
Document Number: MCUXSDKAPIRM
Rev 2.16.000
Jul 2024

MCUXpresso SDK API Reference Manual

NXP Semiconductors



Contents

Chapter 1 Introduction

Chapter 2 Trademarks

Chapter 3 Architectural Overview

Chapter 4 Clock Driver

4.1	Overview	7
4.2	Data Structure Documentation	26
4.2.1	struct _cgc_rtd_sys_clk_config	26
4.2.2	struct _cgc_hifi_sys_clk_config	27
4.2.3	struct _cgc_lpav_sys_clk_config	27
4.2.4	struct _cgc_ddr_sys_clk_config	28
4.2.5	struct _cgc_sosc_config	29
4.2.6	struct _cgc_fro_config	29
4.2.7	struct _cgc_lposc_config	30
4.2.8	struct _cgc_pll0_config	30
4.2.9	struct _cgc_rosc_config	31
4.2.10	struct _cgc_pll1_config	31
4.2.11	struct _cgc_pll4_config	32
4.3	Macro Definition Documentation	33
4.3.1	FSL_SDK_DISABLE_DRIVER_CLOCK_CONTROL	33
4.3.2	FSL_CLOCK_DRIVER_VERSION	34
4.3.3	PCC_PCS_VAL	34
4.3.4	RGPIO_CLOCKS	34
4.3.5	SAI_CLOCKS	34
4.3.6	PCTL_CLOCKS	34
4.3.7	LPI2C_CLOCKS	34
4.3.8	I3C_CLOCKS	35
4.3.9	FLEXIO_CLOCKS	35
4.3.10	FLEXCAN_CLOCKS	35
4.3.11	PDM_CLOCKS	35
4.3.12	LCDIF_CLOCKS	35
4.3.13	MIPI_DSI_HOST_CLOCKS	36
4.3.14	EDMA_CLOCKS	36

Section No.	Title	Page No.
4.3.15	EDMA_CHAN_CLOCKS	36
4.3.16	LPUART_CLOCKS	36
4.3.17	DAC_CLOCKS	36
4.3.18	LPTMR_CLOCKS	37
4.3.19	LPADC_CLOCKS	37
4.3.20	LPSPI_CLOCKS	37
4.3.21	TPM_CLOCKS	37
4.3.22	LPIT_CLOCKS	37
4.3.23	CMP_CLOCKS	38
4.3.24	WDOG_CLOCKS	38
4.3.25	SEMA42_CLOCKS	38
4.3.26	TPIU_CLOCKS	38
4.3.27	FLEXSPI_CLOCKS	38
4.3.28	MRT_CLOCKS	39
4.3.29	BBNSM_CLOCKS	39
4.3.30	PXP_CLOCKS	39
4.3.31	EPDC_CLOCKS	39
4.4	Typedef Documentation	39
4.4.1	clock_name_t	39
4.4.2	clock_ip_name_t	39
4.4.3	cgc_sosc_mode_t	40
4.5	Enumeration Type Documentation	40
4.5.1	_clock_name	40
4.5.2	_clock_ip_src	42
4.5.3	_clock_lptmr_src	46
4.5.4	_clock_ip_name	46
4.5.5	anonymous enum	46
4.5.6	_cgc_sys_clk	46
4.5.7	_cgc_rtd_sys_clk_src	46
4.5.8	_cgc_nic_sys_clk_src	47
4.5.9	_cgc_hifi_sys_clk_src	47
4.5.10	_cgc_lpav_sys_clk_src	47
4.5.11	_cgc_ddr_sys_clk_src	47
4.5.12	_clock_rtd_clkout_src	48
4.5.13	_clock_lpav_clkout_src	48
4.5.14	_cgc_async_clk	48
4.5.15	_cgc_sosc_monitor_mode	49
4.5.16	_cgc_sosc_mode	49
4.5.17	_cgc_sosc_enable_mode	49
4.5.18	_cgc_fro_enable_mode	49
4.5.19	_cgc_lposc_enable_mode	49
4.5.20	_cgc_pll_src	50
4.5.21	_cgc_pll_enable_mode	50

Section No.	Title	Page No.
4.5.22	_cgc_pll_pfd_clkout	50
4.5.23	_cgc_pll0_mult	50
4.5.24	_cgc_rosc_monitor_mode	50
4.5.25	_cgc_pll1_mult	51
4.5.26	_cgc_pll4_mult	51
4.5.27	_cgc_rtd_audclk_src	51
4.5.28	_cgc_ad_audclk_src	51
4.5.29	_cgc_lpav_audclk_src	52
4.6	Function Documentation	52
4.6.1	CLOCK_EnableClock	52
4.6.2	CLOCK_DisableClock	52
4.6.3	CLOCK_IsEnabledByOtherCore	52
4.6.4	CLOCK_SetIpSrc	53
4.6.5	CLOCK_SetIpSrcDiv	53
4.6.6	CLOCK_SetRtdAudClkSrc	53
4.6.7	CLOCK_SetAdAudClkSrc	54
4.6.8	CLOCK_SetLpavAudClkSrc	54
4.6.9	CLOCK_GetFreq	54
4.6.10	CLOCK_GetCm33CorePlatClkFreq	55
4.6.11	CLOCK_GetCm33BusClkFreq	55
4.6.12	CLOCK_GetCm33SlowClkFreq	55
4.6.13	CLOCK_GetFusionDspCorePlatClkFreq	55
4.6.14	CLOCK_GetFusionDspBusClkFreq	55
4.6.15	CLOCK_GetFusionDspSlowClkFreq	55
4.6.16	CLOCK_GetLvdsClkFreq	56
4.6.17	CLOCK_GetIpFreq	56
4.6.18	CLOCK_GetCm33SysClkFreq	56
4.6.19	CLOCK_SetCm33SysClkConfig	56
4.6.20	CLOCK_GetFusionDspSysClkFreq	57
4.6.21	CLOCK_SetFusionSysClkConfig	57
4.6.22	CLOCK_GetCm33SysClkConfig	57
4.6.23	CLOCK_GetFusionDspSysClkConfig	57
4.6.24	CLOCK_SetRtdClkOutConfig	58
4.6.25	CLOCK_SetRtcClkOutConfig	58
4.6.26	CLOCK_GetXbarBusClkFreq	58
4.6.27	CLOCK_GetHifiDspSysClkFreq	58
4.6.28	CLOCK_SetHifiDspSysClkConfig	59
4.6.29	CLOCK_GetHifiDspSysClkConfig	59
4.6.30	CLOCK_GetLpavSysClkFreq	59
4.6.31	CLOCK_SetLpavSysClkConfig	60
4.6.32	CLOCK_GetLpavSysClkConfig	61
4.6.33	CLOCK_GetDdrClkFreq	61
4.6.34	CLOCK_SetLpavClkOutConfig	61
4.6.35	CLOCK_InitSysOsc	61

Section No.	Title	Page No.
4.6.36	CLOCK_DeinitSysOsc	62
4.6.37	CLOCK_SetRtdSysOscAsyncClkDiv	62
4.6.38	CLOCK_SetAdSysOscAsyncClkDiv	63
4.6.39	CLOCK_SetLpavSysOscAsyncClkDiv	63
4.6.40	CLOCK_GetSysOscFreq	63
4.6.41	CLOCK_GetRtdSysOscAsyncFreq	63
4.6.42	CLOCK_GetAdSysOscAsyncFreq	64
4.6.43	CLOCK_GetLpavSysOscAsyncFreq	64
4.6.44	CLOCK_IsSysOscErr	64
4.6.45	CLOCK_SetSysOscMonitorMode	64
4.6.46	CLOCK_IsSysOscSelected	65
4.6.47	CLOCK_IsSysOscValid	65
4.6.48	CLOCK_InitFro	65
4.6.49	CLOCK_DeinitFro	65
4.6.50	CLOCK_SetRtdFroAsyncClkDiv	66
4.6.51	CLOCK_SetAdFroAsyncClkDiv	66
4.6.52	CLOCK_SetLpavFroAsyncClkDiv	66
4.6.53	CLOCK_EnableFroTuning	67
4.6.54	CLOCK_GetFroFreq	67
4.6.55	CLOCK_GetRtdFroAsyncFreq	67
4.6.56	CLOCK_GetAdFroAsyncFreq	67
4.6.57	CLOCK_GetLpavFroAsyncFreq	68
4.6.58	CLOCK_IsFroSelected	68
4.6.59	CLOCK_IsFroValid	68
4.6.60	CLOCK_InitLposc	68
4.6.61	CLOCK_DeinitLposc	69
4.6.62	CLOCK_IsLpOscValid	69
4.6.63	CLOCK_GetLpOscFreq	69
4.6.64	CLOCK_GetRtcOscFreq	69
4.6.65	CLOCK_IsRtcOscErr	70
4.6.66	CLOCK_SetRtcOscMonitorMode	70
4.6.67	CLOCK_IsRtcOscSelected	70
4.6.68	CLOCK_IsRtcOscValid	70
4.6.69	CLOCK_InitPll0	70
4.6.70	CLOCK_DeinitPll0	71
4.6.71	CLOCK_SetPll0AsyncClkDiv	71
4.6.72	CLOCK_GetPll0Freq	72
4.6.73	CLOCK_GetPll0AsyncFreq	72
4.6.74	CLOCK_GetPll0PfdFreq	72
4.6.75	CLOCK_EnablePll0PfdClkout	72
4.6.76	CLOCK_SetPll0LockTime	73
4.6.77	CLOCK_IsPll0Selected	73
4.6.78	CLOCK_IsPll0Valid	73
4.6.79	CLOCK_InitPll1	73
4.6.80	CLOCK_DeinitPll1	74

Section No.	Title	Page No.
4.6.81	CLOCK_SetPll1AsyncClkDiv	74
4.6.82	CLOCK_GetPll1Freq	75
4.6.83	CLOCK_GetPll1AsyncFreq	75
4.6.84	CLOCK_GetPll1PfdFreq	75
4.6.85	CLOCK_EnablePll1PfdClkout	76
4.6.86	CLOCK_EnablePll1SpectrumModulation	76
4.6.87	CLOCK_SetPll1LockTime	76
4.6.88	CLOCK_IsPll1Selected	77
4.6.89	CLOCK_IsPll1Valid	77
4.6.90	CLOCK_GetPll3Freq	77
4.6.91	CLOCK_GetPll3AsyncFreq	77
4.6.92	CLOCK_GetPll3PfdFreq	77
4.6.93	CLOCK_InitPll4	78
4.6.94	CLOCK_DeinitPll4	79
4.6.95	CLOCK_SetPll4AsyncClkDiv	80
4.6.96	CLOCK_GetPll4Freq	80
4.6.97	CLOCK_GetPll4AsyncFreq	80
4.6.98	CLOCK_GetPll4PfdFreq	80
4.6.99	CLOCK_EnablePll4PfdClkout	81
4.6.100	CLOCK_EnablePll4SpectrumModulation	81
4.6.101	CLOCK_SetPll4LockTime	82
4.6.102	CLOCK_IsPll4Selected	82
4.6.103	CLOCK_IsPll4Valid	82
4.6.104	CLOCK_SetXtal0Freq	82
4.6.105	CLOCK_SetXtal32Freq	82
4.6.106	CLOCK_SetLvdsFreq	83
4.6.107	CLOCK_SetMclkFreq	83
4.6.108	CLOCK_SetRxBclkFreq	83
4.6.109	CLOCK_SetTxBclkFreq	83
4.6.110	CLOCK_SetSpdifRxFreq	84
4.6.111	CLOCK_GetWdogClkFreq	85
4.6.112	CLOCK_GetFlexspiClkFreq	85
4.6.113	CLOCK_GetLpitClkFreq	85
4.6.114	CLOCK_GetFlexioClkFreq	85
4.6.115	CLOCK_GetI3cClkFreq	85
4.6.116	CLOCK_GetLpspiClkFreq	86
4.6.117	CLOCK_GetAdcClkFreq	86
4.6.118	CLOCK_GetDacClkFreq	86
4.6.119	CLOCK_GetTpiuClkFreq	87
4.6.120	CLOCK_GetSwoClkFreq	87
4.6.121	CLOCK_GetTpmClkFreq	87
4.6.122	CLOCK_GetLpi2cClkFreq	87
4.6.123	CLOCK_GetLpuartClkFreq	87
4.6.124	CLOCK_GetFlexcanClkFreq	88
4.6.125	CLOCK_GetCsiClkFreq	88

Section No.	Title	Page No.
4.6.126	CLOCK_GetDsiClkFreq	88
4.6.127	CLOCK_GetEpdcClkFreq	88
4.6.128	CLOCK_GetGpu2dClkFreq	88
4.6.129	CLOCK_GetGpu3dClkFreq	89
4.6.130	CLOCK_GetDcnanoClkFreq	89
4.6.131	CLOCK_GetCsiUiClkFreq	89
4.6.132	CLOCK_GetCsiEscClkFreq	89
4.6.133	CLOCK_GetRtdAudClkFreq	89
4.6.134	CLOCK_GetAdAudClkFreq	89
4.6.135	CLOCK_GetLpavAudClkFreq	90
4.6.136	CLOCK_GetSaiFreq	90
4.6.137	CLOCK_GetSpdifFreq	90
4.6.138	CLOCK_GetMqsFreq	90
4.6.139	CLOCK_GetMicfilFreq	90
4.6.140	CLOCK_GetMrtFreq	91
4.6.141	CLOCK_GetRtdSysClkFreq	91
4.7	Variable Documentation	91
4.7.1	g_xtal0Freq	91
4.7.2	g_xtal32Freq	91
4.7.3	g_lvdsFreq	92
4.7.4	g_mclkFreq	92
4.7.5	g_rxBclkFreq	92
4.7.6	g_txBclkFreq	92
4.7.7	g_spdifRxFreq	92
 Chapter 5 Fusion Driver		
 Chapter 6 IOMUXC: IOMUX Controller		
 Chapter 7 Reset Driver		
 Chapter 8 Sentinel Driver		
 Chapter 9 Upower Driver		
9.1	System Clock Generator (SCG)	98
9.1.1	Function description	98
9.1.2	Typical use case	100
 Chapter 10 ACMP: Analog Comparator Driver		
10.1	Overview	102

Section No.	Title	Page No.
10.2	Typical use case	102
10.2.1	Normal Configuration	102
10.2.2	Interrupt Configuration	102
10.2.3	Round robin Configuration	102
10.3	Data Structure Documentation	105
10.3.1	struct _acmp_config	105
10.3.2	struct _acmp_channel_config	106
10.3.3	struct _acmp_filter_config	106
10.3.4	struct _acmp_dac_config	107
10.3.5	struct _acmp_round_robin_config	107
10.3.6	struct _acmp_discrete_mode_config	108
10.4	Macro Definition Documentation	108
10.4.1	FSL_ACMP_DRIVER_VERSION	108
10.4.2	CMP_C0_CFx_MASK	108
10.5	Typedef Documentation	108
10.5.1	acmp_hysteresis_mode_t	108
10.5.2	acmp_reference_voltage_source_t	109
10.5.3	acmp_fixed_port_t	109
10.5.4	acmp_dac_work_mode_t	109
10.5.5	acmp_config_t	109
10.5.6	acmp_channel_config_t	109
10.5.7	acmp_filter_config_t	109
10.5.8	acmp_dac_config_t	109
10.5.9	acmp_round_robin_config_t	109
10.5.10	acmp_discrete_mode_config_t	109
10.6	Enumeration Type Documentation	109
10.6.1	_acmp_interrupt_enable	109
10.6.2	_acmp_status_flags	109
10.6.3	_acmp_hysteresis_mode	110
10.6.4	_acmp_reference_voltage_source	110
10.6.5	_acmp_fixed_port	110
10.6.6	_acmp_dac_work_mode	110
10.7	Function Documentation	110
10.7.1	ACMP_Init	110
10.7.2	ACMP_Deinit	111
10.7.3	ACMP_GetDefaultConfig	111
10.7.4	ACMP_Enable	111
10.7.5	ACMP_EnableLinkToDAC	111
10.7.6	ACMP_SetChannelConfig	112
10.7.7	ACMP_EnableDMA	112

Section No.	Title	Page No.
10.7.8	ACMP_EnableWindowMode	112
10.7.9	ACMP_SetFilterConfig	113
10.7.10	ACMP_SetDACConfig	113
10.7.11	ACMP_SetRoundRobinConfig	114
10.7.12	ACMP_SetRoundRobinPreState	114
10.7.13	ACMP_GetRoundRobinStatusFlags	114
10.7.14	ACMP_ClearRoundRobinStatusFlags	115
10.7.15	ACMP_GetRoundRobinResult	115
10.7.16	ACMP_EnableInterrupts	115
10.7.17	ACMP_DisableInterrupts	116
10.7.18	ACMP_GetStatusFlags	116
10.7.19	ACMP_ClearStatusFlags	116
10.7.20	ACMP_SetDiscreteModeConfig	116
10.7.21	ACMP_GetDefaultDiscreteModeConfig	117
 Chapter 11 CACHE: LMEM CACHE Memory Controller		
11.1	Overview	118
11.2	Function groups	118
11.2.1	L1 CACHE Operation	118
11.3	Macro Definition Documentation	119
11.3.1	FSL_CACHE_DRIVER_VERSION	119
11.3.2	L1CODEBUSCACHE_LINESIZE_BYTE	119
11.3.3	L1SYSTEMBUSCACHE_LINESIZE_BYTE	119
11.4	Function Documentation	119
11.4.1	ICACHE_InvalidateByRange	119
11.4.2	DCACHE_InvalidateByRange	119
11.4.3	DCACHE_CleanByRange	120
11.4.4	DCACHE_CleanInvalidateByRange	120
 Chapter 12 Common Driver		
12.1	Overview	121
12.2	Macro Definition Documentation	124
12.2.1	FSL_DRIVER_TRANSFER_DOUBLE_WEAK_IRQ	124
12.2.2	MAKE_STATUS	124
12.2.3	MAKE_VERSION	124
12.2.4	FSL_COMMON_DRIVER_VERSION	125
12.2.5	DEBUG_CONSOLE_DEVICE_TYPE_NONE	125
12.2.6	DEBUG_CONSOLE_DEVICE_TYPE_UART	125
12.2.7	DEBUG_CONSOLE_DEVICE_TYPE_LPUART	125

Section No.	Title	Page No.
12.2.8	DEBUG_CONSOLE_DEVICE_TYPE_LPSCI	125
12.2.9	DEBUG_CONSOLE_DEVICE_TYPE_USBCDC	125
12.2.10	DEBUG_CONSOLE_DEVICE_TYPE_FLEXCOMM	125
12.2.11	DEBUG_CONSOLE_DEVICE_TYPE_IUART	125
12.2.12	DEBUG_CONSOLE_DEVICE_TYPE_VUSART	125
12.2.13	DEBUG_CONSOLE_DEVICE_TYPE_MINI_USART	125
12.2.14	DEBUG_CONSOLE_DEVICE_TYPE_SWO	125
12.2.15	DEBUG_CONSOLE_DEVICE_TYPE_QSCI	125
12.2.16	MIN	125
12.2.17	MAX	125
12.2.18	ARRAY_SIZE	125
12.2.19	UINT16_MAX	125
12.2.20	UINT32_MAX	125
12.2.21	SUPPRESS_FALL_THROUGH_WARNING	125
12.3	Typedef Documentation	126
12.3.1	status_t	126
12.4	Enumeration Type Documentation	126
12.4.1	_status_groups	126
12.4.2	anonymous enum	129
12.5	Function Documentation	129
12.5.1	SDK_Malloc	129
12.5.2	SDK_Free	129
12.5.3	SDK_DelayAtLeastUs	129
 Chapter 13 CRC: Cyclic Redundancy Check Driver		
13.1	Overview	131
13.2	CRC Driver Initialization and Configuration	131
13.3	CRC Write Data	131
13.4	CRC Get Checksum	131
13.5	Comments about API usage in RTOS	132
13.6	Data Structure Documentation	133
13.6.1	struct_crc_config	133
13.7	Macro Definition Documentation	134
13.7.1	FSL_CRC_DRIVER_VERSION	134
13.7.2	CRC_DRIVER_USE_CRC16_CCIT_FALSE_AS_DEFAULT	134

Section No.	Title	Page No.
13.8	Typedef Documentation	134
13.8.1	crc_config_t	134
13.9	Enumeration Type Documentation	134
13.9.1	_crc_bits	134
13.9.2	_crc_result	135
13.10	Function Documentation	135
13.10.1	CRC_Init	135
13.10.2	CRC_Deinit	135
13.10.3	CRC_GetDefaultConfig	135
13.10.4	CRC_WriteData	136
13.10.5	CRC_Get32bitResult	136
13.10.6	CRC_Get16bitResult	136
Chapter 14 DAC12: 12-bit Digital-to-Analog Converter Driver		
14.1	Overview	138
14.2	Typical use case	138
14.2.1	A simple use case to output the user-defined DAC12 value.	138
14.2.2	Working with the trigger	138
14.3	Data Structure Documentation	141
14.3.1	struct _dac12_hardware_info	141
14.3.2	struct dac12_config_t	142
14.4	Macro Definition Documentation	143
14.4.1	FSL_DAC12_DRIVER_VERSION	143
14.4.2	DAC12_CR_W1C_FLAGS_MASK	143
14.4.3	DAC12_CR_ALL_FLAGS_MASK	143
14.5	Typedef Documentation	143
14.5.1	dac12_reference_current_source_t	143
14.6	Enumeration Type Documentation	143
14.6.1	_dac12_status_flags	143
14.6.2	_dac12_interrupt_enable	143
14.6.3	_dac12_fifo_size_info	144
14.6.4	_dac12_fifo_work_mode	144
14.6.5	_dac12_reference_voltage_source	144
14.6.6	_dac12_fifo_trigger_mode	144
14.6.7	_dac12_reference_current_source	145
14.6.8	_dac12_speed_mode	145
14.7	Function Documentation	145

Section No.	Title	Page No.
14.7.1	DAC12_GetHardwareInfo	145
14.7.2	DAC12_Init	145
14.7.3	DAC12_GetDefaultConfig	146
14.7.4	DAC12_Deinit	146
14.7.5	DAC12_Enable	146
14.7.6	DAC12_ResetConfig	146
14.7.7	DAC12_ResetFIFO	147
14.7.8	DAC12_GetStatusFlags	147
14.7.9	DAC12_ClearStatusFlags	147
14.7.10	DAC12_EnableInterrupts	147
14.7.11	DAC12_DisableInterrupts	148
14.7.12	DAC12_EnableDMA	148
14.7.13	DAC12_SetData	148
14.7.14	DAC12_DoSoftwareTrigger	148
14.7.15	DAC12_GetFIFOReadPointer	149
14.7.16	DAC12_GetFIFOWritePointer	149
 Chapter 15 DMAMUX: Direct Memory Access Multiplexer Driver		
15.1	Overview	150
15.2	Typical use case	150
15.2.1	DMAMUX Operation	150
15.3	Macro Definition Documentation	150
15.3.1	FSL_DMAMUX_DRIVER_VERSION	150
15.4	Function Documentation	150
15.4.1	DMAMUX_Init	150
15.4.2	DMAMUX_Deinit	151
15.4.3	DMAMUX_EnableChannel	151
15.4.4	DMAMUX_DisableChannel	151
15.4.5	DMAMUX_SetSource	152
 Chapter 16 eDMA: Enhanced Direct Memory Access (eDMA) Controller Driver		
16.1	Overview	153
16.2	Typical use case	153
16.2.1	eDMA Operation	153
16.3	Data Structure Documentation	159
16.3.1	struct_edma_config	159
16.3.2	struct_edma_transfer_config	160
16.3.3	struct_edma_channel_Preemption_config	161

Section No.	Title	Page No.
16.3.4	struct _edma_minor_offset_config	161
16.3.5	struct _edma_tcd	162
16.3.6	struct _edma_handle	163
16.4	Macro Definition Documentation	164
16.4.1	FSL_EDMA_DRIVER_VERSION	164
16.5	Typedef Documentation	164
16.5.1	edma_config_t	164
16.5.2	edma_transfer_config_t	164
16.5.3	edma_tcd_t	164
16.5.4	edma_callback	164
16.6	Enumeration Type Documentation	165
16.6.1	_edma_transfer_size	165
16.6.2	_edma_modulo	165
16.6.3	_edma_bandwidth	166
16.6.4	_edma_channel_link_type	166
16.6.5	anonymous enum	166
16.6.6	anonymous enum	167
16.6.7	_edma_interrupt_enable	167
16.6.8	_edma_transfer_type	167
16.6.9	anonymous enum	167
16.7	Function Documentation	168
16.7.1	EDMA_Init	168
16.7.2	EDMA_Deinit	169
16.7.3	EDMA_InstallTCD	169
16.7.4	EDMA_GetDefaultConfig	169
16.7.5	EDMA_EnableContinuousChannelLinkMode	170
16.7.6	EDMA_EnableMinorLoopMapping	170
16.7.7	EDMA_ResetChannel	170
16.7.8	EDMA_SetTransferConfig	171
16.7.9	EDMA_SetMinorOffsetConfig	171
16.7.10	EDMA_SetChannelPreemptionConfig	172
16.7.11	EDMA_SetChannelLink	172
16.7.12	EDMA_SetBandWidth	173
16.7.13	EDMA_SetModulo	173
16.7.14	EDMA_EnableAsyncRequest	174
16.7.15	EDMA_EnableAutoStopRequest	174
16.7.16	EDMA_EnableChannelInterrupts	174
16.7.17	EDMA_DisableChannelInterrupts	174
16.7.18	EDMA_SetMajorOffsetConfig	175
16.7.19	EDMA_TcdReset	175
16.7.20	EDMA_TcdSetTransferConfig	175

Section No.	Title	Page No.
16.7.21	EDMA_TcdSetMinorOffsetConfig	176
16.7.22	EDMA_TcdSetChannelLink	176
16.7.23	EDMA_TcdSetBandWidth	177
16.7.24	EDMA_TcdSetModulo	177
16.7.25	EDMA_TcdEnableAutoStopRequest	178
16.7.26	EDMA_TcdEnableInterrupts	179
16.7.27	EDMA_TcdDisableInterrupts	179
16.7.28	EDMA_TcdSetMajorOffsetConfig	179
16.7.29	EDMA_EnableChannelRequest	179
16.7.30	EDMA_DisableChannelRequest	180
16.7.31	EDMA_TriggerChannelStart	180
16.7.32	EDMA_GetRemainingMajorLoopCount	180
16.7.33	EDMA_GetErrorStatusFlags	181
16.7.34	EDMA_GetChannelStatusFlags	181
16.7.35	EDMA_ClearChannelStatusFlags	181
16.7.36	EDMA_CreateHandle	182
16.7.37	EDMA_InstallTCDMemory	182
16.7.38	EDMA_SetCallback	182
16.7.39	EDMA_PrepareTransferConfig	183
16.7.40	EDMA_PrepareTransfer	183
16.7.41	EDMA_SubmitTransfer	184
16.7.42	EDMA_StartTransfer	185
16.7.43	EDMA_StopTransfer	186
16.7.44	EDMA_AbortTransfer	186
16.7.45	EDMA_GetUnusedTCDNumber	186
16.7.46	EDMA_GetNextTCDAddress	186
16.7.47	EDMA_HandleIRQ	187

Chapter 17 EWM: External Watchdog Monitor Driver

17.1	Overview	188
17.2	Typical use case	188
17.3	Data Structure Documentation	189
17.3.1	struct _ewm_config	189
17.4	Macro Definition Documentation	189
17.4.1	FSL_EWM_DRIVER_VERSION	189
17.5	Typedef Documentation	189
17.5.1	ewm_config_t	189
17.6	Enumeration Type Documentation	190
17.6.1	_ewm_interrupt_enable_t	190

Section No.	Title	Page No.
17.6.2	_ewm_status_flags_t	190
17.7	Function Documentation	190
17.7.1	EWM_Init	190
17.7.2	EWM_Deinit	190
17.7.3	EWM_GetDefaultConfig	191
17.7.4	EWM_EnableInterrupts	191
17.7.5	EWM_DisableInterrupts	191
17.7.6	EWM_GetStatusFlags	192
17.7.7	EWM_Refresh	192
 Chapter 18 FlexIO: FlexIO Driver		
18.1	Overview	194
18.2	FlexIO Driver	195
18.2.1	Overview	195
18.2.2	Data Structure Documentation	202
18.2.3	Macro Definition Documentation	207
18.2.4	Typedef Documentation	207
18.2.5	Enumeration Type Documentation	208
18.2.6	Function Documentation	212
18.2.7	Variable Documentation	226
18.3	FlexIO I2C Master Driver	227
18.3.1	Overview	227
18.3.2	Typical use case	227
18.3.3	Data Structure Documentation	231
18.3.4	Macro Definition Documentation	234
18.3.5	Typedef Documentation	234
18.3.6	Enumeration Type Documentation	234
18.3.7	Function Documentation	235
18.4	FlexIO I2S Driver	245
18.4.1	Overview	245
18.4.2	Typical use case	245
18.4.3	Data Structure Documentation	251
18.4.4	Macro Definition Documentation	253
18.4.5	Typedef Documentation	253
18.4.6	Enumeration Type Documentation	253
18.4.7	Function Documentation	254
18.5	FlexIO SPI Driver	266
18.5.1	Overview	266
18.5.2	Typical use case	266

Section No.	Title	Page No.
18.5.3	Data Structure Documentation	273
18.5.4	Macro Definition Documentation	279
18.5.5	Typedef Documentation	279
18.5.6	Enumeration Type Documentation	279
18.5.7	Function Documentation	281
18.6	FlexIO UART Driver	296
18.6.1	Overview	296
18.6.2	Typical use case	296
18.6.3	Data Structure Documentation	305
18.6.4	Macro Definition Documentation	309
18.6.5	Typedef Documentation	309
18.6.6	Enumeration Type Documentation	309
18.6.7	Function Documentation	310
Chapter 19 GPIO: General-Purpose Input/Output Driver		
19.1	Overview	322
19.2	Data Structure Documentation	323
19.2.1	struct _gpio_pin_config	323
19.3	Macro Definition Documentation	323
19.3.1	FSL_GPIO_DRIVER_VERSION	323
19.4	Typedef Documentation	323
19.4.1	gpio_pin_config_t	323
19.4.2	gpio_interrupt_config_t	323
19.5	Enumeration Type Documentation	323
19.5.1	_gpio_pin_direction	323
19.5.2	_gpio_interrupt_config	324
19.6	GPIO Driver	325
19.6.1	Overview	325
19.6.2	Typical use case	325
19.6.3	Function Documentation	326
19.7	FGPIO Driver	332
19.7.1	Typical use case	332
Chapter 20 LLWU: Low-Leakage Wakeup Unit Driver		
20.1	Overview	333
20.2	External wakeup pins configurations	333

Section No.	Title	Page No.
20.3	Internal wakeup modules configurations	333
20.4	Digital pin filter for external wakeup pin configurations	333
20.5	Macro Definition Documentation	334
20.5.1	FSL_LLWU_DRIVER_VERSION	334
20.6	Enumeration Type Documentation	334
20.6.1	_llwu_external_pin_mode	334
20.6.2	_llwu_pin_filter_mode	334
Chapter 21 LPADC: 12-bit SAR Analog-to-Digital Converter Driver		
21.1	Overview	335
21.2	Typical use case	335
21.2.1	Polling Configuration	335
21.2.2	Interrupt Configuration	335
21.3	Data Structure Documentation	341
21.3.1	struct lpadc_config_t	341
21.3.2	struct lpadc_conv_command_config_t	343
21.3.3	struct lpadc_conv_trigger_config_t	344
21.3.4	struct lpadc_conv_result_t	345
21.4	Macro Definition Documentation	345
21.4.1	FSL_LPADC_DRIVER_VERSION	345
21.4.2	LPADC_GET_ACTIVE_COMMAND_STATUS	345
21.4.3	LPADC_GET_ACTIVE_TRIGGER_STATUE	345
21.5	Typedef Documentation	345
21.5.1	lpadc_sample_scale_mode_t	345
21.5.2	lpadc_sample_channel_mode_t	346
21.5.3	lpadc_hardware_average_mode_t	346
21.5.4	lpadc_sample_time_mode_t	346
21.5.5	lpadc_hardware_compare_mode_t	346
21.5.6	lpadc_reference_voltage_source_t	346
21.5.7	lpadc_power_level_mode_t	346
21.5.8	lpadc_trigger_priority_policy_t	347
21.6	Enumeration Type Documentation	347
21.6.1	_lpadc_status_flags	347
21.6.2	_lpadc_interrupt_enable	347
21.6.3	_lpadc_trigger_status_flags	348
21.6.4	_lpadc_sample_scale_mode	350
21.6.5	_lpadc_sample_channel_mode	350

Section No.	Title	Page No.
21.6.6	_lpadc_hardware_average_mode	350
21.6.7	_lpadc_sample_time_mode	351
21.6.8	_lpadc_hardware_compare_mode	351
21.6.9	_lpadc_reference_voltage_mode	351
21.6.10	_lpadc_power_level_mode	352
21.6.11	_lpadc_trigger_priority_policy	352
21.7	Function Documentation	353
21.7.1	LPADC_Init	353
21.7.2	LPADC_GetDefaultConfig	353
21.7.3	LPADC_Deinit	354
21.7.4	LPADC_Enable	354
21.7.5	LPADC_DoResetFIFO	354
21.7.6	LPADC_DoResetConfig	354
21.7.7	LPADC_GetStatusFlags	355
21.7.8	LPADC_ClearStatusFlags	355
21.7.9	LPADC_GetTriggerStatusFlags	355
21.7.10	LPADC_ClearTriggerStatusFlags	355
21.7.11	LPADC_EnableInterrupts	356
21.7.12	LPADC_DisableInterrupts	356
21.7.13	LPADC_EnableFIFOWatermarkDMA	356
21.7.14	LPADC_GetConvResultCount	356
21.7.15	LPADC_GetConvResult	357
21.7.16	LPADC_GetConvResultBlocking	357
21.7.17	LPADC_SetConvTriggerConfig	357
21.7.18	LPADC_GetDefaultConvTriggerConfig	358
21.7.19	LPADC_DoSoftwareTrigger	358
21.7.20	LPADC_SetConvCommandConfig	358
21.7.21	LPADC_GetDefaultConvCommandConfig	359
Chapter 22	LPI2C: Low Power Inter-Integrated Circuit Driver	
22.1	Overview	360
22.2	Macro Definition Documentation	361
22.2.1	FSL_LPI2C_DRIVER_VERSION	361
22.2.2	I2C_RETRY_TIMES	361
22.3	Enumeration Type Documentation	361
22.3.1	anonymous enum	361
22.4	Function Documentation	361
22.4.1	LPI2C_GetInstance	361
22.5	Variable Documentation	362

Section No.	Title	Page No.
22.5.1	kLpi2cIrqs	362
22.5.2	s_lpi2cMasterIsr	362
22.5.3	s_lpi2cMasterHandle	362
22.6	LPI2C Master Driver	363
22.6.1	Overview	363
22.6.2	Data Structure Documentation	367
22.6.3	Typedef Documentation	371
22.6.4	Enumeration Type Documentation	373
22.6.5	Function Documentation	375
22.7	LPI2C Slave Driver	390
22.7.1	Overview	390
22.7.2	Data Structure Documentation	393
22.7.3	Typedef Documentation	396
22.7.4	Enumeration Type Documentation	397
22.7.5	Function Documentation	399
22.8	LPI2C Master DMA Driver	408
22.8.1	Overview	408
22.8.2	Data Structure Documentation	408
22.8.3	Typedef Documentation	410
22.8.4	Function Documentation	411
22.9	LPI2C FreeRTOS Driver	414
22.10	LPI2C CMSIS Driver	415
22.10.1	LPI2C CMSIS Driver	415
Chapter 23 LPIT: Low-Power Interrupt Timer		
23.1	Overview	417
23.2	Function groups	417
23.2.1	Initialization and deinitialization	417
23.2.2	Timer period Operations	417
23.2.3	Start and Stop timer operations	417
23.2.4	Status	418
23.2.5	Interrupt	418
23.3	Typical use case	418
23.3.1	LPIT tick example	418
23.4	Data Structure Documentation	421
23.4.1	struct _lpit_chnl_params	421
23.4.2	struct _lpit_config	421

Section No.	Title	Page No.
23.5	Typedef Documentation	421
23.5.1	lpit_chnl_t	421
23.5.2	lpit_timer_modes_t	422
23.5.3	lpit_trigger_select_t	422
23.5.4	lpit_interrupt_enable_t	422
23.5.5	lpit_status_flags_t	422
23.5.6	lpit_chnl_params_t	422
23.5.7	lpit_config_t	422
23.6	Enumeration Type Documentation	422
23.6.1	_lpit_chnl	422
23.6.2	_lpit_timer_modes	423
23.6.3	_lpit_trigger_select	423
23.6.4	_lpit_trigger_source	424
23.6.5	_lpit_interrupt_enable	424
23.6.6	_lpit_status_flags	424
23.7	Function Documentation	424
23.7.1	LPIT_Init	424
23.7.2	LPIT_Deinit	425
23.7.3	LPIT_GetDefaultConfig	425
23.7.4	LPIT_SetupChannel	425
23.7.5	LPIT_EnableInterrupts	426
23.7.6	LPIT_DisableInterrupts	426
23.7.7	LPIT_GetEnabledInterrupts	426
23.7.8	LPIT_GetStatusFlags	426
23.7.9	LPIT_ClearStatusFlags	427
23.7.10	LPIT_SetTimerPeriod	427
23.7.11	LPIT_SetTimerValue	427
23.7.12	LPIT_GetCurrentTimerCount	428
23.7.13	LPIT_StartTimer	428
23.7.14	LPIT_StopTimer	429
23.7.15	LPIT_Reset	429
Chapter 24 LPSPI: Low Power Serial Peripheral Interface		
24.1	Overview	430
24.2	LPSPI Peripheral driver	431
24.2.1	Overview	431
24.2.2	Function groups	431
24.2.3	Typical use case	431
24.2.4	Data Structure Documentation	439
24.2.5	Macro Definition Documentation	445
24.2.6	Typedef Documentation	447

Section No.	Title	Page No.
24.2.7	Enumeration Type Documentation	448
24.2.8	Function Documentation	453
24.2.9	Variable Documentation	470
Chapter 25 LPTMR: Low-Power Timer		
25.1	Overview	471
25.2	Function groups	471
25.2.1	Initialization and deinitialization	471
25.2.2	Timer period Operations	471
25.2.3	Start and Stop timer operations	471
25.2.4	Status	472
25.2.5	Interrupt	472
25.3	Typical use case	472
25.3.1	LPTMR tick example	472
25.4	Data Structure Documentation	475
25.4.1	struct _lptmr_config	475
25.5	Typedef Documentation	475
25.5.1	lptmr_pin_select_t	475
25.5.2	lptmr_pin_polarity_t	475
25.5.3	lptmr_timer_mode_t	475
25.5.4	lptmr_prescaler_clock_select_t	475
25.5.5	lptmr_config_t	476
25.6	Enumeration Type Documentation	476
25.6.1	_lptmr_pin_select	476
25.6.2	_lptmr_pin_polarity	476
25.6.3	_lptmr_timer_mode	476
25.6.4	_lptmr_prescaler_glitch_value	476
25.6.5	_lptmr_prescaler_clock_select	477
25.6.6	_lptmr_interrupt_enable	477
25.6.7	_lptmr_status_flags	477
25.7	Function Documentation	478
25.7.1	LPTMR_Init	478
25.7.2	LPTMR_Deinit	478
25.7.3	LPTMR_GetDefaultConfig	478
25.7.4	LPTMR_EnableInterrupts	478
25.7.5	LPTMR_DisableInterrupts	479
25.7.6	LPTMR_GetEnabledInterrupts	479
25.7.7	LPTMR_EnableTimerDMA	479
25.7.8	LPTMR_GetStatusFlags	480

Section No.	Title	Page No.
25.7.9	LPTMR_ClearStatusFlags	480
25.7.10	LPTMR_SetTimerPeriod	480
25.7.11	LPTMR_GetCurrentTimerCount	481
25.7.12	LPTMR_StartTimer	481
25.7.13	LPTMR_StopTimer	481
 Chapter 26 LPUART: Low Power Universal Asynchronous Receiver/Transmitter Driver		
26.1	Overview	483
26.2	LPUART Driver	484
26.2.1	Overview	484
26.2.2	Typical use case	484
26.2.3	Data Structure Documentation	490
26.2.4	Macro Definition Documentation	494
26.2.5	Typedef Documentation	494
26.2.6	Enumeration Type Documentation	495
26.2.7	Function Documentation	498
 Chapter 27 LTC: LP Trusted Cryptography		
27.1	Overview	516
27.2	LTC Driver Initialization and Configuration	516
27.3	Comments about API usage in RTOS	516
27.4	Comments about API usage in interrupt handler	516
27.5	LTC Driver Examples	517
27.5.1	Simple examples	517
27.6	Macro Definition Documentation	517
27.6.1	FSL_LTC_DRIVER_VERSION	517
27.7	Function Documentation	518
27.7.1	LTC_Init	518
27.7.2	LTC_Deinit	518
27.8	LTC Blocking APIs	520
27.8.1	Overview	520
27.8.2	LTC DES driver	521
27.8.3	LTC AES driver	522
27.8.4	LTC HASH driver	530
27.8.5	LTC PKHA driver	535

Section No.	Title	Page No.
Chapter 28 MSMC: Multicore System Mode Controller		
28.1	Overview	537
28.2	Typical use case	537
28.2.1	Set Core 0 from RUN to VLPR mode	537
28.2.2	Set Core 0 from VLPR/HSRUN to RUN mode	537
28.3	Typical use case	537
28.3.1	Set Core 0 from RUN to HSRUN mode	537
28.3.2	Enter wait or stop modes	537
28.4	Data Structure Documentation	541
28.4.1	struct _smc_reset_pin_filter_config	541
28.5	Macro Definition Documentation	542
28.5.1	FSL_MSMC_DRIVER_VERSION	542
28.6	Enumeration Type Documentation	542
28.6.1	_smc_power_mode_protection	542
28.6.2	_smc_power_state	542
28.6.3	_smc_power_stop_entry_status	542
28.6.4	_smc_run_mode	543
28.6.5	_smc_stop_mode	543
28.6.6	_smc_partial_stop_mode	543
28.6.7	anonymous enum	543
28.6.8	_smc_reset_source	543
28.6.9	_smc_interrupt_enable	544
28.7	Function Documentation	544
28.7.1	SMC_SetPowerModeProtection	544
28.7.2	SMC_GetPowerModeState	545
28.7.3	SMC_PreEnterStopModes	545
28.7.4	SMC_PostExitStopModes	545
28.7.5	SMC_PreEnterWaitModes	545
28.7.6	SMC_PostExitWaitModes	545
28.7.7	SMC_SetPowerModeRun	546
28.7.8	SMC_SetPowerModeHsrn	547
28.7.9	SMC_SetPowerModeWait	547
28.7.10	SMC_SetPowerModeStop	547
28.7.11	SMC_SetPowerModeVlpr	548
28.7.12	SMC_SetPowerModeVlps	548
28.7.13	SMC_SetPowerModeVlps	548
28.7.14	SMC_SetPowerModeLls	548
28.7.15	SMC_SetPowerModeVlls	549
28.7.16	SMC_GetPreviousResetSources	549

Section No.	Title	Page No.
28.7.17	SMC_GetStickyResetSources	550
28.7.18	SMC_ClearStickyResetSources	551
28.7.19	SMC_ConfigureResetPinFilter	551
28.7.20	SMC_SetSystemResetInterruptConfig	551
28.7.21	SMC_GetResetInterruptSourcesStatus	552
28.7.22	SMC_ClearResetInterruptSourcesStatus	552
28.7.23	SMC_GetBootOptionConfig	553
 Chapter 29 MU: Messaging Unit		
29.1	Overview	554
29.2	Function description	554
29.2.1	MU initialization	554
29.2.2	MU message	554
29.2.3	MU flags	555
29.2.4	Status and interrupt	555
29.2.5	MU misc functions	555
29.3	Macro Definition Documentation	559
29.3.1	FSL_MU_DRIVER_VERSION	559
29.4	Enumeration Type Documentation	559
29.4.1	_mu_status_flags	559
29.4.2	_mu_interrupt_enable	560
29.4.3	_mu_interrupt_trigger	561
29.4.4	_mu_core_status_flags	561
29.4.5	_mu_msg_reg_index	561
29.4.6	_mu_general_purpose_interrupt	562
29.5	Function Documentation	562
29.5.1	MU_GetInstance	562
29.5.2	MU_Init	562
29.5.3	MU_Deinit	562
29.5.4	MU_SendMsgNonBlocking	562
29.5.5	MU_SendMsg	563
29.5.6	MU_ReceiveMsgNonBlocking	563
29.5.7	MU_ReceiveMsg	564
29.5.8	MU_SetFlagsNonBlocking	564
29.5.9	MU_SetFlags	565
29.5.10	MU_GetFlags	565
29.5.11	MU_GetCoreStatusFlags	565
29.5.12	MU_GetStatusFlags	566
29.5.13	MU_ClearStatusFlags	566
29.5.14	MU_EnableInterrupts	567

Section No.	Title	Page No.
29.5.15	MU_DisableInterrupts	567
29.5.16	MU_TriggerInterrupts	568
29.5.17	MU_TriggerNmi	568
29.5.18	MU_ClearNmi	569
29.5.19	MU_EnableGeneralPurposeInterrupts	569
29.5.20	MU_DisableGeneralPurposeInterrupts	569
29.5.21	MU_GetGeneralPurposeStatusFlags	570
29.5.22	MU_ClearGeneralPurposeStatusFlags	570
29.5.23	MU_TriggerGeneralPurposeInterrupts	571
29.5.24	MU_BootOtherCore	571
29.5.25	MU_HoldOtherCoreReset	572
29.5.26	MU_ResetBothSides	572
29.5.27	MU_SetClockOnOtherCoreEnable	572
29.5.28	MU_HardwareResetOtherCore	573
 Chapter 30 PMC0: Power Management Controller		
30.1	Overview	575
30.2	Typical use case	575
30.2.1	Turn on the PMC 1 using LDO Regulator	575
30.2.2	Turn on the PMC 1 using the PMIC	575
30.2.3	Turn off the LDO Regulator	576
30.2.4	Turn on the LDO Regulator	576
30.2.5	Change the Core Regulator voltage level in PMC 0 RUN or HSRUN mode	576
30.2.6	Change the SRAMs power mode during PMC 0 RUN mode	576
30.3	Data Structure Documentation	581
30.3.1	struct _pmc0_hsruntime_mode_config	581
30.3.2	struct _pmc0_run_mode_config	582
30.3.3	struct _pmc0_vlpr_mode_config	582
30.3.4	struct _pmc0_stop_mode_config	584
30.3.5	struct _pmc0_vlps_mode_config	584
30.3.6	struct _pmc0_lls_mode_config	586
30.3.7	struct _pmc0_vlls_mode_config	587
30.3.8	struct _pmc0_bias_config	588
30.4	Enumeration Type Documentation	589
30.4.1	_pmc0_high_volt_detect_monitor_select	589
30.4.2	_pmc0_low_volt_detect_monitor_select	589
30.4.3	_pmc0_core_regulator_select	589
30.4.4	_pmc0_array_regulator_select	589
30.4.5	_pmc0_vlls_array_regulator_select	590
30.4.6	_pmc0_fbb_p_well_voltage_level_select	590
30.4.7	_pmc0_fbb_n_well_voltage_level_select	590

Section No.	Title	Page No.
30.4.8	_pmc0_rbb_p_well_voltage_level_select	591
30.4.9	_pmc0_rbb_n_well_voltage_level_select	591
30.4.10	_pmc0_status_flags	591
30.4.11	_pmc0_power_mode_status_flags	592
30.5	Function Documentation	592
30.5.1	PMC0_ConfigureHsrunMode	592
30.5.2	PMC0_ConfigureRunMode	592
30.5.3	PMC0_ConfigureVlprMode	593
30.5.4	PMC0_ConfigureStopMode	593
30.5.5	PMC0_ConfigureVlpsMode	593
30.5.6	PMC0_ConfigureLlsMode	593
30.5.7	PMC0_ConfigureVllsMode	595
30.5.8	PMC0_GetPMC0PowerModeStatusFlags	595
30.5.9	PMC0_GetPMC0PowerTransitionStatus	595
30.5.10	PMC0_GetPMC1PowerModeStatusFlags	596
30.5.11	PMC0_GetPMC1PowerTransitionStatus	596
30.5.12	PMC0_GetStatusFlags	596
30.5.13	PMC0_EnableLowVoltDetectInterrupt	596
30.5.14	PMC0_DisableLowVoltDetectInterrupt	597
30.5.15	PMC0_ClearLowVoltDetectFlag	597
30.5.16	PMC0_EnableHighVoltDetectInterrupt	597
30.5.17	PMC0_DisableHighVoltDetectInterrupt	597
30.5.18	PMC0_ClearHighVoltDetectFlag	597
30.5.19	PMC0_EnableLowVoltDetectReset	597
30.5.20	PMC0_EnableHighVoltDetectReset	598
30.5.21	PMC0_ClearPadsIsolation	599
30.5.22	PMC0_PowerOnPmc1	599
30.5.23	PMC0_EnableWaitLdoOkSignal	599
30.5.24	PMC0_EnablePmc1LdoRegulator	599
30.5.25	PMC0_EnablePmc1RBBMode	600
30.5.26	PMC0_SetBiasConfig	600
30.5.27	PMC0_ConfigureSramBankPowerDown	600
30.5.28	PMC0_ConfigureSramBankPowerDownStopMode	600
30.5.29	PMC0_ConfigureSramBankPowerDownStandbyMode	601
30.5.30	PMC0_EnableTemperatureSensor	601
30.5.31	PMC0_SetTemperatureSensorMode	601
Chapter 31	PORT: Port Control and Interrupts	
31.1	Overview	603
31.2	Macro Definition Documentation	604
31.2.1	FSL_PORT_DRIVER_VERSION	604

Section No.	Title	Page No.
31.3	Typedef Documentation	604
31.3.1	port_interrupt_t	604
31.4	Enumeration Type Documentation	604
31.4.1	_port_interrupt	604
31.5	Function Documentation	605
31.5.1	PORT_SetPinInterruptConfig	605
31.5.2	PORT_GetPinsInterruptFlags	605
31.5.3	PORT_ClearPinsInterruptFlags	606
Chapter 32 QSPI: Quad Serial Peripheral Interface		
32.1	Overview	607
32.2	Quad Serial Peripheral Interface Driver	608
32.2.1	Overview	608
32.2.2	Data Structure Documentation	613
32.2.3	Macro Definition Documentation	616
32.2.4	Typedef Documentation	616
32.2.5	Enumeration Type Documentation	616
32.2.6	Function Documentation	619
Chapter 33 SAI: Serial Audio Interface		
33.1	Overview	629
33.2	Typical configurations	629
33.3	Typical use case	630
33.3.1	SAI Send/receive using an interrupt method	630
33.3.2	SAI Send/receive using a DMA method	630
33.4	SAI Driver	631
33.4.1	Overview	631
33.4.2	Data Structure Documentation	639
33.4.3	Macro Definition Documentation	643
33.4.4	Enumeration Type Documentation	643
33.4.5	Function Documentation	648
Chapter 34 SEMA42: Hardware Semaphores Driver		
34.1	Overview	672
34.2	Typical use case	672

Section No.	Title	Page No.
34.3	Macro Definition Documentation	674
34.3.1	SEMA42_GATE_NUM_RESET_ALL	674
34.3.2	SEMA42_GATEn	674
34.4	Enumeration Type Documentation	674
34.4.1	anonymous enum	674
34.4.2	_sema42_gate_status	674
34.5	Function Documentation	675
34.5.1	SEMA42_Init	675
34.5.2	SEMA42_Deinit	675
34.5.3	SEMA42_TryLock	675
34.5.4	SEMA42_Lock	676
34.5.5	SEMA42_Unlock	677
34.5.6	SEMA42_GetGateStatus	677
34.5.7	SEMA42_ResetGate	677
34.5.8	SEMA42_ResetAllGates	678
Chapter 35 TPM: Timer PWM Module		
35.1	Overview	679
35.2	Introduction of TPM	679
35.2.1	Initialization and deinitialization	679
35.2.2	PWM Operations	679
35.2.3	Input capture operations	680
35.2.4	Output compare operations	680
35.2.5	Quad decode	680
35.2.6	Fault operation	680
35.2.7	Status	680
35.2.8	Interrupt	680
35.3	Typical use case	681
35.3.1	PWM output	681
35.4	Data Structure Documentation	686
35.4.1	struct _tpm_chnl_pwm_signal_param	686
35.4.2	struct _tpm_dual_edge_capture_param	687
35.4.3	struct _tpm_phase_param	688
35.4.4	struct _tpm_config	688
35.5	Macro Definition Documentation	689
35.5.1	FSL_TPM_DRIVER_VERSION	689
35.6	Typedef Documentation	689
35.6.1	tpm_chnl_t	689

Section No.	Title	Page No.
35.6.2	tpm_pwm_level_select_t	689
35.6.3	tpm_trigger_select_t	689
35.6.4	tpm_trigger_source_t	689
35.6.5	tpm_ext_trigger_polarity_t	690
35.6.6	tpm_dual_edge_capture_param_t	690
35.6.7	tpm_quad_decode_mode_t	690
35.6.8	tpm_config_t	690
35.6.9	tpm_callback_t	690
35.7	Enumeration Type Documentation	690
35.7.1	_tpm_chnl	690
35.7.2	_tpm_pwm_mode	691
35.7.3	_tpm_pwm_level_select	691
35.7.4	_tpm_pwm_pause_level_select	691
35.7.5	_tpm_chnl_control_bit_mask	692
35.7.6	_tpm_trigger_select	692
35.7.7	_tpm_trigger_source	692
35.7.8	_tpm_ext_trigger_polarity	692
35.7.9	_tpm_output_compare_mode	693
35.7.10	_tpm_input_capture_edge	693
35.7.11	_tpm_quad_decode_mode	693
35.7.12	_tpm_phase_polarity	693
35.7.13	_tpm_clock_source	693
35.7.14	_tpm_clock_prescale	694
35.7.15	_tpm_interrupt_enable	694
35.7.16	_tpm_status_flags	694
35.8	Function Documentation	695
35.8.1	TPM_GetInstance	695
35.8.2	TPM_Init	695
35.8.3	TPM_Deinit	695
35.8.4	TPM_GetDefaultConfig	695
35.8.5	TPM_CalculateCounterClkDiv	696
35.8.6	TPM_SetupPwm	696
35.8.7	TPM_UpdatePwmDutycycle	697
35.8.8	TPM_UpdateChnlEdgeLevelSelect	697
35.8.9	TPM_GetChannelContorlBits	698
35.8.10	TPM_DisableChannel	698
35.8.11	TPM_EnableChannel	698
35.8.12	TPM_SetupInputCapture	699
35.8.13	TPM_SetupOutputCompare	699
35.8.14	TPM_SetupDualEdgeCapture	700
35.8.15	TPM_SetupQuadDecode	701
35.8.16	TPM_SetChannelPolarity	701
35.8.17	TPM_EnableChannelExtTrigger	701

Section No.	Title	Page No.
35.8.18	TPM_EnableInterrupts	702
35.8.19	TPM_DisableInterrupts	702
35.8.20	TPM_GetEnabledInterrupts	702
35.8.21	TPM_RegisterCallBack	703
35.8.22	TPM_GetChannelValue	703
35.8.23	TPM_GetStatusFlags	703
35.8.24	TPM_ClearStatusFlags	704
35.8.25	TPM_SetTimerPeriod	704
35.8.26	TPM_GetCurrentTimerCount	704
35.8.27	TPM_StartTimer	705
35.8.28	TPM_StopTimer	705
35.8.29	TPM_Reset	705
 Chapter 36 TRGMUX: Trigger Mux Driver		
36.1	Overview	707
36.2	Typical use case	707
36.3	Macro Definition Documentation	707
36.3.1	FSL_TRGMUX_DRIVER_VERSION	707
36.4	Enumeration Type Documentation	708
36.4.1	anonymous enum	708
36.4.2	_trgmux_trigger_input	708
36.5	Function Documentation	708
36.5.1	TRGMUX_LockRegister	708
36.5.2	TRGMUX_SetTriggerSource	708
 Chapter 37 TRNG: True Random Number Generator		
37.1	TRNG Initialization	710
37.2	Get random data from TRNG	710
 Chapter 38 TSTMR: Timestamp Timer Driver		
38.1	Overview	711
38.2	Function Documentation	711
38.2.1	TSTMR_ReadTimeStamp	711
38.2.2	TSTMR_DelayUs	711

Section No.	Title	Page No.
Chapter 39	WDOG32: 32-bit Watchdog Timer	
39.1	Overview	713
39.2	Typical use case	713
39.3	Data Structure Documentation	715
39.3.1	struct _wdog32_work_mode	715
39.3.2	struct _wdog32_config	715
39.4	Macro Definition Documentation	716
39.4.1	FSL_WDOG32_DRIVER_VERSION	716
39.5	Typedef Documentation	716
39.5.1	wdog32_clock_source_t	716
39.5.2	wdog32_clock_prescaler_t	716
39.5.3	wdog32_work_mode_t	716
39.5.4	wdog32_test_mode_t	716
39.5.5	wdog32_config_t	716
39.6	Enumeration Type Documentation	716
39.6.1	_wdog32_clock_source	716
39.6.2	_wdog32_clock_prescaler	716
39.6.3	_wdog32_test_mode	717
39.6.4	_wdog32_interrupt_enable_t	717
39.6.5	_wdog32_status_flags_t	717
39.7	Function Documentation	717
39.7.1	WDOG32_GetDefaultConfig	717
39.7.2	WDOG32_Init	718
39.7.3	WDOG32_Deinit	718
39.7.4	WDOG32_Unlock	719
39.7.5	WDOG32_Enable	719
39.7.6	WDOG32_Disable	719
39.7.7	WDOG32_EnableInterrupts	720
39.7.8	WDOG32_DisableInterrupts	720
39.7.9	WDOG32_GetStatusFlags	720
39.7.10	WDOG32_ClearStatusFlags	721
39.7.11	WDOG32_SetTimeoutValue	721
39.7.12	WDOG32_SetWindowValue	722
39.7.13	WDOG32_Refresh	722
39.7.14	WDOG32_GetCounterValue	722
Chapter 40	Debug Console	
40.1	Overview	723

Section No.	Title	Page No.
40.2	Function groups	723
40.2.1	Initialization	723
40.2.2	Advanced Feature	724
40.2.3	SDK_DEBUGCONSOLE and SDK_DEBUGCONSOLE_UART	728
40.3	Typical use case	729
40.4	Macro Definition Documentation	731
40.4.1	DEBUGCONSOLE_REDIRECT_TO_TOOLCHAIN	731
40.4.2	DEBUGCONSOLE_REDIRECT_TO_SDK	731
40.4.3	DEBUGCONSOLE_DISABLE	731
40.4.4	SDK_DEBUGCONSOLE	731
40.4.5	PRINTF	731
40.5	Function Documentation	731
40.5.1	DbgConsole_Init	731
40.5.2	DbgConsole_Deinit	732
40.5.3	DbgConsole_EnterLowpower	732
40.5.4	DbgConsole_ExitLowpower	733
40.5.5	DbgConsole_Printf	733
40.5.6	DbgConsole_Vprintf	733
40.5.7	DbgConsole_Putchar	733
40.5.8	DbgConsole_Scanf	734
40.5.9	DbgConsole_Getchar	734
40.5.10	DbgConsole_BlockingPrintf	735
40.5.11	DbgConsole_BlockingVprintf	735
40.5.12	DbgConsole_Flush	735
40.5.13	DbgConsole_TryGetchar	736
40.6	debug console configuration	738
40.6.1	Overview	738
40.6.2	Macro Definition Documentation	739
Chapter 41 Notification Framework		
41.1	Overview	741
41.2	Notifier Overview	741
41.3	Data Structure Documentation	744
41.3.1	struct_notifier_notification_block	744
41.3.2	struct_notifier_callback_config	744
41.3.3	struct_notifier_handle	745
41.4	Typedef Documentation	746
41.4.1	notifier_policy_t	746

Section No.	Title	Page No.
41.4.2	notifier_notification_type_t	746
41.4.3	notifier_callback_type_t	746
41.4.4	notifier_user_config_t	747
41.4.5	notifier_user_function_t	747
41.4.6	notifier_notification_block_t	747
41.4.7	notifier_callback_t	747
41.4.8	notifier_callback_config_t	748
41.4.9	notifier_handle_t	748
41.5	Enumeration Type Documentation	748
41.5.1	_notifier_status	748
41.5.2	_notifier_policy	748
41.5.3	_notifier_notification_type	749
41.5.4	_notifier_callback_type	749
41.6	Function Documentation	749
41.6.1	NOTIFIER_CreateHandle	749
41.6.2	NOTIFIER_SwitchConfig	750
41.6.3	NOTIFIER_GetErrorCallbackIndex	751
 Chapter 42 Shell		
42.1	Overview	752
42.2	Function groups	752
42.2.1	Initialization	752
42.2.2	Advanced Feature	752
42.2.3	Shell Operation	752
42.3	Data Structure Documentation	754
42.3.1	struct _shell_command	754
42.4	Macro Definition Documentation	755
42.4.1	SHELL_NON_BLOCKING_MODE	755
42.4.2	SHELL_AUTO_COMPLETE	755
42.4.3	SHELL_BUFFER_SIZE	755
42.4.4	SHELL_MAX_ARGS	755
42.4.5	SHELL_HISTORY_COUNT	755
42.4.6	SHELL_HANDLE_SIZE	755
42.4.7	SHELL_USE_COMMON_TASK	756
42.4.8	SHELL_TASK_PRIORITY	756
42.4.9	SHELL_TASK_STACK_SIZE	756
42.4.10	SHELL_PRINT_COPYRIGHT	756
42.4.11	SHELL_HANDLE_DEFINE	756
42.4.12	SHELL_COMMAND_DEFINE	756

Section No.	Title	Page No.
42.4.13	SHELL_COMMAND	757
42.5	Typedef Documentation	757
42.5.1	cmd_function_t	757
42.5.2	shell_command_t	757
42.6	Enumeration Type Documentation	757
42.6.1	_shell_status	757
42.7	Function Documentation	758
42.7.1	SHELL_Init	758
42.7.2	SHELL_RegisterCommand	758
42.7.3	SHELL_UnregisterCommand	759
42.7.4	SHELL_Write	759
42.7.5	SHELL_Printf	760
42.7.6	SHELL_WriteSynchronization	760
42.7.7	SHELL_PrintfSynchronization	760
42.7.8	SHELL_ChangePrompt	761
42.7.9	SHELL_PrintPrompt	761
42.7.10	SHELL_Task	761
42.7.11	SHELL_checkRunningInIsr	762
 Chapter 43 CODEC Driver		
43.1	Overview	763
43.2	CODEC Common Driver	764
43.2.1	Overview	764
43.2.2	Data Structure Documentation	769
43.2.3	Macro Definition Documentation	770
43.2.4	Typedef Documentation	770
43.2.5	Enumeration Type Documentation	770
43.2.6	Function Documentation	775
43.3	CODEC I2C Driver	781
43.3.1	Overview	781
43.3.2	Data Structure Documentation	782
43.3.3	Typedef Documentation	782
43.3.4	Enumeration Type Documentation	782
43.3.5	Function Documentation	783
 Chapter 44 Serial Manager		
44.1	Overview	786
44.2	Data Structure Documentation	789

Section No.	Title	Page No.
44.2.1	struct _serial_manager_config	789
44.2.2	struct _serial_manager_callback_message	790
44.3	Macro Definition Documentation	790
44.3.1	SERIAL_MANAGER_WRITE_TIME_DELAY_DEFAULT_VALUE	790
44.3.2	SERIAL_MANAGER_READ_TIME_DELAY_DEFAULT_VALUE	790
44.3.3	SERIAL_MANAGER_USE_COMMON_TASK	790
44.3.4	SERIAL_MANAGER_HANDLE_SIZE	790
44.3.5	SERIAL_MANAGER_HANDLE_DEFINE	790
44.3.6	SERIAL_MANAGER_WRITE_HANDLE_DEFINE	791
44.3.7	SERIAL_MANAGER_READ_HANDLE_DEFINE	791
44.3.8	SERIAL_MANAGER_TASK_PRIORITY	792
44.3.9	SERIAL_MANAGER_TASK_STACK_SIZE	792
44.4	Enumeration Type Documentation	792
44.4.1	_serial_port_type	792
44.4.2	_serial_manager_type	792
44.4.3	_serial_manager_status	792
44.5	Function Documentation	793
44.5.1	SerialManager_Init	793
44.5.2	SerialManager_Deinit	794
44.5.3	SerialManager_OpenWriteHandle	795
44.5.4	SerialManager_CloseWriteHandle	796
44.5.5	SerialManager_OpenReadHandle	797
44.5.6	SerialManager_CloseReadHandle	798
44.5.7	SerialManager_WriteBlocking	799
44.5.8	SerialManager_ReadBlocking	800
44.5.9	SerialManager_WriteNonBlocking	800
44.5.10	SerialManager_ReadNonBlocking	801
44.5.11	SerialManager_TryRead	802
44.5.12	SerialManager_CancelWriting	803
44.5.13	SerialManager_CancelReading	803
44.5.14	SerialManager_InstallTxCallback	804
44.5.15	SerialManager_InstallRxCallback	804
44.5.16	SerialManager_needPollingIsr	805
44.5.17	SerialManager_EnterLowpower	805
44.5.18	SerialManager_ExitLowpower	805
44.5.19	SerialManager_SetLowpowerCriticalCb	806
Chapter 45 Lpspi_cmsis_driver		
45.1	Function groups	807
45.1.1	LPSPi CMSIS GetVersion Operation	807
45.1.2	LPSPi CMSIS GetCapabilities Operation	807

Section No.	Title	Page No.
45.1.3	LPSPI CMSIS Initialize and Uninitialize Operation	807
45.1.4	LPSPI Transfer Operation	807
45.1.5	LPSPI Status Operation	807
45.1.6	LPSPI CMSIS Control Operation	808
45.2	Typical use case	808
45.2.1	Master Operation	808
45.2.2	Slave Operation	808
 Chapter 46 Lpuart_cmsis_driver		
46.1	Function groups	809
46.1.1	LPUART CMSIS GetVersion Operation	809
46.1.2	LPUART CMSIS GetCapabilities Operation	809
46.1.3	LPUART CMSIS Initialize and Uninitialize Operation	809
46.1.4	LPUART CMSIS Transfer Operation	809
46.1.5	LPUART CMSIS Status Operation	810
46.1.6	LPUART CMSIS Control Operation	810
 Chapter 47 Flexio_edma_i2s		
47.1	Overview	811
47.2	Data Structure Documentation	812
47.2.1	struct_flexio_i2s_edma_handle	812
47.3	Macro Definition Documentation	813
47.3.1	FSL_FLEXIO_I2S_EDMA_DRIVER_VERSION	813
47.4	Function Documentation	813
47.4.1	FLEXIO_I2S_TransferTxCreateHandleEDMA	813
47.4.2	FLEXIO_I2S_TransferRxCreateHandleEDMA	813
47.4.3	FLEXIO_I2S_TransferSetFormatEDMA	814
47.4.4	FLEXIO_I2S_TransferSendEDMA	814
47.4.5	FLEXIO_I2S_TransferReceiveEDMA	815
47.4.6	FLEXIO_I2S_TransferAbortSendEDMA	815
47.4.7	FLEXIO_I2S_TransferAbortReceiveEDMA	815
47.4.8	FLEXIO_I2S_TransferGetSendCountEDMA	816
47.4.9	FLEXIO_I2S_TransferGetReceiveCountEDMA	816
 Chapter 48 Flexio_edma_spi		
48.1	Overview	817
48.2	Data Structure Documentation	818

Section No.	Title	Page No.
48.2.1	struct _flexio_spi_master_edma_handle	818
48.3	Macro Definition Documentation	818
48.3.1	FSL_FLEXIO_SPI_EDMA_DRIVER_VERSION	818
48.4	Typedef Documentation	819
48.4.1	flexio_spi_master_edma_handle_t	819
48.4.2	flexio_spi_slave_edma_handle_t	819
48.5	Function Documentation	819
48.5.1	FLEXIO_SPI_MasterTransferCreateHandleEDMA	819
48.5.2	FLEXIO_SPI_MasterTransferEDMA	819
48.5.3	FLEXIO_SPI_MasterTransferAbortEDMA	820
48.5.4	FLEXIO_SPI_MasterTransferGetCountEDMA	820
48.5.5	FLEXIO_SPI_SlaveTransferCreateHandleEDMA	821
48.5.6	FLEXIO_SPI_SlaveTransferEDMA	822
48.5.7	FLEXIO_SPI_SlaveTransferAbortEDMA	822
48.5.8	FLEXIO_SPI_SlaveTransferGetCountEDMA	823
 Chapter 49 Flexio_edma_uart		
49.1	Overview	824
49.2	Data Structure Documentation	825
49.2.1	struct _flexio_uart_edma_handle	825
49.3	Macro Definition Documentation	825
49.3.1	FSL_FLEXIO_UART_EDMA_DRIVER_VERSION	825
49.4	Typedef Documentation	826
49.4.1	flexio_uart_edma_transfer_callback_t	826
49.5	Function Documentation	826
49.5.1	FLEXIO_UART_TransferCreateHandleEDMA	826
49.5.2	FLEXIO_UART_TransferSendEDMA	826
49.5.3	FLEXIO_UART_TransferReceiveEDMA	827
49.5.4	FLEXIO_UART_TransferAbortSendEDMA	827
49.5.5	FLEXIO_UART_TransferAbortReceiveEDMA	828
49.5.6	FLEXIO_UART_TransferGetSendCountEDMA	829
49.5.7	FLEXIO_UART_TransferGetReceiveCountEDMA	829
 Chapter 50 Lpspi_edma_driver		
50.1	Overview	831
50.2	Data Structure Documentation	832

Section No.	Title	Page No.
50.2.1	struct_lpspi_master_edma_handle	832
50.2.2	struct_lpspi_slave_edma_handle	835
50.3	Macro Definition Documentation	837
50.3.1	FSL_LPSPI_EDMA_DRIVER_VERSION	837
50.4	Typedef Documentation	837
50.4.1	lpspi_master_edma_transfer_callback_t	837
50.4.2	lpspi_slave_edma_transfer_callback_t	838
50.5	Function Documentation	838
50.5.1	LPSPI_MasterTransferCreateHandleEDMA	838
50.5.2	LPSPI_MasterTransferEDMA	839
50.5.3	LPSPI_MasterTransferPrepareEDMALite	839
50.5.4	LPSPI_MasterTransferEDMALite	840
50.5.5	LPSPI_MasterTransferAbortEDMA	841
50.5.6	LPSPI_MasterTransferGetCountEDMA	841
50.5.7	LPSPI_SlaveTransferCreateHandleEDMA	841
50.5.8	LPSPI_SlaveTransferEDMA	842
50.5.9	LPSPI_SlaveTransferAbortEDMA	843
50.5.10	LPSPI_SlaveTransferGetCountEDMA	844
 Chapter 51 Lpuart_edma_driver		
51.1	Overview	845
51.2	Data Structure Documentation	846
51.2.1	struct_lpuart_edma_handle	846
51.3	Macro Definition Documentation	846
51.3.1	FSL_LPUART_EDMA_DRIVER_VERSION	846
51.4	Typedef Documentation	847
51.4.1	lpuart_edma_transfer_callback_t	847
51.5	Function Documentation	847
51.5.1	LPUART_TransferCreateHandleEDMA	847
51.5.2	LPUART_SendEDMA	847
51.5.3	LPUART_ReceiveEDMA	848
51.5.4	LPUART_TransferAbortSendEDMA	848
51.5.5	LPUART_TransferAbortReceiveEDMA	849
51.5.6	LPUART_TransferGetSendCountEDMA	849
51.5.7	LPUART_TransferGetReceiveCountEDMA	849
51.5.8	LPUART_TransferEdmaHandleIRQ	850

Section No.	Title	Page No.
Chapter 52 Ltc_edma_driver		
52.1	Overview	851
52.2	Data Structure Documentation	851
52.2.1	struct_ltc_edma_handle	851
52.3	Macro Definition Documentation	853
52.3.1	FSL_LTC_EDMA_DRIVER_VERSION	853
52.4	Typedef Documentation	854
52.4.1	ltc_edma_callback_t	854
52.4.2	ltc_edma_state_machine_t	854
52.5	Function Documentation	854
52.5.1	LTC_CreateHandleEDMA	854
Chapter 53 Ltc_edma_driver_aes		
53.1	Overview	855
53.2	Function Documentation	855
53.2.1	LTC_AES_EncryptEcbEDMA	855
53.2.2	LTC_AES_DecryptEcbEDMA	856
53.2.3	LTC_AES_EncryptCbcEDMA	857
53.2.4	LTC_AES_DecryptCbcEDMA	858
53.2.5	LTC_AES_CryptCtrEDMA	859
Chapter 54 Qspi_edma_driver		
54.1	Overview	861
54.2	Data Structure Documentation	862
54.2.1	struct_qspi_edma_handle	862
54.3	Macro Definition Documentation	862
54.3.1	FSL_QSPI_EDMA_DRIVER_VERSION	862
54.4	Function Documentation	862
54.4.1	QSPI_TransferTxCreateHandleEDMA	862
54.4.2	QSPI_TransferRxCreateHandleEDMA	863
54.4.3	QSPI_TransferSendEDMA	863
54.4.4	QSPI_TransferReceiveEDMA	863
54.4.5	QSPI_TransferAbortSendEDMA	864
54.4.6	QSPI_TransferAbortReceiveEDMA	864
54.4.7	QSPI_TransferGetSendCountEDMA	864

Section No.	Title	Page No.
54.4.8	QSPI_TransferGetReceiveCountEDMA	865
54.5	SAI EDMA Driver	866
54.5.1	Overview	866
54.5.2	Data Structure Documentation	867
54.5.3	Enumeration Type Documentation	868
54.5.4	Function Documentation	869
54.6	Da7212	878
54.6.1	Overview	878
54.6.2	Data Structure Documentation	881
54.6.3	Macro Definition Documentation	883
54.6.4	Enumeration Type Documentation	883
54.6.5	Function Documentation	885
54.6.6	Da7212_adapter	890
54.6.7	CODEC Adapter	898
54.6.8	Sgtl5000_adapter	899
54.6.9	Wm8960_adapter	907
54.7	Sgtl5000	915
54.7.1	Overview	915
54.7.2	Data Structure Documentation	917
54.7.3	Macro Definition Documentation	919
54.7.4	Typedef Documentation	919
54.7.5	Enumeration Type Documentation	919
54.7.6	Function Documentation	921
54.8	Wm8960	927
54.8.1	Overview	927
54.8.2	Data Structure Documentation	931
54.8.3	Macro Definition Documentation	932
54.8.4	Typedef Documentation	932
54.8.5	Enumeration Type Documentation	932
54.8.6	Function Documentation	935
54.9	Serial_port_swo	941
54.9.1	Overview	941
54.9.2	Data Structure Documentation	941
54.9.3	Enumeration Type Documentation	942
54.10	Serial_port_uart	943
54.10.1	Overview	943
54.10.2	Enumeration Type Documentation	943

Chapter 1

Introduction

The MCUXpresso Software Development Kit (MCUXpresso SDK) is a collection of software enablement for NXP Microcontrollers that includes peripheral drivers, multicore support and integrated RTOS support for FreeRTOS™. In addition to the base enablement, the MCUXpresso SDK is augmented with demo applications, driver example projects, and API documentation to help users quickly leverage the support provided by MCUXpresso SDK. The [MCUXpresso SDK Web Builder](#) is available to provide access to all MCUXpresso SDK packages. See the *MCUXpresso Software Development Kit (SDK) Release Notes* (document MCUXSDKRN) in the Supported Devices section at [MCUXpresso-SDK: Software Development Kit for MCUXpresso](#) for details.

The MCUXpresso SDK is built with the following runtime software components:

- Arm® and DSP standard libraries, and CMSIS-compliant device header files which provide direct access to the peripheral registers.
- Peripheral drivers that provide stateless, high-performance, ease-of-use APIs. Communication drivers provide higher-level transactional APIs for a higher-performance option.
- RTOS wrapper driver built on top of MCUXpresso SDK peripheral drivers and leverage native RTOS services to better comply to the RTOS cases.
- Real time operation systems (RTOS) for FreeRTOS OS.
- Stacks and middleware in source or object formats including:
- CMSIS-DSP, a suite of common signal processing functions.
- The MCUXpresso SDK comes complete with software examples demonstrating the usage of the peripheral drivers, RTOS wrapper drivers, middleware, and RTOSes.

The peripheral drivers and RTOS driver wrappers can be used across multiple devices within the product family without modification. The configuration items for each driver are encapsulated into C language data structures. Device-specific configuration information is provided as part of the MCUXpresso SDK and need not be modified by the user. If necessary, the user is able to modify the peripheral driver and RTOS wrapper driver configuration during runtime. The driver examples demonstrate how to configure the drivers by passing the proper configuration data to the APIs. The folder structure is organized to reduce the total number of includes required to compile a project.

The rest of this document describes the API references in detail for the peripheral drivers and RTOS wrapper drivers. For the latest version of this and other MCUXpresso SDK documents, see the mcuxpresso.nxp.com/apidoc/.

Deliverable	Location
Demo Applications	<install_dir>/boards/<board_name>/demo_apps
Driver Examples	<install_dir>/boards/<board_name>/driver_examples
Documentation	<install_dir>/docs
Middleware	<install_dir>/middleware
Drivers	<install_dir>/<device_name>/drivers/
CMSIS Standard Arm Cortex-M Headers, math and DSP Libraries	<install_dir>/CMSIS
Device Startup and Linker	<install_dir>/<device_name>/<toolchain>/
MCUXpresso SDK Utilities	<install_dir>/devices/<device_name>/utilities
RTOS Kernel Code	<install_dir>/rtos

MCUXpresso SDK Folder Structure

Chapter 2

Trademarks

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

How to Reach Us:

Home Page: nxp.com

Web Support: nxp.com/support

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. “Typical” parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including “typicals,” must be validated for each customer application by customer’s technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. AMBA, Arm, Arm7, Arm7TDMI, Arm9, Arm11, Artisan, big.LITTLE, Cordio, CoreLink, CoreSight, Cortex, DesignStart, DynamIQ, Jazelle, Keil, Mali, Mbed, Mbed Enabled, NEON, POP, RealView, SecurCore, Socrates, Thumb, TrustZone, ULINK, ULINK2, ULINK-ME, ULINK-PLUS, ULINKpro, Vision, Versatile are trademarks or registered trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. The related technology may be protected by any or all of patents, copyrights, designs and trade secrets. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2021 NXP B.V.

Chapter 3

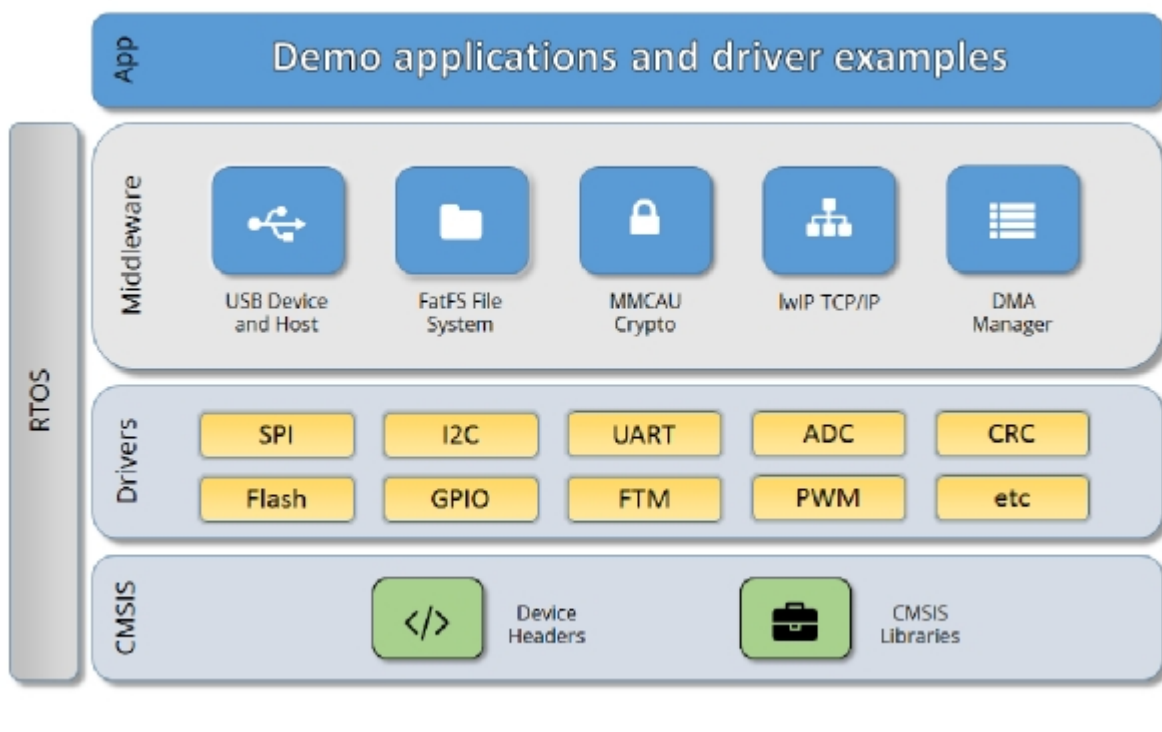
Architectural Overview

This chapter provides the architectural overview for the MCUXpresso Software Development Kit (MCUXpresso SDK). It describes each layer within the architecture and its associated components.

Overview

The MCUXpresso SDK architecture consists of five key components listed below.

1. The Arm Cortex Microcontroller Software Interface Standard (CMSIS) CORE compliance device-specific header files, SOC Header, and CMSIS math/DSP libraries.
2. Peripheral Drivers
3. Real-time Operating Systems (RTOS)
4. Stacks and Middleware that integrate with the MCUXpresso SDK
5. Demo Applications based on the MCUXpresso SDK



MCUXpresso SDK Block Diagram

MCU header files

Each supported MCU device in the MCUXpresso SDK has an overall System-on Chip (SoC) memory-

mapped header file. This header file contains the memory map and register base address for each peripheral and the IRQ vector table with associated vector numbers. The overall SoC header file provides access to the peripheral registers through pointers and predefined bit masks. In addition to the overall SoC memory-mapped header file, the MCUXpresso SDK includes a feature header file for each device. The feature header file allows NXP to deliver a single software driver for a given peripheral. The feature file ensures that the driver is properly compiled for the target SOC.

CMSIS Support

Along with the SoC header files and peripheral extension header files, the MCUXpresso SDK also includes common CMSIS header files for the Arm Cortex-M core and the math and DSP libraries from the latest CMSIS release. The CMSIS DSP library source code is also included for reference.

MCUXpresso SDK Peripheral Drivers

The MCUXpresso SDK peripheral drivers mainly consist of low-level functional APIs for the MCU product family on-chip peripherals and also of high-level transactional APIs for some bus drivers/DM-A driver/eDMA driver to quickly enable the peripherals and perform transfers.

All MCUXpresso SDK peripheral drivers only depend on the CMSIS headers, device feature files, `fsl_common.h`, and `fsl_clock.h` files so that users can easily pull selected drivers and their dependencies into projects. With the exception of the clock/power-relevant peripherals, each peripheral has its own driver. Peripheral drivers handle the peripheral clock gating/ungating inside the drivers during initialization and deinitialization respectively.

Low-level functional APIs provide common peripheral functionality, abstracting the hardware peripheral register accesses into a set of stateless basic functional operations. These APIs primarily focus on the control, configuration, and function of basic peripheral operations. The APIs hide the register access details and various MCU peripheral instantiation differences so that the application can be abstracted from the low-level hardware details. The API prototypes are intentionally similar to help ensure easy portability across supported MCUXpresso SDK devices.

Transactional APIs provide a quick method for customers to utilize higher-level functionality of the peripherals. The transactional APIs utilize interrupts and perform asynchronous operations without user intervention. Transactional APIs operate on high-level logic that requires data storage for internal operation context handling. However, the Peripheral Drivers do not allocate this memory space. Rather, the user passes in the memory to the driver for internal driver operation. Transactional APIs ensure the NVIC is enabled properly inside the drivers. The transactional APIs do not meet all customer needs, but provide a baseline for development of custom user APIs.

Note that the transactional drivers never disable an NVIC after use. This is due to the shared nature of interrupt vectors on devices. It is up to the user to ensure that NVIC interrupts are properly disabled after usage is complete.

Interrupt handling for transactional APIs

A double weak mechanism is introduced for drivers with transactional API. The double weak indicates two levels of weak vector entries. See the examples below:

```
PUBWEAK SPI0_IRQHandler
PUBWEAK SPI0_DriverIRQHandler
SPI0_IRQHandler
```

```
LDR    R0, =SPI0_DriverIRQHandler
BX     R0
```

The first level of the weak implementation are the functions defined in the vector table. In the devices/⟨DEVICE_NAME⟩/⟨TOOLCHAIN⟩/startup_⟨DEVICE_NAME⟩.S file, the implementation of the first layer weak function calls the second layer of weak function. The implementation of the second layer weak function (ex. SPI0_DriverIRQHandler) jumps to itself (B). The MCUXpresso SDK drivers with transactional APIs provide the reimplement of the second layer function inside of the peripheral driver. If the MCUXpresso SDK drivers with transactional APIs are linked into the image, the SPI0_DriverIRQHandler is replaced with the function implemented in the MCUXpresso SDK SPI driver.

The reason for implementing the double weak functions is to provide a better user experience when using the transactional APIs. For drivers with a transactional function, call the transactional APIs and the drivers complete the interrupt-driven flow. Users are not required to redefine the vector entries out of the box. At the same time, if users are not satisfied by the second layer weak function implemented in the MCUXpresso SDK drivers, users can redefine the first layer weak function and implement their own interrupt handler functions to suit their implementation.

The limitation of the double weak mechanism is that it cannot be used for peripherals that share the same vector entry. For this use case, redefine the first layer weak function to enable the desired peripheral interrupt functionality. For example, if the MCU's UART0 and UART1 share the same vector entry, redefine the UART0_UART1_IRQHandler according to the use case requirements.

Feature Header Files

The peripheral drivers are designed to be reusable regardless of the peripheral functional differences from one MCU device to another. An overall Peripheral Feature Header File is provided for the MCUXpresso SDK-supported MCU device to define the features or configuration differences for each sub-family device.

Application

See the *Getting Started with MCUXpresso SDK* document (MCUXSDKGSUG).

Chapter 4

Clock Driver

4.1 Overview

The MCUXpresso SDK provides APIs for MCUXpresso SDK devices' clock operation.

The clock driver supports:

- Clock generator (SOSC, FRO, PLL and so on) configuration
- Clock mux and divider configuration
- Getting clock frequency

Modules

- [System Clock Generator \(SCG\)](#)

Files

- file [fsl_clock.h](#)

Data Structures

- struct [_cgc_rtd_sys_clk_config](#)
CGC system clock configuration for RTD. [More...](#)
- struct [_cgc_hifi_sys_clk_config](#)
CGC system clock configuration for HIFI4 DSP. [More...](#)
- struct [_cgc_lpav_sys_clk_config](#)
CGC system clock configuration for LPAV. [More...](#)
- struct [_cgc_ddr_sys_clk_config](#)
CGC system clock configuration for DDR in LPAV. [More...](#)
- struct [_cgc_sosc_config](#)
CGC system OSC configuration. [More...](#)
- struct [_cgc_fro_config](#)
CGC FRO clock configuration. [More...](#)
- struct [_cgc_lposc_config](#)
CGC LPOSC clock configuration. [More...](#)
- struct [_cgc_pll0_config](#)
CGC PLL0 configuration. [More...](#)
- struct [_cgc_rosc_config](#)
CGC RTC OSC configuration. [More...](#)
- struct [_cgc_pll1_config](#)
CGC PLL1 configuration. [More...](#)
- struct [_cgc_pll4_config](#)
CGC PLL4 configuration. [More...](#)

Macros

- #define **FSL_SDK_DISABLE_DRIVER_CLOCK_CONTROL** 0
Configure whether driver controls clock.
- #define **CGC_PLLPFD_PFD_VAL**(pfdClkout, fracValue) (((uint32_t)((uint32_t)(fracValue) << (uint32_t)(pfdClkout)))
CGC (A/S)PLL PFD[PFDx] value.
- #define **CGC_PLLPFD_PFD_MASK**(pfdClkout) (((uint32_t)((uint32_t)(CGC_PLL0PFD_CFG_PFD0_MASK) << (uint32_t)(pfdClkout)))
CGC (A/S)PLL PFD[PFD] mask.
- #define **CGC_PLLPFD_PFD_VALID_MASK**(pfdClkout) (((uint32_t)((uint32_t)CGC_PLL0PFD_CFG_PFD0_VALID_MASK << (uint32_t)(pfdClkout)))
CGC (A/S)PLL PFD[PFDx_VALID] mask.
- #define **CGC_PLLPFD_PFD_CLKGATE_MASK**(pfdClkout) (((uint32_t)((uint32_t)CGC_PLL0PFD_CFG_PFD0_CLKGATE_MASK << (uint32_t)(pfdClkout)))
CGC (A/S)PLL PFD[PFDx_CLKGATE] mask.
- #define **PCC_CLKCFG_PCD_MASK** (0x7U)
Re-define PCC register masks and bitfield operations to unify the different namings in the soc header file.
- #define **PCC_PCS_VAL**(reg) (((reg)&PCC_CLKCFG_PCS_MASK) >> PCC_CLKCFG_PCS_SHIFT)
Bitfield values for general PCC registers.
- #define **CLOCK_IP_SOURCE_PCC_INDEX**(idx) ((idx) << 8U)
Clock source index macros for clock_ip_src_t.
- #define **PCC_PCS_AVAIL_MASK** (0x2U)
Define PCC bit available mask for clock_ip_name_t.
- #define **IP_NAME_NON_PCC_FLAG_MASK** ((uint32_t)1U << 30)
Define Non-PCC register flag mask for clock_ip_name_t.
- #define **PCC_REG**(name) (*(volatile uint32_t *)((uint32_t)(name) & ~(PCC_PCS_AVAIL_MASK | PCC_PCD_FRAC_AVAIL_MASK)))
Define PCC register content for clock_ip_name_t.
- #define **RGPIO_CLOCKS**
Clock ip name array for RGPIOP.
- #define **SAI_CLOCKS**
Clock ip name array for SAI.
- #define **PCTL_CLOCKS**
Clock ip name array for PCTL.
- #define **LPI2C_CLOCKS**
Clock ip name array for LPI2C.
- #define **I3C_CLOCKS**
Clock ip name array for I3C.
- #define **FLEXIO_CLOCKS**
Clock ip name array for FLEXIO.
- #define **FLEXCAN_CLOCKS**
Clock ip name array for FLEXCAN.
- #define **PDM_CLOCKS**
Clock ip name array for PDM.
- #define **LCDIF_CLOCKS**
Clock ip name array for LCDIF/DCNANO.
- #define **MIPI_DSI_HOST_CLOCKS**
Clock ip name array for MIPI DSI.
- #define **EDMA_CLOCKS**

- *Clock ip name array for EDMA.*
- #define [EDMA_CHAN_CLOCKS](#)
- *Clock ip name array for EDMA Channels.*
- #define [LPUART_CLOCKS](#)
- *Clock ip name array for LPUART.*
- #define [DAC_CLOCKS](#)
- *Clock ip name array for DAC.*
- #define [LPTMR_CLOCKS](#)
- *Clock ip name array for LPTMR.*
- #define [LPADC_CLOCKS](#)
- *Clock ip name array for LPADC.*
- #define [LPSPI_CLOCKS](#)
- *Clock ip name array for LPSPI.*
- #define [TPM_CLOCKS](#)
- *Clock ip name array for TPM.*
- #define [LPIT_CLOCKS](#)
- *Clock ip name array for LPIT.*
- #define [CMP_CLOCKS](#)
- *Clock ip name array for CMP.*
- #define [WDOG_CLOCKS](#)
- *Clock ip name array for MU.*
- #define [SEMA42_CLOCKS](#)
- *Clock ip name array for SEMA42.*
- #define [TPIU_CLOCKS](#)
- *Clock ip name array for TPIU.*
- #define [FLEXSPI_CLOCKS](#)
- *Clock ip name array for QSPI.*
- #define [MRT_CLOCKS](#)
- *Clock ip name array for MRT.*
- #define [BBNSM_CLOCKS](#)
- *Clock ip name array for BBNSM.*
- #define [PXP_CLOCKS](#)
- *Clock ip name array for PXP.*
- #define [EPDC_CLOCKS](#)
- *Clock ip name array for EPDC.*

Typedefs

- typedef enum [_clock_name](#) [clock_name_t](#)
Clock name used to get clock frequency.
- typedef enum [_clock_ip_src](#) [clock_ip_src_t](#)
Clock source for peripherals that support various clock selections.
- typedef enum [_clock_lptmr_src](#) [clock_lptmr_src_t](#)
Clock source for LPTMR.
- typedef enum [_clock_ip_name](#) [clock_ip_name_t](#)
Peripheral clock name difinition used for clock gate, clock source and clock divider setting.
- typedef enum [_cgc_sys_clk](#) [cgc_sys_clk_t](#)
CGC system clock type.
- typedef enum [_cgc_rtd_sys_clk_src](#) [cgc_rtd_sys_clk_src_t](#)
CGC system clock source for RTD.
- typedef enum [_cgc_nic_sys_clk_src](#) [cgc_nic_sys_clk_src_t](#)

- *CGC system clock source for NIC in AD.*
- typedef enum `_cgc_hifi_sys_clk_src cgc_hifi_sys_clk_src_t`
CGC system clock source for HIFI4 in LPAV.
- typedef enum `_cgc_lpav_sys_clk_src cgc_lpav_sys_clk_src_t`
CGC system clock source for LPAV.
- typedef enum `_cgc_ddr_sys_clk_src cgc_ddr_sys_clk_src_t`
CGC system clock source for DDR.
- typedef struct `_cgc_rtd_sys_clk_config cgc_rtd_sys_clk_config_t`
CGC system clock configuration for RTD.
- typedef struct `_cgc_hifi_sys_clk_config cgc_hifi_sys_clk_config_t`
CGC system clock configuration for HIFI4 DSP.
- typedef struct `_cgc_lpav_sys_clk_config cgc_lpav_sys_clk_config_t`
CGC system clock configuration for LPAV.
- typedef struct `_cgc_ddr_sys_clk_config cgc_ddr_sys_clk_config_t`
CGC system clock configuration for DDR in LPAV.
- typedef enum `_clock_rtd_clkout_src clock_rtd_clkout_src_t`
CGC clock out configuration (CLKOUTCFG) in RTD.
- typedef enum `_clock_lpav_clkout_src clock_lpav_clkout_src_t`
CGC clock out configuration (CLKOUTCFG) in LPAV.
- typedef enum `_cgc_async_clk cgc_async_clk_t`
CGC asynchronous clock type.
- typedef enum `_cgc_sosc_monitor_mode cgc_sosc_monitor_mode_t`
CGC system OSC monitor mode.
- typedef enum `_cgc_sosc_mode cgc_sosc_mode_t`
OSC work mode.
- typedef struct `_cgc_sosc_config cgc_sosc_config_t`
CGC system OSC configuration.
- typedef struct `_cgc_fro_config cgc_fro_config_t`
CGC FRO clock configuration.
- typedef struct `_cgc_lposc_config cgc_lposc_config_t`
CGC LPOSC clock configuration.
- typedef enum `_cgc_pll_src cgc_pll_src_t`
CGC PLL clock source.
- typedef enum `_cgc_pll_pfd_clkout cgc_pll_pfd_clkout_t`
CGC PLL PFD clouk out select.
- typedef enum `_cgc_pll0_mult cgc_pll0_mult_t`
PLL0 Multiplication Factor.
- typedef struct `_cgc_pll0_config cgc_pll0_config_t`
CGC PLL0 configuration.
- typedef enum `_cgc_rosc_monitor_mode cgc_rosc_monitor_mode_t`
CGC RTC OSC monitor mode.
- typedef struct `_cgc_rosc_config cgc_rosc_config_t`
CGC RTC OSC configuration.
- typedef enum `_cgc_pll1_mult cgc_pll1_mult_t`
PLL1 Multiplication Factor.
- typedef struct `_cgc_pll1_config cgc_pll1_config_t`
CGC PLL1 configuration.

- typedef enum [_cgc_pll4_mult](#) cgc_pll4_mult_t
PLL4 Multiplication Factor.
- typedef struct [_cgc_pll4_config](#) cgc_pll4_config_t
CGC PLL4 configuration.
- typedef enum [_cgc_rtd_audclk_src](#) cgc_rtd_audclk_src_t
AUD_CLK0 source in RTD.
- typedef enum [_cgc_ad_audclk_src](#) cgc_ad_audclk_src_t
AUD_CLK1 source in AD.
- typedef enum [_cgc_lpav_audclk_src](#) cgc_lpav_audclk_src_t
AUD_CLK2 source in LPAV.

Enumerations

- enum `_clock_name` {
 - `kCLOCK_Cm33CorePlatClk,`
 - `kCLOCK_Cm33BusClk,`
 - `kCLOCK_Cm33SlowClk,`
 - `kCLOCK_FusionDspCorePlatClk,`
 - `kCLOCK_FusionDspBusClk,`
 - `kCLOCK_FusionDspSlowClk,`
 - `kCLOCK_XbarBusClk,`
 - `kCLOCK_HifiDspClk,`
 - `kCLOCK_HifiNicPlatClk,`
 - `kCLOCK_NicLpavAxiClk,`
 - `kCLOCK_NicLpavAhbClk,`
 - `kCLOCK_NicLpavBusClk,`
 - `kCLOCK_DdrClk,`
 - `kCLOCK_SysOscClk,`
 - `kCLOCK_FroClk,`
 - `kCLOCK_LpOscClk,`
 - `kCLOCK_RtcOscClk,`
 - `kCLOCK_LvdsClk,`
 - `kCLOCK_RtdFroDiv1Clk,`
 - `kCLOCK_RtdFroDiv2Clk,`
 - `kCLOCK_RtdFroDiv3Clk,`
 - `kCLOCK_RtdSysOscDiv1Clk,`
 - `kCLOCK_RtdSysOscDiv2Clk,`
 - `kCLOCK_RtdSysOscDiv3Clk,`
 - `kCLOCK_AdFroDiv1Clk,`
 - `kCLOCK_AdFroDiv2Clk,`
 - `kCLOCK_AdFroDiv3Clk,`
 - `kCLOCK_AdSysOscDiv1Clk,`
 - `kCLOCK_AdSysOscDiv2Clk,`
 - `kCLOCK_AdSysOscDiv3Clk,`
 - `kCLOCK_LpavFroDiv1Clk,`
 - `kCLOCK_LpavFroDiv2Clk,`
 - `kCLOCK_LpavFroDiv3Clk,`
 - `kCLOCK_LpavSysOscDiv1Clk,`
 - `kCLOCK_LpavSysOscDiv2Clk,`
 - `kCLOCK_LpavSysOscDiv3Clk,`
 - `kCLOCK_Pll0Clk,`
 - `kCLOCK_Pll1Clk,`
 - `kCLOCK_Pll3Clk,`
 - `kCLOCK_Pll4Clk,`
 - `kCLOCK_Pll0Pfd0Clk,`
 - `kCLOCK_Pll0Pfd1Clk,`
 - `kCLOCK_Pll0Pfd2Clk,`
 - `kCLOCK_Pll0Pfd3Clk,`
 - `kCLOCK_Pll0Pfd4Clk,`
 - `kCLOCK_Pll1Pfd1Clk,`
 - `kCLOCK_Pll1Pfd2Clk,`

`kCLOCK_Pll4Pfd3Div2Clk }`

Clock name used to get clock frequency.

- `enum _clock_ip_src {`

```

kCLOCK_IpSrcNone = 0U,
kCLOCK_Pcc0PlatIpSrcSysOscDiv1,
kCLOCK_Pcc0PlatIpSrcFroDiv1,
kCLOCK_Pcc0PlatIpSrcCm33Plat,
kCLOCK_Pcc0PlatIpSrcFro,
kCLOCK_Pcc0PlatIpSrcPll0Pfd3,
kCLOCK_Pcc0BusIpSrcLpo = CLOCK_IP_SOURCE_PCC_INDEX(0U) | 1U,
kCLOCK_Pcc0BusIpSrcSysOscDiv2,
kCLOCK_Pcc0BusIpSrcFroDiv2,
kCLOCK_Pcc0BusIpSrcCm33Bus,
kCLOCK_Pcc0BusIpSrcPll1Pfd1Div,
kCLOCK_Pcc0BusIpSrcPll0Pfd2Div,
kCLOCK_Pcc0BusIpSrcPll0Pfd1Div,
kCLOCK_Pcc1PlatIpSrcSysOscDiv1,
kCLOCK_Pcc1PlatIpSrcFroDiv1,
kCLOCK_Pcc1PlatIpSrcCm33Plat,
kCLOCK_Pcc1PlatIpSrcFro,
kCLOCK_Pcc1PlatIpSrcPll0Pfd3,
kCLOCK_Pcc1BusIpSrcLpo = CLOCK_IP_SOURCE_PCC_INDEX(1U) | 1U,
kCLOCK_Pcc1BusIpSrcSysOscDiv2,
kCLOCK_Pcc1BusIpSrcFroDiv2,
kCLOCK_Pcc1BusIpSrcCm33Bus,
kCLOCK_Pcc1BusIpSrcPll1VcoDiv,
kCLOCK_Pcc1BusIpSrcPll0Pfd2Div,
kCLOCK_Pcc1BusIpSrcPll0Pfd1Div,
kCLOCK_Pcc2BusIpSrcLpo = CLOCK_IP_SOURCE_PCC_INDEX(2U) | 1U,
kCLOCK_Pcc2BusIpSrcSysOscDiv3,
kCLOCK_Pcc2BusIpSrcFroDiv3,
kCLOCK_Pcc2BusIpSrcFusionDspBus,
kCLOCK_Pcc2BusIpSrcPll1VcoDiv,
kCLOCK_Pcc2BusIpSrcPll0Pfd2Div,
kCLOCK_Pcc2BusIpSrcPll0Pfd1Div,
kCLOCK_Pcc3BusIpSrcLpo = CLOCK_IP_SOURCE_PCC_INDEX(3U) | 1U,
kCLOCK_Pcc3BusIpSrcSysOscDiv2,
kCLOCK_Pcc3BusIpSrcFroDiv2,
kCLOCK_Pcc3BusIpSrcXbarBus = CLOCK_IP_SOURCE_PCC_INDEX(3U) | 4U,
kCLOCK_Pcc3BusIpSrcPll3Pfd1Div1,
kCLOCK_Pcc3BusIpSrcPll3Pfd0Div2,
kCLOCK_Pcc3BusIpSrcPll3Pfd0Div1,
kCLOCK_Pcc4PlatIpSrcSysOscDiv1,
kCLOCK_Pcc4PlatIpSrcFroDiv1,
kCLOCK_Pcc4PlatIpSrcPll3Pfd3Div2,
kCLOCK_Pcc4PlatIpSrcPll3Pfd3Div1,
kCLOCK_Pcc4PlatIpSrcPll3Pfd2Div2,
kCLOCK_Pcc4PlatIpSrcPll3Pfd2Div1,
kCLOCK_Pcc4PlatIpSrcPll3Pfd1Div2,
kCLOCK_Pcc4BusIpSrcLpo = CLOCK_IP_SOURCE_PCC_INDEX(4U) | 2U,
kCLOCK_Pcc4BusIpSrcSysOscDiv2,
kCLOCK_Pcc4BusIpSrcFroDiv2,

```

`kCLOCK_LpavTpm8ClkSrcPll4Pfd3Div1 }`

Clock source for peripherals that support various clock selections.

- `enum _clock_lptmr_src {`
`kCLOCK_LptmrSrcLPO1M = 0U,`
`kCLOCK_LptmrSrcRtc1K = 1U,`
`kCLOCK_LptmrSrcRtc32K = 2U,`
`kCLOCK_LptmrSrcSysOsc = 3U }`

Clock source for LPTMR.

- `enum _clock_ip_name`
Peripheral clock name definition used for clock gate, clock source and clock divider setting.
- `enum {`
`kStatus_CGC_Busy = MAKE_STATUS(kStatusGroup_SCG, 1),`
`kStatus_CGC_InvalidSrc = MAKE_STATUS(kStatusGroup_SCG, 2) }`

CGC status return codes.

- `enum _cgc_sys_clk {`
`kCGC_SysClkSlow,`
`kCGC_SysClkBus,`
`kCGC_SysClkCorePlat,`
`kCGC_SysClkHifi4,`
`kCGC_SysClkNicHifi,`
`kCGC_SysClkLpavAxi,`
`kCGC_SysClkLpavAhb,`
`kCGC_SysClkLpavBus }`

CGC system clock type.

- `enum _cgc_rtd_sys_clk_src {`
`kCGC_RtdSysClkSrcFro = 0U,`
`kCGC_RtdSysClkSrcPll0Pfd0 = 1U,`
`kCGC_RtdSysClkSrcPll1Pfd0 = 2U,`
`kCGC_RtdSysClkSrcSysOsc = 3U,`
`kCGC_RtdSysClkSrcRtcOsc = 4U,`
`kCGC_RtdSysClkSrcLvds = 5U,`
`kCGC_RtdSysClkSrcPll0 = 6U }`

CGC system clock source for RTD.

- `enum _cgc_nic_sys_clk_src {`
`kCGC_NicSysClkSrcFro = 0U,`
`kCGC_NicSysClkSrcPll3Pfd0 = 1U,`
`kCGC_NicSysClkSrcSysOsc = 2U,`
`kCGC_NicSysClkSrcLvds = 3U }`

CGC system clock source for NIC in AD.

- `enum _cgc_hifi_sys_clk_src {`
`kCGC_HifiSysClkSrcFro = 0U,`
`kCGC_HifiSysClkSrcPll4 = 1U,`
`kCGC_HifiSysClkSrcPll4Pfd0 = 2U,`
`kCGC_HifiSysClkSrcSysOsc = 3U,`
`kCGC_HifiSysClkSrcLvds = 4U }`

CGC system clock source for HIFI4 in LPAV.

- `enum _cgc_lpav_sys_clk_src {`

- ```

kCGC_LpavSysClkSrcFro = 0U,
kCGC_LpavSysClkSrcPll4Pfd1 = 1U,
kCGC_LpavSysClkSrcSysOsc = 2U,
kCGC_LpavSysClkSrcLvds = 3U }
 CGC system clock source for LPAV.

```
- ```
enum _cgc_ddr_sys_clk_src {
    kCGC_DdrSysClkSrcFro = 0U,
    kCGC_DdrSysClkSrcPll4Pfd1 = 1U,
    kCGC_DdrSysClkSrcSysOsc = 2U,
    kCGC_DdrSysClkSrcLvds = 3U }
    CGC system clock source for DDR.
```
 - ```
enum _clock_rtd_clkout_src {
 kClockRtdClkoutSelCm33Core = 0U,
 kClockRtdClkoutSelCm33Bus = 1U,
 kClockRtdClkoutSelCm33Slow = 2U,
 kClockRtdClkoutSelFusionDspCore = 3U,
 kClockRtdClkoutSelFusionDspBus = 4U,
 kClockRtdClkoutSelFusionDspSlow = 5U,
 kClockRtdClkoutSelFro48 = 6U,
 kClockRtdClkoutSelPll0VcoDiv = 7U,
 kClockRtdClkoutSelPll1VcoDiv = 8U,
 kClockRtdClkoutSelSysOsc = 9U,
 kClockRtdClkoutSelLpOsc = 10U }
 CGC clock out configuration (CLKOUTCFG) in RTD.
```
  - ```
enum _clock_lpav_clkout_src {
    kClockLpavClkoutSelHifi4 = 0U,
    kClockLpavClkoutSelNicHifi = 1U,
    kClockLpavClkoutSelLpavAxi = 2U,
    kClockLpavClkoutSelLpavAhb = 3U,
    kClockLpavClkoutSelLpavBus = 4U,
    kClockLpavClkoutSelDdr = 5U,
    kClockLpavClkoutSelFro48 = 6U,
    kClockLpavClkoutSelPll4VcoDiv = 7U,
    kClockLpavClkoutSelSysOsc = 9U,
    kClockLpavClkoutSelLpOsc = 10U }
    CGC clock out configuration (CLKOUTCFG) in LPAV.
```
 - ```
enum _cgc_async_clk {
```



- kCGC\_AsyncDiv1Clk = 1U,
- kCGC\_AsyncDiv2Clk = 2U,
- kCGC\_AsyncDiv3Clk = 3U,
- kCGC\_AsyncVcoClk = 4U,
- kCGC\_AsyncPfd0Div1Clk = 5U,
- kCGC\_AsyncPfd0Div2Clk = 6U,
- kCGC\_AsyncPfd1Div1Clk = 7U,
- kCGC\_AsyncPfd1Div2Clk = 8U,
- kCGC\_AsyncPfd2Div1Clk = 9U,
- kCGC\_AsyncPfd2Div2Clk = 10U,
- kCGC\_AsyncPfd3Div1Clk = 11U,
- kCGC\_AsyncPfd3Div2Clk = 12U }
- CGC asynchronous clock type.*
- enum \_cgc\_sosc\_monitor\_mode {  
kCGC\_SysOscMonitorDisable = 0U,  
kCGC\_SysOscMonitorInt = CGC\_SOSCCSR\_SOSCCM\_MASK,  
kCGC\_SysOscMonitorReset }
- CGC system OSC monitor mode.*
- enum \_cgc\_sosc\_mode {  
kCGC\_SysOscModeExt = CGC\_SOSCCFG\_SYSOSC\_BYPASS\_EN\_MASK,  
kCGC\_SysOscModeOscLowPower = 0,  
kCGC\_SysOscModeOscHighGain = CGC\_SOSCCFG\_HGO\_MASK }
- OSC work mode.*
- enum \_cgc\_sosc\_enable\_mode {  
kCGC\_SysOscEnableInDeepSleep = CGC\_SOSCCSR\_SOSCDSEN\_MASK,  
kCGC\_SysOscEnableInPowerDown = CGC\_SOSCCSR\_SOSCPDEN\_MASK }
- OSC enable mode.*
- enum \_cgc\_fro\_enable\_mode { kCGC\_FroEnableInDeepSleep = CGC\_FROCSR\_FRODSSEN\_MASK }
- FRO enable mode.*
- enum \_cgc\_lposc\_enable\_mode {  
kCGC\_LposcEnableInDeepSleep = CGC\_LPOSCCSR\_LPOSCDSEN\_MASK,  
kCGC\_LposcEnableInPowerDown = CGC\_LPOSCCSR\_LPOSCPDEN\_MASK }
- LPOSC enable mode.*
- enum \_cgc\_pll\_src {  
kCGC\_PllSrcSysOsc,  
kCGC\_PllSrcFro24M }
- CGC PLL clock source.*
- enum \_cgc\_pll\_enable\_mode {  
kCGC\_PllEnable = CGC\_PLL0CSR\_PLL0EN\_MASK,  
kCGC\_PllEnableInDeepSleep = CGC\_PLL0CSR\_PLL0DSEN\_MASK }
- PLL enable mode.*
- enum \_cgc\_pll\_pfd\_clkout {  
kCGC\_PllPfd0Clk = 0U,  
kCGC\_PllPfd1Clk = 8U,  
kCGC\_PllPfd2Clk = 16U,  
kCGC\_PllPfd3Clk = 24U }

*CGC PLL PFD clouk out select.*

- enum `_cgc_pll0_mult` {  
`kCGC_Pll0Mult15` = 1U,  
`kCGC_Pll0Mult16` = 2U,  
`kCGC_Pll0Mult20` = 3U,  
`kCGC_Pll0Mult22` = 4U,  
`kCGC_Pll0Mult25` = 5U,  
`kCGC_Pll0Mult30` = 6U }

*PLL0 Multiplication Factor.*

- enum `_cgc_rosc_monitor_mode` {  
`kCGC_RtcOscMonitorDisable` = 0U,  
`kCGC_RtcOscMonitorInt` = CGC\_ROSCCTRL\_ROSCCM\_MASK,  
`kCGC_RtcOscMonitorReset` }

*CGC RTC OSC monitor mode.*

- enum `_cgc_pll1_mult` {  
`kCGC_Pll1Mult16` = 16U,  
`kCGC_Pll1Mult17` = 17U,  
`kCGC_Pll1Mult20` = 20U,  
`kCGC_Pll1Mult22` = 22U,  
`kCGC_Pll1Mult27` = 27U,  
`kCGC_Pll1Mult33` = 33U }

*PLL1 Multiplication Factor.*

- enum `_cgc_pll4_mult` {  
`kCGC_Pll4Mult16` = 16U,  
`kCGC_Pll4Mult17` = 17U,  
`kCGC_Pll4Mult20` = 20U,  
`kCGC_Pll4Mult22` = 22U,  
`kCGC_Pll4Mult27` = 27U,  
`kCGC_Pll4Mult33` = 33U }

*PLL4 Multiplication Factor.*

- enum `_cgc_rtd_audclk_src` {  
`kCGC_RtdAudClkSrcExtMclk0` = 0,  
`kCGC_RtdAudClkSrcExtMclk1` = 1,  
`kCGC_RtdAudClkSrcSai0RxBclk` = 2,  
`kCGC_RtdAudClkSrcSai0TxBclk` = 3,  
`kCGC_RtdAudClkSrcSai1RxBclk` = 4,  
`kCGC_RtdAudClkSrcSai1TxBclk` = 5,  
`kCGC_RtdAudClkSrcSai2RxBclk` = 6,  
`kCGC_RtdAudClkSrcSai2TxBclk` = 7,  
`kCGC_RtdAudClkSrcSai3RxBclk` = 8,  
`kCGC_RtdAudClkSrcSai3TxBclk` = 9 }

*AUD\_CLK0 source in RTD.*

- enum `_cgc_ad_audclk_src` {

```

kCGC_AdAudClkSrcExtMclk2 = 0,
kCGC_AdAudClkSrcSai4RxBclk = 1,
kCGC_AdAudClkSrcSai4TxBclk = 2,
kCGC_AdAudClkSrcSai5RxBclk = 3,
kCGC_AdAudClkSrcSai5TxBclk = 4 }

```

*AUD\_CLK1 source in AD.*

- enum `_cgc_lpav_audclk_src` {  
`kCGC_LpavAudClkSrcExtMclk3` = 0,  
`kCGC_LpavAudClkSrcSai6RxBclk` = 1,  
`kCGC_LpavAudClkSrcSai6TxBclk` = 2,  
`kCGC_LpavAudClkSrcSai7RxBclk` = 3,  
`kCGC_LpavAudClkSrcSai7TxBclk` = 4,  
`kCGC_LpavAudClkSrcSpdifRx` = 5 }

*AUD\_CLK2 source in LPAV.*

## Functions

- static void `CLOCK_EnableClock` (`clock_ip_name_t` name)  
*Enable the clock for specific IP.*
- static void `CLOCK_DisableClock` (`clock_ip_name_t` name)  
*Disable the clock for specific IP.*
- static bool `CLOCK_IsEnabledByOtherCore` (`clock_ip_name_t` name)  
*Check whether the clock is already enabled and configured by any other core.*
- void `CLOCK_SetIpSrc` (`clock_ip_name_t` name, `clock_ip_src_t` src)  
*Set the clock source for specific IP module.*
- void `CLOCK_SetIpSrcDiv` (`clock_ip_name_t` name, `clock_ip_src_t` src, `uint8_t` divValue, `uint8_t` fracValue)  
*Set the clock source and divider for specific IP module.*
- static void `CLOCK_SetRtdAudClkSrc` (`cgc_rtd_audclk_src_t` src)  
*Set the AUD\_CLK0 source in RTD.*
- static void `CLOCK_SetAdAudClkSrc` (`cgc_ad_audclk_src_t` src)  
*Set the AUD\_CLK1 source in AD.*
- static void `CLOCK_SetLpavAudClkSrc` (`cgc_lpav_audclk_src_t` src)  
*Set the AUD\_CLK2 source in LPAV.*
- `uint32_t` `CLOCK_GetFreq` (`clock_name_t` clockName)  
*Gets the clock frequency for a specific clock name.*
- `uint32_t` `CLOCK_GetCm33CorePlatClkFreq` (void)  
*Get the CM33 core/platform clock frequency.*
- `uint32_t` `CLOCK_GetCm33BusClkFreq` (void)  
*Get the CM33 bus clock frequency.*
- `uint32_t` `CLOCK_GetCm33SlowClkFreq` (void)  
*Get the CM33 slow clock frequency.*
- `uint32_t` `CLOCK_GetFusionDspCorePlatClkFreq` (void)  
*Get the Fusion DSP core/platform clock frequency.*
- `uint32_t` `CLOCK_GetFusionDspBusClkFreq` (void)  
*Get the Fusion DSP bus clock frequency.*
- `uint32_t` `CLOCK_GetFusionDspSlowClkFreq` (void)  
*Get the Fusion DSP slow clock frequency.*
- `uint32_t` `CLOCK_GetLvdsClkFreq` (void)  
*Get the external LVDS pad clock frequency (LVDS).*

- uint32\_t [CLOCK\\_GetIpFreq](#) ([clock\\_ip\\_name\\_t](#) name)  
*Gets the clock frequency for a specific IP module.*
- uint32\_t [CLOCK\\_GetRtdSysClkFreq](#) (uint32\_t config, [cgc\\_sys\\_clk\\_t](#) type)  
*Gets the CGC system clock frequency in RTD.*

## Variables

- volatile uint32\_t [g\\_xtal0Freq](#)  
*External XTAL (SYSOSC) clock frequency.*
- volatile uint32\_t [g\\_xtal32Freq](#)  
*External XTAL32/EXTAL32 clock frequency.*
- volatile uint32\_t [g\\_lvdsFreq](#)  
*External LVDS pad clock frequency.*
- volatile uint32\_t [g\\_mclkFreq](#) [4]  
*External MCLK pad clock frequency.*
- volatile uint32\_t [g\\_rxBclkFreq](#) [8]  
*External RX\_BCLK pad clock frequency.*
- volatile uint32\_t [g\\_txBclkFreq](#) [8]  
*External TX\_BCLK pad clock frequency.*
- volatile uint32\_t [g\\_spdifRxFreq](#)  
*Recovered SPDIF\_RX clock frequency.*

## Driver version

- #define [FSL\\_CLOCK\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 0, 5))  
*CLOCK driver version 2.0.5.*

## MCU System Clock.

- uint32\_t [CLOCK\\_GetCm33SysClkFreq](#) ([cgc\\_sys\\_clk\\_t](#) type)  
*Gets the CGC CM33 system clock frequency.*
- static void [CLOCK\\_SetCm33SysClkConfig](#) ([cgc\\_rtd\\_sys\\_clk\\_config\\_t](#) \*config)  
*Sets the system clock configuration for CM33 domain.*
- uint32\_t [CLOCK\\_GetFusionDspSysClkFreq](#) ([cgc\\_sys\\_clk\\_t](#) type)  
*Gets the CGC Fusion DSP system clock frequency.*
- static void [CLOCK\\_SetFusionSysClkConfig](#) (const [cgc\\_rtd\\_sys\\_clk\\_config\\_t](#) \*config)  
*Sets the system clock configuration for FusionF1 DSP domain.*
- static void [CLOCK\\_GetCm33SysClkConfig](#) ([cgc\\_rtd\\_sys\\_clk\\_config\\_t](#) \*config)  
*Gets the system clock configuration for CM33 domain.*
- static void [CLOCK\\_GetFusionDspSysClkConfig](#) ([cgc\\_rtd\\_sys\\_clk\\_config\\_t](#) \*config)  
*Gets the system clock configuration for FusionF1 DSP domain.*
- static void [CLOCK\\_SetRtdClkOutConfig](#) ([clock\\_rtd\\_clkout\\_src\\_t](#) setting, uint8\_t div, bool enable)  
*Sets the clock out configuration in RTD.*
- static void [CLOCK\\_SetRtcClkOutConfig](#) (uint8\_t div)  
*Sets the RTC\_CLOCKOUT configuration.*
- uint32\_t [CLOCK\\_GetXbarBusClkFreq](#) (void)  
*Gets the CGC XBAR bus clock frequency in AD.*
- uint32\_t [CLOCK\\_GetHifiDspSysClkFreq](#) ([cgc\\_sys\\_clk\\_t](#) type)  
*Gets the CGC HIFI DSP system clock frequency in LPAV.*
- static void [CLOCK\\_SetHifiDspSysClkConfig](#) (const [cgc\\_hifi\\_sys\\_clk\\_config\\_t](#) \*config)  
*Sets the system clock configuration for HIFI4 DSP domain.*

- static void `CLOCK_GetHifiDspSysClkConfig` (`cgc_hifi_sys_clk_config_t *config`)  
*Gets the system clock configuration for HIFI4 DSP domain.*
- `uint32_t` `CLOCK_GetLpavSysClkFreq` (`cgc_sys_clk_t` type)  
*Gets the CGC NIC LPAV system clock frequency in LPAV.*
- static void `CLOCK_SetLpavSysClkConfig` (`const cgc_lpav_sys_clk_config_t *config`)  
*Sets the system clock configuration for NIC LPAV domain.*
- static void `CLOCK_GetLpavSysClkConfig` (`cgc_lpav_sys_clk_config_t *config`)  
*Gets the system clock configuration for NIC LPAV domain.*
- `uint32_t` `CLOCK_GetDdrClkFreq` (void)  
*Gets the CGC DDR clock frequency in LPAV.*
- static void `CLOCK_SetLpavClkOutConfig` (`clock_lpav_clkout_src_t` setting, `uint8_t` div, `bool` enable)  
*Sets the clock out configuration in LPAV.*

## CGC System OSC Clock.

- `status_t` `CLOCK_InitSysOsc` (`const cgc_sosc_config_t *config`)  
*Initializes the CGC system OSC.*
- `status_t` `CLOCK_DeinitSysOsc` (void)  
*De-initializes the CGC system OSC.*
- void `CLOCK_SetRtdSysOscAsyncClkDiv` (`cgc_async_clk_t` asyncClk, `uint8_t` divider)  
*Set the asynchronous clock divider in RTD.*
- void `CLOCK_SetAdSysOscAsyncClkDiv` (`cgc_async_clk_t` asyncClk, `uint8_t` divider)  
*Set the asynchronous clock divider in AD.*
- void `CLOCK_SetLpavSysOscAsyncClkDiv` (`cgc_async_clk_t` asyncClk, `uint8_t` divider)  
*Set the asynchronous clock divider in LPAV.*
- `uint32_t` `CLOCK_GetSysOscFreq` (void)  
*Gets the CGC system OSC clock frequency (SYSOSC).*
- `uint32_t` `CLOCK_GetRtdSysOscAsyncFreq` (`cgc_async_clk_t` type)  
*Gets the CGC asynchronous clock frequency from the system OSC in RTD.*
- `uint32_t` `CLOCK_GetAdSysOscAsyncFreq` (`cgc_async_clk_t` type)  
*Gets the CGC asynchronous clock frequency from the system OSC in AD.*
- `uint32_t` `CLOCK_GetLpavSysOscAsyncFreq` (`cgc_async_clk_t` type)  
*Gets the CGC asynchronous clock frequency from the system OSC in LPAV.*
- static `bool` `CLOCK_IsSysOscErr` (void)  
*Checks whether the system OSC clock error occurs.*
- static void `CLOCK_ClearSysOscErr` (void)  
*Clears the system OSC clock error.*
- static void `CLOCK_SetSysOscMonitorMode` (`cgc_sosc_monitor_mode_t` mode)  
*Sets the system OSC monitor mode.*
- static `bool` `CLOCK_IsSysOscSelected` (void)  
*Checks whether the system OSC clock is used as clock source.*
- static `bool` `CLOCK_IsSysOscValid` (void)  
*Checks whether the system OSC clock is valid.*

## CGC FRO Clock.

- `status_t` `CLOCK_InitFro` (`const cgc_fro_config_t *config`)  
*Initializes the CGC FRO clock.*
- `status_t` `CLOCK_DeinitFro` (void)  
*De-initializes the CGC FRO.*

- void [CLOCK\\_SetRtdFroAsyncClkDiv](#) ([cgc\\_async\\_clk\\_t](#) asyncClk, [uint8\\_t](#) divider)  
*Set the asynchronous clock divider in RTD.*
- void [CLOCK\\_SetAdFroAsyncClkDiv](#) ([cgc\\_async\\_clk\\_t](#) asyncClk, [uint8\\_t](#) divider)  
*Set the asynchronous clock divider in AD.*
- void [CLOCK\\_SetLpavFroAsyncClkDiv](#) ([cgc\\_async\\_clk\\_t](#) asyncClk, [uint8\\_t](#) divider)  
*Set the asynchronous clock divider in LPAV.*
- void [CLOCK\\_EnableFroTuning](#) (bool enable)  
*Enable/Disable FRO tuning.*
- [uint32\\_t](#) [CLOCK\\_GetFroFreq](#) (void)  
*Gets the CGC FRO clock frequency.*
- [uint32\\_t](#) [CLOCK\\_GetRtdFroAsyncFreq](#) ([cgc\\_async\\_clk\\_t](#) type)  
*Gets the CGC asynchronous clock frequency from the FRO in RTD.*
- [uint32\\_t](#) [CLOCK\\_GetAdFroAsyncFreq](#) ([cgc\\_async\\_clk\\_t](#) type)  
*Gets the CGC asynchronous clock frequency from the FRO in AD.*
- [uint32\\_t](#) [CLOCK\\_GetLpavFroAsyncFreq](#) ([cgc\\_async\\_clk\\_t](#) type)  
*Gets the CGC asynchronous clock frequency from the FRO in LPAV.*
- static bool [CLOCK\\_IsFroSelected](#) (void)  
*Checks whether the FRO clock is used as clock source.*
- static bool [CLOCK\\_IsFroValid](#) (void)  
*Checks whether the FRO clock is valid.*

## CGC LPOSC Clock.

- [status\\_t](#) [CLOCK\\_InitLposc](#) (const [cgc\\_lposc\\_config\\_t](#) \*config)  
*Initializes the CGC LPOSC clock.*
- [status\\_t](#) [CLOCK\\_DeinitLposc](#) (void)  
*De-initializes the CGC LPOSC.*
- static bool [CLOCK\\_IsLpOscValid](#) (void)  
*Checks whether the LPOSC clock is valid.*
- [uint32\\_t](#) [CLOCK\\_GetLpOscFreq](#) (void)  
*Gets the CGC LPOSC clock frequency.*

## CGC RTCOSC Clock.

- [uint32\\_t](#) [CLOCK\\_GetRtcOscFreq](#) (void)  
*Gets the CGC RTC OSC clock frequency.*
- static bool [CLOCK\\_IsRtcOscErr](#) (void)  
*Checks whether the RTC OSC clock error occurs.*
- static void [CLOCK\\_ClearRtcOscErr](#) (void)  
*Clears the RTC OSC clock error.*
- void [CLOCK\\_SetRtcOscMonitorMode](#) ([cgc\\_rosc\\_monitor\\_mode\\_t](#) mode)  
*Sets the RTC OSC monitor mode.*
- static bool [CLOCK\\_IsRtcOscSelected](#) (void)  
*Checks whether the RTCOSC clock is used as clock source.*
- static bool [CLOCK\\_IsRtcOscValid](#) (void)  
*Checks whether the RTC OSC clock is valid.*

## CGC PLL0 Clock.

- [status\\_t](#) [CLOCK\\_InitPllo](#) (const [cgc\\_pll0\\_config\\_t](#) \*config)  
*Initializes the CGC PLL0.*



- `status_t CLOCK_DeinitPll0` (void)  
*De-initializes the CGC PLL0.*
- void `CLOCK_SetPll0AsyncClkDiv` (`cgc_async_clk_t` asyncClk, uint8\_t divider)  
*Set the asynchronous clock divider.*
- uint32\_t `CLOCK_GetPll0Freq` (void)  
*Gets the CGC PLL0 clock frequency.*
- uint32\_t `CLOCK_GetPll0AsyncFreq` (`cgc_async_clk_t` type)  
*Gets the CGC asynchronous clock frequency from the PLL0.*
- uint32\_t `CLOCK_GetPll0PfdFreq` (`cgc_pll_pfd_clkout_t` pfdClkout)  
*Gets the CGC PLL0 PFD clock frequency.*
- void `CLOCK_EnablePll0PfdClkout` (`cgc_pll_pfd_clkout_t` pfdClkout, uint8\_t fracValue)  
*Enables the CGC PLL0 Fractional Divide (PFD) clock out with configurations.*
- static void `CLOCK_DisablePll0PfdClkout` (`cgc_pll_pfd_clkout_t` pfdClkout)  
*Disables the CGC PLL0 Fractional Divide (PFD) clock out.*
- static void `CLOCK_SetPll0LockTime` (uint16\_t lockTime)  
*Sets the CGC PLL0 lock time.*
- static bool `CLOCK_IsPll0Selected` (void)  
*Checks whether the PLL0 clock is used as clock source.*
- static bool `CLOCK_IsPll0Valid` (void)  
*Checks whether the PLL0 clock is valid.*

## CGC PLL1 Clock.

- `status_t CLOCK_InitPll1` (const `cgc_pll1_config_t` \*config)  
*Initializes the CGC PLL1.*
- `status_t CLOCK_DeinitPll1` (void)  
*De-initializes the CGC PLL1.*
- void `CLOCK_SetPll1AsyncClkDiv` (`cgc_async_clk_t` asyncClk, uint8\_t divider)  
*Set the asynchronous clock divider.*
- uint32\_t `CLOCK_GetPll1Freq` (void)  
*Gets the CGC PLL1 clock frequency.*
- uint32\_t `CLOCK_GetPll1AsyncFreq` (`cgc_async_clk_t` type)  
*Gets the CGC asynchronous clock frequency from the PLL1.*
- uint32\_t `CLOCK_GetPll1PfdFreq` (`cgc_pll_pfd_clkout_t` pfdClkout)  
*Gets the CGC PLL1 PFD clock frequency.*
- void `CLOCK_EnablePll1PfdClkout` (`cgc_pll_pfd_clkout_t` pfdClkout, uint8\_t fracValue)  
*Enables the CGC PLL1 Fractional Divide (PFD) clock out with configurations.*
- static void `CLOCK_DisablePll1PfdClkout` (`cgc_pll_pfd_clkout_t` pfdClkout)  
*Disables the CGC PLL1 Fractional Divide (PFD) clock out.*
- static void `CLOCK_EnablePll1SpectrumModulation` (uint16\_t step, uint16\_t stop)  
*Enables the CGC PLL1 spread spectrum modulation feature with configurations.*
- static void `CLOCK_DisablePll1SpectrumModulation` (void)  
*Disables the CGC PLL1 spread spectrum modulation.*
- static void `CLOCK_SetPll1LockTime` (uint16\_t lockTime)  
*Sets the CGC PLL1 lock time.*
- static bool `CLOCK_IsPll1Selected` (void)  
*Checks whether the PLL1 clock is used as clock source.*
- static bool `CLOCK_IsPll1Valid` (void)  
*Checks whether the PLL1 clock is valid.*

## CGC PLL3 Clock.

- uint32\_t [CLOCK\\_GetPll3Freq](#) (void)  
*Gets the CGC PLL3 clock frequency.*
- uint32\_t [CLOCK\\_GetPll3AsyncFreq](#) (cgc\_async\_clk\_t type)  
*Gets the CGC asynchronous clock frequency from the PLL3.*
- uint32\_t [CLOCK\\_GetPll3PfdFreq](#) (cgc\_pll\_pfd\_clkout\_t pfdClkout)  
*Gets the CGC PLL3 PFD clock frequency.*

## CGC PLL4 Clock.

- status\_t [CLOCK\\_InitPll4](#) (const cgc\_pll4\_config\_t \*config)  
*Initializes the CGC PLL4.*
- status\_t [CLOCK\\_DeinitPll4](#) (void)  
*De-initializes the CGC PLL4.*
- void [CLOCK\\_SetPll4AsyncClkDiv](#) (cgc\_async\_clk\_t asyncClk, uint8\_t divider)  
*Set the asynchronous clock divider.*
- uint32\_t [CLOCK\\_GetPll4Freq](#) (void)  
*Gets the CGC PLL4 clock frequency.*
- uint32\_t [CLOCK\\_GetPll4AsyncFreq](#) (cgc\_async\_clk\_t type)  
*Gets the CGC asynchronous clock frequency from the PLL4.*
- uint32\_t [CLOCK\\_GetPll4PfdFreq](#) (cgc\_pll\_pfd\_clkout\_t pfdClkout)  
*Gets the CGC PLL4 PFD clock frequency.*
- void [CLOCK\\_EnablePll4PfdClkout](#) (cgc\_pll\_pfd\_clkout\_t pfdClkout, uint8\_t fracValue)  
*Enables the CGC PLL4 Fractional Divide (PFD) clock out with configurations.*
- static void [CLOCK\\_DisablePll4PfdClkout](#) (cgc\_pll\_pfd\_clkout\_t pfdClkout)  
*Disables the CGC PLL4 Fractional Divide (PFD) clock out.*
- static void [CLOCK\\_EnablePll4SpectrumModulation](#) (uint16\_t step, uint16\_t stop)  
*Enables the CGC PLL4 spread spectrum modulation feature with configurations.*
- static void [CLOCK\\_SetPll4LockTime](#) (uint16\_t lockTime)  
*Sets the CGC PLL4 lock time.*
- static bool [CLOCK\\_IsPll4Selected](#) (void)  
*Checks whether the PLL4 clock is used as clock source.*
- static bool [CLOCK\\_IsPll4Valid](#) (void)  
*Checks whether the PLL4 clock is valid.*

## External clock frequency

- static void [CLOCK\\_SetXtal0Freq](#) (uint32\_t freq)  
*Sets the XTAL0 frequency based on board settings.*
- static void [CLOCK\\_SetXtal32Freq](#) (uint32\_t freq)  
*Sets the XTAL32 frequency based on board settings.*
- static void [CLOCK\\_SetLvdsFreq](#) (uint32\_t freq)  
*Sets the LVDS pad frequency based on board settings.*
- static void [CLOCK\\_SetMclkFreq](#) (uint32\_t index, uint32\_t freq)  
*Sets the MCLK pad frequency based on Audio settings.*
- static void [CLOCK\\_SetRxBclkFreq](#) (uint32\_t instance, uint32\_t freq)  
*Sets the RX\_BCLK pad frequency based on Audio settings.*
- static void [CLOCK\\_SetTxBclkFreq](#) (uint32\_t instance, uint32\_t freq)  
*Sets the TX\_BCLK pad frequency based on Audio settings.*
- static void [CLOCK\\_SetSpdifRxFreq](#) (uint32\_t freq)  
*Sets the SPDIF\_RX frequency based on Audio settings.*



## Get peripheral frequency

- uint32\_t [CLOCK\\_GetWdogClkFreq](#) (uint32\_t instance)  
*Gets the WDOG clock frequency in RTD and LPAV.*
- uint32\_t [CLOCK\\_GetFlexspiClkFreq](#) (uint32\_t instance)  
*Gets the FlexSPI clock frequency in RTD.*
- uint32\_t [CLOCK\\_GetLpitClkFreq](#) (void)  
*Gets the LPIT clock frequency in RTD.*
- uint32\_t [CLOCK\\_GetFlexioClkFreq](#) (void)  
*Gets the FlexIO clock frequency in RTD.*
- uint32\_t [CLOCK\\_GetI3cClkFreq](#) (uint32\_t instance)  
*Gets the I3C clock frequency in RTD and LPAV.*
- uint32\_t [CLOCK\\_GetLpspiClkFreq](#) (uint32\_t instance)  
*Gets the LPSPI clock frequency in RTD.*
- uint32\_t [CLOCK\\_GetAdcClkFreq](#) (uint32\_t instance)  
*Gets the ADC clock frequency.*
- uint32\_t [CLOCK\\_GetDacClkFreq](#) (uint32\_t instance)  
*Gets the DAC clock frequency.*
- uint32\_t [CLOCK\\_GetTpiuClkFreq](#) (void)  
*Gets the TPIU clock frequency.*
- uint32\_t [CLOCK\\_GetSwoClkFreq](#) (void)  
*Gets the SWO clock frequency.*
- uint32\_t [CLOCK\\_GetTpmClkFreq](#) (uint32\_t instance)  
*Gets the TPM clock frequency.*
- uint32\_t [CLOCK\\_GetLpi2cClkFreq](#) (uint32\_t instance)  
*Gets the LPI2C clock frequency in RTD.*
- uint32\_t [CLOCK\\_GetLpuartClkFreq](#) (uint32\_t instance)  
*Gets the LPUART clock frequency in RTD.*
- uint32\_t [CLOCK\\_GetFlexcanClkFreq](#) (void)  
*Gets the FlexCAN clock frequency.*
- uint32\_t [CLOCK\\_GetCsiClkFreq](#) (void)  
*Gets the CSI clock frequency.*
- uint32\_t [CLOCK\\_GetDsiClkFreq](#) (void)  
*Gets the DSI clock frequency.*
- uint32\_t [CLOCK\\_GetEpdcClkFreq](#) (void)  
*Gets the EPDC clock frequency.*
- uint32\_t [CLOCK\\_GetGpu2dClkFreq](#) (void)  
*Gets the GPU2D clock frequency.*
- uint32\_t [CLOCK\\_GetGpu3dClkFreq](#) (void)  
*Gets the GPU3D clock frequency.*
- uint32\_t [CLOCK\\_GetDcnanoClkFreq](#) (void)  
*Gets the DC Nano clock frequency.*
- uint32\_t [CLOCK\\_GetCsiUiClkFreq](#) (void)  
*Gets the CSI clk\_ui clock frequency.*
- uint32\_t [CLOCK\\_GetCsiEscClkFreq](#) (void)  
*Gets the CSI clk\_esc clock frequency.*
- uint32\_t [CLOCK\\_GetRtdAudClkFreq](#) (void)  
*Gets the audio clock frequency in RTD.*
- uint32\_t [CLOCK\\_GetAdAudClkFreq](#) (void)  
*Gets the audio clock frequency in AD.*
- uint32\_t [CLOCK\\_GetLpavAudClkFreq](#) (void)  
*Gets the audio clock frequency in LPAV.*

- uint32\_t [CLOCK\\_GetSaiFreq](#) (uint32\_t instance)  
*Gets the SAI clock frequency.*
- uint32\_t [CLOCK\\_GetSpdifFreq](#) (void)  
*Gets the SPDIF clock frequency.*
- uint32\_t [CLOCK\\_GetMqsFreq](#) (uint32\_t instance)  
*Gets the MQS clock frequency.*
- uint32\_t [CLOCK\\_GetMicfilFreq](#) (void)  
*Gets the EMICFIL clock frequency.*
- uint32\_t [CLOCK\\_GetMrtFreq](#) (void)  
*Gets the MRT clock frequency.*

## 4.2 Data Structure Documentation

### 4.2.1 struct \_cgc\_rtd\_sys\_clk\_config

#### Data Fields

- uint32\_t [divSlow](#): 6  
*Slow clock divider, selected division is the value of the field + 1.*
- uint32\_t [\\_\\_pad0\\_\\_](#): 1  
*Reserved.*
- uint32\_t [divBus](#): 6  
*Bus clock divider, selected division is the value of the field + 1.*
- uint32\_t [\\_\\_pad1\\_\\_](#): 8  
*Reserved.*
- uint32\_t [divCore](#): 6  
*Core/Platform clock divider, selected division is the value of the field + 1.*
- uint32\_t [switchFin](#): 1  
*1: Clock is running.*
- uint32\_t [src](#): 3  
*System clock source, see [cgc\\_rtd\\_sys\\_clk\\_src\\_t](#).*
- uint32\_t [locked](#): 1  
*Clock register locked.*

#### Field Documentation

- (1) uint32\_t \_cgc\_rtd\_sys\_clk\_config::\_\_pad0\_\_
- (2) uint32\_t \_cgc\_rtd\_sys\_clk\_config::divBus
- (3) uint32\_t \_cgc\_rtd\_sys\_clk\_config::\_\_pad1\_\_
- (4) uint32\_t \_cgc\_rtd\_sys\_clk\_config::divCore
- (5) uint32\_t \_cgc\_rtd\_sys\_clk\_config::switchFin

0: Clock is not running.

(6) `uint32_t _cgc_rtd_sys_clk_config::src`

(7) `uint32_t _cgc_rtd_sys_clk_config::locked`

#### 4.2.2 `struct _cgc_hifi_sys_clk_config`

##### Data Fields

- `uint32_t __pad0__`: 14  
*Reserved.*
- `uint32_t divPlat`: 6  
*Platform clock divider, selected division is the value of the field + 1.*
- `uint32_t __pad1__`: 1  
*Reserved.*
- `uint32_t divCore`: 6  
*Core clock divider, selected division is the value of the field + 1.*
- `uint32_t switchFin`: 1  
*1: Clock is running.*
- `uint32_t src`: 3  
*System clock source, see `cgc_hifi_sys_clk_src_t`.*
- `uint32_t locked`: 1  
*Clock register locked.*

##### Field Documentation

(1) `uint32_t _cgc_hifi_sys_clk_config::__pad0__`

(2) `uint32_t _cgc_hifi_sys_clk_config::divPlat`

(3) `uint32_t _cgc_hifi_sys_clk_config::__pad1__`

(4) `uint32_t _cgc_hifi_sys_clk_config::divCore`

(5) `uint32_t _cgc_hifi_sys_clk_config::switchFin`

0: Clock is not running.

(6) `uint32_t _cgc_hifi_sys_clk_config::src`

(7) `uint32_t _cgc_hifi_sys_clk_config::locked`

#### 4.2.3 `struct _cgc_lpav_sys_clk_config`

##### Data Fields

- `uint32_t __pad0__`: 7  
*Reserved.*
- `uint32_t divBus`: 6  
*Platform clock divider, selected division is the value of the field + 1.*

- uint32\_t `__pad1__`: 1  
*Reserved.*
- uint32\_t `divAhb`: 6  
*Platform clock divider, selected division is the value of the field + 1.*
- uint32\_t `__pad2__`: 1  
*Reserved.*
- uint32\_t `divAxi`: 6  
*Core clock divider, selected division is the value of the field + 1.*
- uint32\_t `switchFin`: 1  
*1: Clock is running.*
- uint32\_t `src`: 2  
*System clock source, see [cgc\\_lpav\\_sys\\_clk\\_src\\_t](#).*
- uint32\_t `__pad3__`: 1  
*Reserved.*
- uint32\_t `locked`: 1  
*Clock register locked.*

### Field Documentation

- (1) uint32\_t `_cgc_lpav_sys_clk_config::__pad0__`
  - (2) uint32\_t `_cgc_lpav_sys_clk_config::divBus`
  - (3) uint32\_t `_cgc_lpav_sys_clk_config::__pad1__`
  - (4) uint32\_t `_cgc_lpav_sys_clk_config::divAhb`
  - (5) uint32\_t `_cgc_lpav_sys_clk_config::__pad2__`
  - (6) uint32\_t `_cgc_lpav_sys_clk_config::divAxi`
  - (7) uint32\_t `_cgc_lpav_sys_clk_config::switchFin`
- 0: Clock is not running.
- (8) uint32\_t `_cgc_lpav_sys_clk_config::src`
  - (9) uint32\_t `_cgc_lpav_sys_clk_config::__pad3__`
  - (10) uint32\_t `_cgc_lpav_sys_clk_config::locked`

### 4.2.4 struct `_cgc_ddr_sys_clk_config`

#### Data Fields

- uint32\_t `__pad0__`: 21  
*Reserved.*
- uint32\_t `divDdr`: 6  
*DDR clock divider, selected division is the value of the field + 1.*
- uint32\_t `switchFin`: 1

- 1: Clock is running.
- uint32\_t [src](#): 3  
System clock source, see [cgc\\_lpav\\_sys\\_clk\\_src\\_t](#).
- uint32\_t [locked](#): 1  
Clock register locked.

#### Field Documentation

(1) uint32\_t [\\_cgc\\_ddr\\_sys\\_clk\\_config::\\_\\_pad0\\_\\_](#)

(2) uint32\_t [\\_cgc\\_ddr\\_sys\\_clk\\_config::divDdr](#)

(3) uint32\_t [\\_cgc\\_ddr\\_sys\\_clk\\_config::switchFin](#)

0: Clock is not running.

(4) uint32\_t [\\_cgc\\_ddr\\_sys\\_clk\\_config::src](#)

(5) uint32\_t [\\_cgc\\_ddr\\_sys\\_clk\\_config::locked](#)

### 4.2.5 struct [\\_cgc\\_sosc\\_config](#)

#### Data Fields

- uint32\_t [freq](#)  
System OSC frequency.
- [cgc\\_sosc\\_monitor\\_mode\\_t](#) [monitorMode](#)  
Clock monitor mode selected.
- uint8\_t [enableMode](#)  
Enable mode, OR'ed value of [\\_cgc\\_sosc\\_enable\\_mode](#).
- [cgc\\_sosc\\_mode\\_t](#) [workMode](#)  
OSC work mode.

#### Field Documentation

(1) uint32\_t [\\_cgc\\_sosc\\_config::freq](#)

(2) [cgc\\_sosc\\_monitor\\_mode\\_t](#) [\\_cgc\\_sosc\\_config::monitorMode](#)

(3) uint8\_t [\\_cgc\\_sosc\\_config::enableMode](#)

(4) [cgc\\_sosc\\_mode\\_t](#) [\\_cgc\\_sosc\\_config::workMode](#)

### 4.2.6 struct [\\_cgc\\_fro\\_config](#)

#### Data Fields

- uint32\_t [enableMode](#)  
Enable mode, OR'ed value of [\\_cgc\\_fro\\_enable\\_mode](#).

**Field Documentation****(1) uint32\_t \_cgc\_fro\_config::enableMode****4.2.7 struct \_cgc\_lposc\_config****Data Fields**

- uint32\_t [enableMode](#)  
*Enable mode, OR'ed value of \_cgc\_lposc\_enable\_mode.*

**Field Documentation****(1) uint32\_t \_cgc\_lposc\_config::enableMode****4.2.8 struct \_cgc\_pll0\_config****Data Fields**

- uint8\_t [enableMode](#)  
*Enable mode, OR'ed value of \_cgc\_pll\_enable\_mode.*
- uint8\_t [div1](#)  
*PLLDIV\_VCO divider value.*
- uint8\_t [pfd1Div](#)  
*PLLDIV\_PFD\_0 DIV1 divider value.*
- uint8\_t [pfd2Div](#)  
*PLLDIV\_PFD\_0 DIV2 divider value.*
- [cgc\\_pll\\_src\\_t](#) [src](#)  
*Clock source.*
- [cgc\\_pll0\\_mult\\_t](#) [mult](#)  
*PLL multiplier.*

**Field Documentation****(1) uint8\_t \_cgc\_pll0\_config::div1**

Disabled when div1 == 0.

**(2) uint8\_t \_cgc\_pll0\_config::pfd1Div**

Disabled when pfd1Div == 0.

**(3) uint8\_t \_cgc\_pll0\_config::pfd2Div**

Disabled when pfd2Div == 0.

(4) `cgc_pll_src_t _cgc_pll0_config::src`

(5) `cgc_pll0_mult_t _cgc_pll0_config::mult`

#### 4.2.9 struct \_cgc\_rosc\_config

##### Data Fields

- `cgc_rosc_monitor_mode_t monitorMode`  
*Clock monitor mode selected.*

##### Field Documentation

(1) `cgc_rosc_monitor_mode_t _cgc_rosc_config::monitorMode`

#### 4.2.10 struct \_cgc\_pll1\_config

##### Data Fields

- `uint8_t enableMode`  
*Enable mode, OR'ed value of \_cgc\_pll\_enable\_mode.*
- `uint8_t div1`  
*PLLDIV\_VCO divider value.*
- `uint8_t pfd1Div`  
*PLLDIV\_PFD\_0 DIV1 divider value.*
- `uint8_t pfd2Div`  
*PLLDIV\_PFD\_0 DIV2 divider value.*
- `cgc_pll_src_t src`  
*Clock source.*
- `cgc_pll1_mult_t mult`  
*PLL1 multiplier.*
- `uint32_t num: 30`  
*30-bit numerator of the PLL1 Fractional-Loop divider.*
- `uint32_t denom: 30`  
*30-bit denominator of the PLL1 Fractional-Loop divider.*

##### Field Documentation

(1) `uint8_t _cgc_pll1_config::div1`

Disabled when `div1 == 0`.

(2) `uint8_t _cgc_pll1_config::pfd1Div`

Disabled when `pfd1Div == 0`.

(3) `uint8_t _cgc_pll1_config::pfd2Div`

Disabled when `pfd2Div == 0`.

- (4) `cgc_pll_src_t _cgc_pll1_config::src`
- (5) `cgc_pll1_mult_t _cgc_pll1_config::mult`
- (6) `uint32_t _cgc_pll1_config::num`
- (7) `uint32_t _cgc_pll1_config::denom`

#### 4.2.11 struct \_cgc\_pll4\_config

##### Data Fields

- `uint8_t enableMode`  
*Enable mode, OR'ed value of \_cgc\_pll\_enable\_mode.*
- `uint8_t div1`  
*PLLDIV\_VCO divider value.*
- `uint8_t pfd0Div1`  
*PLLDIV\_PFD\_0 DIV1 divider value.*
- `uint8_t pfd0Div2`  
*PLLDIV\_PFD\_0 DIV2 divider value.*
- `uint8_t pfd1Div1`  
*PLLDIV\_PFD\_0 DIV3 divider value.*
- `uint8_t pfd1Div2`  
*PLLDIV\_PFD\_0 DIV4 divider value.*
- `uint8_t pfd2Div1`  
*PLLDIV\_PFD\_1 DIV1 divider value.*
- `uint8_t pfd2Div2`  
*PLLDIV\_PFD\_1 DIV2 divider value.*
- `uint8_t pfd3Div1`  
*PLLDIV\_PFD\_2 DIV3 divider value.*
- `uint8_t pfd3Div2`  
*PLLDIV\_PFD\_2 DIV4 divider value.*
- `cgc_pll_src_t src`  
*Clock source.*
- `cgc_pll4_mult_t mult`  
*PLL4 multiplier.*
- `uint32_t num: 30`  
*30-bit numerator of the PLL4 Fractional-Loop divider.*
- `uint32_t denom: 30`  
*30-bit denominator of the PLL4 Fractional-Loop divider.*

##### Field Documentation

###### (1) `uint8_t _cgc_pll4_config::div1`

Disabled when `div1 == 0`.

###### (2) `uint8_t _cgc_pll4_config::pfd0Div1`

Disabled when `pfd0Div1 == 0`.



**(3) uint8\_t \_cgc\_pll4\_config::pfd0Div2**

Disabled when pfd0Div2 == 0.

**(4) uint8\_t \_cgc\_pll4\_config::pfd1Div1**

Disabled when pfd1Div1 == 0.

**(5) uint8\_t \_cgc\_pll4\_config::pfd1Div2**

Disabled when pfd1Div2 == 0.

**(6) uint8\_t \_cgc\_pll4\_config::pfd2Div1**

Disabled when pfd2Div1 == 0.

**(7) uint8\_t \_cgc\_pll4\_config::pfd2Div2**

Disabled when pfd2Div2 == 0.

**(8) uint8\_t \_cgc\_pll4\_config::pfd3Div1**

Disabled when pfd3Div1 == 0.

**(9) uint8\_t \_cgc\_pll4\_config::pfd3Div2**

Disabled when pfd3Div2 == 0.

**(10) cgc\_pll\_src\_t \_cgc\_pll4\_config::src****(11) cgc\_pll4\_mult\_t \_cgc\_pll4\_config::mult****(12) uint32\_t \_cgc\_pll4\_config::num****(13) uint32\_t \_cgc\_pll4\_config::denom****4.3 Macro Definition Documentation****4.3.1 #define FSL\_SDK\_DISABLE\_DRIVER\_CLOCK\_CONTROL 0**

When set to 0, peripheral drivers will enable clock in initialize function and disable clock in de-initialize function. When set to 1, peripheral driver will not control the clock, application could control the clock out of the driver.

**Note**

All drivers share this feature switcher. If it is set to 1, application should handle clock enable and disable for all drivers.

**4.3.2 #define FSL\_CLOCK\_DRIVER\_VERSION (MAKE\_VERSION(2, 0, 5))****4.3.3 #define PCC\_PCS\_VAL( *reg* ) (((reg)&PCC\_CLKCFG\_PCS\_MASK) >> PCC\_CLKCFG\_PCS\_SHIFT)****4.3.4 #define RGPIO\_CLOCKS****Value:**

```
{
 kCLOCK_RgpioA, kCLOCK_RgpioB, kCLOCK_RgpioC, kCLOCK_RgpioD, kCLOCK_RgpioE, kCLOCK_RgpioF \
}
```

**4.3.5 #define SAI\_CLOCKS****Value:**

```
{
 \
 kCLOCK_Sai0, kCLOCK_Sai1, kCLOCK_Sai2, kCLOCK_Sai3, kCLOCK_Sai4, kCLOCK_Sai5, kCLOCK_Sai6,
 kCLOCK_Sai7 \
}
```

**4.3.6 #define PCTL\_CLOCKS****Value:**

```
{
 kCLOCK_PctlA, kCLOCK_PctlB, kCLOCK_IpInvalid, kCLOCK_IpInvalid, kCLOCK_PctlE, kCLOCK_PctlF \
}
```

**4.3.7 #define LPI2C\_CLOCKS****Value:**

```
{
 \
 kCLOCK_Lpi2c0, kCLOCK_Lpi2c1, kCLOCK_Lpi2c2, kCLOCK_Lpi2c3, kCLOCK_Lpi2c4, kCLOCK_Lpi2c5,
 kCLOCK_Lpi2c6, \
 kCLOCK_Lpi2c7
 \
}
```

### 4.3.8 #define I3C\_CLOCKS

Value:

```
{
 kCLOCK_I3c0, kCLOCK_I3c1, kCLOCK_I3c2 \
}
```

### 4.3.9 #define FLEXIO\_CLOCKS

Value:

```
{
 kCLOCK_Flexio0, kCLOCK_Flexio1 \
}
```

### 4.3.10 #define FLEXCAN\_CLOCKS

Value:

```
{
 kCLOCK_Flexcan \
}
```

### 4.3.11 #define PDM\_CLOCKS

Value:

```
{
 kCLOCK_Micfil \
}
```

### 4.3.12 #define LCDIF\_CLOCKS

Value:

```
{
 kCLOCK_Dcnano \
}
```

### 4.3.13 #define MIPI\_DSI\_HOST\_CLOCKS

Value:

```
{
 kCLOCK_Dsi \
}
```

### 4.3.14 #define EDMA\_CLOCKS

Value:

```
{
 kCLOCK_Dma0, kCLOCK_Dma1, kCLOCK_Dma2 \
}
```

### 4.3.15 #define EDMA\_CHAN\_CLOCKS

Value:

```
{
 kCLOCK_Dma0Ch0, kCLOCK_Dma1Ch0, kCLOCK_Dma2Ch0 \
}
```

### 4.3.16 #define LPUART\_CLOCKS

Value:

```
{
 kCLOCK_Lpuart0, kCLOCK_Lpuart1, kCLOCK_Lpuart2, kCLOCK_Lpuart3, kCLOCK_Lpuart4, kCLOCK_Lpuart5, \
 kCLOCK_Lpuart6, kCLOCK_Lpuart7 \
}
```

### 4.3.17 #define DAC\_CLOCKS

Value:

```
{
 kCLOCK_Dac0, kCLOCK_Dac1 \
}
```

### 4.3.18 #define LPTMR\_CLOCKS

Value:

```
{
 kCLOCK_Lptmr0, kCLOCK_Lptmr1 \
}
```

### 4.3.19 #define LPADC\_CLOCKS

Value:

```
{
 kCLOCK_Adc0, kCLOCK_Adc1 \
}
```

### 4.3.20 #define LPSPI\_CLOCKS

Value:

```
{
 kCLOCK_Lpspi0, kCLOCK_Lpspi1, kCLOCK_Lpspi2, kCLOCK_Lpspi3, kCLOCK_Lpspi4, kCLOCK_Lpspi5 \
}
```

### 4.3.21 #define TPM\_CLOCKS

Value:

```
{
 kCLOCK_Tpm0, kCLOCK_Tpm1, kCLOCK_Tpm2, kCLOCK_Tpm3, kCLOCK_Tpm4, kCLOCK_Tpm5, kCLOCK_Tpm6,
 kCLOCK_Tpm7, \
 kCLOCK_Tpm8 \
}
```

### 4.3.22 #define LPIT\_CLOCKS

Value:

```
{
 kCLOCK_Lpit0, kCLOCK_Lpit1 \
}
```

### 4.3.23 #define CMP\_CLOCKS

Value:

```
{
 kCLOCK_Cmp0, kCLOCK_Cmp1 \
}
```

### 4.3.24 #define WDOG\_CLOCKS

Value:

```
{
 kCLOCK_Wdog0, kCLOCK_Wdog1, kCLOCK_Wdog2, kCLOCK_Wdog3, kCLOCK_Wdog4, kCLOCK_Wdog5 \
}
```

Clock ip name array for WDOG.

### 4.3.25 #define SEMA42\_CLOCKS

Value:

```
{
 kCLOCK_Sema420, kCLOCK_Sema421, kCLOCK_Sema422 \
}
```

### 4.3.26 #define TPIU\_CLOCKS

Value:

```
{
 kCLOCK_Tpiu \
}
```

### 4.3.27 #define FLEXSPI\_CLOCKS

Value:

```
{
 kCLOCK_Flexspi0, kCLOCK_Flexspi1, kCLOCK_Flexspi2 \
}
```

### 4.3.28 #define MRT\_CLOCKS

Value:

```
{
 \
 kCLOCK_Mrt \
}
```

### 4.3.29 #define BBNSM\_CLOCKS

Value:

```
{
 \
 kCLOCK_Bbnsn \
}
```

### 4.3.30 #define PXP\_CLOCKS

Value:

```
{
 \
 kCLOCK_Pxp \
}
```

### 4.3.31 #define EPDC\_CLOCKS

Value:

```
{
 \
 kCLOCK_Epdc \
}
```

## 4.4 Typedef Documentation

### 4.4.1 typedef enum \_clock\_name clock\_name\_t

### 4.4.2 typedef enum \_clock\_ip\_name clock\_ip\_name\_t

[31:2] is defined as the corresponding register address. [ 1:1] is used as indicator of existing of PCS. [ 0:0] is used as indicator of existing of PCD/FRC.

#### 4.4.3 typedef enum \_cgc\_sosc\_mode cgc\_sosc\_mode\_t

### 4.5 Enumeration Type Documentation

#### 4.5.1 enum \_clock\_name

Enumerator

*kCLOCK\_Cm33CorePlatClk* RTD : CM33 Core/Platform clock.  
*kCLOCK\_Cm33BusClk* RTD : CM33 Bus clock.  
*kCLOCK\_Cm33SlowClk* RTD : CM33 Slow clock.  
*kCLOCK\_FusionDspCorePlatClk* RTD : FusionF1 DSP Core/Platform clock.  
*kCLOCK\_FusionDspBusClk* RTD : FusionF1 DSP Bus clock.  
*kCLOCK\_FusionDspSlowClk* RTD : FusionF1 DSP Slow clock.  
*kCLOCK\_XbarBusClk* AD : XBAR Bus clock.  
*kCLOCK\_HifiDspClk* LPAV: HIFI4 clock.  
*kCLOCK\_HifiNicPlatClk* LPAV: NIC HIFI clock.  
*kCLOCK\_NicLpavAxiClk* LPAV: NIC LPAV AXI clock.  
*kCLOCK\_NicLpavAhbClk* LPAV: NIC LPAV AHB clock.  
*kCLOCK\_NicLpavBusClk* LPAV: LPAV Bus clock.  
*kCLOCK\_DdrClk* LPAV: DDR clock.  
*kCLOCK\_SysOscClk* CGC system OSC clock. (SYSOSC)  
*kCLOCK\_FroClk* CGC FRO 192MHz clock.  
*kCLOCK\_LpOscClk* CGC LPOSC clock. (LPOSC)  
*kCLOCK\_RtcOscClk* CGC RTC OSC clock. (RTCOSC)  
*kCLOCK\_LvdsClk* LVDS pad input clock frequency.  
*kCLOCK\_RtdFroDiv1Clk* FRODIV1\_CLK in RTD.  
*kCLOCK\_RtdFroDiv2Clk* FRODIV2\_CLK in RTD.  
*kCLOCK\_RtdFroDiv3Clk* FRODIV3\_CLK in RTD.  
*kCLOCK\_RtdSysOscDiv1Clk* SOSCDIV1\_CLK in RTD.  
*kCLOCK\_RtdSysOscDiv2Clk* SOSCDIV2\_CLK in RTD.  
*kCLOCK\_RtdSysOscDiv3Clk* SOSCDIV3\_CLK in RTD.  
*kCLOCK\_AdFroDiv1Clk* FRODIV1\_CLK in AD.  
*kCLOCK\_AdFroDiv2Clk* FRODIV2\_CLK in AD.  
*kCLOCK\_AdFroDiv3Clk* FRODIV3\_CLK in AD.  
*kCLOCK\_AdSysOscDiv1Clk* SOSCDIV1\_CLK in AD.  
*kCLOCK\_AdSysOscDiv2Clk* SOSCDIV2\_CLK in AD.  
*kCLOCK\_AdSysOscDiv3Clk* SOSCDIV3\_CLK in AD.  
*kCLOCK\_LpavFroDiv1Clk* FRODIV1\_CLK in LPAV.  
*kCLOCK\_LpavFroDiv2Clk* FRODIV2\_CLK in LPAV.  
*kCLOCK\_LpavFroDiv3Clk* FRODIV3\_CLK in LPAV.  
*kCLOCK\_LpavSysOscDiv1Clk* SOSCDIV1\_CLK in LPAV.  
*kCLOCK\_LpavSysOscDiv2Clk* SOSCDIV2\_CLK in LPAV.  
*kCLOCK\_LpavSysOscDiv3Clk* SOSCDIV3\_CLK in LPAV.  
*kCLOCK\_Pll0Clk* CGC PLL0 clock. (PLL0CLK)



*kCLOCK\_Pll1Clk* CGC PLL1 clock. (PLL1CLK)  
*kCLOCK\_Pll3Clk* CGC PLL3 clock. (PLL3CLK)  
*kCLOCK\_Pll4Clk* CGC PLL4 clock. (PLL4CLK)  
*kCLOCK\_Pll0Pfd0Clk* pll0 pfd0.  
*kCLOCK\_Pll0Pfd1Clk* pll0 pfd1.  
*kCLOCK\_Pll0Pfd2Clk* pll0 pfd2.  
*kCLOCK\_Pll0Pfd3Clk* pll0 pfd3.  
*kCLOCK\_Pll1Pfd0Clk* pll1 pfd0.  
*kCLOCK\_Pll1Pfd1Clk* pll1 pfd1.  
*kCLOCK\_Pll1Pfd2Clk* pll1 pfd2.  
*kCLOCK\_Pll1Pfd3Clk* pll1 pfd3.  
*kCLOCK\_Pll3Pfd0Clk* pll3 pfd0.  
*kCLOCK\_Pll3Pfd1Clk* pll3 pfd1.  
*kCLOCK\_Pll3Pfd2Clk* pll3 pfd2.  
*kCLOCK\_Pll3Pfd3Clk* pll3 pfd3.  
*kCLOCK\_Pll4Pfd0Clk* pll4 pfd0.  
*kCLOCK\_Pll4Pfd1Clk* pll4 pfd1.  
*kCLOCK\_Pll4Pfd2Clk* pll4 pfd2.  
*kCLOCK\_Pll4Pfd3Clk* pll4 pfd3.  
*kCLOCK\_Pll0VcoDivClk* PLL0VCODIV.  
*kCLOCK\_Pll0Pfd1DivClk* PLL0PFD1DIV.  
*kCLOCK\_Pll0Pfd2DivClk* PLL0PFD2DIV.  
*kCLOCK\_Pll1VcoDivClk* PLL1VCODIV.  
*kCLOCK\_Pll1Pfd1DivClk* PLL1PFD1DIV.  
*kCLOCK\_Pll1Pfd2DivClk* PLL1PFD2DIV.  
*kCLOCK\_Pll3VcoDivClk* PLL3VCODIV.  
*kCLOCK\_Pll3Pfd0Div1Clk* PLL3PFD0DIV1.  
*kCLOCK\_Pll3Pfd0Div2Clk* PLL3PFD0DIV2.  
*kCLOCK\_Pll3Pfd1Div1Clk* PLL3PFD1DIV1.  
*kCLOCK\_Pll3Pfd1Div2Clk* PLL3PFD1DIV2.  
*kCLOCK\_Pll3Pfd2Div1Clk* PLL3PFD2DIV1.  
*kCLOCK\_Pll3Pfd2Div2Clk* PLL3PFD2DIV2.  
*kCLOCK\_Pll3Pfd3Div1Clk* PLL3PFD3DIV1.  
*kCLOCK\_Pll3Pfd3Div2Clk* PLL3PFD3DIV2.  
*kCLOCK\_Pll4VcoDivClk* PLL4VCODIV.  
*kCLOCK\_Pll4Pfd0Div1Clk* PLL4PFD0DIV1.  
*kCLOCK\_Pll4Pfd0Div2Clk* PLL4PFD0DIV2.  
*kCLOCK\_Pll4Pfd1Div1Clk* PLL4PFD1DIV1.  
*kCLOCK\_Pll4Pfd1Div2Clk* PLL4PFD1DIV2.  
*kCLOCK\_Pll4Pfd2Div1Clk* PLL4PFD2DIV1.  
*kCLOCK\_Pll4Pfd2Div2Clk* PLL4PFD2DIV2.  
*kCLOCK\_Pll4Pfd3Div1Clk* PLL4PFD3DIV1.  
*kCLOCK\_Pll4Pfd3Div2Clk* PLL4PFD3DIV2.

## 4.5.2 enum \_clock\_ip\_src

Enumerator

***kCLOCK\_IpSrcNone*** Clock is off.

***kCLOCK\_Pcc0PlatIpSrcSysOscDiv1*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_RtdSysOscDiv1Clk***.

***kCLOCK\_Pcc0PlatIpSrcFroDiv1*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_RtdFroDiv1Clk***.

***kCLOCK\_Pcc0PlatIpSrcCm33Plat*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_Cm33CorePlatClk***.

***kCLOCK\_Pcc0PlatIpSrcFro*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_FroClk***.

***kCLOCK\_Pcc0PlatIpSrcPll0Pfd3*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_Pll0Pfd3Clk***.

***kCLOCK\_Pcc0BusIpSrcLpo*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_LpOscClk***.

***kCLOCK\_Pcc0BusIpSrcSysOscDiv2*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_RtdSysOscDiv2Clk***.

***kCLOCK\_Pcc0BusIpSrcFroDiv2*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_RtdFroDiv2Clk***.

***kCLOCK\_Pcc0BusIpSrcCm33Bus*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_Cm33BusClk***.

***kCLOCK\_Pcc0BusIpSrcPll1Pfd1Div*** PCC0 Bus clock selection: ***kCLOCK\_Pll1Pfd1DivClk***.

***kCLOCK\_Pcc0BusIpSrcPll0Pfd2Div*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_Pll0Pfd2DivClk***.

***kCLOCK\_Pcc0BusIpSrcPll0Pfd1Div*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_Pll0Pfd1DivClk***.

***kCLOCK\_Pcc1PlatIpSrcSysOscDiv1*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_RtdSysOscDiv1Clk***.

***kCLOCK\_Pcc1PlatIpSrcFroDiv1*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_RtdFroDiv1Clk***.

***kCLOCK\_Pcc1PlatIpSrcCm33Plat*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_Cm33CorePlatClk***.

***kCLOCK\_Pcc1PlatIpSrcFro*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_FroClk***.

***kCLOCK\_Pcc1PlatIpSrcPll0Pfd3*** PCC0, PCC1 Platform clock selection: ***kCLOCK\_Pll0Pfd3Clk***.

***kCLOCK\_Pcc1BusIpSrcLpo*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_LpOscClk***.

***kCLOCK\_Pcc1BusIpSrcSysOscDiv2*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_RtdSysOscDiv2Clk***.

***kCLOCK\_Pcc1BusIpSrcFroDiv2*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_RtdFroDiv2Clk***.

***kCLOCK\_Pcc1BusIpSrcCm33Bus*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_Cm33BusClk***.

***kCLOCK\_Pcc1BusIpSrcPll1VcoDiv*** PCC1 Bus clock selection: ***kCLOCK\_Pll1VcoDivClk***.

***kCLOCK\_Pcc1BusIpSrcPll0Pfd2Div*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_Pll0Pfd2DivClk***.

***kCLOCK\_Pcc1BusIpSrcPll0Pfd1Div*** PCC0, PCC1 Bus clock selection: ***kCLOCK\_Pll0Pfd1DivClk***.

***kCLOCK\_Pcc2BusIpSrcLpo*** PCC2 Bus clock selection: ***kCLOCK\_LpOscClk***.

***kCLOCK\_Pcc2BusIpSrcSysOscDiv3*** PCC2 Bus clock selection: ***kCLOCK\_RtdSysOscDiv3Clk***.

***kCLOCK\_Pcc2BusIpSrcFroDiv3*** PCC2 Bus clock selection: ***kCLOCK\_RtdFroDiv3Clk***.

***kCLOCK\_Pcc2BusIpSrcFusionDspBus*** PCC2 Bus clock selection: ***kCLOCK\_FusionDspBusClk***.

*kCLOCK\_Pcc2BusIpSrcPll1VcoDiv* PCC2 Bus clock selection: kCLOCK\_Pll1VcoDivClk.

*kCLOCK\_Pcc2BusIpSrcPll0Pfd2Div* PCC2 Bus clock selection: kCLOCK\_Pll0Pfd2DivClk.

*kCLOCK\_Pcc2BusIpSrcPll0Pfd1Div* PCC2 Bus clock selection: kCLOCK\_Pll0Pfd1DivClk.

*kCLOCK\_Pcc3BusIpSrcLpo* PCC3 Bus clock selection: kCLOCK\_LpOscClk.

*kCLOCK\_Pcc3BusIpSrcSysOscDiv2* PCC3 Bus clock selection: kCLOCK\_AdSysOscDiv2Clk.

*kCLOCK\_Pcc3BusIpSrcFroDiv2* PCC3 Bus clock selection: kCLOCK\_AdFroDiv2Clk.

*kCLOCK\_Pcc3BusIpSrcXbarBus* PCC3 Bus clock selection: kCLOCK\_XbarBusClk.

*kCLOCK\_Pcc3BusIpSrcPll3Pfd1Div1* PCC3 Bus clock selection: kCLOCK\_Pll3Pfd1Div1Clk.

*kCLOCK\_Pcc3BusIpSrcPll3Pfd0Div2* PCC3 Bus clock selection: kCLOCK\_Pll3Pfd0Div2Clk.

*kCLOCK\_Pcc3BusIpSrcPll3Pfd0Div1* PCC3 Bus clock selection: kCLOCK\_Pll3Pfd0Div1Clk.

*kCLOCK\_Pcc4PlatIpSrcSysOscDiv1* PCC4 Platform clock selection: kCLOCK\_AdSysOscDiv1-Clk.

*kCLOCK\_Pcc4PlatIpSrcFroDiv1* PCC4 Platform clock selection: kCLOCK\_AdFroDiv1Clk.

*kCLOCK\_Pcc4PlatIpSrcPll3Pfd3Div2* PCC4 Platform clock selection: kCLOCK\_Pll3Pfd3Div2-Clk.

*kCLOCK\_Pcc4PlatIpSrcPll3Pfd3Div1* PCC4 Platform clock selection: kCLOCK\_Pll3Pfd3Div1-Clk.

*kCLOCK\_Pcc4PlatIpSrcPll3Pfd2Div2* PCC4 Platform clock selection: kCLOCK\_Pll3Pfd2Div2-Clk.

*kCLOCK\_Pcc4PlatIpSrcPll3Pfd2Div1* PCC4 Platform clock selection: kCLOCK\_Pll3Pfd2Div1-Clk.

*kCLOCK\_Pcc4PlatIpSrcPll3Pfd1Div2* PCC4 Platform clock selection: kCLOCK\_Pll3Pfd1Div2-Clk.

*kCLOCK\_Pcc4BusIpSrcLpo* PCC4 Bus clock selection: kCLOCK\_LpOscClk.

*kCLOCK\_Pcc4BusIpSrcSysOscDiv2* PCC4 Bus clock selection: kCLOCK\_AdSysOscDiv2Clk.

*kCLOCK\_Pcc4BusIpSrcFroDiv2* PCC4 Bus clock selection: kCLOCK\_AdFroDiv2Clk.

*kCLOCK\_Pcc4BusIpSrcXbarBus* PCC4 Bus clock selection: kCLOCK\_XbarBusClk.

*kCLOCK\_Pcc4BusIpSrcPll3VcoDiv* PCC4 Bus clock selection: kCLOCK\_Pll3VcoDivClk.

*kCLOCK\_Pcc4BusIpSrcPll3Pfd0Div1* PCC4 Bus clock selection: kCLOCK\_Pll3Pfd0Div1Clk.

*kCLOCK\_Pcc5PlatIpSrcPll4Pfd3Div2* PCC5 Platform clock selection: kCLOCK\_Pll4Pfd3Div2-Clk.

*kCLOCK\_Pcc5PlatIpSrcPll4Pfd2Div2* PCC5 Platform clock selection: kCLOCK\_Pll4Pfd2Div2-Clk.

*kCLOCK\_Pcc5PlatIpSrcPll4Pfd2Div1* PCC5 Platform clock selection: kCLOCK\_Pll4Pfd2Div1-Clk.

*kCLOCK\_Pcc5PlatIpSrcPll4Pfd1Div2* PCC5 Platform clock selection: kCLOCK\_Pll4Pfd1Div2-Clk.

*kCLOCK\_Pcc5PlatIpSrcPll4Pfd1Div1* PCC5 Platform clock selection: kCLOCK\_Pll4Pfd1Div1-Clk.

*kCLOCK\_Pcc5PlatIpSrcPll4Pfd0Div2* PCC5 Platform clock selection: kCLOCK\_Pll4Pfd0Div2-Clk.

*kCLOCK\_Pcc5PlatIpSrcPll4Pfd0Div1* PCC5 Platform clock selection: kCLOCK\_Pll4Pfd0Div1-Clk.

*kCLOCK\_Pcc5BusIpSrcLpo* PCC5 Bus clock selection: kCLOCK\_LpOscClk.

*kCLOCK\_Pcc5BusIpSrcSysOscDiv2* PCC5 Bus clock selection: kCLOCK\_LpavSysOscDiv2Clk.

***kCLOCK\_Pcc5BusIpSrcFroDiv2*** PCC5 Bus clock selection: *kCLOCK\_LpavFroDiv2Clk*.

***kCLOCK\_Pcc5BusIpSrcLpavBus*** PCC5 Bus clock selection: *kCLOCK\_NicLpavBusClk*.

***kCLOCK\_Pcc5BusIpSrcPll4VcoDiv*** PCC5 Bus clock selection: *kCLOCK\_Pll4VcoDivClk*.

***kCLOCK\_Pcc5BusIpSrcPll4Pfd3Div1*** PCC5 Bus clock selection: *kCLOCK\_Pll4Pfd3Div1Clk*.

***kCLOCK\_Cm33SaiClkSrcPll1Pfd2Div*** PLL1 PFD2 DIV in RTD: *kCLOCK\_Pll1Pfd2DivClk*.

***kCLOCK\_Cm33SaiClkSrcRtdAudClk*** Common audio clock in RTD, see *cgc\_rtd\_audclk\_src\_t*.

***kCLOCK\_Cm33SaiClkSrcLpavAudClk*** Common audio clock in LPAV, see *cgc\_lpav\_audclk\_src\_t*.

***kCLOCK\_Cm33SaiClkSrcSysOsc*** SYSOSC main reference clock to the chip: *kCLOCK\_SysOscClk*.

***kCLOCK\_FusionSaiClkSrcPll1Pfd2Div*** PLL1 PFD2 DIV in RTD: *kCLOCK\_Pll1Pfd2DivClk*.

***kCLOCK\_FusionSaiClkSrcExtMclk1*** External audio master clock input 1.

***kCLOCK\_FusionSaiClkSrcSai3Rx*** SAI3 receiver serial bit clock. Only for SAI2.

***kCLOCK\_FusionSaiClkSrcSai2Tx*** SAI2 transmitter serial bit clock. Only for SAI3.

***kCLOCK\_FusionSaiClkSrcSysOsc*** SYSOSC main reference clock to the chip: *kCLOCK\_SysOscClk*.

***kCLOCK\_FusionMicfilClkSrcPll1Pfd2Div*** PLL1 PFD2 DIV in RTD: *kCLOCK\_Pll1Pfd2DivClk*.

***kCLOCK\_FusionMicfilClkSrcFro24*** FRO24: *kCLOCK\_FroClk/8*.

***kCLOCK\_FusionMicfilClkSrcSysOsc*** SYSOSC main reference clock to the chip: *kCLOCK\_SysOscClk*.

***kCLOCK\_FusionMicfilClkSrcExtMclk1*** External audio master clock input 1.

***kCLOCK\_FusionMicfilClkSrcRtcOsc*** *kCLOCK\_RtcOscClk*.

***kCLOCK\_FusionMicfilClkSrcLpo*** *kCLOCK\_LpOscClk*.

***kCLOCK\_FusionTpm2ClkSrcPll1Pfd2Div*** PLL1 PFD2 DIV in RTD: *kCLOCK\_Pll1Pfd2DivClk*.

***kCLOCK\_FusionTpm2ClkSrcExtMclk1*** External audio master clock input 1.

***kCLOCK\_FusionTpm2ClkSrcLpo*** PCC2 Bus clock selection: *kCLOCK\_LpOscClk*.

***kCLOCK\_FusionTpm2ClkSrcSysOscDiv3*** PCC2 Bus clock selection: *kCLOCK\_RtdSysOscDiv3Clk*.

***kCLOCK\_FusionTpm2ClkSrcFroDiv3*** PCC2 Bus clock selection: *kCLOCK\_RtdFroDiv3Clk*.

***kCLOCK\_FusionTpm2ClkSrcFusionDspBus*** PCC2 Bus clock selection: *kCLOCK\_FusionDspBusClk*.

***kCLOCK\_FusionTpm2ClkSrcPll1VcoDiv*** PCC2 Bus clock selection: *kCLOCK\_Pll1VcoDivClk*.

***kCLOCK\_FusionTpm2ClkSrcPll0Pfd2Div*** PCC2 Bus clock selection: *kCLOCK\_Pll0Pfd2DivClk*.

***kCLOCK\_FusionTpm2ClkSrcPll0Pfd1Div*** PCC2 Bus clock selection: *kCLOCK\_Pll0Pfd1DivClk*.

***kCLOCK\_FusionTpm3ClkSrcPll1Pfd2Div*** PLL1 PFD2 DIV in RTD: *kCLOCK\_Pll1Pfd2DivClk*.

***kCLOCK\_FusionTpm3ClkSrcRtdAudClk*** Common audio clock in RTD, see *cgc\_rtd\_audclk\_src\_t*.

***kCLOCK\_FusionTpm3ClkSrcLpavAudClk*** Common audio clock in LPAV, see *cgc\_lpav\_audclk\_src\_t*.

***kCLOCK\_FusionTpm3ClkSrcLpo*** PCC2 Bus clock selection: *kCLOCK\_LpOscClk*.

***kCLOCK\_FusionTpm3ClkSrcSysOscDiv3*** PCC2 Bus clock selection: *kCLOCK\_RtdSysOscDiv3Clk*.

***kCLOCK\_FusionTpm3ClkSrcFroDiv3*** PCC2 Bus clock selection: *kCLOCK\_RtdFroDiv3Clk*.

***kCLOCK\_FusionTpm3ClkSrcFusionDspBus*** PCC2 Bus clock selection: *kCLOCK\_FusionDsp-BusClk*.

***kCLOCK\_FusionTpm3ClkSrcPll1VcoDiv*** PCC2 Bus clock selection: *kCLOCK\_Pll1VcoDivClk*.

***kCLOCK\_FusionTpm3ClkSrcPll0Pfd2Div*** PCC2 Bus clock selection: *kCLOCK\_Pll0Pfd2DivClk*.

***kCLOCK\_FusionTpm3ClkSrcPll0Pfd1Div*** PCC2 Bus clock selection: *kCLOCK\_Pll0Pfd1DivClk*.

***kCLOCK\_AdSaiClkSrcPll3Pfd1Div1*** PLL3 PFD1 DIV1 in AD: *kCLOCK\_Pll3Pfd1Div2Clk*.

***kCLOCK\_AdSaiClkSrcAdAudClk*** Common audio clock in AD, see *cgc\_ad\_audclk\_src\_t*.

***kCLOCK\_AdSaiClkSrcLpavAudClk*** Common audio clock in LPAV, see *cgc\_lpav\_audclk\_src\_t*.

***kCLOCK\_AdSaiClkSrcSysOsc*** SYSOSC main reference clock to the chip: *kCLOCK\_SysOscClk*.

***kCLOCK\_AdTpm67ClkSrcPll3Pfd1Div1*** PLL3 PFD1 DIV1 in AD: *kCLOCK\_Pll3Pfd1Div2Clk*.

***kCLOCK\_AdTpm67ClkSrcAdAudClk*** Common audio clock in AD, see *cgc\_ad\_audclk\_src\_t*.

***kCLOCK\_AdTpm67ClkSrcLpavAudClk*** Common audio clock in LPAV, see *cgc\_lpav\_audclk\_src\_t*.

***kCLOCK\_AdTpm67ClkSrcLpo*** PCC4 Bus clock selection: *kCLOCK\_LpOscClk*.

***kCLOCK\_AdTpm67ClkSrcSysOscDiv2*** PCC4 Bus clock selection: *kCLOCK\_AdSysOscDiv2Clk*.

***kCLOCK\_AdTpm67ClkSrcFroDiv2*** PCC4 Bus clock selection: *kCLOCK\_AdFroDiv2Clk*.

***kCLOCK\_AdTpm67ClkSrcXbarBus*** PCC4 Bus clock selection: *kCLOCK\_XbarBusClk*.

***kCLOCK\_AdTpm67ClkSrcPll3VcoDiv*** PCC4 Bus clock selection: *kCLOCK\_Pll3VcoDivClk*.

***kCLOCK\_AdTpm67ClkSrcPll3Pfd0Div1*** PCC4 Bus clock selection: *kCLOCK\_Pll3Pfd0Div1Clk*.

***kCLOCK\_LpavSaiClkSrcPll1Pfd2Div*** PLL1 PFD2 DIV in RTD: *kCLOCK\_Pll1Pfd2DivClk*.

***kCLOCK\_LpavSaiClkSrcPll3Pfd1Div1*** PLL3 PFD1 DIV1 in AD: *kCLOCK\_Pll3Pfd1Div1Clk*.

***kCLOCK\_LpavSaiClkSrcRtdAudClk*** Common audio clock in RTD, see *cgc\_rtd\_audclk\_src\_t*.

***kCLOCK\_LpavSaiClkSrcAdAudClk*** Common audio clock in AD, see *cgc\_ad\_audclk\_src\_t*.

***kCLOCK\_LpavSaiClkSrcLpavAudClk*** Common audio clock in LPAV, see *cgc\_lpav\_audclk\_src\_t*.

***kCLOCK\_LpavSaiClkSrcSysOsc*** SYSOSC main reference clock to the chip.

***kCLOCK\_LpavTpm8ClkSrcPll1Pfd2Div*** PLL1 PFD2 DIV in RTD: *kCLOCK\_Pll1Pfd2DivClk*.

***kCLOCK\_LpavTpm8ClkSrcPll3Pfd1Div1*** PLL3 PFD1 DIV1 in AD: *kCLOCK\_Pll3Pfd1Div1Clk*.

***kCLOCK\_LpavTpm8ClkSrcRtdAudClk*** Common audio clock in RTD, see *cgc\_rtd\_audclk\_src\_t*.

***kCLOCK\_LpavTpm8ClkSrcAdAudClk*** Common audio clock in AD, see *cgc\_ad\_audclk\_src\_t*.

***kCLOCK\_LpavTpm8ClkSrcLpavAudClk*** Common audio clock in LPAV, see *cgc\_lpav\_audclk\_src\_t*.

***kCLOCK\_LpavTpm8ClkSrcLpo*** PCC5 Bus clock selection: *kCLOCK\_LpOscClk*.

***kCLOCK\_LpavTpm8ClkSrcSysOscDiv2*** PCC5 Bus clock selection: *kCLOCK\_LpavSysOscDiv2Clk*.

***kCLOCK\_LpavTpm8ClkSrcFroDiv2*** PCC5 Bus clock selection: *kCLOCK\_LpavFroDiv2Clk*.

***kCLOCK\_LpavTpm8ClkSrcLpavBus*** PCC5 Bus clock selection: *kCLOCK\_NicLpavBusClk*.

***kCLOCK\_LpavTpm8ClkSrcPll4VcoDiv*** PCC5 Bus clock selection: *kCLOCK\_Pll4VcoDivClk*.

***kCLOCK\_LpavTpm8ClkSrcPll4Pfd3Div1*** PCC5 Bus clock selection: *kCLOCK\_Pll4Pfd3Div1Clk*.

### 4.5.3 enum \_clock\_lptmr\_src

Enumerator

*kCLOCK\_LptmrSrcLPO1M* LPO 1MHz clock.  
*kCLOCK\_LptmrSrcRtc1K* RTC 1KHz clock.  
*kCLOCK\_LptmrSrcRtc32K* RTC 32KHz clock.  
*kCLOCK\_LptmrSrcSysOsc* system OSC clock.

### 4.5.4 enum \_clock\_ip\_name

[31:2] is defined as the corresponding register address. [ 1:1] is used as indicator of existing of PCS. [ 0:0] is used as indicator of existing of PCD/Frac.

### 4.5.5 anonymous enum

Enumerator

*kStatus\_CGC\_Busy* Clock is busy.  
*kStatus\_CGC\_InvalidSrc* Invalid source.

### 4.5.6 enum \_cgc\_sys\_clk

Enumerator

*kCGC\_SysClkSlow* System slow clock.  
*kCGC\_SysClkBus* Bus clock.  
*kCGC\_SysClkCorePlat* Core/Platform clock.  
*kCGC\_SysClkHifi4* Hifi4 core clock.  
*kCGC\_SysClkNicHifi* Hifi4 platform clock.  
*kCGC\_SysClkLpavAxi* LPAV AXI clock.  
*kCGC\_SysClkLpavAhb* LPAV AHB clock.  
*kCGC\_SysClkLpavBus* LPAV Bus clock.

### 4.5.7 enum \_cgc\_rtd\_sys\_clk\_src

Enumerator

*kCGC\_RtdSysClkSrcFro* FRO192.  
*kCGC\_RtdSysClkSrcPll0Pfd0* PLL0 PFD0.  
*kCGC\_RtdSysClkSrcPll1Pfd0* PLL1 PFD0.

*kCGC\_RtdSysClkSrcSysOsc* System OSC.  
*kCGC\_RtdSysClkSrcRtcOsc* RTC OSC.  
*kCGC\_RtdSysClkSrcLvds* LVDS XCVR.  
*kCGC\_RtdSysClkSrcPll0* PLL0.

#### 4.5.8 enum \_cgc\_nic\_sys\_clk\_src

Enumerator

*kCGC\_NicSysClkSrcFro* FRO192.  
*kCGC\_NicSysClkSrcPll3Pfd0* PLL0 PFD0.  
*kCGC\_NicSysClkSrcSysOsc* System OSC.  
*kCGC\_NicSysClkSrcLvds* LVDS XCVR.

#### 4.5.9 enum \_cgc\_hifi\_sys\_clk\_src

Enumerator

*kCGC\_HifiSysClkSrcFro* FRO192.  
*kCGC\_HifiSysClkSrcPll4* PLL4.  
*kCGC\_HifiSysClkSrcPll4Pfd0* PLL4 PFD0.  
*kCGC\_HifiSysClkSrcSysOsc* System OSC.  
*kCGC\_HifiSysClkSrcLvds* LVDS XCVR.

#### 4.5.10 enum \_cgc\_lpav\_sys\_clk\_src

Enumerator

*kCGC\_LpavSysClkSrcFro* FRO192.  
*kCGC\_LpavSysClkSrcPll4Pfd1* PLL4 PFD1.  
*kCGC\_LpavSysClkSrcSysOsc* System OSC.  
*kCGC\_LpavSysClkSrcLvds* LVDS XCVR.

#### 4.5.11 enum \_cgc\_ddr\_sys\_clk\_src

Enumerator

*kCGC\_DdrSysClkSrcFro* FRO192.  
*kCGC\_DdrSysClkSrcPll4Pfd1* PLL4 PFD1.  
*kCGC\_DdrSysClkSrcSysOsc* System OSC.  
*kCGC\_DdrSysClkSrcLvds* LVDS XCVR.

#### 4.5.12 enum \_clock\_rtd\_clkout\_src

Enumerator

*kClockRtdClkoutSelCm33Core* CGC CM33 Core/Platform clock.  
*kClockRtdClkoutSelCm33Bus* CGC CM33 Bus clock.  
*kClockRtdClkoutSelCm33Slow* CGC CM33 Slow clock.  
*kClockRtdClkoutSelFusionDspCore* CGC Fusion DSP Core/Platform clock.  
*kClockRtdClkoutSelFusionDspBus* CGC Fusion DSP Bus clock.  
*kClockRtdClkoutSelFusionDspSlow* CGC Fusion DSP Slow clock.  
*kClockRtdClkoutSelFro48* FRO48: kCLOCK\_FroClk/4.  
*kClockRtdClkoutSelPll0VcoDiv* PLL0 VCO DIV: kCLOCK\_Pll0VcoDivClk.  
*kClockRtdClkoutSelPll1VcoDiv* PLL1 VCO DIV: kCLOCK\_Pll1VcoDivClk.  
*kClockRtdClkoutSelSysOsc* System OSC.  
*kClockRtdClkoutSelLpOsc* CGC LPOSC clock.

#### 4.5.13 enum \_clock\_lpav\_clkout\_src

Enumerator

*kClockLpavClkoutSelHifi4* CGC HIFI4 core clock.  
*kClockLpavClkoutSelNicHifi* CGC HIFI4 platform clock.  
*kClockLpavClkoutSelLpavAxi* CGC NIC LPAV AXI clock.  
*kClockLpavClkoutSelLpavAhb* CGC NIC LPAV AHB clock.  
*kClockLpavClkoutSelLpavBus* CGC LPAV Bus clock.  
*kClockLpavClkoutSelDdr* CGC DDR clock.  
*kClockLpavClkoutSelFro48* FRO48: kCLOCK\_FroClk/4.  
*kClockLpavClkoutSelPll4VcoDiv* PLL4 VCO DIV: kCLOCK\_Pll4VcoDivClk.  
*kClockLpavClkoutSelSysOsc* System OSC.  
*kClockLpavClkoutSelLpOsc* CGC LPOSC clock.

#### 4.5.14 enum \_cgc\_async\_clk

Enumerator

*kCGC\_AsyncDiv1Clk* The async clock by DIV1, e.g. SOSCDIV1\_CLK, FRODIV1\_CLK.  
*kCGC\_AsyncDiv2Clk* The async clock by DIV2, e.g. SOSCDIV2\_CLK, FRODIV2\_CLK.  
*kCGC\_AsyncDiv3Clk* The async clock by DIV3, e.g. SOSCDIV3\_CLK, FRODIV3\_CLK.  
*kCGC\_AsyncVcoClk* The async clock by PLL VCO DIV.  
*kCGC\_AsyncPfd0Div1Clk* The async clock by PLL PFD0 DIV1.  
*kCGC\_AsyncPfd0Div2Clk* The async clock by PLL PFD0 DIV2.  
*kCGC\_AsyncPfd1Div1Clk* The async clock by PLL PFD1 DIV or DIV1.  
*kCGC\_AsyncPfd1Div2Clk* The async clock by PLL PFD1 DIV2.



***kCGC\_AsyncPfd2Div1Clk*** The async clock by PLL PFD2 DIV or DIV1.  
***kCGC\_AsyncPfd2Div2Clk*** The async clock by PLL PFD2 DIV2.  
***kCGC\_AsyncPfd3Div1Clk*** The async clock by PLL PFD3 DIV1.  
***kCGC\_AsyncPfd3Div2Clk*** The async clock by PLL PFD3 DIV2.

#### 4.5.15 enum \_cgc\_sosc\_monitor\_mode

Enumerator

***kCGC\_SysOscMonitorDisable*** Monitor disabled.  
***kCGC\_SysOscMonitorInt*** Interrupt when the system OSC error is detected.  
***kCGC\_SysOscMonitorReset*** Reset when the system OSC error is detected.

#### 4.5.16 enum \_cgc\_sosc\_mode

Enumerator

***kCGC\_SysOscModeExt*** Use external clock.  
***kCGC\_SysOscModeOscLowPower*** Oscillator low power.  
***kCGC\_SysOscModeOscHighGain*** Oscillator high gain.

#### 4.5.17 enum \_cgc\_sosc\_enable\_mode

Enumerator

***kCGC\_SysOscEnableInDeepSleep*** Enable OSC in deep sleep mode.  
***kCGC\_SysOscEnableInPowerDown*** Enable OSC in low power mode.

#### 4.5.18 enum \_cgc\_fro\_enable\_mode

Enumerator

***kCGC\_FroEnableInDeepSleep*** Enable FRO in deep sleep mode.

#### 4.5.19 enum \_cgc\_lposc\_enable\_mode

Enumerator

***kCGC\_LposcEnableInDeepSleep*** Enable OSC in deep sleep mode.  
***kCGC\_LposcEnableInPowerDown*** Enable OSC in low power mode.

#### 4.5.20 enum \_cgc\_pll\_src

Enumerator

*kCGC\_PllSrcSysOsc* PLL clock source is system OSC.

*kCGC\_PllSrcFro24M* PLL clock source is FRO 24M.

#### 4.5.21 enum \_cgc\_pll\_enable\_mode

Enumerator

*kCGC\_PllEnable* Enable PLL clock.

*kCGC\_PllEnableInDeepSleep* Enable PLL in deep sleep mode.

#### 4.5.22 enum \_cgc\_pll\_pfd\_clkout

Enumerator

*kCGC\_PllPfd0Clk* PFD0 output clock selected.

*kCGC\_PllPfd1Clk* PFD1 output clock selected.

*kCGC\_PllPfd2Clk* PFD2 output clock selected.

*kCGC\_PllPfd3Clk* PFD3 output clock selected.

#### 4.5.23 enum \_cgc\_pll0\_mult

Enumerator

*kCGC\_Pll0Mult15* Divide by 15.

*kCGC\_Pll0Mult16* Divide by 16.

*kCGC\_Pll0Mult20* Divide by 20.

*kCGC\_Pll0Mult22* Divide by 22.

*kCGC\_Pll0Mult25* Divide by 25.

*kCGC\_Pll0Mult30* Divide by 30.

#### 4.5.24 enum \_cgc\_rosc\_monitor\_mode

Enumerator

*kCGC\_RtcOscMonitorDisable* Monitor disabled.

*kCGC\_RtcOscMonitorInt* Interrupt when the RTC OSC error is detected.

*kCGC\_RtcOscMonitorReset* Reset when the RTC OSC error is detected.

#### 4.5.25 enum\_cgc\_pll1\_mult

Enumerator

*kCGC\_Pll1Mult16* Divide by 16.  
*kCGC\_Pll1Mult17* Divide by 17.  
*kCGC\_Pll1Mult20* Divide by 20.  
*kCGC\_Pll1Mult22* Divide by 22.  
*kCGC\_Pll1Mult27* Divide by 27.  
*kCGC\_Pll1Mult33* Divide by 33.

#### 4.5.26 enum\_cgc\_pll4\_mult

Enumerator

*kCGC\_Pll4Mult16* Divide by 16.  
*kCGC\_Pll4Mult17* Divide by 17.  
*kCGC\_Pll4Mult20* Divide by 20.  
*kCGC\_Pll4Mult22* Divide by 22.  
*kCGC\_Pll4Mult27* Divide by 27.  
*kCGC\_Pll4Mult33* Divide by 33.

#### 4.5.27 enum\_cgc\_rtd\_audclk\_src

Enumerator

*kCGC\_RtdAudClkSrcExtMclk0* External audio master clock input 0 from pin.  
*kCGC\_RtdAudClkSrcExtMclk1* External audio master clock input 1 from pin.  
*kCGC\_RtdAudClkSrcSai0RxBclk* SAI0 receiver serial bit clock.  
*kCGC\_RtdAudClkSrcSai0TxBclk* SAI0 transmitter serial bit clock.  
*kCGC\_RtdAudClkSrcSai1RxBclk* SAI1 receiver serial bit clock.  
*kCGC\_RtdAudClkSrcSai1TxBclk* SAI1 transmitter serial bit clock.  
*kCGC\_RtdAudClkSrcSai2RxBclk* SAI2 receiver serial bit clock.  
*kCGC\_RtdAudClkSrcSai2TxBclk* SAI2 transmitter serial bit clock.  
*kCGC\_RtdAudClkSrcSai3RxBclk* SAI3 receiver serial bit clock.  
*kCGC\_RtdAudClkSrcSai3TxBclk* SAI3 transmitter serial bit clock.

#### 4.5.28 enum\_cgc\_ad\_audclk\_src

Enumerator

*kCGC\_AdAudClkSrcExtMclk2* External audio master clock input 2 from pin.

***kCGC\_AdAudClkSrcSai4RxBclk*** SAI4 receiver serial bit clock.  
***kCGC\_AdAudClkSrcSai4TxBclk*** SAI4 transmitter serial bit clock.  
***kCGC\_AdAudClkSrcSai5RxBclk*** SAI5 receiver serial bit clock.  
***kCGC\_AdAudClkSrcSai5TxBclk*** SAI5 transmitter serial bit clock.

#### 4.5.29 enum \_cgc\_lpav\_audclk\_src

Enumerator

***kCGC\_LpavAudClkSrcExtMclk3*** External audio master clock input 3 from pin.  
***kCGC\_LpavAudClkSrcSai6RxBclk*** SAI6 receiver serial bit clock.  
***kCGC\_LpavAudClkSrcSai6TxBclk*** SAI6 transmitter serial bit clock.  
***kCGC\_LpavAudClkSrcSai7RxBclk*** SAI7 receiver serial bit clock.  
***kCGC\_LpavAudClkSrcSai7TxBclk*** SAI7 transmitter serial bit clock.  
***kCGC\_LpavAudClkSrcSpdifRx*** SPDIF receiver clock.

## 4.6 Function Documentation

### 4.6.1 static void CLOCK\_EnableClock ( clock\_ip\_name\_t *name* ) [inline], [static]

Parameters

|             |                                                              |
|-------------|--------------------------------------------------------------|
| <i>name</i> | Which clock to enable, see <a href="#">clock_ip_name_t</a> . |
|-------------|--------------------------------------------------------------|

### 4.6.2 static void CLOCK\_DisableClock ( clock\_ip\_name\_t *name* ) [inline], [static]

Parameters

|             |                                                               |
|-------------|---------------------------------------------------------------|
| <i>name</i> | Which clock to disable, see <a href="#">clock_ip_name_t</a> . |
|-------------|---------------------------------------------------------------|

### 4.6.3 static bool CLOCK\_IsEnabledByOtherCore ( clock\_ip\_name\_t *name* ) [inline], [static]

## Parameters

|             |                                                                  |
|-------------|------------------------------------------------------------------|
| <i>name</i> | Which peripheral to check, see <a href="#">clock_ip_name_t</a> . |
|-------------|------------------------------------------------------------------|

## Returns

True if clock is already enabled, otherwise false.

#### 4.6.4 void CLOCK\_SetIpSrc ( clock\_ip\_name\_t *name*, clock\_ip\_src\_t *src* )

Set the clock source for specific IP, not all modules need to set the clock source, should only use this function for the modules need source setting.

## Parameters

|             |                                                                  |
|-------------|------------------------------------------------------------------|
| <i>name</i> | Which peripheral to check, see <a href="#">clock_ip_name_t</a> . |
| <i>src</i>  | Clock source to set.                                             |

#### 4.6.5 void CLOCK\_SetIpSrcDiv ( clock\_ip\_name\_t *name*, clock\_ip\_src\_t *src*, uint8\_t *divValue*, uint8\_t *fracValue* )

Set the clock source and divider for specific IP, not all modules need to set the clock source and divider, should only use this function for the modules need source and divider setting.

Divider output clock = Divider input clock x [(fracValue+1)/(divValue+1)].

## Parameters

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| <i>name</i>      | Which peripheral to check, see <a href="#">clock_ip_name_t</a> . |
| <i>src</i>       | Clock source to set.                                             |
| <i>divValue</i>  | The divider value.                                               |
| <i>fracValue</i> | The fraction multiply value.                                     |

#### 4.6.6 static void CLOCK\_SetRtdAudClkSrc ( cgc\_rtd\_audclk\_src\_t *src* ) [inline], [static]

NOTE: The audio clock pin frequency is decided by SAI, but clock driver cannot depend on SAI driver to get its pin frequencies, user need to explicitly set the MCLK/BCLK frequencies if other modules use the AUD\_CLK0 source and need to get correct frequency.

## Parameters

|            |                      |
|------------|----------------------|
| <i>src</i> | Clock source to set. |
|------------|----------------------|

#### 4.6.7 static void CLOCK\_SetAdAudClkSrc ( cgc\_ad\_audclk\_src\_t *src* ) [inline], [static]

NOTE: The audio clock pin frequency is decided by SAI, but clock driver cannot depend on SAI driver to get its pin frequencies, user need to explicitly set the MCLK/BCLK frequencies if other modules use the AUD\_CLK1 source and need to get correct frequency.

## Parameters

|            |                      |
|------------|----------------------|
| <i>src</i> | Clock source to set. |
|------------|----------------------|

#### 4.6.8 static void CLOCK\_SetLpavAudClkSrc ( cgc\_lpav\_audclk\_src\_t *src* ) [inline], [static]

NOTE: The audio clock pin frequency is decided by SAI, but clock driver cannot depend on SAI driver to get its pin frequencies, user need to explicitly set the MCLK/BCLK frequencies if other modules use the AUD\_CLK2 source and need to get correct frequency.

## Parameters

|            |                      |
|------------|----------------------|
| <i>src</i> | Clock source to set. |
|------------|----------------------|

#### 4.6.9 uint32\_t CLOCK\_GetFreq ( clock\_name\_t *clockName* )

This function checks the current clock configurations and then calculates the clock frequency for a specific clock name defined in clock\_name\_t.

## Parameters

|                  |                                     |
|------------------|-------------------------------------|
| <i>clockName</i> | Clock names defined in clock_name_t |
|------------------|-------------------------------------|

## Returns

Clock frequency value in hertz

**4.6.10 uint32\_t CLOCK\_GetCm33CorePlatClkFreq ( void )**

Returns

Clock frequency in Hz.

**4.6.11 uint32\_t CLOCK\_GetCm33BusClkFreq ( void )**

Returns

Clock frequency in Hz.

**4.6.12 uint32\_t CLOCK\_GetCm33SlowClkFreq ( void )**

Returns

Clock frequency in Hz.

**4.6.13 uint32\_t CLOCK\_GetFusionDspCorePlatClkFreq ( void )**

Returns

Clock frequency in Hz.

**4.6.14 uint32\_t CLOCK\_GetFusionDspBusClkFreq ( void )**

Returns

Clock frequency in Hz.

**4.6.15 uint32\_t CLOCK\_GetFusionDspSlowClkFreq ( void )**

Returns

Clock frequency in Hz.

**4.6.16 uint32\_t CLOCK\_GetLvdsClkFreq ( void )**

Returns

Clock frequency in Hz.

**4.6.17 uint32\_t CLOCK\_GetIpFreq ( clock\_ip\_name\_t *name* )**

This function gets the IP module clock frequency. It is only used for the IP modules which could select clock source by [CLOCK\\_SetIpSrc\(\)](#).

Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>name</i> | Which peripheral to get, see <a href="#">clock_ip_name_t</a> . |
|-------------|----------------------------------------------------------------|

Returns

Clock frequency value in hertz

**4.6.18 uint32\_t CLOCK\_GetCm33SysClkFreq ( cgc\_sys\_clk\_t *type* )**

This function gets the CGC CM33 system clock frequency. These clocks are used for core, platform, bus and slow clock domains.

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>type</i> | Which type of clock to get. |
|-------------|-----------------------------|

Returns

Clock frequency.

**4.6.19 static void CLOCK\_SetCm33SysClkConfig ( cgc\_rtd\_sys\_clk\_config\_t \* *config* ) [inline], [static]**

This function sets the system clock configuration for CM33 domain.



## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.20 **uint32\_t CLOCK\_GetFusionDspSysClkFreq ( cgc\_sys\_clk\_t type )**

This function gets the CGC Fusion DSP system clock frequency. These clocks are used for core, platform, bus and slow clock domains.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>type</i> | Which type of clock to get. |
|-------------|-----------------------------|

## Returns

Clock frequency.

#### 4.6.21 **static void CLOCK\_SetFusionSysClkConfig ( const cgc\_rtd\_sys\_clk\_config\_t \* config ) [inline], [static]**

This function sets the system clock configuration for FusionF1 DSP domain.

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.22 **static void CLOCK\_GetCm33SysClkConfig ( cgc\_rtd\_sys\_clk\_config\_t \* config ) [inline], [static]**

This function gets the system configuration for CM33 domain.

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.23 **static void CLOCK\_GetFusionDspSysClkConfig ( cgc\_rtd\_sys\_clk\_config\_t \* config ) [inline], [static]**

This function gets the system configuration for FusionF1 DSP domain.

Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.24 static void CLOCK\_SetRtdClkOutConfig ( clock\_rtd\_clkout\_src\_t *setting*, uint8\_t *div*, bool *enable* ) [inline], [static]

This function sets the clock out configuration.

Parameters

|                |                               |
|----------------|-------------------------------|
| <i>setting</i> | The selection to set.         |
| <i>div</i>     | The divider to set (div > 0). |
| <i>enable</i>  | Enable clock out.             |

#### 4.6.25 static void CLOCK\_SetRtcClkOutConfig ( uint8\_t *div* ) [inline], [static]

This function sets the RTC\_CLOCKOUT configuration.

Parameters

|            |                               |
|------------|-------------------------------|
| <i>div</i> | The divider to set (div > 0). |
|------------|-------------------------------|

#### 4.6.26 uint32\_t CLOCK\_GetXbarBusClkFreq ( void )

This function gets the CGC XBAR bus clock frequency.

Returns

Clock frequency.

#### 4.6.27 uint32\_t CLOCK\_GetHifiDspSysClkFreq ( cgc\_sys\_clk\_t *type* )

This function gets the CGC HIFI DSP system clock frequency. These clocks are used for core, platform domains.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>type</i> | Which type of clock to get. |
|-------------|-----------------------------|

## Returns

Clock frequency.

#### 4.6.28 static void CLOCK\_SetHifiDspSysClkConfig ( const cgc\_hifi\_sys\_clk\_config\_t \* *config* ) [inline], [static]

This function sets the system clock configuration for HIFI4 DSP domain.

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.29 static void CLOCK\_GetHifiDspSysClkConfig ( cgc\_hifi\_sys\_clk\_config\_t \* *config* ) [inline], [static]

This function gets the system configuration for HIFI4 DSP domain.

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.30 uint32\_t CLOCK\_GetLpavSysClkFreq ( cgc\_sys\_clk\_t *type* )

This function gets the CGC NIC LPAV system clock frequency. These clocks are used for AXI, AHB, Bus domains.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>type</i> | Which type of clock to get. |
|-------------|-----------------------------|

## Returns

Clock frequency.

**4.6.31** `static void CLOCK_SetLpavSysClkConfig ( const cgc_lpav_sys_clk_config_t  
* config ) [inline], [static]`

This function sets the system clock configuration for NIC LPAV domain.

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.32 static void CLOCK\_GetLpavSysClkConfig ( cgc\_lpav\_sys\_clk\_config\_t \* *config* ) [inline], [static]

This function gets the system configuration for NIC LPAV domain.

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | Pointer to the configuration. |
|---------------|-------------------------------|

#### 4.6.33 uint32\_t CLOCK\_GetDdrClkFreq ( void )

This function gets the CGC DDR clock frequency.

## Returns

Clock frequency.

#### 4.6.34 static void CLOCK\_SetLpavClkOutConfig ( clock\_lpav\_clkout\_src\_t *setting*, uint8\_t *div*, bool *enable* ) [inline], [static]

This function sets the clock out configuration.

## Parameters

|                |                                       |
|----------------|---------------------------------------|
| <i>setting</i> | The selection to set.                 |
| <i>div</i>     | The divider to set ( <i>div</i> > 0). |
| <i>enable</i>  | Enable clock out.                     |

#### 4.6.35 status\_t CLOCK\_InitSysOsc ( const cgc\_sosc\_config\_t \* *config* )

This function enables the CGC system OSC clock according to the configuration.

## Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>config</i> | Pointer to the configuration structure. |
|---------------|-----------------------------------------|

## Return values

|                         |                                                              |
|-------------------------|--------------------------------------------------------------|
| <i>kStatus_Success</i>  | System OSC is initialized.                                   |
| <i>kStatus_CGC_Busy</i> | System OSC has been enabled and is used by the system clock. |
| <i>kStatus_ReadOnly</i> | System OSC control register is locked.                       |

## Note

This function can't detect whether the system OSC has been enabled and used by an IP.

#### 4.6.36 **status\_t** CLOCK\_DeinitSysOsc ( void )

This function disables the CGC system OSC clock.

## Return values

|                         |                                         |
|-------------------------|-----------------------------------------|
| <i>kStatus_Success</i>  | System OSC is deinitialized.            |
| <i>kStatus_CGC_Busy</i> | System OSC is used by the system clock. |
| <i>kStatus_ReadOnly</i> | System OSC control register is locked.  |

## Note

This function can't detect whether the system OSC is used by an IP.

#### 4.6.37 **void** CLOCK\_SetRtdSysOscAsyncClkDiv ( cgc\_async\_clk\_t *asyncClk*, uint8\_t *divider* )

## Parameters

|                 |                                        |
|-----------------|----------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure. |
|-----------------|----------------------------------------|

|                |                                                       |
|----------------|-------------------------------------------------------|
| <i>divider</i> | The divider value to set. Disabled when divider == 0. |
|----------------|-------------------------------------------------------|

## Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.38 void CLOCK\_SetAdSysOscAsyncClkDiv ( cgc\_async\_clk\_t *asyncClk*, uint8\_t *divider* )

## Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure.                |
| <i>divider</i>  | The divider value to set. Disabled when divider == 0. |

## Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.39 void CLOCK\_SetLpavSysOscAsyncClkDiv ( cgc\_async\_clk\_t *asyncClk*, uint8\_t *divider* )

## Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure.                |
| <i>divider</i>  | The divider value to set. Disabled when divider == 0. |

## Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.40 uint32\_t CLOCK\_GetSysOscFreq ( void )

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.41 uint32\_t CLOCK\_GetRtdSysOscAsyncFreq ( cgc\_async\_clk\_t *type* )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.42 **uint32\_t** **CLOCK\_GetAdSysOscAsyncFreq** ( *cgc\_async\_clk\_t type* )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.43 **uint32\_t** **CLOCK\_GetLpavSysOscAsyncFreq** ( *cgc\_async\_clk\_t type* )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.44 **static bool** **CLOCK\_IsSysOscErr** ( **void** ) [**inline**], [**static**]

## Returns

True if the error occurs, false if not.

#### 4.6.45 **static void** **CLOCK\_SetSysOscMonitorMode** ( *cgc\_sosc\_monitor\_mode\_t mode* ) [**inline**], [**static**]

This function sets the system OSC monitor mode. The mode can be disabled, it can generate an interrupt when the error is disabled, or reset when the error is detected.



## Parameters

|             |                      |
|-------------|----------------------|
| <i>mode</i> | Monitor mode to set. |
|-------------|----------------------|

**4.6.46 static bool CLOCK\_IsSysOscSelected ( void ) [inline], [static]**

## Returns

True if system OSC is used as clock source, false if not.

**4.6.47 static bool CLOCK\_IsSysOscValid ( void ) [inline], [static]**

## Returns

True if clock is valid, false if not.

**4.6.48 status\_t CLOCK\_InitFro ( const cgc\_fro\_config\_t \* *config* )**

This function initializes the CGC FRO clock according to the configuration.

## Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>config</i> | Pointer to the configuration structure. |
|---------------|-----------------------------------------|

## Return values

|                         |                                                   |
|-------------------------|---------------------------------------------------|
| <i>kStatus_Success</i>  | FRO is initialized.                               |
| <i>kStatus_CGC_Busy</i> | FRO has been enabled and is used by system clock. |
| <i>kStatus_ReadOnly</i> | FRO control register is locked.                   |

## Note

This function can't detect whether the FRO has been enabled and used by an IP.

**4.6.49 status\_t CLOCK\_DeinitFro ( void )**

This function deinitializes the CGC FRO.

Return values

|                         |                                 |
|-------------------------|---------------------------------|
| <i>kStatus_Success</i>  | FRO is deinitialized.           |
| <i>kStatus_CGC_Busy</i> | FRO is used by system clock.    |
| <i>kStatus_ReadOnly</i> | FRO control register is locked. |

Note

This function can't detect whether the FRO is used by an IP.

#### 4.6.50 void CLOCK\_SetRtdFroAsyncClkDiv ( cgc\_async\_clk\_t *asyncClk*, uint8\_t *divider* )

Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure.                |
| <i>divider</i>  | The divider value to set. Disabled when divider == 0. |

Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.51 void CLOCK\_SetAdFroAsyncClkDiv ( cgc\_async\_clk\_t *asyncClk*, uint8\_t *divider* )

Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure.                |
| <i>divider</i>  | The divider value to set. Disabled when divider == 0. |

Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.52 void CLOCK\_SetLpavFroAsyncClkDiv ( cgc\_async\_clk\_t *asyncClk*, uint8\_t *divider* )

## Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure.                |
| <i>divider</i>  | The divider value to set. Disabled when divider == 0. |

## Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.53 void CLOCK\_EnableFroTuning ( bool *enable* )

On enable, the function will wait until FRO is close to the target frequency.

#### 4.6.54 uint32\_t CLOCK\_GetFroFreq ( void )

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.55 uint32\_t CLOCK\_GetRtdFroAsyncFreq ( cgc\_async\_clk\_t *type* )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.56 uint32\_t CLOCK\_GetAdFroAsyncFreq ( cgc\_async\_clk\_t *type* )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.57 **uint32\_t** **CLOCK\_GetLpavFroAsyncFreq** ( **cgc\_async\_clk\_t** *type* )

Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.58 **static bool** **CLOCK\_IsFroSelected** ( **void** ) [**inline**], [**static**]

Returns

True if FRO is used as clock source, false if not.

#### 4.6.59 **static bool** **CLOCK\_IsFroValid** ( **void** ) [**inline**], [**static**]

Returns

True if clock is valid, false if not.

#### 4.6.60 **status\_t** **CLOCK\_InitLposc** ( **const cgc\_lposc\_config\_t** \* *config* )

This function initializes the CGC LPOSC clock according to the configuration.

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>config</i> | Pointer to the configuration structure. |
|---------------|-----------------------------------------|

Return values

|                         |                                 |
|-------------------------|---------------------------------|
| <i>kStatus_Success</i>  | LPOSC is initialized.           |
| <i>kStatus_ReadOnly</i> | FRO control register is locked. |

Note

This function can't detect whether the LPOSC has been enabled and used by an IP.

#### 4.6.61 **status\_t** CLOCK\_DeinitLposc ( void )

This function deinitializes the CGC LPOSC.

Return values

|                         |                                   |
|-------------------------|-----------------------------------|
| <i>kStatus_Success</i>  | LPOSC is deinitialized.           |
| <i>kStatus_ReadOnly</i> | LPOSC control register is locked. |

Note

This function can't detect whether the LPOSC is used by an IP.

#### 4.6.62 **static bool** CLOCK\_IsLpOscValid ( void ) [inline], [static]

Returns

True if clock is valid, false if not.

#### 4.6.63 **uint32\_t** CLOCK\_GetLpOscFreq ( void )

Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.64 **uint32\_t** CLOCK\_GetRtcOscFreq ( void )

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.65 static bool CLOCK\_IsRtcOscErr ( void ) [inline], [static]**

Returns

True if error occurs, false if not.

**4.6.66 void CLOCK\_SetRtcOscMonitorMode ( cgc\_rosc\_monitor\_mode\_t mode )**

This function sets the RTC OSC monitor mode. The mode can be disabled. It can generate an interrupt when the error is disabled, or reset when the error is detected.

Parameters

|             |                      |
|-------------|----------------------|
| <i>mode</i> | Monitor mode to set. |
|-------------|----------------------|

**4.6.67 static bool CLOCK\_IsRtcOscSelected ( void ) [inline], [static]**

Returns

True if RTCOSC is used as clock source, false if not.

**4.6.68 static bool CLOCK\_IsRtcOscValid ( void ) [inline], [static]**

Returns

True if the clock is valid, false if not.

**4.6.69 status\_t CLOCK\_InitPll0 ( const cgc\_pll0\_config\_t \* config )**

This function enables the CGC PLL0 clock according to the configuration. The PLL0 can use the OSC or FRO as the clock source. Ensure that the source clock is valid before calling this function.

Example code for initializing PLL0 clock output:

```
* const cgc_pll0_config_t g_cgcPll0Config = {.enableMode =
 kCGC_PllEnable,
* .div1 = 1U,
* .pfd1Div = 2U,
* .pfd2Div = 0U,
* .src = kCGC_PllSrcSysOsc,
* .mult = kCGC_Pll0Mult20};
* CLOCK_InitPll0(&g_cgcPll0Config);
*
```

## Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>config</i> | Pointer to the configuration structure. |
|---------------|-----------------------------------------|

## Return values

|                         |                                                        |
|-------------------------|--------------------------------------------------------|
| <i>kStatus_Success</i>  | PLL0 is initialized.                                   |
| <i>kStatus_CGC_Busy</i> | PLL0 has been enabled and is used by the system clock. |
| <i>kStatus_ReadOnly</i> | PLL0 control register is locked.                       |

## Note

This function can't detect whether the PLL0 has been enabled and used by an IP.

#### 4.6.70 **status\_t CLOCK\_DeinitPll0 ( void )**

This function disables the CGC PLL0.

## Return values

|                         |                                   |
|-------------------------|-----------------------------------|
| <i>kStatus_Success</i>  | PLL0 is deinitialized.            |
| <i>kStatus_CGC_Busy</i> | PLL0 is used by the system clock. |
| <i>kStatus_ReadOnly</i> | PLL0 control register is locked.  |

## Note

This function can't detect whether the PLL0 is used by an IP.

#### 4.6.71 **void CLOCK\_SetPll0AsyncClkDiv ( cgc\_async\_clk\_t asyncClk, uint8\_t divider )**

## Parameters

|                 |                                        |
|-----------------|----------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure. |
|-----------------|----------------------------------------|

|                |                                                       |
|----------------|-------------------------------------------------------|
| <i>divider</i> | The divider value to set. Disabled when divider == 0. |
|----------------|-------------------------------------------------------|

## Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.72 uint32\_t CLOCK\_GetPll0Freq ( void )

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.73 uint32\_t CLOCK\_GetPll0AsyncFreq ( cgc\_async\_clk\_t type )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.74 uint32\_t CLOCK\_GetPll0PfdFreq ( cgc\_pll\_pfd\_clkout\_t pfdClkout )

## Parameters

|                  |                                                         |
|------------------|---------------------------------------------------------|
| <i>pfdClkout</i> | The selected PFD clock out. See "cgc_pll_pfd_clkout_t". |
|------------------|---------------------------------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.75 void CLOCK\_EnablePll0PfdClkout ( cgc\_pll\_pfd\_clkout\_t pfdClkout, uint8\_t fracValue )

PLL Frequency = Fref \* MULT PFD Clock Frequency = PLL output frequency \* 18/frac value



```

* Example code for configuring PLL0 PFD0 clock output:
* const cgc_pll0_config_t g_cgcPll0Config = {.enableMode =
* kCGC_PllEnable,
*
* .div1 = 1U,
* .pfd1Div = 2U,
* .pfd2Div = 0U,
* .src = kCGC_PllSrcSysOsc,
* .mult = kCGC_Pll0Mult20};
* CLOCK_InitPll0(&g_cgcPll0Config);
* CLOCK_EnablePll0PfdClkout(kCGC_PllPfd0Clk, 15U);
*

```

## Parameters

|                  |                                                                              |
|------------------|------------------------------------------------------------------------------|
| <i>pfdClkout</i> | PLL0 PFD clock out select.                                                   |
| <i>fracValue</i> | Fractional Divider value. Recommended to be kept between 12-35 for all PFDs. |

#### 4.6.76 static void CLOCK\_SetPll0LockTime ( uint16\_t *lockTime* ) [inline], [static]

## Parameters

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>lockTime</i> | Reference clocks to count before PLL0 is considered locked and valid. |
|-----------------|-----------------------------------------------------------------------|

#### 4.6.77 static bool CLOCK\_IsPll0Selected ( void ) [inline], [static]

## Returns

True if PLL0 is used as clock source, false if not.

#### 4.6.78 static bool CLOCK\_IsPll0Valid ( void ) [inline], [static]

## Returns

True if the clock is valid, false if not.

#### 4.6.79 status\_t CLOCK\_InitPll1 ( const cgc\_pll1\_config\_t \* *config* )

This function enables the CGC PLL1 clock according to the configuration. The PLL1 can use the system OSC or FRO as the clock source. Ensure that the source clock is valid before calling this function.

Example code for initializing PLL1 clock output:

```

* const cgc_pll1_config_t g_cgcPll1Config = {.enableMode =
* kCGC_PllEnable,
*
* .div1 = 0U,
* .pfd1Div = 0U,
* .pfd2Div = 0U,
* .src = kCGC_PllSrcFro24M,
* .mult = kCGC_PllMult22,
* .num = 578,
* .denom = 1000};
* CLOCK_InitPll1(&g_cgcPll1Config);
*

```

## Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>config</i> | Pointer to the configuration structure. |
|---------------|-----------------------------------------|

## Return values

|                         |                                                        |
|-------------------------|--------------------------------------------------------|
| <i>kStatus_Success</i>  | PLL1 is initialized.                                   |
| <i>kStatus_CGC_Busy</i> | PLL1 has been enabled and is used by the system clock. |
| <i>kStatus_ReadOnly</i> | PLL1 control register is locked.                       |

## Note

This function can't detect whether the PLL1 has been enabled and is used by an IP.

#### 4.6.80 status\_t CLOCK\_DeinitPll1 ( void )

This function disables the CGC PLL1.

## Return values

|                         |                                   |
|-------------------------|-----------------------------------|
| <i>kStatus_Success</i>  | PLL1 is deinitialized.            |
| <i>kStatus_CGC_Busy</i> | PLL1 is used by the system clock. |
| <i>kStatus_ReadOnly</i> | PLL1 control register is locked.  |

## Note

This function can't detect whether the PLL1 is used by an IP.

#### 4.6.81 void CLOCK\_SetPll1AsyncClkDiv ( cgc\_async\_clk\_t asyncClk, uint8\_t divider )

## Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure.                |
| <i>divider</i>  | The divider value to set. Disabled when divider == 0. |

## Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.82 uint32\_t CLOCK\_GetPll1Freq ( void )

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.83 uint32\_t CLOCK\_GetPll1AsyncFreq ( cgc\_async\_clk\_t type )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.84 uint32\_t CLOCK\_GetPll1PfdFreq ( cgc\_pll\_pfd\_clkout\_t pfdClkout )

## Parameters

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <i>pfdClkout</i> | The selected PFD clocks out. See "cgc_pll_pfd_clkout_t". |
|------------------|----------------------------------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.85 void CLOCK\_EnablePll1PfdClkout ( cgc\_pll\_pfd\_clkout\_t *pfdClkout*, uint8\_t *fracValue* )

PLL1 Frequency = Fref \* (MULT + NUM/DENOM) PFD Clock Frequency = PLL output frequency \* 18/frac value

Example code for configuring PLL1 as PLL1 PFD clock output:

```
* const cgc_pll1_config_t g_cgcPll1Config = {.enableMode =
 kCGC_PllEnable,
*
* .div1 = 0U,
* .pfd1Div = 0U,
* .pfd2Div = 0U,
* .src = kCGC_PllSrcFro24M,
* .mult = kCGC_Pll1Mult22,
* .num = 578,
* .denom = 1000};
* CLOCK_InitPll1(&g_cgcPll1Config);
* CLOCK_EnablePll1PfdClkout(kCGC_Pll1Pfd0Clk, 15U);
*
```

Parameters

|                  |                                                                              |
|------------------|------------------------------------------------------------------------------|
| <i>pfdClkout</i> | PLL1 PFD clock out select.                                                   |
| <i>fracValue</i> | Fractional Divider value. Recommended to be kept between 12-35 for all PFDs. |

#### 4.6.86 static void CLOCK\_EnablePll1SpectrumModulation ( uint16\_t *step*, uint16\_t *stop* ) [inline], [static]

This function sets the CGC PLL1 spread spectrum modulation configurations. STOP and STEP together control the modulation depth (maximum frequency change) and modulation frequency.

Modulation Depth = (STOP/MFD)\*Fref where MFD is the DENOM field value in DENOM register.  
Modulation Frequency = (STEP/(2\*STOP))\*Fref.

Parameters

|             |                            |
|-------------|----------------------------|
| <i>step</i> | PLL1 Spread Spectrum STEP. |
| <i>stop</i> | PLL1 Spread Spectrum STOP. |

#### 4.6.87 static void CLOCK\_SetPll1LockTime ( uint16\_t *lockTime* ) [inline], [static]

## Parameters

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>lockTime</i> | Reference clocks to count before PLL1 is considered locked and valid. |
|-----------------|-----------------------------------------------------------------------|

**4.6.88 static bool CLOCK\_IsPll1Selected ( void ) [inline], [static]**

## Returns

True if PLL1 is used as clock source, false if not.

**4.6.89 static bool CLOCK\_IsPll1Valid ( void ) [inline], [static]**

## Returns

True if the clock is valid, false if not.

**4.6.90 uint32\_t CLOCK\_GetPll3Freq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.91 uint32\_t CLOCK\_GetPll3AsyncFreq ( cgc\_async\_clk\_t type )**

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.92 uint32\_t CLOCK\_GetPll3PfdFreq ( cgc\_pll\_pfd\_clkout\_t pfdClkout )**

## Parameters

|                  |                                                         |
|------------------|---------------------------------------------------------|
| <i>pfdClkout</i> | The selected PFD clock out. See "cgc_pll_pfd_clkout_t". |
|------------------|---------------------------------------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.93 status\_t CLOCK\_InitPll4 ( const cgc\_pll4\_config\_t \* config )

This function enables the CGC PLL4 clock according to the configuration. The PLL4 can use the OSC or FRO as the clock source. Ensure that the source clock is valid before calling this function.

Example code for initializing PLL4 clock output:

```
* const cgc_pll4_config_t g_cgcPll4Config = {.enableMode =
 kCGC_PllEnable,
*
* .div1 = 0U,
* .pfd0Div1 = 0U,
* .pfd0Div2 = 0U,
* .pfd1Div1 = 0U,
* .pfd1Div2 = 0U,
* .pfd2Div1 = 0U,
* .pfd2Div2 = 0U,
* .pfd3Div1 = 0U,
* .pfd3Div2 = 0U,
* .src = kCGC_PllSrcFro24M,
* .mult = kCGC_Pll4Mult22,
* .num = 578,
* .denom = 1000};
* CLOCK_InitPll4(&g_cgcPll4Config);
*
```

## Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>config</i> | Pointer to the configuration structure. |
|---------------|-----------------------------------------|

## Return values

|                         |                                                        |
|-------------------------|--------------------------------------------------------|
| <i>kStatus_Success</i>  | PLL4 is initialized.                                   |
| <i>kStatus_CGC_Busy</i> | PLL4 has been enabled and is used by the system clock. |
| <i>kStatus_ReadOnly</i> | PLL4 control register is locked.                       |

## Note

This function can't detect whether the PLL4 has been enabled and used by an IP.

#### 4.6.94 status\_t CLOCK\_DeinitPll4 ( void )

This function disables the CGC PLL4.

Return values

|                         |                                   |
|-------------------------|-----------------------------------|
| <i>kStatus_Success</i>  | PLL4 is deinitialized.            |
| <i>kStatus_CGC_Busy</i> | PLL4 is used by the system clock. |
| <i>kStatus_ReadOnly</i> | PLL4 control register is locked.  |

Note

This function can't detect whether the PLL4 is used by an IP.

#### 4.6.95 void CLOCK\_SetPll4AsyncClkDiv ( cgc\_async\_clk\_t *asyncClk*, uint8\_t *divider* )

Parameters

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| <i>asyncClk</i> | Which asynchronous clock to configure.                |
| <i>divider</i>  | The divider value to set. Disabled when divider == 0. |

Note

There might be glitch when changing the asynchronous divider, so make sure the asynchronous clock is not used while changing divider.

#### 4.6.96 uint32\_t CLOCK\_GetPll4Freq ( void )

Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.97 uint32\_t CLOCK\_GetPll4AsyncFreq ( cgc\_async\_clk\_t *type* )

Parameters



|             |                              |
|-------------|------------------------------|
| <i>type</i> | The asynchronous clock type. |
|-------------|------------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.98 uint32\_t CLOCK\_GetPll4PfdFreq ( cgc\_pll\_pfd\_clkout\_t *pfdClkout* )

Parameters

|                  |                                                         |
|------------------|---------------------------------------------------------|
| <i>pfdClkout</i> | The selected PFD clock out. See "cgc_pll_pfd_clkout_t". |
|------------------|---------------------------------------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.99 void CLOCK\_EnablePll4PfdClkout ( cgc\_pll\_pfd\_clkout\_t *pfdClkout*, uint8\_t *fracValue* )

PLL Frequency = Fref \* MULT PFD Clock Frequency = PLL output frequency \* 18/frac value

```
* Example code for configuring PLL4 PFD0 clock output:
* const cgc_pll4_config_t g_cgcPll4Config = {.enableMode =
* kCGC_PllEnable,
*
* .div1 = 0U,
* .pfd0Div1 = 0U,
* .pfd0Div2 = 0U,
* .pfd1Div1 = 0U,
* .pfd1Div2 = 0U,
* .pfd2Div1 = 0U,
* .pfd2Div2 = 0U,
* .pfd3Div1 = 0U,
* .pfd3Div2 = 0U,
* .src = kCGC_PllSrcFro24M,
* .mult = kCGC_Pll4Mult22,
* .num = 578,
* .denom = 1000};
* CLOCK_InitPll4(&g_cgcPll4Config);
* CLOCK_EnablePll4PfdClkout(kCGC_PllPfd0Clk, 15U);
*
```

## Parameters

|                  |                                                                              |
|------------------|------------------------------------------------------------------------------|
| <i>pfdClkout</i> | PLL4 PFD clock out select.                                                   |
| <i>fracValue</i> | Fractional Divider value. Recommended to be kept between 12-35 for all PFDs. |

#### 4.6.100 static void CLOCK\_EnablePll4SpectrumModulation ( uint16\_t *step*, uint16\_t *stop* ) [inline], [static]

This function sets the CGC PLL4 spread spectrum modulation configurations. STOP and STEP together control the modulation depth (maximum frequency change) and modulation frequency.

Modulation Depth = (STOP/MFD)\*Fref where MFD is the DENOM field value in DENOM register.  
Modulation Frequency = (STEP/(2\*STOP))\*Fref.

## Parameters

|             |                            |
|-------------|----------------------------|
| <i>step</i> | PLL4 Spread Spectrum STEP. |
| <i>stop</i> | PLL4 Spread Spectrum STOP. |

#### 4.6.101 static void CLOCK\_SetPll4LockTime ( uint16\_t *lockTime* ) [inline], [static]

## Parameters

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>lockTime</i> | Reference clocks to count before PLL4 is considered locked and valid. |
|-----------------|-----------------------------------------------------------------------|

#### 4.6.102 static bool CLOCK\_IsPll4Selected ( void ) [inline], [static]

## Returns

True if PLL4 is used as clock source, false if not.

#### 4.6.103 static bool CLOCK\_IsPll4Valid ( void ) [inline], [static]

## Returns

True if the clock is valid, false if not.

**4.6.104**   `static void CLOCK_SetXtal0Freq ( uint32_t freq ) [inline], [static]`

## Parameters

|             |                                               |
|-------------|-----------------------------------------------|
| <i>freq</i> | The XTAL0/EXTAL0 input clock frequency in Hz. |
|-------------|-----------------------------------------------|

**4.6.105 static void CLOCK\_SetXtal32Freq ( uint32\_t *freq* ) [inline], [static]**

## Parameters

|             |                                                 |
|-------------|-------------------------------------------------|
| <i>freq</i> | The XTAL32/EXTAL32 input clock frequency in Hz. |
|-------------|-------------------------------------------------|

**4.6.106 static void CLOCK\_SetLvdsFreq ( uint32\_t *freq* ) [inline], [static]**

## Parameters

|             |                                           |
|-------------|-------------------------------------------|
| <i>freq</i> | The LVDS pad input clock frequency in Hz. |
|-------------|-------------------------------------------|

**4.6.107 static void CLOCK\_SetMclkFreq ( uint32\_t *index*, uint32\_t *freq* ) [inline], [static]**

## Parameters

|              |                                           |
|--------------|-------------------------------------------|
| <i>index</i> | The MCLK index.                           |
| <i>freq</i>  | The MCLK pad input clock frequency in Hz. |

**4.6.108 static void CLOCK\_SetRxBclkFreq ( uint32\_t *instance*, uint32\_t *freq* ) [inline], [static]**

## Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>instance</i> | The SAI instance to contribute to this RX_BCLK pad. |
| <i>freq</i>     | The RX_BCLK pad input clock frequency in Hz.        |

**4.6.109 static void CLOCK\_SetTxBclkFreq ( uint32\_t *instance*, uint32\_t *freq* ) [inline], [static]**

## Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>instance</i> | The SAI instance to contribute to this TX_BCLK pad. |
| <i>freq</i>     | The TX_BCLK pad input clock frequency in Hz.        |

#### 4.6.110 static void CLOCK\_SetSpdifRxFreq ( uint32\_t *freq* ) [inline], [static]

## Parameters

|             |                                           |
|-------------|-------------------------------------------|
| <i>freq</i> | The SPDIF_RX input clock frequency in Hz. |
|-------------|-------------------------------------------|

#### 4.6.111 uint32\_t CLOCK\_GetWdogClkFreq ( uint32\_t *instance* )

## Parameters

|                 |                            |
|-----------------|----------------------------|
| <i>instance</i> | The WDOG instance (0-2,5). |
|-----------------|----------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.112 uint32\_t CLOCK\_GetFlexspiClkFreq ( uint32\_t *instance* )

## Parameters

|                 |                             |
|-----------------|-----------------------------|
| <i>instance</i> | The FlexSPI instance (0-1). |
|-----------------|-----------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

#### 4.6.113 uint32\_t CLOCK\_GetLpitClkFreq ( void )

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.114 uint32\_t CLOCK\_GetFlexioClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.115 uint32\_t CLOCK\_GetI3cClkFreq ( uint32\_t *instance* )**

Parameters

|                 |                         |
|-----------------|-------------------------|
| <i>instance</i> | The I3C instance (0-1). |
|-----------------|-------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.116 uint32\_t CLOCK\_GetLpspiClkFreq ( uint32\_t *instance* )**

Parameters

|                 |                           |
|-----------------|---------------------------|
| <i>instance</i> | The LPSPI instance (0-3). |
|-----------------|---------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.117 uint32\_t CLOCK\_GetAdcClkFreq ( uint32\_t *instance* )**

Parameters

|                 |                         |
|-----------------|-------------------------|
| <i>instance</i> | The ADC instance (0-1). |
|-----------------|-------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.118 uint32\_t CLOCK\_GetDacClkFreq ( uint32\_t *instance* )**

## Parameters

|                 |                         |
|-----------------|-------------------------|
| <i>instance</i> | The DAC instance (0-1). |
|-----------------|-------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.119 uint32\_t CLOCK\_GetTpiuClkFreq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.120 uint32\_t CLOCK\_GetSwoClkFreq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.121 uint32\_t CLOCK\_GetTpmClkFreq ( uint32\_t *instance* )**

## Parameters

|                 |                         |
|-----------------|-------------------------|
| <i>instance</i> | The TPM instance (0-8). |
|-----------------|-------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.122 uint32\_t CLOCK\_GetLpi2cClkFreq ( uint32\_t *instance* )**

## Parameters

|                 |                           |
|-----------------|---------------------------|
| <i>instance</i> | The LPI2C instance (0-3). |
|-----------------|---------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.123**   `uint32_t` **CLOCK\_GetLpuartClkFreq** ( `uint32_t` *instance* )



## Parameters

|                 |                            |
|-----------------|----------------------------|
| <i>instance</i> | The LPUART instance (0-3). |
|-----------------|----------------------------|

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.124 uint32\_t CLOCK\_GetFlexcanClkFreq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.125 uint32\_t CLOCK\_GetCsiClkFreq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.126 uint32\_t CLOCK\_GetDsiClkFreq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.127 uint32\_t CLOCK\_GetEpdcClkFreq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.128 uint32\_t CLOCK\_GetGpu2dClkFreq ( void )**

## Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.129 uint32\_t CLOCK\_GetGpu3dClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.130 uint32\_t CLOCK\_GetDcnanoClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.131 uint32\_t CLOCK\_GetCsiUiClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.132 uint32\_t CLOCK\_GetCsiEscClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.133 uint32\_t CLOCK\_GetRtdAudClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.134 uint32\_t CLOCK\_GetAdAudClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.135 uint32\_t CLOCK\_GetLpavAudClkFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.136 uint32\_t CLOCK\_GetSaiFreq ( uint32\_t *instance* )**

Parameters

|                 |                         |
|-----------------|-------------------------|
| <i>instance</i> | The SAI instance (0-7). |
|-----------------|-------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.137 uint32\_t CLOCK\_GetSpdifFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.138 uint32\_t CLOCK\_GetMqsFreq ( uint32\_t *instance* )**

Parameters

|                 |                         |
|-----------------|-------------------------|
| <i>instance</i> | The MQS instance (0-1). |
|-----------------|-------------------------|

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.139 uint32\_t CLOCK\_GetMicfilFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.140 uint32\_t CLOCK\_GetMrtFreq ( void )**

Returns

Clock frequency; If the clock is invalid, returns 0.

**4.6.141 uint32\_t CLOCK\_GetRtdSysClkFreq ( uint32\_t *config*, cgc\_sys\_clk\_t *type* )**

This function gets the CGC system clock frequency. These clocks are used for core, platform, bus and slow clock domains.

Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>config</i> | Config value from CGC register. |
| <i>type</i>   | Which type of clock to get.     |

Returns

Clock frequency.

**4.7 Variable Documentation****4.7.1 volatile uint32\_t g\_xtal0Freq**

The XTAL (SYSOSC) clock frequency in Hz. When the clock is set up, use the function CLOCK\_SetXtalFreq to set the value in the clock driver. For example, if XTAL is 24 MHz:

```
* CLOCK_InitSysOsc(...);
* CLOCK_SetXtalFreq(24000000);
*
```

This is important for the multicore platforms where only one core needs to set up the OSC/SYSOSC using CLOCK\_InitSysOsc. All other cores need to call the CLOCK\_SetXtalFreq to get a valid clock frequency.

**4.7.2 volatile uint32\_t g\_xtal32Freq**

The XTAL32/EXTAL32 clock frequency in Hz. When the clock is set up, use the function CLOCK\_SetXtal32Freq to set the value in the clock driver.

This is important for the multicore platforms where only one core needs to set up the clock. All other cores need to call the CLOCK\_SetXtal32Freq to get a valid clock frequency.

#### 4.7.3 volatile uint32\_t g\_lvdsFreq

The LVDS pad clock frequency in Hz. When the clock is set up, use the function `CLOCK_SetLvdsFreq` to set the value in the clock driver.

#### 4.7.4 volatile uint32\_t g\_mclkFreq[4]

The MCLK pad clock frequency in Hz. When the clock is set up, use the function `CLOCK_SetMclkFreq` to set the value in the clock driver.

#### 4.7.5 volatile uint32\_t g\_rxBclkFreq[8]

The RX\_BCLK pad clock frequency in Hz. When the clock is set up, use the function `CLOCK_SetRx-BclkFreq` to set the value in the clock driver.

#### 4.7.6 volatile uint32\_t g\_txBclkFreq[8]

The TX\_BCLK pad clock frequency in Hz. When the clock is set up, use the function `CLOCK_SetTx-BclkFreq` to set the value in the clock driver.

#### 4.7.7 volatile uint32\_t g\_spdifRxFreq

The SPDIF\_RX clock frequency in Hz. When the clock is sampled, use the function `CLOCK_SetSpdif-RxFreq` to set the value in the clock driver.



## Chapter 5

# Fusion Driver

Fusion driver provides APIs for Fusion (DSP Domain) operation.

The fusion driver supports:

- Fusion (DSP Domain) init
- Fusion core start and stop



## Chapter 6

### IOMUXC: IOMUX Controller

IOMUXC driver provides APIs for pin configuration. It also supports the miscellaneous functions integrated in IOMUXC.



## **Chapter 7**

### **Reset Driver**

Reset driver supports peripheral reset.





## **Chapter 8**

# **Sentinel Driver**

Sentinel driver supports Sentinel(S400) system.



## **Chapter 9**

# **Upower Driver**

Upower driver supports uPower Controller.

## 9.1 System Clock Generator (SCG)

The MCUXpresso SDK provides a peripheral driver for the System Clock Generator (SCG) module of MCUXpresso SDK devices.

### 9.1.1 Function description

The SCG module contains the system PLL (SPLL), a slow internal reference clock (SIRC), a fast internal reference clock (FIRC), a low power FLL, and the system oscillator clock (SOSC). They can be configured separately as the source of MCU system clocks. Accordingly, the SCG driver provides these functions:

- MCU system clock configuration.
- SCG SOSC configuration.
- SCG SIRC configuration.
- SCG FIRC configuration.
- SCG SPLL configuration.
- SCG LPFLL configuration.

#### 9.1.1.1 MCU System Clock

MCU system clock configurations include the clock source selection and the clock dividers. The configurations for VLPR, RUN, and HSRUN modes are set separately using the `CLOCK_SetVlprModeSysClkConfig()`, `CLOCK_SetRunModeSysClkConfig()`, and the `CLOCK_SetHsrunModeSysClkConfig()` functions to configure the MCU system clock.

The current MCU system clock configuration can be obtained with the function `CLOCK_GetCurSysClkConfig()`. The current MCU system clock frequency can be obtained with the `CLOCK_GetSysClkFreq()` function.

#### 9.1.1.2 SCG System OSC Clock

The functions `CLOCK_InitSysOsc()`/`CLOCK_DeinitSysOsc()` are used for the SOSC clock initialization. The function `CLOCK_InitSysOsc` disables the SOSC internally and re-configures it. As a result, ensure that the SOSC is not used while calling these functions.

The SOSC clock can be used directly as the MCU system clock source. The `SOSCDIV1_CLK`, `SOSCDIV2_CLK`, and `SOSCDIV3_CLK` can be used as the peripheral clock source. The clocks frequencies can be obtained by functions `CLOCK_GetSysOscFreq()` and `CLOCK_GetSysOscAsyncFreq()`.

To configure the SOSC monitor mode, use the function `CLOCK_SetSysOscMonitorMode()`. The clock error status can be received and cleared with the `CLOCK_IsSysOscErr()` and `CLOCK_ClearSysOscErr()` functions.

### 9.1.1.3 SCG Slow IRC Clock

The functions `CLOCK_InitSirc()/CLOCK_DeinitSirc()` are used for the SIRC clock initialization. The function `CLOCK_InitSirc` disables the SIRC internally and re-configures it. Ensure that the SIRC is not used while calling these functions.

The SIRC clock can be used directly as the MCU system clock source. The `SIRCDIV1_CLK`, `SIRCDIV2_CLK`, and `SIRCDIV3_CLK` can be used as the peripheral clock source. The clocks frequencies can be received with functions `CLOCK_GetSircFreq()` and `CLOCK_GetSircAsyncFreq()`.

### 9.1.1.4 SCG Fast IRC Clock

The functions `CLOCK_InitFirc()/CLOCK_DeinitFirc()` are used for the FIRC clock initialization. The function `CLOCK_InitFirc` disables the FIRC internally and re-configures it. Ensure that the FIRC is not used while calling these functions.

The FIRC clock can be used directly as the MCU system clock source. The `FIRCDIV1_CLK`, `FIRCDIV2_CLK`, and `FIRCDIV3_CLK` can be used as the peripheral clock source. The clocks frequencies could be obtained by functions `CLOCK_GetFircFreq()` and `CLOCK_GetFircAsyncFreq()`.

The FIRC can be trimmed by the external clock. See the Section "Typical use case" to enable the FIRC trim.

### 9.1.1.5 SCG Low Power FLL Clock

The functions `CLOCK_InitLpFll()/CLOCK_DeinitLpFll()` are used for the LPFLL clock initialization. The function `CLOCK_InitLpFll` disables the LPFLL internally and re-configures it. Ensure that the LPFLL is not used while calling these functions.

The LPFLL clock can be used directly as the MCU system clock source. The `LPFLLDIV1_CLK`, `LPFLLDIV2_CLK`, and `LPFLLDIV3_CLK` can be used as the peripheral clock source. The clocks frequencies could be obtained by functions `CLOCK_GetLpFllFreq()` and `CLOCK_GetLpFllAsyncFreq()`.

The LPFLL can be trimmed by the external clock, specific the `trimConfig` in `scg_lpfl_config_t` to enable the clock trim.

### 9.1.1.6 SCG System PLL Clock

The functions `CLOCK_InitSysPll()/CLOCK_DeinitSysPll()` are used for the SPLL clock initialization. The function `CLOCK_InitSysPll` disables the SPLL internally and re-configures it. Ensure that the SPLL is not used while calling these functions.

To generate the desired SPLL frequency, `PREDIV` and `MULT` value must be set properly while initializing the SPLL. The function `CLOCK_GetSysPllMultDiv()` calculates the `PREDIV` and `MULT`. Passing in the reference clock frequency and the desired output frequency, the function returns the `PREDIV` and `MULT` which generate the frequency closest to the desired frequency.

Because the SPLL is based on the FIRC or SOSC, the FIRC or SOSC must be enabled first before the SPLL initialization. Also, when re-configuring the FIRC or SOSC, be careful with the SPLL.

The SPLL clock can be used directly as the MCU system clock source. The SPLLDIV1\_CLK, SPLLDIV2\_CLK, and SPLLDIV3\_CLK can be used as the peripheral clock source. The clocks frequencies can be obtained with functions `CLOCK_GetSysPllFreq()` and `CLOCK_GetSysPllAsyncFreq()`.

To configure the SPLL monitor mode, use the function `CLOCK_SetSysPllMonitorMode()`. The clock error status can be received and cleared by the `CLOCK_IsSysPllErr()` and `CLOCK_ClearSysPllErr()`.

### 9.1.1.7 SCG clock valid check

The functions such as the `CLOCK_IsFircValid()` are used to check whether a specific clock is valid or not. See "Typical use case" for details.

The clocks are valid after the initialization functions such as the `CLOCK_InitFirc()`. As a result, it is not necessary to call the `CLOCK_IsFircValid()` after the `CLOCK_InitFirc()`.

## 9.1.2 Typical use case

### 9.1.2.1 FIRC clock trim

During the FIRC initialization, applications can choose whether to enable trim or not.

1. Trim is not enabled. Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/scg`
2. Trim is enabled. Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/scg`

### 9.1.2.2 SPLL initialization

The following code shows how to set up the SCG SPLL. The SPLL uses the SOSC as a reference clock. Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/scg`

### 9.1.2.3 System clock configuration

While changing the system clock configuration, the actual system clock does not change until the target clock source is valid. Ensure that the clock source is valid before using it. The functions such as `CLOCK_IsSircValid()` are used for this purpose.

The SCG has a dedicated system clock configuration registers for VLPR, RUN, and HSRUN modes. During the power mode change, the system clock configuration may change too. In this case, check whether the clock source is valid during the power mode change.

In the following example, the SIRC is used as the system clock source in VLPR mode, the FIRC is used as a system clock source in RUN mode, and the SPLL is used as a system clock source in HSRUN mode.

The example work flow:

1. SIRC, FIRC, and SPLN are all enabled in RUN mode.
2. MCU enters VLPR mode. In VLPR mode, FIRC, and SPLN are disabled automatically.
3. MCU enters RUN mode. Wait for the FIRC to become valid.
4. MCU enters HSRUN mode. In step 3, the SPLN is already enabled, but may not be valid. Wait for it to become valid when entering HSRUN mode. Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/scg`

# Chapter 10

## ACMP: Analog Comparator Driver

### 10.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Comparator (ACMP) module of MCUXpresso SDK devices.

The ACMP driver is created to help the user operate the ACMP module better. This driver can be considered as a basic comparator with advanced features. The APIs for basic comparator can make the C-MP work as a general comparator, which compares the two input channel's voltage and creates the output of the comparator result immediately. The APIs for advanced feature can be used as the plug-in function based on the basic comparator, and can provide more ways to process the comparator's output.

### 10.2 Typical use case

#### 10.2.1 Normal Configuration

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/acmp

#### 10.2.2 Interrupt Configuration

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/acmp

#### 10.2.3 Round robin Configuration

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/acmp

### Data Structures

- struct [\\_acmp\\_config](#)  
*Configuration for ACMP. [More...](#)*
- struct [\\_acmp\\_channel\\_config](#)  
*Configuration for channel. [More...](#)*
- struct [\\_acmp\\_filter\\_config](#)  
*Configuration for filter. [More...](#)*
- struct [\\_acmp\\_dac\\_config](#)  
*Configuration for DAC. [More...](#)*
- struct [\\_acmp\\_round\\_robin\\_config](#)  
*Configuration for round robin mode. [More...](#)*
- struct [\\_acmp\\_discrete\\_mode\\_config](#)  
*Configuration for discrete mode. [More...](#)*

## Macros

- #define `CMP_C0_CFx_MASK` (`CMP_C0_CFR_MASK` | `CMP_C0_CFF_MASK`)  
*The mask of status flags cleared by writing 1.*

## Typedefs

- typedef enum `_acmp_hysteresis_mode` `acmp_hysteresis_mode_t`  
*Comparator hard block hysteresis control.*
- typedef enum `_acmp_reference_voltage_source` `acmp_reference_voltage_source_t`  
*CMP Voltage Reference source.*
- typedef enum `_acmp_fixed_port` `acmp_fixed_port_t`  
*Fixed mux port.*
- typedef enum `_acmp_dac_work_mode` `acmp_dac_work_mode_t`  
*Internal DAC's work mode.*
- typedef struct `_acmp_config` `acmp_config_t`  
*Configuration for ACMP.*
- typedef struct `_acmp_channel_config` `acmp_channel_config_t`  
*Configuration for channel.*
- typedef struct `_acmp_filter_config` `acmp_filter_config_t`  
*Configuration for filter.*
- typedef struct `_acmp_dac_config` `acmp_dac_config_t`  
*Configuration for DAC.*
- typedef struct `_acmp_round_robin_config` `acmp_round_robin_config_t`  
*Configuration for round robin mode.*
- typedef struct `_acmp_discrete_mode_config` `acmp_discrete_mode_config_t`  
*Configuration for discrete mode.*

## Enumerations

- enum `_acmp_interrupt_enable` {  
    `kACMP_OutputRisingInterruptEnable` = (1U << 0U),  
    `kACMP_OutputFallingInterruptEnable` = (1U << 1U),  
    `kACMP_RoundRobinInterruptEnable` = (1U << 2U) }  
*Interrupt enable/disable mask.*
- enum `_acmp_status_flags` {  
    `kACMP_OutputRisingEventFlag` = `CMP_C0_CFR_MASK`,  
    `kACMP_OutputFallingEventFlag` = `CMP_C0_CFF_MASK`,  
    `kACMP_OutputAssertEventFlag` = `CMP_C0_COUT_MASK` }  
*Status flag mask.*
- enum `_acmp_hysteresis_mode` {  
    `kACMP_HysteresisLevel0` = 0U,  
    `kACMP_HysteresisLevel1` = 1U,  
    `kACMP_HysteresisLevel2` = 2U,  
    `kACMP_HysteresisLevel3` = 3U }  
*Comparator hard block hysteresis control.*



- enum `_acmp_reference_voltage_source` {  
`kACMP_VrefSourceVin1` = 0U,  
`kACMP_VrefSourceVin2` = 1U }  
*CMP Voltage Reference source.*
- enum `_acmp_fixed_port` {  
`kACMP_FixedPlusPort` = 0U,  
`kACMP_FixedMinusPort` = 1U }  
*Fixed mux port.*
- enum `_acmp_dac_work_mode` {  
`kACMP_DACWorkLowSpeedMode` = 0U,  
`kACMP_DACWorkHighSpeedMode` = 1U }  
*Internal DAC's work mode.*

## Driver version

- #define `FSL_ACMP_DRIVER_VERSION` (`MAKE_VERSION(2, 3, 0)`)  
*ACMP driver version 2.3.0.*

## Initialization and deinitialization

- void `ACMP_Init` (`CMP_Type *base`, const `acmp_config_t *config`)  
*Initializes the ACMP.*
- void `ACMP_Deinit` (`CMP_Type *base`)  
*Deinitializes the ACMP.*
- void `ACMP_GetDefaultConfig` (`acmp_config_t *config`)  
*Gets the default configuration for ACMP.*

## Basic Operations

- void `ACMP_Enable` (`CMP_Type *base`, bool enable)  
*Enables or disables the ACMP.*
- void `ACMP_EnableLinkToDAC` (`CMP_Type *base`, bool enable)  
*Enables the link from CMP to DAC enable.*
- void `ACMP_SetChannelConfig` (`CMP_Type *base`, const `acmp_channel_config_t *config`)  
*Sets the channel configuration.*

## Advanced Operations

- void `ACMP_EnableDMA` (`CMP_Type *base`, bool enable)  
*Enables or disables DMA.*
- void `ACMP_EnableWindowMode` (`CMP_Type *base`, bool enable)  
*Enables or disables window mode.*
- void `ACMP_SetFilterConfig` (`CMP_Type *base`, const `acmp_filter_config_t *config`)  
*Configures the filter.*
- void `ACMP_SetDACConfig` (`CMP_Type *base`, const `acmp_dac_config_t *config`)  
*Configures the internal DAC.*
- void `ACMP_SetRoundRobinConfig` (`CMP_Type *base`, const `acmp_round_robin_config_t *config`)  
*Configures the round robin mode.*
- void `ACMP_SetRoundRobinPreState` (`CMP_Type *base`, `uint32_t mask`)  
*Defines the pre-set state of channels in round robin mode.*

- static uint32\_t [ACMP\\_GetRoundRobinStatusFlags](#) (CMP\_Type \*base)  
*Gets the channel input changed flags in round robin mode.*
- void [ACMP\\_ClearRoundRobinStatusFlags](#) (CMP\_Type \*base, uint32\_t mask)  
*Clears the channel input changed flags in round robin mode.*
- static uint32\_t [ACMP\\_GetRoundRobinResult](#) (CMP\_Type \*base)  
*Gets the round robin result.*

## Interrupts

- void [ACMP\\_EnableInterrupts](#) (CMP\_Type \*base, uint32\_t mask)  
*Enables interrupts.*
- void [ACMP\\_DisableInterrupts](#) (CMP\_Type \*base, uint32\_t mask)  
*Disables interrupts.*

## Status

- uint32\_t [ACMP\\_GetStatusFlags](#) (CMP\_Type \*base)  
*Gets status flags.*
- void [ACMP\\_ClearStatusFlags](#) (CMP\_Type \*base, uint32\_t mask)  
*Clears status flags.*

## Discrete mode

- void [ACMP\\_SetDiscreteModeConfig](#) (CMP\_Type \*base, const [acmp\\_discrete\\_mode\\_config\\_t](#) \*config)  
*Configure the discrete mode.*
- void [ACMP\\_GetDefaultDiscreteModeConfig](#) ([acmp\\_discrete\\_mode\\_config\\_t](#) \*config)  
*Get the default configuration for discrete mode setting.*

## 10.3 Data Structure Documentation

### 10.3.1 struct \_acmp\_config

#### Data Fields

- bool [enableHighSpeed](#)  
*Enable High Speed (HS) comparison mode.*
- bool [enableInvertOutput](#)  
*Enable inverted comparator output.*
- bool [useUnfilteredOutput](#)  
*Set compare output(COUT) to equal COUTA(true) or COUT(false).*
- bool [enablePinOut](#)  
*The comparator output is available on the associated pin.*

## Field Documentation

- (1) `bool _acmp_config::enableHighSpeed`
- (2) `bool _acmp_config::enableInvertOutput`
- (3) `bool _acmp_config::useUnfilteredOutput`
- (4) `bool _acmp_config::enablePinOut`

10.3.2 `struct _acmp_channel_config`

The comparator's port can be input from channel mux or DAC. If port input is from channel mux, detailed channel number for the mux should be configured.

## Data Fields

- `uint32_t plusMuxInput`  
*Plus mux input channel(0~7).*
- `uint32_t minusMuxInput`  
*Minus mux input channel(0~7).*

## Field Documentation

- (1) `uint32_t _acmp_channel_config::plusMuxInput`
- (2) `uint32_t _acmp_channel_config::minusMuxInput`

10.3.3 `struct _acmp_filter_config`

## Data Fields

- `bool enableSample`  
*Using external SAMPLE as sampling clock input, or using divided bus clock.*
- `uint32_t filterCount`  
*Filter Sample Count.*
- `uint32_t filterPeriod`  
*Filter Sample Period.*

## Field Documentation

- (1) `bool _acmp_filter_config::enableSample`
- (2) `uint32_t _acmp_filter_config::filterCount`

Available range is 1-7, 0 would cause the filter disabled.

**(3) uint32\_t \_acmp\_filter\_config::filterPeriod**

The divider to bus clock. Available range is 0-255.

**10.3.4 struct \_acmp\_dac\_config****Data Fields**

- [acmp\\_reference\\_voltage\\_source\\_t](#) **referenceVoltageSource**  
*Supply voltage reference source.*
- uint32\_t [DACValue](#)  
*Value for DAC Output Voltage.*

**Field Documentation****(1) acmp\_reference\_voltage\_source\_t \_acmp\_dac\_config::referenceVoltageSource****(2) uint32\_t \_acmp\_dac\_config::DACValue**

Available range is 0-255.

**10.3.5 struct \_acmp\_round\_robin\_config****Data Fields**

- [acmp\\_fixed\\_port\\_t](#) **fixedPort**  
*Fixed mux port.*
- uint32\_t [fixedChannelNumber](#)  
*Indicates which channel is fixed in the fixed mux port.*
- uint32\_t [checkerChannelMask](#)  
*Mask of checker channel index.*
- uint32\_t [sampleClockCount](#)  
*Specifies how many round-robin clock cycles(0~3) later the sample takes place.*
- uint32\_t [delayModulus](#)  
*Comparator and DAC initialization delay modulus.*

**Field Documentation****(1) acmp\_fixed\_port\_t \_acmp\_round\_robin\_config::fixedPort****(2) uint32\_t \_acmp\_round\_robin\_config::fixedChannelNumber****(3) uint32\_t \_acmp\_round\_robin\_config::checkerChannelMask**

Available range is channel0:0x01 to channel7:0x80 for round-robin checker.

(4) `uint32_t _acmp_round_robin_config::sampleClockCount`

(5) `uint32_t _acmp_round_robin_config::delayModulus`

### 10.3.6 struct \_acmp\_discrete\_mode\_config

#### Data Fields

- bool `enablePositiveChannelDiscreteMode`  
*Positive Channel Continuous Mode Enable.*
- bool `enableNegativeChannelDiscreteMode`  
*Negative Channel Continuous Mode Enable.*

#### Field Documentation

(1) `bool _acmp_discrete_mode_config::enablePositiveChannelDiscreteMode`

By default, the continuous mode is used.

(2) `bool _acmp_discrete_mode_config::enableNegativeChannelDiscreteMode`

By default, the continuous mode is used.

## 10.4 Macro Definition Documentation

10.4.1 `#define FSL_ACMP_DRIVER_VERSION (MAKE_VERSION(2, 3, 0))`

10.4.2 `#define CMP_C0_CFx_MASK (CMP_C0_CFR_MASK | CMP_C0_CFF_MASK)`

## 10.5 Typedef Documentation

10.5.1 `typedef enum _acmp_hysteresis_mode acmp_hysteresis_mode_t`

See chip data sheet to get the actual hysteresis value with each level.

**10.5.2** `typedef enum _acmp_reference_voltage_source acmp_reference_voltage_source_t`

**10.5.3** `typedef enum _acmp_fixed_port acmp_fixed_port_t`

**10.5.4** `typedef enum _acmp_dac_work_mode acmp_dac_work_mode_t`

**10.5.5** `typedef struct _acmp_config acmp_config_t`

**10.5.6** `typedef struct _acmp_channel_config acmp_channel_config_t`

The comparator's port can be input from channel mux or DAC. If port input is from channel mux, detailed channel number for the mux should be configured.

**10.5.7** `typedef struct _acmp_filter_config acmp_filter_config_t`

**10.5.8** `typedef struct _acmp_dac_config acmp_dac_config_t`

**10.5.9** `typedef struct _acmp_round_robin_config acmp_round_robin_config_t`

**10.5.10** `typedef struct _acmp_discrete_mode_config acmp_discrete_mode_config_t`

## 10.6 Enumeration Type Documentation

**10.6.1** `enum _acmp_interrupt_enable`

Enumerator

*kACMP\_OutputRisingInterruptEnable* Enable the interrupt when comparator outputs rising.  
*kACMP\_OutputFallingInterruptEnable* Enable the interrupt when comparator outputs falling.  
*kACMP\_RoundRobinInterruptEnable* Enable the Round-Robin interrupt.

**10.6.2** `enum _acmp_status_flags`

Enumerator

*kACMP\_OutputRisingEventFlag* Rising-edge on compare output has occurred.  
*kACMP\_OutputFallingEventFlag* Falling-edge on compare output has occurred.  
*kACMP\_OutputAssertEventFlag* Return the current value of the analog comparator output.

### 10.6.3 enum \_acmp\_hysteresis\_mode

See chip data sheet to get the actual hysteresis value with each level.

Enumerator

- kACMP\_HysteresisLevel0* Offset is level 0 and Hysteresis is level 0.
- kACMP\_HysteresisLevel1* Offset is level 0 and Hysteresis is level 1.
- kACMP\_HysteresisLevel2* Offset is level 0 and Hysteresis is level 2.
- kACMP\_HysteresisLevel3* Offset is level 0 and Hysteresis is level 3.

### 10.6.4 enum \_acmp\_reference\_voltage\_source

Enumerator

- kACMP\_VrefSourceVin1* Vin1 is selected as resistor ladder network supply reference Vin.
- kACMP\_VrefSourceVin2* Vin2 is selected as resistor ladder network supply reference Vin.

### 10.6.5 enum \_acmp\_fixed\_port

Enumerator

- kACMP\_FixedPlusPort* Only the inputs to the Minus port are swept in each round.
- kACMP\_FixedMinusPort* Only the inputs to the Plus port are swept in each round.

### 10.6.6 enum \_acmp\_dac\_work\_mode

Enumerator

- kACMP\_DACWorkLowSpeedMode* DAC is selected to work in low speed and low power mode.
- kACMP\_DACWorkHighSpeedMode* DAC is selected to work in high speed high power mode.

## 10.7 Function Documentation

### 10.7.1 void ACMP\_Init ( CMP\_Type \* *base*, const acmp\_config\_t \* *config* )

The default configuration can be got by calling [ACMP\\_GetDefaultConfig\(\)](#).

## Parameters

|               |                                          |
|---------------|------------------------------------------|
| <i>base</i>   | ACMP peripheral base address.            |
| <i>config</i> | Pointer to ACMP configuration structure. |

**10.7.2 void ACMP\_Deinit ( CMP\_Type \* *base* )**

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | ACMP peripheral base address. |
|-------------|-------------------------------|

**10.7.3 void ACMP\_GetDefaultConfig ( acmp\_config\_t \* *config* )**

This function initializes the user configuration structure to default value. The default value are:

Example:

```
config->enableHighSpeed = false;
config->enableInvertOutput = false;
config->useUnfilteredOutput = false;
config->enablePinOut = false;
config->enableHysteresisBothDirections = false;
config->hysteresisMode = kACMP_hysteresisMode0;
```

## Parameters

|               |                                          |
|---------------|------------------------------------------|
| <i>config</i> | Pointer to ACMP configuration structure. |
|---------------|------------------------------------------|

**10.7.4 void ACMP\_Enable ( CMP\_Type \* *base*, bool *enable* )**

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>base</i>   | ACMP peripheral base address. |
| <i>enable</i> | True to enable the ACMP.      |

**10.7.5 void ACMP\_EnableLinkToDAC ( CMP\_Type \* *base*, bool *enable* )**

When this bit is set, the DAC enable/disable is controlled by the bit CMP\_C0[EN] instead of CMP\_C1[D-ACEN].



## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>base</i>   | ACMP peripheral base address. |
| <i>enable</i> | Enable the feature or not.    |

### 10.7.6 void ACMP\_SetChannelConfig ( CMP\_Type \* *base*, const acmp\_channel\_config\_t \* *config* )

Note that the plus/minus mux's setting is only valid when the positive/negative port's input isn't from DAC but from channel mux.

Example:

```
acmp_channel_config_t configStruct = {0};
configStruct.positivePortInput = kACMP_PortInputFromDAC;
configStruct.negativePortInput = kACMP_PortInputFromMux;
configStruct.minusMuxInput = 1U;
ACMP_SetChannelConfig(CMP0, &configStruct);
```

## Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>base</i>   | ACMP peripheral base address.               |
| <i>config</i> | Pointer to channel configuration structure. |

### 10.7.7 void ACMP\_EnableDMA ( CMP\_Type \* *base*, bool *enable* )

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>base</i>   | ACMP peripheral base address. |
| <i>enable</i> | True to enable DMA.           |

### 10.7.8 void ACMP\_EnableWindowMode ( CMP\_Type \* *base*, bool *enable* )

## Parameters

---

|               |                               |
|---------------|-------------------------------|
| <i>base</i>   | ACMP peripheral base address. |
| <i>enable</i> | True to enable window mode.   |

### 10.7.9 void ACMP\_SetFilterConfig ( CMP\_Type \* *base*, const acmp\_filter\_config\_t \* *config* )

The filter can be enabled when the filter count is bigger than 1, the filter period is greater than 0 and the sample clock is from divided bus clock or the filter is bigger than 1 and the sample clock is from external clock. Detailed usage can be got from the reference manual.

Example:

```
acmp_filter_config_t configStruct = {0};
configStruct.filterCount = 5U;
configStruct.filterPeriod = 200U;
configStruct.enableSample = false;
ACMP_SetFilterConfig(CMP0, &configStruct);
```

Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | ACMP peripheral base address.              |
| <i>config</i> | Pointer to filter configuration structure. |

### 10.7.10 void ACMP\_SetDACConfig ( CMP\_Type \* *base*, const acmp\_dac\_config\_t \* *config* )

Example:

```
acmp_dac_config_t configStruct = {0};
configStruct.referenceVoltageSource = kACMP_VrefSourceVin1;
configStruct.DACValue = 20U;
configStruct.enableOutput = false;
configStruct.workMode = kACMP_DACWorkLowSpeedMode;
ACMP_SetDACConfig(CMP0, &configStruct);
```

Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | ACMP peripheral base address. |
|-------------|-------------------------------|

|               |                                                                              |
|---------------|------------------------------------------------------------------------------|
| <i>config</i> | Pointer to DAC configuration structure. "NULL" is for disabling the feature. |
|---------------|------------------------------------------------------------------------------|

### 10.7.11 void ACMP\_SetRoundRobinConfig ( CMP\_Type \* *base*, const *acmp\_round\_robin\_config\_t* \* *config* )

Example:

```
acmp_round_robin_config_t configStruct = {0};
configStruct.fixedPort = kACMP_FixedPlusPort;
configStruct.fixedChannelNumber = 3U;
configStruct.checkerChannelMask = 0xF7U;
configStruct.sampleClockCount = 0U;
configStruct.delayModulus = 0U;
ACMP_SetRoundRobinConfig(CMP0, &configStruct);
```

Parameters

|               |                                                                                           |
|---------------|-------------------------------------------------------------------------------------------|
| <i>base</i>   | ACMP peripheral base address.                                                             |
| <i>config</i> | Pointer to round robin mode configuration structure. "NULL" is for disabling the feature. |

### 10.7.12 void ACMP\_SetRoundRobinPreState ( CMP\_Type \* *base*, uint32\_t *mask* )

Note: The pre-state has different circuit with get-round-robin-result in the SOC even though they are same bits. So get-round-robin-result can't return the same value as the value are set by pre-state.

Parameters

|             |                                                                                       |
|-------------|---------------------------------------------------------------------------------------|
| <i>base</i> | ACMP peripheral base address.                                                         |
| <i>mask</i> | Mask of round robin channel index. Available range is channel0:0x01 to channel7:0x80. |

### 10.7.13 static uint32\_t ACMP\_GetRoundRobinStatusFlags ( CMP\_Type \* *base* ) [inline], [static]

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | ACMP peripheral base address. |
|-------------|-------------------------------|

## Returns

Mask of channel input changed asserted flags. Available range is channel0:0x01 to channel17:0x80.

#### 10.7.14 void ACMP\_ClearRoundRobinStatusFlags ( CMP\_Type \* *base*, uint32\_t *mask* )

## Parameters

|             |                                                                            |
|-------------|----------------------------------------------------------------------------|
| <i>base</i> | ACMP peripheral base address.                                              |
| <i>mask</i> | Mask of channel index. Available range is channel0:0x01 to channel17:0x80. |

#### 10.7.15 static uint32\_t ACMP\_GetRoundRobinResult ( CMP\_Type \* *base* ) [inline], [static]

Note that the set-pre-state has different circuit with get-round-robin-result in the SOC even though they are same bits. So [ACMP\\_GetRoundRobinResult\(\)](#) can't return the same value as the value are set by [ACMP\\_SetRoundRobinPreState](#).

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | ACMP peripheral base address. |
|-------------|-------------------------------|

## Returns

Mask of round robin channel result. Available range is channel0:0x01 to channel17:0x80.

#### 10.7.16 void ACMP\_EnableInterrupts ( CMP\_Type \* *base*, uint32\_t *mask* )

## Parameters

---

|             |                                                |
|-------------|------------------------------------------------|
| <i>base</i> | ACMP peripheral base address.                  |
| <i>mask</i> | Interrupts mask. See "_acmp_interrupt_enable". |

### 10.7.17 void ACMP\_DisableInterrupts ( CMP\_Type \* *base*, uint32\_t *mask* )

Parameters

|             |                                                |
|-------------|------------------------------------------------|
| <i>base</i> | ACMP peripheral base address.                  |
| <i>mask</i> | Interrupts mask. See "_acmp_interrupt_enable". |

### 10.7.18 uint32\_t ACMP\_GetStatusFlags ( CMP\_Type \* *base* )

Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | ACMP peripheral base address. |
|-------------|-------------------------------|

Returns

Status flags asserted mask. See "\_acmp\_status\_flags".

### 10.7.19 void ACMP\_ClearStatusFlags ( CMP\_Type \* *base*, uint32\_t *mask* )

Parameters

|             |                                              |
|-------------|----------------------------------------------|
| <i>base</i> | ACMP peripheral base address.                |
| <i>mask</i> | Status flags mask. See "_acmp_status_flags". |

### 10.7.20 void ACMP\_SetDiscreteModeConfig ( CMP\_Type \* *base*, const acmp\_discrete\_mode\_config\_t \* *config* )

Configure the discrete mode when supporting 3V domain with 1.8V core.

## Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | ACMP peripheral base address.                                          |
| <i>config</i> | Pointer to configuration structure. See "acmp_discrete_mode_config_t". |

### 10.7.21 void ACMP\_GetDefaultDiscreteModeConfig ( acmp\_discrete\_mode\_config\_t \* *config* )

## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>config</i> | Pointer to configuration structure to be restored with the setting values. |
|---------------|----------------------------------------------------------------------------|

# Chapter 11

## CACHE: LMEM CACHE Memory Controller

### 11.1 Overview

The MCUXpresso SDK provides a peripheral driver for the CACHE Controller of MCUXpresso SDK devices.

The CACHE driver is created to help the user more easily operate the cache memory. The APIs for basic operations are including the following three levels: 1L. The L1 cache driver API. This level provides the level 1 caches controller drivers. The L1 caches in this arch is the previous the local memory controller (LMEM).

2L. The unified cache driver API. This level provides many APIs for unified cache driver APIs for combined L1 and L2 cache maintain operations. This is provided for SDK drivers (DMA, ENET, US-DHC, etc) which should do the cache maintenance in their transactional APIs. Because in this arch, there is no L2 cache so the unified cache driver API directly calls only L1 driver APIs.

### 11.2 Function groups

#### 11.2.1 L1 CACHE Operation

The L1 CACHE has both code cache and data cache. This function group provides two independent API groups for both code cache and data cache. There are Enable/Disable APIs for code cache and data cache control and cache maintenance operations as Invalidate/Clean/CleanInvalidate by all and by address range.

### Macros

- #define [L1CODEBUSCACHE\\_LINESIZE\\_BYTE](#) FSL\_FEATURE\_L1ICACHE\_LINESIZE\_BYTE  
*code bus cache line size is equal to system bus line size, so the unified I/D cache line size equals too.*
- #define [L1SYSTEMBUSCACHE\\_LINESIZE\\_BYTE](#) [L1CODEBUSCACHE\\_LINESIZE\\_BYTE](#)  
*The system bus CACHE line size is 16B = 128b.*

### Driver version

- #define [FSL\\_CACHE\\_DRIVER\\_VERSION](#) (MAKE\_VERSION(2, 0, 6))  
*cache driver version.*

### Unified Cache Control for all caches

- static void [ICACHE\\_InvalidateByRange](#) (uint32\_t address, uint32\_t size\_byte)  
*Invalidates instruction cache by range.*
- static void [DCACHE\\_InvalidateByRange](#) (uint32\_t address, uint32\_t size\_byte)  
*Invalidates data cache by range.*

- static void [DCACHE\\_CleanByRange](#) (uint32\_t address, uint32\_t size\_byte)  
*Clean data cache by range.*
- static void [DCACHE\\_CleanInvalidateByRange](#) (uint32\_t address, uint32\_t size\_byte)  
*Cleans and Invalidates data cache by range.*

## 11.3 Macro Definition Documentation

**11.3.1 #define FSL\_CACHE\_DRIVER\_VERSION (MAKE\_VERSION(2, 0, 6))**

**11.3.2 #define L1CODEBUSCACHE\_LINESIZE\_BYTE FSL\_FEATURE\_L1ICACHE\_LINESIZE\_BYTE**

The code bus CACHE line size is 16B = 128b.

**11.3.3 #define L1SYSTEMBUSCACHE\_LINESIZE\_BYTE L1CODEBUSCACHE\_LINESIZE\_BYTE**

## 11.4 Function Documentation

**11.4.1 static void ICACHE\_InvalidateByRange ( uint32\_t address, uint32\_t size\_byte ) [inline], [static]**

Parameters

|                  |                                       |
|------------------|---------------------------------------|
| <i>address</i>   | The physical address.                 |
| <i>size_byte</i> | size of the memory to be invalidated. |

Note

Address and size should be aligned to 16-Byte due to the cache operation unit FSL\_FEATURE\_L1ICACHE\_LINESIZE\_BYTE. The startAddr here will be forced to align to the cache line size if startAddr is not aligned. For the size\_byte, application should make sure the alignment or make sure the right operation order if the size\_byte is not aligned.

**11.4.2 static void DCACHE\_InvalidateByRange ( uint32\_t address, uint32\_t size\_byte ) [inline], [static]**



## Parameters

|                  |                                       |
|------------------|---------------------------------------|
| <i>address</i>   | The physical address.                 |
| <i>size_byte</i> | size of the memory to be invalidated. |

## Note

Address and size should be aligned to 16-Byte due to the cache operation unit FSL\_FEATURE\_L1DCACHE\_LINESIZE\_BYTE. The startAddr here will be forced to align to the cache line size if startAddr is not aligned. For the size\_byte, application should make sure the alignment or make sure the right operation order if the size\_byte is not aligned.

### 11.4.3 static void DCACHE\_CleanByRange ( uint32\_t address, uint32\_t size\_byte ) [inline], [static]

## Parameters

|                  |                                   |
|------------------|-----------------------------------|
| <i>address</i>   | The physical address.             |
| <i>size_byte</i> | size of the memory to be cleaned. |

## Note

Address and size should be aligned to 16-Byte due to the cache operation unit FSL\_FEATURE\_L1DCACHE\_LINESIZE\_BYTE. The startAddr here will be forced to align to the cache line size if startAddr is not aligned. For the size\_byte, application should make sure the alignment or make sure the right operation order if the size\_byte is not aligned.

### 11.4.4 static void DCACHE\_CleanInvalidateByRange ( uint32\_t address, uint32\_t size\_byte ) [inline], [static]

## Parameters

|                  |                                                   |
|------------------|---------------------------------------------------|
| <i>address</i>   | The physical address.                             |
| <i>size_byte</i> | size of the memory to be Cleaned and Invalidated. |

## Note

Address and size should be aligned to 16-Byte due to the cache operation unit FSL\_FEATURE\_L1DCACHE\_LINESIZE\_BYTE. The startAddr here will be forced to align to the cache line size if startAddr is not aligned. For the size\_byte, application should make sure the alignment or make sure the right operation order if the size\_byte is not aligned.

# Chapter 12

## Common Driver

### 12.1 Overview

The MCUXpresso SDK provides a driver for the common module of MCUXpresso SDK devices.

#### Macros

- #define `FSL_DRIVER_TRANSFER_DOUBLE_WEAK_IRQ` 1  
*Macro to use the default weak IRQ handler in drivers.*
- #define `MAKE_STATUS`(group, code) (((group)\*100L) + (code))  
*Construct a status code value from a group and code number.*
- #define `MAKE_VERSION`(major, minor, bugfix) (((major)\*65536L) + ((minor)\*256L) + (bugfix))  
*Construct the version number for drivers.*
- #define `ARRAY_SIZE`(x) (sizeof(x) / sizeof((x)[0]))  
*Computes the number of elements in an array.*
- #define `UINT64_H`(X) (((uint32\_t)((uint64\_t) (X)) >> 32U) & 0xFFFFFFFFFULL)  
*Macro to get upper 32 bits of a 64-bit value.*
- #define `UINT64_L`(X) ((uint32\_t)((uint64\_t) (X)) & 0xFFFFFFFFFULL)  
*Macro to get lower 32 bits of a 64-bit value.*
- #define `SUPPRESS_FALL_THROUGH_WARNING`()  
*For switch case code block, if case section ends without "break;" statement, there will be fallthrough warning with compiler flag -Wextra or -Wimplicit-fallthrough=n when using armgcc.*

#### Typedefs

- typedef int32\_t `status_t`  
*Type used for all status and error return values.*

## Enumerations

- enum `_status_groups` {
  - `kStatusGroup_Generic` = 0,
  - `kStatusGroup_FLASH` = 1,
  - `kStatusGroup_LPSPI` = 4,
  - `kStatusGroup_FLEXIO_SPI` = 5,
  - `kStatusGroup_DSPI` = 6,
  - `kStatusGroup_FLEXIO_UART` = 7,
  - `kStatusGroup_FLEXIO_I2C` = 8,
  - `kStatusGroup_LPI2C` = 9,
  - `kStatusGroup_UART` = 10,
  - `kStatusGroup_I2C` = 11,
  - `kStatusGroup_LPSCI` = 12,
  - `kStatusGroup_LPUART` = 13,
  - `kStatusGroup_SPI` = 14,
  - `kStatusGroup_XRDC` = 15,
  - `kStatusGroup_SEMA42` = 16,
  - `kStatusGroup_SDHC` = 17,
  - `kStatusGroup_SDMMC` = 18,
  - `kStatusGroup_SAI` = 19,
  - `kStatusGroup_MCG` = 20,
  - `kStatusGroup_SCG` = 21,
  - `kStatusGroup_SDSPI` = 22,
  - `kStatusGroup_FLEXIO_I2S` = 23,
  - `kStatusGroup_FLEXIO_MCULCD` = 24,
  - `kStatusGroup_FLASHIAP` = 25,
  - `kStatusGroup_FLEXCOMM_I2C` = 26,
  - `kStatusGroup_I2S` = 27,
  - `kStatusGroup_IUART` = 28,
  - `kStatusGroup_CSI` = 29,
  - `kStatusGroup_MIPI_DSI` = 30,
  - `kStatusGroup_SDRAMC` = 35,
  - `kStatusGroup_POWER` = 39,
  - `kStatusGroup_ENET` = 40,
  - `kStatusGroup_PHY` = 41,
  - `kStatusGroup_TRGMUX` = 42,
  - `kStatusGroup_SMARTCARD` = 43,
  - `kStatusGroup_LMEM` = 44,
  - `kStatusGroup_QSPI` = 45,
  - `kStatusGroup_DMA` = 50,
  - `kStatusGroup_EDMA` = 51,
  - `kStatusGroup_DMAMGR` = 52,
  - `kStatusGroup_FLEXCAN` = 53,
  - `kStatusGroup_LTC` = 54,
  - `kStatusGroup_FLEXIO_CAMERA` = 55,
  - `kStatusGroup_LPC_SPI` = 56,
  - `kStatusGroup_LPC_USMCI` = 57,
  - `kStatusGroup_DMIC` = 58,
  - `kStatusGroup_SDIF` = 59,

```
kStatusGroup_GLIKEY = 168 }
```

*Status group numbers.*

- enum {
 

```
kStatus_Success = MAKE_STATUS(kStatusGroup_Generic, 0),
kStatus_Fail = MAKE_STATUS(kStatusGroup_Generic, 1),
kStatus_ReadOnly = MAKE_STATUS(kStatusGroup_Generic, 2),
kStatus_OutOfRange = MAKE_STATUS(kStatusGroup_Generic, 3),
kStatus_InvalidArgument = MAKE_STATUS(kStatusGroup_Generic, 4),
kStatus_Timeout = MAKE_STATUS(kStatusGroup_Generic, 5),
kStatus_NoTransferInProgress,
kStatus_Busy = MAKE_STATUS(kStatusGroup_Generic, 7),
kStatus_NoData }
```

*Generic status return codes.*

## Functions

- void \* [SDK\\_Malloc](#) (size\_t size, size\_t alignbytes)  
*Allocate memory with given alignment and aligned size.*
- void [SDK\\_Free](#) (void \*ptr)  
*Free memory.*
- void [SDK\\_DelayAtLeastUs](#) (uint32\_t delayTime\_us, uint32\_t coreClock\_Hz)  
*Delay at least for some time.*

## Driver version

- #define [FSL\\_COMMON\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 4, 1))  
*common driver version.*

## Debug console type definition.

- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_NONE](#) 0U  
*No debug console.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_UART](#) 1U  
*Debug console based on UART.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_LPUART](#) 2U  
*Debug console based on LPUART.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_LPSCI](#) 3U  
*Debug console based on LPSCI.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_USBCDC](#) 4U  
*Debug console based on USBCDC.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_FLEXCOMM](#) 5U  
*Debug console based on FLEXCOMM.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_IUART](#) 6U  
*Debug console based on i.MX UART.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_VUSART](#) 7U  
*Debug console based on LPC\_VUSART.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_MINI\\_USART](#) 8U  
*Debug console based on LPC\_USART.*
- #define [DEBUG\\_CONSOLE\\_DEVICE\\_TYPE\\_SWO](#) 9U

- *Debug console based on SWO.*  
• #define **DEBUG\_CONSOLE\_DEVICE\_TYPE\_QSCI** 10U  
*Debug console based on QSCI.*

## Min/max macros

- #define **MIN**(a, b) (((a) < (b)) ? (a) : (b))  
*Computes the minimum of a and b.*
- #define **MAX**(a, b) (((a) > (b)) ? (a) : (b))  
*Computes the maximum of a and b.*

## UINT16\_MAX/UINT32\_MAX value

- #define **UINT16\_MAX** ((uint16\_t)-1)  
*Max value of uint16\_t type.*
- #define **UINT32\_MAX** ((uint32\_t)-1)  
*Max value of uint32\_t type.*

## 12.2 Macro Definition Documentation

### 12.2.1 #define FSL\_DRIVER\_TRANSFER\_DOUBLE\_WEAK\_IRQ 1

### 12.2.2 #define MAKE\_STATUS( group, code ) (((group)\*100L) + (code))

### 12.2.3 #define MAKE\_VERSION( major, minor, bugfix ) (((major)\*65536L) + ((minor)\*256L) + (bugfix))

The driver version is a 32-bit number, for both 32-bit platforms(such as Cortex M) and 16-bit platforms(such as DSC).

|        |               |    |               |    |         |   |   |
|--------|---------------|----|---------------|----|---------|---|---|
| Unused | Major Version |    | Minor Version |    | Bug Fix |   |   |
| 31     | 25            | 24 | 17            | 16 | 9       | 8 | 0 |

**12.2.4 #define FSL\_COMMON\_DRIVER\_VERSION (MAKE\_VERSION(2, 4, 1))**

**12.2.5 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_NONE 0U**

**12.2.6 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_UART 1U**

**12.2.7 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_LPUART 2U**

**12.2.8 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_LPSCI 3U**

**12.2.9 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_USBCDC 4U**

**12.2.10 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_FLEXCOMM 5U**

**12.2.11 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_IUART 6U**

**12.2.12 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_VUSART 7U**

**12.2.13 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_MINI\_USART 8U**

**12.2.14 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_SWO 9U**

**12.2.15 #define DEBUG\_CONSOLE\_DEVICE\_TYPE\_QSCI 10U**

**12.2.16 #define MIN( a, b ) (((a) < (b)) ? (a) : (b))**

**12.2.17 #define MAX( a, b ) (((a) > (b)) ? (a) : (b))**

**12.2.18 #define ARRAY\_SIZE( x ) (sizeof(x) / sizeof((x)[0]))**

**12.2.19 #define UINT16\_MAX ((uint16\_t)-1)**

**12.2.20 #define UINT32\_MAX ((uint32\_t)-1)**

**12.2.21 #define SUPPRESS\_FALL\_THROUGH\_WARNING( )**

To suppress this warning, "SUPPRESS\_FALL\_THROUGH\_WARNING();" need to be added at the end of each case section which misses "break;" statement.

## 12.3 Typedef Documentation

### 12.3.1 typedef int32\_t status\_t

## 12.4 Enumeration Type Documentation

### 12.4.1 enum \_status\_groups

Enumerator

*kStatusGroup\_Generic* Group number for generic status codes.  
*kStatusGroup\_FLASH* Group number for FLASH status codes.  
*kStatusGroup\_LPSPI* Group number for LPSPI status codes.  
*kStatusGroup\_FLEXIO\_SPI* Group number for FLEXIO SPI status codes.  
*kStatusGroup\_DSPI* Group number for DSPI status codes.  
*kStatusGroup\_FLEXIO\_UART* Group number for FLEXIO UART status codes.  
*kStatusGroup\_FLEXIO\_I2C* Group number for FLEXIO I2C status codes.  
*kStatusGroup\_LPI2C* Group number for LPI2C status codes.  
*kStatusGroup\_UART* Group number for UART status codes.  
*kStatusGroup\_I2C* Group number for UART status codes.  
*kStatusGroup\_LPSCI* Group number for LPSCI status codes.  
*kStatusGroup\_LPUART* Group number for LPUART status codes.  
*kStatusGroup\_SPI* Group number for SPI status code.  
*kStatusGroup\_XRDC* Group number for XRDC status code.  
*kStatusGroup\_SEMA42* Group number for SEMA42 status code.  
*kStatusGroup\_SDHC* Group number for SDHC status code.  
*kStatusGroup\_SDMMC* Group number for SDMMC status code.  
*kStatusGroup\_SAI* Group number for SAI status code.  
*kStatusGroup\_MCG* Group number for MCG status codes.  
*kStatusGroup\_SCG* Group number for SCG status codes.  
*kStatusGroup\_SDSPI* Group number for SDSPI status codes.  
*kStatusGroup\_FLEXIO\_I2S* Group number for FLEXIO I2S status codes.  
*kStatusGroup\_FLEXIO\_MCULCD* Group number for FLEXIO LCD status codes.  
*kStatusGroup\_FLASHIAP* Group number for FLASHIAP status codes.  
*kStatusGroup\_FLEXCOMM\_I2C* Group number for FLEXCOMM I2C status codes.  
*kStatusGroup\_I2S* Group number for I2S status codes.  
*kStatusGroup\_IUART* Group number for IUART status codes.  
*kStatusGroup\_CSI* Group number for CSI status codes.  
*kStatusGroup\_MIPI\_DSI* Group number for MIPI DSI status codes.  
*kStatusGroup\_SDRAMC* Group number for SDRAMC status codes.  
*kStatusGroup\_POWER* Group number for POWER status codes.  
*kStatusGroup\_ENET* Group number for ENET status codes.  
*kStatusGroup\_PHY* Group number for PHY status codes.  
*kStatusGroup\_TRGMUX* Group number for TRGMUX status codes.  
*kStatusGroup\_SMARTCARD* Group number for SMARTCARD status codes.  
*kStatusGroup\_LMEM* Group number for LMEM status codes.

*kStatusGroup\_QSPI* Group number for QSPI status codes.  
*kStatusGroup\_DMA* Group number for DMA status codes.  
*kStatusGroup\_EDMA* Group number for EDMA status codes.  
*kStatusGroup\_DMAMGR* Group number for DMAMGR status codes.  
*kStatusGroup\_FLEXCAN* Group number for FlexCAN status codes.  
*kStatusGroup\_LTC* Group number for LTC status codes.  
*kStatusGroup\_FLEXIO\_CAMERA* Group number for FLEXIO CAMERA status codes.  
*kStatusGroup\_LPC\_SPI* Group number for LPC\_SPI status codes.  
*kStatusGroup\_LPC\_USART* Group number for LPC\_USART status codes.  
*kStatusGroup\_DMIC* Group number for DMIC status codes.  
*kStatusGroup\_SDIF* Group number for SDIF status codes.  
*kStatusGroup\_SPIFI* Group number for SPIFI status codes.  
*kStatusGroup\_OTP* Group number for OTP status codes.  
*kStatusGroup\_MCAN* Group number for MCAN status codes.  
*kStatusGroup\_CAAM* Group number for CAAM status codes.  
*kStatusGroup\_ECSPI* Group number for ECSPI status codes.  
*kStatusGroup\_USDHC* Group number for USDHC status codes.  
*kStatusGroup\_LPC\_I2C* Group number for LPC\_I2C status codes.  
*kStatusGroup\_DCP* Group number for DCP status codes.  
*kStatusGroup\_MSCAN* Group number for MSCAN status codes.  
*kStatusGroup\_ESAI* Group number for ESAI status codes.  
*kStatusGroup\_FLEXSPI* Group number for FLEXSPI status codes.  
*kStatusGroup\_MMDC* Group number for MMDC status codes.  
*kStatusGroup\_PDM* Group number for MIC status codes.  
*kStatusGroup\_SDMA* Group number for SDMA status codes.  
*kStatusGroup\_ICS* Group number for ICS status codes.  
*kStatusGroup\_SPDIF* Group number for SPDIF status codes.  
*kStatusGroup\_LPC\_MINISPI* Group number for LPC\_MINISPI status codes.  
*kStatusGroup\_HASHCRYPT* Group number for Hashcrypt status codes.  
*kStatusGroup\_LPC\_SPI\_SSP* Group number for LPC\_SPI\_SSP status codes.  
*kStatusGroup\_I3C* Group number for I3C status codes.  
*kStatusGroup\_LPC\_I2C\_1* Group number for LPC\_I2C\_1 status codes.  
*kStatusGroup\_NOTIFIER* Group number for NOTIFIER status codes.  
*kStatusGroup\_DebugConsole* Group number for debug console status codes.  
*kStatusGroup\_SEMC* Group number for SEMC status codes.  
*kStatusGroup\_ApplicationRangeStart* Starting number for application groups.  
*kStatusGroup\_IAP* Group number for IAP status codes.  
*kStatusGroup\_SFA* Group number for SFA status codes.  
*kStatusGroup\_SPC* Group number for SPC status codes.  
*kStatusGroup\_PUF* Group number for PUF status codes.  
*kStatusGroup\_TOUCH\_PANEL* Group number for touch panel status codes.  
*kStatusGroup\_VBAT* Group number for VBAT status codes.  
*kStatusGroup\_XSPI* Group number for XSPI status codes.  
*kStatusGroup\_PNGDEC* Group number for PNGDEC status codes.  
*kStatusGroup\_JPEGDEC* Group number for JPEGDEC status codes.



***kStatusGroup\_HAL\_GPIO*** Group number for HAL GPIO status codes.

***kStatusGroup\_HAL\_UART*** Group number for HAL UART status codes.

***kStatusGroup\_HAL\_TIMER*** Group number for HAL TIMER status codes.

***kStatusGroup\_HAL\_SPI*** Group number for HAL SPI status codes.

***kStatusGroup\_HAL\_I2C*** Group number for HAL I2C status codes.

***kStatusGroup\_HAL\_FLASH*** Group number for HAL FLASH status codes.

***kStatusGroup\_HAL\_PWM*** Group number for HAL PWM status codes.

***kStatusGroup\_HAL\_RNG*** Group number for HAL RNG status codes.

***kStatusGroup\_HAL\_I2S*** Group number for HAL I2S status codes.

***kStatusGroup\_HAL\_ADC\_SENSOR*** Group number for HAL ADC SENSOR status codes.

***kStatusGroup\_TIMERMANAGER*** Group number for TiMER MANAGER status codes.

***kStatusGroup\_SERIALMANAGER*** Group number for SERIAL MANAGER status codes.

***kStatusGroup\_LED*** Group number for LED status codes.

***kStatusGroup\_BUTTON*** Group number for BUTTON status codes.

***kStatusGroup\_EXTERN\_EEPROM*** Group number for EXTERN EEPROM status codes.

***kStatusGroup\_SHELL*** Group number for SHELL status codes.

***kStatusGroup\_MEM\_MANAGER*** Group number for MEM MANAGER status codes.

***kStatusGroup\_LIST*** Group number for List status codes.

***kStatusGroup\_OSA*** Group number for OSA status codes.

***kStatusGroup\_COMMON\_TASK*** Group number for Common task status codes.

***kStatusGroup\_MSG*** Group number for messaging status codes.

***kStatusGroup\_SDK\_OCOTP*** Group number for OCOTP status codes.

***kStatusGroup\_SDK\_FLEXSPINOR*** Group number for FLEXSPINOR status codes.

***kStatusGroup\_CODEC*** Group number for codec status codes.

***kStatusGroup\_ASRC*** Group number for codec status ASRC.

***kStatusGroup\_OTFAD*** Group number for codec status codes.

***kStatusGroup\_SDIOSLV*** Group number for SDIOSLV status codes.

***kStatusGroup\_MECC*** Group number for MECC status codes.

***kStatusGroup\_ENET\_QOS*** Group number for ENET\_QOS status codes.

***kStatusGroup\_LOG*** Group number for LOG status codes.

***kStatusGroup\_I3CBUS*** Group number for I3CBUS status codes.

***kStatusGroup\_QSCI*** Group number for QSCI status codes.

***kStatusGroup\_ELEMU*** Group number for ELEMU status codes.

***kStatusGroup\_QUEUEDSPI*** Group number for QSPI status codes.

***kStatusGroup\_POWER\_MANAGER*** Group number for POWER\_MANAGER status codes.

***kStatusGroup\_IPED*** Group number for IPED status codes.

***kStatusGroup\_ELS\_PKC*** Group number for ELS PKC status codes.

***kStatusGroup\_CSS\_PKC*** Group number for CSS PKC status codes.

***kStatusGroup\_HOSTIF*** Group number for HOSTIF status codes.

***kStatusGroup\_CLIF*** Group number for CLIF status codes.

***kStatusGroup\_BMA*** Group number for BMA status codes.

***kStatusGroup\_NETC*** Group number for NETC status codes.

***kStatusGroup\_ELE*** Group number for ELE status codes.

***kStatusGroup\_GLIKEY*** Group number for GLIKEY status codes.

## 12.4.2 anonymous enum

Enumerator

***kStatus\_Success*** Generic status for Success.

***kStatus\_Fail*** Generic status for Fail.

***kStatus\_ReadOnly*** Generic status for read only failure.

***kStatus\_OutOfRange*** Generic status for out of range access.

***kStatus\_InvalidArgument*** Generic status for invalid argument check.

***kStatus\_Timeout*** Generic status for timeout.

***kStatus\_NoTransferInProgress*** Generic status for no transfer in progress.

***kStatus\_Busy*** Generic status for module is busy.

***kStatus\_NoData*** Generic status for no data is found for the operation.

## 12.5 Function Documentation

### 12.5.1 void\* SDK\_Malloc ( size\_t size, size\_t alignbytes )

This is provided to support the dynamically allocated memory used in cache-able region.

Parameters

|                   |                                |
|-------------------|--------------------------------|
| <i>size</i>       | The length required to malloc. |
| <i>alignbytes</i> | The alignment size.            |

Return values

|            |                   |
|------------|-------------------|
| <i>The</i> | allocated memory. |
|------------|-------------------|

### 12.5.2 void SDK\_Free ( void \* ptr )

Parameters

|            |                           |
|------------|---------------------------|
| <i>ptr</i> | The memory to be release. |
|------------|---------------------------|

### 12.5.3 void SDK\_DelayAtLeastUs ( uint32\_t delayTime\_us, uint32\_t coreClock\_Hz )

Please note that, this API uses while loop for delay, different run-time environments make the time not precise, if precise delay count was needed, please implement a new delay function with hardware timer.

## Parameters

|                     |                                    |
|---------------------|------------------------------------|
| <i>delayTime_us</i> | Delay time in unit of microsecond. |
| <i>coreClock_Hz</i> | Core clock frequency with Hz.      |

## Chapter 13

# CRC: Cyclic Redundancy Check Driver

### 13.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Cyclic Redundancy Check (CRC) module of MCUXpresso SDK devices.

The cyclic redundancy check (CRC) module generates 16/32-bit CRC code for error detection. The CRC module also provides a programmable polynomial, seed, and other parameters required to implement a 16-bit or 32-bit CRC standard.

### 13.2 CRC Driver Initialization and Configuration

[CRC\\_Init\(\)](#) function enables the clock gate for the CRC module in the SIM module and fully (re-)configures the CRC module according to the configuration structure. The seed member of the configuration structure is the initial checksum for which new data can be added to. When starting a new checksum computation, the seed is set to the initial checksum per the CRC protocol specification. For continued checksum operation, the seed is set to the intermediate checksum value as obtained from previous calls to [CRC\\_Get16bitResult\(\)](#) or [CRC\\_Get32bitResult\(\)](#) function. After calling the [CRC\\_Init\(\)](#), one or multiple [CRC\\_WriteData\(\)](#) calls follow to update the checksum with data and [CRC\\_Get16bitResult\(\)](#) or [CRC\\_Get32bitResult\(\)](#) follow to read the result. The `crcResult` member of the configuration structure determines whether the [CRC\\_Get16bitResult\(\)](#) or [CRC\\_Get32bitResult\(\)](#) return value is a final checksum or an intermediate checksum. The [CRC\\_Init\(\)](#) function can be called as many times as required allowing for runtime changes of the CRC protocol.

[CRC\\_GetDefaultConfig\(\)](#) function can be used to set the module configuration structure with parameters for CRC-16/CCIT-FALSE protocol.

### 13.3 CRC Write Data

The [CRC\\_WriteData\(\)](#) function adds data to the CRC. Internally, it tries to use 32-bit reads and writes for all aligned data in the user buffer and 8-bit reads and writes for all unaligned data in the user buffer. This function can update the CRC with user-supplied data chunks of an arbitrary size, so one can update the CRC byte by byte or with all bytes at once. Prior to calling the CRC configuration function [CRC\\_Init\(\)](#) fully specifies the CRC module configuration for the [CRC\\_WriteData\(\)](#) call.

### 13.4 CRC Get Checksum

The [CRC\\_Get16bitResult\(\)](#) or [CRC\\_Get32bitResult\(\)](#) function reads the CRC module data register. Depending on the prior CRC module usage, the return value is either an intermediate checksum or the final checksum. For example, for 16-bit CRCs the following call sequences can be used.

[CRC\\_Init\(\)](#) / [CRC\\_WriteData\(\)](#) / [CRC\\_Get16bitResult\(\)](#) to get the final checksum.

[CRC\\_Init\(\)](#) / [CRC\\_WriteData\(\)](#) / ... / [CRC\\_WriteData\(\)](#) / [CRC\\_Get16bitResult\(\)](#) to get the final checksum.

[CRC\\_Init\(\)](#) / [CRC\\_WriteData\(\)](#) / [CRC\\_Get16bitResult\(\)](#) to get an intermediate checksum.

[CRC\\_Init\(\)](#) / [CRC\\_WriteData\(\)](#) / ... / [CRC\\_WriteData\(\)](#) / [CRC\\_Get16bitResult\(\)](#) to get an intermediate checksum.

### 13.5 Comments about API usage in RTOS

If multiple RTOS tasks share the CRC module to compute checksums with different data and/or protocols, the following needs to be implemented by the user.

The triplets

[CRC\\_Init\(\)](#) / [CRC\\_WriteData\(\)](#) / [CRC\\_Get16bitResult\(\)](#) or [CRC\\_Get32bitResult\(\)](#)

The triplets are protected by the RTOS mutex to protect the CRC module against concurrent accesses from different tasks. This is an example. Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/crcRefer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/crcRefer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/crcRefer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/crcRefer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/crc

### Data Structures

- struct [\\_crc\\_config](#)  
CRC protocol configuration. [More...](#)

### Macros

- #define [CRC\\_DRIVER\\_USE\\_CRC16\\_CCIT\\_FALSE\\_AS\\_DEFAULT](#) 1  
Default configuration structure filled by [CRC\\_GetDefaultConfig\(\)](#).

### Typedefs

- typedef enum [\\_crc\\_bits](#) [crc\\_bits\\_t](#)  
CRC bit width.
- typedef enum [\\_crc\\_result](#) [crc\\_result\\_t](#)  
CRC result type.
- typedef struct [\\_crc\\_config](#) [crc\\_config\\_t](#)  
CRC protocol configuration.

### Enumerations

- enum [\\_crc\\_bits](#) {  
    [kCrcBits16](#) = 0U,  
    [kCrcBits32](#) = 1U }  
CRC bit width.
- enum [\\_crc\\_result](#) {  
    [kCrcFinalChecksum](#) = 0U,  
    [kCrcIntermediateChecksum](#) = 1U }  
CRC result type.

## Functions

- void [CRC\\_Init](#) (CRC\_Type \*base, const [crc\\_config\\_t](#) \*config)  
*Enables and configures the CRC peripheral module.*
- static void [CRC\\_Deinit](#) (CRC\_Type \*base)  
*Disables the CRC peripheral module.*
- void [CRC\\_GetDefaultConfig](#) ([crc\\_config\\_t](#) \*config)  
*Loads default values to the CRC protocol configuration structure.*
- void [CRC\\_WriteData](#) (CRC\_Type \*base, const uint8\_t \*data, size\_t dataSize)  
*Writes data to the CRC module.*
- uint32\_t [CRC\\_Get32bitResult](#) (CRC\_Type \*base)  
*Reads the 32-bit checksum from the CRC module.*
- uint16\_t [CRC\\_Get16bitResult](#) (CRC\_Type \*base)  
*Reads a 16-bit checksum from the CRC module.*

## Driver version

- #define [FSL\\_CRC\\_DRIVER\\_VERSION](#) (MAKE\_VERSION(2, 0, 4))  
*CRC driver version.*

## 13.6 Data Structure Documentation

### 13.6.1 struct \_crc\_config

This structure holds the configuration for the CRC protocol.

## Data Fields

- uint32\_t [polynomial](#)  
*CRC Polynomial, MSBit first.*
- uint32\_t [seed](#)  
*Starting checksum value.*
- bool [reflectIn](#)  
*Reflect bits on input.*
- bool [reflectOut](#)  
*Reflect bits on output.*
- bool [complementChecksum](#)  
*True if the result shall be complement of the actual checksum.*
- [crc\\_bits\\_t](#) [crcBits](#)  
*Selects 16- or 32- bit CRC protocol.*
- [crc\\_result\\_t](#) [crcResult](#)  
*Selects final or intermediate checksum return from [CRC\\_Get16bitResult\(\)](#) or [CRC\\_Get32bitResult\(\)](#)*

## Field Documentation

### (1) uint32\_t \_crc\_config::polynomial

Example polynomial: 0x1021 = 1\_0000\_0010\_0001 =  $x^{12} + x^5 + 1$

- (2) `bool _crc_config::reflectIn`
- (3) `bool _crc_config::reflectOut`
- (4) `bool _crc_config::complementChecksum`
- (5) `crc_bits_t _crc_config::crcBits`

## 13.7 Macro Definition Documentation

### 13.7.1 `#define FSL_CRC_DRIVER_VERSION (MAKE_VERSION(2, 0, 4))`

Version 2.0.4.

Current version: 2.0.4

Change log:

- Version 2.0.4
  - Release peripheral from reset if necessary in init function.
- Version 2.0.3
  - Fix MISRA issues
- Version 2.0.2
  - Fix MISRA issues
- Version 2.0.1
  - move DATA and DATALL macro definition from header file to source file

### 13.7.2 `#define CRC_DRIVER_USE_CRC16_CCIT_FALSE_AS_DEFAULT 1`

Use CRC16-CCIT-FALSE as default.

## 13.8 Typedef Documentation

### 13.8.1 `typedef struct _crc_config crc_config_t`

This structure holds the configuration for the CRC protocol.

## 13.9 Enumeration Type Documentation

### 13.9.1 `enum _crc_bits`

Enumerator

***kCrcBits16*** Generate 16-bit CRC code.

***kCrcBits32*** Generate 32-bit CRC code.

### 13.9.2 enum \_crc\_result

Enumerator

***kCrcFinalChecksum*** CRC data register read value is the final checksum. Reflect out and final xor protocol features are applied.

***kCrcIntermediateChecksum*** CRC data register read value is intermediate checksum (raw value). Reflect out and final xor protocol feature are not applied. Intermediate checksum can be used as a seed for [CRC\\_Init\(\)](#) to continue adding data to this checksum.

## 13.10 Function Documentation

### 13.10.1 void CRC\_Init ( CRC\_Type \* *base*, const crc\_config\_t \* *config* )

This function enables the clock gate in the SIM module for the CRC peripheral. It also configures the CRC module and starts a checksum computation by writing the seed.

Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | CRC peripheral address.             |
| <i>config</i> | CRC module configuration structure. |

### 13.10.2 static void CRC\_Deinit ( CRC\_Type \* *base* ) [inline], [static]

This function disables the clock gate in the SIM module for the CRC peripheral.

Parameters

|             |                         |
|-------------|-------------------------|
| <i>base</i> | CRC peripheral address. |
|-------------|-------------------------|

### 13.10.3 void CRC\_GetDefaultConfig ( crc\_config\_t \* *config* )

Loads default values to the CRC protocol configuration structure. The default values are as follows.

```
* config->polynomial = 0x1021;
* config->seed = 0xFFFF;
* config->reflectIn = false;
* config->reflectOut = false;
* config->complementChecksum = false;
* config->crcBits = kCrcBits16;
* config->crcResult = kCrcFinalChecksum;
*
```



## Parameters

|               |                                       |
|---------------|---------------------------------------|
| <i>config</i> | CRC protocol configuration structure. |
|---------------|---------------------------------------|

### 13.10.4 void CRC\_WriteData ( CRC\_Type \* *base*, const uint8\_t \* *data*, size\_t *dataSize* )

Writes input data buffer bytes to the CRC data register. The configured type of transpose is applied.

## Parameters

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>base</i>     | CRC peripheral address.                 |
| <i>data</i>     | Input data stream, MSByte in data[0].   |
| <i>dataSize</i> | Size in bytes of the input data buffer. |

### 13.10.5 uint32\_t CRC\_Get32bitResult ( CRC\_Type \* *base* )

Reads the CRC data register (either an intermediate or the final checksum). The configured type of transpose and complement is applied.

## Parameters

|             |                         |
|-------------|-------------------------|
| <i>base</i> | CRC peripheral address. |
|-------------|-------------------------|

## Returns

An intermediate or the final 32-bit checksum, after configured transpose and complement operations.

### 13.10.6 uint16\_t CRC\_Get16bitResult ( CRC\_Type \* *base* )

Reads the CRC data register (either an intermediate or the final checksum). The configured type of transpose and complement is applied.

## Parameters

|             |                         |
|-------------|-------------------------|
| <i>base</i> | CRC peripheral address. |
|-------------|-------------------------|

#### Returns

An intermediate or the final 16-bit checksum, after configured transpose and complement operations.

## Chapter 14

# DAC12: 12-bit Digital-to-Analog Converter Driver

### 14.1 Overview

The MCUXpresso SDK provides a peripheral driver for the 12-bit Digital-to-Analog Converter (DAC12) module of MCUXpresso SDK devices.

This DAC is the 12-bit resolution digital-to-analog converters with programmable reference generator output. Its output data items are loaded into a FIFO, so that various FIFO mode can be used to output the value for user-defined sequence.

The DAC driver provides a user-friendly interface to operate the DAC peripheral. The user can initialize/deinitialize the DAC driver, set data into FIFO, or enable the interrupt DMA for special events so that the hardware can process the DAC output data automatically. Also, the configuration for software and hardware trigger are also included in the driver.

### 14.2 Typical use case

#### 14.2.1 A simple use case to output the user-defined DAC12 value.

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/dac12`

#### 14.2.2 Working with the trigger

Once more than one data is filled into the FIFO, the output pointer moves into configured mode when a trigger comes. This trigger can be from software or hardware, and moves one item for each trigger. Also, the interrupt/DMA event can be activated when the output pointer hits to the configured position.

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/dac12`

### Files

- file [fsl\\_dac12.h](#)

### Data Structures

- struct [\\_dac12\\_hardware\\_info](#)  
DAC12 hardware information. [More...](#)
- struct [dac12\\_config\\_t](#)  
DAC12 module configuration. [More...](#)

## Macros

- #define `DAC12_CR_W1C_FLAGS_MASK` (`DAC_CR_OVFF_MASK` | `DAC_CR_UDFF_MASK`)  
*Define "write 1 to clear" flags.*
- #define `DAC12_CR_ALL_FLAGS_MASK` (`DAC12_CR_W1C_FLAGS_MASK` | `DAC_CR_WMF_MASK` | `DAC_CR_NEMPTF_MASK` | `DAC_CR_FULLF_MASK`)  
*Define all the flag bits in DACx\_CR register.*

## Typedefs

- typedef enum `_dac12_fifo_size_info` `dac12_fifo_size_info_t`  
*DAC12 FIFO size information provided by hardware.*
- typedef enum `_dac12_fifo_work_mode` `dac12_fifo_work_mode_t`  
*DAC12 FIFO work mode.*
- typedef enum `_dac12_reference_voltage_source` `dac12_reference_voltage_source_t`  
*DAC12 reference voltage source.*
- typedef enum `_dac12_fifo_trigger_mode` `dac12_fifo_trigger_mode_t`  
*DAC12 FIFO trigger mode.*
- typedef enum `_dac12_reference_current_source` `dac12_reference_current_source_t`  
*DAC internal reference current source.*
- typedef enum `_dac12_speed_mode` `dac12_speed_mode_t`  
*DAC analog buffer speed mode for conversion.*
- typedef struct `_dac12_hardware_info` `dac12_hardware_info_t`  
*DAC12 hardware information.*

## Enumerations

- enum `_dac12_status_flags` {  
  `kDAC12_OverflowFlag` = `DAC_CR_OVFF_MASK`,  
  `kDAC12_UnderflowFlag` = `DAC_CR_UDFF_MASK`,  
  `kDAC12_WatermarkFlag` = `DAC_CR_WMF_MASK`,  
  `kDAC12_NearlyEmptyFlag` = `DAC_CR_NEMPTF_MASK`,  
  `kDAC12_FullFlag` = `DAC_CR_FULLF_MASK` }  
*DAC12 flags.*
- enum `_dac12_interrupt_enable` {  
  `kDAC12_UnderOrOverflowInterruptEnable` = `DAC_CR_UVIE_MASK`,  
  `kDAC12_WatermarkInterruptEnable` = `DAC_CR_WTMIE_MASK`,  
  `kDAC12_NearlyEmptyInterruptEnable` = `DAC_CR_EMPTYIE_MASK`,  
  `kDAC12_FullInterruptEnable` = `DAC_CR_FULLLIE_MASK` }  
*DAC12 interrupts.*
- enum `_dac12_fifo_size_info` {

```

kDAC12_FIFOSize2 = 0U,
kDAC12_FIFOSize4 = 1U,
kDAC12_FIFOSize8 = 2U,
kDAC12_FIFOSize16 = 3U,
kDAC12_FIFOSize32 = 4U,
kDAC12_FIFOSize64 = 5U,
kDAC12_FIFOSize128 = 6U,
kDAC12_FIFOSize256 = 7U }

```

*DAC12 FIFO size information provided by hardware.*

- enum `_dac12_fifo_work_mode` {  
`kDAC12_FIFODisabled` = 0U,  
`kDAC12_FIFOWorkAsNormalMode` = 1U,  
`kDAC12_FIFOWorkAsSwingMode` = 2U }

*DAC12 FIFO work mode.*

- enum `_dac12_reference_voltage_source` {  
`kDAC12_ReferenceVoltageSourceAlt1` = 0U,  
`kDAC12_ReferenceVoltageSourceAlt2` = 1U }

*DAC12 reference voltage source.*

- enum `_dac12_fifo_trigger_mode` {  
`kDAC12_FIFOTriggerByHardwareMode` = 0U,  
`kDAC12_FIFOTriggerBySoftwareMode` = 1U }

*DAC12 FIFO trigger mode.*

- enum `_dac12_reference_current_source` {  
`kDAC12_ReferenceCurrentSourceDisabled` = 0U,  
`kDAC12_ReferenceCurrentSourceAlt0` = 1U,  
`kDAC12_ReferenceCurrentSourceAlt1` = 2U,  
`kDAC12_ReferenceCurrentSourceAlt2` = 3U }

*DAC internal reference current source.*

- enum `_dac12_speed_mode` {  
`kDAC12_SpeedLowMode` = 0U,  
`kDAC12_SpeedMiddleMode` = 1U,  
`kDAC12_SpeedHighMode` = 2U }

*DAC analog buffer speed mode for conversion.*

## Driver version

- #define `FSL_DAC12_DRIVER_VERSION` (`MAKE_VERSION(2, 1, 1)`)  
*DAC12 driver version 2.1.1.*

## Initialization and de-initialization

- void `DAC12_GetHardwareInfo` (`DAC_Type *base`, `dac12_hardware_info_t *info`)  
*Get hardware information about this module.*
- void `DAC12_Init` (`DAC_Type *base`, const `dac12_config_t *config`)  
*Initialize the DAC12 module.*
- void `DAC12_GetDefaultConfig` (`dac12_config_t *config`)  
*Initializes the DAC12 user configuration structure.*
- void `DAC12_Deinit` (`DAC_Type *base`)

- *De-initialize the DAC12 module.*
- static void [DAC12\\_Enable](#) (DAC\_Type \*base, bool enable)  
*Enable the DAC12's converter or not.*
- static void [DAC12\\_ResetConfig](#) (DAC\_Type \*base)  
*Reset all internal logic and registers.*
- static void [DAC12\\_ResetFIFO](#) (DAC\_Type \*base)  
*Reset the FIFO pointers.*

## Status

- static uint32\_t [DAC12\\_GetStatusFlags](#) (DAC\_Type \*base)  
*Get status flags.*
- static void [DAC12\\_ClearStatusFlags](#) (DAC\_Type \*base, uint32\_t flags)  
*Clear status flags.*

## Interrupts

- static void [DAC12\\_EnableInterrupts](#) (DAC\_Type \*base, uint32\_t mask)  
*Enable interrupts.*
- static void [DAC12\\_DisableInterrupts](#) (DAC\_Type \*base, uint32\_t mask)  
*Disable interrupts.*

## DMA control

- static void [DAC12\\_EnableDMA](#) (DAC\_Type \*base, bool enable)  
*Enable DMA or not.*

## Functional feature

- static void [DAC12\\_SetData](#) (DAC\_Type \*base, uint32\_t value)  
*Set data into the entry of FIFO buffer.*
- static void [DAC12\\_DoSoftwareTrigger](#) (DAC\_Type \*base)  
*Do trigger the FIFO by software.*
- static uint32\_t [DAC12\\_GetFIFOReadPointer](#) (DAC\_Type \*base)  
*Get the current read pointer of FIFO.*
- static uint32\_t [DAC12\\_GetFIFOWritePointer](#) (DAC\_Type \*base)  
*Get the current write pointer of FIFO.*

## 14.3 Data Structure Documentation

### 14.3.1 struct \_dac12\_hardware\_info

#### Data Fields

- [dac12\\_fifo\\_size\\_info\\_t fifoSizeInfo](#)  
*The number of words in this device's DAC buffer.*

## Field Documentation

(1) `dac12_fifo_size_info_t _dac12_hardware_info::fifoSizeInfo`

### 14.3.2 struct `dac12_config_t`

Actually, the most fields are for FIFO buffer.

## Data Fields

- `uint32_t fifoWatermarkLevel`  
*FIFO's watermark, the max value can be the hardware FIFO size.*
- `dac12_fifo_work_mode_t fifoWorkMode`  
*FIFO's work mode about pointers.*
- `dac12_reference_voltage_source_t referenceVoltageSource`  
*Select the reference voltage source.*
- `dac12_reference_current_source_t referenceCurrentSource`  
*Select the trigger mode for FIFO.*
- `dac12_speed_mode_t speedMode`  
*Select the speed mode for conversion.*
- `bool enableAnalogBuffer`  
*Enable analog buffer for high drive.*

## Field Documentation

(1) `uint32_t dac12_config_t::fifoWatermarkLevel`

(2) `dac12_fifo_work_mode_t dac12_config_t::fifoWorkMode`

(3) `dac12_reference_voltage_source_t dac12_config_t::referenceVoltageSource`

(4) `dac12_reference_current_source_t dac12_config_t::referenceCurrentSource`

Select the reference current source.

(5) `dac12_speed_mode_t dac12_config_t::speedMode`

(6) `bool dac12_config_t::enableAnalogBuffer`

## 14.4 Macro Definition Documentation

14.4.1 **#define FSL\_DAC12\_DRIVER\_VERSION (MAKE\_VERSION(2, 1, 1))**

14.4.2 **#define DAC12\_CR\_W1C\_FLAGS\_MASK (DAC\_CR\_OVFF\_MASK | DAC\_CR\_UDFF\_MASK)**

14.4.3 **#define DAC12\_CR\_ALL\_FLAGS\_MASK (DAC12\_CR\_W1C\_FLAGS\_MASK | DAC\_CR\_WMF\_MASK | DAC\_CR\_NEMPTF\_MASK | DAC\_CR\_FULLF\_MASK)**

## 14.5 Typedef Documentation

14.5.1 **typedef enum \_dac12\_reference\_current\_source dac12\_reference\_current\_source\_t**

Analog module needs reference current to keep working . Such reference current can generated by IP itself, or by on-chip PMC's "reference part". If no current reference be selected, analog module can't working normally ,even when other register can still be assigned, DAC would waste current but no function. To make the DAC work, either kDAC12\_ReferenceCurrentSourceAltx should be selected.

## 14.6 Enumeration Type Documentation

### 14.6.1 enum \_dac12\_status\_flags

Enumerator

***kDAC12\_OverflowFlag*** FIFO overflow status flag, which indicates that more data has been written into FIFO than it can hold.

***kDAC12\_UnderflowFlag*** FIFO underflow status flag, which means that there is a new trigger after the FIFO is nearly empty.

***kDAC12\_WatermarkFlag*** FIFO watermark status flag, which indicates the remaining FIFO data is less than the watermark setting.

***kDAC12\_NearlyEmptyFlag*** FIFO nearly empty flag, which means there is only one data remaining in FIFO.

***kDAC12\_FullFlag*** FIFO full status flag, which means that the FIFO read pointer equals the write pointer, as the write pointer increase.

### 14.6.2 enum \_dac12\_interrupt\_enable

Enumerator

***kDAC12\_UnderOrOverflowInterruptEnable*** Underflow and overflow interrupt enable.

***kDAC12\_WatermarkInterruptEnable*** Watermark interrupt enable.



***kDAC12\_NearlyEmptyInterruptEnable*** Nearly empty interrupt enable.

***kDAC12\_FullInterruptEnable*** Full interrupt enable.

### 14.6.3 enum \_dac12\_fifo\_size\_info

Enumerator

***kDAC12\_FIFOSize2*** FIFO depth is 2.

***kDAC12\_FIFOSize4*** FIFO depth is 4.

***kDAC12\_FIFOSize8*** FIFO depth is 8.

***kDAC12\_FIFOSize16*** FIFO depth is 16.

***kDAC12\_FIFOSize32*** FIFO depth is 32.

***kDAC12\_FIFOSize64*** FIFO depth is 64.

***kDAC12\_FIFOSize128*** FIFO depth is 128.

***kDAC12\_FIFOSize256*** FIFO depth is 256.

### 14.6.4 enum \_dac12\_fifo\_work\_mode

Enumerator

***kDAC12\_FIFODisabled*** FIFO disabled and only one level buffer is enabled. Any data written from this buffer goes to conversion.

***kDAC12\_FIFOWorkAsNormalMode*** Data will first read from FIFO to buffer then go to conversion.

***kDAC12\_FIFOWorkAsSwingMode*** In Swing mode, the FIFO must be set up to be full. In Swing back mode, a trigger changes the read pointer to make it swing between the FIFO Full and Nearly Empty state. That is, the trigger increases the read pointer till FIFO is nearly empty and decreases the read pointer till the FIFO is full.

### 14.6.5 enum \_dac12\_reference\_voltage\_source

Enumerator

***kDAC12\_ReferenceVoltageSourceAlt1*** The DAC selects DACREF\_1 as the reference voltage.

***kDAC12\_ReferenceVoltageSourceAlt2*** The DAC selects DACREF\_2 as the reference voltage.

### 14.6.6 enum \_dac12\_fifo\_trigger\_mode

Enumerator

***kDAC12\_FIFOTriggerByHardwareMode*** Buffer would be triggered by hardware.

***kDAC12\_FIFOTriggerBySoftwareMode*** Buffer would be triggered by software.

### 14.6.7 enum \_dac12\_reference\_current\_source

Analog module needs reference current to keep working . Such reference current can generated by IP itself, or by on-chip PMC's "reference part". If no current reference be selected, analog module can't working normally ,even when other register can still be assigned, DAC would waste current but no function. To make the DAC work, either kDAC12\_ReferenceCurrentSourceAlt0 should be selected.

Enumerator

***kDAC12\_ReferenceCurrentSourceDisabled*** None of reference current source is enabled.

***kDAC12\_ReferenceCurrentSourceAlt0*** Use the internal reference current generated by the module itself.

***kDAC12\_ReferenceCurrentSourceAlt1*** Use the ZTC(Zero Temperature Coefficient) reference current generated by on-chip power management module.

***kDAC12\_ReferenceCurrentSourceAlt2*** Use the PTAT(Proportional To Absolution Temperature) reference current generated by power management module.

### 14.6.8 enum \_dac12\_speed\_mode

Enumerator

***kDAC12\_SpeedLowMode*** Low speed mode.

***kDAC12\_SpeedMiddleMode*** Middle speed mode.

***kDAC12\_SpeedHighMode*** High speed mode.

## 14.7 Function Documentation

### 14.7.1 void DAC12\_GetHardwareInfo ( DAC\_Type \* *base*, dac12\_hardware\_info\_t \* *info* )

Parameters

|             |                                                                           |
|-------------|---------------------------------------------------------------------------|
| <i>base</i> | DAC12 peripheral base address.                                            |
| <i>info</i> | Pointer to info structure, see to <a href="#">dac12_hardware_info_t</a> . |

### 14.7.2 void DAC12\_Init ( DAC\_Type \* *base*, const dac12\_config\_t \* *config* )

## Parameters

|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| <i>base</i>   | DAC12 peripheral base address.                                              |
| <i>config</i> | Pointer to configuration structure, see to <a href="#">dac12_config_t</a> . |

**14.7.3 void DAC12\_GetDefaultConfig ( dac12\_config\_t \* *config* )**

This function initializes the user configuration structure to a default value. The default values are:

```
* config->fifoWatermarkLevel = 0U;
* config->fifoWorkMode = kDAC12_FIFODisabled;
* config->referenceVoltageSource = kDAC12_ReferenceVoltageSourceAlt1;
* config->fifoTriggerMode = kDAC12_FIFOTriggerByHardwareMode;
* config->referenceCurrentSource = kDAC12_ReferenceCurrentSourceAlt0;
* config->speedMode = kDAC12_SpeedLowMode;
* config->speedMode = false;
* config->currentReferenceInternalTrimValue = 0x4;
*
```

## Parameters

|               |                                                               |
|---------------|---------------------------------------------------------------|
| <i>config</i> | Pointer to the configuration structure. See "dac12_config_t". |
|---------------|---------------------------------------------------------------|

**14.7.4 void DAC12\_Deinit ( DAC\_Type \* *base* )**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | DAC12 peripheral base address. |
|-------------|--------------------------------|

**14.7.5 static void DAC12\_Enable ( DAC\_Type \* *base*, bool *enable* ) [inline], [static]**

## Parameters

|               |                                      |
|---------------|--------------------------------------|
| <i>base</i>   | DAC12 peripheral base address.       |
| <i>enable</i> | Enable the DAC12's converter or not. |

**14.7.6 static void DAC12\_ResetConfig ( DAC\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | DAC12 peripheral base address. |
|-------------|--------------------------------|

**14.7.7 static void DAC12\_ResetFIFO ( DAC\_Type \* *base* ) [inline], [static]**

FIFO pointers should only be reset when the DAC12 is disabled. This function can be used to configure both pointers to the same address to reset the FIFO as empty.

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | DAC12 peripheral base address. |
|-------------|--------------------------------|

**14.7.8 static uint32\_t DAC12\_GetStatusFlags ( DAC\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | DAC12 peripheral base address. |
|-------------|--------------------------------|

## Returns

Mask of current status flags. See to [\\_dac12\\_status\\_flags](#).

**14.7.9 static void DAC12\_ClearStatusFlags ( DAC\_Type \* *base*, uint32\_t *flags* ) [inline], [static]**

Note: Not all the flags can be cleared by this API. Several flags need special condition to clear them according to target chip's reference manual document.

## Parameters

|              |                                                                                  |
|--------------|----------------------------------------------------------------------------------|
| <i>base</i>  | DAC12 peripheral base address.                                                   |
| <i>flags</i> | Mask of status flags to be cleared. See to <a href="#">_dac12_status_flags</a> . |

**14.7.10 static void DAC12\_EnableInterrupts ( DAC\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**

## Parameters

|             |                                                                                          |
|-------------|------------------------------------------------------------------------------------------|
| <i>base</i> | DAC12 peripheral base address.                                                           |
| <i>mask</i> | Mask value of interrupts to be enabled. See to <a href="#">_dac12_interrupt_enable</a> . |

**14.7.11 static void DAC12\_DisableInterrupts ( DAC\_Type \* *base*, uint32\_t *mask* )**  
**[inline], [static]**

## Parameters

|             |                                                                                           |
|-------------|-------------------------------------------------------------------------------------------|
| <i>base</i> | DAC12 peripheral base address.                                                            |
| <i>mask</i> | Mask value of interrupts to be disabled. See to <a href="#">_dac12_interrupt_enable</a> . |

**14.7.12 static void DAC12\_EnableDMA ( DAC\_Type \* *base*, bool *enable* )**  
**[inline], [static]**

When DMA is enabled, the DMA request will be generated by original interrupts. The interrupts will not be presented on this module at the same time.

**14.7.13 static void DAC12\_SetData ( DAC\_Type \* *base*, uint32\_t *value* )**  
**[inline], [static]**

When the DAC FIFO is disabled, and the one entry buffer is enabled, the DAC converts the data in the buffer to analog output voltage. Any write to the DATA register will replace the data in the buffer and push data to analog conversion without trigger support. When the DAC FIFO is enabled. Writing data would increase the write pointer of FIFO. Also, the data would be restored into the FIFO buffer.

## Parameters

|              |                                 |
|--------------|---------------------------------|
| <i>base</i>  | DAC12 peripheral base address.  |
| <i>value</i> | Setting value into FIFO buffer. |

**14.7.14 static void DAC12\_DoSoftwareTrigger ( DAC\_Type \* *base* ) [inline],**  
**[static]**

When the DAC FIFO is enabled, and software trigger is used. Doing trigger would increase the read pointer, and the data in the entry pointed by read pointer would be converted as new output.

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | DAC12 peripheral base address. |
|-------------|--------------------------------|

**14.7.15 static uint32\_t DAC12\_GetFIFOReadPointer ( DAC\_Type \* *base* )  
[inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | DAC12 peripheral base address. |
|-------------|--------------------------------|

## Returns

Read pointer index of FIFO buffer.

**14.7.16 static uint32\_t DAC12\_GetFIFOWritePointer ( DAC\_Type \* *base* )  
[inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | DAC12 peripheral base address. |
|-------------|--------------------------------|

## Returns

Write pointer index of FIFO buffer

## Chapter 15

# DMAMUX: Direct Memory Access Multiplexer Driver

### 15.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Direct Memory Access Multiplexer (DMAMUX) of MCUXpresso SDK devices.

### 15.2 Typical use case

#### 15.2.1 DMAMUX Operation

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/dmamux

#### Driver version

- #define [FSL\\_DMAMUX\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 1, 0))  
*DMAMUX driver version 2.1.0.*

#### DMAMUX Initialization and de-initialization

- void [DMAMUX\\_Init](#) (DMAMUX\_Type \*base)  
*Initializes the DMAMUX peripheral.*
- void [DMAMUX\\_Deinit](#) (DMAMUX\_Type \*base)  
*Deinitializes the DMAMUX peripheral.*

#### DMAMUX Channel Operation

- static void [DMAMUX\\_EnableChannel](#) (DMAMUX\_Type \*base, uint32\_t channel)  
*Enables the DMAMUX channel.*
- static void [DMAMUX\\_DisableChannel](#) (DMAMUX\_Type \*base, uint32\_t channel)  
*Disables the DMAMUX channel.*
- static void [DMAMUX\\_SetSource](#) (DMAMUX\_Type \*base, uint32\_t channel, int32\_t source)  
*Configures the DMAMUX channel source.*

### 15.3 Macro Definition Documentation

#### 15.3.1 #define FSL\_DMAMUX\_DRIVER\_VERSION (MAKE\_VERSION(2, 1, 0))

### 15.4 Function Documentation

#### 15.4.1 void DMAMUX\_Init ( DMAMUX\_Type \* *base* )

This function ungates the DMAMUX clock.

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | DMAMUX peripheral base address. |
|-------------|---------------------------------|

**15.4.2 void DMAMUX\_Deinit ( DMAMUX\_Type \* *base* )**

This function gates the DMAMUX clock.

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | DMAMUX peripheral base address. |
|-------------|---------------------------------|

**15.4.3 static void DMAMUX\_EnableChannel ( DMAMUX\_Type \* *base*, uint32\_t *channel* ) [inline], [static]**

This function enables the DMAMUX channel.

## Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | DMAMUX peripheral base address. |
| <i>channel</i> | DMAMUX channel number.          |

**15.4.4 static void DMAMUX\_DisableChannel ( DMAMUX\_Type \* *base*, uint32\_t *channel* ) [inline], [static]**

This function disables the DMAMUX channel.

## Note

The user must disable the DMAMUX channel before configuring it.

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | DMAMUX peripheral base address. |
|-------------|---------------------------------|



|                |                        |
|----------------|------------------------|
| <i>channel</i> | DMAMUX channel number. |
|----------------|------------------------|

**15.4.5 static void DMAMUX\_SetSource ( DMAMUX\_Type \* *base*, uint32\_t *channel*, int32\_t *source* ) [inline], [static]**

Parameters

|                |                                                                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>    | DMAMUX peripheral base address.                                                                                                   |
| <i>channel</i> | DMAMUX channel number.                                                                                                            |
| <i>source</i>  | Channel source, which is used to trigger the DMA transfer. User need to use the dma-request_source_t type as the input parameter. |

## Chapter 16

# eDMA: Enhanced Direct Memory Access (eDMA) Controller Driver

### 16.1 Overview

The MCUXpresso SDK provides a peripheral driver for the enhanced Direct Memory Access (eDMA) of MCUXpresso SDK devices.

### 16.2 Typical use case

#### 16.2.1 eDMA Operation

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/edma

### Data Structures

- struct [\\_edma\\_config](#)  
*eDMA global configuration structure. [More...](#)*
- struct [\\_edma\\_transfer\\_config](#)  
*eDMA transfer configuration [More...](#)*
- struct [\\_edma\\_channel\\_Preemption\\_config](#)  
*eDMA channel priority configuration [More...](#)*
- struct [\\_edma\\_minor\\_offset\\_config](#)  
*eDMA minor offset configuration [More...](#)*
- struct [\\_edma\\_tcd](#)  
*eDMA TCD. [More...](#)*
- struct [\\_edma\\_handle](#)  
*eDMA transfer handle structure [More...](#)*

### Macros

- #define [DMA\\_DCHPRI\\_INDEX](#)(channel) (((channel) & ~0x03U) | (3U - ((channel)&0x03U)))  
*Compute the offset unit from DCHPRI3.*

### Typedefs

- typedef enum [\\_edma\\_transfer\\_size](#) [edma\\_transfer\\_size\\_t](#)  
*eDMA transfer configuration*
- typedef enum [\\_edma\\_modulo](#) [edma\\_modulo\\_t](#)  
*eDMA modulo configuration*
- typedef enum [\\_edma\\_bandwidth](#) [edma\\_bandwidth\\_t](#)  
*Bandwidth control.*
- typedef enum [\\_edma\\_channel\\_link\\_type](#) [edma\\_channel\\_link\\_type\\_t](#)

- *Channel link type.*
- typedef enum `_edma_interrupt_enable edma_interrupt_enable_t`  
*eDMA interrupt source*
- typedef enum `_edma_transfer_type edma_transfer_type_t`  
*eDMA transfer type*
- typedef struct `_edma_config edma_config_t`  
*eDMA global configuration structure.*
- typedef struct `_edma_transfer_config edma_transfer_config_t`  
*eDMA transfer configuration*
- typedef struct `_edma_channel_Preemption_config edma_channel_Preemption_config_t`  
*eDMA channel priority configuration*
- typedef struct `_edma_minor_offset_config edma_minor_offset_config_t`  
*eDMA minor offset configuration*
- typedef struct `_edma_tcd edma_tcd_t`  
*eDMA TCD.*
- typedef void(\* `edma_callback` )(struct `_edma_handle` \*handle, void \*userData, bool transferDone, uint32\_t tcDs)  
*Define callback function for eDMA.*
- typedef struct `_edma_handle edma_handle_t`  
*eDMA transfer handle structure*

## Enumerations

- enum `_edma_transfer_size` {  
`kEDMA_TransferSize1Bytes` = 0x0U,  
`kEDMA_TransferSize2Bytes` = 0x1U,  
`kEDMA_TransferSize4Bytes` = 0x2U,  
`kEDMA_TransferSize8Bytes` = 0x3U,  
`kEDMA_TransferSize16Bytes` = 0x4U,  
`kEDMA_TransferSize32Bytes` = 0x5U }  
*eDMA transfer configuration*
- enum `_edma_modulo` {

```

kEDMA_ModuloDisable = 0x0U,
kEDMA_Modulo2bytes,
kEDMA_Modulo4bytes,
kEDMA_Modulo8bytes,
kEDMA_Modulo16bytes,
kEDMA_Modulo32bytes,
kEDMA_Modulo64bytes,
kEDMA_Modulo128bytes,
kEDMA_Modulo256bytes,
kEDMA_Modulo512bytes,
kEDMA_Modulo1Kbytes,
kEDMA_Modulo2Kbytes,
kEDMA_Modulo4Kbytes,
kEDMA_Modulo8Kbytes,
kEDMA_Modulo16Kbytes,
kEDMA_Modulo32Kbytes,
kEDMA_Modulo64Kbytes,
kEDMA_Modulo128Kbytes,
kEDMA_Modulo256Kbytes,
kEDMA_Modulo512Kbytes,
kEDMA_Modulo1Mbytes,
kEDMA_Modulo2Mbytes,
kEDMA_Modulo4Mbytes,
kEDMA_Modulo8Mbytes,
kEDMA_Modulo16Mbytes,
kEDMA_Modulo32Mbytes,
kEDMA_Modulo64Mbytes,
kEDMA_Modulo128Mbytes,
kEDMA_Modulo256Mbytes,
kEDMA_Modulo512Mbytes,
kEDMA_Modulo1Gbytes,
kEDMA_Modulo2Gbytes }
 eDMA modulo configuration
• enum _edma_bandwidth {
 kEDMA_BandwidthStallNone = 0x0U,
 kEDMA_BandwidthStall4Cycle = 0x2U,
 kEDMA_BandwidthStall8Cycle = 0x3U }
 Bandwidth control.
• enum _edma_channel_link_type {
 kEDMA_LinkNone = 0x0U,
 kEDMA_MinorLink,
 kEDMA_MajorLink }
 Channel link type.
• enum {

```

```

kEDMA_DoneFlag = 0x1U,
kEDMA_ErrorFlag = 0x2U,
kEDMA_InterruptFlag = 0x4U }
 _edma_channel_status_flags eDMA channel status flags.
• enum {
 kEDMA_DestinationBusErrorFlag = DMA_ES_DBE_MASK,
 kEDMA_SourceBusErrorFlag = DMA_ES_SBE_MASK,
 kEDMA_ScatterGatherErrorFlag = DMA_ES_SGE_MASK,
 kEDMA_NbytesErrorFlag = DMA_ES_NCE_MASK,
 kEDMA_DestinationOffsetErrorFlag = DMA_ES_DOE_MASK,
 kEDMA_DestinationAddressErrorFlag = DMA_ES_DAE_MASK,
 kEDMA_SourceOffsetErrorFlag = DMA_ES_SOE_MASK,
 kEDMA_SourceAddressErrorFlag = DMA_ES_SAE_MASK,
 kEDMA_ErrorChannelFlag = DMA_ES_ERRCHN_MASK,
 kEDMA_ChannelPriorityErrorFlag = DMA_ES_CPE_MASK,
 kEDMA_TransferCanceledFlag = DMA_ES_ECX_MASK,
 kEDMA_ValidFlag = (int)DMA_ES_VLD_MASK }
 _edma_error_status_flags eDMA channel error status flags.
• enum _edma_interrupt_enable {
 kEDMA_ErrorInterruptEnable = 0x1U,
 kEDMA_MajorInterruptEnable = DMA_CSR_INTMAJOR_MASK,
 kEDMA_HalfInterruptEnable = DMA_CSR_INTHALF_MASK }
 eDMA interrupt source
• enum _edma_transfer_type {
 kEDMA_MemoryToMemory = 0x0U,
 kEDMA_PeripheralToMemory,
 kEDMA_MemoryToPeripheral,
 kEDMA_PeripheralToPeripheral }
 eDMA transfer type
• enum {
 kStatus_EDMA_QueueFull = MAKE_STATUS(kStatusGroup_EDMA, 0),
 kStatus_EDMA_Busy = MAKE_STATUS(kStatusGroup_EDMA, 1) }
 _edma_transfer_status eDMA transfer status

```

## Driver version

- #define **FSL\_EDMA\_DRIVER\_VERSION** (**MAKE\_VERSION**(2, 4, 4))  
eDMA driver version

## eDMA initialization and de-initialization

- void **EDMA\_Init** (DMA\_Type \*base, const **edma\_config\_t** \*config)  
Initializes the eDMA peripheral.
- void **EDMA\_Deinit** (DMA\_Type \*base)  
Deinitializes the eDMA peripheral.
- void **EDMA\_InstallTCD** (DMA\_Type \*base, uint32\_t channel, **edma\_tcd\_t** \*tcd)  
Push content of TCD structure into hardware TCD register.
- void **EDMA\_GetDefaultConfig** (**edma\_config\_t** \*config)

- Gets the eDMA default configuration structure.*
- static void [EDMA\\_EnableContinuousChannelLinkMode](#) (DMA\_Type \*base, bool enable)  
*Enable/Disable continuous channel link mode.*
- static void [EDMA\\_EnableMinorLoopMapping](#) (DMA\_Type \*base, bool enable)  
*Enable/Disable minor loop mapping.*

## eDMA Channel Operation

- void [EDMA\\_ResetChannel](#) (DMA\_Type \*base, uint32\_t channel)  
*Sets all TCD registers to default values.*
- void [EDMA\\_SetTransferConfig](#) (DMA\_Type \*base, uint32\_t channel, const [edma\\_transfer\\_config\\_t](#) \*config, [edma\\_tcd\\_t](#) \*nextTcd)  
*Configures the eDMA transfer attribute.*
- void [EDMA\\_SetMinorOffsetConfig](#) (DMA\_Type \*base, uint32\_t channel, const [edma\\_minor\\_offset\\_config\\_t](#) \*config)  
*Configures the eDMA minor offset feature.*
- void [EDMA\\_SetChannelPreemptionConfig](#) (DMA\_Type \*base, uint32\_t channel, const [edma\\_channel\\_preemption\\_config\\_t](#) \*config)  
*Configures the eDMA channel preemption feature.*
- void [EDMA\\_SetChannelLink](#) (DMA\_Type \*base, uint32\_t channel, [edma\\_channel\\_link\\_type\\_t](#) linkType, uint32\_t linkedChannel)  
*Sets the channel link for the eDMA transfer.*
- void [EDMA\\_SetBandWidth](#) (DMA\_Type \*base, uint32\_t channel, [edma\\_bandwidth\\_t](#) bandWidth)  
*Sets the bandwidth for the eDMA transfer.*
- void [EDMA\\_SetModulo](#) (DMA\_Type \*base, uint32\_t channel, [edma\\_modulo\\_t](#) srcModulo, [edma\\_modulo\\_t](#) destModulo)  
*Sets the source modulo and the destination modulo for the eDMA transfer.*
- static void [EDMA\\_EnableAsyncRequest](#) (DMA\_Type \*base, uint32\_t channel, bool enable)  
*Enables an async request for the eDMA transfer.*
- static void [EDMA\\_EnableAutoStopRequest](#) (DMA\_Type \*base, uint32\_t channel, bool enable)  
*Enables an auto stop request for the eDMA transfer.*
- void [EDMA\\_EnableChannelInterrupts](#) (DMA\_Type \*base, uint32\_t channel, uint32\_t mask)  
*Enables the interrupt source for the eDMA transfer.*
- void [EDMA\\_DisableChannelInterrupts](#) (DMA\_Type \*base, uint32\_t channel, uint32\_t mask)  
*Disables the interrupt source for the eDMA transfer.*
- void [EDMA\\_SetMajorOffsetConfig](#) (DMA\_Type \*base, uint32\_t channel, int32\_t sourceOffset, int32\_t destOffset)  
*Configures the eDMA channel TCD major offset feature.*

## eDMA TCD Operation

- void [EDMA\\_TcdReset](#) ([edma\\_tcd\\_t](#) \*tcd)  
*Sets all fields to default values for the TCD structure.*
- void [EDMA\\_TcdSetTransferConfig](#) ([edma\\_tcd\\_t](#) \*tcd, const [edma\\_transfer\\_config\\_t](#) \*config, [edma\\_tcd\\_t](#) \*nextTcd)  
*Configures the eDMA TCD transfer attribute.*
- void [EDMA\\_TcdSetMinorOffsetConfig](#) ([edma\\_tcd\\_t](#) \*tcd, const [edma\\_minor\\_offset\\_config\\_t](#) \*config)  
*Configures the eDMA TCD minor offset feature.*

- void [EDMA\\_TcdSetChannelLink](#) (edma\_tcd\_t \*tcd, edma\_channel\_link\_type\_t linkType, uint32\_t linkedChannel)  
*Sets the channel link for the eDMA TCD.*
- static void [EDMA\\_TcdSetBandWidth](#) (edma\_tcd\_t \*tcd, edma\_bandwidth\_t bandWidth)  
*Sets the bandwidth for the eDMA TCD.*
- void [EDMA\\_TcdSetModulo](#) (edma\_tcd\_t \*tcd, edma\_modulo\_t srcModulo, edma\_modulo\_t destModulo)  
*Sets the source modulo and the destination modulo for the eDMA TCD.*
- static void [EDMA\\_TcdEnableAutoStopRequest](#) (edma\_tcd\_t \*tcd, bool enable)  
*Sets the auto stop request for the eDMA TCD.*
- void [EDMA\\_TcdEnableInterrupts](#) (edma\_tcd\_t \*tcd, uint32\_t mask)  
*Enables the interrupt source for the eDMA TCD.*
- void [EDMA\\_TcdDisableInterrupts](#) (edma\_tcd\_t \*tcd, uint32\_t mask)  
*Disables the interrupt source for the eDMA TCD.*
- void [EDMA\\_TcdSetMajorOffsetConfig](#) (edma\_tcd\_t \*tcd, int32\_t sourceOffset, int32\_t destOffset)  
*Configures the eDMA TCD major offset feature.*

## eDMA Channel Transfer Operation

- static void [EDMA\\_EnableChannelRequest](#) (DMA\_Type \*base, uint32\_t channel)  
*Enables the eDMA hardware channel request.*
- static void [EDMA\\_DisableChannelRequest](#) (DMA\_Type \*base, uint32\_t channel)  
*Disables the eDMA hardware channel request.*
- static void [EDMA\\_TriggerChannelStart](#) (DMA\_Type \*base, uint32\_t channel)  
*Starts the eDMA transfer by using the software trigger.*

## eDMA Channel Status Operation

- uint32\_t [EDMA\\_GetRemainingMajorLoopCount](#) (DMA\_Type \*base, uint32\_t channel)  
*Gets the remaining major loop count from the eDMA current channel TCD.*
- static uint32\_t [EDMA\\_GetErrorStatusFlags](#) (DMA\_Type \*base)  
*Gets the eDMA channel error status flags.*
- uint32\_t [EDMA\\_GetChannelStatusFlags](#) (DMA\_Type \*base, uint32\_t channel)  
*Gets the eDMA channel status flags.*
- void [EDMA\\_ClearChannelStatusFlags](#) (DMA\_Type \*base, uint32\_t channel, uint32\_t mask)  
*Clears the eDMA channel status flags.*

## eDMA Transactional Operation

- void [EDMA\\_CreateHandle](#) (edma\_handle\_t \*handle, DMA\_Type \*base, uint32\_t channel)  
*Creates the eDMA handle.*
- void [EDMA\\_InstallTCDMemory](#) (edma\_handle\_t \*handle, edma\_tcd\_t \*tcdPool, uint32\_t tcdSize)  
*Installs the TCDs memory pool into the eDMA handle.*
- void [EDMA\\_SetCallback](#) (edma\_handle\_t \*handle, edma\_callback callback, void \*userData)  
*Installs a callback function for the eDMA transfer.*
- void [EDMA\\_PrepareTransferConfig](#) (edma\_transfer\_config\_t \*config, void \*srcAddr, uint32\_t srcWidth, int16\_t srcOffset, void \*destAddr, uint32\_t destWidth, int16\_t destOffset, uint32\_t bytes-EachRequest, uint32\_t transferBytes)  
*Prepares the eDMA transfer structure configurations.*

- void [EDMA\\_PrepareTransfer](#) ([edma\\_transfer\\_config\\_t](#) \*config, void \*srcAddr, uint32\_t srcWidth, void \*destAddr, uint32\_t destWidth, uint32\_t bytesEachRequest, uint32\_t transferBytes, [edma\\_transfer\\_type\\_t](#) transferType)  
*Prepares the eDMA transfer structure.*
- [status\\_t](#) [EDMA\\_SubmitTransfer](#) ([edma\\_handle\\_t](#) \*handle, const [edma\\_transfer\\_config\\_t](#) \*config)  
*Submits the eDMA transfer request.*
- void [EDMA\\_StartTransfer](#) ([edma\\_handle\\_t](#) \*handle)  
*eDMA starts transfer.*
- void [EDMA\\_StopTransfer](#) ([edma\\_handle\\_t](#) \*handle)  
*eDMA stops transfer.*
- void [EDMA\\_AbortTransfer](#) ([edma\\_handle\\_t](#) \*handle)  
*eDMA aborts transfer.*
- static uint32\_t [EDMA\\_GetUnusedTCDNumber](#) ([edma\\_handle\\_t](#) \*handle)  
*Get unused TCD slot number.*
- static uint32\_t [EDMA\\_GetNextTCDAddress](#) ([edma\\_handle\\_t](#) \*handle)  
*Get the next tcd address.*
- void [EDMA\\_HandleIRQ](#) ([edma\\_handle\\_t](#) \*handle)  
*eDMA IRQ handler for the current major loop transfer completion.*

## 16.3 Data Structure Documentation

### 16.3.1 struct \_edma\_config

#### Data Fields

- bool [enableContinuousLinkMode](#)  
*Enable (true) continuous link mode.*
- bool [enableHaltOnError](#)  
*Enable (true) transfer halt on error.*
- bool [enableRoundRobinArbitration](#)  
*Enable (true) round robin channel arbitration method or fixed priority arbitration is used for channel selection.*
- bool [enableDebugMode](#)  
*Enable(true) eDMA debug mode.*

#### Field Documentation

##### (1) bool \_edma\_config::enableContinuousLinkMode

Upon minor loop completion, the channel activates again if that channel has a minor loop channel link enabled and the link channel is itself.

##### (2) bool \_edma\_config::enableHaltOnError

Any error causes the HALT bit to set. Subsequently, all service requests are ignored until the HALT bit is cleared.



**(3) bool \_edma\_config::enableDebugMode**

When in debug mode, the eDMA stalls the start of a new channel. Executing channels are allowed to complete.

**16.3.2 struct \_edma\_transfer\_config**

This structure configures the source/destination transfer attribute.

**Data Fields**

- uint32\_t [srcAddr](#)  
*Source data address.*
- uint32\_t [destAddr](#)  
*Destination data address.*
- [edma\\_transfer\\_size\\_t](#) [srcTransferSize](#)  
*Source data transfer size.*
- [edma\\_transfer\\_size\\_t](#) [destTransferSize](#)  
*Destination data transfer size.*
- int16\_t [srcOffset](#)  
*Sign-extended offset applied to the current source address to form the next-state value as each source read is completed.*
- int16\_t [destOffset](#)  
*Sign-extended offset applied to the current destination address to form the next-state value as each destination write is completed.*
- uint32\_t [minorLoopBytes](#)  
*Bytes to transfer in a minor loop.*
- uint32\_t [majorLoopCounts](#)  
*Major loop iteration count.*

## Field Documentation

- (1) `uint32_t _edma_transfer_config::srcAddr`
- (2) `uint32_t _edma_transfer_config::destAddr`
- (3) `edma_transfer_size_t _edma_transfer_config::srcTransferSize`
- (4) `edma_transfer_size_t _edma_transfer_config::destTransferSize`
- (5) `int16_t _edma_transfer_config::srcOffset`
- (6) `int16_t _edma_transfer_config::destOffset`
- (7) `uint32_t _edma_transfer_config::majorLoopCounts`

16.3.3 `struct _edma_channel_Preemption_config`

## Data Fields

- `bool enableChannelPreemption`  
*If true: a channel can be suspended by other channel with higher priority.*
- `bool enablePreemptAbility`  
*If true: a channel can suspend other channel with low priority.*
- `uint8_t channelPriority`  
*Channel priority.*

16.3.4 `struct _edma_minor_offset_config`

## Data Fields

- `bool enableSrcMinorOffset`  
*Enable(true) or Disable(false) source minor loop offset.*
- `bool enableDestMinorOffset`  
*Enable(true) or Disable(false) destination minor loop offset.*
- `uint32_t minorOffset`  
*Offset for a minor loop mapping.*

## Field Documentation

- (1) `bool _edma_minor_offset_config::enableSrcMinorOffset`
- (2) `bool _edma_minor_offset_config::enableDestMinorOffset`
- (3) `uint32_t _edma_minor_offset_config::minorOffset`

## 16.3.5 struct \_edma\_tcd

This structure is same as TCD register which is described in reference manual, and is used to configure the scatter/gather feature as a next hardware TCD.

## Data Fields

- `__IO uint32_t SADDR`  
*SADDR register, used to save source address.*
- `__IO uint16_t SOFF`  
*SOFF register, save offset bytes every transfer.*
- `__IO uint16_t ATTR`  
*ATTR register, source/destination transfer size and modulo.*
- `__IO uint32_t NBYTES`  
*Nbytes register, minor loop length in bytes.*
- `__IO uint32_t SLAST`  
*SLAST register.*
- `__IO uint32_t DADDR`  
*DADDR register, used for destination address.*
- `__IO uint16_t DOFF`  
*DOFF register, used for destination offset.*
- `__IO uint16_t CITER`  
*CITER register, current minor loop numbers, for unfinished minor loop.*
- `__IO uint32_t DLAST_SGA`  
*DLASTSGA register, next tcd address used in scatter-gather mode.*
- `__IO uint16_t CSR`  
*CSR register, for TCD control status.*
- `__IO uint16_t BITER`  
*BITER register, begin minor loop count.*

## Field Documentation

(1) `__IO uint16_t _edma_tcd::CITER`(2) `__IO uint16_t _edma_tcd::BITER`16.3.6 `struct _edma_handle`

## Data Fields

- `edma_callback callback`  
*Callback function for major count exhausted.*
- `void * userData`  
*Callback function parameter.*
- `DMA_Type * base`  
*eDMA peripheral base address.*
- `edma_tcd_t * tcdPool`  
*Pointer to memory stored TCDs.*
- `uint8_t channel`  
*eDMA channel number.*
- `volatile int8_t header`  
*The first TCD index.*
- `volatile int8_t tail`  
*The last TCD index.*
- `volatile int8_t tcdUsed`  
*The number of used TCD slots.*
- `volatile int8_t tcdSize`  
*The total number of TCD slots in the queue.*
- `uint8_t flags`  
*The status of the current channel.*

## Field Documentation

(1) `edma_callback _edma_handle::callback`(2) `void* _edma_handle::userData`(3) `DMA_Type* _edma_handle::base`(4) `edma_tcd_t* _edma_handle::tcdPool`(5) `uint8_t _edma_handle::channel`(6) `volatile int8_t _edma_handle::header`

Should point to the next TCD to be loaded into the eDMA engine.

(7) `volatile int8_t _edma_handle::tail`

Should point to the next TCD to be stored into the memory pool.

(8) **volatile int8\_t \_edma\_handle::tcdUsed**

Should reflect the number of TCDs can be used/loaded in the memory.

(9) **volatile int8\_t \_edma\_handle::tcdSize**

(10) **uint8\_t \_edma\_handle::flags**

## 16.4 Macro Definition Documentation

**16.4.1 #define FSL\_EDMA\_DRIVER\_VERSION (MAKE\_VERSION(2, 4, 4))**

Version 2.4.4.

## 16.5 Typedef Documentation

**16.5.1 typedef struct \_edma\_config edma\_config\_t**

**16.5.2 typedef struct \_edma\_transfer\_config edma\_transfer\_config\_t**

This structure configures the source/destination transfer attribute.

**16.5.3 typedef struct \_edma\_tcd edma\_tcd\_t**

This structure is same as TCD register which is described in reference manual, and is used to configure the scatter/gather feature as a next hardware TCD.

**16.5.4 typedef void(\* edma\_callback)(struct \_edma\_handle \*handle, void \*userData, bool transferDone, uint32\_t tcDs)**

This callback function is called in the EDMA interrupt handle. In normal mode, run into callback function means the transfer users need is done. In scatter gather mode, run into callback function means a transfer control block (tcd) is finished. Not all transfer finished, users can get the finished tcd numbers using interface EDMA\_GetUnusedTCDNumber.

Parameters

|               |                                                               |
|---------------|---------------------------------------------------------------|
| <i>handle</i> | EDMA handle pointer, users shall not touch the values inside. |
|---------------|---------------------------------------------------------------|

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>userData</i>     | The callback user parameter pointer. Users can use this parameter to involve things users need to change in EDMA callback function.                                                                                                                                                                                                                                                                                                             |
| <i>transferDone</i> | If the current loaded transfer done. In normal mode it means if all transfer done. In scatter gather mode, this parameter shows is the current transfer block in EDMA register is done. As the load of core is different, it will be different if the new tcd loaded into EDMA registers while this callback called. If true, it always means new tcd still not loaded into registers, while false means new tcd already loaded into registers. |
| <i>tcds</i>         | How many tcds are done from the last callback. This parameter only used in scatter gather mode. It tells user how many tcds are finished between the last callback and this.                                                                                                                                                                                                                                                                    |

## 16.6 Enumeration Type Documentation

### 16.6.1 enum\_edma\_transfer\_size

Enumerator

***kEDMA\_TransferSize1Bytes*** Source/Destination data transfer size is 1 byte every time.  
***kEDMA\_TransferSize2Bytes*** Source/Destination data transfer size is 2 bytes every time.  
***kEDMA\_TransferSize4Bytes*** Source/Destination data transfer size is 4 bytes every time.  
***kEDMA\_TransferSize8Bytes*** Source/Destination data transfer size is 8 bytes every time.  
***kEDMA\_TransferSize16Bytes*** Source/Destination data transfer size is 16 bytes every time.  
***kEDMA\_TransferSize32Bytes*** Source/Destination data transfer size is 32 bytes every time.

### 16.6.2 enum\_edma\_modulo

Enumerator

***kEDMA\_ModuloDisable*** Disable modulo.  
***kEDMA\_Modulo2bytes*** Circular buffer size is 2 bytes.  
***kEDMA\_Modulo4bytes*** Circular buffer size is 4 bytes.  
***kEDMA\_Modulo8bytes*** Circular buffer size is 8 bytes.  
***kEDMA\_Modulo16bytes*** Circular buffer size is 16 bytes.  
***kEDMA\_Modulo32bytes*** Circular buffer size is 32 bytes.  
***kEDMA\_Modulo64bytes*** Circular buffer size is 64 bytes.  
***kEDMA\_Modulo128bytes*** Circular buffer size is 128 bytes.  
***kEDMA\_Modulo256bytes*** Circular buffer size is 256 bytes.  
***kEDMA\_Modulo512bytes*** Circular buffer size is 512 bytes.  
***kEDMA\_Modulo1Kbytes*** Circular buffer size is 1 K bytes.  
***kEDMA\_Modulo2Kbytes*** Circular buffer size is 2 K bytes.  
***kEDMA\_Modulo4Kbytes*** Circular buffer size is 4 K bytes.  
***kEDMA\_Modulo8Kbytes*** Circular buffer size is 8 K bytes.

*kEDMA\_Modulo16Kbytes* Circular buffer size is 16 K bytes.  
*kEDMA\_Modulo32Kbytes* Circular buffer size is 32 K bytes.  
*kEDMA\_Modulo64Kbytes* Circular buffer size is 64 K bytes.  
*kEDMA\_Modulo128Kbytes* Circular buffer size is 128 K bytes.  
*kEDMA\_Modulo256Kbytes* Circular buffer size is 256 K bytes.  
*kEDMA\_Modulo512Kbytes* Circular buffer size is 512 K bytes.  
*kEDMA\_Modulo1Mbytes* Circular buffer size is 1 M bytes.  
*kEDMA\_Modulo2Mbytes* Circular buffer size is 2 M bytes.  
*kEDMA\_Modulo4Mbytes* Circular buffer size is 4 M bytes.  
*kEDMA\_Modulo8Mbytes* Circular buffer size is 8 M bytes.  
*kEDMA\_Modulo16Mbytes* Circular buffer size is 16 M bytes.  
*kEDMA\_Modulo32Mbytes* Circular buffer size is 32 M bytes.  
*kEDMA\_Modulo64Mbytes* Circular buffer size is 64 M bytes.  
*kEDMA\_Modulo128Mbytes* Circular buffer size is 128 M bytes.  
*kEDMA\_Modulo256Mbytes* Circular buffer size is 256 M bytes.  
*kEDMA\_Modulo512Mbytes* Circular buffer size is 512 M bytes.  
*kEDMA\_Modulo1Gbytes* Circular buffer size is 1 G bytes.  
*kEDMA\_Modulo2Gbytes* Circular buffer size is 2 G bytes.

### 16.6.3 enum\_edma\_bandwidth

Enumerator

*kEDMA\_BandwidthStallNone* No eDMA engine stalls.  
*kEDMA\_BandwidthStall4Cycle* eDMA engine stalls for 4 cycles after each read/write.  
*kEDMA\_BandwidthStall8Cycle* eDMA engine stalls for 8 cycles after each read/write.

### 16.6.4 enum\_edma\_channel\_link\_type

Enumerator

*kEDMA\_LinkNone* No channel link.  
*kEDMA\_MinorLink* Channel link after each minor loop.  
*kEDMA\_MajorLink* Channel link while major loop count exhausted.

### 16.6.5 anonymous enum

Enumerator

*kEDMA\_DoneFlag* DONE flag, set while transfer finished, CITER value exhausted.  
*kEDMA\_ErrorFlag* eDMA error flag, an error occurred in a transfer  
*kEDMA\_InterruptFlag* eDMA interrupt flag, set while an interrupt occurred of this channel

### 16.6.6 anonymous enum

Enumerator

***kEDMA\_DestinationBusErrorFlag*** Bus error on destination address.  
***kEDMA\_SourceBusErrorFlag*** Bus error on the source address.  
***kEDMA\_ScatterGatherErrorFlag*** Error on the Scatter/Gather address, not 32byte aligned.  
***kEDMA\_NbytesErrorFlag*** NBYTES/CITER configuration error.  
***kEDMA\_DestinationOffsetErrorFlag*** Destination offset not aligned with destination size.  
***kEDMA\_DestinationAddressErrorFlag*** Destination address not aligned with destination size.  
***kEDMA\_SourceOffsetErrorFlag*** Source offset not aligned with source size.  
***kEDMA\_SourceAddressErrorFlag*** Source address not aligned with source size.  
***kEDMA\_ErrorChannelFlag*** Error channel number of the cancelled channel number.  
***kEDMA\_ChannelPriorityErrorFlag*** Channel priority is not unique.  
***kEDMA\_TransferCanceledFlag*** Transfer cancelled.  
***kEDMA\_ValidFlag*** No error occurred, this bit is 0. Otherwise, it is 1.

### 16.6.7 enum \_edma\_interrupt\_enable

Enumerator

***kEDMA\_ErrorInterruptEnable*** Enable interrupt while channel error occurs.  
***kEDMA\_MajorInterruptEnable*** Enable interrupt while major count exhausted.  
***kEDMA\_HalfInterruptEnable*** Enable interrupt while major count to half value.

### 16.6.8 enum \_edma\_transfer\_type

Enumerator

***kEDMA\_MemoryToMemory*** Transfer from memory to memory.  
***kEDMA\_PeripheralToMemory*** Transfer from peripheral to memory.  
***kEDMA\_MemoryToPeripheral*** Transfer from memory to peripheral.  
***kEDMA\_PeripheralToPeripheral*** Transfer from Peripheral to peripheral.

### 16.6.9 anonymous enum

Enumerator

***kStatus\_EDMA\_QueueFull*** TCD queue is full.  
***kStatus\_EDMA\_Busy*** Channel is busy and can't handle the transfer request.



## 16.7 Function Documentation

### 16.7.1 void EDMA\_Init ( DMA\_Type \* *base*, const edma\_config\_t \* *config* )

This function ungates the eDMA clock and configures the eDMA peripheral according to the configuration structure.

## Parameters

|               |                                                                |
|---------------|----------------------------------------------------------------|
| <i>base</i>   | eDMA peripheral base address.                                  |
| <i>config</i> | A pointer to the configuration structure, see "edma_config_t". |

## Note

This function enables the minor loop map feature.

### 16.7.2 void EDMA\_Deinit ( DMA\_Type \* *base* )

This function gates the eDMA clock.

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | eDMA peripheral base address. |
|-------------|-------------------------------|

### 16.7.3 void EDMA\_InstallTCD ( DMA\_Type \* *base*, uint32\_t *channel*, edma\_tcd\_t \* *tcd* )

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | EDMA peripheral base address. |
| <i>channel</i> | EDMA channel number.          |
| <i>tcd</i>     | Point to TCD structure.       |

### 16.7.4 void EDMA\_GetDefaultConfig ( edma\_config\_t \* *config* )

This function sets the configuration structure to default values. The default configuration is set to the following values.

```
* config.enableContinuousLinkMode = false;
* config.enableHaltOnError = true;
* config.enableRoundRobinArbitration = false;
* config.enableDebugMode = false;
*
```

## Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>config</i> | A pointer to the eDMA configuration structure. |
|---------------|------------------------------------------------|

### 16.7.5 static void EDMA\_EnableContinuousChannelLinkMode ( DMA\_Type \* *base*, bool *enable* ) [inline], [static]

## Note

Do not use continuous link mode with a channel linking to itself if there is only one minor loop iteration per service request, for example, if the channel's NBYTES value is the same as either the source or destination size. The same data transfer profile can be achieved by simply increasing the NBYTES value, which provides more efficient, faster processing.

## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | EDMA peripheral base address.     |
| <i>enable</i> | true is enable, false is disable. |

### 16.7.6 static void EDMA\_EnableMinorLoopMapping ( DMA\_Type \* *base*, bool *enable* ) [inline], [static]

The TCDn.word2 is redefined to include individual enable fields, an offset field, and the NBYTES field.

## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | EDMA peripheral base address.     |
| <i>enable</i> | true is enable, false is disable. |

### 16.7.7 void EDMA\_ResetChannel ( DMA\_Type \* *base*, uint32\_t *channel* )

This function sets TCD registers for this channel to default values.

## Parameters

---

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | eDMA peripheral base address. |
| <i>channel</i> | eDMA channel number.          |

## Note

This function must not be called while the channel transfer is ongoing or it causes unpredictable results.

This function enables the auto stop request feature.

### 16.7.8 void EDMA\_SetTransferConfig ( DMA\_Type \* *base*, uint32\_t *channel*, const edma\_transfer\_config\_t \* *config*, edma\_tcd\_t \* *nextTcd* )

This function configures the transfer attribute, including source address, destination address, transfer size, address offset, and so on. It also configures the scatter gather feature if the user supplies the TCD address.

Example:

```
* edma_transfer_t config;
* edma_tcd_t tcd;
* config.srcAddr = ..;
* config.destAddr = ..;
* ...
* EDMA_SetTransferConfig(DMA0, channel, &config, &tcd);
*
```

## Parameters

|                |                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                                                                 |
| <i>channel</i> | eDMA channel number.                                                                          |
| <i>config</i>  | Pointer to eDMA transfer configuration structure.                                             |
| <i>nextTcd</i> | Point to TCD structure. It can be NULL if users do not want to enable scatter/gather feature. |

## Note

If nextTcd is not NULL, it means scatter gather feature is enabled and DREQ bit is cleared in the previous transfer configuration, which is set in the EDMA\_ResetChannel.

### 16.7.9 void EDMA\_SetMinorOffsetConfig ( DMA\_Type \* *base*, uint32\_t *channel*, const edma\_minor\_offset\_config\_t \* *config* )

The minor offset means that the signed-extended value is added to the source address or destination address after each minor loop.

## Parameters

|                |                                                        |
|----------------|--------------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                          |
| <i>channel</i> | eDMA channel number.                                   |
| <i>config</i>  | A pointer to the minor offset configuration structure. |

### 16.7.10 void EDMA\_SetChannelPreemptionConfig ( DMA\_Type \* *base*, uint32\_t *channel*, const edma\_channel\_Preemption\_config\_t \* *config* )

This function configures the channel preemption attribute and the priority of the channel.

## Parameters

|                |                                                              |
|----------------|--------------------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                                |
| <i>channel</i> | eDMA channel number                                          |
| <i>config</i>  | A pointer to the channel preemption configuration structure. |

### 16.7.11 void EDMA\_SetChannelLink ( DMA\_Type \* *base*, uint32\_t *channel*, edma\_channel\_link\_type\_t *linkType*, uint32\_t *linkedChannel* )

This function configures either the minor link or the major link mode. The minor link means that the channel link is triggered every time CITER decreases by 1. The major link means that the channel link is triggered when the CITER is exhausted.

## Parameters

|                 |                                                                                                                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>     | eDMA peripheral base address.                                                                                                                                                    |
| <i>channel</i>  | eDMA channel number.                                                                                                                                                             |
| <i>linkType</i> | A channel link type, which can be one of the following: <ul style="list-style-type: none"> <li>• kEDMA_LinkNone</li> <li>• kEDMA_MinorLink</li> <li>• kEDMA_MajorLink</li> </ul> |

|                      |                            |
|----------------------|----------------------------|
| <i>linkedChannel</i> | The linked channel number. |
|----------------------|----------------------------|

## Note

Users should ensure that DONE flag is cleared before calling this interface, or the configuration is invalid.

### 16.7.12 void EDMA\_SetBandWidth ( DMA\_Type \* *base*, uint32\_t *channel*, edma\_bandwidth\_t *bandWidth* )

Because the eDMA processes the minor loop, it continuously generates read/write sequences until the minor count is exhausted. The bandwidth forces the eDMA to stall after the completion of each read/write access to control the bus request bandwidth seen by the crossbar switch.

## Parameters

|                  |                                                                                                                                                                                                               |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>      | eDMA peripheral base address.                                                                                                                                                                                 |
| <i>channel</i>   | eDMA channel number.                                                                                                                                                                                          |
| <i>bandWidth</i> | A bandwidth setting, which can be one of the following: <ul style="list-style-type: none"> <li>• kEDMABandwidthStallNone</li> <li>• kEDMABandwidthStall4Cycle</li> <li>• kEDMABandwidthStall8Cycle</li> </ul> |

### 16.7.13 void EDMA\_SetModulo ( DMA\_Type \* *base*, uint32\_t *channel*, edma\_modulo\_t *srcModulo*, edma\_modulo\_t *destModulo* )

This function defines a specific address range specified to be the value after (SADDR + SOFF)/(DADDR + DOFF) calculation is performed or the original register value. It provides the ability to implement a circular data queue easily.

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | eDMA peripheral base address. |
| <i>channel</i> | eDMA channel number.          |

|                   |                             |
|-------------------|-----------------------------|
| <i>srcModulo</i>  | A source modulo value.      |
| <i>destModulo</i> | A destination modulo value. |

**16.7.14 static void EDMA\_EnableAsyncRequest ( DMA\_Type \* *base*, uint32\_t *channel*, bool *enable* ) [inline], [static]**

Parameters

|                |                                                  |
|----------------|--------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                    |
| <i>channel</i> | eDMA channel number.                             |
| <i>enable</i>  | The command to enable (true) or disable (false). |

**16.7.15 static void EDMA\_EnableAutoStopRequest ( DMA\_Type \* *base*, uint32\_t *channel*, bool *enable* ) [inline], [static]**

If enabling the auto stop request, the eDMA hardware automatically disables the hardware channel request.

Parameters

|                |                                                  |
|----------------|--------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                    |
| <i>channel</i> | eDMA channel number.                             |
| <i>enable</i>  | The command to enable (true) or disable (false). |

**16.7.16 void EDMA\_EnableChannelInterrupts ( DMA\_Type \* *base*, uint32\_t *channel*, uint32\_t *mask* )**

Parameters

|                |                                                                                                     |
|----------------|-----------------------------------------------------------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                                                                       |
| <i>channel</i> | eDMA channel number.                                                                                |
| <i>mask</i>    | The mask of interrupt source to be set. Users need to use the defined edma_interrupt_enable_t type. |

**16.7.17 void EDMA\_DisableChannelInterrupts ( DMA\_Type \* *base*, uint32\_t *channel*, uint32\_t *mask* )**

## Parameters

|                |                                                                                                        |
|----------------|--------------------------------------------------------------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                                                                          |
| <i>channel</i> | eDMA channel number.                                                                                   |
| <i>mask</i>    | The mask of the interrupt source to be set. Use the defined <code>edma_interrupt_enable_t</code> type. |

### 16.7.18 void EDMA\_SetMajorOffsetConfig ( DMA\_Type \* *base*, uint32\_t *channel*, int32\_t *sourceOffset*, int32\_t *destOffset* )

Adjustment value added to the source address at the completion of the major iteration count

## Parameters

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| <i>base</i>         | eDMA peripheral base address.                                                       |
| <i>channel</i>      | edma channel number.                                                                |
| <i>sourceOffset</i> | source address offset will be applied to source address after major loop done.      |
| <i>destOffset</i>   | destination address offset will be applied to source address after major loop done. |

### 16.7.19 void EDMA\_TcdReset ( edma\_tcd\_t \* *tcd* )

This function sets all fields for this TCD structure to default value.

## Parameters

|            |                               |
|------------|-------------------------------|
| <i>tcd</i> | Pointer to the TCD structure. |
|------------|-------------------------------|

## Note

This function enables the auto stop request feature.

### 16.7.20 void EDMA\_TcdSetTransferConfig ( edma\_tcd\_t \* *tcd*, const edma\_transfer\_config\_t \* *config*, edma\_tcd\_t \* *nextTcd* )

The TCD is a transfer control descriptor. The content of the TCD is the same as the hardware TCD registers. The TCD is used in the scatter-gather mode. This function configures the TCD transfer attribute, including source address, destination address, transfer size, address offset, and so on. It also configures the scatter gather feature if the user supplies the next TCD address. Example:



```

* edma_transfer_t config = {
* ...
* }
* edma_tcd_t tcd __aligned(32);
* edma_tcd_t nextTcd __aligned(32);
* EDMA_TcdSetTransferConfig(&tcd, &config, &nextTcd);
*

```

## Parameters

|                |                                                                                                          |
|----------------|----------------------------------------------------------------------------------------------------------|
| <i>tcd</i>     | Pointer to the TCD structure.                                                                            |
| <i>config</i>  | Pointer to eDMA transfer configuration structure.                                                        |
| <i>nextTcd</i> | Pointer to the next TCD structure. It can be NULL if users do not want to enable scatter/gather feature. |

## Note

TCD address should be 32 bytes aligned or it causes an eDMA error.

If the nextTcd is not NULL, the scatter gather feature is enabled and DREQ bit is cleared in the previous transfer configuration, which is set in the EDMA\_TcdReset.

### 16.7.21 void EDMA\_TcdSetMinorOffsetConfig ( edma\_tcd\_t \* *tcd*, const edma\_minor\_offset\_config\_t \* *config* )

A minor offset is a signed-extended value added to the source address or a destination address after each minor loop.

## Parameters

|               |                                                        |
|---------------|--------------------------------------------------------|
| <i>tcd</i>    | A point to the TCD structure.                          |
| <i>config</i> | A pointer to the minor offset configuration structure. |

### 16.7.22 void EDMA\_TcdSetChannelLink ( edma\_tcd\_t \* *tcd*, edma\_channel\_link\_type\_t *linkType*, uint32\_t *linkedChannel* )

This function configures either a minor link or a major link. The minor link means the channel link is triggered every time CITER decreases by 1. The major link means that the channel link is triggered when the CITER is exhausted.

## Note

Users should ensure that DONE flag is cleared before calling this interface, or the configuration is invalid.

## Parameters

|                      |                                                                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>tcd</i>           | Point to the TCD structure.                                                                                                                                   |
| <i>linkType</i>      | Channel link type, it can be one of: <ul style="list-style-type: none"> <li>• kEDMA_LinkNone</li> <li>• kEDMA_MinorLink</li> <li>• kEDMA_MajorLink</li> </ul> |
| <i>linkedChannel</i> | The linked channel number.                                                                                                                                    |

### 16.7.23 static void EDMA\_TcdSetBandWidth ( edma\_tcd\_t \* *tcd*, edma\_bandwidth\_t *bandWidth* ) [inline], [static]

Because the eDMA processes the minor loop, it continuously generates read/write sequences until the minor count is exhausted. The bandwidth forces the eDMA to stall after the completion of each read/write access to control the bus request bandwidth seen by the crossbar switch.

## Parameters

|                  |                                                                                                                                                                                                               |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>tcd</i>       | A pointer to the TCD structure.                                                                                                                                                                               |
| <i>bandWidth</i> | A bandwidth setting, which can be one of the following: <ul style="list-style-type: none"> <li>• kEDMABandwidthStallNone</li> <li>• kEDMABandwidthStall4Cycle</li> <li>• kEDMABandwidthStall8Cycle</li> </ul> |

### 16.7.24 void EDMA\_TcdSetModulo ( edma\_tcd\_t \* *tcd*, edma\_modulo\_t *srcModulo*, edma\_modulo\_t *destModulo* )

This function defines a specific address range specified to be the value after (SADDR + SOFF)/(DADDR + DOFF) calculation is performed or the original register value. It provides the ability to implement a circular data queue easily.

## Parameters

|            |                                 |
|------------|---------------------------------|
| <i>tcd</i> | A pointer to the TCD structure. |
|------------|---------------------------------|

|                   |                             |
|-------------------|-----------------------------|
| <i>srcModulo</i>  | A source modulo value.      |
| <i>destModulo</i> | A destination modulo value. |

#### 16.7.25 static void EDMA\_TcdEnableAutoStopRequest ( edma\_tcd\_t \* *tcd*, bool *enable* ) [inline], [static]

If enabling the auto stop request, the eDMA hardware automatically disables the hardware channel request.

Parameters

|               |                                                  |
|---------------|--------------------------------------------------|
| <i>tcd</i>    | A pointer to the TCD structure.                  |
| <i>enable</i> | The command to enable (true) or disable (false). |

#### 16.7.26 void EDMA\_TcdEnableInterrupts ( edma\_tcd\_t \* *tcd*, uint32\_t *mask* )

Parameters

|             |                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------|
| <i>tcd</i>  | Point to the TCD structure.                                                                         |
| <i>mask</i> | The mask of interrupt source to be set. Users need to use the defined edma_interrupt_enable_t type. |

#### 16.7.27 void EDMA\_TcdDisableInterrupts ( edma\_tcd\_t \* *tcd*, uint32\_t *mask* )

Parameters

|             |                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------|
| <i>tcd</i>  | Point to the TCD structure.                                                                         |
| <i>mask</i> | The mask of interrupt source to be set. Users need to use the defined edma_interrupt_enable_t type. |

#### 16.7.28 void EDMA\_TcdSetMajorOffsetConfig ( edma\_tcd\_t \* *tcd*, int32\_t *sourceOffset*, int32\_t *destOffset* )

Adjustment value added to the source address at the completion of the major iteration count

## Parameters

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| <i>tcd</i>          | A point to the TCD structure.                                                       |
| <i>sourceOffset</i> | source address offset will be applied to source address after major loop done.      |
| <i>destOffset</i>   | destination address offset will be applied to source address after major loop done. |

### 16.7.29 static void EDMA\_EnableChannelRequest ( DMA\_Type \* *base*, uint32\_t *channel* ) [inline], [static]

This function enables the hardware channel request.

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | eDMA peripheral base address. |
| <i>channel</i> | eDMA channel number.          |

### 16.7.30 static void EDMA\_DisableChannelRequest ( DMA\_Type \* *base*, uint32\_t *channel* ) [inline], [static]

This function disables the hardware channel request.

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | eDMA peripheral base address. |
| <i>channel</i> | eDMA channel number.          |

### 16.7.31 static void EDMA\_TriggerChannelStart ( DMA\_Type \* *base*, uint32\_t *channel* ) [inline], [static]

This function starts a minor loop transfer.

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | eDMA peripheral base address. |
| <i>channel</i> | eDMA channel number.          |

### 16.7.32 `uint32_t` EDMA\_GetRemainingMajorLoopCount ( `DMA_Type` \* *base*, `uint32_t` *channel* )

This function checks the TCD (Task Control Descriptor) status for a specified eDMA channel and returns the number of major loop count that has not finished.

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | eDMA peripheral base address. |
| <i>channel</i> | eDMA channel number.          |

## Returns

Major loop count which has not been transferred yet for the current TCD.

## Note

1. This function can only be used to get unfinished major loop count of transfer without the next TCD, or it might be inaccuracy.
1. The unfinished/remaining transfer bytes cannot be obtained directly from registers while the channel is running. Because to calculate the remaining bytes, the initial NBYTES configured in DMA\_TCDn\_NBYTES\_MLNO register is needed while the eDMA IP does not support getting it while a channel is active. In another word, the NBYTES value reading is always the actual (decrementing) NBYTES value the dma\_engine is working with while a channel is running. Consequently, to get the remaining transfer bytes, a software-saved initial value of NBYTES (for example copied before enabling the channel) is needed. The formula to calculate it is shown below:  $\text{RemainingBytes} = \text{RemainingMajorLoopCount} * \text{NBYTES}(\text{initially configured})$

### 16.7.33 static uint32\_t EDMA\_GetErrorStatusFlags ( DMA\_Type \* *base* ) [inline], [static]

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | eDMA peripheral base address. |
|-------------|-------------------------------|

## Returns

The mask of error status flags. Users need to use the \_edma\_error\_status\_flags type to decode the return variables.

### 16.7.34 uint32\_t EDMA\_GetChannelStatusFlags ( DMA\_Type \* *base*, uint32\_t *channel* )

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | eDMA peripheral base address. |
| <i>channel</i> | eDMA channel number.          |

## Returns

The mask of channel status flags. Users need to use the `_edma_channel_status_flags` type to decode the return variables.

### 16.7.35 void EDMA\_ClearChannelStatusFlags ( DMA\_Type \* *base*, uint32\_t *channel*, uint32\_t *mask* )

## Parameters

|                |                                                                                                                       |
|----------------|-----------------------------------------------------------------------------------------------------------------------|
| <i>base</i>    | eDMA peripheral base address.                                                                                         |
| <i>channel</i> | eDMA channel number.                                                                                                  |
| <i>mask</i>    | The mask of channel status to be cleared. Users need to use the defined <code>_edma_channel_status_flags</code> type. |

### 16.7.36 void EDMA\_CreateHandle ( edma\_handle\_t \* *handle*, DMA\_Type \* *base*, uint32\_t *channel* )

This function is called if using the transactional API for eDMA. This function initializes the internal state of the eDMA handle.

## Parameters

|                |                                                                               |
|----------------|-------------------------------------------------------------------------------|
| <i>handle</i>  | eDMA handle pointer. The eDMA handle stores callback function and parameters. |
| <i>base</i>    | eDMA peripheral base address.                                                 |
| <i>channel</i> | eDMA channel number.                                                          |

### 16.7.37 void EDMA\_InstallTCDMemory ( edma\_handle\_t \* *handle*, edma\_tcd\_t \* *tcdPool*, uint32\_t *tcdSize* )

This function is called after the `EDMA_CreateHandle` to use scatter/gather feature. This function shall only be used while users need to use scatter gather mode. Scatter gather mode enables EDMA to load a new transfer control block (tcd) in hardware, and automatically reconfigure that DMA channel for a

new transfer. Users need to prepare tcd memory and also configure tcDs using interface EDMA\_Submit-Transfer.



## Parameters

|                |                                                           |
|----------------|-----------------------------------------------------------|
| <i>handle</i>  | eDMA handle pointer.                                      |
| <i>tcdPool</i> | A memory pool to store TCDs. It must be 32 bytes aligned. |
| <i>tcdSize</i> | The number of TCD slots.                                  |

### 16.7.38 void EDMA\_SetCallback ( edma\_handle\_t \* *handle*, edma\_callback *callback*, void \* *userData* )

This callback is called in the eDMA IRQ handler. Use the callback to do something after the current major loop transfer completes. This function will be called every time one tcd finished transfer.

## Parameters

|                 |                                        |
|-----------------|----------------------------------------|
| <i>handle</i>   | eDMA handle pointer.                   |
| <i>callback</i> | eDMA callback function pointer.        |
| <i>userData</i> | A parameter for the callback function. |

### 16.7.39 void EDMA\_PrepareTransferConfig ( edma\_transfer\_config\_t \* *config*, void \* *srcAddr*, uint32\_t *srcWidth*, int16\_t *srcOffset*, void \* *destAddr*, uint32\_t *destWidth*, int16\_t *destOffset*, uint32\_t *bytesEachRequest*, uint32\_t *transferBytes* )

This function prepares the transfer configuration structure according to the user input.

## Parameters

|                         |                                                           |
|-------------------------|-----------------------------------------------------------|
| <i>config</i>           | The user configuration structure of type edma_transfer_t. |
| <i>srcAddr</i>          | eDMA transfer source address.                             |
| <i>srcWidth</i>         | eDMA transfer source address width(bytes).                |
| <i>srcOffset</i>        | source address offset.                                    |
| <i>destAddr</i>         | eDMA transfer destination address.                        |
| <i>destWidth</i>        | eDMA transfer destination address width(bytes).           |
| <i>destOffset</i>       | destination address offset.                               |
| <i>bytesEachRequest</i> | eDMA transfer bytes per channel request.                  |
| <i>transferBytes</i>    | eDMA transfer bytes to be transferred.                    |

## Note

The data address and the data width must be consistent. For example, if the SRC is 4 bytes, the source address must be 4 bytes aligned, or it results in source address error (SAE).

**16.7.40 void EDMA\_PrepareTransfer ( edma\_transfer\_config\_t \* *config*, void \* *srcAddr*, uint32\_t *srcWidth*, void \* *destAddr*, uint32\_t *destWidth*, uint32\_t *bytesEachRequest*, uint32\_t *transferBytes*, edma\_transfer\_type\_t *transferType* )**

This function prepares the transfer configuration structure according to the user input.

## Parameters

|                         |                                                           |
|-------------------------|-----------------------------------------------------------|
| <i>config</i>           | The user configuration structure of type edma_transfer_t. |
| <i>srcAddr</i>          | eDMA transfer source address.                             |
| <i>srcWidth</i>         | eDMA transfer source address width(bytes).                |
| <i>destAddr</i>         | eDMA transfer destination address.                        |
| <i>destWidth</i>        | eDMA transfer destination address width(bytes).           |
| <i>bytesEachRequest</i> | eDMA transfer bytes per channel request.                  |
| <i>transferBytes</i>    | eDMA transfer bytes to be transferred.                    |
| <i>transferType</i>     | eDMA transfer type.                                       |

## Note

The data address and the data width must be consistent. For example, if the SRC is 4 bytes, the source address must be 4 bytes aligned, or it results in source address error (SAE).

**16.7.41 status\_t EDMA\_SubmitTransfer ( edma\_handle\_t \* *handle*, const edma\_transfer\_config\_t \* *config* )**

This function submits the eDMA transfer request according to the transfer configuration structure. In scatter gather mode, call this function will add a configured tcd to the circular list of tcd pool. The tcd pools is setup by call function EDMA\_InstallTCDMemory before.

## Parameters

|               |                                                   |
|---------------|---------------------------------------------------|
| <i>handle</i> | eDMA handle pointer.                              |
| <i>config</i> | Pointer to eDMA transfer configuration structure. |

## Return values

|                                |                                                                     |
|--------------------------------|---------------------------------------------------------------------|
| <i>kStatus_EDMA_Success</i>    | It means submit transfer request succeed.                           |
| <i>kStatus_EDMA_Queue-Full</i> | It means TCD queue is full. Submit transfer request is not allowed. |
| <i>kStatus_EDMA_Busy</i>       | It means the given channel is busy, need to submit request later.   |

**16.7.42 void EDMA\_StartTransfer ( edma\_handle\_t \* *handle* )**

This function enables the channel request. Users can call this function after submitting the transfer request or before submitting the transfer request.

## Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | eDMA handle pointer. |
|---------------|----------------------|

**16.7.43 void EDMA\_StopTransfer ( edma\_handle\_t \* *handle* )**

This function disables the channel request to pause the transfer. Users can call [EDMA\\_StartTransfer\(\)](#) again to resume the transfer.

## Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | eDMA handle pointer. |
|---------------|----------------------|

**16.7.44 void EDMA\_AbortTransfer ( edma\_handle\_t \* *handle* )**

This function disables the channel request and clear transfer status bits. Users can submit another transfer after calling this API.

## Parameters

|               |                     |
|---------------|---------------------|
| <i>handle</i> | DMA handle pointer. |
|---------------|---------------------|

#### 16.7.45 static uint32\_t EDMA\_GetUnusedTCDNumber ( edma\_handle\_t \* *handle* ) [inline], [static]

This function gets current tcd index which is run. If the TCD pool pointer is NULL, it will return 0.

## Parameters

|               |                     |
|---------------|---------------------|
| <i>handle</i> | DMA handle pointer. |
|---------------|---------------------|

## Returns

The unused tcd slot number.

#### 16.7.46 static uint32\_t EDMA\_GetNextTCDAddress ( edma\_handle\_t \* *handle* ) [inline], [static]

This function gets the next tcd address. If this is last TCD, return 0.

## Parameters

|               |                     |
|---------------|---------------------|
| <i>handle</i> | DMA handle pointer. |
|---------------|---------------------|

## Returns

The next TCD address.

#### 16.7.47 void EDMA\_HandleIRQ ( edma\_handle\_t \* *handle* )

This function clears the channel major interrupt flag and calls the callback function if it is not NULL.

Note: For the case using TCD queue, when the major iteration count is exhausted, additional operations are performed. These include the final address adjustments and reloading of the BITER field into the CITER. Assertion of an optional interrupt request also occurs at this time, as does a possible fetch of a new TCD from memory using the scatter/gather address pointer included in the descriptor (if scatter/gather is enabled).

For instance, when the time interrupt of TCD[0] happens, the TCD[1] has already been loaded into the eDMA engine. As `sga` and `sga_index` are calculated based on the `DLAST_SGA` bitfield lies in the `TCD_CSR` register, the `sga_index` in this case should be 2 (`DLAST_SGA` of TCD[1] stores the address of TCD[2]). Thus, the "tcdUsed" updated should be (`tcdUsed - 2U`) which indicates the number of TCDs can be loaded in the memory pool (because TCD[0] and TCD[1] have been loaded into the eDMA engine at this point already.).

For the last two continuous ISRs in a scatter/gather process, they both load the last TCD (The last ISR does not load a new TCD) from the memory pool to the eDMA engine when major loop completes. Therefore, ensure that the header and `tcdUsed` updated are identical for them. `tcdUsed` are both 0 in this case as no TCD to be loaded.

See the "eDMA basic data flow" in the eDMA Functional description section of the Reference Manual for further details.

#### Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | eDMA handle pointer. |
|---------------|----------------------|

## Chapter 17

# EWM: External Watchdog Monitor Driver

### 17.1 Overview

The MCUXpresso SDK provides a peripheral driver for the External Watchdog (EWM) Driver module of MCUXpresso SDK devices.

### 17.2 Typical use case

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/ewm`

### Data Structures

- struct `_ewm_config`  
*Describes EWM clock source. [More...](#)*

### Typedefs

- typedef struct `_ewm_config` `ewm_config_t`  
*Describes EWM clock source.*

### Enumerations

- enum `_ewm_interrupt_enable_t` { `kEWM_InterruptEnable` = `EWM_CTRL_INTEN_MASK` }  
*EWM interrupt configuration structure with default settings all disabled.*
- enum `_ewm_status_flags_t` { `kEWM_RunningFlag` = `EWM_CTRL_EWMEN_MASK` }  
*EWM status flags.*

### Driver version

- #define `FSL_EWM_DRIVER_VERSION` (`MAKE_VERSION`(2, 0, 3))  
*EWM driver version 2.0.3.*

### EWM initialization and de-initialization

- void `EWM_Init` (`EWM_Type` \*base, const `ewm_config_t` \*config)  
*Initializes the EWM peripheral.*
- void `EWM_Deinit` (`EWM_Type` \*base)  
*Deinitializes the EWM peripheral.*
- void `EWM_GetDefaultConfig` (`ewm_config_t` \*config)  
*Initializes the EWM configuration structure.*

## EWM functional Operation

- static void [EWM\\_EnableInterrupts](#) (EWM\_Type \*base, uint32\_t mask)  
*Enables the EWM interrupt.*
- static void [EWM\\_DisableInterrupts](#) (EWM\_Type \*base, uint32\_t mask)  
*Disables the EWM interrupt.*
- static uint32\_t [EWM\\_GetStatusFlags](#) (EWM\_Type \*base)  
*Gets all status flags.*
- void [EWM\\_Refresh](#) (EWM\_Type \*base)  
*Services the EWM.*

## 17.3 Data Structure Documentation

### 17.3.1 struct \_ewm\_config

Data structure for EWM configuration.

This structure is used to configure the EWM.

#### Data Fields

- bool [enableEwm](#)  
*Enable EWM module.*
- bool [enableEwmInput](#)  
*Enable EWM\_in input.*
- bool [setInputAssertLogic](#)  
*EWM\_in signal assertion state.*
- bool [enableInterrupt](#)  
*Enable EWM interrupt.*
- uint8\_t [prescaler](#)  
*Clock prescaler value.*
- uint8\_t [compareLowValue](#)  
*Compare low-register value.*
- uint8\_t [compareHighValue](#)  
*Compare high-register value.*

## 17.4 Macro Definition Documentation

### 17.4.1 #define FSL\_EWM\_DRIVER\_VERSION (MAKE\_VERSION(2, 0, 3))

## 17.5 Typedef Documentation

### 17.5.1 typedef struct \_ewm\_config ewm\_config\_t

Data structure for EWM configuration.

This structure is used to configure the EWM.

## 17.6 Enumeration Type Documentation

### 17.6.1 enum \_ewm\_interrupt\_enable\_t

This structure contains the settings for all of EWM interrupt configurations.

Enumerator

***kEWM\_InterruptEnable*** Enable the EWM to generate an interrupt.

### 17.6.2 enum \_ewm\_status\_flags\_t

This structure contains the constants for the EWM status flags for use in the EWM functions.

Enumerator

***kEWM\_RunningFlag*** Running flag, set when EWM is enabled.

## 17.7 Function Documentation

### 17.7.1 void EWM\_Init ( EWM\_Type \* *base*, const ewm\_config\_t \* *config* )

This function is used to initialize the EWM. After calling, the EWM runs immediately according to the configuration. Note that, except for the interrupt enable control bit, other control bits and registers are write once after a CPU reset. Modifying them more than once generates a bus transfer error.

This is an example.

```
* ewm_config_t config;
* EWM_GetDefaultConfig(&config);
* config.compareHighValue = 0xAAU;
* EWM_Init(ewm_base, &config);
*
```

Parameters

|               |                              |
|---------------|------------------------------|
| <i>base</i>   | EWM peripheral base address  |
| <i>config</i> | The configuration of the EWM |

### 17.7.2 void EWM\_Deinit ( EWM\_Type \* *base* )

This function is used to shut down the EWM.



## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | EWM peripheral base address |
|-------------|-----------------------------|

**17.7.3 void EWM\_GetDefaultConfig ( ewm\_config\_t \* *config* )**

This function initializes the EWM configuration structure to default values. The default values are as follows.

```
* ewmConfig->enableEwm = true;
* ewmConfig->enableEwmInput = false;
* ewmConfig->setInputAssertLogic = false;
* ewmConfig->enableInterrupt = false;
* ewmConfig->ewm_lpo_clock_source_t = kEWM_LpoClockSource0;
* ewmConfig->prescaler = 0;
* ewmConfig->compareLowValue = 0;
* ewmConfig->compareHighValue = 0xFEU;
*
```

## Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Pointer to the EWM configuration structure. |
|---------------|---------------------------------------------|

See Also

[ewm\\_config\\_t](#)

**17.7.4 static void EWM\_EnableInterrupts ( EWM\_Type \* *base*, uint32\_t *mask* )  
[inline], [static]**

This function enables the EWM interrupt.

## Parameters

|             |                                                                                                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | EWM peripheral base address                                                                                                                                           |
| <i>mask</i> | The interrupts to enable The parameter can be combination of the following source if defined <ul style="list-style-type: none"> <li>• kEWM_InterruptEnable</li> </ul> |

**17.7.5 static void EWM\_DisableInterrupts ( EWM\_Type \* *base*, uint32\_t *mask* )  
[inline], [static]**

This function enables the EWM interrupt.

## Parameters

|             |                                                                                                                                                                        |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | EWM peripheral base address                                                                                                                                            |
| <i>mask</i> | The interrupts to disable The parameter can be combination of the following source if defined <ul style="list-style-type: none"> <li>• kEWM_InterruptEnable</li> </ul> |

### 17.7.6 static uint32\_t EWM\_GetStatusFlags ( EWM\_Type \* *base* ) [inline], [static]

This function gets all status flags.

This is an example for getting the running flag.

```
* uint32_t status;
* status = EWM_GetStatusFlags(ewm_base) & kEWM_RunningFlag;
*
```

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | EWM peripheral base address |
|-------------|-----------------------------|

## Returns

State of the status flag: asserted (true) or not-asserted (false).

## See Also

[\\_ewm\\_status\\_flags\\_t](#)

- True: a related status flag has been set.
- False: a related status flag is not set.

### 17.7.7 void EWM\_Refresh ( EWM\_Type \* *base* )

This function resets the EWM counter to zero.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | EWM peripheral base address |
|-------------|-----------------------------|

# Chapter 18

## FlexIO: FlexIO Driver

### 18.1 Overview

The MCUXpresso SDK provides a generic driver and multiple protocol-specific FlexIO drivers for the FlexIO module of MCUXpresso SDK devices.

#### Modules

- [FlexIO Driver](#)
- [FlexIO I2C Master Driver](#)
- [FlexIO I2S Driver](#)
- [FlexIO SPI Driver](#)
- [FlexIO UART Driver](#)

## 18.2 FlexIO Driver

### 18.2.1 Overview

#### Data Structures

- struct [\\_flexio\\_config\\_](#)  
Define FlexIO user configuration structure. [More...](#)
- struct [\\_flexio\\_timer\\_config](#)  
Define FlexIO timer configuration structure. [More...](#)
- struct [\\_flexio\\_shifter\\_config](#)  
Define FlexIO shifter configuration structure. [More...](#)
- struct [\\_flexio\\_gpio\\_config](#)  
The FLEXIO pin configuration structure. [More...](#)

#### Macros

- #define [FLEXIO\\_TIMER\\_TRIGGER\\_SEL\\_PININPUT\(x\)](#) ((uint32\_t)(x) << 1U)  
Calculate FlexIO timer trigger.

#### Typedefs

- typedef enum [\\_flexio\\_timer\\_trigger\\_polarity flexio\\_timer\\_trigger\\_polarity\\_t](#)  
Define time of timer trigger polarity.
- typedef enum [\\_flexio\\_timer\\_trigger\\_source flexio\\_timer\\_trigger\\_source\\_t](#)  
Define type of timer trigger source.
- typedef enum [\\_flexio\\_pin\\_config flexio\\_pin\\_config\\_t](#)  
Define type of timer/shifter pin configuration.
- typedef enum [\\_flexio\\_pin\\_polarity flexio\\_pin\\_polarity\\_t](#)  
Definition of pin polarity.
- typedef enum [\\_flexio\\_timer\\_mode flexio\\_timer\\_mode\\_t](#)  
Define type of timer work mode.
- typedef enum [\\_flexio\\_timer\\_output flexio\\_timer\\_output\\_t](#)  
Define type of timer initial output or timer reset condition.
- typedef enum [\\_flexio\\_timer\\_decrement\\_source flexio\\_timer\\_decrement\\_source\\_t](#)  
Define type of timer decrement.
- typedef enum [\\_flexio\\_timer\\_reset\\_condition flexio\\_timer\\_reset\\_condition\\_t](#)  
Define type of timer reset condition.
- typedef enum [\\_flexio\\_timer\\_disable\\_condition flexio\\_timer\\_disable\\_condition\\_t](#)  
Define type of timer disable condition.
- typedef enum [\\_flexio\\_timer\\_enable\\_condition flexio\\_timer\\_enable\\_condition\\_t](#)

- *Define type of timer enable condition.*  
typedef enum  
    [\\_flexio\\_timer\\_stop\\_bit\\_condition](#) flexio\_timer\_stop\_bit\_condition\_t  
    *Define type of timer stop bit generate condition.*
- typedef enum  
    [\\_flexio\\_timer\\_start\\_bit\\_condition](#) flexio\_timer\_start\_bit\_condition\_t  
    *Define type of timer start bit generate condition.*
- typedef enum  
    [\\_flexio\\_timer\\_output\\_state](#) flexio\_timer\_output\_state\_t  
    *FlexIO as PWM channel output state.*
- typedef enum  
    [\\_flexio\\_shifter\\_timer\\_polarity](#) flexio\_shifter\_timer\_polarity\_t  
    *Define type of timer polarity for shifter control.*
- typedef enum [\\_flexio\\_shifter\\_mode](#) flexio\_shifter\_mode\_t  
    *Define type of shifter working mode.*
- typedef enum  
    [\\_flexio\\_shifter\\_input\\_source](#) flexio\_shifter\_input\_source\_t  
    *Define type of shifter input source.*
- typedef enum  
    [\\_flexio\\_shifter\\_stop\\_bit](#) flexio\_shifter\_stop\_bit\_t  
    *Define of STOP bit configuration.*
- typedef enum  
    [\\_flexio\\_shifter\\_start\\_bit](#) flexio\_shifter\_start\_bit\_t  
    *Define type of START bit configuration.*
- typedef enum  
    [\\_flexio\\_shifter\\_buffer\\_type](#) flexio\_shifter\_buffer\_type\_t  
    *Define FlexIO shifter buffer type.*
- typedef struct [\\_flexio\\_config](#) flexio\_config\_t  
    *Define FlexIO user configuration structure.*
- typedef struct [\\_flexio\\_timer\\_config](#) flexio\_timer\_config\_t  
    *Define FlexIO timer configuration structure.*
- typedef struct  
    [\\_flexio\\_shifter\\_config](#) flexio\_shifter\_config\_t  
    *Define FlexIO shifter configuration structure.*
- typedef enum [\\_flexio\\_gpio\\_direction](#) flexio\_gpio\_direction\_t  
    *FLEXIO gpio direction definition.*
- typedef enum  
    [\\_flexio\\_pin\\_input\\_config](#) flexio\_pin\_input\_config\_t  
    *FLEXIO gpio input config.*
- typedef struct [\\_flexio\\_gpio\\_config](#) flexio\_gpio\_config\_t  
    *The FLEXIO pin configuration structure.*
- typedef void(\* [flexio\\_isr\\_t](#) )(void \*base, void \*handle)  
    *typedef for FlexIO simulated driver interrupt handler.*

## Enumerations

- enum [\\_flexio\\_timer\\_trigger\\_polarity](#) {  
    [kFLEXIO\\_TimerTriggerPolarityActiveHigh](#) = 0x0U,

kFLEXIO\_TimerTriggerPolarityActiveLow = 0x1U }

*Define time of timer trigger polarity.*

- enum \_flexio\_timer\_trigger\_source {  
kFLEXIO\_TimerTriggerSourceExternal = 0x0U,  
kFLEXIO\_TimerTriggerSourceInternal = 0x1U }

*Define type of timer trigger source.*

- enum \_flexio\_pin\_config {  
kFLEXIO\_PinConfigOutputDisabled = 0x0U,  
kFLEXIO\_PinConfigOpenDrainOrBidirection = 0x1U,  
kFLEXIO\_PinConfigBidirectionOutputData = 0x2U,  
kFLEXIO\_PinConfigOutput = 0x3U }

*Define type of timer/shifter pin configuration.*

- enum \_flexio\_pin\_polarity {  
kFLEXIO\_PinActiveHigh = 0x0U,  
kFLEXIO\_PinActiveLow = 0x1U }

*Definition of pin polarity.*

- enum \_flexio\_timer\_mode {  
kFLEXIO\_TimerModeDisabled = 0x0U,  
kFLEXIO\_TimerModeDual8BitBaudBit = 0x1U,  
kFLEXIO\_TimerModeDual8BitPWM = 0x2U,  
kFLEXIO\_TimerModeSingle16Bit = 0x3U }

*Define type of timer work mode.*

- enum \_flexio\_timer\_output {  
kFLEXIO\_TimerOutputOneNotAffectedByReset = 0x0U,  
kFLEXIO\_TimerOutputZeroNotAffectedByReset = 0x1U,  
kFLEXIO\_TimerOutputOneAffectedByReset = 0x2U,  
kFLEXIO\_TimerOutputZeroAffectedByReset = 0x3U }

*Define type of timer initial output or timer reset condition.*

- enum \_flexio\_timer\_decrement\_source {  
kFLEXIO\_TimerDecSrcOnFlexIOClockShiftTimerOutput = 0x0U,  
kFLEXIO\_TimerDecSrcOnTriggerInputShiftTimerOutput,  
kFLEXIO\_TimerDecSrcOnPinInputShiftPinInput,  
kFLEXIO\_TimerDecSrcOnTriggerInputShiftTriggerInput,  
kFLEXIO\_TimerDecSrcDiv16OnFlexIOClockShiftTimerOutput,  
kFLEXIO\_TimerDecSrcDiv256OnFlexIOClockShiftTimerOutput,  
kFLEXIO\_TimerRisSrcOnPinInputShiftPinInput,  
kFLEXIO\_TimerRisSrcOnTriggerInputShiftTriggerInput }

*Define type of timer decrement.*

- enum \_flexio\_timer\_reset\_condition {  
kFLEXIO\_TimerResetNever = 0x0U,  
kFLEXIO\_TimerResetOnTimerPinEqualToTimerOutput = 0x2U,  
kFLEXIO\_TimerResetOnTimerTriggerEqualToTimerOutput = 0x3U,  
kFLEXIO\_TimerResetOnTimerPinRisingEdge = 0x4U,  
kFLEXIO\_TimerResetOnTimerTriggerRisingEdge = 0x6U,  
kFLEXIO\_TimerResetOnTimerTriggerBothEdge = 0x7U }

*Define type of timer reset condition.*

- enum \_flexio\_timer\_disable\_condition {

```

kFLEXIO_TimerDisableNever = 0x0U,
kFLEXIO_TimerDisableOnPreTimerDisable = 0x1U,
kFLEXIO_TimerDisableOnTimerCompare = 0x2U,
kFLEXIO_TimerDisableOnTimerCompareTriggerLow = 0x3U,
kFLEXIO_TimerDisableOnPinBothEdge = 0x4U,
kFLEXIO_TimerDisableOnPinBothEdgeTriggerHigh = 0x5U,
kFLEXIO_TimerDisableOnTriggerFallingEdge = 0x6U }

```

*Define type of timer disable condition.*

- enum `_flexio_timer_enable_condition` {  
`kFLEXIO_TimerEnabledAlways` = 0x0U,  
`kFLEXIO_TimerEnableOnPrevTimerEnable` = 0x1U,  
`kFLEXIO_TimerEnableOnTriggerHigh` = 0x2U,  
`kFLEXIO_TimerEnableOnTriggerHighPinHigh` = 0x3U,  
`kFLEXIO_TimerEnableOnPinRisingEdge` = 0x4U,  
`kFLEXIO_TimerEnableOnPinRisingEdgeTriggerHigh` = 0x5U,  
`kFLEXIO_TimerEnableOnTriggerRisingEdge` = 0x6U,  
`kFLEXIO_TimerEnableOnTriggerBothEdge` = 0x7U }

*Define type of timer enable condition.*

- enum `_flexio_timer_stop_bit_condition` {  
`kFLEXIO_TimerStopBitDisabled` = 0x0U,  
`kFLEXIO_TimerStopBitEnableOnTimerCompare` = 0x1U,  
`kFLEXIO_TimerStopBitEnableOnTimerDisable` = 0x2U,  
`kFLEXIO_TimerStopBitEnableOnTimerCompareDisable` = 0x3U }

*Define type of timer stop bit generate condition.*

- enum `_flexio_timer_start_bit_condition` {  
`kFLEXIO_TimerStartBitDisabled` = 0x0U,  
`kFLEXIO_TimerStartBitEnabled` = 0x1U }

*Define type of timer start bit generate condition.*

- enum `_flexio_timer_output_state` {  
`kFLEXIO_PwmLow` = 0,  
`kFLEXIO_PwmHigh` }

*FlexIO as PWM channel output state.*

- enum `_flexio_shifter_timer_polarity` {  
`kFLEXIO_ShifterTimerPolarityOnPositive` = 0x0U,  
`kFLEXIO_ShifterTimerPolarityOnNegative` = 0x1U }

*Define type of timer polarity for shifter control.*

- enum `_flexio_shifter_mode` {  
`kFLEXIO_ShifterDisabled` = 0x0U,  
`kFLEXIO_ShifterModeReceive` = 0x1U,  
`kFLEXIO_ShifterModeTransmit` = 0x2U,  
`kFLEXIO_ShifterModeMatchStore` = 0x4U,  
`kFLEXIO_ShifterModeMatchContinuous` = 0x5U,  
`kFLEXIO_ShifterModeState` = 0x6U,  
`kFLEXIO_ShifterModeLogic` = 0x7U }

*Define type of shifter working mode.*

- enum `_flexio_shifter_input_source` {



```

kFLEXIO_ShifterInputFromPin = 0x0U,
kFLEXIO_ShifterInputFromNextShifterOutput = 0x1U }
 Define type of shifter input source.
• enum _flexio_shifter_stop_bit {
 kFLEXIO_ShifterStopBitDisable = 0x0U,
 kFLEXIO_ShifterStopBitLow = 0x2U,
 kFLEXIO_ShifterStopBitHigh = 0x3U }
 Define of STOP bit configuration.
• enum _flexio_shifter_start_bit {
 kFLEXIO_ShifterStartBitDisabledLoadDataOnEnable = 0x0U,
 kFLEXIO_ShifterStartBitDisabledLoadDataOnShift = 0x1U,
 kFLEXIO_ShifterStartBitLow = 0x2U,
 kFLEXIO_ShifterStartBitHigh = 0x3U }
 Define type of START bit configuration.
• enum _flexio_shifter_buffer_type {
 kFLEXIO_ShifterBuffer = 0x0U,
 kFLEXIO_ShifterBufferBitSwapped = 0x1U,
 kFLEXIO_ShifterBufferByteSwapped = 0x2U,
 kFLEXIO_ShifterBufferBitByteSwapped = 0x3U,
 kFLEXIO_ShifterBufferNibbleByteSwapped = 0x4U,
 kFLEXIO_ShifterBufferHalfWordSwapped = 0x5U,
 kFLEXIO_ShifterBufferNibbleSwapped = 0x6U }
 Define FlexIO shifter buffer type.
• enum _flexio_gpio_direction {
 kFLEXIO_DigitalInput = 0U,
 kFLEXIO_DigitalOutput = 1U }
 FLEXIO gpio direction definition.
• enum _flexio_pin_input_config {
 kFLEXIO_InputInterruptDisabled = 0x0U,
 kFLEXIO_InputInterruptEnable = 0x1U,
 kFLEXIO_FlagRisingEdgeEnable = 0x2U,
 kFLEXIO_FlagFallingEdgeEnable = 0x4U }
 FLEXIO gpio input config.

```

## Functions

- void **FLEXIO\_SetPinConfig** (FLEXIO\_Type \*base, uint32\_t pin, flexio\_gpio\_config\_t \*config)  
*Configure a FLEXIO pin used by the board.*

## Variables

- FLEXIO\_Type \*const s\_flexioBases []  
*Pointers to flexio bases for each instance.*
- const clock\_ip\_name\_t s\_flexioClocks []  
*Pointers to flexio clocks for each instance.*

## Driver version

- #define **FSL\_FLEXIO\_DRIVER\_VERSION** (**MAKE\_VERSION**(2, 2, 2))  
*FlexIO driver version.*

## FlexIO Initialization and De-initialization

- void **FLEXIO\_GetDefaultConfig** (**flexio\_config\_t** \*userConfig)  
*Gets the default configuration to configure the FlexIO module.*
- void **FLEXIO\_Init** (**FLEXIO\_Type** \*base, const **flexio\_config\_t** \*userConfig)  
*Configures the FlexIO with a FlexIO configuration.*
- void **FLEXIO\_Deinit** (**FLEXIO\_Type** \*base)  
*Gates the FlexIO clock.*
- uint32\_t **FLEXIO\_GetInstance** (**FLEXIO\_Type** \*base)  
*Get instance number for FLEXIO module.*

## FlexIO Basic Operation

- void **FLEXIO\_Reset** (**FLEXIO\_Type** \*base)  
*Resets the FlexIO module.*
- static void **FLEXIO\_Enable** (**FLEXIO\_Type** \*base, bool enable)  
*Enables the FlexIO module operation.*
- static uint32\_t **FLEXIO\_ReadPinInput** (**FLEXIO\_Type** \*base)  
*Reads the input data on each of the FlexIO pins.*
- static uint8\_t **FLEXIO\_GetShifterState** (**FLEXIO\_Type** \*base)  
*Gets the current state pointer for state mode use.*
- void **FLEXIO\_SetShifterConfig** (**FLEXIO\_Type** \*base, uint8\_t index, const **flexio\_shifter\_config\_t** \*shifterConfig)  
*Configures the shifter with the shifter configuration.*
- void **FLEXIO\_SetTimerConfig** (**FLEXIO\_Type** \*base, uint8\_t index, const **flexio\_timer\_config\_t** \*timerConfig)  
*Configures the timer with the timer configuration.*
- static void **FLEXIO\_SetClockMode** (**FLEXIO\_Type** \*base, uint8\_t index, **flexio\_timer\_decrement\_source\_t** clocksource)  
*This function set the value of the prescaler on flexio channels.*

## FlexIO Interrupt Operation

- static void **FLEXIO\_EnableShifterStatusInterrupts** (**FLEXIO\_Type** \*base, uint32\_t mask)  
*Enables the shifter status interrupt.*
- static void **FLEXIO\_DisableShifterStatusInterrupts** (**FLEXIO\_Type** \*base, uint32\_t mask)  
*Disables the shifter status interrupt.*
- static void **FLEXIO\_EnableShifterErrorInterrupts** (**FLEXIO\_Type** \*base, uint32\_t mask)  
*Enables the shifter error interrupt.*
- static void **FLEXIO\_DisableShifterErrorInterrupts** (**FLEXIO\_Type** \*base, uint32\_t mask)  
*Disables the shifter error interrupt.*

- static void [FLEXIO\\_EnableTimerStatusInterrupts](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Enables the timer status interrupt.*
- static void [FLEXIO\\_DisableTimerStatusInterrupts](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Disables the timer status interrupt.*

## FlexIO Status Operation

- static uint32\_t [FLEXIO\\_GetShifterStatusFlags](#) (FLEXIO\_Type \*base)  
*Gets the shifter status flags.*
- static void [FLEXIO\\_ClearShifterStatusFlags](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Clears the shifter status flags.*
- static uint32\_t [FLEXIO\\_GetShifterErrorFlags](#) (FLEXIO\_Type \*base)  
*Gets the shifter error flags.*
- static void [FLEXIO\\_ClearShifterErrorFlags](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Clears the shifter error flags.*
- static uint32\_t [FLEXIO\\_GetTimerStatusFlags](#) (FLEXIO\_Type \*base)  
*Gets the timer status flags.*
- static void [FLEXIO\\_ClearTimerStatusFlags](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Clears the timer status flags.*

## FlexIO DMA Operation

- static void [FLEXIO\\_EnableShifterStatusDMA](#) (FLEXIO\_Type \*base, uint32\_t mask, bool enable)  
*Enables/disables the shifter status DMA.*
- uint32\_t [FLEXIO\\_GetShifterBufferAddress](#) (FLEXIO\_Type \*base, flexio\_shifter\_buffer\_type\_t type, uint8\_t index)  
*Gets the shifter buffer address for the DMA transfer usage.*
- status\_t [FLEXIO\\_RegisterHandleIRQ](#) (void \*base, void \*handle, flexio\_isr\_t isr)  
*Registers the handle and the interrupt handler for the FlexIO-simulated peripheral.*
- status\_t [FLEXIO\\_UnregisterHandleIRQ](#) (void \*base)  
*Unregisters the handle and the interrupt handler for the FlexIO-simulated peripheral.*

## GPIO Output Operations

- static void [FLEXIO\\_ClearPortOutput](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Sets the output level of the multiple FLEXIO pins to the logic 0.*
- static void [FLEXIO\\_SetPortOutput](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Sets the output level of the multiple FLEXIO pins to the logic 1.*
- static void [FLEXIO\\_TogglePortOutput](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Reverses the current output logic of the multiple FLEXIO pins.*
- static void [FLEXIO\\_PinWrite](#) (FLEXIO\_Type \*base, uint32\_t pin, uint8\_t output)  
*Sets the output level of the FLEXIO pins to the logic 1 or 0.*
- static void [FLEXIO\\_EnablePinOutput](#) (FLEXIO\_Type \*base, uint32\_t pin)  
*Enables the FLEXIO output pin function.*

## FLEXIO PIN Input Operations

- static uint32\_t [FLEXIO\\_PinRead](#) (FLEXIO\_Type \*base, uint32\_t pin)  
*Reads the current input value of the FLEXIO pin.*
- static uint32\_t [FLEXIO\\_GetPinStatus](#) (FLEXIO\_Type \*base, uint32\_t pin)  
*Gets the FLEXIO input pin status.*
- static void [FLEXIO\\_ClearPortStatus](#) (FLEXIO\_Type \*base, uint32\_t mask)  
*Clears the multiple FLEXIO input pins status.*

## 18.2.2 Data Structure Documentation

### 18.2.2.1 struct \_flexio\_config\_

#### Data Fields

- bool [enableFlexio](#)  
*Enable/disable FlexIO module.*
- bool [enableInDoze](#)  
*Enable/disable FlexIO operation in doze mode.*
- bool [enableInDebug](#)  
*Enable/disable FlexIO operation in debug mode.*
- bool [enableFastAccess](#)  
*Enable/disable fast access to FlexIO registers, fast access requires the FlexIO clock to be at least twice the frequency of the bus clock.*

#### Field Documentation

#### (1) bool \_flexio\_config\_::enableFastAccess

### 18.2.2.2 struct \_flexio\_timer\_config

#### Data Fields

- uint32\_t [triggerSelect](#)  
*The internal trigger selection number using MACROs.*
- [flexio\\_timer\\_trigger\\_polarity\\_t](#) triggerPolarity  
*Trigger Polarity.*
- [flexio\\_timer\\_trigger\\_source\\_t](#) triggerSource  
*Trigger Source, internal (see 'trgsel') or external.*
- [flexio\\_pin\\_config\\_t](#) pinConfig  
*Timer Pin Configuration.*
- uint32\_t [pinSelect](#)  
*Timer Pin number Select.*
- [flexio\\_pin\\_polarity\\_t](#) pinPolarity  
*Timer Pin Polarity.*
- [flexio\\_timer\\_mode\\_t](#) timerMode  
*Timer work Mode.*
- [flexio\\_timer\\_output\\_t](#) timerOutput  
*Configures the initial state of the Timer Output and*

- whether it is affected by the Timer reset.*
- [flexio\\_timer\\_decrement\\_source\\_t timerDecrement](#)  
 Configures the source of the Timer decrement and the source of the Shift clock.
- [flexio\\_timer\\_reset\\_condition\\_t timerReset](#)  
 Configures the condition that causes the timer counter (and optionally the timer output) to be reset.
- [flexio\\_timer\\_disable\\_condition\\_t timerDisable](#)  
 Configures the condition that causes the Timer to be disabled and stop decrementing.
- [flexio\\_timer\\_enable\\_condition\\_t timerEnable](#)  
 Configures the condition that causes the Timer to be enabled and start decrementing.
- [flexio\\_timer\\_stop\\_bit\\_condition\\_t timerStop](#)  
 Timer STOP Bit generation.
- [flexio\\_timer\\_start\\_bit\\_condition\\_t timerStart](#)  
 Timer STRAT Bit generation.
- [uint32\\_t timerCompare](#)  
 Value for Timer Compare N Register.

## Field Documentation

- (1) `uint32_t flexio_timer_config::triggerSelect`
- (2) `flexio_timer_trigger_polarity_t flexio_timer_config::triggerPolarity`
- (3) `flexio_timer_trigger_source_t flexio_timer_config::triggerSource`
- (4) `flexio_pin_config_t flexio_timer_config::pinConfig`
- (5) `uint32_t flexio_timer_config::pinSelect`
- (6) `flexio_pin_polarity_t flexio_timer_config::pinPolarity`
- (7) `flexio_timer_mode_t flexio_timer_config::timerMode`
- (8) `flexio_timer_output_t flexio_timer_config::timerOutput`
- (9) `flexio_timer_decrement_source_t flexio_timer_config::timerDecrement`
- (10) `flexio_timer_reset_condition_t flexio_timer_config::timerReset`
- (11) `flexio_timer_disable_condition_t flexio_timer_config::timerDisable`
- (12) `flexio_timer_enable_condition_t flexio_timer_config::timerEnable`
- (13) `flexio_timer_stop_bit_condition_t flexio_timer_config::timerStop`
- (14) `flexio_timer_start_bit_condition_t flexio_timer_config::timerStart`
- (15) `uint32_t flexio_timer_config::timerCompare`

### 18.2.2.3 struct `flexio_shifter_config`

#### Data Fields

- `uint32_t timerSelect`  
Selects which Timer is used for controlling the logic/shift register and generating the Shift clock.
- `flexio_shifter_timer_polarity_t timerPolarity`  
Timer Polarity.
- `flexio_pin_config_t pinConfig`  
Shifter Pin Configuration.
- `uint32_t pinSelect`  
Shifter Pin number Select.
- `flexio_pin_polarity_t pinPolarity`  
Shifter Pin Polarity.
- `flexio_shifter_mode_t shifterMode`  
Configures the mode of the Shifter.
- `uint32_t parallelWidth`  
Configures the parallel width when using parallel mode.

- [flexio\\_shifter\\_input\\_source\\_t](#) `inputSource`  
*Selects the input source for the shifter.*
- [flexio\\_shifter\\_stop\\_bit\\_t](#) `shifterStop`  
*Shifter STOP bit.*
- [flexio\\_shifter\\_start\\_bit\\_t](#) `shifterStart`  
*Shifter START bit.*

## Field Documentation

- (1) `uint32_t flexio_shifter_config::timerSelect`
- (2) `flexio_shifter_timer_polarity_t flexio_shifter_config::timerPolarity`
- (3) `flexio_pin_config_t flexio_shifter_config::pinConfig`
- (4) `uint32_t flexio_shifter_config::pinSelect`
- (5) `flexio_pin_polarity_t flexio_shifter_config::pinPolarity`
- (6) `flexio_shifter_mode_t flexio_shifter_config::shifterMode`
- (7) `uint32_t flexio_shifter_config::parallelWidth`
- (8) `flexio_shifter_input_source_t flexio_shifter_config::inputSource`
- (9) `flexio_shifter_stop_bit_t flexio_shifter_config::shifterStop`
- (10) `flexio_shifter_start_bit_t flexio_shifter_config::shifterStart`

### 18.2.2.4 struct `flexio_gpio_config`

Each pin can only be configured as either an output pin or an input pin at a time. If configured as an input pin, use `inputConfig` param. If configured as an output pin, use `outputLogic`.

## Data Fields

- [flexio\\_gpio\\_direction\\_t](#) `pinDirection`  
*FLEXIO pin direction, input or output.*
- `uint8_t` [outputLogic](#)  
*Set a default output logic, which has no use in input.*
- `uint8_t` [inputConfig](#)  
*Set an input config.*





### 18.2.3 Macro Definition Documentation

18.2.3.1 `#define FSL_FLEXIO_DRIVER_VERSION (MAKE_VERSION(2, 2, 2))`

18.2.3.2 `#define FLEXIO_TIMER_TRIGGER_SEL_PININPUT( x ) ((uint32_t)(x) << 1U)`

### 18.2.4 Typedef Documentation

18.2.4.1 `typedef enum _flexio_timer_trigger_polarity flexio_timer_trigger_polarity_t`

18.2.4.2 `typedef enum _flexio_timer_trigger_source flexio_timer_trigger_source_t`

18.2.4.3 `typedef enum _flexio_pin_config flexio_pin_config_t`

18.2.4.4 `typedef enum _flexio_pin_polarity flexio_pin_polarity_t`

18.2.4.5 `typedef enum _flexio_timer_mode flexio_timer_mode_t`

18.2.4.6 `typedef enum _flexio_timer_output flexio_timer_output_t`

18.2.4.7 `typedef enum _flexio_timer_decrement_source flexio_timer_decrement_source_t`

18.2.4.8 `typedef enum _flexio_timer_reset_condition flexio_timer_reset_condition_t`

18.2.4.9 `typedef enum _flexio_timer_disable_condition flexio_timer_disable_condition_t`

18.2.4.10 `typedef enum _flexio_timer_enable_condition flexio_timer_enable_condition_t`

18.2.4.11 `typedef enum _flexio_timer_stop_bit_condition flexio_timer_stop_bit_condition_t`

18.2.4.12 `typedef enum _flexio_timer_start_bit_condition flexio_timer_start_bit_condition_t`

18.2.4.13 `typedef enum _flexio_shifter_timer_polarity flexio_shifter_timer_polarity_t`

18.2.4.14 `typedef enum _flexio_shifter_mode flexio_shifter_mode_t`

18.2.4.15 `typedef enum _flexio_shifter_input_source flexio_shifter_input_source_t`

18.2.4.16 `typedef enum _flexio_shifter_stop_bit flexio_shifter_stop_bit_t`

18.2.4.17 `typedef enum _flexio_shifter_start_bit flexio_shifter_start_bit_t`

18.2.4.18 `typedef struct _flexio_config flexio_config_t`

18.2.4.19 `typedef struct _flexio_timer_config flexio_timer_config_t`

18.2.4.20 `typedef struct flexio_shifter_config flexio_shifter_config_t`

**18.2.4.22** typedef void(\* flexio\_isr\_t)(void \*base, void \*handle)

## **18.2.5 Enumeration Type Documentation**

### **18.2.5.1 enum \_flexio\_timer\_trigger\_polarity**

Enumerator

***kFLEXIO\_TimerTriggerPolarityActiveHigh*** Active high.

***kFLEXIO\_TimerTriggerPolarityActiveLow*** Active low.

### **18.2.5.2 enum \_flexio\_timer\_trigger\_source**

Enumerator

***kFLEXIO\_TimerTriggerSourceExternal*** External trigger selected.

***kFLEXIO\_TimerTriggerSourceInternal*** Internal trigger selected.

### **18.2.5.3 enum \_flexio\_pin\_config**

Enumerator

***kFLEXIO\_PinConfigOutputDisabled*** Pin output disabled.

***kFLEXIO\_PinConfigOpenDrainOrBidirection*** Pin open drain or bidirectional output enable.

***kFLEXIO\_PinConfigBidirectionOutputData*** Pin bidirectional output data.

***kFLEXIO\_PinConfigOutput*** Pin output.

### **18.2.5.4 enum \_flexio\_pin\_polarity**

Enumerator

***kFLEXIO\_PinActiveHigh*** Active high.

***kFLEXIO\_PinActiveLow*** Active low.

### **18.2.5.5 enum \_flexio\_timer\_mode**

Enumerator

***kFLEXIO\_TimerModeDisabled*** Timer Disabled.

***kFLEXIO\_TimerModeDual8BitBaudBit*** Dual 8-bit counters baud/bit mode.

***kFLEXIO\_TimerModeDual8BitPWM*** Dual 8-bit counters PWM mode.

***kFLEXIO\_TimerModeSingle16Bit*** Single 16-bit counter mode.

### 18.2.5.6 enum \_flexio\_timer\_output

Enumerator

- kFLEXIO\_TimerOutputOneNotAffectedByReset*** Logic one when enabled and is not affected by timer reset.
- kFLEXIO\_TimerOutputZeroNotAffectedByReset*** Logic zero when enabled and is not affected by timer reset.
- kFLEXIO\_TimerOutputOneAffectedByReset*** Logic one when enabled and on timer reset.
- kFLEXIO\_TimerOutputZeroAffectedByReset*** Logic zero when enabled and on timer reset.

### 18.2.5.7 enum \_flexio\_timer\_decrement\_source

Enumerator

- kFLEXIO\_TimerDecSrcOnFlexIOClockShiftTimerOutput*** Decrement counter on FlexIO clock, Shift clock equals Timer output.
- kFLEXIO\_TimerDecSrcOnTriggerInputShiftTimerOutput*** Decrement counter on Trigger input (both edges), Shift clock equals Timer output.
- kFLEXIO\_TimerDecSrcOnPinInputShiftPinInput*** Decrement counter on Pin input (both edges), Shift clock equals Pin input.
- kFLEXIO\_TimerDecSrcOnTriggerInputShiftTriggerInput*** Decrement counter on Trigger input (both edges), Shift clock equals Trigger input.
- kFLEXIO\_TimerDecSrcDiv16OnFlexIOClockShiftTimerOutput*** Decrement counter on FlexIO clock divided by 16, Shift clock equals Timer output.
- kFLEXIO\_TimerDecSrcDiv256OnFlexIOClockShiftTimerOutput*** Decrement counter on FlexIO clock divided by 256, Shift clock equals Timer output.
- kFLEXIO\_TimerRisSrcOnPinInputShiftPinInput*** Decrement counter on Pin input (rising edges), Shift clock equals Pin input.
- kFLEXIO\_TimerRisSrcOnTriggerInputShiftTriggerInput*** Decrement counter on Trigger input (rising edges), Shift clock equals Trigger input.

### 18.2.5.8 enum \_flexio\_timer\_reset\_condition

Enumerator

- kFLEXIO\_TimerResetNever*** Timer never reset.
- kFLEXIO\_TimerResetOnTimerPinEqualToTimerOutput*** Timer reset on Timer Pin equal to Timer Output.
- kFLEXIO\_TimerResetOnTimerTriggerEqualToTimerOutput*** Timer reset on Timer Trigger equal to Timer Output.
- kFLEXIO\_TimerResetOnTimerPinRisingEdge*** Timer reset on Timer Pin rising edge.
- kFLEXIO\_TimerResetOnTimerTriggerRisingEdge*** Timer reset on Trigger rising edge.
- kFLEXIO\_TimerResetOnTimerTriggerBothEdge*** Timer reset on Trigger rising or falling edge.

### 18.2.5.9 enum \_flexio\_timer\_disable\_condition

Enumerator

***kFLEXIO\_TimerDisableNever*** Timer never disabled.  
***kFLEXIO\_TimerDisableOnPreTimerDisable*** Timer disabled on Timer N-1 disable.  
***kFLEXIO\_TimerDisableOnTimerCompare*** Timer disabled on Timer compare.  
***kFLEXIO\_TimerDisableOnTimerCompareTriggerLow*** Timer disabled on Timer compare and Trigger Low.  
***kFLEXIO\_TimerDisableOnPinBothEdge*** Timer disabled on Pin rising or falling edge.  
***kFLEXIO\_TimerDisableOnPinBothEdgeTriggerHigh*** Timer disabled on Pin rising or falling edge provided Trigger is high.  
***kFLEXIO\_TimerDisableOnTriggerFallingEdge*** Timer disabled on Trigger falling edge.

### 18.2.5.10 enum \_flexio\_timer\_enable\_condition

Enumerator

***kFLEXIO\_TimerEnabledAlways*** Timer always enabled.  
***kFLEXIO\_TimerEnableOnPrevTimerEnable*** Timer enabled on Timer N-1 enable.  
***kFLEXIO\_TimerEnableOnTriggerHigh*** Timer enabled on Trigger high.  
***kFLEXIO\_TimerEnableOnTriggerHighPinHigh*** Timer enabled on Trigger high and Pin high.  
***kFLEXIO\_TimerEnableOnPinRisingEdge*** Timer enabled on Pin rising edge.  
***kFLEXIO\_TimerEnableOnPinRisingEdgeTriggerHigh*** Timer enabled on Pin rising edge and Trigger high.  
***kFLEXIO\_TimerEnableOnTriggerRisingEdge*** Timer enabled on Trigger rising edge.  
***kFLEXIO\_TimerEnableOnTriggerBothEdge*** Timer enabled on Trigger rising or falling edge.

### 18.2.5.11 enum \_flexio\_timer\_stop\_bit\_condition

Enumerator

***kFLEXIO\_TimerStopBitDisabled*** Stop bit disabled.  
***kFLEXIO\_TimerStopBitEnableOnTimerCompare*** Stop bit is enabled on timer compare.  
***kFLEXIO\_TimerStopBitEnableOnTimerDisable*** Stop bit is enabled on timer disable.  
***kFLEXIO\_TimerStopBitEnableOnTimerCompareDisable*** Stop bit is enabled on timer compare and timer disable.

### 18.2.5.12 enum \_flexio\_timer\_start\_bit\_condition

Enumerator

***kFLEXIO\_TimerStartBitDisabled*** Start bit disabled.  
***kFLEXIO\_TimerStartBitEnabled*** Start bit enabled.

**18.2.5.13 enum \_flexio\_timer\_output\_state**

Enumerator

*kFLEXIO\_PwmLow* The output state of PWM channel is low.*kFLEXIO\_PwmHigh* The output state of PWM channel is high.**18.2.5.14 enum \_flexio\_shifter\_timer\_polarity**

Enumerator

*kFLEXIO\_ShifterTimerPolarityOnPositive* Shift on positive edge of shift clock.*kFLEXIO\_ShifterTimerPolarityOnNegative* Shift on negative edge of shift clock.**18.2.5.15 enum \_flexio\_shifter\_mode**

Enumerator

*kFLEXIO\_ShifterDisabled* Shifter is disabled.*kFLEXIO\_ShifterModeReceive* Receive mode.*kFLEXIO\_ShifterModeTransmit* Transmit mode.*kFLEXIO\_ShifterModeMatchStore* Match store mode.*kFLEXIO\_ShifterModeMatchContinuous* Match continuous mode.*kFLEXIO\_ShifterModeState* SHIFTBUF contents are used for storing programmable state attributes.*kFLEXIO\_ShifterModeLogic* SHIFTBUF contents are used for implementing programmable logic look up table.**18.2.5.16 enum \_flexio\_shifter\_input\_source**

Enumerator

*kFLEXIO\_ShifterInputFromPin* Shifter input from pin.*kFLEXIO\_ShifterInputFromNextShifterOutput* Shifter input from Shifter N+1.**18.2.5.17 enum \_flexio\_shifter\_stop\_bit**

Enumerator

*kFLEXIO\_ShifterStopBitDisable* Disable shifter stop bit.*kFLEXIO\_ShifterStopBitLow* Set shifter stop bit to logic low level.*kFLEXIO\_ShifterStopBitHigh* Set shifter stop bit to logic high level.

**18.2.5.18 enum \_flexio\_shifter\_start\_bit**

Enumerator

***kFLEXIO\_ShifterStartBitDisabledLoadDataOnEnable*** Disable shifter start bit, transmitter loads data on enable.

***kFLEXIO\_ShifterStartBitDisabledLoadDataOnShift*** Disable shifter start bit, transmitter loads data on first shift.

***kFLEXIO\_ShifterStartBitLow*** Set shifter start bit to logic low level.

***kFLEXIO\_ShifterStartBitHigh*** Set shifter start bit to logic high level.

**18.2.5.19 enum \_flexio\_shifter\_buffer\_type**

Enumerator

***kFLEXIO\_ShifterBuffer*** Shifter Buffer N Register.

***kFLEXIO\_ShifterBufferBitSwapped*** Shifter Buffer N Bit Byte Swapped Register.

***kFLEXIO\_ShifterBufferByteSwapped*** Shifter Buffer N Byte Swapped Register.

***kFLEXIO\_ShifterBufferBitByteSwapped*** Shifter Buffer N Bit Swapped Register.

***kFLEXIO\_ShifterBufferNibbleByteSwapped*** Shifter Buffer N Nibble Byte Swapped Register.

***kFLEXIO\_ShifterBufferHalfWordSwapped*** Shifter Buffer N Half Word Swapped Register.

***kFLEXIO\_ShifterBufferNibbleSwapped*** Shifter Buffer N Nibble Swapped Register.

**18.2.5.20 enum \_flexio\_gpio\_direction**

Enumerator

***kFLEXIO\_DigitalInput*** Set current pin as digital input.

***kFLEXIO\_DigitalOutput*** Set current pin as digital output.

**18.2.5.21 enum \_flexio\_pin\_input\_config**

Enumerator

***kFLEXIO\_InputInterruptDisabled*** Interrupt request is disabled.

***kFLEXIO\_InputInterruptEnable*** Interrupt request is enable.

***kFLEXIO\_FlagRisingEdgeEnable*** Input pin flag on rising edge.

***kFLEXIO\_FlagFallingEdgeEnable*** Input pin flag on falling edge.

**18.2.6 Function Documentation****18.2.6.1 void FLEXIO\_GetDefaultConfig ( flexio\_config\_t \* userConfig )**

The configuration can be used directly to call the FLEXIO\_Configure().

Example:

```
flexio_config_t config;
FLEXIO_GetDefaultConfig(&config);
```

Parameters

|                   |                                      |
|-------------------|--------------------------------------|
| <i>userConfig</i> | pointer to flexio_config_t structure |
|-------------------|--------------------------------------|

### 18.2.6.2 void FLEXIO\_Init ( FLEXIO\_Type \* *base*, const flexio\_config\_t \* *userConfig* )

The configuration structure can be filled by the user or be set with default values by [FLEXIO\\_GetDefaultConfig\(\)](#).

Example

```
flexio_config_t config = {
 .enableFlexio = true,
 .enableInDoze = false,
 .enableInDebug = true,
 .enableFastAccess = false
};
FLEXIO_Configure(base, &config);
```

Parameters

|                   |                                      |
|-------------------|--------------------------------------|
| <i>base</i>       | FlexIO peripheral base address       |
| <i>userConfig</i> | pointer to flexio_config_t structure |

### 18.2.6.3 void FLEXIO\_Deinit ( FLEXIO\_Type \* *base* )

Call this API to stop the FlexIO clock.

Note

After calling this API, call the FLEXIO\_Init to use the FlexIO module.

Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
|-------------|--------------------------------|

#### 18.2.6.4 uint32\_t FLEXIO\_GetInstance ( FLEXIO\_Type \* *base* )

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | FLEXIO peripheral base address. |
|-------------|---------------------------------|

#### 18.2.6.5 void FLEXIO\_Reset ( FLEXIO\_Type \* *base* )

Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
|-------------|--------------------------------|

#### 18.2.6.6 static void FLEXIO\_Enable ( FLEXIO\_Type \* *base*, bool *enable* ) [inline], [static]

Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | FlexIO peripheral base address    |
| <i>enable</i> | true to enable, false to disable. |

#### 18.2.6.7 static uint32\_t FLEXIO\_ReadPinInput ( FLEXIO\_Type \* *base* ) [inline], [static]

Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
|-------------|--------------------------------|

Returns

FlexIO pin input data

#### 18.2.6.8 static uint8\_t FLEXIO\_GetShifterState ( FLEXIO\_Type \* *base* ) [inline], [static]



## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
|-------------|--------------------------------|

## Returns

current State pointer

### 18.2.6.9 void FLEXIO\_SetShifterConfig ( FLEXIO\_Type \* *base*, uint8\_t *index*, const flexio\_shifter\_config\_t \* *shifterConfig* )

The configuration structure covers both the SHIFTCTL and SHIFTCFG registers. To configure the shifter to the proper mode, select which timer controls the shifter to shift, whether to generate start bit/stop bit, and the polarity of start bit and stop bit.

## Example

```
flexio_shifter_config_t config = {
 .timerSelect = 0,
 .timerPolarity = kFLEXIO_ShifterTimerPolarityOnPositive,
 .pinConfig = kFLEXIO_PinConfigOpenDrainOrBidirection,
 .pinPolarity = kFLEXIO_PinActiveLow,
 .shifterMode = kFLEXIO_ShifterModeTransmit,
 .inputSource = kFLEXIO_ShifterInputFromPin,
 .shifterStop = kFLEXIO_ShifterStopBitHigh,
 .shifterStart = kFLEXIO_ShifterStartBitLow
};
FLEXIO_SetShifterConfig(base, &config);
```

## Parameters

|                      |                                              |
|----------------------|----------------------------------------------|
| <i>base</i>          | FlexIO peripheral base address               |
| <i>index</i>         | Shifter index                                |
| <i>shifterConfig</i> | Pointer to flexio_shifter_config_t structure |

### 18.2.6.10 void FLEXIO\_SetTimerConfig ( FLEXIO\_Type \* *base*, uint8\_t *index*, const flexio\_timer\_config\_t \* *timerConfig* )

The configuration structure covers both the TIMCTL and TIMCFG registers. To configure the timer to the proper mode, select trigger source for timer and the timer pin output and the timing for timer.

## Example

```
flexio_timer_config_t config = {
 .triggerSelect = FLEXIO_TIMER_TRIGGER_SEL_SHIFtnSTAT(0),
 .triggerPolarity = kFLEXIO_TimerTriggerPolarityActiveLow,
 .triggerSource = kFLEXIO_TimerTriggerSourceInternal,
```

```

.pinConfig = kFLEXIO_PinConfigOpenDrainOrBidirection,
.pinSelect = 0,
.pinPolarity = kFLEXIO_PinActiveHigh,
.timerMode = kFLEXIO_TimerModeDual8BitBaudBit,
.timerOutput = kFLEXIO_TimerOutputZeroNotAffectedByReset,
.timerDecrement = kFLEXIO_TimerDecSrcOnFlexIOClockShiftTimerOutput
,
.timerReset = kFLEXIO_TimerResetOnTimerPinEqualToTimerOutput,
.timerDisable = kFLEXIO_TimerDisableOnTimerCompare,
.timerEnable = kFLEXIO_TimerEnableOnTriggerHigh,
.timerStop = kFLEXIO_TimerStopBitEnableOnTimerDisable,
.timerStart = kFLEXIO_TimerStartBitEnabled
};
FLEXIO_SetTimerConfig(base, &config);

```

#### Parameters

|                    |                                                |
|--------------------|------------------------------------------------|
| <i>base</i>        | FlexIO peripheral base address                 |
| <i>index</i>       | Timer index                                    |
| <i>timerConfig</i> | Pointer to the flexio_timer_config_t structure |

#### 18.2.6.11 static void FLEXIO\_SetClockMode ( FLEXIO\_Type \* *base*, uint8\_t *index*, flexio\_timer\_decrement\_source\_t *clocksource* ) [inline], [static]

#### Parameters

|                    |                                                  |
|--------------------|--------------------------------------------------|
| <i>base</i>        | Pointer to the FlexIO simulated peripheral type. |
| <i>index</i>       | Timer index                                      |
| <i>clocksource</i> | Set clock value                                  |

#### 18.2.6.12 static void FLEXIO\_EnableShifterStatusInterrupts ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

The interrupt generates when the corresponding SSF is set.

#### Parameters

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                    |
| <i>mask</i> | The shifter status mask which can be calculated by $(1 \ll \text{shifter index})$ |

#### Note

For multiple shifter status interrupt enable, for example, two shifter status enable, can calculate the mask by using  $((1 \ll \text{shifter index0}) | (1 \ll \text{shifter index1}))$

**18.2.6.13** `static void FLEXIO_DisableShifterStatusInterrupts ( FLEXIO_Type * base,  
uint32_t mask ) [inline], [static]`

The interrupt won't generate when the corresponding SSF is set.

## Parameters

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                    |
| <i>mask</i> | The shifter status mask which can be calculated by $(1 \ll \text{shifter index})$ |

## Note

For multiple shifter status interrupt enable, for example, two shifter status enable, can calculate the mask by using  $((1 \ll \text{shifter index0}) \mid (1 \ll \text{shifter index1}))$

#### 18.2.6.14 static void FLEXIO\_EnableShifterErrorInterrupts ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

The interrupt generates when the corresponding SEF is set.

## Parameters

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                   |
| <i>mask</i> | The shifter error mask which can be calculated by $(1 \ll \text{shifter index})$ |

## Note

For multiple shifter error interrupt enable, for example, two shifter error enable, can calculate the mask by using  $((1 \ll \text{shifter index0}) \mid (1 \ll \text{shifter index1}))$

#### 18.2.6.15 static void FLEXIO\_DisableShifterErrorInterrupts ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

The interrupt won't generate when the corresponding SEF is set.

## Parameters

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                   |
| <i>mask</i> | The shifter error mask which can be calculated by $(1 \ll \text{shifter index})$ |

## Note

For multiple shifter error interrupt enable, for example, two shifter error enable, can calculate the mask by using  $((1 \ll \text{shifter index0}) \mid (1 \ll \text{shifter index1}))$

**18.2.6.16** `static void FLEXIO_EnableTimerStatusInterrupts ( FLEXIO_Type * base,  
uint32_t mask ) [inline], [static]`

The interrupt generates when the corresponding SSF is set.

## Parameters

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                |
| <i>mask</i> | The timer status mask which can be calculated by $(1 \ll \text{timer index})$ |

## Note

For multiple timer status interrupt enable, for example, two timer status enable, can calculate the mask by using  $((1 \ll \text{timer index0}) \mid (1 \ll \text{timer index1}))$

#### 18.2.6.17 static void FLEXIO\_DisableTimerStatusInterrupts ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

The interrupt won't generate when the corresponding SSF is set.

## Parameters

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                |
| <i>mask</i> | The timer status mask which can be calculated by $(1 \ll \text{timer index})$ |

## Note

For multiple timer status interrupt enable, for example, two timer status enable, can calculate the mask by using  $((1 \ll \text{timer index0}) \mid (1 \ll \text{timer index1}))$

#### 18.2.6.18 static uint32\_t FLEXIO\_GetShifterStatusFlags ( FLEXIO\_Type \* *base* ) [inline], [static]

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
|-------------|--------------------------------|

## Returns

Shifter status flags

#### 18.2.6.19 static void FLEXIO\_ClearShifterStatusFlags ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                    |
| <i>mask</i> | The shifter status mask which can be calculated by $(1 \ll \text{shifter index})$ |

## Note

For clearing multiple shifter status flags, for example, two shifter status flags, can calculate the mask by using  $((1 \ll \text{shifter index0}) \mid (1 \ll \text{shifter index1}))$

#### 18.2.6.20 static uint32\_t FLEXIO\_GetShifterErrorFlags ( FLEXIO\_Type \* *base* ) [inline], [static]

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
|-------------|--------------------------------|

## Returns

Shifter error flags

#### 18.2.6.21 static void FLEXIO\_ClearShifterErrorFlags ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                   |
| <i>mask</i> | The shifter error mask which can be calculated by $(1 \ll \text{shifter index})$ |

## Note

For clearing multiple shifter error flags, for example, two shifter error flags, can calculate the mask by using  $((1 \ll \text{shifter index0}) \mid (1 \ll \text{shifter index1}))$

#### 18.2.6.22 static uint32\_t FLEXIO\_GetTimerStatusFlags ( FLEXIO\_Type \* *base* ) [inline], [static]

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
|-------------|--------------------------------|

## Returns

Timer status flags

### 18.2.6.23 static void FLEXIO\_ClearTimerStatusFlags ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                               |
|-------------|-------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                |
| <i>mask</i> | The timer status mask which can be calculated by $(1 \ll \text{timer index})$ |

## Note

For clearing multiple timer status flags, for example, two timer status flags, can calculate the mask by using  $((1 \ll \text{timer index0}) | (1 \ll \text{timer index1}))$

### 18.2.6.24 static void FLEXIO\_EnableShifterStatusDMA ( FLEXIO\_Type \* *base*, uint32\_t *mask*, bool *enable* ) [inline], [static]

The DMA request generates when the corresponding SSF is set.

## Note

For multiple shifter status DMA enables, for example, calculate the mask by using  $((1 \ll \text{shifter index0}) | (1 \ll \text{shifter index1}))$

## Parameters

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| <i>base</i> | FlexIO peripheral base address                                                    |
| <i>mask</i> | The shifter status mask which can be calculated by $(1 \ll \text{shifter index})$ |



|               |                                   |
|---------------|-----------------------------------|
| <i>enable</i> | True to enable, false to disable. |
|---------------|-----------------------------------|

#### 18.2.6.25 **uint32\_t FLEXIO\_GetShifterBufferAddress ( FLEXIO\_Type \* *base*, flexio\_shifter\_buffer\_type\_t *type*, uint8\_t *index* )**

Parameters

|              |                                              |
|--------------|----------------------------------------------|
| <i>base</i>  | FlexIO peripheral base address               |
| <i>type</i>  | Shifter type of flexio_shifter_buffer_type_t |
| <i>index</i> | Shifter index                                |

Returns

Corresponding shifter buffer index

#### 18.2.6.26 **status\_t FLEXIO\_RegisterHandleIRQ ( void \* *base*, void \* *handle*, flexio\_isr\_t *isr* )**

Parameters

|               |                                                         |
|---------------|---------------------------------------------------------|
| <i>base</i>   | Pointer to the FlexIO simulated peripheral type.        |
| <i>handle</i> | Pointer to the handler for FlexIO simulated peripheral. |
| <i>isr</i>    | FlexIO simulated peripheral interrupt handler.          |

Return values

|                           |                                                |
|---------------------------|------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                |
| <i>kStatus_OutOfRange</i> | The FlexIO type/handle/ISR table out of range. |

#### 18.2.6.27 **status\_t FLEXIO\_UnregisterHandleIRQ ( void \* *base* )**

Parameters

|             |                                                  |
|-------------|--------------------------------------------------|
| <i>base</i> | Pointer to the FlexIO simulated peripheral type. |
|-------------|--------------------------------------------------|

## Return values

|                           |                                                |
|---------------------------|------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                |
| <i>kStatus_OutOfRange</i> | The FlexIO type/handle/ISR table out of range. |

#### 18.2.6.28 void FLEXIO\_SetPinConfig ( FLEXIO\_Type \* *base*, uint32\_t *pin*, flexio\_gpio\_config\_t \* *config* )

To Config the FLEXIO PIN, define a pin configuration, as either input or output, in the user file. Then, call the [FLEXIO\\_SetPinConfig\(\)](#) function.

This is an example to define an input pin or an output pin configuration.

```
* Define a digital input pin configuration,
* flexio_gpio_config_t config =
* {
* kFLEXIO_DigitalInput,
* 0U,
* kFLEXIO_FlagRisingEdgeEnable |
* kFLEXIO_InputInterruptEnable,
* }
* Define a digital output pin configuration,
* flexio_gpio_config_t config =
* {
* kFLEXIO_DigitalOutput,
* 0U,
* 0U
* }
*
```

## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | FlexIO peripheral base address    |
| <i>pin</i>    | FLEXIO pin number.                |
| <i>config</i> | FLEXIO pin configuration pointer. |

#### 18.2.6.29 static void FLEXIO\_ClearPortOutput ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
| <i>mask</i> | FLEXIO pin number mask         |

#### 18.2.6.30 static void FLEXIO\_SetPortOutput ( FLEXIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
| <i>mask</i> | FLEXIO pin number mask         |

**18.2.6.31** `static void FLEXIO_TogglePortOutput ( FLEXIO_Type * base, uint32_t mask )`  
`[inline], [static]`

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
| <i>mask</i> | FLEXIO pin number mask         |

**18.2.6.32** `static void FLEXIO_PinWrite ( FLEXIO_Type * base, uint32_t pin, uint8_t`  
`output ) [inline], [static]`

## Parameters

|               |                                                                                                                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>   | FlexIO peripheral base address                                                                                                                                                              |
| <i>pin</i>    | FLEXIO pin number.                                                                                                                                                                          |
| <i>output</i> | FLEXIO pin output logic level.<br><ul style="list-style-type: none"> <li>• 0: corresponding pin output low-logic level.</li> <li>• 1: corresponding pin output high-logic level.</li> </ul> |

**18.2.6.33** `static void FLEXIO_EnablePinOutput ( FLEXIO_Type * base, uint32_t pin )`  
`[inline], [static]`

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
| <i>pin</i>  | FLEXIO pin number.             |

**18.2.6.34** `static uint32_t FLEXIO_PinRead ( FLEXIO_Type * base, uint32_t pin )`  
`[inline], [static]`

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
| <i>pin</i>  | FLEXIO pin number.             |

## Return values

|               |                                                                                                                                                                          |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>FLEXIO</i> | port input value <ul style="list-style-type: none"> <li>• 0: corresponding pin input low-logic level.</li> <li>• 1: corresponding pin input high-logic level.</li> </ul> |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**18.2.6.35** `static uint32_t FLEXIO_GetPinStatus ( FLEXIO_Type * base, uint32_t pin )`  
**[inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
| <i>pin</i>  | FLEXIO pin number.             |

## Return values

|               |                                                                                                                                                                                           |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>FLEXIO</i> | port input status <ul style="list-style-type: none"> <li>• 0: corresponding pin input capture no status.</li> <li>• 1: corresponding pin input capture rising or falling edge.</li> </ul> |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**18.2.6.36** `static void FLEXIO_ClearPortStatus ( FLEXIO_Type * base, uint32_t mask )`  
**[inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | FlexIO peripheral base address |
| <i>mask</i> | FLEXIO pin number mask         |

## 18.2.7 Variable Documentation

18.2.7.1 `FLEXIO_Type* const s_flexioBases[]`

18.2.7.2 `const clock_ip_name_t s_flexioClocks[]`

## 18.3 FlexIO I2C Master Driver

### 18.3.1 Overview

The MCUXpresso SDK provides a peripheral driver for I2C master function using Flexible I/O module of MCUXpresso SDK devices.

The FlexIO I2C master driver includes functional APIs and transactional APIs.

Functional APIs target low level APIs. Functional APIs can be used for the FlexIO I2C master initialization/configuration/operation for the optimization/customization purpose. Using the functional APIs requires the knowledge of the FlexIO I2C master peripheral and how to organize functional APIs to meet the application requirements. The FlexIO I2C master functional operation groups provide the functional APIs set.

Transactional APIs target high-level APIs. The transactional APIs can be used to enable the peripheral quickly and also in the application if the code size and performance of transactional APIs satisfy the requirements. If the code size and performance are critical requirements, see the transactional API implementation and write custom code using the functional APIs or accessing the hardware registers.

Transactional APIs support an asynchronous transfer. This means that the functions [FLEXIO\\_I2C\\_MasterTransferNonBlocking\(\)](#) set up the interrupt non-blocking transfer. When the transfer completes, the upper layer is notified through a callback function with the `kStatus_Success` status.

### 18.3.2 Typical use case

#### 18.3.2.1 FlexIO I2C master transfer using an interrupt method

```
flexio_i2c_master_handle_t g_m_handle;
flexio_i2c_master_config_t masterConfig;
flexio_i2c_master_transfer_t masterXfer;
volatile bool completionFlag = false;
const uint8_t sendData[] = {.....};
FLEXIO_I2C_Type i2cDev;

void FLEXIO_I2C_MasterCallback(FLEXIO_I2C_Type *base, status_t status, void *
 userData)
{
 userData = userData;

 if (kStatus_Success == status)
 {
 completionFlag = true;
 }
}

void main(void)
{
 //...

 FLEXIO_I2C_MasterGetDefaultConfig(&masterConfig);

 FLEXIO_I2C_MasterInit(&i2cDev, &user_config);
 FLEXIO_I2C_MasterTransferCreateHandle(&i2cDev, &g_m_handle,
 FLEXIO_I2C_MasterCallback, NULL);
```

```

// Prepares to send.
masterXfer.slaveAddress = g_accel_address[0];
masterXfer.direction = kI2C_Read;
masterXfer.subaddress = &who_am_i_reg;
masterXfer.subaddressSize = 1;
masterXfer.data = &who_am_i_value;
masterXfer.dataSize = 1;
masterXfer.flags = kI2C_TransferDefaultFlag;

// Sends out.
FLEXIO_I2C_MasterTransferNonBlocking(&i2cDev, &g_m_handle, &
 masterXfer);

// Wait for sending is complete.
while (!completionFlag)
{
}

// ...
}

```

## Data Structures

- struct [\\_flexio\\_i2c\\_type](#)  
Define FlexIO I2C master access structure typedef. [More...](#)
- struct [\\_flexio\\_i2c\\_master\\_config](#)  
Define FlexIO I2C master user configuration structure. [More...](#)
- struct [\\_flexio\\_i2c\\_master\\_transfer](#)  
Define FlexIO I2C master transfer structure. [More...](#)
- struct [\\_flexio\\_i2c\\_master\\_handle](#)  
Define FlexIO I2C master handle structure. [More...](#)

## Macros

- #define [I2C\\_RETRY\\_TIMES](#) 0U /\* Define to zero means keep waiting until the flag is assert/deassert. \*/  
Retry times for waiting flag.

## Typedefs

- typedef enum [\\_flexio\\_i2c\\_direction](#) flexio\_i2c\_direction\_t  
Direction of master transfer.
- typedef struct [\\_flexio\\_i2c\\_type](#) FLEXIO\_I2C\_Type  
Define FlexIO I2C master access structure typedef.
- typedef struct [\\_flexio\\_i2c\\_master\\_config](#) flexio\_i2c\_master\_config\_t  
Define FlexIO I2C master user configuration structure.
- typedef struct [\\_flexio\\_i2c\\_master\\_transfer](#) flexio\_i2c\_master\_transfer\_t  
Define FlexIO I2C master transfer structure.

- typedef struct  
     [\\_flexio\\_i2c\\_master\\_handle](#) [flexio\\_i2c\\_master\\_handle\\_t](#)  
     *FlexIO I2C master handle typedef.*
- typedef void(\* [flexio\\_i2c\\_master\\_transfer\\_callback\\_t](#) )(FLEXIO\_I2C\_Type \*base, [flexio\\_i2c\\_master\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
     *FlexIO I2C master transfer callback typedef.*

## Enumerations

- enum {  
     [kStatus\\_FLEXIO\\_I2C\\_Busy](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2C, 0),  
     [kStatus\\_FLEXIO\\_I2C\\_Idle](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2C, 1),  
     [kStatus\\_FLEXIO\\_I2C\\_Nak](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2C, 2),  
     [kStatus\\_FLEXIO\\_I2C\\_Timeout](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2C, 3) }  
     *FlexIO I2C transfer status.*
- enum [\\_flexio\\_i2c\\_master\\_interrupt](#) {  
     [kFLEXIO\\_I2C\\_TxEmptyInterruptEnable](#) = 0x1U,  
     [kFLEXIO\\_I2C\\_RxFullInterruptEnable](#) = 0x2U }  
     *Define FlexIO I2C master interrupt mask.*
- enum [\\_flexio\\_i2c\\_master\\_status\\_flags](#) {  
     [kFLEXIO\\_I2C\\_TxEmptyFlag](#) = 0x1U,  
     [kFLEXIO\\_I2C\\_RxFullFlag](#) = 0x2U,  
     [kFLEXIO\\_I2C\\_ReceiveNakFlag](#) = 0x4U }  
     *Define FlexIO I2C master status mask.*
- enum [\\_flexio\\_i2c\\_direction](#) {  
     [kFLEXIO\\_I2C\\_Write](#) = 0x0U,  
     [kFLEXIO\\_I2C\\_Read](#) = 0x1U }  
     *Direction of master transfer.*

## Driver version

- #define FSL\_FLEXIO\_I2C\_MASTER\_DRIVER\_VERSION ([MAKE\\_VERSION](#)(2, 5, 0))

## Initialization and deinitialization

- [status\\_t FLEXIO\\_I2C\\_CheckForBusyBus](#) (FLEXIO\_I2C\_Type \*base)  
     *Make sure the bus isn't already pulled down.*
- [status\\_t FLEXIO\\_I2C\\_MasterInit](#) (FLEXIO\_I2C\_Type \*base, [flexio\\_i2c\\_master\\_config\\_t](#) \*masterConfig, uint32\_t srcClock\_Hz)  
     *Un gates the FlexIO clock, resets the FlexIO module, and configures the FlexIO I2C hardware configuration.*
- void [FLEXIO\\_I2C\\_MasterDeinit](#) (FLEXIO\_I2C\_Type \*base)  
     *De-initializes the FlexIO I2C master peripheral.*
- void [FLEXIO\\_I2C\\_MasterGetDefaultConfig](#) ([flexio\\_i2c\\_master\\_config\\_t](#) \*masterConfig)  
     *Gets the default configuration to configure the FlexIO module.*



- static void [FLEXIO\\_I2C\\_MasterEnable](#) ([FLEXIO\\_I2C\\_Type](#) \*base, bool enable)  
*Enables/disables the FlexIO module operation.*

## Status

- uint32\_t [FLEXIO\\_I2C\\_MasterGetStatusFlags](#) ([FLEXIO\\_I2C\\_Type](#) \*base)  
*Gets the FlexIO I2C master status flags.*
- void [FLEXIO\\_I2C\\_MasterClearStatusFlags](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint32\_t mask)  
*Clears the FlexIO I2C master status flags.*

## Interrupts

- void [FLEXIO\\_I2C\\_MasterEnableInterrupts](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint32\_t mask)  
*Enables the FlexIO i2c master interrupt requests.*
- void [FLEXIO\\_I2C\\_MasterDisableInterrupts](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint32\_t mask)  
*Disables the FlexIO I2C master interrupt requests.*

## Bus Operations

- void [FLEXIO\\_I2C\\_MasterSetBaudRate](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint32\_t baudRate\_Bps, uint32\_t srcClock\_Hz)  
*Sets the FlexIO I2C master transfer baudrate.*
- void [FLEXIO\\_I2C\\_MasterStart](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint8\_t address, [flexio\\_i2c\\_direction\\_t](#) direction)  
*Sends START + 7-bit address to the bus.*
- void [FLEXIO\\_I2C\\_MasterStop](#) ([FLEXIO\\_I2C\\_Type](#) \*base)  
*Sends the stop signal on the bus.*
- void [FLEXIO\\_I2C\\_MasterRepeatedStart](#) ([FLEXIO\\_I2C\\_Type](#) \*base)  
*Sends the repeated start signal on the bus.*
- void [FLEXIO\\_I2C\\_MasterAbortStop](#) ([FLEXIO\\_I2C\\_Type](#) \*base)  
*Sends the stop signal when transfer is still on-going.*
- void [FLEXIO\\_I2C\\_MasterEnableAck](#) ([FLEXIO\\_I2C\\_Type](#) \*base, bool enable)  
*Configures the sent ACK/NAK for the following byte.*
- [status\\_t](#) [FLEXIO\\_I2C\\_MasterSetTransferCount](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint16\_t count)  
*Sets the number of bytes to be transferred from a start signal to a stop signal.*
- static void [FLEXIO\\_I2C\\_MasterWriteByte](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint32\_t data)  
*Writes one byte of data to the I2C bus.*
- static uint8\_t [FLEXIO\\_I2C\\_MasterReadByte](#) ([FLEXIO\\_I2C\\_Type](#) \*base)  
*Reads one byte of data from the I2C bus.*
- [status\\_t](#) [FLEXIO\\_I2C\\_MasterWriteBlocking](#) ([FLEXIO\\_I2C\\_Type](#) \*base, const uint8\_t \*txBuff, uint8\_t txSize)  
*Sends a buffer of data in bytes.*
- [status\\_t](#) [FLEXIO\\_I2C\\_MasterReadBlocking](#) ([FLEXIO\\_I2C\\_Type](#) \*base, uint8\_t \*rxBuff, uint8\_t rxSize)  
*Receives a buffer of bytes.*

- `status_t FLEXIO_I2C_MasterTransferBlocking` (`FLEXIO_I2C_Type *base`, `flexio_i2c_master_transfer_t *xfer`)  
*Performs a master polling transfer on the I2C bus.*

## Transactional

- `status_t FLEXIO_I2C_MasterTransferCreateHandle` (`FLEXIO_I2C_Type *base`, `flexio_i2c_master_handle_t *handle`, `flexio_i2c_master_transfer_callback_t callback`, `void *userData`)  
*Initializes the I2C handle which is used in transactional functions.*
- `status_t FLEXIO_I2C_MasterTransferNonBlocking` (`FLEXIO_I2C_Type *base`, `flexio_i2c_master_handle_t *handle`, `flexio_i2c_master_transfer_t *xfer`)  
*Performs a master interrupt non-blocking transfer on the I2C bus.*
- `status_t FLEXIO_I2C_MasterTransferGetCount` (`FLEXIO_I2C_Type *base`, `flexio_i2c_master_handle_t *handle`, `size_t *count`)  
*Gets the master transfer status during a interrupt non-blocking transfer.*
- `void FLEXIO_I2C_MasterTransferAbort` (`FLEXIO_I2C_Type *base`, `flexio_i2c_master_handle_t *handle`)  
*Aborts an interrupt non-blocking transfer early.*
- `void FLEXIO_I2C_MasterTransferHandleIRQ` (`void *i2cType`, `void *i2cHandle`)  
*Master interrupt handler.*

## 18.3.3 Data Structure Documentation

### 18.3.3.1 struct\_flexio\_i2c\_type

#### Data Fields

- `FLEXIO_Type * flexioBase`  
*FlexIO base pointer.*
- `uint8_t SDAPinIndex`  
*Pin select for I2C SDA.*
- `uint8_t SCLPinIndex`  
*Pin select for I2C SCL.*
- `uint8_t shifterIndex [2]`  
*Shifter index used in FlexIO I2C.*
- `uint8_t timerIndex [3]`  
*Timer index used in FlexIO I2C.*
- `uint32_t baudrate`  
*Master transfer baudrate, used to calculate delay time.*

## Field Documentation

- (1) `FLEXIO_Type* _flexio_i2c_type::flexioBase`
- (2) `uint8_t _flexio_i2c_type::SDAPinIndex`
- (3) `uint8_t _flexio_i2c_type::SCLPinIndex`
- (4) `uint8_t _flexio_i2c_type::shifterIndex[2]`
- (5) `uint8_t _flexio_i2c_type::timerIndex[3]`
- (6) `uint32_t _flexio_i2c_type::baudrate`

18.3.3.2 `struct _flexio_i2c_master_config`

## Data Fields

- `bool enableMaster`  
*Enables the FlexIO I2C peripheral at initialization time.*
- `bool enableInDoze`  
*Enable/disable FlexIO operation in doze mode.*
- `bool enableInDebug`  
*Enable/disable FlexIO operation in debug mode.*
- `bool enableFastAccess`  
*Enable/disable fast access to FlexIO registers, fast access requires the FlexIO clock to be at least twice the frequency of the bus clock.*
- `uint32_t baudRate_Bps`  
*Baud rate in Bps.*

## Field Documentation

- (1) `bool _flexio_i2c_master_config::enableMaster`
- (2) `bool _flexio_i2c_master_config::enableInDoze`
- (3) `bool _flexio_i2c_master_config::enableInDebug`
- (4) `bool _flexio_i2c_master_config::enableFastAccess`
- (5) `uint32_t _flexio_i2c_master_config::baudRate_Bps`

18.3.3.3 `struct _flexio_i2c_master_transfer`

## Data Fields

- `uint32_t flags`  
*Transfer flag which controls the transfer, reserved for FlexIO I2C.*
- `uint8_t slaveAddress`  
*7-bit slave address.*
- `flexio_i2c_direction_t direction`

- `uint32_t subaddress`  
*Transfer direction, read or write.*
- `uint8_t subaddressSize`  
*Sub address.*
- `uint8_t volatile * data`  
*Size of command buffer.*
- `volatile size_t dataSize`  
*Transfer buffer.*
- `volatile size_t dataSize`  
*Transfer size.*

## Field Documentation

- (1) `uint32_t flexio_i2c_master_transfer::flags`
  - (2) `uint8_t flexio_i2c_master_transfer::slaveAddress`
  - (3) `flexio_i2c_direction_t flexio_i2c_master_transfer::direction`
  - (4) `uint32_t flexio_i2c_master_transfer::subaddress`  
Transferred MSB first.
  - (5) `uint8_t flexio_i2c_master_transfer::subaddressSize`
  - (6) `uint8_t volatile* flexio_i2c_master_transfer::data`
  - (7) `volatile size_t flexio_i2c_master_transfer::dataSize`
- ### 18.3.3.4 struct flexio\_i2c\_master\_handle

## Data Fields

- `flexio_i2c_master_transfer_t transfer`  
*FlexIO I2C master transfer copy.*
- `size_t transferSize`  
*Total bytes to be transferred.*
- `uint8_t state`  
*Transfer state maintained during transfer.*
- `flexio_i2c_master_transfer_callback_t completionCallback`  
*Callback function called at transfer event.*
- `void * userData`  
*Callback parameter passed to callback function.*
- `bool needRestart`  
*Whether master needs to send re-start signal.*

## Field Documentation

- (1) `flexio_i2c_master_transfer_t _flexio_i2c_master_handle::transfer`
- (2) `size_t _flexio_i2c_master_handle::transferSize`
- (3) `uint8_t _flexio_i2c_master_handle::state`
- (4) `flexio_i2c_master_transfer_callback_t _flexio_i2c_master_handle::completionCallback`

Callback function called at transfer event.

- (5) `void* _flexio_i2c_master_handle::userData`
- (6) `bool _flexio_i2c_master_handle::needRestart`

## 18.3.4 Macro Definition Documentation

- 18.3.4.1 `#define I2C_RETRY_TIMES 0U /* Define to zero means keep waiting until the flag is assert/deassert. */`

## 18.3.5 Typedef Documentation

- 18.3.5.1 `typedef enum _flexio_i2c_direction flexio_i2c_direction_t`
- 18.3.5.2 `typedef struct _flexio_i2c_type FLEXIO_I2C_Type`
- 18.3.5.3 `typedef struct _flexio_i2c_master_config flexio_i2c_master_config_t`
- 18.3.5.4 `typedef struct _flexio_i2c_master_transfer flexio_i2c_master_transfer_t`
- 18.3.5.5 `typedef struct _flexio_i2c_master_handle flexio_i2c_master_handle_t`
- 18.3.5.6 `typedef void(* flexio_i2c_master_transfer_callback_t)(FLEXIO_I2C_Type *base, flexio_i2c_master_handle_t *handle, status_t status, void *userData)`

## 18.3.6 Enumeration Type Documentation

### 18.3.6.1 anonymous enum

Enumerator

*kStatus\_FLEXIO\_I2C\_Busy* I2C is busy doing transfer.  
*kStatus\_FLEXIO\_I2C\_Idle* I2C is busy doing transfer.  
*kStatus\_FLEXIO\_I2C\_Nak* NAK received during transfer.  
*kStatus\_FLEXIO\_I2C\_Timeout* Timeout polling status flags.

### 18.3.6.2 enum \_flexio\_i2c\_master\_interrupt

Enumerator

***kFLEXIO\_I2C\_TxEmptyInterruptEnable*** Tx buffer empty interrupt enable.

***kFLEXIO\_I2C\_RxFullInterruptEnable*** Rx buffer full interrupt enable.

### 18.3.6.3 enum \_flexio\_i2c\_master\_status\_flags

Enumerator

***kFLEXIO\_I2C\_TxEmptyFlag*** Tx shifter empty flag.

***kFLEXIO\_I2C\_RxFullFlag*** Rx shifter full/Transfer complete flag.

***kFLEXIO\_I2C\_ReceiveNakFlag*** Receive NAK flag.

### 18.3.6.4 enum \_flexio\_i2c\_direction

Enumerator

***kFLEXIO\_I2C\_Write*** Master send to slave.

***kFLEXIO\_I2C\_Read*** Master receive from slave.

## 18.3.7 Function Documentation

### 18.3.7.1 status\_t FLEXIO\_I2C\_CheckForBusyBus ( FLEXIO\_I2C\_Type \* *base* )

Check the FLEXIO pin status to see whether either of SDA and SCL pin is pulled down.

Parameters

|             |                                        |
|-------------|----------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure.. |
|-------------|----------------------------------------|

Return values

|                                |  |
|--------------------------------|--|
| <i>kStatus_Success</i>         |  |
| <i>kStatus_FLEXIO_I2C_Busy</i> |  |

### 18.3.7.2 status\_t FLEXIO\_I2C\_MasterInit ( FLEXIO\_I2C\_Type \* *base*, flexio\_i2c\_master\_config\_t \* *masterConfig*, uint32\_t *srcClock\_Hz* )

Example

```

FLEXIO_I2C_Type base = {
 .flexioBase = FLEXIO,
 .SDAPinIndex = 0,
 .SCLPinIndex = 1,
 .shifterIndex = {0,1},
 .timerIndex = {0,1}
};
flexio_i2c_master_config_t config = {
 .enableInDoze = false,
 .enableInDebug = true,
 .enableFastAccess = false,
 .baudRate_Bps = 100000
};
FLEXIO_I2C_MasterInit(base, &config, srcClock_Hz);

```

### Parameters

|                     |                                                  |
|---------------------|--------------------------------------------------|
| <i>base</i>         | Pointer to FLEXIO_I2C_Type structure.            |
| <i>masterConfig</i> | Pointer to flexio_i2c_master_config_t structure. |
| <i>srcClock_Hz</i>  | FlexIO source clock in Hz.                       |

### Return values

|                                |                                                |
|--------------------------------|------------------------------------------------|
| <i>kStatus_Success</i>         | Initialization successful                      |
| <i>kStatus_InvalidArgument</i> | The source clock exceed upper range limitation |

#### 18.3.7.3 void FLEXIO\_I2C\_MasterDeinit ( FLEXIO\_I2C\_Type \* *base* )

Calling this API Resets the FlexIO I2C master shifer and timer config, module can't work unless the FLEXIO\_I2C\_MasterInit is called.

### Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | pointer to FLEXIO_I2C_Type structure. |
|-------------|---------------------------------------|

#### 18.3.7.4 void FLEXIO\_I2C\_MasterGetDefaultConfig ( flexio\_i2c\_master\_config\_t \* *masterConfig* )

The configuration can be used directly for calling the [FLEXIO\\_I2C\\_MasterInit\(\)](#).

Example:

```

flexio_i2c_master_config_t config;
FLEXIO_I2C_MasterGetDefaultConfig(&config);

```

## Parameters

|                     |                                                  |
|---------------------|--------------------------------------------------|
| <i>masterConfig</i> | Pointer to flexio_i2c_master_config_t structure. |
|---------------------|--------------------------------------------------|

**18.3.7.5 static void FLEXIO\_I2C\_MasterEnable ( FLEXIO\_I2C\_Type \* *base*, bool *enable* ) [inline], [static]**

## Parameters

|               |                                                             |
|---------------|-------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2C_Type structure.                       |
| <i>enable</i> | Pass true to enable module, false does not have any effect. |

**18.3.7.6 uint32\_t FLEXIO\_I2C\_MasterGetStatusFlags ( FLEXIO\_I2C\_Type \* *base* )**

## Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure |
|-------------|--------------------------------------|

## Returns

Status flag, use status flag to AND [\\_flexio\\_i2c\\_master\\_status\\_flags](#) can get the related status.

**18.3.7.7 void FLEXIO\_I2C\_MasterClearStatusFlags ( FLEXIO\_I2C\_Type \* *base*, uint32\_t *mask* )**

## Parameters

|             |                                                                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure.                                                                                                                                                       |
| <i>mask</i> | Status flag. The parameter can be any combination of the following values: <ul style="list-style-type: none"> <li>• kFLEXIO_I2C_RxFullFlag</li> <li>• kFLEXIO_I2C_ReceiveNakFlag</li> </ul> |

**18.3.7.8 void FLEXIO\_I2C\_MasterEnableInterrupts ( FLEXIO\_I2C\_Type \* *base*, uint32\_t *mask* )**



## Parameters

|             |                                                                                                                                                                |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure.                                                                                                                          |
| <i>mask</i> | Interrupt source. Currently only one interrupt request source: <ul style="list-style-type: none"> <li>• kFLEXIO_I2C_TransferCompleteInterruptEnable</li> </ul> |

### 18.3.7.9 void FLEXIO\_I2C\_MasterDisableInterrupts ( FLEXIO\_I2C\_Type \* *base*, uint32\_t *mask* )

## Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure. |
| <i>mask</i> | Interrupt source.                     |

### 18.3.7.10 void FLEXIO\_I2C\_MasterSetBaudRate ( FLEXIO\_I2C\_Type \* *base*, uint32\_t *baudRate\_Bps*, uint32\_t *srcClock\_Hz* )

## Parameters

|                     |                                      |
|---------------------|--------------------------------------|
| <i>base</i>         | Pointer to FLEXIO_I2C_Type structure |
| <i>baudRate_Bps</i> | the baud rate value in HZ            |
| <i>srcClock_Hz</i>  | source clock in HZ                   |

### 18.3.7.11 void FLEXIO\_I2C\_MasterStart ( FLEXIO\_I2C\_Type \* *base*, uint8\_t *address*, flexio\_i2c\_direction\_t *direction* )

## Note

This API should be called when the transfer configuration is ready to send a START signal and 7-bit address to the bus. This is a non-blocking API, which returns directly after the address is put into the data register but the address transfer is not finished on the bus. Ensure that the kFLEXIO\_I2C\_RxFullFlag status is asserted before calling this API.

## Parameters

|                  |                                                                                                                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>      | Pointer to FLEXIO_I2C_Type structure.                                                                                                                                                                   |
| <i>address</i>   | 7-bit address.                                                                                                                                                                                          |
| <i>direction</i> | transfer direction. This parameter is one of the values in flexio_i2c_direction_t: <ul style="list-style-type: none"> <li>• kFLEXIO_I2C_Write: Transmit</li> <li>• kFLEXIO_I2C_Read: Receive</li> </ul> |

**18.3.7.12 void FLEXIO\_I2C\_MasterStop ( FLEXIO\_I2C\_Type \* *base* )**

## Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure. |
|-------------|---------------------------------------|

**18.3.7.13 void FLEXIO\_I2C\_MasterRepeatedStart ( FLEXIO\_I2C\_Type \* *base* )**

## Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure. |
|-------------|---------------------------------------|

**18.3.7.14 void FLEXIO\_I2C\_MasterAbortStop ( FLEXIO\_I2C\_Type \* *base* )**

## Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure. |
|-------------|---------------------------------------|

**18.3.7.15 void FLEXIO\_I2C\_MasterEnableAck ( FLEXIO\_I2C\_Type \* *base*, bool *enable* )**

## Parameters

|               |                                                          |
|---------------|----------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2C_Type structure.                    |
| <i>enable</i> | True to configure send ACK, false configure to send NAK. |

**18.3.7.16 status\_t FLEXIO\_I2C\_MasterSetTransferCount ( FLEXIO\_I2C\_Type \* *base*, uint16\_t *count* )**

## Note

Call this API before a transfer begins because the timer generates a number of clocks according to the number of bytes that need to be transferred.

## Parameters

|              |                                                                                      |
|--------------|--------------------------------------------------------------------------------------|
| <i>base</i>  | Pointer to FLEXIO_I2C_Type structure.                                                |
| <i>count</i> | Number of bytes need to be transferred from a start signal to a re-start/stop signal |

## Return values

|                                |                                    |
|--------------------------------|------------------------------------|
| <i>kStatus_Success</i>         | Successfully configured the count. |
| <i>kStatus_InvalidArgument</i> | Input argument is invalid.         |

**18.3.7.17 static void FLEXIO\_I2C\_MasterWriteByte ( FLEXIO\_I2C\_Type \* *base*, uint32\_t *data* ) [inline], [static]**

## Note

This is a non-blocking API, which returns directly after the data is put into the data register but the data transfer is not finished on the bus. Ensure that the TxEmptyFlag is asserted before calling this API.

## Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure. |
| <i>data</i> | a byte of data.                       |

**18.3.7.18 static uint8\_t FLEXIO\_I2C\_MasterReadByte ( FLEXIO\_I2C\_Type \* *base* ) [inline], [static]**

## Note

This is a non-blocking API, which returns directly after the data is read from the data register. Ensure that the data is ready in the register.

## Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure. |
|-------------|---------------------------------------|

## Returns

data byte read.

### 18.3.7.19 status\_t FLEXIO\_I2C\_MasterWriteBlocking ( FLEXIO\_I2C\_Type \* *base*, const uint8\_t \* *txBuff*, uint8\_t *txSize* )

## Note

This function blocks via polling until all bytes have been sent.

## Parameters

|               |                                       |
|---------------|---------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2C_Type structure. |
| <i>txBuff</i> | The data bytes to send.               |
| <i>txSize</i> | The number of data bytes to send.     |

## Return values

|                                    |                                  |
|------------------------------------|----------------------------------|
| <i>kStatus_Success</i>             | Successfully write data.         |
| <i>kStatus_FLEXIO_I2C_-Nak</i>     | Receive NAK during writing data. |
| <i>kStatus_FLEXIO_I2C_-Timeout</i> | Timeout polling status flags.    |

### 18.3.7.20 status\_t FLEXIO\_I2C\_MasterReadBlocking ( FLEXIO\_I2C\_Type \* *base*, uint8\_t \* *rxBuff*, uint8\_t *rxSize* )

## Note

This function blocks via polling until all bytes have been received.

## Parameters

|               |                                          |
|---------------|------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2C_Type structure.    |
| <i>rxBuff</i> | The buffer to store the received bytes.  |
| <i>rxSize</i> | The number of data bytes to be received. |

## Return values

|                                   |                               |
|-----------------------------------|-------------------------------|
| <i>kStatus_Success</i>            | Successfully read data.       |
| <i>kStatus_FLEXIO_I2C_Timeout</i> | Timeout polling status flags. |

### 18.3.7.21 status\_t FLEXIO\_I2C\_MasterTransferBlocking ( FLEXIO\_I2C\_Type \* *base*, flexio\_i2c\_master\_transfer\_t \* *xfer* )

## Note

The API does not return until the transfer succeeds or fails due to receiving NAK.

## Parameters

|             |                                                    |
|-------------|----------------------------------------------------|
| <i>base</i> | pointer to FLEXIO_I2C_Type structure.              |
| <i>xfer</i> | pointer to flexio_i2c_master_transfer_t structure. |

## Returns

status of status\_t.

### 18.3.7.22 status\_t FLEXIO\_I2C\_MasterTransferCreateHandle ( FLEXIO\_I2C\_Type \* *base*, flexio\_i2c\_master\_handle\_t \* *handle*, flexio\_i2c\_master\_transfer\_callback\_t *callback*, void \* *userData* )

## Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2C_Type structure. |
|-------------|---------------------------------------|

|                 |                                                                              |
|-----------------|------------------------------------------------------------------------------|
| <i>handle</i>   | Pointer to flexio_i2c_master_handle_t structure to store the transfer state. |
| <i>callback</i> | Pointer to user callback function.                                           |
| <i>userData</i> | User param passed to the callback function.                                  |

Return values

|                           |                                                |
|---------------------------|------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                |
| <i>kStatus_OutOfRange</i> | The FlexIO type/handle/isr table out of range. |

#### 18.3.7.23 status\_t FLEXIO\_I2C\_MasterTransferNonBlocking ( FLEXIO\_I2C\_Type \* *base*, flexio\_i2c\_master\_handle\_t \* *handle*, flexio\_i2c\_master\_transfer\_t \* *xfer* )

Note

The API returns immediately after the transfer initiates. Call FLEXIO\_I2C\_MasterTransferGetCount to poll the transfer status to check whether the transfer is finished. If the return status is not kStatus\_FLEXIO\_I2C\_Busy, the transfer is finished.

Parameters

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2C_Type structure                                            |
| <i>handle</i> | Pointer to flexio_i2c_master_handle_t structure which stores the transfer state |
| <i>xfer</i>   | pointer to flexio_i2c_master_transfer_t structure                               |

Return values

|                                |                                                      |
|--------------------------------|------------------------------------------------------|
| <i>kStatus_Success</i>         | Successfully start a transfer.                       |
| <i>kStatus_FLEXIO_I2C_Busy</i> | FlexIO I2C is not idle, is running another transfer. |

#### 18.3.7.24 status\_t FLEXIO\_I2C\_MasterTransferGetCount ( FLEXIO\_I2C\_Type \* *base*, flexio\_i2c\_master\_handle\_t \* *handle*, size\_t \* *count* )

Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2C_Type structure.                                            |
| <i>handle</i> | Pointer to flexio_i2c_master_handle_t structure which stores the transfer state. |
| <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction.              |

## Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_InvalidArgument</i>      | count is Invalid.                                              |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |
| <i>kStatus_Success</i>              | Successfully return the count.                                 |

### 18.3.7.25 void FLEXIO\_I2C\_MasterTransferAbort ( FLEXIO\_I2C\_Type \* *base*, flexio\_i2c\_master\_handle\_t \* *handle* )

## Note

This API can be called at any time when an interrupt non-blocking transfer initiates to abort the transfer early.

## Parameters

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2C_Type structure                                            |
| <i>handle</i> | Pointer to flexio_i2c_master_handle_t structure which stores the transfer state |

### 18.3.7.26 void FLEXIO\_I2C\_MasterTransferHandleIRQ ( void \* *i2cType*, void \* *i2cHandle* )

## Parameters

|                  |                                                   |
|------------------|---------------------------------------------------|
| <i>i2cType</i>   | Pointer to FLEXIO_I2C_Type structure              |
| <i>i2cHandle</i> | Pointer to flexio_i2c_master_transfer_t structure |

## 18.4 FlexIO I2S Driver

### 18.4.1 Overview

The MCUXpresso SDK provides a peripheral driver for I2S function using Flexible I/O module of MCU-Xpresso SDK devices.

The FlexIO I2S driver includes functional APIs and transactional APIs.

Functional APIs are feature/property target low level APIs.

Functional APIs can be used for FlexIO I2S initialization/configuration/operation for optimization/customization purpose. Using the functional API requires the knowledge of the FlexIO I2S peripheral and how to organize functional APIs to meet the application requirements. All functional API use the peripheral base address as the first parameter. FlexIO I2S functional operation groups provide the functional APIs set.

Transactional APIs are transaction target high level APIs. The transactional APIs can be used to enable the peripheral and also in the application if the code size and performance of transactional APIs can satisfy requirements. If the code size and performance are critical requirements, see the transactional API implementation and write custom code. All transactional APIs use the `sai_handle_t` as the first parameter. Initialize the handle by calling the `FlexIO_I2S_TransferTxCreateHandle()` or `FlexIO_I2S_TransferRxCreateHandle()` API.

Transactional APIs support asynchronous transfer. This means that the functions [FLEXIO\\_I2S\\_TransferSendNonBlocking\(\)](#) and [FLEXIO\\_I2S\\_TransferReceiveNonBlocking\(\)](#) set up an interrupt for data transfer. When the transfer completes, the upper layer is notified through a callback function with the `kStatus_FLEXIO_I2S_TxIdle` and `kStatus_FLEXIO_I2S_RxIdle` status.

### 18.4.2 Typical use case

#### 18.4.2.1 FlexIO I2S send/receive using an interrupt method

```
sai_handle_t g_saiTxHandle;
sai_config_t user_config;
sai_transfer_t sendXfer;
volatile bool txFinished;
volatile bool rxFinished;
const uint8_t sendData[] = [.....];

void FLEXIO_I2S_UserCallback(sai_handle_t *handle, status_t status, void *userData)
{
 userData = userData;

 if (kStatus_FLEXIO_I2S_TxIdle == status)
 {
 txFinished = true;
 }
}

void main(void)
{
 //...

 FLEXIO_I2S_TxGetDefaultConfig(&user_config);
```



```

FLEXIO_I2S_TxInit(FLEXIO_I2S0, &user_config);
FLEXIO_I2S_TransferTxCreateHandle(FLEXIO_I2S0, &g_saiHandle,
 FLEXIO_I2S_UserCallback, NULL);

//Configures the SAI format.
FLEXIO_I2S_TransferTxSetTransferFormat(FLEXIO_I2S0, &g_saiHandle, mclkSource, mclk);

// Prepares to send.
sendXfer.data = sendData
sendXfer.dataSize = sizeof(sendData)/sizeof(sendData[0]);
txFinished = false;

// Sends out.
FLEXIO_I2S_TransferSendNonBlocking(FLEXIO_I2S0, &g_saiHandle, &
 sendXfer);

// Waiting to send is finished.
while (!txFinished)
{
}

// ...
}

```

#### 18.4.2.2 FLEXIO\_I2S send/receive using a DMA method

```

sai_handle_t g_saiHandle;
dma_handle_t g_saiTxDmaHandle;
dma_handle_t g_saiRxDmaHandle;
sai_config_t user_config;
sai_transfer_t sendXfer;
volatile bool txFinished;
uint8_t sendData[] = ...;

void FLEXIO_I2S_UserCallback(sai_handle_t *handle, status_t status, void *userData)
{
 userData = userData;

 if (kStatus_FLEXIO_I2S_TxIdle == status)
 {
 txFinished = true;
 }
}

void main(void)
{
 //...

 FLEXIO_I2S_TxGetDefaultConfig(&user_config);
 FLEXIO_I2S_TxInit(FLEXIO_I2S0, &user_config);

 // Sets up the DMA.
 DMAMUX_Init(DMAMUX0);
 DMAMUX_SetSource(DMAMUX0, FLEXIO_I2S_TX_DMA_CHANNEL, FLEXIO_I2S_TX_DMA_REQUEST);
 DMAMUX_EnableChannel(DMAMUX0, FLEXIO_I2S_TX_DMA_CHANNEL);

 DMA_Init(DMA0);

 /* Creates the DMA handle. */
 DMA_TransferTxCreateHandle(&g_saiTxDmaHandle, DMA0, FLEXIO_I2S_TX_DMA_CHANNEL);

 FLEXIO_I2S_TransferTxCreateHandleDMA(FLEXIO_I2S0, &g_saiTxDmaHandle, FLEXIO_I2S_UserCallback, NULL);

 // Prepares to send.
 sendXfer.data = sendData

```

```

sendXfer.dataSize = sizeof(sendData)/sizeof(sendData[0]);
txFinished = false;

// Sends out.
FLEXIO_I2S_TransferSendDMA(&g_saiHandle, &sendXfer);

// Waiting to send is finished.
while (!txFinished)
{
}

// ...
}

```

## Data Structures

- struct [\\_flexio\\_i2s\\_type](#)  
Define FlexIO I2S access structure typedef. [More...](#)
- struct [\\_flexio\\_i2s\\_config](#)  
FlexIO I2S configure structure. [More...](#)
- struct [\\_flexio\\_i2s\\_format](#)  
FlexIO I2S audio format, FlexIO I2S only support the same format in Tx and Rx. [More...](#)
- struct [\\_flexio\\_i2s\\_transfer](#)  
Define FlexIO I2S transfer structure. [More...](#)
- struct [\\_flexio\\_i2s\\_handle](#)  
Define FlexIO I2S handle structure. [More...](#)

## Macros

- #define [I2S\\_RETRY\\_TIMES](#) 0U /\* Define to zero means keep waiting until the flag is assert/deassert. \*/  
Retry times for waiting flag.
- #define [FLEXIO\\_I2S\\_XFER\\_QUEUE\\_SIZE](#) (4U)  
FlexIO I2S transfer queue size, user can refine it according to use case.

## Typedefs

- typedef struct [\\_flexio\\_i2s\\_type](#) FLEXIO\_I2S\_Type  
Define FlexIO I2S access structure typedef.
- typedef enum  
[\\_flexio\\_i2s\\_master\\_slave](#) flexio\_i2s\_master\_slave\_t  
Master or slave mode.
- typedef struct [\\_flexio\\_i2s\\_config](#) flexio\_i2s\_config\_t  
FlexIO I2S configure structure.
- typedef struct [\\_flexio\\_i2s\\_format](#) flexio\_i2s\_format\_t  
FlexIO I2S audio format, FlexIO I2S only support the same format in Tx and Rx.
- typedef enum  
[\\_flexio\\_i2s\\_sample\\_rate](#) flexio\_i2s\_sample\_rate\_t  
Audio sample rate.
- typedef enum [\\_flexio\\_i2s\\_word\\_width](#) flexio\_i2s\_word\_width\_t

- *Audio word width.*
- typedef struct `_flexio_i2s_transfer flexio_i2s_transfer_t`  
*Define FlexIO I2S transfer structure.*
- typedef void(\* `flexio_i2s_callback_t` )(FLEXIO\_I2S\_Type \*base, `flexio_i2s_handle_t` \*handle, `status_t` status, void \*userData)  
*FlexIO I2S xfer callback prototype.*

## Enumerations

- enum {  
`kStatus_FLEXIO_I2S_Idle` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2S, 0),  
`kStatus_FLEXIO_I2S_TxBusy` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2S, 1),  
`kStatus_FLEXIO_I2S_RxBusy` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2S, 2),  
`kStatus_FLEXIO_I2S_Error` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2S, 3),  
`kStatus_FLEXIO_I2S_QueueFull` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_I2S, 4),  
`kStatus_FLEXIO_I2S_Timeout` }  
*FlexIO I2S transfer status.*
- enum `_flexio_i2s_master_slave` {  
`kFLEXIO_I2S_Master` = 0x0U,  
`kFLEXIO_I2S_Slave` = 0x1U }  
*Master or slave mode.*
- enum {  
`kFLEXIO_I2S_TxDataRegEmptyInterruptEnable` = 0x1U,  
`kFLEXIO_I2S_RxDataRegFullInterruptEnable` = 0x2U }  
*\_flexio\_i2s\_interrupt\_enable Define FlexIO FlexIO I2S interrupt mask.*
- enum {  
`kFLEXIO_I2S_TxDataRegEmptyFlag` = 0x1U,  
`kFLEXIO_I2S_RxDataRegFullFlag` = 0x2U }  
*\_flexio\_i2s\_status\_flags Define FlexIO FlexIO I2S status mask.*
- enum `_flexio_i2s_sample_rate` {  
`kFLEXIO_I2S_SampleRate8KHz` = 8000U,  
`kFLEXIO_I2S_SampleRate11025Hz` = 11025U,  
`kFLEXIO_I2S_SampleRate12KHz` = 12000U,  
`kFLEXIO_I2S_SampleRate16KHz` = 16000U,  
`kFLEXIO_I2S_SampleRate22050Hz` = 22050U,  
`kFLEXIO_I2S_SampleRate24KHz` = 24000U,  
`kFLEXIO_I2S_SampleRate32KHz` = 32000U,  
`kFLEXIO_I2S_SampleRate44100Hz` = 44100U,  
`kFLEXIO_I2S_SampleRate48KHz` = 48000U,  
`kFLEXIO_I2S_SampleRate96KHz` = 96000U }  
*Audio sample rate.*
- enum `_flexio_i2s_word_width` {  
`kFLEXIO_I2S_WordWidth8bits` = 8U,  
`kFLEXIO_I2S_WordWidth16bits` = 16U,  
`kFLEXIO_I2S_WordWidth24bits` = 24U,  
`kFLEXIO_I2S_WordWidth32bits` = 32U }

*Audio word width.*

## Driver version

- #define `FSL_FLEXIO_I2S_DRIVER_VERSION` (`MAKE_VERSION(2, 2, 0)`)  
*FlexIO I2S driver version 2.2.0.*

## Initialization and deinitialization

- void `FLEXIO_I2S_Init` (`FLEXIO_I2S_Type` \*base, const `flexio_i2s_config_t` \*config)  
*Initializes the FlexIO I2S.*
- void `FLEXIO_I2S_GetDefaultConfig` (`flexio_i2s_config_t` \*config)  
*Sets the FlexIO I2S configuration structure to default values.*
- void `FLEXIO_I2S_Deinit` (`FLEXIO_I2S_Type` \*base)  
*De-initializes the FlexIO I2S.*
- static void `FLEXIO_I2S_Enable` (`FLEXIO_I2S_Type` \*base, bool enable)  
*Enables/disables the FlexIO I2S module operation.*

## Status

- uint32\_t `FLEXIO_I2S_GetStatusFlags` (`FLEXIO_I2S_Type` \*base)  
*Gets the FlexIO I2S status flags.*

## Interrupts

- void `FLEXIO_I2S_EnableInterrupts` (`FLEXIO_I2S_Type` \*base, uint32\_t mask)  
*Enables the FlexIO I2S interrupt.*
- void `FLEXIO_I2S_DisableInterrupts` (`FLEXIO_I2S_Type` \*base, uint32\_t mask)  
*Disables the FlexIO I2S interrupt.*

## DMA Control

- static void `FLEXIO_I2S_TxEnableDMA` (`FLEXIO_I2S_Type` \*base, bool enable)  
*Enables/disables the FlexIO I2S Tx DMA requests.*
- static void `FLEXIO_I2S_RxEnableDMA` (`FLEXIO_I2S_Type` \*base, bool enable)  
*Enables/disables the FlexIO I2S Rx DMA requests.*
- static uint32\_t `FLEXIO_I2S_TxGetDataRegisterAddress` (`FLEXIO_I2S_Type` \*base)  
*Gets the FlexIO I2S send data register address.*
- static uint32\_t `FLEXIO_I2S_RxGetDataRegisterAddress` (`FLEXIO_I2S_Type` \*base)  
*Gets the FlexIO I2S receive data register address.*

## Bus Operations

- void `FLEXIO_I2S_MasterSetFormat` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_format_t` \*format, `uint32_t` srcClock\_Hz)  
*Configures the FlexIO I2S audio format in master mode.*
- void `FLEXIO_I2S_SlaveSetFormat` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_format_t` \*format)  
*Configures the FlexIO I2S audio format in slave mode.*
- `status_t` `FLEXIO_I2S_WriteBlocking` (`FLEXIO_I2S_Type` \*base, `uint8_t` bitWidth, `uint8_t` \*tx-Data, `size_t` size)  
*Sends data using a blocking method.*
- static void `FLEXIO_I2S_WriteData` (`FLEXIO_I2S_Type` \*base, `uint8_t` bitWidth, `uint32_t` data)  
*Writes data into a data register.*
- `status_t` `FLEXIO_I2S_ReadBlocking` (`FLEXIO_I2S_Type` \*base, `uint8_t` bitWidth, `uint8_t` \*rx-Data, `size_t` size)  
*Receives a piece of data using a blocking method.*
- static `uint32_t` `FLEXIO_I2S_ReadData` (`FLEXIO_I2S_Type` \*base)  
*Reads a data from the data register.*

## Transactional

- void `FLEXIO_I2S_TransferTxCreateHandle` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle, `flexio_i2s_callback_t` callback, void \*userData)  
*Initializes the FlexIO I2S handle.*
- void `FLEXIO_I2S_TransferSetFormat` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle, `flexio_i2s_format_t` \*format, `uint32_t` srcClock\_Hz)  
*Configures the FlexIO I2S audio format.*
- void `FLEXIO_I2S_TransferRxCreateHandle` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle, `flexio_i2s_callback_t` callback, void \*userData)  
*Initializes the FlexIO I2S receive handle.*
- `status_t` `FLEXIO_I2S_TransferSendNonBlocking` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle, `flexio_i2s_transfer_t` \*xfer)  
*Performs an interrupt non-blocking send transfer on FlexIO I2S.*
- `status_t` `FLEXIO_I2S_TransferReceiveNonBlocking` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle, `flexio_i2s_transfer_t` \*xfer)  
*Performs an interrupt non-blocking receive transfer on FlexIO I2S.*
- void `FLEXIO_I2S_TransferAbortSend` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle)  
*Aborts the current send.*
- void `FLEXIO_I2S_TransferAbortReceive` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle)  
*Aborts the current receive.*
- `status_t` `FLEXIO_I2S_TransferGetSendCount` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle, `size_t` \*count)  
*Gets the remaining bytes to be sent.*
- `status_t` `FLEXIO_I2S_TransferGetReceiveCount` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_handle_t` \*handle, `size_t` \*count)  
*Gets the remaining bytes to be received.*
- void `FLEXIO_I2S_TransferTxHandleIRQ` (void \*i2sBase, void \*i2sHandle)  
*Tx interrupt handler.*
- void `FLEXIO_I2S_TransferRxHandleIRQ` (void \*i2sBase, void \*i2sHandle)

*Rx interrupt handler.*

### 18.4.3 Data Structure Documentation

#### 18.4.3.1 struct \_flexio\_i2s\_type

##### Data Fields

- FLEXIO\_Type \* [flexioBase](#)  
*FlexIO base pointer.*
- uint8\_t [txPinIndex](#)  
*Tx data pin index in FlexIO pins.*
- uint8\_t [rxPinIndex](#)  
*Rx data pin index.*
- uint8\_t [bclkPinIndex](#)  
*Bit clock pin index.*
- uint8\_t [fsPinIndex](#)  
*Frame sync pin index.*
- uint8\_t [txShifterIndex](#)  
*Tx data shifter index.*
- uint8\_t [rxShifterIndex](#)  
*Rx data shifter index.*
- uint8\_t [bclkTimerIndex](#)  
*Bit clock timer index.*
- uint8\_t [fsTimerIndex](#)  
*Frame sync timer index.*

#### 18.4.3.2 struct \_flexio\_i2s\_config

##### Data Fields

- bool [enableI2S](#)  
*Enable FlexIO I2S.*
- [flexio\\_i2s\\_master\\_slave\\_t](#) masterSlave  
*Master or slave.*
- [flexio\\_pin\\_polarity\\_t](#) txPinPolarity  
*Tx data pin polarity, active high or low.*
- [flexio\\_pin\\_polarity\\_t](#) rxPinPolarity  
*Rx data pin polarity.*
- [flexio\\_pin\\_polarity\\_t](#) bclkPinPolarity  
*Bit clock pin polarity.*
- [flexio\\_pin\\_polarity\\_t](#) fsPinPolarity  
*Frame sync pin polarity.*
- [flexio\\_shifter\\_timer\\_polarity\\_t](#) txTimerPolarity  
*Tx data valid on bclk rising or falling edge.*
- [flexio\\_shifter\\_timer\\_polarity\\_t](#) rxTimerPolarity  
*Rx data valid on bclk rising or falling edge.*

### 18.4.3.3 struct \_flexio\_i2s\_format

#### Data Fields

- uint8\_t [bitWidth](#)  
*Bit width of audio data, always 8/16/24/32 bits.*
- uint32\_t [sampleRate\\_Hz](#)  
*Sample rate of the audio data.*

### 18.4.3.4 struct \_flexio\_i2s\_transfer

#### Data Fields

- uint8\_t \* [data](#)  
*Data buffer start pointer.*
- size\_t [dataSize](#)  
*Bytes to be transferred.*

#### Field Documentation

(1) [size\\_t \\_flexio\\_i2s\\_transfer::dataSize](#)

### 18.4.3.5 struct \_flexio\_i2s\_handle

#### Data Fields

- uint32\_t [state](#)  
*Internal state.*
- [flexio\\_i2s\\_callback\\_t](#) [callback](#)  
*Callback function called at transfer event.*
- void \* [userData](#)  
*Callback parameter passed to callback function.*
- uint8\_t [bitWidth](#)  
*Bit width for transfer, 8/16/24/32bits.*
- [flexio\\_i2s\\_transfer\\_t](#) [queue](#) [[FLEXIO\\_I2S\\_XFER\\_QUEUE\\_SIZE](#)]  
*Transfer queue storing queued transfer.*
- size\_t [transferSize](#) [[FLEXIO\\_I2S\\_XFER\\_QUEUE\\_SIZE](#)]  
*Data bytes need to transfer.*
- volatile uint8\_t [queueUser](#)  
*Index for user to queue transfer.*
- volatile uint8\_t [queueDriver](#)  
*Index for driver to get the transfer data and size.*

## 18.4.4 Macro Definition Documentation

18.4.4.1 **#define FSL\_FLEXIO\_I2S\_DRIVER\_VERSION (MAKE\_VERSION(2, 2, 0))**

18.4.4.2 **#define I2S\_RETRY\_TIMES 0U** /\* Define to zero means keep waiting until the flag is assert/deassert. \*/

18.4.4.3 **#define FLEXIO\_I2S\_XFER\_QUEUE\_SIZE (4U)**

## 18.4.5 Typedef Documentation

18.4.5.1 **typedef struct \_flexio\_i2s\_transfer flexio\_i2s\_transfer\_t**

## 18.4.6 Enumeration Type Documentation

### 18.4.6.1 anonymous enum

Enumerator

*kStatus\_FLEXIO\_I2S\_Idle* FlexIO I2S is in idle state.  
*kStatus\_FLEXIO\_I2S\_TxBusy* FlexIO I2S Tx is busy.  
*kStatus\_FLEXIO\_I2S\_RxBusy* FlexIO I2S Rx is busy.  
*kStatus\_FLEXIO\_I2S\_Error* FlexIO I2S error occurred.  
*kStatus\_FLEXIO\_I2S\_QueueFull* FlexIO I2S transfer queue is full.  
*kStatus\_FLEXIO\_I2S\_Timeout* FlexIO I2S timeout polling status flags.

### 18.4.6.2 enum \_flexio\_i2s\_master\_slave

Enumerator

*kFLEXIO\_I2S\_Master* Master mode.  
*kFLEXIO\_I2S\_Slave* Slave mode.

### 18.4.6.3 anonymous enum

Enumerator

*kFLEXIO\_I2S\_TxDataRegEmptyInterruptEnable* Transmit buffer empty interrupt enable.  
*kFLEXIO\_I2S\_RxDataRegFullInterruptEnable* Receive buffer full interrupt enable.

### 18.4.6.4 anonymous enum

Enumerator

*kFLEXIO\_I2S\_TxDataRegEmptyFlag* Transmit buffer empty flag.



*kFLEXIO\_I2S\_RxDataRegFullFlag* Receive buffer full flag.

#### 18.4.6.5 enum \_flexio\_i2s\_sample\_rate

Enumerator

*kFLEXIO\_I2S\_SampleRate8KHz* Sample rate 8000Hz.  
*kFLEXIO\_I2S\_SampleRate11025Hz* Sample rate 11025Hz.  
*kFLEXIO\_I2S\_SampleRate12KHz* Sample rate 12000Hz.  
*kFLEXIO\_I2S\_SampleRate16KHz* Sample rate 16000Hz.  
*kFLEXIO\_I2S\_SampleRate22050Hz* Sample rate 22050Hz.  
*kFLEXIO\_I2S\_SampleRate24KHz* Sample rate 24000Hz.  
*kFLEXIO\_I2S\_SampleRate32KHz* Sample rate 32000Hz.  
*kFLEXIO\_I2S\_SampleRate44100Hz* Sample rate 44100Hz.  
*kFLEXIO\_I2S\_SampleRate48KHz* Sample rate 48000Hz.  
*kFLEXIO\_I2S\_SampleRate96KHz* Sample rate 96000Hz.

#### 18.4.6.6 enum \_flexio\_i2s\_word\_width

Enumerator

*kFLEXIO\_I2S\_WordWidth8bits* Audio data width 8 bits.  
*kFLEXIO\_I2S\_WordWidth16bits* Audio data width 16 bits.  
*kFLEXIO\_I2S\_WordWidth24bits* Audio data width 24 bits.  
*kFLEXIO\_I2S\_WordWidth32bits* Audio data width 32 bits.

### 18.4.7 Function Documentation

#### 18.4.7.1 void FLEXIO\_I2S\_Init ( FLEXIO\_I2S\_Type \* *base*, const flexio\_i2s\_config\_t \* *config* )

This API configures FlexIO pins and shifter to I2S and configures the FlexIO I2S with a configuration structure. The configuration structure can be filled by the user, or be set with default values by [FLEXIO\\_I2S\\_GetDefaultConfig\(\)](#).

Note

This API should be called at the beginning of the application to use the FlexIO I2S driver. Otherwise, any access to the FlexIO I2S module can cause hard fault because the clock is not enabled.

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | FlexIO I2S base pointer         |
| <i>config</i> | FlexIO I2S configure structure. |

**18.4.7.2 void FLEXIO\_I2S\_GetDefaultConfig ( flexio\_i2s\_config\_t \* *config* )**

The purpose of this API is to get the configuration structure initialized for use in [FLEXIO\\_I2S\\_Init\(\)](#). Users may use the initialized structure unchanged in [FLEXIO\\_I2S\\_Init\(\)](#) or modify some fields of the structure before calling [FLEXIO\\_I2S\\_Init\(\)](#).

## Parameters

|               |                                           |
|---------------|-------------------------------------------|
| <i>config</i> | pointer to master configuration structure |
|---------------|-------------------------------------------|

**18.4.7.3 void FLEXIO\_I2S\_Deinit ( FLEXIO\_I2S\_Type \* *base* )**

Calling this API resets the FlexIO I2S shifter and timer config. After calling this API, call the FLEXIO\_I2S\_Init to use the FlexIO I2S module.

## Parameters

|             |                         |
|-------------|-------------------------|
| <i>base</i> | FlexIO I2S base pointer |
|-------------|-------------------------|

**18.4.7.4 static void FLEXIO\_I2S\_Enable ( FLEXIO\_I2S\_Type \* *base*, bool *enable* )  
[inline], [static]**

## Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type                      |
| <i>enable</i> | True to enable, false dose not have any effect. |

**18.4.7.5 uint32\_t FLEXIO\_I2S\_GetStatusFlags ( FLEXIO\_I2S\_Type \* *base* )**

## Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2S_Type structure |
|-------------|--------------------------------------|

Returns

Status flag, which are ORed by the enumerators in the `_flexio_i2s_status_flags`.

#### 18.4.7.6 void FLEXIO\_I2S\_EnableInterrupts ( FLEXIO\_I2S\_Type \* *base*, uint32\_t *mask* )

This function enables the FlexIO UART interrupt.

Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2S_Type structure |
| <i>mask</i> | interrupt source                     |

#### 18.4.7.7 void FLEXIO\_I2S\_DisableInterrupts ( FLEXIO\_I2S\_Type \* *base*, uint32\_t *mask* )

This function enables the FlexIO UART interrupt.

Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>base</i> | pointer to FLEXIO_I2S_Type structure |
| <i>mask</i> | interrupt source                     |

#### 18.4.7.8 static void FLEXIO\_I2S\_TxEnableDMA ( FLEXIO\_I2S\_Type \* *base*, bool *enable* ) [inline], [static]

Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>base</i>   | FlexIO I2S base pointer                         |
| <i>enable</i> | True means enable DMA, false means disable DMA. |

#### 18.4.7.9 static void FLEXIO\_I2S\_RxEnableDMA ( FLEXIO\_I2S\_Type \* *base*, bool *enable* ) [inline], [static]

## Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>base</i>   | FlexIO I2S base pointer                         |
| <i>enable</i> | True means enable DMA, false means disable DMA. |

#### 18.4.7.10 **static uint32\_t FLEXIO\_I2S\_TxGetDataRegisterAddress ( FLEXIO\_I2S\_Type \* *base* ) [inline], [static]**

This function returns the I2S data register address, mainly used by DMA/eDMA.

## Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2S_Type structure |
|-------------|--------------------------------------|

## Returns

FlexIO i2s send data register address.

#### 18.4.7.11 **static uint32\_t FLEXIO\_I2S\_RxGetDataRegisterAddress ( FLEXIO\_I2S\_Type \* *base* ) [inline], [static]**

This function returns the I2S data register address, mainly used by DMA/eDMA.

## Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>base</i> | Pointer to FLEXIO_I2S_Type structure |
|-------------|--------------------------------------|

## Returns

FlexIO i2s receive data register address.

#### 18.4.7.12 **void FLEXIO\_I2S\_MasterSetFormat ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_format\_t \* *format*, uint32\_t *srcClock\_Hz* )**

Audio format can be changed in run-time of FlexIO I2S. This function configures the sample rate and audio data format to be transferred.

## Parameters

|                    |                                                    |
|--------------------|----------------------------------------------------|
| <i>base</i>        | Pointer to FLEXIO_I2S_Type structure               |
| <i>format</i>      | Pointer to FlexIO I2S audio data format structure. |
| <i>srcClock_Hz</i> | I2S master clock source frequency in Hz.           |

#### 18.4.7.13 void FLEXIO\_I2S\_SlaveSetFormat ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_format\_t \* *format* )

Audio format can be changed in run-time of FlexIO I2S. This function configures the sample rate and audio data format to be transferred.

## Parameters

|               |                                                    |
|---------------|----------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type structure               |
| <i>format</i> | Pointer to FlexIO I2S audio data format structure. |

#### 18.4.7.14 status\_t FLEXIO\_I2S\_WriteBlocking ( FLEXIO\_I2S\_Type \* *base*, uint8\_t *bitWidth*, uint8\_t \* *txData*, size\_t *size* )

## Note

This function blocks via polling until data is ready to be sent.

## Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>base</i>     | FlexIO I2S base pointer.                                |
| <i>bitWidth</i> | How many bits in a audio word, usually 8/16/24/32 bits. |
| <i>txData</i>   | Pointer to the data to be written.                      |
| <i>size</i>     | Bytes to be written.                                    |

## Return values

|                                   |                               |
|-----------------------------------|-------------------------------|
| <i>kStatus_Success</i>            | Successfully write data.      |
| <i>kStatus_FLEXIO_I2C_Timeout</i> | Timeout polling status flags. |

#### 18.4.7.15 static void FLEXIO\_I2S\_WriteData ( FLEXIO\_I2S\_Type \* *base*, uint8\_t *bitWidth*, uint32\_t *data* ) [inline], [static]

## Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>base</i>     | FlexIO I2S base pointer.                                |
| <i>bitWidth</i> | How many bits in a audio word, usually 8/16/24/32 bits. |
| <i>data</i>     | Data to be written.                                     |

#### 18.4.7.16 **status\_t FLEXIO\_I2S\_ReadBlocking ( FLEXIO\_I2S\_Type \* *base*, uint8\_t *bitWidth*, uint8\_t \* *rxData*, size\_t *size* )**

## Note

This function blocks via polling until data is ready to be sent.

## Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>base</i>     | FlexIO I2S base pointer                                 |
| <i>bitWidth</i> | How many bits in a audio word, usually 8/16/24/32 bits. |
| <i>rxData</i>   | Pointer to the data to be read.                         |
| <i>size</i>     | Bytes to be read.                                       |

## Return values

|                                    |                               |
|------------------------------------|-------------------------------|
| <i>kStatus_Success</i>             | Successfully read data.       |
| <i>kStatus_FLEXIO_I2C_-Timeout</i> | Timeout polling status flags. |

#### 18.4.7.17 **static uint32\_t FLEXIO\_I2S\_ReadData ( FLEXIO\_I2S\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                         |
|-------------|-------------------------|
| <i>base</i> | FlexIO I2S base pointer |
|-------------|-------------------------|

## Returns

Data read from data register.

**18.4.7.18 void FLEXIO\_I2S\_TransferTxCreateHandle ( FLEXIO\_I2S\_Type \* *base*,  
flexio\_i2s\_handle\_t \* *handle*, flexio\_i2s\_callback\_t *callback*, void \* *userData* )**

This function initializes the FlexIO I2S handle which can be used for other FlexIO I2S transactional APIs. Call this API once to get the initialized handle.

## Parameters

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>base</i>     | Pointer to FLEXIO_I2S_Type structure                                  |
| <i>handle</i>   | Pointer to flexio_i2s_handle_t structure to store the transfer state. |
| <i>callback</i> | FlexIO I2S callback function, which is called while finished a block. |
| <i>userData</i> | User parameter for the FlexIO I2S callback.                           |

**18.4.7.19 void FLEXIO\_I2S\_TransferSetFormat ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_handle\_t \* *handle*, flexio\_i2s\_format\_t \* *format*, uint32\_t *srcClock\_Hz* )**

Audio format can be changed at run-time of FlexIO I2S. This function configures the sample rate and audio data format to be transferred.

## Parameters

|                    |                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------|
| <i>base</i>        | Pointer to FLEXIO_I2S_Type structure.                                                        |
| <i>handle</i>      | FlexIO I2S handle pointer.                                                                   |
| <i>format</i>      | Pointer to audio data format structure.                                                      |
| <i>srcClock_Hz</i> | FlexIO I2S bit clock source frequency in Hz. This parameter should be 0 while in slave mode. |

**18.4.7.20 void FLEXIO\_I2S\_TransferRxCreateHandle ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_handle\_t \* *handle*, flexio\_i2s\_callback\_t *callback*, void \* *userData* )**

This function initializes the FlexIO I2S handle which can be used for other FlexIO I2S transactional APIs. Call this API once to get the initialized handle.

## Parameters

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>base</i>     | Pointer to FLEXIO_I2S_Type structure.                                 |
| <i>handle</i>   | Pointer to flexio_i2s_handle_t structure to store the transfer state. |
| <i>callback</i> | FlexIO I2S callback function, which is called while finished a block. |
| <i>userData</i> | User parameter for the FlexIO I2S callback.                           |

**18.4.7.21 status\_t FLEXIO\_I2S\_TransferSendNonBlocking ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_handle\_t \* *handle*, flexio\_i2s\_transfer\_t \* *xfer* )**



## Note

The API returns immediately after transfer initiates. Call FLEXIO\_I2S\_GetRemainingBytes to poll the transfer status and check whether the transfer is finished. If the return status is 0, the transfer is finished.

## Parameters

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type structure.                                    |
| <i>handle</i> | Pointer to flexio_i2s_handle_t structure which stores the transfer state |
| <i>xfer</i>   | Pointer to flexio_i2s_transfer_t structure                               |

## Return values

|                                   |                                                                                    |
|-----------------------------------|------------------------------------------------------------------------------------|
| <i>kStatus_Success</i>            | Successfully start the data transmission.                                          |
| <i>kStatus_FLEXIO_I2S_Tx-Busy</i> | Previous transmission still not finished, data not all written to TX register yet. |
| <i>kStatus_InvalidArgument</i>    | The input parameter is invalid.                                                    |

#### 18.4.7.22 status\_t FLEXIO\_I2S\_TransferReceiveNonBlocking ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_handle\_t \* *handle*, flexio\_i2s\_transfer\_t \* *xfer* )

## Note

The API returns immediately after transfer initiates. Call FLEXIO\_I2S\_GetRemainingBytes to poll the transfer status to check whether the transfer is finished. If the return status is 0, the transfer is finished.

## Parameters

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type structure.                                    |
| <i>handle</i> | Pointer to flexio_i2s_handle_t structure which stores the transfer state |
| <i>xfer</i>   | Pointer to flexio_i2s_transfer_t structure                               |

## Return values

|                        |                                      |
|------------------------|--------------------------------------|
| <i>kStatus_Success</i> | Successfully start the data receive. |
|------------------------|--------------------------------------|

|                                  |                                      |
|----------------------------------|--------------------------------------|
| <i>kStatus_FLEXIO_I2S_RxBusy</i> | Previous receive still not finished. |
| <i>kStatus_InvalidArgument</i>   | The input parameter is invalid.      |

#### 18.4.7.23 void FLEXIO\_I2S\_TransferAbortSend ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_handle\_t \* *handle* )

##### Note

This API can be called at any time when interrupt non-blocking transfer initiates to abort the transfer in a early time.

##### Parameters

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type structure.                                    |
| <i>handle</i> | Pointer to flexio_i2s_handle_t structure which stores the transfer state |

#### 18.4.7.24 void FLEXIO\_I2S\_TransferAbortReceive ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_handle\_t \* *handle* )

##### Note

This API can be called at any time when interrupt non-blocking transfer initiates to abort the transfer in a early time.

##### Parameters

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type structure.                                    |
| <i>handle</i> | Pointer to flexio_i2s_handle_t structure which stores the transfer state |

#### 18.4.7.25 status\_t FLEXIO\_I2S\_TransferGetSendCount ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_handle\_t \* *handle*, size\_t \* *count* )

##### Parameters

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type structure.                                    |
| <i>handle</i> | Pointer to flexio_i2s_handle_t structure which stores the transfer state |
| <i>count</i>  | Bytes sent.                                                              |

Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

**18.4.7.26** `status_t FLEXIO_I2S_TransferGetReceiveCount ( FLEXIO_I2S_Type * base, flexio_i2s_handle_t * handle, size_t * count )`

Parameters

|               |                                                                          |
|---------------|--------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_I2S_Type structure.                                    |
| <i>handle</i> | Pointer to flexio_i2s_handle_t structure which stores the transfer state |
| <i>count</i>  | Bytes recieved.                                                          |

Returns

*count* Bytes received.

Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

**18.4.7.27** `void FLEXIO_I2S_TransferTxHandleIRQ ( void * i2sBase, void * i2sHandle )`

Parameters

|                  |                                          |
|------------------|------------------------------------------|
| <i>i2sBase</i>   | Pointer to FLEXIO_I2S_Type structure.    |
| <i>i2sHandle</i> | Pointer to flexio_i2s_handle_t structure |

**18.4.7.28** `void FLEXIO_I2S_TransferRxHandleIRQ ( void * i2sBase, void * i2sHandle )`

## Parameters

|                  |                                           |
|------------------|-------------------------------------------|
| <i>i2sBase</i>   | Pointer to FLEXIO_I2S_Type structure.     |
| <i>i2sHandle</i> | Pointer to flexio_i2s_handle_t structure. |

## 18.5 FlexIO SPI Driver

### 18.5.1 Overview

The MCUXpresso SDK provides a peripheral driver for an SPI function using the Flexible I/O module of MCUXpresso SDK devices.

FlexIO SPI driver includes functional APIs and transactional APIs.

Functional APIs target low-level APIs. Functional APIs can be used for FlexIO SPI initialization/configuration/operation for optimization/customization purpose. Using the functional API requires the knowledge of the FlexIO SPI peripheral and how to organize functional APIs to meet the application requirements. All functional API use the `FLEXIO_SPI_Type *base` as the first parameter. FlexIO SPI functional operation groups provide the functional API set.

Transactional APIs target high-level APIs. Transactional APIs can be used to enable the peripheral and also in the application if the code size and performance of transactional APIs can satisfy requirements. If the code size and performance are critical requirements, see the transactional API implementation and write custom code. All transactional APIs use the `flexio_spi_master_handle_t/flexio_spi_slave_handle_t` as the second parameter. Initialize the handle by calling the [FLEXIO\\_SPI\\_MasterTransferCreateHandle\(\)](#) or [FLEXIO\\_SPI\\_SlaveTransferCreateHandle\(\)](#) API.

Transactional APIs support asynchronous transfer. This means that the functions [FLEXIO\\_SPI\\_MasterTransferNonBlocking\(\)](#)/[FLEXIO\\_SPI\\_SlaveTransferNonBlocking\(\)](#) set up an interrupt for data transfer. When the transfer is complete, the upper layer is notified through a callback function with the `kStatus_FLEXIO_SPI_Idle` status.

Note that the FlexIO SPI slave driver only supports discontinuous PCS access, which is a limitation. The FlexIO SPI slave driver can support continuous PCS, but the slave cannot adapt discontinuous and continuous PCS automatically. Users can change the timer disable mode in `FLEXIO_SPI_SlaveInit` manually, from `kFLEXIO_TimerDisableOnTimerCompare` to `kFLEXIO_TimerDisableNever` to enable a discontinuous PCS access. Only `CPHA = 0` is supported.

### 18.5.2 Typical use case

#### 18.5.2.1 FlexIO SPI send/receive using an interrupt method

```
flexio_spi_master_handle_t g_spiHandle;
FLEXIO_SPI_Type spiDev;
volatile bool txFinished;
static uint8_t srcBuff[BUFFER_SIZE];
static uint8_t destBuff[BUFFER_SIZE];

void FLEXIO_SPI_MasterUserCallback(FLEXIO_SPI_Type *base,
 flexio_spi_master_handle_t *handle, status_t status, void *userData)
{
 userData = userData;

 if (kStatus_FLEXIO_SPI_Idle == status)
 {
 txFinished = true;
 }
}
```

```

}

void main(void)
{
 //...
 flexio_spi_transfer_t xfer = {0};
 flexio_spi_master_config_t userConfig;

 FLEXIO_SPI_MasterGetDefaultConfig(&userConfig);
 userConfig.baudRate_Bps = 500000U;

 spiDev.flexioBase = BOARD_FLEXIO_BASE;
 spiDev.SDOPinIndex = FLEXIO_SPI_MOSI_PIN;
 spiDev.SDIPinIndex = FLEXIO_SPI_MISO_PIN;
 spiDev.SCKPinIndex = FLEXIO_SPI_SCK_PIN;
 spiDev.CSnPinIndex = FLEXIO_SPI_CSn_PIN;
 spiDev.shifterIndex[0] = 0U;
 spiDev.shifterIndex[1] = 1U;
 spiDev.timerIndex[0] = 0U;
 spiDev.timerIndex[1] = 1U;

 FLEXIO_SPI_MasterInit(&spiDev, &userConfig, FLEXIO_CLOCK_FREQUENCY);

 xfer.txData = srcBuff;
 xfer.rxData = destBuff;
 xfer.dataSize = BUFFER_SIZE;
 xfer.flags = kFLEXIO_SPI_8bitMsb;
 FLEXIO_SPI_MasterTransferCreateHandle(&spiDev, &g_spiHandle,
 FLEXIO_SPI_MasterUserCallback, NULL);
 FLEXIO_SPI_MasterTransferNonBlocking(&spiDev, &g_spiHandle, &xfer);

 // Send finished.
 while (!txFinished)
 {
 // ...
 }
}

```

### 18.5.2.2 FlexIO\_SPI Send/Receive in DMA way

```

dma_handle_t g_spiTxDmaHandle;
dma_handle_t g_spiRxDmaHandle;
flexio_spi_master_handle_t g_spiHandle;
FLEXIO_SPI_Type spiDev;
volatile bool txFinished;
static uint8_t srcBuff[BUFFER_SIZE];
static uint8_t destBuff[BUFFER_SIZE];
void FLEXIO_SPI_MasterUserCallback(FLEXIO_SPI_Type *base, flexio_spi_master_dma_handle_t
 *handle, status_t status, void *userData)
{
 userData = userData;

 if (kStatus_FLEXIO_SPI_Idle == status)
 {
 txFinished = true;
 }
}

void main(void)
{
 flexio_spi_transfer_t xfer = {0};
 flexio_spi_master_config_t userConfig;

 FLEXIO_SPI_MasterGetDefaultConfig(&userConfig);

```

```

userConfig.baudRate_Bps = 500000U;

spiDev.flexioBase = BOARD_FLEXIO_BASE;
spiDev.SDOPinIndex = FLEXIO_SPI_MOSI_PIN;
spiDev.SDIPinIndex = FLEXIO_SPI_MISO_PIN;
spiDev.SCKPinIndex = FLEXIO_SPI_SCK_PIN;
spiDev.CSnPinIndex = FLEXIO_SPI_CSn_PIN;
spiDev.shifterIndex[0] = 0U;
spiDev.shifterIndex[1] = 1U;
spiDev.timerIndex[0] = 0U;
spiDev.timerIndex[1] = 1U;

/* Init DMAMUX. */
DMAMUX_Init(EXAMPLE_FLEXIO_SPI_DMAMUX_BASEADDR)

/* Init the DMA/EDMA module */
#if defined(FSL_FEATURE_SOC_DMA_COUNT) && FSL_FEATURE_SOC_DMA_COUNT > 0U
 DMA_Init(EXAMPLE_FLEXIO_SPI_DMA_BASEADDR);
 DMA_CreateHandle(&txHandle, EXAMPLE_FLEXIO_SPI_DMA_BASEADDR, FLEXIO_SPI_TX_DMA_CHANNEL);
 DMA_CreateHandle(&rxHandle, EXAMPLE_FLEXIO_SPI_DMA_BASEADDR, FLEXIO_SPI_RX_DMA_CHANNEL);
#endif /* FSL_FEATURE_SOC_DMA_COUNT */

#if defined(FSL_FEATURE_SOC_EDMA_COUNT) && FSL_FEATURE_SOC_EDMA_COUNT > 0U
 edma_config_t edmaConfig;

 EDMA_GetDefaultConfig(&edmaConfig);
 EDMA_Init(EXAMPLE_FLEXIO_SPI_DMA_BASEADDR, &edmaConfig);
 EDMA_CreateHandle(&txHandle, EXAMPLE_FLEXIO_SPI_DMA_BASEADDR,
FLEXIO_SPI_TX_DMA_CHANNEL);
 EDMA_CreateHandle(&rxHandle, EXAMPLE_FLEXIO_SPI_DMA_BASEADDR,
FLEXIO_SPI_RX_DMA_CHANNEL);
#endif /* FSL_FEATURE_SOC_EDMA_COUNT */

 dma_request_source_tx = (dma_request_source_t)(FLEXIO_DMA_REQUEST_BASE + spiDev.
shifterIndex[0]);
 dma_request_source_rx = (dma_request_source_t)(FLEXIO_DMA_REQUEST_BASE + spiDev.
shifterIndex[1]);

 /* Requests DMA channels for transmit and receive. */
 DMAMUX_SetSource(EXAMPLE_FLEXIO_SPI_DMAMUX_BASEADDR, FLEXIO_SPI_TX_DMA_CHANNEL, (
dma_request_source_t)dma_request_source_tx);
 DMAMUX_SetSource(EXAMPLE_FLEXIO_SPI_DMAMUX_BASEADDR, FLEXIO_SPI_RX_DMA_CHANNEL, (
dma_request_source_t)dma_request_source_rx);
 DMAMUX_EnableChannel(EXAMPLE_FLEXIO_SPI_DMAMUX_BASEADDR,
FLEXIO_SPI_TX_DMA_CHANNEL);
 DMAMUX_EnableChannel(EXAMPLE_FLEXIO_SPI_DMAMUX_BASEADDR,
FLEXIO_SPI_RX_DMA_CHANNEL);

 FLEXIO_SPI_MasterInit(&spiDev, &userConfig, FLEXIO_CLOCK_FREQUENCY);

/* Initializes the buffer. */
for (i = 0; i < BUFFER_SIZE; i++)
{
 srcBuff[i] = i;
}

/* Sends to the slave. */
xfer.txData = srcBuff;
xfer.rxData = destBuff;
xfer.dataSize = BUFFER_SIZE;
xfer.flags = kFLEXIO_SPI_8bitMsb;
FLEXIO_SPI_MasterTransferCreateHandleDMA(&spiDev, &g_spiHandle, FLEXIO_SPI_MasterUserCallback, NULL
, &g_spiTxDmaHandle, &g_spiRxDmaHandle);
FLEXIO_SPI_MasterTransferDMA(&spiDev, &g_spiHandle, &xfer);

// Send finished.
while (!txFinished)
{

```

```

 }
 // ...
}

```

## Data Structures

- struct `_flexio_spi_type`  
Define FlexIO SPI access structure typedef. [More...](#)
- struct `_flexio_spi_master_config`  
Define FlexIO SPI master configuration structure. [More...](#)
- struct `_flexio_spi_slave_config`  
Define FlexIO SPI slave configuration structure. [More...](#)
- struct `_flexio_spi_transfer`  
Define FlexIO SPI transfer structure. [More...](#)
- struct `_flexio_spi_master_handle`  
Define FlexIO SPI handle structure. [More...](#)

## Macros

- #define `FLEXIO_SPI_DUMMYDATA` (0x00U)  
FlexIO SPI dummy transfer data, the data is sent while txData is NULL.
- #define `SPI_RETRY_TIMES` 0U /\* Define to zero means keep waiting until the flag is assert/deassert. \*/  
Retry times for waiting flag.
- #define `FLEXIO_SPI_XFER_DATA_FORMAT`(flag) ((flag) & (0x7U))  
Get the transfer data format of width and bit order.

## Typedefs

- typedef enum  
`_flexio_spi_clock_phase flexio_spi_clock_phase_t`  
FlexIO SPI clock phase configuration.
- typedef enum  
`_flexio_spi_shift_direction flexio_spi_shift_direction_t`  
FlexIO SPI data shifter direction options.
- typedef enum  
`_flexio_spi_data_bitcount_mode flexio_spi_data_bitcount_mode_t`  
FlexIO SPI data length mode options.
- typedef struct `_flexio_spi_type FLEXIO_SPI_Type`  
Define FlexIO SPI access structure typedef.
- typedef struct  
`_flexio_spi_master_config flexio_spi_master_config_t`  
Define FlexIO SPI master configuration structure.
- typedef struct  
`_flexio_spi_slave_config flexio_spi_slave_config_t`  
Define FlexIO SPI slave configuration structure.



- typedef struct `_flexio_spi_transfer` `flexio_spi_transfer_t`  
*Define FlexIO SPI transfer structure.*
- typedef struct  
`_flexio_spi_master_handle` `flexio_spi_master_handle_t`  
*typedef for flexio\_spi\_master\_handle\_t in advance.*
- typedef `flexio_spi_master_handle_t` `flexio_spi_slave_handle_t`  
*Slave handle is the same with master handle.*
- typedef void(\* `flexio_spi_master_transfer_callback_t` )(FLEXIO\_SPI\_Type \*base, `flexio_spi_master_handle_t` \*handle, `status_t` status, void \*userData)  
*FlexIO SPI master callback for finished transmit.*
- typedef void(\* `flexio_spi_slave_transfer_callback_t` )(FLEXIO\_SPI\_Type \*base, `flexio_spi_slave_handle_t` \*handle, `status_t` status, void \*userData)  
*FlexIO SPI slave callback for finished transmit.*

## Enumerations

- enum {  
`kStatus_FLEXIO_SPI_Busy` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_SPI, 1),  
`kStatus_FLEXIO_SPI_Idle` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_SPI, 2),  
`kStatus_FLEXIO_SPI_Error` = MAKE\_STATUS(kStatusGroup\_FLEXIO\_SPI, 3),  
`kStatus_FLEXIO_SPI_Timeout` }  
*Error codes for the FlexIO SPI driver.*
- enum `_flexio_spi_clock_phase` {  
`kFLEXIO_SPI_ClockPhaseFirstEdge` = 0x0U,  
`kFLEXIO_SPI_ClockPhaseSecondEdge` = 0x1U }  
*FlexIO SPI clock phase configuration.*
- enum `_flexio_spi_shift_direction` {  
`kFLEXIO_SPI_MsbFirst` = 0,  
`kFLEXIO_SPI_LsbFirst` = 1 }  
*FlexIO SPI data shifter direction options.*
- enum `_flexio_spi_data_bitcount_mode` {  
`kFLEXIO_SPI_8BitMode` = 0x08U,  
`kFLEXIO_SPI_16BitMode` = 0x10U,  
`kFLEXIO_SPI_32BitMode` = 0x20U }  
*FlexIO SPI data length mode options.*
- enum `_flexio_spi_interrupt_enable` {  
`kFLEXIO_SPI_TxEmptyInterruptEnable` = 0x1U,  
`kFLEXIO_SPI_RxFullInterruptEnable` = 0x2U }  
*Define FlexIO SPI interrupt mask.*
- enum `_flexio_spi_status_flags` {  
`kFLEXIO_SPI_TxBufferEmptyFlag` = 0x1U,  
`kFLEXIO_SPI_RxBufferFullFlag` = 0x2U }  
*Define FlexIO SPI status mask.*
- enum `_flexio_spi_dma_enable` {  
`kFLEXIO_SPI_TxDmaEnable` = 0x1U,  
`kFLEXIO_SPI_RxDmaEnable` = 0x2U,  
`kFLEXIO_SPI_DmaAllEnable` = 0x3U }

- *Define FlexIO SPI DMA mask.*
- enum `_flexio_spi_transfer_flags` {  
`kFLEXIO_SPI_8bitMsb` = 0x0U,  
`kFLEXIO_SPI_8bitLsb` = 0x1U,  
`kFLEXIO_SPI_16bitMsb` = 0x2U,  
`kFLEXIO_SPI_16bitLsb` = 0x3U,  
`kFLEXIO_SPI_32bitMsb` = 0x4U,  
`kFLEXIO_SPI_32bitLsb` = 0x5U,  
`kFLEXIO_SPI_csContinuous` = 0x8U }  
*Define FlexIO SPI transfer flags.*

## Driver version

- #define `FSL_FLEXIO_SPI_DRIVER_VERSION` (`MAKE_VERSION`(2, 3, 4))  
*FlexIO SPI driver version.*

## FlexIO SPI Configuration

- void `FLEXIO_SPI_MasterInit` (`FLEXIO_SPI_Type` \*base, `flexio_spi_master_config_t` \*masterConfig, uint32\_t srcClock\_Hz)  
*Ungates the FlexIO clock, resets the FlexIO module, configures the FlexIO SPI master hardware, and configures the FlexIO SPI with FlexIO SPI master configuration.*
- void `FLEXIO_SPI_MasterDeinit` (`FLEXIO_SPI_Type` \*base)  
*Resets the FlexIO SPI timer and shifter config.*
- void `FLEXIO_SPI_MasterGetDefaultConfig` (`flexio_spi_master_config_t` \*masterConfig)  
*Gets the default configuration to configure the FlexIO SPI master.*
- void `FLEXIO_SPI_SlaveInit` (`FLEXIO_SPI_Type` \*base, `flexio_spi_slave_config_t` \*slaveConfig)  
*Ungates the FlexIO clock, resets the FlexIO module, configures the FlexIO SPI slave hardware configuration, and configures the FlexIO SPI with FlexIO SPI slave configuration.*
- void `FLEXIO_SPI_SlaveDeinit` (`FLEXIO_SPI_Type` \*base)  
*Gates the FlexIO clock.*
- void `FLEXIO_SPI_SlaveGetDefaultConfig` (`flexio_spi_slave_config_t` \*slaveConfig)  
*Gets the default configuration to configure the FlexIO SPI slave.*

## Status

- uint32\_t `FLEXIO_SPI_GetStatusFlags` (`FLEXIO_SPI_Type` \*base)  
*Gets FlexIO SPI status flags.*
- void `FLEXIO_SPI_ClearStatusFlags` (`FLEXIO_SPI_Type` \*base, uint32\_t mask)  
*Clears FlexIO SPI status flags.*

## Interrupts

- void `FLEXIO_SPI_EnableInterrupts` (`FLEXIO_SPI_Type` \*base, uint32\_t mask)

- *Enables the FlexIO SPI interrupt.*
- void **FLEXIO\_SPI\_DisableInterrupts** (**FLEXIO\_SPI\_Type** \*base, uint32\_t mask)  
*Disables the FlexIO SPI interrupt.*

## DMA Control

- void **FLEXIO\_SPI\_EnableDMA** (**FLEXIO\_SPI\_Type** \*base, uint32\_t mask, bool enable)  
*Enables/disables the FlexIO SPI transmit DMA.*
- static uint32\_t **FLEXIO\_SPI\_GetTxDataRegisterAddress** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_shift\_direction\_t direction)  
*Gets the FlexIO SPI transmit data register address for MSB first transfer.*
- static uint32\_t **FLEXIO\_SPI\_GetRxDataRegisterAddress** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_shift\_direction\_t direction)  
*Gets the FlexIO SPI receive data register address for the MSB first transfer.*

## Bus Operations

- static void **FLEXIO\_SPI\_Enable** (**FLEXIO\_SPI\_Type** \*base, bool enable)  
*Enables/disables the FlexIO SPI module operation.*
- void **FLEXIO\_SPI\_MasterSetBaudRate** (**FLEXIO\_SPI\_Type** \*base, uint32\_t baudRate\_Bps, uint32\_t srcClockHz)  
*Sets baud rate for the FlexIO SPI transfer, which is only used for the master.*
- static void **FLEXIO\_SPI\_WriteData** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_shift\_direction\_t direction, uint32\_t data)  
*Writes one byte of data, which is sent using the MSB method.*
- static uint32\_t **FLEXIO\_SPI\_ReadData** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_shift\_direction\_t direction)  
*Reads 8 bit/16 bit data.*
- status\_t **FLEXIO\_SPI\_WriteBlocking** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_shift\_direction\_t direction, const uint8\_t \*buffer, size\_t size)  
*Sends a buffer of data bytes.*
- status\_t **FLEXIO\_SPI\_ReadBlocking** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_shift\_direction\_t direction, uint8\_t \*buffer, size\_t size)  
*Receives a buffer of bytes.*
- status\_t **FLEXIO\_SPI\_MasterTransferBlocking** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_transfer\_t \*xfer)  
*Receives a buffer of bytes.*
- void **FLEXIO\_SPI\_FlushShifters** (**FLEXIO\_SPI\_Type** \*base)  
*Flush tx/rx shifters.*

## Transactional

- status\_t **FLEXIO\_SPI\_MasterTransferCreateHandle** (**FLEXIO\_SPI\_Type** \*base, flexio\_spi\_master\_handle\_t \*handle, flexio\_spi\_master\_transfer\_callback\_t callback, void \*userData)  
*Initializes the FlexIO SPI Master handle, which is used in transactional functions.*

- `status_t FLEXIO_SPI_MasterTransferNonBlocking` (`FLEXIO_SPI_Type *base`, `flexio_spi_master_handle_t *handle`, `flexio_spi_transfer_t *xfer`)  
*Master transfer data using IRQ.*
- `void FLEXIO_SPI_MasterTransferAbort` (`FLEXIO_SPI_Type *base`, `flexio_spi_master_handle_t *handle`)  
*Aborts the master data transfer, which used IRQ.*
- `status_t FLEXIO_SPI_MasterTransferGetCount` (`FLEXIO_SPI_Type *base`, `flexio_spi_master_handle_t *handle`, `size_t *count`)  
*Gets the data transfer status which used IRQ.*
- `void FLEXIO_SPI_MasterTransferHandleIRQ` (`void *spiType`, `void *spiHandle`)  
*FlexIO SPI master IRQ handler function.*
- `status_t FLEXIO_SPI_SlaveTransferCreateHandle` (`FLEXIO_SPI_Type *base`, `flexio_spi_slave_handle_t *handle`, `flexio_spi_slave_transfer_callback_t callback`, `void *userData`)  
*Initializes the FlexIO SPI Slave handle, which is used in transactional functions.*
- `status_t FLEXIO_SPI_SlaveTransferNonBlocking` (`FLEXIO_SPI_Type *base`, `flexio_spi_slave_handle_t *handle`, `flexio_spi_transfer_t *xfer`)  
*Slave transfer data using IRQ.*
- `static void FLEXIO_SPI_SlaveTransferAbort` (`FLEXIO_SPI_Type *base`, `flexio_spi_slave_handle_t *handle`)  
*Aborts the slave data transfer which used IRQ, share same API with master.*
- `static status_t FLEXIO_SPI_SlaveTransferGetCount` (`FLEXIO_SPI_Type *base`, `flexio_spi_slave_handle_t *handle`, `size_t *count`)  
*Gets the data transfer status which used IRQ, share same API with master.*
- `void FLEXIO_SPI_SlaveTransferHandleIRQ` (`void *spiType`, `void *spiHandle`)  
*FlexIO SPI slave IRQ handler function.*

### 18.5.3 Data Structure Documentation

#### 18.5.3.1 struct \_flexio\_spi\_type

##### Data Fields

- `FLEXIO_Type * flexioBase`  
*FlexIO base pointer.*
- `uint8_t SDOPinIndex`  
*Pin select for data output.*
- `uint8_t SDIPinIndex`  
*Pin select for data input.*
- `uint8_t SCKPinIndex`  
*Pin select for clock.*
- `uint8_t CSnPinIndex`  
*Pin select for enable.*
- `uint8_t shifterIndex` [2]  
*Shifter index used in FlexIO SPI.*
- `uint8_t timerIndex` [2]  
*Timer index used in FlexIO SPI.*

## Field Documentation

(1) `FLEXIO_Type* _flexio_spi_type::flexioBase`

(2) `uint8_t _flexio_spi_type::SDOPinIndex`

To set SDO pin in Hi-Z state, user needs to mux the pin as GPIO input and disable all pull up/down in application.

(3) `uint8_t _flexio_spi_type::SDIPinIndex`

(4) `uint8_t _flexio_spi_type::SCKPinIndex`

(5) `uint8_t _flexio_spi_type::CSnPinIndex`

(6) `uint8_t _flexio_spi_type::shifterIndex[2]`

(7) `uint8_t _flexio_spi_type::timerIndex[2]`

### 18.5.3.2 struct \_flexio\_spi\_master\_config

#### Data Fields

- bool `enableMaster`  
*Enable/disable FlexIO SPI master after configuration.*
- bool `enableInDoze`  
*Enable/disable FlexIO operation in doze mode.*
- bool `enableInDebug`  
*Enable/disable FlexIO operation in debug mode.*
- bool `enableFastAccess`  
*Enable/disable fast access to FlexIO registers, fast access requires the FlexIO clock to be at least twice the frequency of the bus clock.*
- uint32\_t `baudRate_Bps`  
*Baud rate in Bps.*
- `flexio_spi_clock_phase_t` `phase`  
*Clock phase.*
- `flexio_spi_data_bitcount_mode_t` `dataMode`  
*8bit or 16bit mode.*

## Field Documentation

- (1) `bool _flexio_spi_master_config::enableMaster`
- (2) `bool _flexio_spi_master_config::enableInDoze`
- (3) `bool _flexio_spi_master_config::enableInDebug`
- (4) `bool _flexio_spi_master_config::enableFastAccess`
- (5) `uint32_t _flexio_spi_master_config::baudRate_Bps`
- (6) `flexio_spi_clock_phase_t _flexio_spi_master_config::phase`
- (7) `flexio_spi_data_bitcount_mode_t _flexio_spi_master_config::dataMode`

### 18.5.3.3 struct \_flexio\_spi\_slave\_config

#### Data Fields

- `bool enableSlave`  
*Enable/disable FlexIO SPI slave after configuration.*
- `bool enableInDoze`  
*Enable/disable FlexIO operation in doze mode.*
- `bool enableInDebug`  
*Enable/disable FlexIO operation in debug mode.*
- `bool enableFastAccess`  
*Enable/disable fast access to FlexIO registers,  
fast access requires the FlexIO clock to be at least twice the frequency of the bus clock.*
- `flexio_spi_clock_phase_t phase`  
*Clock phase.*
- `flexio_spi_data_bitcount_mode_t dataMode`  
*8bit or 16bit mode.*

### Field Documentation

- (1) `bool _flexio_spi_slave_config::enableSlave`
- (2) `bool _flexio_spi_slave_config::enableInDoze`
- (3) `bool _flexio_spi_slave_config::enableInDebug`
- (4) `bool _flexio_spi_slave_config::enableFastAccess`
- (5) `flexio_spi_clock_phase_t _flexio_spi_slave_config::phase`
- (6) `flexio_spi_data_bitcount_mode_t _flexio_spi_slave_config::dataMode`

### 18.5.3.4 struct \_flexio\_spi\_transfer

#### Data Fields

- `const uint8_t * txData`  
*Send buffer.*
- `uint8_t * rxData`  
*Receive buffer.*
- `size_t dataSize`  
*Transfer bytes.*
- `uint8_t flags`  
*FlexIO SPI control flag, MSB first or LSB first.*

### Field Documentation

- (1) `const uint8_t* _flexio_spi_transfer::txData`
- (2) `uint8_t* _flexio_spi_transfer::rxData`
- (3) `size_t _flexio_spi_transfer::dataSize`
- (4) `uint8_t _flexio_spi_transfer::flags`

### 18.5.3.5 struct \_flexio\_spi\_master\_handle

#### Data Fields

- `const uint8_t * txData`  
*Transfer buffer.*
- `uint8_t * rxData`  
*Receive buffer.*
- `size_t transferSize`  
*Total bytes to be transferred.*
- `volatile size_t txRemainingBytes`  
*Send data remaining in bytes.*
- `volatile size_t rxRemainingBytes`  
*Receive data remaining in bytes.*

- volatile uint32\_t [state](#)  
*FlexIO SPI internal state.*
- uint8\_t [bytePerFrame](#)  
*SPI mode, 2bytes or 1byte in a frame.*
- [flexio\\_spi\\_shift\\_direction\\_t](#) [direction](#)  
*Shift direction.*
- [flexio\\_spi\\_master\\_transfer\\_callback\\_t](#) [callback](#)  
*FlexIO SPI callback.*
- void \* [userData](#)  
*Callback parameter.*





## Field Documentation

- (1) `const uint8_t* _flexio_spi_master_handle::txData`
- (2) `uint8_t* _flexio_spi_master_handle::rxData`
- (3) `size_t _flexio_spi_master_handle::transferSize`
- (4) `volatile size_t _flexio_spi_master_handle::txRemainingBytes`
- (5) `volatile size_t _flexio_spi_master_handle::rxRemainingBytes`
- (6) `volatile uint32_t _flexio_spi_master_handle::state`
- (7) `flexio_spi_shift_direction_t _flexio_spi_master_handle::direction`
- (8) `flexio_spi_master_transfer_callback_t _flexio_spi_master_handle::callback`
- (9) `void* _flexio_spi_master_handle::userData`

## 18.5.4 Macro Definition Documentation

18.5.4.1 `#define FSL_FLEXIO_SPI_DRIVER_VERSION (MAKE_VERSION(2, 3, 4))`

18.5.4.2 `#define FLEXIO_SPI_DUMMYDATA (0x00U)`

18.5.4.3 `#define SPI_RETRY_TIMES 0U /* Define to zero means keep waiting until the flag is assert/deassert. */`

18.5.4.4 `#define FLEXIO_SPI_XFER_DATA_FORMAT( flag ) ((flag) & (0x7U))`

## 18.5.5 Typedef Documentation

18.5.5.1 `typedef enum _flexio_spi_clock_phase flexio_spi_clock_phase_t`

18.5.5.2 `typedef enum _flexio_spi_shift_direction flexio_spi_shift_direction_t`

18.5.5.3 `typedef enum _flexio_spi_data_bitcount_mode flexio_spi_data_bitcount_mode_t`

18.5.5.4 `typedef struct _flexio_spi_type FLEXIO_SPI_Type`

18.5.5.5 `typedef struct _flexio_spi_master_config flexio_spi_master_config_t`

18.5.5.6 `typedef struct _flexio_spi_slave_config flexio_spi_slave_config_t`

18.5.5.7 `typedef struct _flexio_spi_transfer flexio_spi_transfer_t`

18.5.5.8 `typedef struct _flexio_spi_master_handle flexio_spi_master_handle_t`

18.5.5.9 `typedef flexio_spi_master_handle_t flexio_spi_slave_handle_t`

*kStatus\_FLEXIO\_SPI\_Idle* SPI is idle.

*kStatus\_FLEXIO\_SPI\_Error* FlexIO SPI error.

*kStatus\_FLEXIO\_SPI\_Timeout* FlexIO SPI timeout polling status flags.

### 18.5.6.2 enum \_flexio\_spi\_clock\_phase

Enumerator

*kFLEXIO\_SPI\_ClockPhaseFirstEdge* First edge on SPSCCK occurs at the middle of the first cycle of a data transfer.

*kFLEXIO\_SPI\_ClockPhaseSecondEdge* First edge on SPSCCK occurs at the start of the first cycle of a data transfer.

### 18.5.6.3 enum \_flexio\_spi\_shift\_direction

Enumerator

*kFLEXIO\_SPI\_MsbFirst* Data transfers start with most significant bit.

*kFLEXIO\_SPI\_LsbFirst* Data transfers start with least significant bit.

### 18.5.6.4 enum \_flexio\_spi\_data\_bitcount\_mode

Enumerator

*kFLEXIO\_SPI\_8BitMode* 8-bit data transmission mode.

*kFLEXIO\_SPI\_16BitMode* 16-bit data transmission mode.

*kFLEXIO\_SPI\_32BitMode* 32-bit data transmission mode.

### 18.5.6.5 enum \_flexio\_spi\_interrupt\_enable

Enumerator

*kFLEXIO\_SPI\_TxEmptyInterruptEnable* Transmit buffer empty interrupt enable.

*kFLEXIO\_SPI\_RxFullInterruptEnable* Receive buffer full interrupt enable.

### 18.5.6.6 enum \_flexio\_spi\_status\_flags

Enumerator

*kFLEXIO\_SPI\_TxBufferEmptyFlag* Transmit buffer empty flag.

*kFLEXIO\_SPI\_RxBufferFullFlag* Receive buffer full flag.

### 18.5.6.7 enum \_flexio\_spi\_dma\_enable

Enumerator

***kFLEXIO\_SPI\_TxDmaEnable*** Tx DMA request source.  
***kFLEXIO\_SPI\_RxDmaEnable*** Rx DMA request source.  
***kFLEXIO\_SPI\_DmaAllEnable*** All DMA request source.

### 18.5.6.8 enum \_flexio\_spi\_transfer\_flags

Note

Use **kFLEXIO\_SPI\_csContinuous** and one of the other flags to OR together to form the transfer flag.

Enumerator

***kFLEXIO\_SPI\_8bitMsb*** FlexIO SPI 8-bit MSB first.  
***kFLEXIO\_SPI\_8bitLsb*** FlexIO SPI 8-bit LSB first.  
***kFLEXIO\_SPI\_16bitMsb*** FlexIO SPI 16-bit MSB first.  
***kFLEXIO\_SPI\_16bitLsb*** FlexIO SPI 16-bit LSB first.  
***kFLEXIO\_SPI\_32bitMsb*** FlexIO SPI 32-bit MSB first.  
***kFLEXIO\_SPI\_32bitLsb*** FlexIO SPI 32-bit LSB first.  
***kFLEXIO\_SPI\_csContinuous*** Enable the CS signal continuous mode.

## 18.5.7 Function Documentation

### 18.5.7.1 void FLEXIO\_SPI\_MasterInit ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_master\_config\_t \* *masterConfig*, uint32\_t *srcClock\_Hz* )

The configuration structure can be filled by the user, or be set with default values by the [FLEXIO\\_SPI\\_MasterGetDefaultConfig\(\)](#).

Note

1.FlexIO SPI master only support CPOL = 0, which means clock inactive low. 2.For FlexIO SPI master, the input valid time is 1.5 clock cycles, for slave the output valid time is 2.5 clock cycles. So if FlexIO SPI master communicates with other spi IPs, the maximum baud rate is FlexIO clock frequency divided by  $2 \times 2 = 4$ . If FlexIO SPI master communicates with FlexIO SPI slave, the maximum baud rate is FlexIO clock frequency divided by  $(1.5 + 2.5) \times 2 = 8$ .

Example

```
FLEXIO_SPI_Type spiDev = {
 .flexioBase = FLEXIO,
 .SDOPinIndex = 0,
 .SDIPinIndex = 1,
```

```

.SCKPinIndex = 2,
.CSnPinIndex = 3,
.shifterIndex = {0,1},
.timerIndex = {0,1}
};
flexio_spi_master_config_t config = {
.enableMaster = true,
.enableInDoze = false,
.enableInDebug = true,
.enableFastAccess = false,
.baudRate_Bps = 500000,
.phase = kFLEXIO_SPI_ClockPhaseFirstEdge,
.direction = kFLEXIO_SPI_MsbFirst,
.dataMode = kFLEXIO_SPI_8BitMode
};
FLEXIO_SPI_MasterInit(&spiDev, &config, srcClock_Hz);

```

## Parameters

|                     |                                                      |
|---------------------|------------------------------------------------------|
| <i>base</i>         | Pointer to the FLEXIO_SPI_Type structure.            |
| <i>masterConfig</i> | Pointer to the flexio_spi_master_config_t structure. |
| <i>srcClock_Hz</i>  | FlexIO source clock in Hz.                           |

### 18.5.7.2 void FLEXIO\_SPI\_MasterDeinit ( FLEXIO\_SPI\_Type \* *base* )

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | Pointer to the FLEXIO_SPI_Type. |
|-------------|---------------------------------|

### 18.5.7.3 void FLEXIO\_SPI\_MasterGetDefaultConfig ( flexio\_spi\_master\_config\_t \* *masterConfig* )

The configuration can be used directly by calling the FLEXIO\_SPI\_MasterConfigure(). Example:

```

flexio_spi_master_config_t masterConfig;
FLEXIO_SPI_MasterGetDefaultConfig(&masterConfig);

```

## Parameters

|                     |                                                      |
|---------------------|------------------------------------------------------|
| <i>masterConfig</i> | Pointer to the flexio_spi_master_config_t structure. |
|---------------------|------------------------------------------------------|

### 18.5.7.4 void FLEXIO\_SPI\_SlaveInit ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_slave\_config\_t \* *slaveConfig* )

The configuration structure can be filled by the user, or be set with default values by the [FLEXIO\\_SPI\\_SlaveGetDefaultConfig\(\)](#).

## Note

1. Only one timer is needed in the FlexIO SPI slave. As a result, the second timer index is ignored. 2. FlexIO SPI slave only support CPOL = 0, which means clock inactive low. 3. For FlexIO SPI master, the input valid time is 1.5 clock cycles, for slave the output valid time is 2.5 clock cycles. So if FlexIO SPI slave communicates with other spi IPs, the maximum baud rate is FlexIO clock frequency divided by  $3 \times 2 = 6$ . If FlexIO SPI slave communicates with FlexIO SPI master, the maximum baud rate is FlexIO clock frequency divided by  $(1.5 + 2.5) \times 2 = 8$ . Example

```
FLEXIO_SPI_Type spiDev = {
 .flexioBase = FLEXIO,
 .SDOPinIndex = 0,
 .SDIPinIndex = 1,
 .SCKPinIndex = 2,
 .CSnPinIndex = 3,
 .shifterIndex = {0,1},
 .timerIndex = {0}
};
flexio_spi_slave_config_t config = {
 .enableSlave = true,
 .enableInDoze = false,
 .enableInDebug = true,
 .enableFastAccess = false,
 .phase = kFLEXIO_SPI_ClockPhaseFirstEdge,
 .direction = kFLEXIO_SPI_MsbFirst,
 .dataMode = kFLEXIO_SPI_8BitMode
};
FLEXIO_SPI_SlaveInit(&spiDev, &config);
```

## Parameters

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| <i>base</i>        | Pointer to the FLEXIO_SPI_Type structure.           |
| <i>slaveConfig</i> | Pointer to the flexio_spi_slave_config_t structure. |

### 18.5.7.5 void FLEXIO\_SPI\_SlaveDeinit ( FLEXIO\_SPI\_Type \* *base* )

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | Pointer to the FLEXIO_SPI_Type. |
|-------------|---------------------------------|

### 18.5.7.6 void FLEXIO\_SPI\_SlaveGetDefaultConfig ( flexio\_spi\_slave\_config\_t \* *slaveConfig* )

The configuration can be used directly for calling the FLEXIO\_SPI\_SlaveConfigure(). Example:

```
flexio_spi_slave_config_t slaveConfig;
FLEXIO_SPI_SlaveGetDefaultConfig(&slaveConfig);
```

## Parameters

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| <i>slaveConfig</i> | Pointer to the flexio_spi_slave_config_t structure. |
|--------------------|-----------------------------------------------------|

**18.5.7.7 uint32\_t FLEXIO\_SPI\_GetStatusFlags ( FLEXIO\_SPI\_Type \* *base* )**

## Parameters

|             |                                           |
|-------------|-------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_SPI_Type structure. |
|-------------|-------------------------------------------|

## Returns

status flag; Use the status flag to AND the following flag mask and get the status.

- kFLEXIO\_SPI\_TxEmptyFlag
- kFLEXIO\_SPI\_RxEmptyFlag

**18.5.7.8 void FLEXIO\_SPI\_ClearStatusFlags ( FLEXIO\_SPI\_Type \* *base*, uint32\_t *mask* )**

## Parameters

|             |                                                                                                                                                                                          |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_SPI_Type structure.                                                                                                                                                |
| <i>mask</i> | status flag The parameter can be any combination of the following values: <ul style="list-style-type: none"> <li>• kFLEXIO_SPI_TxEmptyFlag</li> <li>• kFLEXIO_SPI_RxEmptyFlag</li> </ul> |

**18.5.7.9 void FLEXIO\_SPI\_EnableInterrupts ( FLEXIO\_SPI\_Type \* *base*, uint32\_t *mask* )**

This function enables the FlexIO SPI interrupt.

## Parameters

|             |                                           |
|-------------|-------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_SPI_Type structure. |
|-------------|-------------------------------------------|

|             |                                                                                                                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>mask</i> | interrupt source. The parameter can be any combination of the following values: <ul style="list-style-type: none"> <li>• kFLEXIO_SPI_RxFullInterruptEnable</li> <li>• kFLEXIO_SPI_TxEmptyInterruptEnable</li> </ul> |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### 18.5.7.10 void FLEXIO\_SPI\_DisableInterrupts ( FLEXIO\_SPI\_Type \* *base*, uint32\_t *mask* )

This function disables the FlexIO SPI interrupt.

Parameters

|             |                                                                                                                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_SPI_Type structure.                                                                                                                                                                          |
| <i>mask</i> | interrupt source The parameter can be any combination of the following values: <ul style="list-style-type: none"> <li>• kFLEXIO_SPI_RxFullInterruptEnable</li> <li>• kFLEXIO_SPI_TxEmptyInterruptEnable</li> </ul> |

#### 18.5.7.11 void FLEXIO\_SPI\_EnableDMA ( FLEXIO\_SPI\_Type \* *base*, uint32\_t *mask*, bool *enable* )

This function enables/disables the FlexIO SPI Tx DMA, which means that asserting the kFLEXIO\_SPI\_TxEmptyFlag does/doesn't trigger the DMA request.

Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type structure.       |
| <i>mask</i>   | SPI DMA source.                                 |
| <i>enable</i> | True means enable DMA, false means disable DMA. |

#### 18.5.7.12 static uint32\_t FLEXIO\_SPI\_GetTxDataRegisterAddress ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_shift\_direction\_t *direction* ) [inline], [static]

This function returns the SPI data register address, which is mainly used by DMA/eDMA.

Parameters

|                  |                                            |
|------------------|--------------------------------------------|
| <i>base</i>      | Pointer to the FLEXIO_SPI_Type structure.  |
| <i>direction</i> | Shift direction of MSB first or LSB first. |



## Returns

FlexIO SPI transmit data register address.

**18.5.7.13** `static uint32_t FLEXIO_SPI_GetRxDataRegisterAddress ( FLEXIO_SPI_Type * base, flexio_spi_shift_direction_t direction ) [inline], [static]`

This function returns the SPI data register address, which is mainly used by DMA/eDMA.

## Parameters

|                  |                                            |
|------------------|--------------------------------------------|
| <i>base</i>      | Pointer to the FLEXIO_SPI_Type structure.  |
| <i>direction</i> | Shift direction of MSB first or LSB first. |

## Returns

FlexIO SPI receive data register address.

**18.5.7.14** `static void FLEXIO_SPI_Enable ( FLEXIO_SPI_Type * base, bool enable ) [inline], [static]`

## Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type.                 |
| <i>enable</i> | True to enable, false does not have any effect. |

**18.5.7.15** `void FLEXIO_SPI_MasterSetBaudRate ( FLEXIO_SPI_Type * base, uint32_t baudRate_Bps, uint32_t srcClockHz )`

## Parameters

|                     |                                           |
|---------------------|-------------------------------------------|
| <i>base</i>         | Pointer to the FLEXIO_SPI_Type structure. |
| <i>baudRate_Bps</i> | Baud Rate needed in Hz.                   |
| <i>srcClockHz</i>   | SPI source clock frequency in Hz.         |

**18.5.7.16** `static void FLEXIO_SPI_WriteData ( FLEXIO_SPI_Type * base, flexio_spi_shift_direction_t direction, uint32_t data ) [inline], [static]`

## Note

This is a non-blocking API, which returns directly after the data is put into the data register but the data transfer is not finished on the bus. Ensure that the TxEmptyFlag is asserted before calling this API.

## Parameters

|                  |                                            |
|------------------|--------------------------------------------|
| <i>base</i>      | Pointer to the FLEXIO_SPI_Type structure.  |
| <i>direction</i> | Shift direction of MSB first or LSB first. |
| <i>data</i>      | 8/16/32 bit data.                          |

**18.5.7.17 static uint32\_t FLEXIO\_SPI\_ReadData ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_shift\_direction\_t *direction* ) [inline], [static]**

## Note

This is a non-blocking API, which returns directly after the data is read from the data register. Ensure that the RxFullFlag is asserted before calling this API.

## Parameters

|                  |                                            |
|------------------|--------------------------------------------|
| <i>base</i>      | Pointer to the FLEXIO_SPI_Type structure.  |
| <i>direction</i> | Shift direction of MSB first or LSB first. |

## Returns

8 bit/16 bit data received.

**18.5.7.18 status\_t FLEXIO\_SPI\_WriteBlocking ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_shift\_direction\_t *direction*, const uint8\_t \* *buffer*, size\_t *size* )**

## Note

This function blocks using the polling method until all bytes have been sent.

## Parameters

|                  |                                            |
|------------------|--------------------------------------------|
| <i>base</i>      | Pointer to the FLEXIO_SPI_Type structure.  |
| <i>direction</i> | Shift direction of MSB first or LSB first. |
| <i>buffer</i>    | The data bytes to send.                    |
| <i>size</i>      | The number of data bytes to send.          |

Return values

|                                   |                                         |
|-----------------------------------|-----------------------------------------|
| <i>kStatus_Success</i>            | Successfully create the handle.         |
| <i>kStatus_FLEXIO_SPI_Timeout</i> | The transfer timed out and was aborted. |

#### 18.5.7.19 **status\_t FLEXIO\_SPI\_ReadBlocking ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_shift\_direction\_t *direction*, uint8\_t \* *buffer*, size\_t *size* )**

Note

This function blocks using the polling method until all bytes have been received.

Parameters

|                  |                                            |
|------------------|--------------------------------------------|
| <i>base</i>      | Pointer to the FLEXIO_SPI_Type structure.  |
| <i>direction</i> | Shift direction of MSB first or LSB first. |
| <i>buffer</i>    | The buffer to store the received bytes.    |
| <i>size</i>      | The number of data bytes to be received.   |

Return values

|                                   |                                         |
|-----------------------------------|-----------------------------------------|
| <i>kStatus_Success</i>            | Successfully create the handle.         |
| <i>kStatus_FLEXIO_SPI_Timeout</i> | The transfer timed out and was aborted. |

#### 18.5.7.20 **status\_t FLEXIO\_SPI\_MasterTransferBlocking ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_transfer\_t \* *xfer* )**

Note

This function blocks via polling until all bytes have been received.

## Parameters

|             |                                                                            |
|-------------|----------------------------------------------------------------------------|
| <i>base</i> | pointer to FLEXIO_SPI_Type structure                                       |
| <i>xfer</i> | FlexIO SPI transfer structure, see <a href="#">flexio_spi_transfer_t</a> . |

## Return values

|                                   |                                         |
|-----------------------------------|-----------------------------------------|
| <i>kStatus_Success</i>            | Successfully create the handle.         |
| <i>kStatus_FLEXIO_SPI_Timeout</i> | The transfer timed out and was aborted. |

**18.5.7.21 void FLEXIO\_SPI\_FlushShifters ( FLEXIO\_SPI\_Type \* *base* )**

## Parameters

|             |                                           |
|-------------|-------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_SPI_Type structure. |
|-------------|-------------------------------------------|

**18.5.7.22 status\_t FLEXIO\_SPI\_MasterTransferCreateHandle ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_master\_handle\_t \* *handle*, flexio\_spi\_master\_transfer\_callback\_t *callback*, void \* *userData* )**

## Parameters

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>base</i>     | Pointer to the FLEXIO_SPI_Type structure.                                        |
| <i>handle</i>   | Pointer to the flexio_spi_master_handle_t structure to store the transfer state. |
| <i>callback</i> | The callback function.                                                           |
| <i>userData</i> | The parameter of the callback function.                                          |

## Return values

|                           |                                                |
|---------------------------|------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                |
| <i>kStatus_OutOfRange</i> | The FlexIO type/handle/ISR table out of range. |

**18.5.7.23 status\_t FLEXIO\_SPI\_MasterTransferNonBlocking ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_master\_handle\_t \* *handle*, flexio\_spi\_transfer\_t \* *xfer* )**

This function sends data using IRQ. This is a non-blocking function, which returns right away. When all data is sent out/received, the callback function is called.

## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type structure.                                        |
| <i>handle</i> | Pointer to the flexio_spi_master_handle_t structure to store the transfer state. |
| <i>xfer</i>   | FlexIO SPI transfer structure. See <a href="#">flexio_spi_transfer_t</a> .       |

## Return values

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| <i>kStatus_Success</i>         | Successfully start a transfer.                |
| <i>kStatus_InvalidArgument</i> | Input argument is invalid.                    |
| <i>kStatus_FLEXIO_SPI_Busy</i> | SPI is not idle, is running another transfer. |

#### 18.5.7.24 void FLEXIO\_SPI\_MasterTransferAbort ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_master\_handle\_t \* *handle* )

## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type structure.                                        |
| <i>handle</i> | Pointer to the flexio_spi_master_handle_t structure to store the transfer state. |

#### 18.5.7.25 status\_t FLEXIO\_SPI\_MasterTransferGetCount ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_master\_handle\_t \* *handle*, size\_t \* *count* )

## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type structure.                                        |
| <i>handle</i> | Pointer to the flexio_spi_master_handle_t structure to store the transfer state. |
| <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction.              |

## Return values

|                                |                                |
|--------------------------------|--------------------------------|
| <i>kStatus_InvalidArgument</i> | count is Invalid.              |
| <i>kStatus_Success</i>         | Successfully return the count. |

#### 18.5.7.26 void FLEXIO\_SPI\_MasterTransferHandleIRQ ( void \* *spiType*, void \* *spiHandle* )

## Parameters

|                  |                                                                                  |
|------------------|----------------------------------------------------------------------------------|
| <i>spiType</i>   | Pointer to the FLEXIO_SPI_Type structure.                                        |
| <i>spiHandle</i> | Pointer to the flexio_spi_master_handle_t structure to store the transfer state. |

**18.5.7.27** `status_t FLEXIO_SPI_SlaveTransferCreateHandle ( FLEXIO_SPI_Type * base, flexio_spi_slave_handle_t * handle, flexio_spi_slave_transfer_callback_t callback, void * userData )`

## Parameters

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>base</i>     | Pointer to the FLEXIO_SPI_Type structure.                                       |
| <i>handle</i>   | Pointer to the flexio_spi_slave_handle_t structure to store the transfer state. |
| <i>callback</i> | The callback function.                                                          |
| <i>userData</i> | The parameter of the callback function.                                         |

## Return values

|                           |                                                |
|---------------------------|------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                |
| <i>kStatus_OutOfRange</i> | The FlexIO type/handle/ISR table out of range. |

**18.5.7.28** `status_t FLEXIO_SPI_SlaveTransferNonBlocking ( FLEXIO_SPI_Type * base, flexio_spi_slave_handle_t * handle, flexio_spi_transfer_t * xfer )`

This function sends data using IRQ. This is a non-blocking function, which returns right away. When all data is sent out/received, the callback function is called.

## Parameters

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <i>handle</i> | Pointer to the flexio_spi_slave_handle_t structure to store the transfer state. |
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type structure.                                       |
| <i>xfer</i>   | FlexIO SPI transfer structure. See <a href="#">flexio_spi_transfer_t</a> .      |

## Return values

|                                |                                                  |
|--------------------------------|--------------------------------------------------|
| <i>kStatus_Success</i>         | Successfully start a transfer.                   |
| <i>kStatus_InvalidArgument</i> | Input argument is invalid.                       |
| <i>kStatus_FLEXIO_SPI_Busy</i> | SPI is not idle; it is running another transfer. |

**18.5.7.29** `static void FLEXIO_SPI_SlaveTransferAbort ( FLEXIO_SPI_Type * base,  
flexio_spi_slave_handle_t * handle ) [inline], [static]`

## Parameters

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type structure.                                       |
| <i>handle</i> | Pointer to the flexio_spi_slave_handle_t structure to store the transfer state. |

**18.5.7.30 static status\_t FLEXIO\_SPI\_SlaveTransferGetCount ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_slave\_handle\_t \* *handle*, size\_t \* *count* ) [inline], [static]**

## Parameters

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_SPI_Type structure.                                       |
| <i>handle</i> | Pointer to the flexio_spi_slave_handle_t structure to store the transfer state. |
| <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction.             |

## Return values

|                                |                                |
|--------------------------------|--------------------------------|
| <i>kStatus_InvalidArgument</i> | count is Invalid.              |
| <i>kStatus_Success</i>         | Successfully return the count. |

**18.5.7.31 void FLEXIO\_SPI\_SlaveTransferHandleIRQ ( void \* *spiType*, void \* *spiHandle* )**

## Parameters

|                  |                                                                                 |
|------------------|---------------------------------------------------------------------------------|
| <i>spiType</i>   | Pointer to the FLEXIO_SPI_Type structure.                                       |
| <i>spiHandle</i> | Pointer to the flexio_spi_slave_handle_t structure to store the transfer state. |



## 18.6 FlexIO UART Driver

### 18.6.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Universal Asynchronous Receiver/Transmitter (UART) function using the Flexible I/O.

FlexIO UART driver includes functional APIs and transactional APIs. Functional APIs target low-level APIs. Functional APIs can be used for the FlexIO UART initialization/configuration/operation for optimization/customization purpose. Using the functional APIs requires the knowledge of the FlexIO UART peripheral and how to organize functional APIs to meet the application requirements. All functional API use the `FLEXIO_UART_Type *` as the first parameter. FlexIO UART functional operation groups provide the functional APIs set.

Transactional APIs target high-level APIs. Transactional APIs can be used to enable the peripheral and also in the application if the code size and performance of transactional APIs satisfy requirements. If the code size and performance are critical requirements, see the transactional API implementation and write custom code. All transactional APIs use the `flexio_uart_handle_t` as the second parameter. Initialize the handle by calling the [FLEXIO\\_UART\\_TransferCreateHandle\(\)](#) API.

Transactional APIs support asynchronous transfer. This means that the functions `FLEXIO_UART_SendNonBlocking()` and `FLEXIO_UART_ReceiveNonBlocking()` set up an interrupt for data transfer. When the transfer is complete, the upper layer is notified through a callback function with the `kStatus_FLEXIO_UART_TxIdle` and `kStatus_FLEXIO_UART_RxIdle` status.

Transactional receive APIs support the ring buffer. Prepare the memory for the ring buffer and pass in the start address and size through calling the `FLEXIO_UART_InstallRingBuffer()`. When the ring buffer is enabled, the received data is saved to the ring buffer in the background. The function `FLEXIO_UART_ReceiveNonBlocking()` first gets data from the ring buffer. If ring buffer does not have enough data, the function returns the data to the ring buffer and saves the received data to user memory. When all data is received, the upper layer is informed through a callback with the `kStatus_FLEXIO_UART_RxIdle` status.

If the receive ring buffer is full, the upper layer is informed through a callback with status `kStatus_FLEXIO_UART_RxRingBufferOverflow`. In the callback function, the upper layer reads data from the ring buffer. If not, the oldest data is overwritten by the new data.

The ring buffer size is specified when calling the `FLEXIO_UART_InstallRingBuffer`. Note that one byte is reserved for the ring buffer maintenance. Create a handle as follows.

```
FLEXIO_UART_InstallRingBuffer(&uartDev, &handle, &ringBuffer, 32);
```

In this example, the buffer size is 32. However, only 31 bytes are used for saving data.

### 18.6.2 Typical use case

#### 18.6.2.1 FlexIO UART send/receive using a polling method

```
uint8_t ch;
```

```

FLEXIO_UART_Type uartDev;
status_t result = kStatus_Success;
flexio_uart_user_config user_config;
FLEXIO_UART_GetDefaultConfig(&user_config);
user_config.baudRate_Bps = 115200U;
user_config.enableUart = true;

uartDev.flexioBase = BOARD_FLEXIO_BASE;
uartDev.TxPinIndex = FLEXIO_UART_TX_PIN;
uartDev.RxPinIndex = FLEXIO_UART_RX_PIN;
uartDev.shifterIndex[0] = 0U;
uartDev.shifterIndex[1] = 1U;
uartDev.timerIndex[0] = 0U;
uartDev.timerIndex[1] = 1U;

result = FLEXIO_UART_Init(&uartDev, &user_config, 48000000U);
//Check if configuration is correct.
if(result != kStatus_Success)
{
 return;
}
FLEXIO_UART_WriteBlocking(&uartDev, txbuff, sizeof(txbuff));

while(1)
{
 FLEXIO_UART_ReadBlocking(&uartDev, &ch, 1);
 FLEXIO_UART_WriteBlocking(&uartDev, &ch, 1);
}

```

### 18.6.2.2 FlexIO UART send/receive using an interrupt method

```

FLEXIO_UART_Type uartDev;
flexio_uart_handle_t g_uartHandle;
flexio_uart_config_t user_config;
flexio_uart_transfer_t sendXfer;
flexio_uart_transfer_t receiveXfer;
volatile bool txFinished;
volatile bool rxFinished;
uint8_t sendData[] = {'H', 'e', 'l', 'l', 'o'};
uint8_t receiveData[32];

void FLEXIO_UART_UserCallback(FLEXIO_UART_Type *base,
 flexio_uart_handle_t *handle, status_t status, void *userData)
{
 userData = userData;

 if (kStatus_FLEXIO_UART_TxIdle == status)
 {
 txFinished = true;
 }

 if (kStatus_FLEXIO_UART_RxIdle == status)
 {
 rxFinished = true;
 }
}

void main(void)
{
 //...

 FLEXIO_UART_GetDefaultConfig(&user_config);
 user_config.baudRate_Bps = 115200U;
 user_config.enableUart = true;

```

```

uartDev.flexioBase = BOARD_FLEXIO_BASE;
uartDev.TxPinIndex = FLEXIO_UART_TX_PIN;
uartDev.RxPinIndex = FLEXIO_UART_RX_PIN;
uartDev.shifterIndex[0] = 0U;
uartDev.shifterIndex[1] = 1U;
uartDev.timerIndex[0] = 0U;
uartDev.timerIndex[1] = 1U;

result = FLEXIO_UART_Init(&uartDev, &user_config, 120000000U);
//Check if configuration is correct.
if(result != kStatus_Success)
{
 return;
}

FLEXIO_UART_TransferCreateHandle(&uartDev, &g_uartHandle,
 FLEXIO_UART_UserCallback, NULL);

// Prepares to send.
sendXfer.data = sendData;
sendXfer.dataSize = sizeof(sendData)/sizeof(sendData[0]);
txFinished = false;

// Sends out.
FLEXIO_UART_SendNonBlocking(&uartDev, &g_uartHandle, &sendXfer);

// Send finished.
while (!txFinished)
{
}

// Prepares to receive.
receiveXfer.data = receiveData;
receiveXfer.dataSize = sizeof(receiveData)/sizeof(receiveData[0]);
rxFinished = false;

// Receives.
FLEXIO_UART_ReceiveNonBlocking(&uartDev, &g_uartHandle, &receiveXfer, NULL);

// Receive finished.
while (!rxFinished)
{
}

// ...
}

```

### 18.6.2.3 FlexIO UART receive using the ringbuffer feature

```

#define RING_BUFFER_SIZE 64
#define RX_DATA_SIZE 32

FLEXIO_UART_Type uartDev;
flexio_uart_handle_t g_uartHandle;
flexio_uart_config_t user_config;
flexio_uart_transfer_t sendXfer;
flexio_uart_transfer_t receiveXfer;
volatile bool txFinished;
volatile bool rxFinished;
uint8_t receiveData[RX_DATA_SIZE];
uint8_t ringBuffer[RING_BUFFER_SIZE];

void FLEXIO_UART_UserCallback(FLEXIO_UART_Type *base,
 flexio_uart_handle_t *handle, status_t status, void *userData)
{

```

```

 userData = userData;

 if (kStatus_FLEXIO_UART_RxIdle == status)
 {
 rxFinished = true;
 }
}

void main(void)
{
 size_t bytesRead;
 //...

 FLEXIO_UART_GetDefaultConfig(&user_config);
 user_config.baudRate_Bps = 115200U;
 user_config.enableUart = true;

 uartDev.flexioBase = BOARD_FLEXIO_BASE;
 uartDev.TxPinIndex = FLEXIO_UART_TX_PIN;
 uartDev.RxPinIndex = FLEXIO_UART_RX_PIN;
 uartDev.shifterIndex[0] = 0U;
 uartDev.shifterIndex[1] = 1U;
 uartDev.timerIndex[0] = 0U;
 uartDev.timerIndex[1] = 1U;

 result = FLEXIO_UART_Init(&uartDev, &user_config, 48000000U);
 //Check if configuration is correct.
 if(result != kStatus_Success)
 {
 return;
 }

 FLEXIO_UART_TransferCreateHandle(&uartDev, &g_uartHandle,
 FLEXIO_UART_UserCallback, NULL);
 FLEXIO_UART_InstallRingBuffer(&uartDev, &g_uartHandle, ringBuffer, RING_BUFFER_SIZE);

 // Receive is working in the background to the ring buffer.

 // Prepares to receive.
 receiveXfer.data = receiveData;
 receiveXfer.dataSize = RX_DATA_SIZE;
 rxFinished = false;

 // Receives.
 FLEXIO_UART_ReceiveNonBlocking(&uartDev, &g_uartHandle, &receiveXfer, &bytesRead);

 if (bytesRead == RX_DATA_SIZE) /* Have read enough data. */
 {
 ;
 }
 else
 {
 if (bytesRead) /* Received some data, process first. */
 {
 ;
 }

 // Receive finished.
 while (!rxFinished)
 {
 }
 }

 // ...
}

```

### 18.6.2.4 FlexIO UART send/receive using a DMA method

```

FLEXIO_UART_Type uartDev;
flexio_uart_handle_t g_uartHandle;
dma_handle_t g_uartTxDmaHandle;
dma_handle_t g_uartRxDmaHandle;
flexio_uart_config_t user_config;
flexio_uart_transfer_t sendXfer;
flexio_uart_transfer_t receiveXfer;
volatile bool txFinished;
volatile bool rxFinished;
uint8_t sendData[] = {'H', 'e', 'l', 'l', 'o'};
uint8_t receiveData[32];

void FLEXIO_UART_UserCallback(FLEXIO_UART_Type *base,
 flexio_uart_handle_t *handle, status_t status, void *userData)
{
 userData = userData;

 if (kStatus_FLEXIO_UART_TxIdle == status)
 {
 txFinished = true;
 }

 if (kStatus_FLEXIO_UART_RxIdle == status)
 {
 rxFinished = true;
 }
}

void main(void)
{
 //...

 FLEXIO_UART_GetDefaultConfig(&user_config);
 user_config.baudRate_Bps = 115200U;
 user_config.enableUart = true;

 uartDev.flexioBase = BOARD_FLEXIO_BASE;
 uartDev.TxPinIndex = FLEXIO_UART_TX_PIN;
 uartDev.RxPinIndex = FLEXIO_UART_RX_PIN;
 uartDev.shifterIndex[0] = 0U;
 uartDev.shifterIndex[1] = 1U;
 uartDev.timerIndex[0] = 0U;
 uartDev.timerIndex[1] = 1U;
 result = FLEXIO_UART_Init(&uartDev, &user_config, 48000000U);
 //Check if configuration is correct.
 if(result != kStatus_Success)
 {
 return;
 }

 /* Init DMAMUX. */
 DMAMUX_Init(EXAMPLE_FLEXIO_UART_DMAMUX_BASEADDR)

 /* Init the DMA/EDMA module */
#ifdef FSL_FEATURE_SOC_DMA_COUNT && FSL_FEATURE_SOC_DMA_COUNT > 0U
 DMA_Init(EXAMPLE_FLEXIO_UART_DMA_BASEADDR);
 DMA_CreateHandle(&g_uartTxDmaHandle, EXAMPLE_FLEXIO_UART_DMA_BASEADDR, FLEXIO_UART_TX_DMA_CHANNEL);
 DMA_CreateHandle(&g_uartRxDmaHandle, EXAMPLE_FLEXIO_UART_DMA_BASEADDR, FLEXIO_UART_RX_DMA_CHANNEL);
#endif /* FSL_FEATURE_SOC_DMA_COUNT */

#ifdef FSL_FEATURE_SOC_EDMA_COUNT && FSL_FEATURE_SOC_EDMA_COUNT > 0U
 edma_config_t edmaConfig;

 EDMA_GetDefaultConfig(&edmaConfig);
 EDMA_Init(EXAMPLE_FLEXIO_UART_DMA_BASEADDR, &edmaConfig);

```

```

 EDMA_CreateHandle(&g_uartTxDmaHandle, EXAMPLE_FLEXIO_UART_DMA_BASEADDR,
FLEXIO_UART_TX_DMA_CHANNEL);
 EDMA_CreateHandle(&g_uartRxDmaHandle, EXAMPLE_FLEXIO_UART_DMA_BASEADDR,
FLEXIO_UART_RX_DMA_CHANNEL);
#endif /* FSL_FEATURE_SOC_EDMA_COUNT */

 dma_request_source_tx = (dma_request_source_t)(FLEXIO_DMA_REQUEST_BASE + uartDev.
shifterIndex[0]);
 dma_request_source_rx = (dma_request_source_t)(FLEXIO_DMA_REQUEST_BASE + uartDev.
shifterIndex[1]);

 /* Requests DMA channels for transmit and receive. */
 DMAMUX_SetSource(EXAMPLE_FLEXIO_UART_DMAMUX_BASEADDR, FLEXIO_UART_TX_DMA_CHANNEL, (
dma_request_source_t)dma_request_source_tx);
 DMAMUX_SetSource(EXAMPLE_FLEXIO_UART_DMAMUX_BASEADDR, FLEXIO_UART_RX_DMA_CHANNEL, (
dma_request_source_t)dma_request_source_rx);
 DMAMUX_EnableChannel(EXAMPLE_FLEXIO_UART_DMAMUX_BASEADDR,
FLEXIO_UART_TX_DMA_CHANNEL);
 DMAMUX_EnableChannel(EXAMPLE_FLEXIO_UART_DMAMUX_BASEADDR,
FLEXIO_UART_RX_DMA_CHANNEL);

 FLEXIO_UART_TransferCreateHandleDMA(&uartDev, &g_uartHandle, FLEXIO_UART_UserCallback, NULL, &
g_uartTxDmaHandle, &g_uartRxDmaHandle);

 // Prepares to send.
 sendXfer.data = sendData
 sendXfer.dataSize = sizeof(sendData)/sizeof(sendData[0]);
 txFinished = false;

 // Sends out.
 FLEXIO_UART_SendDMA(&uartDev, &g_uartHandle, &sendXfer);

 // Send finished.
 while (!txFinished)
 {
 }

 // Prepares to receive.
 receiveXfer.data = receiveData;
 receiveXfer.dataSize = sizeof(receiveData)/sizeof(receiveData[0]);
 rxFinished = false;

 // Receives.
 FLEXIO_UART_ReceiveDMA(&uartDev, &g_uartHandle, &receiveXfer, NULL);

 // Receive finished.
 while (!rxFinished)
 {
 }

 // ...
}

```

## Data Structures

- struct [\\_flexio\\_uart\\_type](#)  
Define FlexIO UART access structure typedef. [More...](#)
- struct [\\_flexio\\_uart\\_config](#)  
Define FlexIO UART user configuration structure. [More...](#)
- struct [\\_flexio\\_uart\\_transfer](#)  
Define FlexIO UART transfer structure. [More...](#)
- struct [\\_flexio\\_uart\\_handle](#)

Define FLEXIO UART handle structure. [More...](#)

## Macros

- `#define UART_RETRY_TIMES 0U` /\* Defining to zero means to keep waiting for the flag until it is assert/deassert. \*/  
*Retry times for waiting flag.*

## Typedefs

- typedef enum  
[\\_flexio\\_uart\\_bit\\_count\\_per\\_char](#) flexio\_uart\_bit\_count\_per\_char\_t  
*FlexIO UART bit count per char.*
- typedef struct [\\_flexio\\_uart\\_type](#) FLEXIO\_UART\_Type  
*Define FlexIO UART access structure typedef.*
- typedef struct [\\_flexio\\_uart\\_config](#) flexio\_uart\_config\_t  
*Define FlexIO UART user configuration structure.*
- typedef struct  
[\\_flexio\\_uart\\_transfer](#) flexio\_uart\_transfer\_t  
*Define FlexIO UART transfer structure.*
- typedef void(\* [flexio\\_uart\\_transfer\\_callback\\_t](#) )(FLEXIO\_UART\_Type \*base, [flexio\\_uart\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
*FlexIO UART transfer callback function.*

## Enumerations

- enum {  
[kStatus\\_FLEXIO\\_UART\\_TxBusy](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_UART, 0),  
[kStatus\\_FLEXIO\\_UART\\_RxBusy](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_UART, 1),  
[kStatus\\_FLEXIO\\_UART\\_TxIdle](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_UART, 2),  
[kStatus\\_FLEXIO\\_UART\\_RxIdle](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_UART, 3),  
[kStatus\\_FLEXIO\\_UART\\_ERROR](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_UART, 4),  
[kStatus\\_FLEXIO\\_UART\\_RxRingBufferOverrun](#),  
[kStatus\\_FLEXIO\\_UART\\_RxHardwareOverrun](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_UART, 6),  
[kStatus\\_FLEXIO\\_UART\\_Timeout](#) = MAKE\_STATUS(kStatusGroup\_FLEXIO\_UART, 7),  
[kStatus\\_FLEXIO\\_UART\\_BaudrateNotSupport](#) }  
*Error codes for the UART driver.*
- enum [\\_flexio\\_uart\\_bit\\_count\\_per\\_char](#) {  
[kFLEXIO\\_UART\\_7BitsPerChar](#) = 7U,  
[kFLEXIO\\_UART\\_8BitsPerChar](#) = 8U,  
[kFLEXIO\\_UART\\_9BitsPerChar](#) = 9U }  
*FlexIO UART bit count per char.*

- enum `_flexio_uart_interrupt_enable` {  
`kFLEXIO_UART_TxDataRegEmptyInterruptEnable` = 0x1U,  
`kFLEXIO_UART_RxDataRegFullInterruptEnable` = 0x2U }  
*Define FlexIO UART interrupt mask.*
- enum `_flexio_uart_status_flags` {  
`kFLEXIO_UART_TxDataRegEmptyFlag` = 0x1U,  
`kFLEXIO_UART_RxDataRegFullFlag` = 0x2U,  
`kFLEXIO_UART_RxOverRunFlag` = 0x4U }  
*Define FlexIO UART status mask.*

## Driver version

- #define `FSL_FLEXIO_UART_DRIVER_VERSION` (`MAKE_VERSION(2, 5, 0)`)  
*FlexIO UART driver version.*

## Initialization and deinitialization

- `status_t FLEXIO_UART_Init` (`FLEXIO_UART_Type *base`, const `flexio_uart_config_t *userConfig`, `uint32_t srcClock_Hz`)  
*Ungates the FlexIO clock, resets the FlexIO module, configures FlexIO UART hardware, and configures the FlexIO UART with FlexIO UART configuration.*
- void `FLEXIO_UART_Deinit` (`FLEXIO_UART_Type *base`)  
*Resets the FlexIO UART shifter and timer config.*
- void `FLEXIO_UART_GetDefaultConfig` (`flexio_uart_config_t *userConfig`)  
*Gets the default configuration to configure the FlexIO UART.*

## Status

- `uint32_t FLEXIO_UART_GetStatusFlags` (`FLEXIO_UART_Type *base`)  
*Gets the FlexIO UART status flags.*
- void `FLEXIO_UART_ClearStatusFlags` (`FLEXIO_UART_Type *base`, `uint32_t mask`)  
*Gets the FlexIO UART status flags.*

## Interrupts

- void `FLEXIO_UART_EnableInterrupts` (`FLEXIO_UART_Type *base`, `uint32_t mask`)  
*Enables the FlexIO UART interrupt.*
- void `FLEXIO_UART_DisableInterrupts` (`FLEXIO_UART_Type *base`, `uint32_t mask`)  
*Disables the FlexIO UART interrupt.*

## DMA Control

- static `uint32_t FLEXIO_UART_GetTxDataRegisterAddress` (`FLEXIO_UART_Type *base`)



- *Gets the FlexIO UART transmit data register address.*
- static uint32\_t **FLEXIO\_UART\_GetRxDataRegisterAddress** (FLEXIO\_UART\_Type \*base)
- *Gets the FlexIO UART receive data register address.*
- static void **FLEXIO\_UART\_EnableTxDMA** (FLEXIO\_UART\_Type \*base, bool enable)
- *Enables/disables the FlexIO UART transmit DMA.*
- static void **FLEXIO\_UART\_EnableRxDMA** (FLEXIO\_UART\_Type \*base, bool enable)
- *Enables/disables the FlexIO UART receive DMA.*

## Bus Operations

- static void **FLEXIO\_UART\_Enable** (FLEXIO\_UART\_Type \*base, bool enable)
- *Enables/disables the FlexIO UART module operation.*
- static void **FLEXIO\_UART\_WriteByte** (FLEXIO\_UART\_Type \*base, const uint8\_t \*buffer)
- *Writes one byte of data.*
- static void **FLEXIO\_UART\_ReadByte** (FLEXIO\_UART\_Type \*base, uint8\_t \*buffer)
- *Reads one byte of data.*
- **status\_t FLEXIO\_UART\_WriteBlocking** (FLEXIO\_UART\_Type \*base, const uint8\_t \*txData, size\_t txSize)
- *Sends a buffer of data bytes.*
- **status\_t FLEXIO\_UART\_ReadBlocking** (FLEXIO\_UART\_Type \*base, uint8\_t \*rxData, size\_t rxSize)
- *Receives a buffer of bytes.*

## Transactional

- **status\_t FLEXIO\_UART\_TransferCreateHandle** (FLEXIO\_UART\_Type \*base, flexio\_uart\_handle\_t \*handle, flexio\_uart\_transfer\_callback\_t callback, void \*userData)
- *Initializes the UART handle.*
- void **FLEXIO\_UART\_TransferStartRingBuffer** (FLEXIO\_UART\_Type \*base, flexio\_uart\_handle\_t \*handle, uint8\_t \*ringBuffer, size\_t ringBufferSize)
- *Sets up the RX ring buffer.*
- void **FLEXIO\_UART\_TransferStopRingBuffer** (FLEXIO\_UART\_Type \*base, flexio\_uart\_handle\_t \*handle)
- *Aborts the background transfer and uninstalls the ring buffer.*
- **status\_t FLEXIO\_UART\_TransferSendNonBlocking** (FLEXIO\_UART\_Type \*base, flexio\_uart\_handle\_t \*handle, flexio\_uart\_transfer\_t \*xfer)
- *Transmits a buffer of data using the interrupt method.*
- void **FLEXIO\_UART\_TransferAbortSend** (FLEXIO\_UART\_Type \*base, flexio\_uart\_handle\_t \*handle)
- *Aborts the interrupt-driven data transmit.*
- **status\_t FLEXIO\_UART\_TransferGetSendCount** (FLEXIO\_UART\_Type \*base, flexio\_uart\_handle\_t \*handle, size\_t \*count)
- *Gets the number of bytes sent.*
- **status\_t FLEXIO\_UART\_TransferReceiveNonBlocking** (FLEXIO\_UART\_Type \*base, flexio\_uart\_handle\_t \*handle, flexio\_uart\_transfer\_t \*xfer, size\_t \*receivedBytes)
- *Receives a buffer of data using the interrupt method.*

- void [FLEXIO\\_UART\\_TransferAbortReceive](#) ([FLEXIO\\_UART\\_Type](#) \*base, [flexio\\_uart\\_handle\\_t](#) \*handle)  
*Aborts the receive data which was using IRQ.*
- [status\\_t FLEXIO\\_UART\\_TransferGetReceiveCount](#) ([FLEXIO\\_UART\\_Type](#) \*base, [flexio\\_uart\\_handle\\_t](#) \*handle, [size\\_t](#) \*count)  
*Gets the number of bytes received.*
- void [FLEXIO\\_UART\\_TransferHandleIRQ](#) (void \*uartType, void \*uartHandle)  
*FlexIO UART IRQ handler function.*
- void [FLEXIO\\_UART\\_FlushShifters](#) ([FLEXIO\\_UART\\_Type](#) \*base)  
*Flush tx/rx shifters.*

### 18.6.3 Data Structure Documentation

#### 18.6.3.1 struct [\\_flexio\\_uart\\_type](#)

##### Data Fields

- [FLEXIO\\_Type](#) \* [flexioBase](#)  
*FlexIO base pointer.*
- [uint8\\_t](#) [TxPinIndex](#)  
*Pin select for UART\_Tx.*
- [uint8\\_t](#) [RxPinIndex](#)  
*Pin select for UART\_Rx.*
- [uint8\\_t](#) [shifterIndex](#) [2]  
*Shifter index used in FlexIO UART.*
- [uint8\\_t](#) [timerIndex](#) [2]  
*Timer index used in FlexIO UART.*

##### Field Documentation

(1) [FLEXIO\\_Type](#)\* [\\_flexio\\_uart\\_type::flexioBase](#)

(2) [uint8\\_t](#) [\\_flexio\\_uart\\_type::TxPinIndex](#)

(3) [uint8\\_t](#) [\\_flexio\\_uart\\_type::RxPinIndex](#)

(4) [uint8\\_t](#) [\\_flexio\\_uart\\_type::shifterIndex](#)[2]

(5) [uint8\\_t](#) [\\_flexio\\_uart\\_type::timerIndex](#)[2]

#### 18.6.3.2 struct [\\_flexio\\_uart\\_config](#)

##### Data Fields

- bool [enableUart](#)  
*Enable/disable FlexIO UART TX & RX.*
- bool [enableInDoze](#)  
*Enable/disable FlexIO operation in doze mode.*
- bool [enableInDebug](#)

- *Enable/disable FlexIO operation in debug mode.*
- bool [enableFastAccess](#)  
Enable/disable fast access to FlexIO registers,  
*fast access requires the FlexIO clock to be at least twice the frequency of the bus clock.*
- uint32\_t [baudRate\\_Bps](#)  
*Baud rate in Bps.*
- [flexio\\_uart\\_bit\\_count\\_per\\_char\\_t](#) [bitCountPerChar](#)  
*number of bits, 7/8/9 -bit*

### Field Documentation

- (1) bool [\\_flexio\\_uart\\_config::enableUart](#)
- (2) bool [\\_flexio\\_uart\\_config::enableFastAccess](#)
- (3) uint32\_t [\\_flexio\\_uart\\_config::baudRate\\_Bps](#)

### 18.6.3.3 struct [\\_flexio\\_uart\\_transfer](#)

#### Data Fields

- size\_t [dataSize](#)  
*Transfer size.*
- uint8\_t \* [data](#)  
*The buffer of data to be transfer.*
- uint8\_t \* [rxData](#)  
*The buffer to receive data.*
- const uint8\_t \* [txData](#)  
*The buffer of data to be sent.*

### Field Documentation

- (1) uint8\_t\* [\\_flexio\\_uart\\_transfer::data](#)
- (2) uint8\_t\* [\\_flexio\\_uart\\_transfer::rxData](#)
- (3) const uint8\_t\* [\\_flexio\\_uart\\_transfer::txData](#)

### 18.6.3.4 struct [\\_flexio\\_uart\\_handle](#)

#### Data Fields

- const uint8\_t \*volatile [txData](#)  
*Address of remaining data to send.*
- volatile size\_t [txDataSize](#)  
*Size of the remaining data to send.*
- uint8\_t \*volatile [rxData](#)  
*Address of remaining data to receive.*
- volatile size\_t [rxDataSize](#)  
*Size of the remaining data to receive.*
- size\_t [txDataSizeAll](#)

- *Total bytes to be sent.*  
size\_t [rxDataSizeAll](#)
- *Total bytes to be received.*  
uint8\_t \* [rxRingBuffer](#)
- *Start address of the receiver ring buffer.*  
size\_t [rxRingBufferSize](#)
- *Size of the ring buffer.*  
volatile uint16\_t [rxRingBufferHead](#)
- *Index for the driver to store received data into ring buffer.*  
volatile uint16\_t [rxRingBufferTail](#)
- *Index for the user to get data from the ring buffer.*  
[flexio\\_uart\\_transfer\\_callback\\_t](#) [callback](#)
- *Callback function.*  
void \* [userData](#)
- *UART callback function parameter.*  
volatile uint8\_t [txState](#)
- *TX transfer state.*  
volatile uint8\_t [rxState](#)
- *RX transfer state.*



## Field Documentation

- (1) `const uint8_t* volatile _flexio_uart_handle::txData`
- (2) `volatile size_t _flexio_uart_handle::txDataSize`
- (3) `uint8_t* volatile _flexio_uart_handle::rxData`
- (4) `volatile size_t _flexio_uart_handle::rxDataSize`
- (5) `size_t _flexio_uart_handle::txDataSizeAll`
- (6) `size_t _flexio_uart_handle::rxDataSizeAll`
- (7) `uint8_t* _flexio_uart_handle::rxRingBuffer`
- (8) `size_t _flexio_uart_handle::rxRingBufferSize`
- (9) `volatile uint16_t _flexio_uart_handle::rxRingBufferHead`
- (10) `volatile uint16_t _flexio_uart_handle::rxRingBufferTail`
- (11) `flexio_uart_transfer_callback_t _flexio_uart_handle::callback`
- (12) `void* _flexio_uart_handle::userData`
- (13) `volatile uint8_t _flexio_uart_handle::txState`

## 18.6.4 Macro Definition Documentation

18.6.4.1 `#define FSL_FLEXIO_UART_DRIVER_VERSION (MAKE_VERSION(2, 5, 0))`

18.6.4.2 `#define UART_RETRY_TIMES 0U /* Defining to zero means to keep waiting for the flag until it is assert/deassert. */`

## 18.6.5 Typedef Documentation

18.6.5.1 `typedef enum _flexio_uart_bit_count_per_char flexio_uart_bit_count_per_char_t`

18.6.5.2 `typedef struct _flexio_uart_type FLEXIO_UART_Type`

18.6.5.3 `typedef struct _flexio_uart_config flexio_uart_config_t`

18.6.5.4 `typedef struct _flexio_uart_transfer flexio_uart_transfer_t`

18.6.5.5 `typedef void(* flexio_uart_transfer_callback_t)(FLEXIO_UART_Type *base, flexio_uart_handle_t *handle, status_t status, void *userData)`

## 18.6.6 Enumeration Type Documentation

### 18.6.6.1 anonymous enum

*kStatus\_FLEXIO\_UART\_RxBusy* Receiver is busy.  
*kStatus\_FLEXIO\_UART\_TxIdle* UART transmitter is idle.  
*kStatus\_FLEXIO\_UART\_RxIdle* UART receiver is idle.  
*kStatus\_FLEXIO\_UART\_ERROR* ERROR happens on UART.  
*kStatus\_FLEXIO\_UART\_RxRingBufferOverflow* UART RX software ring buffer overrun.  
*kStatus\_FLEXIO\_UART\_RxHardwareOverflow* UART RX receiver overrun.  
*kStatus\_FLEXIO\_UART\_Timeout* UART times out.  
*kStatus\_FLEXIO\_UART\_BaudrateNotSupport* Baudrate is not supported in current clock source.

### 18.6.6.2 enum \_flexio\_uart\_bit\_count\_per\_char

Enumerator

*kFLEXIO\_UART\_7BitsPerChar* 7-bit data characters  
*kFLEXIO\_UART\_8BitsPerChar* 8-bit data characters  
*kFLEXIO\_UART\_9BitsPerChar* 9-bit data characters

### 18.6.6.3 enum \_flexio\_uart\_interrupt\_enable

Enumerator

*kFLEXIO\_UART\_TxDataRegEmptyInterruptEnable* Transmit buffer empty interrupt enable.  
*kFLEXIO\_UART\_RxDataRegFullInterruptEnable* Receive buffer full interrupt enable.

### 18.6.6.4 enum \_flexio\_uart\_status\_flags

Enumerator

*kFLEXIO\_UART\_TxDataRegEmptyFlag* Transmit buffer empty flag.  
*kFLEXIO\_UART\_RxDataRegFullFlag* Receive buffer full flag.  
*kFLEXIO\_UART\_RxOverRunFlag* Receive buffer over run flag.

## 18.6.7 Function Documentation

### 18.6.7.1 status\_t FLEXIO\_UART\_Init ( FLEXIO\_UART\_Type \* *base*, const flexio\_uart\_config\_t \* *userConfig*, uint32\_t *srcClock\_Hz* )

The configuration structure can be filled by the user or be set with default values by [FLEXIO\\_UART\\_GetDefaultConfig\(\)](#).

Example

```

FLEXIO_UART_Type base = {
 .flexioBase = FLEXIO,
 .TxPinIndex = 0,
 .RxPinIndex = 1,
 .shifterIndex = {0,1},
 .timerIndex = {0,1}
};
flexio_uart_config_t config = {
 .enableInDoze = false,
 .enableInDebug = true,
 .enableFastAccess = false,
 .baudRate_Bps = 115200U,
 .bitCountPerChar = 8
};
FLEXIO_UART_Init(base, &config, srcClock_Hz);

```

### Parameters

|                    |                                                |
|--------------------|------------------------------------------------|
| <i>base</i>        | Pointer to the FLEXIO_UART_Type structure.     |
| <i>userConfig</i>  | Pointer to the flexio_uart_config_t structure. |
| <i>srcClock_Hz</i> | FlexIO source clock in Hz.                     |

### Return values

|                                               |                                                               |
|-----------------------------------------------|---------------------------------------------------------------|
| <i>kStatus_Success</i>                        | Configuration success.                                        |
| <i>kStatus_FLEXIO_UART-BaudrateNotSupport</i> | Baudrate is not supported for current clock source frequency. |

### 18.6.7.2 void FLEXIO\_UART\_Deinit ( FLEXIO\_UART\_Type \* *base* )

#### Note

After calling this API, call the FLEXIO\_UART\_Init to use the FlexIO UART module.

### Parameters

|             |                                       |
|-------------|---------------------------------------|
| <i>base</i> | Pointer to FLEXIO_UART_Type structure |
|-------------|---------------------------------------|

### 18.6.7.3 void FLEXIO\_UART\_GetDefaultConfig ( flexio\_uart\_config\_t \* *userConfig* )

The configuration can be used directly for calling the [FLEXIO\\_UART\\_Init\(\)](#). Example:

```

flexio_uart_config_t config;
FLEXIO_UART_GetDefaultConfig(&userConfig);

```



## Parameters

|                   |                                                |
|-------------------|------------------------------------------------|
| <i>userConfig</i> | Pointer to the flexio_uart_config_t structure. |
|-------------------|------------------------------------------------|

**18.6.7.4 uint32\_t FLEXIO\_UART\_GetStatusFlags ( FLEXIO\_UART\_Type \* *base* )**

## Parameters

|             |                                            |
|-------------|--------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure. |
|-------------|--------------------------------------------|

## Returns

FlexIO UART status flags.

**18.6.7.5 void FLEXIO\_UART\_ClearStatusFlags ( FLEXIO\_UART\_Type \* *base*, uint32\_t *mask* )**

## Parameters

|             |                                                                                                                                                                                                                                          |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure.                                                                                                                                                                                               |
| <i>mask</i> | Status flag. The parameter can be any combination of the following values: <ul style="list-style-type: none"> <li>• kFLEXIO_UART_TxDataRegEmptyFlag</li> <li>• kFLEXIO_UART_RxEmptyFlag</li> <li>• kFLEXIO_UART_RxOverRunFlag</li> </ul> |

**18.6.7.6 void FLEXIO\_UART\_EnableInterrupts ( FLEXIO\_UART\_Type \* *base*, uint32\_t *mask* )**

This function enables the FlexIO UART interrupt.

## Parameters

|             |                                            |
|-------------|--------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure. |
| <i>mask</i> | Interrupt source.                          |

**18.6.7.7 void FLEXIO\_UART\_DisableInterrupts ( FLEXIO\_UART\_Type \* *base*, uint32\_t *mask* )**

This function disables the FlexIO UART interrupt.

## Parameters

|             |                                            |
|-------------|--------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure. |
| <i>mask</i> | Interrupt source.                          |

#### 18.6.7.8 static uint32\_t FLEXIO\_UART\_GetTxDataRegisterAddress ( FLEXIO\_UART\_Type \* *base* ) [inline], [static]

This function returns the UART data register address, which is mainly used by DMA/eDMA.

## Parameters

|             |                                            |
|-------------|--------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure. |
|-------------|--------------------------------------------|

## Returns

FlexIO UART transmit data register address.

#### 18.6.7.9 static uint32\_t FLEXIO\_UART\_GetRxDataRegisterAddress ( FLEXIO\_UART\_Type \* *base* ) [inline], [static]

This function returns the UART data register address, which is mainly used by DMA/eDMA.

## Parameters

|             |                                            |
|-------------|--------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure. |
|-------------|--------------------------------------------|

## Returns

FlexIO UART receive data register address.

#### 18.6.7.10 static void FLEXIO\_UART\_EnableTxDMA ( FLEXIO\_UART\_Type \* *base*, bool *enable* ) [inline], [static]

This function enables/disables the FlexIO UART Tx DMA, which means asserting the kFLEXIO\_UART\_TxDataRegEmptyFlag does/doesn't trigger the DMA request.

## Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure. |
| <i>enable</i> | True to enable, false to disable.          |

#### 18.6.7.11 static void FLEXIO\_UART\_EnableRxDMA ( FLEXIO\_UART\_Type \* *base*, bool *enable* ) [inline], [static]

This function enables/disables the FlexIO UART Rx DMA, which means asserting kFLEXIO\_UART\_RxDataRegFullFlag does/doesn't trigger the DMA request.

## Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure. |
| <i>enable</i> | True to enable, false to disable.          |

#### 18.6.7.12 static void FLEXIO\_UART\_Enable ( FLEXIO\_UART\_Type \* *base*, bool *enable* ) [inline], [static]

## Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type.                |
| <i>enable</i> | True to enable, false does not have any effect. |

#### 18.6.7.13 static void FLEXIO\_UART\_WriteByte ( FLEXIO\_UART\_Type \* *base*, const uint8\_t \* *buffer* ) [inline], [static]

## Note

This is a non-blocking API, which returns directly after the data is put into the data register. Ensure that the TxEmptyFlag is asserted before calling this API.

## Parameters

|             |                                            |
|-------------|--------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure. |
|-------------|--------------------------------------------|

|               |                         |
|---------------|-------------------------|
| <i>buffer</i> | The data bytes to send. |
|---------------|-------------------------|

**18.6.7.14 static void FLEXIO\_UART\_ReadByte ( FLEXIO\_UART\_Type \* *base*, uint8\_t \* *buffer* ) [inline], [static]**

Note

This is a non-blocking API, which returns directly after the data is read from the data register. Ensure that the RxFullFlag is asserted before calling this API.

Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure. |
| <i>buffer</i> | The buffer to store the received bytes.    |

**18.6.7.15 status\_t FLEXIO\_UART\_WriteBlocking ( FLEXIO\_UART\_Type \* *base*, const uint8\_t \* *txData*, size\_t *txSize* )**

Note

This function blocks using the polling method until all bytes have been sent.

Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure. |
| <i>txData</i> | The data bytes to send.                    |
| <i>txSize</i> | The number of data bytes to send.          |

Return values

|                                          |                                         |
|------------------------------------------|-----------------------------------------|
| <i>kStatus_FLEXIO_UART-<br/>_Timeout</i> | Transmission timed out and was aborted. |
| <i>kStatus_Success</i>                   | Successfully wrote all data.            |

**18.6.7.16 status\_t FLEXIO\_UART\_ReadBlocking ( FLEXIO\_UART\_Type \* *base*, uint8\_t \* *rxData*, size\_t *rxSize* )**

Note

This function blocks using the polling method until all bytes have been received.

## Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure. |
| <i>rxData</i> | The buffer to store the received bytes.    |
| <i>rxSize</i> | The number of data bytes to be received.   |

## Return values

|                                          |                                         |
|------------------------------------------|-----------------------------------------|
| <i>kStatus_FLEXIO_UART-<br/>_Timeout</i> | Transmission timed out and was aborted. |
| <i>kStatus_Success</i>                   | Successfully received all data.         |

#### 18.6.7.17 status\_t FLEXIO\_UART\_TransferCreateHandle ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle*, flexio\_uart\_transfer\_callback\_t *callback*, void \* *userData* )

This function initializes the FlexIO UART handle, which can be used for other FlexIO UART transactional APIs. Call this API once to get the initialized handle.

The UART driver supports the "background" receiving, which means that users can set up a RX ring buffer optionally. Data received is stored into the ring buffer even when the user doesn't call the [FLEXIO\\_UART\\_TransferReceiveNonBlocking\(\)](#) API. If there is already data received in the ring buffer, users can get the received data from the ring buffer directly. The ring buffer is disabled if passing NULL as *ringBuffer*.

## Parameters

|                 |                                                                            |
|-----------------|----------------------------------------------------------------------------|
| <i>base</i>     | to FLEXIO_UART_Type structure.                                             |
| <i>handle</i>   | Pointer to the flexio_uart_handle_t structure to store the transfer state. |
| <i>callback</i> | The callback function.                                                     |
| <i>userData</i> | The parameter of the callback function.                                    |

## Return values

|                           |                                                |
|---------------------------|------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                |
| <i>kStatus_OutOfRange</i> | The FlexIO type/handle/ISR table out of range. |

#### 18.6.7.18 void FLEXIO\_UART\_TransferStartRingBuffer ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle*, uint8\_t \* *ringBuffer*, size\_t *ringBufferSize* )

This function sets up the RX ring buffer to a specific UART handle.

When the RX ring buffer is used, data received is stored into the ring buffer even when the user doesn't

call the `UART_ReceiveNonBlocking()` API. If there is already data received in the ring buffer, users can get the received data from the ring buffer directly.

#### Note

When using the RX ring buffer, one byte is reserved for internal use. In other words, if `ringBufferSize` is 32, only 31 bytes are used for saving data.

#### Parameters

|                       |                                                                                              |
|-----------------------|----------------------------------------------------------------------------------------------|
| <i>base</i>           | Pointer to the <code>FLEXIO_UART_Type</code> structure.                                      |
| <i>handle</i>         | Pointer to the <code>flexio_uart_handle_t</code> structure to store the transfer state.      |
| <i>ringBuffer</i>     | Start address of ring buffer for background receiving. Pass NULL to disable the ring buffer. |
| <i>ringBufferSize</i> | Size of the ring buffer.                                                                     |

#### 18.6.7.19 void FLEXIO\_UART\_TransferStopRingBuffer ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle* )

This function aborts the background transfer and uninstalls the ring buffer.

#### Parameters

|               |                                                                                         |
|---------------|-----------------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the <code>FLEXIO_UART_Type</code> structure.                                 |
| <i>handle</i> | Pointer to the <code>flexio_uart_handle_t</code> structure to store the transfer state. |

#### 18.6.7.20 status\_t FLEXIO\_UART\_TransferSendNonBlocking ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle*, flexio\_uart\_transfer\_t \* *xfer* )

This function sends data using an interrupt method. This is a non-blocking function, which returns directly without waiting for all data to be written to the TX register. When all data is written to the TX register in ISR, the FlexIO UART driver calls the callback function and passes the [kStatus\\_FLEXIO\\_UART\\_TxIdle](#) as status parameter.

#### Note

The `kStatus_FLEXIO_UART_TxIdle` is passed to the upper layer when all data is written to the TX register. However, it does not ensure that all data is sent out.

## Parameters

|               |                                                                              |
|---------------|------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure.                                   |
| <i>handle</i> | Pointer to the flexio_uart_handle_t structure to store the transfer state.   |
| <i>xfer</i>   | FlexIO UART transfer structure. See <a href="#">flexio_uart_transfer_t</a> . |

## Return values

|                            |                                                                                |
|----------------------------|--------------------------------------------------------------------------------|
| <i>kStatus_Success</i>     | Successfully starts the data transmission.                                     |
| <i>kStatus_UART_TxBusy</i> | Previous transmission still not finished, data not written to the TX register. |

### 18.6.7.21 void FLEXIO\_UART\_TransferAbortSend ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle* )

This function aborts the interrupt-driven data sending. Get the remainBytes to find out how many bytes are still not sent out.

## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure.                                 |
| <i>handle</i> | Pointer to the flexio_uart_handle_t structure to store the transfer state. |

### 18.6.7.22 status\_t FLEXIO\_UART\_TransferGetSendCount ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle*, size\_t \* *count* )

This function gets the number of bytes sent driven by interrupt.

## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure.                                 |
| <i>handle</i> | Pointer to the flexio_uart_handle_t structure to store the transfer state. |
| <i>count</i>  | Number of bytes sent so far by the non-blocking transaction.               |

## Return values

|                                     |                                                   |
|-------------------------------------|---------------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | transfer has finished or no transfer in progress. |
|-------------------------------------|---------------------------------------------------|

|                        |                                |
|------------------------|--------------------------------|
| <i>kStatus_Success</i> | Successfully return the count. |
|------------------------|--------------------------------|

#### 18.6.7.23 **status\_t FLEXIO\_UART\_TransferReceiveNonBlocking ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle*, flexio\_uart\_transfer\_t \* *xfer*, size\_t \* *receivedBytes* )**

This function receives data using the interrupt method. This is a non-blocking function, which returns without waiting for all data to be received. If the RX ring buffer is used and not empty, the data in ring buffer is copied and the parameter *receivedBytes* shows how many bytes are copied from the ring buffer. After copying, if the data in ring buffer is not enough to read, the receive request is saved by the UART driver. When new data arrives, the receive request is serviced first. When all data is received, the UART driver notifies the upper layer through a callback function and passes the status parameter *kStatus\_UART\_RxIdle*. For example, if the upper layer needs 10 bytes but there are only 5 bytes in the ring buffer, the 5 bytes are copied to *xfer->data*. This function returns with the parameter *receivedBytes* set to 5. For the last 5 bytes, newly arrived data is saved from the *xfer->data[5]*. When 5 bytes are received, the UART driver notifies upper layer. If the RX ring buffer is not enabled, this function enables the RX and RX interrupt to receive data to *xfer->data*. When all data is received, the upper layer is notified.

##### Parameters

|                      |                                                                            |
|----------------------|----------------------------------------------------------------------------|
| <i>base</i>          | Pointer to the FLEXIO_UART_Type structure.                                 |
| <i>handle</i>        | Pointer to the flexio_uart_handle_t structure to store the transfer state. |
| <i>xfer</i>          | UART transfer structure. See <a href="#">flexio_uart_transfer_t</a> .      |
| <i>receivedBytes</i> | Bytes received from the ring buffer directly.                              |

##### Return values

|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| <i>kStatus_Success</i>            | Successfully queue the transfer into the transmit queue. |
| <i>kStatus_FLEXIO_UART-RxBusy</i> | Previous receive request is not finished.                |

#### 18.6.7.24 **void FLEXIO\_UART\_TransferAbortReceive ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle* )**

This function aborts the receive data which was using IRQ.



## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure.                                 |
| <i>handle</i> | Pointer to the flexio_uart_handle_t structure to store the transfer state. |

### 18.6.7.25 status\_t FLEXIO\_UART\_TransferGetReceiveCount ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_handle\_t \* *handle*, size\_t \* *count* )

This function gets the number of bytes received driven by interrupt.

## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>base</i>   | Pointer to the FLEXIO_UART_Type structure.                                 |
| <i>handle</i> | Pointer to the flexio_uart_handle_t structure to store the transfer state. |
| <i>count</i>  | Number of bytes received so far by the non-blocking transaction.           |

## Return values

|                                     |                                                   |
|-------------------------------------|---------------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | transfer has finished or no transfer in progress. |
| <i>kStatus_Success</i>              | Successfully return the count.                    |

### 18.6.7.26 void FLEXIO\_UART\_TransferHandleIRQ ( void \* *uartType*, void \* *uartHandle* )

This function processes the FlexIO UART transmit and receives the IRQ request.

## Parameters

|                   |                                                                            |
|-------------------|----------------------------------------------------------------------------|
| <i>uartType</i>   | Pointer to the FLEXIO_UART_Type structure.                                 |
| <i>uartHandle</i> | Pointer to the flexio_uart_handle_t structure to store the transfer state. |

### 18.6.7.27 void FLEXIO\_UART\_FlushShifters ( FLEXIO\_UART\_Type \* *base* )

## Parameters

---

|             |                                            |
|-------------|--------------------------------------------|
| <i>base</i> | Pointer to the FLEXIO_UART_Type structure. |
|-------------|--------------------------------------------|

## Chapter 19

# GPIO: General-Purpose Input/Output Driver

### 19.1 Overview

#### Modules

- [FGPIO Driver](#)
- [GPIO Driver](#)

#### Data Structures

- struct [\\_gpio\\_pin\\_config](#)  
*The GPIO pin configuration structure. [More...](#)*

#### Typedefs

- typedef enum [\\_gpio\\_pin\\_direction](#) [gpio\\_pin\\_direction\\_t](#)  
*GPIO direction definition.*
- typedef struct [\\_gpio\\_pin\\_config](#) [gpio\\_pin\\_config\\_t](#)  
*The GPIO pin configuration structure.*
- typedef enum [\\_gpio\\_interrupt\\_config](#) [gpio\\_interrupt\\_config\\_t](#)  
*Configures the interrupt generation condition.*

#### Enumerations

- enum [\\_gpio\\_pin\\_direction](#) {  
    [kGPIO\\_DigitalInput](#) = 0U,  
    [kGPIO\\_DigitalOutput](#) = 1U }  
*GPIO direction definition.*
- enum [\\_gpio\\_interrupt\\_config](#) {  
    [kGPIO\\_InterruptStatusFlagDisabled](#) = 0x0U,  
    [kGPIO\\_DMARisingEdge](#) = 0x1U,  
    [kGPIO\\_DMAFallingEdge](#) = 0x2U,  
    [kGPIO\\_DMAEitherEdge](#) = 0x3U,  
    [kGPIO\\_FlagRisingEdge](#) = 0x05U,  
    [kGPIO\\_FlagFallingEdge](#) = 0x06U,  
    [kGPIO\\_FlagEitherEdge](#) = 0x07U,  
    [kGPIO\\_InterruptLogicZero](#) = 0x8U,  
    [kGPIO\\_InterruptRisingEdge](#) = 0x9U,  
    [kGPIO\\_InterruptFallingEdge](#) = 0xAU,  
    [kGPIO\\_InterruptEitherEdge](#) = 0xBU,  
    [kGPIO\\_InterruptLogicOne](#) = 0xCU,  
    [kGPIO\\_ActiveHighTriggerOutputEnable](#) = 0xDU,  
    [kGPIO\\_ActiveLowTriggerOutputEnable](#) = 0xEU }  
*Configures the interrupt generation condition.*

*Configures the interrupt generation condition.*

## Driver version

- #define [FSL\\_GPIO\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 7, 3))  
*GPIO driver version.*

## 19.2 Data Structure Documentation

### 19.2.1 struct \_gpio\_pin\_config

Each pin can only be configured as either an output pin or an input pin at a time. If configured as an input pin, leave the outputConfig unused. Note that in some use cases, the corresponding port property should be configured in advance with the `PORT_SetPinConfig()`.

#### Data Fields

- [gpio\\_pin\\_direction\\_t](#) `pinDirection`  
*GPIO direction, input or output.*
- `uint8_t` [outputLogic](#)  
*Set a default output logic, which has no use in input.*

## 19.3 Macro Definition Documentation

### 19.3.1 #define FSL\_GPIO\_DRIVER\_VERSION (MAKE\_VERSION(2, 7, 3))

## 19.4 Typedef Documentation

### 19.4.1 typedef struct \_gpio\_pin\_config gpio\_pin\_config\_t

Each pin can only be configured as either an output pin or an input pin at a time. If configured as an input pin, leave the outputConfig unused. Note that in some use cases, the corresponding port property should be configured in advance with the `PORT_SetPinConfig()`.

### 19.4.2 typedef enum \_gpio\_interrupt\_config gpio\_interrupt\_config\_t

## 19.5 Enumeration Type Documentation

### 19.5.1 enum \_gpio\_pin\_direction

Enumerator

- kGPIO\_DigitalInput*** Set current pin as digital input.
- kGPIO\_DigitalOutput*** Set current pin as digital output.

### 19.5.2 enum \_gpio\_interrupt\_config

Enumerator

***kGPIO\_InterruptStatusFlagDisabled*** Interrupt status flag is disabled.  
***kGPIO\_DMARisingEdge*** ISF flag and DMA request on rising edge.  
***kGPIO\_DMAFallingEdge*** ISF flag and DMA request on falling edge.  
***kGPIO\_DMAEitherEdge*** ISF flag and DMA request on either edge.  
***kGPIO\_FlagRisingEdge*** Flag sets on rising edge.  
***kGPIO\_FlagFallingEdge*** Flag sets on falling edge.  
***kGPIO\_FlagEitherEdge*** Flag sets on either edge.  
***kGPIO\_InterruptLogicZero*** Interrupt when logic zero.  
***kGPIO\_InterruptRisingEdge*** Interrupt on rising edge.  
***kGPIO\_InterruptFallingEdge*** Interrupt on falling edge.  
***kGPIO\_InterruptEitherEdge*** Interrupt on either edge.  
***kGPIO\_InterruptLogicOne*** Interrupt when logic one.  
***kGPIO\_ActiveHighTriggerOutputEnable*** Enable active high-trigger output.  
***kGPIO\_ActiveLowTriggerOutputEnable*** Enable active low-trigger output.

## 19.6 GPIO Driver

### 19.6.1 Overview

The MCUXpresso SDK provides a peripheral driver for the General-Purpose Input/Output (GPIO) module of MCUXpresso SDK devices.

### 19.6.2 Typical use case

#### 19.6.2.1 Output Operation

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/gpio`

#### 19.6.2.2 Input Operation

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/gpio`

## GPIO Configuration

- void [GPIO\\_PinInit](#) (GPIO\_Type \*base, uint32\_t pin, const [gpio\\_pin\\_config\\_t](#) \*config)  
*Initializes a GPIO pin used by the board.*

## GPIO Output Operations

- static void [GPIO\\_PinWrite](#) (GPIO\_Type \*base, uint32\_t pin, uint8\_t output)  
*Sets the output level of the multiple GPIO pins to the logic 1 or 0.*
- static void [GPIO\\_PortSet](#) (GPIO\_Type \*base, uint32\_t mask)  
*Sets the output level of the multiple GPIO pins to the logic 1.*
- static void [GPIO\\_PortClear](#) (GPIO\_Type \*base, uint32\_t mask)  
*Sets the output level of the multiple GPIO pins to the logic 0.*
- static void [GPIO\\_PortToggle](#) (GPIO\_Type \*base, uint32\_t mask)  
*Reverses the current output logic of the multiple GPIO pins.*

## GPIO Input Operations

- static uint32\_t [GPIO\\_PinRead](#) (GPIO\_Type \*base, uint32\_t pin)  
*Reads the current input value of the GPIO port.*

## GPIO Interrupt

- static void [GPIO\\_SetPinInterruptConfig](#) (GPIO\_Type \*base, uint32\_t pin, [gpio\\_interrupt\\_config\\_t](#) config)

- Configures the gpio pin interrupt/DMA request.*
- uint32\_t [GPIO\\_GpioGetInterruptFlags](#) (GPIO\_Type \*base)  
*Read the GPIO interrupt status flags.*
- uint8\_t [GPIO\\_PinGetInterruptFlag](#) (GPIO\_Type \*base, uint32\_t pin)  
*Read individual pin's interrupt status flag.*
- void [GPIO\\_GpioClearInterruptFlags](#) (GPIO\_Type \*base, uint32\_t mask)  
*Clears GPIO pin interrupt status flags.*
- void [GPIO\\_PinClearInterruptFlag](#) (GPIO\_Type \*base, uint32\_t pin)  
*Clear GPIO individual pin's interrupt status flag.*
- static uint32\_t [GPIO\\_GetPinsDMARequestFlags](#) (GPIO\_Type \*base)  
*Reads the GPIO DMA request flags.*
- static void [GPIO\\_SetMultipleInterruptPinsConfig](#) (GPIO\_Type \*base, uint32\_t mask, [gpio\\_interrupt\\_config\\_t](#) config)  
*Sets the GPIO interrupt configuration in PCR register for multiple pins.*

### 19.6.3 Function Documentation

#### 19.6.3.1 void GPIO\_PinInit ( GPIO\_Type \* *base*, uint32\_t *pin*, const gpio\_pin\_config\_t \* *config* )

To initialize the GPIO, define a pin configuration, as either input or output, in the user file. Then, call the [GPIO\\_PinInit\(\)](#) function.

This is an example to define an input pin or an output pin configuration.

```
* Define a digital input pin configuration,
* gpio_pin_config_t config =
* {
* kGPIO_DigitalInput,
* 0,
* }
* Define a digital output pin configuration,
* gpio_pin_config_t config =
* {
* kGPIO_DigitalOutput,
* 0,
* }
*
```

#### Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on.) |
| <i>pin</i>  | GPIO port pin number                                           |

|               |                                |
|---------------|--------------------------------|
| <i>config</i> | GPIO pin configuration pointer |
|---------------|--------------------------------|

**19.6.3.2 static void GPIO\_PinWrite ( GPIO\_Type \* *base*, uint32\_t *pin*, uint8\_t *output* ) [inline], [static]**

Parameters

|               |                                                                                                                                                                                        |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>   | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on.)                                                                                                                         |
| <i>pin</i>    | GPIO pin number                                                                                                                                                                        |
| <i>output</i> | GPIO pin output logic level. <ul style="list-style-type: none"> <li>• 0: corresponding pin output low-logic level.</li> <li>• 1: corresponding pin output high-logic level.</li> </ul> |

**19.6.3.3 static void GPIO\_PortSet ( GPIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**

Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on.) |
| <i>mask</i> | GPIO pin number macro                                          |

**19.6.3.4 static void GPIO\_PortClear ( GPIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**

Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on.) |
| <i>mask</i> | GPIO pin number macro                                          |

**19.6.3.5 static void GPIO\_PortToggle ( GPIO\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**



## Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on.) |
| <i>mask</i> | GPIO pin number macro                                          |

### 19.6.3.6 static uint32\_t GPIO\_PinRead ( GPIO\_Type \* *base*, uint32\_t *pin* ) [inline], [static]

## Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on.) |
| <i>pin</i>  | GPIO pin number                                                |

## Return values

|             |                                                                                                                                                                          |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>GPIO</i> | port input value <ul style="list-style-type: none"> <li>• 0: corresponding pin input low-logic level.</li> <li>• 1: corresponding pin input high-logic level.</li> </ul> |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 19.6.3.7 static void GPIO\_SetPinInterruptConfig ( GPIO\_Type \* *base*, uint32\_t *pin*, gpio\_interrupt\_config\_t *config* ) [inline], [static]

Parameters

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>   | GPIO peripheral base pointer.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>pin</i>    | GPIO pin number.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>config</i> | <p>GPIO pin interrupt configuration.</p> <ul style="list-style-type: none"> <li>• <a href="#">kGPIO_InterruptStatusFlagDisabled</a>: Interrupt/DMA request disabled.</li> <li>• <a href="#">kGPIO_DMARisingEdge</a> : DMA request on rising edge(if the DMA requests exit).</li> <li>• <a href="#">kGPIO_DMAFallingEdge</a>: DMA request on falling edge(if the DMA requests exit).</li> <li>• <a href="#">kGPIO_DMAEitherEdge</a> : DMA request on either edge(if the DMA requests exit).</li> <li>• <a href="#">kGPIO_FlagRisingEdge</a> : Flag sets on rising edge(if the Flag states exit).</li> <li>• <a href="#">kGPIO_FlagFallingEdge</a> : Flag sets on falling edge(if the Flag states exit).</li> <li>• <a href="#">kGPIO_FlagEitherEdge</a> : Flag sets on either edge(if the Flag states exit).</li> <li>• <a href="#">kGPIO_InterruptLogicZero</a> : Interrupt when logic zero.</li> <li>• <a href="#">kGPIO_InterruptRisingEdge</a> : Interrupt on rising edge.</li> <li>• <a href="#">kGPIO_InterruptFallingEdge</a>: Interrupt on falling edge.</li> <li>• <a href="#">kGPIO_InterruptEitherEdge</a> : Interrupt on either edge.</li> <li>• <a href="#">kGPIO_InterruptLogicOne</a> : Interrupt when logic one.</li> <li>• <a href="#">kGPIO_ActiveHighTriggerOutputEnable</a> : Enable active high-trigger output (if the trigger states exit).</li> <li>• <a href="#">kGPIO_ActiveLowTriggerOutputEnable</a> : Enable active low-trigger output (if the trigger states exit).</li> </ul> |

### 19.6.3.8 uint32\_t GPIO\_GpioGetInterruptFlags ( GPIO\_Type \* *base* )

Parameters

|             |                                                                 |
|-------------|-----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer. (GPIOA, GPIOB, GPIOC, and so on.) |
|-------------|-----------------------------------------------------------------|

Returns

The current GPIO's interrupt status flag. '1' means the related pin's flag is set, '0' means the related pin's flag not set. For example, the return value 0x00010001 means the pin 0 and 17 have the interrupt pending.

### 19.6.3.9 uint8\_t GPIO\_PinGetInterruptFlag ( GPIO\_Type \* *base*, uint32\_t *pin* )

## Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer. (GPIOA, GPIOB, GPIOC, and so on) |
| <i>pin</i>  | GPIO specific pin number.                                      |

## Returns

The current selected pin's interrupt status flag.

### 19.6.3.10 void GPIO\_GpioClearInterruptFlags ( GPIO\_Type \* *base*, uint32\_t *mask* )

## Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on.) |
| <i>mask</i> | GPIO pin number macro                                          |

### 19.6.3.11 void GPIO\_PinClearInterruptFlag ( GPIO\_Type \* *base*, uint32\_t *pin* )

## Parameters

|             |                                                                |
|-------------|----------------------------------------------------------------|
| <i>base</i> | GPIO peripheral base pointer (GPIOA, GPIOB, GPIOC, and so on). |
| <i>pin</i>  | GPIO specific pin number.                                      |

### 19.6.3.12 static uint32\_t GPIO\_GetPinsDMAResponseFlags ( GPIO\_Type \* *base* ) [inline], [static]

The corresponding flag will be cleared automatically at the completion of the requested DMA transfer

### 19.6.3.13 static void GPIO\_SetMultipleInterruptPinsConfig ( GPIO\_Type \* *base*, uint32\_t *mask*, gpio\_interrupt\_config\_t *config* ) [inline], [static]

## Parameters

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>   | GPIO peripheral base pointer.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <i>mask</i>   | GPIO pin number macro.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>config</i> | <p>GPIO pin interrupt configuration.</p> <ul style="list-style-type: none"> <li>• <a href="#">kGPIO_InterruptStatusFlagDisabled</a>: Interrupt disabled.</li> <li>• <a href="#">kGPIO_DMARisingEdge</a> : DMA request on rising edge(if the DMA requests exit).</li> <li>• <a href="#">kGPIO_DMAFallingEdge</a>: DMA request on falling edge(if the DMA requests exit).</li> <li>• <a href="#">kGPIO_DMAEitherEdge</a> : DMA request on either edge(if the DMA requests exit).</li> <li>• <a href="#">kGPIO_FlagRisingEdge</a> : Flag sets on rising edge(if the Flag states exit).</li> <li>• <a href="#">kGPIO_FlagFallingEdge</a> : Flag sets on falling edge(if the Flag states exit).</li> <li>• <a href="#">kGPIO_FlagEitherEdge</a> : Flag sets on either edge(if the Flag states exit).</li> <li>• <a href="#">kGPIO_InterruptLogicZero</a> : Interrupt when logic zero.</li> <li>• <a href="#">kGPIO_InterruptRisingEdge</a> : Interrupt on rising edge.</li> <li>• <a href="#">kGPIO_InterruptFallingEdge</a>: Interrupt on falling edge.</li> <li>• <a href="#">kGPIO_InterruptEitherEdge</a> : Interrupt on either edge.</li> <li>• <a href="#">kGPIO_InterruptLogicOne</a> : Interrupt when logic one.</li> <li>• <a href="#">kGPIO_ActiveHighTriggerOutputEnable</a> : Enable active high-trigger output (if the trigger states exit).</li> <li>• <a href="#">kGPIO_ActiveLowTriggerOutputEnable</a> : Enable active low-trigger output (if the trigger states exit)..</li> </ul> |

## 19.7 FGPIO Driver

This section describes the programming interface of the FGPIO driver. The FGPIO driver configures the FGPIO module and provides a functional interface to build the GPIO application.

### Note

FGPIO (Fast GPIO) is only available in a few MCUs. FGPIO and GPIO share the same peripheral but use different registers. FGPIO is closer to the core than the regular GPIO and it's faster to read and write.

### 19.7.1 Typical use case

#### 19.7.1.1 Output Operation

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/gpio`

#### 19.7.1.2 Input Operation

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/gpio`

## Chapter 20

# LLWU: Low-Leakage Wakeup Unit Driver

### 20.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Low-Leakage Wakeup Unit (LLWU) module of MCUXpresso SDK devices. The LLWU module allows the user to select external pin sources and internal modules as a wake-up source from low-leakage power modes.

### 20.2 External wakeup pins configurations

Configures the external wakeup pins' working modes, gets, and clears the wake pin flags. External wakeup pins are accessed by the `pinIndex`, which is started from 1. Numbers of the external pins depend on the SoC configuration.

### 20.3 Internal wakeup modules configurations

Enables/disables the internal wakeup modules and gets the module flags. Internal modules are accessed by `moduleIndex`, which is started from 1. Numbers of external pins depend the on SoC configuration.

### 20.4 Digital pin filter for external wakeup pin configurations

Configures the digital pin filter of the external wakeup pins' working modes, gets, and clears the pin filter flags. Digital pin filters are accessed by the `filterIndex`, which is started from 1. Numbers of external pins depend on the SoC configuration.

### Typedefs

- typedef enum  
`_llwu_external_pin_mode llwu_external_pin_mode_t`  
*External input pin control modes.*
- typedef enum `_llwu_pin_filter_mode llwu_pin_filter_mode_t`  
*Digital filter control modes.*

### Enumerations

- enum `_llwu_external_pin_mode` {  
`kLLWU_ExternalPinDisable` = 0U,  
`kLLWU_ExternalPinRisingEdge` = 1U,  
`kLLWU_ExternalPinFallingEdge` = 2U,  
`kLLWU_ExternalPinAnyEdge` = 3U }  
*External input pin control modes.*

- enum `_llwu_pin_filter_mode` {  
`kLLWU_PinFilterDisable` = 0U,  
`kLLWU_PinFilterRisingEdge` = 1U,  
`kLLWU_PinFilterFallingEdge` = 2U,  
`kLLWU_PinFilterAnyEdge` = 3U }

*Digital filter control modes.*

## Driver version

- #define `FSL_LLWU_DRIVER_VERSION` (`MAKE_VERSION(2, 0, 5)`)  
*LLWU driver version.*

## 20.5 Macro Definition Documentation

### 20.5.1 #define FSL\_LLWU\_DRIVER\_VERSION (MAKE\_VERSION(2, 0, 5))

## 20.6 Enumeration Type Documentation

### 20.6.1 enum \_llwu\_external\_pin\_mode

Enumerator

***kLLWU\_ExternalPinDisable*** Pin disabled as a wakeup input.

***kLLWU\_ExternalPinRisingEdge*** Pin enabled with the rising edge detection.

***kLLWU\_ExternalPinFallingEdge*** Pin enabled with the falling edge detection.

***kLLWU\_ExternalPinAnyEdge*** Pin enabled with any change detection.

### 20.6.2 enum \_llwu\_pin\_filter\_mode

Enumerator

***kLLWU\_PinFilterDisable*** Filter disabled.

***kLLWU\_PinFilterRisingEdge*** Filter positive edge detection.

***kLLWU\_PinFilterFallingEdge*** Filter negative edge detection.

***kLLWU\_PinFilterAnyEdge*** Filter any edge detection.

## Chapter 21

# LPADC: 12-bit SAR Analog-to-Digital Converter Driver

### 21.1 Overview

The MCUXpresso SDK provides a peripheral driver for the 12-bit SAR Analog-to-Digital Converter (LP-ADC) module of MCUXpresso SDK devices.

### 21.2 Typical use case

#### 21.2.1 Polling Configuration

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/lpadc`

#### 21.2.2 Interrupt Configuration

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/lpadc`

### Files

- file [fsl\\_lpadc.h](#)

### Data Structures

- struct [lpadc\\_config\\_t](#)  
*LPADC global configuration. [More...](#)*
- struct [lpadc\\_conv\\_command\\_config\\_t](#)  
*Define structure to keep the configuration for conversion command. [More...](#)*
- struct [lpadc\\_conv\\_trigger\\_config\\_t](#)  
*Define structure to keep the configuration for conversion trigger. [More...](#)*
- struct [lpadc\\_conv\\_result\\_t](#)  
*Define the structure to keep the conversion result. [More...](#)*

### Macros

- #define [LPADC\\_GET\\_ACTIVE\\_COMMAND\\_STATUS](#)(statusVal) ((statusVal & ADC\_STAT\_CMDACT\_MASK) >> ADC\_STAT\_CMDACT\_SHIFT)  
*Define the MACRO function to get command status from status value.*
- #define [LPADC\\_GET\\_ACTIVE\\_TRIGGER\\_STATUE](#)(statusVal) ((statusVal & ADC\_STAT\_TRGACT\_MASK) >> ADC\_STAT\_TRGACT\_SHIFT)  
*Define the MACRO function to get trigger status from status value.*



## Typedefs

- typedef enum  
[\\_lpadc\\_sample\\_scale\\_mode](#) [lpadc\\_sample\\_scale\\_mode\\_t](#)  
*Define enumeration of sample scale mode.*
- typedef enum  
[\\_lpadc\\_sample\\_channel\\_mode](#) [lpadc\\_sample\\_channel\\_mode\\_t](#)  
*Define enumeration of channel sample mode.*
- typedef enum  
[\\_lpadc\\_hardware\\_average\\_mode](#) [lpadc\\_hardware\\_average\\_mode\\_t](#)  
*Define enumeration of hardware average selection.*
- typedef enum  
[\\_lpadc\\_sample\\_time\\_mode](#) [lpadc\\_sample\\_time\\_mode\\_t](#)  
*Define enumeration of sample time selection.*
- typedef enum  
[\\_lpadc\\_hardware\\_compare\\_mode](#) [lpadc\\_hardware\\_compare\\_mode\\_t](#)  
*Define enumeration of hardware compare mode.*
- typedef enum  
[\\_lpadc\\_reference\\_voltage\\_mode](#) [lpadc\\_reference\\_voltage\\_source\\_t](#)  
*Define enumeration of reference voltage source.*
- typedef enum  
[\\_lpadc\\_power\\_level\\_mode](#) [lpadc\\_power\\_level\\_mode\\_t](#)  
*Define enumeration of power configuration.*
- typedef enum  
[\\_lpadc\\_trigger\\_priority\\_policy](#) [lpadc\\_trigger\\_priority\\_policy\\_t](#)  
*Define enumeration of trigger priority policy.*

## Enumerations

- enum [\\_lpadc\\_status\\_flags](#) {  
[kLPADC\\_ResultFIFO0OverflowFlag](#) = [ADC\\_STAT\\_FOF0\\_MASK](#),  
[kLPADC\\_ResultFIFO0ReadyFlag](#) = [ADC\\_STAT\\_RDY0\\_MASK](#),  
[kLPADC\\_TriggerExceptionFlag](#) = [ADC\\_STAT\\_TEXC\\_INT\\_MASK](#),  
[kLPADC\\_TriggerCompletionFlag](#) = [ADC\\_STAT\\_TCOMP\\_INT\\_MASK](#),  
[kLPADC\\_ActiveFlag](#) = [ADC\\_STAT\\_ADC\\_ACTIVE\\_MASK](#),  
[kLPADC\\_ResultFIFOOverflowFlag](#) = [kLPADC\\_ResultFIFO0OverflowFlag](#),  
[kLPADC\\_ResultFIFOReadyFlag](#) = [kLPADC\\_ResultFIFO0ReadyFlag](#) }  
*Define hardware flags of the module.*
- enum [\\_lpadc\\_interrupt\\_enable](#) {  
[kLPADC\\_ResultFIFO0OverflowInterruptEnable](#) = [ADC\\_IE\\_FOFIE0\\_MASK](#),  
[kLPADC\\_FIFO0WatermarkInterruptEnable](#) = [ADC\\_IE\\_FWMIE0\\_MASK](#),  
[kLPADC\\_ResultFIFOOverflowInterruptEnable](#) = [kLPADC\\_ResultFIFO0OverflowInterrupt-](#)

Enable,

```

kLPADC_FIFOWatermarkInterruptEnable = kLPADC_FIFO0WatermarkInterruptEnable,
kLPADC_TriggerExceptionInterruptEnable = ADC_IE_TEXC_IE_MASK,
kLPADC_Trigger0CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 0UL),
kLPADC_Trigger1CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 1UL),
kLPADC_Trigger2CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 2UL),
kLPADC_Trigger3CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 3UL),
kLPADC_Trigger4CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 4UL),
kLPADC_Trigger5CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 5UL),
kLPADC_Trigger6CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 6UL),
kLPADC_Trigger7CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 7UL),
kLPADC_Trigger8CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 8UL),
kLPADC_Trigger9CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 9UL),
kLPADC_Trigger10CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 10UL),
kLPADC_Trigger11CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 11UL),
kLPADC_Trigger12CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 12UL),
kLPADC_Trigger13CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 13UL),
kLPADC_Trigger14CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 14UL),
kLPADC_Trigger15CompletionInterruptEnable = ADC_IE_TCOMP_IE(1UL << 15UL) }

```

*Define interrupt switchers of the module.*

- `enum _lpadc_trigger_status_flags {`

```

kLPADC_Trigger0InterruptedFlag = 1UL << 0UL,
kLPADC_Trigger1InterruptedFlag = 1UL << 1UL,
kLPADC_Trigger2InterruptedFlag = 1UL << 2UL,
kLPADC_Trigger3InterruptedFlag = 1UL << 3UL,
kLPADC_Trigger4InterruptedFlag = 1UL << 4UL,
kLPADC_Trigger5InterruptedFlag = 1UL << 5UL,
kLPADC_Trigger6InterruptedFlag = 1UL << 6UL,
kLPADC_Trigger7InterruptedFlag = 1UL << 7UL,
kLPADC_Trigger8InterruptedFlag = 1UL << 8UL,
kLPADC_Trigger9InterruptedFlag = 1UL << 9UL,
kLPADC_Trigger10InterruptedFlag = 1UL << 10UL,
kLPADC_Trigger11InterruptedFlag = 1UL << 11UL,
kLPADC_Trigger12InterruptedFlag = 1UL << 12UL,
kLPADC_Trigger13InterruptedFlag = 1UL << 13UL,
kLPADC_Trigger14InterruptedFlag = 1UL << 14UL,
kLPADC_Trigger15InterruptedFlag = 1UL << 15UL,
kLPADC_Trigger0CompletedFlag = 1UL << 16UL,
kLPADC_Trigger1CompletedFlag = 1UL << 17UL,
kLPADC_Trigger2CompletedFlag = 1UL << 18UL,
kLPADC_Trigger3CompletedFlag = 1UL << 19UL,
kLPADC_Trigger4CompletedFlag = 1UL << 20UL,
kLPADC_Trigger5CompletedFlag = 1UL << 21UL,
kLPADC_Trigger6CompletedFlag = 1UL << 22UL,
kLPADC_Trigger7CompletedFlag = 1UL << 23UL,
kLPADC_Trigger8CompletedFlag = 1UL << 24UL,
kLPADC_Trigger9CompletedFlag = 1UL << 25UL,
kLPADC_Trigger10CompletedFlag = 1UL << 26UL,
kLPADC_Trigger11CompletedFlag = 1UL << 27UL,
kLPADC_Trigger12CompletedFlag = 1UL << 28UL,
kLPADC_Trigger13CompletedFlag = 1UL << 29UL,
kLPADC_Trigger14CompletedFlag = 1UL << 30UL,
kLPADC_Trigger15CompletedFlag = 1UL << 31UL }

```

*The enumerator of lpadc trigger status flags, including interrupted flags and completed flags.*

- enum `_lpadc_sample_scale_mode` {  
`kLPADC_SamplePartScale` = 0U,  
`kLPADC_SampleFullScale` = 1U }
- Define enumeration of sample scale mode.*
- enum `_lpadc_sample_channel_mode` {  
`kLPADC_SampleChannelSingleEndSideA` = 0x0U,  
`kLPADC_SampleChannelSingleEndSideB` = 0x1U,  
`kLPADC_SampleChannelDiffBothSideAB` = 0x2U,  
`kLPADC_SampleChannelDiffBothSideBA` = 0x3U }

*Define enumeration of channel sample mode.*

- enum `_lpadc_hardware_average_mode` {

```

kLPADC_HardwareAverageCount1 = 0U,
kLPADC_HardwareAverageCount2 = 1U,
kLPADC_HardwareAverageCount4 = 2U,
kLPADC_HardwareAverageCount8 = 3U,
kLPADC_HardwareAverageCount16 = 4U,
kLPADC_HardwareAverageCount32 = 5U,
kLPADC_HardwareAverageCount64 = 6U,
kLPADC_HardwareAverageCount128 = 7U }

```

*Define enumeration of hardware average selection.*

- enum `_lpadc_sample_time_mode` {  
`kLPADC_SampleTimeADCK3` = 0U,  
`kLPADC_SampleTimeADCK5` = 1U,  
`kLPADC_SampleTimeADCK7` = 2U,  
`kLPADC_SampleTimeADCK11` = 3U,  
`kLPADC_SampleTimeADCK19` = 4U,  
`kLPADC_SampleTimeADCK35` = 5U,  
`kLPADC_SampleTimeADCK67` = 6U,  
`kLPADC_SampleTimeADCK131` = 7U }

*Define enumeration of sample time selection.*

- enum `_lpadc_hardware_compare_mode` {  
`kLPADC_HardwareCompareDisabled` = 0U,  
`kLPADC_HardwareCompareStoreOnTrue` = 2U,  
`kLPADC_HardwareCompareRepeatUntilTrue` = 3U }

*Define enumeration of hardware compare mode.*

- enum `_lpadc_reference_voltage_mode` {  
`kLPADC_ReferenceVoltageAlt1` = 0U,  
`kLPADC_ReferenceVoltageAlt2` = 1U,  
`kLPADC_ReferenceVoltageAlt3` = 2U }

*Define enumeration of reference voltage source.*

- enum `_lpadc_power_level_mode` {  
`kLPADC_PowerLevelAlt1` = 0U,  
`kLPADC_PowerLevelAlt2` = 1U,  
`kLPADC_PowerLevelAlt3` = 2U,  
`kLPADC_PowerLevelAlt4` = 3U }

*Define enumeration of power configuration.*

- enum `_lpadc_trigger_priority_policy` {

```

kLPADC_ConvPreemptImmediatelyNotAutoResumed = 0x0U,
kLPADC_ConvPreemptSoftlyNotAutoResumed = 0x1U,
kLPADC_ConvPreemptImmediatelyAutoRestarted = 0x4U,
kLPADC_ConvPreemptSoftlyAutoRestarted = 0x5U,
kLPADC_ConvPreemptImmediatelyAutoResumed = 0xCU,
kLPADC_ConvPreemptSoftlyAutoResumed = 0xDU,
kLPADC_TriggerPriorityPreemptImmediately,
kLPADC_TriggerPriorityPreemptSoftly,
kLPADC_ConvPreemptSubsequentlyNotAutoResumed = 0x2U,
kLPADC_ConvPreemptSubsequentlyAutoRestarted = 0x6U,
kLPADC_ConvPreemptSubsequentlyAutoResumed = 0xEU,
kLPADC_TriggerPriorityPreemptSubsequently,
kLPADC_TriggerPriorityExceptionDisabled = 0x10U }
 Define enumeration of trigger priority policy.

```

## Driver version

- #define `FSL_LPADC_DRIVER_VERSION` (`MAKE_VERSION(2, 8, 4)`)  
LPADC driver version 2.8.4.

## Initialization & de-initialization.

- void `LPADC_Init` (`ADC_Type *base`, const `lpadc_config_t *config`)  
*Initializes the LPADC module.*
- void `LPADC_GetDefaultConfig` (`lpadc_config_t *config`)  
*Gets an available pre-defined settings for initial configuration.*
- void `LPADC_Deinit` (`ADC_Type *base`)  
*De-initializes the LPADC module.*
- static void `LPADC_Enable` (`ADC_Type *base`, bool enable)  
*Switch on/off the LPADC module.*
- static void `LPADC_DoResetFIFO` (`ADC_Type *base`)  
*Do reset the conversion FIFO.*
- static void `LPADC_DoResetConfig` (`ADC_Type *base`)  
*Do reset the module's configuration.*

## Status

- static uint32\_t `LPADC_GetStatusFlags` (`ADC_Type *base`)  
*Get status flags.*
- static void `LPADC_ClearStatusFlags` (`ADC_Type *base`, uint32\_t mask)  
*Clear status flags.*
- static uint32\_t `LPADC_GetTriggerStatusFlags` (`ADC_Type *base`)  
*Get trigger status flags to indicate which trigger sequences have been completed or interrupted by a high priority trigger exception.*
- static void `LPADC_ClearTriggerStatusFlags` (`ADC_Type *base`, uint32\_t mask)  
*Clear trigger status flags.*

## Interrupts

- static void `LPADC_EnableInterrupts` (`ADC_Type *base`, uint32\_t mask)

*Enable interrupts.*

- static void [LPADC\\_DisableInterrupts](#) (ADC\_Type \*base, uint32\_t mask)  
*Disable interrupts.*

## DMA Control

- static void [LPADC\\_EnableFIFOWatermarkDMA](#) (ADC\_Type \*base, bool enable)  
*Switch on/off the DMA trigger for FIFO watermark event.*

## Trigger and conversion with FIFO.

- static uint32\_t [LPADC\\_GetConvResultCount](#) (ADC\_Type \*base)  
*Get the count of result kept in conversion FIFO.*
- bool [LPADC\\_GetConvResult](#) (ADC\_Type \*base, [lpadc\\_conv\\_result\\_t](#) \*result)  
*Get the result in conversion FIFO.*
- void [LPADC\\_GetConvResultBlocking](#) (ADC\_Type \*base, [lpadc\\_conv\\_result\\_t](#) \*result)  
*Get the result in conversion FIFO using blocking method.*
- void [LPADC\\_SetConvTriggerConfig](#) (ADC\_Type \*base, uint32\_t triggerId, const [lpadc\\_conv\\_trigger\\_config\\_t](#) \*config)  
*Configure the conversion trigger source.*
- void [LPADC\\_GetDefaultConvTriggerConfig](#) ([lpadc\\_conv\\_trigger\\_config\\_t](#) \*config)  
*Gets an available pre-defined settings for trigger's configuration.*
- static void [LPADC\\_DoSoftwareTrigger](#) (ADC\_Type \*base, uint32\_t triggerIdMask)  
*Do software trigger to conversion command.*
- void [LPADC\\_SetConvCommandConfig](#) (ADC\_Type \*base, uint32\_t commandId, const [lpadc\\_conv\\_command\\_config\\_t](#) \*config)  
*Configure conversion command.*
- void [LPADC\\_GetDefaultConvCommandConfig](#) ([lpadc\\_conv\\_command\\_config\\_t](#) \*config)  
*Gets an available pre-defined settings for conversion command's configuration.*

## 21.3 Data Structure Documentation

### 21.3.1 struct [lpadc\\_config\\_t](#)

This structure would used to keep the settings for initialization.

### Data Fields

- bool [enableInDozeMode](#)  
*Control system transition to Stop and Wait power modes while ADC is converting.*
- bool [enableAnalogPreliminary](#)  
*ADC analog circuits are pre-enabled and ready to execute conversions without startup delays(at the cost of higher DC current consumption).*
- uint32\_t [powerUpDelay](#)  
*When the analog circuits are not pre-enabled, the ADC analog circuits are only powered while the ADC is active and there is a counted delay defined by this field after an initial trigger transitions the ADC from its Idle state to allow time for the analog circuits to stabilize.*
- [lpadc\\_reference\\_voltage\\_source\\_t](#) [referenceVoltageSource](#)

- *Selects the voltage reference high used for conversions.*  
[lpadc\\_power\\_level\\_mode\\_t powerLevelMode](#)  
*Power Configuration Selection.*
- [lpadc\\_trigger\\_priority\\_policy\\_t triggerPriorityPolicy](#)  
*Control how higher priority triggers are handled, see to lpadc\_trigger\_priority\_policy\_t.*
- [bool enableConvPause](#)  
*Enables the ADC pausing function.*
- [uint32\\_t convPauseDelay](#)  
*Controls the duration of pausing during command execution sequencing.*
- [uint32\\_t FIFOWatermark](#)  
*FIFOWatermark is a programmable threshold setting.*

## Field Documentation

### (1) **bool lpadc\_config\_t::enableInDozeMode**

When enabled in Doze mode, immediate entries to Wait or Stop are allowed. When disabled, the ADC will wait for the current averaging iteration/FIFO storage to complete before acknowledging stop or wait mode entry.

### (2) **bool lpadc\_config\_t::enableAnalogPreliminary**

### (3) **uint32\_t lpadc\_config\_t::powerUpDelay**

The startup delay count of  $(\text{powerUpDelay} * 4)$  ADCK cycles must result in a longer delay than the analog startup time.

### (4) **lpadc\_reference\_voltage\_source\_t lpadc\_config\_t::referenceVoltageSource**

### (5) **lpadc\_power\_level\_mode\_t lpadc\_config\_t::powerLevelMode**

### (6) **lpadc\_trigger\_priority\_policy\_t lpadc\_config\_t::triggerPriorityPolicy**

### (7) **bool lpadc\_config\_t::enableConvPause**

When enabled, a programmable delay is inserted during command execution sequencing between LOOP iterations, between commands in a sequence, and between conversions when command is executing in "Compare Until True" configuration.

### (8) **uint32\_t lpadc\_config\_t::convPauseDelay**

The pause delay is a count of  $(\text{convPauseDelay} * 4)$  ADCK cycles. Only available when ADC pausing function is enabled. The available value range is in 9-bit.

### (9) **uint32\_t lpadc\_config\_t::FIFOWatermark**

When the number of datawords stored in the ADC Result FIFO is greater than the value in this field, the ready flag would be asserted to indicate stored data has reached the programmable threshold.



### 21.3.2 struct lpadc\_conv\_command\_config\_t

#### Data Fields

- [lpadc\\_sample\\_scale\\_mode\\_t](#) sampleScaleMode  
*Sample scale mode.*
- [lpadc\\_sample\\_channel\\_mode\\_t](#) sampleChannelMode  
*Channel sample mode.*
- [uint32\\_t](#) channelNumber  
*Channel number, select the channel or channel pair.*
- [uint32\\_t](#) chainedNextCommandNumber  
*Selects the next command to be executed after this command completes.*
- [bool](#) enableAutoChannelIncrement  
*Loop with increment: when disabled, the "loopCount" field selects the number of times the selected channel is converted consecutively; when enabled, the "loopCount" field defines how many consecutive channels are converted as part of the command execution.*
- [uint32\\_t](#) loopCount  
*Selects how many times this command executes before finish and transition to the next command or Idle state.*
- [lpadc\\_hardware\\_average\\_mode\\_t](#) hardwareAverageMode  
*Hardware average selection.*
- [lpadc\\_sample\\_time\\_mode\\_t](#) sampleTimeMode  
*Sample time selection.*
- [lpadc\\_hardware\\_compare\\_mode\\_t](#) hardwareCompareMode  
*Hardware compare selection.*
- [uint32\\_t](#) hardwareCompareValueHigh  
*Compare Value High.*
- [uint32\\_t](#) hardwareCompareValueLow  
*Compare Value Low.*
- [bool](#) enableWaitTrigger  
*Wait for trigger assertion before execution: when disabled, this command will be automatically executed; when enabled, the active trigger must be asserted again before executing this command.*

#### Field Documentation

- (1) [lpadc\\_sample\\_scale\\_mode\\_t](#) lpadc\_conv\_command\_config\_t::sampleScaleMode
- (2) [lpadc\\_sample\\_channel\\_mode\\_t](#) lpadc\_conv\_command\_config\_t::sampleChannelMode
- (3) [uint32\\_t](#) lpadc\_conv\_command\_config\_t::channelNumber
- (4) [uint32\\_t](#) lpadc\_conv\_command\_config\_t::chainedNextCommandNumber

1-15 is available, 0 is to terminate the chain after this command.

- (5) [bool](#) lpadc\_conv\_command\_config\_t::enableAutoChannelIncrement
- (6) [uint32\\_t](#) lpadc\_conv\_command\_config\_t::loopCount

Command executes LOOP+1 times. 0-15 is available.



- (7) `lpadc_hardware_average_mode_t lpadc_conv_command_config_t::hardwareAverageMode`
- (8) `lpadc_sample_time_mode_t lpadc_conv_command_config_t::sampleTimeMode`
- (9) `lpadc_hardware_compare_mode_t lpadc_conv_command_config_t::hardwareCompareMode`
- (10) `uint32_t lpadc_conv_command_config_t::hardwareCompareValueHigh`

The available value range is in 16-bit.

- (11) `uint32_t lpadc_conv_command_config_t::hardwareCompareValueLow`

The available value range is in 16-bit.

- (12) `bool lpadc_conv_command_config_t::enableWaitTrigger`

### 21.3.3 struct `lpadc_conv_trigger_config_t`

#### Data Fields

- `uint32_t targetCommandId`  
*Select the command from command buffer to execute upon detect of the associated trigger event.*
- `uint32_t delayPower`  
*Select the trigger delay duration to wait at the start of servicing a trigger event.*
- `uint32_t priority`  
*Sets the priority of the associated trigger source.*
- `bool enableHardwareTrigger`  
*Enable hardware trigger source to initiate conversion on the rising edge of the input trigger source or not.*

#### Field Documentation

- (1) `uint32_t lpadc_conv_trigger_config_t::targetCommandId`

- (2) `uint32_t lpadc_conv_trigger_config_t::delayPower`

When this field is clear, then no delay is incurred. When this field is set to a non-zero value, the duration for the delay is  $2^{\text{delayPower}}$  ADCK cycles. The available value range is 4-bit.

- (3) `uint32_t lpadc_conv_trigger_config_t::priority`

If two or more triggers have the same priority level setting, the lower order trigger event has the higher priority. The lower value for this field is for the higher priority, the available value range is 1-bit.

- (4) `bool lpadc_conv_trigger_config_t::enableHardwareTrigger`

The software trigger is always available.

### 21.3.4 struct lpadc\_conv\_result\_t

#### Data Fields

- uint32\_t [commandIdSource](#)  
*Indicate the command buffer being executed that generated this result.*
- uint32\_t [loopCountIndex](#)  
*Indicate the loop count value during command execution that generated this result.*
- uint32\_t [triggerIdSource](#)  
*Indicate the trigger source that initiated a conversion and generated this result.*
- uint16\_t [convValue](#)  
*Data result.*

#### Field Documentation

(1) uint32\_t lpadc\_conv\_result\_t::commandIdSource

(2) uint32\_t lpadc\_conv\_result\_t::loopCountIndex

(3) uint32\_t lpadc\_conv\_result\_t::triggerIdSource

(4) uint16\_t lpadc\_conv\_result\_t::convValue

## 21.4 Macro Definition Documentation

21.4.1 #define FSL\_LPADC\_DRIVER\_VERSION (MAKE\_VERSION(2, 8, 4))

21.4.2 #define LPADC\_GET\_ACTIVE\_COMMAND\_STATUS( *statusVal* ) ((statusVal & ADC\_STAT\_CMDACT\_MASK) >> ADC\_STAT\_CMDACT\_SHIFT)

The statusVal is the return value from [LPADC\\_GetStatusFlags\(\)](#).

21.4.3 #define LPADC\_GET\_ACTIVE\_TRIGGER\_STATUE( *statusVal* ) ((statusVal & ADC\_STAT\_TRGACT\_MASK) >> ADC\_STAT\_TRGACT\_SHIFT)

The statusVal is the return value from [LPADC\\_GetStatusFlags\(\)](#).

## 21.5 Typedef Documentation

21.5.1 typedef enum \_lpadc\_sample\_scale\_mode lpadc\_sample\_scale\_mode\_t

The sample scale mode is used to reduce the selected ADC analog channel input voltage level by a factor. The maximum possible voltage on the ADC channel input should be considered when selecting a scale mode to ensure that the reducing factor always results voltage level at or below the VREFH reference. This reducing capability allows conversion of analog inputs higher than VREFH. A-side and B-side channel inputs are both scaled using the scale mode.

### 21.5.2 typedef enum `_lpadc_sample_channel_mode` `lpadc_sample_channel_mode_t`

The channel sample mode configures the channel with single-end/differential/dual-single-end, side A/B.

### 21.5.3 typedef enum `_lpadc_hardware_average_mode` `lpadc_hardware_average_mode_t`

It Selects how many ADC conversions are averaged to create the ADC result. An internal storage buffer is used to capture temporary results while the averaging iterations are executed.

Note

Some enumerator values are not available on some devices, mainly depends on the size of AVGS field in CMDH register.

### 21.5.4 typedef enum `_lpadc_sample_time_mode` `lpadc_sample_time_mode_t`

The shortest sample time maximizes conversion speed for lower impedance inputs. Extending sample time allows higher impedance inputs to be accurately sampled. Longer sample times can also be used to lower overall power consumption when command looping and sequencing is configured and high conversion rates are not required.

### 21.5.5 typedef enum `_lpadc_hardware_compare_mode` `lpadc_hardware_compare_mode_t`

After an ADC channel input is sampled and converted and any averaging iterations are performed, this mode setting guides operation of the automatic compare function to optionally only store when the compare operation is true. When compare is enabled, the conversion result is compared to the compare values.

### 21.5.6 typedef enum `_lpadc_reference_voltage_mode` `lpadc_reference_voltage_source_t`

For detail information, need to check the SoC's specification.

### 21.5.7 typedef enum `_lpadc_power_level_mode` `lpadc_power_level_mode_t`

Configures the ADC for power and performance. In the highest power setting the highest conversion rates will be possible. Refer to the device data sheet for power and performance capabilities for each setting.

### 21.5.8 typedef enum \_lpadc\_trigger\_priority\_policy lpadc\_trigger\_priority\_policy\_t

This selection controls how higher priority triggers are handled.

Note

**kLPADC\_TriggerPriorityPreemptSubsequently** is not available on some devices, mainly depends on the size of TPRICTRL field in CFG register.

## 21.6 Enumeration Type Documentation

### 21.6.1 enum \_lpadc\_status\_flags

Enumerator

**kLPADC\_ResultFIFO0OverflowFlag** Indicates that more data has been written to the Result FIFO 0 than it can hold.

**kLPADC\_ResultFIFO0ReadyFlag** Indicates when the number of valid datawords in the result FIFO 0 is greater than the setting watermark level.

**kLPADC\_TriggerExceptionFlag** Indicates that a trigger exception event has occurred.

**kLPADC\_TriggerCompletionFlag** Indicates that a trigger completion event has occurred.

**kLPADC\_ActiveFlag** Indicates that the ADC is in active state.

**kLPADC\_ResultFIFO0OverflowFlag** To compilitable with old version, do not recommend using this, please use [kLPADC\\_ResultFIFO0OverflowFlag](#) as instead.

**kLPADC\_ResultFIFOReadyFlag** To compilitable with old version, do not recommend using this, please use [kLPADC\\_ResultFIFOReadyFlag](#) as instead.

### 21.6.2 enum \_lpadc\_interrupt\_enable

Note: LPADC of different chips supports different number of trigger sources, please check the Reference Manual for details.

Enumerator

**kLPADC\_ResultFIFO0OverflowInterruptEnable** Configures ADC to generate overflow interrupt requests when FOF0 flag is asserted.

**kLPADC\_FIFO0WatermarkInterruptEnable** Configures ADC to generate watermark interrupt requests when RDY0 flag is asserted.

**kLPADC\_ResultFIFO0OverflowInterruptEnable** To compilitable with old version, do not recommend using this, please use [kLPADC\\_ResultFIFO0OverflowInterruptEnable](#) as instead.

**kLPADC\_FIFOWatermarkInterruptEnable** To compilitable with old version, do not recommend using this, please use [kLPADC\\_FIFO0WatermarkInterruptEnable](#) as instead.

**kLPADC\_TriggerExceptionInterruptEnable** Configures ADC to generate trigger exception interrupt.

|                                                         |                                                                  |
|---------------------------------------------------------|------------------------------------------------------------------|
| <b><i>kLPADC_Trigger0CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 0 completion.  |
| <b><i>kLPADC_Trigger1CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 1 completion.  |
| <b><i>kLPADC_Trigger2CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 2 completion.  |
| <b><i>kLPADC_Trigger3CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 3 completion.  |
| <b><i>kLPADC_Trigger4CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 4 completion.  |
| <b><i>kLPADC_Trigger5CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 5 completion.  |
| <b><i>kLPADC_Trigger6CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 6 completion.  |
| <b><i>kLPADC_Trigger7CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 7 completion.  |
| <b><i>kLPADC_Trigger8CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 8 completion.  |
| <b><i>kLPADC_Trigger9CompletionInterruptEnable</i></b>  | Configures ADC to generate interrupt when trigger 9 completion.  |
| <b><i>kLPADC_Trigger10CompletionInterruptEnable</i></b> | Configures ADC to generate interrupt when trigger 10 completion. |
| <b><i>kLPADC_Trigger11CompletionInterruptEnable</i></b> | Configures ADC to generate interrupt when trigger 11 completion. |
| <b><i>kLPADC_Trigger12CompletionInterruptEnable</i></b> | Configures ADC to generate interrupt when trigger 12 completion. |
| <b><i>kLPADC_Trigger13CompletionInterruptEnable</i></b> | Configures ADC to generate interrupt when trigger 13 completion. |
| <b><i>kLPADC_Trigger14CompletionInterruptEnable</i></b> | Configures ADC to generate interrupt when trigger 14 completion. |
| <b><i>kLPADC_Trigger15CompletionInterruptEnable</i></b> | Configures ADC to generate interrupt when trigger 15 completion. |

### 21.6.3 enum \_lpadc\_trigger\_status\_flags

Note: LPADC of different chips supports different number of trigger sources, please check the Reference Manual for details.

Enumerator

|                                              |                                                        |
|----------------------------------------------|--------------------------------------------------------|
| <b><i>kLPADC_Trigger0InterruptedFlag</i></b> | Trigger 0 is interrupted by a high priority exception. |
| <b><i>kLPADC_Trigger1InterruptedFlag</i></b> | Trigger 1 is interrupted by a high priority exception. |
| <b><i>kLPADC_Trigger2InterruptedFlag</i></b> | Trigger 2 is interrupted by a high priority exception. |
| <b><i>kLPADC_Trigger3InterruptedFlag</i></b> | Trigger 3 is interrupted by a high priority exception. |

|                                               |                                                                           |
|-----------------------------------------------|---------------------------------------------------------------------------|
| <b><i>kLPADC_Trigger4InterruptedFlag</i></b>  | Trigger 4 is interrupted by a high priority exception.                    |
| <b><i>kLPADC_Trigger5InterruptedFlag</i></b>  | Trigger 5 is interrupted by a high priority exception.                    |
| <b><i>kLPADC_Trigger6InterruptedFlag</i></b>  | Trigger 6 is interrupted by a high priority exception.                    |
| <b><i>kLPADC_Trigger7InterruptedFlag</i></b>  | Trigger 7 is interrupted by a high priority exception.                    |
| <b><i>kLPADC_Trigger8InterruptedFlag</i></b>  | Trigger 8 is interrupted by a high priority exception.                    |
| <b><i>kLPADC_Trigger9InterruptedFlag</i></b>  | Trigger 9 is interrupted by a high priority exception.                    |
| <b><i>kLPADC_Trigger10InterruptedFlag</i></b> | Trigger 10 is interrupted by a high priority exception.                   |
| <b><i>kLPADC_Trigger11InterruptedFlag</i></b> | Trigger 11 is interrupted by a high priority exception.                   |
| <b><i>kLPADC_Trigger12InterruptedFlag</i></b> | Trigger 12 is interrupted by a high priority exception.                   |
| <b><i>kLPADC_Trigger13InterruptedFlag</i></b> | Trigger 13 is interrupted by a high priority exception.                   |
| <b><i>kLPADC_Trigger14InterruptedFlag</i></b> | Trigger 14 is interrupted by a high priority exception.                   |
| <b><i>kLPADC_Trigger15InterruptedFlag</i></b> | Trigger 15 is interrupted by a high priority exception.                   |
| <b><i>kLPADC_Trigger0CompletedFlag</i></b>    | Trigger 0 is completed and trigger 0 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger1CompletedFlag</i></b>    | Trigger 1 is completed and trigger 1 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger2CompletedFlag</i></b>    | Trigger 2 is completed and trigger 2 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger3CompletedFlag</i></b>    | Trigger 3 is completed and trigger 3 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger4CompletedFlag</i></b>    | Trigger 4 is completed and trigger 4 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger5CompletedFlag</i></b>    | Trigger 5 is completed and trigger 5 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger6CompletedFlag</i></b>    | Trigger 6 is completed and trigger 6 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger7CompletedFlag</i></b>    | Trigger 7 is completed and trigger 7 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger8CompletedFlag</i></b>    | Trigger 8 is completed and trigger 8 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger9CompletedFlag</i></b>    | Trigger 9 is completed and trigger 9 has enabled completion interrupts.   |
| <b><i>kLPADC_Trigger10CompletedFlag</i></b>   | Trigger 10 is completed and trigger 10 has enabled completion interrupts. |
| <b><i>kLPADC_Trigger11CompletedFlag</i></b>   | Trigger 11 is completed and trigger 11 has enabled completion interrupts. |
| <b><i>kLPADC_Trigger12CompletedFlag</i></b>   | Trigger 12 is completed and trigger 12 has enabled completion interrupts. |
| <b><i>kLPADC_Trigger13CompletedFlag</i></b>   | Trigger 13 is completed and trigger 13 has enabled completion interrupts. |
| <b><i>kLPADC_Trigger14CompletedFlag</i></b>   | Trigger 14 is completed and trigger 14 has enabled completion interrupts. |
| <b><i>kLPADC_Trigger15CompletedFlag</i></b>   | Trigger 15 is completed and trigger 15 has enabled completion interrupts. |

### 21.6.4 enum \_lpadc\_sample\_scale\_mode

The sample scale mode is used to reduce the selected ADC analog channel input voltage level by a factor. The maximum possible voltage on the ADC channel input should be considered when selecting a scale mode to ensure that the reducing factor always results voltage level at or below the VREFH reference. This reducing capability allows conversion of analog inputs higher than VREFH. A-side and B-side channel inputs are both scaled using the scale mode.

Enumerator

- kLPADC\_SamplePartScale*** Use divided input voltage signal. (For scale select, please refer to the reference manual).  
***kLPADC\_SampleFullScale*** Full scale (Factor of 1).

### 21.6.5 enum \_lpadc\_sample\_channel\_mode

The channel sample mode configures the channel with single-end/differential/dual-single-end, side A/B.

Enumerator

- kLPADC\_SampleChannelSingleEndSideA*** Single-end mode, only A-side channel is converted.  
***kLPADC\_SampleChannelSingleEndSideB*** Single-end mode, only B-side channel is converted.  
***kLPADC\_SampleChannelDiffBothSideAB*** Differential mode, the ADC result is (CHnA-CHnB).  
***kLPADC\_SampleChannelDiffBothSideBA*** Differential mode, the ADC result is (CHnB-CHnA).

### 21.6.6 enum \_lpadc\_hardware\_average\_mode

It Selects how many ADC conversions are averaged to create the ADC result. An internal storage buffer is used to capture temporary results while the averaging iterations are executed.

Note

Some enumerator values are not available on some devices, mainly depends on the size of AVGS field in CMDH register.

Enumerator

- kLPADC\_HardwareAverageCount1*** Single conversion.  
***kLPADC\_HardwareAverageCount2*** 2 conversions averaged.  
***kLPADC\_HardwareAverageCount4*** 4 conversions averaged.  
***kLPADC\_HardwareAverageCount8*** 8 conversions averaged.  
***kLPADC\_HardwareAverageCount16*** 16 conversions averaged.  
***kLPADC\_HardwareAverageCount32*** 32 conversions averaged.  
***kLPADC\_HardwareAverageCount64*** 64 conversions averaged.  
***kLPADC\_HardwareAverageCount128*** 128 conversions averaged.

### 21.6.7 enum \_lpadc\_sample\_time\_mode

The shortest sample time maximizes conversion speed for lower impedance inputs. Extending sample time allows higher impedance inputs to be accurately sampled. Longer sample times can also be used to lower overall power consumption when command looping and sequencing is configured and high conversion rates are not required.

Enumerator

*kLPADC\_SampleTimeADCK3* 3 ADCK cycles total sample time.  
*kLPADC\_SampleTimeADCK5* 5 ADCK cycles total sample time.  
*kLPADC\_SampleTimeADCK7* 7 ADCK cycles total sample time.  
*kLPADC\_SampleTimeADCK11* 11 ADCK cycles total sample time.  
*kLPADC\_SampleTimeADCK19* 19 ADCK cycles total sample time.  
*kLPADC\_SampleTimeADCK35* 35 ADCK cycles total sample time.  
*kLPADC\_SampleTimeADCK67* 69 ADCK cycles total sample time.  
*kLPADC\_SampleTimeADCK131* 131 ADCK cycles total sample time.

### 21.6.8 enum \_lpadc\_hardware\_compare\_mode

After an ADC channel input is sampled and converted and any averaging iterations are performed, this mode setting guides operation of the automatic compare function to optionally only store when the compare operation is true. When compare is enabled, the conversion result is compared to the compare values.

Enumerator

*kLPADC\_HardwareCompareDisabled* Compare disabled.  
*kLPADC\_HardwareCompareStoreOnTrue* Compare enabled. Store on true.  
*kLPADC\_HardwareCompareRepeatUntilTrue* Compare enabled. Repeat channel acquisition until true.

### 21.6.9 enum \_lpadc\_reference\_voltage\_mode

For detail information, need to check the SoC's specification.

Enumerator

*kLPADC\_ReferenceVoltageAlt1* Option 1 setting.  
*kLPADC\_ReferenceVoltageAlt2* Option 2 setting.  
*kLPADC\_ReferenceVoltageAlt3* Option 3 setting.



## 21.6.10 enum \_lpadc\_power\_level\_mode

Configures the ADC for power and performance. In the highest power setting the highest conversion rates will be possible. Refer to the device data sheet for power and performance capabilities for each setting.

Enumerator

***kLPADC\_PowerLevelAlt1*** Lowest power setting.  
***kLPADC\_PowerLevelAlt2*** Next lowest power setting.  
***kLPADC\_PowerLevelAlt3*** ...  
***kLPADC\_PowerLevelAlt4*** Highest power setting.

## 21.6.11 enum \_lpadc\_trigger\_priority\_policy

This selection controls how higher priority triggers are handled.

Note

***kLPADC\_TriggerPriorityPreemptSubsequently*** is not available on some devices, mainly depends on the size of TPRICTRL field in CFG register.

Enumerator

***kLPADC\_ConvPreemptImmediatelyNotAutoResumed*** If a higher priority trigger is detected during command processing, the current conversion is aborted and the new command specified by the trigger is started, when higher priority conversion finishes, the preempted conversion is not automatically resumed or restarted.

***kLPADC\_ConvPreemptSoftlyNotAutoResumed*** If a higher priority trigger is received during command processing, the current conversion is completed (including averaging iterations and compare function if enabled) and stored to the result FIFO before the higher priority trigger/command is initiated, when higher priority conversion finishes, the preempted conversion is not resumed or restarted.

***kLPADC\_ConvPreemptImmediatelyAutoRestarted*** If a higher priority trigger is detected during command processing, the current conversion is aborted and the new command specified by the trigger is started, when higher priority conversion finishes, the preempted conversion will automatically be restarted.

***kLPADC\_ConvPreemptSoftlyAutoRestarted*** If a higher priority trigger is received during command processing, the current conversion is completed (including averaging iterations and compare function if enabled) and stored to the result FIFO before the higher priority trigger/command is initiated, when higher priority conversion finishes, the preempted conversion will automatically be restarted.

***kLPADC\_ConvPreemptImmediatelyAutoResumed*** If a higher priority trigger is detected during command processing, the current conversion is aborted and the new command specified by the trigger is started, when higher priority conversion finishes, the preempted conversion will automatically be resumed.

***kLPADC\_ConvPreemptSoftlyAutoResumed*** If a higher priority trigger is received during command processing, the current conversion is completed (including averaging iterations and compare function if enabled) and stored to the result FIFO before the higher priority trigger/command is initiated, when higher priority conversion finishes, the preempted conversion will be automatically be resumed.

***kLPADC\_TriggerPriorityPreemptImmediately*** Legacy support is not recommended as it only ensures compatibility with older versions.

***kLPADC\_TriggerPriorityPreemptSoftly*** Legacy support is not recommended as it only ensures compatibility with older versions.

***kLPADC\_ConvPreemptSubsequentlyNotAutoResumed*** If a higher priority trigger is received during command processing, the current command will be completed (averaging, looping, compare) before servicing the higher priority trigger, when higher priority conversion finishes, the preempted conversion will not automatically be restarted or resumed.

***kLPADC\_ConvPreemptSubsequentlyAutoRestarted*** If a higher priority trigger is received during command processing, the current command will be completed (averaging, looping, compare) before servicing the higher priority trigger, when higher priority conversion finishes, the preempted conversion will be automatically restarted.

***kLPADC\_ConvPreemptSubsequentlyAutoResumed*** If a higher priority trigger is received during command processing, the current command will be completed (averaging, looping, compare) before servicing the higher priority trigger, when higher priority conversion finishes, the preempted conversion will be automatically resumed.

***kLPADC\_TriggerPriorityPreemptSubsequently*** Legacy support is not recommended as it only ensures compatibility with older versions.

***kLPADC\_TriggerPriorityExceptionDisabled*** High priority trigger exception disabled.

## 21.7 Function Documentation

### 21.7.1 void LPADC\_Init ( ADC\_Type \* *base*, const lpadc\_config\_t \* *config* )

Parameters

|               |                                                           |
|---------------|-----------------------------------------------------------|
| <i>base</i>   | LPADC peripheral base address.                            |
| <i>config</i> | Pointer to configuration structure. See "lpadc_config_t". |

### 21.7.2 void LPADC\_GetDefaultConfig ( lpadc\_config\_t \* *config* )

This function initializes the converter configuration structure with an available settings. The default values are:

```
* config->enableInDozeMode = true;
* config->enableAnalogPreliminary = false;
* config->powerUpDelay = 0x80;
* config->referenceVoltageSource = kLPADC_ReferenceVoltageAlt1;
* config->powerLevelMode = kLPADC_PowerLevelAlt1;
```

```

* config->triggerPriorityPolicy = kLPADC_TriggerPriorityPreemptImmediately
* ;
* config->enableConvPause = false;
* config->convPauseDelay = 0U;
* config->FIFOWatermark = 0U;
*

```

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>config</i> | Pointer to configuration structure. |
|---------------|-------------------------------------|

### 21.7.3 void LPADC\_Deinit ( ADC\_Type \* *base* )

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPADC peripheral base address. |
|-------------|--------------------------------|

### 21.7.4 static void LPADC\_Enable ( ADC\_Type \* *base*, bool *enable* ) [inline], [static]

## Parameters

|               |                                |
|---------------|--------------------------------|
| <i>base</i>   | LPADC peripheral base address. |
| <i>enable</i> | switcher to the module.        |

### 21.7.5 static void LPADC\_DoResetFIFO ( ADC\_Type \* *base* ) [inline], [static]

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPADC peripheral base address. |
|-------------|--------------------------------|

### 21.7.6 static void LPADC\_DoResetConfig ( ADC\_Type \* *base* ) [inline], [static]

Reset all ADC internal logic and registers, except the Control Register (ADCx\_CTRL).

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPADC peripheral base address. |
|-------------|--------------------------------|

**21.7.7 static uint32\_t LPADC\_GetStatusFlags ( ADC\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPADC peripheral base address. |
|-------------|--------------------------------|

## Returns

status flags' mask. See to [\\_lpadc\\_status\\_flags](#).

**21.7.8 static void LPADC\_ClearStatusFlags ( ADC\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**

Only the flags can be cleared by writing ADCx\_STATUS register would be cleared by this API.

## Parameters

|             |                                                                                  |
|-------------|----------------------------------------------------------------------------------|
| <i>base</i> | LPADC peripheral base address.                                                   |
| <i>mask</i> | Mask value for flags to be cleared. See to <a href="#">_lpadc_status_flags</a> . |

**21.7.9 static uint32\_t LPADC\_GetTriggerStatusFlags ( ADC\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPADC peripheral base address. |
|-------------|--------------------------------|

## Returns

The OR'ed value of [\\_lpadc\\_trigger\\_status\\_flags](#).

**21.7.10 static void LPADC\_ClearTriggerStatusFlags ( ADC\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**

## Parameters

|             |                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | LPADC peripheral base address.                                                                                             |
| <i>mask</i> | The mask of trigger status flags to be cleared, should be the OR'ed value of <a href="#">_lpadc_trigger_status_flags</a> . |

**21.7.11 static void LPADC\_EnableInterrupts ( ADC\_Type \* *base*, uint32\_t *mask* )**  
**[inline], [static]**

## Parameters

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| <i>base</i> | LPADC peripheral base address.                                                    |
| <i>mask</i> | Mask value for interrupt events. See to <a href="#">_lpadc_interrupt_enable</a> . |

**21.7.12 static void LPADC\_DisableInterrupts ( ADC\_Type \* *base*, uint32\_t *mask* )**  
**[inline], [static]**

## Parameters

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| <i>base</i> | LPADC peripheral base address.                                                    |
| <i>mask</i> | Mask value for interrupt events. See to <a href="#">_lpadc_interrupt_enable</a> . |

**21.7.13 static void LPADC\_EnableFIFOWatermarkDMA ( ADC\_Type \* *base*, bool *enable* )**  
**[inline], [static]**

## Parameters

|               |                                |
|---------------|--------------------------------|
| <i>base</i>   | LPADC peripheral base address. |
| <i>enable</i> | Switcher to the event.         |

**21.7.14 static uint32\_t LPADC\_GetConvResultCount ( ADC\_Type \* *base* )**  
**[inline], [static]**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPADC peripheral base address. |
|-------------|--------------------------------|

## Returns

The count of result kept in conversion FIFO.

### 21.7.15 **bool LPADC\_GetConvResult ( ADC\_Type \* *base*, lpadc\_conv\_result\_t \* *result* )**

## Parameters

|               |                                                                                    |
|---------------|------------------------------------------------------------------------------------|
| <i>base</i>   | LPADC peripheral base address.                                                     |
| <i>result</i> | Pointer to structure variable that keeps the conversion result in conversion FIFO. |

## Returns

Status whether FIFO entry is valid.

### 21.7.16 **void LPADC\_GetConvResultBlocking ( ADC\_Type \* *base*, lpadc\_conv\_result\_t \* *result* )**

## Parameters

|               |                                                                                    |
|---------------|------------------------------------------------------------------------------------|
| <i>base</i>   | LPADC peripheral base address.                                                     |
| <i>result</i> | Pointer to structure variable that keeps the conversion result in conversion FIFO. |

### 21.7.17 **void LPADC\_SetConvTriggerConfig ( ADC\_Type \* *base*, uint32\_t *triggerId*, const lpadc\_conv\_trigger\_config\_t \* *config* )**

Each programmable trigger can launch the conversion command in command buffer.

## Parameters

|                  |                                                                                          |
|------------------|------------------------------------------------------------------------------------------|
| <i>base</i>      | LPADC peripheral base address.                                                           |
| <i>triggerId</i> | ID for each trigger. Typically, the available value range is from 0.                     |
| <i>config</i>    | Pointer to configuration structure. See to <a href="#">lpadc_conv_trigger_config_t</a> . |

### 21.7.18 void LPADC\_GetDefaultConvTriggerConfig ( lpadc\_conv\_trigger\_config\_t \* *config* )

This function initializes the trigger's configuration structure with an available settings. The default values are:

```
* config->targetCommandId = 0U;
* config->delayPower = 0U;
* config->priority = 0U;
* config->channelAFIFOSelect = 0U;
* config->channelBFIFOSelect = 0U;
* config->enableHardwareTrigger = false;
*
```

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>config</i> | Pointer to configuration structure. |
|---------------|-------------------------------------|

### 21.7.19 static void LPADC\_DoSoftwareTrigger ( ADC\_Type \* *base*, uint32\_t *triggerIdMask* ) [inline], [static]

## Parameters

|                      |                                                                 |
|----------------------|-----------------------------------------------------------------|
| <i>base</i>          | LPADC peripheral base address.                                  |
| <i>triggerIdMask</i> | Mask value for software trigger indexes, which count from zero. |

### 21.7.20 void LPADC\_SetConvCommandConfig ( ADC\_Type \* *base*, uint32\_t *commandId*, const lpadc\_conv\_command\_config\_t \* *config* )

## Note

The number of compare value register on different chips is different, that is mean in some chips, some command buffers do not have the compare functionality.

## Parameters

|                  |                                                                                          |
|------------------|------------------------------------------------------------------------------------------|
| <i>base</i>      | LPADC peripheral base address.                                                           |
| <i>commandId</i> | ID for command in command buffer. Typically, the available value range is 1 - 15.        |
| <i>config</i>    | Pointer to configuration structure. See to <a href="#">lpadc_conv_command_config_t</a> . |

### 21.7.21 void LPADC\_GetDefaultConvCommandConfig ( lpadc\_conv\_command\_config\_t \* config )

This function initializes the conversion command's configuration structure with an available settings. The default values are:

```

* config->sampleScaleMode = kLPADC_SampleFullScale;
* config->channelBScaleMode = kLPADC_SampleFullScale;
* config->sampleChannelMode = kLPADC_SampleChannelSingleEndSideA
*
* config->channelNumber = 0U;
* config->channelBNumber = 0U;
* config->chainedNextCommandNumber = 0U;
* config->enableAutoChannelIncrement = false;
* config->loopCount = 0U;
* config->hardwareAverageMode = kLPADC_HardwareAverageCount1;
* config->sampleTimeMode = kLPADC_SampleTimeADCK3;
* config->hardwareCompareMode = kLPADC_HardwareCompareDisabled;
* config->hardwareCompareValueHigh = 0U;
* config->hardwareCompareValueLow = 0U;
* config->conversionResolutionMode = kLPADC_ConversionResolutionStandard;
* config->enableWaitTrigger = false;
* config->enableChannelB = false;
*

```

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>config</i> | Pointer to configuration structure. |
|---------------|-------------------------------------|



## Chapter 22

# LPI2C: Low Power Inter-Integrated Circuit Driver

## 22.1 Overview

### Modules

- [LPI2C CMSIS Driver](#)
- [LPI2C FreeRTOS Driver](#)
- [LPI2C Master DMA Driver](#)
- [LPI2C Master Driver](#)
- [LPI2C Slave Driver](#)

### Macros

- `#define I2C\_RETRY\_TIMES 0U` /\* Define to zero means keep waiting until the flag is assert/deassert. \*/  
*Retry times for waiting flag.*

### Enumerations

- enum {  
    [kStatus\\_LPI2C\\_Busy](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 0),  
    [kStatus\\_LPI2C\\_Idle](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 1),  
    [kStatus\\_LPI2C\\_Nak](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 2),  
    [kStatus\\_LPI2C\\_FifoError](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 3),  
    [kStatus\\_LPI2C\\_BitError](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 4),  
    [kStatus\\_LPI2C\\_ArbitrationLost](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 5),  
    [kStatus\\_LPI2C\\_PinLowTimeout](#),  
    [kStatus\\_LPI2C\\_NoTransferInProgress](#),  
    [kStatus\\_LPI2C\\_DmaRequestFail](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 8),  
    [kStatus\\_LPI2C\\_Timeout](#) = MAKE\_STATUS(kStatusGroup\_LPI2C, 9) }  
*LPI2C status return codes.*

### Functions

- `uint32_t LPI2C\_GetInstance (LPI2C_Type *base)`  
*Returns an instance number given a base address.*

### Variables

- `IRQn_Type const kLpi2cIrqs []`  
*Array to map LPI2C instance number to IRQ number, used internally for LPI2C master int APIs.*
- `lpi2c\_master\_isr\_t s_lpi2cMasterIsr`

Pointer to master IRQ handler for each instance, used internally for LPI2C master interrupt APIs.

- void \* [s\\_lpi2cMasterHandle](#) []

Pointers to master handles for each instance, used internally for LPI2C master interrupt APIs.

## Driver version

- #define [FSL\\_LPI2C\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 5, 4))  
LPI2C driver version.

## 22.2 Macro Definition Documentation

### 22.2.1 #define FSL\_LPI2C\_DRIVER\_VERSION (MAKE\_VERSION(2, 5, 4))

### 22.2.2 #define I2C\_RETRY\_TIMES 0U /\* Define to zero means keep waiting until the flag is assert/deassert. \*/

## 22.3 Enumeration Type Documentation

### 22.3.1 anonymous enum

Enumerator

*kStatus\_LPI2C\_Busy* The master is already performing a transfer.

*kStatus\_LPI2C\_Idle* The slave driver is idle.

*kStatus\_LPI2C\_Nak* The slave device sent a NAK in response to a byte.

*kStatus\_LPI2C\_FifoError* FIFO under run or overrun.

*kStatus\_LPI2C\_BitError* Transferred bit was not seen on the bus.

*kStatus\_LPI2C\_ArbitrationLost* Arbitration lost error.

*kStatus\_LPI2C\_PinLowTimeout* SCL or SDA were held low longer than the timeout.

*kStatus\_LPI2C\_NoTransferInProgress* Attempt to abort a transfer when one is not in progress.

*kStatus\_LPI2C\_DmaRequestFail* DMA request failed.

*kStatus\_LPI2C\_Timeout* Timeout polling status flags.

## 22.4 Function Documentation

### 22.4.1 uint32\_t LPI2C\_GetInstance ( LPI2C\_Type \* *base* )

If an invalid base address is passed, debug builds will assert. Release builds will just return instance number 0.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Returns

LPI2C instance number starting from 0.

## 22.5 Variable Documentation

### 22.5.1 IRQn\_Type const kLpi2clrqs[]

### 22.5.2 lpi2c\_master\_isr\_t s\_lpi2cMasterIsr

### 22.5.3 void\* s\_lpi2cMasterHandle[]

## 22.6 LPI2C Master Driver

### 22.6.1 Overview

#### Data Structures

- struct [\\_lpi2c\\_master\\_config](#)  
*Structure with settings to initialize the LPI2C master module. [More...](#)*
- struct [\\_lpi2c\\_match\\_config](#)  
*LPI2C master data match configuration structure. [More...](#)*
- struct [\\_lpi2c\\_master\\_transfer](#)  
*Non-blocking transfer descriptor structure. [More...](#)*
- struct [\\_lpi2c\\_master\\_handle](#)  
*Driver handle for master non-blocking APIs. [More...](#)*

#### Typedefs

- typedef enum [\\_lpi2c\\_direction](#) [lpi2c\\_direction\\_t](#)  
*Direction of master and slave transfers.*
- typedef enum [\\_lpi2c\\_master\\_pin\\_config](#) [lpi2c\\_master\\_pin\\_config\\_t](#)  
*LPI2C pin configuration.*
- typedef enum [\\_lpi2c\\_host\\_request\\_source](#) [lpi2c\\_host\\_request\\_source\\_t](#)  
*LPI2C master host request selection.*
- typedef enum [\\_lpi2c\\_host\\_request\\_polarity](#) [lpi2c\\_host\\_request\\_polarity\\_t](#)  
*LPI2C master host request pin polarity configuration.*
- typedef struct [\\_lpi2c\\_master\\_config](#) [lpi2c\\_master\\_config\\_t](#)  
*Structure with settings to initialize the LPI2C master module.*
- typedef enum [\\_lpi2c\\_data\\_match\\_config\\_mode](#) [lpi2c\\_data\\_match\\_config\\_mode\\_t](#)  
*LPI2C master data match configuration modes.*
- typedef struct [\\_lpi2c\\_match\\_config](#) [lpi2c\\_data\\_match\\_config\\_t](#)  
*LPI2C master data match configuration structure.*
- typedef struct [\\_lpi2c\\_master\\_transfer](#) [lpi2c\\_master\\_transfer\\_t](#)  
*LPI2C master descriptor of the transfer.*
- typedef struct [\\_lpi2c\\_master\\_handle](#) [lpi2c\\_master\\_handle\\_t](#)  
*LPI2C master handle of the transfer.*
- typedef void(\* [lpi2c\\_master\\_transfer\\_callback\\_t](#) )(LPI2C\_Type \*base, [lpi2c\\_master\\_handle\\_t](#) \*handle, [status\\_t](#) completionStatus, void \*userData)  
*Master completion callback function pointer type.*
- typedef void(\* [lpi2c\\_master\\_isr\\_t](#) )(LPI2C\_Type \*base, void \*handle)  
*Typedef for master interrupt handler, used internally for LPI2C master interrupt and EDMA transactional APIs.*

## Enumerations

- enum `_lpi2c_master_flags` {  
`kLPI2C_MasterTxReadyFlag` = `LPI2C_MSR_TDF_MASK`,  
`kLPI2C_MasterRxReadyFlag` = `LPI2C_MSR_RDF_MASK`,  
`kLPI2C_MasterEndOfPacketFlag` = `LPI2C_MSR_EPF_MASK`,  
`kLPI2C_MasterStopDetectFlag` = `LPI2C_MSR_SDF_MASK`,  
`kLPI2C_MasterNackDetectFlag` = `LPI2C_MSR_NDF_MASK`,  
`kLPI2C_MasterArbitrationLostFlag` = `LPI2C_MSR_ALF_MASK`,  
`kLPI2C_MasterFifoErrFlag` = `LPI2C_MSR_FEF_MASK`,  
`kLPI2C_MasterPinLowTimeoutFlag` = `LPI2C_MSR_PLTF_MASK`,  
`kLPI2C_MasterDataMatchFlag` = `LPI2C_MSR_DMF_MASK`,  
`kLPI2C_MasterBusyFlag` = `LPI2C_MSR_MBF_MASK`,  
`kLPI2C_MasterBusBusyFlag` = `LPI2C_MSR_BBF_MASK`,  
`kLPI2C_MasterClearFlags`,  
`kLPI2C_MasterIrqFlags`,  
`kLPI2C_MasterErrorFlags` }  
*LPI2C master peripheral flags.*
- enum `_lpi2c_direction` {  
`kLPI2C_Write` = 0U,  
`kLPI2C_Read` = 1U }  
*Direction of master and slave transfers.*
- enum `_lpi2c_master_pin_config` {  
`kLPI2C_2PinOpenDrain` = 0x0U,  
`kLPI2C_2PinOutputOnly` = 0x1U,  
`kLPI2C_2PinPushPull` = 0x2U,  
`kLPI2C_4PinPushPull` = 0x3U,  
`kLPI2C_2PinOpenDrainWithSeparateSlave`,  
`kLPI2C_2PinOutputOnlyWithSeparateSlave`,  
`kLPI2C_2PinPushPullWithSeparateSlave`,  
`kLPI2C_4PinPushPullWithInvertedOutput` = 0x7U }  
*LPI2C pin configuration.*
- enum `_lpi2c_host_request_source` {  
`kLPI2C_HostRequestExternalPin` = 0x0U,  
`kLPI2C_HostRequestInputTrigger` = 0x1U }  
*LPI2C master host request selection.*
- enum `_lpi2c_host_request_polarity` {  
`kLPI2C_HostRequestPinActiveLow` = 0x0U,  
`kLPI2C_HostRequestPinActiveHigh` = 0x1U }  
*LPI2C master host request pin polarity configuration.*
- enum `_lpi2c_data_match_config_mode` {

```

kLPI2C_MatchDisabled = 0x0U,
kLPI2C_1stWordEqualsM0OrM1 = 0x2U,
kLPI2C_AnyWordEqualsM0OrM1 = 0x3U,
kLPI2C_1stWordEqualsM0And2ndWordEqualsM1,
kLPI2C_AnyWordEqualsM0AndNextWordEqualsM1,
kLPI2C_1stWordAndM1EqualsM0AndM1,
kLPI2C_AnyWordAndM1EqualsM0AndM1 }

```

*LPI2C master data match configuration modes.*

- enum `_lpi2c_master_transfer_flags` {  
`kLPI2C_TransferDefaultFlag` = 0x00U,  
`kLPI2C_TransferNoStartFlag` = 0x01U,  
`kLPI2C_TransferRepeatedStartFlag` = 0x02U,  
`kLPI2C_TransferNoStopFlag` = 0x04U }

*Transfer option flags.*

## Initialization and deinitialization

- void `LPI2C_MasterGetDefaultConfig` (`lpi2c_master_config_t` \*masterConfig)  
*Provides a default configuration for the LPI2C master peripheral.*
- void `LPI2C_MasterInit` (`LPI2C_Type` \*base, const `lpi2c_master_config_t` \*masterConfig, `uint32_t` sourceClock\_Hz)  
*Initializes the LPI2C master peripheral.*
- void `LPI2C_MasterDeinit` (`LPI2C_Type` \*base)  
*Deinitializes the LPI2C master peripheral.*
- void `LPI2C_MasterConfigureDataMatch` (`LPI2C_Type` \*base, const `lpi2c_data_match_config_t` \*matchConfig)  
*Configures LPI2C master data match feature.*
- `status_t` `LPI2C_MasterCheckAndClearError` (`LPI2C_Type` \*base, `uint32_t` status)  
*Convert provided flags to status code, and clear any errors if present.*
- `status_t` `LPI2C_CheckForBusyBus` (`LPI2C_Type` \*base)  
*Make sure the bus isn't already busy.*
- static void `LPI2C_MasterReset` (`LPI2C_Type` \*base)  
*Performs a software reset.*
- static void `LPI2C_MasterEnable` (`LPI2C_Type` \*base, bool enable)  
*Enables or disables the LPI2C module as master.*

## Status

- static `uint32_t` `LPI2C_MasterGetStatusFlags` (`LPI2C_Type` \*base)  
*Gets the LPI2C master status flags.*
- static void `LPI2C_MasterClearStatusFlags` (`LPI2C_Type` \*base, `uint32_t` statusMask)  
*Clears the LPI2C master status flag state.*

## Interrupts

- static void [LPI2C\\_MasterEnableInterrupts](#) (LPI2C\_Type \*base, uint32\_t interruptMask)  
*Enables the LPI2C master interrupt requests.*
- static void [LPI2C\\_MasterDisableInterrupts](#) (LPI2C\_Type \*base, uint32\_t interruptMask)  
*Disables the LPI2C master interrupt requests.*
- static uint32\_t [LPI2C\\_MasterGetEnabledInterrupts](#) (LPI2C\_Type \*base)  
*Returns the set of currently enabled LPI2C master interrupt requests.*

## DMA control

- static void [LPI2C\\_MasterEnableDMA](#) (LPI2C\_Type \*base, bool enableTx, bool enableRx)  
*Enables or disables LPI2C master DMA requests.*
- static uint32\_t [LPI2C\\_MasterGetTxFifoAddress](#) (LPI2C\_Type \*base)  
*Gets LPI2C master transmit data register address for DMA transfer.*
- static uint32\_t [LPI2C\\_MasterGetRxFifoAddress](#) (LPI2C\_Type \*base)  
*Gets LPI2C master receive data register address for DMA transfer.*

## FIFO control

- static void [LPI2C\\_MasterSetWatermarks](#) (LPI2C\_Type \*base, size\_t txWords, size\_t rxWords)  
*Sets the watermarks for LPI2C master FIFOs.*
- static void [LPI2C\\_MasterGetFifoCounts](#) (LPI2C\_Type \*base, size\_t \*rxCount, size\_t \*txCount)  
*Gets the current number of words in the LPI2C master FIFOs.*

## Bus operations

- void [LPI2C\\_MasterSetBaudRate](#) (LPI2C\_Type \*base, uint32\_t sourceClock\_Hz, uint32\_t baudRate\_Hz)  
*Sets the I2C bus frequency for master transactions.*
- static bool [LPI2C\\_MasterGetBusIdleState](#) (LPI2C\_Type \*base)  
*Returns whether the bus is idle.*
- [status\\_t](#) [LPI2C\\_MasterStart](#) (LPI2C\_Type \*base, uint8\_t address, [lpi2c\\_direction\\_t](#) dir)  
*Sends a START signal and slave address on the I2C bus.*
- [status\\_t](#) [LPI2C\\_MasterRepeatedStart](#) (LPI2C\_Type \*base, uint8\_t address, [lpi2c\\_direction\\_t](#) dir)  
*Sends a repeated START signal and slave address on the I2C bus.*
- [status\\_t](#) [LPI2C\\_MasterSend](#) (LPI2C\_Type \*base, void \*txBuff, size\_t txSize)  
*Performs a polling send transfer on the I2C bus.*
- [status\\_t](#) [LPI2C\\_MasterReceive](#) (LPI2C\_Type \*base, void \*rxBuff, size\_t rxSize)  
*Performs a polling receive transfer on the I2C bus.*
- [status\\_t](#) [LPI2C\\_MasterStop](#) (LPI2C\_Type \*base)  
*Sends a STOP signal on the I2C bus.*
- [status\\_t](#) [LPI2C\\_MasterTransferBlocking](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_transfer\\_t](#) \*transfer)  
*Performs a master polling transfer on the I2C bus.*

## Non-blocking

- void [LPI2C\\_MasterTransferCreateHandle](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_handle\\_t](#) \*handle, [lpi2c\\_master\\_transfer\\_callback\\_t](#) callback, void \*userData)  
*Creates a new handle for the LPI2C master non-blocking APIs.*
- [status\\_t LPI2C\\_MasterTransferNonBlocking](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_handle\\_t](#) \*handle, [lpi2c\\_master\\_transfer\\_t](#) \*transfer)  
*Performs a non-blocking transaction on the I2C bus.*
- [status\\_t LPI2C\\_MasterTransferGetCount](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_handle\\_t](#) \*handle, [size\\_t](#) \*count)  
*Returns number of bytes transferred so far.*
- void [LPI2C\\_MasterTransferAbort](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_handle\\_t](#) \*handle)  
*Terminates a non-blocking LPI2C master transmission early.*

## IRQ handler

- void [LPI2C\\_MasterTransferHandleIRQ](#) (LPI2C\_Type \*base, void \*lpi2cMasterHandle)  
*Reusable routine to handle master interrupts.*

## 22.6.2 Data Structure Documentation

### 22.6.2.1 struct [\\_lpi2c\\_master\\_config](#)

This structure holds configuration settings for the LPI2C peripheral. To initialize this structure to reasonable defaults, call the [LPI2C\\_MasterGetDefaultConfig\(\)](#) function and pass a pointer to your configuration structure instance.

The configuration structure can be made constant so it resides in flash.

### Data Fields

- bool [enableMaster](#)  
*Whether to enable master mode.*
- bool [enableDoze](#)  
*Whether master is enabled in doze mode.*
- bool [debugEnable](#)  
*Enable transfers to continue when halted in debug mode.*
- bool [ignoreAck](#)  
*Whether to ignore ACK/NACK.*
- [lpi2c\\_master\\_pin\\_config\\_t](#) [pinConfig](#)  
*The pin configuration option.*
- [uint32\\_t](#) [baudRate\\_Hz](#)  
*Desired baud rate in Hertz.*
- [uint32\\_t](#) [busIdleTimeout\\_ns](#)  
*Bus idle timeout in nanoseconds.*
- [uint32\\_t](#) [pinLowTimeout\\_ns](#)



- *Pin low timeout in nanoseconds.*  
uint8\_t [sdaGlitchFilterWidth\\_ns](#)  
*Width in nanoseconds of glitch filter on SDA pin.*
- uint8\_t [sclGlitchFilterWidth\\_ns](#)  
*Width in nanoseconds of glitch filter on SCL pin.*
- struct {  
    bool [enable](#)  
        *Enable host request.*  
    lpi2c\_host\_request\_source\_t [source](#)  
        *Host request source.*  
    lpi2c\_host\_request\_polarity\_t [polarity](#)  
        *Host request pin polarity.*  
} [hostRequest](#)

*Host request options.*

### Field Documentation

- (1) **bool \_lpi2c\_master\_config::enableMaster**
- (2) **bool \_lpi2c\_master\_config::enableDoze**
- (3) **bool \_lpi2c\_master\_config::debugEnable**
- (4) **bool \_lpi2c\_master\_config::ignoreAck**
- (5) **lpi2c\_master\_pin\_config\_t \_lpi2c\_master\_config::pinConfig**
- (6) **uint32\_t \_lpi2c\_master\_config::baudRate\_Hz**
- (7) **uint32\_t \_lpi2c\_master\_config::busIdleTimeout\_ns**  
Set to 0 to disable.
- (8) **uint32\_t \_lpi2c\_master\_config::pinLowTimeout\_ns**  
Set to 0 to disable.
- (9) **uint8\_t \_lpi2c\_master\_config::sdaGlitchFilterWidth\_ns**  
Set to 0 to disable.
- (10) **uint8\_t \_lpi2c\_master\_config::sclGlitchFilterWidth\_ns**  
Set to 0 to disable.

- (11) `bool _lpi2c_master_config::enable`
- (12) `lpi2c_host_request_source_t _lpi2c_master_config::source`
- (13) `lpi2c_host_request_polarity_t _lpi2c_master_config::polarity`
- (14) `struct { ... } _lpi2c_master_config::hostRequest`

### 22.6.2.2 struct \_lpi2c\_match\_config

#### Data Fields

- `lpi2c_data_match_config_mode_t matchMode`  
*Data match configuration setting.*
- `bool rxDataMatchOnly`  
*When set to true, received data is ignored until a successful match.*
- `uint32_t match0`  
*Match value 0.*
- `uint32_t match1`  
*Match value 1.*

#### Field Documentation

- (1) `lpi2c_data_match_config_mode_t _lpi2c_match_config::matchMode`
- (2) `bool _lpi2c_match_config::rxDataMatchOnly`
- (3) `uint32_t _lpi2c_match_config::match0`
- (4) `uint32_t _lpi2c_match_config::match1`

### 22.6.2.3 struct \_lpi2c\_master\_transfer

This structure is used to pass transaction parameters to the `LPI2C_MasterTransferNonBlocking()` API.

#### Data Fields

- `uint32_t flags`  
*Bit mask of options for the transfer.*
- `uint16_t slaveAddress`  
*The 7-bit slave address.*
- `lpi2c_direction_t direction`  
*Either `kLPI2C_Read` or `kLPI2C_Write`.*
- `uint32_t subaddress`  
*Sub address.*
- `size_t subaddressSize`  
*Length of sub address to send in bytes.*
- `void * data`  
*Pointer to data to transfer.*
- `size_t dataSize`

*Number of bytes to transfer.*

## Field Documentation

### (1) `uint32_t _lpi2c_master_transfer::flags`

See enumeration `_lpi2c_master_transfer_flags` for available options. Set to 0 or `kLPI2C_TransferDefaultFlag` for normal transfers.

### (2) `uint16_t _lpi2c_master_transfer::slaveAddress`

### (3) `lpi2c_direction_t _lpi2c_master_transfer::direction`

### (4) `uint32_t _lpi2c_master_transfer::subaddress`

Transferred MSB first.

### (5) `size_t _lpi2c_master_transfer::subaddressSize`

Maximum size is 4 bytes.

### (6) `void* _lpi2c_master_transfer::data`

### (7) `size_t _lpi2c_master_transfer::dataSize`

## 22.6.2.4 `struct _lpi2c_master_handle`

Note

The contents of this structure are private and subject to change.

## Data Fields

- `uint8_t state`  
*Transfer state machine current state.*
- `uint16_t remainingBytes`  
*Remaining byte count in current state.*
- `uint8_t * buf`  
*Buffer pointer for current state.*
- `uint16_t commandBuffer [6]`  
*LPI2C command sequence.*
- `lpi2c_master_transfer_t transfer`  
*Copy of the current transfer info.*
- `lpi2c_master_transfer_callback_t completionCallback`  
*Callback function pointer.*
- `void * userData`  
*Application data passed to callback.*

## Field Documentation

- (1) `uint8_t _lpi2c_master_handle::state`
- (2) `uint16_t _lpi2c_master_handle::remainingBytes`
- (3) `uint8_t* _lpi2c_master_handle::buf`
- (4) `uint16_t _lpi2c_master_handle::commandBuffer[6]`

When all 6 command words are used: Start&addr&write[1 word] + subaddr[4 words] + restart&addr&read[1 word]

- (5) `lpi2c_master_transfer_t _lpi2c_master_handle::transfer`
- (6) `lpi2c_master_transfer_callback_t _lpi2c_master_handle::completionCallback`
- (7) `void* _lpi2c_master_handle::userData`

## 22.6.3 Typedef Documentation

**22.6.3.1** `typedef enum _lpi2c_direction lpi2c_direction_t`

**22.6.3.2** `typedef enum _lpi2c_master_pin_config lpi2c_master_pin_config_t`

**22.6.3.3** `typedef enum _lpi2c_host_request_source lpi2c_host_request_source_t`

**22.6.3.4** `typedef enum _lpi2c_host_request_polarity lpi2c_host_request_polarity_t`

**22.6.3.5** `typedef struct _lpi2c_master_config lpi2c_master_config_t`

This structure holds configuration settings for the LPI2C peripheral. To initialize this structure to reasonable defaults, call the [LPI2C\\_MasterGetDefaultConfig\(\)](#) function and pass a pointer to your configuration structure instance.

The configuration structure can be made constant so it resides in flash.

**22.6.3.6** `typedef enum _lpi2c_data_match_config_mode lpi2c_data_match_config_mode_t`

**22.6.3.7** `typedef struct _lpi2c_match_config lpi2c_data_match_config_t`

**22.6.3.8** `typedef struct _lpi2c_master_transfer lpi2c_master_transfer_t`

**22.6.3.9** `typedef struct _lpi2c_master_handle lpi2c_master_handle_t`

**22.6.3.10** `typedef void(* lpi2c_master_transfer_callback_t)(LPI2C_Type *base,  
lpi2c_master_handle_t *handle, status_t completionStatus, void *userData)`

This callback is used only for the non-blocking master transfer API. Specify the callback you wish to use in the call to [LPI2C\\_MasterTransferCreateHandle\(\)](#).

## Parameters

|                          |                                                                                |
|--------------------------|--------------------------------------------------------------------------------|
| <i>base</i>              | The LPI2C peripheral base address.                                             |
| <i>handle</i>            | Pointer to the LPI2C master driver handle.                                     |
| <i>completion-Status</i> | Either kStatus_Success or an error code describing how the transfer completed. |
| <i>userData</i>          | Arbitrary pointer-sized value passed from the application.                     |

## 22.6.4 Enumeration Type Documentation

### 22.6.4.1 enum \_lpi2c\_master\_flags

The following status register flags can be cleared:

- [kLPI2C\\_MasterEndOfPacketFlag](#)
- [kLPI2C\\_MasterStopDetectFlag](#)
- [kLPI2C\\_MasterNackDetectFlag](#)
- [kLPI2C\\_MasterArbitrationLostFlag](#)
- [kLPI2C\\_MasterFifoErrFlag](#)
- [kLPI2C\\_MasterPinLowTimeoutFlag](#)
- [kLPI2C\\_MasterDataMatchFlag](#)

All flags except [kLPI2C\\_MasterBusyFlag](#) and [kLPI2C\\_MasterBusBusyFlag](#) can be enabled as interrupts.

#### Note

These enums are meant to be OR'd together to form a bit mask.

#### Enumerator

***kLPI2C\_MasterTxReadyFlag*** Transmit data flag.  
***kLPI2C\_MasterRxReadyFlag*** Receive data flag.  
***kLPI2C\_MasterEndOfPacketFlag*** End Packet flag.  
***kLPI2C\_MasterStopDetectFlag*** Stop detect flag.  
***kLPI2C\_MasterNackDetectFlag*** NACK detect flag.  
***kLPI2C\_MasterArbitrationLostFlag*** Arbitration lost flag.  
***kLPI2C\_MasterFifoErrFlag*** FIFO error flag.  
***kLPI2C\_MasterPinLowTimeoutFlag*** Pin low timeout flag.  
***kLPI2C\_MasterDataMatchFlag*** Data match flag.  
***kLPI2C\_MasterBusyFlag*** Master busy flag.  
***kLPI2C\_MasterBusBusyFlag*** Bus busy flag.  
***kLPI2C\_MasterClearFlags*** All flags which are cleared by the driver upon starting a transfer.  
***kLPI2C\_MasterIrqFlags*** IRQ sources enabled by the non-blocking transactional API.  
***kLPI2C\_MasterErrorFlags*** Errors to check for.

### 22.6.4.2 enum \_lpi2c\_direction

Enumerator

*kLPI2C\_Write* Master transmit.  
*kLPI2C\_Read* Master receive.

### 22.6.4.3 enum \_lpi2c\_master\_pin\_config

Enumerator

*kLPI2C\_2PinOpenDrain* LPI2C Configured for 2-pin open drain mode.  
*kLPI2C\_2PinOutputOnly* LPI2C Configured for 2-pin output only mode (ultra-fast mode)  
*kLPI2C\_2PinPushPull* LPI2C Configured for 2-pin push-pull mode.  
*kLPI2C\_4PinPushPull* LPI2C Configured for 4-pin push-pull mode.  
*kLPI2C\_2PinOpenDrainWithSeparateSlave* LPI2C Configured for 2-pin open drain mode with separate LPI2C slave.  
*kLPI2C\_2PinOutputOnlyWithSeparateSlave* LPI2C Configured for 2-pin output only mode(ultra-fast mode) with separate LPI2C slave.  
*kLPI2C\_2PinPushPullWithSeparateSlave* LPI2C Configured for 2-pin push-pull mode with separate LPI2C slave.  
*kLPI2C\_4PinPushPullWithInvertedOutput* LPI2C Configured for 4-pin push-pull mode(inverted outputs)

### 22.6.4.4 enum \_lpi2c\_host\_request\_source

Enumerator

*kLPI2C\_HostRequestExternalPin* Select the LPI2C\_HREQ pin as the host request input.  
*kLPI2C\_HostRequestInputTrigger* Select the input trigger as the host request input.

### 22.6.4.5 enum \_lpi2c\_host\_request\_polarity

Enumerator

*kLPI2C\_HostRequestPinActiveLow* Configure the LPI2C\_HREQ pin active low.  
*kLPI2C\_HostRequestPinActiveHigh* Configure the LPI2C\_HREQ pin active high.

### 22.6.4.6 enum \_lpi2c\_data\_match\_config\_mode

Enumerator

*kLPI2C\_MatchDisabled* LPI2C Match Disabled.

***kLPI2C\_1stWordEqualsM0OrM1*** LPI2C Match Enabled and 1st data word equals MATCH0 OR MATCH1.

***kLPI2C\_AnyWordEqualsM0OrM1*** LPI2C Match Enabled and any data word equals MATCH0 OR MATCH1.

***kLPI2C\_1stWordEqualsM0And2ndWordEqualsM1*** LPI2C Match Enabled and 1st data word equals MATCH0, 2nd data equals MATCH1.

***kLPI2C\_AnyWordEqualsM0AndNextWordEqualsM1*** LPI2C Match Enabled and any data word equals MATCH0, next data equals MATCH1.

***kLPI2C\_1stWordAndM1EqualsM0AndM1*** LPI2C Match Enabled and 1st data word and MATCH0 equals MATCH0 and MATCH1.

***kLPI2C\_AnyWordAndM1EqualsM0AndM1*** LPI2C Match Enabled and any data word and MATCH0 equals MATCH0 and MATCH1.

#### 22.6.4.7 enum \_lpi2c\_master\_transfer\_flags

Note

These enumerations are intended to be OR'd together to form a bit mask of options for the `_lpi2c_master_transfer::flags` field.

Enumerator

***kLPI2C\_TransferDefaultFlag*** Transfer starts with a start signal, stops with a stop signal.

***kLPI2C\_TransferNoStartFlag*** Don't send a start condition, address, and sub address.

***kLPI2C\_TransferRepeatedStartFlag*** Send a repeated start condition.

***kLPI2C\_TransferNoStopFlag*** Don't send a stop condition.

### 22.6.5 Function Documentation

#### 22.6.5.1 void LPI2C\_MasterGetDefaultConfig ( lpi2c\_master\_config\_t \* masterConfig )

This function provides the following default configuration for the LPI2C master peripheral:

```
* masterConfig->enableMaster = true;
* masterConfig->debugEnable = false;
* masterConfig->ignoreAck = false;
* masterConfig->pinConfig = kLPI2C_2PinOpenDrain;
* masterConfig->baudRate_Hz = 100000U;
* masterConfig->busIdleTimeout_ns = 0;
* masterConfig->pinLowTimeout_ns = 0;
* masterConfig->sdaGlitchFilterWidth_ns = 0;
* masterConfig->sclGlitchFilterWidth_ns = 0;
* masterConfig->hostRequest.enable = false;
* masterConfig->hostRequest.source = kLPI2C_HostRequestExternalPin;
* masterConfig->hostRequest.polarity = kLPI2C_HostRequestPinActiveHigh;
*
```

After calling this function, you can override any settings in order to customize the configuration, prior to initializing the master driver with `LPI2C_MasterInit()`.



## Parameters

|     |                     |                                                                                                            |
|-----|---------------------|------------------------------------------------------------------------------------------------------------|
| out | <i>masterConfig</i> | User provided configuration structure for default values. Refer to <a href="#">lpi2c_master_config_t</a> . |
|-----|---------------------|------------------------------------------------------------------------------------------------------------|

### 22.6.5.2 void LPI2C\_MasterInit ( LPI2C\_Type \* *base*, const lpi2c\_master\_config\_t \* *masterConfig*, uint32\_t *sourceClock\_Hz* )

This function enables the peripheral clock and initializes the LPI2C master peripheral as described by the user provided configuration. A software reset is performed prior to configuration.

## Parameters

|                       |                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>           | The LPI2C peripheral base address.                                                                                                         |
| <i>masterConfig</i>   | User provided peripheral configuration. Use <a href="#">LPI2C_MasterGetDefaultConfig()</a> to get a set of defaults that you can override. |
| <i>sourceClock_Hz</i> | Frequency in Hertz of the LPI2C functional clock. Used to calculate the baud rate divisors, filter widths, and timeout periods.            |

### 22.6.5.3 void LPI2C\_MasterDeinit ( LPI2C\_Type \* *base* )

This function disables the LPI2C master peripheral and gates the clock. It also performs a software reset to restore the peripheral to reset conditions.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

### 22.6.5.4 void LPI2C\_MasterConfigureDataMatch ( LPI2C\_Type \* *base*, const lpi2c\_data\_match\_config\_t \* *matchConfig* )

## Parameters

|                    |                                      |
|--------------------|--------------------------------------|
| <i>base</i>        | The LPI2C peripheral base address.   |
| <i>matchConfig</i> | Settings for the data match feature. |

### 22.6.5.5 status\_t LPI2C\_MasterCheckAndClearError ( LPI2C\_Type \* *base*, uint32\_t *status* )

## Parameters

|               |                                                  |
|---------------|--------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.               |
| <i>status</i> | Current status flags value that will be checked. |

## Return values

|                                       |  |
|---------------------------------------|--|
| <i>kStatus_Success</i>                |  |
| <i>kStatus_LPI2C_PinLow-Timeout</i>   |  |
| <i>kStatus_LPI2C_-ArbitrationLost</i> |  |
| <i>kStatus_LPI2C_Nak</i>              |  |
| <i>kStatus_LPI2C_FifoError</i>        |  |

**22.6.5.6 status\_t LPI2C\_CheckForBusyBus ( LPI2C\_Type \* *base* )**

A busy bus is allowed if we are the one driving it.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Return values

|                           |  |
|---------------------------|--|
| <i>kStatus_Success</i>    |  |
| <i>kStatus_LPI2C_Busy</i> |  |

**22.6.5.7 static void LPI2C\_MasterReset ( LPI2C\_Type \* *base* ) [inline], [static]**

Restores the LPI2C master peripheral to reset conditions.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

**22.6.5.8 static void LPI2C\_MasterEnable ( LPI2C\_Type \* *base*, bool *enable* ) [inline], [static]**

## Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.                                     |
| <i>enable</i> | Pass true to enable or false to disable the specified LPI2C as master. |

### 22.6.5.9 static uint32\_t LPI2C\_MasterGetStatusFlags ( LPI2C\_Type \* *base* ) [inline], [static]

A bit mask with the state of all LPI2C master status flags is returned. For each flag, the corresponding bit in the return value is set if the flag is asserted.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Returns

State of the status flags:

- 1: related status flag is set.
- 0: related status flag is not set.

## See Also

[\\_lpi2c\\_master\\_flags](#)

### 22.6.5.10 static void LPI2C\_MasterClearStatusFlags ( LPI2C\_Type \* *base*, uint32\_t *statusMask* ) [inline], [static]

The following status register flags can be cleared:

- [kLPI2C\\_MasterEndOfPacketFlag](#)
- [kLPI2C\\_MasterStopDetectFlag](#)
- [kLPI2C\\_MasterNackDetectFlag](#)
- [kLPI2C\\_MasterArbitrationLostFlag](#)
- [kLPI2C\\_MasterFifoErrFlag](#)
- [kLPI2C\\_MasterPinLowTimeoutFlag](#)
- [kLPI2C\\_MasterDataMatchFlag](#)

Attempts to clear other flags has no effect.

## Parameters

|                   |                                                                                                                                                                                                                                    |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>       | The LPI2C peripheral base address.                                                                                                                                                                                                 |
| <i>statusMask</i> | A bitmask of status flags that are to be cleared. The mask is composed of <code>_lpi2c_master_flags</code> enumerators OR'd together. You may pass the result of a previous call to <a href="#">LPI2C_MasterGetStatusFlags()</a> . |

## See Also

[\\_lpi2c\\_master\\_flags](#).

#### 22.6.5.11 **static void LPI2C\_MasterEnableInterrupts ( LPI2C\_Type \* *base*, uint32\_t *interruptMask* ) [inline], [static]**

All flags except [kLPI2C\\_MasterBusyFlag](#) and [kLPI2C\\_MasterBusBusyFlag](#) can be enabled as interrupts.

## Parameters

|                      |                                                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>          | The LPI2C peripheral base address.                                                                                                                 |
| <i>interruptMask</i> | Bit mask of interrupts to enable. See <code>_lpi2c_master_flags</code> for the set of constants that should be OR'd together to form the bit mask. |

#### 22.6.5.12 **static void LPI2C\_MasterDisableInterrupts ( LPI2C\_Type \* *base*, uint32\_t *interruptMask* ) [inline], [static]**

All flags except [kLPI2C\\_MasterBusyFlag](#) and [kLPI2C\\_MasterBusBusyFlag](#) can be enabled as interrupts.

## Parameters

|                      |                                                                                                                                                     |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>          | The LPI2C peripheral base address.                                                                                                                  |
| <i>interruptMask</i> | Bit mask of interrupts to disable. See <code>_lpi2c_master_flags</code> for the set of constants that should be OR'd together to form the bit mask. |

#### 22.6.5.13 **static uint32\_t LPI2C\_MasterGetEnabledInterrupts ( LPI2C\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Returns

A bitmask composed of `_lpi2c_master_flags` enumerators OR'd together to indicate the set of enabled interrupts.

**22.6.5.14** `static void LPI2C_MasterEnableDMA ( LPI2C_Type * base, bool enableTx, bool enableRx ) [inline], [static]`

## Parameters

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>base</i>     | The LPI2C peripheral base address.                                             |
| <i>enableTx</i> | Enable flag for transmit DMA request. Pass true for enable, false for disable. |
| <i>enableRx</i> | Enable flag for receive DMA request. Pass true for enable, false for disable.  |

**22.6.5.15** `static uint32_t LPI2C_MasterGetTxFifoAddress ( LPI2C_Type * base ) [inline], [static]`

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Returns

The LPI2C Master Transmit Data Register address.

**22.6.5.16** `static uint32_t LPI2C_MasterGetRxFifoAddress ( LPI2C_Type * base ) [inline], [static]`

## Parameters

|  |  |
|--|--|
|  |  |
|--|--|

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

Returns

The LPI2C Master Receive Data Register address.

**22.6.5.17 static void LPI2C\_MasterSetWatermarks ( LPI2C\_Type \* *base*, size\_t *txWords*, size\_t *rxWords* ) [inline], [static]**

Parameters

|                |                                                                                                                                                                                                                                                             |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>    | The LPI2C peripheral base address.                                                                                                                                                                                                                          |
| <i>txWords</i> | Transmit FIFO watermark value in words. The <a href="#">kLPI2C_MasterTxReadyFlag</a> flag is set whenever the number of words in the transmit FIFO is equal or less than <i>txWords</i> . Writing a value equal or greater than the FIFO size is truncated. |
| <i>rxWords</i> | Receive FIFO watermark value in words. The <a href="#">kLPI2C_MasterRxReadyFlag</a> flag is set whenever the number of words in the receive FIFO is greater than <i>rxWords</i> . Writing a value equal or greater than the FIFO size is truncated.         |

**22.6.5.18 static void LPI2C\_MasterGetFifoCounts ( LPI2C\_Type \* *base*, size\_t \* *rxCount*, size\_t \* *txCount* ) [inline], [static]**

Parameters

|     |                |                                                                                                                              |
|-----|----------------|------------------------------------------------------------------------------------------------------------------------------|
|     | <i>base</i>    | The LPI2C peripheral base address.                                                                                           |
| out | <i>txCount</i> | Pointer through which the current number of words in the transmit FIFO is returned. Pass NULL if this value is not required. |
| out | <i>rxCount</i> | Pointer through which the current number of words in the receive FIFO is returned. Pass NULL if this value is not required.  |

**22.6.5.19 void LPI2C\_MasterSetBaudRate ( LPI2C\_Type \* *base*, uint32\_t *sourceClock\_Hz*, uint32\_t *baudRate\_Hz* )**

The LPI2C master is automatically disabled and re-enabled as necessary to configure the baud rate. Do not call this function during a transfer, or the transfer is aborted.

**Note**

Please note that the second parameter is the clock frequency of LPI2C module, the third parameter means user configured bus baudrate, this implementation is different from other I2C drivers which use baudrate configuration as second parameter and source clock frequency as third parameter.

## Parameters

|                       |                                            |
|-----------------------|--------------------------------------------|
| <i>base</i>           | The LPI2C peripheral base address.         |
| <i>sourceClock_Hz</i> | LPI2C functional clock frequency in Hertz. |
| <i>baudRate_Hz</i>    | Requested bus frequency in Hertz.          |

### 22.6.5.20 static bool LPI2C\_MasterGetBusIdleState ( LPI2C\_Type \* *base* ) [inline], [static]

Requires the master mode to be enabled.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Return values

|              |              |
|--------------|--------------|
| <i>true</i>  | Bus is busy. |
| <i>false</i> | Bus is idle. |

### 22.6.5.21 status\_t LPI2C\_MasterStart ( LPI2C\_Type \* *base*, uint8\_t *address*, lpi2c\_direction\_t *dir* )

This function is used to initiate a new master mode transfer. First, the bus state is checked to ensure that another master is not occupying the bus. Then a START signal is transmitted, followed by the 7-bit address specified in the *address* parameter. Note that this function does not actually wait until the START and address are successfully sent on the bus before returning.

## Parameters

|                |                                                                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>    | The LPI2C peripheral base address.                                                                                                                                                  |
| <i>address</i> | 7-bit slave device address, in bits [6:0].                                                                                                                                          |
| <i>dir</i>     | Master transfer direction, either <a href="#">kLPI2C_Read</a> or <a href="#">kLPI2C_Write</a> . This parameter is used to set the R/w bit (bit 0) in the transmitted slave address. |



## Return values

|                           |                                                                           |
|---------------------------|---------------------------------------------------------------------------|
| <i>kStatus_Success</i>    | START signal and address were successfully enqueued in the transmit FIFO. |
| <i>kStatus_LPI2C_Busy</i> | Another master is currently utilizing the bus.                            |

#### 22.6.5.22 static status\_t LPI2C\_MasterRepeatedStart ( LPI2C\_Type \* *base*, uint8\_t *address*, lpi2c\_direction\_t *dir* ) [inline], [static]

This function is used to send a Repeated START signal when a transfer is already in progress. Like [LPI2C\\_MasterStart\(\)](#), it also sends the specified 7-bit address.

## Note

This function exists primarily to maintain compatible APIs between LPI2C and I2C drivers, as well as to better document the intent of code that uses these APIs.

## Parameters

|                |                                                                                                                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>    | The LPI2C peripheral base address.                                                                                                                                                  |
| <i>address</i> | 7-bit slave device address, in bits [6:0].                                                                                                                                          |
| <i>dir</i>     | Master transfer direction, either <a href="#">kLPI2C_Read</a> or <a href="#">kLPI2C_Write</a> . This parameter is used to set the R/w bit (bit 0) in the transmitted slave address. |

## Return values

|                           |                                                                                    |
|---------------------------|------------------------------------------------------------------------------------|
| <i>kStatus_Success</i>    | Repeated START signal and address were successfully enqueued in the transmit FIFO. |
| <i>kStatus_LPI2C_Busy</i> | Another master is currently utilizing the bus.                                     |

#### 22.6.5.23 status\_t LPI2C\_MasterSend ( LPI2C\_Type \* *base*, void \* *txBuff*, size\_t *txSize* )

Sends up to *txSize* number of bytes to the previously addressed slave device. The slave may reply with a NAK to any byte in order to terminate the transfer early. If this happens, this function returns [kStatus\\_LPI2C\\_Nak](#).

## Parameters

|               |                                                    |
|---------------|----------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.                 |
| <i>txBuff</i> | The pointer to the data to be transferred.         |
| <i>txSize</i> | The length in bytes of the data to be transferred. |

## Return values

|                                      |                                                    |
|--------------------------------------|----------------------------------------------------|
| <i>kStatus_Success</i>               | Data was sent successfully.                        |
| <i>kStatus_LPI2C_Busy</i>            | Another master is currently utilizing the bus.     |
| <i>kStatus_LPI2C_Nak</i>             | The slave device sent a NAK in response to a byte. |
| <i>kStatus_LPI2C_FifoError</i>       | FIFO under run or over run.                        |
| <i>kStatus_LPI2C_ArbitrationLost</i> | Arbitration lost error.                            |
| <i>kStatus_LPI2C_PinLowTimeout</i>   | SCL or SDA were held low longer than the timeout.  |

#### 22.6.5.24 status\_t LPI2C\_MasterReceive ( LPI2C\_Type \* *base*, void \* *rxBuff*, size\_t *rxSize* )

## Parameters

|               |                                                    |
|---------------|----------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.                 |
| <i>rxBuff</i> | The pointer to the data to be transferred.         |
| <i>rxSize</i> | The length in bytes of the data to be transferred. |

## Return values

|                                      |                                                    |
|--------------------------------------|----------------------------------------------------|
| <i>kStatus_Success</i>               | Data was received successfully.                    |
| <i>kStatus_LPI2C_Busy</i>            | Another master is currently utilizing the bus.     |
| <i>kStatus_LPI2C_Nak</i>             | The slave device sent a NAK in response to a byte. |
| <i>kStatus_LPI2C_FifoError</i>       | FIFO under run or overrun.                         |
| <i>kStatus_LPI2C_ArbitrationLost</i> | Arbitration lost error.                            |
| <i>kStatus_LPI2C_PinLowTimeout</i>   | SCL or SDA were held low longer than the timeout.  |

#### 22.6.5.25 status\_t LPI2C\_MasterStop ( LPI2C\_Type \* *base* )

This function does not return until the STOP signal is seen on the bus, or an error occurs.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Return values

|                                      |                                                                                  |
|--------------------------------------|----------------------------------------------------------------------------------|
| <i>kStatus_Success</i>               | The STOP signal was successfully sent on the bus and the transaction terminated. |
| <i>kStatus_LPI2C_Busy</i>            | Another master is currently utilizing the bus.                                   |
| <i>kStatus_LPI2C_Nak</i>             | The slave device sent a NAK in response to a byte.                               |
| <i>kStatus_LPI2C_FifoError</i>       | FIFO under run or overrun.                                                       |
| <i>kStatus_LPI2C_ArbitrationLost</i> | Arbitration lost error.                                                          |
| <i>kStatus_LPI2C_PinLowTimeout</i>   | SCL or SDA were held low longer than the timeout.                                |

### 22.6.5.26 status\_t LPI2C\_MasterTransferBlocking ( LPI2C\_Type \* *base*, lpi2c\_master\_transfer\_t \* *transfer* )

## Note

The API does not return until the transfer succeeds or fails due to error happens during transfer.

## Parameters

|                 |                                    |
|-----------------|------------------------------------|
| <i>base</i>     | The LPI2C peripheral base address. |
| <i>transfer</i> | Pointer to the transfer structure. |

## Return values

|                                      |                                                    |
|--------------------------------------|----------------------------------------------------|
| <i>kStatus_Success</i>               | Data was received successfully.                    |
| <i>kStatus_LPI2C_Busy</i>            | Another master is currently utilizing the bus.     |
| <i>kStatus_LPI2C_Nak</i>             | The slave device sent a NAK in response to a byte. |
| <i>kStatus_LPI2C_FifoError</i>       | FIFO under run or overrun.                         |
| <i>kStatus_LPI2C_ArbitrationLost</i> | Arbitration lost error.                            |
| <i>kStatus_LPI2C_PinLowTimeout</i>   | SCL or SDA were held low longer than the timeout.  |

**22.6.5.27 void LPI2C\_MasterTransferCreateHandle ( LPI2C\_Type \* *base*,  
lpi2c\_master\_handle\_t \* *handle*, lpi2c\_master\_transfer\_callback\_t *callback*,  
void \* *userData* )**

The creation of a handle is for use with the non-blocking APIs. Once a handle is created, there is not a corresponding destroy handle. If the user wants to terminate a transfer, the [LPI2C\\_MasterTransferAbort\(\)](#) API shall be called.

#### Note

The function also enables the NVIC IRQ for the input LPI2C. Need to notice that on some SoCs the LPI2C IRQ is connected to INTMUX, in this case user needs to enable the associated INTMUX IRQ in application.

#### Parameters

|     |                 |                                                              |
|-----|-----------------|--------------------------------------------------------------|
|     | <i>base</i>     | The LPI2C peripheral base address.                           |
| out | <i>handle</i>   | Pointer to the LPI2C master driver handle.                   |
|     | <i>callback</i> | User provided pointer to the asynchronous callback function. |
|     | <i>userData</i> | User provided pointer to the application callback data.      |

**22.6.5.28 status\_t LPI2C\_MasterTransferNonBlocking ( LPI2C\_Type \* *base*,  
lpi2c\_master\_handle\_t \* *handle*, lpi2c\_master\_transfer\_t \* *transfer* )**

#### Parameters

|                 |                                            |
|-----------------|--------------------------------------------|
| <i>base</i>     | The LPI2C peripheral base address.         |
| <i>handle</i>   | Pointer to the LPI2C master driver handle. |
| <i>transfer</i> | The pointer to the transfer descriptor.    |

#### Return values

|                           |                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------|
| <i>kStatus_Success</i>    | The transaction was started successfully.                                                                   |
| <i>kStatus_LPI2C_Busy</i> | Either another master is currently utilizing the bus, or a non-blocking transaction is already in progress. |

**22.6.5.29 status\_t LPI2C\_MasterTransferGetCount ( LPI2C\_Type \* *base*,  
lpi2c\_master\_handle\_t \* *handle*, size\_t \* *count* )**

## Parameters

|     |               |                                                                     |
|-----|---------------|---------------------------------------------------------------------|
|     | <i>base</i>   | The LPI2C peripheral base address.                                  |
|     | <i>handle</i> | Pointer to the LPI2C master driver handle.                          |
| out | <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction. |

## Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              |                                                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

### 22.6.5.30 void LPI2C\_MasterTransferAbort ( LPI2C\_Type \* *base*, lpi2c\_master\_handle\_t \* *handle* )

## Note

It is not safe to call this function from an IRQ handler that has a higher priority than the LPI2C peripheral's IRQ priority.

## Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.         |
| <i>handle</i> | Pointer to the LPI2C master driver handle. |

### 22.6.5.31 void LPI2C\_MasterTransferHandleIRQ ( LPI2C\_Type \* *base*, void \* *lpi2cMasterHandle* )

## Note

This function does not need to be called unless you are reimplementing the nonblocking API's interrupt handler routines to add special functionality.

## Parameters

|                          |                                            |
|--------------------------|--------------------------------------------|
| <i>base</i>              | The LPI2C peripheral base address.         |
| <i>lpi2cMasterHandle</i> | Pointer to the LPI2C master driver handle. |

## 22.7 LPI2C Slave Driver

### 22.7.1 Overview

#### Data Structures

- struct [\\_lpi2c\\_slave\\_config](#)  
*Structure with settings to initialize the LPI2C slave module. [More...](#)*
- struct [\\_lpi2c\\_slave\\_transfer](#)  
*LPI2C slave transfer structure. [More...](#)*
- struct [\\_lpi2c\\_slave\\_handle](#)  
*LPI2C slave handle structure. [More...](#)*

#### Typedefs

- typedef enum  
[\\_lpi2c\\_slave\\_address\\_match](#) [lpi2c\\_slave\\_address\\_match\\_t](#)  
*LPI2C slave address match options.*
- typedef struct [\\_lpi2c\\_slave\\_config](#) [lpi2c\\_slave\\_config\\_t](#)  
*Structure with settings to initialize the LPI2C slave module.*
- typedef enum  
[\\_lpi2c\\_slave\\_transfer\\_event](#) [lpi2c\\_slave\\_transfer\\_event\\_t](#)  
*Set of events sent to the callback for non blocking slave transfers.*
- typedef struct  
[\\_lpi2c\\_slave\\_transfer](#) [lpi2c\\_slave\\_transfer\\_t](#)  
*LPI2C slave transfer structure.*
- typedef struct [\\_lpi2c\\_slave\\_handle](#) [lpi2c\\_slave\\_handle\\_t](#)  
*LPI2C slave handle structure.*
- typedef void(\* [lpi2c\\_slave\\_transfer\\_callback\\_t](#) )(LPI2C\_Type \*base, [lpi2c\\_slave\\_transfer\\_t](#) \*transfer, void \*userData)  
*Slave event callback function pointer type.*

## Enumerations

- enum `_lpi2c_slave_flags` {  
`kLPI2C_SlaveTxReadyFlag` = `LPI2C_SSR_TDF_MASK`,  
`kLPI2C_SlaveRxReadyFlag` = `LPI2C_SSR_RDF_MASK`,  
`kLPI2C_SlaveAddressValidFlag` = `LPI2C_SSR_AVF_MASK`,  
`kLPI2C_SlaveTransmitAckFlag` = `LPI2C_SSR_TAF_MASK`,  
`kLPI2C_SlaveRepeatedStartDetectFlag` = `LPI2C_SSR_RSF_MASK`,  
`kLPI2C_SlaveStopDetectFlag` = `LPI2C_SSR_SDF_MASK`,  
`kLPI2C_SlaveBitErrFlag` = `LPI2C_SSR_BEF_MASK`,  
`kLPI2C_SlaveFifoErrFlag` = `LPI2C_SSR_FEF_MASK`,  
`kLPI2C_SlaveAddressMatch0Flag` = `LPI2C_SSR_AM0F_MASK`,  
`kLPI2C_SlaveAddressMatch1Flag` = `LPI2C_SSR_AM1F_MASK`,  
`kLPI2C_SlaveGeneralCallFlag` = `LPI2C_SSR_GCF_MASK`,  
`kLPI2C_SlaveBusyFlag` = `LPI2C_SSR_SBF_MASK`,  
`kLPI2C_SlaveBusBusyFlag` = `LPI2C_SSR_BBF_MASK`,  
`kLPI2C_SlaveClearFlags`,  
`kLPI2C_SlaveIrqFlags`,  
`kLPI2C_SlaveErrorFlags` = `kLPI2C_SlaveFifoErrFlag` | `kLPI2C_SlaveBitErrFlag` }  
*LPI2C slave peripheral flags.*
- enum `_lpi2c_slave_address_match` {  
`kLPI2C_MatchAddress0` = 0U,  
`kLPI2C_MatchAddress0OrAddress1` = 2U,  
`kLPI2C_MatchAddress0ThroughAddress1` = 6U }  
*LPI2C slave address match options.*
- enum `_lpi2c_slave_transfer_event` {  
`kLPI2C_SlaveAddressMatchEvent` = 0x01U,  
`kLPI2C_SlaveTransmitEvent` = 0x02U,  
`kLPI2C_SlaveReceiveEvent` = 0x04U,  
`kLPI2C_SlaveTransmitAckEvent` = 0x08U,  
`kLPI2C_SlaveRepeatedStartEvent` = 0x10U,  
`kLPI2C_SlaveCompletionEvent` = 0x20U,  
`kLPI2C_SlaveAllEvents` }  
*Set of events sent to the callback for non blocking slave transfers.*

## Slave initialization and deinitialization

- void `LPI2C_SlaveGetDefaultConfig` (`lpi2c_slave_config_t` \*slaveConfig)  
*Provides a default configuration for the LPI2C slave peripheral.*
- void `LPI2C_SlaveInit` (`LPI2C_Type` \*base, const `lpi2c_slave_config_t` \*slaveConfig, uint32\_t sourceClock\_Hz)  
*Initializes the LPI2C slave peripheral.*
- void `LPI2C_SlaveDeinit` (`LPI2C_Type` \*base)  
*Deinitializes the LPI2C slave peripheral.*
- static void `LPI2C_SlaveReset` (`LPI2C_Type` \*base)  
*Performs a software reset of the LPI2C slave peripheral.*

- static void [LPI2C\\_SlaveEnable](#) (LPI2C\_Type \*base, bool enable)  
*Enables or disables the LPI2C module as slave.*

## Slave status

- static uint32\_t [LPI2C\\_SlaveGetStatusFlags](#) (LPI2C\_Type \*base)  
*Gets the LPI2C slave status flags.*
- static void [LPI2C\\_SlaveClearStatusFlags](#) (LPI2C\_Type \*base, uint32\_t statusMask)  
*Clears the LPI2C status flag state.*

## Slave interrupts

- static void [LPI2C\\_SlaveEnableInterrupts](#) (LPI2C\_Type \*base, uint32\_t interruptMask)  
*Enables the LPI2C slave interrupt requests.*
- static void [LPI2C\\_SlaveDisableInterrupts](#) (LPI2C\_Type \*base, uint32\_t interruptMask)  
*Disables the LPI2C slave interrupt requests.*
- static uint32\_t [LPI2C\\_SlaveGetEnabledInterrupts](#) (LPI2C\_Type \*base)  
*Returns the set of currently enabled LPI2C slave interrupt requests.*

## Slave DMA control

- static void [LPI2C\\_SlaveEnableDMA](#) (LPI2C\_Type \*base, bool enableAddressValid, bool enable-Rx, bool enableTx)  
*Enables or disables the LPI2C slave peripheral DMA requests.*

## Slave bus operations

- static bool [LPI2C\\_SlaveGetBusIdleState](#) (LPI2C\_Type \*base)  
*Returns whether the bus is idle.*
- static void [LPI2C\\_SlaveTransmitAck](#) (LPI2C\_Type \*base, bool ackOrNack)  
*Transmits either an ACK or NAK on the I2C bus in response to a byte from the master.*
- static void [LPI2C\\_SlaveEnableAckStall](#) (LPI2C\_Type \*base, bool enable)  
*Enables or disables ACKSTALL.*
- static uint32\_t [LPI2C\\_SlaveGetReceivedAddress](#) (LPI2C\_Type \*base)  
*Returns the slave address sent by the I2C master.*
- [status\\_t](#) [LPI2C\\_SlaveSend](#) (LPI2C\_Type \*base, void \*txBuff, size\_t txSize, size\_t \*actualTxSize)  
*Performs a polling send transfer on the I2C bus.*
- [status\\_t](#) [LPI2C\\_SlaveReceive](#) (LPI2C\_Type \*base, void \*rxBuff, size\_t rxSize, size\_t \*actualRx-Size)  
*Performs a polling receive transfer on the I2C bus.*



## Slave non-blocking

- void [LPI2C\\_SlaveTransferCreateHandle](#) (LPI2C\_Type \*base, [lpi2c\\_slave\\_handle\\_t](#) \*handle, [lpi2c\\_slave\\_transfer\\_callback\\_t](#) callback, void \*userData)  
*Creates a new handle for the LPI2C slave non-blocking APIs.*
- [status\\_t LPI2C\\_SlaveTransferNonBlocking](#) (LPI2C\_Type \*base, [lpi2c\\_slave\\_handle\\_t](#) \*handle, [uint32\\_t](#) eventMask)  
*Starts accepting slave transfers.*
- [status\\_t LPI2C\\_SlaveTransferGetCount](#) (LPI2C\_Type \*base, [lpi2c\\_slave\\_handle\\_t](#) \*handle, [size\\_t](#) \*count)  
*Gets the slave transfer status during a non-blocking transfer.*
- void [LPI2C\\_SlaveTransferAbort](#) (LPI2C\_Type \*base, [lpi2c\\_slave\\_handle\\_t](#) \*handle)  
*Aborts the slave non-blocking transfers.*

## Slave IRQ handler

- void [LPI2C\\_SlaveTransferHandleIRQ](#) (LPI2C\_Type \*base, [lpi2c\\_slave\\_handle\\_t](#) \*handle)  
*Reusable routine to handle slave interrupts.*

## 22.7.2 Data Structure Documentation

### 22.7.2.1 struct [\\_lpi2c\\_slave\\_config](#)

This structure holds configuration settings for the LPI2C slave peripheral. To initialize this structure to reasonable defaults, call the [LPI2C\\_SlaveGetDefaultConfig\(\)](#) function and pass a pointer to your configuration structure instance.

The configuration structure can be made constant so it resides in flash.

### Data Fields

- bool [enableSlave](#)  
*Enable slave mode.*
- [uint8\\_t address0](#)  
*Slave's 7-bit address.*
- [uint8\\_t address1](#)  
*Alternate slave 7-bit address.*
- [lpi2c\\_slave\\_address\\_match\\_t addressMatchMode](#)  
*Address matching options.*
- bool [filterDozeEnable](#)  
*Enable digital glitch filter in doze mode.*
- bool [filterEnable](#)  
*Enable digital glitch filter.*
- bool [enableGeneralCall](#)  
*Enable general call address matching.*
- struct {

bool [enableAck](#)

*Enables SCL clock stretching during slave-transmit address byte(s) and slave-receiver address and data b*

bool [enableTx](#)

*Enables SCL clock stretching when the transmit data flag is set during a slave-transmit transfer.*

bool [enableRx](#)

*Enables SCL clock stretching when receive data flag is set during a slave-receive transfer.*

bool [enableAddress](#)

*Enables SCL clock stretching when the address valid flag is asserted.*

} [sclStall](#)

*SCL stall enable options.*

- bool [ignoreAck](#)

*Continue transfers after a NACK is detected.*

- bool [enableReceivedAddressRead](#)

*Enable reading the address received address as the first byte of data.*

- uint32\_t [sdaGlitchFilterWidth\\_ns](#)

*Width in nanoseconds of the digital filter on the SDA signal.*

- uint32\_t [sclGlitchFilterWidth\\_ns](#)

*Width in nanoseconds of the digital filter on the SCL signal.*

- uint32\_t [dataValidDelay\\_ns](#)

*Width in nanoseconds of the data valid delay.*

- uint32\_t [clockHoldTime\\_ns](#)

*Width in nanoseconds of the clock hold time.*

## Field Documentation

(1) **bool \_lpi2c\_slave\_config::enableSlave**

(2) **uint8\_t \_lpi2c\_slave\_config::address0**

(3) **uint8\_t \_lpi2c\_slave\_config::address1**

(4) **lpi2c\_slave\_address\_match\_t \_lpi2c\_slave\_config::addressMatchMode**

(5) **bool \_lpi2c\_slave\_config::filterDozeEnable**

(6) **bool \_lpi2c\_slave\_config::filterEnable**

(7) **bool \_lpi2c\_slave\_config::enableGeneralCall**

(8) **bool \_lpi2c\_slave\_config::enableAck**

Clock stretching occurs when transmitting the 9th bit. When `enableAckSCLStall` is enabled, there is no need to set either `enableRxDataSCLStall` or `enableAddressSCLStall`.

- (9) `bool _lpi2c_slave_config::enableTx`
- (10) `bool _lpi2c_slave_config::enableRx`
- (11) `bool _lpi2c_slave_config::enableAddress`
- (12) `struct { ... } _lpi2c_slave_config::sclStall`
- (13) `bool _lpi2c_slave_config::ignoreAck`
- (14) `bool _lpi2c_slave_config::enableReceivedAddressRead`
- (15) `uint32_t _lpi2c_slave_config::sdaGlitchFilterWidth_ns`  
Set to 0 to disable.
- (16) `uint32_t _lpi2c_slave_config::sclGlitchFilterWidth_ns`  
Set to 0 to disable.
- (17) `uint32_t _lpi2c_slave_config::dataValidDelay_ns`
- (18) `uint32_t _lpi2c_slave_config::clockHoldTime_ns`

### 22.7.2.2 struct \_lpi2c\_slave\_transfer

#### Data Fields

- `lpi2c_slave_transfer_event_t event`  
*Reason the callback is being invoked.*
- `uint8_t receivedAddress`  
*Matching address send by master.*
- `uint8_t * data`  
*Transfer buffer.*
- `size_t dataSize`  
*Transfer size.*
- `status_t completionStatus`  
*Success or error code describing how the transfer completed.*
- `size_t transferredCount`  
*Number of bytes actually transferred since start or last repeated start.*

#### Field Documentation

- (1) `lpi2c_slave_transfer_event_t _lpi2c_slave_transfer::event`
- (2) `uint8_t _lpi2c_slave_transfer::receivedAddress`
- (3) `status_t _lpi2c_slave_transfer::completionStatus`

Only applies for `kLPI2C_SlaveCompletionEvent`.

(4) `size_t _lpi2c_slave_transfer::transferredCount`

### 22.7.2.3 struct \_lpi2c\_slave\_handle

Note

The contents of this structure are private and subject to change.

#### Data Fields

- `lpi2c_slave_transfer_t transfer`  
*LPI2C slave transfer copy.*
- `bool isBusy`  
*Whether transfer is busy.*
- `bool wasTransmit`  
*Whether the last transfer was a transmit.*
- `uint32_t eventMask`  
*Mask of enabled events.*
- `uint32_t transferredCount`  
*Count of bytes transferred.*
- `lpi2c_slave_transfer_callback_t callback`  
*Callback function called at transfer event.*
- `void * userData`  
*Callback parameter passed to callback.*

#### Field Documentation

- (1) `lpi2c_slave_transfer_t _lpi2c_slave_handle::transfer`
- (2) `bool _lpi2c_slave_handle::isBusy`
- (3) `bool _lpi2c_slave_handle::wasTransmit`
- (4) `uint32_t _lpi2c_slave_handle::eventMask`
- (5) `uint32_t _lpi2c_slave_handle::transferredCount`
- (6) `lpi2c_slave_transfer_callback_t _lpi2c_slave_handle::callback`
- (7) `void* _lpi2c_slave_handle::userData`

### 22.7.3 Typedef Documentation

22.7.3.1 `typedef enum _lpi2c_slave_address_match lpi2c_slave_address_match_t`

22.7.3.2 `typedef struct _lpi2c_slave_config lpi2c_slave_config_t`

This structure holds configuration settings for the LPI2C slave peripheral. To initialize this structure to reasonable defaults, call the `LPI2C_SlaveGetDefaultConfig()` function and pass a pointer to your

configuration structure instance.

The configuration structure can be made constant so it resides in flash.

### 22.7.3.3 typedef enum \_lpi2c\_slave\_transfer\_event lpi2c\_slave\_transfer\_event\_t

These event enumerations are used for two related purposes. First, a bit mask created by OR'ing together events is passed to [LPI2C\\_SlaveTransferNonBlocking\(\)](#) in order to specify which events to enable. Then, when the slave callback is invoked, it is passed the current event through its *transfer* parameter.

Note

These enumerations are meant to be OR'd together to form a bit mask of events.

### 22.7.3.4 typedef struct \_lpi2c\_slave\_handle lpi2c\_slave\_handle\_t

### 22.7.3.5 typedef void(\* lpi2c\_slave\_transfer\_callback\_t)(LPI2C\_Type \*base, lpi2c\_slave\_transfer\_t \*transfer, void \*userData)

This callback is used only for the slave non-blocking transfer API. To install a callback, use the [LPI2C\\_SlaveSetCallback\(\)](#) function after you have created a handle.

Parameters

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>base</i>     | Base address for the LPI2C instance on which the event occurred.                     |
| <i>transfer</i> | Pointer to transfer descriptor containing values passed to and/or from the callback. |
| <i>userData</i> | Arbitrary pointer-sized value passed from the application.                           |

## 22.7.4 Enumeration Type Documentation

### 22.7.4.1 enum \_lpi2c\_slave\_flags

The following status register flags can be cleared:

- [kLPI2C\\_SlaveRepeatedStartDetectFlag](#)
- [kLPI2C\\_SlaveStopDetectFlag](#)
- [kLPI2C\\_SlaveBitErrFlag](#)
- [kLPI2C\\_SlaveFifoErrFlag](#)

All flags except [kLPI2C\\_SlaveBusyFlag](#) and [kLPI2C\\_SlaveBusBusyFlag](#) can be enabled as interrupts.

## Note

These enumerations are meant to be OR'd together to form a bit mask.

## Enumerator

***kLPI2C\_SlaveTxReadyFlag*** Transmit data flag.  
***kLPI2C\_SlaveRxReadyFlag*** Receive data flag.  
***kLPI2C\_SlaveAddressValidFlag*** Address valid flag.  
***kLPI2C\_SlaveTransmitAckFlag*** Transmit ACK flag.  
***kLPI2C\_SlaveRepeatedStartDetectFlag*** Repeated start detect flag.  
***kLPI2C\_SlaveStopDetectFlag*** Stop detect flag.  
***kLPI2C\_SlaveBitErrFlag*** Bit error flag.  
***kLPI2C\_SlaveFifoErrFlag*** FIFO error flag.  
***kLPI2C\_SlaveAddressMatch0Flag*** Address match 0 flag.  
***kLPI2C\_SlaveAddressMatch1Flag*** Address match 1 flag.  
***kLPI2C\_SlaveGeneralCallFlag*** General call flag.  
***kLPI2C\_SlaveBusyFlag*** Master busy flag.  
***kLPI2C\_SlaveBusBusyFlag*** Bus busy flag.  
***kLPI2C\_SlaveClearFlags*** All flags which are cleared by the driver upon starting a transfer.  
***kLPI2C\_SlaveIrqFlags*** IRQ sources enabled by the non-blocking transactional API.  
***kLPI2C\_SlaveErrorFlags*** Errors to check for.

## 22.7.4.2 enum \_lpi2c\_slave\_address\_match

## Enumerator

***kLPI2C\_MatchAddress0*** Match only address 0.  
***kLPI2C\_MatchAddress0OrAddress1*** Match either address 0 or address 1.  
***kLPI2C\_MatchAddress0ThroughAddress1*** Match a range of slave addresses from address 0 through address 1.

## 22.7.4.3 enum \_lpi2c\_slave\_transfer\_event

These event enumerations are used for two related purposes. First, a bit mask created by OR'ing together events is passed to [LPI2C\\_SlaveTransferNonBlocking\(\)](#) in order to specify which events to enable. Then, when the slave callback is invoked, it is passed the current event through its *transfer* parameter.

## Note

These enumerations are meant to be OR'd together to form a bit mask of events.

## Enumerator

***kLPI2C\_SlaveAddressMatchEvent*** Received the slave address after a start or repeated start.

***kLPI2C\_SlaveTransmitEvent*** Callback is requested to provide data to transmit (slave-transmitter role).

***kLPI2C\_SlaveReceiveEvent*** Callback is requested to provide a buffer in which to place received data (slave-receiver role).

***kLPI2C\_SlaveTransmitAckEvent*** Callback needs to either transmit an ACK or NACK.

***kLPI2C\_SlaveRepeatedStartEvent*** A repeated start was detected.

***kLPI2C\_SlaveCompletionEvent*** A stop was detected, completing the transfer.

***kLPI2C\_SlaveAllEvents*** Bit mask of all available events.

## 22.7.5 Function Documentation

### 22.7.5.1 void LPI2C\_SlaveGetDefaultConfig ( lpi2c\_slave\_config\_t \* *slaveConfig* )

This function provides the following default configuration for the LPI2C slave peripheral:

```
* slaveConfig->enableSlave = true;
* slaveConfig->address0 = 0U;
* slaveConfig->address1 = 0U;
* slaveConfig->addressMatchMode = kLPI2C_MatchAddress0;
* slaveConfig->filterDozeEnable = true;
* slaveConfig->filterEnable = true;
* slaveConfig->enableGeneralCall = false;
* slaveConfig->sclStall.enableAck = false;
* slaveConfig->sclStall.enableTx = true;
* slaveConfig->sclStall.enableRx = true;
* slaveConfig->sclStall.enableAddress = true;
* slaveConfig->ignoreAck = false;
* slaveConfig->enableReceivedAddressRead = false;
* slaveConfig->sdaGlitchFilterWidth_ns = 0;
* slaveConfig->sclGlitchFilterWidth_ns = 0;
* slaveConfig->dataValidDelay_ns = 0;
* slaveConfig->clockHoldTime_ns = 0;
*
```

After calling this function, override any settings to customize the configuration, prior to initializing the master driver with [LPI2C\\_SlaveInit\(\)](#). Be sure to override at least the *address0* member of the configuration structure with the desired slave address.

Parameters

|     |                    |                                                                                                                      |
|-----|--------------------|----------------------------------------------------------------------------------------------------------------------|
| out | <i>slaveConfig</i> | User provided configuration structure that is set to default values. Refer to <a href="#">lpi2c_slave_config_t</a> . |
|-----|--------------------|----------------------------------------------------------------------------------------------------------------------|

### 22.7.5.2 void LPI2C\_SlaveInit ( LPI2C\_Type \* *base*, const lpi2c\_slave\_config\_t \* *slaveConfig*, uint32\_t *sourceClock\_Hz* )

This function enables the peripheral clock and initializes the LPI2C slave peripheral as described by the user provided configuration.

## Parameters

|                       |                                                                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>           | The LPI2C peripheral base address.                                                                                                        |
| <i>slaveConfig</i>    | User provided peripheral configuration. Use <a href="#">LPI2C_SlaveGetDefaultConfig()</a> to get a set of defaults that you can override. |
| <i>sourceClock_Hz</i> | Frequency in Hertz of the LPI2C functional clock. Used to calculate the filter widths, data valid delay, and clock hold time.             |

**22.7.5.3 void LPI2C\_SlaveDeinit ( LPI2C\_Type \* *base* )**

This function disables the LPI2C slave peripheral and gates the clock. It also performs a software reset to restore the peripheral to reset conditions.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

**22.7.5.4 static void LPI2C\_SlaveReset ( LPI2C\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

**22.7.5.5 static void LPI2C\_SlaveEnable ( LPI2C\_Type \* *base*, bool *enable* ) [inline], [static]**

## Parameters

|               |                                                                       |
|---------------|-----------------------------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.                                    |
| <i>enable</i> | Pass true to enable or false to disable the specified LPI2C as slave. |

**22.7.5.6 static uint32\_t LPI2C\_SlaveGetStatusFlags ( LPI2C\_Type \* *base* ) [inline], [static]**

A bit mask with the state of all LPI2C slave status flags is returned. For each flag, the corresponding bit in the return value is set if the flag is asserted.



## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Returns

State of the status flags:

- 1: related status flag is set.
- 0: related status flag is not set.

## See Also

[\\_lpi2c\\_slave\\_flags](#)

### 22.7.5.7 static void LPI2C\_SlaveClearStatusFlags ( LPI2C\_Type \* *base*, uint32\_t *statusMask* ) [inline], [static]

The following status register flags can be cleared:

- [kLPI2C\\_SlaveRepeatedStartDetectFlag](#)
- [kLPI2C\\_SlaveStopDetectFlag](#)
- [kLPI2C\\_SlaveBitErrFlag](#)
- [kLPI2C\\_SlaveFifoErrFlag](#)

Attempts to clear other flags has no effect.

## Parameters

|                   |                                                                                                                                                                                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>       | The LPI2C peripheral base address.                                                                                                                                                                                                  |
| <i>statusMask</i> | A bitmask of status flags that are to be cleared. The mask is composed of <a href="#">_lpi2c_slave_flags</a> enumerators OR'd together. You may pass the result of a previous call to <a href="#">LPI2C_SlaveGetStatusFlags()</a> . |

## See Also

[\\_lpi2c\\_slave\\_flags](#).

### 22.7.5.8 static void LPI2C\_SlaveEnableInterrupts ( LPI2C\_Type \* *base*, uint32\_t *interruptMask* ) [inline], [static]

All flags except [kLPI2C\\_SlaveBusyFlag](#) and [kLPI2C\\_SlaveBusBusyFlag](#) can be enabled as interrupts.

## Parameters

|                      |                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>          | The LPI2C peripheral base address.                                                                                                                   |
| <i>interruptMask</i> | Bit mask of interrupts to enable. See <a href="#">_lpi2c_slave_flags</a> for the set of constants that should be OR'd together to form the bit mask. |

### 22.7.5.9 static void LPI2C\_SlaveDisableInterrupts ( LPI2C\_Type \* *base*, uint32\_t *interruptMask* ) [inline], [static]

All flags except [kLPI2C\\_SlaveBusyFlag](#) and [kLPI2C\\_SlaveBusBusyFlag](#) can be enabled as interrupts.

## Parameters

|                      |                                                                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>          | The LPI2C peripheral base address.                                                                                                                    |
| <i>interruptMask</i> | Bit mask of interrupts to disable. See <a href="#">_lpi2c_slave_flags</a> for the set of constants that should be OR'd together to form the bit mask. |

### 22.7.5.10 static uint32\_t LPI2C\_SlaveGetEnabledInterrupts ( LPI2C\_Type \* *base* ) [inline], [static]

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Returns

A bitmask composed of [\\_lpi2c\\_slave\\_flags](#) enumerators OR'd together to indicate the set of enabled interrupts.

### 22.7.5.11 static void LPI2C\_SlaveEnableDMA ( LPI2C\_Type \* *base*, bool *enableAddressValid*, bool *enableRx*, bool *enableTx* ) [inline], [static]

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

|                            |                                                                                                                                                                    |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>enableAddress-Valid</i> | Enable flag for the address valid DMA request. Pass true for enable, false for disable. The address valid DMA request is shared with the receive data DMA request. |
| <i>enableRx</i>            | Enable flag for the receive data DMA request. Pass true for enable, false for disable.                                                                             |
| <i>enableTx</i>            | Enable flag for the transmit data DMA request. Pass true for enable, false for disable.                                                                            |

#### 22.7.5.12 static bool LPI2C\_SlaveGetBusIdleState ( LPI2C\_Type \* *base* ) [inline], [static]

Requires the slave mode to be enabled.

Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

Return values

|              |              |
|--------------|--------------|
| <i>true</i>  | Bus is busy. |
| <i>false</i> | Bus is idle. |

#### 22.7.5.13 static void LPI2C\_SlaveTransmitAck ( LPI2C\_Type \* *base*, bool *ackOrNack* ) [inline], [static]

Use this function to send an ACK or NAK when the [kLPI2C\\_SlaveTransmitAckFlag](#) is asserted. This only happens if you enable the `sclStall.enableAck` field of the [lpi2c\\_slave\\_config\\_t](#) configuration structure used to initialize the slave peripheral.

Parameters

|                  |                                          |
|------------------|------------------------------------------|
| <i>base</i>      | The LPI2C peripheral base address.       |
| <i>ackOrNack</i> | Pass true for an ACK or false for a NAK. |

#### 22.7.5.14 static void LPI2C\_SlaveEnableAckStall ( LPI2C\_Type \* *base*, bool *enable* ) [inline], [static]

When enables ACKSTALL, software can transmit either an ACK or NAK on the I2C bus in response to a byte from the master.

## Parameters

|               |                                                         |
|---------------|---------------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.                      |
| <i>enable</i> | True will enable ACKSTALL, false will disable ACKSTALL. |

### 22.7.5.15 static uint32\_t LPI2C\_SlaveGetReceivedAddress ( LPI2C\_Type \* *base* ) [inline], [static]

This function should only be called if the [kLPI2C\\_SlaveAddressValidFlag](#) is asserted.

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | The LPI2C peripheral base address. |
|-------------|------------------------------------|

## Returns

The 8-bit address matched by the LPI2C slave. Bit 0 contains the R/w direction bit, and the 7-bit slave address is in the upper 7 bits.

### 22.7.5.16 status\_t LPI2C\_SlaveSend ( LPI2C\_Type \* *base*, void \* *txBuff*, size\_t *txSize*, size\_t \* *actualTxSize* )

## Parameters

|     |                     |                                                    |
|-----|---------------------|----------------------------------------------------|
|     | <i>base</i>         | The LPI2C peripheral base address.                 |
|     | <i>txBuff</i>       | The pointer to the data to be transferred.         |
|     | <i>txSize</i>       | The length in bytes of the data to be transferred. |
| out | <i>actualTxSize</i> |                                                    |

## Returns

Error or success status returned by API.

### 22.7.5.17 status\_t LPI2C\_SlaveReceive ( LPI2C\_Type \* *base*, void \* *rxBuff*, size\_t *rxSize*, size\_t \* *actualRxSize* )

## Parameters

|     |                     |                                                    |
|-----|---------------------|----------------------------------------------------|
|     | <i>base</i>         | The LPI2C peripheral base address.                 |
|     | <i>rxBuff</i>       | The pointer to the data to be transferred.         |
|     | <i>rxSize</i>       | The length in bytes of the data to be transferred. |
| out | <i>actualRxSize</i> |                                                    |

## Returns

Error or success status returned by API.

**22.7.5.18 void LPI2C\_SlaveTransferCreateHandle ( LPI2C\_Type \* *base*,  
lpi2c\_slave\_handle\_t \* *handle*, lpi2c\_slave\_transfer\_callback\_t *callback*, void \*  
*userData* )**

The creation of a handle is for use with the non-blocking APIs. Once a handle is created, there is not a corresponding destroy handle. If the user wants to terminate a transfer, the [LPI2C\\_SlaveTransferAbort\(\)](#) API shall be called.

## Note

The function also enables the NVIC IRQ for the input LPI2C. Need to notice that on some SoCs the LPI2C IRQ is connected to INTMUX, in this case user needs to enable the associated INTMUX IRQ in application.

## Parameters

|     |                 |                                                              |
|-----|-----------------|--------------------------------------------------------------|
|     | <i>base</i>     | The LPI2C peripheral base address.                           |
| out | <i>handle</i>   | Pointer to the LPI2C slave driver handle.                    |
|     | <i>callback</i> | User provided pointer to the asynchronous callback function. |
|     | <i>userData</i> | User provided pointer to the application callback data.      |

**22.7.5.19 status\_t LPI2C\_SlaveTransferNonBlocking ( LPI2C\_Type \* *base*,  
lpi2c\_slave\_handle\_t \* *handle*, uint32\_t *eventMask* )**

Call this API after calling I2C\_SlaveInit() and [LPI2C\\_SlaveTransferCreateHandle\(\)](#) to start processing transactions driven by an I2C master. The slave monitors the I2C bus and pass events to the callback that was passed into the call to [LPI2C\\_SlaveTransferCreateHandle\(\)](#). The callback is always invoked from the interrupt context.

The set of events received by the callback is customizable. To do so, set the *eventMask* parameter to the OR'd combination of [lpi2c\\_slave\\_transfer\\_event\\_t](#) enumerators for the events you wish to receive. The

[kLPI2C\\_SlaveTransmitEvent](#) and [kLPI2C\\_SlaveReceiveEvent](#) events are always enabled and do not need to be included in the mask. Alternatively, you can pass 0 to get a default set of only the transmit and receive events that are always enabled. In addition, the [kLPI2C\\_SlaveAllEvents](#) constant is provided as a convenient way to enable all events.

## Parameters

|                  |                                                                                                                                                                                                                                                                                                        |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>      | The LPI2C peripheral base address.                                                                                                                                                                                                                                                                     |
| <i>handle</i>    | Pointer to <code>lpi2c_slave_handle_t</code> structure which stores the transfer state.                                                                                                                                                                                                                |
| <i>eventMask</i> | Bit mask formed by OR'ing together <a href="#">lpi2c_slave_transfer_event_t</a> enumerators to specify which events to send to the callback. Other accepted values are 0 to get a default set of only the transmit and receive events, and <a href="#">kLPI2C_SlaveAllEvents</a> to enable all events. |

## Return values

|                           |                                                           |
|---------------------------|-----------------------------------------------------------|
| <i>kStatus_Success</i>    | Slave transfers were successfully started.                |
| <i>kStatus_LPI2C_Busy</i> | Slave transfers have already been started on this handle. |

#### 22.7.5.20 `status_t LPI2C_SlaveTransferGetCount ( LPI2C_Type * base, lpi2c_slave_handle_t * handle, size_t * count )`

## Parameters

|     |               |                                                                                                       |
|-----|---------------|-------------------------------------------------------------------------------------------------------|
|     | <i>base</i>   | The LPI2C peripheral base address.                                                                    |
|     | <i>handle</i> | Pointer to <code>i2c_slave_handle_t</code> structure.                                                 |
| out | <i>count</i>  | Pointer to a value to hold the number of bytes transferred. May be NULL if the count is not required. |

## Return values

|                                     |  |
|-------------------------------------|--|
| <i>kStatus_Success</i>              |  |
| <i>kStatus_NoTransferInProgress</i> |  |

#### 22.7.5.21 `void LPI2C_SlaveTransferAbort ( LPI2C_Type * base, lpi2c_slave_handle_t * handle )`

## Note

This API could be called at any time to stop slave for handling the bus events.

## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.                                         |
| <i>handle</i> | Pointer to lpi2c_slave_handle_t structure which stores the transfer state. |

#### 22.7.5.22 void LPI2C\_SlaveTransferHandleIRQ ( LPI2C\_Type \* *base*, lpi2c\_slave\_handle\_t \* *handle* )

## Note

This function does not need to be called unless you are reimplementing the non blocking API's interrupt handler routines to add special functionality.

## Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.                                         |
| <i>handle</i> | Pointer to lpi2c_slave_handle_t structure which stores the transfer state. |

## 22.8 LPI2C Master DMA Driver

### 22.8.1 Overview

#### Data Structures

- struct [\\_lpi2c\\_master\\_edma\\_handle](#)  
*Driver handle for master DMA APIs. [More...](#)*

#### Typedefs

- typedef struct  
[\\_lpi2c\\_master\\_edma\\_handle](#) [lpi2c\\_master\\_edma\\_handle\\_t](#)  
*LPI2C master EDMA handle of the transfer.*
- typedef void(\* [lpi2c\\_master\\_edma\\_transfer\\_callback\\_t](#))(LPI2C\_Type \*base, [lpi2c\\_master\\_edma\\_handle\\_t](#) \*handle, [status\\_t](#) completionStatus, void \*userData)  
*Master DMA completion callback function pointer type.*

#### Master DMA

- void [LPI2C\\_MasterCreateEDMAHandle](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_edma\\_handle\\_t](#) \*handle, [edma\\_handle\\_t](#) \*rxDmaHandle, [edma\\_handle\\_t](#) \*txDmaHandle, [lpi2c\\_master\\_edma\\_transfer\\_callback\\_t](#) callback, void \*userData)  
*Create a new handle for the LPI2C master DMA APIs.*
- [status\\_t](#) [LPI2C\\_MasterTransferEDMA](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_edma\\_handle\\_t](#) \*handle, [lpi2c\\_master\\_transfer\\_t](#) \*transfer)  
*Performs a non-blocking DMA-based transaction on the I2C bus.*
- [status\\_t](#) [LPI2C\\_MasterTransferGetCountEDMA](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_edma\\_handle\\_t](#) \*handle, [size\\_t](#) \*count)  
*Returns number of bytes transferred so far.*
- [status\\_t](#) [LPI2C\\_MasterTransferAbortEDMA](#) (LPI2C\_Type \*base, [lpi2c\\_master\\_edma\\_handle\\_t](#) \*handle)  
*Terminates a non-blocking LPI2C master transmission early.*

### 22.8.2 Data Structure Documentation

#### 22.8.2.1 struct [\\_lpi2c\\_master\\_edma\\_handle](#)

Note

The contents of this structure are private and subject to change.

#### Data Fields

- LPI2C\_Type \* [base](#)



- *LPI2C base pointer.*
- bool **isBusy**  
*Transfer state machine current state.*
- uint8\_t **nbytes**  
*eDMA minor byte transfer count initially configured.*
- uint16\_t **commandBuffer** [10]  
*LPI2C command sequence.*
- **lpi2c\_master\_transfer\_t** **transfer**  
*Copy of the current transfer info.*
- **lpi2c\_master\_edma\_transfer\_callback\_t** **completionCallback**  
*Callback function pointer.*
- void \* **userData**  
*Application data passed to callback.*
- **edma\_handle\_t** \* **rx**  
*Handle for receive DMA channel.*
- **edma\_handle\_t** \* **tx**  
*Handle for transmit DMA channel.*
- **edma\_tcd\_t** **tcds** [3]  
*Software TCD.*

### Field Documentation

(1) **LPI2C\_Type\* \_lpi2c\_master\_edma\_handle::base**

(2) **bool \_lpi2c\_master\_edma\_handle::isBusy**

(3) **uint8\_t \_lpi2c\_master\_edma\_handle::nbytes**

(4) **uint16\_t \_lpi2c\_master\_edma\_handle::commandBuffer[10]**

When all 10 command words are used: Start&addr&write[1 word] + subaddr[4 words] + restart&addr&read[1 word] + receive&Size[4 words]

(5) **lpi2c\_master\_transfer\_t \_lpi2c\_master\_edma\_handle::transfer**

(6) **lpi2c\_master\_edma\_transfer\_callback\_t \_lpi2c\_master\_edma\_handle::completionCallback**

(7) **void\* \_lpi2c\_master\_edma\_handle::userData**

(8) **edma\_handle\_t\* \_lpi2c\_master\_edma\_handle::rx**

(9) **edma\_handle\_t\* \_lpi2c\_master\_edma\_handle::tx**

(10) **edma\_tcd\_t \_lpi2c\_master\_edma\_handle::tcds[3]**

Three are allocated to provide enough room to align to 32-bytes.

### 22.8.3 Typedef Documentation

**22.8.3.1** `typedef struct _lpi2c_master_edma_handle lpi2c_master_edma_handle_t`

**22.8.3.2** `typedef void(* lpi2c_master_edma_transfer_callback_t)(LPI2C_Type *base,  
lpi2c_master_edma_handle_t *handle, status_t completionStatus, void  
*userData)`

This callback is used only for the non-blocking master transfer API. Specify the callback you wish to use in the call to [LPI2C\\_MasterCreateEDMAHandle\(\)](#).

## Parameters

|                          |                                                                                |
|--------------------------|--------------------------------------------------------------------------------|
| <i>base</i>              | The LPI2C peripheral base address.                                             |
| <i>handle</i>            | Handle associated with the completed transfer.                                 |
| <i>completion-Status</i> | Either kStatus_Success or an error code describing how the transfer completed. |
| <i>userData</i>          | Arbitrary pointer-sized value passed from the application.                     |

## 22.8.4 Function Documentation

**22.8.4.1 void LPI2C\_MasterCreateEDMAHandle ( LPI2C\_Type \* *base*, lpi2c\_master\_edma\_handle\_t \* *handle*, edma\_handle\_t \* *rxDmaHandle*, edma\_handle\_t \* *txDmaHandle*, lpi2c\_master\_edma\_transfer\_callback\_t *callback*, void \* *userData* )**

The creation of a handle is for use with the DMA APIs. Once a handle is created, there is not a corresponding destroy handle. If the user wants to terminate a transfer, the [LPI2C\\_MasterTransferAbortEDMA\(\)](#) API shall be called.

For devices where the LPI2C send and receive DMA requests are OR'd together, the *txDmaHandle* parameter is ignored and may be set to NULL.

## Parameters

|     |                    |                                                                                           |
|-----|--------------------|-------------------------------------------------------------------------------------------|
|     | <i>base</i>        | The LPI2C peripheral base address.                                                        |
| out | <i>handle</i>      | Pointer to the LPI2C master driver handle.                                                |
|     | <i>rxDmaHandle</i> | Handle for the eDMA receive channel. Created by the user prior to calling this function.  |
|     | <i>txDmaHandle</i> | Handle for the eDMA transmit channel. Created by the user prior to calling this function. |
|     | <i>callback</i>    | User provided pointer to the asynchronous callback function.                              |
|     | <i>userData</i>    | User provided pointer to the application callback data.                                   |

**22.8.4.2 status\_t LPI2C\_MasterTransferEDMA ( LPI2C\_Type \* *base*, lpi2c\_master\_edma\_handle\_t \* *handle*, lpi2c\_master\_transfer\_t \* *transfer* )**

The callback specified when the *handle* was created is invoked when the transaction has completed.

## Parameters

|                 |                                            |
|-----------------|--------------------------------------------|
| <i>base</i>     | The LPI2C peripheral base address.         |
| <i>handle</i>   | Pointer to the LPI2C master driver handle. |
| <i>transfer</i> | The pointer to the transfer descriptor.    |

## Return values

|                           |                                                                                                          |
|---------------------------|----------------------------------------------------------------------------------------------------------|
| <i>kStatus_Success</i>    | The transaction was started successfully.                                                                |
| <i>kStatus_LPI2C_Busy</i> | Either another master is currently utilizing the bus, or another DMA transaction is already in progress. |

#### 22.8.4.3 **status\_t LPI2C\_MasterTransferGetCountEDMA ( LPI2C\_Type \* *base*, lpi2c\_master\_edma\_handle\_t \* *handle*, size\_t \* *count* )**

## Parameters

|     |               |                                                                     |
|-----|---------------|---------------------------------------------------------------------|
|     | <i>base</i>   | The LPI2C peripheral base address.                                  |
|     | <i>handle</i> | Pointer to the LPI2C master driver handle.                          |
| out | <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction. |

## Return values

|                                     |                                                       |
|-------------------------------------|-------------------------------------------------------|
| <i>kStatus_Success</i>              |                                                       |
| <i>kStatus_NoTransferInProgress</i> | There is not a DMA transaction currently in progress. |

#### 22.8.4.4 **status\_t LPI2C\_MasterTransferAbortEDMA ( LPI2C\_Type \* *base*, lpi2c\_master\_edma\_handle\_t \* *handle* )**

## Note

It is not safe to call this function from an IRQ handler that has a higher priority than the eDMA peripheral's IRQ priority.

## Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | The LPI2C peripheral base address.         |
| <i>handle</i> | Pointer to the LPI2C master driver handle. |

## Return values

|                           |                                                       |
|---------------------------|-------------------------------------------------------|
| <i>kStatus_Success</i>    | A transaction was successfully aborted.               |
| <i>kStatus_LPI2C_Idle</i> | There is not a DMA transaction currently in progress. |

## **22.9 LPI2C FreeRTOS Driver**

## 22.10 LPI2C CMSIS Driver

This section describes the programming interface of the LPI2C Cortex Microcontroller Software Interface Standard (CMSIS) driver. And this driver defines generic peripheral driver interfaces for middleware making it reusable across a wide range of supported microcontroller devices. The API connects microcontroller peripherals with middleware that implements for example communication stacks, file systems, or graphic user interfaces. More information and usage method see <http://www.keil.com/pack/doc/cmsis/Driver/html/index.html>.

The LPI2C CMSIS driver includes transactional APIs.

Transactional APIs are transaction target high-level APIs. The transactional APIs can be used to enable the peripheral quickly and also in the application if the code size and performance of transactional APIs satisfy the requirements. If the code size and performance are critical requirements, see the transactional API implementation and write custom code accessing the hardware registers.

### 22.10.1 LPI2C CMSIS Driver

#### 22.10.1.1 Master Operation in interrupt transactional method

```
void I2C_MasterSignalEvent_t(uint32_t event)
{
 if (event == ARM_I2C_EVENT_TRANSFER_DONE)
 {
 g_MasterCompletionFlag = true;
 }
}

/*Init I2C0*/
Driver_I2C0.Initialize(I2C_MasterSignalEvent_t);

Driver_I2C0.PowerControl(ARM_POWER_FULL);

/*config transmit speed*/
Driver_I2C0.Control(ARM_I2C_BUS_SPEED, ARM_I2C_BUS_SPEED_STANDARD);

/*start transmit*/
Driver_I2C0.MasterTransmit(I2C_MASTER_SLAVE_ADDR, g_master_buff, I2C_DATA_LENGTH, false);

/* Wait for transfer completed. */
while (!g_MasterCompletionFlag)
{
}
g_MasterCompletionFlag = false;
```

#### 22.10.1.2 Master Operation in DMA transactional method

```
void I2C_MasterSignalEvent_t(uint32_t event)
{
 /* Transfer done */
 if (event == ARM_I2C_EVENT_TRANSFER_DONE)
 {
 g_MasterCompletionFlag = true;
 }
}

/* DMAMux init and EDMA init. */
DMAMUX_Init(EXAMPLE_LPI2C_DMAMUX_BASEADDR);
```

```

edma_config_t edmaConfig;
EDMA_GetDefaultConfig(&edmaConfig);
EDMA_Init(EXAMPLE_LPI2C_DMA_BASEADDR, &edmaConfig);

/*Init I2C0*/
Driver_I2C0.Initialize(I2C_MasterSignalEvent_t);

Driver_I2C0.PowerControl(ARM_POWER_FULL);

/*config transmit speed*/
Driver_I2C0.Control(ARM_I2C_BUS_SPEED, ARM_I2C_BUS_SPEED_STANDARD);

/*start transfer*/
Driver_I2C0.MasterReceive(I2C_MASTER_SLAVE_ADDR, g_master_buff, I2C_DATA_LENGTH, false);

/* Wait for transfer completed. */
while (!g_MasterCompletionFlag)
{
}
g_MasterCompletionFlag = false;

```

### 22.10.1.3 Slave Operation in interrupt transactional method

```

void I2C_SlaveSignalEvent_t(uint32_t event)
{
 /* Transfer done */
 if (event == ARM_I2C_EVENT_TRANSFER_DONE)
 {
 g_SlaveCompletionFlag = true;
 }
}

/*Init I2C1*/
Driver_I2C1.Initialize(I2C_SlaveSignalEvent_t);

Driver_I2C1.PowerControl(ARM_POWER_FULL);

/*config slave addr*/
Driver_I2C1.Control(ARM_I2C_OWN_ADDRESS, I2C_MASTER_SLAVE_ADDR);

/*start transfer*/
Driver_I2C1.SlaveReceive(g_slave_buff, I2C_DATA_LENGTH);

/* Wait for transfer completed. */
while (!g_SlaveCompletionFlag)
{
}
g_SlaveCompletionFlag = false;

```



## Chapter 23

# LPIT: Low-Power Interrupt Timer

### 23.1 Overview

The MCUXpresso SDK provides a driver for the Low-Power Interrupt Timer (LPIT) of MCUXpresso SDK devices.

### 23.2 Function groups

The LPIT driver supports operating the module as a time counter.

#### 23.2.1 Initialization and deinitialization

The function [LPIT\\_Init\(\)](#) initializes the LPIT with specified configurations. The function [LPIT\\_GetDefaultConfig\(\)](#) gets the default configurations. The initialization function configures the LPIT operation in doze mode and debug mode.

The function [LPIT\\_SetupChannel\(\)](#) configures the operation of each LPIT channel.

The function [LPIT\\_Deinit\(\)](#) disables the LPIT module and disables the module clock.

#### 23.2.2 Timer period Operations

The function [LPITR\\_SetTimerPeriod\(\)](#) sets the timer period in units of count. Timers begin counting down from the value set by this function until it reaches 0.

The function [LPIT\\_GetCurrentTimerCount\(\)](#) reads the current timer counting value. This function returns the real-time timer counting value, in a range from 0 to a timer period.

The timer period operation functions takes the count value in ticks. User can call the utility macros provided in `fsl_common.h` to convert to microseconds or milliseconds

#### 23.2.3 Start and Stop timer operations

The function [LPIT\\_StartTimer\(\)](#) starts the timer counting. After calling this function, the timer loads the period value set earlier via the [LPIT\\_SetPeriod\(\)](#) function and starts counting down to 0. When the timer reaches 0, it generates a trigger pulse and sets the timeout interrupt flag.

The function [LPIT\\_StopTimer\(\)](#) stops the timer counting.

### 23.2.4 Status

Provides functions to get and clear the LPIT status.

### 23.2.5 Interrupt

Provides functions to enable/disable LPIT interrupts and get current enabled interrupts.

## 23.3 Typical use case

### 23.3.1 LPIT tick example

Updates the LPIT period and toggles an LED periodically. Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/lpit`

## Data Structures

- struct [\\_lpit\\_chnl\\_params](#)  
*Structure to configure the channel timer. [More...](#)*
- struct [\\_lpit\\_config](#)  
*LPIT configuration structure. [More...](#)*

## Functions

- static void [LPIT\\_Reset](#) (LPIT\_Type \*base)  
*Performs a software reset on the LPIT module.*

## Driver version

- enum [\\_lpit\\_chnl](#) {  
  [kLPIT\\_Chnl\\_0](#) = 0U,  
  [kLPIT\\_Chnl\\_1](#),  
  [kLPIT\\_Chnl\\_2](#),  
  [kLPIT\\_Chnl\\_3](#) }  
*List of LPIT channels.*
- enum [\\_lpit\\_timer\\_modes](#) {  
  [kLPIT\\_PeriodicCounter](#) = 0U,  
  [kLPIT\\_DualPeriodicCounter](#),  
  [kLPIT\\_TriggerAccumulator](#),  
  [kLPIT\\_InputCapture](#) }  
*Mode options available for the LPIT timer.*
- enum [\\_lpit\\_trigger\\_select](#) {

```

kLPIT_Trigger_TimerChn0 = 0U,
kLPIT_Trigger_TimerChn1,
kLPIT_Trigger_TimerChn2,
kLPIT_Trigger_TimerChn3,
kLPIT_Trigger_TimerChn4,
kLPIT_Trigger_TimerChn5,
kLPIT_Trigger_TimerChn6,
kLPIT_Trigger_TimerChn7,
kLPIT_Trigger_TimerChn8,
kLPIT_Trigger_TimerChn9,
kLPIT_Trigger_TimerChn10,
kLPIT_Trigger_TimerChn11,
kLPIT_Trigger_TimerChn12,
kLPIT_Trigger_TimerChn13,
kLPIT_Trigger_TimerChn14,
kLPIT_Trigger_TimerChn15 }

```

*Trigger options available.*

- enum `_lpit_trigger_source` {  
`kLPIT_TriggerSource_External` = 0U,  
`kLPIT_TriggerSource_Internal` }  
*Trigger source options available.*
- enum `_lpit_interrupt_enable` {  
`kLPIT_Channel0TimerInterruptEnable` = (1U << 0),  
`kLPIT_Channel1TimerInterruptEnable` = (1U << 1),  
`kLPIT_Channel2TimerInterruptEnable` = (1U << 2),  
`kLPIT_Channel3TimerInterruptEnable` = (1U << 3) }  
*List of LPIT interrupts.*
- enum `_lpit_status_flags` {  
`kLPIT_Channel0TimerFlag` = (1U << 0),  
`kLPIT_Channel1TimerFlag` = (1U << 1),  
`kLPIT_Channel2TimerFlag` = (1U << 2),  
`kLPIT_Channel3TimerFlag` = (1U << 3) }  
*List of LPIT status flags.*
- typedef enum `_lpit_chnl` `lpit_chnl_t`  
*List of LPIT channels.*
- typedef enum `_lpit_timer_modes` `lpit_timer_modes_t`  
*Mode options available for the LPIT timer.*
- typedef enum `_lpit_trigger_select` `lpit_trigger_select_t`  
*Trigger options available.*
- typedef enum `_lpit_trigger_source` `lpit_trigger_source_t`  
*Trigger source options available.*
- typedef enum `_lpit_interrupt_enable` `lpit_interrupt_enable_t`  
*List of LPIT interrupts.*
- typedef enum `_lpit_status_flags` `lpit_status_flags_t`  
*List of LPIT status flags.*
- typedef struct `_lpit_chnl_params` `lpit_chnl_params_t`  
*Structure to configure the channel timer.*

- typedef struct `_lpit_config lpit_config_t`  
*LPIT configuration structure.*
- #define `FSL_LPIT_DRIVER_VERSION` (`MAKE_VERSION(2, 1, 1)`)  
*Version 2.1.1.*

## Initialization and deinitialization

- void `LPIT_Init` (`LPIT_Type *base`, const `lpit_config_t *config`)  
*Ungates the LPIT clock and configures the peripheral for a basic operation.*
- void `LPIT_Deinit` (`LPIT_Type *base`)  
*Disables the module and gates the LPIT clock.*
- void `LPIT_GetDefaultConfig` (`lpit_config_t *config`)  
*Fills in the LPIT configuration structure with default settings.*
- `status_t LPIT_SetupChannel` (`LPIT_Type *base`, `lpit_chnl_t channel`, const `lpit_chnl_params_t *chnlSetup`)  
*Sets up an LPIT channel based on the user's preference.*

## Interrupt Interface

- static void `LPIT_EnableInterrupts` (`LPIT_Type *base`, `uint32_t mask`)  
*Enables the selected PIT interrupts.*
- static void `LPIT_DisableInterrupts` (`LPIT_Type *base`, `uint32_t mask`)  
*Disables the selected PIT interrupts.*
- static `uint32_t LPIT_GetEnabledInterrupts` (`LPIT_Type *base`)  
*Gets the enabled LPIT interrupts.*

## Status Interface

- static `uint32_t LPIT_GetStatusFlags` (`LPIT_Type *base`)  
*Gets the LPIT status flags.*
- static void `LPIT_ClearStatusFlags` (`LPIT_Type *base`, `uint32_t mask`)  
*Clears the LPIT status flags.*

## Read and Write the timer period

- static void `LPIT_SetTimerPeriod` (`LPIT_Type *base`, `lpit_chnl_t channel`, `uint32_t ticks`)  
*Sets the timer period in units of count.*
- static void `LPIT_SetTimerValue` (`LPIT_Type *base`, `lpit_chnl_t channel`, `uint32_t ticks`)  
*Sets the timer period in units of count.*
- static `uint32_t LPIT_GetCurrentTimerCount` (`LPIT_Type *base`, `lpit_chnl_t channel`)  
*Reads the current timer counting value.*

## Timer Start and Stop

- static void `LPIT_StartTimer` (`LPIT_Type *base`, `lpit_chnl_t channel`)  
*Starts the timer counting.*
- static void `LPIT_StopTimer` (`LPIT_Type *base`, `lpit_chnl_t channel`)  
*Stops the timer counting.*

## 23.4 Data Structure Documentation

### 23.4.1 struct \_lpit\_chnl\_params

#### Data Fields

- bool [chainChannel](#)  
*true: Timer chained to previous timer; false: Timer not chained*
- [lpit\\_timer\\_modes\\_t](#) timerMode  
*Timers mode of operation.*
- [lpit\\_trigger\\_select\\_t](#) triggerSelect  
*Trigger selection for the timer.*
- [lpit\\_trigger\\_source\\_t](#) triggerSource  
*Decides if we use external or internal trigger.*
- bool [enableReloadOnTrigger](#)  
*true: Timer reloads when a trigger is detected; false: No effect*
- bool [enableStopOnTimeout](#)  
*true: Timer will stop after timeout; false: does not stop after timeout*
- bool [enableStartOnTrigger](#)  
*true: Timer starts when a trigger is detected; false: decrement immediately*

#### Field Documentation

(1) [lpit\\_timer\\_modes\\_t](#) \_lpit\_chnl\_params::timerMode

(2) [lpit\\_trigger\\_source\\_t](#) \_lpit\_chnl\_params::triggerSource

### 23.4.2 struct \_lpit\_config

This structure holds the configuration settings for the LPIT peripheral. To initialize this structure to reasonable defaults, call the [LPIT\\_GetDefaultConfig\(\)](#) function and pass a pointer to the configuration structure instance.

The configuration structure can be made constant so as to reside in flash.

#### Data Fields

- bool [enableRunInDebug](#)  
*true: Timers run in debug mode; false: Timers stop in debug mode*
- bool [enableRunInDoze](#)  
*true: Timers run in doze mode; false: Timers stop in doze mode*

## 23.5 Typedef Documentation

### 23.5.1 typedef enum \_lpit\_chnl lpit\_chnl\_t

Note

Actual number of available channels is SoC-dependent

### 23.5.2 typedef enum \_lpit\_timer\_modes lpit\_timer\_modes\_t

### 23.5.3 typedef enum \_lpit\_trigger\_select lpit\_trigger\_select\_t

This is used for both internal and external trigger sources. The actual trigger options available is SoC-specific, user should refer to the reference manual.

### 23.5.4 typedef enum \_lpit\_interrupt\_enable lpit\_interrupt\_enable\_t

Note

Number of timer channels are SoC-specific. See the SoC Reference Manual.

### 23.5.5 typedef enum \_lpit\_status\_flags lpit\_status\_flags\_t

Note

Number of timer channels are SoC-specific. See the SoC Reference Manual.

### 23.5.6 typedef struct \_lpit\_chnl\_params lpit\_chnl\_params\_t

### 23.5.7 typedef struct \_lpit\_config lpit\_config\_t

This structure holds the configuration settings for the LPIT peripheral. To initialize this structure to reasonable defaults, call the [LPIT\\_GetDefaultConfig\(\)](#) function and pass a pointer to the configuration structure instance.

The configuration structure can be made constant so as to reside in flash.

## 23.6 Enumeration Type Documentation

### 23.6.1 enum \_lpit\_chnl

Note

Actual number of available channels is SoC-dependent

Enumerator

***kLPIT\_Chnl\_0*** LPIT channel number 0.  
***kLPIT\_Chnl\_1*** LPIT channel number 1.  
***kLPIT\_Chnl\_2*** LPIT channel number 2.  
***kLPIT\_Chnl\_3*** LPIT channel number 3.

### 23.6.2 enum \_lpit\_timer\_modes

Enumerator

***kLPIT\_PeriodicCounter*** Use the all 32-bits, counter loads and decrements to zero.  
***kLPIT\_DualPeriodicCounter*** Counter loads, lower 16-bits decrement to zero, then upper 16-bits decrement.  
***kLPIT\_TriggerAccumulator*** Counter loads on first trigger and decrements on each trigger.  
***kLPIT\_InputCapture*** Counter loads with 0xFFFFFFFF, decrements to zero. It stores the inverse of the current value when a input trigger is detected

### 23.6.3 enum \_lpit\_trigger\_select

This is used for both internal and external trigger sources. The actual trigger options available is SoC-specific, user should refer to the reference manual.

Enumerator

***kLPIT\_Trigger\_TimerChn0*** Channel 0 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn1*** Channel 1 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn2*** Channel 2 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn3*** Channel 3 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn4*** Channel 4 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn5*** Channel 5 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn6*** Channel 6 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn7*** Channel 7 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn8*** Channel 8 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn9*** Channel 9 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn10*** Channel 10 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn11*** Channel 11 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn12*** Channel 12 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn13*** Channel 13 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn14*** Channel 14 is selected as a trigger source.  
***kLPIT\_Trigger\_TimerChn15*** Channel 15 is selected as a trigger source.

### 23.6.4 enum \_lpit\_trigger\_source

Enumerator

*kLPIT\_TriggerSource\_External* Use external trigger input.  
*kLPIT\_TriggerSource\_Internal* Use internal trigger.

### 23.6.5 enum \_lpit\_interrupt\_enable

Note

Number of timer channels are SoC-specific. See the SoC Reference Manual.

Enumerator

*kLPIT\_Channel0TimerInterruptEnable* Channel 0 Timer interrupt.  
*kLPIT\_Channel1TimerInterruptEnable* Channel 1 Timer interrupt.  
*kLPIT\_Channel2TimerInterruptEnable* Channel 2 Timer interrupt.  
*kLPIT\_Channel3TimerInterruptEnable* Channel 3 Timer interrupt.

### 23.6.6 enum \_lpit\_status\_flags

Note

Number of timer channels are SoC-specific. See the SoC Reference Manual.

Enumerator

*kLPIT\_Channel0TimerFlag* Channel 0 Timer interrupt flag.  
*kLPIT\_Channel1TimerFlag* Channel 1 Timer interrupt flag.  
*kLPIT\_Channel2TimerFlag* Channel 2 Timer interrupt flag.  
*kLPIT\_Channel3TimerFlag* Channel 3 Timer interrupt flag.

## 23.7 Function Documentation

### 23.7.1 void LPIT\_Init ( LPIT\_Type \* *base*, const lpit\_config\_t \* *config* )

This function issues a software reset to reset all channels and registers except the Module Control register.

Note

This API should be called at the beginning of the application using the LPIT driver.



Parameters

|               |                                              |
|---------------|----------------------------------------------|
| <i>base</i>   | LPIT peripheral base address.                |
| <i>config</i> | Pointer to the user configuration structure. |

### 23.7.2 void LPIT\_Deinit ( LPIT\_Type \* *base* )

Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPIT peripheral base address. |
|-------------|-------------------------------|

### 23.7.3 void LPIT\_GetDefaultConfig ( lpit\_config\_t \* *config* )

The default values are:

```
* config->enableRunInDebug = false;
* config->enableRunInDoze = false;
*
```

Parameters

|               |                                              |
|---------------|----------------------------------------------|
| <i>config</i> | Pointer to the user configuration structure. |
|---------------|----------------------------------------------|

### 23.7.4 status\_t LPIT\_SetupChannel ( LPIT\_Type \* *base*, lpit\_chnl\_t *channel*, const lpit\_chnl\_params\_t \* *chnlSetup* )

This function sets up the operation mode to one of the options available in the enumeration [lpit\\_timer\\_modes\\_t](#). It sets the trigger source as either internal or external, trigger selection and the timers behaviour when a timeout occurs. It also chains the timer if a prior timer if requested by the user.

Parameters

|                |                                   |
|----------------|-----------------------------------|
| <i>base</i>    | LPIT peripheral base address.     |
| <i>channel</i> | Channel that is being configured. |

|                  |                           |
|------------------|---------------------------|
| <i>chnlSetup</i> | Configuration parameters. |
|------------------|---------------------------|

### 23.7.5 static void LPIT\_EnableInterrupts ( LPIT\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

Parameters

|             |                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | LPIT peripheral base address.                                                                                        |
| <i>mask</i> | The interrupts to enable. This is a logical OR of members of the enumeration <a href="#">lpit_interrupt_enable_t</a> |

### 23.7.6 static void LPIT\_DisableInterrupts ( LPIT\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

Parameters

|             |                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | LPIT peripheral base address.                                                                                        |
| <i>mask</i> | The interrupts to enable. This is a logical OR of members of the enumeration <a href="#">lpit_interrupt_enable_t</a> |

### 23.7.7 static uint32\_t LPIT\_GetEnabledInterrupts ( LPIT\_Type \* *base* ) [inline], [static]

Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPIT peripheral base address. |
|-------------|-------------------------------|

Returns

The enabled interrupts. This is the logical OR of members of the enumeration [lpit\\_interrupt\\_enable\\_t](#)

### 23.7.8 static uint32\_t LPIT\_GetStatusFlags ( LPIT\_Type \* *base* ) [inline], [static]

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPIT peripheral base address. |
|-------------|-------------------------------|

## Returns

The status flags. This is the logical OR of members of the enumeration [lpit\\_status\\_flags\\_t](#)

### 23.7.9 static void LPIT\_ClearStatusFlags ( LPIT\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                                                   |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | LPIT peripheral base address.                                                                                     |
| <i>mask</i> | The status flags to clear. This is a logical OR of members of the enumeration <a href="#">lpit_status_flags_t</a> |

### 23.7.10 static void LPIT\_SetTimerPeriod ( LPIT\_Type \* *base*, lpit\_chnl\_t *channel*, uint32\_t *ticks* ) [inline], [static]

Timers begin counting down from the value set by this function until it reaches 0, at which point it generates an interrupt and loads this register value again. Writing a new value to this register does not restart the timer. Instead, the value is loaded after the timer expires.

## Note

User can call the utility macros provided in `fsl_common.h` to convert to ticks.

## Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | LPIT peripheral base address.   |
| <i>channel</i> | Timer channel number.           |
| <i>ticks</i>   | Timer period in units of ticks. |

### 23.7.11 static void LPIT\_SetTimerValue ( LPIT\_Type \* *base*, lpit\_chnl\_t *channel*, uint32\_t *ticks* ) [inline], [static]

In the Dual 16-bit Periodic Counter mode, the counter will load and then the lower 16-bits will decrement down to zero, which will assert the output pre-trigger. The upper 16-bits will then decrement down to zero, which will negate the output pre-trigger and set the timer interrupt flag.

## Note

Set TVAL register to 0 or 1 is invalid in compare mode.

## Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | LPIT peripheral base address.   |
| <i>channel</i> | Timer channel number.           |
| <i>ticks</i>   | Timer period in units of ticks. |

### 23.7.12 static uint32\_t LPIT\_GetCurrentTimerCount ( LPIT\_Type \* *base*, lpit\_chnl\_t *channel* ) [inline], [static]

This function returns the real-time timer counting value, in a range from 0 to a timer period.

## Note

User can call the utility macros provided in fsl\_common.h to convert ticks to microseconds or milliseconds.

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | LPIT peripheral base address. |
| <i>channel</i> | Timer channel number.         |

## Returns

Current timer counting value in ticks.

### 23.7.13 static void LPIT\_StartTimer ( LPIT\_Type \* *base*, lpit\_chnl\_t *channel* ) [inline], [static]

After calling this function, timers load the period value and count down to 0. When the timer reaches 0, it generates a trigger pulse and sets the timeout interrupt flag.

## Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | LPIT peripheral base address. |
| <i>channel</i> | Timer channel number.         |

**23.7.14 static void LPIT\_StopTimer ( LPIT\_Type \* *base*, lpit\_chnl\_t *channel* )**  
**[inline], [static]**

Parameters

|                |                               |
|----------------|-------------------------------|
| <i>base</i>    | LPIT peripheral base address. |
| <i>channel</i> | Timer channel number.         |

**23.7.15 static void LPIT\_Reset ( LPIT\_Type \* *base* ) [inline], [static]**

This resets all channels and registers except the Module Control Register.

Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPIT peripheral base address. |
|-------------|-------------------------------|



## Chapter 24

# LPSPI: Low Power Serial Peripheral Interface

### 24.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Low Power Serial Peripheral Interface (LPSPI) module of MCUXpresso SDK devices.

### Modules

- [LPSPI Peripheral driver](#)

## 24.2 LPSPI Peripheral driver

### 24.2.1 Overview

This section describes the programming interface of the LPSPI Peripheral driver. The LPSPI driver configures LPSPI module, provides the functional and transactional interfaces to build the LPSPI application.

### 24.2.2 Function groups

#### 24.2.2.1 LPSPI Initialization and De-initialization

This function group initializes the default configuration structure for master and slave, initializes the LPSPI master with a master configuration, initializes the LPSPI slave with a slave configuration, and de-initializes the LPSPI module.

#### 24.2.2.2 LPSPI Basic Operation

This function group enables/disables the LPSPI module both interrupt and DMA, gets the data register address for the DMA transfer, sets master and slave, starts and stops the transfer, and so on.

#### 24.2.2.3 LPSPI Transfer Operation

This function group controls the transfer, master send/receive data, and slave send/receive data.

#### 24.2.2.4 LPSPI Status Operation

This function group gets/clears the LPSPI status.

#### 24.2.2.5 LPSPI Block Transfer Operation

This function group transfers a block of data, gets the transfer status, and aborts the transfer.

### 24.2.3 Typical use case

#### 24.2.3.1 Master Operation

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/lpspi

### 24.2.3.2 Slave Operation

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/lpspi

## Data Structures

- struct [\\_lpspi\\_master\\_config](#)  
LPSPI master configuration structure. [More...](#)
- struct [\\_lpspi\\_slave\\_config](#)  
LPSPI slave configuration structure. [More...](#)
- struct [\\_lpspi\\_transfer](#)  
LPSPI master/slave transfer structure. [More...](#)
- struct [\\_lpspi\\_master\\_handle](#)  
LPSPI master transfer handle structure used for transactional API. [More...](#)
- struct [\\_lpspi\\_slave\\_handle](#)  
LPSPI slave transfer handle structure used for transactional API. [More...](#)

## Macros

- #define [LPSPI\\_DUMMY\\_DATA](#) (0x00U)  
LPSPI dummy data if no Tx data.
- #define [SPI\\_RETRY\\_TIMES](#) 0U /\* Define to zero means keep waiting until the flag is assert/deassert. \*/  
Retry times for waiting flag.
- #define [LPSPI\\_MASTER\\_PCS\\_SHIFT](#) (4U)  
LPSPI master PCS shift macro , internal used.
- #define [LPSPI\\_MASTER\\_PCS\\_MASK](#) (0xF0U)  
LPSPI master PCS shift macro , internal used.
- #define [LPSPI\\_MASTER\\_WIDTH\\_SHIFT](#) (16U)  
LPSPI master width shift macro, internal used.
- #define [LPSPI\\_MASTER\\_WIDTH\\_MASK](#) (0x30000U)  
LPSPI master width shift mask, internal used.
- #define [LPSPI\\_SLAVE\\_PCS\\_SHIFT](#) (4U)  
LPSPI slave PCS shift macro , internal used.
- #define [LPSPI\\_SLAVE\\_PCS\\_MASK](#) (0xF0U)  
LPSPI slave PCS shift macro , internal used.

## Typedefs

- typedef enum  
[\\_lpspi\\_master\\_slave\\_mode](#) [lpspi\\_master\\_slave\\_mode\\_t](#)  
LPSPI master or slave mode configuration.
- typedef enum  
[\\_lpspi\\_which\\_pcs\\_config](#) [lpspi\\_which\\_pcs\\_t](#)  
LPSPI Peripheral Chip Select (PCS) configuration (which PCS to configure).



- typedef enum  
[\\_lpspi\\_pcs\\_polarity\\_config](#) [lpspi\\_pcs\\_polarity\\_config\\_t](#)  
*LPSPI Peripheral Chip Select (PCS) Polarity configuration.*
- typedef enum [\\_lpspi\\_clock\\_polarity](#) [lpspi\\_clock\\_polarity\\_t](#)  
*LPSPI clock polarity configuration.*
- typedef enum [\\_lpspi\\_clock\\_phase](#) [lpspi\\_clock\\_phase\\_t](#)  
*LPSPI clock phase configuration.*
- typedef enum [\\_lpspi\\_shift\\_direction](#) [lpspi\\_shift\\_direction\\_t](#)  
*LPSPI data shifter direction options.*
- typedef enum  
[\\_lpspi\\_host\\_request\\_select](#) [lpspi\\_host\\_request\\_select\\_t](#)  
*LPSPI Host Request select configuration.*
- typedef enum [\\_lpspi\\_match\\_config](#) [lpspi\\_match\\_config\\_t](#)  
*LPSPI Match configuration options.*
- typedef enum [\\_lpspi\\_pin\\_config](#) [lpspi\\_pin\\_config\\_t](#)  
*LPSPI pin (SDO and SDI) configuration.*
- typedef enum [\\_lpspi\\_data\\_out\\_config](#) [lpspi\\_data\\_out\\_config\\_t](#)  
*LPSPI data output configuration.*
- typedef enum  
[\\_lpspi\\_pcs\\_function\\_config](#) [lpspi\\_pcs\\_function\\_config\\_t](#)  
*LPSPI cs function configuration.*
- typedef enum [\\_lpspi\\_transfer\\_width](#) [lpspi\\_transfer\\_width\\_t](#)  
*LPSPI transfer width configuration.*
- typedef enum [\\_lpspi\\_delay\\_type](#) [lpspi\\_delay\\_type\\_t](#)  
*LPSPI delay type selection.*
- typedef struct [\\_lpspi\\_master\\_config](#) [lpspi\\_master\\_config\\_t](#)  
*LPSPI master configuration structure.*
- typedef struct [\\_lpspi\\_slave\\_config](#) [lpspi\\_slave\\_config\\_t](#)  
*LPSPI slave configuration structure.*
- typedef struct [\\_lpspi\\_master\\_handle](#) [lpspi\\_master\\_handle\\_t](#)  
*Forward declaration of the [\\_lpspi\\_master\\_handle](#) typedefs.*
- typedef struct [\\_lpspi\\_slave\\_handle](#) [lpspi\\_slave\\_handle\\_t](#)  
*Forward declaration of the [\\_lpspi\\_slave\\_handle](#) typedefs.*
- typedef void(\* [lpspi\\_master\\_transfer\\_callback\\_t](#) )(LPSPI\_Type \*base, [lpspi\\_master\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
*Master completion callback function pointer type.*
- typedef void(\* [lpspi\\_slave\\_transfer\\_callback\\_t](#) )(LPSPI\_Type \*base, [lpspi\\_slave\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
*Slave completion callback function pointer type.*
- typedef struct [\\_lpspi\\_transfer](#) [lpspi\\_transfer\\_t](#)  
*LPSPI master/slave transfer structure.*

## Enumerations

- enum {  
[kStatus\\_LPSPI\\_Busy](#) = MAKE\_STATUS(kStatusGroup\_LPSPI, 0),  
[kStatus\\_LPSPI\\_Error](#) = MAKE\_STATUS(kStatusGroup\_LPSPI, 1),  
[kStatus\\_LPSPI\\_Idle](#) = MAKE\_STATUS(kStatusGroup\_LPSPI, 2),  
[kStatus\\_LPSPI\\_OutOfRange](#) = MAKE\_STATUS(kStatusGroup\_LPSPI, 3),

`kStatus_LPSPI_Timeout = MAKE_STATUS(kStatusGroup_LPSPI, 4) }`

*Status for the LPSPI driver.*

- `enum _lpspi_flags {`  
`kLPSPI_TxDataRequestFlag = LPSPI_SR_TDF_MASK,`  
`kLPSPI_RxDataReadyFlag = LPSPI_SR_RDF_MASK,`  
`kLPSPI_WordCompleteFlag = LPSPI_SR_WCF_MASK,`  
`kLPSPI_FrameCompleteFlag = LPSPI_SR_FCF_MASK,`  
`kLPSPI_TransferCompleteFlag = LPSPI_SR_TCF_MASK,`  
`kLPSPI_TransmitErrorFlag = LPSPI_SR_TEF_MASK,`  
`kLPSPI_ReceiveErrorFlag = LPSPI_SR_REF_MASK,`  
`kLPSPI_DataMatchFlag = LPSPI_SR_DMF_MASK,`  
`kLPSPI_ModuleBusyFlag = LPSPI_SR_MBF_MASK,`  
`kLPSPI_AllStatusFlag }`
- LPSPI status flags in SPIx\_SR register.*
- `enum _lpspi_interrupt_enable {`  
`kLPSPI_TxInterruptEnable = LPSPI_IER_TDIE_MASK,`  
`kLPSPI_RxInterruptEnable = LPSPI_IER_RDIE_MASK,`  
`kLPSPI_WordCompleteInterruptEnable = LPSPI_IER_WCIE_MASK,`  
`kLPSPI_FrameCompleteInterruptEnable = LPSPI_IER_FCIE_MASK,`  
`kLPSPI_TransferCompleteInterruptEnable = LPSPI_IER_TCIE_MASK,`  
`kLPSPI_TransmitErrorInterruptEnable = LPSPI_IER_TEIE_MASK,`  
`kLPSPI_ReceiveErrorInterruptEnable = LPSPI_IER_REIE_MASK,`  
`kLPSPI_DataMatchInterruptEnable = LPSPI_IER_DMIE_MASK,`  
`kLPSPI_AllInterruptEnable }`
- LPSPI interrupt source.*
- `enum _lpspi_dma_enable {`  
`kLPSPI_TxDmaEnable = LPSPI_DER_TDDE_MASK,`  
`kLPSPI_RxDmaEnable = LPSPI_DER_RDDE_MASK }`
- LPSPI DMA source.*
- `enum _lpspi_master_slave_mode {`  
`kLPSPI_Master = 1U,`  
`kLPSPI_Slave = 0U }`
- LPSPI master or slave mode configuration.*
- `enum _lpspi_which_pcs_config {`  
`kLPSPI_Pcs0 = 0U,`  
`kLPSPI_Pcs1 = 1U,`  
`kLPSPI_Pcs2 = 2U,`  
`kLPSPI_Pcs3 = 3U }`
- LPSPI Peripheral Chip Select (PCS) configuration (which PCS to configure).*
- `enum _lpspi_pcs_polarity_config {`  
`kLPSPI_PcsActiveHigh = 1U,`  
`kLPSPI_PcsActiveLow = 0U }`
- LPSPI Peripheral Chip Select (PCS) Polarity configuration.*
- `enum _lpspi_pcs_polarity {`

- `kLPSPI_Pcs0ActiveLow = 1U << 0,`
- `kLPSPI_Pcs1ActiveLow = 1U << 1,`
- `kLPSPI_Pcs2ActiveLow = 1U << 2,`
- `kLPSPI_Pcs3ActiveLow = 1U << 3,`
- `kLPSPI_PcsAllActiveLow = 0xFU }`
- LPSPI Peripheral Chip Select (PCS) Polarity.*
- `enum _lpspi_clock_polarity {`
  - `kLPSPI_ClockPolarityActiveHigh = 0U,`
  - `kLPSPI_ClockPolarityActiveLow = 1U }`
- LPSPI clock polarity configuration.*
- `enum _lpspi_clock_phase {`
  - `kLPSPI_ClockPhaseFirstEdge = 0U,`
  - `kLPSPI_ClockPhaseSecondEdge = 1U }`
- LPSPI clock phase configuration.*
- `enum _lpspi_shift_direction {`
  - `kLPSPI_MsbFirst = 0U,`
  - `kLPSPI_LsbFirst = 1U }`
- LPSPI data shifter direction options.*
- `enum _lpspi_host_request_select {`
  - `kLPSPI_HostReqExtPin = 0U,`
  - `kLPSPI_HostReqInternalTrigger = 1U }`
- LPSPI Host Request select configuration.*
- `enum _lpspi_match_config {`
  - `kLPSI_MatchDisabled = 0x0U,`
  - `kLPSI_1stWordEqualsM0orM1 = 0x2U,`
  - `kLPSI_AnyWordEqualsM0orM1 = 0x3U,`
  - `kLPSI_1stWordEqualsM0and2ndWordEqualsM1 = 0x4U,`
  - `kLPSI_AnyWordEqualsM0andNxtWordEqualsM1 = 0x5U,`
  - `kLPSI_1stWordAndM1EqualsM0andM1 = 0x6U,`
  - `kLPSI_AnyWordAndM1EqualsM0andM1 = 0x7U }`
- LPSPI Match configuration options.*
- `enum _lpspi_pin_config {`
  - `kLPSPI_SdiInSdoOut = 0U,`
  - `kLPSPI_SdiInSdiOut = 1U,`
  - `kLPSPI_SdoInSdoOut = 2U,`
  - `kLPSPI_SdoInSdiOut = 3U }`
- LPSPI pin (SDO and SDI) configuration.*
- `enum _lpspi_data_out_config {`
  - `kLpspiDataOutRetained = 0U,`
  - `kLpspiDataOutTristate = 1U }`
- LPSPI data output configuration.*
- `enum _lpspi_pcs_function_config {`
  - `kLPSPI_PcsAsCs = 0U,`
  - `kLPSPI_PcsAsData = 1U }`
- LPSPI cs function configuration.*
- `enum _lpspi_transfer_width {`

```
kLPSPI_SingleBitXfer = 0U,
kLPSPI_TwoBitXfer = 1U,
kLPSPI_FourBitXfer = 2U }
```

*LPSPI transfer width configuration.*

- enum `_lpspi_delay_type` {  
    kLPSPI\_PcsToSck = 1U,  
    kLPSPI\_LastSckToPcs,  
    kLPSPI\_BetweenTransfer }

*LPSPI delay type selection.*

- enum `_lpspi_transfer_config_flag_for_master` {  
    kLPSPI\_MasterPcs0 = 0U << LPSPI\_MASTER\_PCS\_SHIFT,  
    kLPSPI\_MasterPcs1 = 1U << LPSPI\_MASTER\_PCS\_SHIFT,  
    kLPSPI\_MasterPcs2 = 2U << LPSPI\_MASTER\_PCS\_SHIFT,  
    kLPSPI\_MasterPcs3 = 3U << LPSPI\_MASTER\_PCS\_SHIFT,  
    kLPSPI\_MasterWidth1 = 0U << LPSPI\_MASTER\_WIDTH\_SHIFT,  
    kLPSPI\_MasterWidth2 = 1U << LPSPI\_MASTER\_WIDTH\_SHIFT,  
    kLPSPI\_MasterWidth4 = 2U << LPSPI\_MASTER\_WIDTH\_SHIFT,  
    kLPSPI\_MasterPcsContinuous = 1U << 20,  
    kLPSPI\_MasterByteSwap }

*Use this enumeration for LPSPi master transfer configFlags.*

- enum `_lpspi_transfer_config_flag_for_slave` {  
    kLPSPI\_SlavePcs0 = 0U << LPSPI\_SLAVE\_PCS\_SHIFT,  
    kLPSPI\_SlavePcs1 = 1U << LPSPI\_SLAVE\_PCS\_SHIFT,  
    kLPSPI\_SlavePcs2 = 2U << LPSPI\_SLAVE\_PCS\_SHIFT,  
    kLPSPI\_SlavePcs3 = 3U << LPSPI\_SLAVE\_PCS\_SHIFT,  
    kLPSPI\_SlaveByteSwap }

*Use this enumeration for LPSPi slave transfer configFlags.*

- enum `_lpspi_transfer_state` {  
    kLPSPI\_Idle = 0x0U,  
    kLPSPI\_Busy,  
    kLPSPI\_Error }

*LPSPI transfer state, which is used for LPSPi transactional API state machine.*

## Variables

- volatile uint8\_t `g_lpspiDummyData` []  
*Global variable for dummy data value setting.*

## Driver version

- #define `FSL_LPSPI_DRIVER_VERSION` (`MAKE_VERSION`(2, 6, 8))  
*LPSPi driver version.*

## Initialization and deinitialization

- void [LPSPI\\_MasterInit](#) (LPSPI\_Type \*base, const [lpspi\\_master\\_config\\_t](#) \*masterConfig, uint32\_t srcClock\_Hz)  
*Initializes the LPSPI master.*
- void [LPSPI\\_MasterGetDefaultConfig](#) ([lpspi\\_master\\_config\\_t](#) \*masterConfig)  
*Sets the [lpspi\\_master\\_config\\_t](#) structure to default values.*
- void [LPSPI\\_SlaveInit](#) (LPSPI\_Type \*base, const [lpspi\\_slave\\_config\\_t](#) \*slaveConfig)  
*LPSPI slave configuration.*
- void [LPSPI\\_SlaveGetDefaultConfig](#) ([lpspi\\_slave\\_config\\_t](#) \*slaveConfig)  
*Sets the [lpspi\\_slave\\_config\\_t](#) structure to default values.*
- void [LPSPI\\_Deinit](#) (LPSPI\_Type \*base)  
*De-initializes the LPSPI peripheral.*
- void [LPSPI\\_Reset](#) (LPSPI\_Type \*base)  
*Restores the LPSPI peripheral to reset state.*
- uint32\_t [LPSPI\\_GetInstance](#) (LPSPI\_Type \*base)  
*Get the LPSPI instance from peripheral base address.*
- static void [LPSPI\\_Enable](#) (LPSPI\_Type \*base, bool enable)  
*Enables the LPSPI peripheral and sets the MCR MDIS to 0.*

## Status

- static uint32\_t [LPSPI\\_GetStatusFlags](#) (LPSPI\_Type \*base)  
*Gets the LPSPI status flag state.*
- static uint8\_t [LPSPI\\_GetTxFifoSize](#) (LPSPI\_Type \*base)  
*Gets the LPSPI Tx FIFO size.*
- static uint8\_t [LPSPI\\_GetRxFifoSize](#) (LPSPI\_Type \*base)  
*Gets the LPSPI Rx FIFO size.*
- static uint32\_t [LPSPI\\_GetTxFifoCount](#) (LPSPI\_Type \*base)  
*Gets the LPSPI Tx FIFO count.*
- static uint32\_t [LPSPI\\_GetRxFifoCount](#) (LPSPI\_Type \*base)  
*Gets the LPSPI Rx FIFO count.*
- static void [LPSPI\\_ClearStatusFlags](#) (LPSPI\_Type \*base, uint32\_t statusFlags)  
*Clears the LPSPI status flag.*

## Interrupts

- static void [LPSPI\\_EnableInterrupts](#) (LPSPI\_Type \*base, uint32\_t mask)  
*Enables the LPSPI interrupts.*
- static void [LPSPI\\_DisableInterrupts](#) (LPSPI\_Type \*base, uint32\_t mask)  
*Disables the LPSPI interrupts.*

## DMA Control

- static void [LPSPI\\_EnableDMA](#) (LPSPI\_Type \*base, uint32\_t mask)  
*Enables the LPSPI DMA request.*
- static void [LPSPI\\_DisableDMA](#) (LPSPI\_Type \*base, uint32\_t mask)

- *Disables the LPSPI DMA request.*
- static uint32\_t [LPSPI\\_GetTxRegisterAddress](#) (LPSPI\_Type \*base)  
*Gets the LPSPI Transmit Data Register address for a DMA operation.*
- static uint32\_t [LPSPI\\_GetRxRegisterAddress](#) (LPSPI\_Type \*base)  
*Gets the LPSPI Receive Data Register address for a DMA operation.*

## Bus Operations

- bool [LPSPI\\_CheckTransferArgument](#) (LPSPI\_Type \*base, [lpspi\\_transfer\\_t](#) \*transfer, bool isEdma)  
*Check the argument for transfer .*
- static void [LPSPI\\_SetMasterSlaveMode](#) (LPSPI\_Type \*base, [lpspi\\_master\\_slave\\_mode\\_t](#) mode)  
*Configures the LPSPI for either master or slave.*
- static void [LPSPI\\_SelectTransferPCS](#) (LPSPI\_Type \*base, [lpspi\\_which\\_pcs\\_t](#) select)  
*Configures the peripheral chip select used for the transfer.*
- static void [LPSPI\\_SetPCSContinuous](#) (LPSPI\_Type \*base, bool IsContinuous)  
*Set the PCS signal to continuous or uncontinuous mode.*
- static bool [LPSPI\\_IsMaster](#) (LPSPI\_Type \*base)  
*Returns whether the LPSPI module is in master mode.*
- static void [LPSPI\\_FlushFifo](#) (LPSPI\_Type \*base, bool flushTxFifo, bool flushRxFifo)  
*Flushes the LPSPI FIFOs.*
- static void [LPSPI\\_SetFifoWatermarks](#) (LPSPI\_Type \*base, uint32\_t txWater, uint32\_t rxWater)  
*Sets the transmit and receive FIFO watermark values.*
- static void [LPSPI\\_SetAllPcsPolarity](#) (LPSPI\_Type \*base, uint32\_t mask)  
*Configures all LPSPI peripheral chip select polarities simultaneously.*
- static void [LPSPI\\_SetFrameSize](#) (LPSPI\_Type \*base, uint32\_t frameSize)  
*Configures the frame size.*
- uint32\_t [LPSPI\\_MasterSetBaudRate](#) (LPSPI\_Type \*base, uint32\_t baudRate\_Bps, uint32\_t srcClock\_Hz, uint32\_t \*tcrPrescaleValue)  
*Sets the LPSPI baud rate in bits per second.*
- void [LPSPI\\_MasterSetDelayScaler](#) (LPSPI\_Type \*base, uint32\_t scaler, [lpspi\\_delay\\_type\\_t](#) whichDelay)  
*Manually configures a specific LPSPI delay parameter (module must be disabled to change the delay values).*
- uint32\_t [LPSPI\\_MasterSetDelayTimes](#) (LPSPI\_Type \*base, uint32\_t delayTimeInNanoSec, [lpspi\\_delay\\_type\\_t](#) whichDelay, uint32\_t srcClock\_Hz)  
*Calculates the delay based on the desired delay input in nanoseconds (module must be disabled to change the delay values).*
- static void [LPSPI\\_WriteData](#) (LPSPI\_Type \*base, uint32\_t data)  
*Writes data into the transmit data buffer.*
- static uint32\_t [LPSPI\\_ReadData](#) (LPSPI\_Type \*base)  
*Reads data from the data buffer.*
- void [LPSPI\\_SetDummyData](#) (LPSPI\_Type \*base, uint8\_t dummyData)  
*Set up the dummy data.*

## Transactional

- void [LPSPI\\_MasterTransferCreateHandle](#) (LPSPI\_Type \*base, [lpspi\\_master\\_handle\\_t](#) \*handle, [lpspi\\_master\\_transfer\\_callback\\_t](#) callback, void \*userData)



- *Initializes the LPSPI master handle.*
- [status\\_t LPSPI\\_MasterTransferBlocking](#) (LPSPI\_Type \*base, [lpspi\\_transfer\\_t](#) \*transfer)  
*LPSPI master transfer data using a polling method.*
- [status\\_t LPSPI\\_MasterTransferNonBlocking](#) (LPSPI\_Type \*base, [lpspi\\_master\\_handle\\_t](#) \*handle, [lpspi\\_transfer\\_t](#) \*transfer)  
*LPSPI master transfer data using an interrupt method.*
- [status\\_t LPSPI\\_MasterTransferGetCount](#) (LPSPI\_Type \*base, [lpspi\\_master\\_handle\\_t](#) \*handle, [size\\_t](#) \*count)  
*Gets the master transfer remaining bytes.*
- [void LPSPI\\_MasterTransferAbort](#) (LPSPI\_Type \*base, [lpspi\\_master\\_handle\\_t](#) \*handle)  
*LPSPI master abort transfer which uses an interrupt method.*
- [void LPSPI\\_MasterTransferHandleIRQ](#) (LPSPI\_Type \*base, [lpspi\\_master\\_handle\\_t](#) \*handle)  
*LPSPI Master IRQ handler function.*
- [void LPSPI\\_SlaveTransferCreateHandle](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_handle\\_t](#) \*handle, [lpspi\\_slave\\_transfer\\_callback\\_t](#) callback, void \*userData)  
*Initializes the LPSPI slave handle.*
- [status\\_t LPSPI\\_SlaveTransferNonBlocking](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_handle\\_t](#) \*handle, [lpspi\\_transfer\\_t](#) \*transfer)  
*LPSPI slave transfer data using an interrupt method.*
- [status\\_t LPSPI\\_SlaveTransferGetCount](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_handle\\_t](#) \*handle, [size\\_t](#) \*count)  
*Gets the slave transfer remaining bytes.*
- [void LPSPI\\_SlaveTransferAbort](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_handle\\_t](#) \*handle)  
*LPSPI slave aborts a transfer which uses an interrupt method.*
- [void LPSPI\\_SlaveTransferHandleIRQ](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_handle\\_t](#) \*handle)  
*LPSPI Slave IRQ handler function.*
- [bool LPSPI\\_WaitTxFifoEmpty](#) (LPSPI\_Type \*base)  
*Wait for tx FIFO to be empty.*

## 24.2.4 Data Structure Documentation

### 24.2.4.1 struct [\\_lpspi\\_master\\_config](#)

#### Data Fields

- [uint32\\_t baudRate](#)  
*Baud Rate for LPSPI.*
- [uint32\\_t bitsPerFrame](#)  
*Bits per frame, minimum 8, maximum 4096.*
- [lpspi\\_clock\\_polarity\\_t cpol](#)  
*Clock polarity.*
- [lpspi\\_clock\\_phase\\_t cpha](#)  
*Clock phase.*
- [lpspi\\_shift\\_direction\\_t direction](#)  
*MSB or LSB data shift direction.*
- [uint32\\_t pcsToSckDelayInNanoSec](#)  
*PCS to SCK delay time in nanoseconds, setting to 0 sets the minimum delay.*
- [uint32\\_t lastSckToPcsDelayInNanoSec](#)  
*Last SCK to PCS delay time in nanoseconds, setting to 0 sets the minimum delay.*

- `uint32_t betweenTransferDelayInNanoSec`  
After the SCK delay time with nanoseconds, setting to 0 sets the minimum delay.
- `lpspi_which_pcs_t whichPcs`  
*Desired Peripheral Chip Select (PCS).*
- `lpspi_pcs_polarity_config_t pcsActiveHighOrLow`  
*Desired PCS active high or low.*
- `lpspi_pin_config_t pinCfg`  
*Configures which pins are used for input and output data during single bit transfers.*
- `lpspi_pcs_function_config_t pcsFunc`  
*Configures cs pins function.*
- `lpspi_data_out_config_t dataOutConfig`  
*Configures if the output data is tristated between accesses (LPSPI\_PCS is negated).*
- `bool enableInputDelay`  
*Enable master to sample the input data on a delayed SCK.*

### Field Documentation

- (1) `uint32_t _lpspi_master_config::baudRate`
- (2) `uint32_t _lpspi_master_config::bitsPerFrame`
- (3) `lpspi_clock_polarity_t _lpspi_master_config::cpol`
- (4) `lpspi_clock_phase_t _lpspi_master_config::cpha`
- (5) `lpspi_shift_direction_t _lpspi_master_config::direction`
- (6) `uint32_t _lpspi_master_config::pcsToSckDelayInNanoSec`

It sets the boundary value if out of range.

- (7) `uint32_t _lpspi_master_config::lastSckToPcsDelayInNanoSec`

It sets the boundary value if out of range.

- (8) `uint32_t _lpspi_master_config::betweenTransferDelayInNanoSec`

It sets the boundary value if out of range.

- (9) `lpspi_which_pcs_t _lpspi_master_config::whichPcs`

- (10) `lpspi_pin_config_t _lpspi_master_config::pinCfg`

- (11) `lpspi_pcs_function_config_t _lpspi_master_config::pcsFunc`

- (12) `lpspi_data_out_config_t _lpspi_master_config::dataOutConfig`

- (13) `bool _lpspi_master_config::enableInputDelay`

This can help improve slave setup time. Refer to device data sheet for specific time length.



#### 24.2.4.2 struct \_lpspi\_slave\_config

##### Data Fields

- uint32\_t [bitsPerFrame](#)  
*Bits per frame, minimum 8, maximum 4096.*
- [lpspi\\_clock\\_polarity\\_t](#) [cpol](#)  
*Clock polarity.*
- [lpspi\\_clock\\_phase\\_t](#) [cpha](#)  
*Clock phase.*
- [lpspi\\_shift\\_direction\\_t](#) [direction](#)  
*MSB or LSB data shift direction.*
- [lpspi\\_which\\_pcs\\_t](#) [whichPcs](#)  
*Desired Peripheral Chip Select (pcs)*
- [lpspi\\_pcs\\_polarity\\_config\\_t](#) [pcsActiveHighOrLow](#)  
*Desired PCS active high or low.*
- [lpspi\\_pin\\_config\\_t](#) [pinCfg](#)  
*Configures which pins are used for input and output data during single bit transfers.*
- [lpspi\\_data\\_out\\_config\\_t](#) [dataOutConfig](#)  
*Configures if the output data is tristated between accesses (LPSPI\_PCS is negated).*

##### Field Documentation

- (1) [uint32\\_t \\_lpspi\\_slave\\_config::bitsPerFrame](#)
- (2) [lpspi\\_clock\\_polarity\\_t \\_lpspi\\_slave\\_config::cpol](#)
- (3) [lpspi\\_clock\\_phase\\_t \\_lpspi\\_slave\\_config::cpha](#)
- (4) [lpspi\\_shift\\_direction\\_t \\_lpspi\\_slave\\_config::direction](#)
- (5) [lpspi\\_pin\\_config\\_t \\_lpspi\\_slave\\_config::pinCfg](#)
- (6) [lpspi\\_data\\_out\\_config\\_t \\_lpspi\\_slave\\_config::dataOutConfig](#)

#### 24.2.4.3 struct \_lpspi\_transfer

##### Data Fields

- const uint8\_t \* [txData](#)  
*Send buffer.*
- uint8\_t \* [rxData](#)  
*Receive buffer.*
- volatile size\_t [dataSize](#)  
*Transfer bytes.*
- uint32\_t [configFlags](#)  
*Transfer transfer configuration flags.*

## Field Documentation

- (1) `const uint8_t* _lpspi_transfer::txData`
- (2) `uint8_t* _lpspi_transfer::rxData`
- (3) `volatile size_t _lpspi_transfer::dataSize`
- (4) `uint32_t _lpspi_transfer::configFlags`

Set from `_lpspi_transfer_config_flag_for_master` if the transfer is used for master or `_lpspi_transfer_config_flag_for_slave` enumeration if the transfer is used for slave.

### 24.2.4.4 struct \_lpspi\_master\_handle

#### Data Fields

- volatile bool `isPcsContinuous`  
*Is PCS continuous in transfer.*
- volatile bool `writeTcrInIsr`  
*A flag that whether should write TCR in ISR.*
- volatile bool `isByteSwap`  
*A flag that whether should byte swap.*
- volatile bool `isTxMask`  
*A flag that whether TCR[TXMSK] is set.*
- volatile uint16\_t `bytesPerFrame`  
*Number of bytes in each frame.*
- volatile uint8\_t `fifoSize`  
*FIFO dataSize.*
- volatile uint8\_t `rxWatermark`  
*Rx watermark.*
- volatile uint8\_t `bytesEachWrite`  
*Bytes for each write TDR.*
- volatile uint8\_t `bytesEachRead`  
*Bytes for each read RDR.*
- const uint8\_t \*volatile `txData`  
*Send buffer.*
- uint8\_t \*volatile `rxData`  
*Receive buffer.*
- volatile size\_t `txRemainingByteCount`  
*Number of bytes remaining to send.*
- volatile size\_t `rxRemainingByteCount`  
*Number of bytes remaining to receive.*
- volatile uint32\_t `writeRegRemainingTimes`  
*Write TDR register remaining times.*
- volatile uint32\_t `readRegRemainingTimes`  
*Read RDR register remaining times.*
- uint32\_t `totalByteCount`  
*Number of transfer bytes.*
- uint32\_t `txBuffIfNull`

- *Used if the txData is NULL.*
- volatile uint8\_t [state](#)  
*LPSPI transfer state , \_lpspi\_transfer\_state.*
- [lpspi\\_master\\_transfer\\_callback\\_t](#) *callback*  
*Completion callback.*
- void \* [userData](#)  
*Callback user data.*

## Field Documentation

- (1) volatile bool [\\_lpspi\\_master\\_handle::isPcsContinuous](#)
- (2) volatile bool [\\_lpspi\\_master\\_handle::writeTcrInlSr](#)
- (3) volatile bool [\\_lpspi\\_master\\_handle::isByteSwap](#)
- (4) volatile bool [\\_lpspi\\_master\\_handle::isTxMask](#)
- (5) volatile uint8\_t [\\_lpspi\\_master\\_handle::fifoSize](#)
- (6) volatile uint8\_t [\\_lpspi\\_master\\_handle::rxWatermark](#)
- (7) volatile uint8\_t [\\_lpspi\\_master\\_handle::bytesEachWrite](#)
- (8) volatile uint8\_t [\\_lpspi\\_master\\_handle::bytesEachRead](#)
- (9) const uint8\_t\* volatile [\\_lpspi\\_master\\_handle::txData](#)
- (10) uint8\_t\* volatile [\\_lpspi\\_master\\_handle::rxData](#)
- (11) volatile size\_t [\\_lpspi\\_master\\_handle::txRemainingByteCount](#)
- (12) volatile size\_t [\\_lpspi\\_master\\_handle::rxRemainingByteCount](#)
- (13) volatile uint32\_t [\\_lpspi\\_master\\_handle::writeRegRemainingTimes](#)
- (14) volatile uint32\_t [\\_lpspi\\_master\\_handle::readRegRemainingTimes](#)
- (15) uint32\_t [\\_lpspi\\_master\\_handle::txBuffIfNull](#)
- (16) volatile uint8\_t [\\_lpspi\\_master\\_handle::state](#)
- (17) [lpspi\\_master\\_transfer\\_callback\\_t](#) [\\_lpspi\\_master\\_handle::callback](#)
- (18) void\* [\\_lpspi\\_master\\_handle::userData](#)

### 24.2.4.5 struct [\\_lpspi\\_slave\\_handle](#)

#### Data Fields

- volatile bool [isByteSwap](#)

- *A flag that whether should byte swap.*  
volatile uint8\_t `fifoSize`
- *FIFO dataSize.*  
volatile uint8\_t `rxWatermark`
- *Rx watermark.*  
volatile uint8\_t `bytesEachWrite`
- *Bytes for each write TDR.*  
volatile uint8\_t `bytesEachRead`
- *Bytes for each read RDR.*  
const uint8\_t \*volatile `txData`
- *Send buffer.*  
uint8\_t \*volatile `rxData`
- *Receive buffer.*  
volatile size\_t `txRemainingByteCount`
- *Number of bytes remaining to send.*  
volatile size\_t `rxRemainingByteCount`
- *Number of bytes remaining to receive.*  
volatile uint32\_t `writeRegRemainingTimes`
- *Write TDR register remaining times.*  
volatile uint32\_t `readRegRemainingTimes`
- *Read RDR register remaining times.*  
uint32\_t `totalByteCount`
- *Number of transfer bytes.*  
volatile uint8\_t `state`
- *LPSPI transfer state , \_lpspi\_transfer\_state.*  
volatile uint32\_t `errorCount`
- *Error count for slave transfer.*  
`lpspi_slave_transfer_callback_t` `callback`
- *Completion callback.*  
void \* `userData`
- *Callback user data.*

## Field Documentation

- (1) `volatile bool _lpspi_slave_handle::isByteSwap`
- (2) `volatile uint8_t _lpspi_slave_handle::fifoSize`
- (3) `volatile uint8_t _lpspi_slave_handle::rxWatermark`
- (4) `volatile uint8_t _lpspi_slave_handle::bytesEachWrite`
- (5) `volatile uint8_t _lpspi_slave_handle::bytesEachRead`
- (6) `const uint8_t* volatile _lpspi_slave_handle::txData`
- (7) `uint8_t* volatile _lpspi_slave_handle::rxData`
- (8) `volatile size_t _lpspi_slave_handle::txRemainingByteCount`
- (9) `volatile size_t _lpspi_slave_handle::rxRemainingByteCount`
- (10) `volatile uint32_t _lpspi_slave_handle::writeRegRemainingTimes`
- (11) `volatile uint32_t _lpspi_slave_handle::readRegRemainingTimes`
- (12) `volatile uint8_t _lpspi_slave_handle::state`
- (13) `volatile uint32_t _lpspi_slave_handle::errorCount`
- (14) `lpspi_slave_transfer_callback_t _lpspi_slave_handle::callback`
- (15) `void* _lpspi_slave_handle::userData`

## 24.2.5 Macro Definition Documentation

**24.2.5.1 `#define FSL_LPSPI_DRIVER_VERSION (MAKE_VERSION(2, 6, 8))`**

**24.2.5.2 `#define LPSPI_DUMMY_DATA (0x00U)`**

Dummy data used for tx if there is not txData.



**24.2.5.3** `#define SPI_RETRY_TIMES 0U /* Define to zero means keep waiting until the flag is assert/deassert. */`

**24.2.5.4** `#define LPSPI_MASTER_PCS_SHIFT (4U)`

**24.2.5.5** `#define LPSPI_MASTER_PCS_MASK (0xF0U)`

**24.2.5.6** `#define LPSPI_SLAVE_PCS_SHIFT (4U)`

**24.2.5.7** `#define LPSPI_SLAVE_PCS_MASK (0xF0U)`

## **24.2.6 Typedef Documentation**

**24.2.6.1** `typedef enum _lpspi_master_slave_mode lpspi_master_slave_mode_t`

**24.2.6.2** `typedef enum _lpspi_which_pcs_config lpspi_which_pcs_t`

**24.2.6.3** `typedef enum _lpspi_pcs_polarity_config lpspi_pcs_polarity_config_t`

**24.2.6.4** `typedef enum _lpspi_clock_polarity lpspi_clock_polarity_t`

**24.2.6.5** `typedef enum _lpspi_clock_phase lpspi_clock_phase_t`

**24.2.6.6** `typedef enum _lpspi_shift_direction lpspi_shift_direction_t`

**24.2.6.7** `typedef enum _lpspi_host_request_select lpspi_host_request_select_t`

**24.2.6.8** `typedef enum _lpspi_match_config lpspi_match_config_t`

**24.2.6.9** `typedef enum _lpspi_pin_config lpspi_pin_config_t`

**24.2.6.10** `typedef enum _lpspi_data_out_config lpspi_data_out_config_t`

**24.2.6.11** `typedef enum _lpspi_pcs_function_config lpspi_pcs_function_config_t`

**24.2.6.12** `typedef enum _lpspi_transfer_width lpspi_transfer_width_t`

**24.2.6.13** `typedef enum _lpspi_delay_type lpspi_delay_type_t`

**24.2.6.14** `typedef struct _lpspi_master_config lpspi_master_config_t`

**24.2.6.15** `typedef struct _lpspi_slave_config lpspi_slave_config_t`

**24.2.6.16** `typedef void(* lpspi_master_transfer_callback_t)(LPSPi_Type *base, lpspi_master_handle_t *handle, status_t status, void *userData)`

## Parameters

|                 |                                                                     |
|-----------------|---------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral address.                                           |
| <i>handle</i>   | Pointer to the handle for the LPSPI master.                         |
| <i>status</i>   | Success or error code describing whether the transfer is completed. |
| <i>userData</i> | Arbitrary pointer-dataSized value passed from the application.      |

**24.2.6.17** `typedef void(* lpspi_slave_transfer_callback_t)(LPSPI_Type *base, lpspi_slave_handle_t *handle, status_t status, void *userData)`

## Parameters

|                 |                                                                     |
|-----------------|---------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral address.                                           |
| <i>handle</i>   | Pointer to the handle for the LPSPI slave.                          |
| <i>status</i>   | Success or error code describing whether the transfer is completed. |
| <i>userData</i> | Arbitrary pointer-dataSized value passed from the application.      |

**24.2.6.18** `typedef struct _lpspi_transfer lpspi_transfer_t`

## 24.2.7 Enumeration Type Documentation

### 24.2.7.1 anonymous enum

## Enumerator

*kStatus\_LPSPI\_Busy* LPSPI transfer is busy.  
*kStatus\_LPSPI\_Error* LPSPI driver error.  
*kStatus\_LPSPI\_Idle* LPSPI is idle.  
*kStatus\_LPSPI\_OutOfRange* LPSPI transfer out Of range.  
*kStatus\_LPSPI\_Timeout* LPSPI timeout polling status flags.

### 24.2.7.2 enum \_lpspi\_flags

## Enumerator

*kLPSPI\_TxDataRequestFlag* Transmit data flag.  
*kLPSPI\_RxDataReadyFlag* Receive data flag.  
*kLPSPI\_WordCompleteFlag* Word Complete flag.  
*kLPSPI\_FrameCompleteFlag* Frame Complete flag.  
*kLPSPI\_TransferCompleteFlag* Transfer Complete flag.



***kLPSPI\_TransmitErrorFlag*** Transmit Error flag (FIFO underrun)  
***kLPSPI\_ReceiveErrorFlag*** Receive Error flag (FIFO overrun)  
***kLPSPI\_DataMatchFlag*** Data Match flag.  
***kLPSPI\_ModuleBusyFlag*** Module Busy flag.  
***kLPSPI\_AllStatusFlag*** Used for clearing all w1c status flags.

### 24.2.7.3 enum \_lpspi\_interrupt\_enable

Enumerator

***kLPSPI\_TxInterruptEnable*** Transmit data interrupt enable.  
***kLPSPI\_RxInterruptEnable*** Receive data interrupt enable.  
***kLPSPI\_WordCompleteInterruptEnable*** Word complete interrupt enable.  
***kLPSPI\_FrameCompleteInterruptEnable*** Frame complete interrupt enable.  
***kLPSPI\_TransferCompleteInterruptEnable*** Transfer complete interrupt enable.  
***kLPSPI\_TransmitErrorInterruptEnable*** Transmit error interrupt enable(FIFO underrun)  
***kLPSPI\_ReceiveErrorInterruptEnable*** Receive Error interrupt enable (FIFO overrun)  
***kLPSPI\_DataMatchInterruptEnable*** Data Match interrupt enable.  
***kLPSPI\_AllInterruptEnable*** All above interrupts enable.

### 24.2.7.4 enum \_lpspi\_dma\_enable

Enumerator

***kLPSPI\_TxDmaEnable*** Transmit data DMA enable.  
***kLPSPI\_RxDmaEnable*** Receive data DMA enable.

### 24.2.7.5 enum \_lpspi\_master\_slave\_mode

Enumerator

***kLPSPI\_Master*** LPSPi peripheral operates in master mode.  
***kLPSPI\_Slave*** LPSPi peripheral operates in slave mode.

### 24.2.7.6 enum \_lpspi\_which\_pcs\_config

Enumerator

***kLPSPI\_Pcs0*** PCS[0].  
***kLPSPI\_Pcs1*** PCS[1].  
***kLPSPI\_Pcs2*** PCS[2].  
***kLPSPI\_Pcs3*** PCS[3].

#### 24.2.7.7 enum \_lpspi\_pcs\_polarity\_config

Enumerator

***kLPSPi\_PcsActiveHigh*** PCS Active High (idles low)  
***kLPSPi\_PcsActiveLow*** PCS Active Low (idles high)

#### 24.2.7.8 enum \_lpspi\_pcs\_polarity

Enumerator

***kLPSPi\_Pcs0ActiveLow*** Pcs0 Active Low (idles high).  
***kLPSPi\_Pcs1ActiveLow*** Pcs1 Active Low (idles high).  
***kLPSPi\_Pcs2ActiveLow*** Pcs2 Active Low (idles high).  
***kLPSPi\_Pcs3ActiveLow*** Pcs3 Active Low (idles high).  
***kLPSPi\_PcsAllActiveLow*** Pcs0 to Pcs5 Active Low (idles high).

#### 24.2.7.9 enum \_lpspi\_clock\_polarity

Enumerator

***kLPSPi\_ClockPolarityActiveHigh*** CPOL=0. Active-high LPSPi clock (idles low)  
***kLPSPi\_ClockPolarityActiveLow*** CPOL=1. Active-low LPSPi clock (idles high)

#### 24.2.7.10 enum \_lpspi\_clock\_phase

Enumerator

***kLPSPi\_ClockPhaseFirstEdge*** CPHA=0. Data is captured on the leading edge of the SCK and changed on the following edge.  
***kLPSPi\_ClockPhaseSecondEdge*** CPHA=1. Data is changed on the leading edge of the SCK and captured on the following edge.

#### 24.2.7.11 enum \_lpspi\_shift\_direction

Enumerator

***kLPSPi\_MsbFirst*** Data transfers start with most significant bit.  
***kLPSPi\_LsbFirst*** Data transfers start with least significant bit.

**24.2.7.12 enum \_lpspi\_host\_request\_select**

Enumerator

*kLPSPI\_HostReqExtPin* Host Request is an ext pin.*kLPSPI\_HostReqInternalTrigger* Host Request is an internal trigger.**24.2.7.13 enum \_lpspi\_match\_config**

Enumerator

*kLPSI\_MatchDisabled* LPSPI Match Disabled.*kLPSI\_1stWordEqualsM0orM1* LPSPI Match Enabled.*kLPSI\_AnyWordEqualsM0orM1* LPSPI Match Enabled.*kLPSI\_1stWordEqualsM0and2ndWordEqualsM1* LPSPI Match Enabled.*kLPSI\_AnyWordEqualsM0andNxtWordEqualsM1* LPSPI Match Enabled.*kLPSI\_1stWordAndM1EqualsM0andM1* LPSPI Match Enabled.*kLPSI\_AnyWordAndM1EqualsM0andM1* LPSPI Match Enabled.**24.2.7.14 enum \_lpspi\_pin\_config**

Enumerator

*kLPSPI\_SdiInSdoOut* LPSPI SDI input, SDO output.*kLPSPI\_SdiInSdiOut* LPSPI SDI input, SDI output.*kLPSPI\_SdoInSdoOut* LPSPI SDO input, SDO output.*kLPSPI\_SdoInSdiOut* LPSPI SDO input, SDI output.**24.2.7.15 enum \_lpspi\_data\_out\_config**

Enumerator

*kLpspiDataOutRetained* Data out retains last value when chip select is de-asserted.*kLpspiDataOutTristate* Data out is tristated when chip select is de-asserted.**24.2.7.16 enum \_lpspi\_pcs\_function\_config**

Enumerator

*kLPSPI\_PcsAsCs* PCS pin select as cs function.*kLPSPI\_PcsAsData* PCS pin select as date function.

#### 24.2.7.17 enum \_lpspi\_transfer\_width

Enumerator

***kLPSPI\_SingleBitXfer*** 1-bit shift at a time, data out on SDO, in on SDI (normal mode)  
***kLPSPI\_TwoBitXfer*** 2-bits shift out on SDO/SDI and in on SDO/SDI  
***kLPSPI\_FourBitXfer*** 4-bits shift out on SDO/SDI/PCS[3:2] and in on SDO/SDI/PCS[3:2]

#### 24.2.7.18 enum \_lpspi\_delay\_type

Enumerator

***kLPSPI\_PcsToSck*** PCS-to-SCK delay.  
***kLPSPI\_LastSckToPcs*** Last SCK edge to PCS delay.  
***kLPSPI\_BetweenTransfer*** Delay between transfers.

#### 24.2.7.19 enum \_lpspi\_transfer\_config\_flag\_for\_master

Enumerator

***kLPSPI\_MasterPcs0*** LPSPI master transfer use PCS0 signal.  
***kLPSPI\_MasterPcs1*** LPSPI master transfer use PCS1 signal.  
***kLPSPI\_MasterPcs2*** LPSPI master transfer use PCS2 signal.  
***kLPSPI\_MasterPcs3*** LPSPI master transfer use PCS3 signal.  
***kLPSPI\_MasterWidth1*** LPSPI master transfer 1bit.  
***kLPSPI\_MasterWidth2*** LPSPI master transfer 2bit.  
***kLPSPI\_MasterWidth4*** LPSPI master transfer 4bit.  
***kLPSPI\_MasterPcsContinuous*** Is PCS signal continuous.  
***kLPSPI\_MasterByteSwap*** Is master swap the byte. For example, when want to send data 1 2 3 4 5 6 7 8 (suppose you set lpspi\_shift\_direction\_t to MSB).  
 1. If you set bitPerFrame = 8 , no matter the kLPSPI\_MasterByteSwap flag is used or not, the waveform is 1 2 3 4 5 6 7 8.  
 2. If you set bitPerFrame = 16 : (1) the waveform is 2 1 4 3 6 5 8 7 if you do not use the kLPSPI\_MasterByteSwap flag. (2) the waveform is 1 2 3 4 5 6 7 8 if you use the kLPSPI\_MasterByteSwap flag.  
 3. If you set bitPerFrame = 32 : (1) the waveform is 4 3 2 1 8 7 6 5 if you do not use the kLPSPI\_MasterByteSwap flag. (2) the waveform is 1 2 3 4 5 6 7 8 if you use the kLPSPI\_MasterByteSwap flag.

#### 24.2.7.20 enum \_lpspi\_transfer\_config\_flag\_for\_slave

Enumerator

***kLPSPI\_SlavePcs0*** LPSPI slave transfer use PCS0 signal.

***kLPSPI\_SlavePcs1*** LPSPI slave transfer use PCS1 signal.

***kLPSPI\_SlavePcs2*** LPSPI slave transfer use PCS2 signal.

***kLPSPI\_SlavePcs3*** LPSPI slave transfer use PCS3 signal.

***kLPSPI\_SlaveByteSwap*** Is slave swap the byte. For example, when want to send data 1 2 3 4 5 6 7 8 (suppose you set *lpspi\_shift\_direction\_t* to MSB).

1. If you set *bitPerFrame* = 8 , no matter the *kLPSPI\_SlaveByteSwap* flag is used or not, the waveform is 1 2 3 4 5 6 7 8.
2. If you set *bitPerFrame* = 16 : (1) the waveform is 2 1 4 3 6 5 8 7 if you do not use the *kLPSPI\_SlaveByteSwap* flag. (2) the waveform is 1 2 3 4 5 6 7 8 if you use the *kLPSPI\_SlaveByteSwap* flag.
3. If you set *bitPerFrame* = 32 : (1) the waveform is 4 3 2 1 8 7 6 5 if you do not use the *kLPSPI\_SlaveByteSwap* flag. (2) the waveform is 1 2 3 4 5 6 7 8 if you use the *kLPSPI\_SlaveByteSwap* flag.

#### 24.2.7.21 enum\_lpspi\_transfer\_state

Enumerator

***kLPSPI\_Idle*** Nothing in the transmitter/receiver.

***kLPSPI\_Busy*** Transfer queue is not finished.

***kLPSPI\_Error*** Transfer error.

### 24.2.8 Function Documentation

#### 24.2.8.1 void LPSPI\_MasterInit ( LPSPI\_Type \* *base*, const lpspi\_master\_config\_t \* *masterConfig*, uint32\_t *srcClock\_Hz* )

Parameters

|                     |                                                     |
|---------------------|-----------------------------------------------------|
| <i>base</i>         | LPSPI peripheral address.                           |
| <i>masterConfig</i> | Pointer to structure <i>lpspi_master_config_t</i> . |
| <i>srcClock_Hz</i>  | Module source input clock in Hertz                  |

#### 24.2.8.2 void LPSPI\_MasterGetDefaultConfig ( lpspi\_master\_config\_t \* *masterConfig* )

This API initializes the configuration structure for [LPSPI\\_MasterInit\(\)](#). The initialized structure can remain unchanged in [LPSPI\\_MasterInit\(\)](#), or can be modified before calling the [LPSPI\\_MasterInit\(\)](#). Example:

```
* lpspi_master_config_t masterConfig;
* LPSPI_MasterGetDefaultConfig(&masterConfig);
*
```

Parameters

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>masterConfig</i> | pointer to <code>lpspi_master_config_t</code> structure |
|---------------------|---------------------------------------------------------|

### 24.2.8.3 void LPSPI\_SlaveInit ( LPSPI\_Type \* *base*, const `lpspi_slave_config_t` \* *slaveConfig* )

Parameters

|                    |                                                            |
|--------------------|------------------------------------------------------------|
| <i>base</i>        | LPSPi peripheral address.                                  |
| <i>slaveConfig</i> | Pointer to a structure <code>lpspi_slave_config_t</code> . |

### 24.2.8.4 void LPSPI\_SlaveGetDefaultConfig ( `lpspi_slave_config_t` \* *slaveConfig* )

This API initializes the configuration structure for [LPSPI\\_SlaveInit\(\)](#). The initialized structure can remain unchanged in [LPSPI\\_SlaveInit\(\)](#) or can be modified before calling the [LPSPI\\_SlaveInit\(\)](#). Example:

```
* lpspi_slave_config_t slaveConfig;
* LPSPI_SlaveGetDefaultConfig(&slaveConfig);
*
```

Parameters

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| <i>slaveConfig</i> | pointer to <code>lpspi_slave_config_t</code> structure. |
|--------------------|---------------------------------------------------------|

### 24.2.8.5 void LPSPI\_Deinit ( LPSPI\_Type \* *base* )

Call this API to disable the LPSPi clock.

Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPi peripheral address. |
|-------------|---------------------------|

### 24.2.8.6 void LPSPI\_Reset ( LPSPI\_Type \* *base* )

Note that this function sets all registers to reset state. As a result, the LPSPi module can't work after calling this API.

## Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

**24.2.8.7 uint32\_t LPSPI\_GetInstance ( LPSPI\_Type \* *base* )**

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPSPI peripheral base address. |
|-------------|--------------------------------|

## Returns

LPSPI instance.

**24.2.8.8 static void LPSPI\_Enable ( LPSPI\_Type \* *base*, bool *enable* ) [inline], [static]**

## Parameters

|               |                                                      |
|---------------|------------------------------------------------------|
| <i>base</i>   | LPSPI peripheral address.                            |
| <i>enable</i> | Pass true to enable module, false to disable module. |

**24.2.8.9 static uint32\_t LPSPI\_GetStatusFlags ( LPSPI\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

## Returns

The LPSPI status(in SR register).

**24.2.8.10 static uint8\_t LPSPI\_GetTxFifoSize ( LPSPI\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

## Returns

The LPSPI Tx FIFO size.

**24.2.8.11** `static uint8_t LPSPI_GetRxFifoSize ( LPSPI_Type * base ) [inline],  
[static]`

## Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

## Returns

The LPSPI Rx FIFO size.

**24.2.8.12** `static uint32_t LPSPI_GetTxFifoCount ( LPSPI_Type * base ) [inline],  
[static]`

## Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

## Returns

The number of words in the transmit FIFO.

**24.2.8.13** `static uint32_t LPSPI_GetRxFifoCount ( LPSPI_Type * base ) [inline],  
[static]`

## Parameters

---



|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

#### Returns

The number of words in the receive FIFO.

#### 24.2.8.14 static void LPSPi\_ClearStatusFlags ( LPSPi\_Type \* *base*, uint32\_t *statusFlags* ) [inline], [static]

This function clears the desired status bit by using a write-1-to-clear. The user passes in the base and the desired status flag bit to clear. The list of status flags is defined in the `_lpspi_flags`. Example usage:

```
* LPSPi_ClearStatusFlags(base, kLPSPi_TxDataRequestFlag |
 kLPSPi_RxDataReadyFlag);
*
```

#### Parameters

|                    |                                                            |
|--------------------|------------------------------------------------------------|
| <i>base</i>        | LPSPI peripheral address.                                  |
| <i>statusFlags</i> | The status flag used from type <code>_lpspi_flags</code> . |

< The status flags are cleared by writing 1 (w1c).

#### 24.2.8.15 static void LPSPi\_EnableInterrupts ( LPSPi\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

This function configures the various interrupt masks of the LPSPi. The parameters are base and an interrupt mask. Note that, for Tx fill and Rx FIFO drain requests, enabling the interrupt request disables the DMA request.

```
* LPSPi_EnableInterrupts(base, kLPSPi_TxInterruptEnable |
 kLPSPi_RxInterruptEnable);
*
```

#### Parameters

|             |                                                                         |
|-------------|-------------------------------------------------------------------------|
| <i>base</i> | LPSPI peripheral address.                                               |
| <i>mask</i> | The interrupt mask; Use the enum <code>_lpspi_interrupt_enable</code> . |

#### 24.2.8.16 static void LPSPi\_DisableInterrupts ( LPSPi\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

```
* LPSPi_DisableInterrupts(base, kLPSPi_TxInterruptEnable |
 kLPSPi_RxInterruptEnable);
*
```

#### Parameters

|             |                                                                         |
|-------------|-------------------------------------------------------------------------|
| <i>base</i> | LPSPi peripheral address.                                               |
| <i>mask</i> | The interrupt mask; Use the enum <code>_lpspi_interrupt_enable</code> . |

#### 24.2.8.17 **static void LPSPi\_EnableDMA ( LPSPi\_Type \* *base*, uint32\_t *mask* )** **[inline], [static]**

This function configures the Rx and Tx DMA mask of the LPSPi. The parameters are base and a DMA mask.

```
* LPSPi_EnableDMA(base, kLPSPi_TxDmaEnable |
* kLPSPi_RxDmaEnable);
*
```

#### Parameters

|             |                                                                   |
|-------------|-------------------------------------------------------------------|
| <i>base</i> | LPSPi peripheral address.                                         |
| <i>mask</i> | The interrupt mask; Use the enum <code>_lpspi_dma_enable</code> . |

#### 24.2.8.18 **static void LPSPi\_DisableDMA ( LPSPi\_Type \* *base*, uint32\_t *mask* )** **[inline], [static]**

This function configures the Rx and Tx DMA mask of the LPSPi. The parameters are base and a DMA mask.

```
* SPI_DisableDMA(base, kLPSPi_TxDmaEnable |
* kLPSPi_RxDmaEnable);
*
```

#### Parameters

|             |                                                                   |
|-------------|-------------------------------------------------------------------|
| <i>base</i> | LPSPi peripheral address.                                         |
| <i>mask</i> | The interrupt mask; Use the enum <code>_lpspi_dma_enable</code> . |

#### 24.2.8.19 **static uint32\_t LPSPi\_GetTxRegisterAddress ( LPSPi\_Type \* *base* )** **[inline], [static]**

This function gets the LPSPi Transmit Data Register address because this value is needed for the DMA operation. This function can be used for either master or slave mode.

#### Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

#### Returns

The LPSPI Transmit Data Register address.

**24.2.8.20 static uint32\_t LPSPI\_GetRxRegisterAddress ( LPSPI\_Type \* *base* )  
[inline], [static]**

This function gets the LPSPI Receive Data Register address because this value is needed for the DMA operation. This function can be used for either master or slave mode.

#### Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

#### Returns

The LPSPI Receive Data Register address.

**24.2.8.21 bool LPSPI\_CheckTransferArgument ( LPSPI\_Type \* *base*, lpspi\_transfer\_t \* *transfer*, bool *isEdma* )**

#### Parameters

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral address.                                                       |
| <i>transfer</i> | the transfer struct to be used.                                                 |
| <i>isEdma</i>   | True to check for EDMA transfer, false to check interrupt non-blocking transfer |

#### Returns

Return true for right and false for wrong.

**24.2.8.22 static void LPSPI\_SetMasterSlaveMode ( LPSPI\_Type \* *base*,  
lpspi\_master\_slave\_mode\_t *mode* ) [inline], [static]**

Note that the CFGR1 should only be written when the LPSPI is disabled (LPSPIx\_CR\_MEN = 0).

Parameters

|             |                                                                                 |
|-------------|---------------------------------------------------------------------------------|
| <i>base</i> | LPSPI peripheral address.                                                       |
| <i>mode</i> | Mode setting (master or slave) of type <code>lpspi_master_slave_mode_t</code> . |

**24.2.8.23** `static void LPSPI_SelectTransferPCS ( LPSPI_Type * base, lpspi_which_pcs_t select ) [inline], [static]`

Parameters

|               |                                                   |
|---------------|---------------------------------------------------|
| <i>base</i>   | LPSPI peripheral address.                         |
| <i>select</i> | LPSPI Peripheral Chip Select (PCS) configuration. |

**24.2.8.24** `static void LPSPI_SetPCSContinuous ( LPSPI_Type * base, bool IsContinuous ) [inline], [static]`

Note

In master mode, continuous transfer will keep the PCS asserted at the end of the frame size, until a command word is received that starts a new frame. So PCS must be set back to uncontinuous when transfer finishes. In slave mode, when continuous transfer is enabled, the LPSPI will only transmit the first frame size bits, after that the LPSPI will transmit received data back (assuming a 32-bit shift register).

Parameters

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| <i>base</i>         | LPSPI peripheral address.                                                           |
| <i>IsContinuous</i> | True to set the transfer PCS to continuous mode, false to set to uncontinuous mode. |

**24.2.8.25** `static bool LPSPI_IsMaster ( LPSPI_Type * base ) [inline], [static]`

Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

Returns

Returns true if the module is in master mode or false if the module is in slave mode.

**24.2.8.26** `static void LPSPI_FlushFifo ( LPSPI_Type * base, bool flushTxFifo, bool flushRxFifo ) [inline], [static]`

## Parameters

|                    |                                                                    |
|--------------------|--------------------------------------------------------------------|
| <i>base</i>        | LPSPI peripheral address.                                          |
| <i>flushTxFifo</i> | Flushes (true) the Tx FIFO, else do not flush (false) the Tx FIFO. |
| <i>flushRxFifo</i> | Flushes (true) the Rx FIFO, else do not flush (false) the Rx FIFO. |

#### 24.2.8.27 static void LPSPI\_SetFifoWatermarks ( LPSPI\_Type \* *base*, uint32\_t *txWater*, uint32\_t *rxWater* ) [inline], [static]

This function allows the user to set the receive and transmit FIFO watermarks. The function does not compare the watermark settings to the FIFO size. The FIFO watermark should not be equal to or greater than the FIFO size. It is up to the higher level driver to make this check.

## Parameters

|                |                                                                                                |
|----------------|------------------------------------------------------------------------------------------------|
| <i>base</i>    | LPSPI peripheral address.                                                                      |
| <i>txWater</i> | The TX FIFO watermark value. Writing a value equal or greater than the FIFO size is truncated. |
| <i>rxWater</i> | The RX FIFO watermark value. Writing a value equal or greater than the FIFO size is truncated. |

#### 24.2.8.28 static void LPSPI\_SetAllPcsPolarity ( LPSPI\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

Note that the CFGR1 should only be written when the LPSPI is disabled (LPSPIx\_CR\_MEN = 0).

This is an example: PCS0 and PCS1 set to active low and other PCSs set to active high. Note that the number of PCS is device-specific.

```
* LPSPI_SetAllPcsPolarity(base, kLPSPI_Pcs0ActiveLow |
 kLPSPI_Pcs1ActiveLow);
*
```

## Parameters

|             |                                                                        |
|-------------|------------------------------------------------------------------------|
| <i>base</i> | LPSPI peripheral address.                                              |
| <i>mask</i> | The PCS polarity mask; Use the enum <code>_lpspi_pcs_polarity</code> . |

#### 24.2.8.29 static void LPSPI\_SetFrameSize ( LPSPI\_Type \* *base*, uint32\_t *frameSize* ) [inline], [static]

The minimum frame size is 8-bits and the maximum frame size is 4096-bits. If the frame size is less than or equal to 32-bits, the word size and frame size are identical. If the frame size is greater than 32-bits, the word size is 32-bits for each word except the last (the last word contains the remainder bits if the frame size is not divisible by 32). The minimum word size is 2-bits. A frame size of 33-bits (or similar) is not supported.

Note 1: The transmit command register should be initialized before enabling the LPSPI in slave mode, although the command register does not update until after the LPSPI is enabled. After it is enabled, the transmit command register should only be changed if the LPSPI is idle.

Note 2: The transmit and command FIFO is a combined FIFO that includes both transmit data and command words. That means the TCR register should be written to when the Tx FIFO is not full.

Parameters

|                  |                                   |
|------------------|-----------------------------------|
| <i>base</i>      | LPSPI peripheral address.         |
| <i>frameSize</i> | The frame size in number of bits. |

#### 24.2.8.30 uint32\_t LPSPI\_MasterSetBaudRate ( LPSPI\_Type \* *base*, uint32\_t *baudRate\_Bps*, uint32\_t *srcClock\_Hz*, uint32\_t \* *tcrPrescaleValue* )

This function takes in the desired bitsPerSec (baud rate) and calculates the nearest possible baud rate without exceeding the desired baud rate and returns the calculated baud rate in bits-per-second. It requires the caller to provide the frequency of the module source clock (in Hertz). Note that the baud rate does not go into effect until the Transmit Control Register (TCR) is programmed with the prescale value. Hence, this function returns the prescale *tcrPrescaleValue* parameter for later programming in the TCR. The higher level peripheral driver should alert the user of an out of range baud rate input.

Note that the LPSPI module must first be disabled before configuring this. Note that the LPSPI module must be configured for master mode before configuring this.

Parameters

|                     |                                           |
|---------------------|-------------------------------------------|
| <i>base</i>         | LPSPI peripheral address.                 |
| <i>baudRate_Bps</i> | The desired baud rate in bits per second. |
| <i>srcClock_Hz</i>  | Module source input clock in Hertz.       |

|                          |                                                   |
|--------------------------|---------------------------------------------------|
| <i>tcrPrescale-Value</i> | The TCR prescale value needed to program the TCR. |
|--------------------------|---------------------------------------------------|

#### Returns

The actual calculated baud rate. This function may also return a "0" if the LPSPi is not configured for master mode or if the LPSPi module is not disabled.

#### 24.2.8.31 void LPSPi\_MasterSetDelayScaler ( LPSPi\_Type \* *base*, uint32\_t *scaler*, lpspi\_delay\_type\_t *whichDelay* )

This function configures the following: SCK to PCS delay, or PCS to SCK delay, or The configurations must occur between the transfer delay.

The delay names are available in type lpspi\_delay\_type\_t.

The user passes the desired delay along with the delay value. This allows the user to directly set the delay values if they have pre-calculated them or if they simply wish to manually increment the value.

Note that the LPSPi module must first be disabled before configuring this. Note that the LPSPi module must be configured for master mode before configuring this.

#### Parameters

|                   |                                                                     |
|-------------------|---------------------------------------------------------------------|
| <i>base</i>       | LPSPi peripheral address.                                           |
| <i>scaler</i>     | The 8-bit delay value 0x00 to 0xFF (255).                           |
| <i>whichDelay</i> | The desired delay to configure, must be of type lpspi_delay_type_t. |

#### 24.2.8.32 uint32\_t LPSPi\_MasterSetDelayTimes ( LPSPi\_Type \* *base*, uint32\_t *delayTimeInNanoSec*, lpspi\_delay\_type\_t *whichDelay*, uint32\_t *srcClock\_Hz* )

This function calculates the values for the following: SCK to PCS delay, or PCS to SCK delay, or The configurations must occur between the transfer delay.

The delay names are available in type lpspi\_delay\_type\_t.

The user passes the desired delay and the desired delay value in nano-seconds. The function calculates the value needed for the desired delay parameter and returns the actual calculated delay because an exact delay match may not be possible. In this case, the closest match is calculated without going below the desired delay value input. It is possible to input a very large delay value that exceeds the capability of the part, in which case the maximum supported delay is returned. It is up to the higher level peripheral driver to alert the user of an out of range delay input.

Note that the LPSPi module must be configured for master mode before configuring this. And note that the  $\text{delayTime} = \text{LPSPi\_clockSource} / (\text{PRESCALE} * \text{Delay\_scaler})$ .



#### Parameters

|                            |                                                                                             |
|----------------------------|---------------------------------------------------------------------------------------------|
| <i>base</i>                | LPSPi peripheral address.                                                                   |
| <i>delayTimeIn-NanoSec</i> | The desired delay value in nano-seconds.                                                    |
| <i>whichDelay</i>          | The desired delay to configuration, which must be of type <code>lpspi_delay_type_t</code> . |
| <i>srcClock_Hz</i>         | Module source input clock in Hertz.                                                         |

#### Returns

actual Calculated delay value in nano-seconds.

#### 24.2.8.33 static void LPSPi\_WriteData ( LPSPi\_Type \* *base*, uint32\_t *data* ) [inline], [static]

This function writes data passed in by the user to the Transmit Data Register (TDR). The user can pass up to 32-bits of data to load into the TDR. If the frame size exceeds 32-bits, the user has to manage sending the data one 32-bit word at a time. Any writes to the TDR result in an immediate push to the transmit FIFO. This function can be used for either master or slave modes.

#### Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPi peripheral address. |
| <i>data</i> | The data word to be sent. |

#### 24.2.8.34 static uint32\_t LPSPi\_ReadData ( LPSPi\_Type \* *base* ) [inline], [static]

This function reads the data from the Receive Data Register (RDR). This function can be used for either master or slave mode.

#### Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPi peripheral address. |
|-------------|---------------------------|

#### Returns

The data read from the data buffer.

#### 24.2.8.35 void LPSPi\_SetDummyData ( LPSPi\_Type \* *base*, uint8\_t *dummyData* )

#### Parameters

|                  |                                                                                                                                                                                                                                                 |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>      | LPSPi peripheral address.                                                                                                                                                                                                                       |
| <i>dummyData</i> | Data to be transferred when tx buffer is NULL. Note: This API has no effect when LPSPi in slave interrupt mode, because driver will set the TXMSK bit to 1 if txData is NULL, no data is loaded from transmit FIFO and output pin is tristated. |

**24.2.8.36 void LPSPi\_MasterTransferCreateHandle ( LPSPi\_Type \* *base*,  
lpspi\_master\_handle\_t \* *handle*, lpspi\_master\_transfer\_callback\_t *callback*,  
void \* *userData* )**

This function initializes the LPSPi handle, which can be used for other LPSPi transactional APIs. Usually, for a specified LPSPi instance, call this API once to get the initialized handle.

#### Parameters

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>base</i>     | LPSPi peripheral address.                      |
| <i>handle</i>   | LPSPi handle pointer to lpspi_master_handle_t. |
| <i>callback</i> | DSPI callback.                                 |
| <i>userData</i> | callback function parameter.                   |

**24.2.8.37 status\_t LPSPi\_MasterTransferBlocking ( LPSPi\_Type \* *base*, lpspi\_transfer\_t \* *transfer* )**

This function transfers data using a polling method. This is a blocking function, which does not return until all transfers have been completed.

Note: The transfer data size should be integer multiples of bytesPerFrame if bytesPerFrame is less than or equal to 4. For bytesPerFrame greater than 4: The transfer data size should be equal to bytesPerFrame if the bytesPerFrame is not integer multiples of 4. Otherwise, the transfer data size can be an integer multiple of bytesPerFrame.

#### Parameters

|                 |                                        |
|-----------------|----------------------------------------|
| <i>base</i>     | LPSPi peripheral address.              |
| <i>transfer</i> | pointer to lpspi_transfer_t structure. |

#### Returns

status of status\_t.

#### 24.2.8.38 **status\_t LPSPI\_MasterTransferNonBlocking ( LPSPI\_Type \* *base*, lpspi\_master\_handle\_t \* *handle*, lpspi\_transfer\_t \* *transfer* )**

This function transfers data using an interrupt method. This is a non-blocking function, which returns right away. When all data is transferred, the callback function is called.

Note: The transfer data size should be integer multiples of bytesPerFrame if bytesPerFrame is less than or equal to 4. For bytesPerFrame greater than 4: The transfer data size should be equal to bytesPerFrame if the bytesPerFrame is not integer multiples of 4. Otherwise, the transfer data size can be an integer multiple of bytesPerFrame.

Parameters

|                 |                                                                             |
|-----------------|-----------------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral address.                                                   |
| <i>handle</i>   | pointer to lpspi_master_handle_t structure which stores the transfer state. |
| <i>transfer</i> | pointer to lpspi_transfer_t structure.                                      |

Returns

status of status\_t.

#### 24.2.8.39 **status\_t LPSPI\_MasterTransferGetCount ( LPSPI\_Type \* *base*, lpspi\_master\_handle\_t \* *handle*, size\_t \* *count* )**

This function gets the master transfer remaining bytes.

Parameters

|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| <i>base</i>   | LPSPI peripheral address.                                                   |
| <i>handle</i> | pointer to lpspi_master_handle_t structure which stores the transfer state. |
| <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction.         |

Returns

status of status\_t.

#### 24.2.8.40 **void LPSPI\_MasterTransferAbort ( LPSPI\_Type \* *base*, lpspi\_master\_handle\_t \* *handle* )**

This function aborts a transfer which uses an interrupt method.

Parameters

|               |                                                                                          |
|---------------|------------------------------------------------------------------------------------------|
| <i>base</i>   | LPSPI peripheral address.                                                                |
| <i>handle</i> | pointer to <code>lpspi_master_handle_t</code> structure which stores the transfer state. |

**24.2.8.41 void LPSPI\_MasterTransferHandleIRQ ( LPSPI\_Type \* *base*,  
lpspi\_master\_handle\_t \* *handle* )**

This function processes the LPSPI transmit and receive IRQ.

Parameters

|               |                                                                                          |
|---------------|------------------------------------------------------------------------------------------|
| <i>base</i>   | LPSPI peripheral address.                                                                |
| <i>handle</i> | pointer to <code>lpspi_master_handle_t</code> structure which stores the transfer state. |

**24.2.8.42 void LPSPI\_SlaveTransferCreateHandle ( LPSPI\_Type \* *base*,  
lpspi\_slave\_handle\_t \* *handle*, lpspi\_slave\_transfer\_callback\_t *callback*, void \*  
*userData* )**

This function initializes the LPSPI handle, which can be used for other LPSPI transactional APIs. Usually, for a specified LPSPI instance, call this API once to get the initialized handle.

Parameters

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral address.                                   |
| <i>handle</i>   | LPSPI handle pointer to <code>lpspi_slave_handle_t</code> . |
| <i>callback</i> | DSPI callback.                                              |
| <i>userData</i> | callback function parameter.                                |

**24.2.8.43 status\_t LPSPI\_SlaveTransferNonBlocking ( LPSPI\_Type \* *base*,  
lpspi\_slave\_handle\_t \* *handle*, lpspi\_transfer\_t \* *transfer* )**

This function transfer data using an interrupt method. This is a non-blocking function, which returns right away. When all data is transferred, the callback function is called.

Note: The transfer data size should be integer multiples of bytesPerFrame if bytesPerFrame is less than or equal to 4. For bytesPerFrame greater than 4: The transfer data size should be equal to bytesPerFrame if the bytesPerFrame is not an integer multiple of 4. Otherwise, the transfer data size can be an integer multiple of bytesPerFrame.

Parameters

|                 |                                                                                         |
|-----------------|-----------------------------------------------------------------------------------------|
| <i>base</i>     | LPSPi peripheral address.                                                               |
| <i>handle</i>   | pointer to <code>lpspi_slave_handle_t</code> structure which stores the transfer state. |
| <i>transfer</i> | pointer to <code>lpspi_transfer_t</code> structure.                                     |

Returns

status of `status_t`.

#### 24.2.8.44 `status_t LPSPI_SlaveTransferGetCount ( LPSPI_Type * base, lpspi_slave_handle_t * handle, size_t * count )`

This function gets the slave transfer remaining bytes.

Parameters

|               |                                                                                         |
|---------------|-----------------------------------------------------------------------------------------|
| <i>base</i>   | LPSPi peripheral address.                                                               |
| <i>handle</i> | pointer to <code>lpspi_slave_handle_t</code> structure which stores the transfer state. |
| <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction.                     |

Returns

status of `status_t`.

#### 24.2.8.45 `void LPSPI_SlaveTransferAbort ( LPSPI_Type * base, lpspi_slave_handle_t * handle )`

This function aborts a transfer which uses an interrupt method.

Parameters

|               |                                                                                         |
|---------------|-----------------------------------------------------------------------------------------|
| <i>base</i>   | LPSPi peripheral address.                                                               |
| <i>handle</i> | pointer to <code>lpspi_slave_handle_t</code> structure which stores the transfer state. |

#### 24.2.8.46 `void LPSPI_SlaveTransferHandleIRQ ( LPSPI_Type * base, lpspi_slave_handle_t * handle )`

This function processes the LPSPi transmit and receives an IRQ.

## Parameters

|               |                                                                                         |
|---------------|-----------------------------------------------------------------------------------------|
| <i>base</i>   | LPSPI peripheral address.                                                               |
| <i>handle</i> | pointer to <code>lpspi_slave_handle_t</code> structure which stores the transfer state. |

**24.2.8.47 bool LPSPI\_WaitTxFifoEmpty ( LPSPI\_Type \* *base* )**

This function wait the tx fifo empty

## Parameters

|             |                           |
|-------------|---------------------------|
| <i>base</i> | LPSPI peripheral address. |
|-------------|---------------------------|

## Returns

true for the tx FIFO is ready, false is not.

**24.2.9 Variable Documentation****24.2.9.1 volatile uint8\_t g\_lpspiDummyData[]**

## Chapter 25

# LPTMR: Low-Power Timer

### 25.1 Overview

The MCUXpresso SDK provides a driver for the Low-Power Timer (LPTMR) of MCUXpresso SDK devices.

### 25.2 Function groups

The LPTMR driver supports operating the module as a time counter or as a pulse counter.

#### 25.2.1 Initialization and deinitialization

The function [LPTMR\\_Init\(\)](#) initializes the LPTMR with specified configurations. The function [LPTMR\\_GetDefaultConfig\(\)](#) gets the default configurations. The initialization function configures the LPTMR for a timer or a pulse counter mode. It also sets up the LPTMR's free running mode operation and a clock source.

The function [LPTMR\\_DeInit\(\)](#) disables the LPTMR module and gates the module clock.

#### 25.2.2 Timer period Operations

The function [LPTMR\\_SetTimerPeriod\(\)](#) sets the timer period in units of count. Timers counts from 0 to the count value set here.

The function [LPTMR\\_GetCurrentTimerCount\(\)](#) reads the current timer counting value. This function returns the real-time timer counting value ranging from 0 to a timer period.

The timer period operation function takes the count value in ticks. Call the utility macros provided in the `fsl_common.h` file to convert to microseconds or milliseconds.

#### 25.2.3 Start and Stop timer operations

The function [LPTMR\\_StartTimer\(\)](#) starts the timer counting. After calling this function, the timer counts up to the counter value set earlier by using the [LPTMR\\_SetPeriod\(\)](#) function. Each time the timer reaches the count value and increments, it generates a trigger pulse and sets the timeout interrupt flag. An interrupt is also triggered if the timer interrupt is enabled.

The function [LPTMR\\_StopTimer\(\)](#) stops the timer counting and resets the timer's counter register.

## 25.2.4 Status

Provides functions to get and clear the LPTMR status.

## 25.2.5 Interrupt

Provides functions to enable/disable LPTMR interrupts and get the currently enabled interrupts.

## 25.3 Typical use case

### 25.3.1 LPTMR tick example

Updates the LPTMR period and toggles an LED periodically. Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/lptmr`

## Data Structures

- struct [\\_lptmr\\_config](#)  
*LPTMR config structure. [More...](#)*

## Typedefs

- typedef enum [\\_lptmr\\_pin\\_select](#) [lptmr\\_pin\\_select\\_t](#)  
*LPTMR pin selection used in pulse counter mode.*
- typedef enum [\\_lptmr\\_pin\\_polarity](#) [lptmr\\_pin\\_polarity\\_t](#)  
*LPTMR pin polarity used in pulse counter mode.*
- typedef enum [\\_lptmr\\_timer\\_mode](#) [lptmr\\_timer\\_mode\\_t](#)  
*LPTMR timer mode selection.*
- typedef enum [\\_lptmr\\_prescaler\\_glitch\\_value](#) [lptmr\\_prescaler\\_glitch\\_value\\_t](#)  
*LPTMR prescaler/glitch filter values.*
- typedef enum [\\_lptmr\\_prescaler\\_clock\\_select](#) [lptmr\\_prescaler\\_clock\\_select\\_t](#)  
*LPTMR prescaler/glitch filter clock select.*
- typedef enum [\\_lptmr\\_interrupt\\_enable](#) [lptmr\\_interrupt\\_enable\\_t](#)  
*List of the LPTMR interrupts.*
- typedef enum [\\_lptmr\\_status\\_flags](#) [lptmr\\_status\\_flags\\_t](#)  
*List of the LPTMR status flags.*
- typedef struct [\\_lptmr\\_config](#) [lptmr\\_config\\_t](#)  
*LPTMR config structure.*



## Enumerations

- enum `_lptmr_pin_select` {  
`kLPTMR_PinSelectInput_0` = 0x0U,  
`kLPTMR_PinSelectInput_1` = 0x1U,  
`kLPTMR_PinSelectInput_2` = 0x2U,  
`kLPTMR_PinSelectInput_3` = 0x3U }  
*LPTMR pin selection used in pulse counter mode.*
  - enum `_lptmr_pin_polarity` {  
`kLPTMR_PinPolarityActiveHigh` = 0x0U,  
`kLPTMR_PinPolarityActiveLow` = 0x1U }  
*LPTMR pin polarity used in pulse counter mode.*
  - enum `_lptmr_timer_mode` {  
`kLPTMR_TimerModeTimeCounter` = 0x0U,  
`kLPTMR_TimerModePulseCounter` = 0x1U }  
*LPTMR timer mode selection.*
  - enum `_lptmr_prescaler_glitch_value` {  
`kLPTMR_Prescale_Glitch_0` = 0x0U,  
`kLPTMR_Prescale_Glitch_1` = 0x1U,  
`kLPTMR_Prescale_Glitch_2` = 0x2U,  
`kLPTMR_Prescale_Glitch_3` = 0x3U,  
`kLPTMR_Prescale_Glitch_4` = 0x4U,  
`kLPTMR_Prescale_Glitch_5` = 0x5U,  
`kLPTMR_Prescale_Glitch_6` = 0x6U,  
`kLPTMR_Prescale_Glitch_7` = 0x7U,  
`kLPTMR_Prescale_Glitch_8` = 0x8U,  
`kLPTMR_Prescale_Glitch_9` = 0x9U,  
`kLPTMR_Prescale_Glitch_10` = 0xAU,  
`kLPTMR_Prescale_Glitch_11` = 0xBU,  
`kLPTMR_Prescale_Glitch_12` = 0xCU,  
`kLPTMR_Prescale_Glitch_13` = 0xDU,  
`kLPTMR_Prescale_Glitch_14` = 0xEU,  
`kLPTMR_Prescale_Glitch_15` = 0xFU }  
*LPTMR prescaler/glitch filter values.*
  - enum `_lptmr_prescaler_clock_select` {  
`kLPTMR_PrescalerClock_0` = 0x0U,  
`kLPTMR_PrescalerClock_1` = 0x1U,  
`kLPTMR_PrescalerClock_2` = 0x2U,  
`kLPTMR_PrescalerClock_3` = 0x3U }  
*LPTMR prescaler/glitch filter clock select.*
  - enum `_lptmr_interrupt_enable` { `kLPTMR_TimerInterruptEnable` = LPTMR\_CSR\_TIE\_MASK }
  - enum `_lptmr_status_flags` { `kLPTMR_TimerCompareFlag` = LPTMR\_CSR\_TCF\_MASK }
- List of the LPTMR interrupts.*
- List of the LPTMR status flags.*

## Functions

- static void `LPTMR_EnableTimerDMA` (LPTMR\_Type \*base, bool enable)

*Enable or disable timer DMA request.*

## Driver version

- #define `FSL_LPTMR_DRIVER_VERSION` (`MAKE_VERSION(2, 2, 0)`)  
*Driver Version.*

## Initialization and deinitialization

- void `LPTMR_Init` (`LPTMR_Type *base`, const `lptmr_config_t *config`)  
*Ungates the LPTMR clock and configures the peripheral for a basic operation.*
- void `LPTMR_Deinit` (`LPTMR_Type *base`)  
*Gates the LPTMR clock.*
- void `LPTMR_GetDefaultConfig` (`lptmr_config_t *config`)  
*Fills in the LPTMR configuration structure with default settings.*

## Interrupt Interface

- static void `LPTMR_EnableInterrupts` (`LPTMR_Type *base`, `uint32_t mask`)  
*Enables the selected LPTMR interrupts.*
- static void `LPTMR_DisableInterrupts` (`LPTMR_Type *base`, `uint32_t mask`)  
*Disables the selected LPTMR interrupts.*
- static `uint32_t` `LPTMR_GetEnabledInterrupts` (`LPTMR_Type *base`)  
*Gets the enabled LPTMR interrupts.*

## Status Interface

- static `uint32_t` `LPTMR_GetStatusFlags` (`LPTMR_Type *base`)  
*Gets the LPTMR status flags.*
- static void `LPTMR_ClearStatusFlags` (`LPTMR_Type *base`, `uint32_t mask`)  
*Clears the LPTMR status flags.*

## Read and write the timer period

- static void `LPTMR_SetTimerPeriod` (`LPTMR_Type *base`, `uint32_t ticks`)  
*Sets the timer period in units of count.*
- static `uint32_t` `LPTMR_GetCurrentTimerCount` (`LPTMR_Type *base`)  
*Reads the current timer counting value.*

## Timer Start and Stop

- static void `LPTMR_StartTimer` (`LPTMR_Type *base`)  
*Starts the timer.*
- static void `LPTMR_StopTimer` (`LPTMR_Type *base`)  
*Stops the timer.*

## 25.4 Data Structure Documentation

### 25.4.1 struct \_lptmr\_config

This structure holds the configuration settings for the LPTMR peripheral. To initialize this structure to reasonable defaults, call the [LPTMR\\_GetDefaultConfig\(\)](#) function and pass a pointer to your configuration structure instance.

The configuration struct can be made constant so it resides in flash.

#### Data Fields

- [lptmr\\_timer\\_mode\\_t](#) timerMode  
*Time counter mode or pulse counter mode.*
- [lptmr\\_pin\\_select\\_t](#) pinSelect  
*LPTMR pulse input pin select; used only in pulse counter mode.*
- [lptmr\\_pin\\_polarity\\_t](#) pinPolarity  
*LPTMR pulse input pin polarity; used only in pulse counter mode.*
- bool [enableFreeRunning](#)  
*True: enable free running, counter is reset on overflow False: counter is reset when the compare flag is set.*
- bool [bypassPrescaler](#)  
*True: bypass prescaler; false: use clock from prescaler.*
- [lptmr\\_prescaler\\_clock\\_select\\_t](#) prescalerClockSource  
*LPTMR clock source.*
- [lptmr\\_prescaler\\_glitch\\_value\\_t](#) value  
*Prescaler or glitch filter value.*

## 25.5 Typedef Documentation

### 25.5.1 typedef enum \_lptmr\_pin\_select lptmr\_pin\_select\_t

### 25.5.2 typedef enum \_lptmr\_pin\_polarity lptmr\_pin\_polarity\_t

### 25.5.3 typedef enum \_lptmr\_timer\_mode lptmr\_timer\_mode\_t

### 25.5.4 typedef enum \_lptmr\_prescaler\_clock\_select lptmr\_prescaler\_clock\_select\_t

Note

Clock connections are SoC-specific

### 25.5.5 typedef struct \_lptmr\_config lptmr\_config\_t

This structure holds the configuration settings for the LPTMR peripheral. To initialize this structure to reasonable defaults, call the [LPTMR\\_GetDefaultConfig\(\)](#) function and pass a pointer to your configuration structure instance.

The configuration struct can be made constant so it resides in flash.

## 25.6 Enumeration Type Documentation

### 25.6.1 enum \_lptmr\_pin\_select

Enumerator

*kLPTMR\_PinSelectInput\_0* Pulse counter input 0 is selected.  
*kLPTMR\_PinSelectInput\_1* Pulse counter input 1 is selected.  
*kLPTMR\_PinSelectInput\_2* Pulse counter input 2 is selected.  
*kLPTMR\_PinSelectInput\_3* Pulse counter input 3 is selected.

### 25.6.2 enum \_lptmr\_pin\_polarity

Enumerator

*kLPTMR\_PinPolarityActiveHigh* Pulse Counter input source is active-high.  
*kLPTMR\_PinPolarityActiveLow* Pulse Counter input source is active-low.

### 25.6.3 enum \_lptmr\_timer\_mode

Enumerator

*kLPTMR\_TimerModeTimeCounter* Time Counter mode.  
*kLPTMR\_TimerModePulseCounter* Pulse Counter mode.

### 25.6.4 enum \_lptmr\_prescaler\_glitch\_value

Enumerator

*kLPTMR\_Prescale\_Glitch\_0* Prescaler divide 2, glitch filter does not support this setting.

|                                         |                                              |
|-----------------------------------------|----------------------------------------------|
| <b><i>kLPTMR_Prescale_Glitch_1</i></b>  | Prescaler divide 4, glitch filter 2.         |
| <b><i>kLPTMR_Prescale_Glitch_2</i></b>  | Prescaler divide 8, glitch filter 4.         |
| <b><i>kLPTMR_Prescale_Glitch_3</i></b>  | Prescaler divide 16, glitch filter 8.        |
| <b><i>kLPTMR_Prescale_Glitch_4</i></b>  | Prescaler divide 32, glitch filter 16.       |
| <b><i>kLPTMR_Prescale_Glitch_5</i></b>  | Prescaler divide 64, glitch filter 32.       |
| <b><i>kLPTMR_Prescale_Glitch_6</i></b>  | Prescaler divide 128, glitch filter 64.      |
| <b><i>kLPTMR_Prescale_Glitch_7</i></b>  | Prescaler divide 256, glitch filter 128.     |
| <b><i>kLPTMR_Prescale_Glitch_8</i></b>  | Prescaler divide 512, glitch filter 256.     |
| <b><i>kLPTMR_Prescale_Glitch_9</i></b>  | Prescaler divide 1024, glitch filter 512.    |
| <b><i>kLPTMR_Prescale_Glitch_10</i></b> | Prescaler divide 2048 glitch filter 1024.    |
| <b><i>kLPTMR_Prescale_Glitch_11</i></b> | Prescaler divide 4096, glitch filter 2048.   |
| <b><i>kLPTMR_Prescale_Glitch_12</i></b> | Prescaler divide 8192, glitch filter 4096.   |
| <b><i>kLPTMR_Prescale_Glitch_13</i></b> | Prescaler divide 16384, glitch filter 8192.  |
| <b><i>kLPTMR_Prescale_Glitch_14</i></b> | Prescaler divide 32768, glitch filter 16384. |
| <b><i>kLPTMR_Prescale_Glitch_15</i></b> | Prescaler divide 65536, glitch filter 32768. |

### 25.6.5 enum \_lptmr\_prescaler\_clock\_select

Note

Clock connections are SoC-specific

Enumerator

|                                       |                                           |
|---------------------------------------|-------------------------------------------|
| <b><i>kLPTMR_PrescalerClock_0</i></b> | Prescaler/glitch filter clock 0 selected. |
| <b><i>kLPTMR_PrescalerClock_1</i></b> | Prescaler/glitch filter clock 1 selected. |
| <b><i>kLPTMR_PrescalerClock_2</i></b> | Prescaler/glitch filter clock 2 selected. |
| <b><i>kLPTMR_PrescalerClock_3</i></b> | Prescaler/glitch filter clock 3 selected. |

### 25.6.6 enum \_lptmr\_interrupt\_enable

Enumerator

|                                           |                         |
|-------------------------------------------|-------------------------|
| <b><i>kLPTMR_TimerInterruptEnable</i></b> | Timer interrupt enable. |
|-------------------------------------------|-------------------------|

### 25.6.7 enum \_lptmr\_status\_flags

Enumerator

|                                       |                     |
|---------------------------------------|---------------------|
| <b><i>kLPTMR_TimerCompareFlag</i></b> | Timer compare flag. |
|---------------------------------------|---------------------|

## 25.7 Function Documentation

### 25.7.1 void LPTMR\_Init ( LPTMR\_Type \* *base*, const lptmr\_config\_t \* *config* )

Note

This API should be called at the beginning of the application using the LPTMR driver.

Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>base</i>   | LPTMR peripheral base address                   |
| <i>config</i> | A pointer to the LPTMR configuration structure. |

### 25.7.2 void LPTMR\_Deinit ( LPTMR\_Type \* *base* )

Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPTMR peripheral base address |
|-------------|-------------------------------|

### 25.7.3 void LPTMR\_GetDefaultConfig ( lptmr\_config\_t \* *config* )

The default values are as follows.

```
* config->timerMode = kLPTMR_TimerModeTimeCounter;
* config->pinSelect = kLPTMR_PinSelectInput_0;
* config->pinPolarity = kLPTMR_PinPolarityActiveHigh;
* config->enableFreeRunning = false;
* config->bypassPrescaler = true;
* config->prescalerClockSource = kLPTMR_PrescalerClock_1;
* config->value = kLPTMR_Prescale_Glitch_0;
*
```

Parameters

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>config</i> | A pointer to the LPTMR configuration structure. |
|---------------|-------------------------------------------------|

### 25.7.4 static void LPTMR\_EnableInterrupts ( LPTMR\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                                                       |
|-------------|-----------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | LPTMR peripheral base address                                                                                         |
| <i>mask</i> | The interrupts to enable. This is a logical OR of members of the enumeration <a href="#">lptmr_interrupt_enable_t</a> |

**25.7.5 static void LPTMR\_DisableInterrupts ( LPTMR\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**

## Parameters

|             |                                                                                                                          |
|-------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | LPTMR peripheral base address                                                                                            |
| <i>mask</i> | The interrupts to disable. This is a logical OR of members of the enumeration <a href="#">lptmr_interrupt_enable_t</a> . |

**25.7.6 static uint32\_t LPTMR\_GetEnabledInterrupts ( LPTMR\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPTMR peripheral base address |
|-------------|-------------------------------|

## Returns

The enabled interrupts. This is the logical OR of members of the enumeration [lptmr\\_interrupt\\_enable\\_t](#)

**25.7.7 static void LPTMR\_EnableTimerDMA ( LPTMR\_Type \* *base*, bool *enable* ) [inline], [static]**

## Parameters

|             |                                    |
|-------------|------------------------------------|
| <i>base</i> | base LPTMR peripheral base address |
|-------------|------------------------------------|

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>enable</i> | Switcher of timer DMA feature. "true" means to enable, "false" means to disable. |
|---------------|----------------------------------------------------------------------------------|

### 25.7.8 static uint32\_t LPTMR\_GetStatusFlags ( LPTMR\_Type \* *base* ) [inline], [static]

Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPTMR peripheral base address |
|-------------|-------------------------------|

Returns

The status flags. This is the logical OR of members of the enumeration [lptmr\\_status\\_flags\\_t](#)

### 25.7.9 static void LPTMR\_ClearStatusFlags ( LPTMR\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

Parameters

|             |                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | LPTMR peripheral base address                                                                                        |
| <i>mask</i> | The status flags to clear. This is a logical OR of members of the enumeration <a href="#">lptmr_status_flags_t</a> . |

### 25.7.10 static void LPTMR\_SetTimerPeriod ( LPTMR\_Type \* *base*, uint32\_t *ticks* ) [inline], [static]

Timers counts from 0 until it equals the count value set here. The count value is written to the CMR register.

Note

1. The TCF flag is set with the CNR equals the count provided here and then increments.
2. Call the utility macros provided in the fsl\_common.h to convert to ticks.



## Parameters

|              |                                                                            |
|--------------|----------------------------------------------------------------------------|
| <i>base</i>  | LPTMR peripheral base address                                              |
| <i>ticks</i> | A timer period in units of ticks, which should be equal or greater than 1. |

### 25.7.11 **static uint32\_t LPTMR\_GetCurrentTimerCount ( LPTMR\_Type \* *base* ) [inline], [static]**

This function returns the real-time timer counting value in a range from 0 to a timer period.

## Note

Call the utility macros provided in the fsl\_common.h to convert ticks to usec or msec.

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPTMR peripheral base address |
|-------------|-------------------------------|

## Returns

The current counter value in ticks

### 25.7.12 **static void LPTMR\_StartTimer ( LPTMR\_Type \* *base* ) [inline], [static]**

After calling this function, the timer counts up to the CMR register value. Each time the timer reaches the CMR value and then increments, it generates a trigger pulse and sets the timeout interrupt flag. An interrupt is also triggered if the timer interrupt is enabled.

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPTMR peripheral base address |
|-------------|-------------------------------|

### 25.7.13 **static void LPTMR\_StopTimer ( LPTMR\_Type \* *base* ) [inline], [static]**

This function stops the timer and resets the timer's counter register.

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | LPTMR peripheral base address |
|-------------|-------------------------------|



## Chapter 26

# LPUART: Low Power Universal Asynchronous Receiver/-Transmitter Driver

## 26.1 Overview

### Modules

- [LPUART Driver](#)

## 26.2 LPUART Driver

### 26.2.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Low Power UART (LPUART) module of MCUXpresso SDK devices.

### 26.2.2 Typical use case

#### 26.2.2.1 LPUART Operation

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/lpuart

### Data Structures

- struct [\\_lpuart\\_config](#)  
*LPUART configuration structure. [More...](#)*
- struct [\\_lpuart\\_transfer](#)  
*LPUART transfer structure. [More...](#)*
- struct [\\_lpuart\\_handle](#)  
*LPUART handle structure. [More...](#)*

### Macros

- #define [UART\\_RETRY\\_TIMES](#) 0U /\* Defining to zero means to keep waiting for the flag until it is assert/deassert. \*/  
*Retry times for waiting flag.*

### Typedefs

- typedef enum [\\_lpuart\\_parity\\_mode](#) lpuart\_parity\_mode\_t  
*LPUART parity mode.*
- typedef enum [\\_lpuart\\_data\\_bits](#) lpuart\_data\_bits\_t  
*LPUART data bits count.*
- typedef enum [\\_lpuart\\_stop\\_bit\\_count](#) lpuart\_stop\_bit\_count\_t  
*LPUART stop bit count.*
- typedef enum [\\_lpuart\\_transmit\\_cts\\_source](#) lpuart\_transmit\_cts\_source\_t  
*LPUART transmit CTS source.*
- typedef enum [\\_lpuart\\_transmit\\_cts\\_config](#) lpuart\_transmit\_cts\_config\_t  
*LPUART transmit CTS configure.*
- typedef enum [\\_lpuart\\_idle\\_type\\_select](#) lpuart\_idle\_type\_select\_t

- *LPUART idle flag type defines when the receiver starts counting.*  
typedef enum `_lpuart_idle_config` `lpuart_idle_config_t`  
*LPUART idle detected configuration.*
- typedef struct `_lpuart_config` `lpuart_config_t`  
*LPUART configuration structure.*
- typedef struct `_lpuart_transfer` `lpuart_transfer_t`  
*LPUART transfer structure.*
- typedef void(\* `lpuart_transfer_callback_t`)(LPUART\_Type \*base, `lpuart_handle_t` \*handle, `status_t` status, void \*userData)  
*LPUART transfer callback function.*

## Enumerations

- enum {  
`kStatus_LPUART_TxBusy` = MAKE\_STATUS(kStatusGroup\_LPUART, 0),  
`kStatus_LPUART_RxBusy` = MAKE\_STATUS(kStatusGroup\_LPUART, 1),  
`kStatus_LPUART_TxIdle` = MAKE\_STATUS(kStatusGroup\_LPUART, 2),  
`kStatus_LPUART_RxIdle` = MAKE\_STATUS(kStatusGroup\_LPUART, 3),  
`kStatus_LPUART_TxWatermarkTooLarge` = MAKE\_STATUS(kStatusGroup\_LPUART, 4),  
`kStatus_LPUART_RxWatermarkTooLarge` = MAKE\_STATUS(kStatusGroup\_LPUART, 5),  
`kStatus_LPUART_FlagCannotClearManually` = MAKE\_STATUS(kStatusGroup\_LPUART, 6),  
`kStatus_LPUART_Error` = MAKE\_STATUS(kStatusGroup\_LPUART, 7),  
`kStatus_LPUART_RxRingBufferOverflow`,  
`kStatus_LPUART_RxHardwareOverflow` = MAKE\_STATUS(kStatusGroup\_LPUART, 9),  
`kStatus_LPUART_NoiseError` = MAKE\_STATUS(kStatusGroup\_LPUART, 10),  
`kStatus_LPUART_FramingError` = MAKE\_STATUS(kStatusGroup\_LPUART, 11),  
`kStatus_LPUART_ParityError` = MAKE\_STATUS(kStatusGroup\_LPUART, 12),  
`kStatus_LPUART_BaudrateNotSupport`,  
`kStatus_LPUART_IdleLineDetected` = MAKE\_STATUS(kStatusGroup\_LPUART, 14),  
`kStatus_LPUART_Timeout` = MAKE\_STATUS(kStatusGroup\_LPUART, 15) }  
*Error codes for the LPUART driver.*
- enum `_lpuart_parity_mode` {  
`kLPUART_ParityDisabled` = 0x0U,  
`kLPUART_ParityEven` = 0x2U,  
`kLPUART_ParityOdd` = 0x3U }  
*LPUART parity mode.*
- enum `_lpuart_data_bits` {  
`kLPUART_EightDataBits` = 0x0U,  
`kLPUART_SevenDataBits` = 0x1U }  
*LPUART data bits count.*
- enum `_lpuart_stop_bit_count` {  
`kLPUART_OneStopBit` = 0U,  
`kLPUART_TwoStopBit` = 1U }  
*LPUART stop bit count.*
- enum `_lpuart_transmit_cts_source` {  
`kLPUART_CtsSourcePin` = 0U,

- `kLPUART_CtsSourceMatchResult = 1U }`  
*LPUART transmit CTS source.*
- `enum _lpuart_transmit_cts_config {`  
`kLPUART_CtsSampleAtStart = 0U,`  
`kLPUART_CtsSampleAtIdle = 1U }`  
*LPUART transmit CTS configure.*
- `enum _lpuart_idle_type_select {`  
`kLPUART_IdleTypeStartBit = 0U,`  
`kLPUART_IdleTypeStopBit = 1U }`  
*LPUART idle flag type defines when the receiver starts counting.*
- `enum _lpuart_idle_config {`  
`kLPUART_IdleCharacter1 = 0U,`  
`kLPUART_IdleCharacter2 = 1U,`  
`kLPUART_IdleCharacter4 = 2U,`  
`kLPUART_IdleCharacter8 = 3U,`  
`kLPUART_IdleCharacter16 = 4U,`  
`kLPUART_IdleCharacter32 = 5U,`  
`kLPUART_IdleCharacter64 = 6U,`  
`kLPUART_IdleCharacter128 = 7U }`  
*LPUART idle detected configuration.*
- `enum _lpuart_interrupt_enable {`  
`kLPUART_LinBreakInterruptEnable = (LPUART_BAUD_LBKDIE_MASK >> 8U),`  
`kLPUART_RxActiveEdgeInterruptEnable = (LPUART_BAUD_RXEDGIE_MASK >> 8U),`  
`kLPUART_TxDataRegEmptyInterruptEnable = (LPUART_CTRL_TIE_MASK),`  
`kLPUART_TransmissionCompleteInterruptEnable = (LPUART_CTRL_TCIE_MASK),`  
`kLPUART_RxDataRegFullInterruptEnable = (LPUART_CTRL_RIE_MASK),`  
`kLPUART_IdleLineInterruptEnable = (LPUART_CTRL_ILIE_MASK),`  
`kLPUART_RxOverrunInterruptEnable = (LPUART_CTRL_ORIE_MASK),`  
`kLPUART_NoiseErrorInterruptEnable = (LPUART_CTRL_NEIE_MASK),`  
`kLPUART_FramingErrorInterruptEnable = (LPUART_CTRL_FEIE_MASK),`  
`kLPUART_ParityErrorInterruptEnable = (LPUART_CTRL_PEIE_MASK),`  
`kLPUART_Match1InterruptEnable = (LPUART_CTRL_MA1IE_MASK),`  
`kLPUART_Match2InterruptEnable = (LPUART_CTRL_MA2IE_MASK),`  
`kLPUART_TxFifoOverflowInterruptEnable = (LPUART_FIFO_TXOFE_MASK),`  
`kLPUART_RxFifoUnderflowInterruptEnable = (LPUART_FIFO_RXUFE_MASK) }`  
*LPUART interrupt configuration structure, default settings all disabled.*
- `enum _lpuart_flags {`

```

kLPUART_TxDataRegEmptyFlag,
kLPUART_TransmissionCompleteFlag,
kLPUART_RxDataRegFullFlag = (LPUART_STAT_RDRF_MASK),
kLPUART_IdleLineFlag = (LPUART_STAT_IDLE_MASK),
kLPUART_RxOverrunFlag = (LPUART_STAT_OR_MASK),
kLPUART_NoiseErrorFlag = (LPUART_STAT_NF_MASK),
kLPUART_FramingErrorFlag,
kLPUART_ParityErrorFlag = (LPUART_STAT_PF_MASK),
kLPUART_LinBreakFlag = (LPUART_STAT_LBKDIF_MASK),
kLPUART_RxActiveEdgeFlag = (LPUART_STAT_RXEDGIF_MASK),
kLPUART_RxActiveFlag,
kLPUART_DataMatch1Flag,
kLPUART_DataMatch2Flag,
kLPUART_TxFifoEmptyFlag,
kLPUART_RxFifoEmptyFlag,
kLPUART_TxFifoOverflowFlag,
kLPUART_RxFifoUnderflowFlag }
 LPUART status flags.

```

## Driver version

- #define **FSL\_LPUART\_DRIVER\_VERSION** (**MAKE\_VERSION**(2, 8, 2))  
*LPUART driver version.*

## Software Reset

- static void **LPUART\_SoftwareReset** (LPUART\_Type \*base)  
*Resets the LPUART using software.*

## Initialization and deinitialization

- **status\_t LPUART\_Init** (LPUART\_Type \*base, const **lpuart\_config\_t** \*config, uint32\_t srcClock\_Hz)  
*Initializes an LPUART instance with the user configuration structure and the peripheral clock.*
- void **LPUART\_Deinit** (LPUART\_Type \*base)  
*Deinitializes a LPUART instance.*
- void **LPUART\_GetDefaultConfig** (**lpuart\_config\_t** \*config)  
*Gets the default configuration structure.*

## Module configuration

- **status\_t LPUART\_SetBaudRate** (LPUART\_Type \*base, uint32\_t baudRate\_Bps, uint32\_t srcClock\_Hz)

- *Sets the LPUART instance baudrate.*
- void [LPUART\\_Enable9bitMode](#) (LPUART\_Type \*base, bool enable)  
*Enable 9-bit data mode for LPUART.*
- static void [LPUART\\_SetMatchAddress](#) (LPUART\_Type \*base, uint16\_t address1, uint16\_t address2)  
*Set the LPUART address.*
- static void [LPUART\\_EnableMatchAddress](#) (LPUART\_Type \*base, bool match1, bool match2)  
*Enable the LPUART match address feature.*
- static void [LPUART\\_SetRxFifoWatermark](#) (LPUART\_Type \*base, uint8\_t water)  
*Sets the rx FIFO watermark.*
- static void [LPUART\\_SetTxFifoWatermark](#) (LPUART\_Type \*base, uint8\_t water)  
*Sets the tx FIFO watermark.*
- static void [LPUART\\_TransferEnable16Bit](#) (lpuart\_handle\_t \*handle, bool enable)  
*Sets the LPUART using 16bit transmit, only for 9bit or 10bit mode.*

## Status

- uint32\_t [LPUART\\_GetStatusFlags](#) (LPUART\_Type \*base)  
*Gets LPUART status flags.*
- [status\\_t LPUART\\_ClearStatusFlags](#) (LPUART\_Type \*base, uint32\_t mask)  
*Clears status flags with a provided mask.*

## Interrupts

- void [LPUART\\_EnableInterrupts](#) (LPUART\_Type \*base, uint32\_t mask)  
*Enables LPUART interrupts according to a provided mask.*
- void [LPUART\\_DisableInterrupts](#) (LPUART\_Type \*base, uint32\_t mask)  
*Disables LPUART interrupts according to a provided mask.*
- uint32\_t [LPUART\\_GetEnabledInterrupts](#) (LPUART\_Type \*base)  
*Gets enabled LPUART interrupts.*

## DMA Configuration

- static uintptr\_t [LPUART\\_GetDataRegisterAddress](#) (LPUART\_Type \*base)  
*Gets the LPUART data register address.*
- static void [LPUART\\_EnableTxDMA](#) (LPUART\_Type \*base, bool enable)  
*Enables or disables the LPUART transmitter DMA request.*
- static void [LPUART\\_EnableRxDMA](#) (LPUART\_Type \*base, bool enable)  
*Enables or disables the LPUART receiver DMA.*

## Bus Operations

- uint32\_t [LPUART\\_GetInstance](#) (LPUART\_Type \*base)  
*Get the LPUART instance from peripheral base address.*
- static void [LPUART\\_EnableTx](#) (LPUART\_Type \*base, bool enable)  
*Enables or disables the LPUART transmitter.*



- static void [LPUART\\_EnableRx](#) (LPUART\_Type \*base, bool enable)  
*Enables or disables the LPUART receiver.*
- static void [LPUART\\_WriteByte](#) (LPUART\_Type \*base, uint8\_t data)  
*Writes to the transmitter register.*
- static uint8\_t [LPUART\\_ReadByte](#) (LPUART\_Type \*base)  
*Reads the receiver register.*
- static uint8\_t [LPUART\\_GetRxFifoCount](#) (LPUART\_Type \*base)  
*Gets the rx FIFO data count.*
- static uint8\_t [LPUART\\_GetTxFifoCount](#) (LPUART\_Type \*base)  
*Gets the tx FIFO data count.*
- void [LPUART\\_SendAddress](#) (LPUART\_Type \*base, uint8\_t address)  
*Transmit an address frame in 9-bit data mode.*
- status\_t [LPUART\\_WriteBlocking](#) (LPUART\_Type \*base, const uint8\_t \*data, size\_t length)  
*Writes to the transmitter register using a blocking method.*
- status\_t [LPUART\\_WriteBlocking16bit](#) (LPUART\_Type \*base, const uint16\_t \*data, size\_t length)  
*Writes to the transmitter register using a blocking method in 9bit or 10bit mode.*
- status\_t [LPUART\\_ReadBlocking](#) (LPUART\_Type \*base, uint8\_t \*data, size\_t length)  
*Reads the receiver data register using a blocking method.*
- status\_t [LPUART\\_ReadBlocking16bit](#) (LPUART\_Type \*base, uint16\_t \*data, size\_t length)  
*Reads the receiver data register in 9bit or 10bit mode.*

## Transactional

- void [LPUART\\_TransferCreateHandle](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle, lpuart\_transfer\_callback\_t callback, void \*userData)  
*Initializes the LPUART handle.*
- status\_t [LPUART\\_TransferSendNonBlocking](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle, lpuart\_transfer\_t \*xfer)  
*Transmits a buffer of data using the interrupt method.*
- void [LPUART\\_TransferStartRingBuffer](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle, uint8\_t \*ringBuffer, size\_t ringBufferSize)  
*Sets up the RX ring buffer.*
- void [LPUART\\_TransferStopRingBuffer](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle)  
*Aborts the background transfer and uninstalls the ring buffer.*
- size\_t [LPUART\\_TransferGetRxRingBufferLength](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle)  
*Get the length of received data in RX ring buffer.*
- void [LPUART\\_TransferAbortSend](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle)  
*Aborts the interrupt-driven data transmit.*
- status\_t [LPUART\\_TransferGetSendCount](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle, uint32\_t \*count)  
*Gets the number of bytes that have been sent out to bus.*
- status\_t [LPUART\\_TransferReceiveNonBlocking](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle, lpuart\_transfer\_t \*xfer, size\_t \*receivedBytes)  
*Receives a buffer of data using the interrupt method.*
- void [LPUART\\_TransferAbortReceive](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle)  
*Aborts the interrupt-driven data receiving.*
- status\_t [LPUART\\_TransferGetReceiveCount](#) (LPUART\_Type \*base, lpuart\_handle\_t \*handle, uint32\_t \*count)

- *Gets the number of bytes that have been received.*
- void [LPUART\\_TransferHandleIRQ](#) (LPUART\_Type \*base, void \*irqHandle)  
*LPUART IRQ handle function.*
- void [LPUART\\_TransferHandleErrorIRQ](#) (LPUART\_Type \*base, void \*irqHandle)  
*LPUART Error IRQ handle function.*

## 26.2.3 Data Structure Documentation

### 26.2.3.1 struct \_lpuart\_config

#### Data Fields

- uint32\_t [baudRate\\_Bps](#)  
*LPUART baud rate.*
- [lpuart\\_parity\\_mode\\_t](#) parityMode  
*Parity mode, disabled (default), even, odd.*
- [lpuart\\_data\\_bits\\_t](#) dataBitsCount  
*Data bits count, eight (default), seven.*
- bool [isMsb](#)  
*Data bits order, LSB (default), MSB.*
- [lpuart\\_stop\\_bit\\_count\\_t](#) stopBitCount  
*Number of stop bits, 1 stop bit (default) or 2 stop bits.*
- uint8\_t [txFifoWatermark](#)  
*TX FIFO watermark.*
- uint8\_t [rxFifoWatermark](#)  
*RX FIFO watermark.*
- bool [enableRxRTS](#)  
*RX RTS enable.*
- bool [enableTxCTS](#)  
*TX CTS enable.*
- [lpuart\\_transmit\\_cts\\_source\\_t](#) txCtsSource  
*TX CTS source.*
- [lpuart\\_transmit\\_cts\\_config\\_t](#) txCtsConfig  
*TX CTS configure.*
- [lpuart\\_idle\\_type\\_select\\_t](#) rxIdleType  
*RX IDLE type.*
- [lpuart\\_idle\\_config\\_t](#) rxIdleConfig  
*RX IDLE configuration.*
- bool [enableTx](#)  
*Enable TX.*
- bool [enableRx](#)  
*Enable RX.*

## Field Documentation

(1) `lpuart_idle_type_select_t _lpuart_config::rxIdleType`

(2) `lpuart_idle_config_t _lpuart_config::rxIdleConfig`

### 26.2.3.2 struct \_lpuart\_transfer

#### Data Fields

- `size_t dataSize`  
*The byte count to be transfer.*
- `uint8_t * data`  
*The buffer of data to be transfer.*
- `uint8_t * rxData`  
*The buffer to receive data.*
- `uint16_t * rxData16`  
*The buffer to receive data.*
- `const uint8_t * txData`  
*The buffer of data to be sent.*
- `const uint16_t * txData16`  
*The buffer of data to be sent.*

## Field Documentation

(1) `uint8_t* _lpuart_transfer::data`

(2) `uint8_t* _lpuart_transfer::rxData`

(3) `uint16_t* _lpuart_transfer::rxData16`

(4) `const uint8_t* _lpuart_transfer::txData`

(5) `const uint16_t* _lpuart_transfer::txData16`

(6) `size_t _lpuart_transfer::dataSize`

### 26.2.3.3 struct \_lpuart\_handle

#### Data Fields

- `volatile size_t txDataSize`  
*Size of the remaining data to send.*
- `size_t txDataSizeAll`  
*Size of the data to send out.*
- `volatile size_t rxDataSize`  
*Size of the remaining data to receive.*
- `size_t rxDataSizeAll`  
*Size of the data to receive.*
- `size_t rxRingBufferSize`  
*Size of the ring buffer.*

- volatile uint16\_t [rxRingBufferHead](#)  
*Index for the driver to store received data into ring buffer.*
- volatile uint16\_t [rxRingBufferTail](#)  
*Index for the user to get data from the ring buffer.*
- [lpuart\\_transfer\\_callback\\_t](#) callback  
*Callback function.*
- void \* [userData](#)  
*LPUART callback function parameter.*
- volatile uint8\_t [txState](#)  
*TX transfer state.*
- volatile uint8\_t [rxState](#)  
*RX transfer state.*
- bool [isSevenDataBits](#)  
*Seven data bits flag.*
- bool [is16bitData](#)  
*16bit data bits flag, only used for 9bit or 10bit data*
- const uint8\_t \*volatile [txData](#)  
*Address of remaining data to send.*
- const uint16\_t \*volatile [txData16](#)  
*Address of remaining data to send.*
- uint8\_t \*volatile [rxData](#)  
*Address of remaining data to receive.*
- uint16\_t \*volatile [rxData16](#)  
*Address of remaining data to receive.*
- uint8\_t \* [rxRingBuffer](#)  
*Start address of the receiver ring buffer.*
- uint16\_t \* [rxRingBuffer16](#)  
*Start address of the receiver ring buffer.*



## Field Documentation

- (1) `const uint8_t* volatile _lpuart_handle::txData`
- (2) `const uint16_t* volatile _lpuart_handle::txData16`
- (3) `volatile size_t _lpuart_handle::txDataSize`
- (4) `size_t _lpuart_handle::txDataSizeAll`
- (5) `uint8_t* volatile _lpuart_handle::rxData`
- (6) `uint16_t* volatile _lpuart_handle::rxData16`
- (7) `volatile size_t _lpuart_handle::rxDataSize`
- (8) `size_t _lpuart_handle::rxDataSizeAll`
- (9) `uint8_t* _lpuart_handle::rxRingBuffer`
- (10) `uint16_t* _lpuart_handle::rxRingBuffer16`
- (11) `size_t _lpuart_handle::rxRingBufferSize`
- (12) `volatile uint16_t _lpuart_handle::rxRingBufferHead`
- (13) `volatile uint16_t _lpuart_handle::rxRingBufferTail`
- (14) `lpuart_transfer_callback_t _lpuart_handle::callback`
- (15) `void* _lpuart_handle::userData`
- (16) `volatile uint8_t _lpuart_handle::txState`
- (17) `volatile uint8_t _lpuart_handle::rxState`
- (18) `bool _lpuart_handle::isSevenDataBits`

## 26.2.4 Macro Definition Documentation

26.2.4.1 `#define FSL_LPUART_DRIVER_VERSION (MAKE_VERSION(2, 8, 2))`

26.2.4.2 `#define UART_RETRY_TIMES 0U /* Defining to zero means to keep waiting for the flag until it is assert/deassert. */`

## 26.2.5 Typedef Documentation

26.2.5.1 `typedef enum _lpuart_parity_mode lpuart_parity_mode_t`

26.2.5.2 `typedef enum _lpuart_data_bits lpuart_data_bits_t`

26.2.5.3 `typedef enum _lpuart_stop_bit_count lpuart_stop_bit_count_t`

26.2.5.4 `typedef enum _lpuart_transmit_cts_source lpuart_transmit_cts_source_t`

**26.2.5.8** `typedef struct _lpuart_config lpuart_config_t`

**26.2.5.9** `typedef struct _lpuart_transfer lpuart_transfer_t`

**26.2.5.10** `typedef void(* lpuart_transfer_callback_t)(LPUART_Type *base,  
lpuart_handle_t *handle, status_t status, void *userData)`

## 26.2.6 Enumeration Type Documentation

### 26.2.6.1 anonymous enum

Enumerator

*kStatus\_LPUART\_TxBusy* TX busy.  
*kStatus\_LPUART\_RxBusy* RX busy.  
*kStatus\_LPUART\_TxIdle* LPUART transmitter is idle.  
*kStatus\_LPUART\_RxIdle* LPUART receiver is idle.  
*kStatus\_LPUART\_TxWatermarkTooLarge* TX FIFO watermark too large.  
*kStatus\_LPUART\_RxWatermarkTooLarge* RX FIFO watermark too large.  
*kStatus\_LPUART\_FlagCannotClearManually* Some flag can't manually clear.  
*kStatus\_LPUART\_Error* Error happens on LPUART.  
*kStatus\_LPUART\_RxRingBufferOverflow* LPUART RX software ring buffer overrun.  
*kStatus\_LPUART\_RxHardwareOverflow* LPUART RX receiver overrun.  
*kStatus\_LPUART\_NoiseError* LPUART noise error.  
*kStatus\_LPUART\_FramingError* LPUART framing error.  
*kStatus\_LPUART\_ParityError* LPUART parity error.  
*kStatus\_LPUART\_BaudrateNotSupport* Baudrate is not support in current clock source.  
*kStatus\_LPUART\_IdleLineDetected* IDLE flag.  
*kStatus\_LPUART\_Timeout* LPUART times out.

### 26.2.6.2 enum \_lpuart\_parity\_mode

Enumerator

*kLPUART\_ParityDisabled* Parity disabled.  
*kLPUART\_ParityEven* Parity enabled, type even, bit setting: PE|PT = 10.  
*kLPUART\_ParityOdd* Parity enabled, type odd, bit setting: PE|PT = 11.

### 26.2.6.3 enum \_lpuart\_data\_bits

Enumerator

*kLPUART\_EightDataBits* Eight data bit.  
*kLPUART\_SevenDataBits* Seven data bit.

#### 26.2.6.4 enum \_lpuart\_stop\_bit\_count

Enumerator

*kLPUART\_OneStopBit* One stop bit.  
*kLPUART\_TwoStopBit* Two stop bits.

#### 26.2.6.5 enum \_lpuart\_transmit\_cts\_source

Enumerator

*kLPUART\_CtsSourcePin* CTS resource is the LPUART\_CTS pin.  
*kLPUART\_CtsSourceMatchResult* CTS resource is the match result.

#### 26.2.6.6 enum \_lpuart\_transmit\_cts\_config

Enumerator

*kLPUART\_CtsSampleAtStart* CTS input is sampled at the start of each character.  
*kLPUART\_CtsSampleAtIdle* CTS input is sampled when the transmitter is idle.

#### 26.2.6.7 enum \_lpuart\_idle\_type\_select

Enumerator

*kLPUART\_IdleTypeStartBit* Start counting after a valid start bit.  
*kLPUART\_IdleTypeStopBit* Start counting after a stop bit.

#### 26.2.6.8 enum \_lpuart\_idle\_config

This structure defines the number of idle characters that must be received before the IDLE flag is set.

Enumerator

*kLPUART\_IdleCharacter1* the number of idle characters.  
*kLPUART\_IdleCharacter2* the number of idle characters.  
*kLPUART\_IdleCharacter4* the number of idle characters.  
*kLPUART\_IdleCharacter8* the number of idle characters.  
*kLPUART\_IdleCharacter16* the number of idle characters.  
*kLPUART\_IdleCharacter32* the number of idle characters.  
*kLPUART\_IdleCharacter64* the number of idle characters.  
*kLPUART\_IdleCharacter128* the number of idle characters.



### 26.2.6.9 enum \_lpuart\_interrupt\_enable

This structure contains the settings for all LPUART interrupt configurations.

Enumerator

***kLPUART\_LinBreakInterruptEnable*** LIN break detect. bit 7  
***kLPUART\_RxActiveEdgeInterruptEnable*** Receive Active Edge. bit 6  
***kLPUART\_TxDataRegEmptyInterruptEnable*** Transmit data register empty. bit 23  
***kLPUART\_TransmissionCompleteInterruptEnable*** Transmission complete. bit 22  
***kLPUART\_RxDataRegFullInterruptEnable*** Receiver data register full. bit 21  
***kLPUART\_IdleLineInterruptEnable*** Idle line. bit 20  
***kLPUART\_RxOverrunInterruptEnable*** Receiver Overrun. bit 27  
***kLPUART\_NoiseErrorInterruptEnable*** Noise error flag. bit 26  
***kLPUART\_FramingErrorInterruptEnable*** Framing error flag. bit 25  
***kLPUART\_ParityErrorInterruptEnable*** Parity error flag. bit 24  
***kLPUART\_Match1InterruptEnable*** Parity error flag. bit 15  
***kLPUART\_Match2InterruptEnable*** Parity error flag. bit 14  
***kLPUART\_TxFifoOverflowInterruptEnable*** Transmit FIFO Overflow. bit 9  
***kLPUART\_RxFifoUnderflowInterruptEnable*** Receive FIFO Underflow. bit 8

### 26.2.6.10 enum \_lpuart\_flags

This provides constants for the LPUART status flags for use in the LPUART functions.

Enumerator

***kLPUART\_TxDataRegEmptyFlag*** Transmit data register empty flag, sets when transmit buffer is empty. bit 23  
***kLPUART\_TransmissionCompleteFlag*** Transmission complete flag, sets when transmission activity complete. bit 22  
***kLPUART\_RxDataRegFullFlag*** Receive data register full flag, sets when the receive data buffer is full. bit 21  
***kLPUART\_IdleLineFlag*** Idle line detect flag, sets when idle line detected. bit 20  
***kLPUART\_RxOverrunFlag*** Receive Overrun, sets when new data is received before data is read from receive register. bit 19  
***kLPUART\_NoiseErrorFlag*** Receive takes 3 samples of each received bit. If any of these samples differ, noise flag sets. bit 18  
***kLPUART\_FramingErrorFlag*** Frame error flag, sets if logic 0 was detected where stop bit expected. bit 17  
***kLPUART\_ParityErrorFlag*** If parity enabled, sets upon parity error detection. bit 16  
***kLPUART\_LinBreakFlag*** LIN break detect interrupt flag, sets when LIN break char detected and LIN circuit enabled. bit 31  
***kLPUART\_RxActiveEdgeFlag*** Receive pin active edge interrupt flag, sets when active edge detected. bit 30

- kLPUART\_RxActiveFlag*** Receiver Active Flag (RAF), sets at beginning of valid start. bit 24
- kLPUART\_DataMatch1Flag*** The next character to be read from LPUART\_DATA matches MA1. bit 15
- kLPUART\_DataMatch2Flag*** The next character to be read from LPUART\_DATA matches MA2. bit 14
- kLPUART\_TxFifoEmptyFlag*** TXEMPT bit, sets if transmit buffer is empty. bit 7
- kLPUART\_RxFifoEmptyFlag*** RXEMPT bit, sets if receive buffer is empty. bit 6
- kLPUART\_TxFifoOverflowFlag*** TXOF bit, sets if transmit buffer overflow occurred. bit 1
- kLPUART\_RxFifoUnderflowFlag*** RXUF bit, sets if receive buffer underflow occurred. bit 0

## 26.2.7 Function Documentation

### 26.2.7.1 static void LPUART\_SoftwareReset ( LPUART\_Type \* *base* ) [inline], [static]

This function resets all internal logic and registers except the Global Register. Remains set until cleared by software.

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

### 26.2.7.2 status\_t LPUART\_Init ( LPUART\_Type \* *base*, const lpuart\_config\_t \* *config*, uint32\_t *srcClock\_Hz* )

This function configures the LPUART module with user-defined settings. Call the [LPUART\\_GetDefaultConfig\(\)](#) function to configure the configuration structure and get the default configuration. The example below shows how to use this API to configure the LPUART.

```
* lpuart_config_t lpuartConfig;
* lpuartConfig.baudRate_Bps = 115200U;
* lpuartConfig.parityMode = kLPUART_ParityDisabled;
* lpuartConfig.dataBitsCount = kLPUART_EightDataBits;
* lpuartConfig.isMsb = false;
* lpuartConfig.stopBitCount = kLPUART_OneStopBit;
* lpuartConfig.txFifoWatermark = 0;
* lpuartConfig.rxFifoWatermark = 1;
* LPUART_Init(LPUART1, &lpuartConfig, 20000000U);
*
```

Parameters

|                    |                                                    |
|--------------------|----------------------------------------------------|
| <i>base</i>        | LPUART peripheral base address.                    |
| <i>config</i>      | Pointer to a user-defined configuration structure. |
| <i>srcClock_Hz</i> | LPUART clock source frequency in HZ.               |

Return values

|                                           |                                                  |
|-------------------------------------------|--------------------------------------------------|
| <i>kStatus_LPUART_-BaudrateNotSupport</i> | Baudrate is not support in current clock source. |
| <i>kStatus_Success</i>                    | LPUART initialize succeed                        |

### 26.2.7.3 void LPUART\_Deinit ( LPUART\_Type \* *base* )

This function waits for transmit to complete, disables TX and RX, and disables the LPUART clock.

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

### 26.2.7.4 void LPUART\_GetDefaultConfig ( lpuart\_config\_t \* *config* )

This function initializes the LPUART configuration structure to a default value. The default values are:  
: lpuartConfig->baudRate\_Bps = 115200U; lpuartConfig->parityMode = kLPUART\_ParityDisabled;  
lpuartConfig->dataBitsCount = kLPUART\_EightDataBits; lpuartConfig->isMsb = false; lpuartConfig->stopBitCount = kLPUART\_OneStopBit; lpuartConfig->txFifoWatermark = 0; lpuartConfig->rxFifoWatermark = 1; lpuartConfig->rxIdleType = kLPUART\_IdleTypeStartBit; lpuartConfig->rxIdleConfig = kLPUART\_IdleCharacter1; lpuartConfig->enableTx = false; lpuartConfig->enableRx = false;

Parameters

|               |                                       |
|---------------|---------------------------------------|
| <i>config</i> | Pointer to a configuration structure. |
|---------------|---------------------------------------|

### 26.2.7.5 status\_t LPUART\_SetBaudRate ( LPUART\_Type \* *base*, uint32\_t *baudRate\_Bps*, uint32\_t *srcClock\_Hz* )

This function configures the LPUART module baudrate. This function is used to update the LPUART module baudrate after the LPUART module is initialized by the LPUART\_Init.

```
* LPUART_SetBaudRate(LPUART1, 115200U, 200000000U);
*
```

## Parameters

|                     |                                      |
|---------------------|--------------------------------------|
| <i>base</i>         | LPUART peripheral base address.      |
| <i>baudRate_Bps</i> | LPUART baudrate to be set.           |
| <i>srcClock_Hz</i>  | LPUART clock source frequency in HZ. |

## Return values

|                                           |                                                        |
|-------------------------------------------|--------------------------------------------------------|
| <i>kStatus_LPUART_-BaudrateNotSupport</i> | Baudrate is not supported in the current clock source. |
| <i>kStatus_Success</i>                    | Set baudrate succeeded.                                |

**26.2.7.6 void LPUART\_Enable9bitMode ( LPUART\_Type \* *base*, bool *enable* )**

This function set the 9-bit mode for LPUART module. The 9th bit is not used for parity thus can be modified by user.

## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | LPUART peripheral base address.   |
| <i>enable</i> | true to enable, false to disable. |

**26.2.7.7 static void LPUART\_SetMatchAddress ( LPUART\_Type \* *base*, uint16\_t *address1*, uint16\_t *address2* ) [inline], [static]**

This function configures the address for LPUART module that works as slave in 9-bit data mode. One or two address fields can be configured. When the address field's match enable bit is set, the frame it receives with MSB being 1 is considered as an address frame, otherwise it is considered as data frame. Once the address frame matches one of slave's own addresses, this slave is addressed. This address frame and its following data frames are stored in the receive buffer, otherwise the frames will be discarded. To un-address a slave, just send an address frame with unmatched address.

## Note

Any LPUART instance joined in the multi-slave system can work as slave. The position of the address mark is the same as the parity bit when parity is enabled for 8 bit and 9 bit data formats.

## Parameters

|                 |                                 |
|-----------------|---------------------------------|
| <i>base</i>     | LPUART peripheral base address. |
| <i>address1</i> | LPUART slave address1.          |
| <i>address2</i> | LPUART slave address2.          |

**26.2.7.8 static void LPUART\_EnableMatchAddress ( LPUART\_Type \* *base*, bool *match1*, bool *match2* ) [inline], [static]**

## Parameters

|               |                                                  |
|---------------|--------------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.                  |
| <i>match1</i> | true to enable match address1, false to disable. |
| <i>match2</i> | true to enable match address2, false to disable. |

**26.2.7.9 static void LPUART\_SetRxFifoWatermark ( LPUART\_Type \* *base*, uint8\_t *water* ) [inline], [static]**

## Parameters

|              |                                 |
|--------------|---------------------------------|
| <i>base</i>  | LPUART peripheral base address. |
| <i>water</i> | Rx FIFO watermark.              |

**26.2.7.10 static void LPUART\_SetTxFifoWatermark ( LPUART\_Type \* *base*, uint8\_t *water* ) [inline], [static]**

## Parameters

|              |                                 |
|--------------|---------------------------------|
| <i>base</i>  | LPUART peripheral base address. |
| <i>water</i> | Tx FIFO watermark.              |

**26.2.7.11 static void LPUART\_TransferEnable16Bit ( lpuart\_handle\_t \* *handle*, bool *enable* ) [inline], [static]**

This function Enable 16bit Data transmit in lpuart\_handle\_t.

## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>handle</i> | LPUART handle pointer.            |
| <i>enable</i> | true to enable, false to disable. |

**26.2.7.12 uint32\_t LPUART\_GetStatusFlags ( LPUART\_Type \* *base* )**

This function gets all LPUART status flags. The flags are returned as the logical OR value of the enumerators `_lpuart_flags`. To check for a specific status, compare the return value with enumerators in the `_lpuart_flags`. For example, to check whether the TX is empty:

```
* if (kLPUART_TxDataRegEmptyFlag &
* LPUART_GetStatusFlags(LPUART1))
* {
* ...
* }
*
```

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

## Returns

LPUART status flags which are ORed by the enumerators in the `_lpuart_flags`.

**26.2.7.13 status\_t LPUART\_ClearStatusFlags ( LPUART\_Type \* *base*, uint32\_t *mask* )**

This function clears LPUART status flags with a provided mask. Automatically cleared flags can't be cleared by this function. Flags that can only be cleared or set by hardware are: `kLPUART_TxDataRegEmptyFlag`, `kLPUART_TransmissionCompleteFlag`, `kLPUART_RxDataRegFullFlag`, `kLPUART_RxActiveFlag`, `kLPUART_NoiseErrorFlag`, `kLPUART_ParityErrorFlag`, `kLPUART_TxFifoEmptyFlag`, `kLPUART_RxFifoEmptyFlag`. Note: This API should be called when the Tx/Rx is idle, otherwise it takes no effects.

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

|             |                                                                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>mask</i> | the status flags to be cleared. The user can use the enumerators in the <code>_lpuart_status_flag_t</code> to do the OR operation and get the mask. |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|

## Returns

0 succeed, others failed.

## Return values

|                                                     |                                                                                         |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------|
| <i>kStatus_LPUART_Flag-<br/>CannotClearManually</i> | The flag can't be cleared by this function but it is cleared automatically by hardware. |
| <i>kStatus_Success</i>                              | Status in the mask are cleared.                                                         |

#### 26.2.7.14 void LPUART\_EnableInterrupts ( LPUART\_Type \* *base*, uint32\_t *mask* )

This function enables the LPUART interrupts according to a provided mask. The mask is a logical OR of enumeration members. See the [\\_lpuart\\_interrupt\\_enable](#). This examples shows how to enable TX empty interrupt and RX full interrupt:

```
* LPUART_EnableInterrupts(LPUART1,
 kLPUART_TxDataRegEmptyInterruptEnable |
 kLPUART_RxDataRegFullInterruptEnable);
*
```

## Parameters

|             |                                                                                    |
|-------------|------------------------------------------------------------------------------------|
| <i>base</i> | LPUART peripheral base address.                                                    |
| <i>mask</i> | The interrupts to enable. Logical OR of <a href="#">_lpuart_interrupt_enable</a> . |

#### 26.2.7.15 void LPUART\_DisableInterrupts ( LPUART\_Type \* *base*, uint32\_t *mask* )

This function disables the LPUART interrupts according to a provided mask. The mask is a logical OR of enumeration members. See [\\_lpuart\\_interrupt\\_enable](#). This example shows how to disable the TX empty interrupt and RX full interrupt:

```
* LPUART_DisableInterrupts(LPUART1,
 kLPUART_TxDataRegEmptyInterruptEnable |
 kLPUART_RxDataRegFullInterruptEnable);
*
```

## Parameters

|             |                                                                                     |
|-------------|-------------------------------------------------------------------------------------|
| <i>base</i> | LPUART peripheral base address.                                                     |
| <i>mask</i> | The interrupts to disable. Logical OR of <a href="#">_lpuart_interrupt_enable</a> . |

### 26.2.7.16 uint32\_t LPUART\_GetEnabledInterrupts ( LPUART\_Type \* *base* )

This function gets the enabled LPUART interrupts. The enabled interrupts are returned as the logical OR value of the enumerators [\\_lpuart\\_interrupt\\_enable](#). To check a specific interrupt enable status, compare the return value with enumerators in [\\_lpuart\\_interrupt\\_enable](#). For example, to check whether the TX empty interrupt is enabled:

```
* uint32_t enabledInterrupts = LPUART_GetEnabledInterrupts(LPUART1);
*
* if (kLPUART_TxDataRegEmptyInterruptEnable & enabledInterrupts)
* {
* ...
* }
*
```

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

## Returns

LPUART interrupt flags which are logical OR of the enumerators in [\\_lpuart\\_interrupt\\_enable](#).

### 26.2.7.17 static uintptr\_t LPUART\_GetDataRegisterAddress ( LPUART\_Type \* *base* ) [inline], [static]

This function returns the LPUART data register address, which is mainly used by the DMA/eDMA.

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

## Returns

LPUART data register addresses which are used both by the transmitter and receiver.

### 26.2.7.18 static void LPUART\_EnableTxDMA ( LPUART\_Type \* *base*, bool *enable* ) [inline], [static]

This function enables or disables the transmit data register empty flag, STAT[TDRE], to generate DMA requests.



## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | LPUART peripheral base address.   |
| <i>enable</i> | True to enable, false to disable. |

### 26.2.7.19 static void LPUART\_EnableRxDMA ( LPUART\_Type \* *base*, bool *enable* ) [inline], [static]

This function enables or disables the receiver data register full flag, STAT[RDRF], to generate DMA requests.

## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | LPUART peripheral base address.   |
| <i>enable</i> | True to enable, false to disable. |

### 26.2.7.20 uint32\_t LPUART\_GetInstance ( LPUART\_Type \* *base* )

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

## Returns

LPUART instance.

### 26.2.7.21 static void LPUART\_EnableTx ( LPUART\_Type \* *base*, bool *enable* ) [inline], [static]

This function enables or disables the LPUART transmitter.

## Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | LPUART peripheral base address.   |
| <i>enable</i> | True to enable, false to disable. |

### 26.2.7.22 static void LPUART\_EnableRx ( LPUART\_Type \* *base*, bool *enable* ) [inline], [static]

This function enables or disables the LPUART receiver.

Parameters

|               |                                   |
|---------------|-----------------------------------|
| <i>base</i>   | LPUART peripheral base address.   |
| <i>enable</i> | True to enable, false to disable. |

### 26.2.7.23 static void LPUART\_WriteByte ( LPUART\_Type \* *base*, uint8\_t *data* ) [inline], [static]

This function writes data to the transmitter register directly. The upper layer must ensure that the TX register is empty or that the TX FIFO has room before calling this function.

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
| <i>data</i> | Data write to the TX register.  |

### 26.2.7.24 static uint8\_t LPUART\_ReadByte ( LPUART\_Type \* *base* ) [inline], [static]

This function reads data from the receiver register directly. The upper layer must ensure that the receiver register is full or that the RX FIFO has data before calling this function.

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

Returns

Data read from data register.

### 26.2.7.25 static uint8\_t LPUART\_GetRxFifoCount ( LPUART\_Type \* *base* ) [inline], [static]

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

Returns

rx FIFO data count.

**26.2.7.26** `static uint8_t LPUART_GetTxFifoCount ( LPUART_Type * base ) [inline],  
[static]`

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | LPUART peripheral base address. |
|-------------|---------------------------------|

Returns

tx FIFO data count.

**26.2.7.27** `void LPUART_SendAddress ( LPUART_Type * base, uint8_t address )`

Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | LPUART peripheral base address. |
| <i>address</i> | LPUART slave address.           |

**26.2.7.28** `status_t LPUART_WriteBlocking ( LPUART_Type * base, const uint8_t * data,  
size_t length )`

This function polls the transmitter register, first waits for the register to be empty or TX FIFO to have room, and writes data to the transmitter buffer, then waits for the data to be sent out to the bus.

Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | LPUART peripheral base address.     |
| <i>data</i>   | Start address of the data to write. |
| <i>length</i> | Size of the data to write.          |

#### Return values

|                               |                                         |
|-------------------------------|-----------------------------------------|
| <i>kStatus_LPUART_Timeout</i> | Transmission timed out and was aborted. |
| <i>kStatus_Success</i>        | Successfully wrote all data.            |

#### 26.2.7.29 **status\_t LPUART\_WriteBlocking16bit ( LPUART\_Type \* *base*, const uint16\_t \* *data*, size\_t *length* )**

##### Note

This function only support 9bit or 10bit transfer. Please make sure only 10bit of data is valid and other bits are 0.

#### Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | LPUART peripheral base address.     |
| <i>data</i>   | Start address of the data to write. |
| <i>length</i> | Size of the data to write.          |

#### Return values

|                               |                                         |
|-------------------------------|-----------------------------------------|
| <i>kStatus_LPUART_Timeout</i> | Transmission timed out and was aborted. |
| <i>kStatus_Success</i>        | Successfully wrote all data.            |

#### 26.2.7.30 **status\_t LPUART\_ReadBlocking ( LPUART\_Type \* *base*, uint8\_t \* *data*, size\_t *length* )**

This function polls the receiver register, waits for the receiver register full or receiver FIFO has data, and reads data from the TX register.

#### Parameters

|               |                                                         |
|---------------|---------------------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.                         |
| <i>data</i>   | Start address of the buffer to store the received data. |
| <i>length</i> | Size of the buffer.                                     |

## Return values

|                                          |                                                 |
|------------------------------------------|-------------------------------------------------|
| <i>kStatus_LPUART_Rx-HardwareOverrun</i> | Receiver overrun happened while receiving data. |
| <i>kStatus_LPUART_Noise-Error</i>        | Noise error happened while receiving data.      |
| <i>kStatus_LPUART_-FramingError</i>      | Framing error happened while receiving data.    |
| <i>kStatus_LPUART_Parity-Error</i>       | Parity error happened while receiving data.     |
| <i>kStatus_LPUART_-Timeout</i>           | Transmission timed out and was aborted.         |
| <i>kStatus_Success</i>                   | Successfully received all data.                 |

### 26.2.7.31 **status\_t LPUART\_ReadBlocking16bit ( LPUART\_Type \* *base*, uint16\_t \* *data*, size\_t *length* )**

## Note

This function only support 9bit or 10bit transfer.

## Parameters

|               |                                                                                       |
|---------------|---------------------------------------------------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.                                                       |
| <i>data</i>   | Start address of the buffer to store the received data by 16bit, only 10bit is valid. |
| <i>length</i> | Size of the buffer.                                                                   |

## Return values

|                                          |                                                 |
|------------------------------------------|-------------------------------------------------|
| <i>kStatus_LPUART_Rx-HardwareOverrun</i> | Receiver overrun happened while receiving data. |
| <i>kStatus_LPUART_Noise-Error</i>        | Noise error happened while receiving data.      |
| <i>kStatus_LPUART_-FramingError</i>      | Framing error happened while receiving data.    |
| <i>kStatus_LPUART_Parity-Error</i>       | Parity error happened while receiving data.     |
| <i>kStatus_LPUART_-Timeout</i>           | Transmission timed out and was aborted.         |
| <i>kStatus_Success</i>                   | Successfully received all data.                 |

### 26.2.7.32 void LPUART\_TransferCreateHandle ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle*, lpuart\_transfer\_callback\_t *callback*, void \* *userData* )

This function initializes the LPUART handle, which can be used for other LPUART transactional APIs. Usually, for a specified LPUART instance, call this API once to get the initialized handle.

The LPUART driver supports the "background" receiving, which means that user can set up an RX ring buffer optionally. Data received is stored into the ring buffer even when the user doesn't call the [LPUART\\_TransferReceiveNonBlocking\(\)](#) API. If there is already data received in the ring buffer, the user can get the received data from the ring buffer directly. The ring buffer is disabled if passing NULL as *ringBuffer*.

Parameters

|                 |                                 |
|-----------------|---------------------------------|
| <i>base</i>     | LPUART peripheral base address. |
| <i>handle</i>   | LPUART handle pointer.          |
| <i>callback</i> | Callback function.              |
| <i>userData</i> | User data.                      |

### 26.2.7.33 status\_t LPUART\_TransferSendNonBlocking ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle*, lpuart\_transfer\_t \* *xfer* )

This function send data using an interrupt method. This is a non-blocking function, which returns directly without waiting for all data written to the transmitter register. When all data is written to the TX register in the ISR, the LPUART driver calls the callback function and passes the [kStatus\\_LPUART\\_TxIdle](#) as status parameter.

Note

The [kStatus\\_LPUART\\_TxIdle](#) is passed to the upper layer when all data are written to the TX register. However, there is no check to ensure that all the data sent out. Before disabling the T-X, check the [kLPUART\\_TransmissionCompleteFlag](#) to ensure that the transmit is finished.

Parameters

|               |                                                                    |
|---------------|--------------------------------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.                                    |
| <i>handle</i> | LPUART handle pointer.                                             |
| <i>xfer</i>   | LPUART transfer structure, see <a href="#">lpuart_transfer_t</a> . |

Return values

|                                |                                                                                    |
|--------------------------------|------------------------------------------------------------------------------------|
| <i>kStatus_Success</i>         | Successfully start the data transmission.                                          |
| <i>kStatus_LPUART_TxBusy</i>   | Previous transmission still not finished, data not all written to the TX register. |
| <i>kStatus_InvalidArgument</i> | Invalid argument.                                                                  |

#### 26.2.7.34 void LPUART\_TransferStartRingBuffer ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle*, uint8\_t \* *ringBuffer*, size\_t *ringBufferSize* )

This function sets up the RX ring buffer to a specific UART handle.

When the RX ring buffer is used, data received is stored into the ring buffer even when the user doesn't call the UART\_TransferReceiveNonBlocking() API. If there is already data received in the ring buffer, the user can get the received data from the ring buffer directly.

Note

When using RX ring buffer, one byte is reserved for internal use. In other words, if *ringBufferSize* is 32, then only 31 bytes are used for saving data.

Parameters

|                       |                                                                                              |
|-----------------------|----------------------------------------------------------------------------------------------|
| <i>base</i>           | LPUART peripheral base address.                                                              |
| <i>handle</i>         | LPUART handle pointer.                                                                       |
| <i>ringBuffer</i>     | Start address of ring buffer for background receiving. Pass NULL to disable the ring buffer. |
| <i>ringBufferSize</i> | size of the ring buffer.                                                                     |

#### 26.2.7.35 void LPUART\_TransferStopRingBuffer ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle* )

This function aborts the background transfer and uninstalls the ring buffer.

Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |

**26.2.7.36**   **size\_t** LPUART\_TransferGetRxRingBufferLength ( LPUART\_Type \* *base*,  
lpuart\_handle\_t \* *handle* )



## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |

## Returns

Length of received data in RX ring buffer.

### 26.2.7.37 void LPUART\_TransferAbortSend ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle* )

This function aborts the interrupt driven data sending. The user can get the remainBtyes to find out how many bytes are not sent out.

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |

### 26.2.7.38 status\_t LPUART\_TransferGetSendCount ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle*, uint32\_t \* *count* )

This function gets the number of bytes that have been sent out to bus by an interrupt method.

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |
| <i>count</i>  | Send bytes count.               |

## Return values

|                                     |                                               |
|-------------------------------------|-----------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | No send in progress.                          |
| <i>kStatus_InvalidArgument</i>      | Parameter is invalid.                         |
| <i>kStatus_Success</i>              | Get successfully through the parameter count; |

### 26.2.7.39 **status\_t LPUART\_TransferReceiveNonBlocking ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle*, lpuart\_transfer\_t \* *xfer*, size\_t \* *receivedBytes* )**

This function receives data using an interrupt method. This is a non-blocking function which returns without waiting to ensure that all data are received. If the RX ring buffer is used and not empty, the data in the ring buffer is copied and the parameter *receivedBytes* shows how many bytes are copied from the ring buffer. After copying, if the data in the ring buffer is not enough for read, the receive request is saved by the LPUART driver. When the new data arrives, the receive request is serviced first. When all data is received, the LPUART driver notifies the upper layer through a callback function and passes a status parameter *kStatus\_UART\_RxIdle*. For example, the upper layer needs 10 bytes but there are only 5 bytes in ring buffer. The 5 bytes are copied to *xfer->data*, which returns with the parameter *receivedBytes* set to 5. For the remaining 5 bytes, the newly arrived data is saved from *xfer->data[5]*. When 5 bytes are received, the LPUART driver notifies the upper layer. If the RX ring buffer is not enabled, this function enables the RX and RX interrupt to receive data to *xfer->data*. When all data is received, the upper layer is notified.

Parameters

|                      |                                                         |
|----------------------|---------------------------------------------------------|
| <i>base</i>          | LPUART peripheral base address.                         |
| <i>handle</i>        | LPUART handle pointer.                                  |
| <i>xfer</i>          | LPUART transfer structure, see <i>uart_transfer_t</i> . |
| <i>receivedBytes</i> | Bytes received from the ring buffer directly.           |

Return values

|                                |                                                          |
|--------------------------------|----------------------------------------------------------|
| <i>kStatus_Success</i>         | Successfully queue the transfer into the transmit queue. |
| <i>kStatus_LPUART_Rx-Busy</i>  | Previous receive request is not finished.                |
| <i>kStatus_InvalidArgument</i> | Invalid argument.                                        |

### 26.2.7.40 **void LPUART\_TransferAbortReceive ( LPUART\_Type \* *base*, lpuart\_handle\_t \* *handle* )**

This function aborts the interrupt-driven data receiving. The user can get the *remainBytes* to find out how many bytes not received yet.

Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |

**26.2.7.41** `status_t LPUART_TransferGetReceiveCount ( LPUART_Type * base,  
lpuart_handle_t * handle, uint32_t * count )`

This function gets the number of bytes that have been received.

#### Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |
| <i>count</i>  | Receive bytes count.            |

#### Return values

|                                     |                                               |
|-------------------------------------|-----------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | No receive in progress.                       |
| <i>kStatus_InvalidArgument</i>      | Parameter is invalid.                         |
| <i>kStatus_Success</i>              | Get successfully through the parameter count; |

#### 26.2.7.42 void LPUART\_TransferHandleIRQ ( LPUART\_Type \* *base*, void \* *irqHandle* )

This function handles the LPUART transmit and receive IRQ request.

#### Parameters

|                  |                                 |
|------------------|---------------------------------|
| <i>base</i>      | LPUART peripheral base address. |
| <i>irqHandle</i> | LPUART handle pointer.          |

#### 26.2.7.43 void LPUART\_TransferHandleErrorIRQ ( LPUART\_Type \* *base*, void \* *irqHandle* )

This function handles the LPUART error IRQ request.

#### Parameters

|                  |                                 |
|------------------|---------------------------------|
| <i>base</i>      | LPUART peripheral base address. |
| <i>irqHandle</i> | LPUART handle pointer.          |

## Chapter 27

# LTC: LP Trusted Cryptography

### 27.1 Overview

The MCUXpresso SDK provides a peripheral driver for the LP Trusted Cryptography (LTC) module of MCUXpresso SDK devices. LP Trusted Cryptography is a set of cryptographic hardware accelerator engines that share common registers. LTC architecture can support AES, DES, 3DES, MDHA (SHA), RSA, and ECC. The actual list of implemented cryptographic hardware accelerator engines depends on the specific microcontroller.

The driver comprises two sets of API functions.

In the first set, blocking synchronous APIs are provided, for all operations supported by LTC hardware. The LTC operations are complete (and results are made available for further usage) when a function returns. When called, these functions do not return until an LTC operation is complete. These functions use main CPU for simple polling loops to determine operation complete or error status and also for plaintext or ciphertext data movements. The driver functions are not re-entrant. These functions provide typical interface to upper layer or application software.

In the second set, DMA support for symmetric LTC processing is provided, for AES and DES engines. APIs in the second set use DMA for data movement to and from the LTC input and output FIFOs. By using these functions, main CPU is not used for plaintext or ciphertext data movements (DMA is used instead). Thus, CPU processing power can be used for other application tasks, at cost of decreased maximum data throughput (because of DMA module and transactions management overhead). These functions provide less typical interface, for applications that must offload main CPU while ciphertext or plaintext is being processed, at cost of longer cryptographic processing time.

### 27.2 LTC Driver Initialization and Configuration

LTC Driver is initialized by calling the [LTC\\_Init\(\)](#) function, it enables the LTC module clock in the SIM module. If AES or DES engine is used and the LTC module implementation features the LTC DPA Mask Seed register, seed the DPA mask generator by using the seed from a random number generator. The [LTC\\_SetDpaMaskSeed\(\)](#) function is provided to set the DPA mask seed.

### 27.3 Comments about API usage in RTOS

LTC operations provided by this driver are not re-entrant. Thus, application software shall ensure the LTC module operation is not requested from different tasks or interrupt service routines while an operation is in progress.

### 27.4 Comments about API usage in interrupt handler

All APIs can be used from interrupt handler although execution time shall be considered (interrupt latency of equal and lower priority interrupts increases).

## 27.5 LTC Driver Examples

### 27.5.1 Simple examples

Initialize LTC after Power On Reset or reset cycle Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/ltcEncrypt plaintext by DES engine Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/ltc Encrypt plaintext by AES engine Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/ltc Compute keyed hash by AES engine (CMAC) Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/ltc Compute hash by MDHA engine (SHA-256) Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/ltc Compute modular integer exponentiation Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/ltc Compute elliptic curve point multiplication Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/ltc

### Modules

- [LTC Blocking APIs](#)

### Functions

- void [LTC\\_Init](#) (LTC\_Type \*base)  
*Initializes the LTC driver.*
- void [LTC\\_Deinit](#) (LTC\_Type \*base)  
*Deinitializes the LTC driver.*

### Driver version

- #define [FSL\\_LTC\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 0, 16))  
*LTC driver version.*

## 27.6 Macro Definition Documentation

### 27.6.1 #define FSL\_LTC\_DRIVER\_VERSION (MAKE\_VERSION(2, 0, 16))

Version 2.0.16.

Current version: 2.0.16

Change log:

- Version 2.0.1
  - fixed warning during g++ compilation
- Version 2.0.2
  - fixed [KPSDK-10932][LTC][SHA] [LTC\\_HASH\(\)](#) blocks indefinitely when message size exceeds 4080 bytes
- Version 2.0.3
  - fixed LTC\_PKHA\_CompareBigNum() in case an integer argument is an array of all zeroes

- Version 2.0.4
  - constant LTC\_PKHA\_CompareBigNum() processing time
- Version 2.0.5
  - Fix MISRA issues
- Version 2.0.6
  - fixed [KPSDK-23603][LTC] AES Decrypt in ECB and CBC modes fail when ciphertext size > 0xff0 bytes
- Version 2.0.7
  - Fix MISRA-2012 issues
- Version 2.0.8
  - Fix Coverity issues
- Version 2.0.9
  - Fix sign-compare warning in ltc\_set\_context and in ltc\_get\_context
- Version 2.0.10
  - Fix MISRA-2012 issues
- Version 2.0.11
  - Fix MISRA-2012 issues
- Version 2.0.12
  - Fix AES Decrypt in CBC modes fail when used kLTC\_DeCryptKey.
- Version 2.0.13
  - Add feature macro FSL\_FEATURE\_LTC\_HAS\_NO\_CLOCK\_CONTROL\_BIT into LTC\_Init function.
- Version 2.0.14
  - Add feature macro FSL\_FEATURE\_LTC\_HAS\_NO\_CLOCK\_CONTROL\_BIT into LTC\_Deinit function.
- Version 2.0.15
  - Fix MISRA-2012 issues
- Version 2.0.16
  - Fix uninitialized GCC warning in [LTC\\_AES\\_GenerateDecryptKey\(\)](#)

## 27.7 Function Documentation

### 27.7.1 void LTC\_Init ( LTC\_Type \* *base* )

This function initializes the LTC driver.

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | LTC peripheral base address |
|-------------|-----------------------------|

### 27.7.2 void LTC\_Deinit ( LTC\_Type \* *base* )

This function deinitializes the LTC driver.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | LTC peripheral base address |
|-------------|-----------------------------|



## 27.8 LTC Blocking APIs

### 27.8.1 Overview

This section describes the programming interface of the LTC Synchronous Blocking functions

#### Modules

- [LTC AES driver](#)
- [LTC DES driver](#)
- [LTC HASH driver](#)
- [LTC PKHA driver](#)

## 27.8.2 LTC DES driver

### 27.8.2.1 Overview

This section describes the programming interface of the LTC DES driver.

#### Macros

- #define [LTC\\_DES\\_KEY\\_SIZE](#) 8  
*LTC DES key size - 64 bits.*
- #define [LTC\\_DES\\_IV\\_SIZE](#) 8  
*LTC DES IV size - 8 bytes.*

### 27.8.2.2 Macro Definition Documentation

#### 27.8.2.2.1 #define LTC\_DES\_KEY\_SIZE 8

## 27.8.3 LTC AES driver

### 27.8.3.1 Overview

This section describes the programming interface of the LTC AES driver.

#### Macros

- `#define LTC_AES_BLOCK_SIZE 16U`  
*AES block size in bytes.*
- `#define LTC_AES_IV_SIZE 16`  
*AES Input Vector size in bytes.*
- `#define LTC_AES_DecryptCtr(base, input, output, size, counter, key, keySize, counterlast, szLeft) LTC_AES_CryptCtr(base, input, output, size, counter, key, keySize, counterlast, szLeft)`  
*AES CTR decrypt is mapped to the AES CTR generic operation.*
- `#define LTC_AES_EncryptCtr(base, input, output, size, counter, key, keySize, counterlast, szLeft) LTC_AES_CryptCtr(base, input, output, size, counter, key, keySize, counterlast, szLeft)`  
*AES CTR encrypt is mapped to the AES CTR generic operation.*

#### Typedefs

- `typedef enum _ltc_aes_key_t ltc_aes_key_t`  
*Type of AES key for ECB and CBC decrypt operations.*

#### Enumerations

- `enum _ltc_aes_key_t {`  
    `kLTC_EncryptKey = 0U,`  
    `kLTC_DecryptKey = 1U }`  
*Type of AES key for ECB and CBC decrypt operations.*

#### Functions

- `status_t LTC_AES_GenerateDecryptKey (LTC_Type *base, const uint8_t *encryptKey, uint8_t *decryptKey, uint32_t keySize)`  
*Transforms an AES encrypt key (forward AES) into the decrypt key (inverse AES).*
- `status_t LTC_AES_EncryptEcb (LTC_Type *base, const uint8_t *plaintext, uint8_t *ciphertext, uint32_t size, const uint8_t *key, uint32_t keySize)`  
*Encrypts AES using the ECB block mode.*
- `status_t LTC_AES_DecryptEcb (LTC_Type *base, const uint8_t *ciphertext, uint8_t *plaintext, uint32_t size, const uint8_t *key, uint32_t keySize, ltc_aes_key_t keyType)`  
*Decrypts AES using ECB block mode.*
- `status_t LTC_AES_EncryptCbc (LTC_Type *base, const uint8_t *plaintext, uint8_t *ciphertext, uint32_t size, const uint8_t iv[LTC_AES_IV_SIZE], const uint8_t *key, uint32_t keySize)`  
*Encrypts AES using CBC block mode.*

- **status\_t LTC\_AES\_DecryptCbc** (LTC\_Type \*base, const uint8\_t \*ciphertext, uint8\_t \*plaintext, uint32\_t size, const uint8\_t iv[LTC\_AES\_IV\_SIZE], const uint8\_t \*key, uint32\_t keySize, **ltc\_aes\_key\_t** keyType)  
*Decrypts AES using CBC block mode.*
- **status\_t LTC\_AES\_CryptCtr** (LTC\_Type \*base, const uint8\_t \*input, uint8\_t \*output, uint32\_t size, uint8\_t counter[LTC\_AES\_BLOCK\_SIZE], const uint8\_t \*key, uint32\_t keySize, uint8\_t counterlast[LTC\_AES\_BLOCK\_SIZE], uint32\_t \*szLeft)  
*Encrypts or decrypts AES using CTR block mode.*
- **status\_t LTC\_AES\_EncryptTagCcm** (LTC\_Type \*base, const uint8\_t \*plaintext, uint8\_t \*ciphertext, uint32\_t size, const uint8\_t \*iv, uint32\_t ivSize, const uint8\_t \*aad, uint32\_t aadSize, const uint8\_t \*key, uint32\_t keySize, uint8\_t \*tag, uint32\_t tagSize)  
*Encrypts AES and tags using CCM block mode.*
- **status\_t LTC\_AES\_DecryptTagCcm** (LTC\_Type \*base, const uint8\_t \*ciphertext, uint8\_t \*plaintext, uint32\_t size, const uint8\_t \*iv, uint32\_t ivSize, const uint8\_t \*aad, uint32\_t aadSize, const uint8\_t \*key, uint32\_t keySize, const uint8\_t \*tag, uint32\_t tagSize)  
*Decrypts AES and authenticates using CCM block mode.*

### 27.8.3.2 Typedef Documentation

#### 27.8.3.2.1 typedef enum \_ltc\_aes\_key\_t ltc\_aes\_key\_t

### 27.8.3.3 Enumeration Type Documentation

#### 27.8.3.3.1 enum \_ltc\_aes\_key\_t

Enumerator

**kLTC\_EncryptKey** Input key is an encrypt key.

**kLTC\_DecryptKey** Input key is a decrypt key.

### 27.8.3.4 Function Documentation

#### 27.8.3.4.1 status\_t LTC\_AES\_GenerateDecryptKey ( LTC\_Type \* base, const uint8\_t \* encryptKey, uint8\_t \* decryptKey, uint32\_t keySize )

Transforms the AES encrypt key (forward AES) into the decrypt key (inverse AES). The key derived by this function can be used as a direct load decrypt key for AES ECB and CBC decryption operations (keyType argument).

Parameters

---

|     |                   |                                                                       |
|-----|-------------------|-----------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                           |
|     | <i>encryptKey</i> | Input key for decrypt key transformation                              |
| out | <i>decryptKey</i> | Output key, the decrypt form of the AES key.                          |
|     | <i>keySize</i>    | Size of the input key and output key in bytes. Must be 16, 24, or 32. |

Returns

Status from key generation operation

**27.8.3.4.2** `status_t LTC_AES_EncryptEcb ( LTC_Type * base, const uint8_t * plaintext, uint8_t * ciphertext, uint32_t size, const uint8_t * key, uint32_t keySize )`

Encrypts AES using the ECB block mode.

Parameters

|     |                   |                                                                       |
|-----|-------------------|-----------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                           |
|     | <i>plaintext</i>  | Input plain text to encrypt                                           |
| out | <i>ciphertext</i> | Output cipher text                                                    |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes. |
|     | <i>key</i>        | Input key to use for encryption                                       |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.               |

Returns

Status from encrypt operation

**27.8.3.4.3** `status_t LTC_AES_DecryptEcb ( LTC_Type * base, const uint8_t * ciphertext, uint8_t * plaintext, uint32_t size, const uint8_t * key, uint32_t keySize, ltc_aes_key_t keyType )`

Decrypts AES using ECB block mode.

Parameters

|     |                   |                                                                                            |
|-----|-------------------|--------------------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                                |
|     | <i>ciphertext</i> | Input cipher text to decrypt                                                               |
| out | <i>plaintext</i>  | Output plain text                                                                          |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes.                      |
|     | <i>key</i>        | Input key.                                                                                 |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.                                    |
|     | <i>keyType</i>    | Input type of the key (allows to directly load decrypt key for AES ECB decrypt operation.) |

## Returns

Status from decrypt operation

**27.8.3.4.4** `status_t LTC_AES_EncryptCbc ( LTC_Type * base, const uint8_t * plaintext, uint8_t * ciphertext, uint32_t size, const uint8_t iv[LTC_AES_IV_SIZE], const uint8_t * key, uint32_t keySize )`

## Parameters

|     |                   |                                                                       |
|-----|-------------------|-----------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                           |
|     | <i>plaintext</i>  | Input plain text to encrypt                                           |
| out | <i>ciphertext</i> | Output cipher text                                                    |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes. |
|     | <i>iv</i>         | Input initial vector to combine with the first input block.           |
|     | <i>key</i>        | Input key to use for encryption                                       |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.               |

## Returns

Status from encrypt operation

**27.8.3.4.5** `status_t LTC_AES_DecryptCbc ( LTC_Type * base, const uint8_t * ciphertext, uint8_t * plaintext, uint32_t size, const uint8_t iv[LTC_AES_IV_SIZE], const uint8_t * key, uint32_t keySize, ltc_aes_key_t keyType )`

## Parameters

|     |                   |                                                                                            |
|-----|-------------------|--------------------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                                |
|     | <i>ciphertext</i> | Input cipher text to decrypt                                                               |
| out | <i>plaintext</i>  | Output plain text                                                                          |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes.                      |
|     | <i>iv</i>         | Input initial vector to combine with the first input block.                                |
|     | <i>key</i>        | Input key to use for decryption                                                            |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.                                    |
|     | <i>keyType</i>    | Input type of the key (allows to directly load decrypt key for AES CBC decrypt operation.) |

## Returns

Status from decrypt operation

**27.8.3.4.6** `status_t LTC_AES_CryptCtr ( LTC_Type * base, const uint8_t * input, uint8_t * output, uint32_t size, uint8_t counter[LTC_AES_BLOCK_SIZE], const uint8_t * key, uint32_t keySize, uint8_t counterlast[LTC_AES_BLOCK_SIZE], uint32_t * szLeft )`

Encrypts or decrypts AES using CTR block mode. AES CTR mode uses only forward AES cipher and same algorithm for encryption and decryption. The only difference between encryption and decryption is that, for encryption, the input argument is plain text and the output argument is cipher text. For decryption, the input argument is cipher text and the output argument is plain text.

## Parameters

|         |                |                                         |
|---------|----------------|-----------------------------------------|
|         | <i>base</i>    | LTC peripheral base address             |
|         | <i>input</i>   | Input data for CTR block mode           |
| out     | <i>output</i>  | Output data for CTR block mode          |
|         | <i>size</i>    | Size of input and output data in bytes  |
| in, out | <i>counter</i> | Input counter (updates on return)       |
|         | <i>key</i>     | Input key to use for forward AES cipher |

|     |                    |                                                                                                               |
|-----|--------------------|---------------------------------------------------------------------------------------------------------------|
|     | <i>keySize</i>     | Size of the input key, in bytes. Must be 16, 24, or 32.                                                       |
| out | <i>counterlast</i> | Output cipher of last counter, for chained CTR calls. NULL can be passed if chained calls are not used.       |
| out | <i>szLeft</i>      | Output number of bytes in left unused in counterlast block. NULL can be passed if chained calls are not used. |

## Returns

Status from encrypt operation

**27.8.3.4.7** `status_t LTC_AES_EncryptTagCcm ( LTC_Type * base, const uint8_t * plaintext, uint8_t * ciphertext, uint32_t size, const uint8_t * iv, uint32_t ivSize, const uint8_t * aad, uint32_t aadSize, const uint8_t * key, uint32_t keySize, uint8_t * tag, uint32_t tagSize )`

Encrypts AES and optionally tags using CCM block mode.

## Parameters

|     |                   |                                                                                  |
|-----|-------------------|----------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                      |
|     | <i>plaintext</i>  | Input plain text to encrypt                                                      |
| out | <i>ciphertext</i> | Output cipher text.                                                              |
|     | <i>size</i>       | Size of input and output data in bytes. Zero means authentication only.          |
|     | <i>iv</i>         | Nonce                                                                            |
|     | <i>ivSize</i>     | Length of the Nonce in bytes. Must be 7, 8, 9, 10, 11, 12, or 13.                |
|     | <i>aad</i>        | Input additional authentication data. Can be NULL if aadSize is zero.            |
|     | <i>aadSize</i>    | Input size in bytes of AAD. Zero means data mode only (authentication skipped).  |
|     | <i>key</i>        | Input key to use for encryption                                                  |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.                          |
| out | <i>tag</i>        | Generated output tag. Set to NULL to skip tag processing.                        |
|     | <i>tagSize</i>    | Input size of the tag to generate, in bytes. Must be 4, 6, 8, 10, 12, 14, or 16. |

## Returns

Status from encrypt operation



**27.8.3.4.8** `status_t LTC_AES_DecryptTagCcm ( LTC_Type * base, const uint8_t * ciphertext,  
uint8_t * plaintext, uint32_t size, const uint8_t * iv, uint32_t ivSize, const uint8_t *  
aad, uint32_t aadSize, const uint8_t * key, uint32_t keySize, const uint8_t * tag,  
uint32_t tagSize )`

Decrypts AES and optionally authenticates using CCM block mode.

## Parameters

|     |                   |                                                                                                                |
|-----|-------------------|----------------------------------------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                                                    |
|     | <i>ciphertext</i> | Input cipher text to decrypt                                                                                   |
| out | <i>plaintext</i>  | Output plain text.                                                                                             |
|     | <i>size</i>       | Size of input and output data in bytes. Zero means authentication only.                                        |
|     | <i>iv</i>         | Nonce                                                                                                          |
|     | <i>ivSize</i>     | Length of the Nonce in bytes. Must be 7, 8, 9, 10, 11, 12, or 13.                                              |
|     | <i>aad</i>        | Input additional authentication data. Can be NULL if aadSize is zero.                                          |
|     | <i>aadSize</i>    | Input size in bytes of AAD. Zero means data mode only (authentication skipped).                                |
|     | <i>key</i>        | Input key to use for decryption                                                                                |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.                                                        |
|     | <i>tag</i>        | Received tag. Set to NULL to skip tag processing.                                                              |
|     | <i>tagSize</i>    | Input size of the received tag to compare with the computed tag, in bytes. Must be 4, 6, 8, 10, 12, 14, or 16. |

## Returns

Status from decrypt operation

## 27.8.4 LTC HASH driver

### 27.8.4.1 Overview

This section describes the programming interface of the LTC HASH driver.

#### Data Structures

- struct [\\_ltc\\_hash\\_ctx\\_t](#)  
*Storage type used to save hash context. [More...](#)*

#### Macros

- #define [LTC\\_HASH\\_CTX\\_SIZE](#) 29  
*LTC HASH Context size.*

#### Typedefs

- typedef enum [\\_ltc\\_hash\\_algo\\_t](#) [ltc\\_hash\\_algo\\_t](#)  
*Supported cryptographic block cipher functions for HASH creation.*
- typedef struct [\\_ltc\\_hash\\_ctx\\_t](#) [ltc\\_hash\\_ctx\\_t](#)  
*Storage type used to save hash context.*

#### Enumerations

- enum [\\_ltc\\_hash\\_algo\\_t](#) {  
    [kLTC\\_XcbcMac](#) = 0,  
    [kLTC\\_Cmac](#) }  
*Supported cryptographic block cipher functions for HASH creation.*

#### Functions

- [status\\_t LTC\\_HASH\\_Init](#) (LTC\_Type \*base, [ltc\\_hash\\_ctx\\_t](#) \*ctx, [ltc\\_hash\\_algo\\_t](#) algo, const uint8\_t \*key, uint32\_t keySize)  
*Initialize HASH context.*
- [status\\_t LTC\\_HASH\\_Update](#) ([ltc\\_hash\\_ctx\\_t](#) \*ctx, const uint8\_t \*input, uint32\_t inputSize)  
*Add data to current HASH.*
- [status\\_t LTC\\_HASH\\_Finish](#) ([ltc\\_hash\\_ctx\\_t](#) \*ctx, uint8\_t \*output, uint32\_t \*outputSize)  
*Finalize hashing.*
- [status\\_t LTC\\_HASH](#) (LTC\_Type \*base, [ltc\\_hash\\_algo\\_t](#) algo, const uint8\_t \*input, uint32\_t inputSize, const uint8\_t \*key, uint32\_t keySize, uint8\_t \*output, uint32\_t \*outputSize)  
*Create HASH on given data.*

## 27.8.4.2 Data Structure Documentation

### 27.8.4.2.1 struct \_ltc\_hash\_ctx\_t

## 27.8.4.3 Macro Definition Documentation

### 27.8.4.3.1 #define LTC\_HASH\_CTX\_SIZE 29

## 27.8.4.4 Typedef Documentation

### 27.8.4.4.1 typedef struct \_ltc\_hash\_ctx\_t ltc\_hash\_ctx\_t

## 27.8.4.5 Enumeration Type Documentation

### 27.8.4.5.1 enum \_ltc\_hash\_algo\_t

Enumerator

*kLTC\_XcbcMac* XCBC-MAC (AES engine)

*kLTC\_Cmac* CMAC (AES engine)

## 27.8.4.6 Function Documentation

### 27.8.4.6.1 status\_t LTC\_HASH\_Init ( LTC\_Type \* *base*, ltc\_hash\_ctx\_t \* *ctx*, ltc\_hash\_algo\_t *algo*, const uint8\_t \* *key*, uint32\_t *keySize* )

This function initialize the HASH. Key shall be supplied if the underlying algorithm is AES XCBC-MAC or CMAC. Key shall be NULL if the underlying algorithm is SHA.

For XCBC-MAC, the key length must be 16. For CMAC, the key length can be the AES key lengths supported by AES engine. For MDHA the key length argument is ignored.

Parameters

|     |                |                                                   |
|-----|----------------|---------------------------------------------------|
|     | <i>base</i>    | LTC peripheral base address                       |
| out | <i>ctx</i>     | Output hash context                               |
|     | <i>algo</i>    | Underlying algorithm to use for hash computation. |
|     | <i>key</i>     | Input key (NULL if underlying algorithm is SHA)   |
|     | <i>keySize</i> | Size of input key in bytes                        |

Returns

Status of initialization

**27.8.4.6.2** `status_t LTC_HASH_Update ( ltc_hash_ctx_t * ctx, const uint8_t * input, uint32_t inputSize )`

Add data to current HASH. This can be called repeatedly with an arbitrary amount of data to be hashed.

## Parameters

|                |                  |                             |
|----------------|------------------|-----------------------------|
| <i>in, out</i> | <i>ctx</i>       | HASH context                |
|                | <i>input</i>     | Input data                  |
|                | <i>inputSize</i> | Size of input data in bytes |

## Returns

Status of the hash update operation

**27.8.4.6.3** `status_t LTC_HASH_Finish ( ltc_hash_ctx_t * ctx, uint8_t * output, uint32_t * outputSize )`

Outputs the final hash and erases the context.

## Parameters

|                |                   |                                                               |
|----------------|-------------------|---------------------------------------------------------------|
| <i>in, out</i> | <i>ctx</i>        | Input hash context                                            |
| <i>out</i>     | <i>output</i>     | Output hash data                                              |
| <i>out</i>     | <i>outputSize</i> | Output parameter storing the size of the output hash in bytes |

## Returns

Status of the hash finish operation

**27.8.4.6.4** `status_t LTC_HASH ( LTC_Type * base, ltc_hash_algo_t algo, const uint8_t * input, uint32_t inputSize, const uint8_t * key, uint32_t keySize, uint8_t * output, uint32_t * outputSize )`

Perform the full keyed HASH in one function call.

## Parameters

|  |             |                                                 |
|--|-------------|-------------------------------------------------|
|  | <i>base</i> | LTC peripheral base address                     |
|  | <i>algo</i> | Block cipher algorithm to use for CMAC creation |

|     |                   |                                                               |
|-----|-------------------|---------------------------------------------------------------|
|     | <i>input</i>      | Input data                                                    |
|     | <i>inputSize</i>  | Size of input data in bytes                                   |
|     | <i>key</i>        | Input key                                                     |
|     | <i>keySize</i>    | Size of input key in bytes                                    |
| out | <i>output</i>     | Output hash data                                              |
| out | <i>outputSize</i> | Output parameter storing the size of the output hash in bytes |

#### Returns

Status of the one call hash operation.

## 27.8.5 LTC PKHA driver

### 27.8.5.1 Overview

This section describes the programming interface of the LTC PKHA driver.

#### Data Structures

- struct `_ltc_pkha_ecc_point_t`  
*PKHA ECC point structure. [More...](#)*

#### Typedefs

- typedef struct  
`_ltc_pkha_ecc_point_t ltc_pkha_ecc_point_t`  
*PKHA ECC point structure.*
- typedef enum `_ltc_pkha_timing_t ltc_pkha_timing_t`  
*Use of timing equalized version of a PKHA function.*
- typedef enum `_ltc_pkha_f2m_t ltc_pkha_f2m_t`  
*Integer vs binary polynomial arithmetic selection.*
- typedef enum  
`_ltc_pkha_montgomery_form_t ltc_pkha_montgomery_form_t`  
*Montgomery or normal PKHA input format.*

#### Enumerations

- enum `_ltc_pkha_timing_t` {  
`kLTC_PKHA_NoTimingEqualized` = 0U,  
`kLTC_PKHA_TimingEqualized` = 1U }  
*Use of timing equalized version of a PKHA function.*
- enum `_ltc_pkha_f2m_t` {  
`kLTC_PKHA_IntegerArith` = 0U,  
`kLTC_PKHA_F2mArith` = 1U }  
*Integer vs binary polynomial arithmetic selection.*
- enum `_ltc_pkha_montgomery_form_t` {  
`kLTC_PKHA_NormalValue` = 0U,  
`kLTC_PKHA_MontgomeryFormat` = 1U }  
*Montgomery or normal PKHA input format.*

### 27.8.5.2 Data Structure Documentation

#### 27.8.5.2.1 struct `_ltc_pkha_ecc_point_t`

##### Data Fields

- `uint8_t * X`



- $X \text{ coordinate (affine)}$   
 $\text{uint8\_t} * Y$   
 $Y \text{ coordinate (affine)}$

### 27.8.5.3 Typedef Documentation

27.8.5.3.1 typedef enum \_ltc\_pkha\_timing\_t ltc\_pkha\_timing\_t

27.8.5.3.2 typedef enum \_ltc\_pkha\_f2m\_t ltc\_pkha\_f2m\_t

27.8.5.3.3 typedef enum \_ltc\_pkha\_montgomery\_form\_t ltc\_pkha\_montgomery\_form\_t

### 27.8.5.4 Enumeration Type Documentation

27.8.5.4.1 enum \_ltc\_pkha\_timing\_t

Enumerator

- kLTC\_PKHA\_NoTimingEqualized* Normal version of a PKHA operation.  
*kLTC\_PKHA\_TimingEqualized* Timing-equalized version of a PKHA operation.

27.8.5.4.2 enum \_ltc\_pkha\_f2m\_t

Enumerator

- kLTC\_PKHA\_IntegerArith* Use integer arithmetic.  
*kLTC\_PKHA\_F2mArith* Use binary polynomial arithmetic.

27.8.5.4.3 enum \_ltc\_pkha\_montgomery\_form\_t

Enumerator

- kLTC\_PKHA\_NormalValue* PKHA number is normal integer.  
*kLTC\_PKHA\_MontgomeryFormat* PKHA number is in montgomery format.

## Chapter 28

# MSMC: Multicore System Mode Controller

### 28.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Multicore System Mode Controller (MSMC) module of MCUXpresso SDK devices.

### 28.2 Typical use case

#### 28.2.1 Set Core 0 from RUN to VLPR mode

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/msmc`

#### 28.2.2 Set Core 0 from VLPR/HSRUN to RUN mode

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/msmc`

### 28.3 Typical use case

#### 28.3.1 Set Core 0 from RUN to HSRUN mode

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/msmc`

#### 28.3.2 Enter wait or stop modes

SMC driver provides APIs to set MCU to different wait modes and stop modes. At the same time, there are pre-function and post-function for the modes setting. The pre-function and post-function are used for:

1. Disable/enable the interrupt through PRIMASK. In practise, there is such scenario: the application sets the wakeup interrupt and calls SMC function [SMC\\_SetPowerModeStop](#) to set MCU to STOP mode, but the wakeup interrupt happens so quickly that the ISR completed before the function [SMC\\_SetPowerModeStop](#), as a result, the MCU enters STOP mode and never be wakeup by the interrupt. In this case, application could first disable interrupt through PRIMASK, then set the wakeup interrupt and enter STOP mode. After wakeup, the first thing is enable the interrupt through PRIMASK. The MCU could still be wakeup when disable interrupt through PRIMASK. The pre- and post- functions handle the PRIMASK inside.

```
SMC_PreEnterStopModes();

/* Enable the wakeup interrupt here. */

SMC_SetPowerModeStop(SMC0, kSMC_PartialStop);

SMC_PostExitStopModes();
```

For other use cases, please refer to the comments of the MSMC driver header file. Some example codes are also provided.

## Files

- file [fsl\\_msmc.h](#)

## Data Structures

- struct [\\_smc\\_reset\\_pin\\_filter\\_config](#)  
*Reset pin filter configuration. [More...](#)*

## Typedefs

- typedef enum  
[\\_smc\\_power\\_mode\\_protection](#) [smc\\_power\\_mode\\_protection\\_t](#)  
*Power Modes Protection.*
- typedef enum [\\_smc\\_power\\_state](#) [smc\\_power\\_state\\_t](#)  
*Power Modes in PMSTAT.*
- typedef enum  
[\\_smc\\_power\\_stop\\_entry\\_status](#) [smc\\_power\\_stop\\_entry\\_status\\_t](#)  
*Power Stop Entry Status in PMSTAT.*
- typedef enum [\\_smc\\_run\\_mode](#) [smc\\_run\\_mode\\_t](#)  
*Run mode definition.*
- typedef enum [\\_smc\\_stop\\_mode](#) [smc\\_stop\\_mode\\_t](#)  
*Stop mode definition.*
- typedef enum [\\_smc\\_partial\\_stop\\_mode](#) [smc\\_partial\\_stop\\_option\\_t](#)  
*Partial STOP option.*
- typedef enum [\\_smc\\_reset\\_source](#) [smc\\_reset\\_source\\_t](#)  
*System Reset Source Name definitions.*
- typedef enum [\\_smc\\_interrupt\\_enable](#) [smc\\_interrupt\\_enable\\_t](#)  
*System reset interrupt enable bit definitions.*
- typedef struct  
[\\_smc\\_reset\\_pin\\_filter\\_config](#) [smc\\_reset\\_pin\\_filter\\_config\\_t](#)  
*Reset pin filter configuration.*

## Enumerations

- enum [\\_smc\\_power\\_mode\\_protection](#) {  
    [kSMC\\_AllowPowerModeVlls](#) = SMC\_PMPROT\_AVLLS\_MASK,  
    [kSMC\\_AllowPowerModeLls](#) = SMC\_PMPROT\_ALLS\_MASK,  
    [kSMC\\_AllowPowerModeVlp](#) = SMC\_PMPROT\_AVLP\_MASK,  
    [kSMC\\_AllowPowerModeHsrn](#) = SMC\_PMPROT\_AHSRUN\_MASK,  
    [kSMC\\_AllowPowerModeAll](#) }  
}

*Power Modes Protection.*

- enum `_smc_power_state` {  
`kSMC_PowerStateRun` = 1U,  
`kSMC_PowerStateStop` = 1U << 1U,  
`kSMC_PowerStateVlpr` = 1U << 2U,  
`kSMC_PowerStateHsruntime` = 1U << 7U }

*Power Modes in PMSTAT.*

- enum `_smc_power_stop_entry_status` {  
`kSMC_PowerStopEntryAlt0` = 1U,  
`kSMC_PowerStopEntryAlt1` = 1U << 1,  
`kSMC_PowerStopEntryAlt2` = 1U << 2,  
`kSMC_PowerStopEntryAlt3` = 1U << 3,  
`kSMC_PowerStopEntryAlt4` = 1U << 4,  
`kSMC_PowerStopEntryAlt5` = 1U << 5 }

*Power Stop Entry Status in PMSTAT.*

- enum `_smc_run_mode` {  
`kSMC_RunNormal` = 0U,  
`kSMC_RunVlpr` = 2U,  
`kSMC_Hsruntime` = 3U }

*Run mode definition.*

- enum `_smc_stop_mode` {  
`kSMC_StopNormal` = 0U,  
`kSMC_StopVlps` = 2U,  
`kSMC_StopLls` = 3U,  
`kSMC_StopVlls` = 4U }

*Stop mode definition.*

- enum `_smc_partial_stop_mode` {  
`kSMC_PartialStop` = 0U,  
`kSMC_PartialStop1` = 1U,  
`kSMC_PartialStop2` = 2U,  
`kSMC_PartialStop3` = 3U }

*Partial STOP option.*

- enum { `kStatus_SMC_StopAbort` = MAKE\_STATUS(kStatusGroup\_POWER, 0) }

*SMC configuration status.*

- enum `_smc_reset_source` {

```

kSMC_SourceWakeup = SMC_SRS_WAKEUP_MASK,
kSMC_SourcePor = SMC_SRS_POR_MASK,
kSMC_SourceLvd = SMC_SRS_LVD_MASK,
kSMC_SourceHvd = SMC_SRS_HVD_MASK,
kSMC_SourceWarm = SMC_SRS_WARM_MASK,
kSMC_SourceFatal = SMC_SRS_FATAL_MASK,
kSMC_SourceCore,
kSMC_SourcePin = SMC_SRS_PIN_MASK,
kSMC_SourceMdm = SMC_SRS_MDM_MASK,
kSMC_SourceRstAck = SMC_SRS_RSTACK_MASK,
kSMC_SourceStopAck = SMC_SRS_STOPACK_MASK,
kSMC_SourceScg = SMC_SRS_SCG_MASK,
kSMC_SourceWdog = SMC_SRS_WDOG_MASK,
kSMC_SourceSoftware = SMC_SRS_SW_MASK,
kSMC_SourceLockup = SMC_SRS_LOCKUP_MASK,
kSMC_SourceJtag = SMC_SRS_JTAG_MASK }

```

*System Reset Source Name definitions.*

- enum `_smc_interrupt_enable` {  
`kSMC_IntNone` = 0U,  
`kSMC_IntPin` = SMC\_SRIE\_PIN\_MASK,  
`kSMC_IntMdm` = SMC\_SRIE\_MDM\_MASK,  
`kSMC_IntStopAck` = SMC\_SRIE\_STOPACK\_MASK,  
`kSMC_IntWdog` = SMC\_SRIE\_WDOG\_MASK,  
`kSMC_IntSoftware` = SMC\_SRIE\_SW\_MASK,  
`kSMC_IntLockup` = SMC\_SRIE\_LOCKUP\_MASK,  
`kSMC_IntAll` }

*System reset interrupt enable bit definitions.*

## Driver version

- #define `FSL_MSMC_DRIVER_VERSION` (`MAKE_VERSION`(2, 1, 2))  
*MSMC driver version.*

## System mode controller APIs

- static void `SMC_SetPowerModeProtection` (SMC\_Type \*base, uint8\_t allowedModes)  
*Configures all power mode protection settings.*
- static `smc_power_state_t` `SMC_GetPowerModeState` (SMC\_Type \*base)  
*Gets the current power mode status.*
- static void `SMC_PreEnterStopModes` (void)  
*Prepare to enter stop modes.*
- static void `SMC_PostExitStopModes` (void)  
*Recovering after wake up from stop modes.*
- static void `SMC_PreEnterWaitModes` (void)  
*Prepare to enter wait modes.*
- static void `SMC_PostExitWaitModes` (void)  
*Recovering after wake up from stop modes.*
- `status_t` `SMC_SetPowerModeRun` (SMC\_Type \*base)

- *Configure the system to RUN power mode.*
- [status\\_t SMC\\_SetPowerModeHsrun](#) (SMC\_Type \*base)
- *Configure the system to HSRUN power mode.*
- [status\\_t SMC\\_SetPowerModeWait](#) (SMC\_Type \*base)
- *Configure the system to WAIT power mode.*
- [status\\_t SMC\\_SetPowerModeStop](#) (SMC\_Type \*base, [smc\\_partial\\_stop\\_option\\_t](#) option)
- *Configure the system to Stop power mode.*
- [status\\_t SMC\\_SetPowerModeVlpr](#) (SMC\_Type \*base)
- *Configure the system to VLPR power mode.*
- [status\\_t SMC\\_SetPowerModeVlpw](#) (SMC\_Type \*base)
- *Configure the system to VLPW power mode.*
- [status\\_t SMC\\_SetPowerModeVlps](#) (SMC\_Type \*base)
- *Configure the system to VLPS power mode.*
- [status\\_t SMC\\_SetPowerModeLls](#) (SMC\_Type \*base)
- *Configure the system to LLS power mode.*
- [status\\_t SMC\\_SetPowerModeVlls](#) (SMC\_Type \*base)
- *Configure the system to VLLS power mode.*
- [static uint32\\_t SMC\\_GetPreviousResetSources](#) (SMC\_Type \*base)
- *Gets the reset source status which caused a previous reset.*
- [static uint32\\_t SMC\\_GetStickyResetSources](#) (SMC\_Type \*base)
- *Gets the sticky reset source status.*
- [static void SMC\\_ClearStickyResetSources](#) (SMC\_Type \*base, uint32\_t sourceMasks)
- *Clears the sticky reset source status.*
- [void SMC\\_ConfigureResetPinFilter](#) (SMC\_Type \*base, const [smc\\_reset\\_pin\\_filter\\_config\\_t](#) \*config)
- *Configures the reset pin filter.*
- [static void SMC\\_SetSystemResetInterruptConfig](#) (SMC\_Type \*base, uint32\_t intMask)
- *Sets the system reset interrupt configuration.*
- [static uint32\\_t SMC\\_GetResetInterruptSourcesStatus](#) (SMC\_Type \*base)
- *Gets the source status of the system reset interrupt.*
- [static void SMC\\_ClearResetInterruptSourcesStatus](#) (SMC\_Type \*base, uint32\_t intMask)
- *Clears the source status of the system reset interrupt.*
- [static uint32\\_t SMC\\_GetBootOptionConfig](#) (SMC\_Type \*base)
- *Gets the boot option configuration.*

## 28.4 Data Structure Documentation

### 28.4.1 struct \_smc\_reset\_pin\_filter\_config

#### Data Fields

- [uint8\\_t slowClockFilterCount](#)
- *Reset pin bus clock filter width from 1 to 32 slow clock cycles.*
- [bool enableFilter](#)
- *Reset pin filter enable/disable.*

## Field Documentation

(1) `uint8_t _smc_reset_pin_filter_config::slowClockFilterCount`

(2) `bool _smc_reset_pin_filter_config::enableFilter`

## 28.5 Macro Definition Documentation

28.5.1 `#define FSL_MSMC_DRIVER_VERSION (MAKE_VERSION(2, 1, 2))`

## 28.6 Enumeration Type Documentation

### 28.6.1 `enum _smc_power_mode_protection`

#### Enumerator

*kSMC\_AllowPowerModeVlls* Allow Very-Low-Leakage Stop Mode.

*kSMC\_AllowPowerModeLls* Allow Low-Leakage Stop Mode.

*kSMC\_AllowPowerModeVlp* Allow Very-Low-Power Mode.

*kSMC\_AllowPowerModeHsruntime* Allow High Speed Run mode.

*kSMC\_AllowPowerModeAll* Allow all power mode.

### 28.6.2 `enum _smc_power_state`

#### Enumerator

*kSMC\_PowerStateRun* 0000\_0001 - Current power mode is RUN

*kSMC\_PowerStateStop* 0000\_0010 - Current power mode is any STOP mode

*kSMC\_PowerStateVlpr* 0000\_0100 - Current power mode is VLPR

*kSMC\_PowerStateHsruntime* 1000\_0000 - Current power mode is HSRUN

### 28.6.3 `enum _smc_power_stop_entry_status`

#### Enumerator

*kSMC\_PowerStopEntryAlt0* Indicates a Stop mode entry since this field was last cleared.

*kSMC\_PowerStopEntryAlt1* Indicates the system bus masters acknowledged the Stop mode entry.

*kSMC\_PowerStopEntryAlt2* Indicates the system clock peripherals acknowledged the Stop mode entry.

*kSMC\_PowerStopEntryAlt3* Indicates the bus clock peripherals acknowledged the Stop mode entry.

*kSMC\_PowerStopEntryAlt4* Indicates the slow clock peripherals acknowledged the Stop mode entry.

*kSMC\_PowerStopEntryAlt5* Indicates Stop mode entry completed.

### 28.6.4 enum \_smc\_run\_mode

Enumerator

*kSMC\_RunNormal* normal RUN mode.  
*kSMC\_RunVlpr* Very-Low-Power RUN mode.  
*kSMC\_Hsrun* High Speed Run mode (HSRUN).

### 28.6.5 enum \_smc\_stop\_mode

Enumerator

*kSMC\_StopNormal* Normal STOP mode.  
*kSMC\_StopVlps* Very-Low-Power STOP mode.  
*kSMC\_StopLls* Low-Leakage Stop mode.  
*kSMC\_StopVlls* Very-Low-Leakage Stop mode.

### 28.6.6 enum \_smc\_partial\_stop\_mode

Enumerator

*kSMC\_PartialStop* STOP - Normal Stop mode.  
*kSMC\_PartialStop1* Partial Stop with both system and bus clocks disabled.  
*kSMC\_PartialStop2* Partial Stop with system clock disabled and bus clock enabled.  
*kSMC\_PartialStop3* Partial Stop with system clock enabled and bus clock disabled.

### 28.6.7 anonymous enum

Enumerator

*kStatus\_SMC\_StopAbort* Entering Stop mode is abort.

### 28.6.8 enum \_smc\_reset\_source

Enumerator

*kSMC\_SourceWakeup* Very low-leakage wakeup reset.  
*kSMC\_SourcePor* Power on reset.  
*kSMC\_SourceLvd* Low-voltage detect reset.  
*kSMC\_SourceHvd* High-voltage detect reset.  
*kSMC\_SourceWarm* Warm reset. Warm Reset flag will assert if any of the system reset sources in this register assert (SRS[31:8])



*kSMC\_SourceFatal* Fatal reset.  
*kSMC\_SourceCore* Software reset that only reset the core, NOT a sticky system reset source.  
*kSMC\_SourcePin* RESET\_B pin reset.  
*kSMC\_SourceMdm* MDM reset.  
*kSMC\_SourceRstAck* Reset Controller timeout reset.  
*kSMC\_SourceStopAck* Stop timeout reset.  
*kSMC\_SourceScg* SCG loss of lock or loss of clock.  
*kSMC\_SourceWdog* Watchdog reset.  
*kSMC\_SourceSoftware* Software reset.  
*kSMC\_SourceLockup* Lockup reset. Core lockup or exception.  
*kSMC\_SourceJtag* JTAG system reset.

## 28.6.9 enum \_smc\_interrupt\_enable

Enumerator

*kSMC\_IntNone* No interrupt enabled.  
*kSMC\_IntPin* Pin reset interrupt.  
*kSMC\_IntMdm* MDM reset interrupt.  
*kSMC\_IntStopAck* Stop timeout reset interrupt.  
*kSMC\_IntWdog* Watchdog interrupt.  
*kSMC\_IntSoftware* Software reset interrupts.  
*kSMC\_IntLockup* Lock up interrupt.  
*kSMC\_IntAll* All system reset interrupts.

## 28.7 Function Documentation

### 28.7.1 static void SMC\_SetPowerModeProtection ( SMC\_Type \* *base*, uint8\_t *allowedModes* ) [inline], [static]

This function configures the power mode protection settings for supported power modes in the specified chip family. The available power modes are defined in the `smc_power_mode_protection_t`. This should be done at an early system level initialization stage. See the reference manual for details. This register can only write once after the power reset.

The allowed modes are passed as bit map, for example, to allow LLS and VLLS, use `SMC_SetPowerModeProtection(kSMC_AllowPowerModeLLs | kSMC_AllowPowerModeVlls)`. To allow all modes, use `SMC_SetPowerModeProtection(kSMC_AllowPowerModeAll)`.

## Parameters

|                     |                                    |
|---------------------|------------------------------------|
| <i>base</i>         | SMC peripheral base address.       |
| <i>allowedModes</i> | Bitmap of the allowed power modes. |

### 28.7.2 static smc\_power\_state\_t SMC\_GetPowerModeState ( SMC\_Type \* *base* ) [inline], [static]

This function returns the current power mode stat. Once application switches the power mode, it should always check the stat to check whether it runs into the specified mode or not. An application should check this mode before switching to a different mode. The system requires that only certain modes can switch to other specific modes. See the reference manual for details and the `smc_power_state_t` for information about the power stat.

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

Current power mode status.

### 28.7.3 static void SMC\_PreEnterStopModes ( void ) [inline], [static]

This function should be called before entering STOP/VLPS/LLS/VLLS modes.

### 28.7.4 static void SMC\_PostExitStopModes ( void ) [inline], [static]

This function should be called after wake up from STOP/VLPS/LLS/VLLS modes. It is used together with [SMC\\_PreEnterStopModes](#).

### 28.7.5 static void SMC\_PreEnterWaitModes ( void ) [inline], [static]

This function should be called before entering WAIT/VLPW modes..

### 28.7.6 static void SMC\_PostExitWaitModes ( void ) [inline], [static]

This function should be called after wake up from WAIT/VLPW modes. It is used together with [SMC\\_PreEnterWaitModes](#).

### 28.7.7 status\_t SMC\_SetPowerModeRun ( SMC\_Type \* *base* )

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

SMC configuration error code.

### 28.7.8 **status\_t SMC\_SetPowerModeHsrn ( SMC\_Type \* *base* )**

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

SMC configuration error code.

### 28.7.9 **status\_t SMC\_SetPowerModeWait ( SMC\_Type \* *base* )**

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

SMC configuration error code.

### 28.7.10 **status\_t SMC\_SetPowerModeStop ( SMC\_Type \* *base*, smc\_partial\_stop\_option\_t *option* )**

## Parameters

|               |                              |
|---------------|------------------------------|
| <i>base</i>   | SMC peripheral base address. |
| <i>option</i> | Partial Stop mode option.    |

Returns

SMC configuration error code.

### 28.7.11 **status\_t SMC\_SetPowerModeVlpr ( SMC\_Type \* *base* )**

Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

Returns

SMC configuration error code.

### 28.7.12 **status\_t SMC\_SetPowerModeVlpw ( SMC\_Type \* *base* )**

Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

Returns

SMC configuration error code.

### 28.7.13 **status\_t SMC\_SetPowerModeVlps ( SMC\_Type \* *base* )**

Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

Returns

SMC configuration error code.

### 28.7.14 **status\_t SMC\_SetPowerModeLls ( SMC\_Type \* *base* )**

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

SMC configuration error code.

### 28.7.15 `status_t SMC_SetPowerModeVlls ( SMC_Type * base )`

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

SMC configuration error code.

### 28.7.16 `static uint32_t SMC_GetPreviousResetSources ( SMC_Type * base )` `[inline], [static]`

This function gets the current reset source status. Use source masks defined in the `smc_reset_source_t` to get the desired source status.

Example: To get all reset source statuses.

```
resetStatus = SMC_GetPreviousResetSources(SMC0) & kSMC_SourceAll;
```

Example: To test whether the MCU is reset using Watchdog.

```
uint32_t resetStatus;

resetStatus = SMC_GetPreviousResetSources(SMC0) &
 kSMC_SourceWdog;
```

Example: To test multiple reset sources.

```
uint32_t resetStatus;

resetStatus = SMC_GetPreviousResetSources(SMC0) & (
 kSMC_SourceWdog | kSMC_SourcePin);
```

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

All reset source status bit map.

### 28.7.17 static uint32\_t SMC\_GetStickyResetSources ( SMC\_Type \* *base* ) [inline], [static]

This function gets the current reset source status that has not been cleared by software for some specific source.

Example: To get all reset source statuses.

```
uint32_t resetStatus;

resetStatus = SMC_GetStickyResetSources(SMC0) & kSMC_SourceAll;
```

Example, To test whether the MCU is reset using Watchdog.

```
uint32_t resetStatus;

resetStatus = SMC_GetStickyResetSources(SMC0) &
 kSMC_SourceWdog;
```

Example To test multiple reset sources.

```
uint32_t resetStatus;

resetStatus = SMC_GetStickyResetSources(SMC0) & (
 kSMC_SourceWdog | kSMC_SourcePin);
```

## Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

## Returns

All reset source status bit map.

### 28.7.18 static void SMC\_ClearStickyResetSources ( SMC\_Type \* *base*, uint32\_t *sourceMasks* ) [inline], [static]

This function clears the sticky system reset flags indicated by source masks.

Example: Clears multiple reset sources.

```
SMC_ClearStickyResetSources(SMC0, (kSMC_SourceWdog |
 kSMC_SourcePin));
```

Parameters

|                    |                              |
|--------------------|------------------------------|
| <i>base</i>        | SMC peripheral base address. |
| <i>sourceMasks</i> | reset source status bit map  |

### 28.7.19 void SMC\_ConfigureResetPinFilter ( SMC\_Type \* *base*, const smc\_reset\_pin\_filter\_config\_t \* *config* )

This function sets the reset pin filter including the enablement/disablement and filter width.

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | SMC peripheral base address.            |
| <i>config</i> | Pointer to the configuration structure. |

### 28.7.20 static void SMC\_SetSystemResetInterruptConfig ( SMC\_Type \* *base*, uint32\_t *intMask* ) [inline], [static]

For a graceful shut down, the MSMC supports delaying the assertion of the system reset for a period of time when the reset interrupt is generated. This function can be used to enable the interrupt. The interrupts are passed in as bit mask. See `smc_interrupt_enable_t` for details. For example, to delay a reset after the WDOG timeout or PIN reset occurs, configure as follows: `SMC_SetSystemResetInterruptConfig(SMC0, (kSMC_IntWdog | kSMC_IntPin));`

Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|



|                |                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------|
| <i>intMask</i> | Bit mask of the system reset interrupts to enable. See <code>smc_interrupt_enable_t</code> for details. |
|----------------|---------------------------------------------------------------------------------------------------------|

### 28.7.21 `static uint32_t SMC_GetResetInterruptSourcesStatus ( SMC_Type * base ) [inline], [static]`

This function gets the source status of the reset interrupt. Use source masks defined in the `smc_interrupt_enable_t` to get the desired source status.

Example: To get all reset interrupt source statuses.

```
uint32_t interruptStatus;

interruptStatus = SMC_GetResetInterruptSourcesStatus(SMC0) &
 kSMC_IntAll;
```

Example: To test whether the reset interrupt of Watchdog is pending.

```
uint32_t interruptStatus;

interruptStatus = SMC_GetResetInterruptSourcesStatus(SMC0) &
 kSMC_IntWdog;
```

Example: To test multiple reset interrupt sources.

```
uint32_t interruptStatus;

interruptStatus = SMC_GetResetInterruptSourcesStatus(SMC0) & (
 kSMC_IntWdog | kSMC_IntPin);
```

#### Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

#### Returns

All reset interrupt source status bit map.

### 28.7.22 `static void SMC_ClearResetInterruptSourcesStatus ( SMC_Type * base, uint32_t intMask ) [inline], [static]`

This function clears the source status of the reset interrupt. Use source masks defined in the `smc_interrupt_enable_t` to get the desired source status.

Example: To clear all reset interrupt source statuses.

```
uint32_t interruptStatus;

MMC_ClearResetInterruptSourcesStatus(SMC0, kSMC_IntAll);
```

Example, To clear the reset interrupt of Watchdog.

```
uint32_t interruptStatus;

SMC_ClearResetInterruptSourcesStatus(SMC0,
 kSMC_IntWdog);
```

Example, To clear multiple reset interrupt sources status.

```
uint32_t interruptStatus;

SMC_ClearResetInterruptSourcesStatus(SMC0, (
 kSMC_IntWdog | kSMC_IntPin));
```

Parameters

|                |                                                     |
|----------------|-----------------------------------------------------|
| <i>base</i>    | SMC peripheral base address.                        |
| <i>intMask</i> | All reset interrupt source status bit map to clear. |

### 28.7.23 static uint32\_t SMC\_GetBootOptionConfig ( SMC\_Type \* *base* ) [inline], [static]

This function gets the boot option configuration of MSMC.

Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | SMC peripheral base address. |
|-------------|------------------------------|

Returns

The boot option configuration. 1 means boot option enabled. 0 means not.

## Chapter 29

# MU: Messaging Unit

### 29.1 Overview

The MCUXpresso SDK provides a driver for the MU module of MCUXpresso SDK devices.

### 29.2 Function description

The MU driver provides these functions:

- Functions to initialize the MU module.
- Functions to send and receive messages.
- Functions for MU flags for both MU sides.
- Functions for status flags and interrupts.
- Other miscellaneous functions.

#### 29.2.1 MU initialization

The function [MU\\_Init\(\)](#) initializes the MU module and enables the MU clock. It should be called before any other MU functions.

The function [MU\\_Deinit\(\)](#) deinitializes the MU module and disables the MU clock. No MU functions can be called after this function.

#### 29.2.2 MU message

The MU message must be sent when the transmit register is empty. The MU driver provides blocking API and non-blocking API to send message.

The [MU\\_SendMsgNonBlocking\(\)](#) function writes a message to the MU transmit register without checking the transmit register status. The upper layer should check that the transmit register is empty before calling this function. This function can be used in the ISR for better performance.

The [MU\\_SendMsg\(\)](#) function is a blocking function. It waits until the transmit register is empty and sends the message.

Correspondingly, there are blocking and non-blocking APIs for receiving a message. The [MU\\_ReadMsgNonBlocking\(\)](#) function is a non-blocking API. The [MU\\_ReadMsg\(\)](#) function is the blocking API.

### 29.2.3 MU flags

The MU driver provides 3-bit general purpose flags. When the flags are set on one side, they are reflected on the other side.

The MU flags must be set when the previous flags have been updated to the other side. The MU driver provides a non-blocking function and a blocking function. The blocking function [MU\\_SetFlags\(\)](#) waits until previous flags have been updated to the other side and then sets flags. The non-blocking function sets the flags directly. Ensure that the `kMU_FlagsUpdatingFlag` is not pending before calling this function.

The function [MU\\_GetFlags\(\)](#) gets the MU flags on the current side.

### 29.2.4 Status and interrupt

The function [MU\\_GetStatusFlags\(\)](#) returns all MU status flags. Use the `_mu_status_flags` to check for specific flags, for example, to check RX0 and RX1 register full, use the following code:

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/mu`. The receive full flags are cleared automatically after messages are read out. The transmit empty flags are cleared automatically after new messages are written to the transmit register. The general purpose interrupt flags must be cleared manually using the function [MU\\_ClearStatusFlags\(\)](#).

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/mu`. To enable or disable a specific interrupt, use [MU\\_EnableInterrupts\(\)](#) and [MU\\_DisableInterrupts\(\)](#) functions. The interrupts to enable or disable should be passed in as a bit mask of the `_mu_interrupt_enable`.

The [MU\\_TriggerInterrupts\(\)](#) function triggers general purpose interrupts and NMI to the other core. The interrupts to trigger are passed in as a bit mask of the `_mu_interrupt_trigger`. If previously triggered interrupts have not been processed by the other side, this function returns an error.

### 29.2.5 MU misc functions

The [MU\\_ResetBothSides\(\)](#) function resets MU at both A and B sides. However, only the A side can call this function.

If a core enters stop mode, the platform clock of this core is disabled by default. The function [MU\\_SetClockOnOtherCoreEnable\(\)](#) forces the other core's platform clock to remain enabled even after that core has entered a stop mode. In this case, the other core's platform clock keeps running until the current core enters stop mode too.

### Typedefs

- typedef enum [\\_mu\\_msg\\_reg\\_index](#) [mu\\_msg\\_reg\\_index\\_t](#)  
*MU message register index.*
- typedef enum [\\_mu\\_general\\_purpose\\_interrupt](#) [mu\\_general\\_purpose\\_interrupt\\_t](#)

*MU general purpose interrupts.*

## Enumerations

- enum `_mu_status_flags` {
  - `kMU_Tx0EmptyFlag` = `MU_TX_FLAG(1UL << 0U)`,
  - `kMU_Tx1EmptyFlag` = `MU_TX_FLAG(1UL << 1U)`,
  - `kMU_Tx2EmptyFlag` = `MU_TX_FLAG(1UL << 2U)`,
  - `kMU_Tx3EmptyFlag` = `MU_TX_FLAG(1UL << 3U)`,
  - `kMU_Rx0FullFlag` = `MU_RX_FLAG(1UL << 0U)`,
  - `kMU_Rx1FullFlag` = `MU_RX_FLAG(1UL << 1U)`,
  - `kMU_Rx2FullFlag` = `MU_RX_FLAG(1UL << 2U)`,
  - `kMU_Rx3FullFlag` = `MU_RX_FLAG(1UL << 3U)`,
  - `kMU_GenInt0Flag` = `MU_GI_FLAG(1UL << 0U)`,
  - `kMU_GenInt1Flag` = `MU_GI_FLAG(1UL << 1U)`,
  - `kMU_GenInt2Flag` = `MU_GI_FLAG(1UL << 2U)`,
  - `kMU_GenInt3Flag` = `MU_GI_FLAG(1UL << 3U)`,
  - `kMU_CoreEventPendingFlag` = `MU_STAT_FLAG(MU_SR_CEP_MASK)`,
  - `kMU_RxFullPendingFlag` = `MU_STAT_FLAG(MU_SR_RFP_MASK)`,
  - `kMU_TxEmptyPendingFlag` = `MU_STAT_FLAG(MU_SR_TEP_MASK)`,
  - `kMU_GenIntPendingFlag` = `MU_STAT_FLAG(MU_SR_GIRP_MASK)`,
  - `kMU_EventPendingFlag` = `MU_STAT_FLAG(MU_SR_EP_MASK)`,
  - `kMU_FlagsUpdatingFlag` = `MU_STAT_FLAG(MU_SR_FUP_MASK)`,
  - `kMU_MuInResetFlag` = `MU_STAT_FLAG(MU_SR_MURS_MASK)`,
  - `kMU_MuResetInterruptFlag` = `MU_STAT_FLAG(MU_SR_MURIP_MASK)`,
  - `kMU_OtherSideEnterRunInterruptFlag` = `MU_CORE_FLAG(MU_CSSR0_RUN_MASK)`,
  - `kMU_OtherSideEnterHaltInterruptFlag` = `MU_CORE_FLAG(MU_CSSR0_HALT_MASK)`,
  - `kMU_OtherSideEnterWaitInterruptFlag` = `MU_CORE_FLAG(MU_CSSR0_WAIT_MASK)`,
  - `kMU_OtherSideEnterStopInterruptFlag` = `MU_CORE_FLAG(MU_CSSR0_STOP_MASK)`,
  - `kMU_OtherSideEnterPowerDownInterruptFlag` = `MU_CORE_FLAG(MU_CSSR0_PD_MASK)`,
  - `kMU_ResetAssertInterruptFlag` = `MU_CORE_FLAG(MU_CSSR0_RAIP_MASK)`,
  - `kMU_HardwareResetInterruptFlag` = `MU_CORE_FLAG(MU_CSSR0_HRIP_MASK)` }

*MU status flags.*
- enum `_mu_interrupt_enable` {

```

kMU_Tx0EmptyInterruptEnable = MU_TX_INTR(1UL << 0U),
kMU_Tx1EmptyInterruptEnable = MU_TX_INTR(1UL << 1U),
kMU_Tx2EmptyInterruptEnable = MU_TX_INTR(1UL << 2U),
kMU_Tx3EmptyInterruptEnable = MU_TX_INTR(1UL << 3U),
kMU_Rx0FullInterruptEnable = MU_RX_INTR(1UL << 0U),
kMU_Rx1FullInterruptEnable = MU_RX_INTR(1UL << 1U),
kMU_Rx2FullInterruptEnable = MU_RX_INTR(1UL << 2U),
kMU_Rx3FullInterruptEnable = MU_RX_INTR(1UL << 3U),
kMU_GenInt0InterruptEnable = MU_GI_INTR(1UL << 0U),
kMU_GenInt1InterruptEnable = MU_GI_INTR(1UL << 1U),
kMU_GenInt2InterruptEnable = MU_GI_INTR(1UL << 2U),
kMU_GenInt3InterruptEnable = MU_GI_INTR(1UL << 3U),
kMU_OtherSideEnterRunInterruptEnable = MU_CORE_INTR(MU_CIER0_RUNIE_MASK),
kMU_OtherSideEnterHaltInterruptEnable = MU_CORE_INTR(MU_CIER0_HALTIE_MASK),
kMU_OtherSideEnterWaitInterruptEnable = MU_CORE_INTR(MU_CIER0_WAITIE_MASK),
kMU_OtherSideEnterStopInterruptEnable = MU_CORE_INTR(MU_CIER0_STOPIE_MASK),
kMU_OtherSideEnterPowerDownInterruptEnable = MU_CORE_INTR(MU_CIER0_PDIE_MASK),
kMU_ResetAssertInterruptEnable = MU_CORE_INTR(MU_CIER0_RAIE_MASK),
kMU_HardwareResetInterruptEnable = MU_CORE_INTR(MU_CIER0_HRIE_MASK),
kMU_MuResetInterruptEnable = MU_MISC_INTR(MU_CR_MURIE_MASK) }

 MU interrupt source to enable.
• enum _mu_interrupt_trigger {
 kMU_GenInt0InterruptTrigger = MU_GI_INTR(1UL << 0U),
 kMU_GenInt1InterruptTrigger = MU_GI_INTR(1UL << 1U),
 kMU_GenInt2InterruptTrigger = MU_GI_INTR(1UL << 2U),
 kMU_GenInt3InterruptTrigger = MU_GI_INTR(1UL << 3U) }

 MU interrupt that could be triggered to the other core.
• enum _mu_core_status_flags {
 kMU_OtherSideEnterRunFlag = MU_CSSR0_RUN_MASK,
 kMU_OtherSideEnterHaltFlag = MU_CSSR0_HALT_MASK,
 kMU_OtherSideEnterWaitFlag = MU_CSSR0_WAIT_MASK,
 kMU_OtherSideEnterStopFlag = MU_CSSR0_STOP_MASK,
 kMU_OtherSideEnterPowerDownFlag = MU_CSSR0_PD_MASK,
 kMU_OtherSideEnterResetFlag = MU_CSSR0_RAIP_MASK,
 kMU_HardwareResetFlag = MU_CSSR0_HRIP_MASK }

 MU core status flags.
• enum _mu_msg_reg_index {
 kMU_MsgReg0 = 0,
 kMU_MsgReg1,
 kMU_MsgReg2,
 kMU_MsgReg3 }

 MU message register index.
• enum _mu_general_purpose_interrupt {

```

```

kMU_GeneralPurposeInterrupt0 = 1UL << 0U,
kMU_GeneralPurposeInterrupt1 = 1UL << 1U,
kMU_GeneralPurposeInterrupt2 = 1UL << 2U,
kMU_GeneralPurposeInterrupt3 = 1UL << 3U }

```

*MU general purpose interrupts.*

## Driver version

- #define `FSL_MU_DRIVER_VERSION` (`MAKE_VERSION(2, 5, 0)`)  
*MU driver version.*

## MU instance.

- uint32\_t `MU_GetInstance` (MU\_Type \*base)  
*Get the MU instance index.*

## MU initialization.

- void `MU_Init` (MU\_Type \*base)  
*Initializes the MU module.*
- void `MU_Deinit` (MU\_Type \*base)  
*De-initializes the MU module.*

## MU Message

- static void `MU_SendMsgNonBlocking` (MU\_Type \*base, uint32\_t regIndex, uint32\_t msg)  
*Writes a message to the TX register.*
- void `MU_SendMsg` (MU\_Type \*base, uint32\_t regIndex, uint32\_t msg)  
*Blocks to send a message.*
- static uint32\_t `MU_ReceiveMsgNonBlocking` (MU\_Type \*base, uint32\_t regIndex)  
*Reads a message from the RX register.*
- uint32\_t `MU_ReceiveMsg` (MU\_Type \*base, uint32\_t regIndex)  
*Blocks to receive a message.*

## MU Flags

- static void `MU_SetFlagsNonBlocking` (MU\_Type \*base, uint32\_t flags)  
*Sets the 3-bit MU flags reflect on the other MU side.*
- void `MU_SetFlags` (MU\_Type \*base, uint32\_t flags)  
*Blocks setting the 3-bit MU flags reflect on the other MU side.*
- static uint32\_t `MU_GetFlags` (MU\_Type \*base)  
*Gets the current value of the 3-bit MU flags set by the other side.*

## Status and Interrupt.

- static uint32\_t `MU_GetCoreStatusFlags` (MU\_Type \*base)  
*Gets the MU core status flags.*
- uint32\_t `MU_GetStatusFlags` (MU\_Type \*base)  
*Gets the MU status flags.*

- static void [MU\\_ClearStatusFlags](#) (MU\_Type \*base, uint32\_t flags)  
*Clears the specific MU status flags.*
- static void [MU\\_EnableInterrupts](#) (MU\_Type \*base, uint32\_t interrupts)  
*Enables the specific MU interrupts.*
- static void [MU\\_DisableInterrupts](#) (MU\_Type \*base, uint32\_t interrupts)  
*Disables the specific MU interrupts.*
- [status\\_t MU\\_TriggerInterrupts](#) (MU\_Type \*base, uint32\_t interrupts)  
*Triggers interrupts to the other core.*
- [status\\_t MU\\_TriggerNmi](#) (MU\_Type \*base)  
*Triggers NMI to the other core.*
- static void [MU\\_ClearNmi](#) (MU\_Type \*base)  
*Clear non-maskable interrupt (NMI) sent by the other core.*

## MU general purpose interrupt

- static void [MU\\_EnableGeneralPurposeInterrupts](#) (MU\_Type \*base, uint32\_t interrupts)  
*Enables the MU general purpose interrupts.*
- static void [MU\\_DisableGeneralPurposeInterrupts](#) (MU\_Type \*base, uint32\_t interrupts)  
*Disables the MU general purpose interrupts.*
- static uint32\_t [MU\\_GetGeneralPurposeStatusFlags](#) (MU\_Type \*base)  
*Gets the MU general purpose interrupt status flags.*
- static void [MU\\_ClearGeneralPurposeStatusFlags](#) (MU\_Type \*base, uint32\_t flags)  
*Clear the MU general purpose interrupt status flags.*
- [status\\_t MU\\_TriggerGeneralPurposeInterrupts](#) (MU\_Type \*base, uint32\_t interrupts)  
*Triggers general purpose interrupts to the other core.*

## MU misc functions

- void [MU\\_BootOtherCore](#) (MU\_Type \*base, mu\_core\_boot\_mode\_t mode)  
*Boots the other core.*
- void [MU\\_HoldOtherCoreReset](#) (MU\_Type \*base)  
*Holds the other core reset.*
- static void [MU\\_ResetBothSides](#) (MU\_Type \*base)  
*Resets the MU for both A side and B side.*
- static void [MU\\_SetClockOnOtherCoreEnable](#) (MU\_Type \*base, bool enable)  
*Enables or disables the clock on the other core.*
- void [MU\\_HardwareResetOtherCore](#) (MU\_Type \*base, bool waitReset, bool holdReset, mu\_core\_boot\_mode\_t bootMode)  
*Hardware reset the other core.*

## 29.3 Macro Definition Documentation

### 29.3.1 #define FSL\_MU\_DRIVER\_VERSION (MAKE\_VERSION(2, 5, 0))

## 29.4 Enumeration Type Documentation

### 29.4.1 enum \_mu\_status\_flags

Enumerator

*kMU\_Tx0EmptyFlag* TX0 empty.



***kMU\_Tx1EmptyFlag*** TX1 empty.  
***kMU\_Tx2EmptyFlag*** TX2 empty.  
***kMU\_Tx3EmptyFlag*** TX3 empty.  
***kMU\_Rx0FullFlag*** RX0 full.  
***kMU\_Rx1FullFlag*** RX1 full.  
***kMU\_Rx2FullFlag*** RX2 full.  
***kMU\_Rx3FullFlag*** RX3 full.  
***kMU\_GenInt0Flag*** General purpose interrupt 0 pending.  
***kMU\_GenInt1Flag*** General purpose interrupt 1 pending.  
***kMU\_GenInt2Flag*** General purpose interrupt 2 pending.  
***kMU\_GenInt3Flag*** General purpose interrupt 3 pending.  
***kMU\_CoreEventPendingFlag*** The other core mode entry event pending.  
***kMU\_RxFullPendingFlag*** Any RX full flag is pending.  
***kMU\_TxEmptyPendingFlag*** Any TX empty flag is pending.  
***kMU\_GenIntPendingFlag*** Any general interrupt flag is pending.  
***kMU\_EventPendingFlag*** MU event pending.  
***kMU\_FlagsUpdatingFlag*** MU flags update is on-going.  
***kMU\_MuInResetFlag*** MU of any side is in reset.  
***kMU\_MuResetInterruptFlag*** The other side initializes MU reset.  
***kMU\_OtherSideEnterRunInterruptFlag*** The other side enters run mode.  
***kMU\_OtherSideEnterHaltInterruptFlag*** The other side enters halt mode.  
***kMU\_OtherSideEnterWaitInterruptFlag*** The other side enters wait mode.  
***kMU\_OtherSideEnterStopInterruptFlag*** The other side enters stop mode.  
***kMU\_OtherSideEnterPowerDownInterruptFlag*** The other side enters power down mode.  
***kMU\_ResetAssertInterruptFlag*** The other core reset assert interrupt.  
***kMU\_HardwareResetInterruptFlag*** Current side has been hardware reset by the other side.

### 29.4.2 enum \_mu\_interrupt\_enable

Enumerator

***kMU\_Tx0EmptyInterruptEnable*** TX0 empty.  
***kMU\_Tx1EmptyInterruptEnable*** TX1 empty.  
***kMU\_Tx2EmptyInterruptEnable*** TX2 empty.  
***kMU\_Tx3EmptyInterruptEnable*** TX3 empty.  
***kMU\_Rx0FullInterruptEnable*** RX0 full.  
***kMU\_Rx1FullInterruptEnable*** RX1 full.  
***kMU\_Rx2FullInterruptEnable*** RX2 full.  
***kMU\_Rx3FullInterruptEnable*** RX3 full.  
***kMU\_GenInt0InterruptEnable*** General purpose interrupt 0.  
***kMU\_GenInt1InterruptEnable*** General purpose interrupt 1.  
***kMU\_GenInt2InterruptEnable*** General purpose interrupt 2.  
***kMU\_GenInt3InterruptEnable*** General purpose interrupt 3.  
***kMU\_OtherSideEnterRunInterruptEnable*** The other side enters run mode.

***kMU\_OtherSideEnterHaltInterruptEnable*** The other side enters halt mode.  
***kMU\_OtherSideEnterWaitInterruptEnable*** The other side enters wait mode.  
***kMU\_OtherSideEnterStopInterruptEnable*** The other side enters stop mode.  
***kMU\_OtherSideEnterPowerDownInterruptEnable*** The other side enters power down mode.  
***kMU\_ResetAssertInterruptEnable*** The other core reset assert interrupt.  
***kMU\_HardwareResetInterruptEnable*** Current side has been hardware reset by the other side.  
***kMU\_MuResetInterruptEnable*** The other side initializes MU reset.

### 29.4.3 enum \_mu\_interrupt\_trigger

Enumerator

***kMU\_GenInt0InterruptTrigger*** General purpose interrupt 0.  
***kMU\_GenInt1InterruptTrigger*** General purpose interrupt 1.  
***kMU\_GenInt2InterruptTrigger*** General purpose interrupt 2.  
***kMU\_GenInt3InterruptTrigger*** General purpose interrupt 3.

### 29.4.4 enum \_mu\_core\_status\_flags

Enumerator

***kMU\_OtherSideEnterRunFlag*** The other side in run mode.  
***kMU\_OtherSideEnterHaltFlag*** The other side in halt mode.  
***kMU\_OtherSideEnterWaitFlag*** The other side in wait mode.  
***kMU\_OtherSideEnterStopFlag*** The other side in stop mode.  
***kMU\_OtherSideEnterPowerDownFlag*** The other side in power down mode.  
***kMU\_OtherSideEnterResetFlag*** The other core entered reset.  
***kMU\_HardwareResetFlag*** Current side has been hardware reset by the other side.

### 29.4.5 enum \_mu\_msg\_reg\_index

Enumerator

***kMU\_MsgReg0*** Message register 0.  
***kMU\_MsgReg1*** Message register 1.  
***kMU\_MsgReg2*** Message register 2.  
***kMU\_MsgReg3*** Message register 3.

## 29.4.6 enum \_mu\_general\_purpose\_interrupt

Enumerator

***kMU\_GeneralPurposeInterrupt0*** General purpose interrupt 0.  
***kMU\_GeneralPurposeInterrupt1*** General purpose interrupt 1.  
***kMU\_GeneralPurposeInterrupt2*** General purpose interrupt 2.  
***kMU\_GeneralPurposeInterrupt3*** General purpose interrupt 3.

## 29.5 Function Documentation

### 29.5.1 uint32\_t MU\_GetInstance ( MU\_Type \* *base* )

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

Returns

MU instance index.

### 29.5.2 void MU\_Init ( MU\_Type \* *base* )

This function enables the MU clock only.

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

### 29.5.3 void MU\_Deinit ( MU\_Type \* *base* )

This function disables the MU clock only.

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

### 29.5.4 static void MU\_SendMsgNonBlocking ( MU\_Type \* *base*, uint32\_t *regIndex*, uint32\_t *msg* ) [inline], [static]

This function writes a message to the specific TX register. It does not check whether the TX register is empty or not. The upper layer should make sure the TX register is empty before calling this function. This

function can be used in ISR for better performance.

```
* while (!(kMU_Tx0EmptyFlag & MU_GetStatusFlags(base))) { } Wait for TX0
 register empty.
* MU_SendMsgNonBlocking(base, kMU_MsgReg0, MSG_VAL); Write message to the
 TX0 register.
*
```

#### Parameters

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| <i>base</i>     | MU peripheral base address.                                 |
| <i>regIndex</i> | TX register index, see <a href="#">mu_msg_reg_index_t</a> . |
| <i>msg</i>      | Message to send.                                            |

### 29.5.5 void MU\_SendMsg ( MU\_Type \* *base*, uint32\_t *regIndex*, uint32\_t *msg* )

This function waits until the TX register is empty and sends the message.

#### Parameters

|                 |                                                               |
|-----------------|---------------------------------------------------------------|
| <i>base</i>     | MU peripheral base address.                                   |
| <i>regIndex</i> | MU message register, see <a href="#">mu_msg_reg_index_t</a> . |
| <i>msg</i>      | Message to send.                                              |

### 29.5.6 static uint32\_t MU\_ReceiveMsgNonBlocking ( MU\_Type \* *base*, uint32\_t *regIndex* ) [inline], [static]

This function reads a message from the specific RX register. It does not check whether the RX register is full or not. The upper layer should make sure the RX register is full before calling this function. This function can be used in ISR for better performance.

```
* uint32_t msg;
* while (!(kMU_Rx0FullFlag & MU_GetStatusFlags(base)))
* {
* } Wait for the RX0 register full.
*
* msg = MU_ReceiveMsgNonBlocking(base, kMU_MsgReg0); Read message from
 RX0 register.
*
```

## Parameters

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| <i>base</i>     | MU peripheral base address.                                 |
| <i>regIndex</i> | RX register index, see <a href="#">mu_msg_reg_index_t</a> . |

## Returns

The received message.

### 29.5.7 uint32\_t MU\_ReceiveMsg ( MU\_Type \* *base*, uint32\_t *regIndex* )

This function waits until the RX register is full and receives the message.

## Parameters

|                 |                                                             |
|-----------------|-------------------------------------------------------------|
| <i>base</i>     | MU peripheral base address.                                 |
| <i>regIndex</i> | MU message register, see <a href="#">mu_msg_reg_index_t</a> |

## Returns

The received message.

### 29.5.8 static void MU\_SetFlagsNonBlocking ( MU\_Type \* *base*, uint32\_t *flags* ) [inline], [static]

This function sets the 3-bit MU flags directly. Every time the 3-bit MU flags are changed, the status flag kMU\_FlagsUpdatingFlag asserts indicating the 3-bit MU flags are updating to the other side. After the 3-bit MU flags are updated, the status flag kMU\_FlagsUpdatingFlag is cleared by hardware. During the flags updating period, the flags cannot be changed. The upper layer should make sure the status flag kMU\_FlagsUpdatingFlag is cleared before calling this function.

```
* while (kMU_FlagsUpdatingFlag & MU_GetStatusFlags(base))
* {
* } Wait for previous MU flags updating.
*
* MU_SetFlagsNonBlocking(base, 0U); Set the mU flags.
*
```

## Parameters

|              |                             |
|--------------|-----------------------------|
| <i>base</i>  | MU peripheral base address. |
| <i>flags</i> | The 3-bit MU flags to set.  |

**29.5.9 void MU\_SetFlags ( MU\_Type \* *base*, uint32\_t *flags* )**

This function blocks setting the 3-bit MU flags. Every time the 3-bit MU flags are changed, the status flag kMU\_FlagsUpdatingFlag asserts indicating the 3-bit MU flags are updating to the other side. After the 3-bit MU flags are updated, the status flag kMU\_FlagsUpdatingFlag is cleared by hardware. During the flags updating period, the flags cannot be changed. This function waits for the MU status flag kMU\_FlagsUpdatingFlag cleared and sets the 3-bit MU flags.

## Parameters

|              |                             |
|--------------|-----------------------------|
| <i>base</i>  | MU peripheral base address. |
| <i>flags</i> | The 3-bit MU flags to set.  |

**29.5.10 static uint32\_t MU\_GetFlags ( MU\_Type \* *base* ) [inline], [static]**

This function gets the current 3-bit MU flags on the current side.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

## Returns

flags Current value of the 3-bit flags.

**29.5.11 static uint32\_t MU\_GetCoreStatusFlags ( MU\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

Returns

Bit mask of the MU status flags, see [\\_mu\\_core\\_status\\_flags](#).

### 29.5.12 uint32\_t MU\_GetStatusFlags ( MU\_Type \* *base* )

This function returns the bit mask of the MU status flags. See [\\_mu\\_status\\_flags](#).

```
* uint32_t flags;
* flags = MU_GetStatusFlags(base); Get all status flags.
* if (kMU_Tx0EmptyFlag & flags)
* {
* The TX0 register is empty. Message can be sent.
* MU_SendMsgNonBlocking(base, kMU_MsgReg0, MSG0_VAL);
* }
* if (kMU_Tx1EmptyFlag & flags)
* {
* The TX1 register is empty. Message can be sent.
* MU_SendMsgNonBlocking(base, kMU_MsgReg1, MSG1_VAL);
* }
*
```

If there are more than 4 general purpose interrupts, use [MU\\_GetGeneralPurposeStatusFlags](#).

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

Returns

Bit mask of the MU status flags, see [\\_mu\\_status\\_flags](#).

### 29.5.13 static void MU\_ClearStatusFlags ( MU\_Type \* *base*, uint32\_t *flags* ) [inline], [static]

This function clears the specific MU status flags. The flags to clear should be passed in as bit mask. See [\\_mu\\_status\\_flags](#).

```
* Clear general interrupt 0 and general interrupt 1 pending flags.
* MU_ClearStatusFlags(base, kMU_GenInt0Flag |
* kMU_GenInt1Flag);
*
```

If there are more than 4 general purpose interrupts, use [MU\\_ClearGeneralPurposeStatusFlags](#).

## Parameters

|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>  | MU peripheral base address.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <i>flags</i> | Bit mask of the MU status flags. See <code>_mu_status_flags</code> . Only the following flags can be cleared by software, other flags are cleared by hardware: <ul style="list-style-type: none"> <li>• <a href="#">kMU_GenInt0Flag</a></li> <li>• <a href="#">kMU_GenInt1Flag</a></li> <li>• <a href="#">kMU_GenInt2Flag</a></li> <li>• <a href="#">kMU_GenInt3Flag</a></li> <li>• <a href="#">kMU_MuResetInterruptFlag</a></li> <li>• <a href="#">kMU_OtherSideEnterRunInterruptFlag</a></li> <li>• <a href="#">kMU_OtherSideEnterHaltInterruptFlag</a></li> <li>• <a href="#">kMU_OtherSideEnterWaitInterruptFlag</a></li> <li>• <a href="#">kMU_OtherSideEnterStopInterruptFlag</a></li> <li>• <a href="#">kMU_OtherSideEnterPowerDownInterruptFlag</a></li> <li>• <a href="#">kMU_ResetAssertInterruptFlag</a></li> <li>• <a href="#">kMU_HardwareResetInterruptFlag</a></li> </ul> |

### 29.5.14 static void MU\_EnableInterrupts ( MU\_Type \* *base*, uint32\_t *interrupts* ) [inline], [static]

This function enables the specific MU interrupts. The interrupts to enable should be passed in as bit mask. See `_mu_interrupt_enable`.

```
* Enable general interrupt 0 and TX0 empty interrupt.
* MU_EnableInterrupts(base, kMU_GenInt0InterruptEnable |
* kMU_Tx0EmptyInterruptEnable);
*
```

If there are more than 4 general purpose interrupts, use [MU\\_EnableGeneralPurposeInterrupts](#).

## Parameters

|                   |                                                                        |
|-------------------|------------------------------------------------------------------------|
| <i>base</i>       | MU peripheral base address.                                            |
| <i>interrupts</i> | Bit mask of the MU interrupts. See <code>_mu_interrupt_enable</code> . |

### 29.5.15 static void MU\_DisableInterrupts ( MU\_Type \* *base*, uint32\_t *interrupts* ) [inline], [static]

This function disables the specific MU interrupts. The interrupts to disable should be passed in as bit mask. See `_mu_interrupt_enable`.



```

* Disable general interrupt 0 and TX0 empty interrupt.
* MU_DisableInterrupts(base, kMU_GenInt0InterruptEnable |
* kMU_Tx0EmptyInterruptEnable);
*

```

If there are more than 4 general purpose interrupts, use [MU\\_DisableGeneralPurposeInterrupts](#).

Parameters

|                   |                                                                        |
|-------------------|------------------------------------------------------------------------|
| <i>base</i>       | MU peripheral base address.                                            |
| <i>interrupts</i> | Bit mask of the MU interrupts. See <code>_mu_interrupt_enable</code> . |

### 29.5.16 `status_t MU_TriggerInterrupts ( MU_Type * base, uint32_t interrupts )`

This function triggers the specific interrupts to the other core. The interrupts to trigger are passed in as bit mask. See [\\_mu\\_interrupt\\_trigger](#). The MU should not trigger an interrupt to the other core when the previous interrupt has not been processed by the other core. This function checks whether the previous interrupts have been processed. If not, it returns an error.

```

* if (kStatus_Success != MU_TriggerInterrupts(base,
* kMU_GenInt0InterruptTrigger |
* kMU_GenInt2InterruptTrigger))
* {
* Previous general purpose interrupt 0 or general purpose interrupt 2
* has not been processed by the other core.
* }
*

```

If there are more than 4 general purpose interrupts, use [MU\\_TriggerGeneralPurposeInterrupts](#).

Parameters

|                   |                                                                                 |
|-------------------|---------------------------------------------------------------------------------|
| <i>base</i>       | MU peripheral base address.                                                     |
| <i>interrupts</i> | Bit mask of the interrupts to trigger. See <code>_mu_interrupt_trigger</code> . |

Return values

|                        |                                              |
|------------------------|----------------------------------------------|
| <i>kStatus_Success</i> | Interrupts have been triggered successfully. |
| <i>kStatus_Fail</i>    | Previous interrupts have not been accepted.  |

### 29.5.17 `status_t MU_TriggerNmi ( MU_Type * base )`

This function triggers the NMI to the other core. The MU should not trigger NMI to the other core when the previous interrupt has not been processed by the other core. This function checks whether the previous interrupts have been processed. If not, it returns an error.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

## Return values

|                        |                                              |
|------------------------|----------------------------------------------|
| <i>kStatus_Success</i> | Interrupts have been triggered successfully. |
| <i>kStatus_Fail</i>    | Previous interrupts have not been accepted.  |

### 29.5.18 static void MU\_ClearNmi ( MU\_Type \* *base* ) [inline], [static]

This function clears non-maskable interrupt (NMI) sent by the other core.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

### 29.5.19 static void MU\_EnableGeneralPurposeInterrupts ( MU\_Type \* *base*, uint32\_t *interrupts* ) [inline], [static]

This function enables the MU general purpose interrupts. The interrupts to enable should be passed in as bit mask of [mu\\_general\\_purpose\\_interrupt\\_t](#). The function [MU\\_EnableInterrupts](#) only support general interrupt 0~3, this function supports all general interrupts.

For example, to enable general purpose interrupt 0 and 3, use like this:

```
MU_EnableGeneralPurposeInterrupts (MU,
 kMU_GeneralPurposeInterrupt0 |
 kMU_GeneralPurposeInterrupt3);
```

## Parameters

|                   |                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------|
| <i>base</i>       | MU peripheral base address.                                                                         |
| <i>interrupts</i> | Bit mask of the MU general purpose interrupts, see <a href="#">mu_general_purpose_interrupt_t</a> . |

### 29.5.20 static void MU\_DisableGeneralPurposeInterrupts ( MU\_Type \* *base*, uint32\_t *interrupts* ) [inline], [static]

This function disables the MU general purpose interrupts. The interrupts to disable should be passed in as bit mask of [mu\\_general\\_purpose\\_interrupt\\_t](#). The function [MU\\_DisableInterrupts](#) only support general interrupt 0~3, this function supports all general interrupts.

For example, to disable general purpose interrupt 0 and 3, use like this:

```
MU_EnableGeneralPurposeInterrupts (MU,
 kMU_GeneralPurposeInterrupt0 |
 kMU_GeneralPurposeInterrupt3);
```

Parameters

|                   |                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------|
| <i>base</i>       | MU peripheral base address.                                                                         |
| <i>interrupts</i> | Bit mask of the MU general purpose interrupts. see <a href="#">mu_general_purpose_interrupt_t</a> . |

### 29.5.21 static uint32\_t MU\_GetGeneralPurposeStatusFlags ( MU\_Type \* *base* ) [inline], [static]

This function returns the bit mask of the MU general purpose interrupt status flags. [MU\\_GetStatusFlags](#) can only get general purpose interrupt status 0~3, this function can get all general purpose interrupts status.

This example shows to check whether general purpose interrupt 0 and 3 happened.

```
uint32_t flags;
flags = MU_GetGeneralPurposeStatusFlags (base);
if (kMU_GeneralPurposeInterrupt0 & flags)
{
}
if (kMU_GeneralPurposeInterrupt3 & flags)
{
}
```

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

Returns

Bit mask of the MU general purpose interrupt status flags.

### 29.5.22 static void MU\_ClearGeneralPurposeStatusFlags ( MU\_Type \* *base*, uint32\_t *flags* ) [inline], [static]

This function clears the specific MU general purpose interrupt status flags. The flags to clear should be passed in as bit mask. [mu\\_general\\_purpose\\_interrupt\\_t\\_mu\\_status\\_flags](#).

Example to clear general purpose interrupt 0 and general interrupt 1 pending flags.

```
MU_ClearGeneralPurposeStatusFlags (base,
 kMU_GeneralPurposeInterrupt0 |
 kMU_GeneralPurposeInterrupt1);
```

## Parameters

|              |                                                                                                                 |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| <i>base</i>  | MU peripheral base address.                                                                                     |
| <i>flags</i> | Bit mask of the MU general purpose interrupt status flags. See <a href="#">mu_general_purpose_interrupt_t</a> . |

### 29.5.23 **status\_t MU\_TriggerGeneralPurposeInterrupts ( MU\_Type \* *base*, uint32\_t *interrupts* )**

This function triggers the specific general purpose interrupts to the other core. The interrupts to trigger are passed in as bit mask. See [mu\\_general\\_purpose\\_interrupt\\_t](#). The MU should not trigger an interrupt to the other core when the previous interrupt has not been processed by the other core. This function checks whether the previous interrupts have been processed. If not, it returns an error.

```
status_t status;
status = MU_TriggerGeneralPurposeInterrupts (base,
 kMU_GeneralPurposeInterrupt0 |
 kMU_GeneralPurposeInterrupt2);

if (kStatus_Success != status)
{
 Previous general purpose interrupt 0 or general purpose interrupt 2
 has not been processed by the other core.
}
```

## Parameters

|                   |                                                                                             |
|-------------------|---------------------------------------------------------------------------------------------|
| <i>base</i>       | MU peripheral base address.                                                                 |
| <i>interrupts</i> | Bit mask of the interrupts to trigger. See <a href="#">mu_general_purpose_interrupt_t</a> . |

## Return values

|                        |                                              |
|------------------------|----------------------------------------------|
| <i>kStatus_Success</i> | Interrupts have been triggered successfully. |
| <i>kStatus_Fail</i>    | Previous interrupts have not been accepted.  |

### 29.5.24 **void MU\_BootOtherCore ( MU\_Type \* *base*, mu\_core\_boot\_mode\_t *mode* )**

This function boots the other core with a boot configuration.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
| <i>mode</i> | The other core boot mode.   |

**29.5.25 void MU\_HoldOtherCoreReset ( MU\_Type \* *base* )**

This function causes the other core to be held in reset following any reset event.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

**29.5.26 static void MU\_ResetBothSides ( MU\_Type \* *base* ) [inline], [static]**

This function resets the MU for both A side and B side. Before reset, it is recommended to interrupt processor B, because this function may affect the ongoing processor B programs.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | MU peripheral base address. |
|-------------|-----------------------------|

**29.5.27 static void MU\_SetClockOnOtherCoreEnable ( MU\_Type \* *base*, bool *enable* ) [inline], [static]**

This function enables or disables the platform clock on the other core when that core enters a stop mode. If disabled, the platform clock for the other core is disabled when it enters stop mode. If enabled, the platform clock keeps running on the other core in stop mode, until this core also enters stop mode.

## Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>base</i>   | MU peripheral base address.                    |
| <i>enable</i> | Enable or disable the clock on the other core. |

### 29.5.28 void MU\_HardwareResetOtherCore ( MU\_Type \* *base*, bool *waitReset*, bool *holdReset*, mu\_core\_boot\_mode\_t *bootMode* )

This function resets the other core, the other core could mask the hardware reset by calling MU\_MaskHardwareReset. The hardware reset mask feature is only available for some platforms. This function could be used together with MU\_BootOtherCore to control the other core reset workflow.

Example 1: Reset the other core, and no hold reset

```
* MU_HardwareResetOtherCore(MU_A, true, false, bootMode);
*
```

In this example, the core at MU side B will reset with the specified boot mode.

Example 2: Reset the other core and hold it, then boot the other core later.

```
* Here the other core enters reset, and the reset is hold
* MU_HardwareResetOtherCore(MU_A, true, true, modeDontCare);
* Current core boot the other core when necessary.
* MU_BootOtherCore(MU_A, bootMode);
*
```

#### Note

The feature *waitReset*, *holdReset*, and *bootMode* might be not supported for some platforms. *waitReset* is only available for platforms that FSL\_FEATURE\_MU\_NO\_CORE\_STATUS not defined as 1 and FSL\_FEATURE\_MU\_HAS\_RESET\_ASSERT\_INT not defined as 0. *holdReset* is only available for platforms that FSL\_FEATURE\_MU\_HAS\_RSTH not defined as 0. *bootMode* is only available for platforms that FSL\_FEATURE\_MU\_HAS\_BOOT not defined as 0.

#### Parameters

|                  |                                                                                                                                                                                                                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>      | MU peripheral base address.                                                                                                                                                                                                                                                                          |
| <i>waitReset</i> | Wait the other core enters reset. Only work when there is CSSR0[RAIP] <ul style="list-style-type: none"> <li>• true: Wait until the other core enters reset, if the other core has masked the hardware reset, then this function will be blocked.</li> <li>• false: Don't wait the reset.</li> </ul> |

|                  |                                                                                                                                                                                                                                                                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>holdReset</i> | Hold the other core reset or not. Only work when there is CCR0[RSTH] <ul style="list-style-type: none"><li>• true: Hold the other core in reset, this function returns directly when the other core enters reset.</li><li>• false: Don't hold the other core in reset, this function waits until the other core out of reset.</li></ul> |
| <i>bootMode</i>  | Boot mode of the other core, if <code>holdReset</code> is true, this parameter is useless.                                                                                                                                                                                                                                              |

## Chapter 30

# PMC0: Power Management Controller

### 30.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Power Management Controller (PMC) module of MCUXpresso SDK devices. The Power Management Controller (PMC) can be divided in two parts: PMC 0 and PMC 1. The PMC 0 controls the Core 0, its SoG and RAM, and the PMC 1 controls the Core 1, its SoG and RAM. This driver is for PMC 0 only.

The PMC 0 has:

- the high-power (HP) and low-power (LP) Core Regulator;
- the high-power (HP) and low-power (LP) Array Regulator;
- the high-power (HP) and low-power (LP) 1.2V Low Voltage Detector (LVD) monitor (in regulators input);
- the high-power (HP) and low-power (LP) 1.2V High Voltage Detector (HVD) monitor (in regulators input);
- the bandgap;
- the forward bias (FBB) and the reverse back bias (RBB). In addition, the PMC has a 1.8 V POR (Power-On Reset) monitor to assure the voltage level in the Always-On power domain would be in the correct range to the correct functionality of the internal digital and analog blocks. Both PMCs receive requests from the MSMC to change the current power mode. Each PMC allows the customer to choose what features are enabled or disabled for each power mode using the PMC registers.

### 30.2 Typical use case

#### 30.2.1 Turn on the PMC 1 using LDO Regulator

After a POR event, when the PMC 0 is during RUN mode and the PMC 1 is turned off. The procedure to turn on the PMC 1 using the internal LDO Regulator.

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/pmc0`

#### 30.2.2 Turn on the PMC 1 using the PMIC

After a POR event, when the PMC 0 is during RUN mode and the PMC 1 is turned off. The procedure to turn on the PMC 1 using the external PMIC

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/pmc0`



### 30.2.3 Turn off the LDO Regulator

When the PMC 1 is during RUN mode, the LDO Regulator can be programmed to be turned off in the next transition from RUN to VLLS power mode. As in VLLS the regulator is disconnected from the load by the switches (switches are OFF), a external regulator can assume the power supply (PMIC).

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/pmc0`

### 30.2.4 Turn on the LDO Regulator

When the PMC 1 is during VLLS mode, the LDO Regulator can be turned on in a transition to RUN mode.

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/pmc0`

### 30.2.5 Change the Core Regulator voltage level in PMC 0 RUN or HSRUN mode

To change the Core Regulator voltage level when the PMC 0 is in RUN mode:

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/pmc0`

To change the Core Regulator voltage level when the PMC 0 is in HSRUN mode:

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/pmc0`

### 30.2.6 Change the SRAMs power mode during PMC 0 RUN mode

To change the SRAMs power mode during the PMC 0 RUN mode.

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/pmc0`

## Files

- file [fsl\\_pmc0.h](#)

## Data Structures

- struct [\\_pmc0\\_hsruntime\\_config](#)  
*PMC 0 HSRUN mode configuration. [More...](#)*
- struct [\\_pmc0\\_run\\_mode\\_config](#)  
*PMC 0 RUN mode configuration. [More...](#)*
- struct [\\_pmc0\\_vlpr\\_mode\\_config](#)  
*PMC 0 VLPR mode configuration. [More...](#)*
- struct [\\_pmc0\\_stop\\_mode\\_config](#)  
*PMC 0 STOP mode configuration. [More...](#)*

- struct `_pmc0_vlps_mode_config`  
*PMC 0 VLPS mode configuration. [More...](#)*
- struct `_pmc0_lls_mode_config`  
*PMC 0 LLS mode configuration. [More...](#)*
- struct `_pmc0_vlls_mode_config`  
*PMC 0 VLLS mode configuration. [More...](#)*
- struct `_pmc0_bias_config`  
*PMC 0 bias configuration. [More...](#)*

## Macros

- #define `CORE_REGULATOR_VOLT_LEVEL_MAX` 50U  
*MAX valid values of Core Regulator Voltage Level.*

## Typedefs

- typedef enum  
`_pmc0_high_volt_detect_monitor_select pmc0_high_volt_detect_monitor_select_t`  
*High Voltage Detect Monitor Select.*
- typedef enum  
`_pmc0_low_volt_detect_monitor_select pmc0_low_volt_detect_monitor_select_t`  
*Low Voltage Detect Monitor Select.*
- typedef enum  
`_pmc0_core_regulator_select pmc0_core_regulator_select_t`  
*Core Regulator Select.*
- typedef enum  
`_pmc0_array_regulator_select pmc0_array_regulator_select_t`  
*Array Regulator Select.*
- typedef enum  
`_pmc0_vlls_array_regulator_select pmc0_vlls_array_regulator_select_t`  
*VLLS mode array Regulator Select.*
- typedef enum  
`_pmc0_fbb_p_well_voltage_level_select pmc0_fbb_p_well_voltage_level_select_t`  
*FBB P-Well voltage level select.*
- typedef enum  
`_pmc0_fbb_n_well_voltage_level_select pmc0_fbb_n_well_voltage_level_select_t`  
*FBB N-Well voltage level select.*
- typedef enum  
`_pmc0_rbb_p_well_voltage_level_select pmc0_rbb_p_well_voltage_level_select_t`  
*RBB P-Well voltage level select.*
- typedef enum  
`_pmc0_rbb_n_well_voltage_level_select pmc0_rbb_n_well_voltage_level_select_t`  
*RBB N-Well voltage level select.*
- typedef struct  
`_pmc0_hsruntime_config pmc0_hsruntime_config_t`  
*PMC 0 HSRUN mode configuration.*
- typedef struct  
`_pmc0_run_mode_config pmc0_run_mode_config_t`  
*PMC 0 RUN mode configuration.*

- typedef struct  
`_pmc0_vlpr_mode_config pmc0_vlpr_mode_config_t`  
*PMC 0 VLPR mode configuration.*
- typedef struct  
`_pmc0_stop_mode_config pmc0_stop_mode_config_t`  
*PMC 0 STOP mode configuration.*
- typedef struct  
`_pmc0_vlps_mode_config pmc0_vlps_mode_config_t`  
*PMC 0 VLPS mode configuration.*
- typedef struct  
`_pmc0_lls_mode_config pmc0_lls_mode_config_t`  
*PMC 0 LLS mode configuration.*
- typedef struct  
`_pmc0_vlls_mode_config pmc0_vlls_mode_config_t`  
*PMC 0 VLLS mode configuration.*
- typedef struct `_pmc0_bias_config pmc0_bias_config_t`  
*PMC 0 bias configuration.*

## Enumerations

- enum `_pmc0_high_volt_detect_monitor_select` {  
`kPMC0_HighVoltDetectLowPowerMonitor = 0U,`  
`kPMC0_HighVoltDetectHighPowerMonitor = 1U }`  
*High Voltage Detect Monitor Select.*
- enum `_pmc0_low_volt_detect_monitor_select` {  
`kPMC0_LowVoltDetectLowPowerMonitor = 0U,`  
`kPMC0_LowVoltDetectHighPowerMonitor = 1U }`  
*Low Voltage Detect Monitor Select.*
- enum `_pmc0_core_regulator_select` {  
`kPMC0_CoreLowPowerRegulator = 0U,`  
`kPMC0_CoreHighPowerRegulator = 1U }`  
*Core Regulator Select.*
- enum `_pmc0_array_regulator_select` {  
`kPMC0_ArrayLowPowerRegulator = 0U,`  
`kPMC0_ArrayHighPowerRegulator = 1U }`  
*Array Regulator Select.*
- enum `_pmc0_vlls_array_regulator_select` {  
`kPMC0_VllsArrayRegulatorOff = 0U,`  
`kPMC0_VllsArrayLowPowerRegulator = 2U,`  
`kPMC0_VllsArrayHighPowerRegulator = 3U }`  
*VLLS mode array Regulator Select.*
- enum `_pmc0_fbb_p_well_voltage_level_select` {

```

kPMC0_FbbPWellNoBiasCondition = 0U,
kPMC0_FbbPWellVoltageLevelAt50Mv = 1U,
kPMC0_FbbPWellVoltageLevelAt150Mv = 2U,
kPMC0_FbbPWellVoltageLevelAt100Mv = 3U,
kPMC0_FbbPWellVoltageLevelAt350Mv = 4U,
kPMC0_FbbPWellVoltageLevelAt300Mv = 5U,
kPMC0_FbbPWellVoltageLevelAt200Mv = 6U,
kPMC0_FbbPWellVoltageLevelAt250Mv = 7U }

```

*FBF P-Well voltage level select.*

- enum `_pmc0_fbb_n_well_voltage_level_select` {  
`kPMC0_FbbNWellNoBiasCondition` = 0U,  
`kPMC0_FbbNWellVoltageLevelAtMinus50Mv` = 1U,  
`kPMC0_FbbNWellVoltageLevelAtMinus150Mv` = 2U,  
`kPMC0_FbbNWellVoltageLevelAtMinus100Mv` = 3U,  
`kPMC0_FbbNWellVoltageLevelAtMinus350Mv` = 4U,  
`kPMC0_FbbNWellVoltageLevelAtMinus300Mv` = 5U,  
`kPMC0_FbbNWellVoltageLevelAtMinus200Mv` = 6U,  
`kPMC0_FbbNWellVoltageLevelAtMinus250Mv` = 7U }

*FBF N-Well voltage level select.*

- enum `_pmc0_rbb_p_well_voltage_level_select` {  
`kPMC0_RBBPWellVoltageLevelAtMinus0_5V` = 0U,  
`kPMC0_RBBPWellVoltageLevelAtMinus0_6V` = 1U,  
`kPMC0_RBBPWellVoltageLevelAtMinus0_7V` = 2U,  
`kPMC0_RBBPWellVoltageLevelAtMinus0_8V` = 3U,  
`kPMC0_RBBPWellVoltageLevelAtMinus0_9V` = 4U,  
`kPMC0_RBBPWellVoltageLevelAtMinus1_0V` = 5U,  
`kPMC0_RBBPWellVoltageLevelAtMinus1_1V` = 6U,  
`kPMC0_RBBPWellVoltageLevelAtMinus1_2V` = 7U,  
`kPMC0_RBBPWellVoltageLevelAtMinus1_3V` = 8U }

*RBB P-Well voltage level select.*

- enum `_pmc0_rbb_n_well_voltage_level_select` {  
`kPMC0_RBBNWellVoltageLevelAt0_5V` = 0U,  
`kPMC0_RBBNWellVoltageLevelAt0_6V` = 1U,  
`kPMC0_RBBNWellVoltageLevelAt0_7V` = 2U,  
`kPMC0_RBBNWellVoltageLevelAt0_8V` = 3U,  
`kPMC0_RBBNWellVoltageLevelAt0_9V` = 4U,  
`kPMC0_RBBNWellVoltageLevelAt1_0V` = 5U,  
`kPMC0_RBBNWellVoltageLevelAt1_1V` = 6U,  
`kPMC0_RBBNWellVoltageLevelAt1_2V` = 7U,  
`kPMC0_RBBNWellVoltageLevelAt1_3V` = 8U }

*RBB N-Well voltage level select.*

- enum `_pmc0_status_flags` {

```

kPMC0_LowVoltDetectEventFlag,
kPMC0_LowVoltDetectValueFlag = PMC0_STATUS_LVDV_MASK,
kPMC0_HighVoltDetectEventFlag,
kPMC0_HighVoltDetectValueFlag = PMC0_STATUS_HVDV_MASK,
kPMC0_CoreRegulatorVoltLevelFlag = PMC0_STATUS_COREVLF_MASK,
kPMC0_SramFlag,
kPMC0_PMC1VoltageSourceFlag }
 PMC 0 status flags.
• enum _pmc0_power_mode_status_flags {
 kPMC0_HSRUNModeStatusFlags = 0U,
 kPMC0_RUNModeStatusFlags = 1U,
 kPMC0_STOPModeStatusFlags = 2U,
 kPMC0_VLPRModeStatusFlags = 3U,
 kPMC0_VLPSModeStatusFlags = 4U,
 kPMC0_LLSModeStatusFlags = 5U,
 kPMC0_VLLSModeStatusFlags = 6U }
 PMC 0 power mode status flags.

```

## Driver version

- #define `FSL_PMC0_DRIVER_VERSION` (`MAKE_VERSION(2, 1, 1)`)  
*PMC 0 driver version.*

## Power Management Controller Control APIs

- static void `PMC0_ConfigureHsrunMode` (const `pmc0_hsrun_mode_config_t` \*config)  
*Configure the HSRUN power mode.*
- static void `PMC0_ConfigureRunMode` (const `pmc0_run_mode_config_t` \*config)  
*Configure the RUN power mode.*
- static void `PMC0_ConfigureVlprMode` (const `pmc0_vlpr_mode_config_t` \*config)  
*Configure the VLPR power mode.*
- static void `PMC0_ConfigureStopMode` (const `pmc0_stop_mode_config_t` \*config)  
*Configure the STOP power mode.*
- static void `PMC0_ConfigureVlpsMode` (const `pmc0_vlps_mode_config_t` \*config)  
*Configure the VLPS power mode.*
- static void `PMC0_ConfigureLlsMode` (const `pmc0_lls_mode_config_t` \*config)  
*Configure the LLS power mode.*
- static void `PMC0_ConfigureVllsMode` (const `pmc0_vlls_mode_config_t` \*config)  
*Configure the VLLS power mode.*
- static uint32\_t `PMC0_GetPMC0PowerModeStatusFlags` (void)  
*Get current power mode of PMC 0.*
- static bool `PMC0_GetPMC0PowerTransitionStatus` (void)  
*Get the status of PMC 0 power mode transition.*
- static uint32\_t `PMC0_GetPMC1PowerModeStatusFlags` (void)  
*Get current power mode of PMC 1.*
- static bool `PMC0_GetPMC1PowerTransitionStatus` (void)  
*Get the status of PMC 1 power mode transition.*
- static uint32\_t `PMC0_GetStatusFlags` (void)  
*Gets PMC 0 status flags.*

- static void [PMC0\\_EnableLowVoltDetectInterrupt](#) (void)  
*Enables the 1.2V Low-Voltage Detector interrupt.*
- static void [PMC0\\_DisableLowVoltDetectInterrupt](#) (void)  
*Disables the 1.2V Low-Voltage Detector interrupt.*
- static void [PMC0\\_ClearLowVoltDetectFlag](#) (void)  
*Clears the 1.2V Low-Voltage Detector flag.*
- static void [PMC0\\_EnableHighVoltDetectInterrupt](#) (void)  
*Enables the 1.8V High-Voltage Detector interrupt.*
- static void [PMC0\\_DisableHighVoltDetectInterrupt](#) (void)  
*Disables the 1.8V High-Voltage Detector interrupt.*
- static void [PMC0\\_ClearHighVoltDetectFlag](#) (void)  
*Clears the 1.8V High-Voltage Detector flag.*
- static void [PMC0\\_EnableLowVoltDetectReset](#) (bool enable)  
*Enables the 1.2V Low-Voltage Detector reset.*
- static void [PMC0\\_EnableHighVoltDetectReset](#) (bool enable)  
*Enables the 1.8V High-Voltage Detector reset.*
- static void [PMC0\\_ClearPadsIsolation](#) (void)  
*Releases/clears the isolation in the PADS.*
- static void [PMC0\\_PowerOnPmc1](#) (void)  
*Powers on PMC 1.*
- static void [PMC0\\_EnableWaitLdoOkSignal](#) (bool enable)  
*Enables to wait LDO OK signal.*
- static void [PMC0\\_EnablePmc1LdoRegulator](#) (bool enable)  
*Enables PMC 1 LDO Regulator.*
- static void [PMC0\\_EnablePmc1RBBMode](#) (bool enable)  
*Enable the PMC 1 RBB(reverse back bias) mode.*
- static void [PMC0\\_SetBiasConfig](#) (const [pmc0\\_bias\\_config\\_t](#) \*config)  
*Configure the PMC 0 bias voltage level and enable/disable pull-down.*
- static void [PMC0\\_ConfigureSramBankPowerDown](#) (uint32\_t bankMask)  
*Configures PMC 0 SRAM bank power down.*
- static void [PMC0\\_ConfigureSramBankPowerDownStopMode](#) (uint32\_t bankMask)  
*Configures PMC 0 SRAM bank power down in stop modes.*
- static void [PMC0\\_ConfigureSramBankPowerDownStandbyMode](#) (uint32\_t bankMask)  
*Configures PMC 0 SRAM bank power down in Standby Mode.*
- static void [PMC0\\_EnableTemperatureSensor](#) (bool enable)  
*Enable/disable internal temperature sensor.*
- static void [PMC0\\_SetTemperatureSensorMode](#) (uint8\_t mode)  
*Set temperature sensor mode.*

## 30.3 Data Structure Documentation

### 30.3.1 struct [\\_pmc0\\_hsruntime\\_mode\\_config](#)

#### Data Fields

- uint32\_t [\\_\\_pad0\\_\\_](#): 16  
*Reserved.*
- uint32\_t [coreRegulatorVoltLevel](#): 6  
*Core Regulator Voltage Level.*
- uint32\_t [\\_\\_pad1\\_\\_](#): 2

- *Reserved.*
- uint32\_t [enableForwardBias](#): 1  
*Enable forward bias.*
- uint32\_t [\\_\\_pad2\\_\\_](#): 7  
*Reserved.*

#### Field Documentation

- (1) uint32\_t [\\_pmc0\\_hsrun\\_mode\\_config::\\_\\_pad0\\_\\_](#)
- (2) uint32\_t [\\_pmc0\\_hsrun\\_mode\\_config::coreRegulatorVoltLevel](#)
- (3) uint32\_t [\\_pmc0\\_hsrun\\_mode\\_config::\\_\\_pad1\\_\\_](#)
- (4) uint32\_t [\\_pmc0\\_hsrun\\_mode\\_config::enableForwardBias](#)
- (5) uint32\_t [\\_pmc0\\_hsrun\\_mode\\_config::\\_\\_pad2\\_\\_](#)

### 30.3.2 struct [\\_pmc0\\_run\\_mode\\_config](#)

#### Data Fields

- uint32\_t [\\_\\_pad0\\_\\_](#): 16  
*Reserved.*
- uint32\_t [coreRegulatorVoltLevel](#): 6  
*Core Regulator Voltage Level.*
- uint32\_t [\\_\\_pad1\\_\\_](#): 10  
*Reserved.*

#### Field Documentation

- (1) uint32\_t [\\_pmc0\\_run\\_mode\\_config::\\_\\_pad0\\_\\_](#)
- (2) uint32\_t [\\_pmc0\\_run\\_mode\\_config::coreRegulatorVoltLevel](#)
- (3) uint32\_t [\\_pmc0\\_run\\_mode\\_config::\\_\\_pad1\\_\\_](#)

### 30.3.3 struct [\\_pmc0\\_vlpr\\_mode\\_config](#)

#### Data Fields

- uint32\_t [arrayRegulatorSelect](#): 1  
*Array Regulator Select.*
- uint32\_t [\\_\\_pad0\\_\\_](#): 1  
*Reserved.*
- uint32\_t [coreRegulatorSelect](#): 1  
*Core Regulator Select.*
- uint32\_t [\\_\\_pad1\\_\\_](#): 1  
*Reserved.*

- uint32\_t [lvdMonitorSelect](#): 1  
*1.2V LVD Monitor Select.*
- uint32\_t [hvdMonitorSelect](#): 1  
*1.2V HVD Monitor Select.*
- uint32\_t [\\_\\_pad2\\_\\_](#): 1  
*Reserved.*
- uint32\_t [enableForceHpBandgap](#): 1  
*Enable force HP band-gap.*
- uint32\_t [\\_\\_pad3\\_\\_](#): 8  
*Reserved.*
- uint32\_t [coreRegulatorVoltLevel](#): 6  
*Core Regulator Voltage Level.*
- uint32\_t [\\_\\_pad4\\_\\_](#): 6  
*Reserved.*
- uint32\_t [enableReverseBackBias](#): 1  
*Enable reverse back bias.*
- uint32\_t [\\_\\_pad5\\_\\_](#): 3  
*Reserved.*

## Field Documentation

(1) uint32\_t [\\_pmc0\\_vlpr\\_mode\\_config::arrayRegulatorSelect](#)

[pmc0\\_array\\_regulator\\_select\\_t](#)

(2) uint32\_t [\\_pmc0\\_vlpr\\_mode\\_config::\\_\\_pad0\\_\\_](#)

(3) uint32\_t [\\_pmc0\\_vlpr\\_mode\\_config::coreRegulatorSelect](#)

[pmc0\\_core\\_regulator\\_select\\_t](#)

(4) uint32\_t [\\_pmc0\\_vlpr\\_mode\\_config::\\_\\_pad1\\_\\_](#)

(5) uint32\_t [\\_pmc0\\_vlpr\\_mode\\_config::lvdMonitorSelect](#)

[pmc0\\_low\\_volt\\_detect\\_monitor\\_select\\_t](#)

(6) uint32\_t [\\_pmc0\\_vlpr\\_mode\\_config::hvdMonitorSelect](#)

[pmc0\\_high\\_volt\\_detect\\_monitor\\_select\\_t](#)



- (7) uint32\_t \_pmc0\_vlpr\_mode\_config::\_\_pad2\_\_
- (8) uint32\_t \_pmc0\_vlpr\_mode\_config::enableForceHpBandgap
- (9) uint32\_t \_pmc0\_vlpr\_mode\_config::\_\_pad3\_\_
- (10) uint32\_t \_pmc0\_vlpr\_mode\_config::coreRegulatorVoltLevel
- (11) uint32\_t \_pmc0\_vlpr\_mode\_config::\_\_pad4\_\_
- (12) uint32\_t \_pmc0\_vlpr\_mode\_config::enableReverseBackBias
- (13) uint32\_t \_pmc0\_vlpr\_mode\_config::\_\_pad5\_\_

### 30.3.4 struct \_pmc0\_stop\_mode\_config

#### Data Fields

- uint32\_t \_\_pad0\_\_: 16  
*Reserved.*
- uint32\_t coreRegulatorVoltLevel: 6  
*Core Regulator Voltage Level.*
- uint32\_t \_\_pad1\_\_: 10  
*Reserved.*

#### Field Documentation

- (1) uint32\_t \_pmc0\_stop\_mode\_config::\_\_pad0\_\_
- (2) uint32\_t \_pmc0\_stop\_mode\_config::coreRegulatorVoltLevel
- (3) uint32\_t \_pmc0\_stop\_mode\_config::\_\_pad1\_\_

### 30.3.5 struct \_pmc0\_vlps\_mode\_config

#### Data Fields

- uint32\_t arrayRegulatorSelect: 1  
*Array Regulator Select.*
- uint32\_t \_\_pad0\_\_: 1  
*Reserved.*
- uint32\_t coreRegulatorSelect: 1  
*Core Regulator Select.*
- uint32\_t \_\_pad1\_\_: 1  
*Reserved.*
- uint32\_t lvdMonitorSelect: 1  
*1.2V LVD Monitor Select.*
- uint32\_t hvdMonitorSelect: 1  
*1.2V HVD Monitor Select.*

- uint32\_t [\\_\\_pad2\\_\\_](#): 1  
*Reserved.*
- uint32\_t [enableForceHpBandgap](#): 1  
*Enable force HP band-gap.*
- uint32\_t [\\_\\_pad3\\_\\_](#): 8  
*Reserved.*
- uint32\_t [coreRegulatorVoltLevel](#): 6  
*Core Regulator Voltage Level.*
- uint32\_t [\\_\\_pad4\\_\\_](#): 6  
*Reserved.*
- uint32\_t [enableReverseBackBias](#): 1  
*Enable reverse back bias.*
- uint32\_t [\\_\\_pad5\\_\\_](#): 3  
*Reserved.*

### Field Documentation

(1) uint32\_t [\\_pmc0\\_vlps\\_mode\\_config::arrayRegulatorSelect](#)

[pmc0\\_array\\_regulator\\_select\\_t](#)

(2) uint32\_t [\\_pmc0\\_vlps\\_mode\\_config::\\_\\_pad0\\_\\_](#)

(3) uint32\_t [\\_pmc0\\_vlps\\_mode\\_config::coreRegulatorSelect](#)

[pmc0\\_core\\_regulator\\_select\\_t](#)

(4) uint32\_t [\\_pmc0\\_vlps\\_mode\\_config::\\_\\_pad1\\_\\_](#)

(5) uint32\_t [\\_pmc0\\_vlps\\_mode\\_config::lvdMonitorSelect](#)

[pmc0\\_low\\_volt\\_detect\\_monitor\\_select\\_t](#)

(6) uint32\_t [\\_pmc0\\_vlps\\_mode\\_config::hvdMonitorSelect](#)

[pmc0\\_high\\_volt\\_detect\\_monitor\\_select\\_t](#)

- (7) uint32\_t \_pmc0\_vlps\_mode\_config::\_\_pad2\_\_
- (8) uint32\_t \_pmc0\_vlps\_mode\_config::enableForceHpBandgap
- (9) uint32\_t \_pmc0\_vlps\_mode\_config::\_\_pad3\_\_
- (10) uint32\_t \_pmc0\_vlps\_mode\_config::coreRegulatorVoltLevel
- (11) uint32\_t \_pmc0\_vlps\_mode\_config::\_\_pad4\_\_
- (12) uint32\_t \_pmc0\_vlps\_mode\_config::enableReverseBackBias
- (13) uint32\_t \_pmc0\_vlps\_mode\_config::\_\_pad5\_\_

### 30.3.6 struct \_pmc0\_lls\_mode\_config

#### Data Fields

- uint32\_t [arrayRegulatorSelect](#): 1  
*Array Regulator Select.*
- uint32\_t [\\_\\_pad0\\_\\_](#): 1  
*Reserved.*
- uint32\_t [coreRegulatorSelect](#): 1  
*Core Regulator Select.*
- uint32\_t [\\_\\_pad1\\_\\_](#): 1  
*Reserved.*
- uint32\_t [lvdMonitorSelect](#): 1  
*1.2V LVD Monitor Select.*
- uint32\_t [hvdMonitorSelect](#): 1  
*1.2V HVD Monitor Select.*
- uint32\_t [\\_\\_pad2\\_\\_](#): 1  
*Reserved.*
- uint32\_t [enableForceHpBandgap](#): 1  
*Enable force HP band-gap.*
- uint32\_t [\\_\\_pad3\\_\\_](#): 8  
*Reserved.*
- uint32\_t [coreRegulatorVoltLevel](#): 6  
*Core Regulator Voltage Level.*
- uint32\_t [\\_\\_pad4\\_\\_](#): 6  
*Reserved.*
- uint32\_t [enableReverseBackBias](#): 1  
*Enable reverse back bias.*
- uint32\_t [\\_\\_pad5\\_\\_](#): 3  
*Reserved.*

#### Field Documentation

- (1) uint32\_t \_pmc0\_lls\_mode\_config::arrayRegulatorSelect

[pmc0\\_array\\_regulator\\_select\\_t](#)

- (2) `uint32_t _pmc0_lls_mode_config::__pad0__`
- (3) `uint32_t _pmc0_lls_mode_config::coreRegulatorSelect`  
[pmc0\\_core\\_regulator\\_select\\_t](#)
- (4) `uint32_t _pmc0_lls_mode_config::__pad1__`
- (5) `uint32_t _pmc0_lls_mode_config::lvdMonitorSelect`  
[pmc0\\_low\\_volt\\_detect\\_monitor\\_select\\_t](#)
- (6) `uint32_t _pmc0_lls_mode_config::hvdMonitorSelect`  
[pmc0\\_high\\_volt\\_detect\\_monitor\\_select\\_t](#)
- (7) `uint32_t _pmc0_lls_mode_config::__pad2__`
- (8) `uint32_t _pmc0_lls_mode_config::enableForceHpBandgap`
- (9) `uint32_t _pmc0_lls_mode_config::__pad3__`
- (10) `uint32_t _pmc0_lls_mode_config::coreRegulatorVoltLevel`
- (11) `uint32_t _pmc0_lls_mode_config::__pad4__`
- (12) `uint32_t _pmc0_lls_mode_config::enableReverseBackBias`
- (13) `uint32_t _pmc0_lls_mode_config::__pad5__`

### 30.3.7 struct `_pmc0_vlls_mode_config`

#### Data Fields

- `uint32_t arrayRegulatorSelect`: 2  
*Array Regulator Select.*
- `uint32_t __pad0__`: 2  
*Reserved.*
- `uint32_t lvdMonitorSelect`: 1  
*1.2V LVD Monitor Select.*
- `uint32_t hvdMonitorSelect`: 1  
*1.2V HVD Monitor Select.*
- `uint32_t __pad1__`: 1  
*Reserved.*
- `uint32_t enableForceHpBandgap`: 1  
*Enable force HP band-gap.*
- `uint32_t __pad2__`: 24  
*Reserved.*

**Field Documentation**

(1) `uint32_t _pmc0_vlls_mode_config::arrayRegulatorSelect`

[pmc0\\_vlls\\_array\\_regulator\\_select\\_t](#)

(2) `uint32_t _pmc0_vlls_mode_config::__pad0__`

(3) `uint32_t _pmc0_vlls_mode_config::lvdMonitorSelect`

[pmc0\\_low\\_volt\\_detect\\_monitor\\_select\\_t](#)

(4) `uint32_t _pmc0_vlls_mode_config::hvdMonitorSelect`

[pmc0\\_high\\_volt\\_detect\\_monitor\\_select\\_t](#)

(5) `uint32_t _pmc0_vlls_mode_config::__pad1__`

(6) `uint32_t _pmc0_vlls_mode_config::enableForceHpBandgap`

(7) `uint32_t _pmc0_vlls_mode_config::__pad2__`

### 30.3.8 struct `_pmc0_bias_config`

**Data Fields**

- `uint32_t __pad0__`: 4  
*Reserved.*
- `uint32_t RBBPWellVoltageLevelSelect`: 4  
*Select PMC0 RBB P-Well voltage level.*
- `uint32_t __pad1__`: 3  
*Reserved.*
- `uint32_t DisableRBBPullDown`: 1  
*Disable RBB pull-down.*
- `uint32_t FBBNWellVoltageLevelSelect`: 4  
*Select PMC0 FBB N-Well voltage level.*
- `uint32_t __pad2__`: 4  
*Reserved.*
- `uint32_t FBBPWellVoltageLevelSelect`: 4  
*Select PMC0 FBB P-Well voltage level.*
- `uint32_t __pad3__`: 4  
*Reserved.*

**Field Documentation**

(1) `uint32_t _pmc0_bias_config::__pad0__`

(2) `uint32_t _pmc0_bias_config::RBBPWellVoltageLevelSelect`

[pmc0\\_rbb\\_p\\_well\\_voltage\\_level\\_select\\_t](#)

(3) `uint32_t _pmc0_bias_config::__pad1__`

(4) `uint32_t _pmc0_bias_config::DisableRBBPullDown`

'1' means to disable pull-down. '0' means to enable pull-down.

(5) `uint32_t _pmc0_bias_config::FBBNWellVoltageLevelSelect`

[pmc0\\_fbb\\_n\\_well\\_voltage\\_level\\_select\\_t](#)

(6) `uint32_t _pmc0_bias_config::__pad2__`

(7) `uint32_t _pmc0_bias_config::FBBPWellVoltageLevelSelect`

[pmc0\\_fbb\\_p\\_well\\_voltage\\_level\\_select\\_t](#)

(8) `uint32_t _pmc0_bias_config::__pad3__`

## 30.4 Enumeration Type Documentation

### 30.4.1 `enum _pmc0_high_volt_detect_monitor_select`

Enumerator

***kPMC0\_HighVoltDetectLowPowerMonitor*** LP monitor is selected.

***kPMC0\_HighVoltDetectHighPowerMonitor*** HP monitor is selected.

### 30.4.2 `enum _pmc0_low_volt_detect_monitor_select`

Enumerator

***kPMC0\_LowVoltDetectLowPowerMonitor*** LP monitor is selected.

***kPMC0\_LowVoltDetectHighPowerMonitor*** HP monitor is selected.

### 30.4.3 `enum _pmc0_core_regulator_select`

Enumerator

***kPMC0\_CoreLowPowerRegulator*** Core LP regulator is selected.

***kPMC0\_CoreHighPowerRegulator*** Core HP regulator is selected.

### 30.4.4 `enum _pmc0_array_regulator_select`

Enumerator

***kPMC0\_ArrayLowPowerRegulator*** Array LP regulator is selected.

***kPMC0\_ArrayHighPowerRegulator*** Array HP regulator is selected.

### 30.4.5 enum \_pmc0\_vlls\_array\_regulator\_select

Enumerator

***kPMC0\_VllsArrayRegulatorOff*** Array regulator is selected OFF. This is selectable only for VLLS mode.

***kPMC0\_VllsArrayLowPowerRegulator*** Array LP regulator is selected.

***kPMC0\_VllsArrayHighPowerRegulator*** Array HP regulator is selected.

### 30.4.6 enum \_pmc0\_fbb\_p\_well\_voltage\_level\_select

Enumerator

***kPMC0\_FbbPWellNoBiasCondition*** No BIAS condition is selected.

***kPMC0\_FbbPWellVoltageLevelAt50Mv*** Voltage level at 50mV is selected.

***kPMC0\_FbbPWellVoltageLevelAt150Mv*** Voltage level at 150mV is selected.

***kPMC0\_FbbPWellVoltageLevelAt100Mv*** Voltage level at 100mV is selected.

***kPMC0\_FbbPWellVoltageLevelAt350Mv*** Voltage level at 350mV is selected.

***kPMC0\_FbbPWellVoltageLevelAt300Mv*** Voltage level at 300mV is selected.

***kPMC0\_FbbPWellVoltageLevelAt200Mv*** Voltage level at 200mV is selected.

***kPMC0\_FbbPWellVoltageLevelAt250Mv*** Voltage level at 250mV is selected.

### 30.4.7 enum \_pmc0\_fbb\_n\_well\_voltage\_level\_select

Enumerator

***kPMC0\_FbbNWellNoBiasCondition*** No BIAS condition is selected.

***kPMC0\_FbbNWellVoltageLevelAtMinus50Mv*** Voltage level at -50mV is selected.

***kPMC0\_FbbNWellVoltageLevelAtMinus150Mv*** Voltage level at -150mV is selected.

***kPMC0\_FbbNWellVoltageLevelAtMinus100Mv*** Voltage level at -100mV is selected.

***kPMC0\_FbbNWellVoltageLevelAtMinus350Mv*** Voltage level at -350mV is selected.

***kPMC0\_FbbNWellVoltageLevelAtMinus300Mv*** Voltage level at -300mV is selected.

***kPMC0\_FbbNWellVoltageLevelAtMinus200Mv*** Voltage level at -200mV is selected.

***kPMC0\_FbbNWellVoltageLevelAtMinus250Mv*** Voltage level at -250mV is selected.

### 30.4.8 enum \_pmc0\_rbb\_p\_well\_voltage\_level\_select

Enumerator

***kPMC0\_RBBPWellVoltageLevelAtMinus0\_5V*** Voltage level at -0.5V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus0\_6V*** Voltage level at -0.6V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus0\_7V*** Voltage level at -0.7V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus0\_8V*** Voltage level at -0.8V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus0\_9V*** Voltage level at -0.9V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus1\_0V*** Voltage level at -1.0V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus1\_1V*** Voltage level at -1.1V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus1\_2V*** Voltage level at -1.2V is selected.  
***kPMC0\_RBBPWellVoltageLevelAtMinus1\_3V*** Voltage level at -1.3V is selected.

### 30.4.9 enum \_pmc0\_rbb\_n\_well\_voltage\_level\_select

Enumerator

***kPMC0\_RBBNWellVoltageLevelAt0\_5V*** Voltage level at 0.5V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt0\_6V*** Voltage level at 0.6V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt0\_7V*** Voltage level at 0.7V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt0\_8V*** Voltage level at 0.8V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt0\_9V*** Voltage level at 0.9V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt1\_0V*** Voltage level at 1.0V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt1\_1V*** Voltage level at 1.1V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt1\_2V*** Voltage level at 1.2V is selected.  
***kPMC0\_RBBNWellVoltageLevelAt1\_3V*** Voltage level at 1.3V is selected.

### 30.4.10 enum \_pmc0\_status\_flags

Enumerator

***kPMC0\_LowVoltDetectEventFlag*** 1.2V Low-Voltage Detector Flag, sets when low-voltage event was detected.  
***kPMC0\_LowVoltDetectValueFlag*** 1.2V Low-Voltage Detector Value, sets when current value of the 1.2V LVD monitor output is 1.  
***kPMC0\_HighVoltDetectEventFlag*** 1.8V High-Voltage Detector Flag, sets when high-voltage event was detected.  
***kPMC0\_HighVoltDetectValueFlag*** 1.8V High-Voltage Detector Value, sets when current value of the 1.8V HVD monitor output is 1.  
***kPMC0\_CoreRegulatorVoltLevelFlag*** Core Regulator Voltage Level Flag, sets when core regulator voltage level is changing (not stable).



***kPMC0\_SramFlag*** SRAM Flag, sets when a change mode request is being processed in the SRAMs.

***kPMC0\_PMC1VoltageSourceFlag*** This flag indicates what is the voltage source selected to supply the PMC 1 and where the sense point of the PMC 1's LVD/HVD is placed. '0' means internal LDO supplies the PMC 1. '1' means external PMIC supplies the PMC 1.

### 30.4.11 enum \_pmc0\_power\_mode\_status\_flags

Enumerator

***kPMC0\_HSRUNModeStatusFlags*** The PMC 0 is in HSRUN mode.

***kPMC0\_RUNModeStatusFlags*** The PMC 0 is in RUN mode.

***kPMC0\_STOPModeStatusFlags*** The PMC 0 is in STOP mode.

***kPMC0\_VLPRModeStatusFlags*** The PMC 0 is in VLPR mode.

***kPMC0\_VLPSModeStatusFlags*** The PMC 0 is in VLPS mode.

***kPMC0\_LLSModeStatusFlags*** The PMC 0 is in LLSM mode.

***kPMC0\_VLLSModeStatusFlags*** The PMC 0 is in VLLS mode.

## 30.5 Function Documentation

### 30.5.1 static void PMC0\_ConfigureHsrunMode ( const pmc0\_hsrun\_mode\_config\_t \* *config* ) [inline], [static]

This function configures the HSRUN power mode, including the core regulator voltage Level setting, enable forward bias or not.

Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Low-Voltage detect configuration structure. |
|---------------|---------------------------------------------|

### 30.5.2 static void PMC0\_ConfigureRunMode ( const pmc0\_run\_mode\_config\_t \* *config* ) [inline], [static]

This function configures the RUN power mode, including the core regulator voltage Level setting.

Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Low-Voltage detect configuration structure. |
|---------------|---------------------------------------------|

### 30.5.3 static void PMC0\_ConfigureVlprMode ( const pmc0\_vlpr\_mode\_config\_t \* *config* ) [inline], [static]

This function configures the VLPR power mode, including the core regulator voltage Level setting, enable reverse back bias or not, turn on force HP band-gap or not, select of HVD/LVD monitor and select of core/array regulator.

Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Low-Voltage detect configuration structure. |
|---------------|---------------------------------------------|

### 30.5.4 static void PMC0\_ConfigureStopMode ( const pmc0\_stop\_mode\_config\_t \* *config* ) [inline], [static]

This function configures the STOP power mode, including the core regulator voltage Level setting.

Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Low-Voltage detect configuration structure. |
|---------------|---------------------------------------------|

### 30.5.5 static void PMC0\_ConfigureVlpsMode ( const pmc0\_vlps\_mode\_config\_t \* *config* ) [inline], [static]

This function configures the VLPS power mode, including the core regulator voltage Level setting, enable reverse back bias or not, turn on force HP band-gap or not, select of HVD/LVD monitor and select of core/array regulator.

Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Low-Voltage detect configuration structure. |
|---------------|---------------------------------------------|

### 30.5.6 static void PMC0\_ConfigureLlsMode ( const pmc0\_lls\_mode\_config\_t \* *config* ) [inline], [static]

This function configures the LLS power mode, including the core regulator voltage Level setting, enable reverse back bias or not, turn on force HP band-gap or not, select of HVD/LVD monitor and select of

core/array regulator.

## Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Low-Voltage detect configuration structure. |
|---------------|---------------------------------------------|

### 30.5.7 static void PMC0\_ConfigureVllsMode ( const pmc0\_vlls\_mode\_config\_t \* *config* ) [inline], [static]

This function configures the VLLS power mode, including turn on force HP band-gap or not, select of HVD/LVD monitor and select of core/array regulator.

The select of array regulator is different from the other mode configurations. PMC 0 VLLS config has three array regulator select options where the others have only the latter two, see [pmc0\\_vlls\\_array\\_regulator\\_select\\_t](#). Three array regulator select options in PMC 0 VLLS config are shown below:

- Regulator is off (differentiator)
- LP Regulator is on
- HP Regulator is on

## Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>config</i> | Low-Voltage detect configuration structure. |
|---------------|---------------------------------------------|

### 30.5.8 static uint32\_t PMC0\_GetPMC0PowerModeStatusFlags ( void ) [inline], [static]

```
*
* if (kPMC0_HSRUNModeStatusFlags ==
* PMC0_GetPMC0PowerModeStatusFlags(void))
* {
* ...
* }
*
```

## Returns

PMC 0 current power mode status flags in the `_pmc0_power_mode_status_flags`.

### 30.5.9 static bool PMC0\_GetPMC0PowerTransitionStatus ( void ) [inline], [static]

## Returns

If return 'true', which means PMC 0 is in a power mode transition. If return 'false', which means PMC 0 is not in a power mode transition.

### 30.5.10 static uint32\_t PMC0\_GetPMC1PowerModeStatusFlags ( void ) [inline], [static]

```
*
* if (kPMC0_HSRUNModeStatusFlags ==
* PMC0_GetPMC1PowerModeStatusFlags(void))
* {
* ...
* }
*
```

Returns

PMC 1 current power mode status flags in the `_pmc0_power_mode_status_flags`.

### 30.5.11 static bool PMC0\_GetPMC1PowerTransitionStatus ( void ) [inline], [static]

Returns

If return 'true', which means PMC 1 is in a power mode transition. If return 'false', which means PMC 1 is not in a power mode transition.

### 30.5.12 static uint32\_t PMC0\_GetStatusFlags ( void ) [inline], [static]

This function gets all PMC 0 status flags. The flags are returned as the logical OR value of the enumerators `_pmc0_status_flags`. To check for a specific status, compare the return value with enumerators in the `_pmc0_status_flags`. For example, to check whether core regulator voltage level is changing:

```
*
* if (kPMC0_CoreRegulatorVoltLevelFlag &
* PMC0_GetStatusFlags(void))
* {
* ...
* }
*
```

Returns

PMC 0 status flags which are ORed by the enumerators in the `_pmc0_status_flags`.

### 30.5.13 static void PMC0\_EnableLowVoltDetectInterrupt ( void ) [inline], [static]

This function enables the 1.2V Low-Voltage Detector interrupt.

**30.5.14 static void PMC0\_DisableLowVoltDetectInterrupt ( void ) [inline],  
[static]**

This function disables the 1.2V Low-Voltage Detector interrupt.

**30.5.15 static void PMC0\_ClearLowVoltDetectFlag ( void ) [inline],  
[static]**

This function enables the 1.2V Low-Voltage Detector flag.

**30.5.16 static void PMC0\_EnableHighVoltDetectInterrupt ( void ) [inline],  
[static]**

This function enables the 1.8V High-Voltage Detector interrupt.

**30.5.17 static void PMC0\_DisableHighVoltDetectInterrupt ( void ) [inline],  
[static]**

This function disables the 1.8V High-Voltage Detector interrupt.

**30.5.18 static void PMC0\_ClearHighVoltDetectFlag ( void ) [inline],  
[static]**

This function enables the 1.8V High-Voltage Detector flag.

**30.5.19 static void PMC0\_EnableLowVoltDetectReset ( bool *enable* ) [inline],  
[static]**

This function enables 1.2V Low-Voltage Detector reset.

Parameters

|               |                                                                                                 |
|---------------|-------------------------------------------------------------------------------------------------|
| <i>enable</i> | Switcher of 1.2V Low-Voltage Detector reset feature. "true" means to enable, "false" means not. |
|---------------|-------------------------------------------------------------------------------------------------|

**30.5.20** `static void PMC0_EnableHighVoltDetectReset ( bool enable ) [inline],  
[static]`

This function enables 1.8V High-Voltage Detector reset.

## Parameters

|               |                                                                                                  |
|---------------|--------------------------------------------------------------------------------------------------|
| <i>enable</i> | Switcher of 1.8V High-Voltage Detector reset feature. "true" means to enable, "false" means not. |
|---------------|--------------------------------------------------------------------------------------------------|

**30.5.21 static void PMC0\_ClearPadsIsolation ( void ) [inline], [static]**

This function releases/clears the isolation in the PADS.

The isolation in the pads only will be asserted during LLS/VLLS. On LLS exit, the isolation will release automatically. ISOACK must be set after a VLLS to RUN mode transition has completed.

**30.5.22 static void PMC0\_PowerOnPmc1 ( void ) [inline], [static]**

This function powers on PMC 1.

When this bit field is asserted the PMC 1 is powered on. This bit would take action only once. This bit will be rearmed after a POR event only. NOTE: USB PHY-related interrupt (NVIC/GIC) and wake-up channels (AWIC/WKPU) must be disabled before turning PMC1 on.

**30.5.23 static void PMC0\_EnableWaitLdoOkSignal ( bool *enable* ) [inline], [static]**

This function enables to wait LDO OK signal.

## Parameters

|               |                                                                                    |
|---------------|------------------------------------------------------------------------------------|
| <i>enable</i> | Switcher of wait LDO OK signal feature. "true" means to enable, "false" means not. |
|---------------|------------------------------------------------------------------------------------|

**30.5.24 static void PMC0\_EnablePmc1LdoRegulator ( bool *enable* ) [inline], [static]**

This function enables PMC 1 LDO Regulator.

## Parameters



|               |                                                                             |
|---------------|-----------------------------------------------------------------------------|
| <i>enable</i> | Switcher of PMC 1 LDO Regulator. "true" means to enable, "false" means not. |
|---------------|-----------------------------------------------------------------------------|

### 30.5.25 static void PMC0\_EnablePmc1RBBMode ( bool *enable* ) [inline], [static]

This function enables PMC1 RBB mode. Since this circuit when enabled has current consumption. It is recommended to use it just in high temperatures when the leakage reduction is much higher than the current consumption.

Parameters

|               |                                                                           |
|---------------|---------------------------------------------------------------------------|
| <i>enable</i> | Switcher of PMC1 RBB mode. "true" means to enable, "false" means disable. |
|---------------|---------------------------------------------------------------------------|

### 30.5.26 static void PMC0\_SetBiasConfig ( const pmc0\_bias\_config\_t \* *config* ) [inline], [static]

This function change the RBB&FBB voltage level and RBB pull-down.

Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>config</i> | PMC 0 bias configuration structure. |
|---------------|-------------------------------------|

### 30.5.27 static void PMC0\_ConfigureSramBankPowerDown ( uint32\_t *bankMask* ) [inline], [static]

This function configures PMC 0 SRAM bank power down.

The bit *i* controls the power mode of the PMC 0 SRAM bank *i*. PMC0\_SRAM\_PD[*i*] = 1'b0 - PMC 0 SRAM bank *i* is not affected. PMC0\_SRAM\_PD[*i*] = 1'b1 - PMC 0 SRAM bank *i* is in ASD or ARRAY\_SHUTDOWN during all modes, except VLLS. During VLLS is in POWER\_DOWN mode.

Example: Enable band 0 and 1 in ASD or ARRAY\_SHUTDOWN during all modes except VLLS

```
* PMC0_ConfigureSramBankPowerDown(0x3U);
*
```

## Parameters

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| <i>bankMask</i> | The bands to enable. Logical OR of all bits of band index to enable. |
|-----------------|----------------------------------------------------------------------|

### 30.5.28 static void PMC0\_ConfigureSramBankPowerDownStopMode ( uint32\_t *bankMask* ) [inline], [static]

This function configures PMC 0 SRAM bank power down in stop modes.

The bit *i* controls the PMC 0 SRAM bank *i*. PMC0\_SRAM\_PDS[*i*] = 1'b0 - PMC 0 SRAM bank *i* is not affected. PMC0\_SRAM\_PDS[*i*] = 1'b1 - PMC 0 SRAM bank *i* is in ASD or ARRAY\_SHUTDOWN mode during STOP, VLPS and LLS modes. During VLLS is in POWER\_DOWN mode.

Example: Enable band 0 and 1 in ASD or ARRAY\_SHUTDOWN during STOP, VLPS and LLS modes

```
* PMC0_ConfigureSramBankPowerDownStopMode(0x3U);
*
```

## Parameters

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| <i>bankMask</i> | The bands to enable. Logical OR of all bits of band index to enable. |
|-----------------|----------------------------------------------------------------------|

### 30.5.29 static void PMC0\_ConfigureSramBankPowerDownStandbyMode ( uint32\_t *bankMask* ) [inline], [static]

This function configures PMC 0 SRAM bank power down in Standby Mode.

The bit *i* controls the PMC 0 SRAM bank *i*. PMC0\_SRAM\_STDY[*i*] = 1'b0 - PMC 0 SRAM bank *i* is not affected. PMC0\_SRAM\_STDY[*i*] = 1'b1 - PMC 0 SRAM bank *i* is in STANDBY mode during all modes (except VLLS and LLS).

Example: Enable band 0 and 1 in STANDBY mode except VLLS and LLS

```
* PMC0_ConfigureSramBankPowerDownStandbyMode(0x3U);
*
```

## Parameters

|                 |                                                                      |
|-----------------|----------------------------------------------------------------------|
| <i>bankMask</i> | The bands to enable. Logical OR of all bits of band index to enable. |
|-----------------|----------------------------------------------------------------------|

### 30.5.30 static void PMC0\_EnableTemperatureSensor ( bool *enable* ) [inline], [static]

## Parameters

|               |                                                                                                                                                                                                                   |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>enable</i> | Used to enable/disable internal temperature sensor. <ul style="list-style-type: none"><li>• <b>true</b> Enable internal temperature sensor.</li><li>• <b>false</b> Disable internal temperature sensor.</li></ul> |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 30.5.31 static void PMCO\_SetTemperatureSensorMode ( uint8\_t *mode* ) [inline], [static]

## Parameters

|             |                                     |
|-------------|-------------------------------------|
| <i>mode</i> | The temperature sensor mode to set. |
|-------------|-------------------------------------|

# Chapter 31

## PORT: Port Control and Interrupts

### 31.1 Overview

The MCUXpresso SDK provides a driver for the Port Control and Interrupts (PORT) module of MCU-Xpresso SDK devices.

#### Typedefs

- typedef enum `_port_interrupt` `port_interrupt_t`  
*Configures the interrupt generation condition.*

#### Enumerations

- enum `_port_interrupt` {  
    `kPORT_InterruptOrDMADisabled` = 0x0U,  
    `kPORT_DMARisingEdge` = 0x1U,  
    `kPORT_DMAFallingEdge` = 0x2U,  
    `kPORT_DMAEitherEdge` = 0x3U,  
    `kPORT_FlagRisingEdge` = 0x05U,  
    `kPORT_FlagFallingEdge` = 0x06U,  
    `kPORT_FlagEitherEdge` = 0x07U,  
    `kPORT_InterruptLogicZero` = 0x8U,  
    `kPORT_InterruptRisingEdge` = 0x9U,  
    `kPORT_InterruptFallingEdge` = 0xAU,  
    `kPORT_InterruptEitherEdge` = 0xBU,  
    `kPORT_InterruptLogicOne` = 0xCU,  
    `kPORT_ActiveHighTriggerOutputEnable` = 0xDU,  
    `kPORT_ActiveLowTriggerOutputEnable` = 0xEU }  
*Configures the interrupt generation condition.*

#### Driver version

- #define `FSL_PORT_DRIVER_VERSION` (`MAKE_VERSION`(2, 5, 0))  
*PORT driver version.*

#### Interrupt

- static void `PORT_SetPinInterruptConfig` (`PORT_Type` \*base, uint32\_t pin, `port_interrupt_t` config)  
*Configures the port pin interrupt/DMA request.*
- static uint32\_t `PORT_GetPinsInterruptFlags` (`PORT_Type` \*base)  
*Reads the whole port status flag.*
- static void `PORT_ClearPinsInterruptFlags` (`PORT_Type` \*base, uint32\_t mask)  
*Clears the multiple pin interrupt status flag.*

## 31.2 Macro Definition Documentation

### 31.2.1 #define FSL\_PORT\_DRIVER\_VERSION (MAKE\_VERSION(2, 5, 0))

## 31.3 Typedef Documentation

### 31.3.1 typedef enum \_port\_interrupt port\_interrupt\_t

## 31.4 Enumeration Type Documentation

### 31.4.1 enum \_port\_interrupt

Enumerator

***kPORT\_InterruptOrDMADisabled*** Interrupt/DMA request is disabled.  
***kPORT\_DMARisingEdge*** DMA request on rising edge.  
***kPORT\_DMAFallingEdge*** DMA request on falling edge.  
***kPORT\_DMAEitherEdge*** DMA request on either edge.  
***kPORT\_FlagRisingEdge*** Flag sets on rising edge.  
***kPORT\_FlagFallingEdge*** Flag sets on falling edge.  
***kPORT\_FlagEitherEdge*** Flag sets on either edge.  
***kPORT\_InterruptLogicZero*** Interrupt when logic zero.  
***kPORT\_InterruptRisingEdge*** Interrupt on rising edge.  
***kPORT\_InterruptFallingEdge*** Interrupt on falling edge.  
***kPORT\_InterruptEitherEdge*** Interrupt on either edge.  
***kPORT\_InterruptLogicOne*** Interrupt when logic one.  
***kPORT\_ActiveHighTriggerOutputEnable*** Enable active high-trigger output.  
***kPORT\_ActiveLowTriggerOutputEnable*** Enable active low-trigger output.

## 31.5 Function Documentation

### 31.5.1 static void PORT\_SetPinInterruptConfig ( PORT\_Type \* *base*, uint32\_t *pin*, port\_interrupt\_t *config* ) [inline], [static]

Parameters

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>   | PORT peripheral base pointer.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <i>pin</i>    | PORT pin number.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <i>config</i> | PORT pin interrupt configuration. <ul style="list-style-type: none"> <li>• <a href="#">kPORT_InterruptOrDMADisabled</a>: Interrupt/DMA request disabled.</li> <li>• <a href="#">kPORT_DMARisingEdge</a>: DMA request on rising edge(if the DMA requests exit).</li> <li>• <a href="#">kPORT_DMAFallingEdge</a>: DMA request on falling edge(if the DMA requests exit).</li> <li>• <a href="#">kPORT_DMAEitherEdge</a>: DMA request on either edge(if the DMA requests exit).</li> <li>• <a href="#">kPORT_FlagRisingEdge</a>: Flag sets on rising edge(if the Flag states exit).</li> <li>• <a href="#">kPORT_FlagFallingEdge</a>: Flag sets on falling edge(if the Flag states exit).</li> <li>• <a href="#">kPORT_FlagEitherEdge</a>: Flag sets on either edge(if the Flag states exit).</li> <li>• <a href="#">kPORT_InterruptLogicZero</a>: Interrupt when logic zero.</li> <li>• <a href="#">kPORT_InterruptRisingEdge</a>: Interrupt on rising edge.</li> <li>• <a href="#">kPORT_InterruptFallingEdge</a>: Interrupt on falling edge.</li> <li>• <a href="#">kPORT_InterruptEitherEdge</a>: Interrupt on either edge.</li> <li>• <a href="#">kPORT_InterruptLogicOne</a>: Interrupt when logic one.</li> <li>• <a href="#">kPORT_ActiveHighTriggerOutputEnable</a>: Enable active high-trigger output (if the trigger states exit).</li> <li>• <a href="#">kPORT_ActiveLowTriggerOutputEnable</a>: Enable active low-trigger output (if the trigger states exit).</li> </ul> |

### 31.5.2 static uint32\_t PORT\_GetPinsInterruptFlags ( PORT\_Type \* *base* ) [inline], [static]

If a pin is configured to generate the DMA request, the corresponding flag is cleared automatically at the completion of the requested DMA transfer. Otherwise, the flag remains set until a logic one is written to that flag. If configured for a level sensitive interrupt that remains asserted, the flag is set again immediately.

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | PORT peripheral base pointer. |
|-------------|-------------------------------|

## Returns

Current port interrupt status flags, for example, 0x00010001 means the pin 0 and 16 have the interrupt.

### 31.5.3 static void PORT\_ClearPinsInterruptFlags ( PORT\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                               |
|-------------|-------------------------------|
| <i>base</i> | PORT peripheral base pointer. |
| <i>mask</i> | PORT pin number macro.        |

## Chapter 32

# QSPI: Quad Serial Peripheral Interface

### 32.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Quad Serial Peripheral Interface (QSPI) module of MCUXpresso SDK devices.

QSPI driver includes functional APIs and EDMA transactional APIs.

Functional APIs are feature/property target low level APIs. Functional APIs can be used for QSPI initialization/configuration/operation for optimization/customization purpose. Using the functional API requires the knowledge of the QSPI peripheral and how to organize functional APIs to meet the application requirements. All functional API use the peripheral base address as the first parameter. QSPI functional operation groups provide the functional API set.

Transactional APIs are transaction target high level APIs. Transactional APIs can be used to enable the peripheral and in the application if the code size and performance of transactional APIs satisfy the requirements. If the code size and performance are a critical requirement, see the transactional API implementation and write a custom code. All transactional APIs use the `qspi_handle_t` as the first parameter. Initialize the handle by calling the [QSPI\\_TransferTxCreateHandleEDMA\(\)](#) or [QSPI\\_Transfer-RxCreateHandleEDMA\(\)](#) API.

### Modules

- [Quad Serial Peripheral Interface Driver](#)



## 32.2 Quad Serial Peripheral Interface Driver

### 32.2.1 Overview

#### Data Structures

- struct [QspiDQSConfig](#)  
*DQS configure features. [More...](#)*
- struct [QspiFlashTiming](#)  
*Flash timing configuration. [More...](#)*
- struct [QspiConfig](#)  
*QSPI configuration structure. [More...](#)*
- struct [\\_qspi\\_flash\\_config](#)  
*External flash configuration items. [More...](#)*
- struct [\\_qspi\\_transfer](#)  
*Transfer structure for QSPI. [More...](#)*
- struct [\\_ip\\_command\\_config](#)  
*16-bit access reg for IPCR register [More...](#)*

#### Macros

- #define [QSPI\\_LUT\\_SEQ](#)(cmd0, pad0, op0, cmd1, pad1, op1)  
*Macro functions for LUT table.*
- #define [QSPI\\_CMD](#) (0x1U)  
*Macro for QSPI LUT command.*
- #define [QSPI\\_PAD\\_1](#) (0x0U)  
*Macro for QSPI PAD.*

#### Typedefs

- typedef enum [\\_qspi\\_read\\_area](#) [qspi\\_read\\_area\\_t](#)  
*QSPI read data area, from IP FIFO or AHB buffer.*
- typedef enum [\\_qspi\\_command\\_seq](#) [qspi\\_command\\_seq\\_t](#)  
*QSPI command sequence type.*
- typedef enum [\\_qspi\\_fifo](#) [qspi\\_fifo\\_t](#)  
*QSPI buffer type.*
- typedef enum [\\_qspi\\_endianness](#) [qspi\\_endianness\\_t](#)  
*QSPI transfer endianness.*
- typedef enum [\\_qspi\\_dqs\\_phrase\\_shift](#) [qspi\\_dqs\\_phrase\\_shift\\_t](#)  
*Phrase shift number for DQS mode.*
- typedef enum [\\_qspi\\_dqs\\_read\\_sample\\_clock](#) [qspi\\_dqs\\_read\\_sample\\_clock\\_t](#)  
*Qspi read sampling option.*
- typedef struct [QspiDQSConfig](#) [qspi\\_dqs\\_config\\_t](#)  
*DQS configure features.*
- typedef struct [QspiFlashTiming](#) [qspi\\_flash\\_timing\\_t](#)  
*Flash timing configuration.*
- typedef struct [QspiConfig](#) [qspi\\_config\\_t](#)

- *QSPI configuration structure.*  
typedef struct `_qspi_flash_config` `qspi_flash_config_t`
- *External flash configuration items.*  
typedef struct `_qspi_transfer` `qspi_transfer_t`
- *Transfer structure for QSPI.*  
typedef struct `_ip_command_config` `ip_command_config_t`
- *16-bit access reg for IPCR register*

## Enumerations

- enum {  
    `kStatus_QSPI_Idle` = MAKE\_STATUS(kStatusGroup\_QSPI, 0),  
    `kStatus_QSPI_Busy` = MAKE\_STATUS(kStatusGroup\_QSPI, 1),  
    `kStatus_QSPI_Error` = MAKE\_STATUS(kStatusGroup\_QSPI, 2) }  
    *Status structure of QSPI.*
- enum `_qspi_read_area` {  
    `kQSPI_ReadAHB` = 0x0U,  
    `kQSPI_ReadIP` }  
    *QSPI read data area, from IP FIFO or AHB buffer.*
- enum `_qspi_command_seq` {  
    `kQSPI_IPSeq` = QuadSPI\_SPTRCLR\_IPPTRC\_MASK,  
    `kQSPI_BufferSeq` = QuadSPI\_SPTRCLR\_BFPTRC\_MASK }  
    *QSPI command sequence type.*
- enum `_qspi_fifo` {  
    `kQSPI_TxFifo` = QuadSPI\_MCR\_CLR\_TXF\_MASK,  
    `kQSPI_RxFifo` = QuadSPI\_MCR\_CLR\_RXF\_MASK,  
    `kQSPI_AllFifo` = QuadSPI\_MCR\_CLR\_TXF\_MASK | QuadSPI\_MCR\_CLR\_RXF\_MASK }  
    *QSPI buffer type.*
- enum `_qspi_endianness` {  
    `kQSPI_64BigEndian` = 0x0U,  
    `kQSPI_32LittleEndian`,  
    `kQSPI_32BigEndian`,  
    `kQSPI_64LittleEndian` }  
    *QSPI transfer endianness.*
- enum `_qspi_error_flags` {

```

kQSPI_DataLearningFail = (int)QuadSPI_FR_DLPFF_MASK,
kQSPI_TxBufferFill = QuadSPI_FR_TBFF_MASK,
kQSPI_TxBufferUnderrun = QuadSPI_FR_TBUF_MASK,
kQSPI_IllegalInstruction = QuadSPI_FR_ILLINE_MASK,
kQSPI_RxBufferOverflow = QuadSPI_FR_RBOF_MASK,
kQSPI_RxBufferDrain = QuadSPI_FR_RBDF_MASK,
kQSPI_AHBSequenceError = QuadSPI_FR_ABSEF_MASK,
kQSPI_AHBIllegalTransaction = QuadSPI_FR_AITEF_MASK,
kQSPI_AHBIllegalBurstSize = QuadSPI_FR_AIBSEF_MASK,
kQSPI_AHBBufferOverflow = QuadSPI_FR_ABOF_MASK,
kQSPI_IPCommandTriggerDuringAHBAccess = QuadSPI_FR_IPAEF_MASK,
kQSPI_IPCommandTriggerDuringIPAccess = QuadSPI_FR_IPIEF_MASK,
kQSPI_IPCommandTriggerDuringAHBGrant = QuadSPI_FR_IPGEF_MASK,
kQSPI_IPCommandTransactionFinished = QuadSPI_FR_TFF_MASK,
kQSPI_FlagAll = (int)0x8C83F8D1U }

```

*QSPI error flags.*

- enum `_qspi_flags` {

```

kQSPI_DataLearningSamplePoint = (int)QuadSPI_SR_DLPSMP_MASK,
kQSPI_TxBufferFull = QuadSPI_SR_TXFULL_MASK,
kQSPI_TxDMA = QuadSPI_SR_TXDMA_MASK,
kQSPI_TxWatermark = QuadSPI_SR_TXWA_MASK,
kQSPI_TxBufferEnoughData = QuadSPI_SR_TXEDA_MASK,
kQSPI_RxDMA = QuadSPI_SR_RXDMA_MASK,
kQSPI_RxBufferFull = QuadSPI_SR_RXFULL_MASK,
kQSPI_RxWatermark = QuadSPI_SR_RXWE_MASK,
kQSPI_AHB3BufferFull = QuadSPI_SR_AHB3FUL_MASK,
kQSPI_AHB2BufferFull = QuadSPI_SR_AHB2FUL_MASK,
kQSPI_AHB1BufferFull = QuadSPI_SR_AHB1FUL_MASK,
kQSPI_AHB0BufferFull = QuadSPI_SR_AHB0FUL_MASK,
kQSPI_AHB3BufferNotEmpty = QuadSPI_SR_AHB3NE_MASK,
kQSPI_AHB2BufferNotEmpty = QuadSPI_SR_AHB2NE_MASK,
kQSPI_AHB1BufferNotEmpty = QuadSPI_SR_AHB1NE_MASK,
kQSPI_AHB0BufferNotEmpty = QuadSPI_SR_AHB0NE_MASK,
kQSPI_AHBTransactionPending = QuadSPI_SR_AHBTRN_MASK,
kQSPI_AHBCommandPriorityGranted = QuadSPI_SR_AHBGNT_MASK,
kQSPI_AHBAccess = QuadSPI_SR_AHB_ACC_MASK,
kQSPI_IPAccess = QuadSPI_SR_IP_ACC_MASK,
kQSPI_Busy = QuadSPI_SR_BUSY_MASK,
kQSPI_StateAll = (int)0xEF897FE7U }

```

*QSPI state bit.*

- enum `_qspi_interrupt_enable` {

```

kQSPI_DataLearningFailInterruptEnable,
kQSPI_TxBufferFillInterruptEnable = QuadSPI_RSER_TBFIE_MASK,
kQSPI_TxBufferUnderrunInterruptEnable = QuadSPI_RSER_TBUIE_MASK,
kQSPI_IllegalInstructionInterruptEnable,
kQSPI_RxBufferOverflowInterruptEnable = QuadSPI_RSER_RBOIE_MASK,
kQSPI_RxBufferDrainInterruptEnable = QuadSPI_RSER_RBDIE_MASK,
kQSPI_AHBSequenceErrorInterruptEnable = QuadSPI_RSER_ABSEIE_MASK,
kQSPI_AHBIllegalTransactionInterruptEnable,
kQSPI_AHBIllegalBurstSizeInterruptEnable,
kQSPI_AHBBufferOverflowInterruptEnable = QuadSPI_RSER_ABOIE_MASK,
kQSPI_IPCommandTriggerDuringAHBAccessInterruptEnable,
kQSPI_IPCommandTriggerDuringIPAccessInterruptEnable,
kQSPI_IPCommandTriggerDuringAHBGrantInterruptEnable,
kQSPI_IPCommandTransactionFinishedInterruptEnable,
kQSPI_AllInterruptEnable = (int)0x8C83F8D1U }

 QSPI interrupt enable.
• enum _qspi_dma_enable {
 kQSPI_TxBufferFillDMAEnable = QuadSPI_RSER_TBFDE_MASK,
 kQSPI_RxBufferDrainDMAEnable = QuadSPI_RSER_RBDDE_MASK,
 kQSPI_AllDDMAEnable = QuadSPI_RSER_TBFDE_MASK | QuadSPI_RSER_RBDDE_MASK
}

 QSPI DMA request flag.
• enum _qspi_dqs_phrase_shift {
 kQSPI_DQSNoPhraseShift = 0x0U,
 kQSPI_DQSPhraseShift45Degree,
 kQSPI_DQSPhraseShift90Degree,
 kQSPI_DQSPhraseShift135Degree }

 Phrase shift number for DQS mode.
• enum _qspi_dqs_read_sample_clock {
 kQSPI_ReadSampleClkInternalLoopback = 0x0U,
 kQSPI_ReadSampleClkLoopbackFromDqsPad = 0x1U,
 kQSPI_ReadSampleClkExternalInputFromDqsPad = 0x2U }

 Qspi read sampling option.

```

## Driver version

- #define `FSL_QSPI_DRIVER_VERSION` (`MAKE_VERSION(2, 2, 5)`)  
*QSPI driver version 2.2.3.*

## Initialization and deinitialization

- uint32\_t `QSPI_GetInstance` (QuadSPI\_Type \*base)  
*Get the instance number for QSPI.*
- void `QSPI_Init` (QuadSPI\_Type \*base, `qspi_config_t` \*config, uint32\_t srcClock\_Hz)  
*Initializes the QSPI module and internal state.*

- void [QSPI\\_GetDefaultQspiConfig](#) ([qspi\\_config\\_t](#) \*config)  
*Gets default settings for QSPI.*
- void [QSPI\\_Deinit](#) ([QuadSPI\\_Type](#) \*base)  
*Deinitializes the QSPI module.*
- void [QSPI\\_SetFlashConfig](#) ([QuadSPI\\_Type](#) \*base, [qspi\\_flash\\_config\\_t](#) \*config)  
*Configures the serial flash parameter.*
- void [QSPI\\_SetDqsConfig](#) ([QuadSPI\\_Type](#) \*base, [qspi\\_dqs\\_config\\_t](#) \*config)  
*Configures the serial flash DQS parameter.*
- void [QSPI\\_SoftwareReset](#) ([QuadSPI\\_Type](#) \*base)  
*Software reset for the QSPI logic.*
- static void [QSPI\\_Enable](#) ([QuadSPI\\_Type](#) \*base, bool enable)  
*Enables or disables the QSPI module.*

## Status

- static uint32\_t [QSPI\\_GetStatusFlags](#) ([QuadSPI\\_Type](#) \*base)  
*Gets the state value of QSPI.*
- static uint32\_t [QSPI\\_GetErrorStatusFlags](#) ([QuadSPI\\_Type](#) \*base)  
*Gets QSPI error status flags.*
- static void [QSPI\\_ClearErrorFlag](#) ([QuadSPI\\_Type](#) \*base, uint32\_t mask)  
*Clears the QSPI error flags.*

## Interrupts

- static void [QSPI\\_EnableInterrupts](#) ([QuadSPI\\_Type](#) \*base, uint32\_t mask)  
*Enables the QSPI interrupts.*
- static void [QSPI\\_DisableInterrupts](#) ([QuadSPI\\_Type](#) \*base, uint32\_t mask)  
*Disables the QSPI interrupts.*

## DMA Control

- static void [QSPI\\_EnableDMA](#) ([QuadSPI\\_Type](#) \*base, uint32\_t mask, bool enable)  
*Enables the QSPI DMA source.*
- static uint32\_t [QSPI\\_GetTxDataRegisterAddress](#) ([QuadSPI\\_Type](#) \*base)  
*Gets the Tx data register address.*
- uint32\_t [QSPI\\_GetRxDataRegisterAddress](#) ([QuadSPI\\_Type](#) \*base)  
*Gets the Rx data register address used for DMA operation.*

## Bus Operations

- static void [QSPI\\_SetIPCommandAddress](#) ([QuadSPI\\_Type](#) \*base, uint32\_t addr)  
*Sets the IP command address.*
- static void [QSPI\\_SetIPCommandSize](#) ([QuadSPI\\_Type](#) \*base, uint32\_t size)  
*Sets the IP command size.*
- void [QSPI\\_ExecuteIPCommand](#) ([QuadSPI\\_Type](#) \*base, uint32\_t index)  
*Executes IP commands located in LUT table.*

- void [QSPI\\_ExecuteAHBCommand](#) (QuadSPI\_Type \*base, uint32\_t index)  
*Executes AHB commands located in LUT table.*
- void [QSPI\\_UpdateLUT](#) (QuadSPI\_Type \*base, uint32\_t index, uint32\_t \*cmd)  
*Updates the LUT table.*
- static void [QSPI\\_ClearFifo](#) (QuadSPI\_Type \*base, uint32\_t mask)  
*Clears the QSPI FIFO logic.*
- static void [QSPI\\_ClearCommandSequence](#) (QuadSPI\_Type \*base, [qspi\\_command\\_seq\\_t](#) seq)  
*@ brief Clears the command sequence for the IP/buffer command.*
- static void [QSPI\\_EnableDDRMMode](#) (QuadSPI\_Type \*base, bool enable)  
*Enable or disable DDR mode.*
- void [QSPI\\_SetReadDataArea](#) (QuadSPI\_Type \*base, [qspi\\_read\\_area\\_t](#) area)  
*@ brief Set the RX buffer readout area.*
- void [QSPI\\_WriteBlocking](#) (QuadSPI\_Type \*base, const uint32\_t \*buffer, size\_t size)  
*Sends a buffer of data bytes using a blocking method.*
- static void [QSPI\\_WriteData](#) (QuadSPI\_Type \*base, uint32\_t data)  
*Writes data into FIFO.*
- void [QSPI\\_ReadBlocking](#) (QuadSPI\_Type \*base, uint32\_t \*buffer, size\_t size)  
*Receives a buffer of data bytes using a blocking method.*
- uint32\_t [QSPI\\_ReadData](#) (QuadSPI\_Type \*base)  
*Receives data from data FIFO.*

## Transactional

- static void [QSPI\\_TransferSendBlocking](#) (QuadSPI\_Type \*base, [qspi\\_transfer\\_t](#) \*xfer)  
*Writes data to the QSPI transmit buffer.*
- static void [QSPI\\_TransferReceiveBlocking](#) (QuadSPI\_Type \*base, [qspi\\_transfer\\_t](#) \*xfer)  
*Reads data from the QSPI receive buffer in polling way.*

## 32.2.2 Data Structure Documentation

### 32.2.2.1 struct QspiDQSConfig

#### Data Fields

- uint32\_t [portADelayTapNum](#)  
*Delay chain tap number selection for QSPI port A DQS.*
- [qspi\\_dqs\\_phrase\\_shift\\_t](#) shift  
*Phase shift for internal DQS generation.*
- [qspi\\_dqs\\_read\\_sample\\_clock\\_t](#) rxSampleClock  
*Read sample clock for Dqs.*
- bool [enableDQSClkInverse](#)  
*Enable inverse clock for internal DQS generation.*

## Field Documentation

(1) `qspi_dqs_read_sample_clock_t QspiDQSConfig::rxSampleClock`32.2.2.2 struct `QspiFlashTiming`

## Data Fields

- `uint32_t dataHoldTime`  
*Serial flash data in hold time.*
- `uint32_t CSHoldTime`  
*Serial flash CS hold time in terms of serial flash clock cycles.*
- `uint32_t CSSetupTime`  
*Serial flash CS setup time in terms of serial flash clock cycles.*

32.2.2.3 struct `QspiConfig`

## Data Fields

- `uint32_t clockSource`  
*Clock source for QSPI module.*
- `uint32_t baudRate`  
*Serial flash clock baud rate.*
- `uint8_t txWatermark`  
*QSPI transmit watermark value.*
- `uint8_t rxWatermark`  
*QSPI receive watermark value.*
- `uint32_t AHBbufferSize` [FSL\_FEATURE\_QSPI\_AHB\_BUFFER\_COUNT]  
*AHB buffer size.*
- `uint8_t AHBbufferMaster` [FSL\_FEATURE\_QSPI\_AHB\_BUFFER\_COUNT]  
*AHB buffer master.*
- `bool enableAHBbuffer3AllMaster`  
*Is AHB buffer3 for all master.*
- `qspi_read_area_t area`  
*Which area Rx data readout.*
- `bool enableQspi`  
*Enable QSPI after initialization.*

## Field Documentation

- (1) `uint8_t QspiConfig::rxWatermark`
- (2) `uint32_t QspiConfig::AHBbufferSize[FSL_FEATURE_QSPI_AHB_BUFFER_COUNT]`
- (3) `uint8_t QspiConfig::AHBbufferMaster[FSL_FEATURE_QSPI_AHB_BUFFER_COUNT]`
- (4) `bool QspiConfig::enableAHBbuffer3AllMaster`

32.2.2.4 `struct _qspi_flash_config`

## Data Fields

- `uint32_t flashA1Size`  
*Flash A1 size.*
- `uint32_t flashA2Size`  
*Flash A2 size.*
- `uint32_t lookuptable [FSL_FEATURE_QSPI_LUT_DEPTH]`  
*Flash command in LUT.*
- `uint32_t dataHoldTime`  
*Data line hold time.*
- `uint32_t CSHoldTime`  
*CS line hold time.*
- `uint32_t CSSetupTime`  
*CS line setup time.*
- `uint32_t cloumnspace`  
*Column space size.*
- `uint32_t dataLearnValue`  
*Data Learn value if enable data learn.*
- `qspi_endianness_t endian`  
*Flash data endianness.*
- `bool enableWordAddress`  
*If enable word address.*

## Field Documentation

- (1) `uint32_t _qspi_flash_config::dataHoldTime`
- (2) `qspi_endianness_t _qspi_flash_config::endian`
- (3) `bool _qspi_flash_config::enableWordAddress`

32.2.2.5 `struct _qspi_transfer`

## Data Fields

- `uint32_t * data`  
*Pointer to data to transmit.*
- `size_t dataSize`  
*Bytes to be transmit.*



**32.2.2.6 struct \_ip\_command\_config****32.2.3 Macro Definition Documentation****32.2.3.1 #define FSL\_QSPI\_DRIVER\_VERSION (MAKE\_VERSION(2, 2, 5))****32.2.4 Typedef Documentation****32.2.4.1 typedef enum \_qspi\_read\_area qspi\_read\_area\_t****32.2.4.2 typedef enum \_qspi\_dqs\_phrase\_shift qspi\_dqs\_phrase\_shift\_t****32.2.4.3 typedef enum \_qspi\_dqs\_read\_sample\_clock qspi\_dqs\_read\_sample\_clock\_t****32.2.4.4 typedef struct QspiFlashTiming qspi\_flash\_timing\_t****32.2.5 Enumeration Type Documentation****32.2.5.1 anonymous enum**

Enumerator

*kStatus\_QSPI\_Idle* QSPI is in idle state.

*kStatus\_QSPI\_Busy* QSPI is busy.

*kStatus\_QSPI\_Error* Error occurred during QSPI transfer.

**32.2.5.2 enum \_qspi\_read\_area**

Enumerator

*kQSPI\_ReadAHB* QSPI read from AHB buffer.

*kQSPI\_ReadIP* QSPI read from IP FIFO.

**32.2.5.3 enum \_qspi\_command\_seq**

Enumerator

*kQSPI\_IPSeq* IP command sequence.

*kQSPI\_BufferSeq* Buffer command sequence.

### 32.2.5.4 enum \_qspi\_fifo

Enumerator

*kQSPI\_TxFifo* QSPI Tx FIFO.  
*kQSPI\_RxFifo* QSPI Rx FIFO.  
*kQSPI\_AllFifo* QSPI all FIFO, including Tx and Rx.

### 32.2.5.5 enum \_qspi\_endianness

Enumerator

*kQSPI\_64BigEndian* 64 bits big endian  
*kQSPI\_32LittleEndian* 32 bit little endian  
*kQSPI\_32BigEndian* 32 bit big endian  
*kQSPI\_64LittleEndian* 64 bit little endian

### 32.2.5.6 enum \_qspi\_error\_flags

Enumerator

*kQSPI\_DataLearningFail* Data learning pattern failure flag.  
*kQSPI\_TxBufferFill* Tx buffer fill flag.  
*kQSPI\_TxBufferUnderrun* Tx buffer underrun flag.  
*kQSPI\_IllegalInstruction* Illegal instruction error flag.  
*kQSPI\_RxBufferOverflow* Rx buffer overflow flag.  
*kQSPI\_RxBufferDrain* Rx buffer drain flag.  
*kQSPI\_AHBSequenceError* AHB sequence error flag.  
*kQSPI\_AHBIllegalTransaction* AHB illegal transaction error flag.  
*kQSPI\_AHBIllegalBurstSize* AHB illegal burst error flag.  
*kQSPI\_AHBBufferOverflow* AHB buffer overflow flag.  
*kQSPI\_IPCommandTriggerDuringAHBAccess* IP command trigger during AHB access error.  
*kQSPI\_IPCommandTriggerDuringIPAccess* IP command trigger cannot be executed.  
*kQSPI\_IPCommandTriggerDuringAHBGrant* IP command trigger during AHB grant error.  
*kQSPI\_IPCommandTransactionFinished* IP command transaction finished flag.  
*kQSPI\_FlagAll* All error flag.

### 32.2.5.7 enum \_qspi\_flags

Enumerator

*kQSPI\_DataLearningSamplePoint* Data learning sample point.  
*kQSPI\_TxBufferFull* Tx buffer full flag.

***kQSPI\_TxDMA*** Tx DMA is requested or running.  
***kQSPI\_TxWatermark*** Tx buffer watermark available.  
***kQSPI\_TxBufferEnoughData*** Tx buffer enough data available.  
***kQSPI\_RxDMA*** Rx DMA is requesting or running.  
***kQSPI\_RxBufferFull*** Rx buffer full.  
***kQSPI\_RxWatermark*** Rx buffer watermark exceeded.  
***kQSPI\_AHB3BufferFull*** AHB buffer 3 full.  
***kQSPI\_AHB2BufferFull*** AHB buffer 2 full.  
***kQSPI\_AHB1BufferFull*** AHB buffer 1 full.  
***kQSPI\_AHB0BufferFull*** AHB buffer 0 full.  
***kQSPI\_AHB3BufferNotEmpty*** AHB buffer 3 not empty.  
***kQSPI\_AHB2BufferNotEmpty*** AHB buffer 2 not empty.  
***kQSPI\_AHB1BufferNotEmpty*** AHB buffer 1 not empty.  
***kQSPI\_AHB0BufferNotEmpty*** AHB buffer 0 not empty.  
***kQSPI\_AHBTransactionPending*** AHB access transaction pending.  
***kQSPI\_AHBCommandPriorityGranted*** AHB command priority granted.  
***kQSPI\_AHBAccess*** AHB access.  
***kQSPI\_IPAccess*** IP access.  
***kQSPI\_Busy*** Module busy.  
***kQSPI\_StateAll*** All flags.

### 32.2.5.8 enum \_qspi\_interrupt\_enable

Enumerator

***kQSPI\_DataLearningFailInterruptEnable*** Data learning pattern failure interrupt enable.  
***kQSPI\_TxBufferFillInterruptEnable*** Tx buffer fill interrupt enable.  
***kQSPI\_TxBufferUnderrunInterruptEnable*** Tx buffer underrun interrupt enable.  
***kQSPI\_IllegalInstructionInterruptEnable*** Illegal instruction error interrupt enable.  
***kQSPI\_RxBufferOverflowInterruptEnable*** Rx buffer overflow interrupt enable.  
***kQSPI\_RxBufferDrainInterruptEnable*** Rx buffer drain interrupt enable.  
***kQSPI\_AHBSequenceErrorInterruptEnable*** AHB sequence error interrupt enable.  
***kQSPI\_AHBIllegalTransactionInterruptEnable*** AHB illegal transaction error interrupt enable.  
***kQSPI\_AHBIllegalBurstSizeInterruptEnable*** AHB illegal burst error interrupt enable.  
***kQSPI\_AHBBufferOverflowInterruptEnable*** AHB buffer overflow interrupt enable.  
***kQSPI\_IPCommandTriggerDuringAHBAccessInterruptEnable*** IP command trigger during AHB access error.  
***kQSPI\_IPCommandTriggerDuringIPAccessInterruptEnable*** IP command trigger cannot be executed.  
***kQSPI\_IPCommandTriggerDuringAHBGrantInterruptEnable*** IP command trigger during AHB grant error.  
***kQSPI\_IPCommandTransactionFinishedInterruptEnable*** IP command transaction finished interrupt enable.  
***kQSPI\_AllInterruptEnable*** All error interrupt enable.

### 32.2.5.9 enum \_qspi\_dma\_enable

Enumerator

*kQSPI\_TxBufferFillDMAEnable* Tx buffer fill DMA.  
*kQSPI\_RxBufferDrainDMAEnable* Rx buffer drain DMA.  
*kQSPI\_AllDDMAEnable* All DMA source.

### 32.2.5.10 enum \_qspi\_dqs\_phrase\_shift

Enumerator

*kQSPI\_DQSNoPhraseShift* No phase shift.  
*kQSPI\_DQSPhraseShift45Degree* Select 45 degree phase shift.  
*kQSPI\_DQSPhraseShift90Degree* Select 90 degree phase shift.  
*kQSPI\_DQSPhraseShift135Degree* Select 135 degree phase shift.

### 32.2.5.11 enum \_qspi\_dqs\_read\_sample\_clock

Enumerator

*kQSPI\_ReadSampleClkInternalLoopback* Read sample clock adopts internal loopback mode.  
*kQSPI\_ReadSampleClkLoopbackFromDqsPad* Dummy Read strobe generated by QSPI Controller and loopback from DQS pad.  
*kQSPI\_ReadSampleClkExternalInputFromDqsPad* Flash provided Read strobe and input from D-QS pad.

## 32.2.6 Function Documentation

### 32.2.6.1 uint32\_t QSPI\_GetInstance ( QuadSPI\_Type \* *base* )

Parameters

|             |                    |
|-------------|--------------------|
| <i>base</i> | QSPI base pointer. |
|-------------|--------------------|

### 32.2.6.2 void QSPI\_Init ( QuadSPI\_Type \* *base*, qspi\_config\_t \* *config*, uint32\_t *srcClock\_Hz* )

This function enables the clock for QSPI and also configures the QSPI with the input configure parameters. Users should call this function before any QSPI operations.

## Parameters

|                    |                                    |
|--------------------|------------------------------------|
| <i>base</i>        | Pointer to QuadSPI Type.           |
| <i>config</i>      | QSPI configure structure.          |
| <i>srcClock_Hz</i> | QSPI source clock frequency in Hz. |

**32.2.6.3 void QSPI\_GetDefaultQspiConfig ( qspi\_config\_t \* *config* )**

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>config</i> | QSPI configuration structure. |
|---------------|-------------------------------|

**32.2.6.4 void QSPI\_Deinit ( QuadSPI\_Type \* *base* )**

Clears the QSPI state and QSPI module registers.

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
|-------------|--------------------------|

**32.2.6.5 void QSPI\_SetFlashConfig ( QuadSPI\_Type \* *base*, qspi\_flash\_config\_t \* *config* )**

This function configures the serial flash relevant parameters, such as the size, command, and so on. The flash configuration value cannot have a default value. The user needs to configure it according to the QSPI features.

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.        |
| <i>config</i> | Flash configuration parameters. |

**32.2.6.6 void QSPI\_SetDqsConfig ( QuadSPI\_Type \* *base*, qspi\_dqs\_config\_t \* *config* )**

This function configures the serial flash DQS relevant parameters, such as the delay chain tap number, . DQS shift phase, whether need to inverse and the rxc sample clock selection.

## Parameters

|               |                               |
|---------------|-------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.      |
| <i>config</i> | Dqs configuration parameters. |

**32.2.6.7 void QSPI\_SoftwareReset ( QuadSPI\_Type \* *base* )**

This function sets the software reset flags for both AHB and buffer domain and resets both AHB buffer and also IP FIFOs.

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
|-------------|--------------------------|

**32.2.6.8 static void QSPI\_Enable ( QuadSPI\_Type \* *base*, bool *enable* ) [inline], [static]**

## Parameters

|               |                                              |
|---------------|----------------------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.                     |
| <i>enable</i> | True means enable QSPI, false means disable. |

**32.2.6.9 static uint32\_t QSPI\_GetStatusFlags ( QuadSPI\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
|-------------|--------------------------|

## Returns

status flag, use status flag to AND [\\_qspi\\_flags](#) could get the related status.

**32.2.6.10 static uint32\_t QSPI\_GetErrorStatusFlags ( QuadSPI\_Type \* *base* ) [inline], [static]**

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
|-------------|--------------------------|

## Returns

status flag, use status flag to AND [\\_qspi\\_error\\_flags](#) could get the related status.

**32.2.6.11 static void QSPI\_ClearErrorFlag ( QuadSPI\_Type \* *base*, uint32\_t *mask* )  
[inline], [static]**

## Parameters

|             |                                                                                              |
|-------------|----------------------------------------------------------------------------------------------|
| <i>base</i> | Pointer to QuadSPI Type.                                                                     |
| <i>mask</i> | Which kind of QSPI flags to be cleared, a combination of <a href="#">_qspi_error_flags</a> . |

**32.2.6.12 static void QSPI\_EnableInterrupts ( QuadSPI\_Type \* *base*, uint32\_t *mask* )  
[inline], [static]**

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
| <i>mask</i> | QSPI interrupt source.   |

**32.2.6.13 static void QSPI\_DisableInterrupts ( QuadSPI\_Type \* *base*, uint32\_t *mask* )  
[inline], [static]**

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
| <i>mask</i> | QSPI interrupt source.   |

**32.2.6.14 static void QSPI\_EnableDMA ( QuadSPI\_Type \* *base*, uint32\_t *mask*, bool *enable* ) [inline], [static]**

## Parameters

|               |                                             |
|---------------|---------------------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.                    |
| <i>mask</i>   | QSPI DMA source.                            |
| <i>enable</i> | True means enable DMA, false means disable. |

### 32.2.6.15 static uint32\_t QSPI\_GetTxDataRegisterAddress ( QuadSPI\_Type \* *base* ) [inline], [static]

It is used for DMA operation.

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
|-------------|--------------------------|

## Returns

QSPI Tx data register address.

### 32.2.6.16 uint32\_t QSPI\_GetRxDataRegisterAddress ( QuadSPI\_Type \* *base* )

This function returns the Rx data register address or Rx buffer address according to the Rx read area settings.

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
|-------------|--------------------------|

## Returns

QSPI Rx data register address.

### 32.2.6.17 static void QSPI\_SetIPCommandAddress ( QuadSPI\_Type \* *base*, uint32\_t *addr* ) [inline], [static]



## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
| <i>addr</i> | IP command address.      |

### 32.2.6.18 static void QSPI\_SetIPCommandSize ( QuadSPI\_Type \* *base*, uint32\_t *size* ) [inline], [static]

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
| <i>size</i> | IP command size.         |

### 32.2.6.19 void QSPI\_ExecuteIPCommand ( QuadSPI\_Type \* *base*, uint32\_t *index* )

## Parameters

|              |                                              |
|--------------|----------------------------------------------|
| <i>base</i>  | Pointer to QuadSPI Type.                     |
| <i>index</i> | IP command located in which LUT table index. |

### 32.2.6.20 void QSPI\_ExecuteAHBCommand ( QuadSPI\_Type \* *base*, uint32\_t *index* )

## Parameters

|              |                                               |
|--------------|-----------------------------------------------|
| <i>base</i>  | Pointer to QuadSPI Type.                      |
| <i>index</i> | AHB command located in which LUT table index. |

### 32.2.6.21 void QSPI\_UpdateLUT ( QuadSPI\_Type \* *base*, uint32\_t *index*, uint32\_t \* *cmd* )

## Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
|-------------|--------------------------|

|              |                                                                            |
|--------------|----------------------------------------------------------------------------|
| <i>index</i> | Which LUT index needs to be located. It should be an integer divided by 4. |
| <i>cmd</i>   | Command sequence array.                                                    |

**32.2.6.22** `static void QSPI_ClearFifo ( QuadSPI_Type * base, uint32_t mask )  
[inline], [static]`

Parameters

|             |                                        |
|-------------|----------------------------------------|
| <i>base</i> | Pointer to QuadSPI Type.               |
| <i>mask</i> | Which kind of QSPI FIFO to be cleared. |

**32.2.6.23** `static void QSPI_ClearCommandSequence ( QuadSPI_Type * base,  
qspi_command_seq_t seq ) [inline], [static]`

This function can reset the command sequence.

Parameters

|             |                                                                           |
|-------------|---------------------------------------------------------------------------|
| <i>base</i> | QSPI base address.                                                        |
| <i>seq</i>  | Which command sequence need to reset, IP command, buffer command or both. |

**32.2.6.24** `static void QSPI_EnableDDRMMode ( QuadSPI_Type * base, bool enable )  
[inline], [static]`

Parameters

|               |                                                           |
|---------------|-----------------------------------------------------------|
| <i>base</i>   | QSPI base pointer                                         |
| <i>enable</i> | True means enable DDR mode, false means disable DDR mode. |

**32.2.6.25** `void QSPI_SetReadDataArea ( QuadSPI_Type * base, qspi_read_area_t area )`

This function can set the RX buffer readout, from AHB bus or IP Bus.

Parameters

|             |                                                               |
|-------------|---------------------------------------------------------------|
| <i>base</i> | QSPI base address.                                            |
| <i>area</i> | QSPI Rx buffer readout area. AHB bus buffer or IP bus buffer. |

**32.2.6.26 void QSPI\_WriteBlocking ( QuadSPI\_Type \* *base*, const uint32\_t \* *buffer*, size\_t *size* )**

Note

This function blocks via polling until all bytes have been sent.

Parameters

|               |                                  |
|---------------|----------------------------------|
| <i>base</i>   | QSPI base pointer                |
| <i>buffer</i> | The data bytes to send           |
| <i>size</i>   | The number of data bytes to send |

**32.2.6.27 static void QSPI\_WriteData ( QuadSPI\_Type \* *base*, uint32\_t *data* )  
[inline], [static]**

Parameters

|             |                        |
|-------------|------------------------|
| <i>base</i> | QSPI base pointer      |
| <i>data</i> | The data bytes to send |

**32.2.6.28 void QSPI\_ReadBlocking ( QuadSPI\_Type \* *base*, uint32\_t \* *buffer*, size\_t *size* )**

Note

This function blocks via polling until all bytes have been sent. Users shall notice that this receive size shall not bigger than 64 bytes. As this interface is used to read flash status registers. For flash contents read, please use AHB bus read, this is much more efficiency.

Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | QSPI base pointer                   |
| <i>buffer</i> | The data bytes to send              |
| <i>size</i>   | The number of data bytes to receive |

### 32.2.6.29 uint32\_t QSPI\_ReadData ( QuadSPI\_Type \* *base* )

Parameters

|             |                   |
|-------------|-------------------|
| <i>base</i> | QSPI base pointer |
|-------------|-------------------|

Returns

The data in the FIFO.

### 32.2.6.30 static void QSPI\_TransferSendBlocking ( QuadSPI\_Type \* *base*, qspi\_transfer\_t \* *xfer* ) [inline], [static]

This function writes a continuous data to the QSPI transmit FIFO. This function is a block function and can return only when finished. This function uses polling methods.

Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
| <i>xfer</i> | QSPI transfer structure. |

### 32.2.6.31 static void QSPI\_TransferReceiveBlocking ( QuadSPI\_Type \* *base*, qspi\_transfer\_t \* *xfer* ) [inline], [static]

This function reads continuous data from the QSPI receive buffer/FIFO. This function is a blocking function and can return only when finished. This function uses polling methods. Users shall notice that this receive size shall not bigger than 64 bytes. As this interface is used to read flash status registers. For flash contents read, please use AHB bus read, this is much more efficiency.

Parameters

|             |                          |
|-------------|--------------------------|
| <i>base</i> | Pointer to QuadSPI Type. |
| <i>xfer</i> | QSPI transfer structure. |

## Chapter 33

# SAI: Serial Audio Interface

### 33.1 Overview

The MCUXpresso SDK provides a peripheral driver for the Serial Audio Interface (SAI) module of MCUXpresso SDK devices.

SAI driver includes functional APIs and transactional APIs.

Functional APIs target low-level APIs. Functional APIs can be used for SAI initialization, configuration and operation, and for optimization and customization purposes. Using the functional API requires the knowledge of the SAI peripheral and how to organize functional APIs to meet the application requirements. All functional API use the peripheral base address as the first parameter. SAI functional operation groups provide the functional API set.

Transactional APIs target high-level APIs. Transactional APIs can be used to enable the peripheral and in the application if the code size and performance of transactional APIs satisfy the requirements. If the code size and performance are a critical requirement, see the transactional API implementation and write a custom code. All transactional APIs use the `sai_handle_t` as the first parameter. Initialize the handle by calling the [SAI\\_TransferTxCreateHandle\(\)](#) or [SAI\\_TransferRxCreateHandle\(\)](#) API.

Transactional APIs support asynchronous transfer. This means that the functions [SAI\\_TransferSendNonBlocking\(\)](#) and [SAI\\_TransferReceiveNonBlocking\(\)](#) set up the interrupt for data transfer. When the transfer completes, the upper layer is notified through a callback function with the `kStatus_SAI_TxIdle` and `kStatus_SAI_RxIdle` status.

### 33.2 Typical configurations

#### Bit width configuration

SAI driver support 8/16/24/32bits stereo/mono raw audio data transfer. SAI EDMA driver support 8/16/32bits stereo/mono raw audio data transfer, since the EDMA doesn't support 24bit data width, so application should pre-convert the 24bit data to 32bit. SAI DMA driver support 8/16/32bits stereo/mono raw audio data transfer, since the EDMA doesn't support 24bit data width, so application should pre-convert the 24bit data to 32bit. SAI SDMA driver support 8/16/24/32bits stereo/mono raw audio data transfer.

#### Frame configuration

SAI driver support I2S, DSP, Left justified, Right justified, TDM mode. Application can call the api directly: `SAI_GetClassicI2SConfig` `SAI_GetLeftJustifiedConfig` `SAI_GetRightJustifiedConfig` `SAI_GetTDMConfig` `SAI_GetDSPConfig`

### 33.3 Typical use case

#### 33.3.1 SAI Send/receive using an interrupt method

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/sai`

#### 33.3.2 SAI Send/receive using a DMA method

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/sai`

### Modules

- [SAI Driver](#)
- [SAI EDMA Driver](#)

## 33.4 SAI Driver

### 33.4.1 Overview

#### Data Structures

- struct [\\_sai\\_config](#)  
*SAI user configuration structure. [More...](#)*
- struct [\\_sai\\_transfer\\_format](#)  
*sai transfer format [More...](#)*
- struct [\\_sai\\_fifo](#)  
*sai fifo configurations [More...](#)*
- struct [\\_sai\\_bit\\_clock](#)  
*sai bit clock configurations [More...](#)*
- struct [\\_sai\\_frame\\_sync](#)  
*sai frame sync configurations [More...](#)*
- struct [\\_sai\\_serial\\_data](#)  
*sai serial data configurations [More...](#)*
- struct [\\_sai\\_transceiver](#)  
*sai transceiver configurations [More...](#)*
- struct [\\_sai\\_transfer](#)  
*SAI transfer structure. [More...](#)*
- struct [\\_sai\\_handle](#)  
*SAI handle structure. [More...](#)*

#### Macros

- #define [SAI\\_XFER\\_QUEUE\\_SIZE](#) (4U)  
*SAI transfer queue size, user can refine it according to use case.*
- #define [FSL\\_SAI\\_HAS\\_FIFO\\_EXTEND\\_FEATURE](#) 1  
*sai fifo feature*

#### Typedefs

- typedef enum [\\_sai\\_protocol](#) [sai\\_protocol\\_t](#)  
*Define the SAI bus type.*
- typedef enum [\\_sai\\_master\\_slave](#) [sai\\_master\\_slave\\_t](#)  
*Master or slave mode.*
- typedef enum [\\_sai\\_mono\\_stereo](#) [sai\\_mono\\_stereo\\_t](#)  
*Mono or stereo audio format.*
- typedef enum [\\_sai\\_data\\_order](#) [sai\\_data\\_order\\_t](#)  
*SAI data order, MSB or LSB.*
- typedef enum [\\_sai\\_clock\\_polarity](#) [sai\\_clock\\_polarity\\_t](#)  
*SAI clock polarity, active high or low.*
- typedef enum [\\_sai\\_sync\\_mode](#) [sai\\_sync\\_mode\\_t](#)  
*Synchronous or asynchronous mode.*
- typedef enum [\\_sai\\_bclk\\_source](#) [sai\\_bclk\\_source\\_t](#)  
*Bit clock source.*



- typedef enum `_sai_reset_type` `sai_reset_type_t`  
*The reset type.*
- typedef enum `_sai_fifo_packing` `sai_fifo_packing_t`  
*The SAI packing mode The mode includes 8 bit and 16 bit packing.*
- typedef struct `_sai_config` `sai_config_t`  
*SAI user configuration structure.*
- typedef enum `_sai_sample_rate` `sai_sample_rate_t`  
*Audio sample rate.*
- typedef enum `_sai_word_width` `sai_word_width_t`  
*Audio word width.*
- typedef enum `_sai_data_pin_state` `sai_data_pin_state_t`  
*sai data pin state definition*
- typedef enum `_sai_fifo_combine` `sai_fifo_combine_t`  
*sai fifo combine mode definition*
- typedef enum `_sai_transceiver_type` `sai_transceiver_type_t`  
*sai transceiver type*
- typedef enum `_sai_frame_sync_len` `sai_frame_sync_len_t`  
*sai frame sync len*
- typedef struct `_sai_transfer_format` `sai_transfer_format_t`  
*sai transfer format*
- typedef struct `_sai_fifo` `sai_fifo_t`  
*sai fifo configurations*
- typedef struct `_sai_bit_clock` `sai_bit_clock_t`  
*sai bit clock configurations*
- typedef struct `_sai_frame_sync` `sai_frame_sync_t`  
*sai frame sync configurations*
- typedef struct `_sai_serial_data` `sai_serial_data_t`  
*sai serial data configurations*
- typedef struct `_sai_transceiver` `sai_transceiver_t`  
*sai transceiver configurations*
- typedef struct `_sai_transfer` `sai_transfer_t`  
*SAI transfer structure.*
- typedef void(\* `sai_transfer_callback_t`)(I2S\_Type \*base, `sai_handle_t` \*handle, `status_t` status, void \*userData)  
*SAI transfer callback prototype.*

## Enumerations

- enum {  
`kStatus_SAI_TxBusy` = MAKE\_STATUS(kStatusGroup\_SAI, 0),  
`kStatus_SAI_RxBusy` = MAKE\_STATUS(kStatusGroup\_SAI, 1),  
`kStatus_SAI_TxError` = MAKE\_STATUS(kStatusGroup\_SAI, 2),  
`kStatus_SAI_RxError` = MAKE\_STATUS(kStatusGroup\_SAI, 3),  
`kStatus_SAI_QueueFull` = MAKE\_STATUS(kStatusGroup\_SAI, 4),  
`kStatus_SAI_TxIdle` = MAKE\_STATUS(kStatusGroup\_SAI, 5),  
`kStatus_SAI_RxIdle` = MAKE\_STATUS(kStatusGroup\_SAI, 6) }  
*\_sai\_status\_t, SAI return status.*
- enum {

```

kSAI_Channel0Mask = 1 << 0U,
kSAI_Channel1Mask = 1 << 1U,
kSAI_Channel2Mask = 1 << 2U,
kSAI_Channel3Mask = 1 << 3U,
kSAI_Channel4Mask = 1 << 4U,
kSAI_Channel5Mask = 1 << 5U,
kSAI_Channel6Mask = 1 << 6U,
kSAI_Channel7Mask = 1 << 7U }

```

*\_sai\_channel\_mask, sai channel mask value, actual channel numbers is depend soc specific*

- enum `_sai_protocol` {  
`kSAI_BusLeftJustified` = 0x0U,  
`kSAI_BusRightJustified`,  
`kSAI_BusI2S`,  
`kSAI_BusPCMA`,  
`kSAI_BusPCMB` }  
*Define the SAI bus type.*
- enum `_sai_master_slave` {  
`kSAI_Master` = 0x0U,  
`kSAI_Slave` = 0x1U,  
`kSAI_Bclk_Master_FrameSync_Slave` = 0x2U,  
`kSAI_Bclk_Slave_FrameSync_Master` = 0x3U }  
*Master or slave mode.*

- enum `_sai_mono_stereo` {  
`kSAI_Stereo` = 0x0U,  
`kSAI_MonoRight`,  
`kSAI_MonoLeft` }  
*Mono or stereo audio format.*

- enum `_sai_data_order` {  
`kSAI_DataLSB` = 0x0U,  
`kSAI_DataMSB` }  
*SAI data order, MSB or LSB.*
- enum `_sai_clock_polarity` {  
`kSAI_PolarityActiveHigh` = 0x0U,  
`kSAI_PolarityActiveLow` = 0x1U,  
`kSAI_SampleOnFallingEdge` = 0x0U,  
`kSAI_SampleOnRisingEdge` = 0x1U }  
*SAI clock polarity, active high or low.*

- enum `_sai_sync_mode` {  
`kSAI_ModeAsync` = 0x0U,  
`kSAI_ModeSync` }  
*Synchronous or asynchronous mode.*
- enum `_sai_bclk_source` {

```

kSAI_BclkSourceBusclk = 0x0U,
kSAI_BclkSourceMclkOption1 = 0x1U,
kSAI_BclkSourceMclkOption2 = 0x2U,
kSAI_BclkSourceMclkOption3 = 0x3U,
kSAI_BclkSourceMclkDiv = 0x1U,
kSAI_BclkSourceOtherSai0 = 0x2U,
kSAI_BclkSourceOtherSai1 = 0x3U }

```

*Bit clock source.*

- enum {
 

```

kSAI_WordStartInterruptEnable,
kSAI_SyncErrorInterruptEnable = I2S_TCSR_SEIE_MASK,
kSAI_FIFOWarningInterruptEnable = I2S_TCSR_FWIE_MASK,
kSAI_FIFOErrorInterruptEnable = I2S_TCSR_FEIE_MASK,
kSAI_FIFORequestInterruptEnable = I2S_TCSR_FRIE_MASK }

```

*\_sai\_interrupt\_enable\_t, The SAI interrupt enable flag*
- enum {
 

```

kSAI_FIFOWarningDMAEnable = I2S_TCSR_FWDE_MASK,
kSAI_FIFORequestDMAEnable = I2S_TCSR_FRDE_MASK }

```

*\_sai\_dma\_enable\_t, The DMA request sources*
- enum {
 

```

kSAI_WordStartFlag = I2S_TCSR_WSF_MASK,
kSAI_SyncErrorFlag = I2S_TCSR_SEF_MASK,
kSAI_FIFOErrorFlag = I2S_TCSR_FEF_MASK,
kSAI_FIFORequestFlag = I2S_TCSR_FRF_MASK,
kSAI_FIFOWarningFlag = I2S_TCSR_FWF_MASK }

```

*\_sai\_flags, The SAI status flag*
- enum *\_sai\_reset\_type* {
 

```

kSAI_ResetTypeSoftware = I2S_TCSR_SR_MASK,
kSAI_ResetTypeFIFO = I2S_TCSR_FR_MASK,
kSAI_ResetAll = I2S_TCSR_SR_MASK | I2S_TCSR_FR_MASK }

```

*The reset type.*
- enum *\_sai\_fifo\_packing* {
 

```

kSAI_FifoPackingDisabled = 0x0U,
kSAI_FifoPacking8bit = 0x2U,
kSAI_FifoPacking16bit = 0x3U }

```

*The SAI packing mode The mode includes 8 bit and 16 bit packing.*
- enum *\_sai\_sample\_rate* {

```

kSAI_SampleRate8KHz = 8000U,
kSAI_SampleRate11025Hz = 11025U,
kSAI_SampleRate12KHz = 12000U,
kSAI_SampleRate16KHz = 16000U,
kSAI_SampleRate22050Hz = 22050U,
kSAI_SampleRate24KHz = 24000U,
kSAI_SampleRate32KHz = 32000U,
kSAI_SampleRate44100Hz = 44100U,
kSAI_SampleRate48KHz = 48000U,
kSAI_SampleRate96KHz = 96000U,
kSAI_SampleRate192KHz = 192000U,
kSAI_SampleRate384KHz = 384000U }

```

*Audio sample rate.*

- enum `_sai_word_width` {  
`kSAI_WordWidth8bits` = 8U,  
`kSAI_WordWidth16bits` = 16U,  
`kSAI_WordWidth24bits` = 24U,  
`kSAI_WordWidth32bits` = 32U }

*Audio word width.*

- enum `_sai_data_pin_state` {  
`kSAI_DataPinStateTriState`,  
`kSAI_DataPinStateOutputZero` = 1U }

*sai data pin state definition*

- enum `_sai_fifo_combine` {  
`kSAI_FifoCombineDisabled` = 0U,  
`kSAI_FifoCombineModeEnabledOnRead`,  
`kSAI_FifoCombineModeEnabledOnWrite`,  
`kSAI_FifoCombineModeEnabledOnReadWrite` }

*sai fifo combine mode definition*

- enum `_sai_transceiver_type` {  
`kSAI_Transmitter` = 0U,  
`kSAI_Receiver` = 1U }

*sai transceiver type*

- enum `_sai_frame_sync_len` {  
`kSAI_FrameSyncLenOneBitClk` = 0U,  
`kSAI_FrameSyncLenPerWordWidth` = 1U }

*sai frame sync len*

## Driver version

- #define `FSL_SAI_DRIVER_VERSION` (`MAKE_VERSION`(2, 4, 2))  
*Version 2.4.2.*

## Initialization and deinitialization

- void [SAI\\_Init](#) (I2S\_Type \*base)  
*Initializes the SAI peripheral.*
- void [SAI\\_Deinit](#) (I2S\_Type \*base)  
*De-initializes the SAI peripheral.*
- void [SAI\\_TxReset](#) (I2S\_Type \*base)  
*Resets the SAI Tx.*
- void [SAI\\_RxReset](#) (I2S\_Type \*base)  
*Resets the SAI Rx.*
- void [SAI\\_TxEnable](#) (I2S\_Type \*base, bool enable)  
*Enables/disables the SAI Tx.*
- void [SAI\\_RxEnable](#) (I2S\_Type \*base, bool enable)  
*Enables/disables the SAI Rx.*
- static void [SAI\\_TxSetBitClockDirection](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave)  
*Set Rx bit clock direction.*
- static void [SAI\\_RxSetBitClockDirection](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave)  
*Set Rx bit clock direction.*
- static void [SAI\\_RxSetFrameSyncDirection](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave)  
*Set Rx frame sync direction.*
- static void [SAI\\_TxSetFrameSyncDirection](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave)  
*Set Tx frame sync direction.*
- void [SAI\\_TxSetBitClockRate](#) (I2S\_Type \*base, uint32\_t sourceClockHz, uint32\_t sampleRate, uint32\_t bitWidth, uint32\_t channelNumbers)  
*Transmitter bit clock rate configurations.*
- void [SAI\\_RxSetBitClockRate](#) (I2S\_Type \*base, uint32\_t sourceClockHz, uint32\_t sampleRate, uint32\_t bitWidth, uint32\_t channelNumbers)  
*Receiver bit clock rate configurations.*
- void [SAI\\_TxSetBitclockConfig](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave, sai\_bit\_clock\_t \*config)  
*Transmitter Bit clock configurations.*
- void [SAI\\_RxSetBitclockConfig](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave, sai\_bit\_clock\_t \*config)  
*Receiver Bit clock configurations.*
- void [SAI\\_TxSetFifoConfig](#) (I2S\_Type \*base, sai\_fifo\_t \*config)  
*SAI transmitter fifo configurations.*
- void [SAI\\_RxSetFifoConfig](#) (I2S\_Type \*base, sai\_fifo\_t \*config)  
*SAI receiver fifo configurations.*
- void [SAI\\_TxSetFrameSyncConfig](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave, sai\_frame\_sync\_t \*config)  
*SAI transmitter Frame sync configurations.*
- void [SAI\\_RxSetFrameSyncConfig](#) (I2S\_Type \*base, sai\_master\_slave\_t masterSlave, sai\_frame\_sync\_t \*config)  
*SAI receiver Frame sync configurations.*
- void [SAI\\_TxSetSerialDataConfig](#) (I2S\_Type \*base, sai\_serial\_data\_t \*config)  
*SAI transmitter Serial data configurations.*
- void [SAI\\_RxSetSerialDataConfig](#) (I2S\_Type \*base, sai\_serial\_data\_t \*config)  
*SAI receiver Serial data configurations.*
- void [SAI\\_TxSetConfig](#) (I2S\_Type \*base, sai\_transceiver\_t \*config)  
*SAI transmitter configurations.*

- void [SAI\\_RxSetConfig](#) (I2S\_Type \*base, [sai\\_transceiver\\_t](#) \*config)  
*SAI receiver configurations.*
- void [SAI\\_GetClassicI2SConfig](#) ([sai\\_transceiver\\_t](#) \*config, [sai\\_word\\_width\\_t](#) bitWidth, [sai\\_mono\\_stereo\\_t](#) mode, [uint32\\_t](#) saiChannelMask)  
*Get classic I2S mode configurations.*
- void [SAI\\_GetLeftJustifiedConfig](#) ([sai\\_transceiver\\_t](#) \*config, [sai\\_word\\_width\\_t](#) bitWidth, [sai\\_mono\\_stereo\\_t](#) mode, [uint32\\_t](#) saiChannelMask)  
*Get left justified mode configurations.*
- void [SAI\\_GetRightJustifiedConfig](#) ([sai\\_transceiver\\_t](#) \*config, [sai\\_word\\_width\\_t](#) bitWidth, [sai\\_mono\\_stereo\\_t](#) mode, [uint32\\_t](#) saiChannelMask)  
*Get right justified mode configurations.*
- void [SAI\\_GetTDMConfig](#) ([sai\\_transceiver\\_t](#) \*config, [sai\\_frame\\_sync\\_len\\_t](#) frameSyncWidth, [sai\\_word\\_width\\_t](#) bitWidth, [uint32\\_t](#) dataWordNum, [uint32\\_t](#) saiChannelMask)  
*Get TDM mode configurations.*
- void [SAI\\_GetDSPConfig](#) ([sai\\_transceiver\\_t](#) \*config, [sai\\_frame\\_sync\\_len\\_t](#) frameSyncWidth, [sai\\_word\\_width\\_t](#) bitWidth, [sai\\_mono\\_stereo\\_t](#) mode, [uint32\\_t](#) saiChannelMask)  
*Get DSP mode configurations.*

## Status

- static [uint32\\_t](#) [SAI\\_TxGetStatusFlag](#) (I2S\_Type \*base)  
*Gets the SAI Tx status flag state.*
- static void [SAI\\_TxClearStatusFlags](#) (I2S\_Type \*base, [uint32\\_t](#) mask)  
*Clears the SAI Tx status flag state.*
- static [uint32\\_t](#) [SAI\\_RxGetStatusFlag](#) (I2S\_Type \*base)  
*Gets the SAI Rx status flag state.*
- static void [SAI\\_RxClearStatusFlags](#) (I2S\_Type \*base, [uint32\\_t](#) mask)  
*Clears the SAI Rx status flag state.*
- void [SAI\\_TxSoftwareReset](#) (I2S\_Type \*base, [sai\\_reset\\_type\\_t](#) resetType)  
*Do software reset or FIFO reset.*
- void [SAI\\_RxSoftwareReset](#) (I2S\_Type \*base, [sai\\_reset\\_type\\_t](#) resetType)  
*Do software reset or FIFO reset.*
- void [SAI\\_TxSetChannelFIFOMask](#) (I2S\_Type \*base, [uint8\\_t](#) mask)  
*Set the Tx channel FIFO enable mask.*
- void [SAI\\_RxSetChannelFIFOMask](#) (I2S\_Type \*base, [uint8\\_t](#) mask)  
*Set the Rx channel FIFO enable mask.*
- void [SAI\\_TxSetDataOrder](#) (I2S\_Type \*base, [sai\\_data\\_order\\_t](#) order)  
*Set the Tx data order.*
- void [SAI\\_RxSetDataOrder](#) (I2S\_Type \*base, [sai\\_data\\_order\\_t](#) order)  
*Set the Rx data order.*
- void [SAI\\_TxSetBitClockPolarity](#) (I2S\_Type \*base, [sai\\_clock\\_polarity\\_t](#) polarity)  
*Set the Tx data order.*
- void [SAI\\_RxSetBitClockPolarity](#) (I2S\_Type \*base, [sai\\_clock\\_polarity\\_t](#) polarity)  
*Set the Rx data order.*
- void [SAI\\_TxSetFrameSyncPolarity](#) (I2S\_Type \*base, [sai\\_clock\\_polarity\\_t](#) polarity)  
*Set the Tx data order.*
- void [SAI\\_RxSetFrameSyncPolarity](#) (I2S\_Type \*base, [sai\\_clock\\_polarity\\_t](#) polarity)  
*Set the Rx data order.*
- void [SAI\\_TxSetFIFOPacking](#) (I2S\_Type \*base, [sai\\_fifo\\_packing\\_t](#) pack)

- *Set Tx FIFO packing feature.*
- void [SAI\\_RxSetFIFOPacking](#) (I2S\_Type \*base, [sai\\_fifo\\_packing\\_t](#) pack)
- *Set Rx FIFO packing feature.*
- static void [SAI\\_TxSetFIFOErrorContinue](#) (I2S\_Type \*base, bool isEnabled)
- *Set Tx FIFO error continue.*
- static void [SAI\\_RxSetFIFOErrorContinue](#) (I2S\_Type \*base, bool isEnabled)
- *Set Rx FIFO error continue.*

## Interrupts

- static void [SAI\\_TxEnableInterrupts](#) (I2S\_Type \*base, uint32\_t mask)
- *Enables the SAI Tx interrupt requests.*
- static void [SAI\\_RxEnableInterrupts](#) (I2S\_Type \*base, uint32\_t mask)
- *Enables the SAI Rx interrupt requests.*
- static void [SAI\\_TxDisableInterrupts](#) (I2S\_Type \*base, uint32\_t mask)
- *Disables the SAI Tx interrupt requests.*
- static void [SAI\\_RxDisableInterrupts](#) (I2S\_Type \*base, uint32\_t mask)
- *Disables the SAI Rx interrupt requests.*

## DMA Control

- static void [SAI\\_TxEnableDMA](#) (I2S\_Type \*base, uint32\_t mask, bool enable)
- *Enables/disables the SAI Tx DMA requests.*
- static void [SAI\\_RxEnableDMA](#) (I2S\_Type \*base, uint32\_t mask, bool enable)
- *Enables/disables the SAI Rx DMA requests.*
- static uintptr\_t [SAI\\_TxGetDataRegisterAddress](#) (I2S\_Type \*base, uint32\_t channel)
- *Gets the SAI Tx data register address.*
- static uintptr\_t [SAI\\_RxGetDataRegisterAddress](#) (I2S\_Type \*base, uint32\_t channel)
- *Gets the SAI Rx data register address.*

## Bus Operations

- void [SAI\\_WriteBlocking](#) (I2S\_Type \*base, uint32\_t channel, uint32\_t bitWidth, uint8\_t \*buffer, uint32\_t size)
- *Sends data using a blocking method.*
- void [SAI\\_WriteMultiChannelBlocking](#) (I2S\_Type \*base, uint32\_t channel, uint32\_t channelMask, uint32\_t bitWidth, uint8\_t \*buffer, uint32\_t size)
- *Sends data to multi channel using a blocking method.*
- static void [SAI\\_WriteData](#) (I2S\_Type \*base, uint32\_t channel, uint32\_t data)
- *Writes data into SAI FIFO.*
- void [SAI\\_ReadBlocking](#) (I2S\_Type \*base, uint32\_t channel, uint32\_t bitWidth, uint8\_t \*buffer, uint32\_t size)
- *Receives data using a blocking method.*
- void [SAI\\_ReadMultiChannelBlocking](#) (I2S\_Type \*base, uint32\_t channel, uint32\_t channelMask, uint32\_t bitWidth, uint8\_t \*buffer, uint32\_t size)
- *Receives multi channel data using a blocking method.*
- static uint32\_t [SAI\\_ReadData](#) (I2S\_Type \*base, uint32\_t channel)



*Reads data from the SAI FIFO.*

## Transactional

- void [SAI\\_TransferTxCreateHandle](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, [sai\\_transfer\\_callback\\_t](#) callback, void \*userData)  
*Initializes the SAI Tx handle.*
- void [SAI\\_TransferRxCreateHandle](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, [sai\\_transfer\\_callback\\_t](#) callback, void \*userData)  
*Initializes the SAI Rx handle.*
- void [SAI\\_TransferTxSetConfig](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, [sai\\_transceiver\\_t](#) \*config)  
*SAI transmitter transfer configurations.*
- void [SAI\\_TransferRxSetConfig](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, [sai\\_transceiver\\_t](#) \*config)  
*SAI receiver transfer configurations.*
- [status\\_t SAI\\_TransferSendNonBlocking](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, [sai\\_transfer\\_t](#) \*xfer)  
*Performs an interrupt non-blocking send transfer on SAI.*
- [status\\_t SAI\\_TransferReceiveNonBlocking](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, [sai\\_transfer\\_t](#) \*xfer)  
*Performs an interrupt non-blocking receive transfer on SAI.*
- [status\\_t SAI\\_TransferGetSendCount](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, size\_t \*count)  
*Gets a set byte count.*
- [status\\_t SAI\\_TransferGetReceiveCount](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle, size\_t \*count)  
*Gets a received byte count.*
- void [SAI\\_TransferAbortSend](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle)  
*Aborts the current send.*
- void [SAI\\_TransferAbortReceive](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle)  
*Aborts the current IRQ receive.*
- void [SAI\\_TransferTerminateSend](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle)  
*Terminate all SAI send.*
- void [SAI\\_TransferTerminateReceive](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle)  
*Terminate all SAI receive.*
- void [SAI\\_TransferTxHandleIRQ](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle)  
*Tx interrupt handler.*
- void [SAI\\_TransferRxHandleIRQ](#) (I2S\_Type \*base, [sai\\_handle\\_t](#) \*handle)  
*Rx interrupt handler.*

## 33.4.2 Data Structure Documentation

### 33.4.2.1 struct\_sai\_config

#### Data Fields

- [sai\\_protocol\\_t protocol](#)  
*Audio bus protocol in SAI.*
- [sai\\_sync\\_mode\\_t syncMode](#)  
*SAI sync mode, control Tx/Rx clock sync.*



- [sai\\_bclk\\_source\\_t](#) `bclkSource`  
*Bit Clock source.*
- [sai\\_master\\_slave\\_t](#) `masterSlave`  
*Master or slave.*

### 33.4.2.2 struct \_sai\_transfer\_format

#### Data Fields

- [uint32\\_t](#) `sampleRate_Hz`  
*Sample rate of audio data.*
- [uint32\\_t](#) `bitWidth`  
*Data length of audio data, usually 8/16/24/32 bits.*
- [sai\\_mono\\_stereo\\_t](#) `stereo`  
*Mono or stereo.*
- [uint8\\_t](#) `watermark`  
*Watermark value.*
- [uint8\\_t](#) `channel`  
*Transfer start channel.*
- [uint8\\_t](#) `channelMask`  
*enabled channel mask value, reference `_sai_channel_mask`*
- [uint8\\_t](#) `endChannel`  
*end channel number*
- [uint8\\_t](#) `channelNums`  
*Total enabled channel numbers.*
- [sai\\_protocol\\_t](#) `protocol`  
*Which audio protocol used.*
- [bool](#) `isFrameSyncCompact`  
*True means Frame sync length is configurable according to bitWidth, false means frame sync length is 64 times of bit clock.*

#### Field Documentation

#### (1) [bool](#) `_sai_transfer_format::isFrameSyncCompact`

### 33.4.2.3 struct \_sai\_fifo

#### Data Fields

- [bool](#) `fifoContinueOnError`  
*fifo continues when error occur*
- [sai\\_fifo\\_combine\\_t](#) `fifoCombine`  
*fifo combine mode*
- [sai\\_fifo\\_packing\\_t](#) `fifoPacking`  
*fifo packing mode*
- [uint8\\_t](#) `fifoWatermark`  
*fifo watermark*

### 33.4.2.4 struct \_sai\_bit\_clock

#### Data Fields

- bool [bclkSrcSwap](#)  
*bit clock source swap*
- bool [bclkInputDelay](#)  
*bit clock actually used by the transmitter is delayed by the pad output delay, this has effect of decreasing the data input setup time, but increasing the data output valid time .*
- [sai\\_clock\\_polarity\\_t](#) [bclkPolarity](#)  
*bit clock polarity*
- [sai\\_bclk\\_source\\_t](#) [bclkSource](#)  
*bit Clock source*

#### Field Documentation

(1) bool \_sai\_bit\_clock::bclkInputDelay

### 33.4.2.5 struct \_sai\_frame\_sync

#### Data Fields

- uint8\_t [frameSyncWidth](#)  
*frame sync width in number of bit clocks*
- bool [frameSyncEarly](#)  
*TRUE is frame sync assert one bit before the first bit of frame FALSE is frame sync assert with the first bit of the frame.*
- bool [frameSyncGenerateOnDemand](#)  
*internal frame sync is generated when FIFO waring flag is clear*
- [sai\\_clock\\_polarity\\_t](#) [frameSyncPolarity](#)  
*frame sync polarity*

### 33.4.2.6 struct \_sai\_serial\_data

#### Data Fields

- [sai\\_data\\_pin\\_state\\_t](#) [dataMode](#)  
*sai data pin state when slots masked or channel disabled*
- [sai\\_data\\_order\\_t](#) [dataOrder](#)  
*configure whether the LSB or MSB is transmitted first*
- uint8\_t [dataWord0Length](#)  
*configure the number of bits in the first word in each frame*
- uint8\_t [dataWordNLength](#)  
*configure the number of bits in the each word in each frame, except the first word*
- uint8\_t [dataWordLength](#)  
*used to record the data length for dma transfer*
- uint8\_t [dataFirstBitShifted](#)  
*Configure the bit index for the first bit transmitted for each word in the frame.*
- uint8\_t [dataWordNum](#)  
*configure the number of words in each frame*

- uint32\_t [dataMaskedWord](#)  
*configure whether the transmit word is masked*

### 33.4.2.7 struct \_sai\_transceiver

#### Data Fields

- [sai\\_serial\\_data\\_t](#) serialData  
*serial data configurations*
- [sai\\_frame\\_sync\\_t](#) frameSync  
*ws configurations*
- [sai\\_bit\\_clock\\_t](#) bitClock  
*bit clock configurations*
- [sai\\_fifo\\_t](#) fifo  
*fifo configurations*
- [sai\\_master\\_slave\\_t](#) masterSlave  
*transceiver is master or slave*
- [sai\\_sync\\_mode\\_t](#) syncMode  
*transceiver sync mode*
- uint8\_t [startChannel](#)  
*Transfer start channel.*
- uint8\_t [channelMask](#)  
*enabled channel mask value, reference `_sai_channel_mask`*
- uint8\_t [endChannel](#)  
*end channel number*
- uint8\_t [channelNums](#)  
*Total enabled channel numbers.*

### 33.4.2.8 struct \_sai\_transfer

#### Data Fields

- uint8\_t \* [data](#)  
*Data start address to transfer.*
- size\_t [dataSize](#)  
*Transfer size.*

#### Field Documentation

(1) uint8\_t\* `_sai_transfer::data`

(2) size\_t `_sai_transfer::dataSize`

### 33.4.2.9 struct \_sai\_handle

#### Data Fields

- I2S\_Type \* [base](#)  
*base address*

- `uint32_t state`  
*Transfer status.*
- `sai_transfer_callback_t callback`  
*Callback function called at transfer event.*
- `void * userData`  
*Callback parameter passed to callback function.*
- `uint8_t bitWidth`  
*Bit width for transfer, 8/16/24/32 bits.*
- `uint8_t channel`  
*Transfer start channel.*
- `uint8_t channelMask`  
*enabled channel mask value, refernece `_sai_channel_mask`*
- `uint8_t endChannel`  
*end channel number*
- `uint8_t channelNums`  
*Total enabled channel numbers.*
- `sai_transfer_t saiQueue [SAI_XFER_QUEUE_SIZE]`  
*Transfer queue storing queued transfer.*
- `size_t transferSize [SAI_XFER_QUEUE_SIZE]`  
*Data bytes need to transfer.*
- `volatile uint8_t queueUser`  
*Index for user to queue transfer.*
- `volatile uint8_t queueDriver`  
*Index for driver to get the transfer data and size.*
- `uint8_t watermark`  
*Watermark value.*

### 33.4.3 Macro Definition Documentation

#### 33.4.3.1 #define SAI\_XFER\_QUEUE\_SIZE (4U)

### 33.4.4 Enumeration Type Documentation

#### 33.4.4.1 anonymous enum

Enumerator

***kStatus\_SAI\_TxBusy*** SAI Tx is busy.  
***kStatus\_SAI\_RxBusy*** SAI Rx is busy.  
***kStatus\_SAI\_TxError*** SAI Tx FIFO error.  
***kStatus\_SAI\_RxError*** SAI Rx FIFO error.  
***kStatus\_SAI\_QueueFull*** SAI transfer queue is full.  
***kStatus\_SAI\_TxIdle*** SAI Tx is idle.  
***kStatus\_SAI\_RxIdle*** SAI Rx is idle.

### 33.4.4.2 anonymous enum

Enumerator

*kSAI\_Channel0Mask* channel 0 mask value  
*kSAI\_Channel1Mask* channel 1 mask value  
*kSAI\_Channel2Mask* channel 2 mask value  
*kSAI\_Channel3Mask* channel 3 mask value  
*kSAI\_Channel4Mask* channel 4 mask value  
*kSAI\_Channel5Mask* channel 5 mask value  
*kSAI\_Channel6Mask* channel 6 mask value  
*kSAI\_Channel7Mask* channel 7 mask value

### 33.4.4.3 enum \_sai\_protocol

Enumerator

*kSAI\_BusLeftJustified* Uses left justified format.  
*kSAI\_BusRightJustified* Uses right justified format.  
*kSAI\_BusI2S* Uses I2S format.  
*kSAI\_BusPCMA* Uses I2S PCM A format.  
*kSAI\_BusPCMB* Uses I2S PCM B format.

### 33.4.4.4 enum \_sai\_master\_slave

Enumerator

*kSAI\_Master* Master mode include bclk and frame sync.  
*kSAI\_Slave* Slave mode include bclk and frame sync.  
*kSAI\_Bclk\_Master\_FrameSync\_Slave* bclk in master mode, frame sync in slave mode  
*kSAI\_Bclk\_Slave\_FrameSync\_Master* bclk in slave mode, frame sync in master mode

### 33.4.4.5 enum \_sai\_mono\_stereo

Enumerator

*kSAI\_Stereo* Stereo sound.  
*kSAI\_MonoRight* Only Right channel have sound.  
*kSAI\_MonoLeft* Only left channel have sound.

**33.4.4.6 enum \_sai\_data\_order**

Enumerator

*kSAI\_DataLSB* LSB bit transferred first.  
*kSAI\_DataMSB* MSB bit transferred first.

**33.4.4.7 enum \_sai\_clock\_polarity**

Enumerator

*kSAI\_PolarityActiveHigh* Drive outputs on rising edge.  
*kSAI\_PolarityActiveLow* Drive outputs on falling edge.  
*kSAI\_SampleOnFallingEdge* Sample inputs on falling edge.  
*kSAI\_SampleOnRisingEdge* Sample inputs on rising edge.

**33.4.4.8 enum \_sai\_sync\_mode**

Enumerator

*kSAI\_ModeAsync* Asynchronous mode.  
*kSAI\_ModeSync* Synchronous mode (with receiver or transmit)

**33.4.4.9 enum \_sai\_bclk\_source**

Enumerator

*kSAI\_BclkSourceBusclk* Bit clock using bus clock.  
*kSAI\_BclkSourceMclkOption1* Bit clock MCLK option 1.  
*kSAI\_BclkSourceMclkOption2* Bit clock MCLK option2.  
*kSAI\_BclkSourceMclkOption3* Bit clock MCLK option3.  
*kSAI\_BclkSourceMclkDiv* Bit clock using master clock divider.  
*kSAI\_BclkSourceOtherSai0* Bit clock from other SAI device.  
*kSAI\_BclkSourceOtherSai1* Bit clock from other SAI device.

**33.4.4.10 anonymous enum**

Enumerator

*kSAI\_WordStartInterruptEnable* Word start flag, means the first word in a frame detected.  
*kSAI\_SyncErrorInterruptEnable* Sync error flag, means the sync error is detected.  
*kSAI\_FIFOWarningInterruptEnable* FIFO warning flag, means the FIFO is empty.  
*kSAI\_FIFOErrorInterruptEnable* FIFO error flag.  
*kSAI\_FIFORequestInterruptEnable* FIFO request, means reached watermark.

**33.4.4.11 anonymous enum**

Enumerator

*kSAI\_FIFOWarningDMAEnable* FIFO warning caused by the DMA request.*kSAI\_FIFORequestDMAEnable* FIFO request caused by the DMA request.**33.4.4.12 anonymous enum**

Enumerator

*kSAI\_WordStartFlag* Word start flag, means the first word in a frame detected.*kSAI\_SyncErrorFlag* Sync error flag, means the sync error is detected.*kSAI\_FIFOErrorFlag* FIFO error flag.*kSAI\_FIFORequestFlag* FIFO request flag.*kSAI\_FIFOWarningFlag* FIFO warning flag.**33.4.4.13 enum \_sai\_reset\_type**

Enumerator

*kSAI\_ResetTypeSoftware* Software reset, reset the logic state.*kSAI\_ResetTypeFIFO* FIFO reset, reset the FIFO read and write pointer.*kSAI\_ResetAll* All reset.**33.4.4.14 enum \_sai\_fifo\_packing**

Enumerator

*kSAI\_FifoPackingDisabled* Packing disabled.*kSAI\_FifoPacking8bit* 8 bit packing enabled*kSAI\_FifoPacking16bit* 16bit packing enabled**33.4.4.15 enum \_sai\_sample\_rate**

Enumerator

*kSAI\_SampleRate8KHz* Sample rate 8000 Hz.*kSAI\_SampleRate11025Hz* Sample rate 11025 Hz.*kSAI\_SampleRate12KHz* Sample rate 12000 Hz.*kSAI\_SampleRate16KHz* Sample rate 16000 Hz.*kSAI\_SampleRate22050Hz* Sample rate 22050 Hz.*kSAI\_SampleRate24KHz* Sample rate 24000 Hz.*kSAI\_SampleRate32KHz* Sample rate 32000 Hz.

***kSAI\_SampleRate44100Hz*** Sample rate 44100 Hz.

***kSAI\_SampleRate48KHz*** Sample rate 48000 Hz.

***kSAI\_SampleRate96KHz*** Sample rate 96000 Hz.

***kSAI\_SampleRate192KHz*** Sample rate 192000 Hz.

***kSAI\_SampleRate384KHz*** Sample rate 384000 Hz.

#### 33.4.4.16 enum \_sai\_word\_width

Enumerator

***kSAI\_WordWidth8bits*** Audio data width 8 bits.

***kSAI\_WordWidth16bits*** Audio data width 16 bits.

***kSAI\_WordWidth24bits*** Audio data width 24 bits.

***kSAI\_WordWidth32bits*** Audio data width 32 bits.

#### 33.4.4.17 enum \_sai\_data\_pin\_state

Enumerator

***kSAI\_DataPinStateTriState*** transmit data pins are tri-stated when slots are masked or channels are disabled

***kSAI\_DataPinStateOutputZero*** transmit data pins are never tri-stated and will output zero when slots are masked or channel disabled

#### 33.4.4.18 enum \_sai\_fifo\_combine

Enumerator

***kSAI\_FifoCombineDisabled*** sai fifo combine mode disabled

***kSAI\_FifoCombineModeEnabledOnRead*** sai fifo combine mode enabled on FIFO reads

***kSAI\_FifoCombineModeEnabledOnWrite*** sai fifo combine mode enabled on FIFO write

***kSAI\_FifoCombineModeEnabledOnReadWrite*** sai fifo combined mode enabled on FIFO read/writes

#### 33.4.4.19 enum \_sai\_transceiver\_type

Enumerator

***kSAI\_Transmitter*** sai transmitter

***kSAI\_Receiver*** sai receiver



### 33.4.4.20 enum \_sai\_frame\_sync\_len

Enumerator

*kSAI\_FrameSyncLenOneBitClk* 1 bit clock frame sync len for DSP mode  
*kSAI\_FrameSyncLenPerWordWidth* Frame sync length decided by word width.

## 33.4.5 Function Documentation

### 33.4.5.1 void SAI\_Init ( I2S\_Type \* *base* )

This API gates the SAI clock. The SAI module can't operate unless SAI\_Init is called to enable the clock.

Parameters

|             |                   |
|-------------|-------------------|
| <i>base</i> | SAI base pointer. |
|-------------|-------------------|

### 33.4.5.2 void SAI\_Deinit ( I2S\_Type \* *base* )

This API gates the SAI clock. The SAI module can't operate unless SAI\_TxInit or SAI\_RxInit is called to enable the clock.

Parameters

|             |                   |
|-------------|-------------------|
| <i>base</i> | SAI base pointer. |
|-------------|-------------------|

### 33.4.5.3 void SAI\_TxReset ( I2S\_Type \* *base* )

This function enables the software reset and FIFO reset of SAI Tx. After reset, clear the reset bit.

Parameters

|             |                  |
|-------------|------------------|
| <i>base</i> | SAI base pointer |
|-------------|------------------|

### 33.4.5.4 void SAI\_RxReset ( I2S\_Type \* *base* )

This function enables the software reset and FIFO reset of SAI Rx. After reset, clear the reset bit.

Parameters

|             |                  |
|-------------|------------------|
| <i>base</i> | SAI base pointer |
|-------------|------------------|

### 33.4.5.5 void SAI\_TxEnable ( I2S\_Type \* *base*, bool *enable* )

Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>base</i>   | SAI base pointer.                              |
| <i>enable</i> | True means enable SAI Tx, false means disable. |

### 33.4.5.6 void SAI\_RxEnable ( I2S\_Type \* *base*, bool *enable* )

Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>base</i>   | SAI base pointer.                              |
| <i>enable</i> | True means enable SAI Rx, false means disable. |

### 33.4.5.7 static void SAI\_TxSetBitClockDirection ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave* ) [inline], [static]

Select bit clock direction, master or slave.

Parameters

|                    |                               |
|--------------------|-------------------------------|
| <i>base</i>        | SAI base pointer.             |
| <i>masterSlave</i> | reference sai_master_slave_t. |

### 33.4.5.8 static void SAI\_RxSetBitClockDirection ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave* ) [inline], [static]

Select bit clock direction, master or slave.

Parameters

---

|                    |                               |
|--------------------|-------------------------------|
| <i>base</i>        | SAI base pointer.             |
| <i>masterSlave</i> | reference sai_master_slave_t. |

#### 33.4.5.9 static void SAI\_RxSetFrameSyncDirection ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave* ) [inline], [static]

Select frame sync direction, master or slave.

Parameters

|                    |                               |
|--------------------|-------------------------------|
| <i>base</i>        | SAI base pointer.             |
| <i>masterSlave</i> | reference sai_master_slave_t. |

#### 33.4.5.10 static void SAI\_TxSetFrameSyncDirection ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave* ) [inline], [static]

Select frame sync direction, master or slave.

Parameters

|                    |                               |
|--------------------|-------------------------------|
| <i>base</i>        | SAI base pointer.             |
| <i>masterSlave</i> | reference sai_master_slave_t. |

#### 33.4.5.11 void SAI\_TxSetBitClockRate ( I2S\_Type \* *base*, uint32\_t *sourceClockHz*, uint32\_t *sampleRate*, uint32\_t *bitWidth*, uint32\_t *channelNumbers* )

Parameters

|                        |                             |
|------------------------|-----------------------------|
| <i>base</i>            | SAI base pointer.           |
| <i>sourceClockHz</i>   | Bit clock source frequency. |
| <i>sampleRate</i>      | Audio data sample rate.     |
| <i>bitWidth</i>        | Audio data bitWidth.        |
| <i>channel-Numbers</i> | Audio channel numbers.      |

#### 33.4.5.12 void SAI\_RxSetBitClockRate ( I2S\_Type \* *base*, uint32\_t *sourceClockHz*, uint32\_t *sampleRate*, uint32\_t *bitWidth*, uint32\_t *channelNumbers* )

## Parameters

|                        |                             |
|------------------------|-----------------------------|
| <i>base</i>            | SAI base pointer.           |
| <i>sourceClockHz</i>   | Bit clock source frequency. |
| <i>sampleRate</i>      | Audio data sample rate.     |
| <i>bitWidth</i>        | Audio data bitWidth.        |
| <i>channel-Numbers</i> | Audio channel numbers.      |

### 33.4.5.13 void SAI\_TxSetBitclockConfig ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave*, sai\_bit\_clock\_t \* *config* )

## Parameters

|                    |                                                            |
|--------------------|------------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                          |
| <i>masterSlave</i> | master or slave.                                           |
| <i>config</i>      | bit clock other configurations, can be NULL in slave mode. |

### 33.4.5.14 void SAI\_RxSetBitclockConfig ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave*, sai\_bit\_clock\_t \* *config* )

## Parameters

|                    |                                                            |
|--------------------|------------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                          |
| <i>masterSlave</i> | master or slave.                                           |
| <i>config</i>      | bit clock other configurations, can be NULL in slave mode. |

### 33.4.5.15 void SAI\_TxSetFifoConfig ( I2S\_Type \* *base*, sai\_fifo\_t \* *config* )

## Parameters

|             |                   |
|-------------|-------------------|
| <i>base</i> | SAI base pointer. |
|-------------|-------------------|

|               |                      |
|---------------|----------------------|
| <i>config</i> | fifo configurations. |
|---------------|----------------------|

#### 33.4.5.16 void SAI\_RxSetFifoConfig ( I2S\_Type \* *base*, sai\_fifo\_t \* *config* )

Parameters

|               |                      |
|---------------|----------------------|
| <i>base</i>   | SAI base pointer.    |
| <i>config</i> | fifo configurations. |

#### 33.4.5.17 void SAI\_TxSetFrameSyncConfig ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave*, sai\_frame\_sync\_t \* *config* )

Parameters

|                    |                                                       |
|--------------------|-------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                     |
| <i>masterSlave</i> | master or slave.                                      |
| <i>config</i>      | frame sync configurations, can be NULL in slave mode. |

#### 33.4.5.18 void SAI\_RxSetFrameSyncConfig ( I2S\_Type \* *base*, sai\_master\_slave\_t *masterSlave*, sai\_frame\_sync\_t \* *config* )

Parameters

|                    |                                                       |
|--------------------|-------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                     |
| <i>masterSlave</i> | master or slave.                                      |
| <i>config</i>      | frame sync configurations, can be NULL in slave mode. |

#### 33.4.5.19 void SAI\_TxSetSerialDataConfig ( I2S\_Type \* *base*, sai\_serial\_data\_t \* *config* )

Parameters

|               |                             |
|---------------|-----------------------------|
| <i>base</i>   | SAI base pointer.           |
| <i>config</i> | serial data configurations. |

#### 33.4.5.20 void SAI\_RxSetSerialDataConfig ( I2S\_Type \* *base*, sai\_serial\_data\_t \* *config* )

Parameters

|               |                             |
|---------------|-----------------------------|
| <i>base</i>   | SAI base pointer.           |
| <i>config</i> | serial data configurations. |

### 33.4.5.21 void SAI\_TxSetConfig ( I2S\_Type \* *base*, sai\_transceiver\_t \* *config* )

Parameters

|               |                             |
|---------------|-----------------------------|
| <i>base</i>   | SAI base pointer.           |
| <i>config</i> | transmitter configurations. |

### 33.4.5.22 void SAI\_RxSetConfig ( I2S\_Type \* *base*, sai\_transceiver\_t \* *config* )

Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>config</i> | receiver configurations. |

### 33.4.5.23 void SAI\_GetClassicI2SConfig ( sai\_transceiver\_t \* *config*, sai\_word\_width\_t *bitWidth*, sai\_mono\_stereo\_t *mode*, uint32\_t *saiChannelMask* )

Parameters

|                       |                                         |
|-----------------------|-----------------------------------------|
| <i>config</i>         | transceiver configurations.             |
| <i>bitWidth</i>       | audio data bitWidth.                    |
| <i>mode</i>           | audio data channel.                     |
| <i>saiChannelMask</i> | mask value of the channel to be enable. |

### 33.4.5.24 void SAI\_GetLeftJustifiedConfig ( sai\_transceiver\_t \* *config*, sai\_word\_width\_t *bitWidth*, sai\_mono\_stereo\_t *mode*, uint32\_t *saiChannelMask* )

## Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>config</i>          | transceiver configurations.             |
| <i>bitWidth</i>        | audio data bitWidth.                    |
| <i>mode</i>            | audio data channel.                     |
| <i>saiChannel-Mask</i> | mask value of the channel to be enable. |

**33.4.5.25 void SAI\_GetRightJustifiedConfig ( sai\_transceiver\_t \* *config*,  
sai\_word\_width\_t *bitWidth*, sai\_mono\_stereo\_t *mode*, uint32\_t *saiChannelMask*  
)**

## Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>config</i>          | transceiver configurations.             |
| <i>bitWidth</i>        | audio data bitWidth.                    |
| <i>mode</i>            | audio data channel.                     |
| <i>saiChannel-Mask</i> | mask value of the channel to be enable. |

**33.4.5.26 void SAI\_GetTDMConfig ( sai\_transceiver\_t \* *config*, sai\_frame\_sync\_len\_t  
*frameSyncWidth*, sai\_word\_width\_t *bitWidth*, uint32\_t *dataWordNum*, uint32\_t  
*saiChannelMask* )**

## Parameters

|                        |                                         |
|------------------------|-----------------------------------------|
| <i>config</i>          | transceiver configurations.             |
| <i>frameSync-Width</i> | length of frame sync.                   |
| <i>bitWidth</i>        | audio data word width.                  |
| <i>dataWordNum</i>     | word number in one frame.               |
| <i>saiChannel-Mask</i> | mask value of the channel to be enable. |

**33.4.5.27 void SAI\_GetDSPConfig ( sai\_transceiver\_t \* *config*, sai\_frame\_sync\_len\_t *frameSyncWidth*, sai\_word\_width\_t *bitWidth*, sai\_mono\_stereo\_t *mode*, uint32\_t *saiChannelMask* )**

Note

DSP mode is also called PCM mode which support MODE A and MODE B, DSP/PCM MODE A configuration flow. RX is similiar but uses SAI\_RxSetConfig instead of SAI\_TxSetConfig:

```
* SAI_GetDSPConfig(config, kSAI_FrameSyncLenOneBitClk, bitWidth,
 kSAI_Stereo, channelMask)
* config->frameSync.frameSyncEarly = true;
* SAI_TxSetConfig(base, config)
*
```

DSP/PCM MODE B configuration flow for TX. RX is similiar but uses SAI\_RxSetConfig instead of SAI\_TxSetConfig:

```
* SAI_GetDSPConfig(config, kSAI_FrameSyncLenOneBitClk, bitWidth,
 kSAI_Stereo, channelMask)
* SAI_TxSetConfig(base, config)
*
```

Parameters

|                        |                                      |
|------------------------|--------------------------------------|
| <i>config</i>          | transceiver configurations.          |
| <i>frameSync-Width</i> | length of frame sync.                |
| <i>bitWidth</i>        | audio data bitWidth.                 |
| <i>mode</i>            | audio data channel.                  |
| <i>saiChannel-Mask</i> | mask value of the channel to enable. |

**33.4.5.28 static uint32\_t SAI\_TxGetStatusFlag ( I2S\_Type \* *base* ) [inline], [static]**

Parameters

|             |                  |
|-------------|------------------|
| <i>base</i> | SAI base pointer |
|-------------|------------------|

Returns

SAI Tx status flag value. Use the Status Mask to get the status value needed.

**33.4.5.29 static void SAI\_TxClearStatusFlags ( I2S\_Type \* *base*, uint32\_t *mask* ) [inline], [static]**



## Parameters

|             |                                                                                                                                                                                                            |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                                                                                           |
| <i>mask</i> | State mask. It can be a combination of the following source if defined: <ul style="list-style-type: none"> <li>• kSAI_WordStartFlag</li> <li>• kSAI_SyncErrorFlag</li> <li>• kSAI_FIFOErrorFlag</li> </ul> |

**33.4.5.30** `static uint32_t SAI_RxGetStatusFlag ( I2S_Type * base ) [inline], [static]`

## Parameters

|             |                  |
|-------------|------------------|
| <i>base</i> | SAI base pointer |
|-------------|------------------|

## Returns

SAI Rx status flag value. Use the Status Mask to get the status value needed.

**33.4.5.31** `static void SAI_RxClearStatusFlags ( I2S_Type * base, uint32_t mask ) [inline], [static]`

## Parameters

|             |                                                                                                                                                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                                                                                            |
| <i>mask</i> | State mask. It can be a combination of the following sources if defined. <ul style="list-style-type: none"> <li>• kSAI_WordStartFlag</li> <li>• kSAI_SyncErrorFlag</li> <li>• kSAI_FIFOErrorFlag</li> </ul> |

**33.4.5.32** `void SAI_TxSoftwareReset ( I2S_Type * base, sai_reset_type_t resetType )`

FIFO reset means clear all the data in the FIFO, and make the FIFO pointer both to 0. Software reset means clear the Tx internal logic, including the bit clock, frame count etc. But software reset will not clear any configuration registers like TCR1~TCR5. This function will also clear all the error flags such as FIFO error, sync error etc.

## Parameters

|                  |                                          |
|------------------|------------------------------------------|
| <i>base</i>      | SAI base pointer                         |
| <i>resetType</i> | Reset type, FIFO reset or software reset |

**33.4.5.33 void SAI\_RxSoftwareReset ( I2S\_Type \* *base*, sai\_reset\_type\_t *resetType* )**

FIFO reset means clear all the data in the FIFO, and make the FIFO pointer both to 0. Software reset means clear the Rx internal logic, including the bit clock, frame count etc. But software reset will not clear any configuration registers like RCR1~RCR5. This function will also clear all the error flags such as FIFO error, sync error etc.

## Parameters

|                  |                                          |
|------------------|------------------------------------------|
| <i>base</i>      | SAI base pointer                         |
| <i>resetType</i> | Reset type, FIFO reset or software reset |

**33.4.5.34 void SAI\_TxSetChannelFIFOMask ( I2S\_Type \* *base*, uint8\_t *mask* )**

## Parameters

|             |                                                                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                 |
| <i>mask</i> | Channel enable mask, 0 means all channel FIFO disabled, 1 means channel 0 enabled, 3 means both channel 0 and channel 1 enabled. |

**33.4.5.35 void SAI\_RxSetChannelFIFOMask ( I2S\_Type \* *base*, uint8\_t *mask* )**

## Parameters

|             |                                                                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                 |
| <i>mask</i> | Channel enable mask, 0 means all channel FIFO disabled, 1 means channel 0 enabled, 3 means both channel 0 and channel 1 enabled. |

**33.4.5.36 void SAI\_TxSetDataOrder ( I2S\_Type \* *base*, sai\_data\_order\_t *order* )**

Parameters

|              |                       |
|--------------|-----------------------|
| <i>base</i>  | SAI base pointer      |
| <i>order</i> | Data order MSB or LSB |

**33.4.5.37 void SAI\_RxSetDataOrder ( I2S\_Type \* *base*, sai\_data\_order\_t *order* )**

Parameters

|              |                       |
|--------------|-----------------------|
| <i>base</i>  | SAI base pointer      |
| <i>order</i> | Data order MSB or LSB |

**33.4.5.38 void SAI\_TxSetBitClockPolarity ( I2S\_Type \* *base*, sai\_clock\_polarity\_t *polarity* )**

Parameters

|                 |                  |
|-----------------|------------------|
| <i>base</i>     | SAI base pointer |
| <i>polarity</i> |                  |

**33.4.5.39 void SAI\_RxSetBitClockPolarity ( I2S\_Type \* *base*, sai\_clock\_polarity\_t *polarity* )**

Parameters

|                 |                  |
|-----------------|------------------|
| <i>base</i>     | SAI base pointer |
| <i>polarity</i> |                  |

**33.4.5.40 void SAI\_TxSetFrameSyncPolarity ( I2S\_Type \* *base*, sai\_clock\_polarity\_t *polarity* )**

Parameters

---

|                 |                  |
|-----------------|------------------|
| <i>base</i>     | SAI base pointer |
| <i>polarity</i> |                  |

#### 33.4.5.41 void SAI\_RxSetFrameSyncPolarity ( I2S\_Type \* *base*, sai\_clock\_polarity\_t *polarity* )

Parameters

|                 |                  |
|-----------------|------------------|
| <i>base</i>     | SAI base pointer |
| <i>polarity</i> |                  |

#### 33.4.5.42 void SAI\_TxSetFIFOPacking ( I2S\_Type \* *base*, sai\_fifo\_packing\_t *pack* )

Parameters

|             |                                                      |
|-------------|------------------------------------------------------|
| <i>base</i> | SAI base pointer.                                    |
| <i>pack</i> | FIFO pack type. It is element of sai_fifo_packing_t. |

#### 33.4.5.43 void SAI\_RxSetFIFOPacking ( I2S\_Type \* *base*, sai\_fifo\_packing\_t *pack* )

Parameters

|             |                                                      |
|-------------|------------------------------------------------------|
| <i>base</i> | SAI base pointer.                                    |
| <i>pack</i> | FIFO pack type. It is element of sai_fifo_packing_t. |

#### 33.4.5.44 static void SAI\_TxSetFIFOErrorContinue ( I2S\_Type \* *base*, bool *isEnabled* ) [inline], [static]

FIFO error continue mode means SAI will keep running while FIFO error occurred. If this feature not enabled, SAI will hang and users need to clear FEF flag in TCSR register.

Parameters

|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| <i>base</i>      | SAI base pointer.                                                       |
| <i>isEnabled</i> | Is FIFO error continue enabled, true means enable, false means disable. |

**33.4.5.45** `static void SAI_RxSetFIFOErrorContinue ( I2S_Type * base, bool isEnabled )`  
`[inline], [static]`

FIFO error continue mode means SAI will keep running while FIFO error occurred. If this feature not enabled, SAI will hang and users need to clear FEF flag in RCSR register.

## Parameters

|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| <i>base</i>      | SAI base pointer.                                                       |
| <i>isEnabled</i> | Is FIFO error continue enabled, true means enable, false means disable. |

#### 33.4.5.46 static void SAI\_TxEnableInterrupts ( I2S\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                                                                                                                                                                                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                                                                                                                                                                                                                                   |
| <i>mask</i> | interrupt source The parameter can be a combination of the following sources if defined. <ul style="list-style-type: none"> <li>• kSAI_WordStartInterruptEnable</li> <li>• kSAI_SyncErrorInterruptEnable</li> <li>• kSAI_FIFOWarningInterruptEnable</li> <li>• kSAI_FIFORequestInterruptEnable</li> <li>• kSAI_FIFOErrorInterruptEnable</li> </ul> |

#### 33.4.5.47 static void SAI\_RxEnableInterrupts ( I2S\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                                                                                                                                                                                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                                                                                                                                                                                                                                   |
| <i>mask</i> | interrupt source The parameter can be a combination of the following sources if defined. <ul style="list-style-type: none"> <li>• kSAI_WordStartInterruptEnable</li> <li>• kSAI_SyncErrorInterruptEnable</li> <li>• kSAI_FIFOWarningInterruptEnable</li> <li>• kSAI_FIFORequestInterruptEnable</li> <li>• kSAI_FIFOErrorInterruptEnable</li> </ul> |

#### 33.4.5.48 static void SAI\_TxDisableInterrupts ( I2S\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                                                                                                                                                                                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                                                                                                                                                                                                                                   |
| <i>mask</i> | interrupt source The parameter can be a combination of the following sources if defined. <ul style="list-style-type: none"> <li>• kSAI_WordStartInterruptEnable</li> <li>• kSAI_SyncErrorInterruptEnable</li> <li>• kSAI_FIFOWarningInterruptEnable</li> <li>• kSAI_FIFORequestInterruptEnable</li> <li>• kSAI_FIFOErrorInterruptEnable</li> </ul> |

#### 33.4.5.49 static void SAI\_RxDisableInterrupts ( I2S\_Type \* *base*, uint32\_t *mask* ) [inline], [static]

## Parameters

|             |                                                                                                                                                                                                                                                                                                                                                    |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                                                                                                                                                                                                                                   |
| <i>mask</i> | interrupt source The parameter can be a combination of the following sources if defined. <ul style="list-style-type: none"> <li>• kSAI_WordStartInterruptEnable</li> <li>• kSAI_SyncErrorInterruptEnable</li> <li>• kSAI_FIFOWarningInterruptEnable</li> <li>• kSAI_FIFORequestInterruptEnable</li> <li>• kSAI_FIFOErrorInterruptEnable</li> </ul> |

#### 33.4.5.50 static void SAI\_TxEnableDMA ( I2S\_Type \* *base*, uint32\_t *mask*, bool *enable* ) [inline], [static]

## Parameters

|             |                                                                                                                                                                                                     |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | SAI base pointer                                                                                                                                                                                    |
| <i>mask</i> | DMA source The parameter can be combination of the following sources if defined. <ul style="list-style-type: none"> <li>• kSAI_FIFOWarningDMAEnable</li> <li>• kSAI_FIFORequestDMAEnable</li> </ul> |

|               |                                                 |
|---------------|-------------------------------------------------|
| <i>enable</i> | True means enable DMA, false means disable DMA. |
|---------------|-------------------------------------------------|

#### 33.4.5.51 static void SAI\_RxEnableDMA ( I2S\_Type \* *base*, uint32\_t *mask*, bool *enable* ) [inline], [static]

Parameters

|               |                                                                                                                                                                                                       |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer                                                                                                                                                                                      |
| <i>mask</i>   | DMA source The parameter can be a combination of the following sources if defined. <ul style="list-style-type: none"> <li>• kSAI_FIFOWarningDMAEnable</li> <li>• kSAI_FIFORequestDMAEnable</li> </ul> |
| <i>enable</i> | True means enable DMA, false means disable DMA.                                                                                                                                                       |

#### 33.4.5.52 static uintptr\_t SAI\_TxGetDataRegisterAddress ( I2S\_Type \* *base*, uint32\_t *channel* ) [inline], [static]

This API is used to provide a transfer address for the SAI DMA transfer configuration.

Parameters

|                |                          |
|----------------|--------------------------|
| <i>base</i>    | SAI base pointer.        |
| <i>channel</i> | Which data channel used. |

Returns

data register address.

#### 33.4.5.53 static uintptr\_t SAI\_RxGetDataRegisterAddress ( I2S\_Type \* *base*, uint32\_t *channel* ) [inline], [static]

This API is used to provide a transfer address for the SAI DMA transfer configuration.

Parameters

|                |                          |
|----------------|--------------------------|
| <i>base</i>    | SAI base pointer.        |
| <i>channel</i> | Which data channel used. |



## Returns

data register address.

**33.4.5.54 void SAI\_WriteBlocking ( I2S\_Type \* *base*, uint32\_t *channel*, uint32\_t *bitWidth*, uint8\_t \* *buffer*, uint32\_t *size* )**

## Note

This function blocks by polling until data is ready to be sent.

## Parameters

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>base</i>     | SAI base pointer.                                        |
| <i>channel</i>  | Data channel used.                                       |
| <i>bitWidth</i> | How many bits in an audio word; usually 8/16/24/32 bits. |
| <i>buffer</i>   | Pointer to the data to be written.                       |
| <i>size</i>     | Bytes to be written.                                     |

**33.4.5.55 void SAI\_WriteMultiChannelBlocking ( I2S\_Type \* *base*, uint32\_t *channel*, uint32\_t *channelMask*, uint32\_t *bitWidth*, uint8\_t \* *buffer*, uint32\_t *size* )**

## Note

This function blocks by polling until data is ready to be sent.

## Parameters

|                    |                                                          |
|--------------------|----------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                        |
| <i>channel</i>     | Data channel used.                                       |
| <i>channelMask</i> | channel mask.                                            |
| <i>bitWidth</i>    | How many bits in an audio word; usually 8/16/24/32 bits. |
| <i>buffer</i>      | Pointer to the data to be written.                       |
| <i>size</i>        | Bytes to be written.                                     |

**33.4.5.56 static void SAI\_WriteData ( I2S\_Type \* *base*, uint32\_t *channel*, uint32\_t *data* )  
[inline], [static]**

## Parameters

|                |                           |
|----------------|---------------------------|
| <i>base</i>    | SAI base pointer.         |
| <i>channel</i> | Data channel used.        |
| <i>data</i>    | Data needs to be written. |

**33.4.5.57 void SAI\_ReadBlocking ( I2S\_Type \* *base*, uint32\_t *channel*, uint32\_t *bitWidth*, uint8\_t \* *buffer*, uint32\_t *size* )**

## Note

This function blocks by polling until data is ready to be sent.

## Parameters

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>base</i>     | SAI base pointer.                                        |
| <i>channel</i>  | Data channel used.                                       |
| <i>bitWidth</i> | How many bits in an audio word; usually 8/16/24/32 bits. |
| <i>buffer</i>   | Pointer to the data to be read.                          |
| <i>size</i>     | Bytes to be read.                                        |

**33.4.5.58 void SAI\_ReadMultiChannelBlocking ( I2S\_Type \* *base*, uint32\_t *channel*, uint32\_t *channelMask*, uint32\_t *bitWidth*, uint8\_t \* *buffer*, uint32\_t *size* )**

## Note

This function blocks by polling until data is ready to be sent.

## Parameters

|                    |                                                          |
|--------------------|----------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                        |
| <i>channel</i>     | Data channel used.                                       |
| <i>channelMask</i> | channel mask.                                            |
| <i>bitWidth</i>    | How many bits in an audio word; usually 8/16/24/32 bits. |
| <i>buffer</i>      | Pointer to the data to be read.                          |
| <i>size</i>        | Bytes to be read.                                        |

**33.4.5.59** `static uint32_t SAI_ReadData ( I2S_Type * base, uint32_t channel )`  
`[inline], [static]`

## Parameters

|                |                    |
|----------------|--------------------|
| <i>base</i>    | SAI base pointer.  |
| <i>channel</i> | Data channel used. |

## Returns

Data in SAI FIFO.

### 33.4.5.60 void SAI\_TransferTxCreateHandle ( I2S\_Type \* *base*, sai\_handle\_t \* *handle*, sai\_transfer\_callback\_t *callback*, void \* *userData* )

This function initializes the Tx handle for the SAI Tx transactional APIs. Call this function once to get the handle initialized.

## Parameters

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>base</i>     | SAI base pointer                               |
| <i>handle</i>   | SAI handle pointer.                            |
| <i>callback</i> | Pointer to the user callback function.         |
| <i>userData</i> | User parameter passed to the callback function |

### 33.4.5.61 void SAI\_TransferRxCreateHandle ( I2S\_Type \* *base*, sai\_handle\_t \* *handle*, sai\_transfer\_callback\_t *callback*, void \* *userData* )

This function initializes the Rx handle for the SAI Rx transactional APIs. Call this function once to get the handle initialized.

## Parameters

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>base</i>     | SAI base pointer.                               |
| <i>handle</i>   | SAI handle pointer.                             |
| <i>callback</i> | Pointer to the user callback function.          |
| <i>userData</i> | User parameter passed to the callback function. |

### 33.4.5.62 void SAI\_TransferTxSetConfig ( I2S\_Type \* *base*, sai\_handle\_t \* *handle*, sai\_transceiver\_t \* *config* )

This function initializes the Tx, include bit clock, frame sync, master clock, serial data and fifo configurations.

## Parameters

|               |                             |
|---------------|-----------------------------|
| <i>base</i>   | SAI base pointer.           |
| <i>handle</i> | SAI handle pointer.         |
| <i>config</i> | transmitter configurations. |

### 33.4.5.63 void SAI\_TransferRxSetConfig ( I2S\_Type \* *base*, sai\_handle\_t \* *handle*, sai\_transceiver\_t \* *config* )

This function initializes the Rx, include bit clock, frame sync, master clock, serial data and fifo configurations.

## Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>handle</i> | SAI handle pointer.      |
| <i>config</i> | receiver configurations. |

### 33.4.5.64 status\_t SAI\_TransferSendNonBlocking ( I2S\_Type \* *base*, sai\_handle\_t \* *handle*, sai\_transfer\_t \* *xfer* )

## Note

This API returns immediately after the transfer initiates. Call the SAI\_TxGetTransferStatusIRQ to poll the transfer status and check whether the transfer is finished. If the return status is not kStatus\_SAI\_Busy, the transfer is finished.

## Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer.                                                      |
| <i>handle</i> | Pointer to the sai_handle_t structure which stores the transfer state. |
| <i>xfer</i>   | Pointer to the sai_transfer_t structure.                               |

## Return values

|                        |                                        |
|------------------------|----------------------------------------|
| <i>kStatus_Success</i> | Successfully started the data receive. |
|------------------------|----------------------------------------|

|                                |                                      |
|--------------------------------|--------------------------------------|
| <i>kStatus_SAI_TxBusy</i>      | Previous receive still not finished. |
| <i>kStatus_InvalidArgument</i> | The input parameter is invalid.      |

#### 33.4.5.65 **status\_t SAI\_TransferReceiveNonBlocking ( I2S\_Type \* *base*, sai\_handle\_t \* *handle*, sai\_transfer\_t \* *xfer* )**

##### Note

This API returns immediately after the transfer initiates. Call the SAI\_RxGetTransferStatusIRQ to poll the transfer status and check whether the transfer is finished. If the return status is not kStatus\_SAI\_Busy, the transfer is finished.

##### Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer                                                       |
| <i>handle</i> | Pointer to the sai_handle_t structure which stores the transfer state. |
| <i>xfer</i>   | Pointer to the sai_transfer_t structure.                               |

##### Return values

|                                |                                        |
|--------------------------------|----------------------------------------|
| <i>kStatus_Success</i>         | Successfully started the data receive. |
| <i>kStatus_SAI_RxBusy</i>      | Previous receive still not finished.   |
| <i>kStatus_InvalidArgument</i> | The input parameter is invalid.        |

#### 33.4.5.66 **status\_t SAI\_TransferGetSendCount ( I2S\_Type \* *base*, sai\_handle\_t \* *handle*, size\_t \* *count* )**

##### Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer.                                                      |
| <i>handle</i> | Pointer to the sai_handle_t structure which stores the transfer state. |
| <i>count</i>  | Bytes count sent.                                                      |

##### Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

**33.4.5.67** `status_t SAI_TransferGetReceiveCount ( I2S_Type * base, sai_handle_t *  
handle, size_t * count )`

## Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer.                                                      |
| <i>handle</i> | Pointer to the sai_handle_t structure which stores the transfer state. |
| <i>count</i>  | Bytes count received.                                                  |

## Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

**33.4.5.68 void SAI\_TransferAbortSend ( I2S\_Type \* *base*, sai\_handle\_t \* *handle* )**

## Note

This API can be called any time when an interrupt non-blocking transfer initiates to abort the transfer early.

## Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer.                                                      |
| <i>handle</i> | Pointer to the sai_handle_t structure which stores the transfer state. |

**33.4.5.69 void SAI\_TransferAbortReceive ( I2S\_Type \* *base*, sai\_handle\_t \* *handle* )**

## Note

This API can be called when an interrupt non-blocking transfer initiates to abort the transfer early.

## Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer                                                       |
| <i>handle</i> | Pointer to the sai_handle_t structure which stores the transfer state. |

**33.4.5.70 void SAI\_TransferTerminateSend ( I2S\_Type \* *base*, sai\_handle\_t \* *handle* )**

This function will clear all transfer slots buffered in the sai queue. If users only want to abort the current transfer slot, please call SAI\_TransferAbortSend.



## Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>handle</i> | SAI eDMA handle pointer. |

**33.4.5.71 void SAI\_TransferTerminateReceive ( I2S\_Type \* *base*, sai\_handle\_t \* *handle* )**

This function will clear all transfer slots buffered in the sai queue. If users only want to abort the current transfer slot, please call SAI\_TransferAbortReceive.

## Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>handle</i> | SAI eDMA handle pointer. |

**33.4.5.72 void SAI\_TransferTxHandleIRQ ( I2S\_Type \* *base*, sai\_handle\_t \* *handle* )**

## Parameters

|               |                                        |
|---------------|----------------------------------------|
| <i>base</i>   | SAI base pointer.                      |
| <i>handle</i> | Pointer to the sai_handle_t structure. |

**33.4.5.73 void SAI\_TransferRxHandleIRQ ( I2S\_Type \* *base*, sai\_handle\_t \* *handle* )**

## Parameters

|               |                                        |
|---------------|----------------------------------------|
| <i>base</i>   | SAI base pointer.                      |
| <i>handle</i> | Pointer to the sai_handle_t structure. |

## Chapter 34

# SEMA42: Hardware Semaphores Driver

### 34.1 Overview

The MCUXpresso SDK provides a driver for the SEMA42 module of MCUXpresso SDK devices.

The SEMA42 driver is used for multicore platforms. Before using the SEMA42, call the [SEMA42\\_Init\(\)](#) function to initialize the module. Note that this function only enables the clock but does not reset the gates because the module might be used by other processors at the same time. To reset the gates, call either the [SEMA42\\_ResetGate\(\)](#) or [SEMA42\\_ResetAllGates\(\)](#) functions. The function [SEMA42\\_Deinit\(\)](#) deinitializes the SEMA42.

The SEMA42 provides two functions to lock the SEMA42 gate. The function [SEMA42\\_TryLock\(\)](#) tries to lock the gate. If the gate has been locked by another processor, this function returns an error immediately. The function [SEMA42\\_Lock\(\)](#) is a blocking method, which waits until the gate is free and locks it.

The [SEMA42\\_Unlock\(\)](#) unlocks the SEMA42 gate. The gate can only be unlocked by the processor which locked it. If the gate is not locked by the current processor, this function takes no effect. The function [SEMA42\\_GetGateStatus\(\)](#) returns a status whether the gate is unlocked and which processor locks the gate.

The SEMA42 gate can be reset to unlock forcefully. The function [SEMA42\\_ResetGate\(\)](#) resets a specific gate. The function [SEMA42\\_ResetAllGates\(\)](#) resets all gates.

### 34.2 Typical use case

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/sema42

### Macros

- #define [SEMA42\\_GATE\\_NUM\\_RESET\\_ALL](#) (64U)  
*The number to reset all SEMA42 gates.*
- #define [SEMA42\\_GATEn](#)(base, n) (((volatile uint8\_t \*)(&((base)->GATE3)))[(n) ^ 3U])  
*SEMA42 gate n register address.*

### Typedefs

- typedef enum [\\_sema42\\_gate\\_status](#) [sema42\\_gate\\_status\\_t](#)  
*SEMA42 gate lock status.*

### Enumerations

- enum {  
    [kStatus\\_SEMA42\\_Busy](#) = MAKE\_STATUS(kStatusGroup\_SEMA42, 0),  
    [kStatus\\_SEMA42\\_Reseting](#) = MAKE\_STATUS(kStatusGroup\_SEMA42, 1) }

*SEMA42 status return codes.*

- enum `_sema42_gate_status` {  
`kSEMA42_Unlocked` = 0U,  
`kSEMA42_LockedByProc0` = 1U,  
`kSEMA42_LockedByProc1` = 2U,  
`kSEMA42_LockedByProc2` = 3U,  
`kSEMA42_LockedByProc3` = 4U,  
`kSEMA42_LockedByProc4` = 5U,  
`kSEMA42_LockedByProc5` = 6U,  
`kSEMA42_LockedByProc6` = 7U,  
`kSEMA42_LockedByProc7` = 8U,  
`kSEMA42_LockedByProc8` = 9U,  
`kSEMA42_LockedByProc9` = 10U,  
`kSEMA42_LockedByProc10` = 11U,  
`kSEMA42_LockedByProc11` = 12U,  
`kSEMA42_LockedByProc12` = 13U,  
`kSEMA42_LockedByProc13` = 14U,  
`kSEMA42_LockedByProc14` = 15U }

*SEMA42 gate lock status.*

## Functions

- void `SEMA42_Init` (SEMA42\_Type \*base)  
*Initializes the SEMA42 module.*
- void `SEMA42_Deinit` (SEMA42\_Type \*base)  
*De-initializes the SEMA42 module.*
- `status_t` `SEMA42_TryLock` (SEMA42\_Type \*base, uint8\_t gateNum, uint8\_t procNum)  
*Tries to lock the SEMA42 gate.*
- void `SEMA42_Lock` (SEMA42\_Type \*base, uint8\_t gateNum, uint8\_t procNum)  
*Locks the SEMA42 gate.*
- static void `SEMA42_Unlock` (SEMA42\_Type \*base, uint8\_t gateNum)  
*Unlocks the SEMA42 gate.*
- static `sema42_gate_status_t` `SEMA42_GetGateStatus` (SEMA42\_Type \*base, uint8\_t gateNum)  
*Gets the status of the SEMA42 gate.*
- `status_t` `SEMA42_ResetGate` (SEMA42\_Type \*base, uint8\_t gateNum)  
*Resets the SEMA42 gate to an unlocked status.*
- static `status_t` `SEMA42_ResetAllGates` (SEMA42\_Type \*base)  
*Resets all SEMA42 gates to an unlocked status.*

## Driver version

- #define `FSL_SEMA42_DRIVER_VERSION` (MAKE\_VERSION(2, 0, 4))  
*SEMA42 driver version.*

### 34.3 Macro Definition Documentation

#### 34.3.1 #define SEMA42\_GATE\_NUM\_RESET\_ALL (64U)

#### 34.3.2 #define SEMA42\_GATEn( base, n ) (((volatile uint8\_t \*)(&((base)->GATE3)))[(n) ^ 3U])

The SEMA42 gates are sorted in the order 3, 2, 1, 0, 7, 6, 5, 4, ... not in the order 0, 1, 2, 3, 4, 5, 6, 7, ... The macro SEMA42\_GATEn gets the SEMA42 gate based on the gate index.

The input gate index is XOR'ed with 3U:  $0 \wedge 3 = 3$   $1 \wedge 3 = 2$   $2 \wedge 3 = 1$   $3 \wedge 3 = 0$   $4 \wedge 3 = 7$   $5 \wedge 3 = 6$   $6 \wedge 3 = 5$   $7 \wedge 3 = 4$  ...

### 34.4 Enumeration Type Documentation

#### 34.4.1 anonymous enum

Enumerator

*kStatus\_SEMA42\_Busy* SEMA42 gate has been locked by other processor.

*kStatus\_SEMA42\_Reseting* SEMA42 gate reseting is ongoing.

#### 34.4.2 enum \_sema42\_gate\_status

Enumerator

*kSEMA42\_Unlocked* The gate is unlocked.

*kSEMA42\_LockedByProc0* The gate is locked by processor 0.

*kSEMA42\_LockedByProc1* The gate is locked by processor 1.

*kSEMA42\_LockedByProc2* The gate is locked by processor 2.

*kSEMA42\_LockedByProc3* The gate is locked by processor 3.

*kSEMA42\_LockedByProc4* The gate is locked by processor 4.

*kSEMA42\_LockedByProc5* The gate is locked by processor 5.

*kSEMA42\_LockedByProc6* The gate is locked by processor 6.

*kSEMA42\_LockedByProc7* The gate is locked by processor 7.

*kSEMA42\_LockedByProc8* The gate is locked by processor 8.

*kSEMA42\_LockedByProc9* The gate is locked by processor 9.

*kSEMA42\_LockedByProc10* The gate is locked by processor 10.

*kSEMA42\_LockedByProc11* The gate is locked by processor 11.

*kSEMA42\_LockedByProc12* The gate is locked by processor 12.

*kSEMA42\_LockedByProc13* The gate is locked by processor 13.

*kSEMA42\_LockedByProc14* The gate is locked by processor 14.

## 34.5 Function Documentation

### 34.5.1 void SEMA42\_Init ( SEMA42\_Type \* *base* )

This function initializes the SEMA42 module. It only enables the clock but does not reset the gates because the module might be used by other processors at the same time. To reset the gates, call either SEMA42\_ResetGate or SEMA42\_ResetAllGates function.

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | SEMA42 peripheral base address. |
|-------------|---------------------------------|

### 34.5.2 void SEMA42\_Deinit ( SEMA42\_Type \* *base* )

This function de-initializes the SEMA42 module. It only disables the clock.

Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | SEMA42 peripheral base address. |
|-------------|---------------------------------|

### 34.5.3 status\_t SEMA42\_TryLock ( SEMA42\_Type \* *base*, uint8\_t *gateNum*, uint8\_t *procNum* )

This function tries to lock the specific SEMA42 gate. If the gate has been locked by another processor, this function returns an error code.

Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | SEMA42 peripheral base address. |
| <i>gateNum</i> | Gate number to lock.            |
| <i>procNum</i> | Current processor number.       |

Return values

|                            |                                                   |
|----------------------------|---------------------------------------------------|
| <i>kStatus_Success</i>     | Lock the sema42 gate successfully.                |
| <i>kStatus_SEMA42_Busy</i> | Sema42 gate has been locked by another processor. |

#### 34.5.4 void SEMA42\_Lock ( SEMA42\_Type \* *base*, uint8\_t *gateNum*, uint8\_t *procNum* )

This function locks the specific SEMA42 gate. If the gate has been locked by other processors, this function waits until it is unlocked and then lock it.

## Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | SEMA42 peripheral base address. |
| <i>gateNum</i> | Gate number to lock.            |
| <i>procNum</i> | Current processor number.       |

### 34.5.5 static void SEMA42\_Unlock ( SEMA42\_Type \* *base*, uint8\_t *gateNum* ) [inline], [static]

This function unlocks the specific SEMA42 gate. It only writes unlock value to the SEMA42 gate register. However, it does not check whether the SEMA42 gate is locked by the current processor or not. As a result, if the SEMA42 gate is not locked by the current processor, this function has no effect.

## Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | SEMA42 peripheral base address. |
| <i>gateNum</i> | Gate number to unlock.          |

### 34.5.6 static sema42\_gate\_status\_t SEMA42\_GetGateStatus ( SEMA42\_Type \* *base*, uint8\_t *gateNum* ) [inline], [static]

This function checks the lock status of a specific SEMA42 gate.

## Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | SEMA42 peripheral base address. |
| <i>gateNum</i> | Gate number.                    |

## Returns

status Current status.

### 34.5.7 status\_t SEMA42\_ResetGate ( SEMA42\_Type \* *base*, uint8\_t *gateNum* )

This function resets a SEMA42 gate to an unlocked status.

## Parameters

|                |                                 |
|----------------|---------------------------------|
| <i>base</i>    | SEMA42 peripheral base address. |
| <i>gateNum</i> | Gate number.                    |

## Return values

|                                 |                                      |
|---------------------------------|--------------------------------------|
| <i>kStatus_Success</i>          | SEMA42 gate is reset successfully.   |
| <i>kStatus_SEMA42_-Reseting</i> | Some other reset process is ongoing. |

### 34.5.8 static status\_t SEMA42\_ResetAllGates ( SEMA42\_Type \* *base* ) [inline], [static]

This function resets all SEMA42 gate to an unlocked status.

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | SEMA42 peripheral base address. |
|-------------|---------------------------------|

## Return values

|                                 |                                      |
|---------------------------------|--------------------------------------|
| <i>kStatus_Success</i>          | SEMA42 is reset successfully.        |
| <i>kStatus_SEMA42_-Reseting</i> | Some other reset process is ongoing. |



## Chapter 35

# TPM: Timer PWM Module

### 35.1 Overview

The MCUXpresso SDK provides a driver for the Timer PWM Module (TPM) of MCUXpresso SDK devices.

The TPM driver supports the generation of PWM signals, input capture, and output compare modes. On some SoCs, the driver supports the generation of combined PWM signals, dual-edge capture, and quadrature decoder modes. The driver also supports configuring each of the TPM fault inputs. The fault input is available only on some SoCs.

### 35.2 Introduction of TPM

#### 35.2.1 Initialization and deinitialization

The function [TPM\\_Init\(\)](#) initializes the TPM with a specified configurations. The function [TPM\\_GetDefaultConfig\(\)](#) gets the default configurations. On some SoCs, the initialization function issues a software reset to reset the TPM internal logic. The initialization function configures the TPM's behavior when it receives a trigger input and its operation in doze and debug modes.

The function [TPM\\_Deinit\(\)](#) disables the TPM counter and turns off the module clock.

#### 35.2.2 PWM Operations

The function [TPM\\_SetupPwm\(\)](#) sets up TPM channels for the PWM output. The function can set up the PWM signal properties for multiple channels. Each channel has its own `tpm_chnl_pwm_signal_param_t` structure that is used to specify the output signals duty cycle and level-mode. However, the same PWM period and PWM mode is applied to all channels requesting a PWM output. The signal duty cycle is provided as a percentage of the PWM period. Its value should be between 0 and 100 where 0=inactive signal (0% duty cycle) and 100=always active signal (100% duty cycle). When generating a combined PWM signal, the channel number passed refers to a channel pair number, for example 0 refers to channel 0 and 1, 1 refers to channels 2 and 3.

The function [TPM\\_UpdatePwmDutycycle\(\)](#) updates the PWM signal duty cycle of a particular TPM channel.

The function [TPM\\_UpdateChnlEdgeLevelSelect\(\)](#) updates the level select bits of a particular TPM channel. This can be used to disable the PWM output when making changes to the PWM signal.

### 35.2.3 Input capture operations

The function `TPM_SetupInputCapture()` sets up a TPM channel for input capture. The user can specify the capture edge.

The function `TPM_SetupDualEdgeCapture()` can be used to measure the pulse width of a signal. This is available only for certain SoCs. A channel pair is used during the capture with the input signal coming through a channel that can be configured. The user can specify the capture edge for each channel and any filter value to be used when processing the input signal.

### 35.2.4 Output compare operations

The function `TPM_SetupOutputCompare()` sets up a TPM channel for output comparison. The user can specify the channel output on a successful comparison and a comparison value.

### 35.2.5 Quad decode

The function `TPM_SetupQuadDecode()` sets up TPM channels 0 and 1 for quad decode, which is available only for certain SoCs. The user can specify the quad decode mode, polarity, and filter properties for each input signal.

### 35.2.6 Fault operation

The function `TPM_SetupFault()` sets up the properties for each fault, which is available only for certain SoCs. The user can specify the fault polarity and whether to use a filter on a fault input. The overall fault filter value and fault control mode are set up during initialization.

### 35.2.7 Status

Provides functions to get and clear the TPM status.

### 35.2.8 Interrupt

Provides functions to enable/disable TPM interrupts and get current enabled interrupts.

## 35.3 Typical use case

### 35.3.1 PWM output

Output the PWM signal on 2 TPM channels with different duty cycles. Periodically update the PWM signal duty cycle. Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/tpm

## Data Structures

- struct [\\_tpm\\_chnl\\_pwm\\_signal\\_param](#)  
*Options to configure a TPM channel's PWM signal. [More...](#)*
- struct [\\_tpm\\_dual\\_edge\\_capture\\_param](#)  
*TPM dual edge capture parameters. [More...](#)*
- struct [\\_tpm\\_phase\\_param](#)  
*TPM quadrature decode phase parameters. [More...](#)*
- struct [\\_tpm\\_config](#)  
*TPM config structure. [More...](#)*

## Macros

- #define [TPM\\_MAX\\_COUNTER\\_VALUE\(x\)](#) ((1U != (uint8\_t)FSL\_FEATURE\_TPM\_HAS\_32BIT\_COUNTERn(x)) ? 0xFFFFFU : 0xFFFFFFFFFU)  
*Help macro to get the max counter value.*

## Typedefs

- typedef enum [\\_tpm\\_chnl\\_tpm\\_chnl\\_t](#)  
*List of TPM channels.*
- typedef enum [\\_tpm\\_pwm\\_mode\\_tpm\\_pwm\\_mode\\_t](#)  
*TPM PWM operation modes.*
- typedef enum [\\_tpm\\_pwm\\_level\\_select\\_tpm\\_pwm\\_level\\_select\\_t](#)  
*TPM PWM output pulse mode: high-true, low-true or no output.*
- typedef enum [\\_tpm\\_pwm\\_pause\\_level\\_select\\_tpm\\_pwm\\_pause\\_level\\_select\\_t](#)  
*TPM PWM output when first enabled or paused: set or clear.*
- typedef enum [\\_tpm\\_chnl\\_control\\_bit\\_mask\\_tpm\\_chnl\\_control\\_bit\\_mask\\_t](#)  
*List of TPM channel modes and level control bit mask.*
- typedef struct [\\_tpm\\_chnl\\_pwm\\_signal\\_param\\_tpm\\_chnl\\_pwm\\_signal\\_param\\_t](#)  
*Options to configure a TPM channel's PWM signal.*
- typedef enum [\\_tpm\\_trigger\\_select\\_tpm\\_trigger\\_select\\_t](#)  
*Trigger sources available.*
- typedef enum [\\_tpm\\_trigger\\_source\\_tpm\\_trigger\\_source\\_t](#)  
*Trigger source options available.*
- typedef enum [\\_tpm\\_ext\\_trigger\\_polarity\\_tpm\\_ext\\_trigger\\_polarity\\_t](#)  
*External trigger source polarity.*

- typedef enum  
[\\_tpm\\_output\\_compare\\_mode](#) tpm\_output\_compare\_mode\_t  
*TPM output compare modes.*
- typedef enum  
[\\_tpm\\_input\\_capture\\_edge](#) tpm\_input\_capture\_edge\_t  
*TPM input capture edge.*
- typedef struct  
[\\_tpm\\_dual\\_edge\\_capture\\_param](#) tpm\_dual\_edge\_capture\_param\_t  
*TPM dual edge capture parameters.*
- typedef enum [\\_tpm\\_quad\\_decode\\_mode](#) tpm\_quad\_decode\_mode\_t  
*TPM quadrature decode modes.*
- typedef enum [\\_tpm\\_phase\\_polarity](#) tpm\_phase\_polarity\_t  
*TPM quadrature phase polarities.*
- typedef struct [\\_tpm\\_phase\\_param](#) tpm\_phase\_params\_t  
*TPM quadrature decode phase parameters.*
- typedef enum [\\_tpm\\_clock\\_source](#) tpm\_clock\_source\_t  
*TPM clock source selection.*
- typedef enum [\\_tpm\\_clock\\_prescale](#) tpm\_clock\_prescale\_t  
*TPM prescale value selection for the clock source.*
- typedef struct [\\_tpm\\_config](#) tpm\_config\_t  
*TPM config structure.*
- typedef enum [\\_tpm\\_interrupt\\_enable](#) tpm\_interrupt\_enable\_t  
*List of TPM interrupts.*
- typedef enum [\\_tpm\\_status\\_flags](#) tpm\_status\_flags\_t  
*List of TPM flags.*
- typedef void(\* [tpm\\_callback\\_t](#))(TPM\_Type \*base)  
*TPM callback function pointer.*

## Enumerations

- enum [\\_tpm\\_chnl](#) {  
[kTPM\\_Chnl\\_0](#) = 0U,  
[kTPM\\_Chnl\\_1](#),  
[kTPM\\_Chnl\\_2](#),  
[kTPM\\_Chnl\\_3](#),  
[kTPM\\_Chnl\\_4](#),  
[kTPM\\_Chnl\\_5](#),  
[kTPM\\_Chnl\\_6](#),  
[kTPM\\_Chnl\\_7](#) }  
*List of TPM channels.*
- enum [\\_tpm\\_pwm\\_mode](#) {  
[kTPM\\_EdgeAlignedPwm](#) = 0U,  
[kTPM\\_CenterAlignedPwm](#),  
[kTPM\\_CombinedPwm](#) }  
*TPM PWM operation modes.*
- enum [\\_tpm\\_pwm\\_level\\_select](#) {  
[kTPM\\_HighTrue](#) = 0U,  
[kTPM\\_LowTrue](#) }  
*TPM PWM output pulse mode: high-true, low-true or no output.*

- enum `_tpm_pwm_pause_level_select` {  
`kTPM_ClearOnPause` = 0U,  
`kTPM_SetOnPause` }  
*TPM PWM output when first enabled or paused: set or clear.*
- enum `_tpm_chnl_control_bit_mask` {  
`kTPM_ChnlELSnAMask` = TPM\_CnSC\_ELSA\_MASK,  
`kTPM_ChnlELSnBMask` = TPM\_CnSC\_ELSB\_MASK,  
`kTPM_ChnlMSAMask` = TPM\_CnSC\_MSA\_MASK,  
`kTPM_ChnlMSBMask` = TPM\_CnSC\_MSB\_MASK }  
*List of TPM channel modes and level control bit mask.*
- enum `_tpm_trigger_select`  
*Trigger sources available.*
- enum `_tpm_trigger_source` {  
`kTPM_TriggerSource_External` = 0U,  
`kTPM_TriggerSource_Internal` }  
*Trigger source options available.*
- enum `_tpm_ext_trigger_polarity` {  
`kTPM_ExtTrigger_Active_High` = 0U,  
`kTPM_ExtTrigger_Active_Low` }  
*External trigger source polarity.*
- enum `_tpm_output_compare_mode` {  
`kTPM_NoOutputSignal` = (1U << TPM\_CnSC\_MSA\_SHIFT),  
`kTPM_ToggleOnMatch` = ((1U << TPM\_CnSC\_MSA\_SHIFT) | (1U << TPM\_CnSC\_ELSA\_SHIFT)),  
`kTPM_ClearOnMatch` = ((1U << TPM\_CnSC\_MSA\_SHIFT) | (2U << TPM\_CnSC\_ELSA\_SHIFT)),  
`kTPM_SetOnMatch` = ((1U << TPM\_CnSC\_MSA\_SHIFT) | (3U << TPM\_CnSC\_ELSA\_SHIFT)),  
`kTPM_HighPulseOutput` = ((3U << TPM\_CnSC\_MSA\_SHIFT) | (1U << TPM\_CnSC\_ELSA\_SHIFT)),  
`kTPM_LowPulseOutput` = ((3U << TPM\_CnSC\_MSA\_SHIFT) | (2U << TPM\_CnSC\_ELSA\_SHIFT)) }  
*TPM output compare modes.*
- enum `_tpm_input_capture_edge` {  
`kTPM_RisingEdge` = (1U << TPM\_CnSC\_ELSA\_SHIFT),  
`kTPM_FallingEdge` = (2U << TPM\_CnSC\_ELSA\_SHIFT),  
`kTPM_RiseAndFallEdge` = (3U << TPM\_CnSC\_ELSA\_SHIFT) }  
*TPM input capture edge.*
- enum `_tpm_quad_decode_mode` {  
`kTPM_QuadPhaseEncode` = 0U,  
`kTPM_QuadCountAndDir` }  
*TPM quadrature decode modes.*
- enum `_tpm_phase_polarity` {  
`kTPM_QuadPhaseNormal` = 0U,  
`kTPM_QuadPhaseInvert` }  
*TPM quadrature phase polarities.*
- enum `_tpm_clock_source` {

```
kTPM_SystemClock = 1U,
kTPM_ExternalClock,
kTPM_ExternalInputTriggerClock }
```

*TPM clock source selection.*

- enum `_tpm_clock_prescale` {  
`kTPM_Prescale_Divide_1` = 0U,  
`kTPM_Prescale_Divide_2`,  
`kTPM_Prescale_Divide_4`,  
`kTPM_Prescale_Divide_8`,  
`kTPM_Prescale_Divide_16`,  
`kTPM_Prescale_Divide_32`,  
`kTPM_Prescale_Divide_64`,  
`kTPM_Prescale_Divide_128` }

*TPM prescale value selection for the clock source.*

- enum `_tpm_interrupt_enable` {  
`kTPM_Chnl0InterruptEnable` = (1U << 0),  
`kTPM_Chnl1InterruptEnable` = (1U << 1),  
`kTPM_Chnl2InterruptEnable` = (1U << 2),  
`kTPM_Chnl3InterruptEnable` = (1U << 3),  
`kTPM_Chnl4InterruptEnable` = (1U << 4),  
`kTPM_Chnl5InterruptEnable` = (1U << 5),  
`kTPM_Chnl6InterruptEnable` = (1U << 6),  
`kTPM_Chnl7InterruptEnable` = (1U << 7),  
`kTPM_TimeOverflowInterruptEnable` = (1U << 8) }

*List of TPM interrupts.*

- enum `_tpm_status_flags` {  
`kTPM_Chnl0Flag` = (1U << 0),  
`kTPM_Chnl1Flag` = (1U << 1),  
`kTPM_Chnl2Flag` = (1U << 2),  
`kTPM_Chnl3Flag` = (1U << 3),  
`kTPM_Chnl4Flag` = (1U << 4),  
`kTPM_Chnl5Flag` = (1U << 5),  
`kTPM_Chnl6Flag` = (1U << 6),  
`kTPM_Chnl7Flag` = (1U << 7),  
`kTPM_TimeOverflowFlag` = (1U << 8) }

*List of TPM flags.*

## Functions

- static void `TPM_Reset` (TPM\_Type \*base)  
*Performs a software reset on the TPM module.*

## Driver version

- #define `FSL_TPM_DRIVER_VERSION` (`MAKE_VERSION`(2, 3, 1))  
*TPM driver version 2.3.1.*

## Initialization and deinitialization

- `uint32_t TPM_GetInstance` (TPM\_Type \*base)  
*Gets the instance from the base address.*
- `void TPM_Init` (TPM\_Type \*base, const `tpm_config_t` \*config)  
*Ungates the TPM clock and configures the peripheral for basic operation.*
- `void TPM_Deinit` (TPM\_Type \*base)  
*Stops the counter and gates the TPM clock.*
- `void TPM_GetDefaultConfig` (`tpm_config_t` \*config)  
*Fill in the TPM config struct with the default settings.*
- `tpm_clock_prescale_t TPM_CalculateCounterClkDiv` (TPM\_Type \*base, uint32\_t counterPeriod\_Hz, uint32\_t srcClock\_Hz)  
*Calculates the counter clock prescaler.*

## Channel mode operations

- `status_t TPM_SetupPwm` (TPM\_Type \*base, const `tpm_chnl_pwm_signal_param_t` \*chnlParams, uint8\_t numOfChnls, `tpm_pwm_mode_t` mode, uint32\_t pwmFreq\_Hz, uint32\_t srcClock\_Hz)  
*Configures the PWM signal parameters.*
- `status_t TPM_UpdatePwmDutycycle` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber, `tpm_pwm_mode_t` currentPwmMode, uint8\_t dutyCyclePercent)  
*Update the duty cycle of an active PWM signal.*
- `void TPM_UpdateChnlEdgeLevelSelect` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber, uint8\_t level)  
*Update the edge level selection for a channel.*
- `static uint8_t TPM_GetChannelControlBits` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber)  
*Get the channel control bits value (mode, edge and level bit fields).*
- `static void TPM_DisableChannel` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber)  
*Disable the channel.*
- `static void TPM_EnableChannel` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber, uint8\_t control)  
*Enable the channel according to mode and level configs.*
- `void TPM_SetupInputCapture` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber, `tpm_input_capture_edge_t` captureMode)  
*Enables capturing an input signal on the channel using the function parameters.*
- `void TPM_SetupOutputCompare` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber, `tpm_output_compare_mode_t` compareMode, uint32\_t compareValue)  
*Configures the TPM to generate timed pulses.*
- `void TPM_SetupDualEdgeCapture` (TPM\_Type \*base, `tpm_chnl_t` chnlPairNumber, const `tpm_dual_edge_capture_param_t` \*edgeParam, uint32\_t filterValue)  
*Configures the dual edge capture mode of the TPM.*
- `void TPM_SetupQuadDecode` (TPM\_Type \*base, const `tpm_phase_params_t` \*phaseAParams, const `tpm_phase_params_t` \*phaseBParams, `tpm_quad_decode_mode_t` quadMode)  
*Configures the parameters and activates the quadrature decode mode.*
- `static void TPM_SetChannelPolarity` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber, bool enable)  
*Set the input and output polarity of each of the channels.*
- `static void TPM_EnableChannelExtTrigger` (TPM\_Type \*base, `tpm_chnl_t` chnlNumber, bool enable)  
*Enable external trigger input to be used by channel.*

## Interrupt Interface

- `void TPM_EnableInterrupts` (TPM\_Type \*base, uint32\_t mask)

- *Enables the selected TPM interrupts.*  
void [TPM\\_DisableInterrupts](#) (TPM\_Type \*base, uint32\_t mask)
- *Disables the selected TPM interrupts.*  
uint32\_t [TPM\\_GetEnabledInterrupts](#) (TPM\_Type \*base)
- *Gets the enabled TPM interrupts.*  
void [TPM\\_RegisterCallBack](#) (TPM\_Type \*base, [tpm\\_callback\\_t](#) callback)  
*Register callback.*

## Status Interface

- static uint32\_t [TPM\\_GetChannelValue](#) (TPM\_Type \*base, [tpm\\_chnl\\_t](#) chnlNumber)  
*Gets the TPM channel value.*
- static uint32\_t [TPM\\_GetStatusFlags](#) (TPM\_Type \*base)  
*Gets the TPM status flags.*
- static void [TPM\\_ClearStatusFlags](#) (TPM\_Type \*base, uint32\_t mask)  
*Clears the TPM status flags.*

## Read and write the timer period

- static void [TPM\\_SetTimerPeriod](#) (TPM\_Type \*base, uint32\_t ticks)  
*Sets the timer period in units of ticks.*
- static uint32\_t [TPM\\_GetCurrentTimerCount](#) (TPM\_Type \*base)  
*Reads the current timer counting value.*

## Timer Start and Stop

- static void [TPM\\_StartTimer](#) (TPM\_Type \*base, [tpm\\_clock\\_source\\_t](#) clockSource)  
*Starts the TPM counter.*
- static void [TPM\\_StopTimer](#) (TPM\_Type \*base)  
*Stops the TPM counter.*

## 35.4 Data Structure Documentation

### 35.4.1 struct [\\_tpm\\_chnl\\_pwm\\_signal\\_param](#)

#### Data Fields

- [tpm\\_chnl\\_t](#) chnlNumber  
*TPM channel to configure.*
- [tpm\\_pwm\\_pause\\_level\\_select\\_t](#) pauseLevel  
*PWM output level when counter first enabled or paused.*
- [tpm\\_pwm\\_level\\_select\\_t](#) level  
*PWM output active level select.*
- uint8\_t [dutyCyclePercent](#)  
*PWM pulse width, value should be between 0 to 100 0=inactive signal(0% duty cycle)...*
- uint8\_t [firstEdgeDelayPercent](#)  
*Used only in combined PWM mode to generate asymmetrical PWM.*
- bool [enableComplementary](#)  
*Used only in combined PWM mode.*
- [tpm\\_pwm\\_pause\\_level\\_select\\_t](#) secPauseLevel



- *Used only in combined PWM mode.*  
 • `uint8_t deadTimeValue[2]`  
*The dead time value for channel  $n$  and  $n+1$  in combined complementary PWM mode.*

### Field Documentation

#### (1) `tpm_chnl_t_tpm_chnl_pwm_signal_param::chnlNumber`

In combined mode (available in some SoC's), this represents the channel pair number

#### (2) `uint8_t_tpm_chnl_pwm_signal_param::dutyCyclePercent`

100=always active signal (100% duty cycle)

#### (3) `uint8_t_tpm_chnl_pwm_signal_param::firstEdgeDelayPercent`

Specifies the delay to the first edge in a PWM period. If unsure, leave as 0. Should be specified as percentage of the PWM period, (dutyCyclePercent + firstEdgeDelayPercent) value should be not greater than 100.

#### (4) `bool_tpm_chnl_pwm_signal_param::enableComplementary`

true: The combined channels output complementary signals; false: The combined channels output same signals;

#### (5) `tpm_pwm_pause_level_select_t_tpm_chnl_pwm_signal_param::secPauseLevel`

Define the second channel output level when counter first enabled or paused

#### (6) `uint8_t_tpm_chnl_pwm_signal_param::deadTimeValue[2]`

Deadtime insertion is disabled when this value is zero, otherwise deadtime insertion for channel  $n/n+1$  is configured as (deadTimeValue \* 4) clock cycles. deadTimeValue's available range is 0 ~ 15.

## 35.4.2 struct `tpm_dual_edge_capture_param`

Note

This mode is available only on some SoC's.

### Data Fields

- `bool enableSwap`  
*true: Use channel  $n+1$  input, channel  $n$  input is ignored; false: Use channel  $n$  input, channel  $n+1$  input is ignored*
- `tpm_input_capture_edge_t currChanEdgeMode`  
*Input capture edge select for channel  $n$ .*
- `tpm_input_capture_edge_t nextChanEdgeMode`

*Input capture edge select for channel n+1.*

### 35.4.3 struct \_tpm\_phase\_param

#### Data Fields

- uint32\_t [phaseFilterVal](#)  
*Filter value, filter is disabled when the value is zero.*
- [tpm\\_phase\\_polarity\\_t](#) [phasePolarity](#)  
*Phase polarity.*

### 35.4.4 struct \_tpm\_config

This structure holds the configuration settings for the TPM peripheral. To initialize this structure to reasonable defaults, call the [TPM\\_GetDefaultConfig\(\)](#) function and pass a pointer to your config structure instance.

The config struct can be made const so it resides in flash

#### Data Fields

- [tpm\\_clock\\_prescale\\_t](#) [prescale](#)  
*Select TPM clock prescale value.*
- bool [useGlobalTimeBase](#)  
*true: The TPM channels use an external global time base (the local counter still use for generate overflow interrupt and DMA request); false: All TPM channels use the local counter as their timebase*
- bool [syncGlobalTimeBase](#)  
*true: The TPM counter is synchronized to the global time base; false: disabled*
- [tpm\\_trigger\\_select\\_t](#) [triggerSelect](#)  
*Input trigger to use for controlling the counter operation.*
- [tpm\\_trigger\\_source\\_t](#) [triggerSource](#)  
*Decides if we use external or internal trigger.*
- [tpm\\_ext\\_trigger\\_polarity\\_t](#) [extTriggerPolarity](#)  
*when using external trigger source, need selects the polarity of it.*
- bool [enableDoze](#)  
*true: TPM counter is paused in doze mode; false: TPM counter continues in doze mode*
- bool [enableDebugMode](#)  
*true: TPM counter continues in debug mode; false: TPM counter is paused in debug mode*
- bool [enableReloadOnTrigger](#)  
*true: TPM counter is reloaded on trigger; false: TPM counter not reloaded*
- bool [enableStopOnOverflow](#)  
*true: TPM counter stops after overflow; false: TPM counter continues running after overflow*
- bool [enableStartOnTrigger](#)  
*true: TPM counter only starts when a trigger is detected; false: TPM counter starts immediately*
- bool [enablePauseOnTrigger](#)  
*true: TPM counter will pause while trigger remains asserted; false: TPM counter continues running*

- uint8\_t [chnlPolarity](#)  
*Defines the input/output polarity of the channels in POL register.*

## Field Documentation

(1) `tpm_trigger_source_t _tpm_config::triggerSource`

(2) `tpm_ext_trigger_polarity_t _tpm_config::extTriggerPolarity`

## 35.5 Macro Definition Documentation

**35.5.1** `#define FSL_TPM_DRIVER_VERSION (MAKE_VERSION(2, 3, 1))`

## 35.6 Typedef Documentation

**35.6.1** `typedef enum _tpm_chnl tpm_chnl_t`

Note

Actual number of available channels is SoC dependent

**35.6.2** `typedef enum _tpm_pwm_level_select tpm_pwm_level_select_t`

Note

When the TPM has PWM pause level select feature, the PWM output cannot be turned off by selecting the output level. In this case, the channel must be closed to close the PWM output.

**35.6.3** `typedef enum _tpm_trigger_select tpm_trigger_select_t`

This is used for both internal & external trigger sources (external trigger sources available in certain SoC's)

Note

The actual trigger sources available is SoC-specific.

**35.6.4** `typedef enum _tpm_trigger_source tpm_trigger_source_t`

Note

This selection is available only on some SoC's. For SoC's without this selection, the only trigger source available is internal trigger.

**35.6.5 typedef enum \_tpm\_ext\_trigger\_polarity tpm\_ext\_trigger\_polarity\_t**

Note

Selects the polarity of the external trigger source.

**35.6.6 typedef struct \_tpm\_dual\_edge\_capture\_param tpm\_dual\_edge\_capture\_param\_t**

Note

This mode is available only on some SoC's.

**35.6.7 typedef enum \_tpm\_quad\_decode\_mode tpm\_quad\_decode\_mode\_t**

Note

This mode is available only on some SoC's.

**35.6.8 typedef struct \_tpm\_config tpm\_config\_t**

This structure holds the configuration settings for the TPM peripheral. To initialize this structure to reasonable defaults, call the [TPM\\_GetDefaultConfig\(\)](#) function and pass a pointer to your config structure instance.

The config struct can be made const so it resides in flash

**35.6.9 typedef void(\* tpm\_callback\_t)(TPM\_Type \*base)**

Parameters

|             |                              |
|-------------|------------------------------|
| <i>base</i> | TPM peripheral base address. |
|-------------|------------------------------|

**35.7 Enumeration Type Documentation****35.7.1 enum \_tpm\_chnl**

Note

Actual number of available channels is SoC dependent

Enumerator

***kTPM\_Chnl\_0*** TPM channel number 0.  
***kTPM\_Chnl\_1*** TPM channel number 1.  
***kTPM\_Chnl\_2*** TPM channel number 2.  
***kTPM\_Chnl\_3*** TPM channel number 3.  
***kTPM\_Chnl\_4*** TPM channel number 4.  
***kTPM\_Chnl\_5*** TPM channel number 5.  
***kTPM\_Chnl\_6*** TPM channel number 6.  
***kTPM\_Chnl\_7*** TPM channel number 7.

### 35.7.2 enum \_tpm\_pwm\_mode

Enumerator

***kTPM\_EdgeAlignedPwm*** Edge aligned PWM.  
***kTPM\_CenterAlignedPwm*** Center aligned PWM.  
***kTPM\_CombinedPwm*** Combined PWM (Edge-aligned, center-aligned, or asymmetrical PWMs can be obtained in combined mode using different software configurations)

### 35.7.3 enum \_tpm\_pwm\_level\_select

Note

When the TPM has PWM pause level select feature, the PWM output cannot be turned off by selecting the output level. In this case, the channel must be closed to close the PWM output.

Enumerator

***kTPM\_HighTrue*** High true pulses.  
***kTPM\_LowTrue*** Low true pulses.

### 35.7.4 enum \_tpm\_pwm\_pause\_level\_select

Enumerator

***kTPM\_ClearOnPause*** Clear Output when counter first enabled or paused.  
***kTPM\_SetOnPause*** Set Output when counter first enabled or paused.

### 35.7.5 enum \_tpm\_chnl\_control\_bit\_mask

Enumerator

***kTPM\_ChnlELSnAMask*** Channel ELSA bit mask.

***kTPM\_ChnlELSnBMask*** Channel ELSB bit mask.

***kTPM\_ChnlMSAMask*** Channel MSA bit mask.

***kTPM\_ChnlMSBMask*** Channel MSB bit mask.

### 35.7.6 enum \_tpm\_trigger\_select

This is used for both internal & external trigger sources (external trigger sources available in certain SoC's)

Note

The actual trigger sources available is SoC-specific.

### 35.7.7 enum \_tpm\_trigger\_source

Note

This selection is available only on some SoC's. For SoC's without this selection, the only trigger source available is internal trigger.

Enumerator

***kTPM\_TriggerSource\_External*** Use external trigger input.

***kTPM\_TriggerSource\_Internal*** Use internal trigger (channel pin input capture)

### 35.7.8 enum \_tpm\_ext\_trigger\_polarity

Note

Selects the polarity of the external trigger source.

Enumerator

***kTPM\_ExtTrigger\_Active\_High*** External trigger input is active high.

***kTPM\_ExtTrigger\_Active\_Low*** External trigger input is active low.

### 35.7.9 enum \_tpm\_output\_compare\_mode

Enumerator

*kTPM\_NoOutputSignal* No channel output when counter reaches CnV.  
*kTPM\_ToggleOnMatch* Toggle output.  
*kTPM\_ClearOnMatch* Clear output.  
*kTPM\_SetOnMatch* Set output.  
*kTPM\_HighPulseOutput* Pulse output high.  
*kTPM\_LowPulseOutput* Pulse output low.

### 35.7.10 enum \_tpm\_input\_capture\_edge

Enumerator

*kTPM\_RisingEdge* Capture on rising edge only.  
*kTPM\_FallingEdge* Capture on falling edge only.  
*kTPM\_RiseAndFallEdge* Capture on rising or falling edge.

### 35.7.11 enum \_tpm\_quad\_decode\_mode

Note

This mode is available only on some SoC's.

Enumerator

*kTPM\_QuadPhaseEncode* Phase A and Phase B encoding mode.  
*kTPM\_QuadCountAndDir* Count and direction encoding mode.

### 35.7.12 enum \_tpm\_phase\_polarity

Enumerator

*kTPM\_QuadPhaseNormal* Phase input signal is not inverted.  
*kTPM\_QuadPhaseInvert* Phase input signal is inverted.

### 35.7.13 enum \_tpm\_clock\_source

Enumerator

*kTPM\_SystemClock* System clock.

***kTPM\_ExternalClock*** External TPM\_EXTCLK pin clock.

***kTPM\_ExternalInputTriggerClock*** Selected external input trigger clock.

### 35.7.14 enum \_tpm\_clock\_prescale

Enumerator

***kTPM\_Prescale\_Divide\_1*** Divide by 1.

***kTPM\_Prescale\_Divide\_2*** Divide by 2.

***kTPM\_Prescale\_Divide\_4*** Divide by 4.

***kTPM\_Prescale\_Divide\_8*** Divide by 8.

***kTPM\_Prescale\_Divide\_16*** Divide by 16.

***kTPM\_Prescale\_Divide\_32*** Divide by 32.

***kTPM\_Prescale\_Divide\_64*** Divide by 64.

***kTPM\_Prescale\_Divide\_128*** Divide by 128.

### 35.7.15 enum \_tpm\_interrupt\_enable

Enumerator

***kTPM\_Chnl0InterruptEnable*** Channel 0 interrupt.

***kTPM\_Chnl1InterruptEnable*** Channel 1 interrupt.

***kTPM\_Chnl2InterruptEnable*** Channel 2 interrupt.

***kTPM\_Chnl3InterruptEnable*** Channel 3 interrupt.

***kTPM\_Chnl4InterruptEnable*** Channel 4 interrupt.

***kTPM\_Chnl5InterruptEnable*** Channel 5 interrupt.

***kTPM\_Chnl6InterruptEnable*** Channel 6 interrupt.

***kTPM\_Chnl7InterruptEnable*** Channel 7 interrupt.

***kTPM\_TimeOverflowInterruptEnable*** Time overflow interrupt.

### 35.7.16 enum \_tpm\_status\_flags

Enumerator

***kTPM\_Chnl0Flag*** Channel 0 flag.

***kTPM\_Chnl1Flag*** Channel 1 flag.

***kTPM\_Chnl2Flag*** Channel 2 flag.

***kTPM\_Chnl3Flag*** Channel 3 flag.

***kTPM\_Chnl4Flag*** Channel 4 flag.

***kTPM\_Chnl5Flag*** Channel 5 flag.

***kTPM\_Chnl6Flag*** Channel 6 flag.

***kTPM\_Chnl7Flag*** Channel 7 flag.

***kTPM\_TimeOverflowFlag*** Time overflow flag.



## 35.8 Function Documentation

### 35.8.1 uint32\_t TPM\_GetInstance ( TPM\_Type \* *base* )

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | TPM peripheral base address |
|-------------|-----------------------------|

Returns

The TPM instance

### 35.8.2 void TPM\_Init ( TPM\_Type \* *base*, const tpm\_config\_t \* *config* )

Note

This API should be called at the beginning of the application using the TPM driver.

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | TPM peripheral base address             |
| <i>config</i> | Pointer to user's TPM config structure. |

### 35.8.3 void TPM\_Deinit ( TPM\_Type \* *base* )

Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | TPM peripheral base address |
|-------------|-----------------------------|

### 35.8.4 void TPM\_GetDefaultConfig ( tpm\_config\_t \* *config* )

The default values are:

```
* config->prescale = kTPM_Prescale_Divide_1;
* config->useGlobalTimeBase = false;
* config->syncGlobalTimeBase = false;
* config->dozeEnable = false;
* config->dbgMode = false;
* config->enableReloadOnTrigger = false;
* config->enableStopOnOverflow = false;
* config->enableStartOnTrigger = false;
*#if FSL_FEATURE_TPM_HAS_PAUSE_COUNTER_ON_TRIGGER
* config->enablePauseOnTrigger = false;
```

```

*#endif
* config->triggerSelect = kTPM_Trigger_Select_0;
*#if FSL_FEATURE_TPM_HAS_EXTERNAL_TRIGGER_SELECTION
* config->triggerSource = kTPM_TriggerSource_External;
* config->extTriggerPolarity = kTPM_ExtTrigger_Active_High;
*#endif
*#if defined(FSL_FEATURE_TPM_HAS_POL) && FSL_FEATURE_TPM_HAS_POL
* config->chnlPolarity = 0U;
*#endif
*

```

## Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>config</i> | Pointer to user's TPM config structure. |
|---------------|-----------------------------------------|

### 35.8.5 tpm\_clock\_prescale\_t TPM\_CalculateCounterClkDiv ( TPM\_Type \* *base*, uint32\_t *counterPeriod\_Hz*, uint32\_t *srcClock\_Hz* )

This function calculates the values for SC[PS].

## Parameters

|                         |                                                                                                    |
|-------------------------|----------------------------------------------------------------------------------------------------|
| <i>base</i>             | TPM peripheral base address                                                                        |
| <i>counterPeriod_Hz</i> | The desired frequency in Hz which corresponding to the time when the counter reaches the mod value |
| <i>srcClock_Hz</i>      | TPM counter clock in Hz                                                                            |

return Calculated clock prescaler value.

### 35.8.6 status\_t TPM\_SetupPwm ( TPM\_Type \* *base*, const tpm\_chnl\_pwm\_signal\_param\_t \* *chnlParams*, uint8\_t *numOfChnls*, tpm\_pwm\_mode\_t *mode*, uint32\_t *pwmFreq\_Hz*, uint32\_t *srcClock\_Hz* )

User calls this function to configure the PWM signals period, mode, dutycycle and edge. Use this function to configure all the TPM channels that will be used to output a PWM signal

## Parameters

|                   |                                                             |
|-------------------|-------------------------------------------------------------|
| <i>base</i>       | TPM peripheral base address                                 |
| <i>chnlParams</i> | Array of PWM channel parameters to configure the channel(s) |

|                    |                                                                                     |
|--------------------|-------------------------------------------------------------------------------------|
| <i>numOfChnls</i>  | Number of channels to configure, this should be the size of the array passed in     |
| <i>mode</i>        | PWM operation mode, options available in enumeration <a href="#">tpm_pwm_mode_t</a> |
| <i>pwmFreq_Hz</i>  | PWM signal frequency in Hz                                                          |
| <i>srcClock_Hz</i> | TPM counter clock in Hz                                                             |

Returns

kStatus\_Success if the PWM setup was successful, kStatus\_Error on failure

**35.8.7** `status_t TPM_UpdatePwmDutycycle ( TPM_Type * base, tpm_chnl_t chnlNumber, tpm_pwm_mode_t currentPwmMode, uint8_t dutyCyclePercent )`

Parameters

|                          |                                                                                                                               |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>              | TPM peripheral base address                                                                                                   |
| <i>chnlNumber</i>        | The channel number. In combined mode, this represents the channel pair number                                                 |
| <i>currentPwm-Mode</i>   | The current PWM mode set during PWM setup                                                                                     |
| <i>dutyCycle-Percent</i> | New PWM pulse width, value should be between 0 to 100 0=inactive signal(0% duty cycle)... 100=active signal (100% duty cycle) |

Returns

kStatus\_Success if the PWM setup was successful, kStatus\_Error on failure

**35.8.8** `void TPM_UpdateChnlEdgeLevelSelect ( TPM_Type * base, tpm_chnl_t chnlNumber, uint8_t level )`

Note

When the TPM has PWM pause level select feature (FSL\_FEATURE\_TPM\_HAS\_PAUSE\_LEVEL\_SELECT = 1), the PWM output cannot be turned off by selecting the output level. In this case, must use TPM\_DisableChannel API to close the PWM output.

## Parameters

|                   |                                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>       | TPM peripheral base address                                                                                                                           |
| <i>chnlNumber</i> | The channel number                                                                                                                                    |
| <i>level</i>      | The level to be set to the ELSnB:ELSnA field; valid values are 00, 01, 10, 11. See the appropriate SoC reference manual for details about this field. |

### 35.8.9 static uint8\_t TPM\_GetChannelContorlBits ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber* ) [inline], [static]

This function disable the channel by clear all mode and level control bits.

## Parameters

|                   |                             |
|-------------------|-----------------------------|
| <i>base</i>       | TPM peripheral base address |
| <i>chnlNumber</i> | The channel number          |

## Returns

The contorl bits value. This is the logical OR of members of the enumeration [tpm\\_chnl\\_control\\_bit\\_mask\\_t](#).

### 35.8.10 static void TPM\_DisableChannel ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber* ) [inline], [static]

This function disable the channel by clear all mode and level control bits.

## Parameters

|                   |                             |
|-------------------|-----------------------------|
| <i>base</i>       | TPM peripheral base address |
| <i>chnlNumber</i> | The channel number          |

### 35.8.11 static void TPM\_EnableChannel ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber*, uint8\_t *control* ) [inline], [static]

This function enable the channel output according to input mode/level config parameters.

## Parameters

|                   |                                                                                                                            |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>       | TPM peripheral base address                                                                                                |
| <i>chnlNumber</i> | The channel number                                                                                                         |
| <i>control</i>    | The control bits value. This is the logical OR of members of the enumeration <a href="#">tpm_chnl_control_bit_mask_t</a> . |

### 35.8.12 void TPM\_SetupInputCapture ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber*, tpm\_input\_capture\_edge\_t *captureMode* )

When the edge specified in the captureMode argument occurs on the channel, the TPM counter is captured into the CnV register. The user has to read the CnV register separately to get this value.

## Parameters

|                    |                                 |
|--------------------|---------------------------------|
| <i>base</i>        | TPM peripheral base address     |
| <i>chnlNumber</i>  | The channel number              |
| <i>captureMode</i> | Specifies which edge to capture |

### 35.8.13 void TPM\_SetupOutputCompare ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber*, tpm\_output\_compare\_mode\_t *compareMode*, uint32\_t *compareValue* )

When the TPM counter matches the value of compareVal argument (this is written into CnV reg), the channel output is changed based on what is specified in the compareMode argument.

## Parameters

|                     |                                                                        |
|---------------------|------------------------------------------------------------------------|
| <i>base</i>         | TPM peripheral base address                                            |
| <i>chnlNumber</i>   | The channel number                                                     |
| <i>compareMode</i>  | Action to take on the channel output when the compare condition is met |
| <i>compareValue</i> | Value to be programmed in the CnV register.                            |

**35.8.14** void TPM\_SetupDualEdgeCapture ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlPairNumber*, const tpm\_dual\_edge\_capture\_param\_t \* *edgeParam*, uint32\_t *filterValue* )

This function allows to measure a pulse width of the signal on the input of channel of a channel pair. The filter function is disabled if the filterVal argument passed is zero.

## Parameters

|                        |                                                     |
|------------------------|-----------------------------------------------------|
| <i>base</i>            | TPM peripheral base address                         |
| <i>chnlPair-Number</i> | The TPM channel pair number; options are 0, 1, 2, 3 |
| <i>edgeParam</i>       | Sets up the dual edge capture function              |
| <i>filterValue</i>     | Filter value, specify 0 to disable filter.          |

**35.8.15 void TPM\_SetupQuadDecode ( TPM\_Type \* *base*, const tpm\_phase\_params\_t \* *phaseAParams*, const tpm\_phase\_params\_t \* *phaseBParams*, tpm\_quad\_decode\_mode\_t *quadMode* )**

## Parameters

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>base</i>         | TPM peripheral base address                           |
| <i>phaseAParams</i> | Phase A configuration parameters                      |
| <i>phaseBParams</i> | Phase B configuration parameters                      |
| <i>quadMode</i>     | Selects encoding mode used in quadrature decoder mode |

**35.8.16 static void TPM\_SetChannelPolarity ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber*, bool *enable* ) [inline], [static]**

## Parameters

|                   |                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------|
| <i>base</i>       | TPM peripheral base address                                                                   |
| <i>chnlNumber</i> | The channel number                                                                            |
| <i>enable</i>     | true: Set the channel polarity to active high; false: Set the channel polarity to active low; |

**35.8.17 static void TPM\_EnableChannelExtTrigger ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber*, bool *enable* ) [inline], [static]**

In input capture mode, configures the trigger input that is used by the channel to capture the counter value. In output compare or PWM mode, configures the trigger input used to modulate the channel output. When modulating the output, the output is forced to the channel initial value whenever the trigger is not asserted.

## Note

No matter how many external trigger sources there are, only input trigger 0 and 1 are used. The even numbered channels share the input trigger 0 and the odd numbered channels share the second input trigger 1.

## Parameters

|                   |                                                                                                                |
|-------------------|----------------------------------------------------------------------------------------------------------------|
| <i>base</i>       | TPM peripheral base address                                                                                    |
| <i>chnlNumber</i> | The channel number                                                                                             |
| <i>enable</i>     | true: Configures trigger input 0 or 1 to be used by channel; false: Trigger input has no effect on the channel |

### 35.8.18 void TPM\_EnableInterrupts ( TPM\_Type \* *base*, uint32\_t *mask* )

## Parameters

|             |                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | TPM peripheral base address                                                                                         |
| <i>mask</i> | The interrupts to enable. This is a logical OR of members of the enumeration <a href="#">tpm_interrupt_enable_t</a> |

### 35.8.19 void TPM\_DisableInterrupts ( TPM\_Type \* *base*, uint32\_t *mask* )

## Parameters

|             |                                                                                                                      |
|-------------|----------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | TPM peripheral base address                                                                                          |
| <i>mask</i> | The interrupts to disable. This is a logical OR of members of the enumeration <a href="#">tpm_interrupt_enable_t</a> |

### 35.8.20 uint32\_t TPM\_GetEnabledInterrupts ( TPM\_Type \* *base* )

## Parameters



|             |                             |
|-------------|-----------------------------|
| <i>base</i> | TPM peripheral base address |
|-------------|-----------------------------|

Returns

The enabled interrupts. This is the logical OR of members of the enumeration [tpm\\_interrupt\\_enable\\_t](#)

### 35.8.21 void TPM\_RegisterCallback ( TPM\_Type \* *base*, tpm\_callback\_t *callback* )

If channel or overflow interrupt is enabled by the user, then a callback can be registered which will be invoked when the interrupt is triggered.

Parameters

|                 |                             |
|-----------------|-----------------------------|
| <i>base</i>     | TPM peripheral base address |
| <i>callback</i> | Callback function           |

### 35.8.22 static uint32\_t TPM\_GetChannelValue ( TPM\_Type \* *base*, tpm\_chnl\_t *chnlNumber* ) [inline], [static]

Note

The TPM channel value contain the captured TPM counter value for the input modes or the match value for the output modes.

Parameters

|                   |                             |
|-------------------|-----------------------------|
| <i>base</i>       | TPM peripheral base address |
| <i>chnlNumber</i> | The channel number          |

Returns

The channle CnV regisyer value.

### 35.8.23 static uint32\_t TPM\_GetStatusFlags ( TPM\_Type \* *base* ) [inline], [static]

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | TPM peripheral base address |
|-------------|-----------------------------|

## Returns

The status flags. This is the logical OR of members of the enumeration [tpm\\_status\\_flags\\_t](#)

**35.8.24 static void TPM\_ClearStatusFlags ( TPM\_Type \* *base*, uint32\_t *mask* )**  
**[inline], [static]**

## Parameters

|             |                                                                                                                  |
|-------------|------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | TPM peripheral base address                                                                                      |
| <i>mask</i> | The status flags to clear. This is a logical OR of members of the enumeration <a href="#">tpm_status_flags_t</a> |

**35.8.25 static void TPM\_SetTimerPeriod ( TPM\_Type \* *base*, uint32\_t *ticks* )**  
**[inline], [static]**

Timers counts from 0 until it equals the count value set here. The count value is written to the MOD register.

## Note

1. This API allows the user to use the TPM module as a timer. Do not mix usage of this API with TPM's PWM setup API's.
2. Call the utility macros provided in the `fsl_common.h` to convert usec or msec to ticks.

## Parameters

|              |                                                                            |
|--------------|----------------------------------------------------------------------------|
| <i>base</i>  | TPM peripheral base address                                                |
| <i>ticks</i> | A timer period in units of ticks, which should be equal or greater than 1. |

**35.8.26 static uint32\_t TPM\_GetCurrentTimerCount ( TPM\_Type \* *base* )**  
**[inline], [static]**

This function returns the real-time timer counting value in a range from 0 to a timer period.

## Note

Call the utility macros provided in the `fsl_common.h` to convert ticks to usec or msec.

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | TPM peripheral base address |
|-------------|-----------------------------|

## Returns

The current counter value in ticks

**35.8.27** `static void TPM_StartTimer ( TPM_Type * base, tpm_clock_source_t clockSource ) [inline], [static]`

## Parameters

|                    |                                                                           |
|--------------------|---------------------------------------------------------------------------|
| <i>base</i>        | TPM peripheral base address                                               |
| <i>clockSource</i> | TPM clock source; once clock source is set the counter will start running |

**35.8.28** `static void TPM_StopTimer ( TPM_Type * base ) [inline], [static]`

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | TPM peripheral base address |
|-------------|-----------------------------|

**35.8.29** `static void TPM_Reset ( TPM_Type * base ) [inline], [static]`

Reset all internal logic and registers, except the Global Register. Remains set until cleared by software.

## Note

TPM software reset is available on certain SoC's only

## Parameters

|             |                             |
|-------------|-----------------------------|
| <i>base</i> | TPM peripheral base address |
|-------------|-----------------------------|

## Chapter 36

# TRGMUX: Trigger Mux Driver

### 36.1 Overview

The MCUXpresso SDK provides driver for the Trigger Mux (TRGMUX) module of MCUXpresso SDK devices.

### 36.2 Typical use case

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/trgmux

#### Typedefs

- typedef enum `_trgmux_trigger_input` `trgmux_trigger_input_t`  
*Defines the MUX select for peripheral trigger input.*

#### Enumerations

- enum { `kStatus_TRGMUX_Locked` = MAKE\_STATUS(kStatusGroup\_TRGMUX, 0) }  
*TRGMUX configure status.*
- enum `_trgmux_trigger_input` {  
    `kTRGMUX_TriggerInput0` = TRGMUX\_TRGCFG\_SEL0\_SHIFT,  
    `kTRGMUX_TriggerInput1` = TRGMUX\_TRGCFG\_SEL1\_SHIFT,  
    `kTRGMUX_TriggerInput2` = TRGMUX\_TRGCFG\_SEL2\_SHIFT,  
    `kTRGMUX_TriggerInput3` = TRGMUX\_TRGCFG\_SEL3\_SHIFT }  
*Defines the MUX select for peripheral trigger input.*

#### Driver version

- #define `FSL_TRGMUX_DRIVER_VERSION` (MAKE\_VERSION(2, 0, 1))  
*TRGMUX driver version.*

#### TRGMUX Functional Operation

- static void `TRGMUX_LockRegister` (TRGMUX\_Type \*base, uint32\_t index)  
*Sets the flag of the register which is used to mark writeable.*
- `status_t` `TRGMUX_SetTriggerSource` (TRGMUX\_Type \*base, uint32\_t index, `trgmux_trigger_input_t` input, uint32\_t trigger\_src)  
*Configures the trigger source of the appointed peripheral.*

### 36.3 Macro Definition Documentation

#### 36.3.1 #define FSL\_TRGMUX\_DRIVER\_VERSION (MAKE\_VERSION(2, 0, 1))

## 36.4 Enumeration Type Documentation

### 36.4.1 anonymous enum

Enumerator

***kStatus\_TRGMUX\_Locked*** Configure failed for register is locked.

### 36.4.2 enum \_trgmux\_trigger\_input

Enumerator

***kTRGMUX\_TriggerInput0*** The MUX select for peripheral trigger input 0.

***kTRGMUX\_TriggerInput1*** The MUX select for peripheral trigger input 1.

***kTRGMUX\_TriggerInput2*** The MUX select for peripheral trigger input 2.

***kTRGMUX\_TriggerInput3*** The MUX select for peripheral trigger input 3.

## 36.5 Function Documentation

### 36.5.1 static void TRGMUX\_LockRegister ( TRGMUX\_Type \* *base*, uint32\_t *index* ) [inline], [static]

The function sets the flag of the register which is used to mark writeable. Example:

```
TRGMUX_LockRegister (TRGMUX0, kTRGMUX_Trgmux0Dmamux0);
```

Parameters

|              |                                                                                     |
|--------------|-------------------------------------------------------------------------------------|
| <i>base</i>  | TRGMUX peripheral base address.                                                     |
| <i>index</i> | The index of the TRGMUX register, see the enum trgmux_device_t defined in <SO-C>.h. |

### 36.5.2 status\_t TRGMUX\_SetTriggerSource ( TRGMUX\_Type \* *base*, uint32\_t *index*, trgmux\_trigger\_input\_t *input*, uint32\_t *trigger\_src* )

The function configures the trigger source of the appointed peripheral. Example:

```
TRGMUX_SetTriggerSource (TRGMUX0, kTRGMUX_Trgmux0Dmamux0,
 kTRGMUX_TriggerInput0, kTRGMUX_SourcePortPin);
```

## Parameters

|                    |                                                                                                                               |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i>        | TRGMUX peripheral base address.                                                                                               |
| <i>index</i>       | The index of the TRGMUX register, see the enum <code>trgmux_device_t</code> defined in <code>&lt;SOC&gt;.h</code> .           |
| <i>input</i>       | The MUX select for peripheral trigger input                                                                                   |
| <i>trigger_src</i> | The trigger inputs for various peripherals. See the enum <code>trgmux_source_t</code> defined in <code>&lt;SOC&gt;.h</code> . |

## Return values

|                              |                                                      |
|------------------------------|------------------------------------------------------|
| <i>kStatus_Success</i>       | Configured successfully.                             |
| <i>kStatus_TRGMUX_Locked</i> | Configuration failed because the register is locked. |

## Chapter 37

# TRNG: True Random Number Generator

The MCUXpresso SDK provides a peripheral driver for the True Random Number Generator (TRNG) module of MCUXpresso SDK devices.

The True Random Number Generator is a hardware accelerator module that generates a 512-bit entropy as needed by an entropy consuming module or by other post processing functions. A typical entropy consumer is a pseudo random number generator (PRNG) which can be implemented to achieve both true randomness and cryptographic strength random numbers using the TRNG output as its entropy seed. The entropy generated by a TRNG is intended for direct use by functions that generate secret keys, per-message secrets, random challenges, and other similar quantities used in cryptographic algorithms.

### 37.1 TRNG Initialization

1. Define the TRNG user configuration structure. Use `TRNG_InitUserConfigDefault()` function to set it to default TRNG configuration values.
2. Initialize the TRNG module, call the `TRNG_Init()` function, and pass the user configuration structure. This function automatically enables the TRNG module and its clock. After that, the TRNG is enabled and the entropy generation starts working.
3. To disable the TRNG module, call the `TRNG_Deinit()` function.

### 37.2 Get random data from TRNG

1. `TRNG_GetRandomData()` function gets random data from the TRNG module.

This example code shows how to initialize and get random data from the TRNG driver.

Refer to the driver examples codes located at `<SDK_ROOT>/boards/<BOARD>/driver_examples/trng`



# Chapter 38

## TSTMR: Timestamp Timer Driver

### 38.1 Overview

The MCUXpresso SDK provides a driver for the TSTMR module of MCUXpresso SDK devices.

### Functions

- static uint64\_t [TSTMR\\_ReadTimeStamp](#) (TSTMR\_Type \*base)  
*Reads the time stamp.*
- static void [TSTMR\\_DelayUs](#) (TSTMR\_Type \*base, uint64\_t delayInUs)  
*Delays for a specified number of microseconds.*

### Driver version

- #define [FSL\\_TSTMR\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 0, 2))  
*Version 2.0.2.*

### 38.2 Function Documentation

#### 38.2.1 static uint64\_t TSTMR\_ReadTimeStamp ( TSTMR\_Type \* *base* ) [inline], [static]

This function reads the low and high registers and returns the 56-bit free running counter value. This can be read by software at any time to determine the software ticks. TSTMR registers can be read with 32-bit accesses only. The TSTMR LOW read should occur first, followed by the TSTMR HIGH read.

Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | TSTMR peripheral base address. |
|-------------|--------------------------------|

Returns

The 56-bit time stamp value.

#### 38.2.2 static void TSTMR\_DelayUs ( TSTMR\_Type \* *base*, uint64\_t *delayInUs* ) [inline], [static]

This function repeatedly reads the timestamp register and waits for the user-specified delay value.

## Parameters

|                  |                                |
|------------------|--------------------------------|
| <i>base</i>      | TSTMR peripheral base address. |
| <i>delayInUs</i> | Delay value in microseconds.   |

## Chapter 39

# WDOG32: 32-bit Watchdog Timer

### 39.1 Overview

The MCUXpresso SDK provides a peripheral driver for the WDOG32 module of MCUXpresso SDK devices.

### 39.2 Typical use case

Refer to the driver examples codes located at <SDK\_ROOT>/boards/<BOARD>/driver\_examples/wdog32

### Data Structures

- struct `_wdog32_work_mode`  
*Defines WDOG32 work mode. [More...](#)*
- struct `_wdog32_config`  
*Describes WDOG32 configuration structure. [More...](#)*

### Typedefs

- typedef enum `_wdog32_clock_source wdog32_clock_source_t`  
*Describes WDOG32 clock source.*
- typedef enum `_wdog32_clock_prescaler wdog32_clock_prescaler_t`  
*Describes the selection of the clock prescaler.*
- typedef struct `_wdog32_work_mode wdog32_work_mode_t`  
*Defines WDOG32 work mode.*
- typedef enum `_wdog32_test_mode wdog32_test_mode_t`  
*Describes WDOG32 test mode.*
- typedef struct `_wdog32_config wdog32_config_t`  
*Describes WDOG32 configuration structure.*

### Enumerations

- enum `_wdog32_clock_source` {  
    `kWDOG32_ClockSource0` = 0U,  
    `kWDOG32_ClockSource1` = 1U,  
    `kWDOG32_ClockSource2` = 2U,  
    `kWDOG32_ClockSource3` = 3U }  
*Describes WDOG32 clock source.*
- enum `_wdog32_clock_prescaler` {  
    `kWDOG32_ClockPrescalerDivide1` = 0x0U,  
    `kWDOG32_ClockPrescalerDivide256` = 0x1U }  
*Describes the selection of the clock prescaler.*

- enum `_wdog32_test_mode` {  
`kWDOG32_TestModeDisabled` = 0U,  
`kWDOG32_UserModeEnabled` = 1U,  
`kWDOG32_LowByteTest` = 2U,  
`kWDOG32_HighByteTest` = 3U }  
*Describes WDOG32 test mode.*
- enum `_wdog32_interrupt_enable_t` { `kWDOG32_InterruptEnable` = WDOG\_CS\_INT\_MASK }  
*WDOG32 interrupt configuration structure.*
- enum `_wdog32_status_flags_t` {  
`kWDOG32_RunningFlag` = WDOG\_CS\_EN\_MASK,  
`kWDOG32_InterruptFlag` = WDOG\_CS\_FLG\_MASK }  
*WDOG32 status flags.*

## Unlock sequence

- #define `WDOG_FIRST_WORD_OF_UNLOCK` (WDOG\_UPDATE\_KEY & 0xFFFFU)  
*First word of unlock sequence.*
- #define `WDOG_SECOND_WORD_OF_UNLOCK` ((WDOG\_UPDATE\_KEY >> 16U) & 0xFFFFU)  
*Second word of unlock sequence.*

## Refresh sequence

- #define `WDOG_FIRST_WORD_OF_REFRESH` (WDOG\_REFRESH\_KEY & 0xFFFFU)  
*First word of refresh sequence.*
- #define `WDOG_SECOND_WORD_OF_REFRESH` ((WDOG\_REFRESH\_KEY >> 16U) & 0xFFFFU)  
*Second word of refresh sequence.*

## Driver version

- #define `FSL_WDOG32_DRIVER_VERSION` (MAKE\_VERSION(2, 1, 0))  
*WDOG32 driver version.*

## WDOG32 Initialization and De-initialization

- void `WDOG32_GetDefaultConfig` (`wdog32_config_t` \*config)  
*Initializes the WDOG32 configuration structure.*
- void `WDOG32_Init` (WDOG\_Type \*base, const `wdog32_config_t` \*config)  
*Initializes the WDOG32 module.*
- void `WDOG32_Deinit` (WDOG\_Type \*base)  
*De-initializes the WDOG32 module.*

## WDOG32 functional Operation

- void `WDOG32_Unlock` (WDOG\_Type \*base)  
*Unlocks the WDOG32 register written.*
- void `WDOG32_Enable` (WDOG\_Type \*base)  
*Enables the WDOG32 module.*

- void [WDOG32\\_Disable](#) (WDOG\_Type \*base)  
*Disables the WDOG32 module.*
- void [WDOG32\\_EnableInterrupts](#) (WDOG\_Type \*base, uint32\_t mask)  
*Enables the WDOG32 interrupt.*
- void [WDOG32\\_DisableInterrupts](#) (WDOG\_Type \*base, uint32\_t mask)  
*Disables the WDOG32 interrupt.*
- static uint32\_t [WDOG32\\_GetStatusFlags](#) (WDOG\_Type \*base)  
*Gets the WDOG32 all status flags.*
- void [WDOG32\\_ClearStatusFlags](#) (WDOG\_Type \*base, uint32\_t mask)  
*Clears the WDOG32 flag.*
- void [WDOG32\\_SetTimeoutValue](#) (WDOG\_Type \*base, uint16\_t timeoutCount)  
*Sets the WDOG32 timeout value.*
- void [WDOG32\\_SetWindowValue](#) (WDOG\_Type \*base, uint16\_t windowValue)  
*Sets the WDOG32 window value.*
- static void [WDOG32\\_Refresh](#) (WDOG\_Type \*base)  
*Refreshes the WDOG32 timer.*
- static uint16\_t [WDOG32\\_GetCounterValue](#) (WDOG\_Type \*base)  
*Gets the WDOG32 counter value.*

## 39.3 Data Structure Documentation

### 39.3.1 struct \_wdog32\_work\_mode

#### Data Fields

- bool [enableWait](#)  
*Enables or disables WDOG32 in wait mode.*
- bool [enableStop](#)  
*Enables or disables WDOG32 in stop mode.*
- bool [enableDebug](#)  
*Enables or disables WDOG32 in debug mode.*

### 39.3.2 struct \_wdog32\_config

#### Data Fields

- bool [enableWdog32](#)  
*Enables or disables WDOG32.*
- [wdog32\\_clock\\_source\\_t](#) clockSource  
*Clock source select.*
- [wdog32\\_clock\\_prescaler\\_t](#) prescaler  
*Clock prescaler value.*
- [wdog32\\_work\\_mode\\_t](#) workMode  
*Configures WDOG32 work mode in debug stop and wait mode.*
- [wdog32\\_test\\_mode\\_t](#) testMode  
*Configures WDOG32 test mode.*
- bool [enableUpdate](#)  
*Update write-once register enable.*

- bool [enableInterrupt](#)  
*Enables or disables WDOG32 interrupt.*
- bool [enableWindowMode](#)  
*Enables or disables WDOG32 window mode.*
- uint16\_t [windowValue](#)  
*Window value.*
- uint16\_t [timeoutValue](#)  
*Timeout value.*

## 39.4 Macro Definition Documentation

### 39.4.1 #define FSL\_WDOG32\_DRIVER\_VERSION (MAKE\_VERSION(2, 1, 0))

## 39.5 Typedef Documentation

### 39.5.1 typedef enum \_wdog32\_clock\_source wdog32\_clock\_source\_t

### 39.5.2 typedef enum \_wdog32\_clock\_prescaler wdog32\_clock\_prescaler\_t

### 39.5.3 typedef struct \_wdog32\_work\_mode wdog32\_work\_mode\_t

### 39.5.4 typedef enum \_wdog32\_test\_mode wdog32\_test\_mode\_t

### 39.5.5 typedef struct \_wdog32\_config wdog32\_config\_t

## 39.6 Enumeration Type Documentation

### 39.6.1 enum \_wdog32\_clock\_source

Enumerator

- kWDOG32\_ClockSource0* Clock source 0.
- kWDOG32\_ClockSource1* Clock source 1.
- kWDOG32\_ClockSource2* Clock source 2.
- kWDOG32\_ClockSource3* Clock source 3.

### 39.6.2 enum \_wdog32\_clock\_prescaler

Enumerator

- kWDOG32\_ClockPrescalerDivide1* Divided by 1.
- kWDOG32\_ClockPrescalerDivide256* Divided by 256.

### 39.6.3 enum \_wdog32\_test\_mode

Enumerator

*kWDOG32\_TestModeDisabled* Test Mode disabled.  
*kWDOG32\_UserModeEnabled* User Mode enabled.  
*kWDOG32\_LowByteTest* Test Mode enabled, only low byte is used.  
*kWDOG32\_HighByteTest* Test Mode enabled, only high byte is used.

### 39.6.4 enum \_wdog32\_interrupt\_enable\_t

This structure contains the settings for all of the WDOG32 interrupt configurations.

Enumerator

*kWDOG32\_InterruptEnable* Interrupt is generated before forcing a reset.

### 39.6.5 enum \_wdog32\_status\_flags\_t

This structure contains the WDOG32 status flags for use in the WDOG32 functions.

Enumerator

*kWDOG32\_RunningFlag* Running flag, set when WDOG32 is enabled.  
*kWDOG32\_InterruptFlag* Interrupt flag, set when interrupt occurs.

## 39.7 Function Documentation

### 39.7.1 void WDOG32\_GetDefaultConfig ( wdog32\_config\_t \* config )

This function initializes the WDOG32 configuration structure to default values. The default values are:

```
* wdog32Config->enableWdog32 = true;
* wdog32Config->clockSource = kWDOG32_ClockSource1;
* wdog32Config->prescaler = kWDOG32_ClockPrescalerDivide1;
* wdog32Config->workMode.enableWait = true;
* wdog32Config->workMode.enableStop = false;
* wdog32Config->workMode.enableDebug = false;
* wdog32Config->testMode = kWDOG32_TestModeDisabled;
* wdog32Config->enableUpdate = true;
* wdog32Config->enableInterrupt = false;
* wdog32Config->enableWindowMode = false;
* wdog32Config->windowValue = 0U;
* wdog32Config->timeoutValue = 0xFFFFU;
*
```

## Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>config</i> | Pointer to the WDOG32 configuration structure. |
|---------------|------------------------------------------------|

## See Also

[wdog32\\_config\\_t](#)

### 39.7.2 void WDOG32\_Init ( WDOG\_Type \* *base*, const wdog32\_config\_t \* *config* )

This function initializes the WDOG32. To reconfigure the WDOG32 without forcing a reset first, enable-Update must be set to true in the configuration.

## Example:

```
* wdog32_config_t config;
* WDOG32_GetDefaultConfig(&config);
* config.timeoutValue = 0x7ffU;
* config.enableUpdate = true;
* WDOG32_Init(wdog_base, &config);
*
```

## Note

If there is errata ERR010536 (FSL\_FEATURE\_WDOG\_HAS\_ERRATA\_010536 defined as 1), then after calling this function, user need delay at least 4 LPO clock cycles before accessing other WDOG32 registers.

## Parameters

|               |                                  |
|---------------|----------------------------------|
| <i>base</i>   | WDOG32 peripheral base address.  |
| <i>config</i> | The configuration of the WDOG32. |

### 39.7.3 void WDOG32\_Deinit ( WDOG\_Type \* *base* )

This function shuts down the WDOG32. Ensure that the WDOG\_CS.UPDATE is 1, which means that the register update is enabled.



## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | WDOG32 peripheral base address. |
|-------------|---------------------------------|

**39.7.4 void WDOG32\_Unlock ( WDOG\_Type \* *base* )**

This function unlocks the WDOG32 register written.

Before starting the unlock sequence and following the configuration, disable the global interrupts. Otherwise, an interrupt could effectively invalidate the unlock sequence and the WCT may expire. After the configuration finishes, re-enable the global interrupts.

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | WDOG32 peripheral base address |
|-------------|--------------------------------|

**39.7.5 void WDOG32\_Enable ( WDOG\_Type \* *base* )**

This function writes a value into the WDOG\_CS register to enable the WDOG32. The WDOG\_CS register is a write-once register. Please check the enableUpdate is set to true for calling [WDOG32\\_Init](#) to do wdog initialize. Before call the re-configuration APIs, ensure that the WCT window is still open and this register has not been written in this WCT while the function is called.

## Parameters

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | WDOG32 peripheral base address. |
|-------------|---------------------------------|

**39.7.6 void WDOG32\_Disable ( WDOG\_Type \* *base* )**

This function writes a value into the WDOG\_CS register to disable the WDOG32. The WDOG\_CS register is a write-once register. Please check the enableUpdate is set to true for calling [WDOG32\\_Init](#) to do wdog initialize. Before call the re-configuration APIs, ensure that the WCT window is still open and this register has not been written in this WCT while the function is called.

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | WDOG32 peripheral base address |
|-------------|--------------------------------|

### 39.7.7 void WDOG32\_EnableInterrupts ( WDOG\_Type \* *base*, uint32\_t *mask* )

This function writes a value into the WDOG\_CS register to enable the WDOG32 interrupt. The WDOG\_CS register is a write-once register. Please check the enableUpdate is set to true for calling [WDOG32\\_Init](#) to do wdog initialize. Before call the re-configuration APIs, ensure that the WCT window is still open and this register has not been written in this WCT while the function is called.

Parameters

|             |                                                                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | WDOG32 peripheral base address.                                                                                                                                            |
| <i>mask</i> | The interrupts to enable. The parameter can be a combination of the following source if defined: <ul style="list-style-type: none"> <li>kWDOG32_InterruptEnable</li> </ul> |

### 39.7.8 void WDOG32\_DisableInterrupts ( WDOG\_Type \* *base*, uint32\_t *mask* )

This function writes a value into the WDOG\_CS register to disable the WDOG32 interrupt. The WDOG\_CS register is a write-once register. Please check the enableUpdate is set to true for calling [WDOG32\\_Init](#) to do wdog initialize. Before call the re-configuration APIs, ensure that the WCT window is still open and this register has not been written in this WCT while the function is called.

Parameters

|             |                                                                                                                                                                              |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | WDOG32 peripheral base address.                                                                                                                                              |
| <i>mask</i> | The interrupts to disabled. The parameter can be a combination of the following source if defined: <ul style="list-style-type: none"> <li>kWDOG32_InterruptEnable</li> </ul> |

### 39.7.9 static uint32\_t WDOG32\_GetStatusFlags ( WDOG\_Type \* *base* ) [inline], [static]

This function gets all status flags.

Example to get the running flag:

```
* uint32_t status;
* status = WDOG32_GetStatusFlags(wdog_base) &
* kWDOG32_RunningFlag;
*
```

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | WDOG32 peripheral base address |
|-------------|--------------------------------|

## Returns

State of the status flag: asserted (true) or not-asserted (false).

## See Also

[\\_wdog32\\_status\\_flags\\_t](#)

- true: related status flag has been set.
- false: related status flag is not set.

### 39.7.10 void WDOG32\_ClearStatusFlags ( WDOG\_Type \* *base*, uint32\_t *mask* )

This function clears the WDOG32 status flag.

Example to clear an interrupt flag:

```
* WDOG32_ClearStatusFlags(wdog_base,
* kWDOG32_InterruptFlag);
*
```

## Parameters

|             |                                                                                                                                                                    |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>base</i> | WDOG32 peripheral base address.                                                                                                                                    |
| <i>mask</i> | The status flags to clear. The parameter can be any combination of the following values: <ul style="list-style-type: none"> <li>• kWDOG32_InterruptFlag</li> </ul> |

### 39.7.11 void WDOG32\_SetTimeoutValue ( WDOG\_Type \* *base*, uint16\_t *timeoutCount* )

This function writes a timeout value into the WDOG\_TOVAL register. The WDOG\_TOVAL register is a write-once register. To ensure the reconfiguration fits the timing of WCT, unlock function will be called inline.

## Parameters

|                     |                                                    |
|---------------------|----------------------------------------------------|
| <i>base</i>         | WDOG32 peripheral base address                     |
| <i>timeoutCount</i> | WDOG32 timeout value, count of WDOG32 clock ticks. |

### 39.7.12 void WDOG32\_SetWindowValue ( WDOG\_Type \* *base*, uint16\_t *windowValue* )

This function writes a window value into the WDOG\_WIN register. The WDOG\_WIN register is a write-once register. Please check the enableUpdate is set to true for calling [WDOG32\\_Init](#) to do wdog initialize. Before call the re-configuration APIs, ensure that the WCT window is still open and this register has not been written in this WCT while the function is called.

## Parameters

|                    |                                 |
|--------------------|---------------------------------|
| <i>base</i>        | WDOG32 peripheral base address. |
| <i>windowValue</i> | WDOG32 window value.            |

### 39.7.13 static void WDOG32\_Refresh ( WDOG\_Type \* *base* ) [inline], [static]

This function feeds the WDOG32. This function should be called before the Watchdog timer is in timeout. Otherwise, a reset is asserted.

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | WDOG32 peripheral base address |
|-------------|--------------------------------|

### 39.7.14 static uint16\_t WDOG32\_GetCounterValue ( WDOG\_Type \* *base* ) [inline], [static]

This function gets the WDOG32 counter value.

## Parameters

---

|             |                                 |
|-------------|---------------------------------|
| <i>base</i> | WDOG32 peripheral base address. |
|-------------|---------------------------------|

## Returns

Current WDOG32 counter value.

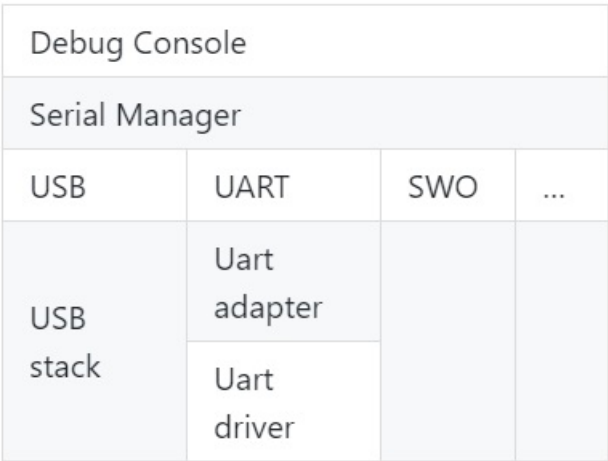
# Chapter 40

## Debug Console

### 40.1 Overview

This chapter describes the programming interface of the debug console driver.

The debug console enables debug log messages to be output via the specified peripheral with frequency of the peripheral source clock and base address at the specified baud rate. Additionally, it provides input and output functions to scan and print formatted data. The below picture shows the layout of debug console.



Debug console overview

### 40.2 Function groups

#### 40.2.1 Initialization

To initialize the debug console, call the [DbgConsole\\_Init\(\)](#) function with these parameters. This function automatically enables the module and the clock.

```
status_t DbgConsole_Init(uint8_t instance, uint32_t baudRate,
 serial_port_type_t device, uint32_t clkSrcFreq);
```

Select the supported debug console hardware device type, such as

```
typedef enum _serial_port_type
{
 kSerialPort_Uart = 1U,
 kSerialPort_UsbCdc,
 kSerialPort_Swo,
} serial_port_type_t;
```

After the initialization is successful, stdout and stdin are connected to the selected peripheral. This example shows how to call the `DbgConsole_Init()` given the user configuration structure.

```
DbgConsole_Init(BOARD_DEBUG_UART_INSTANCE, BOARD_DEBUG_UART_BAUDRATE, BOARD_DEBUG_UART_TYPE,
 BOARD_DEBUG_UART_CLK_FREQ);
```

## 40.2.2 Advanced Feature

The debug console provides input and output functions to scan and print formatted data.

- Support a format specifier for PRINTF following this prototype "`%[flags][width][.precision][length]specifier`", which is explained below

| flags   | Description                                                                                                                                                                                                                                                                                                                                                                             |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -       | Left-justified within the given field width. Right-justified is the default.                                                                                                                                                                                                                                                                                                            |
| +       | Forces to precede the result with a plus or minus sign (+ or -) even for positive numbers. By default, only negative numbers are preceded with a - sign.                                                                                                                                                                                                                                |
| (space) | If no sign is written, a blank space is inserted before the value.                                                                                                                                                                                                                                                                                                                      |
| #       | Used with o, x, or X specifiers the value is preceded with 0, 0x, or 0X respectively for values other than zero. Used with e, E and f, it forces the written output to contain a decimal point even if no digits would follow. By default, if no digits follow, no decimal point is written. Used with g or G the result is the same as with e or E but trailing zeros are not removed. |
| 0       | Left-pads the number with zeroes (0) instead of spaces, where padding is specified (see width sub-specifier).                                                                                                                                                                                                                                                                           |

| Width    | Description                                                                                                                                                                                            |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (number) | A minimum number of characters to be printed. If the value to be printed is shorter than this number, the result is padded with blank spaces. The value is not truncated even if the result is larger. |
| *        | The width is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.                                                          |

| <b>.precision</b> | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| .number           | For integer specifiers (d, i, o, u, x, X) precision specifies the minimum number of digits to be written. If the value to be written is shorter than this number, the result is padded with leading zeros. The value is not truncated even if the result is longer. A precision of 0 means that no character is written for the value 0. For e, E, and f specifiers this is the number of digits to be printed after the decimal point. For g and G specifiers This is the maximum number of significant digits to be printed. For s this is the maximum number of characters to be printed. By default, all characters are printed until the ending null character is encountered. For c type it has no effect. When no precision is specified, the default is 1. If the period is specified without an explicit value for precision, 0 is assumed. |
| .*                | The precision is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

| <b>length</b>  | <b>Description</b> |
|----------------|--------------------|
| Do not support |                    |

| <b>specifier</b> | <b>Description</b>                           |
|------------------|----------------------------------------------|
| d or i           | Signed decimal integer                       |
| f                | Decimal floating point                       |
| F                | Decimal floating point capital letters       |
| x                | Unsigned hexadecimal integer                 |
| X                | Unsigned hexadecimal integer capital letters |
| o                | Signed octal                                 |
| b                | Binary value                                 |
| p                | Pointer address                              |
| u                | Unsigned decimal integer                     |
| c                | Character                                    |
| s                | String of characters                         |
| n                | Nothing printed                              |



| specifier | Description |
|-----------|-------------|
|-----------|-------------|

- Support a format specifier for SCANF following this prototype " %[\*][width][length]specifier", which is explained below

| *                                                                                                                                                                | Description |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| An optional starting asterisk indicates that the data is to be read from the stream but ignored. In other words, it is not stored in the corresponding argument. |             |

| width                                                                                        | Description |
|----------------------------------------------------------------------------------------------|-------------|
| This specifies the maximum number of characters to be read in the current reading operation. |             |

| length      | Description                                                                                                                                                                                             |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| hh          | The argument is interpreted as a signed character or unsigned character (only applies to integer specifiers: i, d, o, u, x, and X).                                                                     |
| h           | The argument is interpreted as a short integer or unsigned short integer (only applies to integer specifiers: i, d, o, u, x, and X).                                                                    |
| l           | The argument is interpreted as a long integer or unsigned long integer for integer specifiers (i, d, o, u, x, and X) and as a wide character or wide character string for specifiers c and s.           |
| ll          | The argument is interpreted as a long long integer or unsigned long long integer for integer specifiers (i, d, o, u, x, and X) and as a wide character or wide character string for specifiers c and s. |
| L           | The argument is interpreted as a long double (only applies to floating point specifiers: e, E, f, g, and G).                                                                                            |
| j or z or t | Not supported                                                                                                                                                                                           |

| specifier              | Qualifying Input                                                                                                                                                                                                                                 | Type of argument |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| c                      | Single character: Reads the next character. If a width different from 1 is specified, the function reads width characters and stores them in the successive locations of the array passed as argument. No null character is appended at the end. | char *           |
| i                      | Integer: : Number optionally preceded with a + or - sign                                                                                                                                                                                         | int *            |
| d                      | Decimal integer: Number optionally preceded with a + or - sign                                                                                                                                                                                   | int *            |
| a, A, e, E, f, F, g, G | Floating point: Decimal number containing a decimal point, optionally preceded by a + or - sign and optionally followed by the e or E character and a decimal number. Two examples of valid entries are -732.103 and 7.12e4                      | float *          |
| o                      | Octal Integer:                                                                                                                                                                                                                                   | int *            |
| s                      | String of characters. This reads subsequent characters until a white space is found (white space characters are considered to be blank, newline, and tab).                                                                                       | char *           |
| u                      | Unsigned decimal integer.                                                                                                                                                                                                                        | unsigned int *   |

The debug console has its own printf/scanf/putchar/getchar functions which are defined in the header file.

```
int DbgConsole_Printf(const char *fmt_s, ...);
int DbgConsole_Putchar(int ch);
int DbgConsole_Scanf(char *fmt_ptr, ...);
int DbgConsole_Getchar(void);
```

This utility supports selecting toolchain's printf/scanf or the MCUXpresso SDK printf/scanf.

```
#if SDK_DEBUGCONSOLE == DEBUGCONSOLE_DISABLE /* Disable debug console */
#define PRINTF
#define SCANF
#define PUTCHAR
#define GETCHAR
#elif SDK_DEBUGCONSOLE == DEBUGCONSOLE_REDIRECT_TO_SDK /* Select printf, scanf, putchar, getchar of SDK
```

```

 version. */
#define PRINTF DbgConsole_Printf
#define SCANF DbgConsole_Scanf
#define PUTCHAR DbgConsole_Putchar
#define GETCHAR DbgConsole_Getchar
#elif SDK_DEBUGCONSOLE == DEBUGCONSOLE_REDIRECT_TO_TOOLCHAIN /* Select printf, scanf, putchar, getchar of
 toolchain. */
#define PRINTF printf
#define SCANF scanf
#define PUTCHAR putchar
#define GETCHAR getchar
#endif /* SDK_DEBUGCONSOLE */

```

### 40.2.3 SDK\_DEBUGCONSOLE and SDK\_DEBUGCONSOLE\_UART

There are two macros `SDK_DEBUGCONSOLE` and `SDK_DEBUGCONSOLE_UART` added to configure `PRINTF` and low level output peripheral.

- The macro `SDK_DEBUGCONSOLE` is used for frontend. Whether debug console redirect to toolchain or SDK or disabled, it decides which is the frontend of the debug console, Tool chain or SDK. The function can be set by the macro `SDK_DEBUGCONSOLE`.
- The macro `SDK_DEBUGCONSOLE_UART` is used for backend. It is used to decide whether provide low level IO implementation to toolchain `printf` and `scanf`. For example, within MCU-Xpresso, if the macro `SDK_DEBUGCONSOLE_UART` is defined, `__sys_write` and `__sys_readc` will be used when `__REDLIB__` is defined; `_write` and `_read` will be used in other cases. The macro does not specifically refer to the peripheral "UART". It refers to the external peripheral similar to UART, like as USB CDC, UART, SWO, etc. So if the macro `SDK_DEBUGCONSOLE_UART` is not defined when tool-chain `printf` is calling, the semihosting will be used.

The following matrix shows the effects of `SDK_DEBUGCONSOLE` and `SDK_DEBUGCONSOLE_UART` on `PRINTF` and `printf`. The green mark is the default setting of the debug console.

| <b>SDK_DEBUGCONSOLE</b>                      | <b>SDK_DEBUGCONSOLE_UART</b> | <b>PRINTF</b>         | <b>printf</b>        |
|----------------------------------------------|------------------------------|-----------------------|----------------------|
| DEBUGCONSOLE_-<br>REDIRECT_TO_SDK            | defined                      | Low level peripheral* | Low level peripheral |
| DEBUGCONSOLE_-<br>REDIRECT_TO_SDK            | undefined                    | Low level peripheral* | semihost             |
| DEBUGCONSOLE_-<br>REDIRECT_TO_TO-<br>OLCHAIN | defined                      | Low level peripheral* | Low level peripheral |
| DEBUGCONSOLE_-<br>REDIRECT_TO_TO-<br>OLCHAIN | undefined                    | semihost              | semihost             |
| DEBUGCONSOLE_-<br>DISABLE                    | defined                      | No output             | Low level peripheral |
| DEBUGCONSOLE_-<br>DISABLE                    | undefined                    | No output             | semihost             |

| SDK_DEBUGCONSOLE | SDK_DEBUGCONSOLE_UART | PRINTF | printf |
|------------------|-----------------------|--------|--------|
|------------------|-----------------------|--------|--------|

\* the **low level peripheral** could be USB CDC, UART, or SWO, and so on.

## 40.3 Typical use case

### Some examples use the PUTCHAR & GETCHAR function

```
ch = GETCHAR();
PUTCHAR(ch);
```

### Some examples use the PRINTF function

Statement prints the string format.

```
PRINTF("%s %s\r\n", "Hello", "world!");
```

Statement prints the hexadecimal format/

```
PRINTF("0x%02X hexadecimal number equivalent 255", 255);
```

Statement prints the decimal floating point and unsigned decimal.

```
PRINTF("Execution timer: %s\r\nTime: %u ticks %2.5f milliseconds\r\n\r\nDONE\r\n\r\n", "1 day", 86400, 86.4);
```

### Some examples use the SCANF function

```
PRINTF("Enter a decimal number: ");
SCANF("%d", &i);
PRINTF("\r\nYou have entered %d.\r\n", i, i);
PRINTF("Enter a hexadecimal number: ");
SCANF("%x", &i);
PRINTF("\r\nYou have entered 0x%X (%d).\r\n", i, i);
```

### Print out failure messages using MCUXpresso SDK \_\_assert\_func:

```
void __assert_func(const char *file, int line, const char *func, const char *failedExpr)
{
 PRINTF("ASSERT ERROR \" %s \": file \"%s\" Line \"%d\" function name \"%s\" \n", failedExpr, file
, line, func);
 for (;;)
 {}
}
```

**Note:**

To use 'printf' and 'scanf' for GNUC Base, add file 'fsl\_sbrk.c' in path: `..\{package}\devices\{subset}\utilities\fsl-_sbrk.c` to your project.

**Modules**

- [debug console configuration](#)  
*The configuration is used for debug console only.*

**Macros**

- `#define DEBUGCONSOLE_REDIRECT_TO_TOOLCHAIN 0U`  
*Definition select redirect toolchain printf, scanf to uart or not.*
- `#define DEBUGCONSOLE_REDIRECT_TO_SDK 1U`  
*Select SDK version printf, scanf.*
- `#define DEBUGCONSOLE_DISABLE 2U`  
*Disable debugconsole function.*
- `#define SDK_DEBUGCONSOLE DEBUGCONSOLE_REDIRECT_TO_SDK`  
*Definition to select sdk or toolchain printf, scanf.*
- `#define PRINTF DbgConsole_Printf`  
*Definition to select redirect toolchain printf, scanf to uart or not.*

**Variables**

- [serial\\_handle\\_t g\\_serialHandle](#)  
*serial manager handle*

**Initialization**

- `status_t DbgConsole_Init` (uint8\_t instance, uint32\_t baudRate, [serial\\_port\\_type\\_t](#) device, uint32\_t clkSrcFreq)  
*Initializes the peripheral used for debug messages.*
- `status_t DbgConsole_Deinit` (void)  
*De-initializes the peripheral used for debug messages.*
- `status_t DbgConsole_EnterLowpower` (void)  
*Prepares to enter low power consumption.*
- `status_t DbgConsole_ExitLowpower` (void)  
*Restores from low power consumption.*
- `int DbgConsole_Printf` (const char \*fmt\_s,...)  
*Writes formatted output to the standard output stream.*
- `int DbgConsole_Vprintf` (const char \*fmt\_s, va\_list formatStringArg)  
*Writes formatted output to the standard output stream.*
- `int DbgConsole_Putchar` (int ch)  
*Writes a character to stdout.*
- `int DbgConsole_Scanf` (char \*fmt\_s,...)  
*Reads formatted data from the standard input stream.*
- `int DbgConsole_Getchar` (void)  
*Reads a character from standard input.*
- `int DbgConsole_BlockingPrintf` (const char \*fmt\_s,...)

- *Writes formatted output to the standard output stream with the blocking mode.*  
 • `int DbgConsole_BlockingVprintf` (const char \*fmt\_s, va\_list formatStringArg)  
*Writes formatted output to the standard output stream with the blocking mode.*
- `status_t DbgConsole_Flush` (void)  
*Debug console flush.*
- `status_t DbgConsole_TryGetchar` (char \*ch)  
*Debug console try to get char This function provides a API which will not block current task, if character is available return it, otherwise return fail.*

## 40.4 Macro Definition Documentation

### 40.4.1 #define DEBUGCONSOLE\_REDIRECT\_TO\_TOOLCHAIN 0U

Select toolchain printf and scanf.

### 40.4.2 #define DEBUGCONSOLE\_REDIRECT\_TO\_SDK 1U

### 40.4.3 #define DEBUGCONSOLE\_DISABLE 2U

### 40.4.4 #define SDK\_DEBUGCONSOLE DEBUGCONSOLE\_REDIRECT\_TO\_SDK

The macro only support to be redefined in project setting.

### 40.4.5 #define PRINTF DbgConsole\_Printf

if SDK\_DEBUGCONSOLE defined to 0,it represents select toolchain printf, scanf. if SDK\_DEBUGCONSOLE defined to 1,it represents select SDK version printf, scanf. if SDK\_DEBUGCONSOLE defined to 2,it represents disable debugconsole function.

## 40.5 Function Documentation

### 40.5.1 status\_t DbgConsole\_Init ( uint8\_t instance, uint32\_t baudRate, serial\_port\_type\_t device, uint32\_t clkSrcFreq )

Call this function to enable debug log messages to be output via the specified peripheral initialized by the serial manager module. After this function has returned, stdout and stdin are connected to the selected peripheral.

## Parameters

|                   |                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>instance</i>   | The instance of the module.If the device is kSerialPort_Uart, the instance is UART peripheral instance. The UART hardware peripheral type is determined by UART adapter. For example, if the instance is 1, if the lpuart_adapter.c is added to the current project, the UART peripheral is LPUART1. If the uart_adapter.c is added to the current project, the UART peripheral is UART1. |
| <i>baudRate</i>   | The desired baud rate in bits per second.                                                                                                                                                                                                                                                                                                                                                 |
| <i>device</i>     | Low level device type for the debug console, can be one of the following. <ul style="list-style-type: none"> <li>• kSerialPort_Uart,</li> <li>• kSerialPort_UsbCdc</li> </ul>                                                                                                                                                                                                             |
| <i>clkSrcFreq</i> | Frequency of peripheral source clock.                                                                                                                                                                                                                                                                                                                                                     |

## Returns

Indicates whether initialization was successful or not.

## Return values

|                        |                        |
|------------------------|------------------------|
| <i>kStatus_Success</i> | Execution successfully |
|------------------------|------------------------|

### 40.5.2 status\_t DbgConsole\_Deinit ( void )

Call this function to disable debug log messages to be output via the specified peripheral initialized by the serial manager module.

## Returns

Indicates whether de-initialization was successful or not.

### 40.5.3 status\_t DbgConsole\_EnterLowpower ( void )

This function is used to prepare to enter low power consumption.

## Returns

Indicates whether de-initialization was successful or not.

**40.5.4 status\_t DbgConsole\_ExitLowpower ( void )**

This function is used to restore from low power consumption.

Returns

Indicates whether de-initialization was successful or not.

**40.5.5 int DbgConsole\_Printf ( const char \* *fmt\_s*, ... )**

Call this function to write a formatted output to the standard output stream.

Parameters

|              |                        |
|--------------|------------------------|
| <i>fmt_s</i> | Format control string. |
|--------------|------------------------|

Returns

Returns the number of characters printed or a negative value if an error occurs.

**40.5.6 int DbgConsole\_Vprintf ( const char \* *fmt\_s*, va\_list *formatStringArg* )**

Call this function to write a formatted output to the standard output stream.

Parameters

|                         |                        |
|-------------------------|------------------------|
| <i>fmt_s</i>            | Format control string. |
| <i>formatString-Arg</i> | Format arguments.      |

Returns

Returns the number of characters printed or a negative value if an error occurs.

**40.5.7 int DbgConsole\_Putchar ( int *ch* )**

Call this function to write a character to stdout.



## Parameters

|           |                          |
|-----------|--------------------------|
| <i>ch</i> | Character to be written. |
|-----------|--------------------------|

## Returns

Returns the character written.

#### 40.5.8 int DbgConsole\_Scanf ( char \* *fmt\_s*, ... )

Call this function to read formatted data from the standard input stream.

## Note

Due the limitation in the BM OSA environment (CPU is blocked in the function, other tasks will not be scheduled), the function cannot be used when the `DEBUG_CONSOLE_TRANSFER_NON_BLOCKING` is set in the BM OSA environment. And an error is returned when the function called in this case. The suggestion is that polling the non-blocking function `DbgConsole_TryGetchar` to get the input char.

## Parameters

|              |                        |
|--------------|------------------------|
| <i>fmt_s</i> | Format control string. |
|--------------|------------------------|

## Returns

Returns the number of fields successfully converted and assigned.

#### 40.5.9 int DbgConsole\_Getchar ( void )

Call this function to read a character from standard input.

## Note

Due the limitation in the BM OSA environment (CPU is blocked in the function, other tasks will not be scheduled), the function cannot be used when the `DEBUG_CONSOLE_TRANSFER_NON_BLOCKING` is set in the BM OSA environment. And an error is returned when the function called in this case. The suggestion is that polling the non-blocking function `DbgConsole_TryGetchar` to get the input char.

## Returns

Returns the character read.

#### 40.5.10 int DbgConsole\_BlockingPrintf ( const char \* *fmt\_s*, ... )

Call this function to write a formatted output to the standard output stream with the blocking mode. The function will send data with blocking mode no matter the DEBUG\_CONSOLE\_TRANSFER\_NON\_BLOCKING set or not. The function could be used in system ISR mode with DEBUG\_CONSOLE\_TRANSFER\_NON\_BLOCKING set.

Parameters

|              |                        |
|--------------|------------------------|
| <i>fmt_s</i> | Format control string. |
|--------------|------------------------|

Returns

Returns the number of characters printed or a negative value if an error occurs.

#### 40.5.11 int DbgConsole\_BlockingVprintf ( const char \* *fmt\_s*, va\_list *formatStringArg* )

Call this function to write a formatted output to the standard output stream with the blocking mode. The function will send data with blocking mode no matter the DEBUG\_CONSOLE\_TRANSFER\_NON\_BLOCKING set or not. The function could be used in system ISR mode with DEBUG\_CONSOLE\_TRANSFER\_NON\_BLOCKING set.

Parameters

|                        |                        |
|------------------------|------------------------|
| <i>fmt_s</i>           | Format control string. |
| <i>formatStringArg</i> | Format arguments.      |

Returns

Returns the number of characters printed or a negative value if an error occurs.

#### 40.5.12 status\_t DbgConsole\_Flush ( void )

Call this function to wait the tx buffer empty. If interrupt transfer is using, make sure the global IRQ is enable before call this function This function should be called when 1, before enter power down mode 2, log is required to print to terminal immediately

Returns

Indicates whether wait idle was successful or not.

#### 40.5.13 status\_t DbgConsole\_TryGetchar ( char \* *ch* )

## Parameters

|           |                                |
|-----------|--------------------------------|
| <i>ch</i> | the address of char to receive |
|-----------|--------------------------------|

## Returns

Indicates get char was successful or not.

## 40.6 debug console configuration

The configuration is used for debug console only.

### 40.6.1 Overview

Please note, it is not used for debug console lite.

### Macros

- #define **DEBUG\_CONSOLE\_TRANSMIT\_BUFFER\_LEN** (512U)  
*If Non-blocking mode is needed, please define it at project setting, otherwise blocking mode is the default transfer mode.*
- #define **DEBUG\_CONSOLE\_RECEIVE\_BUFFER\_LEN** (1024U)  
*define the receive buffer length which is used to store the user input, buffer is enabled automatically when non-blocking transfer is using, This value will affect the RAM's utilization, should be set per platform's capability and software requirement.*
- #define **DEBUG\_CONSOLE\_TX\_RELIABLE\_ENABLE** (1U)  
*Whether enable the reliable TX function If the macro is zero, the reliable TX function of the debug console is disabled.*
- #define **DEBUG\_CONSOLE\_RX\_ENABLE** (1U)  
*Whether enable the RX function If the macro is zero, the receive function of the debug console is disabled.*
- #define **DEBUG\_CONSOLE\_PRINTF\_MAX\_LOG\_LEN** (128U)  
*define the MAX log length debug console support , that is when you call printf("log", x);, the log length can not bigger than this value.*
- #define **DEBUG\_CONSOLE\_SCANF\_MAX\_LOG\_LEN** (20U)  
*define the buffer support buffer scanf log length, that is when you call scanf("log", &x);, the log length can not bigger than this value.*
- #define **DEBUG\_CONSOLE\_SYNCHRONIZATION\_BM** 0  
*Debug console synchronization User should not change these macro for synchronization mode, but add the corresponding synchronization mechanism per different software environment.*
- #define **DEBUG\_CONSOLE\_SYNCHRONIZATION\_FREERTOS** 1  
*synchronization for freertos software*
- #define **DEBUG\_CONSOLE\_SYNCHRONIZATION\_MODE** **DEBUG\_CONSOLE\_SYNCHRONIZATION\_BM**  
*RTOS synchronization mechanism disable If not defined, default is enable, to avoid multitask log print mess.*
- #define **DEBUG\_CONSOLE\_ENABLE\_ECHO\_FUNCTION** 0  
*echo function support If you want to use the echo function, please define **DEBUG\_CONSOLE\_ENABLE\_ECHO** at your project setting.*
- #define **BOARD\_USE\_VIRTUALCOM** 0U  
*Definition to select virtual com(USB CDC) as the debug console.*

## 40.6.2 Macro Definition Documentation

### 40.6.2.1 #define DEBUG\_CONSOLE\_TRANSMIT\_BUFFER\_LEN (512U)

Warning: If you want to use non-blocking transfer, please make sure the corresponding IO interrupt is enable, otherwise there is no output. And non-blocking is combine with buffer, no matter bare-metal or rtos. Below shows how to configure in your project if you want to use non-blocking mode. For IAR, right click project and select "Options", define it in "C/C++ Compiler->Preprocessor->Defined symbols". For KEIL, click "Options for Target. . .", define it in "C/C++->Preprocessor Symbols->Define". For ARM-GCC, open CmakeLists.txt and add the following lines, "SET(CMAKE\_C\_FLAGS\_DEBUG "\${CMAKE\_C\_FLAGS\_DEBUG} -DDEBUG\_CONSOLE\_TRANSFER\_NON\_BLOCKING)" for debug target. "SET(CMAKE\_C\_FLAGS\_RELEASE "\${CMAKE\_C\_FLAGS\_RELEASE} -DDEBUG\_CONSOLE\_TRANSFER\_NON\_BLOCKING)" for release target. For MCUXpresso, right click project and select "Properties", define it in "C/C++ Build->Settings->MCU C Compiler->Preprocessor".

define the transmit buffer length which is used to store the multi task log, buffer is enabled automatically when non-blocking transfer is using, This value will affect the RAM's utilization, should be set per paltform's capability and software requirement. If it is configured too small, log maybe missed , because the log will not be buffered if the buffer is full, and the print will return immediately with -1. And this value should be multiple of 4 to meet memory alignment.

### 40.6.2.2 #define DEBUG\_CONSOLE\_RECEIVE\_BUFFER\_LEN (1024U)

If it is configured too small, log maybe missed, because buffer will be overwritten if buffer is too small. And this value should be multiple of 4 to meet memory alignment.

### 40.6.2.3 #define DEBUG\_CONSOLE\_TX\_RELIABLE\_ENABLE (1U)

When the macro is zero, the string of PRINTF will be thrown away after the transmit buffer is full.

### 40.6.2.4 #define DEBUG\_CONSOLE\_PRINTF\_MAX\_LOG\_LEN (128U)

This macro decide the local log buffer length, the buffer locate at stack, the stack maybe overflow if the buffer is too big and current task stack size not big enough.

### 40.6.2.5 #define DEBUG\_CONSOLE\_SCANF\_MAX\_LOG\_LEN (20U)

As same as the DEBUG\_CONSOLE\_BUFFER\_PRINTF\_MAX\_LOG\_LEN.

#### 40.6.2.6 **#define DEBUG\_CONSOLE\_SYNCHRONIZATION\_BM 0**

Such as, if another RTOS is used, add: `#define DEBUG_CONSOLE_SYNCHRONIZATION_XXXX 3` in this configuration file and implement the synchronization in `fsl.log.c`.

synchronization for baremetal software

#### 40.6.2.7 **#define DEBUG\_CONSOLE\_SYNCHRONIZATION\_MODE DEBUG\_CONSOLE\_SYNCHRONIZATION\_BM**

If other RTOS is used, you can implement the RTOS's specific synchronization mechanism in `fsl.log.c`. If synchronization is disabled, log maybe messed on terminal.

#### 40.6.2.8 **#define BOARD\_USE\_VIRTUALCOM 0U**

# Chapter 41

## Notification Framework

### 41.1 Overview

This section describes the programming interface of the Notifier driver.

### 41.2 Notifier Overview

The Notifier provides a configuration dynamic change service. Based on this service, applications can switch between pre-defined configurations. The Notifier enables drivers and applications to register callback functions to this framework. Each time that the configuration is changed, drivers and applications receive a notification and change their settings. To simplify, the Notifier only supports the static callback registration. This means that, for applications, all callback functions are collected into a static table and passed to the Notifier.

These are the steps for the configuration transition.

1. Before configuration transition, the Notifier sends a "BEFORE" message to the callback table. When this message is received, IP drivers should check whether any current processes can be stopped and stop them. If the processes cannot be stopped, the callback function returns an error.  
The Notifier supports two types of transition policies, a graceful policy and a forceful policy. When the graceful policy is used, if some callbacks return an error while sending a "BEFORE" message, the configuration transition stops and the Notifier sends a "RECOVER" message to all drivers that have stopped. Then, these drivers can recover the previous status and continue to work. When the forceful policy is used, drivers are stopped forcefully.
2. After the "BEFORE" message is processed successfully, the system switches to the new configuration.
3. After the configuration changes, the Notifier sends an "AFTER" message to the callback table to notify drivers that the configuration transition is finished.

This example shows how to use the Notifier in the Power Manager application.

```
#include "fsl_notifier.h"

// Definition of the Power Manager callback.
status_t callback0(notifier_notification_block_t *notify, void *data)
{
 status_t ret = kStatus_Success;

 ...
 ...
 ...

 return ret;
}

// Definition of the Power Manager user function.
status_t APP_PowerModeSwitch(notifier_user_config_t *targetConfig, void *
 userData)
```



```

{
 ...
 ...
 ...
}
...
...
...
...
...
// Main function.
int main(void)
{
 // Define a notifier handle.
 notifier_handle_t powerModeHandle;

 // Callback configuration.
 user_callback_data_t callbackData0;

 notifier_callback_config_t callbackCfg0 = {callback0,
 kNOTIFIER_CallbackBeforeAfter,
 (void *)&callbackData0};

 notifier_callback_config_t callbacks[] = {callbackCfg0};

 // Power mode configurations.
 power_user_config_t vlprConfig;
 power_user_config_t stopConfig;

 notifier_user_config_t *powerConfigs[] = {&vlprConfig, &stopConfig};

 // Definition of a transition to and out the power modes.
 vlprConfig.mode = kAPP_PowerModeVlpr;
 vlprConfig.enableLowPowerWakeUpOnInterrupt = false;

 stopConfig = vlprConfig;
 stopConfig.mode = kAPP_PowerModeStop;

 // Create Notifier handle.
 NOTIFIER_CreateHandle(&powerModeHandle, powerConfigs, 2U, callbacks, 1U,
 APP_PowerModeSwitch, NULL);
 ...
 ...
 // Power mode switch.
 NOTIFIER_switchConfig(&powerModeHandle, targetConfigIndex,
 kNOTIFIER_PolicyAgreement);
}

```

## Data Structures

- struct [\\_notifier\\_notification\\_block](#)  
*notification block passed to the registered callback function. [More...](#)*
- struct [\\_notifier\\_callback\\_config](#)  
*Callback configuration structure. [More...](#)*
- struct [\\_notifier\\_handle](#)  
*Notifier handle structure. [More...](#)*

## Typedefs

- typedef enum [\\_notifier\\_policy](#) [notifier\\_policy\\_t](#)  
*Notifier policies.*
- typedef enum  
[\\_notifier\\_notification\\_type](#) [notifier\\_notification\\_type\\_t](#)

- *Notification type.*
- typedef enum  
   \_notifier\_callback\_type\_notifier\_callback\_type\_t  
   *The callback type, which indicates kinds of notification the callback handles.*
- typedef void\_notifier\_user\_config\_t  
   Notifier user configuration type.
- typedef status\_t(\*\_notifier\_user\_function\_t)(\_notifier\_user\_config\_t \*targetConfig, void \*userData)  
   Notifier user function prototype Use this function to execute specific operations in configuration switch.
- typedef struct  
   \_notifier\_notification\_block\_notifier\_notification\_block\_t  
   notification block passed to the registered callback function.
- typedef status\_t(\*\_notifier\_callback\_t)(\_notifier\_notification\_block\_t \*notify, void \*data)  
   Callback prototype.
- typedef struct  
   \_notifier\_callback\_config\_notifier\_callback\_config\_t  
   Callback configuration structure.
- typedef struct\_notifier\_handle\_notifier\_handle\_t  
   Notifier handle structure.

## Enumerations

- enum\_notifier\_status {  
   kStatus\_NOTIFIER\_ErrorNotificationBefore,  
   kStatus\_NOTIFIER\_ErrorNotificationAfter }  
   Notifier error codes.
- enum\_notifier\_policy {  
   kNOTIFIER\_PolicyAgreement,  
   kNOTIFIER\_PolicyForcible }  
   Notifier policies.
- enum\_notifier\_notification\_type {  
   kNOTIFIER\_NotifyRecover = 0x00U,  
   kNOTIFIER\_NotifyBefore = 0x01U,  
   kNOTIFIER\_NotifyAfter = 0x02U }  
   Notification type.
- enum\_notifier\_callback\_type {  
   kNOTIFIER\_CallbackBefore = 0x01U,  
   kNOTIFIER\_CallbackAfter = 0x02U,  
   kNOTIFIER\_CallbackBeforeAfter = 0x03U }  
   The callback type, which indicates kinds of notification the callback handles.

## Functions

- status\_t NOTIFIER\_CreateHandle (\_notifier\_handle\_t \*notifierHandle, \_notifier\_user\_config\_t \*\*configs, uint8\_t configsNumber, \_notifier\_callback\_config\_t \*callbacks, uint8\_t callbacksNumber, \_notifier\_user\_function\_t userFunction, void \*userData)  
   Creates a Notifier handle.
- status\_t NOTIFIER\_SwitchConfig (\_notifier\_handle\_t \*notifierHandle, uint8\_t configIndex, \_notifier\_policy\_t policy)  
   Switches the configuration according to a pre-defined structure.

- uint8\_t [NOTIFIER\\_GetErrorCallbackIndex](#) (notifier\_handle\_t \*notifierHandle)  
*This function returns the last failed notification callback.*

## 41.3 Data Structure Documentation

### 41.3.1 struct \_notifier\_notification\_block

#### Data Fields

- [notifier\\_user\\_config\\_t](#) \* targetConfig  
*Pointer to target configuration.*
- [notifier\\_policy\\_t](#) policy  
*Configure transition policy.*
- [notifier\\_notification\\_type\\_t](#) notifyType  
*Configure notification type.*

#### Field Documentation

- (1) [notifier\\_user\\_config\\_t](#)\* [\\_notifier\\_notification\\_block::targetConfig](#)
- (2) [notifier\\_policy\\_t](#) [\\_notifier\\_notification\\_block::policy](#)
- (3) [notifier\\_notification\\_type\\_t](#) [\\_notifier\\_notification\\_block::notifyType](#)

### 41.3.2 struct \_notifier\_callback\_config

This structure holds the configuration of callbacks. Callbacks of this type are expected to be statically allocated. This structure contains the following application-defined data. callback - pointer to the callback function callbackType - specifies when the callback is called callbackData - pointer to the data passed to the callback.

#### Data Fields

- [notifier\\_callback\\_t](#) callback  
*Pointer to the callback function.*
- [notifier\\_callback\\_type\\_t](#) callbackType  
*Callback type.*
- void \* [callbackData](#)  
*Pointer to the data passed to the callback.*

## Field Documentation

- (1) `notifier_callback_t_notifier_callback_config::callback`
- (2) `notifier_callback_type_t_notifier_callback_config::callbackType`
- (3) `void*_notifier_callback_config::callbackData`

41.3.3 struct `_notifier_handle`

Notifier handle structure. Contains data necessary for the Notifier proper function. Stores references to registered configurations, callbacks, information about their numbers, user function, user data, and other internal data. [NOTIFIER\\_CreateHandle\(\)](#) must be called to initialize this handle.

## Data Fields

- [notifier\\_user\\_config\\_t](#) \*\* `configsTable`  
*Pointer to configure table.*
- `uint8_t` [configsNumber](#)  
*Number of configurations.*
- [notifier\\_callback\\_config\\_t](#) \* `callbacksTable`  
*Pointer to callback table.*
- `uint8_t` [callbacksNumber](#)  
*Maximum number of callback configurations.*
- `uint8_t` [errorCallbackIndex](#)  
*Index of callback returns error.*
- `uint8_t` [currentConfigIndex](#)  
*Index of current configuration.*
- [notifier\\_user\\_function\\_t](#) `userFunction`  
*User function.*
- `void *` [userData](#)  
*User data passed to user function.*

## Field Documentation

- (1) `notifier_user_config_t** _notifier_handle::configsTable`
- (2) `uint8_t _notifier_handle::configsNumber`
- (3) `notifier_callback_config_t* _notifier_handle::callbacksTable`
- (4) `uint8_t _notifier_handle::callbacksNumber`
- (5) `uint8_t _notifier_handle::errorCallbackIndex`
- (6) `uint8_t _notifier_handle::currentConfigIndex`
- (7) `notifier_user_function_t _notifier_handle::userFunction`
- (8) `void* _notifier_handle::userData`

## 41.4 Typedef Documentation

### 41.4.1 typedef enum \_notifier\_policy notifier\_policy\_t

Defines whether the user function execution is forced or not. For `kNOTIFIER_PolicyForcible`, the user function is executed regardless of the callback results, while `kNOTIFIER_PolicyAgreement` policy is used to exit [NOTIFIER\\_SwitchConfig\(\)](#) when any of the callbacks returns error code. See also [NOTIFIER\\_SwitchConfig\(\)](#) description.

### 41.4.2 typedef enum \_notifier\_notification\_type notifier\_notification\_type\_t

Used to notify registered callbacks

### 41.4.3 typedef enum \_notifier\_callback\_type notifier\_callback\_type\_t

Used in the callback configuration structure (`notifier_callback_config_t`) to specify when the registered callback is called during configuration switch initiated by the [NOTIFIER\\_SwitchConfig\(\)](#). Callback can be invoked in following situations.

- Before the configuration switch (Callback return value can affect [NOTIFIER\\_SwitchConfig\(\)](#) execution. See the [NOTIFIER\\_SwitchConfig\(\)](#) and `notifier_policy_t` documentation).
- After an unsuccessful attempt to switch configuration
- After a successful configuration switch

#### 41.4.4 typedef void notifier\_user\_config\_t

Reference of the user defined configuration is stored in an array; the notifier switches between these configurations based on this array.

#### 41.4.5 typedef status\_t(\* notifier\_user\_function\_t)(notifier\_user\_config\_t \*targetConfig, void \*userData)

Before and after this function execution, different notification is sent to registered callbacks. If this function returns any error code, [NOTIFIER\\_SwitchConfig\(\)](#) exits.

Parameters

|                     |                                                        |
|---------------------|--------------------------------------------------------|
| <i>targetConfig</i> | target Configuration.                                  |
| <i>userData</i>     | Refers to other specific data passed to user function. |

Returns

An error code or kStatus\_Success.

#### 41.4.6 typedef struct \_notifier\_notification\_block notifier\_notification\_block\_t

#### 41.4.7 typedef status\_t(\* notifier\_callback\_t)(notifier\_notification\_block\_t \*notify, void \*data)

Declaration of a callback. It is common for registered callbacks. Reference to function of this type is part of the notifier\_callback\_config\_t callback configuration structure. Depending on callback type, function of this prototype is called (see [NOTIFIER\\_SwitchConfig\(\)](#)) before configuration switch, after it or in both use cases to notify about the switch progress (see notifier\_callback\_type\_t). When called, the type of the notification is passed as a parameter along with the reference to the target configuration structure (see notifier\_notification\_block\_t) and any data passed during the callback registration. When notified before the configuration switch, depending on the configuration switch policy (see notifier\_policy\_t), the callback may deny the execution of the user function by returning an error code different than kStatus\_Success (see [NOTIFIER\\_SwitchConfig\(\)](#)).

Parameters

|               |                                                                                                                                                            |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>notify</i> | Notification block.                                                                                                                                        |
| <i>data</i>   | Callback data. Refers to the data passed during callback registration. Intended to pass any driver or application data such as internal state information. |

Returns

An error code or `kStatus_Success`.

#### 41.4.8 `typedef struct _notifier_callback_config notifier_callback_config_t`

This structure holds the configuration of callbacks. Callbacks of this type are expected to be statically allocated. This structure contains the following application-defined data. `callback` - pointer to the callback function `callbackType` - specifies when the callback is called `callbackData` - pointer to the data passed to the callback.

#### 41.4.9 `typedef struct _notifier_handle notifier_handle_t`

Notifier handle structure. Contains data necessary for the Notifier proper function. Stores references to registered configurations, callbacks, information about their numbers, user function, user data, and other internal data. [NOTIFIER\\_CreateHandle\(\)](#) must be called to initialize this handle.

### 41.5 Enumeration Type Documentation

#### 41.5.1 `enum _notifier_status`

Used as return value of Notifier functions.

Enumerator

***kStatus\_NOTIFIER\_ErrorNotificationBefore*** An error occurs during send "BEFORE" notification.

***kStatus\_NOTIFIER\_ErrorNotificationAfter*** An error occurs during send "AFTER" notification.

#### 41.5.2 `enum _notifier_policy`

Defines whether the user function execution is forced or not. For `kNOTIFIER_PolicyForcible`, the user function is executed regardless of the callback results, while `kNOTIFIER_PolicyAgreement` policy is used to exit [NOTIFIER\\_SwitchConfig\(\)](#) when any of the callbacks returns error code. See also [NOTIFIER\\_SwitchConfig\(\)](#) description.

Enumerator

***kNOTIFIER\_PolicyAgreement*** [NOTIFIER\\_SwitchConfig\(\)](#) method is exited when any of the callbacks returns error code.

***kNOTIFIER\_PolicyForcible*** The user function is executed regardless of the results.

### 41.5.3 enum \_notifier\_notification\_type

Used to notify registered callbacks

Enumerator

***kNOTIFIER\_NotifyRecover*** Notify IP to recover to previous work state.

***kNOTIFIER\_NotifyBefore*** Notify IP that configuration setting is going to change.

***kNOTIFIER\_NotifyAfter*** Notify IP that configuration setting has been changed.

### 41.5.4 enum \_notifier\_callback\_type

Used in the callback configuration structure ([notifier\\_callback\\_config\\_t](#)) to specify when the registered callback is called during configuration switch initiated by the [NOTIFIER\\_SwitchConfig\(\)](#). Callback can be invoked in following situations.

- Before the configuration switch (Callback return value can affect [NOTIFIER\\_SwitchConfig\(\)](#) execution. See the [NOTIFIER\\_SwitchConfig\(\)](#) and [notifier\\_policy\\_t](#) documentation).
- After an unsuccessful attempt to switch configuration
- After a successful configuration switch

Enumerator

***kNOTIFIER\_CallbackBefore*** Callback handles BEFORE notification.

***kNOTIFIER\_CallbackAfter*** Callback handles AFTER notification.

***kNOTIFIER\_CallbackBeforeAfter*** Callback handles BEFORE and AFTER notification.

## 41.6 Function Documentation

**41.6.1** `status_t NOTIFIER_CreateHandle ( notifier_handle_t * notifierHandle,  
notifier_user_config_t ** configs, uint8_t configsNumber, notifier_callback-  
_config_t * callbacks, uint8_t callbacksNumber, notifier_user_function_t  
userFunction, void * userData )`



## Parameters

|                         |                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <i>notifierHandle</i>   | A pointer to the notifier handle.                                                                                                       |
| <i>configs</i>          | A pointer to an array with references to all configurations which is handled by the Notifier.                                           |
| <i>configsNumber</i>    | Number of configurations. Size of the configuration array.                                                                              |
| <i>callbacks</i>        | A pointer to an array of callback configurations. If there are no callbacks to register during Notifier initialization, use NULL value. |
| <i>callbacks-Number</i> | Number of registered callbacks. Size of the callbacks array.                                                                            |
| <i>userFunction</i>     | User function.                                                                                                                          |
| <i>userData</i>         | User data passed to user function.                                                                                                      |

## Returns

An error Code or kStatus\_Success.

#### 41.6.2 **status\_t NOTIFIER\_SwitchConfig ( notifier\_handle\_t \* *notifierHandle*, uint8\_t *configIndex*, notifier\_policy\_t *policy* )**

This function sets the system to the target configuration. Before transition, the Notifier sends notifications to all callbacks registered to the callback table. Callbacks are invoked in the following order: All registered callbacks are notified ordered by index in the callbacks array. The same order is used for before and after switch notifications. The notifications before the configuration switch can be used to obtain confirmation about the change from registered callbacks. If any registered callback denies the configuration change, further execution of this function depends on the notifier policy: the configuration change is either forced (kNOTIFIER\_PolicyForcible) or exited (kNOTIFIER\_PolicyAgreement). When configuration change is forced, the result of the before switch notifications are ignored. If an agreement is required, if any callback returns an error code, further notifications before switch notifications are cancelled and all already notified callbacks are re-invoked. The index of the callback which returned error code during pre-switch notifications is stored (any error codes during callbacks re-invocation are ignored) and NOTIFIER\_GetErrorCallback() can be used to get it. Regardless of the policies, if any callback returns an error code, an error code indicating in which phase the error occurred is returned when [NOTIFIER\\_SwitchConfig\(\)](#) exits.

## Parameters

|                       |                                                                            |
|-----------------------|----------------------------------------------------------------------------|
| <i>notifierHandle</i> | pointer to notifier handle                                                 |
| <i>configIndex</i>    | Index of the target configuration.                                         |
| <i>policy</i>         | Transaction policy, kNOTIFIER_PolicyAgreement or kNOTIFIER_PolicyForcible. |

## Returns

An error code or kStatus\_Success.

### 41.6.3 uint8\_t NOTIFIER\_GetErrorCallbackIndex ( notifier\_handle\_t \* *notifierHandle* )

This function returns an index of the last callback that failed during the configuration switch while the last [NOTIFIER\\_SwitchConfig\(\)](#) was called. If the last [NOTIFIER\\_SwitchConfig\(\)](#) call ended successfully value equal to callbacks number is returned. The returned value represents an index in the array of static call-backs.

## Parameters

|                       |                                |
|-----------------------|--------------------------------|
| <i>notifierHandle</i> | Pointer to the notifier handle |
|-----------------------|--------------------------------|

## Returns

Callback Index of the last failed callback or value equal to callbacks count.

# Chapter 42

## Shell

### 42.1 Overview

This section describes the programming interface of the Shell middleware.

Shell controls MCUs by commands via the specified communication peripheral based on the debug console driver.

### 42.2 Function groups

#### 42.2.1 Initialization

To initialize the Shell middleware, call the [SHELL\\_Init\(\)](#) function with these parameters. This function automatically enables the middleware.

```
shell_status_t SHELL_Init(shell_handle_t shellHandle,
 serial_handle_t serialHandle, char *prompt);
```

Then, after the initialization was successful, call a command to control MCUs.

This example shows how to call the [SHELL\\_Init\(\)](#) given the user configuration structure.

```
SHELL_Init(s_shellHandle, s_serialHandle, "Test@SHELL>");
```

#### 42.2.2 Advanced Feature

- Support to get a character from standard input devices.

```
static shell_status_t SHELL_GetChar(shell_context_handle_t *shellContextHandle, uint8_t *ch);
```

| Commands | Description                       |
|----------|-----------------------------------|
| help     | List all the registered commands. |
| exit     | Exit program.                     |

#### 42.2.3 Shell Operation

```
SHELL_Init(s_shellHandle, s_serialHandle, "Test@SHELL>");
SHELL_Task((s_shellHandle);
```

## Data Structures

- struct [\\_shell\\_command](#)  
User command data configuration structure. [More...](#)

## Macros

- #define [SHELL\\_NON\\_BLOCKING\\_MODE SERIAL\\_MANAGER\\_NON\\_BLOCKING\\_MODE](#)  
Whether use non-blocking mode.
- #define [SHELL\\_AUTO\\_COMPLETE](#) (1U)  
Macro to set on/off auto-complete feature.
- #define [SHELL\\_BUFFER\\_SIZE](#) (64U)  
Macro to set console buffer size.
- #define [SHELL\\_MAX\\_ARGS](#) (8U)  
Macro to set maximum arguments in command.
- #define [SHELL\\_HISTORY\\_COUNT](#) (3U)  
Macro to set maximum count of history commands.
- #define [SHELL\\_IGNORE\\_PARAMETER\\_COUNT](#) (0xFF)  
Macro to bypass arguments check.
- #define [SHELL\\_HANDLE\\_SIZE](#)  
The handle size of the shell module.
- #define [SHELL\\_USE\\_COMMON\\_TASK](#) (0U)  
Macro to determine whether use common task.
- #define [SHELL\\_TASK\\_PRIORITY](#) (2U)  
Macro to set shell task priority.
- #define [SHELL\\_TASK\\_STACK\\_SIZE](#) (1000U)  
Macro to set shell task stack size.
- #define [SHELL\\_PRINT\\_COPYRIGHT](#) (1U)  
Whether print copyright.
- #define [SHELL\\_HANDLE\\_DEFINE](#)(name) uint32\_t name[(([SHELL\\_HANDLE\\_SIZE](#) + sizeof(uint32\_t) - 1U) / sizeof(uint32\_t))]  
Defines the shell handle.
- #define [SHELL\\_COMMAND\\_DEFINE](#)(command, descriptor, callback, paramCount)  
Defines the shell command structure.
- #define [SHELL\\_COMMAND](#)(command) &g\_shellCommand##command  
Gets the shell command pointer.

## Typedefs

- typedef enum [\\_shell\\_status](#) [shell\\_status\\_t](#)  
Shell status.
- typedef void \* [shell\\_handle\\_t](#)  
The handle of the shell module.
- typedef [shell\\_status\\_t](#)(\* [cmd\\_function\\_t](#))([shell\\_handle\\_t](#) shellHandle, int32\_t argc, char \*\*argv)  
User command function prototype.
- typedef struct [\\_shell\\_command](#) [shell\\_command\\_t](#)  
User command data configuration structure.

## Enumerations

- enum `_shell_status` {  
`kStatus_SHELL_Success` = `kStatus_Success`,  
`kStatus_SHELL_Error` = `MAKE_STATUS(kStatusGroup_SHELL, 1)`,  
`kStatus_SHELL_OpenWriteHandleFailed` = `MAKE_STATUS(kStatusGroup_SHELL, 2)`,  
`kStatus_SHELL_OpenReadHandleFailed` = `MAKE_STATUS(kStatusGroup_SHELL, 3)`,  
`kStatus_SHELL_RetUsage` = `MAKE_STATUS(kStatusGroup_SHELL, 4)` }  
*Shell status.*

## Shell functional operation

- `shell_status_t SHELL_Init` (`shell_handle_t` shellHandle, `serial_handle_t` serialHandle, `char *prompt`)  
*Initializes the shell module.*
- `shell_status_t SHELL_RegisterCommand` (`shell_handle_t` shellHandle, `shell_command_t *shellCommand`)  
*Registers the shell command.*
- `shell_status_t SHELL_UnregisterCommand` (`shell_command_t *shellCommand`)  
*Unregisters the shell command.*
- `shell_status_t SHELL_Write` (`shell_handle_t` shellHandle, `const char *buffer`, `uint32_t length`)  
*Sends data to the shell output stream.*
- `int SHELL_Printf` (`shell_handle_t` shellHandle, `const char *formatString`,...)  
*Writes formatted output to the shell output stream.*
- `shell_status_t SHELL_WriteSynchronization` (`shell_handle_t` shellHandle, `const char *buffer`, `uint32_t length`)  
*Sends data to the shell output stream with OS synchronization.*
- `int SHELL_PrintfSynchronization` (`shell_handle_t` shellHandle, `const char *formatString`,...)  
*Writes formatted output to the shell output stream with OS synchronization.*
- `void SHELL_ChangePrompt` (`shell_handle_t` shellHandle, `char *prompt`)  
*Change shell prompt.*
- `void SHELL_PrintPrompt` (`shell_handle_t` shellHandle)  
*Print shell prompt.*
- `void SHELL_Task` (`shell_handle_t` shellHandle)  
*The task function for Shell.*
- `static bool SHELL_checkRunningInIsr` (`void`)  
*Check if code is running in ISR.*

## 42.3 Data Structure Documentation

### 42.3.1 struct `_shell_command`

#### Data Fields

- `const char * pcCommand`  
*The command that is executed.*
- `char * pcHelpString`  
*String that describes how to use the command.*
- `const cmd_function_t pFuncCallBack`

- *A pointer to the callback function that returns the output generated by the command.*
- `uint8_t cExpectedNumberOfParameters`  
*Commands expect a fixed number of parameters, which may be zero.*
- `list_element_t link`  
*link of the element*

## Field Documentation

### (1) `const char* _shell_command::pcCommand`

For example "help". It must be all lower case.

### (2) `char* _shell_command::pcHelpString`

It should start with the command itself, and end with "\r\n". For example "help: Returns a list of all the commands\r\n".

### (3) `const cmd_function_t _shell_command::pFuncCallBack`

### (4) `uint8_t _shell_command::cExpectedNumberOfParameters`

## 42.4 Macro Definition Documentation

### 42.4.1 `#define SHELL_NON_BLOCKING_MODE SERIAL_MANAGER_NON_BLOCKING_MODE`

### 42.4.2 `#define SHELL_AUTO_COMPLETE (1U)`

### 42.4.3 `#define SHELL_BUFFER_SIZE (64U)`

### 42.4.4 `#define SHELL_MAX_ARGS (8U)`

### 42.4.5 `#define SHELL_HISTORY_COUNT (3U)`

### 42.4.6 `#define SHELL_HANDLE_SIZE`

Value:

```
(160U + SHELL_HISTORY_COUNT * SHELL_BUFFER_SIZE +
SHELL_BUFFER_SIZE + SERIAL_MANAGER_READ_HANDLE_SIZE + \
SERIAL_MANAGER_WRITE_HANDLE_SIZE)
```

It is the sum of the `SHELL_HISTORY_COUNT * SHELL_BUFFER_SIZE + SHELL_BUFFER_SIZE + SERIAL_MANAGER_READ_HANDLE_SIZE + SERIAL_MANAGER_WRITE_HANDLE_SIZE`

**42.4.7 #define SHELL\_USE\_COMMON\_TASK (0U)****42.4.8 #define SHELL\_TASK\_PRIORITY (2U)****42.4.9 #define SHELL\_TASK\_STACK\_SIZE (1000U)****42.4.10 #define SHELL\_PRINT\_COPYRIGHT (1U)****42.4.11 #define SHELL\_HANDLE\_DEFINE( *name* ) uint32\_t  
name[(((SHELL\_HANDLE\_SIZE + sizeof(uint32\_t) - 1U) / sizeof(uint32\_t)))]**

This macro is used to define a 4 byte aligned shell handle. Then use "(shell\_handle\_t)name" to get the shell handle.

The macro should be global and could be optional. You could also define shell handle by yourself.

This is an example,

```
* SHELL_HANDLE_DEFINE(shellHandle);
*
```

## Parameters

|             |                                      |
|-------------|--------------------------------------|
| <i>name</i> | The name string of the shell handle. |
|-------------|--------------------------------------|

**42.4.12 #define SHELL\_COMMAND\_DEFINE( *command*, *descriptor*, *callback*,  
*paramCount* )****Value:**

```
\
shell_command_t g_shellCommand##command = {
 (#command), (descriptor), (callback), (paramCount), {0},
}
\
```

This macro is used to define the shell command structure [shell\\_command\\_t](#). And then uses the macro SHELL\_COMMAND to get the command structure pointer. The macro should not be used in any function.

This is a example,

```
* SHELL_COMMAND_DEFINE(exit, "\r\n\"exit\": Exit program\r\n", SHELL_ExitCommand, 0);
* SHELL_RegisterCommand(s_shellHandle, SHELL_COMMAND(exit));
*
```

### Parameters

|                   |                                                                                                                              |
|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| <i>command</i>    | The command string of the command. The double quotes do not need. Such as exit for "exit", help for "Help", read for "read". |
| <i>descriptor</i> | The description of the command is used for showing the command usage when "help" is typing.                                  |
| <i>callback</i>   | The callback of the command is used to handle the command line when the input command is matched.                            |
| <i>paramCount</i> | The max parameter count of the current command.                                                                              |

### 42.4.13 #define SHELL\_COMMAND( *command* ) &g\_shellCommand##command

This macro is used to get the shell command pointer. The macro should not be used before the macro SHELL\_COMMAND\_DEFINE is used.

### Parameters

|                |                                                                                                                              |
|----------------|------------------------------------------------------------------------------------------------------------------------------|
| <i>command</i> | The command string of the command. The double quotes do not need. Such as exit for "exit", help for "Help", read for "read". |
|----------------|------------------------------------------------------------------------------------------------------------------------------|

## 42.5 Typedef Documentation

### 42.5.1 typedef shell\_status\_t(\* cmd\_function\_t)(shell\_handle\_t shellHandle, int32\_t argc, char \*\*argv)

### 42.5.2 typedef struct \_shell\_command shell\_command\_t

## 42.6 Enumeration Type Documentation

### 42.6.1 enum \_shell\_status

### Enumerator

*kStatus\_SHELL\_Success* Success.  
*kStatus\_SHELL\_Error* Failed.  
*kStatus\_SHELL\_OpenWriteHandleFailed* Open write handle failed.  
*kStatus\_SHELL\_OpenReadHandleFailed* Open read handle failed.  
*kStatus\_SHELL\_RetUsage* RetUsage for print cmd usage.



## 42.7 Function Documentation

### 42.7.1 `shell_status_t SHELL_Init ( shell_handle_t shellHandle, serial_handle_t serialHandle, char * prompt )`

This function must be called before calling all other Shell functions. Call operation the Shell commands with user-defined settings. The example below shows how to set up the Shell and how to call the SHELL\_Init function by passing in these parameters. This is an example.

```
* static SHELL_HANDLE_DEFINE(s_shellHandle);
* SHELL_Init((shell_handle_t)s_shellHandle, (
* serial_handle_t)s_serialHandle, "Test@SHELL>");
*
```

#### Parameters

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>shellHandle</i>  | Pointer to point to a memory space of size <a href="#">SHELL_HANDLE_SIZE</a> allocated by the caller. The handle should be 4 byte aligned, because unaligned access doesn't be supported on some devices. You can define the handle in the following two ways: <a href="#">SHELL_HANDLE_DEFINE(shellHandle)</a> ; or <code>uint32_t shellHandle[((SHELL_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t))];</code> |
| <i>serialHandle</i> | The serial manager module handle pointer.                                                                                                                                                                                                                                                                                                                                                                                 |
| <i>prompt</i>       | The string prompt pointer of Shell. Only the global variable can be passed.                                                                                                                                                                                                                                                                                                                                               |

#### Return values

|                                             |                                                  |
|---------------------------------------------|--------------------------------------------------|
| <i>kStatus_SHELL_Success</i>                | The shell initialization succeed.                |
| <i>kStatus_SHELL_Error</i>                  | An error occurred when the shell is initialized. |
| <i>kStatus_SHELL_Open-WriteHandleFailed</i> | Open the write handle failed.                    |
| <i>kStatus_SHELL_Open-ReadHandleFailed</i>  | Open the read handle failed.                     |

### 42.7.2 `shell_status_t SHELL_RegisterCommand ( shell_handle_t shellHandle, shell_command_t * shellCommand )`

This function is used to register the shell command by using the command configuration `shell_command_config_t`. This is a example,

```
* SHELL_COMMAND_DEFINE(exit, "\r\n\"exit\": Exit program\r\n", SHELL_ExitCommand, 0);
* SHELL_RegisterCommand(s_shellHandle, SHELL_COMMAND(exit));
*
```

## Parameters

|                     |                                  |
|---------------------|----------------------------------|
| <i>shellHandle</i>  | The shell module handle pointer. |
| <i>shellCommand</i> | The command element.             |

## Return values

|                              |                                    |
|------------------------------|------------------------------------|
| <i>kStatus_SHELL_Success</i> | Successfully register the command. |
| <i>kStatus_SHELL_Error</i>   | An error occurred.                 |

### 42.7.3 **shell\_status\_t** SHELL\_UnregisterCommand ( **shell\_command\_t** \* **shellCommand** )

This function is used to unregister the shell command.

## Parameters

|                     |                      |
|---------------------|----------------------|
| <i>shellCommand</i> | The command element. |
|---------------------|----------------------|

## Return values

|                              |                                      |
|------------------------------|--------------------------------------|
| <i>kStatus_SHELL_Success</i> | Successfully unregister the command. |
|------------------------------|--------------------------------------|

### 42.7.4 **shell\_status\_t** SHELL\_Write ( **shell\_handle\_t** **shellHandle**, **const char** \* **buffer**, **uint32\_t** **length** )

This function is used to send data to the shell output stream.

## Parameters

|                    |                                     |
|--------------------|-------------------------------------|
| <i>shellHandle</i> | The shell module handle pointer.    |
| <i>buffer</i>      | Start address of the data to write. |
| <i>length</i>      | Length of the data to write.        |

## Return values

|                              |                         |
|------------------------------|-------------------------|
| <i>kStatus_SHELL_Success</i> | Successfully send data. |
| <i>kStatus_SHELL_Error</i>   | An error occurred.      |

#### 42.7.5 int SHELL\_Printf ( shell\_handle\_t *shellHandle*, const char \* *formatString*, ... )

Call this function to write a formatted output to the shell output stream.

Parameters

|                     |                                  |
|---------------------|----------------------------------|
| <i>shellHandle</i>  | The shell module handle pointer. |
| <i>formatString</i> | Format string.                   |

Returns

Returns the number of characters printed or a negative value if an error occurs.

#### 42.7.6 shell\_status\_t SHELL\_WriteSynchronization ( shell\_handle\_t *shellHandle*, const char \* *buffer*, uint32\_t *length* )

This function is used to send data to the shell output stream with OS synchronization, note the function could not be called in ISR.

Parameters

|                    |                                     |
|--------------------|-------------------------------------|
| <i>shellHandle</i> | The shell module handle pointer.    |
| <i>buffer</i>      | Start address of the data to write. |
| <i>length</i>      | Length of the data to write.        |

Return values

|                              |                         |
|------------------------------|-------------------------|
| <i>kStatus_SHELL_Success</i> | Successfully send data. |
| <i>kStatus_SHELL_Error</i>   | An error occurred.      |

#### 42.7.7 int SHELL\_PrintfSynchronization ( shell\_handle\_t *shellHandle*, const char \* *formatString*, ... )

Call this function to write a formatted output to the shell output stream with OS synchronization, note the function could not be called in ISR.

## Parameters

|                     |                                  |
|---------------------|----------------------------------|
| <i>shellHandle</i>  | The shell module handle pointer. |
| <i>formatString</i> | Format string.                   |

## Returns

Returns the number of characters printed or a negative value if an error occurs.

#### 42.7.8 void SHELL\_ChangePrompt ( shell\_handle\_t *shellHandle*, char \* *prompt* )

Call this function to change shell prompt.

## Parameters

|                    |                                                  |
|--------------------|--------------------------------------------------|
| <i>shellHandle</i> | The shell module handle pointer.                 |
| <i>prompt</i>      | The string which will be used for command prompt |

## Returns

NULL.

#### 42.7.9 void SHELL\_PrintPrompt ( shell\_handle\_t *shellHandle* )

Call this function to print shell prompt.

## Parameters

|                    |                                  |
|--------------------|----------------------------------|
| <i>shellHandle</i> | The shell module handle pointer. |
|--------------------|----------------------------------|

## Returns

NULL.

#### 42.7.10 void SHELL\_Task ( shell\_handle\_t *shellHandle* )

The task function for Shell; The function should be polled by upper layer. This function does not return until Shell command exit was called.

## Parameters

|                    |                                  |
|--------------------|----------------------------------|
| <i>shellHandle</i> | The shell module handle pointer. |
|--------------------|----------------------------------|

**42.7.11 static bool SHELL\_checkRunningInIsr ( void ) [inline], [static]**

This function is used to check if code running in ISR.

## Return values

|             |                         |
|-------------|-------------------------|
| <i>TRUE</i> | if code running in ISR. |
|-------------|-------------------------|

## Chapter 43

# CODEC Driver

### 43.1 Overview

The MCUXpresso SDK provides a codec abstraction driver interface to access codec register.

#### Modules

- [CODEC Common Driver](#)
- [CODEC I2C Driver](#)
- [Da7212](#)
- [Sgt15000](#)
- [Wm8960](#)

## 43.2 CODEC Common Driver

### 43.2.1 Overview

The codec common driver provides a codec control abstraction interface.

#### Modules

- [CODEC Adapter](#)
- [Da7212\\_adapter](#)
- [Sgtl5000\\_adapter](#)
- [Wm8960\\_adapter](#)

#### Data Structures

- struct [\\_codec\\_config](#)  
*Initialize structure of the codec. [More...](#)*
- struct [\\_codec\\_capability](#)  
*codec capability [More...](#)*
- struct [\\_codec\\_handle](#)  
*Codec handle definition. [More...](#)*

#### Macros

- `#define` [CODEC\\_VOLUME\\_MAX\\_VALUE](#) (100U)  
*codec maximum volume range*

#### Typedefs

- typedef enum [\\_codec\\_audio\\_protocol](#) [codec\\_audio\\_protocol\\_t](#)  
*AUDIO format definition.*
- typedef enum [\\_codec\\_module](#) [codec\\_module\\_t](#)  
*audio codec module*
- typedef enum [\\_codec\\_module\\_ctrl\\_cmd](#) [codec\\_module\\_ctrl\\_cmd\\_t](#)  
*audio codec module control cmd*
- typedef struct [\\_codec\\_handle](#) [codec\\_handle\\_t](#)  
*codec handle declaration*
- typedef struct [\\_codec\\_config](#) [codec\\_config\\_t](#)  
*Initialize structure of the codec.*
- typedef struct [\\_codec\\_capability](#) [codec\\_capability\\_t](#)  
*codec capability*

## Enumerations

- enum {
  - kStatus\_CODEC\_NotSupport = MAKE\_STATUS(kStatusGroup\_CODEC, 0U),
  - kStatus\_CODEC\_DeviceNotRegistered = MAKE\_STATUS(kStatusGroup\_CODEC, 1U),
  - kStatus\_CODEC\_I2CBusInitialFailed,
  - kStatus\_CODEC\_I2CCommandTransferFailed }*CODEC status.*
- enum \_codec\_audio\_protocol {
  - kCODEC\_BusI2S = 0U,
  - kCODEC\_BusLeftJustified = 1U,
  - kCODEC\_BusRightJustified = 2U,
  - kCODEC\_BusPCMA = 3U,
  - kCODEC\_BusPCMB = 4U,
  - kCODEC\_BusTDM = 5U }*AUDIO format definition.*
- enum {
  - kCODEC\_AudioSampleRate8KHz = 8000U,
  - kCODEC\_AudioSampleRate11025Hz = 11025U,
  - kCODEC\_AudioSampleRate12KHz = 12000U,
  - kCODEC\_AudioSampleRate16KHz = 16000U,
  - kCODEC\_AudioSampleRate22050Hz = 22050U,
  - kCODEC\_AudioSampleRate24KHz = 24000U,
  - kCODEC\_AudioSampleRate32KHz = 32000U,
  - kCODEC\_AudioSampleRate44100Hz = 44100U,
  - kCODEC\_AudioSampleRate48KHz = 48000U,
  - kCODEC\_AudioSampleRate96KHz = 96000U,
  - kCODEC\_AudioSampleRate192KHz = 192000U,
  - kCODEC\_AudioSampleRate384KHz = 384000U }*audio sample rate definition*
- enum {
  - kCODEC\_AudioBitWidth16bit = 16U,
  - kCODEC\_AudioBitWidth20bit = 20U,
  - kCODEC\_AudioBitWidth24bit = 24U,
  - kCODEC\_AudioBitWidth32bit = 32U }*audio bit width*
- enum \_codec\_module {



```

kCODEC_ModuleADC = 0U,
kCODEC_ModuleDAC = 1U,
kCODEC_ModulePGA = 2U,
kCODEC_ModuleHeadphone = 3U,
kCODEC_ModuleSpeaker = 4U,
kCODEC_ModuleLinein = 5U,
kCODEC_ModuleLineout = 6U,
kCODEC_ModuleVref = 7U,
kCODEC_ModuleMicbias = 8U,
kCODEC_ModuleMic = 9U,
kCODEC_ModuleI2SIn = 10U,
kCODEC_ModuleI2SOut = 11U,
kCODEC_ModuleMixer = 12U }
 audio codec module
• enum _codec_module_ctrl_cmd { kCODEC_ModuleSwitchI2SInInterface = 0U }
 audio codec module control cmd
• enum {
 kCODEC_ModuleI2SInInterfacePCM = 0U,
 kCODEC_ModuleI2SInInterfaceDSD = 1U }
 audio codec module digital interface
• enum {
 kCODEC_RecordSourceDifferentialLine = 1U,
 kCODEC_RecordSourceLineInput = 2U,
 kCODEC_RecordSourceDifferentialMic = 4U,
 kCODEC_RecordSourceDigitalMic = 8U,
 kCODEC_RecordSourceSingleEndMic = 16U }
 audio codec module record source value
• enum {
 kCODEC_RecordChannelLeft1 = 1U,
 kCODEC_RecordChannelLeft2 = 2U,
 kCODEC_RecordChannelLeft3 = 4U,
 kCODEC_RecordChannelRight1 = 1U,
 kCODEC_RecordChannelRight2 = 2U,
 kCODEC_RecordChannelRight3 = 4U,
 kCODEC_RecordChannelDifferentialPositive1 = 1U,
 kCODEC_RecordChannelDifferentialPositive2 = 2U,
 kCODEC_RecordChannelDifferentialPositive3 = 4U,
 kCODEC_RecordChannelDifferentialNegative1 = 8U,
 kCODEC_RecordChannelDifferentialNegative2 = 16U,
 kCODEC_RecordChannelDifferentialNegative3 = 32U }
 audio codec record channel
• enum {

```

```

kCODEC_PlaySourcePGA = 1U,
kCODEC_PlaySourceInput = 2U,
kCODEC_PlaySourceDAC = 4U,
kCODEC_PlaySourceMixerIn = 1U,
kCODEC_PlaySourceMixerInLeft = 2U,
kCODEC_PlaySourceMixerInRight = 4U,
kCODEC_PlaySourceAux = 8U }

```

*audio codec module play source value*

- enum {
 

```

kCODEC_PlayChannelHeadphoneLeft = 1U,
kCODEC_PlayChannelHeadphoneRight = 2U,
kCODEC_PlayChannelSpeakerLeft = 4U,
kCODEC_PlayChannelSpeakerRight = 8U,
kCODEC_PlayChannelLineOutLeft = 16U,
kCODEC_PlayChannelLineOutRight = 32U,
kCODEC_PlayChannelLeft0 = 1U,
kCODEC_PlayChannelRight0 = 2U,
kCODEC_PlayChannelLeft1 = 4U,
kCODEC_PlayChannelRight1 = 8U,
kCODEC_PlayChannelLeft2 = 16U,
kCODEC_PlayChannelRight2 = 32U,
kCODEC_PlayChannelLeft3 = 64U,
kCODEC_PlayChannelRight3 = 128U }

```

*codec play channel*

- enum {
 

```

kCODEC_VolumeHeadphoneLeft = 1U,
kCODEC_VolumeHeadphoneRight = 2U,
kCODEC_VolumeSpeakerLeft = 4U,
kCODEC_VolumeSpeakerRight = 8U,
kCODEC_VolumeLineOutLeft = 16U,
kCODEC_VolumeLineOutRight = 32U,
kCODEC_VolumeLeft0 = 1UL << 0U,
kCODEC_VolumeRight0 = 1UL << 1U,
kCODEC_VolumeLeft1 = 1UL << 2U,
kCODEC_VolumeRight1 = 1UL << 3U,
kCODEC_VolumeLeft2 = 1UL << 4U,
kCODEC_VolumeRight2 = 1UL << 5U,
kCODEC_VolumeLeft3 = 1UL << 6U,
kCODEC_VolumeRight3 = 1UL << 7U,
kCODEC_VolumeDAC = 1UL << 8U }

```

*codec volume setting*

- enum {

```

kCODEC_SupportModuleADC = 1U << 0U,
kCODEC_SupportModuleDAC = 1U << 1U,
kCODEC_SupportModulePGA = 1U << 2U,
kCODEC_SupportModuleHeadphone = 1U << 3U,
kCODEC_SupportModuleSpeaker = 1U << 4U,
kCODEC_SupportModuleLinein = 1U << 5U,
kCODEC_SupportModuleLineout = 1U << 6U,
kCODEC_SupportModuleVref = 1U << 7U,
kCODEC_SupportModuleMicbias = 1U << 8U,
kCODEC_SupportModuleMic = 1U << 9U,
kCODEC_SupportModuleI2SIn = 1U << 10U,
kCODEC_SupportModuleI2SOut = 1U << 11U,
kCODEC_SupportModuleMixer = 1U << 12U,
kCODEC_SupportModuleI2SInSwitchInterface = 1U << 13U,
kCODEC_SupportPlayChannelLeft0 = 1U << 0U,
kCODEC_SupportPlayChannelRight0 = 1U << 1U,
kCODEC_SupportPlayChannelLeft1 = 1U << 2U,
kCODEC_SupportPlayChannelRight1 = 1U << 3U,
kCODEC_SupportPlayChannelLeft2 = 1U << 4U,
kCODEC_SupportPlayChannelRight2 = 1U << 5U,
kCODEC_SupportPlayChannelLeft3 = 1U << 6U,
kCODEC_SupportPlayChannelRight3 = 1U << 7U,
kCODEC_SupportPlaySourcePGA = 1U << 8U,
kCODEC_SupportPlaySourceInput = 1U << 9U,
kCODEC_SupportPlaySourceDAC = 1U << 10U,
kCODEC_SupportPlaySourceMixerIn = 1U << 11U,
kCODEC_SupportPlaySourceMixerInLeft = 1U << 12U,
kCODEC_SupportPlaySourceMixerInRight = 1U << 13U,
kCODEC_SupportPlaySourceAux = 1U << 14U,
kCODEC_SupportRecordSourceDifferentialLine = 1U << 0U,
kCODEC_SupportRecordSourceLineInput = 1U << 1U,
kCODEC_SupportRecordSourceDifferentialMic = 1U << 2U,
kCODEC_SupportRecordSourceDigitalMic = 1U << 3U,
kCODEC_SupportRecordSourceSingleEndMic = 1U << 4U,
kCODEC_SupportRecordChannelLeft1 = 1U << 6U,
kCODEC_SupportRecordChannelLeft2 = 1U << 7U,
kCODEC_SupportRecordChannelLeft3 = 1U << 8U,
kCODEC_SupportRecordChannelRight1 = 1U << 9U,
kCODEC_SupportRecordChannelRight2 = 1U << 10U,
kCODEC_SupportRecordChannelRight3 = 1U << 11U }

```

*audio codec capability*

## Functions

- `status_t CODEC_Init (codec_handle_t *handle, codec_config_t *config)`  
*Codec initialization.*
- `status_t CODEC_Deinit (codec_handle_t *handle)`  
*Codec de-initialization.*
- `status_t CODEC_SetFormat (codec_handle_t *handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth)`  
*set audio data format.*
- `status_t CODEC_ModuleControl (codec_handle_t *handle, codec_module_ctrl_cmd_t cmd, uint32_t data)`  
*codec module control.*
- `status_t CODEC_SetVolume (codec_handle_t *handle, uint32_t channel, uint32_t volume)`  
*set audio codec pl volume.*
- `status_t CODEC_SetMute (codec_handle_t *handle, uint32_t channel, bool mute)`  
*set audio codec module mute.*
- `status_t CODEC_SetPower (codec_handle_t *handle, codec_module_t module, bool powerOn)`  
*set audio codec power.*
- `status_t CODEC_SetRecord (codec_handle_t *handle, uint32_t recordSource)`  
*codec set record source.*
- `status_t CODEC_SetRecordChannel (codec_handle_t *handle, uint32_t leftRecordChannel, uint32_t rightRecordChannel)`  
*codec set record channel.*
- `status_t CODEC_SetPlay (codec_handle_t *handle, uint32_t playSource)`  
*codec set play source.*

## Driver version

- `#define FSL_CODEC_DRIVER_VERSION (MAKE_VERSION(2, 3, 1))`  
*CLOCK driver version 2.3.1.*

## 43.2.2 Data Structure Documentation

### 43.2.2.1 struct \_codec\_config

#### Data Fields

- `uint32_t codecDevType`  
*codec type*
- `void * codecDevConfig`  
*Codec device specific configuration.*

### 43.2.2.2 struct \_codec\_capability

#### Data Fields

- uint32\_t [codecModuleCapability](#)  
*codec module capability*
- uint32\_t [codecPlayCapability](#)  
*codec play capability*
- uint32\_t [codecRecordCapability](#)  
*codec record capability*
- uint32\_t [codecVolumeCapability](#)  
*codec volume capability*

### 43.2.2.3 struct \_codec\_handle

- Application should allocate a buffer with CODEC\_HANDLE\_SIZE for handle definition, such as uint8\_t codecHandleBuffer[CODEC\_HANDLE\_SIZE]; codec\_handle\_t \*codecHandle = codecHandleBuffer;

#### Data Fields

- [codec\\_config\\_t](#) \* [codecConfig](#)  
*codec configuration function pointer*
- const [codec\\_capability\\_t](#) \* [codecCapability](#)  
*codec capability*
- uint8\_t [codecDevHandle](#) [HAL\_CODEC\_HANDLER\_SIZE]  
*codec device handle*

## 43.2.3 Macro Definition Documentation

### 43.2.3.1 #define FSL\_CODEC\_DRIVER\_VERSION (MAKE\_VERSION(2, 3, 1))

## 43.2.4 Typedef Documentation

### 43.2.4.1 typedef enum \_codec\_audio\_protocol codec\_audio\_protocol\_t

## 43.2.5 Enumeration Type Documentation

### 43.2.5.1 anonymous enum

Enumerator

- kStatus\_CODEC\_NotSupport*** CODEC not support status.
- kStatus\_CODEC\_DeviceNotRegistered*** CODEC device register failed status.
- kStatus\_CODEC\_I2CBusInitialFailed*** CODEC i2c bus initialization failed status.

*kStatus\_CODEC\_I2CCommandTransferFailed* CODEC i2c bus command transfer failed status.

#### 43.2.5.2 enum \_codec\_audio\_protocol

Enumerator

*kCODEC\_BusI2S* I2S type.  
*kCODEC\_BusLeftJustified* Left justified mode.  
*kCODEC\_BusRightJustified* Right justified mode.  
*kCODEC\_BusPCMA* DSP/PCM A mode.  
*kCODEC\_BusPCMB* DSP/PCM B mode.  
*kCODEC\_BusTDM* TDM mode.

#### 43.2.5.3 anonymous enum

Enumerator

*kCODEC\_AudioSampleRate8KHz* Sample rate 8000 Hz.  
*kCODEC\_AudioSampleRate11025Hz* Sample rate 11025 Hz.  
*kCODEC\_AudioSampleRate12KHz* Sample rate 12000 Hz.  
*kCODEC\_AudioSampleRate16KHz* Sample rate 16000 Hz.  
*kCODEC\_AudioSampleRate22050Hz* Sample rate 22050 Hz.  
*kCODEC\_AudioSampleRate24KHz* Sample rate 24000 Hz.  
*kCODEC\_AudioSampleRate32KHz* Sample rate 32000 Hz.  
*kCODEC\_AudioSampleRate44100Hz* Sample rate 44100 Hz.  
*kCODEC\_AudioSampleRate48KHz* Sample rate 48000 Hz.  
*kCODEC\_AudioSampleRate96KHz* Sample rate 96000 Hz.  
*kCODEC\_AudioSampleRate192KHz* Sample rate 192000 Hz.  
*kCODEC\_AudioSampleRate384KHz* Sample rate 384000 Hz.

#### 43.2.5.4 anonymous enum

Enumerator

*kCODEC\_AudioBitWidth16bit* audio bit width 16  
*kCODEC\_AudioBitWidth20bit* audio bit width 20  
*kCODEC\_AudioBitWidth24bit* audio bit width 24  
*kCODEC\_AudioBitWidth32bit* audio bit width 32

#### 43.2.5.5 enum \_codec\_module

Enumerator

*kCODEC\_ModuleADC* codec module ADC

***kCODEC\_ModuleDAC*** codec module DAC  
***kCODEC\_ModulePGA*** codec module PGA  
***kCODEC\_ModuleHeadphone*** codec module headphone  
***kCODEC\_ModuleSpeaker*** codec module speaker  
***kCODEC\_ModuleLinein*** codec module linein  
***kCODEC\_ModuleLineout*** codec module lineout  
***kCODEC\_ModuleVref*** codec module VREF  
***kCODEC\_ModuleMicbias*** codec module MIC BIAS  
***kCODEC\_ModuleMic*** codec module MIC  
***kCODEC\_ModuleI2SIn*** codec module I2S in  
***kCODEC\_ModuleI2SOut*** codec module I2S out  
***kCODEC\_ModuleMixer*** codec module mixer

#### 43.2.5.6 enum \_codec\_module\_ctrl\_cmd

Enumerator

***kCODEC\_ModuleSwitchI2SInInterface*** module digital interface siwtch.

#### 43.2.5.7 anonymous enum

Enumerator

***kCODEC\_ModuleI2SInInterfacePCM*** Pcm interface.  
***kCODEC\_ModuleI2SInInterfaceDSD*** DSD interface.

#### 43.2.5.8 anonymous enum

Enumerator

***kCODEC\_RecordSourceDifferentialLine*** record source from differential line  
***kCODEC\_RecordSourceLineInput*** record source from line input  
***kCODEC\_RecordSourceDifferentialMic*** record source from differential mic  
***kCODEC\_RecordSourceDigitalMic*** record source from digital microphone  
***kCODEC\_RecordSourceSingleEndMic*** record source from single microphone

#### 43.2.5.9 anonymous enum

Enumerator

***kCODEC\_RecordChannelLeft1*** left record channel 1  
***kCODEC\_RecordChannelLeft2*** left record channel 2  
***kCODEC\_RecordChannelLeft3*** left record channel 3  
***kCODEC\_RecordChannelRight1*** right record channel 1

*kCODEC\_RecordChannelRight2* right record channel 2  
*kCODEC\_RecordChannelRight3* right record channel 3  
*kCODEC\_RecordChannelDifferentialPositive1* differential positive record channel 1  
*kCODEC\_RecordChannelDifferentialPositive2* differential positive record channel 2  
*kCODEC\_RecordChannelDifferentialPositive3* differential positive record channel 3  
*kCODEC\_RecordChannelDifferentialNegative1* differential negative record channel 1  
*kCODEC\_RecordChannelDifferentialNegative2* differential negative record channel 2  
*kCODEC\_RecordChannelDifferentialNegative3* differential negative record channel 3

#### 43.2.5.10 anonymous enum

Enumerator

*kCODEC\_PlaySourcePGA* play source PGA, bypass ADC  
*kCODEC\_PlaySourceInput* play source Input3  
*kCODEC\_PlaySourceDAC* play source DAC  
*kCODEC\_PlaySourceMixerIn* play source mixer in  
*kCODEC\_PlaySourceMixerInLeft* play source mixer in left  
*kCODEC\_PlaySourceMixerInRight* play source mixer in right  
*kCODEC\_PlaySourceAux* play source mixer in AUx

#### 43.2.5.11 anonymous enum

Enumerator

*kCODEC\_PlayChannelHeadphoneLeft* play channel headphone left  
*kCODEC\_PlayChannelHeadphoneRight* play channel headphone right  
*kCODEC\_PlayChannelSpeakerLeft* play channel speaker left  
*kCODEC\_PlayChannelSpeakerRight* play channel speaker right  
*kCODEC\_PlayChannelLineOutLeft* play channel lineout left  
*kCODEC\_PlayChannelLineOutRight* play channel lineout right  
*kCODEC\_PlayChannelLeft0* play channel left0  
*kCODEC\_PlayChannelRight0* play channel right0  
*kCODEC\_PlayChannelLeft1* play channel left1  
*kCODEC\_PlayChannelRight1* play channel right1  
*kCODEC\_PlayChannelLeft2* play channel left2  
*kCODEC\_PlayChannelRight2* play channel right2  
*kCODEC\_PlayChannelLeft3* play channel left3  
*kCODEC\_PlayChannelRight3* play channel right3

#### 43.2.5.12 anonymous enum

Enumerator

*kCODEC\_VolumeHeadphoneLeft* headphone left volume



***kCODEC\_VolumeHeadphoneRight*** headphone right volume  
***kCODEC\_VolumeSpeakerLeft*** speaker left volume  
***kCODEC\_VolumeSpeakerRight*** speaker right volume  
***kCODEC\_VolumeLineOutLeft*** lineout left volume  
***kCODEC\_VolumeLineOutRight*** lineout right volume  
***kCODEC\_VolumeLeft0*** left0 volume  
***kCODEC\_VolumeRight0*** right0 volume  
***kCODEC\_VolumeLeft1*** left1 volume  
***kCODEC\_VolumeRight1*** right1 volume  
***kCODEC\_VolumeLeft2*** left2 volume  
***kCODEC\_VolumeRight2*** right2 volume  
***kCODEC\_VolumeLeft3*** left3 volume  
***kCODEC\_VolumeRight3*** right3 volume  
***kCODEC\_VolumeDAC*** dac volume

#### 43.2.5.13 anonymous enum

Enumerator

***kCODEC\_SupportModuleADC*** codec capability of module ADC  
***kCODEC\_SupportModuleDAC*** codec capability of module DAC  
***kCODEC\_SupportModulePGA*** codec capability of module PGA  
***kCODEC\_SupportModuleHeadphone*** codec capability of module headphone  
***kCODEC\_SupportModuleSpeaker*** codec capability of module speaker  
***kCODEC\_SupportModuleLinein*** codec capability of module linein  
***kCODEC\_SupportModuleLineout*** codec capability of module lineout  
***kCODEC\_SupportModuleVref*** codec capability of module vref  
***kCODEC\_SupportModuleMicbias*** codec capability of module mic bias  
***kCODEC\_SupportModuleMic*** codec capability of module mic bias  
***kCODEC\_SupportModuleI2SIn*** codec capability of module I2S in  
***kCODEC\_SupportModuleI2SOut*** codec capability of module I2S out  
***kCODEC\_SupportModuleMixer*** codec capability of module mixer  
***kCODEC\_SupportModuleI2SInSwitchInterface*** codec capability of module I2S in switch interface  
  
***kCODEC\_SupportPlayChannelLeft0*** codec capability of play channel left 0  
***kCODEC\_SupportPlayChannelRight0*** codec capability of play channel right 0  
***kCODEC\_SupportPlayChannelLeft1*** codec capability of play channel left 1  
***kCODEC\_SupportPlayChannelRight1*** codec capability of play channel right 1  
***kCODEC\_SupportPlayChannelLeft2*** codec capability of play channel left 2  
***kCODEC\_SupportPlayChannelRight2*** codec capability of play channel right 2  
***kCODEC\_SupportPlayChannelLeft3*** codec capability of play channel left 3  
***kCODEC\_SupportPlayChannelRight3*** codec capability of play channel right 3  
***kCODEC\_SupportPlaySourcePGA*** codec capability of set playback source PGA  
***kCODEC\_SupportPlaySourceInput*** codec capability of set playback source INPUT  
***kCODEC\_SupportPlaySourceDAC*** codec capability of set playback source DAC

***kCODEC\_SupportPlaySourceMixerIn*** codec capability of set play source Mixer in  
***kCODEC\_SupportPlaySourceMixerInLeft*** codec capability of set play source Mixer in left  
***kCODEC\_SupportPlaySourceMixerInRight*** codec capability of set play source Mixer in right  
***kCODEC\_SupportPlaySourceAux*** codec capability of set play source aux  
***kCODEC\_SupportRecordSourceDifferentialLine*** codec capability of record source differential line

***kCODEC\_SupportRecordSourceLineInput*** codec capability of record source line input  
***kCODEC\_SupportRecordSourceDifferentialMic*** codec capability of record source differential mic

***kCODEC\_SupportRecordSourceDigitalMic*** codec capability of record digital mic  
***kCODEC\_SupportRecordSourceSingleEndMic*** codec capability of single end mic  
***kCODEC\_SupportRecordChannelLeft1*** left record channel 1  
***kCODEC\_SupportRecordChannelLeft2*** left record channel 2  
***kCODEC\_SupportRecordChannelLeft3*** left record channel 3  
***kCODEC\_SupportRecordChannelRight1*** right record channel 1  
***kCODEC\_SupportRecordChannelRight2*** right record channel 2  
***kCODEC\_SupportRecordChannelRight3*** right record channel 3

## 43.2.6 Function Documentation

### 43.2.6.1 `status_t CODEC_Init ( codec_handle_t * handle, codec_config_t * config )`

Parameters

|               |                       |
|---------------|-----------------------|
| <i>handle</i> | codec handle.         |
| <i>config</i> | codec configurations. |

Returns

kStatus\_Success is success, else de-initial failed.

### 43.2.6.2 `status_t CODEC_Deinit ( codec_handle_t * handle )`

Parameters

|               |               |
|---------------|---------------|
| <i>handle</i> | codec handle. |
|---------------|---------------|

Returns

kStatus\_Success is success, else de-initial failed.

**43.2.6.3** `status_t CODEC_SetFormat ( codec_handle_t * handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth )`

## Parameters

|                   |                               |
|-------------------|-------------------------------|
| <i>handle</i>     | codec handle.                 |
| <i>mclk</i>       | master clock frequency in HZ. |
| <i>sampleRate</i> | sample rate in HZ.            |
| <i>bitWidth</i>   | bit width.                    |

## Returns

kStatus\_Success is success, else configure failed.

#### 43.2.6.4 status\_t CODEC\_ModuleControl ( codec\_handle\_t \* *handle*, codec\_module\_ctrl\_cmd\_t *cmd*, uint32\_t *data* )

This function is used for codec module control, support switch digital interface cmd, can be expand to support codec module specific feature.

## Parameters

|               |                                                                                                                                                                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | codec handle.                                                                                                                                                                                                                                                         |
| <i>cmd</i>    | module control cmd, reference _codec_module_ctrl_cmd.                                                                                                                                                                                                                 |
| <i>data</i>   | value to write, when cmd is kCODEC_ModuleRecordSourceChannel, the data should be a value combine of channel and source, please reference macro CODEC_MODULE_RECORD_SOURCE_CHANNEL(source, LP, LN, RP, RN), reference codec specific driver for detail configurations. |

## Returns

kStatus\_Success is success, else configure failed.

#### 43.2.6.5 status\_t CODEC\_SetVolume ( codec\_handle\_t \* *handle*, uint32\_t *channel*, uint32\_t *volume* )

## Parameters

|                |                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>  | codec handle.                                                                                                                              |
| <i>channel</i> | audio codec volume channel, can be a value or combine value of <code>_codec_volume_capability</code> or <code>_codec_play_channel</code> . |
| <i>volume</i>  | volume value, support 0 ~ 100, 0 is mute, 100 is the maximum volume value.                                                                 |

Returns

`kStatus_Success` is success, else configure failed.

#### 43.2.6.6 **status\_t CODEC\_SetMute ( codec\_handle\_t \* *handle*, uint32\_t *channel*, bool *mute* )**

Parameters

|                |                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>  | codec handle.                                                                                                                              |
| <i>channel</i> | audio codec volume channel, can be a value or combine value of <code>_codec_volume_capability</code> or <code>_codec_play_channel</code> . |
| <i>mute</i>    | true is mute, false is unmute.                                                                                                             |

Returns

`kStatus_Success` is success, else configure failed.

#### 43.2.6.7 **status\_t CODEC\_SetPower ( codec\_handle\_t \* *handle*, codec\_module\_t *module*, bool *powerOn* )**

Parameters

|                |                                        |
|----------------|----------------------------------------|
| <i>handle</i>  | codec handle.                          |
| <i>module</i>  | audio codec module.                    |
| <i>powerOn</i> | true is power on, false is power down. |

Returns

`kStatus_Success` is success, else configure failed.

#### 43.2.6.8 **status\_t CODEC\_SetRecord ( codec\_handle\_t \* *handle*, uint32\_t *recordSource* )**

## Parameters

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <i>handle</i>       | codec handle.                                                                                     |
| <i>recordSource</i> | audio codec record source, can be a value or combine value of <code>_codec_record_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

#### 43.2.6.9 `status_t CODEC_SetRecordChannel ( codec_handle_t * handle, uint32_t leftRecordChannel, uint32_t rightRecordChannel )`

## Parameters

|                            |                                                                                                                                                     |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>              | codec handle.                                                                                                                                       |
| <i>leftRecord-Channel</i>  | audio codec record channel, reference <code>_codec_record_channel</code> , can be a value combine of member in <code>_codec_record_channel</code> . |
| <i>rightRecord-Channel</i> | audio codec record channel, reference <code>_codec_record_channel</code> , can be a value combine of member in <code>_codec_record_channel</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

#### 43.2.6.10 `status_t CODEC_SetPlay ( codec_handle_t * handle, uint32_t playSource )`

## Parameters

|                   |                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------|
| <i>handle</i>     | codec handle.                                                                                 |
| <i>playSource</i> | audio codec play source, can be a value or combine value of <code>_codec_play_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

## 43.3 CODEC I2C Driver

### 43.3.1 Overview

The codec common driver provides a codec control abstraction interface.

### Data Structures

- struct [\\_codec\\_i2c\\_config](#)  
CODEC I2C configurations structure. [More...](#)

### Macros

- #define [CODEC\\_I2C\\_MASTER\\_HANDLER\\_SIZE](#) HAL\_I2C\_MASTER\_HANDLE\_SIZE  
codec i2c handler

### Typedefs

- typedef enum [\\_codec\\_reg\\_addr](#) [codec\\_reg\\_addr\\_t](#)  
CODEC device register address type.
- typedef enum [\\_codec\\_reg\\_width](#) [codec\\_reg\\_width\\_t](#)  
CODEC device register width.
- typedef struct [\\_codec\\_i2c\\_config](#) [codec\\_i2c\\_config\\_t](#)  
CODEC I2C configurations structure.

### Enumerations

- enum [\\_codec\\_reg\\_addr](#) {  
  [kCODEC\\_RegAddr8Bit](#) = 1U,  
  [kCODEC\\_RegAddr16Bit](#) = 2U }  
CODEC device register address type.
- enum [\\_codec\\_reg\\_width](#) {  
  [kCODEC\\_RegWidth8Bit](#) = 1U,  
  [kCODEC\\_RegWidth16Bit](#) = 2U,  
  [kCODEC\\_RegWidth32Bit](#) = 4U }  
CODEC device register width.

### Functions

- [status\\_t CODEC\\_I2C\\_Init](#) (void \*handle, uint32\_t i2cInstance, uint32\_t i2cBaudrate, uint32\_t i2cSourceClockHz)  
Codec i2c bus initialization.
- [status\\_t CODEC\\_I2C\\_Deinit](#) (void \*handle)

- *Codec i2c de-initialization.*
- [status\\_t CODEC\\_I2C\\_Send](#) (void \*handle, uint8\_t deviceAddress, uint32\_t subAddress, uint8\_t subaddressSize, uint8\_t \*txBuff, uint8\_t txBuffSize)  
*codec i2c send function.*
- [status\\_t CODEC\\_I2C\\_Receive](#) (void \*handle, uint8\_t deviceAddress, uint32\_t subAddress, uint8\_t subaddressSize, uint8\_t \*rxBuff, uint8\_t rxBuffSize)  
*codec i2c receive function.*

## 43.3.2 Data Structure Documentation

### 43.3.2.1 struct \_codec\_i2c\_config

#### Data Fields

- uint32\_t [codecI2CInstance](#)  
*i2c bus instance*
- uint32\_t [codecI2CSourceClock](#)  
*i2c bus source clock frequency*

## 43.3.3 Typedef Documentation

### 43.3.3.1 typedef enum \_codec\_reg\_addr codec\_reg\_addr\_t

### 43.3.3.2 typedef enum \_codec\_reg\_width codec\_reg\_width\_t

## 43.3.4 Enumeration Type Documentation

### 43.3.4.1 enum \_codec\_reg\_addr

#### Enumerator

- kCODEC\_RegAddr8Bit* 8-bit register address.  
*kCODEC\_RegAddr16Bit* 16-bit register address.

### 43.3.4.2 enum \_codec\_reg\_width

#### Enumerator

- kCODEC\_RegWidth8Bit* 8-bit register width.  
*kCODEC\_RegWidth16Bit* 16-bit register width.  
*kCODEC\_RegWidth32Bit* 32-bit register width.



### 43.3.5 Function Documentation

**43.3.5.1** `status_t CODEC_I2C_Init ( void * handle, uint32_t i2cInstance, uint32_t i2cBaudrate, uint32_t i2cSourceClockHz )`

## Parameters

|                          |                                                                     |
|--------------------------|---------------------------------------------------------------------|
| <i>handle</i>            | i2c master handle.                                                  |
| <i>i2cInstance</i>       | instance number of the i2c bus, such as 0 is corresponding to I2C0. |
| <i>i2cBaudrate</i>       | i2c baudrate.                                                       |
| <i>i2cSource-ClockHz</i> | i2c source clock frequency.                                         |

## Returns

kStatus\_HAL\_I2cSuccess is success, else initial failed.

### 43.3.5.2 status\_t CODEC\_I2C\_Deinit ( void \* *handle* )

## Parameters

|               |                    |
|---------------|--------------------|
| <i>handle</i> | i2c master handle. |
|---------------|--------------------|

## Returns

kStatus\_HAL\_I2cSuccess is success, else deinitial failed.

### 43.3.5.3 status\_t CODEC\_I2C\_Send ( void \* *handle*, uint8\_t *deviceAddress*, uint32\_t *subAddress*, uint8\_t *subaddressSize*, uint8\_t \* *txBuff*, uint8\_t *txBuffSize* )

## Parameters

|                       |                         |
|-----------------------|-------------------------|
| <i>handle</i>         | i2c master handle.      |
| <i>deviceAddress</i>  | codec device address.   |
| <i>subAddress</i>     | register address.       |
| <i>subaddressSize</i> | register address width. |
| <i>txBuff</i>         | tx buffer pointer.      |
| <i>txBuffSize</i>     | tx buffer size.         |

## Returns

kStatus\_HAL\_I2cSuccess is success, else send failed.

### 43.3.5.4 status\_t CODEC\_I2C\_Receive ( void \* *handle*, uint8\_t *deviceAddress*, uint32\_t *subAddress*, uint8\_t *subaddressSize*, uint8\_t \* *rxBuff*, uint8\_t *rxBuffSize* )

## Parameters

|                       |                         |
|-----------------------|-------------------------|
| <i>handle</i>         | i2c master handle.      |
| <i>deviceAddress</i>  | codec device address.   |
| <i>subAddress</i>     | register address.       |
| <i>subaddressSize</i> | register address width. |
| <i>rxBuff</i>         | rx buffer pointer.      |
| <i>rxBuffSize</i>     | rx buffer size.         |

## Returns

kStatus\_HAL\_I2cSuccess is success, else receive failed.

# Chapter 44

## Serial Manager

### 44.1 Overview

This chapter describes the programming interface of the serial manager component.

The serial manager component provides a series of APIs to operate different serial port types. The port types it supports are UART, USB CDC and SWO.

### Modules

- [Serial\\_port\\_swo](#)
- [Serial\\_port\\_uart](#)

### Data Structures

- struct [\\_serial\\_manager\\_config](#)  
*serial manager config structure [More...](#)*
- struct [\\_serial\\_manager\\_callback\\_message](#)  
*Callback message structure. [More...](#)*

### Macros

- #define [SERIAL\\_MANAGER\\_NON\\_BLOCKING\\_MODE](#) (1U)  
*Enable or disable serial manager non-blocking mode (1 - enable, 0 - disable)*
- #define [SERIAL\\_MANAGER\\_RING\\_BUFFER\\_FLOWCONTROL](#) (0U)  
*Enable or ring buffer flow control (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_UART](#) (0U)  
*Enable or disable uart port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_UART\\_DMA](#) (0U)  
*Enable or disable uart dma port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_USBCDC](#) (0U)  
*Enable or disable USB CDC port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_SWO](#) (0U)  
*Enable or disable SWO port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_VIRTUAL](#) (0U)  
*Enable or disable USB CDC virtual port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_RPMSG](#) (0U)  
*Enable or disable rPMSG port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_SPI\\_MASTER](#) (0U)  
*Enable or disable SPI Master port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_SPI\\_SLAVE](#) (0U)  
*Enable or disable SPI Slave port (1 - enable, 0 - disable)*
- #define [SERIAL\\_PORT\\_TYPE\\_BLE\\_WU](#) (0U)  
*Enable or disable BLE WU port (1 - enable, 0 - disable)*
- #define [SERIAL\\_MANAGER\\_WRITE\\_TIME\\_DELAY\\_DEFAULT\\_VALUE](#) (1U)

- Set the default delay time in ms used by `SerialManager_WriteTimeDelay()`.
- #define `SERIAL_MANAGER_READ_TIME_DELAY_DEFAULT_VALUE` (1U)
- Set the default delay time in ms used by `SerialManager_ReadTimeDelay()`.
- #define `SERIAL_MANAGER_TASK_HANDLE_RX_AVAILABLE_NOTIFY` (0U)
- Enable or disable `SerialManager_Task()` handle RX data available notify.
- #define `SERIAL_MANAGER_WRITE_HANDLE_SIZE` (44U)
- Set serial manager write handle size.
- #define `SERIAL_MANAGER_USE_COMMON_TASK` (0U)
- `SERIAL_PORT_UART_HANDLE_SIZE/SERIAL_PORT_USB_CDC_HANDLE_SIZE + serial manager dedicated size.`
- #define `SERIAL_MANAGER_HANDLE_SIZE` (`SERIAL_MANAGER_HANDLE_SIZE_TEMP + 124U`)
- Definition of serial manager handle size.
- #define `SERIAL_MANAGER_HANDLE_DEFINE(name)` `uint32_t name[(((SERIAL_MANAGER_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t)))]`
- Defines the serial manager handle.
- #define `SERIAL_MANAGER_WRITE_HANDLE_DEFINE(name)` `uint32_t name[(((SERIAL_MANAGER_WRITE_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t)))]`
- Defines the serial manager write handle.
- #define `SERIAL_MANAGER_READ_HANDLE_DEFINE(name)` `uint32_t name[(((SERIAL_MANAGER_READ_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t)))]`
- Defines the serial manager read handle.
- #define `SERIAL_MANAGER_TASK_PRIORITY` (2U)
- Macro to set serial manager task priority.
- #define `SERIAL_MANAGER_TASK_STACK_SIZE` (1000U)
- Macro to set serial manager task stack size.

## Typedefs

- typedef void \* `serial_handle_t`  
The handle of the serial manager module.
- typedef void \* `serial_write_handle_t`  
The write handle of the serial manager module.
- typedef void \* `serial_read_handle_t`  
The read handle of the serial manager module.
- typedef enum `_serial_port_type` `serial_port_type_t`  
serial port type
- typedef enum `_serial_manager_type` `serial_manager_type_t`  
serial manager type
- typedef struct `_serial_manager_config` `serial_manager_config_t`  
serial manager config structure
- typedef enum `_serial_manager_status` `serial_manager_status_t`  
serial manager error code
- typedef struct `_serial_manager_callback_message` `serial_manager_callback_message_t`  
Callback message structure.
- typedef void(\* `serial_manager_callback_t` )(void \*callbackParam, `serial_manager_callback_message_t` \*message, `serial_manager_status_t` status)  
serial manager callback function

- typedef int32\_t(\* serial\_manager\_lowpower\_critical\_callback\_t)(int32\_t power\_mode)  
*serial manager Lowpower Critical callback function*

## Enumerations

- enum \_serial\_port\_type {  
kSerialPort\_None = 0U,  
kSerialPort\_Uart = 1U,  
kSerialPort\_UsbCdc,  
kSerialPort\_Swo,  
kSerialPort\_Virtual,  
kSerialPort\_Rpmsg,  
kSerialPort\_UartDma,  
kSerialPort\_SpiMaster,  
kSerialPort\_SpiSlave,  
kSerialPort\_BleWu }  
*serial port type*
- enum \_serial\_manager\_type {  
kSerialManager\_NonBlocking = 0x0U,  
kSerialManager\_Blocking = 0x8F41U }  
*serial manager type*
- enum \_serial\_manager\_status {  
kStatus\_SerialManager\_Success = kStatus\_Success,  
kStatus\_SerialManager\_Error = MAKE\_STATUS(kStatusGroup\_SERIALMANAGER, 1),  
kStatus\_SerialManager\_Busy = MAKE\_STATUS(kStatusGroup\_SERIALMANAGER, 2),  
kStatus\_SerialManager\_Notify = MAKE\_STATUS(kStatusGroup\_SERIALMANAGER, 3),  
kStatus\_SerialManager\_Canceled,  
kStatus\_SerialManager\_HandleConflict = MAKE\_STATUS(kStatusGroup\_SERIALMANAGER, 5),  
kStatus\_SerialManager\_RingBufferOverflow,  
kStatus\_SerialManager\_NotConnected = MAKE\_STATUS(kStatusGroup\_SERIALMANAGER, 7) }  
*serial manager error code*

## Functions

- serial\_manager\_status\_t SerialManager\_Init (serial\_handle\_t serialHandle, const serial\_manager\_config\_t \*serialConfig)  
*Initializes a serial manager module with the serial manager handle and the user configuration structure.*
- serial\_manager\_status\_t SerialManager\_Deinit (serial\_handle\_t serialHandle)  
*De-initializes the serial manager module instance.*
- serial\_manager\_status\_t SerialManager\_OpenWriteHandle (serial\_handle\_t serialHandle, serial\_write\_handle\_t writeHandle)  
*Opens a writing handle for the serial manager module.*
- serial\_manager\_status\_t SerialManager\_CloseWriteHandle (serial\_write\_handle\_t writeHandle)  
*Closes a writing handle for the serial manager module.*
- serial\_manager\_status\_t SerialManager\_OpenReadHandle (serial\_handle\_t serialHandle, serial\_read\_handle\_t readHandle)

- *Opens a reading handle for the serial manager module.*
- [serial\\_manager\\_status\\_t SerialManager\\_CloseReadHandle](#) ([serial\\_read\\_handle\\_t](#) readHandle)  
*Closes a reading for the serial manager module.*
- [serial\\_manager\\_status\\_t SerialManager\\_WriteBlocking](#) ([serial\\_write\\_handle\\_t](#) writeHandle, [uint8\\_t](#) \*buffer, [uint32\\_t](#) length)  
*Transmits data with the blocking mode.*
- [serial\\_manager\\_status\\_t SerialManager\\_ReadBlocking](#) ([serial\\_read\\_handle\\_t](#) readHandle, [uint8\\_t](#) \*buffer, [uint32\\_t](#) length)  
*Reads data with the blocking mode.*
- [serial\\_manager\\_status\\_t SerialManager\\_WriteNonBlocking](#) ([serial\\_write\\_handle\\_t](#) writeHandle, [uint8\\_t](#) \*buffer, [uint32\\_t](#) length)  
*Transmits data with the non-blocking mode.*
- [serial\\_manager\\_status\\_t SerialManager\\_ReadNonBlocking](#) ([serial\\_read\\_handle\\_t](#) readHandle, [uint8\\_t](#) \*buffer, [uint32\\_t](#) length)  
*Reads data with the non-blocking mode.*
- [serial\\_manager\\_status\\_t SerialManager\\_TryRead](#) ([serial\\_read\\_handle\\_t](#) readHandle, [uint8\\_t](#) \*buffer, [uint32\\_t](#) length, [uint32\\_t](#) \*receivedLength)  
*Tries to read data.*
- [serial\\_manager\\_status\\_t SerialManager\\_CancelWriting](#) ([serial\\_write\\_handle\\_t](#) writeHandle)  
*Cancels unfinished send transmission.*
- [serial\\_manager\\_status\\_t SerialManager\\_CancelReading](#) ([serial\\_read\\_handle\\_t](#) readHandle)  
*Cancels unfinished receive transmission.*
- [serial\\_manager\\_status\\_t SerialManager\\_InstallTxCallback](#) ([serial\\_write\\_handle\\_t](#) writeHandle, [serial\\_manager\\_callback\\_t](#) callback, void \*callbackParam)  
*Installs a TX callback and callback parameter.*
- [serial\\_manager\\_status\\_t SerialManager\\_InstallRxCallback](#) ([serial\\_read\\_handle\\_t](#) readHandle, [serial\\_manager\\_callback\\_t](#) callback, void \*callbackParam)  
*Installs a RX callback and callback parameter.*
- static bool [SerialManager\\_needPollingIsr](#) (void)  
*Check if need polling ISR.*
- [serial\\_manager\\_status\\_t SerialManager\\_EnterLowpower](#) ([serial\\_handle\\_t](#) serialHandle)  
*Prepares to enter low power consumption.*
- [serial\\_manager\\_status\\_t SerialManager\\_ExitLowpower](#) ([serial\\_handle\\_t](#) serialHandle)  
*Restores from low power consumption.*
- void [SerialManager\\_SetLowpowerCriticalCb](#) (const [serial\\_manager\\_lowpower\\_critical\\_CBs\\_t](#) \*pf-Callback)  
*This function performs initialization of the callbacks structure used to disable lowpower when serial manager is active.*

## 44.2 Data Structure Documentation

### 44.2.1 struct\_serial\_manager\_config

#### Data Fields

- [uint8\\_t](#) \* [ringBuffer](#)  
*Ring buffer address, it is used to buffer data received by the hardware.*
- [uint32\\_t](#) [ringBufferSize](#)  
*The size of the ring buffer.*

- `serial_port_type_t` type  
*Serial port type.*
- `serial_manager_type_t` blockType  
*Serial manager port type.*
- `void * portConfig`  
*Serial port configuration.*

## Field Documentation

### (1) `uint8_t* _serial_manager_config::ringBuffer`

Besides, the memory space cannot be free during the lifetime of the serial manager module.

## 44.2.2 `struct _serial_manager_callback_message`

### Data Fields

- `uint8_t * buffer`  
*Transferred buffer.*
- `uint32_t length`  
*Transferred data length.*

## 44.3 Macro Definition Documentation

### 44.3.1 `#define SERIAL_MANAGER_WRITE_TIME_DELAY_DEFAULT_VALUE (1U)`

### 44.3.2 `#define SERIAL_MANAGER_READ_TIME_DELAY_DEFAULT_VALUE (1U)`

### 44.3.3 `#define SERIAL_MANAGER_USE_COMMON_TASK (0U)`

Macro to determine whether use common task.

### 44.3.4 `#define SERIAL_MANAGER_HANDLE_SIZE (SERIAL_MANAGER_HANDLE_SIZE_TEMP + 124U)`

### 44.3.5 `#define SERIAL_MANAGER_HANDLE_DEFINE( name ) uint32_t name[(((SERIAL_MANAGER_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t)))]`

This macro is used to define a 4 byte aligned serial manager handle. Then use "(serial\_handle\_t)name" to get the serial manager handle.

The macro should be global and could be optional. You could also define serial manager handle by yourself.



This is an example,

```
* SERIAL_MANAGER_HANDLE_DEFINE(serialManagerHandle);
*
```

Parameters

|             |                                               |
|-------------|-----------------------------------------------|
| <i>name</i> | The name string of the serial manager handle. |
|-------------|-----------------------------------------------|

#### 44.3.6 **#define SERIAL\_MANAGER\_WRITE\_HANDLE\_DEFINE( *name* ) uint32\_t name[(((SERIAL\_MANAGER\_WRITE\_HANDLE\_SIZE + sizeof(uint32\_t) - 1U) / sizeof(uint32\_t))]**

This macro is used to define a 4 byte aligned serial manager write handle. Then use "(serial\_write\_handle\_\*)\_name" to get the serial manager write handle.

The macro should be global and could be optional. You could also define serial manager write handle by yourself.

This is an example,

```
* SERIAL_MANAGER_WRITE_HANDLE_DEFINE(serialManagerwriteHandle);
*
```

Parameters

|             |                                                     |
|-------------|-----------------------------------------------------|
| <i>name</i> | The name string of the serial manager write handle. |
|-------------|-----------------------------------------------------|

#### 44.3.7 **#define SERIAL\_MANAGER\_READ\_HANDLE\_DEFINE( *name* ) uint32\_t name[(((SERIAL\_MANAGER\_READ\_HANDLE\_SIZE + sizeof(uint32\_t) - 1U) / sizeof(uint32\_t))]**

This macro is used to define a 4 byte aligned serial manager read handle. Then use "(serial\_read\_handle\_\*)\_name" to get the serial manager read handle.

The macro should be global and could be optional. You could also define serial manager read handle by yourself.

This is an example,

```
* SERIAL_MANAGER_READ_HANDLE_DEFINE(serialManagerReadHandle);
*
```

## Parameters

|             |                                                    |
|-------------|----------------------------------------------------|
| <i>name</i> | The name string of the serial manager read handle. |
|-------------|----------------------------------------------------|

**44.3.8 #define SERIAL\_MANAGER\_TASK\_PRIORITY (2U)****44.3.9 #define SERIAL\_MANAGER\_TASK\_STACK\_SIZE (1000U)****44.4 Enumeration Type Documentation****44.4.1 enum \_serial\_port\_type**

## Enumerator

*kSerialPort\_None* Serial port is none.  
*kSerialPort\_Uart* Serial port UART.  
*kSerialPort\_UsbCdc* Serial port USB CDC.  
*kSerialPort\_Swo* Serial port SWO.  
*kSerialPort\_Virtual* Serial port Virtual.  
*kSerialPort\_Rpmsg* Serial port RPMSG.  
*kSerialPort\_UartDma* Serial port UART DMA.  
*kSerialPort\_SpiMaster* Serial port SPIMASTER.  
*kSerialPort\_SpiSlave* Serial port SPISLAVE.  
*kSerialPort\_BleWu* Serial port BLE WU.

**44.4.2 enum \_serial\_manager\_type**

## Enumerator

*kSerialManager\_NonBlocking* None blocking handle.  
*kSerialManager\_Blocking* Blocking handle.

**44.4.3 enum \_serial\_manager\_status**

## Enumerator

*kStatus\_SerialManager\_Success* Success.  
*kStatus\_SerialManager\_Error* Failed.  
*kStatus\_SerialManager\_Busy* Busy.  
*kStatus\_SerialManager\_Notify* Ring buffer is not empty.  
*kStatus\_SerialManager\_Canceled* the non-blocking request is canceled

***kStatus\_SerialManager\_HandleConflict*** The handle is opened.

***kStatus\_SerialManager\_RingBufferOverflow*** The ring buffer is overflowed.

***kStatus\_SerialManager\_NotConnected*** The host is not connected.

## 44.5 Function Documentation

### 44.5.1 serial\_manager\_status\_t SerialManager\_Init ( serial\_handle\_t serialHandle, const serial\_manager\_config\_t \* serialConfig )

This function configures the Serial Manager module with user-defined settings. The user can configure the configuration structure. The parameter serialHandle is a pointer to point to a memory space of size [SERIAL\\_MANAGER\\_HANDLE\\_SIZE](#) allocated by the caller. The Serial Manager module supports three types of serial port, UART (includes UART, USART, LPSCI, LPUART, etc), USB CDC and sww. Please refer to [serial\\_port\\_type\\_t](#) for serial port setting. These three types can be set by using [serial\\_manager\\_config\\_t](#).

Example below shows how to use this API to configure the Serial Manager. For UART,

```
* #define SERIAL_MANAGER_RING_BUFFER_SIZE (256U)
* static SERIAL_MANAGER_HANDLE_DEFINE(s_serialHandle);
* static uint8_t s_ringBuffer[SERIAL_MANAGER_RING_BUFFER_SIZE];
*
* serial_manager_config_t config;
* serial_port_uart_config_t uartConfig;
* config.type = kSerialPort_Uart;
* config.ringBuffer = &s_ringBuffer[0];
* config.ringBufferSize = SERIAL_MANAGER_RING_BUFFER_SIZE;
* uartConfig.instance = 0;
* uartConfig.clockRate = 24000000;
* uartConfig.baudRate = 115200;
* uartConfig.parityMode = kSerialManager_UartParityDisabled;
* uartConfig.stopBitCount = kSerialManager_UartOneStopBit;
* uartConfig.enableRx = 1;
* uartConfig.enableTx = 1;
* uartConfig.enableRxRTS = 0;
* uartConfig.enableTxCTS = 0;
* config.portConfig = &uartConfig;
* SerialManager_Init((serial_handle_t)s_serialHandle, &config);
*
```

For USB CDC,

```
* #define SERIAL_MANAGER_RING_BUFFER_SIZE (256U)
* static SERIAL_MANAGER_HANDLE_DEFINE(s_serialHandle);
* static uint8_t s_ringBuffer[SERIAL_MANAGER_RING_BUFFER_SIZE];
*
* serial_manager_config_t config;
* serial_port_usb_cdc_config_t usbCdcConfig;
* config.type = kSerialPort_UsbCdc;
* config.ringBuffer = &s_ringBuffer[0];
* config.ringBufferSize = SERIAL_MANAGER_RING_BUFFER_SIZE;
* usbCdcConfig.controllerIndex = kSerialManager_UsbControllerKhci0;
* config.portConfig = &usbCdcConfig;
* SerialManager_Init((serial_handle_t)s_serialHandle, &config);
*
```

Example below shows how to use this API to configure the Serial Manager task configuration. For example if user need do specifical configuration(s\_os\_thread\_def\_serialmanager)for the serial manager task,

```

* #define SERIAL_MANAGER_RING_BUFFER_SIZE (256U)
* static SERIAL_MANAGER_HANDLE_DEFINE(s_serialHandle);
* static uint8_t s_ringBuffer[SERIAL_MANAGER_RING_BUFFER_SIZE];
* const osa_task_def_t s_os_thread_def_serialmanager = {
* .tpriority = 4,
* .instances = 1,
* .stacksize = 2048,
* };
* serial_manager_config_t config;
* serial_port_uart_config_t uartConfig;
* config.type = kSerialPort_Uart;
* config.ringBuffer = &s_ringBuffer[0];
* config.ringBufferSize = SERIAL_MANAGER_RING_BUFFER_SIZE;
* config.serialTaskConfig = (osa_task_def_t *)&s_os_thread_def_serialmanager,
* uartConfig.instance = 0;
* uartConfig.clockRate = 24000000;
* uartConfig.baudRate = 115200;
* uartConfig.parityMode = kSerialManager_UartParityDisabled;
* uartConfig.stopBitCount = kSerialManager_UartOneStopBit;
* uartConfig.enableRx = 1;
* uartConfig.enableTx = 1;
* uartConfig.enableRxRTS = 0;
* uartConfig.enableTxCTS = 0;
* config.portConfig = &uartConfig;
* SerialManager_Init((serial_handle_t)s_serialHandle, &config);
*

```

### Parameters

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>serialHandle</i> | Pointer to point to a memory space of size <a href="#">SERIAL_MANAGER_HANDLE_SIZE</a> allocated by the caller. The handle should be 4 byte aligned, because unaligned access doesn't be supported on some devices. You can define the handle in the following two ways: <a href="#">SERIAL_MANAGER_HANDLE_DEFINE(serialHandle)</a> ; or <code>uint32_t serialHandle[((SERIAL_MANAGER_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t))];</code> |
| <i>serialConfig</i> | Pointer to user-defined configuration structure.                                                                                                                                                                                                                                                                                                                                                                                                       |

### Return values

|                                      |                                                   |
|--------------------------------------|---------------------------------------------------|
| <i>kStatus_SerialManager_Error</i>   | An error occurred.                                |
| <i>kStatus_SerialManager_Success</i> | The Serial Manager module initialization succeed. |

## 44.5.2 serial\_manager\_status\_t SerialManager\_Deinit ( serial\_handle\_t serialHandle )

This function de-initializes the serial manager module instance. If the opened writing or reading handle is not closed, the function will return `kStatus_SerialManager_Busy`.

## Parameters

|                     |                                           |
|---------------------|-------------------------------------------|
| <i>serialHandle</i> | The serial manager module handle pointer. |
|---------------------|-------------------------------------------|

## Return values

|                                      |                                                 |
|--------------------------------------|-------------------------------------------------|
| <i>kStatus_SerialManager_Success</i> | The serial manager de-initialization succeed.   |
| <i>kStatus_SerialManager_Busy</i>    | Opened reading or writing handle is not closed. |

### 44.5.3 serial\_manager\_status\_t SerialManager\_OpenWriteHandle ( serial\_handle\_t serialHandle, serial\_write\_handle\_t writeHandle )

This function Opens a writing handle for the serial manager module. If the serial manager needs to be used in different tasks, the task should open a dedicated write handle for itself by calling [SerialManager\\_OpenWriteHandle](#). Since there can only one buffer for transmission for the writing handle at the same time, multiple writing handles need to be opened when the multiple transmission is needed for a task.

## Parameters

|                     |                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>serialHandle</i> | The serial manager module handle pointer. The handle should be 4 byte aligned, because unaligned access doesn't be supported on some devices.                                                                                                                                                                                                                                                       |
| <i>writeHandle</i>  | The serial manager module writing handle pointer. The handle should be 4 byte aligned, because unaligned access doesn't be supported on some devices. You can define the handle in the following two ways: <a href="#">SERIAL_MANAGER_WRITE_HANDLE_DEFINE(writeHandle)</a> ; or <code>uint32_t writeHandle[((SERIAL_MANAGER_WRITE_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t))];</code> |

## Return values

|                                             |                                |
|---------------------------------------------|--------------------------------|
| <i>kStatus_SerialManager_Error</i>          | An error occurred.             |
| <i>kStatus_SerialManager_HandleConflict</i> | The writing handle was opened. |

|                                      |                               |
|--------------------------------------|-------------------------------|
| <i>kStatus_SerialManager_Success</i> | The writing handle is opened. |
|--------------------------------------|-------------------------------|

Example below shows how to use this API to write data. For task 1,

```
* static SERIAL_MANAGER_WRITE_HANDLE_DEFINE(s_serialWriteHandle1);
* static uint8_t s_nonBlockingWelcome1[] = "This is non-blocking writing log for task1!\r\n";
* SerialManager_OpenWriteHandle((serial_handle_t)serialHandle
* , (serial_write_handle_t)s_serialWriteHandle1);
* SerialManager_InstallTxCallback((
* serial_write_handle_t)s_serialWriteHandle1,
* Task1_SerialManagerTxCallback,
* s_serialWriteHandle1);
* SerialManager_WriteNonBlocking((
* serial_write_handle_t)s_serialWriteHandle1,
* s_nonBlockingWelcome1,
* sizeof(s_nonBlockingWelcome1) - 1U);
*
```

For task 2,

```
* static SERIAL_MANAGER_WRITE_HANDLE_DEFINE(s_serialWriteHandle2);
* static uint8_t s_nonBlockingWelcome2[] = "This is non-blocking writing log for task2!\r\n";
* SerialManager_OpenWriteHandle((serial_handle_t)serialHandle
* , (serial_write_handle_t)s_serialWriteHandle2);
* SerialManager_InstallTxCallback((
* serial_write_handle_t)s_serialWriteHandle2,
* Task2_SerialManagerTxCallback,
* s_serialWriteHandle2);
* SerialManager_WriteNonBlocking((
* serial_write_handle_t)s_serialWriteHandle2,
* s_nonBlockingWelcome2,
* sizeof(s_nonBlockingWelcome2) - 1U);
*
```

#### 44.5.4 serial\_manager\_status\_t SerialManager\_CloseWriteHandle ( serial\_write\_handle\_t writeHandle )

This function Closes a writing handle for the serial manager module.

Parameters

|                    |                                                   |
|--------------------|---------------------------------------------------|
| <i>writeHandle</i> | The serial manager module writing handle pointer. |
|--------------------|---------------------------------------------------|

Return values

---

|                                      |                               |
|--------------------------------------|-------------------------------|
| <i>kStatus_SerialManager_Success</i> | The writing handle is closed. |
|--------------------------------------|-------------------------------|

#### 44.5.5 serial\_manager\_status\_t SerialManager\_OpenReadHandle ( serial\_handle\_t serialHandle, serial\_read\_handle\_t readHandle )

This function Opens a reading handle for the serial manager module. The reading handle can not be opened multiple at the same time. The error code `kStatus_SerialManager_Busy` would be returned when the previous reading handle is not closed. And there can only be one buffer for receiving for the reading handle at the same time.

Parameters

|                     |                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>serialHandle</i> | The serial manager module handle pointer. The handle should be 4 byte aligned, because unaligned access doesn't be supported on some devices.                                                                                                                                                                                                                                                      |
| <i>readHandle</i>   | The serial manager module reading handle pointer. The handle should be 4 byte aligned, because unaligned access doesn't be supported on some devices. You can define the handle in the following two ways: <a href="#">SERIAL_MANAGER_READ_HANDLE_DEFINE(readHandle)</a> ; or <code>uint32_t readHandle[(((SERIAL_MANAGER_READ_HANDLE_SIZE + sizeof(uint32_t) - 1U) / sizeof(uint32_t)))]</code> ; |

Return values

|                                      |                                        |
|--------------------------------------|----------------------------------------|
| <i>kStatus_SerialManager_Error</i>   | An error occurred.                     |
| <i>kStatus_SerialManager_Success</i> | The reading handle is opened.          |
| <i>kStatus_SerialManager_Busy</i>    | Previous reading handle is not closed. |

Example below shows how to use this API to read data.

```
* static SERIAL_MANAGER_READ_HANDLE_DEFINE(s_serialReadHandle);
* SerialManager_OpenReadHandle((serial_handle_t)serialHandle,
* (serial_read_handle_t)s_serialReadHandle);
* static uint8_t s_nonBlockingBuffer[64];
* SerialManager_InstallRxCallback((
* serial_read_handle_t)s_serialReadHandle,
* APP_SerialManagerRxCallback,
* s_serialReadHandle);
* SerialManager_ReadNonBlocking((
* serial_read_handle_t)s_serialReadHandle,
* s_nonBlockingBuffer,
* sizeof(s_nonBlockingBuffer));
*
```

#### 44.5.6 serial\_manager\_status\_t SerialManager\_CloseReadHandle ( serial\_read\_handle\_t *readHandle* )

This function Closes a reading for the serial manager module.



## Parameters

|                   |                                                   |
|-------------------|---------------------------------------------------|
| <i>readHandle</i> | The serial manager module reading handle pointer. |
|-------------------|---------------------------------------------------|

## Return values

|                                      |                               |
|--------------------------------------|-------------------------------|
| <i>kStatus_SerialManager_Success</i> | The reading handle is closed. |
|--------------------------------------|-------------------------------|

#### 44.5.7 serial\_manager\_status\_t SerialManager\_WriteBlocking ( serial\_write\_handle\_t writeHandle, uint8\_t \* buffer, uint32\_t length )

This is a blocking function, which polls the sending queue, waits for the sending queue to be empty. This function sends data using an interrupt method. The interrupt of the hardware could not be disabled. And There can only one buffer for transmission for the writing handle at the same time.

## Note

The function [SerialManager\\_WriteBlocking](#) and the function `SerialManager_WriteNonBlocking` cannot be used at the same time. And, the function `SerialManager_CancelWriting` cannot be used to abort the transmission of this function.

## Parameters

|                    |                                           |
|--------------------|-------------------------------------------|
| <i>writeHandle</i> | The serial manager module handle pointer. |
| <i>buffer</i>      | Start address of the data to write.       |
| <i>length</i>      | Length of the data to write.              |

## Return values

|                                      |                                                                  |
|--------------------------------------|------------------------------------------------------------------|
| <i>kStatus_SerialManager_Success</i> | Successfully sent all data.                                      |
| <i>kStatus_SerialManager_Busy</i>    | Previous transmission still not finished; data not all sent yet. |

|                                    |                    |
|------------------------------------|--------------------|
| <i>kStatus_SerialManager_Error</i> | An error occurred. |
|------------------------------------|--------------------|

#### 44.5.8 serial\_manager\_status\_t SerialManager\_ReadBlocking ( serial\_read\_handle\_t readHandle, uint8\_t \* buffer, uint32\_t length )

This is a blocking function, which polls the receiving buffer, waits for the receiving buffer to be full. This function receives data using an interrupt method. The interrupt of the hardware could not be disabled. And There can only one buffer for receiving for the reading handle at the same time.

##### Note

The function [SerialManager\\_ReadBlocking](#) and the function `SerialManager_ReadNonBlocking` cannot be used at the same time. And, the function `SerialManager_CancelReading` cannot be used to abort the transmission of this function.

##### Parameters

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| <i>readHandle</i> | The serial manager module handle pointer.             |
| <i>buffer</i>     | Start address of the data to store the received data. |
| <i>length</i>     | The length of the data to be received.                |

##### Return values

|                                      |                                                                      |
|--------------------------------------|----------------------------------------------------------------------|
| <i>kStatus_SerialManager_Success</i> | Successfully received all data.                                      |
| <i>kStatus_SerialManager_Busy</i>    | Previous transmission still not finished; data not all received yet. |
| <i>kStatus_SerialManager_Error</i>   | An error occurred.                                                   |

#### 44.5.9 serial\_manager\_status\_t SerialManager\_WriteNonBlocking ( serial\_write\_handle\_t writeHandle, uint8\_t \* buffer, uint32\_t length )

This is a non-blocking function, which returns directly without waiting for all data to be sent. When all data is sent, the module notifies the upper layer through a TX callback function and passes the status parameter [kStatus\\_SerialManager\\_Success](#). This function sends data using an interrupt method. The interrupt of the hardware could not be disabled. And There can only one buffer for transmission for the writing handle at the same time.

## Note

The function [SerialManager\\_WriteBlocking](#) and the function [SerialManager\\_WriteNonBlocking](#) cannot be used at the same time. And, the TX callback is mandatory before the function could be used.

## Parameters

|                    |                                           |
|--------------------|-------------------------------------------|
| <i>writeHandle</i> | The serial manager module handle pointer. |
| <i>buffer</i>      | Start address of the data to write.       |
| <i>length</i>      | Length of the data to write.              |

## Return values

|                                      |                                                                  |
|--------------------------------------|------------------------------------------------------------------|
| <i>kStatus_SerialManager_Success</i> | Successfully sent all data.                                      |
| <i>kStatus_SerialManager_Busy</i>    | Previous transmission still not finished; data not all sent yet. |
| <i>kStatus_SerialManager_Error</i>   | An error occurred.                                               |

#### 44.5.10 serial\_manager\_status\_t SerialManager\_ReadNonBlocking ( serial\_read\_handle\_t readHandle, uint8\_t \* buffer, uint32\_t length )

This is a non-blocking function, which returns directly without waiting for all data to be received. When all data is received, the module driver notifies the upper layer through a RX callback function and passes the status parameter [kStatus\\_SerialManager\\_Success](#). This function receives data using an interrupt method. The interrupt of the hardware could not be disabled. And There can only one buffer for receiving for the reading handle at the same time.

## Note

The function [SerialManager\\_ReadBlocking](#) and the function [SerialManager\\_ReadNonBlocking](#) cannot be used at the same time. And, the RX callback is mandatory before the function could be used.

## Parameters

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| <i>readHandle</i> | The serial manager module handle pointer.             |
| <i>buffer</i>     | Start address of the data to store the received data. |
| <i>length</i>     | The length of the data to be received.                |

Return values

|                                            |                                                                      |
|--------------------------------------------|----------------------------------------------------------------------|
| <i>kStatus_SerialManager_-<br/>Success</i> | Successfully received all data.                                      |
| <i>kStatus_SerialManager_-<br/>Busy</i>    | Previous transmission still not finished; data not all received yet. |
| <i>kStatus_SerialManager_-<br/>Error</i>   | An error occurred.                                                   |

#### 44.5.11 serial\_manager\_status\_t SerialManager\_TryRead ( serial\_read\_handle\_t *readHandle*, uint8\_t \* *buffer*, uint32\_t *length*, uint32\_t \* *receivedLength* )

The function tries to read data from internal ring buffer. If the ring buffer is not empty, the data will be copied from ring buffer to up layer buffer. The copied length is the minimum of the ring buffer and up layer length. After the data is copied, the actual data length is passed by the parameter length. And There can only one buffer for receiving for the reading handle at the same time.

Parameters

|                       |                                                       |
|-----------------------|-------------------------------------------------------|
| <i>readHandle</i>     | The serial manager module handle pointer.             |
| <i>buffer</i>         | Start address of the data to store the received data. |
| <i>length</i>         | The length of the data to be received.                |
| <i>receivedLength</i> | Length received from the ring buffer directly.        |

Return values

|                                            |                                                                      |
|--------------------------------------------|----------------------------------------------------------------------|
| <i>kStatus_SerialManager_-<br/>Success</i> | Successfully received all data.                                      |
| <i>kStatus_SerialManager_-<br/>Busy</i>    | Previous transmission still not finished; data not all received yet. |

|                                    |                    |
|------------------------------------|--------------------|
| <i>kStatus_SerialManager_Error</i> | An error occurred. |
|------------------------------------|--------------------|

#### 44.5.12 serial\_manager\_status\_t SerialManager\_CancelWriting ( serial\_write\_handle\_t writeHandle )

The function cancels unfinished send transmission. When the transfer is canceled, the module notifies the upper layer through a TX callback function and passes the status parameter [kStatus\\_SerialManager\\_Canceled](#).

Note

The function [SerialManager\\_CancelWriting](#) cannot be used to abort the transmission of the function [SerialManager\\_WriteBlocking](#).

Parameters

|                    |                                           |
|--------------------|-------------------------------------------|
| <i>writeHandle</i> | The serial manager module handle pointer. |
|--------------------|-------------------------------------------|

Return values

|                                      |                                     |
|--------------------------------------|-------------------------------------|
| <i>kStatus_SerialManager_Success</i> | Get successfully abort the sending. |
| <i>kStatus_SerialManager_Error</i>   | An error occurred.                  |

#### 44.5.13 serial\_manager\_status\_t SerialManager\_CancelReading ( serial\_read\_handle\_t readHandle )

The function cancels unfinished receive transmission. When the transfer is canceled, the module notifies the upper layer through a RX callback function and passes the status parameter [kStatus\\_SerialManager\\_Canceled](#).

Note

The function [SerialManager\\_CancelReading](#) cannot be used to abort the transmission of the function [SerialManager\\_ReadBlocking](#).

## Parameters

|                   |                                           |
|-------------------|-------------------------------------------|
| <i>readHandle</i> | The serial manager module handle pointer. |
|-------------------|-------------------------------------------|

## Return values

|                                      |                                       |
|--------------------------------------|---------------------------------------|
| <i>kStatus_SerialManager_Success</i> | Get successfully abort the receiving. |
| <i>kStatus_SerialManager_Error</i>   | An error occurred.                    |

#### 44.5.14 **serial\_manager\_status\_t SerialManager\_InstallTxCallback ( serial\_write\_handle\_t writeHandle, serial\_manager\_callback\_t callback, void \* callbackParam )**

This function is used to install the TX callback and callback parameter for the serial manager module. When any status of TX transmission changed, the driver will notify the upper layer by the installed callback function. And the status is also passed as status parameter when the callback is called.

## Parameters

|                      |                                           |
|----------------------|-------------------------------------------|
| <i>writeHandle</i>   | The serial manager module handle pointer. |
| <i>callback</i>      | The callback function.                    |
| <i>callbackParam</i> | The parameter of the callback function.   |

## Return values

|                                      |                                    |
|--------------------------------------|------------------------------------|
| <i>kStatus_SerialManager_Success</i> | Successfully install the callback. |
|--------------------------------------|------------------------------------|

#### 44.5.15 **serial\_manager\_status\_t SerialManager\_InstallRxCallback ( serial\_read\_handle\_t readHandle, serial\_manager\_callback\_t callback, void \* callbackParam )**

This function is used to install the RX callback and callback parameter for the serial manager module. When any status of RX transmission changed, the driver will notify the upper layer by the installed callback function. And the status is also passed as status parameter when the callback is called.

## Parameters

|                      |                                           |
|----------------------|-------------------------------------------|
| <i>readHandle</i>    | The serial manager module handle pointer. |
| <i>callback</i>      | The callback function.                    |
| <i>callbackParam</i> | The parameter of the callback function.   |

## Return values

|                                      |                                    |
|--------------------------------------|------------------------------------|
| <i>kStatus_SerialManager_Success</i> | Successfully install the callback. |
|--------------------------------------|------------------------------------|

**44.5.16 static bool SerialManager\_needPollingIsr ( void ) [inline], [static]**

This function is used to check if need polling ISR.

## Return values

|             |                  |
|-------------|------------------|
| <i>TRUE</i> | if need polling. |
|-------------|------------------|

**44.5.17 serial\_manager\_status\_t SerialManager\_EnterLowpower ( serial\_handle\_t serialHandle )**

This function is used to prepare to enter low power consumption.

## Parameters

|                     |                                           |
|---------------------|-------------------------------------------|
| <i>serialHandle</i> | The serial manager module handle pointer. |
|---------------------|-------------------------------------------|

## Return values

|                                      |                       |
|--------------------------------------|-----------------------|
| <i>kStatus_SerialManager_Success</i> | Successful operation. |
|--------------------------------------|-----------------------|

**44.5.18 serial\_manager\_status\_t SerialManager\_ExitLowpower ( serial\_handle\_t serialHandle )**

This function is used to restore from low power consumption.

## Parameters

|                     |                                           |
|---------------------|-------------------------------------------|
| <i>serialHandle</i> | The serial manager module handle pointer. |
|---------------------|-------------------------------------------|

## Return values

|                                      |                       |
|--------------------------------------|-----------------------|
| <i>kStatus_SerialManager_Success</i> | Successful operation. |
|--------------------------------------|-----------------------|

#### 44.5.19 void SerialManager\_SetLowpowerCriticalCb ( const serial\_manager\_lowpower\_critical\_CBs\_t \* *pfCallback* )

## Parameters

|                   |                                                                   |
|-------------------|-------------------------------------------------------------------|
| <i>pfCallback</i> | Pointer to the function structure used to allow/disable lowpower. |
|-------------------|-------------------------------------------------------------------|



## Chapter 45

### Lpspi\_cmsis\_driver

This section describes the programming interface of the LPSPI Cortex Microcontroller Software Interface Standard (CMSIS) driver. And this driver defines generic peripheral driver interfaces for middleware making it reusable across a wide range of supported microcontroller devices. The API connects microcontroller peripherals with middleware that implements for example communication stacks, file systems, or graphic user interfaces. More information and usage method please refer to <http://www.keil.com/pack/doc/cmsis/Driver/html/index.html>.

#### 45.1 Function groups

##### 45.1.1 LPSPI CMSIS GetVersion Operation

This function group will return the DSPI CMSIS Driver version to user.

##### 45.1.2 LPSPI CMSIS GetCapabilities Operation

This function group will return the capabilities of this driver.

##### 45.1.3 LPSPI CMSIS Initialize and Uninitialize Operation

This function will initialize and uninitialize the instance in master mode or slave mode. And this API must be called before you configure an instance or after you Deinit an instance. The right steps to start an instance is that you must initialize the instance which been selected firstly, then you can power on the instance. After these all have been done, you can configure the instance by using control operation. If you want to Uninitialize the instance, you must power off the instance first.

##### 45.1.4 LPSPI Transfer Operation

This function group controls the transfer, master send/receive data, and slave send/receive data.

##### 45.1.5 LPSPI Status Operation

This function group gets the LPSPI transfer status.

## 45.1.6 LPSPI CMSIS Control Operation

This function can select instance as master mode or slave mode, set baudrate for master mode transfer, get current baudrate of master mode transfer, set transfer data bits and set other control command.

## 45.2 Typical use case

### 45.2.1 Master Operation

```
/* Variables */
uint8_t masterRxData[TRANSFER_SIZE] = {0U};
uint8_t masterTxData[TRANSFER_SIZE] = {0U};

/*DSPI master init*/
Driver_SPI0.Initialize(DSPI_MasterSignalEvent_t);
Driver_SPI0.PowerControl(ARM_POWER_FULL);
Driver_SPI0.Control(ARM_SPI_MODE_MASTER, TRANSFER_BAUDRATE);

/* Start master transfer */
Driver_SPI0.Transfer(masterTxData, masterRxData, TRANSFER_SIZE);

/* Master power off */
Driver_SPI0.PowerControl(ARM_POWER_OFF);

/* Master uninitialized */
Driver_SPI0.Uninitialize();
```

### 45.2.2 Slave Operation

```
/* Variables */
uint8_t slaveRxData[TRANSFER_SIZE] = {0U};
uint8_t slaveTxData[TRANSFER_SIZE] = {0U};

/*DSPI slave init*/
Driver_SPI2.Initialize(DSPI_SlaveSignalEvent_t);
Driver_SPI2.PowerControl(ARM_POWER_FULL);
Driver_SPI2.Control(ARM_SPI_MODE_SLAVE, false);

/* Start slave transfer */
Driver_SPI2.Transfer(slaveTxData, slaveRxData, TRANSFER_SIZE);

/* slave power off */
Driver_SPI2.PowerControl(ARM_POWER_OFF);

/* slave uninitialized */
Driver_SPI2.Uninitialize();
```

## Chapter 46

### Lpuart\_cmsis\_driver

This section describes the programming interface of the LPUART Cortex Microcontroller Software Interface Standard (CMSIS) driver. And this driver defines generic peripheral driver interfaces for middleware making it reusable across a wide range of supported microcontroller devices. The API connects microcontroller peripherals with middleware that implements for example communication stacks, file systems, or graphic user interfaces. More information and usage method please refer to <http://www.keil.com/pack/doc/cmsis/Driver/html/index.html>.

The LPUART driver includes transactional APIs.

Transactional APIs can be used to enable the peripheral quickly and in the application if the code size and performance of transactional APIs can satisfy the requirements. If the code size and performance are critical requirements please write custom code.

#### 46.1 Function groups

##### 46.1.1 LPUART CMSIS GetVersion Operation

This function group will return the LPUART CMSIS Driver version to user.

##### 46.1.2 LPUART CMSIS GetCapabilities Operation

This function group will return the capabilities of this driver.

##### 46.1.3 LPUART CMSIS Initialize and Uninitialize Operation

This function will initialize and uninitialize the lpuart instance . And this API must be called before you configure a lpuart instance or after you Deinit a lpuart instance. The right steps to start an instance is that you must initialize the instance which been selected firstly, then you can power on the instance. After these all have been done, you can configure the instance by using control operation. If you want to Uninitialize the instance, you must power off the instance first.

##### 46.1.4 LPUART CMSIS Transfer Operation

This function group controls the transfer, send/receive data.

### **46.1.5 LPUART CMSIS Status Operation**

This function group gets the LPUART transfer status.

### **46.1.6 LPUART CMSIS Control Operation**

This function can configure an instance ,set baudrate for lpuart, get current baudrate ,set transfer data bits and other control command.

## Chapter 47

# Flexio\_edma\_i2s

### 47.1 Overview

#### Data Structures

- struct `_flexio_i2s_edma_handle`  
*FlexIO I2S DMA transfer handle, users should not touch the content of the handle. [More...](#)*

#### Typedefs

- typedef void(\* `flexio_i2s_edma_callback_t`)(`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle, `status_t` status, void \*userData)  
*FlexIO I2S eDMA transfer callback function for finish and error.*

#### Driver version

- #define `FSL_FLEXIO_I2S_EDMA_DRIVER_VERSION` (`MAKE_VERSION`(2, 1, 8))  
*FlexIO I2S EDMA driver version 2.1.8.*

#### eDMA Transactional

- void `FLEXIO_I2S_TransferTxCreateHandleEDMA` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle, `flexio_i2s_edma_callback_t` callback, void \*userData, `edma_handle_t` \*dmaHandle)  
*Initializes the FlexIO I2S eDMA handle.*
- void `FLEXIO_I2S_TransferRxCreateHandleEDMA` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle, `flexio_i2s_edma_callback_t` callback, void \*userData, `edma_handle_t` \*dmaHandle)  
*Initializes the FlexIO I2S Rx eDMA handle.*
- void `FLEXIO_I2S_TransferSetFormatEDMA` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle, `flexio_i2s_format_t` \*format, `uint32_t` srcClock\_Hz)  
*Configures the FlexIO I2S Tx audio format.*
- `status_t` `FLEXIO_I2S_TransferSendEDMA` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle, `flexio_i2s_transfer_t` \*xfer)  
*Performs a non-blocking FlexIO I2S transfer using DMA.*
- `status_t` `FLEXIO_I2S_TransferReceiveEDMA` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle, `flexio_i2s_transfer_t` \*xfer)  
*Performs a non-blocking FlexIO I2S receive using eDMA.*
- void `FLEXIO_I2S_TransferAbortSendEDMA` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle)  
*Aborts a FlexIO I2S transfer using eDMA.*
- void `FLEXIO_I2S_TransferAbortReceiveEDMA` (`FLEXIO_I2S_Type` \*base, `flexio_i2s_edma_handle_t` \*handle)  
*Aborts a FlexIO I2S receive using eDMA.*

- `status_t FLEXIO_I2S_TransferGetSendCountEDMA (FLEXIO_I2S_Type *base, flexio_i2s_edma_handle_t *handle, size_t *count)`  
*Gets the remaining bytes to be sent.*
- `status_t FLEXIO_I2S_TransferGetReceiveCountEDMA (FLEXIO_I2S_Type *base, flexio_i2s_edma_handle_t *handle, size_t *count)`  
*Get the remaining bytes to be received.*

## 47.2 Data Structure Documentation

### 47.2.1 struct flexio\_i2s\_edma\_handle

#### Data Fields

- `edma_handle_t * dmaHandle`  
*DMA handler for FlexIO I2S send.*
- `uint8_t bytesPerFrame`  
*Bytes in a frame.*
- `uint8_t nbytes`  
*eDMA minor byte transfer count initially configured.*
- `uint32_t state`  
*Internal state for FlexIO I2S eDMA transfer.*
- `flexio_i2s_edma_callback_t callback`  
*Callback for users while transfer finish or error occurred.*
- `void * userData`  
*User callback parameter.*
- `edma_tcd_t tcd [FLEXIO_I2S_XFER_QUEUE_SIZE+1U]`  
*TCD pool for eDMA transfer.*
- `flexio_i2s_transfer_t queue [FLEXIO_I2S_XFER_QUEUE_SIZE]`  
*Transfer queue storing queued transfer.*
- `size_t transferSize [FLEXIO_I2S_XFER_QUEUE_SIZE]`  
*Data bytes need to transfer.*
- `volatile uint8_t queueUser`  
*Index for user to queue transfer.*
- `volatile uint8_t queueDriver`  
*Index for driver to get the transfer data and size.*

#### Field Documentation

- (1) `uint8_t flexio_i2s_edma_handle::nbytes`
- (2) `edma_tcd_t flexio_i2s_edma_handle::tcd[FLEXIO_I2S_XFER_QUEUE_SIZE+1U]`
- (3) `flexio_i2s_transfer_t flexio_i2s_edma_handle::queue[FLEXIO_I2S_XFER_QUEUE_SIZE]`
- (4) `volatile uint8_t flexio_i2s_edma_handle::queueUser`

## 47.3 Macro Definition Documentation

**47.3.1 #define FSL\_FLEXIO\_I2S\_EDMA\_DRIVER\_VERSION (MAKE\_VERSION(2, 1, 8))**

## 47.4 Function Documentation

**47.4.1 void FLEXIO\_I2S\_TransferTxCreateHandleEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle*, flexio\_i2s\_edma\_callback\_t *callback*, void \* *userData*, edma\_handle\_t \* *dmaHandle* )**

This function initializes the FlexIO I2S master DMA handle which can be used for other FlexIO I2S master transactional APIs. Usually, for a specified FlexIO I2S instance, call this API once to get the initialized handle.

Parameters

|                  |                                                                               |
|------------------|-------------------------------------------------------------------------------|
| <i>base</i>      | FlexIO I2S peripheral base address.                                           |
| <i>handle</i>    | FlexIO I2S eDMA handle pointer.                                               |
| <i>callback</i>  | FlexIO I2S eDMA callback function called while finished a block.              |
| <i>userData</i>  | User parameter for callback.                                                  |
| <i>dmaHandle</i> | eDMA handle for FlexIO I2S. This handle is a static value allocated by users. |

**47.4.2 void FLEXIO\_I2S\_TransferRxCreateHandleEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle*, flexio\_i2s\_edma\_callback\_t *callback*, void \* *userData*, edma\_handle\_t \* *dmaHandle* )**

This function initializes the FlexIO I2S slave DMA handle which can be used for other FlexIO I2S master transactional APIs. Usually, for a specified FlexIO I2S instance, call this API once to get the initialized handle.

Parameters

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <i>base</i>     | FlexIO I2S peripheral base address.                              |
| <i>handle</i>   | FlexIO I2S eDMA handle pointer.                                  |
| <i>callback</i> | FlexIO I2S eDMA callback function called while finished a block. |

|                  |                                                                               |
|------------------|-------------------------------------------------------------------------------|
| <i>userData</i>  | User parameter for callback.                                                  |
| <i>dmaHandle</i> | eDMA handle for FlexIO I2S. This handle is a static value allocated by users. |

#### 47.4.3 void FLEXIO\_I2S\_TransferSetFormatEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle*, flexio\_i2s\_format\_t \* *format*, uint32\_t *srcClock\_Hz* )

Audio format can be changed in run-time of FlexIO I2S. This function configures the sample rate and audio data format to be transferred. This function also sets the eDMA parameter according to format.

Parameters

|                    |                                                                              |
|--------------------|------------------------------------------------------------------------------|
| <i>base</i>        | FlexIO I2S peripheral base address.                                          |
| <i>handle</i>      | FlexIO I2S eDMA handle pointer                                               |
| <i>format</i>      | Pointer to FlexIO I2S audio data format structure.                           |
| <i>srcClock_Hz</i> | FlexIO I2S clock source frequency in Hz, it should be 0 while in slave mode. |

#### 47.4.4 status\_t FLEXIO\_I2S\_TransferSendEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle*, flexio\_i2s\_transfer\_t \* *xfer* )

Note

This interface returned immediately after transfer initiates. Users should call FLEXIO\_I2S\_GetTransferStatus to poll the transfer status and check whether the FlexIO I2S transfer is finished.

Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | FlexIO I2S peripheral base address. |
| <i>handle</i> | FlexIO I2S DMA handle pointer.      |
| <i>xfer</i>   | Pointer to DMA transfer structure.  |

Return values

|                        |                                            |
|------------------------|--------------------------------------------|
| <i>kStatus_Success</i> | Start a FlexIO I2S eDMA send successfully. |
|------------------------|--------------------------------------------|



|                                |                                  |
|--------------------------------|----------------------------------|
| <i>kStatus_InvalidArgument</i> | The input arguments is invalid.  |
| <i>kStatus_TxBusy</i>          | FlexIO I2S is busy sending data. |

#### 47.4.5 **status\_t FLEXIO\_I2S\_TransferReceiveEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle*, flexio\_i2s\_transfer\_t \* *xfer* )**

##### Note

This interface returned immediately after transfer initiates. Users should call FLEXIO\_I2S\_GetReceiveRemainingBytes to poll the transfer status and check whether the FlexIO I2S transfer is finished.

##### Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | FlexIO I2S peripheral base address. |
| <i>handle</i> | FlexIO I2S DMA handle pointer.      |
| <i>xfer</i>   | Pointer to DMA transfer structure.  |

##### Return values

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| <i>kStatus_Success</i>         | Start a FlexIO I2S eDMA receive successfully. |
| <i>kStatus_InvalidArgument</i> | The input arguments is invalid.               |
| <i>kStatus_RxBusy</i>          | FlexIO I2S is busy receiving data.            |

#### 47.4.6 **void FLEXIO\_I2S\_TransferAbortSendEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle* )**

##### Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | FlexIO I2S peripheral base address. |
| <i>handle</i> | FlexIO I2S DMA handle pointer.      |

#### 47.4.7 **void FLEXIO\_I2S\_TransferAbortReceiveEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle* )**

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | FlexIO I2S peripheral base address. |
| <i>handle</i> | FlexIO I2S DMA handle pointer.      |

#### 47.4.8 **status\_t FLEXIO\_I2S\_TransferGetSendCountEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle*, size\_t \* *count* )**

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | FlexIO I2S peripheral base address. |
| <i>handle</i> | FlexIO I2S DMA handle pointer.      |
| <i>count</i>  | Bytes sent.                         |

## Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

#### 47.4.9 **status\_t FLEXIO\_I2S\_TransferGetReceiveCountEDMA ( FLEXIO\_I2S\_Type \* *base*, flexio\_i2s\_edma\_handle\_t \* *handle*, size\_t \* *count* )**

## Parameters

|               |                                     |
|---------------|-------------------------------------|
| <i>base</i>   | FlexIO I2S peripheral base address. |
| <i>handle</i> | FlexIO I2S DMA handle pointer.      |
| <i>count</i>  | Bytes received.                     |

## Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

# Chapter 48

## Flexio\_edma\_spi

### 48.1 Overview

#### Data Structures

- struct `_flexio_spi_master_edma_handle`  
*FlexIO SPI eDMA transfer handle, users should not touch the content of the handle. [More...](#)*

#### Typedefs

- typedef struct  
    `_flexio_spi_master_edma_handle` `flexio_spi_master_edma_handle_t`  
    *typedef for flexio\_spi\_master\_edma\_handle\_t in advance.*
- typedef  
    `flexio_spi_master_edma_handle_t` `flexio_spi_slave_edma_handle_t`  
    *Slave handle is the same with master handle.*
- typedef void(\* `flexio_spi_master_edma_transfer_callback_t` )(FLEXIO\_SPI\_Type \*base, `flexio_spi_master_edma_handle_t` \*handle, `status_t` status, void \*userData)  
    *FlexIO SPI master callback for finished transmit.*
- typedef void(\* `flexio_spi_slave_edma_transfer_callback_t` )(FLEXIO\_SPI\_Type \*base, `flexio_spi_slave_edma_handle_t` \*handle, `status_t` status, void \*userData)  
    *FlexIO SPI slave callback for finished transmit.*

#### Driver version

- #define `FSL_FLEXIO_SPI_EDMA_DRIVER_VERSION` (`MAKE_VERSION`(2, 3, 0))  
    *FlexIO SPI EDMA driver version.*

#### eDMA Transactional

- `status_t` `FLEXIO_SPI_MasterTransferCreateHandleEDMA` (FLEXIO\_SPI\_Type \*base, `flexio_spi_master_edma_handle_t` \*handle, `flexio_spi_master_edma_transfer_callback_t` callback, void \*userData, `edma_handle_t` \*txHandle, `edma_handle_t` \*rxHandle)  
    *Initializes the FlexIO SPI master eDMA handle.*
- `status_t` `FLEXIO_SPI_MasterTransferEDMA` (FLEXIO\_SPI\_Type \*base, `flexio_spi_master_edma_handle_t` \*handle, `flexio_spi_transfer_t` \*xfer)  
    *Performs a non-blocking FlexIO SPI transfer using eDMA.*
- void `FLEXIO_SPI_MasterTransferAbortEDMA` (FLEXIO\_SPI\_Type \*base, `flexio_spi_master_edma_handle_t` \*handle)  
    *Aborts a FlexIO SPI transfer using eDMA.*
- `status_t` `FLEXIO_SPI_MasterTransferGetCountEDMA` (FLEXIO\_SPI\_Type \*base, `flexio_spi_master_edma_handle_t` \*handle, `size_t` \*count)  
    *Gets the number of bytes transferred so far using FlexIO SPI master eDMA.*

- static void `FLEXIO_SPI_SlaveTransferCreateHandleEDMA` (`FLEXIO_SPI_Type` \*base, `flexio_spi_slave_edma_handle_t` \*handle, `flexio_spi_slave_edma_transfer_callback_t` callback, void \*userData, `edma_handle_t` \*txHandle, `edma_handle_t` \*rxHandle)  
*Initializes the FlexIO SPI slave eDMA handle.*
- `status_t FLEXIO_SPI_SlaveTransferEDMA` (`FLEXIO_SPI_Type` \*base, `flexio_spi_slave_edma_handle_t` \*handle, `flexio_spi_transfer_t` \*xfer)  
*Performs a non-blocking FlexIO SPI transfer using eDMA.*
- static void `FLEXIO_SPI_SlaveTransferAbortEDMA` (`FLEXIO_SPI_Type` \*base, `flexio_spi_slave_edma_handle_t` \*handle)  
*Aborts a FlexIO SPI transfer using eDMA.*
- static `status_t FLEXIO_SPI_SlaveTransferGetCountEDMA` (`FLEXIO_SPI_Type` \*base, `flexio_spi_slave_edma_handle_t` \*handle, `size_t` \*count)  
*Gets the number of bytes transferred so far using FlexIO SPI slave eDMA.*

## 48.2 Data Structure Documentation

### 48.2.1 struct `_flexio_spi_master_edma_handle`

#### Data Fields

- `size_t transferSize`  
*Total bytes to be transferred.*
- `uint8_t nbytes`  
*eDMA minor byte transfer count initially configured.*
- `bool txInProgress`  
*Send transfer in progress.*
- `bool rxInProgress`  
*Receive transfer in progress.*
- `edma_handle_t * txHandle`  
*DMA handler for SPI send.*
- `edma_handle_t * rxHandle`  
*DMA handler for SPI receive.*
- `flexio_spi_master_edma_transfer_callback_t` callback  
*Callback for SPI DMA transfer.*
- void \* `userData`  
*User Data for SPI DMA callback.*

#### Field Documentation

(1) `size_t _flexio_spi_master_edma_handle::transferSize`

(2) `uint8_t _flexio_spi_master_edma_handle::nbytes`

## 48.3 Macro Definition Documentation

48.3.1 `#define FSL_FLEXIO_SPI_EDMA_DRIVER_VERSION (MAKE_VERSION(2, 3, 0))`

## 48.4 Typedef Documentation

**48.4.1** `typedef struct _flexio_spi_master_edma_handle flexio_spi_master_edma_handle_t`

**48.4.2** `typedef flexio_spi_master_edma_handle_t flexio_spi_slave_edma_handle_t`

## 48.5 Function Documentation

**48.5.1** `status_t FLEXIO_SPI_MasterTransferCreateHandleEDMA ( FLEXIO_SPI_Type * base, flexio_spi_master_edma_handle_t * handle, flexio_spi_master_edma_transfer_callback_t callback, void * userData, edma_handle_t * txHandle, edma_handle_t * rxHandle )`

This function initializes the FlexIO SPI master eDMA handle which can be used for other FlexIO SPI master transactional APIs. For a specified FlexIO SPI instance, call this API once to get the initialized handle.

Parameters

|                 |                                                                                   |
|-----------------|-----------------------------------------------------------------------------------|
| <i>base</i>     | Pointer to FLEXIO_SPI_Type structure.                                             |
| <i>handle</i>   | Pointer to flexio_spi_master_edma_handle_t structure to store the transfer state. |
| <i>callback</i> | SPI callback, NULL means no callback.                                             |
| <i>userData</i> | callback function parameter.                                                      |
| <i>txHandle</i> | User requested eDMA handle for FlexIO SPI RX eDMA transfer.                       |
| <i>rxHandle</i> | User requested eDMA handle for FlexIO SPI TX eDMA transfer.                       |

Return values

|                           |                                                     |
|---------------------------|-----------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                     |
| <i>kStatus_OutOfRange</i> | The FlexIO SPI eDMA type/handle table out of range. |

**48.5.2** `status_t FLEXIO_SPI_MasterTransferEDMA ( FLEXIO_SPI_Type * base, flexio_spi_master_edma_handle_t * handle, flexio_spi_transfer_t * xfer )`

Note

This interface returns immediately after transfer initiates. Call FLEXIO\_SPI\_MasterGetTransferCountEDMA to poll the transfer status and check whether the FlexIO SPI transfer is finished.

## Parameters

|               |                                                                                   |
|---------------|-----------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_SPI_Type structure.                                             |
| <i>handle</i> | Pointer to flexio_spi_master_edma_handle_t structure to store the transfer state. |
| <i>xfer</i>   | Pointer to FlexIO SPI transfer structure.                                         |

## Return values

|                                |                                                      |
|--------------------------------|------------------------------------------------------|
| <i>kStatus_Success</i>         | Successfully start a transfer.                       |
| <i>kStatus_InvalidArgument</i> | Input argument is invalid.                           |
| <i>kStatus_FLEXIO_SPI_Busy</i> | FlexIO SPI is not idle, is running another transfer. |

### 48.5.3 void FLEXIO\_SPI\_MasterTransferAbortEDMA ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_master\_edma\_handle\_t \* *handle* )

## Parameters

|               |                                       |
|---------------|---------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_SPI_Type structure. |
| <i>handle</i> | FlexIO SPI eDMA handle pointer.       |

### 48.5.4 status\_t FLEXIO\_SPI\_MasterTransferGetCountEDMA ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_master\_edma\_handle\_t \* *handle*, size\_t \* *count* )

## Parameters

|               |                                                                     |
|---------------|---------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_SPI_Type structure.                               |
| <i>handle</i> | FlexIO SPI eDMA handle pointer.                                     |
| <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction. |

**48.5.5 static void FLEXIO\_SPI\_SlaveTransferCreateHandleEDMA (**  
**FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_slave\_edma\_handle\_t \* *handle*,**  
**flexio\_spi\_slave\_edma\_transfer\_callback\_t *callback*, void \* *userData*,**  
**edma\_handle\_t \* *txHandle*, edma\_handle\_t \* *rxHandle* ) [inline],**  
**[static]**

This function initializes the FlexIO SPI slave eDMA handle.

## Parameters

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>base</i>     | Pointer to FLEXIO_SPI_Type structure.                                            |
| <i>handle</i>   | Pointer to flexio_spi_slave_edma_handle_t structure to store the transfer state. |
| <i>callback</i> | SPI callback, NULL means no callback.                                            |
| <i>userData</i> | callback function parameter.                                                     |
| <i>txHandle</i> | User requested eDMA handle for FlexIO SPI TX eDMA transfer.                      |
| <i>rxHandle</i> | User requested eDMA handle for FlexIO SPI RX eDMA transfer.                      |

#### 48.5.6 status\_t FLEXIO\_SPI\_SlaveTransferEDMA ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_slave\_edma\_handle\_t \* *handle*, flexio\_spi\_transfer\_t \* *xfer* )

## Note

This interface returns immediately after transfer initiates. Call FLEXIO\_SPI\_SlaveGetTransferCountEDMA to poll the transfer status and check whether the FlexIO SPI transfer is finished.

## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_SPI_Type structure.                                            |
| <i>handle</i> | Pointer to flexio_spi_slave_edma_handle_t structure to store the transfer state. |
| <i>xfer</i>   | Pointer to FlexIO SPI transfer structure.                                        |

## Return values

|                                |                                                      |
|--------------------------------|------------------------------------------------------|
| <i>kStatus_Success</i>         | Successfully start a transfer.                       |
| <i>kStatus_InvalidArgument</i> | Input argument is invalid.                           |
| <i>kStatus_FLEXIO_SPI_Busy</i> | FlexIO SPI is not idle, is running another transfer. |

#### 48.5.7 static void FLEXIO\_SPI\_SlaveTransferAbortEDMA ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_slave\_edma\_handle\_t \* *handle* ) [inline], [static]



## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_SPI_Type structure.                                            |
| <i>handle</i> | Pointer to flexio_spi_slave_edma_handle_t structure to store the transfer state. |

**48.5.8 static status\_t FLEXIO\_SPI\_SlaveTransferGetCountEDMA ( FLEXIO\_SPI\_Type \* *base*, flexio\_spi\_slave\_edma\_handle\_t \* *handle*, size\_t \* *count* ) [inline], [static]**

## Parameters

|               |                                                                     |
|---------------|---------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_SPI_Type structure.                               |
| <i>handle</i> | FlexIO SPI eDMA handle pointer.                                     |
| <i>count</i>  | Number of bytes transferred so far by the non-blocking transaction. |

## Chapter 49

# Flexio\_edma\_uart

### 49.1 Overview

#### Data Structures

- struct `_flexio_uart_edma_handle`  
*UART eDMA handle. [More...](#)*

#### Typedefs

- typedef void(\* `flexio_uart_edma_transfer_callback_t`)(`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle, `status_t` status, void \*userData)  
*UART transfer callback function.*

#### Driver version

- #define `FSL_FLEXIO_UART_EDMA_DRIVER_VERSION` (`MAKE_VERSION(2, 4, 1)`)  
*FlexIO UART EDMA driver version.*

#### eDMA transactional

- `status_t` `FLEXIO_UART_TransferCreateHandleEDMA` (`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle, `flexio_uart_edma_transfer_callback_t` callback, void \*userData, `edma_handle_t` \*txEdmaHandle, `edma_handle_t` \*rxEdmaHandle)  
*Initializes the UART handle which is used in transactional functions.*
- `status_t` `FLEXIO_UART_TransferSendEDMA` (`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle, `flexio_uart_transfer_t` \*xfer)  
*Sends data using eDMA.*
- `status_t` `FLEXIO_UART_TransferReceiveEDMA` (`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle, `flexio_uart_transfer_t` \*xfer)  
*Receives data using eDMA.*
- void `FLEXIO_UART_TransferAbortSendEDMA` (`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle)  
*Aborts the sent data which using eDMA.*
- void `FLEXIO_UART_TransferAbortReceiveEDMA` (`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle)  
*Aborts the receive data which using eDMA.*
- `status_t` `FLEXIO_UART_TransferGetSendCountEDMA` (`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle, `size_t` \*count)  
*Gets the number of bytes sent out.*
- `status_t` `FLEXIO_UART_TransferGetReceiveCountEDMA` (`FLEXIO_UART_Type` \*base, `flexio_uart_edma_handle_t` \*handle, `size_t` \*count)  
*Gets the number of bytes received.*

## 49.2 Data Structure Documentation

### 49.2.1 struct \_flexio\_uart\_edma\_handle

#### Data Fields

- [flexio\\_uart\\_edma\\_transfer\\_callback\\_t](#) *callback*  
*Callback function.*
- void \* [userData](#)  
*UART callback function parameter.*
- size\_t [txDataSizeAll](#)  
*Total bytes to be sent.*
- size\_t [rxDataSizeAll](#)  
*Total bytes to be received.*
- [edma\\_handle\\_t](#) \* [txEdmaHandle](#)  
*The eDMA TX channel used.*
- [edma\\_handle\\_t](#) \* [rxEdmaHandle](#)  
*The eDMA RX channel used.*
- uint8\_t [nbytes](#)  
*eDMA minor byte transfer count initially configured.*
- volatile uint8\_t [txState](#)  
*TX transfer state.*
- volatile uint8\_t [rxState](#)  
*RX transfer state.*

#### Field Documentation

- (1) [flexio\\_uart\\_edma\\_transfer\\_callback\\_t \\_flexio\\_uart\\_edma\\_handle::callback](#)
- (2) [void\\* \\_flexio\\_uart\\_edma\\_handle::userData](#)
- (3) [size\\_t \\_flexio\\_uart\\_edma\\_handle::txDataSizeAll](#)
- (4) [size\\_t \\_flexio\\_uart\\_edma\\_handle::rxDataSizeAll](#)
- (5) [edma\\_handle\\_t\\* \\_flexio\\_uart\\_edma\\_handle::txEdmaHandle](#)
- (6) [edma\\_handle\\_t\\* \\_flexio\\_uart\\_edma\\_handle::rxEdmaHandle](#)
- (7) [uint8\\_t \\_flexio\\_uart\\_edma\\_handle::nbytes](#)
- (8) [volatile uint8\\_t \\_flexio\\_uart\\_edma\\_handle::txState](#)

## 49.3 Macro Definition Documentation

### 49.3.1 #define FSL\_FLEXIO\_UART\_EDMA\_DRIVER\_VERSION (MAKE\_VERSION(2, 4, 1))

## 49.4 Typedef Documentation

**49.4.1** `typedef void(* flexio_uart_edma_transfer_callback_t)(FLEXIO_UART_Type *base, flexio_uart_edma_handle_t *handle, status_t status, void *userData)`

## 49.5 Function Documentation

**49.5.1** `status_t FLEXIO_UART_TransferCreateHandleEDMA ( FLEXIO_UART_Type * base, flexio_uart_edma_handle_t * handle, flexio_uart_edma_transfer_callback_t callback, void * userData, edma_handle_t * txEdmaHandle, edma_handle_t * rxEdmaHandle )`

Parameters

|                     |                                                 |
|---------------------|-------------------------------------------------|
| <i>base</i>         | Pointer to FLEXIO_UART_Type.                    |
| <i>handle</i>       | Pointer to flexio_uart_edma_handle_t structure. |
| <i>callback</i>     | The callback function.                          |
| <i>userData</i>     | The parameter of the callback function.         |
| <i>rxEdmaHandle</i> | User requested DMA handle for RX DMA transfer.  |
| <i>txEdmaHandle</i> | User requested DMA handle for TX DMA transfer.  |

Return values

|                           |                                                     |
|---------------------------|-----------------------------------------------------|
| <i>kStatus_Success</i>    | Successfully create the handle.                     |
| <i>kStatus_OutOfRange</i> | The FlexIO SPI eDMA type/handle table out of range. |

**49.5.2** `status_t FLEXIO_UART_TransferSendEDMA ( FLEXIO_UART_Type * base, flexio_uart_edma_handle_t * handle, flexio_uart_transfer_t * xfer )`

This function sends data using eDMA. This is a non-blocking function, which returns right away. When all data is sent out, the send callback function is called.

Parameters

|               |                             |
|---------------|-----------------------------|
| <i>base</i>   | Pointer to FLEXIO_UART_Type |
| <i>handle</i> | UART handle pointer.        |

|             |                                                                            |
|-------------|----------------------------------------------------------------------------|
| <i>xfer</i> | UART eDMA transfer structure, see <a href="#">flexio_uart_transfer_t</a> . |
|-------------|----------------------------------------------------------------------------|

Return values

|                                   |                             |
|-----------------------------------|-----------------------------|
| <i>kStatus_Success</i>            | if succeed, others failed.  |
| <i>kStatus_FLEXIO_UART-TxBusy</i> | Previous transfer on going. |

#### 49.5.3 **status\_t FLEXIO\_UART\_TransferReceiveEDMA ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_edma\_handle\_t \* *handle*, flexio\_uart\_transfer\_t \* *xfer* )**

This function receives data using eDMA. This is a non-blocking function, which returns right away. When all data is received, the receive callback function is called.

Parameters

|               |                                                                            |
|---------------|----------------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_UART_Type                                                |
| <i>handle</i> | Pointer to flexio_uart_edma_handle_t structure                             |
| <i>xfer</i>   | UART eDMA transfer structure, see <a href="#">flexio_uart_transfer_t</a> . |

Return values

|                            |                             |
|----------------------------|-----------------------------|
| <i>kStatus_Success</i>     | if succeed, others failed.  |
| <i>kStatus_UART_RxBusy</i> | Previous transfer on going. |

#### 49.5.4 **void FLEXIO\_UART\_TransferAbortSendEDMA ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_edma\_handle\_t \* *handle* )**

This function aborts sent data which using eDMA.

Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_UART_Type                    |
| <i>handle</i> | Pointer to flexio_uart_edma_handle_t structure |

#### 49.5.5 void FLEXIO\_UART\_TransferAbortReceiveEDMA ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_edma\_handle\_t \* *handle* )

This function aborts the receive data which using eDMA.

## Parameters

|               |                                                |
|---------------|------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_UART_Type                    |
| <i>handle</i> | Pointer to flexio_uart_edma_handle_t structure |

#### 49.5.6 status\_t FLEXIO\_UART\_TransferGetSendCountEDMA ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_edma\_handle\_t \* *handle*, size\_t \* *count* )

This function gets the number of bytes sent out.

## Parameters

|               |                                                              |
|---------------|--------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_UART_Type                                  |
| <i>handle</i> | Pointer to flexio_uart_edma_handle_t structure               |
| <i>count</i>  | Number of bytes sent so far by the non-blocking transaction. |

## Return values

|                                     |                                                   |
|-------------------------------------|---------------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | transfer has finished or no transfer in progress. |
| <i>kStatus_Success</i>              | Successfully return the count.                    |

#### 49.5.7 status\_t FLEXIO\_UART\_TransferGetReceiveCountEDMA ( FLEXIO\_UART\_Type \* *base*, flexio\_uart\_edma\_handle\_t \* *handle*, size\_t \* *count* )

This function gets the number of bytes received.

## Parameters

|               |                                                                  |
|---------------|------------------------------------------------------------------|
| <i>base</i>   | Pointer to FLEXIO_UART_Type                                      |
| <i>handle</i> | Pointer to flexio_uart_edma_handle_t structure                   |
| <i>count</i>  | Number of bytes received so far by the non-blocking transaction. |

## Return values

|                                     |                                                   |
|-------------------------------------|---------------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | transfer has finished or no transfer in progress. |
| <i>kStatus_Success</i>              | Successfully return the count.                    |



# Chapter 50

## Lpspi\_edma\_driver

### 50.1 Overview

#### Data Structures

- struct [\\_lpspi\\_master\\_edma\\_handle](#)  
*LPSPi master eDMA transfer handle structure used for transactional API. [More...](#)*
- struct [\\_lpspi\\_slave\\_edma\\_handle](#)  
*LPSPi slave eDMA transfer handle structure used for transactional API. [More...](#)*

#### Typedefs

- typedef struct  
[\\_lpspi\\_master\\_edma\\_handle](#) [lpspi\\_master\\_edma\\_handle\\_t](#)  
*Forward declaration of the [\\_lpspi\\_master\\_edma\\_handle](#) typedefs.*
- typedef struct  
[\\_lpspi\\_slave\\_edma\\_handle](#) [lpspi\\_slave\\_edma\\_handle\\_t](#)  
*Forward declaration of the [\\_lpspi\\_slave\\_edma\\_handle](#) typedefs.*
- typedef void(\* [lpspi\\_master\\_edma\\_transfer\\_callback\\_t](#) )(LPSPi\_Type \*base, [lpspi\\_master\\_edma\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
*Completion callback function pointer type.*
- typedef void(\* [lpspi\\_slave\\_edma\\_transfer\\_callback\\_t](#) )(LPSPi\_Type \*base, [lpspi\\_slave\\_edma\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
*Completion callback function pointer type.*

#### Functions

- void [LPSPi\\_MasterTransferCreateHandleEDMA](#) (LPSPi\_Type \*base, [lpspi\\_master\\_edma\\_handle\\_t](#) \*handle, [lpspi\\_master\\_edma\\_transfer\\_callback\\_t](#) callback, void \*userData, [edma\\_handle\\_t](#) \*edmaRxRegToRxDataHandle, [edma\\_handle\\_t](#) \*edmaTxDataToTxRegHandle)  
*Initializes the LPSPi master eDMA handle.*
- [status\\_t](#) [LPSPi\\_MasterTransferEDMA](#) (LPSPi\_Type \*base, [lpspi\\_master\\_edma\\_handle\\_t](#) \*handle, [lpspi\\_transfer\\_t](#) \*transfer)  
*LPSPi master transfer data using eDMA.*
- [status\\_t](#) [LPSPi\\_MasterTransferPrepareEDMALite](#) (LPSPi\_Type \*base, [lpspi\\_master\\_edma\\_handle\\_t](#) \*handle, uint32\_t configFlags)  
*LPSPi master config transfer parameter while using eDMA.*
- [status\\_t](#) [LPSPi\\_MasterTransferEDMALite](#) (LPSPi\_Type \*base, [lpspi\\_master\\_edma\\_handle\\_t](#) \*handle, [lpspi\\_transfer\\_t](#) \*transfer)  
*LPSPi master transfer data using eDMA without configs.*
- void [LPSPi\\_MasterTransferAbortEDMA](#) (LPSPi\_Type \*base, [lpspi\\_master\\_edma\\_handle\\_t](#) \*handle)  
*LPSPi master aborts a transfer which is using eDMA.*

- [status\\_t LPSPI\\_MasterTransferGetCountEDMA](#) (LPSPI\_Type \*base, [lpspi\\_master\\_edma\\_handle\\_t](#) \*handle, size\_t \*count)  
*Gets the master eDMA transfer remaining bytes.*
- void [LPSPI\\_SlaveTransferCreateHandleEDMA](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_edma\\_handle\\_t](#) \*handle, [lpspi\\_slave\\_edma\\_transfer\\_callback\\_t](#) callback, void \*userData, [edma\\_handle\\_t](#) \*edma-RxRegToRxDataHandle, [edma\\_handle\\_t](#) \*edmaTxDataToTxRegHandle)  
*Initializes the LPSPI slave eDMA handle.*
- [status\\_t LPSPI\\_SlaveTransferEDMA](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_edma\\_handle\\_t](#) \*handle, [lpspi\\_transfer\\_t](#) \*transfer)  
*LPSPI slave transfers data using eDMA.*
- void [LPSPI\\_SlaveTransferAbortEDMA](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_edma\\_handle\\_t](#) \*handle)  
*LPSPI slave aborts a transfer which is using eDMA.*
- [status\\_t LPSPI\\_SlaveTransferGetCountEDMA](#) (LPSPI\_Type \*base, [lpspi\\_slave\\_edma\\_handle\\_t](#) \*handle, size\_t \*count)  
*Gets the slave eDMA transfer remaining bytes.*

## Driver version

- #define [FSL\\_LPSPI\\_EDMA\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 4, 4))  
*LPSPI EDMA driver version.*
- #define [DMA\\_MAX\\_TRANSFER\\_COUNT](#) 0x7FFFU  
*DMA max transfer size.*

## 50.2 Data Structure Documentation

### 50.2.1 struct [\\_lpspi\\_master\\_edma\\_handle](#)

#### Data Fields

- volatile bool [isPcsContinuous](#)  
*Is PCS continuous in transfer.*
- volatile bool [isByteSwap](#)  
*A flag that whether should byte swap.*
- volatile uint8\_t [fifoSize](#)  
*FIFO dataSize.*
- volatile uint8\_t [rxWatermark](#)  
*Rx watermark.*
- volatile uint8\_t [bytesEachWrite](#)  
*Bytes for each write TDR.*
- volatile uint8\_t [bytesEachRead](#)  
*Bytes for each read RDR.*
- volatile uint8\_t [bytesLastRead](#)  
*Bytes for last read RDR.*
- volatile bool [isThereExtraRxBytes](#)  
*Is there extra RX byte.*
- const uint8\_t \*volatile [txData](#)  
*Send buffer.*
- uint8\_t \*volatile [rxData](#)  
*Receive buffer.*

- volatile size\_t [txRemainingByteCount](#)  
*Number of bytes remaining to send.*
- volatile size\_t [rxRemainingByteCount](#)  
*Number of bytes remaining to receive.*
- volatile uint32\_t [writeRegRemainingTimes](#)  
*Write TDR register remaining times.*
- volatile uint32\_t [readRegRemainingTimes](#)  
*Read RDR register remaining times.*
- uint32\_t [totalByteCount](#)  
*Number of transfer bytes.*
- [edma\\_tcd\\_t](#) \* [lastTimeTCD](#)  
*Pointer to the lastTime TCD.*
- bool [isMultiDMATransmit](#)  
*Is there multi DMA transmit.*
- volatile uint8\_t [dmaTransmitTime](#)  
*DMA Transfer times.*
- uint32\_t [lastTimeDataBytes](#)  
*DMA transmit last Time data Bytes.*
- uint32\_t [dataBytesEveryTime](#)  
*Bytes in a time for DMA transfer, default is DMA\_MAX\_TRANSFER\_COUNT.*
- [edma\\_transfer\\_config\\_t](#) [transferConfigRx](#)  
*Config of DMA rx channel.*
- [edma\\_transfer\\_config\\_t](#) [transferConfigTx](#)  
*Config of DMA tx channel.*
- uint32\_t [txBuffIfNull](#)  
*Used if there is not txData for DMA purpose.*
- uint32\_t [rxBuffIfNull](#)  
*Used if there is not rxData for DMA purpose.*
- uint32\_t [transmitCommand](#)  
*Used to write TCR for DMA purpose.*
- volatile uint8\_t [state](#)  
*LPSPi transfer state , \_lpspi\_transfer\_state.*
- uint8\_t [nbytes](#)  
*eDMA minor byte transfer count initially configured.*
- [lpspi\\_master\\_edma\\_transfer\\_callback\\_t](#) [callback](#)  
*Completion callback.*
- void \* [userData](#)  
*Callback user data.*
- [edma\\_handle\\_t](#) \* [edmaRxRegToRxDataHandle](#)  
*edma\_handle\_t handle point used for RxReg to RxData buff*
- [edma\\_handle\\_t](#) \* [edmaTxDataToTxRegHandle](#)  
*edma\_handle\_t handle point used for TxData to TxReg buff*
- [edma\\_tcd\\_t](#) [lpspiSoftwareTCD](#) [3]  
*SoftwareTCD, internal used.*



## Field Documentation

- (1) volatile bool \_lpspi\_master\_edma\_handle::isPcsContinuous
- (2) volatile bool \_lpspi\_master\_edma\_handle::isByteSwap
- (3) volatile uint8\_t \_lpspi\_master\_edma\_handle::fifoSize
- (4) volatile uint8\_t \_lpspi\_master\_edma\_handle::rxWatermark
- (5) volatile uint8\_t \_lpspi\_master\_edma\_handle::bytesEachWrite
- (6) volatile uint8\_t \_lpspi\_master\_edma\_handle::bytesEachRead
- (7) volatile uint8\_t \_lpspi\_master\_edma\_handle::bytesLastRead
- (8) volatile bool \_lpspi\_master\_edma\_handle::isThereExtraRxBytes
- (9) const uint8\_t\* volatile \_lpspi\_master\_edma\_handle::txData
- (10) uint8\_t\* volatile \_lpspi\_master\_edma\_handle::rxData
- (11) volatile size\_t \_lpspi\_master\_edma\_handle::txRemainingByteCount
- (12) volatile size\_t \_lpspi\_master\_edma\_handle::rxRemainingByteCount
- (13) volatile uint32\_t \_lpspi\_master\_edma\_handle::writeRegRemainingTimes
- (14) volatile uint32\_t \_lpspi\_master\_edma\_handle::readRegRemainingTimes
- (15) volatile uint8\_t \_lpspi\_master\_edma\_handle::dmaTransmitTime
- (16) edma\_transfer\_config\_t \_lpspi\_master\_edma\_handle::transferConfigRx
- (17) edma\_transfer\_config\_t \_lpspi\_master\_edma\_handle::transferConfigTx
- (18) uint32\_t \_lpspi\_master\_edma\_handle::txBuffIfNull
- (19) uint32\_t \_lpspi\_master\_edma\_handle::rxBuffIfNull
- (20) uint32\_t \_lpspi\_master\_edma\_handle::transmitCommand
- (21) volatile uint8\_t \_lpspi\_master\_edma\_handle::state
- (22) uint8\_t \_lpspi\_master\_edma\_handle::nbytes
- (23) lpspi\_master\_edma\_transfer\_callback\_t \_lpspi\_master\_edma\_handle::callback
- (24) void\* \_lpspi\_master\_edma\_handle::userData

## 50.2.2 struct \_lpspi\_slave\_edma\_handle

## Data Fields

- volatile bool [isByteSwap](#)

- *A flag that whether should byte swap.*
- volatile uint8\_t [fifoSize](#)  
*FIFO dataSize.*
- volatile uint8\_t [rxWatermark](#)  
*Rx watermark.*
- volatile uint8\_t [bytesEachWrite](#)  
*Bytes for each write TDR.*
- volatile uint8\_t [bytesEachRead](#)  
*Bytes for each read RDR.*
- volatile uint8\_t [bytesLastRead](#)  
*Bytes for last read RDR.*
- volatile bool [isThereExtraRxBytes](#)  
*Is there extra RX byte.*
- uint8\_t [nbytes](#)  
*eDMA minor byte transfer count initially configured.*
- const uint8\_t \*volatile [txData](#)  
*Send buffer.*
- uint8\_t \*volatile [rxData](#)  
*Receive buffer.*
- volatile size\_t [txRemainingByteCount](#)  
*Number of bytes remaining to send.*
- volatile size\_t [rxRemainingByteCount](#)  
*Number of bytes remaining to receive.*
- volatile uint32\_t [writeRegRemainingTimes](#)  
*Write TDR register remaining times.*
- volatile uint32\_t [readRegRemainingTimes](#)  
*Read RDR register remaining times.*
- uint32\_t [totalByteCount](#)  
*Number of transfer bytes.*
- uint32\_t [txBuffIfNull](#)  
*Used if there is not txData for DMA purpose.*
- uint32\_t [rxBuffIfNull](#)  
*Used if there is not rxData for DMA purpose.*
- volatile uint8\_t [state](#)  
*LPSPI transfer state.*
- uint32\_t [errorCount](#)  
*Error count for slave transfer.*
- [lpspi\\_slave\\_edma\\_transfer\\_callback\\_t](#) callback  
*Completion callback.*
- void \* [userData](#)  
*Callback user data.*
- [edma\\_handle\\_t](#) \* [edmaRxRegToRxDataHandle](#)  
*edma\_handle\_t handle point used for RxReg to RxData buff*
- [edma\\_handle\\_t](#) \* [edmaTxDataToTxRegHandle](#)  
*edma\_handle\_t handle point used for TxData to TxReg*
- [edma\\_tcd\\_t](#) [lpspiSoftwareTCD](#) [2]  
*SoftwareTCD, internal used.*

## Field Documentation

- (1) `volatile bool _lpspi_slave_edma_handle::isByteSwap`
- (2) `volatile uint8_t _lpspi_slave_edma_handle::fifoSize`
- (3) `volatile uint8_t _lpspi_slave_edma_handle::rxWatermark`
- (4) `volatile uint8_t _lpspi_slave_edma_handle::bytesEachWrite`
- (5) `volatile uint8_t _lpspi_slave_edma_handle::bytesEachRead`
- (6) `volatile uint8_t _lpspi_slave_edma_handle::bytesLastRead`
- (7) `volatile bool _lpspi_slave_edma_handle::isThereExtraRxBytes`
- (8) `uint8_t _lpspi_slave_edma_handle::nbytes`
- (9) `const uint8_t* volatile _lpspi_slave_edma_handle::txData`
- (10) `uint8_t* volatile _lpspi_slave_edma_handle::rxData`
- (11) `volatile size_t _lpspi_slave_edma_handle::txRemainingByteCount`
- (12) `volatile size_t _lpspi_slave_edma_handle::rxRemainingByteCount`
- (13) `volatile uint32_t _lpspi_slave_edma_handle::writeRegRemainingTimes`
- (14) `volatile uint32_t _lpspi_slave_edma_handle::readRegRemainingTimes`
- (15) `uint32_t _lpspi_slave_edma_handle::txBuffIfNull`
- (16) `uint32_t _lpspi_slave_edma_handle::rxBuffIfNull`
- (17) `volatile uint8_t _lpspi_slave_edma_handle::state`
- (18) `uint32_t _lpspi_slave_edma_handle::errorCount`
- (19) `lpspi_slave_edma_transfer_callback_t _lpspi_slave_edma_handle::callback`
- (20) `void* _lpspi_slave_edma_handle::userData`

## 50.3 Macro Definition Documentation

### 50.3.1 `#define FSL_LPSPi_EDMA_DRIVER_VERSION (MAKE_VERSION(2, 4, 4))`

## 50.4 Typedef Documentation

### 50.4.1 `typedef void(* lpspi_master_edma_transfer_callback_t)(LPSPi_Type *base, lpspi_master_edma_handle_t *handle, status_t status, void *userData)`

## Parameters

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral base address.                                   |
| <i>handle</i>   | Pointer to the handle for the LPSPI master.                      |
| <i>status</i>   | Success or error code describing whether the transfer completed. |
| <i>userData</i> | Arbitrary pointer-dataSized value passed from the application.   |

#### 50.4.2 **typedef void(\* lpspi\_slave\_edma\_transfer\_callback\_t)(LPSPI\_Type \*base, lpspi\_slave\_edma\_handle\_t \*handle, status\_t status, void \*userData)**

## Parameters

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral base address.                                   |
| <i>handle</i>   | Pointer to the handle for the LPSPI slave.                       |
| <i>status</i>   | Success or error code describing whether the transfer completed. |
| <i>userData</i> | Arbitrary pointer-dataSized value passed from the application.   |

## 50.5 Function Documentation

### 50.5.1 **void LPSPI\_MasterTransferCreateHandleEDMA ( LPSPI\_Type \* *base*, lpspi\_master\_edma\_handle\_t \* *handle*, lpspi\_master\_edma\_transfer\_callback\_t *callback*, void \* *userData*, edma\_handle\_t \* *edmaRxRegToRxDataHandle*, edma\_handle\_t \* *edmaTxDataToTxRegHandle* )**

This function initializes the LPSPI eDMA handle which can be used for other LPSPI transactional APIs. Usually, for a specified LPSPI instance, call this API once to get the initialized handle.

Note that the LPSPI eDMA has a separated (Rx and Tx as two sources) or shared (Rx and Tx are the same source) DMA request source. (1) For a separated DMA request source, enable and set the Rx DMAMUX source for *edmaRxRegToRxDataHandle* and Tx DMAMUX source for *edmaTxDataToTxRegHandle*. (2) For a shared DMA request source, enable and set the Rx/Tx DMAMUX source for *edmaRxRegToRxDataHandle*.

## Parameters

|             |                                |
|-------------|--------------------------------|
| <i>base</i> | LPSPI peripheral base address. |
|-------------|--------------------------------|



|                                  |                                                                              |
|----------------------------------|------------------------------------------------------------------------------|
| <i>handle</i>                    | LPSPI handle pointer to <code>lpspi_master_edma_handle_t</code> .            |
| <i>callback</i>                  | LPSPI callback.                                                              |
| <i>userData</i>                  | callback function parameter.                                                 |
| <i>edmaRxRegTo-RxDataHandle</i>  | <code>edmaRxRegToRxDataHandle</code> pointer to <code>edma_handle_t</code> . |
| <i>edmaTxData-ToTxReg-Handle</i> | <code>edmaTxDataToTxRegHandle</code> pointer to <code>edma_handle_t</code> . |

### 50.5.2 `status_t LPSPI_MasterTransferEDMA ( LPSPI_Type * base, lpspi_master_edma_handle_t * handle, lpspi_transfer_t * transfer )`

This function transfers data using eDMA. This is a non-blocking function, which returns right away. When all data is transferred, the callback function is called.

Note: The transfer data size should be an integer multiple of `bytesPerFrame` if `bytesPerFrame` is less than or equal to 4. For `bytesPerFrame` greater than 4: The transfer data size should be equal to `bytesPerFrame` if the `bytesPerFrame` is not an integer multiple of 4. Otherwise, the transfer data size can be an integer multiple of `bytesPerFrame`.

Parameters

|                 |                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral base address.                                                                |
| <i>handle</i>   | pointer to <code>lpspi_master_edma_handle_t</code> structure which stores the transfer state. |
| <i>transfer</i> | pointer to <code>lpspi_transfer_t</code> structure.                                           |

Returns

status of `status_t`.

### 50.5.3 `status_t LPSPI_MasterTransferPrepareEDMALite ( LPSPI_Type * base, lpspi_master_edma_handle_t * handle, uint32_t configFlags )`

This function is preparing to transfer data using eDMA, work with `LPSPI_MasterTransferEDMALite`.

## Parameters

|                    |                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------|
| <i>base</i>        | LPSPI peripheral base address.                                                                |
| <i>handle</i>      | pointer to <code>lpspi_master_edma_handle_t</code> structure which stores the transfer state. |
| <i>configFlags</i> | transfer configuration flags. <a href="#">_lpspi_transfer_config_flag_for_master</a> .        |

## Returns

Indicates whether LPSPi master transfer was successful or not.

## Return values

|                           |                           |
|---------------------------|---------------------------|
| <i>kStatus_Success</i>    | Execution successfully.   |
| <i>kStatus_LPSPi_Busy</i> | The LPSPi device is busy. |

#### 50.5.4 `status_t LPSPi_MasterTransferEDMALite ( LPSPi_Type * base, lpspi_master_edma_handle_t * handle, lpspi_transfer_t * transfer )`

This function transfers data using eDMA. This is a non-blocking function, which returns right away. When all data is transferred, the callback function is called.

Note: This API is only for transfer through DMA without configuration. Before calling this API, you must call `LPSPi_MasterTransferPrepareEDMALite` to configure it once. The transfer data size should be an integer multiple of `bytesPerFrame` if `bytesPerFrame` is less than or equal to 4. For `bytesPerFrame` greater than 4: The transfer data size should be equal to `bytesPerFrame` if the `bytesPerFrame` is not an integer multiple of 4. Otherwise, the transfer data size can be an integer multiple of `bytesPerFrame`.

## Parameters

|                 |                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------|
| <i>base</i>     | LPSPi peripheral base address.                                                                |
| <i>handle</i>   | pointer to <code>lpspi_master_edma_handle_t</code> structure which stores the transfer state. |
| <i>transfer</i> | pointer to <code>lpspi_transfer_t</code> structure, config field is not used.                 |

## Returns

Indicates whether LPSPi master transfer was successful or not.

Return values

|                                |                                    |
|--------------------------------|------------------------------------|
| <i>kStatus_Success</i>         | Execution successfully.            |
| <i>kStatus_LPSPi_Busy</i>      | The LPSPi device is busy.          |
| <i>kStatus_InvalidArgument</i> | The transfer structure is invalid. |

#### 50.5.5 void LPSPi\_MasterTransferAbortEDMA ( LPSPi\_Type \* *base*, lpspi\_master\_edma\_handle\_t \* *handle* )

This function aborts a transfer which is using eDMA.

Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | LPSPi peripheral base address.                                                   |
| <i>handle</i> | pointer to lpspi_master_edma_handle_t structure which stores the transfer state. |

#### 50.5.6 status\_t LPSPi\_MasterTransferGetCountEDMA ( LPSPi\_Type \* *base*, lpspi\_master\_edma\_handle\_t \* *handle*, size\_t \* *count* )

This function gets the master eDMA transfer remaining bytes.

Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>base</i>   | LPSPi peripheral base address.                                                   |
| <i>handle</i> | pointer to lpspi_master_edma_handle_t structure which stores the transfer state. |
| <i>count</i>  | Number of bytes transferred so far by the EDMA transaction.                      |

Returns

status of status\_t.

#### 50.5.7 void LPSPi\_SlaveTransferCreateHandleEDMA ( LPSPi\_Type \* *base*, lpspi\_slave\_edma\_handle\_t \* *handle*, lpspi\_slave\_edma\_transfer\_callback\_t *callback*, void \* *userData*, edma\_handle\_t \* *edmaRxRegToRxDataHandle*, edma\_handle\_t \* *edmaTxDataToTxRegHandle* )

This function initializes the LPSPi eDMA handle which can be used for other LPSPi transactional APIs. Usually, for a specified LPSPi instance, call this API once to get the initialized handle.

Note that LPSPI eDMA has a separated (Rx and Tx as two sources) or shared (Rx and Tx as the same source) DMA request source.

(1) For a separated DMA request source, enable and set the Rx DMAMUX source for edmaRxRegToRxDataHandle and Tx DMAMUX source for edmaTxDataToTxRegHandle. (2) For a shared DMA request source, enable and set the Rx/Rx DMAMUX source for edmaRxRegToRxDataHandle .

Parameters

|                                  |                                                    |
|----------------------------------|----------------------------------------------------|
| <i>base</i>                      | LPSPI peripheral base address.                     |
| <i>handle</i>                    | LPSPI handle pointer to lpspi_slave_edma_handle_t. |
| <i>callback</i>                  | LPSPI callback.                                    |
| <i>userData</i>                  | callback function parameter.                       |
| <i>edmaRxRegTo-RxDataHandle</i>  | edmaRxRegToRxDataHandle pointer to edma_handle_t.  |
| <i>edmaTxData-ToTxReg-Handle</i> | edmaTxDataToTxRegHandle pointer to edma_handle_t.  |

### 50.5.8 status\_t LPSPI\_SlaveTransferEDMA ( LPSPI\_Type \* *base*, lpspi\_slave\_edma\_handle\_t \* *handle*, lpspi\_transfer\_t \* *transfer* )

This function transfers data using eDMA. This is a non-blocking function, which return right away. When all data is transferred, the callback function is called.

Note: The transfer data size should be an integer multiple of bytesPerFrame if bytesPerFrame is less than or equal to 4. For bytesPerFrame greater than 4: The transfer data size should be equal to bytesPerFrame if the bytesPerFrame is not an integer multiple of 4. Otherwise, the transfer data size can be an integer multiple of bytesPerFrame.

Parameters

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>base</i>     | LPSPI peripheral base address.                                                  |
| <i>handle</i>   | pointer to lpspi_slave_edma_handle_t structure which stores the transfer state. |
| <i>transfer</i> | pointer to lpspi_transfer_t structure.                                          |

Returns

status of status\_t.

**50.5.9 void LPSPI\_SlaveTransferAbortEDMA ( LPSPI\_Type \* *base*,  
lpspi\_slave\_edma\_handle\_t \* *handle* )**

This function aborts a transfer which is using eDMA.

## Parameters

|               |                                                                                              |
|---------------|----------------------------------------------------------------------------------------------|
| <i>base</i>   | LPSPi peripheral base address.                                                               |
| <i>handle</i> | pointer to <code>lpspi_slave_edma_handle_t</code> structure which stores the transfer state. |

### 50.5.10 `status_t LPSPI_SlaveTransferGetCountEDMA ( LPSPI_Type * base, lpspi_slave_edma_handle_t * handle, size_t * count )`

This function gets the slave eDMA transfer remaining bytes.

## Parameters

|               |                                                                                              |
|---------------|----------------------------------------------------------------------------------------------|
| <i>base</i>   | LPSPi peripheral base address.                                                               |
| <i>handle</i> | pointer to <code>lpspi_slave_edma_handle_t</code> structure which stores the transfer state. |
| <i>count</i>  | Number of bytes transferred so far by the eDMA transaction.                                  |

## Returns

status of `status_t`.

# Chapter 51

## Lpuart\_edma\_driver

### 51.1 Overview

#### Data Structures

- struct `_lpuart_edma_handle`  
*LPUART eDMA handle. [More...](#)*

#### Typedefs

- typedef void(\* `lpuart_edma_transfer_callback_t` )(LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle, `status_t` status, void \*userData)  
*LPUART transfer callback function.*

#### Driver version

- #define `FSL_LPUART_EDMA_DRIVER_VERSION` (`MAKE_VERSION`(2, 6, 0))  
*LPUART EDMA driver version.*

#### eDMA transactional

- void `LPUART_TransferCreateHandleEDMA` (LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle, `lpuart_edma_transfer_callback_t` callback, void \*userData, `edma_handle_t` \*txEdmaHandle, `edma_handle_t` \*rxEdmaHandle)  
*Initializes the LPUART handle which is used in transactional functions.*
- `status_t` `LPUART_SendEDMA` (LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle, `lpuart_transfer_t` \*xfer)  
*Sends data using eDMA.*
- `status_t` `LPUART_ReceiveEDMA` (LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle, `lpuart_transfer_t` \*xfer)  
*Receives data using eDMA.*
- void `LPUART_TransferAbortSendEDMA` (LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle)  
*Aborts the sent data using eDMA.*
- void `LPUART_TransferAbortReceiveEDMA` (LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle)  
*Aborts the received data using eDMA.*
- `status_t` `LPUART_TransferGetSendCountEDMA` (LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle, `uint32_t` \*count)  
*Gets the number of bytes written to the LPUART TX register.*
- `status_t` `LPUART_TransferGetReceiveCountEDMA` (LPUART\_Type \*base, `lpuart_edma_handle_t` \*handle, `uint32_t` \*count)  
*Gets the number of received bytes.*
- void `LPUART_TransferEdmaHandleIRQ` (LPUART\_Type \*base, void \*lpuartEdmaHandle)  
*LPUART eDMA IRQ handle function.*

## 51.2 Data Structure Documentation

### 51.2.1 struct \_lpuart\_edma\_handle

#### Data Fields

- [lpuart\\_edma\\_transfer\\_callback\\_t](#) [callback](#)  
*Callback function.*
- void \* [userData](#)  
*LPUART callback function parameter.*
- size\_t [rxDataSizeAll](#)  
*Size of the data to receive.*
- size\_t [txDataSizeAll](#)  
*Size of the data to send out.*
- [edma\\_handle\\_t](#) \* [txEdmaHandle](#)  
*The eDMA TX channel used.*
- [edma\\_handle\\_t](#) \* [rxEdmaHandle](#)  
*The eDMA RX channel used.*
- uint8\_t [nbytes](#)  
*eDMA minor byte transfer count initially configured.*
- volatile uint8\_t [txState](#)  
*TX transfer state.*
- volatile uint8\_t [rxState](#)  
*RX transfer state.*

#### Field Documentation

- (1) [lpuart\\_edma\\_transfer\\_callback\\_t](#) [\\_lpuart\\_edma\\_handle::callback](#)
- (2) void\* [\\_lpuart\\_edma\\_handle::userData](#)
- (3) size\_t [\\_lpuart\\_edma\\_handle::rxDataSizeAll](#)
- (4) size\_t [\\_lpuart\\_edma\\_handle::txDataSizeAll](#)
- (5) [edma\\_handle\\_t](#)\* [\\_lpuart\\_edma\\_handle::txEdmaHandle](#)
- (6) [edma\\_handle\\_t](#)\* [\\_lpuart\\_edma\\_handle::rxEdmaHandle](#)
- (7) uint8\_t [\\_lpuart\\_edma\\_handle::nbytes](#)
- (8) volatile uint8\_t [\\_lpuart\\_edma\\_handle::txState](#)

## 51.3 Macro Definition Documentation

### 51.3.1 #define FSL\_LPUART\_EDMA\_DRIVER\_VERSION (MAKE\_VERSION(2, 6, 0))



## 51.4 Typedef Documentation

**51.4.1** `typedef void(* lpuart_edma_transfer_callback_t)(LPUART_Type *base, lpuart_edma_handle_t *handle, status_t status, void *userData)`

## 51.5 Function Documentation

**51.5.1** `void LPUART_TransferCreateHandleEDMA ( LPUART_Type * base, lpuart_edma_handle_t * handle, lpuart_edma_transfer_callback_t callback, void * userData, edma_handle_t * txEdmaHandle, edma_handle_t * rxEdmaHandle )`

Note

This function disables all LPUART interrupts.

Parameters

|                     |                                                |
|---------------------|------------------------------------------------|
| <i>base</i>         | LPUART peripheral base address.                |
| <i>handle</i>       | Pointer to lpuart_edma_handle_t structure.     |
| <i>callback</i>     | Callback function.                             |
| <i>userData</i>     | User data.                                     |
| <i>txEdmaHandle</i> | User requested DMA handle for TX DMA transfer. |
| <i>rxEdmaHandle</i> | User requested DMA handle for RX DMA transfer. |

**51.5.2** `status_t LPUART_SendEDMA ( LPUART_Type * base, lpuart_edma_handle_t * handle, lpuart_transfer_t * xfer )`

This function sends data using eDMA. This is a non-blocking function, which returns right away. When all data is sent, the send callback function is called.

Parameters

|               |                                                                         |
|---------------|-------------------------------------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.                                         |
| <i>handle</i> | LPUART handle pointer.                                                  |
| <i>xfer</i>   | LPUART eDMA transfer structure. See <a href="#">lpuart_transfer_t</a> . |

Return values

|                                |                             |
|--------------------------------|-----------------------------|
| <i>kStatus_Success</i>         | if succeed, others failed.  |
| <i>kStatus_LPUART_TxBusy</i>   | Previous transfer on going. |
| <i>kStatus_InvalidArgument</i> | Invalid argument.           |

### 51.5.3 status\_t LPUART\_ReceiveEDMA ( LPUART\_Type \* *base*, lpuart\_edma\_handle\_t \* *handle*, lpuart\_transfer\_t \* *xfer* )

This function receives data using eDMA. This is non-blocking function, which returns right away. When all data is received, the receive callback function is called.

Parameters

|               |                                                                         |
|---------------|-------------------------------------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.                                         |
| <i>handle</i> | Pointer to lpuart_edma_handle_t structure.                              |
| <i>xfer</i>   | LPUART eDMA transfer structure, see <a href="#">lpuart_transfer_t</a> . |

Return values

|                                |                            |
|--------------------------------|----------------------------|
| <i>kStatus_Success</i>         | if succeed, others fail.   |
| <i>kStatus_LPUART_Rx-Busy</i>  | Previous transfer ongoing. |
| <i>kStatus_InvalidArgument</i> | Invalid argument.          |

### 51.5.4 void LPUART\_TransferAbortSendEDMA ( LPUART\_Type \* *base*, lpuart\_edma\_handle\_t \* *handle* )

This function aborts the sent data using eDMA.

Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.            |
| <i>handle</i> | Pointer to lpuart_edma_handle_t structure. |

**51.5.5 void LPUART\_TransferAbortReceiveEDMA ( LPUART\_Type \* *base*,  
lpuart\_edma\_handle\_t \* *handle* )**

This function aborts the received data using eDMA.

## Parameters

|               |                                                         |
|---------------|---------------------------------------------------------|
| <i>base</i>   | LPUART peripheral base address.                         |
| <i>handle</i> | Pointer to <code>lpuart_edma_handle_t</code> structure. |

### 51.5.6 `status_t LPUART_TransferGetSendCountEDMA ( LPUART_Type * base, lpuart_edma_handle_t * handle, uint32_t * count )`

This function gets the number of bytes written to the LPUART TX register by DMA.

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |
| <i>count</i>  | Send bytes count.               |

## Return values

|                                     |                                                             |
|-------------------------------------|-------------------------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | No send in progress.                                        |
| <i>kStatus_InvalidArgument</i>      | Parameter is invalid.                                       |
| <i>kStatus_Success</i>              | Get successfully through the parameter <code>count</code> ; |

### 51.5.7 `status_t LPUART_TransferGetReceiveCountEDMA ( LPUART_Type * base, lpuart_edma_handle_t * handle, uint32_t * count )`

This function gets the number of received bytes.

## Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>base</i>   | LPUART peripheral base address. |
| <i>handle</i> | LPUART handle pointer.          |
| <i>count</i>  | Receive bytes count.            |

## Return values

|                                     |                                               |
|-------------------------------------|-----------------------------------------------|
| <i>kStatus_NoTransferInProgress</i> | No receive in progress.                       |
| <i>kStatus_InvalidArgument</i>      | Parameter is invalid.                         |
| <i>kStatus_Success</i>              | Get successfully through the parameter count; |

### 51.5.8 void LPUART\_TransferEdmaHandleIRQ ( LPUART\_Type \* *base*, void \* *lpuartEdmaHandle* )

This function handles the LPUART tx complete IRQ request and invoke user callback. It is not set to static so that it can be used in user application.

## Note

This function is used as default IRQ handler by double weak mechanism. If user's specific IRQ handler is implemented, make sure this function is invoked in the handler.

## Parameters

|                          |                                 |
|--------------------------|---------------------------------|
| <i>base</i>              | LPUART peripheral base address. |
| <i>lpuartEdma-Handle</i> | LPUART handle pointer.          |

## Chapter 52

### Ltc\_edma\_driver

#### 52.1 Overview

##### Data Structures

- struct [\\_ltc\\_edma\\_handle](#)  
*LTC eDMA handle. [More...](#)*

##### Typedefs

- typedef void(\* [ltc\\_edma\\_callback\\_t](#))(LTC\_Type \*base, [ltc\\_edma\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
*LTC eDMA callback function.*
- typedef [status\\_t](#)(\* [ltc\\_edma\\_state\\_machine\\_t](#))(LTC\_Type \*base, [ltc\\_edma\\_handle\\_t](#) \*handle)  
*LTC eDMA state machine function.*

##### Functions

- void [LTC\\_CreateHandleEDMA](#) (LTC\_Type \*base, [ltc\\_edma\\_handle\\_t](#) \*handle, [ltc\\_edma\\_callback\\_t](#) callback, void \*userData, [edma\\_handle\\_t](#) \*inputFifoEdmaHandle, [edma\\_handle\\_t](#) \*outputFifoEdmaHandle)  
*Init the LTC eDMA handle which is used in transactional functions.*

##### Driver version

- #define [FSL\\_LTC\\_EDMA\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 0, 15))  
*LTC EDMA driver version.*

#### 52.2 Data Structure Documentation

##### 52.2.1 struct [\\_ltc\\_edma\\_handle](#)

It is defined only for private usage inside LTC eDMA driver.

##### Data Fields

- [ltc\\_edma\\_callback\\_t](#) callback  
*Callback function.*
- void \* [userData](#)  
*LTC callback function parameter.*
- [edma\\_handle\\_t](#) \* [inputFifoEdmaHandle](#)  
*The eDMA TX channel used.*

- `edma_handle_t * outputFifoEdmaHandle`  
*The eDMA RX channel used.*
- `ltc_edma_state_machine_t state_machine`  
*State machine.*
- `uint32_t state`  
*Internal state.*
- `const uint8_t * inData`  
*Input data.*
- `uint8_t * outData`  
*Output data.*
- `uint32_t size`  
*Size of input and output data in bytes.*
- `uint32_t modeReg`  
*LTC mode register.*
- `uint8_t * counter`  
*Input counter (updates on return)*
- `const uint8_t * key`  
*Input key to use for forward AES cipher.*
- `uint32_t keySize`  
*Size of the input key, in bytes.*
- `uint8_t * counterlast`  
*Output cipher of last counter, for chained CTR calls.*
- `uint32_t * szLeft`  
*Output number of bytes in left unused in counterlast block.*
- `uint32_t lastSize`  
*Last size.*

**Field Documentation**

- (1) `ltc_edma_callback_t _ltc_edma_handle::callback`
- (2) `void* _ltc_edma_handle::userData`
- (3) `edma_handle_t* _ltc_edma_handle::inputFifoEdmaHandle`
- (4) `edma_handle_t* _ltc_edma_handle::outputFifoEdmaHandle`
- (5) `ltc_edma_state_machine_t _ltc_edma_handle::state_machine`
- (6) `uint32_t _ltc_edma_handle::state`
- (7) `const uint8_t* _ltc_edma_handle::inData`
- (8) `uint8_t* _ltc_edma_handle::outData`
- (9) `uint32_t _ltc_edma_handle::size`
- (10) `uint32_t _ltc_edma_handle::modeReg`
- (11) `uint32_t _ltc_edma_handle::keySize`

Must be 16, 24, or 32.

- (12) `uint8_t* _ltc_edma_handle::counterlast`

NULL can be passed if chained calls are not used.

- (13) `uint32_t* _ltc_edma_handle::szLeft`

NULL can be passed if chained calls are not used.

- (14) `uint32_t _ltc_edma_handle::lastSize`

**52.3 Macro Definition Documentation****52.3.1 #define FSL\_LTC\_EDMA\_DRIVER\_VERSION (MAKE\_VERSION(2, 0, 15))**

Version 2.0.15.



## 52.4 Typedef Documentation

**52.4.1** `typedef void(* ltc_edma_callback_t)(LTC_Type *base, ltc_edma_handle_t *handle, status_t status, void *userData)`

**52.4.2** `typedef status_t(* ltc_edma_state_machine_t)(LTC_Type *base, ltc_edma_handle_t *handle)`

It is defined only for private usage inside LTC eDMA driver.

## 52.5 Function Documentation

**52.5.1** `void LTC_CreateHandleEDMA ( LTC_Type * base, ltc_edma_handle_t * handle, ltc_edma_callback_t callback, void * userData, edma_handle_t * inputFifoEdmaHandle, edma_handle_t * outputFifoEdmaHandle )`

Parameters

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| <i>base</i>                  | LTC module base address                          |
| <i>handle</i>                | Pointer to ltc_edma_handle_t structure           |
| <i>callback</i>              | Callback function, NULL means no callback.       |
| <i>userData</i>              | Callback function parameter.                     |
| <i>inputFifo-EdmaHandle</i>  | User requested eDMA handle for Input FIFO eDMA.  |
| <i>outputFifo-EdmaHandle</i> | User requested eDMA handle for Output FIFO eDMA. |

## Chapter 53

### Ltc\_edma\_driver\_aes

#### 53.1 Overview

##### Macros

- #define `LTC_AES_DecryptCtrEDMA`(base, handle, input, output, size, counter, key, keySize, counterlast, szLeft) `LTC_AES_CryptCtrEDMA`(base, handle, input, output, size, counter, key, keySize, counterlast, szLeft)  
*AES CTR decrypt is mapped to the AES CTR generic operation.*
- #define `LTC_AES_EncryptCtrEDMA`(base, handle, input, output, size, counter, key, keySize, counterlast, szLeft) `LTC_AES_CryptCtrEDMA`(base, handle, input, output, size, counter, key, keySize, counterlast, szLeft)  
*AES CTR encrypt is mapped to the AES CTR generic operation.*

##### Functions

- `status_t LTC_AES_EncryptEcbEDMA` (LTC\_Type \*base, `ltc_edma_handle_t` \*handle, const uint8\_t \*plaintext, uint8\_t \*ciphertext, uint32\_t size, const uint8\_t \*key, uint32\_t keySize)  
*Encrypts AES using the ECB block mode.*
- `status_t LTC_AES_DecryptEcbEDMA` (LTC\_Type \*base, `ltc_edma_handle_t` \*handle, const uint8\_t \*ciphertext, uint8\_t \*plaintext, uint32\_t size, const uint8\_t \*key, uint32\_t keySize, `ltc_aes_key_t` keyType)  
*Decrypts AES using ECB block mode.*
- `status_t LTC_AES_EncryptCbcEDMA` (LTC\_Type \*base, `ltc_edma_handle_t` \*handle, const uint8\_t \*plaintext, uint8\_t \*ciphertext, uint32\_t size, const uint8\_t iv[LTC\_AES\_IV\_SIZE], const uint8\_t \*key, uint32\_t keySize)  
*Encrypts AES using CBC block mode.*
- `status_t LTC_AES_DecryptCbcEDMA` (LTC\_Type \*base, `ltc_edma_handle_t` \*handle, const uint8\_t \*ciphertext, uint8\_t \*plaintext, uint32\_t size, const uint8\_t iv[LTC\_AES\_IV\_SIZE], const uint8\_t \*key, uint32\_t keySize, `ltc_aes_key_t` keyType)  
*Decrypts AES using CBC block mode.*
- `status_t LTC_AES_CryptCtrEDMA` (LTC\_Type \*base, `ltc_edma_handle_t` \*handle, const uint8\_t \*input, uint8\_t \*output, uint32\_t size, uint8\_t counter[LTC\_AES\_BLOCK\_SIZE], const uint8\_t \*key, uint32\_t keySize, uint8\_t counterlast[LTC\_AES\_BLOCK\_SIZE], uint32\_t \*szLeft)  
*Encrypts or decrypts AES using CTR block mode.*

#### 53.2 Function Documentation

##### 53.2.1 `status_t LTC_AES_EncryptEcbEDMA` ( LTC\_Type \* *base*, `ltc_edma_handle_t` \* *handle*, const uint8\_t \* *plaintext*, uint8\_t \* *ciphertext*, uint32\_t *size*, const uint8\_t \* *key*, uint32\_t *keySize* )

Encrypts AES using the ECB block mode.

## Parameters

|     |                   |                                                                                         |
|-----|-------------------|-----------------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                             |
|     | <i>handle</i>     | pointer to <code>ltc_edma_handle_t</code> structure which stores the transaction state. |
|     | <i>plaintext</i>  | Input plain text to encrypt                                                             |
| out | <i>ciphertext</i> | Output cipher text                                                                      |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes.                   |
|     | <i>key</i>        | Input key to use for encryption                                                         |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.                                 |

## Returns

Status from encrypt operation

**53.2.2** `status_t LTC_AES_DecryptEcbEDMA ( LTC_Type * base, ltc_edma_handle_t * handle, const uint8_t * ciphertext, uint8_t * plaintext, uint32_t size, const uint8_t * key, uint32_t keySize, ltc_aes_key_t keyType )`

Decrypts AES using ECB block mode.

## Parameters

|     |                   |                                                                                            |
|-----|-------------------|--------------------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                                |
|     | <i>handle</i>     | pointer to <code>ltc_edma_handle_t</code> structure which stores the transaction state.    |
|     | <i>ciphertext</i> | Input cipher text to decrypt                                                               |
| out | <i>plaintext</i>  | Output plain text                                                                          |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes.                      |
|     | <i>key</i>        | Input key.                                                                                 |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.                                    |
|     | <i>keyType</i>    | Input type of the key (allows to directly load decrypt key for AES ECB decrypt operation.) |

## Returns

Status from decrypt operation

**53.2.3** `status_t LTC_AES_EncryptCbcEDMA ( LTC_Type * base, ltc_edma_handle_t * handle, const uint8_t * plaintext, uint8_t * ciphertext, uint32_t size, const uint8_t iv[LTC_AES_IV_SIZE], const uint8_t * key, uint32_t keySize )`

## Parameters

|     |                   |                                                                                         |
|-----|-------------------|-----------------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                             |
|     | <i>handle</i>     | pointer to <code>ltc_edma_handle_t</code> structure which stores the transaction state. |
|     | <i>plaintext</i>  | Input plain text to encrypt                                                             |
| out | <i>ciphertext</i> | Output cipher text                                                                      |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes.                   |
|     | <i>iv</i>         | Input initial vector to combine with the first input block.                             |
|     | <i>key</i>        | Input key to use for encryption                                                         |
|     | <i>keySize</i>    | Size of the input key, in bytes. Must be 16, 24, or 32.                                 |

## Returns

Status from encrypt operation

**53.2.4** `status_t LTC_AES_DecryptCbcEDMA ( LTC_Type * base,  
ltc_edma_handle_t * handle, const uint8_t * ciphertext, uint8_t * plaintext,  
uint32_t size, const uint8_t iv[LTC_AES_IV_SIZE], const uint8_t * key,  
uint32_t keySize, ltc_aes_key_t keyType )`

## Parameters

|     |                   |                                                                                         |
|-----|-------------------|-----------------------------------------------------------------------------------------|
|     | <i>base</i>       | LTC peripheral base address                                                             |
|     | <i>handle</i>     | pointer to <code>ltc_edma_handle_t</code> structure which stores the transaction state. |
|     | <i>ciphertext</i> | Input cipher text to decrypt                                                            |
| out | <i>plaintext</i>  | Output plain text                                                                       |
|     | <i>size</i>       | Size of input and output data in bytes. Must be multiple of 16 bytes.                   |
|     | <i>iv</i>         | Input initial vector to combine with the first input block.                             |
|     | <i>key</i>        | Input key to use for decryption                                                         |

|  |                |                                                                                            |
|--|----------------|--------------------------------------------------------------------------------------------|
|  | <i>keySize</i> | Size of the input key, in bytes. Must be 16, 24, or 32.                                    |
|  | <i>keyType</i> | Input type of the key (allows to directly load decrypt key for AES CBC decrypt operation.) |

## Returns

Status from decrypt operation

**53.2.5** `status_t LTC_AES_CryptCtrEDMA ( LTC_Type * base, ltc_edma_handle_t * handle, const uint8_t * input, uint8_t * output, uint32_t size, uint8_t counter[LTC_AES_BLOCK_SIZE], const uint8_t * key, uint32_t keySize, uint8_t counterlast[LTC_AES_BLOCK_SIZE], uint32_t * szLeft )`

Encrypts or decrypts AES using CTR block mode. AES CTR mode uses only forward AES cipher and same algorithm for encryption and decryption. The only difference between encryption and decryption is that, for encryption, the input argument is plain text and the output argument is cipher text. For decryption, the input argument is cipher text and the output argument is plain text.

## Parameters

|         |                    |                                                                                                         |
|---------|--------------------|---------------------------------------------------------------------------------------------------------|
|         | <i>base</i>        | LTC peripheral base address                                                                             |
|         | <i>handle</i>      | pointer to ltc_edma_handle_t structure which stores the transaction state.                              |
|         | <i>input</i>       | Input data for CTR block mode                                                                           |
| out     | <i>output</i>      | Output data for CTR block mode                                                                          |
|         | <i>size</i>        | Size of input and output data in bytes                                                                  |
| in, out | <i>counter</i>     | Input counter (updates on return)                                                                       |
|         | <i>key</i>         | Input key to use for forward AES cipher                                                                 |
|         | <i>keySize</i>     | Size of the input key, in bytes. Must be 16, 24, or 32.                                                 |
| out     | <i>counterlast</i> | Output cipher of last counter, for chained CTR calls. NULL can be passed if chained calls are not used. |

|     |               |                                                                                                               |
|-----|---------------|---------------------------------------------------------------------------------------------------------------|
| out | <i>szLeft</i> | Output number of bytes in left unused in counterlast block. NULL can be passed if chained calls are not used. |
|-----|---------------|---------------------------------------------------------------------------------------------------------------|

## Returns

Status from encrypt operation

## Chapter 54

### Qspi\_edma\_driver

#### 54.1 Overview

##### Data Structures

- struct `_qspi_edma_handle`  
*QSPI DMA transfer handle, users should not touch the content of the handle. [More...](#)*

##### Typedefs

- typedef void(\* `qspi_edma_callback_t`)(QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle, `status_t` status, void \*userData)  
*QSPI eDMA transfer callback function for finish and error.*

##### Driver version

- #define `FSL_QSPI_EDMA_DRIVER_VERSION` (`MAKE_VERSION`(2, 2, 2))  
*QSPI EDMA driver version 2.2.2.*

##### eDMA Transactional

- void `QSPI_TransferTxCreateHandleEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle, `qspi_edma_callback_t` callback, void \*userData, `edma_handle_t` \*dmaHandle)  
*Initializes the QSPI handle for send which is used in transactional functions and set the callback.*
- void `QSPI_TransferRxCreateHandleEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle, `qspi_edma_callback_t` callback, void \*userData, `edma_handle_t` \*dmaHandle)  
*Initializes the QSPI handle for receive which is used in transactional functions and set the callback.*
- `status_t` `QSPI_TransferSendEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle, `qspi_transfer_t` \*xfer)  
*Transfers QSPI data using an eDMA non-blocking method.*
- `status_t` `QSPI_TransferReceiveEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle, `qspi_transfer_t` \*xfer)  
*Receives data using an eDMA non-blocking method.*
- void `QSPI_TransferAbortSendEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle)  
*Aborts the sent data using eDMA.*
- void `QSPI_TransferAbortReceiveEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle)  
*Aborts the receive data using eDMA.*
- `status_t` `QSPI_TransferGetSendCountEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle, `size_t` \*count)  
*Gets the transferred counts of send.*
- `status_t` `QSPI_TransferGetReceiveCountEDMA` (QuadSPI\_Type \*base, `qspi_edma_handle_t` \*handle, `size_t` \*count)  
*Gets the status of the receive transfer.*



## 54.2 Data Structure Documentation

### 54.2.1 struct \_qspi\_edma\_handle

#### Data Fields

- [edma\\_handle\\_t](#) \* [dmaHandle](#)  
*eDMA handler for QSPI send*
- [size\\_t](#) [transferSize](#)  
*Bytes need to transfer.*
- [uint8\\_t](#) [nbytes](#)  
*eDMA minor byte transfer count initially configured.*
- [uint8\\_t](#) [count](#)  
*The transfer data count in a DMA request.*
- [uint32\\_t](#) [state](#)  
*Internal state for QSPI eDMA transfer.*
- [qspi\\_edma\\_callback\\_t](#) [callback](#)  
*Callback for users while transfer finish or error occurred.*
- [void](#) \* [userData](#)  
*User callback parameter.*

#### Field Documentation

(1) [size\\_t](#) [\\_qspi\\_edma\\_handle::transferSize](#)

(2) [uint8\\_t](#) [\\_qspi\\_edma\\_handle::nbytes](#)

## 54.3 Macro Definition Documentation

54.3.1 **#define FSL\_QSPI\_EDMA\_DRIVER\_VERSION (MAKE\_VERSION(2, 2, 2))**

## 54.4 Function Documentation

54.4.1 **void QSPI\_TransferTxCreateHandleEDMA ( QuadSPI\_Type \* *base*,  
qspi\_edma\_handle\_t \* *handle*, qspi\_edma\_callback\_t *callback*, void \*  
*userData*, edma\_handle\_t \* *dmaHandle* )**

#### Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | QSPI peripheral base address            |
| <i>handle</i> | Pointer to qspi_edma_handle_t structure |

|                  |                                              |
|------------------|----------------------------------------------|
| <i>callback</i>  | QSPI callback, NULL means no callback.       |
| <i>userData</i>  | User callback function data.                 |
| <i>dmaHandle</i> | User requested eDMA handle for eDMA transfer |

**54.4.2 void QSPI\_TransferRxCreateHandleEDMA ( QuadSPI\_Type \* *base*, qspi\_edma\_handle\_t \* *handle*, qspi\_edma\_callback\_t *callback*, void \* *userData*, edma\_handle\_t \* *dmaHandle* )**

Parameters

|                  |                                              |
|------------------|----------------------------------------------|
| <i>base</i>      | QSPI peripheral base address                 |
| <i>handle</i>    | Pointer to qspi_edma_handle_t structure      |
| <i>callback</i>  | QSPI callback, NULL means no callback.       |
| <i>userData</i>  | User callback function data.                 |
| <i>dmaHandle</i> | User requested eDMA handle for eDMA transfer |

**54.4.3 status\_t QSPI\_TransferSendEDMA ( QuadSPI\_Type \* *base*, qspi\_edma\_handle\_t \* *handle*, qspi\_transfer\_t \* *xfer* )**

This function writes data to the QSPI transmit FIFO. This function is non-blocking.

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.                |
| <i>handle</i> | Pointer to qspi_edma_handle_t structure |
| <i>xfer</i>   | QSPI transfer structure.                |

**54.4.4 status\_t QSPI\_TransferReceiveEDMA ( QuadSPI\_Type \* *base*, qspi\_edma\_handle\_t \* *handle*, qspi\_transfer\_t \* *xfer* )**

This function receive data from the QSPI receive buffer/FIFO. This function is non-blocking. Users shall notice that this receive size shall not bigger than 64 bytes. As this interface is used to read flash status registers. For flash contents read, please use AHB bus read, this is much more efficiency.

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.                |
| <i>handle</i> | Pointer to qspi_edma_handle_t structure |
| <i>xfer</i>   | QSPI transfer structure.                |

#### 54.4.5 void QSPI\_TransferAbortSendEDMA ( QuadSPI\_Type \* *base*, qspi\_edma\_handle\_t \* *handle* )

This function aborts the sent data using eDMA.

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | QSPI peripheral base address.           |
| <i>handle</i> | Pointer to qspi_edma_handle_t structure |

#### 54.4.6 void QSPI\_TransferAbortReceiveEDMA ( QuadSPI\_Type \* *base*, qspi\_edma\_handle\_t \* *handle* )

This function abort receive data which using eDMA.

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | QSPI peripheral base address.           |
| <i>handle</i> | Pointer to qspi_edma_handle_t structure |

#### 54.4.7 status\_t QSPI\_TransferGetSendCountEDMA ( QuadSPI\_Type \* *base*, qspi\_edma\_handle\_t \* *handle*, size\_t \* *count* )

Parameters

|               |                                          |
|---------------|------------------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.                 |
| <i>handle</i> | Pointer to qspi_edma_handle_t structure. |

|              |             |
|--------------|-------------|
| <i>count</i> | Bytes sent. |
|--------------|-------------|

Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

#### 54.4.8 **status\_t QSPI\_TransferGetReceiveCountEDMA ( QuadSPI\_Type \* *base*, qspi\_edma\_handle\_t \* *handle*, size\_t \* *count* )**

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>base</i>   | Pointer to QuadSPI Type.                |
| <i>handle</i> | Pointer to qspi_edma_handle_t structure |
| <i>count</i>  | Bytes received.                         |

Return values

|                                     |                                                                |
|-------------------------------------|----------------------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                                |
| <i>kStatus_NoTransferInProgress</i> | There is not a non-blocking transaction currently in progress. |

## 54.5 SAI EDMA Driver

### 54.5.1 Overview

#### Data Structures

- struct [sai\\_edma\\_handle](#)  
SAI DMA transfer handle, users should not touch the content of the handle. [More...](#)

#### Typedefs

- typedef void(\* [sai\\_edma\\_callback\\_t](#))(I2S\_Type \*base, [sai\\_edma\\_handle\\_t](#) \*handle, [status\\_t](#) status, void \*userData)  
SAI eDMA transfer callback function for finish and error.
- typedef enum [\\_sai\\_edma\\_interleave](#) [sai\\_edma\\_interleave\\_t](#)  
sai interleave type

#### Enumerations

- enum [\\_sai\\_edma\\_interleave](#) {  
    [kSAI\\_EDMAInterleavePerChannelSample](#),  
    [kSAI\\_EDMAInterleavePerChannelBlock](#) }  
sai interleave type

#### Driver version

- #define [FSL\\_SAI\\_EDMA\\_DRIVER\\_VERSION](#) ([MAKE\\_VERSION](#)(2, 7, 1))  
Version 2.7.1.

#### eDMA Transactional

- void [SAI\\_TransferTxCreateHandleEDMA](#) (I2S\_Type \*base, [sai\\_edma\\_handle\\_t](#) \*handle, [sai\\_edma\\_callback\\_t](#) callback, void \*userData, [edma\\_handle\\_t](#) \*txDmaHandle)  
Initializes the SAI eDMA handle.
- void [SAI\\_TransferRxCreateHandleEDMA](#) (I2S\_Type \*base, [sai\\_edma\\_handle\\_t](#) \*handle, [sai\\_edma\\_callback\\_t](#) callback, void \*userData, [edma\\_handle\\_t](#) \*rxDmaHandle)  
Initializes the SAI Rx eDMA handle.
- void [SAI\\_TransferSetInterleaveType](#) ([sai\\_edma\\_handle\\_t](#) \*handle, [sai\\_edma\\_interleave\\_t](#) interleaveType)  
Initializes the SAI interleave type.
- void [SAI\\_TransferTxSetConfigEDMA](#) (I2S\_Type \*base, [sai\\_edma\\_handle\\_t](#) \*handle, [sai\\_transceiver\\_t](#) \*saiConfig)  
Configures the SAI Tx.

- void [SAI\\_TransferRxSetConfigEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle, sai\_transceiver\_t \*saiConfig)  
*Configures the SAI Rx.*
- status\_t [SAI\\_TransferSendEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle, sai\_transfer\_t \*xfer)  
*Performs a non-blocking SAI transfer using DMA.*
- status\_t [SAI\\_TransferReceiveEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle, sai\_transfer\_t \*xfer)  
*Performs a non-blocking SAI receive using eDMA.*
- status\_t [SAI\\_TransferSendLoopEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle, sai\_transfer\_t \*xfer, uint32\_t loopTransferCount)  
*Performs a non-blocking SAI loop transfer using eDMA.*
- status\_t [SAI\\_TransferReceiveLoopEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle, sai\_transfer\_t \*xfer, uint32\_t loopTransferCount)  
*Performs a non-blocking SAI loop transfer using eDMA.*
- void [SAI\\_TransferTerminateSendEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle)  
*Terminate all SAI send.*
- void [SAI\\_TransferTerminateReceiveEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle)  
*Terminate all SAI receive.*
- void [SAI\\_TransferAbortSendEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle)  
*Aborts a SAI transfer using eDMA.*
- void [SAI\\_TransferAbortReceiveEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle)  
*Aborts a SAI receive using eDMA.*
- status\_t [SAI\\_TransferGetSendCountEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle, size\_t \*count)  
*Gets byte count sent by SAI.*
- status\_t [SAI\\_TransferGetReceiveCountEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle, size\_t \*count)  
*Gets byte count received by SAI.*
- uint32\_t [SAI\\_TransferGetValidTransferSlotsEDMA](#) (I2S\_Type \*base, sai\_edma\_handle\_t \*handle)  
*Gets valid transfer slot.*

## 54.5.2 Data Structure Documentation

### 54.5.2.1 struct sai\_edma\_handle

#### Data Fields

- edma\_handle\_t \* [dmaHandle](#)  
*DMA handler for SAI send.*
- uint8\_t [nbytes](#)  
*eDMA minor byte transfer count initially configured.*
- uint8\_t [bytesPerFrame](#)  
*Bytes in a frame.*
- uint8\_t [channelMask](#)  
*Enabled channel mask value, reference `_sai_channel_mask`.*
- uint8\_t [channelNums](#)  
*total enabled channel nums*

- `uint8_t channel`  
*Which data channel.*
- `uint8_t count`  
*The transfer data count in a DMA request.*
- `uint32_t state`  
*Internal state for SAI eDMA transfer.*
- `sai_edma_callback_t callback`  
*Callback for users while transfer finish or error occurs.*
- `void * userData`  
*User callback parameter.*
- `uint8_t tcd [(SAI_XFER_QUEUE_SIZE+1U)*sizeof(edma_tcd_t)]`  
*TCD pool for eDMA transfer.*
- `sai_transfer_t saiQueue [SAI_XFER_QUEUE_SIZE]`  
*Transfer queue storing queued transfer.*
- `size_t transferSize [SAI_XFER_QUEUE_SIZE]`  
*Data bytes need to transfer.*
- `sai_edma_interleave_t interleaveType`  
*Transfer interleave type.*
- `volatile uint8_t queueUser`  
*Index for user to queue transfer.*
- `volatile uint8_t queueDriver`  
*Index for driver to get the transfer data and size.*

## Field Documentation

- (1) `uint8_t sai_edma_handle::nbytes`
- (2) `uint8_t sai_edma_handle::tcd[(SAI_XFER_QUEUE_SIZE+1U)*sizeof(edma_tcd_t)]`
- (3) `sai_transfer_t sai_edma_handle::saiQueue[SAI_XFER_QUEUE_SIZE]`
- (4) `volatile uint8_t sai_edma_handle::queueUser`

## 54.5.3 Enumeration Type Documentation

### 54.5.3.1 `enum _sai_edma_interleave`

#### SAI data interleave per channel sample

| LEFT CHANNEL | RIGHT CHANNEL | LEFT CHANNEL | RIGHT CHANNEL | LEFT CHANNEL | RIGHT CHANNEL | .... |

#### SAI data interleave per channel block

| LEFT CHANNEL | LEFT CHANNEL | LEFT CHANNEL | ... | **RIGHT CHANNEL | RIGHT CHANNEL | RIGHT CHANNEL | ...** |

## 54.5.4 Function Documentation

**54.5.4.1 void SAI\_TransferTxCreateHandleEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_edma\_callback\_t *callback*, void \* *userData*, edma\_handle\_t \* *txDmaHandle* )**

This function initializes the SAI master DMA handle, which can be used for other SAI master transactional APIs. Usually, for a specified SAI instance, call this API once to get the initialized handle.



## Parameters

|                    |                                                                      |
|--------------------|----------------------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                                    |
| <i>handle</i>      | SAI eDMA handle pointer.                                             |
| <i>callback</i>    | Pointer to user callback function.                                   |
| <i>userData</i>    | User parameter passed to the callback function.                      |
| <i>txDmaHandle</i> | eDMA handle pointer, this handle shall be static allocated by users. |

**54.5.4.2 void SAI\_TransferRxCreateHandleEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_edma\_callback\_t *callback*, void \* *userData*, edma\_handle\_t \* *rxDmaHandle* )**

This function initializes the SAI slave DMA handle, which can be used for other SAI master transactional APIs. Usually, for a specified SAI instance, call this API once to get the initialized handle.

## Parameters

|                    |                                                                      |
|--------------------|----------------------------------------------------------------------|
| <i>base</i>        | SAI base pointer.                                                    |
| <i>handle</i>      | SAI eDMA handle pointer.                                             |
| <i>callback</i>    | Pointer to user callback function.                                   |
| <i>userData</i>    | User parameter passed to the callback function.                      |
| <i>rxDmaHandle</i> | eDMA handle pointer, this handle shall be static allocated by users. |

**54.5.4.3 void SAI\_TransferSetInterleaveType ( sai\_edma\_handle\_t \* *handle*, sai\_edma\_interleave\_t *interleaveType* )**

This function initializes the SAI DMA handle member *interleaveType*, it shall be called only when application would like to use type `kSAI_EDMAInterleavePerChannelBlock`, since the default *interleaveType* is `kSAI_EDMAInterleavePerChannelSample` always

## Parameters

|                       |                                |
|-----------------------|--------------------------------|
| <i>handle</i>         | SAI eDMA handle pointer.       |
| <i>interleaveType</i> | Multi channel interleave type. |

**54.5.4.4 void SAI\_TransferTxSetConfigEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_transceiver\_t \* *saiConfig* )**

## Enumerator

## Note

***kSAI\_EDMAInterleavePerChannelSample*** ***kSAI\_EDMAInterleavePerChannelBlock*** SAI eDMA supports data transfer in a multiple SAI channels if the FIFO Combine feature is supported. To activate the multi-channel transfer enable SAI channels by filling the channelMask of sai\_transceiver\_t with the corresponding values of \_sai\_channel\_mask enum, enable the FIFO Combine mode by assigning kSAI\_FifoCombineModeEnabledOnWrite to the fifoCombine member of sai\_fifo\_combine\_t which is a member of sai\_transceiver\_t. This is an example of multi-channel data transfer configuration step.

```
* sai_transceiver_t config;
* SAI_GetClassicI2SConfig(&config, kSAI_WordWidth16bits,
* kSAI_Stereo, kSAI_Channel0Mask|kSAI_Channel1Mask);
* config.fifo.fifoCombine = kSAI_FifoCombineModeEnabledOnWrite
* ;
* SAI_TransferTxSetConfigEDMA(I2S0, &edmaHandle, &config);
*
```

## Parameters

|                  |                          |
|------------------|--------------------------|
| <i>base</i>      | SAI base pointer.        |
| <i>handle</i>    | SAI eDMA handle pointer. |
| <i>saiConfig</i> | sai configurations.      |

**54.5.4.5 void SAI\_TransferRxSetConfigEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_transceiver\_t \* *saiConfig* )**

## Note

SAI eDMA supports data transfer in a multiple SAI channels if the FIFO Combine feature is supported. To activate the multi-channel transfer enable SAI channels by filling the channelMask of sai\_transceiver\_t with the corresponding values of \_sai\_channel\_mask enum, enable the FIFO Combine mode by assigning kSAI\_FifoCombineModeEnabledOnRead to the fifoCombine member of sai\_fifo\_combine\_t which is a member of sai\_transceiver\_t. This is an example of multi-channel data transfer configuration step.

```
* sai_transceiver_t config;
* SAI_GetClassicI2SConfig(&config, kSAI_WordWidth16bits,
* kSAI_Stereo, kSAI_Channel0Mask|kSAI_Channel1Mask);
* config.fifo.fifoCombine = kSAI_FifoCombineModeEnabledOnRead
* ;
* SAI_TransferRxSetConfigEDMA(I2S0, &edmaHandle, &config);
*
```

## Parameters

|                  |                          |
|------------------|--------------------------|
| <i>base</i>      | SAI base pointer.        |
| <i>handle</i>    | SAI eDMA handle pointer. |
| <i>saiConfig</i> | sai configurations.      |

#### 54.5.4.6 status\_t SAI\_TransferSendEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_transfer\_t \* *xfer* )

## Note

This interface returns immediately after the transfer initiates. Call SAI\_GetTransferStatus to poll the transfer status and check whether the SAI transfer is finished.

This function support multi channel transfer,

1. for the sai IP support fifo combine mode, application should enable the fifo combine mode, no limitation on channel numbers
2. for the sai IP not support fifo combine mode, sai edma provide another solution which using EDMA modulo feature, but support 2 or 4 channels only.

## Parameters

|               |                                        |
|---------------|----------------------------------------|
| <i>base</i>   | SAI base pointer.                      |
| <i>handle</i> | SAI eDMA handle pointer.               |
| <i>xfer</i>   | Pointer to the DMA transfer structure. |

## Return values

|                                |                                     |
|--------------------------------|-------------------------------------|
| <i>kStatus_Success</i>         | Start a SAI eDMA send successfully. |
| <i>kStatus_InvalidArgument</i> | The input argument is invalid.      |
| <i>kStatus_TxBusy</i>          | SAI is busy sending data.           |

#### 54.5.4.7 status\_t SAI\_TransferReceiveEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_transfer\_t \* *xfer* )

## Note

This interface returns immediately after the transfer initiates. Call the SAI\_GetReceiveRemaining-Bytes to poll the transfer status and check whether the SAI transfer is finished.

This function support multi channel transfer,

1. for the sai IP support fifo combine mode, application should enable the fifo combine mode, no limitation on channel numbers
2. for the sai IP not support fifo combine mode, sai edma provide another solution which using EDMA modulo feature, but support 2 or 4 channels only.

## Parameters

|               |                                    |
|---------------|------------------------------------|
| <i>base</i>   | SAI base pointer                   |
| <i>handle</i> | SAI eDMA handle pointer.           |
| <i>xfer</i>   | Pointer to DMA transfer structure. |

## Return values

|                                |                                        |
|--------------------------------|----------------------------------------|
| <i>kStatus_Success</i>         | Start a SAI eDMA receive successfully. |
| <i>kStatus_InvalidArgument</i> | The input argument is invalid.         |
| <i>kStatus_RxBusy</i>          | SAI is busy receiving data.            |

#### 54.5.4.8 **status\_t SAI\_TransferSendLoopEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_transfer\_t \* *xfer*, uint32\_t *loopTransferCount* )**

## Note

This function support loop transfer only, such as A->B->...->A, application must be aware of that the more counts of the loop transfer, then more tcd memory required, as the function use the tcd pool in sai\_edma\_handle\_t, so application could redefine the SAI\_XFER\_QUEUE\_SIZE to determine the proper TCD pool size. This function support one sai channel only.

Once the loop transfer start, application can use function SAI\_TransferAbortSendEDMA to stop the loop transfer.

## Parameters

|               |                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------|
| <i>base</i>   | SAI base pointer.                                                                                           |
| <i>handle</i> | SAI eDMA handle pointer.                                                                                    |
| <i>xfer</i>   | Pointer to the DMA transfer structure, should be a array with elements counts $\geq 1$ (loopTransferCount). |

|                           |                           |
|---------------------------|---------------------------|
| <i>loopTransfer-Count</i> | the counts of xfer array. |
|---------------------------|---------------------------|

Return values

|                                |                                     |
|--------------------------------|-------------------------------------|
| <i>kStatus_Success</i>         | Start a SAI eDMA send successfully. |
| <i>kStatus_InvalidArgument</i> | The input argument is invalid.      |

#### 54.5.4.9 **status\_t SAI\_TransferReceiveLoopEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, sai\_transfer\_t \* *xfer*, uint32\_t *loopTransferCount* )**

Note

This function support loop transfer only,such as A->B->...->A, application must be aware of that the more counts of the loop transfer, then more tcd memory required, as the function use the tcd pool in sai\_edma\_handle\_t, so application could redefine the SAI\_XFER\_QUEUE\_SIZE to determine the proper TCD pool size. This function support one sai channel only.

Once the loop transfer start, application can use function SAI\_TransferAbortReceiveEDMA to stop the loop transfer.

Parameters

|                           |                                                                                                       |
|---------------------------|-------------------------------------------------------------------------------------------------------|
| <i>base</i>               | SAI base pointer.                                                                                     |
| <i>handle</i>             | SAI eDMA handle pointer.                                                                              |
| <i>xfer</i>               | Pointer to the DMA transfer structure, should be a array with elements counts >=1(loopTransferCount). |
| <i>loopTransfer-Count</i> | the counts of xfer array.                                                                             |

Return values

|                                |                                        |
|--------------------------------|----------------------------------------|
| <i>kStatus_Success</i>         | Start a SAI eDMA receive successfully. |
| <i>kStatus_InvalidArgument</i> | The input argument is invalid.         |

#### 54.5.4.10 **void SAI\_TransferTerminateSendEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle* )**

This function will clear all transfer slots buffered in the sai queue. If users only want to abort the current transfer slot, please call SAI\_TransferAbortSendEDMA.

## Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>handle</i> | SAI eDMA handle pointer. |

#### 54.5.4.11 void SAI\_TransferTerminateReceiveEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle* )

This function will clear all transfer slots buffered in the sai queue. If users only want to abort the current transfer slot, please call SAI\_TransferAbortReceiveEDMA.

## Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>handle</i> | SAI eDMA handle pointer. |

#### 54.5.4.12 void SAI\_TransferAbortSendEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle* )

This function only aborts the current transfer slots, the other transfer slots' information still kept in the handler. If users want to terminate all transfer slots, just call SAI\_TransferTerminateSendEDMA.

## Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>handle</i> | SAI eDMA handle pointer. |

#### 54.5.4.13 void SAI\_TransferAbortReceiveEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle* )

This function only aborts the current transfer slots, the other transfer slots' information still kept in the handler. If users want to terminate all transfer slots, just call SAI\_TransferTerminateReceiveEDMA.

## Parameters

|             |                  |
|-------------|------------------|
| <i>base</i> | SAI base pointer |
|-------------|------------------|

|               |                          |
|---------------|--------------------------|
| <i>handle</i> | SAI eDMA handle pointer. |
|---------------|--------------------------|

#### 54.5.4.14 **status\_t SAI\_TransferGetSendCountEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, size\_t \* *count* )**

Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer.        |
| <i>handle</i> | SAI eDMA handle pointer. |
| <i>count</i>  | Bytes count sent by SAI. |

Return values

|                                     |                                                   |
|-------------------------------------|---------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                   |
| <i>kStatus_NoTransferInProgress</i> | There is no non-blocking transaction in progress. |

#### 54.5.4.15 **status\_t SAI\_TransferGetReceiveCountEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle*, size\_t \* *count* )**

Parameters

|               |                              |
|---------------|------------------------------|
| <i>base</i>   | SAI base pointer             |
| <i>handle</i> | SAI eDMA handle pointer.     |
| <i>count</i>  | Bytes count received by SAI. |

Return values

|                                     |                                                   |
|-------------------------------------|---------------------------------------------------|
| <i>kStatus_Success</i>              | Succeed get the transfer count.                   |
| <i>kStatus_NoTransferInProgress</i> | There is no non-blocking transaction in progress. |

#### 54.5.4.16 **uint32\_t SAI\_TransferGetValidTransferSlotsEDMA ( I2S\_Type \* *base*, sai\_edma\_handle\_t \* *handle* )**

This function can be used to query the valid transfer request slot that the application can submit. It should be called in the critical section, that means the application could call it in the corresponding callback function or disable IRQ before calling it in the application, otherwise, the returned value may not correct.

Parameters

|               |                          |
|---------------|--------------------------|
| <i>base</i>   | SAI base pointer         |
| <i>handle</i> | SAI eDMA handle pointer. |

Return values

|              |                                     |
|--------------|-------------------------------------|
| <i>valid</i> | slot count that application submit. |
|--------------|-------------------------------------|



## 54.6 Da7212

### 54.6.1 Overview

#### Data Structures

- struct [\\_da7212\\_pll\\_config](#)  
*da7212 pll configuration [More...](#)*
- struct [\\_da7212\\_audio\\_format](#)  
*da7212 audio format [More...](#)*
- struct [da7212\\_config](#)  
*DA7212 configure structure. [More...](#)*
- struct [\\_da7212\\_handle](#)  
*da7212 codec handler [More...](#)*

#### Macros

- #define [DA7212\\_I2C\\_HANDLER\\_SIZE](#) [CODEC\\_I2C\\_MASTER\\_HANDLER\\_SIZE](#)  
*da7212 handle size*
- #define [DA7212\\_ADDRESS](#) (0x1A)  
*DA7212 I2C address.*
- #define [DA7212\\_HEADPHONE\\_MAX\\_VOLUME\\_VALUE](#) 0x3FU  
*da7212 volume setting range*

#### Typedefs

- typedef enum [\\_da7212\\_Input](#) [da7212\\_Input\\_t](#)  
*DA7212 input source select.*
- typedef enum [\\_da7212\\_Output](#) [da7212\\_Output\\_t](#)  
*DA7212 output device select.*
- typedef enum [\\_da7212\\_dac\\_source](#) [da7212\\_dac\\_source\\_t](#)  
*DA7212 functionality.*
- typedef enum [\\_da7212\\_volume](#) [da7212\\_volume\\_t](#)  
*DA7212 volume.*
- typedef enum [\\_da7212\\_protocol](#) [da7212\\_protocol\\_t](#)  
*The audio data transfer protocol choice.*
- typedef enum [\\_da7212\\_sys\\_clk\\_source](#) [da7212\\_sys\\_clk\\_source\\_t](#)  
*da7212 system clock source*
- typedef enum [\\_da7212\\_pll\\_clk\\_source](#) [da7212\\_pll\\_clk\\_source\\_t](#)  
*DA7212 pll clock source.*
- typedef enum [\\_da7212\\_pll\\_out\\_clk](#) [da7212\\_pll\\_out\\_clk\\_t](#)  
*DA7212 output clock frequency.*
- typedef enum [\\_da7212\\_master\\_bits](#) [da7212\\_master\\_bits\\_t](#)  
*master mode bits per frame*
- typedef struct [\\_da7212\\_pll\\_config](#) [da7212\\_pll\\_config\\_t](#)  
*da7212 pll configuration*
- typedef struct [\\_da7212\\_audio\\_format](#) [da7212\\_audio\\_format\\_t](#)  
*da7212 audio format*

- typedef struct [da7212\\_config](#) [da7212\\_config\\_t](#)  
*DA7212 configure structure.*
- typedef struct [\\_da7212\\_handle](#) [da7212\\_handle\\_t](#)  
*da7212 codec handler*

## Enumerations

- enum [\\_da7212\\_Input](#) {  
  [kDA7212\\_Input\\_AUX](#) = 0x0,  
  [kDA7212\\_Input\\_MIC1\\_Dig](#),  
  [kDA7212\\_Input\\_MIC1\\_An](#),  
  [kDA7212\\_Input\\_MIC2](#) }  
*DA7212 input source select.*
- enum [\\_da7212\\_play\\_channel](#) {  
  [kDA7212\\_HeadphoneLeft](#) = 1U,  
  [kDA7212\\_HeadphoneRight](#) = 2U,  
  [kDA7212\\_Speaker](#) = 4U }  
*da7212 play channel*
- enum [\\_da7212\\_Output](#) {  
  [kDA7212\\_Output\\_HP](#) = 0x0,  
  [kDA7212\\_Output\\_SP](#) }  
*DA7212 output device select.*
- enum [\\_da7212\\_module](#) {  
  [kDA7212\\_ModuleADC](#),  
  [kDA7212\\_ModuleDAC](#),  
  [kDA7212\\_ModuleHeadphone](#),  
  [kDA7212\\_ModuleSpeaker](#) }  
*DA7212 module.*
- enum [\\_da7212\\_dac\\_source](#) {  
  [kDA7212\\_DACSourceADC](#) = 0x0U,  
  [kDA7212\\_DACSourceInputStream](#) = 0x3U }  
*DA7212 functionality.*
- enum [\\_da7212\\_volume](#) {

```

kDA7212_DACGainMute = 0x7,
kDA7212_DACGainM72DB = 0x17,
kDA7212_DACGainM60DB = 0x1F,
kDA7212_DACGainM54DB = 0x27,
kDA7212_DACGainM48DB = 0x2F,
kDA7212_DACGainM42DB = 0x37,
kDA7212_DACGainM36DB = 0x3F,
kDA7212_DACGainM30DB = 0x47,
kDA7212_DACGainM24DB = 0x4F,
kDA7212_DACGainM18DB = 0x57,
kDA7212_DACGainM12DB = 0x5F,
kDA7212_DACGainM6DB = 0x67,
kDA7212_DACGain0DB = 0x6F,
kDA7212_DACGain6DB = 0x77,
kDA7212_DACGain12DB = 0x7F }

```

*DA7212 volume.*

- enum `_da7212_protocol` {  
`kDA7212_BusI2S` = 0x0,  
`kDA7212_BusLeftJustified`,  
`kDA7212_BusRightJustified`,  
`kDA7212_BusDSPMode` }

*The audio data transfer protocol choice.*

- enum `_da7212_sys_clk_source` {  
`kDA7212_SysClkSourceMCLK` = 0U,  
`kDA7212_SysClkSourcePLL` = 1U << 14 }

*da7212 system clock source*

- enum `_da7212_pll_clk_source` { `kDA7212_PLLClkSourceMCLK` = 0U }

*DA7212 pll clock source.*

- enum `_da7212_pll_out_clk` {  
`kDA7212_PLLOutputClk11289600` = 11289600U,  
`kDA7212_PLLOutputClk12288000` = 12288000U }

*DA7212 output clock frequency.*

- enum `_da7212_master_bits` {  
`kDA7212_MasterBits32PerFrame` = 0U,  
`kDA7212_MasterBits64PerFrame` = 1U,  
`kDA7212_MasterBits128PerFrame` = 2U,  
`kDA7212_MasterBits256PerFrame` = 3U }

*master mode bits per frame*

## Functions

- `status_t DA7212_Init (da7212_handle_t *handle, da7212_config_t *codecConfig)`  
*DA7212 initialize function.*
- `status_t DA7212_ConfigAudioFormat (da7212_handle_t *handle, uint32_t masterClock_Hz, uint32_t sampleRate_Hz, uint32_t dataBits)`  
*Set DA7212 audio format.*

- `status_t DA7212_SetPLLConfig (da7212_handle_t *handle, da7212_pll_config_t *config)`  
*DA7212 set PLL configuration This function will enable the GPIO1 FLL clock output function, so user can see the generated fll output clock frequency from WM8904 GPIO1.*
- `void DA7212_ChangeHPVolume (da7212_handle_t *handle, da7212_volume_t volume)`  
*Set DA7212 playback volume.*
- `void DA7212_Mute (da7212_handle_t *handle, bool isMuted)`  
*Mute or unmute DA7212.*
- `void DA7212_ChangeInput (da7212_handle_t *handle, da7212_input_t DA7212_Input)`  
*Set the input data source of DA7212.*
- `void DA7212_ChangeOutput (da7212_handle_t *handle, da7212_output_t DA7212_Output)`  
*Set the output device of DA7212.*
- `status_t DA7212_SetChannelVolume (da7212_handle_t *handle, uint32_t channel, uint32_t volume)`  
*Set module volume.*
- `status_t DA7212_SetChannelMute (da7212_handle_t *handle, uint32_t channel, bool isMute)`  
*Set module mute.*
- `status_t DA7212_SetProtocol (da7212_handle_t *handle, da7212_protocol_t protocol)`  
*Set protocol for DA7212.*
- `status_t DA7212_SetMasterModeBits (da7212_handle_t *handle, uint32_t bitWidth)`  
*Set master mode bits per frame for DA7212.*
- `status_t DA7212_WriteRegister (da7212_handle_t *handle, uint8_t u8Register, uint8_t u8RegisterData)`  
*Write a register for DA7212.*
- `status_t DA7212_ReadRegister (da7212_handle_t *handle, uint8_t u8Register, uint8_t *pu8RegisterData)`  
*Get a register value of DA7212.*
- `status_t DA7212_Deinit (da7212_handle_t *handle)`  
*Deinit DA7212.*

## Driver version

- `#define FSL_DA7212_DRIVER_VERSION (MAKE_VERSION(2, 3, 1))`  
*CLOCK driver version 2.3.0.*

## 54.6.2 Data Structure Documentation

### 54.6.2.1 struct \_da7212\_pll\_config

#### Data Fields

- `da7212_pll_clk_source_t source`  
*pll reference clock source*
- `uint32_t refClock_HZ`  
*pll reference clock frequency*
- `da7212_pll_out_clk_t outputClock_HZ`  
*pll output clock frequency*

### 54.6.2.2 struct \_da7212\_audio\_format

#### Data Fields

- uint32\_t [mclk\\_HZ](#)  
*master clock frequency*
- uint32\_t [sampleRate](#)  
*sample rate*
- uint32\_t [bitWidth](#)  
*bit width*
- bool [isBclkInvert](#)  
*bit clock interval*

### 54.6.2.3 struct da7212\_config

#### Data Fields

- bool [isMaster](#)  
*If DA7212 is master, true means master, false means slave.*
- [da7212\\_protocol\\_t](#) [protocol](#)  
*Audio bus format, can be I2S, LJ, RJ or DSP mode.*
- [da7212\\_dac\\_source\\_t](#) [dacSource](#)  
*DA7212 data source.*
- [da7212\\_audio\\_format\\_t](#) [format](#)  
*audio format*
- uint8\_t [slaveAddress](#)  
*device address*
- [codec\\_i2c\\_config\\_t](#) [i2cConfig](#)  
*i2c configuration*
- [da7212\\_sys\\_clk\\_source\\_t](#) [sysClkSource](#)  
*system clock source*
- [da7212\\_pll\\_config\\_t](#) \* [pll](#)  
*pll configuration*
- [da7212\\_input\\_t](#) [inputSource](#)  
*AD212 input source.*

#### Field Documentation

- (1) `bool da7212_config::isMaster`
- (2) `da7212_protocol_t da7212_config::protocol`
- (3) `da7212_dac_source_t da7212_config::dacSource`

### 54.6.2.4 struct \_da7212\_handle

#### Data Fields

- [da7212\\_config\\_t](#) \* [config](#)  
*da7212 config pointer*

- uint8\_t *i2cHandle* [DA7212\_I2C\_HANDLER\_SIZE]  
i2c handle

### 54.6.3 Macro Definition Documentation

#### 54.6.3.1 #define FSL\_DA7212\_DRIVER\_VERSION (MAKE\_VERSION(2, 3, 1))

### 54.6.4 Enumeration Type Documentation

#### 54.6.4.1 enum \_da7212\_Input

Enumerator

*kDA7212\_Input\_AUX* Input from AUX.  
*kDA7212\_Input\_MIC1\_Dig* Input from MIC1 Digital.  
*kDA7212\_Input\_MIC1\_An* Input from Mic1 Analog.  
*kDA7212\_Input\_MIC2* Input from MIC2.

#### 54.6.4.2 enum \_da7212\_play\_channel

Enumerator

*kDA7212\_HeadphoneLeft* headphone left  
*kDA7212\_HeadphoneRight* headphone right  
*kDA7212\_Speaker* speaker channel

#### 54.6.4.3 enum \_da7212\_Output

Enumerator

*kDA7212\_Output\_HP* Output to headphone.  
*kDA7212\_Output\_SP* Output to speaker.

#### 54.6.4.4 enum \_da7212\_module

Enumerator

*kDA7212\_ModuleADC* module ADC  
*kDA7212\_ModuleDAC* module DAC  
*kDA7212\_ModuleHeadphone* module headphone  
*kDA7212\_ModuleSpeaker* module speaker

#### 54.6.4.5 enum \_da7212\_dac\_source

Enumerator

***kDA7212\_DACSourceADC*** DAC source from ADC.  
***kDA7212\_DACSourceInputStream*** DAC source from.

#### 54.6.4.6 enum \_da7212\_volume

Enumerator

***kDA7212\_DACGainMute*** Mute DAC.  
***kDA7212\_DACGainM72DB*** DAC volume -72db.  
***kDA7212\_DACGainM60DB*** DAC volume -60db.  
***kDA7212\_DACGainM54DB*** DAC volume -54db.  
***kDA7212\_DACGainM48DB*** DAC volume -48db.  
***kDA7212\_DACGainM42DB*** DAC volume -42db.  
***kDA7212\_DACGainM36DB*** DAC volume -36db.  
***kDA7212\_DACGainM30DB*** DAC volume -30db.  
***kDA7212\_DACGainM24DB*** DAC volume -24db.  
***kDA7212\_DACGainM18DB*** DAC volume -18db.  
***kDA7212\_DACGainM12DB*** DAC volume -12db.  
***kDA7212\_DACGainM6DB*** DAC volume -6db.  
***kDA7212\_DACGain0DB*** DAC volume +0db.  
***kDA7212\_DACGain6DB*** DAC volume +6db.  
***kDA7212\_DACGain12DB*** DAC volume +12db.

#### 54.6.4.7 enum \_da7212\_protocol

Enumerator

***kDA7212\_BusI2S*** I2S Type.  
***kDA7212\_BusLeftJustified*** Left justified.  
***kDA7212\_BusRightJustified*** Right Justified.  
***kDA7212\_BusDSPMode*** DSP mode.

#### 54.6.4.8 enum \_da7212\_sys\_clk\_source

Enumerator

***kDA7212\_SysClkSourceMCLK*** da7212 system clock source from MCLK  
***kDA7212\_SysClkSourcePLL*** da7212 system clock source from PLL

#### 54.6.4.9 enum \_da7212\_pll\_clk\_source

Enumerator

***kDA7212\_PLLClkSourceMCLK*** DA7212 PLL clock source from MCLK.

#### 54.6.4.10 enum \_da7212\_pll\_out\_clk

Enumerator

***kDA7212\_PLLOutputClk11289600*** output 112896000U

***kDA7212\_PLLOutputClk12288000*** output 12288000U

#### 54.6.4.11 enum \_da7212\_master\_bits

Enumerator

***kDA7212\_MasterBits32PerFrame*** master mode bits32 per frame

***kDA7212\_MasterBits64PerFrame*** master mode bits64 per frame

***kDA7212\_MasterBits128PerFrame*** master mode bits128 per frame

***kDA7212\_MasterBits256PerFrame*** master mode bits256 per frame

### 54.6.5 Function Documentation

#### 54.6.5.1 status\_t DA7212\_Init ( da7212\_handle\_t \* *handle*, da7212\_config\_t \* *codecConfig* )

Parameters

|                    |                                                                                                                                                                                                                                                                                 |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>      | DA7212 handle pointer.                                                                                                                                                                                                                                                          |
| <i>codecConfig</i> | <p>Codec configure structure. This parameter can be NULL, if NULL, set as default settings. The default setting:</p> <pre> * sgtl_init_t codec_config * codec_config.route = kDA7212_RoutePlayback * codec_config.bus = kDA7212_BusI2S * codec_config.isMaster = false * </pre> |

#### 54.6.5.2 status\_t DA7212\_ConfigAudioFormat ( da7212\_handle\_t \* *handle*, uint32\_t *masterClock\_Hz*, uint32\_t *sampleRate\_Hz*, uint32\_t *dataBits* )



## Parameters

|                       |                                                                                                                                                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>         | DA7212 handle pointer.                                                                                                                                                                                                                  |
| <i>masterClock_Hz</i> | Master clock frequency in Hz. If DA7212 is slave, use the frequency of master, if DA7212 as master, it should be 1228000 while sample rate frequency is 8k/12K/16-K/24K/32K/48K/96K, 11289600 while sample rate is 11.025K/22.05K/44.1K |
| <i>sampleRate_Hz</i>  | Sample rate frequency in Hz.                                                                                                                                                                                                            |
| <i>dataBits</i>       | How many bits in a word of a audio frame, DA7212 only supports 16/20/24/32 bits.                                                                                                                                                        |

#### 54.6.5.3 **status\_t DA7212\_SetPLLConfig ( da7212\_handle\_t \* *handle*, da7212\_pll\_config\_t \* *config* )**

## Parameters

|               |                            |
|---------------|----------------------------|
| <i>handle</i> | DA7212 handler pointer.    |
| <i>config</i> | PLL configuration pointer. |

#### 54.6.5.4 **void DA7212\_ChangeHPVolume ( da7212\_handle\_t \* *handle*, da7212\_volume\_t *volume* )**

## Parameters

|               |                         |
|---------------|-------------------------|
| <i>handle</i> | DA7212 handle pointer.  |
| <i>volume</i> | The volume of playback. |

#### 54.6.5.5 **void DA7212\_Mute ( da7212\_handle\_t \* *handle*, bool *isMuted* )**

## Parameters

|                |                                      |
|----------------|--------------------------------------|
| <i>handle</i>  | DA7212 handle pointer.               |
| <i>isMuted</i> | True means mute, false means unmute. |

#### 54.6.5.6 **void DA7212\_ChangeInput ( da7212\_handle\_t \* *handle*, da7212\_Input\_t *DA7212\_Input* )**

## Parameters

|                     |                        |
|---------------------|------------------------|
| <i>handle</i>       | DA7212 handle pointer. |
| <i>DA7212_Input</i> | Input data source.     |

#### 54.6.5.7 void DA7212\_ChangeOutput ( da7212\_handle\_t \* *handle*, da7212\_Output\_t *DA7212\_Output* )

## Parameters

|                      |                          |
|----------------------|--------------------------|
| <i>handle</i>        | DA7212 handle pointer.   |
| <i>DA7212_Output</i> | Output device of DA7212. |

#### 54.6.5.8 status\_t DA7212\_SetChannelVolume ( da7212\_handle\_t \* *handle*, uint32\_t *channel*, uint32\_t *volume* )

## Parameters

|                |                                                    |
|----------------|----------------------------------------------------|
| <i>handle</i>  | DA7212 handle pointer.                             |
| <i>channel</i> | shoule be a value of _da7212_channel.              |
| <i>volume</i>  | volume range 0 - 0x3F mapped to range -57dB - 6dB. |

#### 54.6.5.9 status\_t DA7212\_SetChannelMute ( da7212\_handle\_t \* *handle*, uint32\_t *channel*, bool *isMute* )

## Parameters

|                |                                       |
|----------------|---------------------------------------|
| <i>handle</i>  | DA7212 handle pointer.                |
| <i>channel</i> | shoule be a value of _da7212_channel. |
| <i>isMute</i>  | true is mute, false is unmute.        |

#### 54.6.5.10 status\_t DA7212\_SetProtocol ( da7212\_handle\_t \* *handle*, da7212\_protocol\_t *protocol* )

## Parameters

|                 |                        |
|-----------------|------------------------|
| <i>handle</i>   | DA7212 handle pointer. |
| <i>protocol</i> | da7212_protocol_t.     |

#### 54.6.5.11 status\_t DA7212\_SetMasterModeBits ( da7212\_handle\_t \* *handle*, uint32\_t *bitWidth* )

## Parameters

|                 |                        |
|-----------------|------------------------|
| <i>handle</i>   | DA7212 handle pointer. |
| <i>bitWidth</i> | audio data bitwidth.   |

#### 54.6.5.12 status\_t DA7212\_WriteRegister ( da7212\_handle\_t \* *handle*, uint8\_t *u8Register*, uint8\_t *u8RegisterData* )

## Parameters

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>handle</i>         | DA7212 handle pointer.                 |
| <i>u8Register</i>     | DA7212 register address to be written. |
| <i>u8RegisterData</i> | Data to be written into register       |

#### 54.6.5.13 status\_t DA7212\_ReadRegister ( da7212\_handle\_t \* *handle*, uint8\_t *u8Register*, uint8\_t \* *pu8RegisterData* )

## Parameters

|                        |                                                |
|------------------------|------------------------------------------------|
| <i>handle</i>          | DA7212 handle pointer.                         |
| <i>u8Register</i>      | DA7212 register address to be read.            |
| <i>pu8RegisterData</i> | Pointer where the read out value to be stored. |

#### 54.6.5.14 status\_t DA7212\_Deinit ( da7212\_handle\_t \* *handle* )

## Parameters

|               |                        |
|---------------|------------------------|
| <i>handle</i> | DA7212 handle pointer. |
|---------------|------------------------|

## 54.6.6 Da7212\_adapter

### 54.6.6.1 Overview

#### Macros

- #define [HAL\\_CODEC\\_DA7212\\_HANDLER\\_SIZE](#) ([DA7212\\_I2C\\_HANDLER\\_SIZE](#) + 4)  
*codec handler size*

#### Functions

- [status\\_t HAL\\_CODEC\\_DA7212\\_Init](#) (void \*handle, void \*config)  
*Codec initialization.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_Deinit](#) (void \*handle)  
*Codec de-initialization.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_SetFormat](#) (void \*handle, uint32\_t mclk, uint32\_t sampleRate, uint32\_t bitWidth)  
*set audio data format.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_SetVolume](#) (void \*handle, uint32\_t playChannel, uint32\_t volume)  
*set audio codec module volume.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_SetMute](#) (void \*handle, uint32\_t playChannel, bool isMute)  
*set audio codec module mute.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_SetPower](#) (void \*handle, uint32\_t module, bool powerOn)  
*set audio codec module power.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_SetRecord](#) (void \*handle, uint32\_t recordSource)  
*codec set record source.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_SetRecordChannel](#) (void \*handle, uint32\_t leftRecordChannel, uint32\_t rightRecordChannel)  
*codec set record channel.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_SetPlay](#) (void \*handle, uint32\_t playSource)  
*codec set play source.*
- [status\\_t HAL\\_CODEC\\_DA7212\\_ModuleControl](#) (void \*handle, uint32\_t cmd, uint32\_t data)  
*codec module control.*
- static [status\\_t HAL\\_CODEC\\_Init](#) (void \*handle, void \*config)  
*Codec initialization.*
- static [status\\_t HAL\\_CODEC\\_Deinit](#) (void \*handle)  
*Codec de-initialization.*
- static [status\\_t HAL\\_CODEC\\_SetFormat](#) (void \*handle, uint32\_t mclk, uint32\_t sampleRate, uint32\_t bitWidth)  
*set audio data format.*
- static [status\\_t HAL\\_CODEC\\_SetVolume](#) (void \*handle, uint32\_t playChannel, uint32\_t volume)  
*set audio codec module volume.*
- static [status\\_t HAL\\_CODEC\\_SetMute](#) (void \*handle, uint32\_t playChannel, bool isMute)  
*set audio codec module mute.*
- static [status\\_t HAL\\_CODEC\\_SetPower](#) (void \*handle, uint32\_t module, bool powerOn)  
*set audio codec module power.*
- static [status\\_t HAL\\_CODEC\\_SetRecord](#) (void \*handle, uint32\_t recordSource)  
*codec set record source.*

- static `status_t HAL_CODEC_SetRecordChannel` (void \*handle, uint32\_t leftRecordChannel, uint32\_t rightRecordChannel)  
*codec set record channel.*
- static `status_t HAL_CODEC_SetPlay` (void \*handle, uint32\_t playSource)  
*codec set play source.*
- static `status_t HAL_CODEC_ModuleControl` (void \*handle, uint32\_t cmd, uint32\_t data)  
*codec module control.*

## 54.6.6.2 Function Documentation

### 54.6.6.2.1 `status_t HAL_CODEC_DA7212_Init ( void * handle, void * config )`

Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | codec handle.        |
| <i>config</i> | codec configuration. |

Returns

kStatus\_Success is success, else initial failed.

### 54.6.6.2.2 `status_t HAL_CODEC_DA7212_Deinit ( void * handle )`

Parameters

|               |               |
|---------------|---------------|
| <i>handle</i> | codec handle. |
|---------------|---------------|

Returns

kStatus\_Success is success, else de-initial failed.

### 54.6.6.2.3 `status_t HAL_CODEC_DA7212_SetFormat ( void * handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth )`

Parameters

|                   |                               |
|-------------------|-------------------------------|
| <i>handle</i>     | codec handle.                 |
| <i>mclk</i>       | master clock frequency in HZ. |
| <i>sampleRate</i> | sample rate in HZ.            |
| <i>bitWidth</i>   | bit width.                    |

Returns

kStatus\_Success is success, else configure failed.

**54.6.6.2.4** `status_t HAL_CODEC_DA7212_SetVolume ( void * handle, uint32_t playChannel, uint32_t volume )`

Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>volume</i>      | volume value, support 0 ~ 100, 0 is mute, 100 is the maximum volume value.        |

Returns

kStatus\_Success is success, else configure failed.

**54.6.6.2.5** `status_t HAL_CODEC_DA7212_SetMute ( void * handle, uint32_t playChannel, bool isMute )`

Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>isMute</i>      | true is mute, false is unmute.                                                    |

Returns

kStatus\_Success is success, else configure failed.

**54.6.6.2.6** `status_t HAL_CODEC_DA7212_SetPower ( void * handle, uint32_t module, bool powerOn )`

## Parameters

|                |                                        |
|----------------|----------------------------------------|
| <i>handle</i>  | codec handle.                          |
| <i>module</i>  | audio codec module.                    |
| <i>powerOn</i> | true is power on, false is power down. |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.6.2.7 status\_t HAL\_CODEC\_DA7212\_SetRecord ( void \* *handle*, uint32\_t *recordSource* )

## Parameters

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| <i>handle</i>       | codec handle.                                                                       |
| <i>recordSource</i> | audio codec record source, can be a value or combine value of _codec_record_source. |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.6.2.8 status\_t HAL\_CODEC\_DA7212\_SetRecordChannel ( void \* *handle*, uint32\_t *leftRecordChannel*, uint32\_t *rightRecordChannel* )

## Parameters

|                            |                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>              | codec handle.                                                                                                                    |
| <i>leftRecord-Channel</i>  | audio codec record channel, reference _codec_record_channel, can be a value or combine value of member in _codec_record_channel. |
| <i>rightRecord-Channel</i> | audio codec record channel, reference _codec_record_channel, can be a value combine of member in _codec_record_channel.          |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.6.2.9 status\_t HAL\_CODEC\_DA7212\_SetPlay ( void \* *handle*, uint32\_t *playSource* )



## Parameters

|                   |                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------|
| <i>handle</i>     | codec handle.                                                                                 |
| <i>playSource</i> | audio codec play source, can be a value or combine value of <code>_codec_play_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

#### 54.6.6.2.10 `status_t HAL_CODEC_DA7212_ModuleControl ( void * handle, uint32_t cmd, uint32_t data )`

This function is used for codec module control, support switch digital interface cmd, can be expand to support codec module specific feature

## Parameters

|               |                                                                                                                                                                                                                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | codec handle.                                                                                                                                                                                                                                                                                     |
| <i>cmd</i>    | module control cmd, reference <code>_codec_module_ctrl_cmd</code> .                                                                                                                                                                                                                               |
| <i>data</i>   | value to write, when cmd is <code>kCODEC_ModuleRecordSourceChannel</code> , the data should be a value combine of channel and source, please reference macro <code>CODEC_MODULE_RECORD_SOURCE_CHANNEL(source, LP, LN, RP, RN)</code> , reference codec specific driver for detail configurations. |

## Returns

`kStatus_Success` is success, else configure failed.

#### 54.6.6.2.11 `static status_t HAL_CODEC_Init ( void * handle, void * config ) [inline], [static]`

## Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | codec handle.        |
| <i>config</i> | codec configuration. |

## Returns

`kStatus_Success` is success, else initial failed.

#### 54.6.6.2.12 `static status_t HAL_CODEC_Deinit ( void * handle ) [inline], [static]`

## Parameters

|               |               |
|---------------|---------------|
| <i>handle</i> | codec handle. |
|---------------|---------------|

## Returns

kStatus\_Success is success, else de-initial failed.

**54.6.6.2.13** `static status_t HAL_CODEC_SetFormat ( void * handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth ) [inline], [static]`

## Parameters

|                   |                               |
|-------------------|-------------------------------|
| <i>handle</i>     | codec handle.                 |
| <i>mclk</i>       | master clock frequency in HZ. |
| <i>sampleRate</i> | sample rate in HZ.            |
| <i>bitWidth</i>   | bit width.                    |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.6.2.14** `static status_t HAL_CODEC_SetVolume ( void * handle, uint32_t playChannel, uint32_t volume ) [inline], [static]`

## Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>volume</i>      | volume value, support 0 ~ 100, 0 is mute, 100 is the maximum volume value.        |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.6.2.15** `static status_t HAL_CODEC_SetMute ( void * handle, uint32_t playChannel, bool isMute ) [inline], [static]`

## Parameters

|                    |                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                                   |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of <code>_codec_play_channel</code> . |
| <i>isMute</i>      | true is mute, false is unmute.                                                                  |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.6.2.16** `static status_t HAL_CODEC_SetPower ( void * handle, uint32_t module, bool powerOn ) [inline], [static]`

## Parameters

|                |                                        |
|----------------|----------------------------------------|
| <i>handle</i>  | codec handle.                          |
| <i>module</i>  | audio codec module.                    |
| <i>powerOn</i> | true is power on, false is power down. |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.6.2.17** `static status_t HAL_CODEC_SetRecord ( void * handle, uint32_t recordSource ) [inline], [static]`

## Parameters

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <i>handle</i>       | codec handle.                                                                                     |
| <i>recordSource</i> | audio codec record source, can be a value or combine value of <code>_codec_record_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.6.2.18** `static status_t HAL_CODEC_SetRecordChannel ( void * handle, uint32_t leftRecordChannel, uint32_t rightRecordChannel ) [inline], [static]`

## Parameters

|                            |                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>              | codec handle.                                                                                                                    |
| <i>leftRecord-Channel</i>  | audio codec record channel, reference _codec_record_channel, can be a value or combine value of member in _codec_record_channel. |
| <i>rightRecord-Channel</i> | audio codec record channel, reference _codec_record_channel, can be a value or combine of member in _codec_record_channel.       |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.6.2.19** `static status_t HAL_CODEC_SetPlay ( void * handle, uint32_t playSource ) [inline], [static]`

## Parameters

|                   |                                                                                 |
|-------------------|---------------------------------------------------------------------------------|
| <i>handle</i>     | codec handle.                                                                   |
| <i>playSource</i> | audio codec play source, can be a value or combine value of _codec_play_source. |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.6.2.20** `static status_t HAL_CODEC_ModuleControl ( void * handle, uint32_t cmd, uint32_t data ) [inline], [static]`

This function is used for codec module control, support switch digital interface cmd, can be expand to support codec module specific feature

## Parameters

|               |                                                                                                                                                                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | codec handle.                                                                                                                                                                                                                                                         |
| <i>cmd</i>    | module control cmd, reference _codec_module_ctrl_cmd.                                                                                                                                                                                                                 |
| <i>data</i>   | value to write, when cmd is kCODEC_ModuleRecordSourceChannel, the data should be a value combine of channel and source, please reference macro CODEC_MODULE_RECORD_SOURCE_CHANNEL(source, LP, LN, RP, RN), reference codec specific driver for detail configurations. |

## Returns

kStatus\_Success is success, else configure failed.

## 54.6.7 CODEC Adapter

### 54.6.7.1 Overview

#### Enumerations

- enum {  
[kCODEC\\_WM8904](#),  
[kCODEC\\_WM8960](#),  
[kCODEC\\_WM8524](#),  
[kCODEC\\_SGTL5000](#),  
[kCODEC\\_DA7212](#),  
[kCODEC\\_CS42888](#),  
[kCODEC\\_CS42448](#),  
[kCODEC\\_AK4497](#),  
[kCODEC\\_AK4458](#),  
[kCODEC\\_TFA9XXX](#),  
[kCODEC\\_TFA9896](#),  
[kCODEC\\_WM8962](#),  
[kCODEC\\_PCM512X](#),  
[kCODEC\\_PCM186X](#) }  
*codec type*

### 54.6.7.2 Enumeration Type Documentation

#### 54.6.7.2.1 anonymous enum

Enumerator

***kCODEC\_WM8904***   wm8904  
***kCODEC\_WM8960***   wm8960  
***kCODEC\_WM8524***   wm8524  
***kCODEC\_SGTL5000***   sgtl5000  
***kCODEC\_DA7212***   da7212  
***kCODEC\_CS42888***   CS42888.  
***kCODEC\_CS42448***   CS42448.  
***kCODEC\_AK4497***   AK4497.  
***kCODEC\_AK4458***   ak4458  
***kCODEC\_TFA9XXX***   tfa9xxx  
***kCODEC\_TFA9896***   tfa9896  
***kCODEC\_WM8962***   wm8962  
***kCODEC\_PCM512X***   pcm512x  
***kCODEC\_PCM186X***   pcm186x

## 54.6.8 Sgtl5000\_adapter

### 54.6.8.1 Overview

#### Macros

- #define [HAL\\_CODEC\\_SGTL\\_HANDLER\\_SIZE](#) ([SGTL\\_I2C\\_HANDLER\\_SIZE](#) + 4)  
*codec handler size*

#### Functions

- [status\\_t HAL\\_CODEC\\_SGTL5000\\_Init](#) (void \*handle, void \*config)  
*Codec initialization.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_Deinit](#) (void \*handle)  
*Codec de-initialization.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_SetFormat](#) (void \*handle, uint32\_t mclk, uint32\_t sampleRate, uint32\_t bitWidth)  
*set audio data format.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_SetVolume](#) (void \*handle, uint32\_t playChannel, uint32\_t volume)  
*set audio codec module volume.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_SetMute](#) (void \*handle, uint32\_t playChannel, bool isMute)  
*set audio codec module mute.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_SetPower](#) (void \*handle, uint32\_t module, bool powerOn)  
*set audio codec module power.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_SetRecord](#) (void \*handle, uint32\_t recordSource)  
*codec set record source.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_SetRecordChannel](#) (void \*handle, uint32\_t leftRecordChannel, uint32\_t rightRecordChannel)  
*codec set record channel.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_SetPlay](#) (void \*handle, uint32\_t playSource)  
*codec set play source.*
- [status\\_t HAL\\_CODEC\\_SGTL5000\\_ModuleControl](#) (void \*handle, uint32\_t cmd, uint32\_t data)  
*codec module control.*
- static [status\\_t HAL\\_CODEC\\_Init](#) (void \*handle, void \*config)  
*Codec initialization.*
- static [status\\_t HAL\\_CODEC\\_Deinit](#) (void \*handle)  
*Codec de-initialization.*
- static [status\\_t HAL\\_CODEC\\_SetFormat](#) (void \*handle, uint32\_t mclk, uint32\_t sampleRate, uint32\_t bitWidth)  
*set audio data format.*
- static [status\\_t HAL\\_CODEC\\_SetVolume](#) (void \*handle, uint32\_t playChannel, uint32\_t volume)  
*set audio codec module volume.*
- static [status\\_t HAL\\_CODEC\\_SetMute](#) (void \*handle, uint32\_t playChannel, bool isMute)  
*set audio codec module mute.*
- static [status\\_t HAL\\_CODEC\\_SetPower](#) (void \*handle, uint32\_t module, bool powerOn)  
*set audio codec module power.*
- static [status\\_t HAL\\_CODEC\\_SetRecord](#) (void \*handle, uint32\_t recordSource)  
*codec set record source.*

- static `status_t HAL_CODEC_SetRecordChannel` (void \*handle, uint32\_t leftRecordChannel, uint32\_t rightRecordChannel)  
*codec set record channel.*
- static `status_t HAL_CODEC_SetPlay` (void \*handle, uint32\_t playSource)  
*codec set play source.*
- static `status_t HAL_CODEC_ModuleControl` (void \*handle, uint32\_t cmd, uint32\_t data)  
*codec module control.*

## 54.6.8.2 Function Documentation

### 54.6.8.2.1 `status_t HAL_CODEC_SGTL5000_Init ( void * handle, void * config )`

Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | codec handle.        |
| <i>config</i> | codec configuration. |

Returns

kStatus\_Success is success, else initial failed.

### 54.6.8.2.2 `status_t HAL_CODEC_SGTL5000_Deinit ( void * handle )`

Parameters

|               |               |
|---------------|---------------|
| <i>handle</i> | codec handle. |
|---------------|---------------|

Returns

kStatus\_Success is success, else de-initial failed.

### 54.6.8.2.3 `status_t HAL_CODEC_SGTL5000_SetFormat ( void * handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth )`

Parameters

|                   |                               |
|-------------------|-------------------------------|
| <i>handle</i>     | codec handle.                 |
| <i>mclk</i>       | master clock frequency in HZ. |
| <i>sampleRate</i> | sample rate in HZ.            |
| <i>bitWidth</i>   | bit width.                    |

Returns

kStatus\_Success is success, else configure failed.

**54.6.8.2.4** `status_t HAL_CODEC_SGTL5000_SetVolume ( void * handle, uint32_t playChannel, uint32_t volume )`

Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>volume</i>      | volume value, support 0 ~ 100, 0 is mute, 100 is the maximum volume value.        |

Returns

kStatus\_Success is success, else configure failed.

**54.6.8.2.5** `status_t HAL_CODEC_SGTL5000_SetMute ( void * handle, uint32_t playChannel, bool isMute )`

Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>isMute</i>      | true is mute, false is unmute.                                                    |

Returns

kStatus\_Success is success, else configure failed.

**54.6.8.2.6** `status_t HAL_CODEC_SGTL5000_SetPower ( void * handle, uint32_t module, bool powerOn )`



## Parameters

|                |                                        |
|----------------|----------------------------------------|
| <i>handle</i>  | codec handle.                          |
| <i>module</i>  | audio codec module.                    |
| <i>powerOn</i> | true is power on, false is power down. |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.8.2.7 status\_t HAL\_CODEC\_SGTL5000\_SetRecord ( void \* *handle*, uint32\_t *recordSource* )

## Parameters

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| <i>handle</i>       | codec handle.                                                                       |
| <i>recordSource</i> | audio codec record source, can be a value or combine value of _codec_record_source. |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.8.2.8 status\_t HAL\_CODEC\_SGTL5000\_SetRecordChannel ( void \* *handle*, uint32\_t *leftRecordChannel*, uint32\_t *rightRecordChannel* )

## Parameters

|                            |                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>              | codec handle.                                                                                                                    |
| <i>leftRecord-Channel</i>  | audio codec record channel, reference _codec_record_channel, can be a value or combine value of member in _codec_record_channel. |
| <i>rightRecord-Channel</i> | audio codec record channel, reference _codec_record_channel, can be a value combine of member in _codec_record_channel.          |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.8.2.9 status\_t HAL\_CODEC\_SGTL5000\_SetPlay ( void \* *handle*, uint32\_t *playSource* )

## Parameters

|                   |                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------|
| <i>handle</i>     | codec handle.                                                                                 |
| <i>playSource</i> | audio codec play source, can be a value or combine value of <code>_codec_play_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

#### 54.6.8.2.10 `status_t HAL_CODEC_SGTL5000_ModuleControl ( void * handle, uint32_t cmd, uint32_t data )`

This function is used for codec module control, support switch digital interface cmd, can be expand to support codec module specific feature

## Parameters

|               |                                                                                                                                                                                                                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | codec handle.                                                                                                                                                                                                                                                                                     |
| <i>cmd</i>    | module control cmd, reference <code>_codec_module_ctrl_cmd</code> .                                                                                                                                                                                                                               |
| <i>data</i>   | value to write, when cmd is <code>kCODEC_ModuleRecordSourceChannel</code> , the data should be a value combine of channel and source, please reference macro <code>CODEC_MODULE_RECORD_SOURCE_CHANNEL(source, LP, LN, RP, RN)</code> , reference codec specific driver for detail configurations. |

## Returns

`kStatus_Success` is success, else configure failed.

#### 54.6.8.2.11 `static status_t HAL_CODEC_Init ( void * handle, void * config ) [inline], [static]`

## Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | codec handle.        |
| <i>config</i> | codec configuration. |

## Returns

`kStatus_Success` is success, else initial failed.

#### 54.6.8.2.12 `static status_t HAL_CODEC_Deinit ( void * handle ) [inline], [static]`

## Parameters

|               |               |
|---------------|---------------|
| <i>handle</i> | codec handle. |
|---------------|---------------|

## Returns

kStatus\_Success is success, else de-initial failed.

**54.6.8.2.13** `static status_t HAL_CODEC_SetFormat ( void * handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth ) [inline], [static]`

## Parameters

|                   |                               |
|-------------------|-------------------------------|
| <i>handle</i>     | codec handle.                 |
| <i>mclk</i>       | master clock frequency in HZ. |
| <i>sampleRate</i> | sample rate in HZ.            |
| <i>bitWidth</i>   | bit width.                    |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.8.2.14** `static status_t HAL_CODEC_SetVolume ( void * handle, uint32_t playChannel, uint32_t volume ) [inline], [static]`

## Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>volume</i>      | volume value, support 0 ~ 100, 0 is mute, 100 is the maximum volume value.        |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.8.2.15** `static status_t HAL_CODEC_SetMute ( void * handle, uint32_t playChannel, bool isMute ) [inline], [static]`

## Parameters

|                    |                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                                   |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of <code>_codec_play_channel</code> . |
| <i>isMute</i>      | true is mute, false is unmute.                                                                  |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.8.2.16** `static status_t HAL_CODEC_SetPower ( void * handle, uint32_t module, bool powerOn ) [inline], [static]`

## Parameters

|                |                                        |
|----------------|----------------------------------------|
| <i>handle</i>  | codec handle.                          |
| <i>module</i>  | audio codec module.                    |
| <i>powerOn</i> | true is power on, false is power down. |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.8.2.17** `static status_t HAL_CODEC_SetRecord ( void * handle, uint32_t recordSource ) [inline], [static]`

## Parameters

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <i>handle</i>       | codec handle.                                                                                     |
| <i>recordSource</i> | audio codec record source, can be a value or combine value of <code>_codec_record_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.8.2.18** `static status_t HAL_CODEC_SetRecordChannel ( void * handle, uint32_t leftRecordChannel, uint32_t rightRecordChannel ) [inline], [static]`

## Parameters

|                            |                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>              | codec handle.                                                                                                                    |
| <i>leftRecord-Channel</i>  | audio codec record channel, reference _codec_record_channel, can be a value or combine value of member in _codec_record_channel. |
| <i>rightRecord-Channel</i> | audio codec record channel, reference _codec_record_channel, can be a value or combine of member in _codec_record_channel.       |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.8.2.19** static status\_t HAL\_CODEC\_SetPlay ( void \* *handle*, uint32\_t *playSource* )  
[inline], [static]

## Parameters

|                   |                                                                                 |
|-------------------|---------------------------------------------------------------------------------|
| <i>handle</i>     | codec handle.                                                                   |
| <i>playSource</i> | audio codec play source, can be a value or combine value of _codec_play_source. |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.8.2.20** static status\_t HAL\_CODEC\_ModuleControl ( void \* *handle*, uint32\_t *cmd*, uint32\_t *data* ) [inline], [static]

This function is used for codec module control, support switch digital interface cmd, can be expand to support codec module specific feature

## Parameters

|               |                                                                                                                                                                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | codec handle.                                                                                                                                                                                                                                                         |
| <i>cmd</i>    | module control cmd, reference _codec_module_ctrl_cmd.                                                                                                                                                                                                                 |
| <i>data</i>   | value to write, when cmd is kCODEC_ModuleRecordSourceChannel, the data should be a value combine of channel and source, please reference macro CODEC_MODULE_RECORD_SOURCE_CHANNEL(source, LP, LN, RP, RN), reference codec specific driver for detail configurations. |

## Returns

kStatus\_Success is success, else configure failed.

## 54.6.9 Wm8960\_adapter

### 54.6.9.1 Overview

#### Macros

- #define [HAL\\_CODEC\\_WM8960\\_HANDLER\\_SIZE](#) ([WM8960\\_I2C\\_HANDLER\\_SIZE](#) + 4)  
*codec handler size*

#### Functions

- [status\\_t HAL\\_CODEC\\_WM8960\\_Init](#) (void \*handle, void \*config)  
*Codec initialization.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_Deinit](#) (void \*handle)  
*Codec de-initialization.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_SetFormat](#) (void \*handle, uint32\_t mclk, uint32\_t sampleRate, uint32\_t bitWidth)  
*set audio data format.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_SetVolume](#) (void \*handle, uint32\_t playChannel, uint32\_t volume)  
*set audio codec module volume.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_SetMute](#) (void \*handle, uint32\_t playChannel, bool isMute)  
*set audio codec module mute.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_SetPower](#) (void \*handle, uint32\_t module, bool powerOn)  
*set audio codec module power.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_SetRecord](#) (void \*handle, uint32\_t recordSource)  
*codec set record source.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_SetRecordChannel](#) (void \*handle, uint32\_t leftRecordChannel, uint32\_t rightRecordChannel)  
*codec set record channel.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_SetPlay](#) (void \*handle, uint32\_t playSource)  
*codec set play source.*
- [status\\_t HAL\\_CODEC\\_WM8960\\_ModuleControl](#) (void \*handle, uint32\_t cmd, uint32\_t data)  
*codec module control.*
- static [status\\_t HAL\\_CODEC\\_Init](#) (void \*handle, void \*config)  
*Codec initialization.*
- static [status\\_t HAL\\_CODEC\\_Deinit](#) (void \*handle)  
*Codec de-initialization.*
- static [status\\_t HAL\\_CODEC\\_SetFormat](#) (void \*handle, uint32\_t mclk, uint32\_t sampleRate, uint32\_t bitWidth)  
*set audio data format.*
- static [status\\_t HAL\\_CODEC\\_SetVolume](#) (void \*handle, uint32\_t playChannel, uint32\_t volume)  
*set audio codec module volume.*
- static [status\\_t HAL\\_CODEC\\_SetMute](#) (void \*handle, uint32\_t playChannel, bool isMute)  
*set audio codec module mute.*
- static [status\\_t HAL\\_CODEC\\_SetPower](#) (void \*handle, uint32\_t module, bool powerOn)  
*set audio codec module power.*
- static [status\\_t HAL\\_CODEC\\_SetRecord](#) (void \*handle, uint32\_t recordSource)  
*codec set record source.*

- static `status_t HAL_CODEC_SetRecordChannel` (void \*handle, uint32\_t leftRecordChannel, uint32\_t rightRecordChannel)  
*codec set record channel.*
- static `status_t HAL_CODEC_SetPlay` (void \*handle, uint32\_t playSource)  
*codec set play source.*
- static `status_t HAL_CODEC_ModuleControl` (void \*handle, uint32\_t cmd, uint32\_t data)  
*codec module control.*

## 54.6.9.2 Function Documentation

### 54.6.9.2.1 `status_t HAL_CODEC_WM8960_Init ( void * handle, void * config )`

Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | codec handle.        |
| <i>config</i> | codec configuration. |

Returns

kStatus\_Success is success, else initial failed.

### 54.6.9.2.2 `status_t HAL_CODEC_WM8960_Deinit ( void * handle )`

Parameters

|               |               |
|---------------|---------------|
| <i>handle</i> | codec handle. |
|---------------|---------------|

Returns

kStatus\_Success is success, else de-initial failed.

### 54.6.9.2.3 `status_t HAL_CODEC_WM8960_SetFormat ( void * handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth )`

Parameters

|                   |                               |
|-------------------|-------------------------------|
| <i>handle</i>     | codec handle.                 |
| <i>mclk</i>       | master clock frequency in HZ. |
| <i>sampleRate</i> | sample rate in HZ.            |
| <i>bitWidth</i>   | bit width.                    |

Returns

kStatus\_Success is success, else configure failed.

#### 54.6.9.2.4 status\_t HAL\_CODEC\_WM8960\_SetVolume ( void \* *handle*, uint32\_t *playChannel*, uint32\_t *volume* )

Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>volume</i>      | volume value, support 0 ~ 100, 0 is mute, 100 is the maximum volume value.        |

Returns

kStatus\_Success is success, else configure failed.

#### 54.6.9.2.5 status\_t HAL\_CODEC\_WM8960\_SetMute ( void \* *handle*, uint32\_t *playChannel*, bool *isMute* )

Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>isMute</i>      | true is mute, false is unmute.                                                    |

Returns

kStatus\_Success is success, else configure failed.

#### 54.6.9.2.6 status\_t HAL\_CODEC\_WM8960\_SetPower ( void \* *handle*, uint32\_t *module*, bool *powerOn* )



## Parameters

|                |                                        |
|----------------|----------------------------------------|
| <i>handle</i>  | codec handle.                          |
| <i>module</i>  | audio codec module.                    |
| <i>powerOn</i> | true is power on, false is power down. |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.9.2.7 status\_t HAL\_CODEC\_WM8960\_SetRecord ( void \* *handle*, uint32\_t *recordSource* )

## Parameters

|                     |                                                                                     |
|---------------------|-------------------------------------------------------------------------------------|
| <i>handle</i>       | codec handle.                                                                       |
| <i>recordSource</i> | audio codec record source, can be a value or combine value of _codec_record_source. |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.9.2.8 status\_t HAL\_CODEC\_WM8960\_SetRecordChannel ( void \* *handle*, uint32\_t *leftRecordChannel*, uint32\_t *rightRecordChannel* )

## Parameters

|                            |                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>              | codec handle.                                                                                                                    |
| <i>leftRecord-Channel</i>  | audio codec record channel, reference _codec_record_channel, can be a value or combine value of member in _codec_record_channel. |
| <i>rightRecord-Channel</i> | audio codec record channel, reference _codec_record_channel, can be a value combine of member in _codec_record_channel.          |

## Returns

kStatus\_Success is success, else configure failed.

#### 54.6.9.2.9 status\_t HAL\_CODEC\_WM8960\_SetPlay ( void \* *handle*, uint32\_t *playSource* )

## Parameters

|                   |                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------|
| <i>handle</i>     | codec handle.                                                                                 |
| <i>playSource</i> | audio codec play source, can be a value or combine value of <code>_codec_play_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

#### 54.6.9.2.10 `status_t HAL_CODEC_WM8960_ModuleControl ( void * handle, uint32_t cmd, uint32_t data )`

This function is used for codec module control, support switch digital interface cmd, can be expand to support codec module specific feature

## Parameters

|               |                                                                                                                                                                                                                                                                                                   |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | codec handle.                                                                                                                                                                                                                                                                                     |
| <i>cmd</i>    | module control cmd, reference <code>_codec_module_ctrl_cmd</code> .                                                                                                                                                                                                                               |
| <i>data</i>   | value to write, when cmd is <code>kCODEC_ModuleRecordSourceChannel</code> , the data should be a value combine of channel and source, please reference macro <code>CODEC_MODULE_RECORD_SOURCE_CHANNEL(source, LP, LN, RP, RN)</code> , reference codec specific driver for detail configurations. |

## Returns

`kStatus_Success` is success, else configure failed.

#### 54.6.9.2.11 `static status_t HAL_CODEC_Init ( void * handle, void * config ) [inline], [static]`

## Parameters

|               |                      |
|---------------|----------------------|
| <i>handle</i> | codec handle.        |
| <i>config</i> | codec configuration. |

## Returns

`kStatus_Success` is success, else initial failed.

#### 54.6.9.2.12 `static status_t HAL_CODEC_Deinit ( void * handle ) [inline], [static]`

## Parameters

|               |               |
|---------------|---------------|
| <i>handle</i> | codec handle. |
|---------------|---------------|

## Returns

kStatus\_Success is success, else de-initial failed.

**54.6.9.2.13** `static status_t HAL_CODEC_SetFormat ( void * handle, uint32_t mclk, uint32_t sampleRate, uint32_t bitWidth ) [inline], [static]`

## Parameters

|                   |                               |
|-------------------|-------------------------------|
| <i>handle</i>     | codec handle.                 |
| <i>mclk</i>       | master clock frequency in HZ. |
| <i>sampleRate</i> | sample rate in HZ.            |
| <i>bitWidth</i>   | bit width.                    |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.9.2.14** `static status_t HAL_CODEC_SetVolume ( void * handle, uint32_t playChannel, uint32_t volume ) [inline], [static]`

## Parameters

|                    |                                                                                   |
|--------------------|-----------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                     |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of _codec_play_channel. |
| <i>volume</i>      | volume value, support 0 ~ 100, 0 is mute, 100 is the maximum volume value.        |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.9.2.15** `static status_t HAL_CODEC_SetMute ( void * handle, uint32_t playChannel, bool isMute ) [inline], [static]`

## Parameters

|                    |                                                                                                 |
|--------------------|-------------------------------------------------------------------------------------------------|
| <i>handle</i>      | codec handle.                                                                                   |
| <i>playChannel</i> | audio codec play channel, can be a value or combine value of <code>_codec_play_channel</code> . |
| <i>isMute</i>      | true is mute, false is unmute.                                                                  |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.9.2.16** `static status_t HAL_CODEC_SetPower ( void * handle, uint32_t module, bool powerOn ) [inline], [static]`

## Parameters

|                |                                        |
|----------------|----------------------------------------|
| <i>handle</i>  | codec handle.                          |
| <i>module</i>  | audio codec module.                    |
| <i>powerOn</i> | true is power on, false is power down. |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.9.2.17** `static status_t HAL_CODEC_SetRecord ( void * handle, uint32_t recordSource ) [inline], [static]`

## Parameters

|                     |                                                                                                   |
|---------------------|---------------------------------------------------------------------------------------------------|
| <i>handle</i>       | codec handle.                                                                                     |
| <i>recordSource</i> | audio codec record source, can be a value or combine value of <code>_codec_record_source</code> . |

## Returns

`kStatus_Success` is success, else configure failed.

**54.6.9.2.18** `static status_t HAL_CODEC_SetRecordChannel ( void * handle, uint32_t leftRecordChannel, uint32_t rightRecordChannel ) [inline], [static]`

## Parameters

|                            |                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>              | codec handle.                                                                                                                    |
| <i>leftRecord-Channel</i>  | audio codec record channel, reference _codec_record_channel, can be a value or combine value of member in _codec_record_channel. |
| <i>rightRecord-Channel</i> | audio codec record channel, reference _codec_record_channel, can be a value or combine of member in _codec_record_channel.       |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.9.2.19 static status\_t HAL\_CODEC\_SetPlay ( void \* *handle*, uint32\_t *playSource* ) [inline], [static]**

## Parameters

|                   |                                                                                 |
|-------------------|---------------------------------------------------------------------------------|
| <i>handle</i>     | codec handle.                                                                   |
| <i>playSource</i> | audio codec play source, can be a value or combine value of _codec_play_source. |

## Returns

kStatus\_Success is success, else configure failed.

**54.6.9.2.20 static status\_t HAL\_CODEC\_ModuleControl ( void \* *handle*, uint32\_t *cmd*, uint32\_t *data* ) [inline], [static]**

This function is used for codec module control, support switch digital interface cmd, can be expand to support codec module specific feature

## Parameters

|               |                                                                                                                                                                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | codec handle.                                                                                                                                                                                                                                                         |
| <i>cmd</i>    | module control cmd, reference _codec_module_ctrl_cmd.                                                                                                                                                                                                                 |
| <i>data</i>   | value to write, when cmd is kCODEC_ModuleRecordSourceChannel, the data should be a value combine of channel and source, please reference macro CODEC_MODULE_RECORD_SOURCE_CHANNEL(source, LP, LN, RP, RN), reference codec specific driver for detail configurations. |

## Returns

kStatus\_Success is success, else configure failed.

## 54.7 Sgtl5000

### 54.7.1 Overview

#### Data Structures

- struct [\\_sgtl\\_audio\\_format](#)  
*Audio format configuration. [More...](#)*
- struct [\\_sgtl\\_config](#)  
*Initailize structure of sgtl5000. [More...](#)*
- struct [\\_sgtl\\_handle](#)  
*SGTL codec handler. [More...](#)*

#### Macros

- #define [CHIP\\_ID](#) 0x0000U  
*Define the register address of sgtl5000.*
- #define [SGTL5000\\_HEADPHONE\\_MAX\\_VOLUME\\_VALUE](#) 0x7FU  
*SGTL5000 volume setting range.*
- #define [SGTL5000\\_I2C\\_ADDR](#) 0x0A  
*SGTL5000 I2C address.*
- #define [SGTL\\_I2C\\_HANDLER\\_SIZE](#) CODEC\_I2C\_MASTER\_HANDLER\_SIZE  
*sgtl handle size*
- #define [SGTL\\_I2C\\_BITRATE](#) 100000U  
*sgtl i2c baudrate*

#### Typedefs

- typedef enum [\\_sgtl5000\\_module](#) sgtl\_module\_t  
*Modules in Sgtl5000 board.*
- typedef enum [\\_sgtl\\_route](#) sgtl\_route\_t  
*Sgtl5000 data route.*
- typedef enum [\\_sgtl\\_protocol](#) sgtl\_protocol\_t  
*The audio data transfer protocol choice.*
- typedef enum [\\_sgtl\\_sclk\\_edge](#) sgtl\_sclk\_edge\_t  
*SGTL SCLK valid edge.*
- typedef struct [\\_sgtl\\_audio\\_format](#) sgtl\_audio\_format\_t  
*Audio format configuration.*
- typedef struct [\\_sgtl\\_config](#) sgtl\_config\_t  
*Initailize structure of sgtl5000.*
- typedef struct [\\_sgtl\\_handle](#) sgtl\_handle\_t  
*SGTL codec handler.*

## Enumerations

- enum `_sgtl5000_module` {  
`kSGTL_ModuleADC` = 0x0,  
`kSGTL_ModuleDAC`,  
`kSGTL_ModuleDAP`,  
`kSGTL_ModuleHP`,  
`kSGTL_ModuleI2SIN`,  
`kSGTL_ModuleI2SOUT`,  
`kSGTL_ModuleLineIn`,  
`kSGTL_ModuleLineOut`,  
`kSGTL_ModuleMicin` }  
*Modules in Sgtl5000 board.*
- enum `_sgtl_route` {  
`kSGTL_RouteBypass` = 0x0,  
`kSGTL_RoutePlayback`,  
`kSGTL_RoutePlaybackandRecord`,  
`kSGTL_RoutePlaybackwithDAP`,  
`kSGTL_RoutePlaybackwithDAPandRecord`,  
`kSGTL_RouteRecord` }  
*Sgtl5000 data route.*
- enum `_sgtl_protocol` {  
`kSGTL_BusI2S` = 0x0,  
`kSGTL_BusLeftJustified`,  
`kSGTL_BusRightJustified`,  
`kSGTL_BusPCMA`,  
`kSGTL_BusPCMB` }  
*The audio data transfer protocol choice.*
- enum {  
`kSGTL_HeadphoneLeft` = 0,  
`kSGTL_HeadphoneRight` = 1,  
`kSGTL_LineoutLeft` = 2,  
`kSGTL_LineoutRight` = 3 }  
*sgtl play channel*
- enum {  
`kSGTL_RecordSourceLineIn` = 0U,  
`kSGTL_RecordSourceMic` = 1U }  
*sgtl record source \_sgtl\_record\_source*
- enum {  
`kSGTL_PlaySourceLineIn` = 0U,  
`kSGTL_PlaySourceDAC` = 1U }  
*sgtl play source \_sgtl\_play\_source*
- enum `_sgtl_sclk_edge` {  
`kSGTL_SclkValidEdgeRising` = 0U,  
`kSGTL_SclkValidEdgeFailling` = 1U }  
*SGTL SCLK valid edge.*

## Functions

- `status_t SGTL_Init (sgtl_handle_t *handle, sgtl_config_t *config)`  
*sgtl5000 initialize function.*
- `status_t SGTL_SetDataRoute (sgtl_handle_t *handle, sgtl_route_t route)`  
*Set audio data route in sgtl5000.*
- `status_t SGTL_SetProtocol (sgtl_handle_t *handle, sgtl_protocol_t protocol)`  
*Set the audio transfer protocol.*
- `void SGTL_SetMasterSlave (sgtl_handle_t *handle, bool master)`  
*Set sgtl5000 as master or slave.*
- `status_t SGTL_SetVolume (sgtl_handle_t *handle, sgtl_module_t module, uint32_t volume)`  
*Set the volume of different modules in sgtl5000.*
- `uint32_t SGTL_GetVolume (sgtl_handle_t *handle, sgtl_module_t module)`  
*Get the volume of different modules in sgtl5000.*
- `status_t SGTL_SetMute (sgtl_handle_t *handle, sgtl_module_t module, bool mute)`  
*Mute/unmute modules in sgtl5000.*
- `status_t SGTL_EnableModule (sgtl_handle_t *handle, sgtl_module_t module)`  
*Enable expected devices.*
- `status_t SGTL_DisableModule (sgtl_handle_t *handle, sgtl_module_t module)`  
*Disable expected devices.*
- `status_t SGTL_Deinit (sgtl_handle_t *handle)`  
*Deinit the sgtl5000 codec.*
- `status_t SGTL_ConfigDataFormat (sgtl_handle_t *handle, uint32_t mclk, uint32_t sample_rate, uint32_t bits)`  
*Configure the data format of audio data.*
- `status_t SGTL_SetPlay (sgtl_handle_t *handle, uint32_t playSource)`  
*select SGTL codec play source.*
- `status_t SGTL_SetRecord (sgtl_handle_t *handle, uint32_t recordSource)`  
*select SGTL codec record source.*
- `status_t SGTL_WriteReg (sgtl_handle_t *handle, uint16_t reg, uint16_t val)`  
*Write register to sgtl using I2C.*
- `status_t SGTL_ReadReg (sgtl_handle_t *handle, uint16_t reg, uint16_t *val)`  
*Read register from sgtl using I2C.*
- `status_t SGTL_ModifyReg (sgtl_handle_t *handle, uint16_t reg, uint16_t clr_mask, uint16_t val)`  
*Modify some bits in the register using I2C.*

## Driver version

- `#define FSL_SGTL5000_DRIVER_VERSION (MAKE_VERSION(2, 1, 1))`  
*CLOCK driver version 2.1.1.*

## 54.7.2 Data Structure Documentation

### 54.7.2.1 struct \_sgtl\_audio\_format

#### Data Fields

- `uint32_t mclk_HZ`



- *master clock*
- uint32\_t [sampleRate](#)  
*Sample rate.*
- uint32\_t [bitWidth](#)  
*Bit width.*
- [sgtl\\_sclk\\_edge\\_t](#) [sclkEdge](#)  
*sclk valid edge*

#### 54.7.2.2 struct \_sgtl\_config

##### Data Fields

- [sgtl\\_route\\_t](#) [route](#)  
*Audio data route.*
- [sgtl\\_protocol\\_t](#) [bus](#)  
*Audio transfer protocol.*
- bool [master\\_slave](#)  
*Master or slave.*
- [sgtl\\_audio\\_format\\_t](#) [format](#)  
*audio format*
- uint8\_t [slaveAddress](#)  
*code device slave address*
- [codec\\_i2c\\_config\\_t](#) [i2cConfig](#)  
*i2c bus configuration*

##### Field Documentation

(1) [sgtl\\_route\\_t](#) [\\_sgtl\\_config::route](#)

(2) [bool](#) [\\_sgtl\\_config::master\\_slave](#)

True means master, false means slave.

#### 54.7.2.3 struct \_sgtl\_handle

##### Data Fields

- [sgtl\\_config\\_t](#) \* [config](#)  
*sgtl config pointer*
- uint8\_t [i2cHandle](#) [[SGTL\\_I2C\\_HANDLER\\_SIZE](#)]  
*i2c handle*

### 54.7.3 Macro Definition Documentation

54.7.3.1 **#define FSL\_SGTL5000\_DRIVER\_VERSION (MAKE\_VERSION(2, 1, 1))**

54.7.3.2 **#define CHIP\_ID 0x0000U**

54.7.3.3 **#define SGTL5000\_I2C\_ADDR 0x0A**

### 54.7.4 Typedef Documentation

54.7.4.1 **typedef enum \_sgtl5000\_module sgtl\_module\_t**

54.7.4.2 **typedef enum \_sgtl\_route sgtl\_route\_t**

Note

Only provide some typical data route, not all route listed. Users cannot combine any routes, once a new route is set, the previous one would be replaced.

54.7.4.3 **typedef enum \_sgtl\_protocol sgtl\_protocol\_t**

Sgtl5000 only supports I2S format and PCM format.

54.7.4.4 **typedef struct \_sgtl\_audio\_format sgtl\_audio\_format\_t**

### 54.7.5 Enumeration Type Documentation

54.7.5.1 **enum \_sgtl5000\_module**

Enumerator

***kSGTL\_ModuleADC*** ADC module in SGTL5000.  
***kSGTL\_ModuleDAC*** DAC module in SGTL5000.  
***kSGTL\_ModuleDAP*** DAP module in SGTL5000.  
***kSGTL\_ModuleHP*** Headphone module in SGTL5000.  
***kSGTL\_ModuleI2SIN*** I2S-IN module in SGTL5000.  
***kSGTL\_ModuleI2SOUT*** I2S-OUT module in SGTL5000.  
***kSGTL\_ModuleLineIn*** Line-in module in SGTL5000.  
***kSGTL\_ModuleLineOut*** Line-out module in SGTL5000.  
***kSGTL\_ModuleMicin*** Microphone module in SGTL5000.

### 54.7.5.2 enum \_sgtl\_route

#### Note

Only provide some typical data route, not all route listed. Users cannot combine any routes, once a new route is set, the previous one would be replaced.

#### Enumerator

***kSGTL\_RouteBypass*** LINEIN->Headphone.  
***kSGTL\_RoutePlayback*** I2SIN->DAC->Headphone.  
***kSGTL\_RoutePlaybackandRecord*** I2SIN->DAC->Headphone, LINEIN->ADC->I2SOUT.  
***kSGTL\_RoutePlaybackwithDAP*** I2SIN->DAP->DAC->Headphone.  
***kSGTL\_RoutePlaybackwithDAPandRecord*** I2SIN->DAP->DAC->HP, LINEIN->ADC->I2SOUT.  
***kSGTL\_RouteRecord*** LINEIN->ADC->I2SOUT.

### 54.7.5.3 enum \_sgtl\_protocol

Sgtl5000 only supports I2S format and PCM format.

#### Enumerator

***kSGTL\_BusI2S*** I2S Type.  
***kSGTL\_BusLeftJustified*** Left justified.  
***kSGTL\_BusRightJustified*** Right Justified.  
***kSGTL\_BusPCMA*** PCMA.  
***kSGTL\_BusPCMB*** PCMB.

### 54.7.5.4 anonymous enum

#### Enumerator

***kSGTL\_HeadphoneLeft*** headphone left channel  
***kSGTL\_HeadphoneRight*** headphone right channel  
***kSGTL\_LineoutLeft*** lineout left channel  
***kSGTL\_LineoutRight*** lineout right channel

### 54.7.5.5 anonymous enum

#### Enumerator

***kSGTL\_RecordSourceLineIn*** record source line in  
***kSGTL\_RecordSourceMic*** record source single end

### 54.7.5.6 anonymous enum

Enumerator

*kSGTL\_PlaySourceLineIn* play source line in  
*kSGTL\_PlaySourceDAC* play source line in

### 54.7.5.7 enum \_sgtl\_sclk\_edge

Enumerator

*kSGTL\_SclkValidEdgeRising* SCLK valid edge.  
*kSGTL\_SclkValidEdgeFalling* SCLK failling edge.

## 54.7.6 Function Documentation

### 54.7.6.1 status\_t SGTL\_Init ( sgtl\_handle\_t \* *handle*, sgtl\_config\_t \* *config* )

This function calls SGTL\_I2CInit(), and in this function, some configurations are fixed. The second parameter can be NULL. If users want to change the SGTL5000 settings, a configure structure should be prepared.

Note

If the codec\_config is NULL, it would initialize sgtl5000 using default settings. The default setting:

```
* sgtl_init_t codec_config
* codec_config.route = kSGTL_RoutePlaybackandRecord
* codec_config.bus = kSGTL_BusI2S
* codec_config.master = slave
*
```

Parameters

|               |                                                                                                             |
|---------------|-------------------------------------------------------------------------------------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.                                                                                  |
| <i>config</i> | sgtl5000 configuration structure. If this pointer equals to NULL, it means using the default configuration. |

Returns

Initialization status

#### 54.7.6.2 status\_t SGTL\_SetDataRoute ( sgtl\_handle\_t \* *handle*, sgtl\_route\_t *route* )

This function would set the data route according to route. The route cannot be combined, as all route would enable different modules.

Note

If a new route is set, the previous route would not work.

Parameters

|               |                               |
|---------------|-------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.    |
| <i>route</i>  | Audio data route in sgtl5000. |

#### 54.7.6.3 status\_t SGTL\_SetProtocol ( sgtl\_handle\_t \* *handle*, sgtl\_protocol\_t *protocol* )

Sgtl5000 only supports I2S, I2S left, I2S right, PCM A, PCM B format.

Parameters

|                 |                               |
|-----------------|-------------------------------|
| <i>handle</i>   | Sgtl5000 handle structure.    |
| <i>protocol</i> | Audio data transfer protocol. |

#### 54.7.6.4 void SGTL\_SetMasterSlave ( sgtl\_handle\_t \* *handle*, bool *master* )

Parameters

|               |                                        |
|---------------|----------------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.             |
| <i>master</i> | 1 represent master, 0 represent slave. |

#### 54.7.6.5 status\_t SGTL\_SetVolume ( sgtl\_handle\_t \* *handle*, sgtl\_module\_t *module*, uint32\_t *volume* )

This function would set the volume of sgtl5000 modules. This interface set module volume. The function assume that left channel and right channel has the same volume.

kSGTL\_ModuleADC volume range: 0 - 0xF, 0dB - 22.5dB kSGTL\_ModuleDAC volume range: 0x3C - 0xF0, 0dB - -90dB kSGTL\_ModuleHP volume range: 0 - 0x7F, 12dB - -51.5dB kSGTL\_ModuleLineOut volume range: 0 - 0x1F, 0.5dB steps

## Parameters

|               |                                                                        |
|---------------|------------------------------------------------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.                                             |
| <i>module</i> | Sgtl5000 module, such as DAC, ADC and etc.                             |
| <i>volume</i> | Volume value need to be set. The value is the exact value in register. |

#### 54.7.6.6 uint32\_t SGTL\_GetVolume ( sgtl\_handle\_t \* *handle*, sgtl\_module\_t *module* )

This function gets the volume of sgtl5000 modules. This interface get DAC module volume. The function assume that left channel and right channel has the same volume.

## Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.                 |
| <i>module</i> | Sgtl5000 module, such as DAC, ADC and etc. |

## Returns

Module value, the value is exact value in register.

#### 54.7.6.7 status\_t SGTL\_SetMute ( sgtl\_handle\_t \* *handle*, sgtl\_module\_t *module*, bool *mute* )

## Parameters

|               |                                            |
|---------------|--------------------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.                 |
| <i>module</i> | Sgtl5000 module, such as DAC, ADC and etc. |
| <i>mute</i>   | True means mute, and false means unmute.   |

#### 54.7.6.8 status\_t SGTL\_EnableModule ( sgtl\_handle\_t \* *handle*, sgtl\_module\_t *module* )

## Parameters

|               |                            |
|---------------|----------------------------|
| <i>handle</i> | Sgtl5000 handle structure. |
|---------------|----------------------------|

|               |                            |
|---------------|----------------------------|
| <i>module</i> | Module expected to enable. |
|---------------|----------------------------|

#### 54.7.6.9 **status\_t SGTL\_DisableModule ( sgtl\_handle\_t \* *handle*, sgtl\_module\_t *module* )**

Parameters

|               |                            |
|---------------|----------------------------|
| <i>handle</i> | Sgtl5000 handle structure. |
| <i>module</i> | Module expected to enable. |

#### 54.7.6.10 **status\_t SGTL\_Deinit ( sgtl\_handle\_t \* *handle* )**

Shut down Sctl5000 modules.

Parameters

|               |                                    |
|---------------|------------------------------------|
| <i>handle</i> | Sgtl5000 handle structure pointer. |
|---------------|------------------------------------|

#### 54.7.6.11 **status\_t SGTL\_ConfigDataFormat ( sgtl\_handle\_t \* *handle*, uint32\_t *mclk*, uint32\_t *sample\_rate*, uint32\_t *bits* )**

This function would configure the registers about the sample rate, bit depths.

Parameters

|                    |                                                                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>      | Sgtl5000 handle structure pointer.                                                                                                            |
| <i>mclk</i>        | Master clock frequency of I2S.                                                                                                                |
| <i>sample_rate</i> | Sample rate of audio file running in sctl5000. Sctl5000 now supports 8k, 11.025k, 12k, 16k, 22.05k, 24k, 32k, 44.1k, 48k and 96k sample rate. |
| <i>bits</i>        | Bit depth of audio file (Sctl5000 only supports 16bit, 20bit, 24bit and 32 bit in HW).                                                        |

#### 54.7.6.12 **status\_t SGTL\_SetPlay ( sgtl\_handle\_t \* *handle*, uint32\_t *playSource* )**

Parameters

|                   |                                                 |
|-------------------|-------------------------------------------------|
| <i>handle</i>     | Sgtl5000 handle structure pointer.              |
| <i>playSource</i> | play source value, reference _sgtl_play_source. |

Returns

kStatus\_Success, else failed.

#### 54.7.6.13 status\_t SGTL\_SetRecord ( sgtl\_handle\_t \* *handle*, uint32\_t *recordSource* )

Parameters

|                     |                                                     |
|---------------------|-----------------------------------------------------|
| <i>handle</i>       | Sgtl5000 handle structure pointer.                  |
| <i>recordSource</i> | record source value, reference _sgtl_record_source. |

Returns

kStatus\_Success, else failed.

#### 54.7.6.14 status\_t SGTL\_WriteReg ( sgtl\_handle\_t \* *handle*, uint16\_t *reg*, uint16\_t *val* )

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.              |
| <i>reg</i>    | The register address in sgtl.           |
| <i>val</i>    | Value needs to write into the register. |

#### 54.7.6.15 status\_t SGTL\_ReadReg ( sgtl\_handle\_t \* *handle*, uint16\_t *reg*, uint16\_t \* *val* )

Parameters

|               |                               |
|---------------|-------------------------------|
| <i>handle</i> | Sgtl5000 handle structure.    |
| <i>reg</i>    | The register address in sgtl. |



|            |                   |
|------------|-------------------|
| <i>val</i> | Value written to. |
|------------|-------------------|

**54.7.6.16** `status_t SGTL_ModifyReg ( sgtl_handle_t * handle, uint16_t reg, uint16_t clr_mask, uint16_t val )`

Parameters

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>handle</i>   | Sgtl5000 handle structure.                                                       |
| <i>reg</i>      | The register address in sgtl.                                                    |
| <i>clr_mask</i> | The mask code for the bits want to write. The bit you want to write should be 0. |
| <i>val</i>      | Value needs to write into the register.                                          |

## 54.8 Wm8960

### 54.8.1 Overview

#### Data Structures

- struct [\\_wm8960\\_audio\\_format](#)  
*wm8960 audio format [More...](#)*
- struct [\\_wm8960\\_master\\_sysclk\\_config](#)  
*wm8960 master system clock configuration [More...](#)*
- struct [wm8960\\_config](#)  
*Initialize structure of WM8960. [More...](#)*
- struct [\\_wm8960\\_handle](#)  
*wm8960 codec handler [More...](#)*

#### Macros

- #define [WM8960\\_I2C\\_HANDLER\\_SIZE](#) [CODEC\\_I2C\\_MASTER\\_HANDLER\\_SIZE](#)  
*wm8960 handle size*
- #define [WM8960\\_LINVOL](#) 0x0U  
*Define the register address of WM8960.*
- #define [WM8960\\_CACHEREGNUM](#) 56U  
*Cache register number.*
- #define [WM8960\\_CLOCK2\\_BCLK\\_DIV\\_MASK](#) 0xFU  
*WM8960 CLOCK2 bits.*
- #define [WM8960\\_IFACE1\\_FORMAT\\_MASK](#) 0x03U  
*WM8960\_IFACE1 FORMAT bits.*
- #define [WM8960\\_IFACE1\\_WL\\_MASK](#) 0x0CU  
*WM8960\_IFACE1 WL bits.*
- #define [WM8960\\_IFACE1\\_LRP\\_MASK](#) 0x10U  
*WM8960\_IFACE1 LRP bit.*
- #define [WM8960\\_IFACE1\\_DLRSWAP\\_MASK](#) 0x20U  
*WM8960\_IFACE1 DLRSWAP bit.*
- #define [WM8960\\_IFACE1\\_MS\\_MASK](#) 0x40U  
*WM8960\_IFACE1 MS bit.*
- #define [WM8960\\_IFACE1\\_BCLKINV\\_MASK](#) 0x80U  
*WM8960\_IFACE1 BCLKINV bit.*
- #define [WM8960\\_IFACE1\\_ALRSWAP\\_MASK](#) 0x100U  
*WM8960\_IFACE1 ALRSWAP bit.*
- #define [WM8960\\_POWER1\\_VREF\\_MASK](#) 0x40U  
*WM8960\_POWER1.*
- #define [WM8960\\_POWER2\\_DACL\\_MASK](#) 0x100U  
*WM8960\_POWER2.*
- #define [WM8960\\_I2C\\_ADDR](#) 0x1A  
*WM8960 I2C address.*
- #define [WM8960\\_I2C\\_BAUDRATE](#) (100000U)  
*WM8960 I2C baudrate.*
- #define [WM8960\\_ADC\\_MAX\\_VOLUME\\_VALUE](#) 0xFFU  
*WM8960 maximum volume value.*

## Typedefs

- typedef enum [\\_wm8960\\_module](#) [wm8960\\_module\\_t](#)  
*Modules in WM8960 board.*
- typedef enum [\\_wm8960\\_play\\_source](#) [wm8960\\_play\\_source\\_t](#)  
*wm8960 play source*
- typedef enum [\\_wm8960\\_route](#) [wm8960\\_route\\_t](#)  
*WM8960 data route.*
- typedef enum [\\_wm8960\\_protocol](#) [wm8960\\_protocol\\_t](#)  
*The audio data transfer protocol choice.*
- typedef enum [\\_wm8960\\_input](#) [wm8960\\_input\\_t](#)  
*wm8960 input source*
- typedef enum [\\_wm8960\\_sysclk\\_source](#) [wm8960\\_sysclk\\_source\\_t](#)  
*wm8960 sysclk source*
- typedef struct [\\_wm8960\\_audio\\_format](#) [wm8960\\_audio\\_format\\_t](#)  
*wm8960 audio format*
- typedef struct [\\_wm8960\\_master\\_sysclk\\_config](#) [wm8960\\_master\\_sysclk\\_config\\_t](#)  
*wm8960 master system clock configuration*
- typedef struct [wm8960\\_config](#) [wm8960\\_config\\_t](#)  
*Initialize structure of WM8960.*
- typedef struct [\\_wm8960\\_handle](#) [wm8960\\_handle\\_t](#)  
*wm8960 codec handler*

## Enumerations

- enum [\\_wm8960\\_module](#) {  
    [kWM8960\\_ModuleADC](#) = 0,  
    [kWM8960\\_ModuleDAC](#) = 1,  
    [kWM8960\\_ModuleVREF](#) = 2,  
    [kWM8960\\_ModuleHP](#) = 3,  
    [kWM8960\\_ModuleMICB](#) = 4,  
    [kWM8960\\_ModuleMIC](#) = 5,  
    [kWM8960\\_ModuleLineIn](#) = 6,  
    [kWM8960\\_ModuleLineOut](#) = 7,  
    [kWM8960\\_ModuleSpeaker](#) = 8,  
    [kWM8960\\_ModuleOMIX](#) = 9 }  
*Modules in WM8960 board.*
- enum {  
    [kWM8960\\_HeadphoneLeft](#) = 1,  
    [kWM8960\\_HeadphoneRight](#) = 2,  
    [kWM8960\\_SpeakerLeft](#) = 4,  
    [kWM8960\\_SpeakerRight](#) = 8 }  
*wm8960 play channel*
- enum [\\_wm8960\\_play\\_source](#) {  
    [kWM8960\\_PlaySourcePGA](#) = 1,  
    [kWM8960\\_PlaySourceInput](#) = 2,

```

kWM8960_PlaySourceDAC = 4 }
 wm8960 play source
• enum _wm8960_route {
 kWM8960_RouteBypass = 0,
 kWM8960_RoutePlayback = 1,
 kWM8960_RoutePlaybackandRecord = 2,
 kWM8960_RouteRecord = 5 }
 WM8960 data route.
• enum _wm8960_protocol {
 kWM8960_BusI2S = 2,
 kWM8960_BusLeftJustified = 1,
 kWM8960_BusRightJustified = 0,
 kWM8960_BusPCMA = 3,
 kWM8960_BusPCMB = 3 | (1 << 4) }
 The audio data transfer protocol choice.
• enum _wm8960_input {
 kWM8960_InputClosed = 0,
 kWM8960_InputSingleEndedMic = 1,
 kWM8960_InputDifferentialMicInput2 = 2,
 kWM8960_InputDifferentialMicInput3 = 3,
 kWM8960_InputLineINPUT2 = 4,
 kWM8960_InputLineINPUT3 = 5 }
 wm8960 input source
• enum {
 kWM8960_AudioSampleRate8KHz = 8000U,
 kWM8960_AudioSampleRate11025Hz = 11025U,
 kWM8960_AudioSampleRate12KHz = 12000U,
 kWM8960_AudioSampleRate16KHz = 16000U,
 kWM8960_AudioSampleRate22050Hz = 22050U,
 kWM8960_AudioSampleRate24KHz = 24000U,
 kWM8960_AudioSampleRate32KHz = 32000U,
 kWM8960_AudioSampleRate44100Hz = 44100U,
 kWM8960_AudioSampleRate48KHz = 48000U,
 kWM8960_AudioSampleRate96KHz = 96000U,
 kWM8960_AudioSampleRate192KHz = 192000U,
 kWM8960_AudioSampleRate384KHz = 384000U }
 audio sample rate definition
• enum {
 kWM8960_AudioBitWidth16bit = 16U,
 kWM8960_AudioBitWidth20bit = 20U,
 kWM8960_AudioBitWidth24bit = 24U,
 kWM8960_AudioBitWidth32bit = 32U }
 audio bit width
• enum _wm8960_sysclk_source {
 kWM8960_SysClkSourceMclk = 0U,
 kWM8960_SysClkSourceInternalPLL = 1U }

```

*wm8960 sysclk source*

## Functions

- [status\\_t WM8960\\_Init](#) ([wm8960\\_handle\\_t](#) \*handle, const [wm8960\\_config\\_t](#) \*config)  
*WM8960 initialize function.*
- [status\\_t WM8960\\_Deinit](#) ([wm8960\\_handle\\_t](#) \*handle)  
*Deinit the WM8960 codec.*
- [status\\_t WM8960\\_SetDataRoute](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_route\\_t](#) route)  
*Set audio data route in WM8960.*
- [status\\_t WM8960\\_SetLeftInput](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_input\\_t](#) input)  
*Set left audio input source in WM8960.*
- [status\\_t WM8960\\_SetRightInput](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_input\\_t](#) input)  
*Set right audio input source in WM8960.*
- [status\\_t WM8960\\_SetProtocol](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_protocol\\_t](#) protocol)  
*Set the audio transfer protocol.*
- [void WM8960\\_SetMasterSlave](#) ([wm8960\\_handle\\_t](#) \*handle, bool master)  
*Set WM8960 as master or slave.*
- [status\\_t WM8960\\_SetVolume](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_module\\_t](#) module, [uint32\\_t](#) volume)  
*Set the volume of different modules in WM8960.*
- [uint32\\_t WM8960\\_GetVolume](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_module\\_t](#) module)  
*Get the volume of different modules in WM8960.*
- [status\\_t WM8960\\_SetMute](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_module\\_t](#) module, bool is-Enabled)  
*Mute modules in WM8960.*
- [status\\_t WM8960\\_SetModule](#) ([wm8960\\_handle\\_t](#) \*handle, [wm8960\\_module\\_t](#) module, bool is-Enabled)  
*Enable/disable expected devices.*
- [status\\_t WM8960\\_SetPlay](#) ([wm8960\\_handle\\_t](#) \*handle, [uint32\\_t](#) playSource)  
*SET the WM8960 play source.*
- [status\\_t WM8960\\_ConfigDataFormat](#) ([wm8960\\_handle\\_t](#) \*handle, [uint32\\_t](#) sysclk, [uint32\\_t](#) sample\_rate, [uint32\\_t](#) bits)  
*Configure the data format of audio data.*
- [status\\_t WM8960\\_SetJackDetect](#) ([wm8960\\_handle\\_t](#) \*handle, bool isEnabled)  
*Enable/disable jack detect feature.*
- [status\\_t WM8960\\_WriteReg](#) ([wm8960\\_handle\\_t](#) \*handle, [uint8\\_t](#) reg, [uint16\\_t](#) val)  
*Write register to WM8960 using I2C.*
- [status\\_t WM8960\\_ReadReg](#) ([uint8\\_t](#) reg, [uint16\\_t](#) \*val)  
*Read register from WM8960 using I2C.*
- [status\\_t WM8960\\_ModifyReg](#) ([wm8960\\_handle\\_t](#) \*handle, [uint8\\_t](#) reg, [uint16\\_t](#) mask, [uint16\\_t](#) val)  
*Modify some bits in the register using I2C.*

## Driver version

- `#define FSL_WM8960_DRIVER_VERSION (MAKE_VERSION(2, 2, 4))`  
*CLOCK driver version 2.2.4.*

## 54.8.2 Data Structure Documentation

### 54.8.2.1 struct \_wm8960\_audio\_format

#### Data Fields

- uint32\_t [mclk\\_HZ](#)  
*master clock frequency*
- uint32\_t [sampleRate](#)  
*sample rate*
- uint32\_t [bitWidth](#)  
*bit width*

### 54.8.2.2 struct \_wm8960\_master\_sysclk\_config

#### Data Fields

- [wm8960\\_sysclk\\_source\\_t](#) sysclkSource  
*sysclk source*
- uint32\_t [sysclkFreq](#)  
*PLL output frequency value.*

### 54.8.2.3 struct wm8960\_config

#### Data Fields

- [wm8960\\_route\\_t](#) route  
*Audio data route.*
- [wm8960\\_protocol\\_t](#) bus  
*Audio transfer protocol.*
- [wm8960\\_audio\\_format\\_t](#) format  
*Audio format.*
- bool [master\\_slave](#)  
*Master or slave.*
- [wm8960\\_master\\_sysclk\\_config\\_t](#) masterClock  
*master clock configurations*
- bool [enableSpeaker](#)  
*True means enable class D speaker as output, false means no.*
- [wm8960\\_input\\_t](#) leftInputSource  
*Left input source for WM8960.*
- [wm8960\\_input\\_t](#) rightInputSource  
*Right input source for wm8960.*
- [wm8960\\_play\\_source\\_t](#) playSource  
*play source*
- uint8\_t [slaveAddress](#)  
*wm8960 device address*
- [codec\\_i2c\\_config\\_t](#) i2cConfig  
*i2c configuration*

## Field Documentation

(1) `wm8960_route_t wm8960_config::route`

(2) `bool wm8960_config::master_slave`

### 54.8.2.4 struct \_wm8960\_handle

#### Data Fields

- const `wm8960_config_t * config`  
*wm8904 config pointer*
- `uint8_t i2cHandle [WM8960_I2C_HANDLER_SIZE]`  
*i2c handle*

### 54.8.3 Macro Definition Documentation

54.8.3.1 `#define WM8960_LINVOL 0x0U`

54.8.3.2 `#define WM8960_I2C_ADDR 0x1A`

### 54.8.4 Typedef Documentation

54.8.4.1 `typedef enum _wm8960_module wm8960_module_t`

54.8.4.2 `typedef enum _wm8960_route wm8960_route_t`

Only provide some typical data route, not all route listed. Note: Users cannot combine any routes, once a new route is set, the previous one would be replaced.

54.8.4.3 `typedef enum _wm8960_protocol wm8960_protocol_t`

WM8960 only supports I2S format and PCM format.

### 54.8.5 Enumeration Type Documentation

54.8.5.1 `enum _wm8960_module`

Enumerator

- kWM8960\_ModuleADC* ADC module in WM8960.
- kWM8960\_ModuleDAC* DAC module in WM8960.
- kWM8960\_ModuleVREF* VREF module.
- kWM8960\_ModuleHP* Headphone.

***kWM8960\_ModuleMICB*** Mic bias.  
***kWM8960\_ModuleMIC*** Input Mic.  
***kWM8960\_ModuleLineIn*** Analog in PGA.  
***kWM8960\_ModuleLineOut*** Line out module.  
***kWM8960\_ModuleSpeaker*** Speaker module.  
***kWM8960\_ModuleOMIX*** Output mixer.

#### 54.8.5.2 anonymous enum

Enumerator

***kWM8960\_HeadphoneLeft*** wm8960 headphone left channel  
***kWM8960\_HeadphoneRight*** wm8960 headphone right channel  
***kWM8960\_SpeakerLeft*** wm8960 speaker left channel  
***kWM8960\_SpeakerRight*** wm8960 speaker right channel

#### 54.8.5.3 enum \_wm8960\_play\_source

Enumerator

***kWM8960\_PlaySourcePGA*** wm8960 play source PGA  
***kWM8960\_PlaySourceInput*** wm8960 play source Input  
***kWM8960\_PlaySourceDAC*** wm8960 play source DAC

#### 54.8.5.4 enum \_wm8960\_route

Only provide some typical data route, not all route listed. Note: Users cannot combine any routes, once a new route is set, the previous one would be replaced.

Enumerator

***kWM8960\_RouteBypass*** LINEIN->Headphone.  
***kWM8960\_RoutePlayback*** I2SIN->DAC->Headphone.  
***kWM8960\_RoutePlaybackandRecord*** I2SIN->DAC->Headphone, LINEIN->ADC->I2SOUT.  
***kWM8960\_RouteRecord*** LINEIN->ADC->I2SOUT.

#### 54.8.5.5 enum \_wm8960\_protocol

WM8960 only supports I2S format and PCM format.

Enumerator

***kWM8960\_BusI2S*** I2S type.



***kWM8960\_BusLeftJustified*** Left justified mode.  
***kWM8960\_BusRightJustified*** Right justified mode.  
***kWM8960\_BusPCMA*** PCM A mode.  
***kWM8960\_BusPCMB*** PCM B mode.

#### 54.8.5.6 enum \_wm8960\_input

Enumerator

***kWM8960\_InputClosed*** Input device is closed.  
***kWM8960\_InputSingleEndedMic*** Input as single ended mic, only use L/RINPUT1.  
***kWM8960\_InputDifferentialMicInput2*** Input as differential mic, use L/RINPUT1 and L/RINPUT2.  
***kWM8960\_InputDifferentialMicInput3*** Input as differential mic, use L/RINPUT1 and L/RINPUT3.  
***kWM8960\_InputLineINPUT2*** Input as line input, only use L/RINPUT2.  
***kWM8960\_InputLineINPUT3*** Input as line input, only use L/RINPUT3.

#### 54.8.5.7 anonymous enum

Enumerator

***kWM8960\_AudioSampleRate8KHz*** Sample rate 8000 Hz.  
***kWM8960\_AudioSampleRate11025Hz*** Sample rate 11025 Hz.  
***kWM8960\_AudioSampleRate12KHz*** Sample rate 12000 Hz.  
***kWM8960\_AudioSampleRate16KHz*** Sample rate 16000 Hz.  
***kWM8960\_AudioSampleRate22050Hz*** Sample rate 22050 Hz.  
***kWM8960\_AudioSampleRate24KHz*** Sample rate 24000 Hz.  
***kWM8960\_AudioSampleRate32KHz*** Sample rate 32000 Hz.  
***kWM8960\_AudioSampleRate44100Hz*** Sample rate 44100 Hz.  
***kWM8960\_AudioSampleRate48KHz*** Sample rate 48000 Hz.  
***kWM8960\_AudioSampleRate96KHz*** Sample rate 96000 Hz.  
***kWM8960\_AudioSampleRate192KHz*** Sample rate 192000 Hz.  
***kWM8960\_AudioSampleRate384KHz*** Sample rate 384000 Hz.

#### 54.8.5.8 anonymous enum

Enumerator

***kWM8960\_AudioBitWidth16bit*** audio bit width 16  
***kWM8960\_AudioBitWidth20bit*** audio bit width 20  
***kWM8960\_AudioBitWidth24bit*** audio bit width 24  
***kWM8960\_AudioBitWidth32bit*** audio bit width 32

### 54.8.5.9 enum \_wm8960\_sysclk\_source

Enumerator

*kWM8960\_SysClkSourceMclk* sysclk source from external MCLK

*kWM8960\_SysClkSourceInternalPLL* sysclk source from internal PLL

## 54.8.6 Function Documentation

### 54.8.6.1 status\_t WM8960\_Init ( wm8960\_handle\_t \* *handle*, const wm8960\_config\_t \* *config* )

The second parameter is NULL to WM8960 in this version. If users want to change the settings, they have to use wm8960\_write\_reg() or wm8960\_modify\_reg() to set the register value of WM8960. Note: If the codec\_config is NULL, it would initialize WM8960 using default settings. The default setting: codec\_config->route = kWM8960\_RoutePlaybackandRecord codec\_config->bus = kWM8960\_BusI2S codec\_config->master = slave

Parameters

|               |                                 |
|---------------|---------------------------------|
| <i>handle</i> | WM8960 handle structure.        |
| <i>config</i> | WM8960 configuration structure. |

### 54.8.6.2 status\_t WM8960\_Deinit ( wm8960\_handle\_t \* *handle* )

This function close all modules in WM8960 to save power.

Parameters

|               |                                  |
|---------------|----------------------------------|
| <i>handle</i> | WM8960 handle structure pointer. |
|---------------|----------------------------------|

### 54.8.6.3 status\_t WM8960\_SetDataRoute ( wm8960\_handle\_t \* *handle*, wm8960\_route\_t *route* )

This function would set the data route according to route. The route cannot be combined, as all route would enable different modules. Note: If a new route is set, the previous route would not work.

Parameters

|               |                             |
|---------------|-----------------------------|
| <i>handle</i> | WM8960 handle structure.    |
| <i>route</i>  | Audio data route in WM8960. |

#### 54.8.6.4 **status\_t WM8960\_SetLeftInput ( wm8960\_handle\_t \* *handle*, wm8960\_input\_t *input* )**

Parameters

|               |                          |
|---------------|--------------------------|
| <i>handle</i> | WM8960 handle structure. |
| <i>input</i>  | Audio input source.      |

#### 54.8.6.5 **status\_t WM8960\_SetRightInput ( wm8960\_handle\_t \* *handle*, wm8960\_input\_t *input* )**

Parameters

|               |                          |
|---------------|--------------------------|
| <i>handle</i> | WM8960 handle structure. |
| <i>input</i>  | Audio input source.      |

#### 54.8.6.6 **status\_t WM8960\_SetProtocol ( wm8960\_handle\_t \* *handle*, wm8960\_protocol\_t *protocol* )**

WM8960 only supports I2S, left justified, right justified, PCM A, PCM B format.

Parameters

|                 |                               |
|-----------------|-------------------------------|
| <i>handle</i>   | WM8960 handle structure.      |
| <i>protocol</i> | Audio data transfer protocol. |

#### 54.8.6.7 **void WM8960\_SetMasterSlave ( wm8960\_handle\_t \* *handle*, bool *master* )**

Parameters

|               |                                        |
|---------------|----------------------------------------|
| <i>handle</i> | WM8960 handle structure.               |
| <i>master</i> | 1 represent master, 0 represent slave. |

#### 54.8.6.8 **status\_t WM8960\_SetVolume ( wm8960\_handle\_t \* *handle*, wm8960\_module\_t *module*, uint32\_t *volume* )**

This function would set the volume of WM8960 modules. Uses need to appoint the module. The function assume that left channel and right channel has the same volume.

Module:kWM8960\_ModuleADC, volume range value: 0 is mute, 1-255 is -97db to 30db Module:kWM8960\_ModuleDAC, volume range value: 0 is mute, 1-255 is -127db to 0db Module:kWM8960\_ModuleHP, volume range value: 0 - 2F is mute, 0x30 - 0x7F is -73db to 6db Module:kWM8960\_ModuleLineIn, volume range value: 0 - 0x3F is -17.25db to 30db Module:kWM8960\_ModuleSpeaker, volume range value: 0 - 2F is mute, 0x30 - 0x7F is -73db to 6db

Parameters

|               |                                                                |
|---------------|----------------------------------------------------------------|
| <i>handle</i> | WM8960 handle structure.                                       |
| <i>module</i> | Module to set volume, it can be ADC, DAC, Headphone and so on. |
| <i>volume</i> | Volume value need to be set.                                   |

#### 54.8.6.9 **uint32\_t WM8960\_GetVolume ( wm8960\_handle\_t \* *handle*, wm8960\_module\_t *module* )**

This function gets the volume of WM8960 modules. Uses need to appoint the module. The function assume that left channel and right channel has the same volume.

Parameters

|               |                                                                |
|---------------|----------------------------------------------------------------|
| <i>handle</i> | WM8960 handle structure.                                       |
| <i>module</i> | Module to set volume, it can be ADC, DAC, Headphone and so on. |

Returns

Volume value of the module.

#### 54.8.6.10 **status\_t WM8960\_SetMute ( wm8960\_handle\_t \* *handle*, wm8960\_module\_t *module*, bool *isEnabled* )**

## Parameters

|                  |                                   |
|------------------|-----------------------------------|
| <i>handle</i>    | WM8960 handle structure.          |
| <i>module</i>    | Modules need to be mute.          |
| <i>isEnabled</i> | Mute or unmute, 1 represent mute. |

#### 54.8.6.11 **status\_t WM8960\_SetModule ( wm8960\_handle\_t \* *handle*, wm8960\_module\_t *module*, bool *isEnabled* )**

## Parameters

|                  |                            |
|------------------|----------------------------|
| <i>handle</i>    | WM8960 handle structure.   |
| <i>module</i>    | Module expected to enable. |
| <i>isEnabled</i> | Enable or disable moudles. |

#### 54.8.6.12 **status\_t WM8960\_SetPlay ( wm8960\_handle\_t \* *handle*, uint32\_t *playSource* )**

## Parameters

|                   |                                                                                                                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>     | WM8960 handle structure.                                                                                                                                                                            |
| <i>playSource</i> | play source , can be a value combine of kWM8960_ModuleHeadphoneSourcePGA, kWM8960_ModuleHeadphoneSourceDAC, kWM8960_ModulePlaySourceInput, kWM8960_ModulePlayMonoRight, kWM8960_ModulePlayMonoLeft. |

## Returns

kStatus\_WM8904\_Success if successful, different code otherwise..

#### 54.8.6.13 **status\_t WM8960\_ConfigDataFormat ( wm8960\_handle\_t \* *handle*, uint32\_t *sysclk*, uint32\_t *sample\_rate*, uint32\_t *bits* )**

This function would configure the registers about the sample rate, bit depths.

## Parameters

|                    |                                                                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <i>handle</i>      | WM8960 handle structure pointer.                                                                                                          |
| <i>sysclk</i>      | system clock of the codec which can be generated by MCLK or PLL output.                                                                   |
| <i>sample_rate</i> | Sample rate of audio file running in WM8960. WM8960 now supports 8k, 11.025k, 12k, 16k, 22.05k, 24k, 32k, 44.1k, 48k and 96k sample rate. |
| <i>bits</i>        | Bit depth of audio file (WM8960 only supports 16bit, 20bit, 24bit and 32 bit in HW).                                                      |

**54.8.6.14** `status_t WM8960_SetJackDetect ( wm8960_handle_t * handle, bool isEnabled )`

Parameters

|                  |                            |
|------------------|----------------------------|
| <i>handle</i>    | WM8960 handle structure.   |
| <i>isEnabled</i> | Enable or disable moudles. |

**54.8.6.15** `status_t WM8960_WriteReg ( wm8960_handle_t * handle, uint8_t reg, uint16_t val )`

Parameters

|               |                                         |
|---------------|-----------------------------------------|
| <i>handle</i> | WM8960 handle structure.                |
| <i>reg</i>    | The register address in WM8960.         |
| <i>val</i>    | Value needs to write into the register. |

**54.8.6.16** `status_t WM8960_ReadReg ( uint8_t reg, uint16_t * val )`

Parameters

|            |                                 |
|------------|---------------------------------|
| <i>reg</i> | The register address in WM8960. |
| <i>val</i> | Value written to.               |

**54.8.6.17** `status_t WM8960_ModifyReg ( wm8960_handle_t * handle, uint8_t reg, uint16_t mask, uint16_t val )`

## Parameters

|               |                                                                                  |
|---------------|----------------------------------------------------------------------------------|
| <i>handle</i> | WM8960 handle structure.                                                         |
| <i>reg</i>    | The register address in WM8960.                                                  |
| <i>mask</i>   | The mask code for the bits want to write. The bit you want to write should be 0. |
| <i>val</i>    | Value needs to write into the register.                                          |

## 54.9 Serial\_port\_swo

### 54.9.1 Overview

#### Data Structures

- struct [\\_serial\\_port\\_swo\\_config](#)  
*serial port swo config struct [More...](#)*

#### Macros

- #define [SERIAL\\_PORT\\_SWO\\_HANDLE\\_SIZE](#) (12U)  
*serial port swo handle size*

#### Typedefs

- typedef enum  
[\\_serial\\_port\\_swo\\_protocol](#) [serial\\_port\\_swo\\_protocol\\_t](#)  
*serial port swo protocol*
- typedef struct  
[\\_serial\\_port\\_swo\\_config](#) [serial\\_port\\_swo\\_config\\_t](#)  
*serial port swo config struct*

#### Enumerations

- enum [\\_serial\\_port\\_swo\\_protocol](#) {  
    [kSerialManager\\_SwoProtocolManchester](#) = 1U,  
    [kSerialManager\\_SwoProtocolNrz](#) = 2U }  
*serial port swo protocol*

### 54.9.2 Data Structure Documentation

#### 54.9.2.1 struct \_serial\_port\_swo\_config

##### Data Fields

- uint32\_t [clockRate](#)  
*clock rate*
- uint32\_t [baudRate](#)  
*baud rate*
- uint32\_t [port](#)  
*Port used to transfer data.*
- [serial\\_port\\_swo\\_protocol\\_t](#) [protocol](#)  
*SWO protocol.*



### 54.9.3 Enumeration Type Documentation

#### 54.9.3.1 enum \_serial\_port\_swo\_protocol

Enumerator

*kSerialManager\_SwoProtocolManchester* SWO Manchester protocol.

*kSerialManager\_SwoProtocolNrz* SWO UART/NRZ protocol.

## 54.10 Serial\_port\_uart

### 54.10.1 Overview

#### Macros

- #define `SERIAL_PORT_UART_DMA_RECEIVE_DATA_LENGTH` (64U)  
*serial port uart handle size*
- #define `SERIAL_USE_CONFIGURE_STRUCTURE` (0U)  
*Enable or disable the configure structure pointer.*

#### Typedefs

- typedef enum  
`_serial_port_uart_parity_mode` `serial_port_uart_parity_mode_t`  
*serial port uart parity mode*
- typedef enum  
`_serial_port_uart_stop_bit_count` `serial_port_uart_stop_bit_count_t`  
*serial port uart stop bit count*

#### Enumerations

- enum `_serial_port_uart_parity_mode` {  
`kSerialManager_UartParityDisabled` = 0x0U,  
`kSerialManager_UartParityEven` = 0x2U,  
`kSerialManager_UartParityOdd` = 0x3U }  
*serial port uart parity mode*
- enum `_serial_port_uart_stop_bit_count` {  
`kSerialManager_UartOneStopBit` = 0U,  
`kSerialManager_UartTwoStopBit` = 1U }  
*serial port uart stop bit count*

### 54.10.2 Enumeration Type Documentation

#### 54.10.2.1 enum \_serial\_port\_uart\_parity\_mode

##### Enumerator

***kSerialManager\_UartParityDisabled*** Parity disabled.  
***kSerialManager\_UartParityEven*** Parity even enabled.  
***kSerialManager\_UartParityOdd*** Parity odd enabled.

### 54.10.2.2 enum \_serial\_port\_uart\_stop\_bit\_count

Enumerator

*kSerialManager\_UartOneStopBit* One stop bit.

*kSerialManager\_UartTwoStopBit* Two stop bits.

**How to Reach Us:****Home Page:**

[nxp.com](http://nxp.com)

**Web Support:**

[nxp.com/support](http://nxp.com/support)

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: [nxp.com/SalesTermsandConditions](http://nxp.com/SalesTermsandConditions).

While NXP has implemented advanced security features, all products may be subject to unidentified vulnerabilities. Customers are responsible for the design and operation of their applications and products to reduce the effect of these vulnerabilities on customer's applications and products, and NXP accepts no liability for any vulnerability that is discovered. Customers should implement appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, Freescale, the Freescale logo, Kinetis, Processor Expert, and Tower are trademarks of NXP B.V. All other product or service names are the property of their respective owners. Arm, Cortex, Keil, Mbed, Mbed Enabled, and Vision are trademarks or registered trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. The related technology may be protected by any or all of patents, copyrights, designs and trade secrets. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2021 NXP B.V.

